Cryptography From the Ground Up

The Mathematics and Intuition of Keeping Secrets — From Modular Arithmetic to TLS, WiFi, and Beyond

An Entry Path into Real Cryptography

June 2026

Preface

There is a particular kind of frustration that brings people to cryptography. You read that TLS "encrypts your connection," that Signal offers "end-to-end encryption," that your WiFi password is "protected by WPA3," and somewhere underneath all of it sits mathematics you were told to trust but never got to see. The padlock in the browser is a promise, and this book is about the mathematics that backs it.

I wrote it because the books that taught me cryptography each covered only part of the subject. Some gave me clean theorems and security proofs but never showed a packet on a wire. Some gave me practical recipes – use AES-GCM, never roll your own crypto – without ever explaining *why* AES-GCM and not something else, or what "rolling your own" actually breaks. And some gave me beautiful intuition with hand-waving exactly where I most wanted rigor. The goal here is to cover all three at once: the math, the intuition, and the real-world use, side by side on the same page.

What this book is, and who it is for

You are smart, motivated, and willing to do arithmetic by hand when it teaches you something. You do not need to already know number theory or abstract algebra – we build those from the ground up. You should be comfortable with high-school algebra, with the idea of a function, and with the patience to follow a small calculation to its end. If you have written a few hundred lines of code in any language, you will get more out of the toy implementations, but they are not strictly required.

By the end you should be able to start work as an entry-level cryptographer, or a security engineer who actually understands the primitives. Concretely, that means you can:

- read a protocol specification (a TLS RFC, the Signal documentation, the WPA3 standard) and follow the cryptographic reasoning without it being a wall of acronyms;
- look at a scheme and state its threat model – who the attacker is, what they can see, what they can change, and what "secure" is supposed to mean;
- do the math: modular exponentiation, a tiny RSA key, a point addition on an elliptic curve over a small field, a birthday-bound estimate, all by hand on small numbers;
- recognize the classic failure modes – nonce reuse, padding oracles, timing leaks, missing authentication – and explain the attack, not just name it;
- choose the right tool (authenticated encryption, not a raw block cipher; a key-derivation function, not a bare hash) and say why.

That last skill matters most. Memorizing algorithms does not make someone a cryptographer;

reasoning about what an adversary can do, and proving – or at least arguing carefully – that the adversary cannot do the thing you care about, does. Everything in this book is aimed at building that habit.

How to read this book

Each scheme is introduced in the same order: a story, a small worked example, the precise math, and a security argument.

First comes the *story*. Alice wants to send Bob a message; Eve is listening on the wire, and sometimes Mallory is actively tampering with it. We start there because every cryptographic protocol is a contest between parties with conflicting goals, and the algorithms are the moves in that contest. Naming the players keeps you precise about who knows what.

Reading note

The cast you will meet over and over: **Alice** and **Bob** want to communicate. **Eve** is a passive eavesdropper – she reads but does not change. **Mallory** is an active, in-the-middle attacker – she can drop, replay, reorder, and forge messages. **Trent** is a trusted third party (a certificate authority, say). When a scheme protects against Eve but not Mallory, that distinction is usually where the interesting attacks live.

Then come *tiny numbers*. Before any general formula, we trace the idea through an example small enough to check on paper. RSA with primes $p = 61$ and $q = 53$. A four-bit toy block cipher you can encrypt by hand. An elliptic curve over $\mathbb{F}_{11}$ with so few points you can list them all. Working through a calculation like $7^{13} \bmod 23$ by hand makes the abstract idea concrete.

Then the *precise math*, and a *security argument*. We state what the scheme is, what assumption it rests on (factoring is hard, the discrete logarithm is hard, this hash behaves like a random function), and we argue why breaking the scheme would mean breaking the assumption. Some of these arguments are full reductions in the style of Katz and Lindell; others are the informal-but-correct sketches you would give a colleague. Either way you should finish a section able to answer "why do we believe this is secure?" with more than "because the textbook said so."

You do not have to read strictly front to back, but the early chapters genuinely are foundations. The arithmetic and algebra of Part I are the vocabulary every later chapter speaks. If you already know modular arithmetic and groups cold, skim them and start at the symmetric ciphers; if you do not, resist the urge to skip – you will only end up back there confused.

The colored boxes, and what each one signals

The text uses recurring colored boxes so you can tell at a glance what kind of thinking a passage demands. Each type has a fixed purpose:

Intuition

The mental picture – the analogy or the one-sentence "aha" that makes the formal statement make sense. If you remember nothing else from a section, remember the intuition boxes.

Why it works

The argument for *why* something is secure, or why an attack succeeds. This is where the security reasoning lives – reductions, indistinguishability arguments, the logic that connects an assumption to a guarantee.

Formal

The precise definition or theorem, stated the way you would write it in a paper. When intuition and rigor diverge, this box is the authority.

Attack

A concrete break. Not "this could be weak" but "here is exactly how Mallory wins, step by step." Seeing how attacks work teaches defense better than a list of best practices does.

Focus

Key ideas and guiding themes that keep the chapter's main thread in view.

Implement it

Short toy code – usually Python – that turns an idea into something you can run. **Read this warning once and carry it through the whole book:** the code in these boxes is written to be *transparent*, not safe. It leaks timing, it uses weak randomness, it skips the checks that production code must never skip. It exists so you can see the mechanism. *Never* paste it into anything real. Real cryptographic code belongs to vetted, constant-time, well-maintained libraries, and one of the things this book teaches is the judgment to know why.

In the real world

Where the idea shows up in systems you use every day – TLS, WiFi, Signal, SSH, the cellular network. These boxes show why the idea matters in practice, and where abstract guarantees run into engineering constraints.

Connection

A pointer to the companion ground-up readers on probability and on algorithms. When we need a birthday bound, the cost of modular exponentiation, or the analysis of a randomized algorithm, the deep version lives there and we link to it rather than reprove it.

You will also meet `historybox` (historical context and lessons from past failures), `walkbox` (a walking question to chew on away from the page), `trapbox` (a mistake almost everyone makes once), and `recoverybox` (a place to pause and consolidate). Use the boxes to tell what kind of reading a passage needs.

The security mindset

Before any specific scheme, you need a way of thinking, and it is the hardest thing to teach because it runs against ordinary engineering instincts. An ordinary engineer asks "does it work?" – does the system do the right thing when used the right way. A cryptographer asks the adversary's question: "how does it fail when someone is *trying* to make it fail?" You design against the attacker, not the typical user.

This flips the burden of proof. In normal software, a feature is correct until a bug is found. In cryptography, a scheme is *broken* until there is a convincing argument that it is not – and "I couldn't break it" is not such an argument, because the person who breaks it may be cleverer, better funded, or simply have more time than you. That is why the field leans so heavily on definitions and proofs. A security definition is a promise made precise: it says exactly which attacker, with exactly which powers, fails to achieve exactly which goal. Without that precision, "secure" is a marketing word.

Intuition

Think of it as a game. We hand the attacker a generous set of powers – let her choose messages to be encrypted, let her see ciphertexts, sometimes even let her ask for some decryptions – and we define a single thing she must not be able to do (tell two ciphertexts apart, say, or forge one valid tag). The scheme is secure if *even with all those powers* her chance of winning is no better than blind guessing, up to a negligible sliver. We will make "negligible" precise later; for now it means "so small that no realistic amount of computation closes the gap."

Two principles follow from this mindset and recur on nearly every page.

Kerckhoffs's principle: the secret is the key, never the design

In 1883 the Dutch linguist Auguste Kerckhoffs laid down a rule for military ciphers that the entire modern field has adopted: a cryptosystem should remain secure even if everything about it is public knowledge *except the key*. The algorithm, the protocol, the source code, the exact wiring – assume the enemy has all of it. The only thing you get to keep secret is a short string, the key, which is easy to generate fresh, easy to change, and easy to keep in small quantity.

History

Claude Shannon, founding the mathematical theory of secrecy in 1949, restated Kerckhoffs's 1883 rule so bluntly it became a slogan: "the enemy knows the system." Design as if your adversary has read your code, bought your hardware, and reverse-engineered your firmware – because eventually one of them will. Historically, governments and vendors resisted this, relying on secret designs known as "security through obscurity." That stance has a long record of failure: the A5/1 cipher protecting GSM phone calls was kept secret for years, but once its design leaked and was analyzed in the open, it fell quickly (as we will dissect in Chapter 29). Open standards like AES earn trust only after thousands of independent cryptanalysts try to break them; today, the hardest frontier is defending open algorithms against hardware side channels like timing and power leaks.

Why it works

Why insist on this? Because secrets that are hard to change are secrets that will eventually leak and then cannot be repaired. An algorithm baked into a billion devices cannot be swapped overnight; a leaked key can be rotated in seconds. Kerckhoffs's principle pushes all the fragility into the one component that is cheap to replace. It also has a social benefit that turns out to be decisive: a public algorithm can be attacked by the whole world's cryptanalysts for years before you trust it. Every cipher in this book that you should actually use – AES, the SHA-2 and SHA-3 families, Curve25519 – earned that trust by surviving open, published assault. A secret design earns nothing, because no one outside could ever try to break it.

Common trap

"We use a proprietary, undisclosed encryption algorithm for extra security." Whenever you read this in a product's marketing, translate it as "no qualified outsider has ever checked our math." Secrecy of the design is not a second layer of protection on top of a strong cipher; historically it is a substitute for one, and a poor substitute at that. Extra obscurity is fine as a thin outer wrapper – it raises the cost of a casual attacker – but it must never be the thing your security rests on.

Don't roll your own crypto – and why this book seems to do exactly that

The second principle sounds like it contradicts the existence of this book: do not write your own cryptographic code for real use. We will write plenty of cryptographic code. The resolution is in the word *real*. Understanding a scheme well enough to implement a toy version is exactly what makes you competent to *use* the real one correctly – to pick the right mode, supply a proper nonce, check a tag before trusting a plaintext. But the gap between a textbook-correct implementation and a deployment-safe one is enormous, and much of that gap consists of side channels: timing differences, power draw, cache behavior, error messages that leak one bit at a time.

Attack

A textbook RSA decryption can be perfectly correct and still hand an attacker your private key, because the time it takes leaks information about the secret exponent. A textbook AES in CBC mode can be perfectly correct and still surrender the plaintext to a *padding oracle* – an attacker who learns only whether decryption "padded correctly" can peel off the message one byte at a time, without ever recovering the key. We will build these very attacks by hand in later chapters. They are the reason production crypto is constant-time, authenticated, and written by specialists. Once you have seen them, you are much less likely to ship them.

So implement everything here to learn it, and trust none of it in production. Understanding schemes deeply while staying cautious about deploying your own code is the attitude every `implementbox` in this book aims to build.

How the book is organized: from modular arithmetic to TLS, WiFi, and Signal

The book has five parts, each building on the ones before it, and it helps to see the overall plan before you start.

Part I – Foundations. We begin with the mathematics: modular arithmetic, primes and the greatest common divisor, groups, rings, fields, and the finite fields that the rest of the subject is built on. Then probability and a first taste of information theory, and the idea of computational hardness – the notion that some problems are easy to state and effectively impossible to solve at scale, which is the basis of all public-key cryptography. This part says little about secrets, but everything later depends on it, so take it slowly.

Part II – Symmetric cryptography. Alice and Bob already share a secret key; what can they do with it? We reach perfect secrecy and the one-time pad – the one scheme that is provably unbreakable, and a lovely lesson in why provably unbreakable is not the same as practical. Then stream ciphers, block ciphers (the Feistel construction, DES, and AES), the modes that turn a block cipher into something that can encrypt a whole message, hash functions, message authentication codes and HMAC, and finally authenticated encryption – the modern default that gives you confidentiality and integrity together, which it turns out you almost always need at the same time.

Part III – Public-key cryptography. Now we drop the assumption that Alice and Bob already share a key, and confront the question that launched modern cryptography: how do two strangers agree on a secret over a wire that an enemy is reading? Diffie–Hellman key exchange, RSA, the discrete logarithm and ElGamal, elliptic curves – the same hardness assumptions from Part I, now used to build working schemes. Digital signatures, key exchange with forward secrecy, and the certificates and public-key infrastructure (PKI) that let your browser decide whom to trust.

Part IV – Real systems. Here we study protocols you use every day. TLS, the protocol behind every `https`. The evolution of WiFi security up through WPA3. Signal's double ratchet, which gives messaging both forward secrecy and post-compromise recovery. SSH, the cryptography woven through the cellular network, VPNs and tunnels. These chapters are where every primitive from the earlier parts gets composed into a system – and where you learn that composing secure pieces does not automatically yield a secure whole.

Part V – The frontier and the craft. Zero-knowledge proofs, commitments and secret sharing, homomorphic encryption, multiparty computation, post-quantum cryptography, and the cryptography under blockchains. Then the practitioner's chapters that tie the whole book together: side-channel and timing attacks, the catalog of common pitfalls, and how to implement cryptography safely in the real world. Two appendices collect the number theory and algebra for reference.

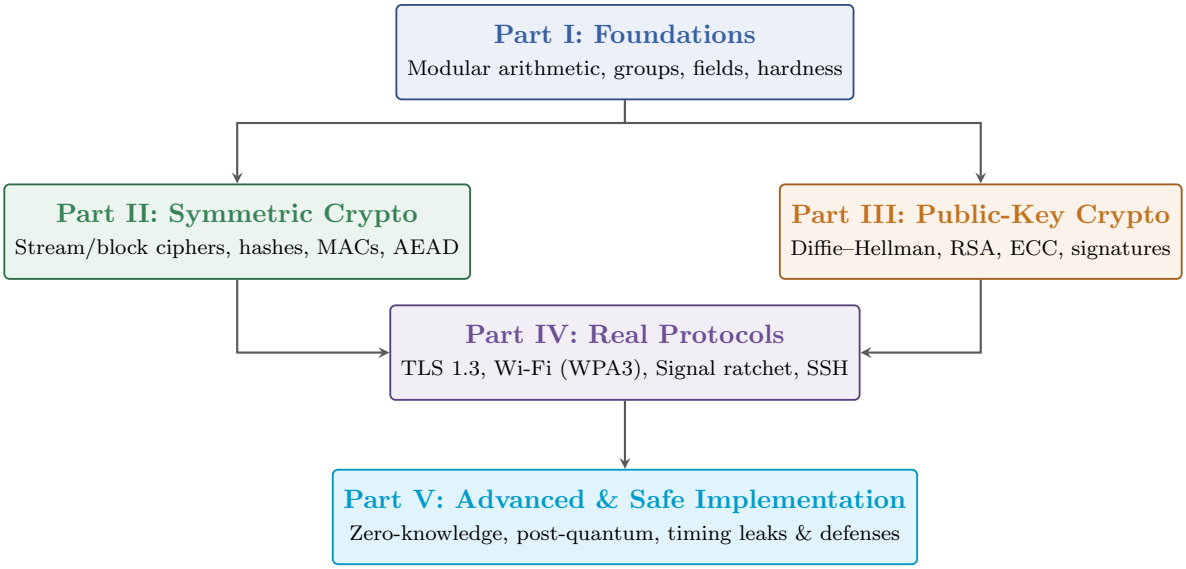
Connection

This is one of three companion ground-up readers. Where we need the deep version of a probabilistic argument – the precise birthday bound behind hash collisions, the analysis of a randomized primality test – the probability book carries the full treatment. Where we need the cost of an algorithm – how many multiplications modular exponentiation really takes, why fast multiplication matters for big keys – the algorithms book has it. Cryptography sits exactly at the meeting point of "what is hard to compute" and "what is likely by chance," so you will see those pointers often.

Focus

One theme runs through the whole book: the move from *information-theoretic* security (the one-time pad, secure even against an adversary with infinite computing power, but crippled by the need for a key as long as the message) to *computational* security (everything else, secure only against adversaries with realistic resources, but practical enough to run the internet). Almost all of modern cryptography lives in that second world, and almost all of its subtlety comes from making "realistic resources" precise. Keep that distinction in mind throughout.

The dependency map below sketches how the parts lean on one another. You can take many paths through it, but no path skips the foundations.



One worked example, so you know what "by hand" means

To set expectations, here is the flavor of the small-number work you will do throughout, using a piece of arithmetic that powers nearly every public-key scheme: modular exponentiation, computed the smart way.

Suppose we want $7^{13} \bmod 23$. The naive route – multiply seven by itself thirteen times, then reduce – works but scales terribly; with the thousand-bit exponents of real cryptography it would never finish. The trick is *square-and-multiply*. Write the exponent in binary: $13 = 1101_2 = 8+4+1$. Then

$$7^{13} = 7^8 \cdot 7^4 \cdot 7^1,$$

and we get the high powers by repeated squaring, reducing mod 23 at every step so the numbers never grow:

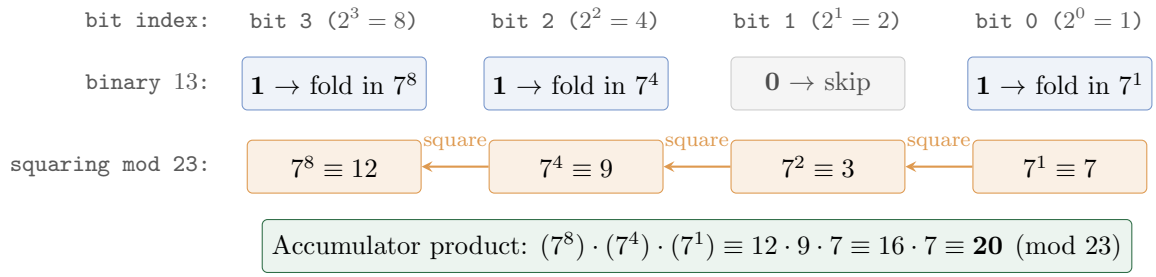
$$7^1 \equiv 7, \qquad 7^2 \equiv 49 \equiv 3, \qquad 7^4 \equiv 3^2 = 9, \qquad 7^8 \equiv 9^2 = 81 \equiv 81 - 69 = 12 \pmod{23}.$$

Now multiply together the pieces the binary digits selected (the 8, the 4, and the 1):

$$7^{13} \equiv 7^8 \cdot 7^4 \cdot 7^1 \equiv 12 \cdot 9 \cdot 7 \pmod{23}.$$

Take it in steps: $12 \cdot 9 = 108 \equiv 108 - 92 = 16$, and $16 \cdot 7 = 112 \equiv 112 - 92 = 20$. So

$$7^{13} \equiv 20 \pmod{23}.$$



Four squarings and two multiplications replace twelve sequential multiplies. That logarithmic growth makes 2048-bit modular exponentiations run in a matter of milliseconds.

Implement it

The same algorithm, written to be read rather than deployed. Python already has a fast, safe `pow(base, exp, mod)` built in – this version exists only to make the squaring visible.

```
def modexp(base, exp, mod):
    # Square-and-multiply. LEARNING ONLY:
    # not constant-time -- leaks the exponent's bits via timing.
    result = 1
    base = base % mod
    while exp > 0:
        if exp & 1:                    # low bit is 1 -> fold in this power
            result = (result * base) % mod
        base = (base * base) % mod     # square for the next bit
        exp >>= 1                      # move to the next bit of the exponent
    return result

print(modexp(7, 13, 23))   # -> 20
```

Take the comment seriously. The branch `if exp & 1` runs a multiplication only when the exponent bit is set, so the running time depends on the secret exponent – exactly the leak that lets an attacker recover a private key by timing. Real libraries spend serious effort erasing that dependence. This is your first encounter with the gap between "correct" and "safe," and it will not be your last.

Connection

Why is square-and-multiply the right algorithm and not merely a clever one? Because it runs in time proportional to the number of *bits* in the exponent, not its magnitude – logarithmic in the exponent itself. The companion algorithms reader develops this cost model carefully and explains why that logarithmic gap is the difference between cryptography that runs in milliseconds and cryptography that never finishes.

That single operation – raise to a power, reduce mod n – is nearly all of the computation in RSA. To make the point concrete, and to show you that "by hand" really does mean by hand, here is a

complete RSA encryption and decryption on numbers small enough to check on a napkin. Take the two primes the running RSA example will use, $p = 61$ and $q = 53$. Then

$$n = pq = 61 \cdot 53 = 3233, \qquad \varphi(n) = (p-1)(q-1) = 60 \cdot 52 = 3120,$$

where $\varphi(n)$ counts the integers below n that share no factor with it – the size of the group we will be exponentiating in. Pick a public exponent $e = 17$, which is coprime to 3120. The private exponent d is the modular inverse of e modulo $\varphi(n)$, the number with $e\,d \equiv 1 \pmod{3120}$; the extended Euclidean algorithm (which we develop in full in Part I) returns $d = 2753$. You can check the defining relation directly: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$, so $17 \cdot 2753 \equiv 1 \pmod{3120}$ as required.

Now encrypt the message $m = 65$ (think of it as a single byte). The ciphertext is

$$c \equiv m^e \equiv 65^{17} \pmod{3233},$$

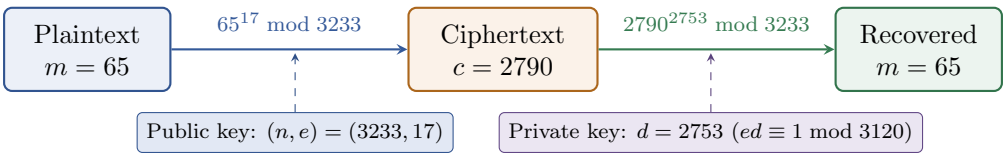
which is exactly a square-and-multiply away. Writing $17 = 10001_2 = 16 + 1$ and squaring mod 3233:

$$65^1 \equiv 65, \quad 65^2 \equiv 4225 \equiv 992, \quad 65^4 \equiv 992^2 = 984064 \equiv 1232, \quad 65^8 \equiv 1232^2 = 1517824 \equiv 1547,$$

and one more squaring gives $65^{16} \equiv 1547^2 = 2393209 \equiv 789 \pmod{3233}$. The exponent 17 selects the 16 and the 1, so

$$c \equiv 65^{16} \cdot 65^1 \equiv 789 \cdot 65 = 51285 \equiv 2790 \pmod{3233}.$$

To decrypt, raise the ciphertext to the private exponent: $m \equiv c^d \equiv 2790^{2753} \pmod{3233}$. By the same square-and-multiply (now over twelve bits) this returns $2790^{2753} \equiv 65$, recovering the original byte. The round trip below puts the whole toy RSA cycle on one line: the public key (n, e) locks the message, and only the modular inverse $d \equiv e^{-1} \pmod{\varphi(n)}$ unlocks it.



The reason it inverts – why $(m^e)^d \equiv m$ – is Euler's theorem, and establishing that guarantee is the central goal of Chapter 19. For now, note that encryption and decryption both used only the operation you just learned.

Common trap

The numbers above are small enough to factor by trial division in well under a second: anyone who sees $n = 3233$ can recover $p = 61$ and $q = 53$, hence $\varphi(n)$, hence d, and read every message. This does not mean RSA is flawed: its security rests entirely on n being too large to factor – real keys make n a number with *six hundred or more* decimal digits. A common beginner error is to admire a scheme working on toy numbers and conclude it is therefore secure. Toy numbers prove correctness, never security. Keep the two questions – "does it compute the right answer?" and "can an adversary undo it?" – rigidly separate in your head; conflating them is the first mistake the security mindset trains out of you.

What it takes to be an entry-level cryptographer

People imagine the job needs a doctorate and a gift for inventing new schemes. A few roles do, but most do not. The entry-level work – and it is real, well-paid, and in demand – is using cryptography correctly: choosing primitives, reading specifications, reviewing designs for the failure modes that recur endlessly, integrating vetted libraries without undoing their guarantees. That work rests on a surprisingly finite base of understanding, and assembling that base is precisely what this book is for.

In the real world

What this looks like on the job: a protocol engineer deciding that a new service should use TLS 1.3 with X25519 key exchange and AES-256-GCM, and being able to defend each choice. A reviewer catching that a teammate reused a GCM nonce across two messages – a one-line bug that collapses the whole scheme's integrity. An embedded developer noticing that a smartcard's signing routine branches on secret key bits and is therefore leaking through timing. None of these require inventing new mathematics. All of them require understanding the mathematics that already exists well enough to see where it is being misused.

Pause

A fair self-test for "ready for entry-level work," which doubles as a map of where you are headed. By the end you should be able to, without notes:

- state Kerckhoffs's principle and explain why a public algorithm is a strength, not a weakness;
- compute a small RSA encryption and decryption by hand and explain why decryption inverts encryption;
- explain the difference between confidentiality and integrity, and why encryption alone gives you neither against an active attacker;
- describe a Diffie–Hellman exchange and say precisely what an eavesdropper does and does not learn from the wire;
- name three real-world attacks (a padding oracle, a nonce reuse, a timing leak) and sketch how each one works;
- read "AES-256-GCM with a random 96-bit nonce" and say what every part of that means and why each part is there;
- explain why you should reach for a vetted library and never the toy code in this book.

If those feel far off right now, that is expected; the book is designed to cover that distance. None of them require talent you do not have; they require the chapters ahead, worked rather than skimmed.

A last word on how to use the book well. *Do the arithmetic.* When a worked example traces $7^{13} \bmod 23$, get a pen and follow it; when an exercise asks you to break a weakened scheme, actually break it. Cryptography punishes the reader who nods along – the understanding that lets you spot a nonce-reuse bug in someone else's protocol is built only by having reused a nonce yourself, on small numbers, and watched the secret fall out. Run the toy code. Trace the attacks. Work through the security proofs step by step until they stop feeling like formal exercises and start reading like plain

common sense.

Every chapter ends with exercises – some to compute, some to break, some to prove. Do them; much of the learning happens there. The compute problems build fluency, the break problems build the adversarial reflex that defines the field, and the prove problems build the precision that separates a guarantee from a hope. To set the tone, the preface has a few of its own. None require machinery we have not already shown; all reward a pen and ten unhurried minutes.

Exercises

Exercise 0.1 (Square-and-multiply by hand)**.** Compute $5^{20} \bmod 23$ using square-and-multiply, reducing mod 23 at every step. Write 20 in binary first, list the squarings $5^1, 5^2, 5^4, 5^8, 5^{16}$ each reduced mod 23, then multiply together only the powers the binary digits of 20 select. How many squarings and how many multiplications did you use, and how does that compare to the nineteen multiplications of the naive method?

Exercise 0.2 (Finish the worked RSA decryption)**.** Using the toy key $n = 3233$, $d = 2753$ from the chapter, verify that the ciphertext $c = 2790$ decrypts to $m = 65$. Do it with square-and-multiply on $2790^{2753} \bmod 3233$: write 2753 in binary, build the squarings $2790^{2^k} \bmod 3233$, and combine the selected ones. (If you have a computer handy, also confirm with `pow(2790, 2753, 3233)` – but do the first few squaring steps by hand so the mechanism is yours.)

Exercise 0.3 (Break the toy key)**.** The public key in the chapter is $(n, e) = (3233, 17)$. Playing the attacker, factor n by trial division, recover p and q, compute $\varphi(n)$, and reconstruct the private exponent d as the inverse of e modulo $\varphi(n)$. Confirm you get $d = 2753$. Then explain in one sentence why repeating this attack is hopeless when n has six hundred digits rather than four.

Exercise 0.4 (Scaling up modular exponentiation)**.** In computing $7^{13} \bmod 23$, we decomposed the exponent 13 into binary (1101_2) and performed 4 squarings and 2 multiplications. Suppose an exponent has 2048 bits, with roughly half of its bits set to 1. Approximately how many squarings and multiplications would square-and-multiply require? Contrast this with the naive method of multiplying the base by itself 2^{2048} times.

Exercise 0.5 (Eavesdroppers versus active adversaries)**.** Alice sends an email to Bob over open coffee-shop WiFi without encryption. Eve captures the raw radio packets. Mallory runs a rogue WiFi access point that intercepts, alters, and drops packets. For each of the following goals, state whether Eve alone can achieve it, Mallory alone can achieve it, or both: (a) reading the body of Alice's email, (b) changing the bank routing number inside the email, (c) preventing the email from reaching Bob altogether.

Exercise 0.6 (Kerckhoffs in the wild)**.** Find a real product or protocol whose marketing claims security comes partly from a secret or proprietary algorithm. State, in the language of this preface, exactly which assumption its security is resting on, and describe the single event (a leak, a reverse-engineering, a disgruntled employee) that would collapse it. Contrast this with what a leaked *key* would cost a Kerckhoffs-compliant design.

Exercise 0.7 (Confidentiality is not integrity)**.** A naive scheme encrypts a one-byte message m as $c = m \oplus k$ for a secret one-byte key k (a one-time pad on a single byte). Eve cannot read m from c without k. But Mallory, who controls the wire, wants to flip the message from the byte `0x00` ("no") to `0xFF` ("yes") without knowing k or m. Show exactly what Mallory XORs into c to do this, and

argue that she succeeds with certainty. This is your first taste of why confidentiality alone buys you nothing against an active attacker – and why authenticated encryption exists.

Exercise 0.8 (A timing leak you can see)**.** Look again at the `modexp` code in this chapter. Suppose the exponent is the secret. Argue informally that two exponents with different numbers of 1-bits will, on average, cause the function to run for measurably different lengths of time. Sketch what an attacker who can time many decryptions might learn about the secret exponent's bits. (You are not expected to mount the attack – only to see why the door is open.)

Cryptography makes a bold mathematical claim: that two people who have never met can communicate privately across a wire the whole world is listening to, and keep their secret even from an adversary who knows everything about the method and has read every word of this book. In Chapter 1 we start by pinning down the ground rules: who the players are, what attack models govern real systems, and what "secure" is actually allowed to mean.

Contents

Preface **iii**

I Foundations and the Mathematics **1**

1 What Cryptography Is **3**

1.1 Four Goals: What Are We Actually Protecting? . . . 3
1.2 What Cryptography Is Not . . . 5
1.3 A Tiny Cipher, and Why It Already Fails . . . 6
1.4 The Cast: Alice, Bob, Eve, and Mallory . . . 7
1.5 Kerckhoffs's Principle: No Security Through Obscurity . . . 9
1.6 Threat Models: Secure Against Whom, Given What? . . . 10
1.7 Randomness: The Resource Everything Rests On . . . 12
1.8 Symmetric vs. Asymmetric: A First Look . . . 13
1.9 What Does "Secure" Even Mean? . . . 15
1.10 The Security Mindset . . . 19
1.11 Composing the Goals: A Single Message, Several Guarantees . . . 20
1.12 What's Ahead . . . 22
1.13 Exercises . . . 22

2 A Short History of Secret Writing **25**

2.1 The cast and the rules of the game . . . 25
2.2 The Caesar cipher: shift and pray . . . 26
2.2.1 Why it fails: there is nowhere for the secret to hide . . . 27
2.3 General substitution: a huge key that still loses . . . 28
2.3.1 Frequency analysis: the key space is irrelevant if the cipher leaks . . . 28

2.3.2 Breaking one by hand . . . 29
2.4 The Vigenere cipher: stacking shifts to flatten the fingerprint . . . 31
2.4.1 How it works . . . 31
2.4.2 Kasiski and Babbage: finding the key length . . . 32
2.4.3 The Friedman test: counting coincidences . . . 34
2.5 Rotor machines and Enigma: mechanizing the substitution . . . 35
2.5.1 What the machine does . . . 36
2.5.2 The flaws that sank it . . . 37
2.5.3 A tiny rotor by hand . . . 38
2.6 What three thousand years of failure teaches . . . 39
Exercises . . . 41

3 Modular Arithmetic 43
3.1 The Clock Already Taught You This . . . 43
3.2 The Integers Modulo n . . . 44
3.2.1 Congruence: the equality of modular arithmetic . . . 44
3.2.2 Congruence is an equivalence relation . . . 45
3.3 Addition, Subtraction, Multiplication All Survive Wrap-around . . . 45
3.3.1 Addition and multiplication tables for a tiny modulus . . . 46
3.4 Where Division Goes Wrong . . . 47
3.4.1 Zero divisors . . . 47
3.5 Modular Inverses and Division . . . 48
3.5.1 The group of units . . . 48
3.5.2 Computing an inverse by hand . . . 49
3.6 Modular Exponentiation . . . 50
3.6.1 Order, and why the powers cycle . . . 50
3.6.2 Why you cannot just multiply k times . . . 51
3.6.3 Square-and-multiply . . . 51
3.6.4 The cost, precisely . . . 52
3.6.5 Toy implementation . . . 52
3.7 Solving Linear Congruences . . . 53
3.8 A Worked Mini-Cipher: Affine Encryption . . . 54
3.9 Negative Numbers, and a Useful Symmetric View . . . 56

3.10 Why This Underlies Cryptography . . . 56
3.11 Exercises . . . 57

4 Number Theory: Primes, GCD, and Euclid 59
4.1 Divisibility . . . 59
4.2 Primes and the Fundamental Theorem . . . 60
4.3 Greatest Common Divisor and the Euclidean Algorithm . . . 62
4.4 Congruences and the Units of $\mathbb{Z}/n\mathbb{Z}$ . . . 66
4.5 Fermat's Little Theorem and Euler's Theorem . . . 67
4.6 The Chinese Remainder Theorem . . . 68
4.7 Primitive Roots . . . 71
4.8 Putting It Together: A Tiny RSA by Hand . . . 73
4.9 Exercises . . . 75

5 Groups, Rings, and Fields 77
5.1 Why crypto needs algebra at all . . . 77
5.2 Groups . . . 78
5.2.1 The definition, and why each axiom earns its place . . . 78
5.2.2 The two groups you already know . . . 78
5.2.3 Group notation: additive vs. multiplicative . . . 79
5.3 Order of an element, and cyclic groups . . . 79
5.3.1 Order of an element . . . 79
5.3.2 Cyclic groups and generators . . . 80
5.3.3 Not every group is cyclic . . . 82
5.4 Subgroups and Lagrange's theorem . . . 82
5.4.1 Subgroups . . . 82
5.4.2 Lagrange's theorem . . . 82
5.4.3 The consequences crypto actually uses . . . 83
5.5 From one operation to two: rings . . . 84
5.5.1 Zero divisors: the wrinkle that composite moduli introduce . . . 85
5.6 Fields: rings with division . . . 85
5.7 A worked tour: the structures inside $\mathbb{Z}/n\mathbb{Z}$ . . . 86
5.8 Toy code: groups by hand . . . 87

5.9 A pitfall: small subgroups and where Lagrange bites back . . . 88
5.10 How the three structures power the rest of the book . . . 89
5.11 Exercises . . . 89

6 Finite Fields **91**
6.1 What a Field Is, Quickly . . . 91
6.1.1 The first family: $\mathbb{F}_p$ . . . 92
6.2 Why Prime Powers, and Nothing Else . . . 92
6.3 Polynomials over a Field . . . 93
6.3.1 Irreducibility with small numbers . . . 94
6.4 Building $\mathbb{F}_4$ by Hand . . . 94
6.5 The Field AES Uses: $\mathbb{F}_{2^8}$ . . . 96
6.5.1 Addition is free . . . 97
6.5.2 Multiplication, worked by hand . . . 98
6.5.3 The doubling trick: `xtime` . . . 99
6.5.4 Finding inverses in $\mathbb{F}_{2^8}$ . . . 100
6.6 The Multiplicative Group Is Cyclic . . . 101
6.6.1 Why AES lives in $\mathbb{F}_{2^8}$, summarized . . . 103
6.7 The Frobenius Map and Square Roots . . . 103
6.8 Summary . . . 104
Exercises . . . 105

7 Probability and Information for Cryptography **107**
7.1 Why probability is the native language of secrecy . . . 107
7.1.1 Sample spaces, events, and uniform distributions . . . 108
7.1.2 Conditional probability and Bayes . . . 108
7.1.3 Independence . . . 109
7.2 Random variables and expectation . . . 110
7.2.1 The indicator trick . . . 110
7.3 Entropy: measuring unpredictability on average . . . 110
7.3.1 The definition and where it comes from . . . 111
7.3.2 Worked entropies on tiny distributions . . . 111
7.3.3 The properties that matter for crypto . . . 112

7.4 Min-entropy: the paranoid measure . . . 113
7.4.1 Guessing entropy: how many tries on average . . . 114
7.5 The birthday paradox and collision security . . . 115
7.5.1 Why 23 is enough . . . 115
7.5.2 The clean bound . . . 116
7.5.3 Counting collisions with the indicator trick . . . 117
7.5.4 What this dictates for hash functions . . . 117
7.6 Statistical distance and indistinguishability . . . 119
7.6.1 Definition and the guessing-game meaning . . . 119
7.6.2 A worked distance . . . 120
7.6.3 From statistical to computational indistinguishability . . . 120
7.6.4 The data-processing inequality for distance . . . 121
7.7 Putting the pieces together . . . 122
7.8 Exercises . . . 122

8 Computational Hardness and One-Way Functions 125
8.1 The asymmetry we are buying . . . 125
8.2 Measuring effort: input size and asymptotics . . . 126
8.3 The adversary: probabilistic polynomial time . . . 128
8.4 Negligible vs. noticeable . . . 129
8.5 Advantage: how much better than guessing . . . 130
8.6 One-way functions . . . 131
8.6.1 Candidate one-way functions . . . 132
8.7 Trapdoors: a hidden shortcut . . . 134
8.8 Reductions: turning hardness into security . . . 136
8.9 Standard hardness assumptions . . . 138
8.10 Putting the pieces together . . . 140
8.11 Exercises . . . 140

II Symmetric Cryptography 143

9 Perfect Secrecy and the One-Time Pad 145
9.1 The setting: Alice, Bob, Eve, and a shared secret . . . 145

9.2 What should "secret" mean? Three tries . . . 146
9.2.1 An equivalent and easier-to-check form . . . 147
9.3 XOR: the operation that makes pads work . . . 148
9.4 The one-time pad . . . 148
9.4.1 A tiny worked example . . . 149
9.4.2 The proof of perfect secrecy . . . 150
9.5 Why the key must be long, random, and fresh . . . 152
9.5.1 Random: a biased pad leaks . . . 152
9.5.2 Long: a short key cannot cover the messages . . . 152
9.6 Shannon's theorem: perfect secrecy forces a huge key . . . 152
9.7 The two-time pad: reuse is fatal . . . 154
9.8 Why we settle for computational security . . . 156
9.9 What to remember . . . 158
Exercises . . . 158

10 Stream Ciphers **161**
10.1 From the one-time pad to a generator . . . 161
10.1.1 Why this cannot be perfectly secret . . . 162
10.2 What we demand of a PRG . . . 163
10.2.1 Next-bit unpredictability . . . 163
10.2.2 Security of the stream cipher, granted a good PRG . . . 164
10.3 Keystream reuse: the two-time pad disaster . . . 165
10.4 RC4: simple, ubiquitous, and broken . . . 166
10.4.1 Why RC4 is broken . . . 168
10.5 ChaCha20: the modern stream cipher . . . 168
10.5.1 The state and the quarter-round . . . 169
10.5.2 From core to cipher . . . 170
10.5.3 Why ChaCha20 is trusted . . . 171
10.6 Linear feedback shift registers and the curse of linearity . . . 172
10.6.1 A tiny LFSR by hand . . . 172
10.6.2 Why linearity is fatal . . . 174
10.6.3 Trying to rescue LFSRs . . . 175
10.7 Summary . . . 175

10.8 Exercises . . . 176

11 Block Ciphers, Feistel Networks, and DES **179**

11.1 What a Block Cipher Is . . . 179

11.1.1 Why "pseudorandom permutation" is the right goal . . . 180

11.2 Confusion and Diffusion . . . 181

11.2.1 The avalanche criterion . . . 181

11.3 The Feistel Construction . . . 182

11.3.1 One Feistel round . . . 182

11.3.2 How secure is a Feistel network? The Luby–Rackoff theorem . . . 183

11.4 A Toy Feistel Cipher, Worked by Hand . . . 184

11.4.1 Watching the avalanche fail (and then start) . . . 186

11.5 DES: The Data Encryption Standard . . . 187

11.5.1 The shape of DES . . . 187

11.5.2 The DES round function f . . . 188

11.5.3 Inside an S-box . . . 188

11.5.4 The key schedule . . . 189

11.5.5 A first taste of differential cryptanalysis . . . 190

11.6 Why DES Died: 56 Bits Is Not Enough . . . 191

11.6.1 The natural fix that mostly fails: Double DES . . . 192

11.7 What to Carry Forward . . . 194

Exercises . . . 194

12 The Advanced Encryption Standard (AES) **197**

12.1 Where AES came from . . . 197

12.2 The state: bytes in a 4×4 grid . . . 198

12.2.1 Number of rounds . . . 199

12.3 The field $\mathbb{F}_{256}$: arithmetic on bytes . . . 200

12.4 SubBytes: the only nonlinear layer . . . 201

12.5 ShiftRows: cheap cross-column movement . . . 203

12.6 MixColumns: diffusion inside a column . . . 204

12.7 AddRoundKey: where the key finally enters . . . 206

12.8 The key schedule . . . 206

12.9 A full round, traced on bytes . . . 208
12.10 Decryption: inverting each layer . . . 210
12.11 Why AES is secure . . . 210
12.12 Performance and where AES runs . . . 212
12.13 Putting the whole cipher together . . . 213
12.14 Exercises . . . 213

13 Modes of Operation **215**

13.1 The block cipher as a tool, and what it is not . . . 215
13.1.1 The security target: IND-CPA . . . 216
13.2 ECB: the mode you must never use . . . 216
13.2.1 A tiny ECB trace . . . 217
13.2.2 The penguin . . . 218
13.3 CBC: chaining blocks with feedback . . . 218
13.3.1 A tiny CBC trace . . . 219
13.3.2 Why the IV must be unpredictable . . . 220
13.3.3 CBC malleability and the bit-flipping attack . . . 221
13.3.4 Sequential, and the padding problem . . . 221
13.4 The padding-oracle attack (a preview) . . . 222
13.5 CTR: turn the block cipher into a stream cipher . . . 223
13.5.1 A tiny CTR trace . . . 225
13.5.2 CTR's strengths and its nonce-uniqueness rule . . . 225
13.5.3 A tiny two-time-pad example . . . 226
13.6 CFB and OFB: the older stream-cipher modes . . . 226
13.7 Putting the modes side by side . . . 228
13.8 Toy code: ECB versus CBC versus CTR . . . 228
13.9 Where the modes actually live . . . 229
13.10 Summary . . . 230
13.11 Exercises . . . 231

14 Hash Functions **233**

14.1 What a hash function is, and what it must promise . . . 233
14.1.1 The random-oracle ideal . . . 235

14.2 The birthday bound: why n-bit hashes give $n/2$-bit collision security . . . 235
14.3 Building a hash from a small component: Merkle–Damgard . . . 237
14.4 Length extension: the Merkle–Damgard wart . . . 239
14.5 The sponge construction and SHA-3 . . . 241
14.6 SHA-2 in practice . . . 243
14.7 Application 1: fingerprints and integrity . . . 244
14.7.1 Merkle trees: fingerprinting a collection . . . 244
14.8 Application 2: commitments . . . 245
14.9 Password Hashing, Memory-Hard Key Derivation, and Phishing-Resistant Authentication . . . 246
14.9.1 Why raw hashes and unsalted hashes fail . . . 246
14.9.2 Iterative CPU-hard stretching and its silicon limits . . . 247
14.9.3 Memory-hard key derivation: scrypt and Argon2 . . . 248
14.9.4 Origin-bound phishing-resistant authentication: WebAuthn and passkeys . . . 251
14.10 The real-world status of named hashes . . . 254
14.11 Exercises . . . 255

15 Message Authentication Codes and HMAC 257

15.1 Confidentiality Is Not Integrity . . . 257
15.2 What a MAC Is, Formally . . . 258
15.3 What “Secure” Means: Unforgeability . . . 259
15.3.1 A pseudorandom function is already a MAC . . . 260
15.4 CBC-MAC: A MAC From a Block Cipher . . . 261
15.4.1 Working it by hand with a 4-bit toy cipher . . . 262
15.4.2 The length-extension forgery on raw CBC-MAC . . . 262
15.4.3 Fixing CBC-MAC . . . 263
15.5 Why You Cannot Just MAC With a Hash . . . 263
15.5.1 The length extension, worked on a toy hash . . . 265
15.5.2 Does secret-suffix help? Partly, and not enough . . . 266
15.6 HMAC: The Right Way to MAC With a Hash . . . 266
15.6.1 A tiny HMAC by hand . . . 268
15.7 Combining Encryption and Authentication . . . 269
15.7.1 Why Encrypt-then-MAC wins . . . 270

15.7.2 Why the other orders are dangerous . . . 271
15.8 Verifying Tags in Constant Time . . . 272
15.9 Putting It Together . . . 274
Exercises . . . 274

16 Authenticated Encryption **277**
16.1 Why Confidentiality Without Integrity Is a Trap . . . 277
16.2 What "Secure" Means Here: AE and AEAD . . . 279
16.3 Three Ways to Glue, Two of Them Broken . . . 281
16.4 AES-GCM: Counter Mode Meets a Polynomial Hash . . . 283
16.4.1 The confidentiality half: CTR mode . . . 283
16.4.2 The integrity half: GHASH over $\mathbb{F}_{2^{128}}$ . . . 283
16.4.3 A worked GHASH in a toy field . . . 284
16.5 The Nonce-Reuse Catastrophe in GCM . . . 286
16.5.1 Why a repeat is so much worse than for a plain stream cipher . . . 286
16.6 ChaCha20-Poly1305: The Software-Friendly AEAD . . . 288
16.7 Choosing and Using AEAD in the Real World . . . 291
Exercises . . . 292

17 Randomness and CSPRNGs **295**
17.1 Why Cryptography Lives or Dies on Randomness . . . 295
17.2 True Randomness versus Pseudorandomness . . . 296
17.3 What Makes a Generator *Cryptographically* Secure . . . 298
17.4 Where Entropy Actually Comes From . . . 300
17.5 Seeding, Reseeding, and the OS CSPRNG . . . 302
17.5.1 Fork Safety: When One Becomes Two . . . 303
17.5.2 Health Tests: Catching a Source That Dies . . . 305
17.6 Famous RNG Failures . . . 305
17.6.1 Debian OpenSSL (2008): The 15-Bit Keyspace . . . 305
17.6.2 Dual_EC_DRBG (c. 2006–2013): A Standardized Backdoor . . . 306
17.6.3 Repeated Nonces in Digital Signatures: The Secret Key Falls Out . . . 308
17.7 Getting Randomness Right on a Real System . . . 310
17.8 Exercises . . . 311

III Public-Key Cryptography 315

18 The Public-Key Idea and Diffie-Hellman 317

18.1 The key-distribution problem . . . 318
18.2 The intuition: mixing paint . . . 319
18.3 Diffie–Hellman key exchange . . . 320
18.3.1 A worked example with tiny numbers . . . 320
18.4 What "secure" means here: three hardness assumptions . . . 323
18.4.1 From shared secret to usable key . . . 324
18.5 The man-in-the-middle problem . . . 324
18.6 Ephemeral Diffie–Hellman and forward secrecy . . . 326
18.7 Looking ahead . . . 328
18.8 Exercises . . . 328

19 RSA 331

19.1 The setup: a one-way operation with a trapdoor . . . 331
19.2 The number theory you need . . . 332
19.3 Key generation . . . 333
19.4 Encryption, decryption, and why it works . . . 334
19.4.1 A miniature toy trace: $p = 5$, $q = 11$, $n = 55$ . . . 334
19.5 A full worked example: $p = 37$, $q = 43$ . . . 335
19.6 The factoring assumption . . . 338
19.7 Why textbook RSA is broken . . . 339
19.8 Padding done right: OAEP . . . 339
19.9 Implementation mistakes that have sunk real systems . . . 341
19.10 The other half: RSA signatures . . . 342
19.11 RSA in the real world . . . 343
19.12 Exercises . . . 344

20 The Discrete Logarithm Problem and ElGamal 347

20.1 The discrete logarithm, precisely . . . 347
20.1.1 The concrete case: $(\mathbb{Z}/p\mathbb{Z})^*$ . . . 348
20.1.2 Why we believe it is hard . . . 348
20.2 Attacking the DLP: generic algorithms . . . 349

20.2.1 Baby-step giant-step . . . 349
20.2.2 Pollard's rho: square-root time, constant memory . . . 351
20.3 From hard problem to encryption: ElGamal . . . 354
20.3.1 The scheme . . . 354
20.3.2 Why it is secure . . . 355
20.3.3 The homomorphic property . . . 356
20.4 Choosing safe groups and parameters . . . 358
20.5 Relationship to Diffie-Hellman . . . 360
20.6 Capabilities and Limitations of ElGamal . . . 360
20.7 Exercises . . . 361

21 Elliptic Curve Cryptography 363
21.1 From a multiplicative group to a curve . . . 363
21.2 The curve and its points . . . 364
21.2.1 The point at infinity . . . 365
21.3 The group law: geometry first . . . 365
21.4 The group law: algebra . . . 367
21.4.1 Projective coordinates: eliminating modular inversions . . . 369
21.5 Curves over a finite field . . . 370
21.5.1 How many points? Hasse's theorem . . . 372
21.6 Scalar multiplication and the ECDLP . . . 372
21.6.1 Why ECC keys are small: generic versus special attacks . . . 373
21.6.2 When the security collapses: pitfalls . . . 374
21.7 Elliptic-Curve Diffie–Hellman (ECDH) . . . 375
21.8 Curve25519 and X25519 . . . 377
21.8.1 Edwards curves and complete addition formulas . . . 378
21.9 Why ECC dominates: the summary . . . 380
21.10Exercises . . . 381

22 Digital Signatures 383
22.1 Signatures Versus MACs . . . 383
22.1.1 The syntax of a signature scheme . . . 384
22.1.2 What "secure" means: existential unforgeability . . . 385

22.2 RSA Signatures . . . 385
22.2.1 Textbook RSA, and why it is broken . . . 386
22.2.2 Hash-then-sign: the fix . . . 387
22.3 Discrete-Log Signatures: DSA . . . 389
22.3.1 Setup and the algorithm . . . 389
22.4 ECDSA: DSA on an Elliptic Curve . . . 390
22.5 The Catastrophic Nonce Attack . . . 392
22.5.1 Reusing one nonce reveals the key . . . 392
22.5.2 The fix: deterministic nonces (RFC 6979) . . . 394
22.6 EdDSA and Ed25519 . . . 395
22.6.1 How Ed25519 signs . . . 395
22.7 What Signatures Guarantee – and What They Do Not . . . 397
22.8 Putting the Family Together . . . 398
22.9 Exercises . . . 399

23 Key Exchange and Forward Secrecy 401
23.1 The threat we are defending against . . . 401
23.2 Recalling the bare handshake, and watching it fail . . . 402
23.3 Authenticating the handshake . . . 403
23.3.1 Signed ephemeral Diffie-Hellman . . . 403
23.3.2 The foundation: ephemeral keys . . . 404
23.3.3 Implicit authentication: long-term DH keys . . . 405
23.4 Forward secrecy . . . 406
23.4.1 Why ephemeral DH gives it, and static encryption does not . . . 406
23.4.2 A worked tiny example . . . 408
23.5 From one group element to many keys: KDFs . . . 408
23.6 Replay and downgrade . . . 410
23.6.1 Replay . . . 411
23.6.2 Downgrade . . . 411
23.7 Putting it together: the shape of TLS 1.3 and Signal . . . 412
23.7.1 TLS 1.3 at a glance . . . 412
23.7.2 Signal at a glance . . . 413
23.8 Exercises . . . 414

IV Protocols and Applications 415

24 Certificates and Public-Key Infrastructure 417

24.1 The binding problem . . . 417
24.1.1 Three ways to bind a name to a key . . . 418
24.2 A certificate is a signed statement . . . 419
24.2.1 A toy certificate with tiny numbers . . . 419
24.3 Inside an X.509 certificate . . . 420
24.4 Chains of trust . . . 421
24.4.1 Tracing a tiny chain by hand . . . 423
24.5 Revocation: taking it back . . . 423
24.5.1 Certificate Revocation Lists (CRLs) . . . 423
24.5.2 OCSP: ask about one certificate . . . 424
24.5.3 OCSP stapling: bring your own proof . . . 424
24.6 When the trust breaks: CA mis-issuance . . . 425
24.6.1 Partial fixes before transparency . . . 427
24.7 Certificate Transparency . . . 427
24.7.1 Merkle trees: proving membership and append-only-ness . . . 427
24.7.2 How CT plugs into TLS . . . 429
24.8 Let's Encrypt and ACME: automating trust . . . 429
24.8.1 Domain validation: proving you control the name . . . 429
24.9 The alternative: web of trust . . . 431
24.10Putting it together: what your browser does . . . 432
Exercises . . . 433

25 TLS: Securing the Web 435

25.1 What TLS provides, and against whom . . . 435
25.2 The shape of a handshake . . . 436
25.3 Step 1: key exchange (the ClientHello) . . . 437
25.3.1 A worked exchange with tiny numbers . . . 438
25.4 Step 2: the ServerHello and the handshake key . . . 438
25.5 Step 3: the key schedule . . . 439
25.6 Step 4: authentication (Certificate and CertificateVerify) . . . 441

25.7 Step 5: Finished (the MAC that seals the handshake) . . . 442
25.8 The record protocol and AEAD . . . 443
25.8.1 Nonces and the sequence number . . . 443
25.8.2 What the record header authenticates . . . 443
25.9 Forward secrecy in TLS 1.3 . . . 444
25.10 Resumption and 0-RTT (and its one sharp edge) . . . 445
25.11 Famous TLS attacks . . . 446
25.12 How the pieces fit together . . . 448
25.13 Exercises . . . 449

26 Securing WiFi: WPA2 and WPA3 451

26.1 The shape of the problem . . . 451
26.2 WEP, and how to break a stream cipher by reusing it . . . 452
26.3 WPA2: the key hierarchy . . . 454
26.3.1 From passphrase to PMK . . . 454
26.3.2 From PMK to PTK: the four-way handshake . . . 455
26.3.3 Protecting data: AES-CCMP . . . 457
26.4 A worked miniature handshake . . . 457
26.5 The offline dictionary attack on WPA2-Personal . . . 458
26.6 KRACK: breaking WPA2 without the password . . . 460
26.7 WPA3 and the SAE/Dragonfly handshake . . . 462
26.7.1 How SAE works . . . 462
26.7.2 Dragonblood: early attacks on WPA3 . . . 464
26.8 The practical attacker: deauth, evil twins, and what's actually at risk . . . 465
26.9 Putting the layers together . . . 467
26.10 Exercises . . . 467

27 Secure Messaging and the Signal Protocol 469

27.1 What End-to-End Really Promises . . . 469
27.2 Why the Old Tools Do Not Fit . . . 471
27.3 X3DH: Agreeing on a Key With Someone Who Is Asleep . . . 471
27.3.1 The keys each party holds . . . 472
27.3.2 Alice's side: three (or four) Diffie-Hellmans . . . 472

27.3.3 Bob's side: he catches up later . . . 473
27.3.4 A tiny worked example . . . 474
27.4 The Double Ratchet: Keys That Never Stop Moving . . . 475
27.4.1 Ratchet one: the symmetric-key (hash) ratchet . . . 475
27.4.2 Ratchet two: the Diffie-Hellman ratchet . . . 476
27.4.3 Putting both gears together . . . 477
27.4.4 Tracing the ratchet with tiny numbers . . . 478
27.4.5 Tracing the DH ratchet self-heal . . . 478
27.4.6 Out-of-order and lost messages . . . 479
27.5 A Toy Double Ratchet in Code . . . 479
27.6 Sealed Sender and the Metadata Problem . . . 481
27.7 Authentication, Deniability, and the Safety Number . . . 482
27.7.1 Verifying you have the right Bob . . . 482
27.7.2 Deniability: authentic now, deniable later . . . 482
27.8 Putting It All Together . . . 483
27.9 Exercises . . . 484

28 SSH and Secure Remote Access 487

28.1 What problem SSH solves . . . 487
28.2 The three layers of SSH . . . 488
28.3 The transport layer: key exchange and host keys . . . 489
28.3.1 Identification and algorithm negotiation . . . 489
28.3.2 The key exchange itself . . . 490
28.3.3 From one secret to six keys . . . 491
28.3.4 The Binary Packet Protocol: encrypt and MAC . . . 492
28.4 Host keys and trust on first use . . . 493
28.4.1 Escaping TOFU: certificates and SSHFP . . . 494
28.5 User authentication: passwords versus public keys . . . 495
28.5.1 Password authentication . . . 495
28.5.2 Public-key authentication . . . 496
28.5.3 Protecting the private key . . . 498
28.6 The agent, and the risk of forwarding it . . . 498
28.6.1 Agent forwarding . . . 498

28.7 Port forwarding and the connection layer . . . 499
28.8 Common misconfigurations . . . 500
28.9 Putting it together . . . 501
28.10Exercises . . . 503

29 Cryptography in Telecoms: GSM, LTE, and 5G 505

29.1 The setting: who is who, and what they trust . . . 505
29.2 GSM authentication: a challenge and a response . . . 506
29.2.1 A worked triplet with tiny numbers . . . 508
29.3 Where GSM broke: COMP128, A5/1, and A5/2 . . . 508
29.3.1 COMP128-1: cloning the SIM . . . 509
29.3.2 The A5 family: the cipher on the air . . . 509
29.3.3 The arithmetic of a 54-bit key . . . 511
29.3.4 No integrity, and the network is never authenticated . . . 512
29.4 IMSI catchers (stingrays): the canonical telecom attack . . . 512
29.5 3G/4G AKA: making the network prove itself . . . 513
29.5.1 A worked SQN trace: replay, freshness, and resync . . . 515
29.5.2 What AKA still leaks: the IMSI on attach . . . 516
29.6 5G: encrypting the identity itself . . . 517
29.7 A toy SUCI: ECIES identity concealment in miniature . . . 518
29.8 The constraints that shaped all of this . . . 519
29.9 Exercises . . . 521

30 VPNs and Tunnels 523

30.1 What a tunnel actually is . . . 523
30.2 The threat model, stated precisely . . . 525
30.3 IPsec: the standards-body design . . . 526
30.3.1 ESP: the data plane . . . 527
30.3.2 IKE: the control plane . . . 527
30.3.3 Rekeying, dead peers, and roaming . . . 529
30.4 WireGuard: the minimal modern design . . . 530
30.4.1 Identity is a public key . . . 531
30.4.2 The handshake: Noise IK . . . 531

30.4.3 The data plane and the counter nonce . . . 533
30.5 A worked nonce-reuse catastrophe . . . 535
30.6 TLS-based VPNs . . . 537
30.7 What a VPN leaks even when the crypto is perfect . . . 538
30.8 Putting the families side by side . . . 539
30.9 Exercises . . . 540

V Advanced Topics **543**

31 Zero-Knowledge Proofs **545**

31.1 The cave: intuition before formalism . . . 545
31.2 Interactive proofs . . . 547
31.3 Commitments: the cryptographic envelope . . . 548
31.4 Schnorr's identification protocol . . . 549
31.5 Fiat-Shamir: from conversation to signature . . . 553
31.6 A second example: graph 3-coloring . . . 555
31.7 A glimpse of zk-SNARKs . . . 556
31.8 Putting the pieces together . . . 558
Exercises . . . 559

32 Commitments and Secret Sharing **561**

32.1 The sealed envelope: commitment schemes . . . 561
32.1.1 The threat model . . . 563
32.2 Building commitments from hash functions . . . 563
32.3 Pedersen commitments: perfect hiding and homomorphism . . . 564
32.3.1 The homomorphic bonus . . . 565
32.4 Splitting a secret: the problem . . . 566
32.5 Shamir secret sharing . . . 566
32.5.1 Setup and why a field . . . 567
32.5.2 Dealing the shares . . . 567
32.5.3 A worked example over $\mathbb{F}_{17}$ . . . 568
32.5.4 Reconstruction by Lagrange interpolation . . . 568
32.5.5 Why $t-1$ shares reveal nothing . . . 570

32.5.6 Properties that make it shine . . . 571
32.6 The dealer problem and verifiable secret sharing . . . 571
32.6.1 Feldman VSS . . . 572
32.7 Distributed key generation: removing the dealer . . . 573
32.8 Where this gets used: key escrow, multisig, distributed trust . . . 574
32.9 Summary . . . 574
32.10Exercises . . . 575

33 Homomorphic Encryption **577**
33.1 Computing on ciphertext: the threat model . . . 577
33.2 Partially homomorphic: schemes you already know . . . 579
33.2.1 Textbook RSA multiplies . . . 579
33.2.2 ElGamal multiplies, with randomness . . . 580
33.3 Paillier: additive homomorphism done right . . . 580
33.3.1 The construction . . . 580
33.3.2 Why it is additive . . . 581
33.3.3 A toy implementation . . . 583
33.4 The leap to fully homomorphic encryption . . . 584
33.4.1 Noise: the enemy and the engine . . . 585
33.4.2 Bootstrapping: refreshing noisy ciphertexts . . . 587
33.4.3 Message flow and the cost of depth . . . 588
33.4.4 What a real homomorphic multiply actually does . . . 589
33.5 What it costs and where it is heading . . . 590
33.6 Putting the pieces together . . . 592
33.7 Exercises . . . 593

34 Secure Multiparty Computation **595**
34.1 The dream: a trusted party who forgets . . . 595
34.1.1 Two adversary models . . . 596
34.2 The millionaires' problem . . . 597
34.3 Oblivious transfer: the atom of MPC . . . 598
34.3.1 A concrete OT from Diffie-Hellman . . . 599
34.3.2 OT extension: making it cheap . . . 600

34.4 Yao's garbled circuits . . . 601
34.4.1 Garbling one wire, one gate . . . 601
34.4.2 Wiring up the whole circuit . . . 602
34.4.3 The optimizations that make it real . . . 603
34.5 Secret-sharing-based MPC for many parties . . . 605
34.5.1 Recap: Shamir shares are linear . . . 605
34.5.2 Addition is free, multiplication is the hard part . . . 605
34.5.3 The BGW multiplication step . . . 606
34.5.4 A worked multiplication with tiny numbers . . . 606
34.5.5 Beaver triples: precompute the hard part . . . 607
34.6 An attack: when "private" inputs aren't enough . . . 608
34.7 Where MPC is used in the real world . . . 609
34.8 What it costs, and when to reach for it . . . 609
Exercises . . . 610

35 Post-Quantum Cryptography 613

35.1 What a quantum computer is, in just enough detail . . . 613
35.2 Shor's algorithm: why factoring and discrete log fall . . . 614
35.2.1 Factoring reduces to finding the order of an element . . . 615
35.2.2 Finding the period with a quantum Fourier transform . . . 615
35.2.3 Discrete log dies the same way . . . 616
35.3 Grover's algorithm and what it does to symmetric crypto . . . 617
35.4 Lattices: the foundation of post-quantum public key . . . 618
35.4.1 What a lattice is . . . 618
35.4.2 Learning With Errors (LWE), the assumption that does the work . . . 619
35.4.3 Encrypting with LWE: Regev's scheme . . . 620
35.4.4 Module-LWE, Kyber, and Dilithium . . . 621
35.5 A toy lattice cryptosystem you can break by hand . . . 622
35.6 Hash-based signatures: security from the primitive you trust most . . . 623
35.6.1 Lamport: signing one bit, then one message, with a hash . . . 623
35.6.2 From one signature to a tree: Merkle, XMSS, SPHINCS+ . . . 624
35.7 Migration, hybrids, and timelines . . . 625
35.7.1 The standardized algorithms . . . 625

35.7.2 Hybrid key exchange: belt and suspenders . . . 626
35.7.3 Timelines, without hand-waving . . . 627
35.8 Where this leaves us . . . 627
35.9 Exercises . . . 628

36 The Cryptography Behind Blockchains **631**
36.1 The job: agreement without a referee . . . 631
36.2 Hashes: the glue . . . 632
36.3 Binding a block to its predecessor . . . 633
36.4 Merkle trees: one root for a million transactions . . . 634
36.4.1 Construction . . . 634
36.4.2 Membership proofs . . . 635
36.5 Keys, addresses, and wallets . . . 637
36.5.1 From private key to public key . . . 637
36.5.2 From public key to address . . . 638
36.5.3 Wallets and key derivation . . . 639
36.6 Signatures: authorizing a spend . . . 639
36.6.1 ECDSA . . . 640
36.6.2 Schnorr signatures . . . 641
36.7 Proof of work: making history expensive to rewrite . . . 642
36.7.1 The puzzle . . . 642
36.7.2 Why rewriting is hard: the longest-chain rule . . . 644
36.8 What the cryptography does and does not guarantee . . . 644
36.9 Putting it together: the life of a transaction . . . 646
Exercises . . . 647

VI Attacks and Safe Implementation **649**

37 Side-Channel and Timing Attacks **651**
37.1 The Shape of the Problem . . . 651
37.2 Timing Attacks: When Time Is a Variable . . . 653
37.2.1 The leaky string comparison . . . 653
37.2.2 The hard part is the noise . . . 654

37.2.3 Constant-time comparison . . . 655
37.2.4 Timing in the math itself: square-and-multiply . . . 656
37.3 Cache Attacks: The Memory System Is Watching . . . 657
37.3.1 Why the cache leaks . . . 657
37.4 Power and Electromagnetic Analysis . . . 659
37.4.1 Simple power analysis (SPA) . . . 659
37.4.2 Differential power analysis (DPA) . . . 660
37.5 Fault Attacks: Breaking the Machine on Purpose . . . 661
37.5.1 The Bellcore attack on RSA-CRT . . . 661
37.6 Padding Oracles: When an Error Message Is the Channel . . . 663
37.6.1 The AES-CBC padding oracle . . . 663
37.6.2 Bleichenbacher: the RSA padding oracle that will not die . . . 666
37.6.3 Lucky 13: the padding oracle hiding in the timing . . . 666
37.7 Constant-Time Programming: The Discipline . . . 668
37.8 Why “Secure on Paper” Is Not Enough . . . 670
Exercises . . . 671

38 Common Pitfalls 673

38.1 The Shape of Every Pitfall . . . 673
38.2 Nonce and IV Reuse . . . 674
38.2.1 The two-time pad . . . 674
38.2.2 CTR mode is a stream cipher, so the same rule binds . . . 676
38.2.3 The GCM catastrophe: from confidentiality loss to forgery . . . 676
38.2.4 Predictable IVs: the CBC case and BEAST . . . 677
38.3 ECB Mode: Encryption That Preserves Pictures . . . 678
38.4 Getting Encrypt-and-Authenticate Backwards . . . 680
38.5 Weak, Predictable, and Duplicated Randomness . . . 682
38.5.1 Predictable seeds . . . 682
38.5.2 Duplicated randomness across devices . . . 683
38.5.3 The forked-counter trap . . . 683
38.6 The Signature Nonce: One Number Leaks the Private Key . . . 684
38.7 Hardcoded and Reused Keys . . . 685
38.8 Reusing One Key for Two Purposes . . . 686

38.9 Broken Primitives: MD5, SHA-1, DES, RC4 . . . 687
38.9.1 What "broken" means, precisely . . . 687
38.10Insecure Comparison: Leaking Secrets Through Time . . . 688
38.11Rolling Your Own Crypto . . . 690
38.12A Field Checklist . . . 692
Exercises . . . 692

39 Implementing Cryptography Safely 695
39.1 Defining the Core Rule . . . 695
39.2 The Library Hierarchy: Where to Stand . . . 697
39.2.1 NaCl and libsodium: the box that hides the nonce . . . 697
39.2.2 Tink: types that refuse to be confused . . . 699
39.3 Misuse-Resistance Made Concrete . . . 699
39.4 Key Management: The Part Nobody Tests . . . 700
39.4.1 The key lifecycle . . . 701
39.4.2 Key hierarchies and key wrapping . . . 701
39.4.3 Rotation, staged so nothing breaks . . . 703
39.4.4 Memory Hygiene: Dead-Store Elimination and Zeroization . . . 703
39.5 Secure Defaults and Crypto-Agility . . . 705
39.6 Testing Crypto: Vectors, Properties, and Failure Paths . . . 705
39.6.1 Known-answer test vectors . . . 705
39.6.2 Property tests and the rejection path . . . 706
39.7 Constant-Time Discipline and Branch Elimination . . . 707
39.7.1 The Constant-Time Multiplexer . . . 707
39.8 Threat Modeling: Decide What You Are Defending Against . . . 709
39.9 When (Almost Never) to Implement a Primitive . . . 710
39.10A Practical Checklist . . . 711
39.11What to Carry Forward . . . 712
Exercises . . . 713

VII Mathematical Background 715

40 Appendix: Number Theory Reference 717

40.1 Notation and conventions . . . 717
40.2 Divisibility and the division algorithm . . . 718
40.3 Greatest common divisor and the Euclidean algorithm . . . 718
40.3.1 Bézout and the extended Euclidean algorithm . . . 719
40.4 Modular arithmetic: the ring $\mathbb{Z}/n\mathbb{Z}$ . . . 720
40.4.1 Fast modular exponentiation . . . 720
40.5 Fermat's little theorem and Euler's theorem . . . 721
40.5.1 Computing the totient . . . 722
40.6 The Chinese Remainder Theorem . . . 722
40.7 Orders, cyclic groups, and primitive roots . . . 723
40.8 Primality testing . . . 724
40.8.1 The Fermat test and its blind spot . . . 724
40.8.2 Miller-Rabin . . . 725
40.9 Quadratic residues . . . 726
40.10 Polynomial Rings, Quotients, and Cyclotomic Reductions . . . 727
40.10.1 The Polynomial Ring $\mathbb{F}[x]$ . . . 728
40.10.2 Quotient Rings and the Cyclotomic Modulus $x^n + 1$ . . . 728
40.11 Lagrange Polynomial Interpolation . . . 729
40.11.1 The Lagrange Basis Construction . . . 730
40.11.2 Application: Reconstructing Shamir's Secret at $x = 0$ . . . 730
40.12 Quick reference . . . 730
40.13 Worked end-to-end: a complete RSA toy with these tools . . . 731
40.14 Exercises . . . 732

41 Appendix: Algebra and Probability Reference 735

41.1 The big picture: why cryptographers care about algebra . . . 736
41.2 Groups . . . 737
41.2.1 Order, generators, and cyclic groups . . . 738
41.2.2 Subgroups and the structure of $(\mathbb{Z}/p\mathbb{Z})^*$ . . . 738
41.3 Rings: where you can add and multiply . . . 739
41.4 Fields . . . 739
41.4.1 Finite fields of prime-power order . . . 740
41.4.2 Worked example: arithmetic in $\mathbb{F}_{2^3}$ . . . 740

41.5 Vector Spaces, Lattices, and Determinants . . . 742
41.5.1 Vector Spaces . . . 742
41.5.2 Lattices and the Fundamental Domain . . . 742
41.6 Bitwise Arithmetic and Two's Complement . . . 743
41.6.1 The Two's Complement Ring $\mathbb{Z}/2^w\mathbb{Z}$ . . . 744
41.6.2 Constant-Time Arithmetic Masking . . . 744
41.7 A pocket card of algebraic facts . . . 745
41.8 Probability: the cryptographer's slice . . . 745
41.8.1 Probability spaces and uniform distributions . . . 745
41.8.2 Expectation, variance, and the union bound . . . 746
41.8.3 Independence and conditional probability . . . 746
41.8.4 Entropy: measuring unpredictability . . . 746
41.8.5 The birthday bound: when do random draws collide? . . . 747
41.8.6 The negligible-function discipline . . . 749
41.9 A pocket card of probability facts . . . 750
41.10How the two halves meet . . . 750
41.11Exercises . . . 750

Part I

Foundations and the Mathematics

Chapter 1

What Cryptography Is

As previewed in the preface, we start with the central problem: how do you share a secret when anyone might be watching? Imagine you write a note, fold it, and slide it across a crowded room to a friend. Between your hand and theirs, a dozen people could read it, copy it, change a word, or replace the note entirely with one of their own. Cryptography is the science of making that slip of paper behave as if the room were empty – as if only your friend could read it, any tampering would be immediately detected, and a forged note could be told apart from a real one. It does this with mathematics rather than locks and walls, and its central claim is that a few hundred bits of carefully chosen numbers can defeat an adversary with more computing power than exists on Earth.

This chapter builds the vocabulary and the worldview you will use for the rest of the book. We will not yet encrypt anything seriously. Instead we pin down four goals that cryptography chases (confidentiality, integrity, authentication, non-repudiation), meet the cast of characters who show up in every protocol story, learn the one principle that separates real cryptography from wishful thinking, and – most importantly – learn what the word "secure" is even allowed to mean. That last point is subtle enough that whole schemes which "felt" secure to clever people have collapsed embarrassingly. By the end you should have the security mindset: a reflex to ask, before trusting anything, "secure against whom, given what, and proven how?"

1.1 Four Goals: What Are We Actually Protecting?

When people first hear "cryptography" they think "secret codes" – hiding the contents of a message. That is one goal, but only one, and historically not even the one that gets people killed most often. A message that nobody can read is useless if an attacker can swap it for a different message, or impersonate the sender, or convince a court they never sent something they did. Cryptography pursues four distinct goals, and a real system usually needs several of them at once.

Confidentiality. Only the intended recipient learns the contents. An eavesdropper who sees every bit on the wire learns nothing useful about the message (ideally, nothing at all beyond its length). This is the "secret writing" goal, the one encryption provides.

Integrity. The recipient can detect if the message was altered in transit – even by one bit, even by an attacker who controls the entire network. Note this does not mean the attacker cannot

change the bits; on an open network they always can. It means the recipient will *notice* and reject the change.

Authentication. The recipient can verify who the message came from. "This really is from Alice, not from someone pretending to be Alice." There are two flavors: *entity authentication* (you are talking to the right party right now) and *message* or *data-origin authentication* (this particular message originated with that party).

Non-repudiation. The sender cannot later deny having sent the message. If Alice signs a contract, she cannot turn around and tell a judge "that wasn't me." This is a stronger and stranger property than the others, because it must convince a *third party* – not just the recipient.

Intuition

A useful way to keep these straight is to ask what kind of lie each one prevents. *Confidentiality* stops the lie "I read your message." *Integrity* stops the lie "the message said X" when it actually said Y. *Authentication* stops the lie "I am Alice." *Non-repudiation* stops the lie "I never said that" – told by Alice herself, after the fact. The first three protect the communicating pair against an outsider. The fourth protects each member of the pair against the *other*, and against future disputes.

Why it works

Why non-repudiation is the odd one out. The first three goals can be achieved with a shared secret key: if only Alice and Bob know k, then a valid authentication tag on a message proves *to Bob* that Alice sent it – nobody else could have made the tag. But that same proof is worthless *to a judge*, because Bob knows k too, so Bob could have forged the message himself. With shared-key authentication, Alice can always plausibly say "Bob faked it," and the judge cannot tell who is lying. Non-repudiation therefore demands an *asymmetry* the other goals do not: Alice must hold a secret that Bob does *not* have, so that a valid tag could only have come from Alice. That is exactly what a digital signature provides – a private signing key only Alice holds, and a public verification key everyone (including the judge) can check. This is the first hint that some goals are unreachable with symmetric cryptography alone, foreshadowing §1.8.

These goals are independent. You can have any one without the others, and confusing them is a classic source of broken systems. A WiFi network that encrypts your traffic (confidentiality) but lets an attacker flip bits undetected (no integrity) can be manipulated in nasty ways – and historically was. An email that is signed (authentication, non-repudiation) but sent in the clear (no confidentiality) is perfectly readable by anyone on the path. Encryption alone gives you neither integrity nor authentication; that surprises newcomers, and we will hammer on it repeatedly.

Common trap

"I encrypted it, so it can't be tampered with." False, and dangerously so. Most basic encryption schemes are *malleable*: an attacker who cannot read the plaintext can still flip chosen bits of the ciphertext and cause predictable changes to the decrypted plaintext. We will see in Chapters 9 and 10 that flipping bit i of a one-time-pad ciphertext flips bit i of the plaintext exactly. Confidentiality and integrity are separate goals requiring separate

mechanisms. Modern designs bundle them with *authenticated encryption*, which we treat later – but the lesson starts here.

In the real world

When you load a banking site over HTTPS, all four goals are in play. *Confidentiality*: nobody on the coffee-shop WiFi reads your password. *Integrity*: the network cannot alter the page to inject a fake login form without detection. *Authentication*: the certificate proves you are talking to the real bank, not an impostor. *Non-repudiation* shows up less in the browser and more in the back end – digitally signed transaction logs that the bank can present if you dispute a transfer. TLS, the protocol behind HTTPS, is designed to answer the question "how do we get the first three over a hostile network, starting from nothing but a trusted list of certificate authorities?" We will build up to it.

1.2 What Cryptography Is Not

It helps to distinguish cryptography from three things people often confuse with it.

Cryptography is not steganography. Steganography *hides the existence* of a message – invisible ink, data tucked in the low bits of an image, a microdot. Cryptography hides the *contents* while making no secret that a message exists; an eavesdropper plainly sees that ciphertext is flying by, just not what it says. The two are complementary and orthogonal. Steganography fails the moment the hiding place is discovered (and per Kerckhoffs we must assume it will be); cryptography is designed to stay secure even when the adversary sees every encrypted bit. Serious systems encrypt first and, only if hiding the existence of communication also matters, steganographically conceal the ciphertext as a second, weaker layer.

Cryptography is not encoding. Base64, ASCII, Morse, and URL-encoding are *encodings*: reversible reformatting with no secret key, meant for compatibility or transport, not secrecy. Anyone can reverse them; there is nothing to know. Calling Base64 "encryption" is a beginner's error with real consequences – people have shipped "encrypted" configuration files that were merely Base64-encoded, readable by anyone in one command. If there is no key, there is no secret, and there is no cryptography.

Cryptography is not a classical "code." Historically a *code* replaced whole words or phrases via a fixed codebook ("BLUEBIRD" means "attack at dawn"). Codes operate on meaning; *ciphers* operate mechanically on symbols (letters, bits) regardless of meaning. Modern cryptography is entirely about ciphers and their mathematical generalizations – the word "code" survives mostly in fossils like "error-correcting code" and "decode," which are unrelated. When we say "code" in this book we mean source code, not a secret codebook.

Common trap

"It's encrypted" is a claim people make about anything they cannot read at a glance – Base64 blobs, compressed data, binary formats, obfuscated source. None of those involve a key or a security argument, so none of them are encryption. The test is simple: *is there a secret key, and is there a stated adversary the scheme is designed to defeat?* No key and no adversary

means no cryptography, however scrambled the output looks.

1.3 A Tiny Cipher, and Why It Already Fails

A concrete example, even a bad one, makes the abstractions easier to follow. So before the formal definitions, let us encrypt something by hand, see what "secure" would have to mean, and watch exactly how the scheme breaks.

Take the oldest cipher there is, the *shift cipher* (Caesar's cipher). Fix the alphabet $A = 0, B = 1, \ldots, Z = 25$ and work modulo 26. The key is a single number $k \in \{0, 1, \ldots, 25\}$. To encrypt, shift every letter forward by k; to decrypt, shift back:

$$\mathrm{Enc}_k(m) = (m + k) \bmod 26, \qquad \mathrm{Dec}_k(c) = (c - k) \bmod 26,$$

applied letter by letter. With $k = 3$, the word `HELLO` becomes:

$$\texttt{H}(7) \to 10\,\texttt{K}, \quad \texttt{E}(4) \to 7\,\texttt{H}, \quad \texttt{L}(11) \to 14\,\texttt{O}, \quad \texttt{L} \to \texttt{O}, \quad \texttt{O}(14) \to 17\,\texttt{R},$$

so the ciphertext is `KHOOR`. Decryption shifts each letter back by 3 and recovers `HELLO`.

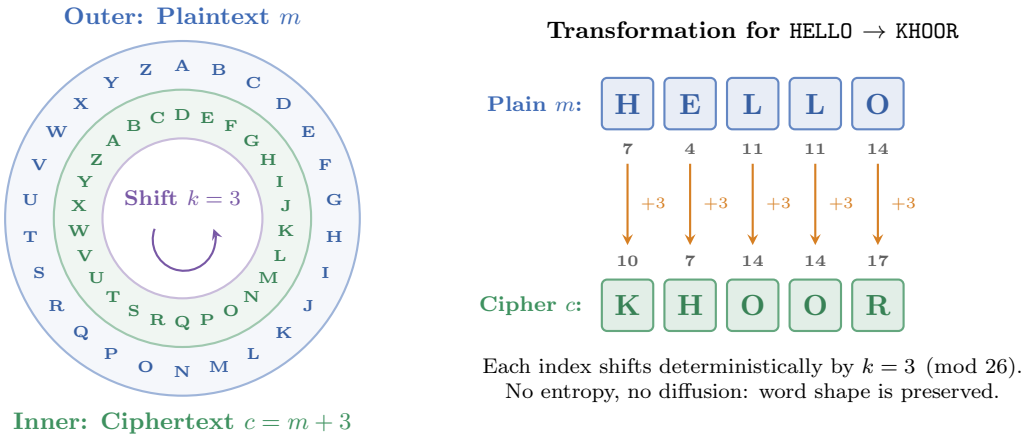


Figure 1.1: The Caesar shift cipher. Left: Concentric wheels shifting the alphabet by $k = 3$. Right: Direct letter-to-letter index mapping showing why structural properties pass straight through to the ciphertext.

The scheme is *correct* – decryption always inverts encryption – which is the first thing any cipher must get right but says nothing yet about security.

Walking question

Before reading on, ask yourself: how many keys are there? If you intercepted `KHOOR` and knew only that a shift cipher was used (Kerckhoffs: assume the attacker knows the algorithm), how long would it take you to read it?

There are exactly 26 keys – and $k = 0$ does nothing, so really 25 useful ones. An attacker simply tries all of them. This is *exhaustive key search*, the most basic attack, and against a 26-key cipher it is instant.

Attack

Breaking the shift cipher two ways. Intercept `KHOOR`. *Brute force:* shift back by $1, 2, 3, \ldots$ and read off the only result that is English. Shifting back by 3 gives `HELLO`; the other 24 candidates are gibberish like `JGNNQ` or `GDKKN`. Twenty-five trials, done by hand in under a minute. *Frequency analysis:* for longer texts you do not even need to try keys. English letter frequencies are lopsided – `E` is about 12.7% of letters, `T` about 9%, while `Z` is near 0.07%. A shift cipher slides the whole frequency histogram sideways by k without changing its shape. Find the most common ciphertext letter, assume it is `E`, and the gap to `E` reveals k. The cipher's fatal flaw is that it *preserves structure*: the statistical fingerprint of the language passes straight through.

The shift cipher is named for Julius Caesar, who according to Suetonius shifted his correspondence by three. It was genuinely useful in an era when most adversaries were illiterate and the very idea of systematic cryptanalysis did not exist. Frequency analysis – the technique that kills it – was first described by the polymath al-Kindi in 9th-century Baghdad, in a manuscript on deciphering encrypted messages. That single idea broke essentially every cipher built on simple letter substitution for the next thousand years, and it is why "it scrambles the letters" is never, by itself, security (as we will see in Chapter 2).

You might try to patch the shift cipher by allowing an *arbitrary* permutation of the alphabet instead of a mere shift – the *substitution cipher*. Now the key is a full scrambling of 26 letters, so there are $26! \approx 4 \times 10^{26}$ keys, a number near 2^{88}. Surely *that* is unbreakable – 2^{88} is roughly three hundred septillion keys, far too many to brute force by hand or even by machine.

Common trap

A huge keyspace does not imply security. The substitution cipher has $\approx 2^{88}$ keys and is broken by a bored teenager in an afternoon – it is the staple of newspaper cryptograms. Why? A permutation, like a shift, maps each plaintext letter to a *fixed* ciphertext letter, so it still preserves the language's frequency fingerprint: the most common ciphertext symbol is almost surely `E`, the commonest digraph is probably `TH`, and so on. Frequency analysis cracks it without ever touching the 2^{88} keys. *The size of the keyspace is necessary for security but nowhere near sufficient.* What matters is whether the ciphertext leaks structure – and these classical ciphers leak it all. We will spend much of the book building ciphers whose output is statistically indistinguishable from random noise, precisely so that no such fingerprint survives.

The shift cipher taught us correctness and exhaustive search; the substitution cipher taught us that keyspace size is not the whole story. Both taught us that "secure" is going to need a careful definition, because our intuition ("billions of keys, must be safe!") turned out to be wrong. These failures motivate much of what follows.

1.4 The Cast: Alice, Bob, Eve, and Mallory

Cryptographers tell stories about a recurring cast, and the names are not just whimsy – each one encodes a specific assumption about who is doing what. Getting the cast right is the first step in stating a problem precisely.

Alice and Bob are the two parties who want to communicate securely. By convention Alice initiates and Bob responds, but they are symmetric in spirit. They are the "good guys," though as we will see, even they do not fully trust each other when non-repudiation is at stake.

Eve is the *eavesdropper*. She is *passive*: she can see everything that crosses the channel, copy it, store it forever, and analyze it at leisure – but she does not alter, inject, or delete anything. Eve models wiretaps, packet sniffers, and the intelligence agency recording all traffic for later. Defeating Eve is the confidentiality problem.

Mallory is the *active* attacker, the adversary in the middle. Mallory can do everything Eve can, *plus* modify messages, drop them, replay old ones, inject new ones, and impersonate either party. Mallory is far more capable than Eve, and defeating Mallory requires integrity and authentication, not just encryption. The name is a mnemonic: **Mal**icious, **Mall**eable, **M**an-in-the-**m**iddle.

Trent is a *trusted third party* – an arbiter everyone agrees to believe, such as a certificate authority. Trent is convenient but dangerous: every trusted party is also a single point of failure and a juicy target. Good design minimizes how much you must trust Trent.

You will also meet **Carol** and **Dave** (additional legitimate parties when two are not enough), **Peggy** the *prover* and **Victor** the *verifier* (in zero-knowledge proofs, where Peggy proves she knows a secret without revealing it), and **Sybil** (an attacker who forges *many* fake identities at once). The names are a shorthand the whole field shares.

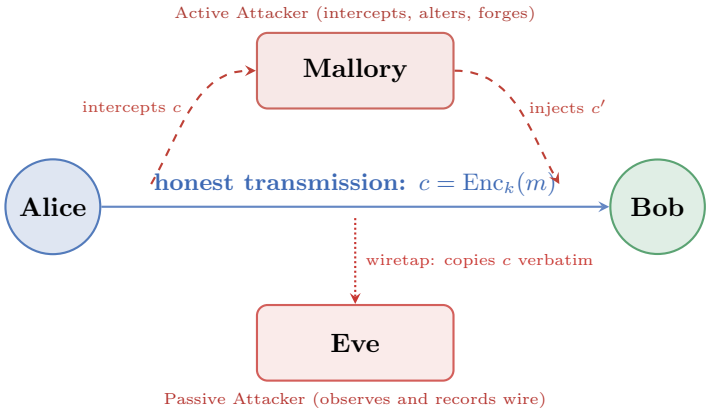


Figure 1.2: The cryptographic threat models. Eve passively records traffic on the channel without disrupting it; confidentiality protects against Eve. Mallory sits directly within the network flow, altering or injecting arbitrary messages; integrity and authentication are required to defeat Mallory.

Focus

The single most important question to ask about any attacker is *passive or active?* A scheme can be perfectly confidential against Eve and yet immediately broken by Mallory, because confidentiality says nothing about a message you did not even encrypt – one Mallory injected himself. Most real-world catastrophes (downgrade attacks, man-in-the-middle interception, replay) come from designers who thought about Eve and forgot about Mallory.

1.5 Kerckhoffs's Principle: No Security Through Obscurity

The foundational rule is counterintuitive enough that beginners resist it: *the security of a cryptographic system must depend only on the secrecy of the key, never on the secrecy of the algorithm.* You should be able to publish your entire design – the cipher, the protocol, the source code, the exact parameters – hand it to your enemy, and remain secure as long as the key stays secret.

Auguste Kerckhoffs, a Dutch linguist, laid out six design rules for military ciphers in an 1883 article in the *Journal des sciences militaires*. The second rule is the one that immortalized his name: a system "must not require secrecy, and it must be able to fall into the hands of the enemy without inconvenience." Claude Shannon, founding the mathematical theory of secrecy in 1949, restated it crisply as "the enemy knows the system" – now called *Shannon's maxim*. The principle predates computers by decades, but every modern standard (AES, RSA, the TLS protocol) is fully public precisely because of it.

There are several reasons to insist on this.

Keys can be changed; algorithms cannot. If your secret leaks and it was a key, you generate a new key and move on in seconds. If your secret was the algorithm and it leaks, you must redesign, re-implement, re-deploy, and re-audit everything, everywhere, often across hardware you cannot recall. In practice, secrets that are expensive to replace often do not get replaced.

Secret algorithms do not get reviewed. A cipher's strength is the absence of a clever attack. The only way to gain confidence that no clever attack exists is to let thousands of clever people try to find one. A design kept secret is reviewed by, at most, the handful of people who built it – the very people least able to see its blind spots. Public algorithms like AES survived years of open cryptanalysis by the world's best before standardization. That public scrutiny is what earns the trust.

Obscurity always leaks. Algorithms get reverse-engineered from binaries, leaked by insiders, extracted from hardware, or simply guessed. Designing as if the algorithm is secret means designing as if a temporary, fragile condition were permanent.

Attack

A5/1 and the GSM phone network. Early mobile phone encryption (the A5/1 cipher protecting 2G GSM voice calls) was designed in secret in the late 1980s and never published. Secrecy was treated as part of the defense. The design leaked and was reverse-engineered by 1999, and once cryptographers could see it, fatal weaknesses fell out quickly: by the late 2000s, attackers with precomputed tables ("rainbow tables") could recover the session key for a call in seconds on commodity hardware. Decades of calls had been protected by a cipher whose only real defense – obscurity – evaporated the moment someone looked. The contrast with AES, which gained resilience and trust through years of open scrutiny, illustrates the point.

Common trap

Kerckhoffs's principle is *not* "you must publish your source code" or "secrecy never helps." Keeping deployment details quiet can raise the bar for an attacker as a bonus layer (defense in depth). The principle is narrower and firmer: you may never *rely* on that secrecy for your security. Your scheme must remain secure under the assumption that the attacker knows

everything except the key. If revealing your design breaks your system, your system was already broken; you just had not met the right adversary yet.

Enigma: a large keyspace, defeated by structure and operators. The German military's Enigma machine of World War II had a daily key drawn from a space of around 10^{23} possibilities ($\approx 2^{77}$) – far beyond brute force with the technology of the day. It was broken anyway, first by Polish mathematicians (notably Marian Rejewski) and then on an industrial scale at Britain's Bletchley Park, where Alan Turing's team built electromechanical *bombes* to mechanize the search. The keyspace was never the weak point. The breaks came from structural flaws (no letter ever encrypted to itself – a constraint the attackers exploited), procedural mistakes (repeated message indicators, predictable plaintext like weather reports and military salutes), and operator laziness. The pattern is the same one the shift cipher showed in miniature and that recurs throughout this book: a giant keyspace is undone by leaked *structure* and human *procedure*, not by counting keys. It is also a clear case for Kerckhoffs – the Allies effectively obtained the system, and security then rested entirely on the daily key, which the flaws let them recover.

1.6 Threat Models: Secure Against Whom, Given What?

"Is it secure?" is not a well-formed question. Secure against a teenager with a laptop, or against a nation-state with a building full of supercomputers and a court order? Secure against someone who only listens, or someone who can also choose which messages you encrypt? A *threat model* is the precise specification of the adversary you are defending against: their goals, their capabilities, and their knowledge. Without it, "secure" is marketing, not engineering.

A threat model answers three questions.

1. **What is the attacker trying to do?** (Their *goal*.) Read one specific message? Recover the key? Distinguish two possible plaintexts? Forge a single valid signature? Weaker goals make stronger attacks – defending against "learns even one bit of the plaintext" is harder, hence better, than defending against "recovers the whole plaintext."
2. **What can the attacker do?** (Their *capabilities*.) This is where we rank attack models by how much access the adversary has to the cryptosystem in action:
   - *Ciphertext-only:* the attacker sees ciphertexts and nothing else. The weakest attacker; surviving this is the bare minimum.
   - *Known-plaintext:* the attacker also sees some plaintext/ciphertext pairs they did not choose (e.g. a message with a predictable greeting).
   - *Chosen-plaintext (CPA):* the attacker can get plaintexts *of their choosing* encrypted. This sounds generous, but it models reality – an attacker can often trick a system into encrypting data they pick (think of a web server that encrypts whatever you type).
   - *Chosen-ciphertext (CCA):* the strongest standard model – the attacker can also get ciphertexts *of their choosing* decrypted (except the one they are trying to break). This models systems that act on decrypted data and leak whether decryption succeeded.
3. **What does the attacker know and have?** Computational power, time, side-channel access (timing, power consumption, electromagnetic leakage), and – per Kerckhoffs – the full algo-

rithm. We almost always assume the attacker is *computationally bounded*: realistically limited to, say, 2^{100} operations, vast but finite. The attacker who can do 2^{256} operations can break almost anything by brute force, but such an attacker does not exist and never will (we will do the physics in a moment).

Intuition

Always design for a *stronger* attacker than you think you face. If your scheme resists chosen-ciphertext attacks, it certainly resists mere eavesdropping; the converse fails badly. Many cryptosystems have been broken because their designers assumed a weaker attacker than the one they *actually* faced.

Connection

The phrase "computationally bounded" carries the main assumption here and can be stated precisely in the language of the companion algorithms book. We treat "feasible" as "runs in time polynomial in the key length" and "infeasible" as "requires super-polynomial (typically exponential) time." That is why a cryptographer cares enormously whether the best known attack on a problem is $O(n^3)$ or 2^n: the first is a tractable computation, the second is a wall. Modular exponentiation – the core operation in RSA and Diffie-Hellman – costs $O(\log e)$ multiplications via repeated squaring, which is why public-key crypto is feasible at all. The algorithms companion covers fast exponentiation; the probability companion covers the birthday bound that governs how big a hash must be. We will lean on both.

How big is 2^{128}, really?

Cryptographers throw around numbers like 2^{128} as "infeasible" and it is worth seeing why. Suppose you build a machine that tries one key every nanosecond – a billion keys per second. To try all 2^{128} keys:

$$\frac{2^{128}}{10^9 \text{ keys/sec}} \approx \frac{3.4 \times 10^{38}}{10^9} = 3.4 \times 10^{29} \text{ seconds.}$$

There are about 3.15×10^7 seconds in a year, so that is roughly 1.1×10^{22} years – about a trillion times the current age of the universe ($\sim 1.4 \times 10^{10}$ years). Now run a billion such machines in parallel: you save nine orders of magnitude and still need 10^{13} years. There is also a hard physical floor: Landauer's principle and the thermodynamics of computation imply that merely *counting* through 2^{128} states would consume more energy than is reasonably harvestable. Brute force is simply not a viable attack on a 128-bit key.

Focus

This is the central bargain of modern cryptography: we make the *legitimate* operation cheap (a fraction of a microsecond to encrypt) and the *attacking* operation astronomically expensive (more than the universe affords), purely by the gap between, say, 2^{30} defender work and 2^{128} attacker work. Every scheme in this book is an attempt to engineer such a gap and to *prove* the gap really exists.

1.7 Randomness: The Resource Everything Rests On

Every secure scheme relies on an assumption that beginners often miss: cryptography consumes *genuine unpredictability*. Keys must be unguessable, which means they must be *random* – chosen by a process the attacker cannot predict or reproduce. A perfect algorithm fed a predictable "random" key is no safer than no algorithm at all, because the attacker simply predicts the key.

To quantify this: if you generate a 128-bit key by sampling 128 uniform, independent bits, the attacker faces 2^{128} equally likely candidates – the astronomical wall of §1.6. But suppose your random-number generator has an unnoticed flaw and only ever produces keys from a pool of, say, 2^{20} possibilities (a million). The key is still 128 bits *long*; it just is not 128 bits *unpredictable*. The attacker who knows the flaw – and per Kerckhoffs we assume they do – searches only $2^{20} \approx 10^6$ keys, a fraction of a second's work. Your 128-bit cipher now offers 20 bits of real protection. What counts is the *entropy* behind a key, not its length.

Intuition

A flawless algorithm is useless if its keys are predictable. Many catastrophic real-world breaks come not from broken *algorithms* – AES and RSA remain unbroken – but from broken *randomness*: predictable keys, reused nonces, weak seeds. When you audit a system, check the randomness first.

Attack

When the dice are loaded. In 2008 a Debian Linux maintainer, cleaning up a memory-handling warning, accidentally removed a line that fed entropy into the OpenSSL random-number generator. For nearly two years, every cryptographic key generated on affected systems was drawn from a pool of only about 2^{15} possibilities – determined essentially by the process ID. Keys that looked like full-strength 2048-bit RSA or strong SSH keys were, in fact, enumerable by an attacker in minutes. Millions of keys across the internet had to be revoked and regenerated. No cipher was broken; the failure was entirely in the randomness. The same theme recurs in embedded devices that generate keys at first boot before they have gathered any entropy, producing predictable or even *shared* keys across thousands of distinct devices – researchers have factored RSA keys from routers simply because two devices independently generated overlapping "random" primes.

This is why cryptographers distinguish a *true random number generator* (TRNG), which harvests physical entropy – thermal noise, timing jitter, radioactive decay – from a *cryptographically secure pseudo-random number generator* (CSPRNG), which stretches a short truly-random seed into a long stream that no efficient adversary can distinguish from true randomness. Practical systems gather a little real entropy into a seed, then expand it with a CSPRNG. The catch is the seed: stretch a guessable seed and you get a guessable, infinitely long "random" stream. The Debian bug was exactly this: a guessable seed.

Common trap

The ordinary random functions in most programming languages – C's `rand()`, Python's `random` module, JavaScript's `Math.random()` – are *not* cryptographically secure. They are built for simulations and games: fast, statistically smooth, and *predictable*, because an at-

tacker who sees a few outputs can recover the internal state and predict all future ones. Never generate keys, nonces, or tokens with them. Use the operating system's cryptographic source (`/dev/urandom`, `getrandom()`, Python's `secrets` or `os.urandom`, Java's `SecureRandom`). The `os.urandom` call in the listing of §1.9 was deliberate – it is the right source – even though that example's *usage* of the output was wrong.

Connection

Whether a stretched seed "looks random" to every efficient adversary is the formal notion of a *pseudo-random generator*, and proving a CSPRNG secure reduces – as always – to some hardness assumption. The probability companion provides the statistical-distance and indistinguishability tools used to state "cannot be told apart from uniform"; the number-theory chapters provide candidate hard problems (like the difficulty of certain residue questions) from which provably secure generators can be built. Randomness is as central to cryptography as the hardness assumptions themselves.

1.8 Symmetric vs. Asymmetric: A First Look

There are two great families of cryptographic schemes, distinguished by how keys are arranged. We preview them now and devote later chapters to each.

Before naming the families, let us write down what an encryption scheme even *is*, as a piece of syntax. This three-algorithm template recurs for every scheme in the book, so it is worth fixing now.

Definition 1.1 (Symmetric encryption scheme)**.** A symmetric (private-key) encryption scheme is a triple of algorithms $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ over a message space $\mathcal{M}$, key space $\mathcal{K}$, and ciphertext space $\mathcal{C}$:

- Gen outputs a key $k \in \mathcal{K}$, typically by sampling uniformly at random: $k \xleftarrow{\$} \mathcal{K}$.
- Enc takes a key k and message $m \in \mathcal{M}$ and outputs a ciphertext $c \leftarrow \mathrm{Enc}_k(m)$. It may be randomized – and for serious security it *must* be.
- Dec takes a key k and ciphertext c and outputs a message $m \leftarrow \mathrm{Dec}_k(c)$.

The scheme is *correct* if decryption inverts encryption: for every key k in the support of Gen and every $m \in \mathcal{M}$,

$$\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = m.$$

Correctness is the part beginners never forget; security is the part the definition above says *nothing* about. The shift cipher of §1.3 fits this template perfectly and is correct – and useless. Syntax is not security. With the template in hand, the two families differ in just one place: how the key for Enc relates to the key for Dec.

Symmetric (secret-key) cryptography. Alice and Bob share one secret key k. The same k encrypts and decrypts:

$$c = \mathrm{Enc}_k(m), \qquad m = \mathrm{Dec}_k(c).$$

Symmetric schemes are fast – modern CPUs encrypt gigabytes per second – and they are how the bulk of real data gets protected. AES is the dominant example. The catch is the *key distribution*

problem: before they can talk, Alice and Bob must already share k through some secure channel. If they could do that reliably, why not just send the message that way? For two people who can meet in person once, fine. For a browser connecting to a website it has never contacted, hopeless.

Asymmetric (public-key) cryptography. Each party has a *key pair*: a public key pk that anyone may know, and a private key sk kept secret. The keys are mathematically linked so that what one does, only the other can undo. For encryption: anyone encrypts to Bob using his public key, and only Bob, holding sk, can decrypt:

$$c = \mathrm{Enc}_{pk_B}(m), \qquad m = \mathrm{Dec}_{sk_B}(c).$$

For signatures the roles flip: Alice signs with her *private* key, and anyone can verify with her public key – giving authentication and non-repudiation in one stroke. Public-key crypto (RSA, Diffie-Hellman, elliptic curves) solves key distribution: Bob publishes pk_B to the world, and strangers can send him secrets with no prior shared secret. The cost is speed – asymmetric operations are orders of magnitude slower than symmetric ones.

Intuition

Symmetric is one key shared by two trusting parties – like a single lock with two copies of its key. *Asymmetric* is a mailbox: anyone can drop a letter through the public slot (the public key), but only the holder of the physical key (the private key) can open it and take the letters out. The mailbox slot can be photographed and broadcast on television; it does not help the thief open the box.

In practice, real systems are *hybrid*. Because public-key operations are slow, you use them only to do the hard part – agree on a fresh symmetric key over an insecure channel – and then switch to fast symmetric encryption for the actual data. Every HTTPS connection works this way: a public-key handshake establishes a shared secret, then AES carries the gigabytes. We get the key-distribution win of asymmetric crypto and the speed of symmetric crypto in one design.

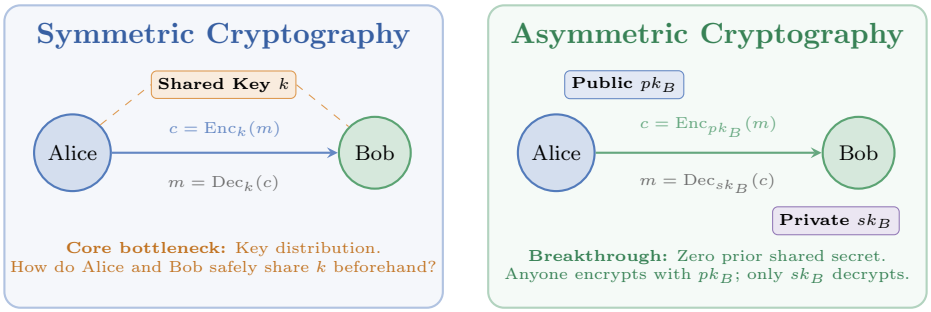


Figure 1.3: Symmetric vs. asymmetric architecture. In symmetric schemes, one shared key does both encryption and decryption, creating a chicken-and-egg distribution problem. In asymmetric schemes, the key pair splits the roles: public encryption is open to all, while private decryption remains bound to a single secret.

History

For all of recorded history until the 1970s, *all* cryptography was symmetric, and the key-distribution problem was solved by couriers, codebooks, and trust – expensively and unreliably.

The conceptual leap that public keys were even *possible* came in 1976 from Whitfield Diffie and Martin Hellman (building on a suggestion by Ralph Merkle), and shortly after in the RSA scheme of Rivest, Shamir, and Adleman. We later learned that researchers at the British intelligence agency GCHQ (James Ellis, Clifford Cocks, Malcolm Williamson) had discovered essentially the same ideas between 1970 and 1973, but their work stayed classified for over two decades. The public discovery sparked a fight with the US National Security Agency, which tried to restrict open academic publication under export controls; when open research prevailed, it made web commerce, TLS, and secure messaging possible, and it is itself an example of Kerckhoffs's principle at work. Today the main open challenge for public-key cryptography is post-quantum migration: because Shor's algorithm on a quantum computer can factor integers and compute discrete logarithms in polynomial time, standard systems are moving toward lattice-based hardness assumptions.

1.9 What Does "Secure" Even Mean?

We have used the word "secure" a dozen times without defining it, and it is easy to misuse. Cryptographic history is full of schemes that everyone, including experts, *felt* were secure until they were broken. Intuition is not enough. The entire modern field exists to replace "I can't see how to break it" with "I can prove no efficient attacker can break it, or breaking it would solve a problem the whole world believes is hard."

Why intuition fails

Consider a tempting argument. "I XOR my message with a long pseudo-random keystream. The keystream looks completely random, so the ciphertext looks completely random, so the attacker learns nothing." That feels airtight. But it quietly assumes the attacker only ever sees *one* ciphertext and never sees any plaintext. Reuse the keystream on two messages m_1, m_2, and the attacker computes

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2,$$

and the key has vanished, leaving the two plaintexts XORed together – often enough to recover both. The intuition was not wrong, exactly; it answered the wrong threat model. This exact mistake – keystream reuse – has broken real systems repeatedly (the WEP WiFi protocol, the wartime Soviet one-time-pad reuse that the VENONA project exploited, Microsoft's PPTP).

Common trap

"It looks random" is not a security argument. "I tried for an afternoon and couldn't break it" is not a security argument. "It's been used for years" is not a security argument (A5/1 was). The only thing that counts is a precise claim about a precise adversary, backed by either a proof or, at minimum, sustained public failure to break it despite strong incentives. Anything less is hope.

The three ingredients of a security definition

A modern security definition always specifies three things together. Drop any one and the claim becomes meaningless.

1. **A security goal** – exactly what the attacker must fail to do. For encryption the gold standard is *indistinguishability*: the attacker, having chosen two equal-length messages m_0, m_1, cannot tell which one a given ciphertext encrypts, any better than flipping a coin. This is far stronger than "cannot recover the plaintext," and deliberately so: it forbids leaking even a single bit, even the answer to "is the salary above $100k?"

2. **A threat model** – exactly what the attacker may do, from §1.6: ciphertext-only, chosen-plaintext, chosen-ciphertext, and how much computation they get.

3. **A quantified bound** – exactly how badly they fail. We do not demand the attacker *never* succeeds; with enough luck anyone can guess a key. We demand their advantage over guessing is *negligible*: it shrinks faster than any inverse polynomial in the key length. Double the key length and the attacker's success chance should drop sharply.

Putting these together: a scheme is *secure* if every efficient (polynomial-time) attacker, operating within the stated threat model, achieves only negligible advantage at the stated goal. That is the shape of every real theorem in this book.

Formal

We will write security as a game. A challenger and an adversary Adv interact by fixed rules; Adv "wins" if it achieves the goal. The scheme is secure if for every probabilistic polynomial-time (PPT) adversary,

$$\Pr[\,\mathsf{Adv}\text{ wins}\,] \le \tfrac{1}{2} + \mathsf{negl}(n),$$

where n is the security parameter (think key length) and $\mathsf{negl}(n)$ is a function smaller than $1/p(n)$ for every polynomial p and all large n. The $\frac{1}{2}$ is the trivial coin-flip baseline for a yes/no challenge; the content of the definition is that no clever strategy beats coin-flipping by more than a vanishing margin. We unfold this machinery carefully in the chapters on perfect and computational secrecy. For now, absorb the *shape*: quantify over all efficient adversaries, bound their advantage by a negligible function.

The indistinguishability game deserves a picture, because its structure is the skeleton of nearly every security proof you will see. The adversary picks two messages it considers maximally telling; the challenger secretly flips a coin b, encrypts m_b, and hands back the ciphertext; the adversary must guess b. If even the cleverest adversary cannot beat a coin flip by a non-negligible margin, the ciphertext is leaking nothing useful about which message it hides.

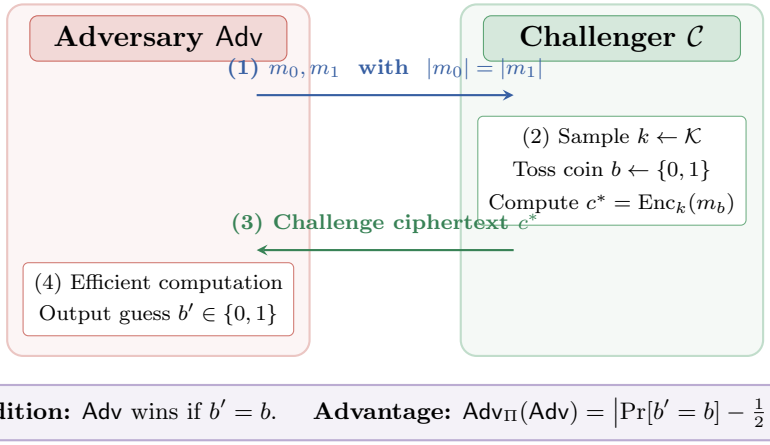


Figure 1.4: The indistinguishability game (IND-EAV / IND-CPA). The adversary picks two candidate messages of equal length; the challenger encrypts one chosen by a fair coin flip. Security guarantees that no efficient algorithm can guess the coin any better than pure chance.

It pays to be careful about *advantage* versus raw probability, because the distinction matters. In a yes/no game an adversary that ignores the ciphertext and guesses $b' \xleftarrow{\$} \{0,1\}$ already wins with probability $\frac{1}{2}$. So $\Pr[b' = b] = \frac{1}{2}$ means "learned nothing," not "always wrong." We therefore measure the *advantage*

$$\mathsf{Adv} = \left|\Pr[b' = b] - \tfrac{1}{2}\right|,$$

the margin by which the adversary beats blind guessing. Security demands Adv be negligible. Concretely, an adversary winning with probability 0.5000001 has advantage 10^{-7} – tiny, plausibly tolerable. An adversary winning with probability 0.75 has advantage 0.25 – a quarter of the way to certainty, completely broken. And an adversary winning with probability 0.5 exactly has advantage 0 – the ideal. Reading a security theorem is largely reading a bound on this one quantity.

Example 1.1 (Why deterministic encryption loses the game)**.** Suppose Enc is *deterministic*: encrypting the same message always yields the same ciphertext (the shift cipher is like this). Then the game is trivial to win. The adversary submits any two distinct messages $m_0 \neq m_1$, and – if it is allowed even a single chosen-plaintext query, or simply knows the encryptions of m_0 and m_1 from earlier traffic – it precomputes $\mathrm{Enc}_k(m_0)$, compares it to the challenge ciphertext c, and answers $b' = 0$ if they match and $b' = 1$ otherwise. It wins with probability 1, not $\frac{1}{2}$. The advantage is $\frac{1}{2}$, the largest possible – catastrophically non-negligible. This is the formal reason *secure encryption must be randomized*: the same plaintext encrypted twice must (with overwhelming probability) give different ciphertexts, or an attacker learns when you repeat yourself. "This message equals that earlier one" is exactly the kind of bit indistinguishability forbids.

Connection

The word "negligible" has a precise probabilistic meaning developed in the companion probability book: a function $\mathsf{negl}(n)$ that is eventually smaller than every inverse polynomial $1/n^c$. The reason this is the right cutoff is an amplification argument – if an attacker had even a $1/n^c$ chance per try, repeating a polynomial number of times would boost success to near certainty, so anything non-negligible is, for a patient attacker, eventually fatal. The probability companion also develops the *birthday bound*, which dictates that a hash function needs a $2n$-bit output to resist collision-finding at the 2^n level, and the union and Markov bounds

we use constantly to push success probabilities around in reductions. Keep that book within reach; every security definition in this book is a quantitative probability statement.

Two routes to confidence: perfect and computational security

There are two ways to back a security claim.

Information-theoretic (perfect) security promises that the ciphertext reveals *nothing* about the plaintext, even to an adversary with unlimited computing power. Shannon proved the one-time pad achieves this – but at a brutal price: the key must be as long as the message and never reused. Perfect security is real, provable, and mostly impractical.

Computational security – the foundation of modern practical protocols – promises something more modest and more useful: the ciphertext might in principle reveal the plaintext, but extracting it would require more computation than any feasible adversary commands. In practical cryptography, these proofs are almost always *conditional*: "if factoring large integers is hard, then RSA is secure." We do not know how to prove factoring is hard outright (that would resolve deep open questions in complexity theory), but we reduce the security of a whole scheme to one clean, ancient, heavily-studied problem. If the scheme breaks, one of a short list of famous problems must have fallen – and we would all have much bigger news to discuss.

Why it works

Why reductions are central to the field. Suppose I prove: "any efficient algorithm that breaks my encryption scheme can be transformed into an efficient algorithm that factors large numbers." Then breaking my scheme is *at least as hard* as factoring. I have not proven factoring is hard – nobody has – but I have concentrated all the risk into a single, well-studied assumption that the world's best mathematicians have failed to crack for centuries. That is enormously more trustworthy than "I, personally, couldn't break it." A reduction converts "trust my cleverness" into "trust the failure of every number theorist since Fermat." This is the discipline that separates cryptography from codemaking. Almost every theorem you meet here has the form: *if problem P is hard, then scheme S is secure*, proved by turning an attacker on S into a solver for P (Figure 1.5).

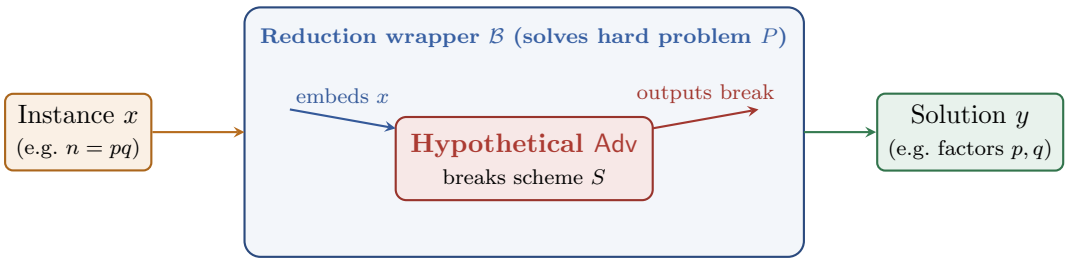


Figure 1.5: Proof by reduction. A wrapper algorithm $\mathcal{B}$ takes a hard-problem instance x, embeds it into a simulated challenge for a hypothetical attacker Adv against scheme S, and translates Adv's break into a solution y for P. Because P is believed hard, no efficient Adv can exist.

Implement it

Here is the keystream-reuse pitfall in a few lines of Python, so you can *see* the key dissolve. This is a demonstration of a **broken** use of encryption – the very mistake §1.9 warns against. **Learning only: never reuse a keystream, and never ship this.**

```python
import os

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

m1 = b"attack at dawn!!"
m2 = b"retreat at noon."   # same length as m1

# THE BUG: one keystream reused for two messages.
keystream = os.urandom(len(m1))
c1 = xor(m1, keystream)
c2 = xor(m2, keystream)

# The attacker never sees the key, yet:
leak = xor(c1, c2)
# leak == m1 XOR m2, completely independent of the key.
assert leak == xor(m1, m2)

# If the attacker guesses (or knows) m1, m2 falls out for free:
recovered_m2 = xor(leak, m1)
assert recovered_m2 == m2
print("recovered:", recovered_m2)   # b"retreat at noon."
```

The key was a perfect, unguessable random string, but that did not help: the *usage* violated the threat model the one-time pad requires (each key used once). Security depends on definitions and correct usage, not on how random the key looks.

1.10 The Security Mindset

Everything above points to a habit of thought. A builder asks "how do I make this work?" A cryptographer asks "how does this fail, and who makes it fail?" The security mindset is the reflex to look at any system and immediately hunt for the gap between what the designer assumed and what an adversary can actually do.

Concretely, when you meet a scheme, interrogate it:

- **What exactly is the secret, and what happens when it leaks?** If the answer is "the algorithm," stop – Kerckhoffs already lost this for you.
- **Who is the attacker, precisely?** Passive Eve or active Mallory? What attack model – ciphertext-only or chosen-ciphertext? Name the strongest realistic adversary, then check the scheme against *that* one, not the polite one.
- **What is the security goal, stated as something the attacker must fail to do?** If you cannot state a crisp win condition for the adversary, you cannot reason about security at all.

- **Where is the proof, or the assumption?** Is the claim "provably as hard as factoring," or merely "nobody's broken it yet"? Both can be acceptable – AES has no reduction, only a long unbroken public record – but you must know which one you are leaning on.
- **What did the model ignore?** The math can be perfect while the implementation leaks the key through timing, power draw, or a careless error message. The most elegant proof says nothing about a side channel it never modeled.

Attack

Padding-oracle attacks: when the model and the world diverge. A cipher can be provably secure against a chosen-plaintext attacker and still be torn apart in deployment. The padding-oracle attack targets systems that, on receiving a malformed ciphertext, reveal – through an error message, a timing difference, or a behavioral quirk – *whether the decryption padding was valid*. That single leaked bit per query, repeated, lets an attacker decrypt traffic without ever knowing the key. It works because real systems often act as a *chosen-ciphertext oracle* that the original "CPA-secure" analysis never accounted for. The proof assumed one model, the deployed system presented another, and the exploit lived in the difference. The fix – authenticated encryption that rejects tampered ciphertexts before they reach the padding check – closes the gap by enlarging the model to match reality.

Intuition

A cryptographer is professionally paranoid in a specific, disciplined way. Not "everything is dangerous" (that is paralysis) but "state your assumptions, then assume the adversary violates every one you did not nail down." The discipline is what turns paranoia into proofs. You assume the worst about the adversary so that you can make precise, defensible promises about the system – and then you test those promises by handing your design to the smartest enemies you can find and inviting them to break it.

In the real world

The Signal messaging protocol is a good example of this mindset in a widely deployed system. Beyond encrypting messages, it is designed to resist a very strong attacker. *Forward secrecy*: if your phone is seized and keys extracted today, yesterday's messages stay safe, because each message used a key already deleted. *Post-compromise security* ("future secrecy"): even after a temporary key compromise, the protocol heals and future messages regain protection as fresh key material ratchets in. These are not goals a casual designer would think to chase – they answer the question "what if the attacker eventually gets the key anyway?" Asking that question early and answering it in the design is what the security mindset looks like in practice. We build toward Signal's ratchet in Chapter 27.

1.11 Composing the Goals: A Single Message, Several Guarantees

Real messages usually need more than one guarantee at once, and the order in which you bolt mechanisms together is itself a security decision with a wrong answer. Consider Alice sending Bob a message that must be *both* confidential (Eve learns nothing) *and* integrity-protected (Mallory cannot alter it undetected). She has two primitives: an encryption scheme for confidentiality and a message-

authentication code (MAC), a keyed function that produces a short tag $t = \mathrm{MAC}_k(x)$ which Bob can recompute to verify x was not tampered with. How should she combine them?

There are three natural orders, and they are not equally safe.

- **Encrypt-and-MAC**: send $\mathrm{Enc}_{k_1}(m)$ alongside $\mathrm{MAC}_{k_2}(m)$. The tag is computed on the *plaintext* and sent in the clear-ish next to the ciphertext. Problem: a MAC guarantees integrity, not secrecy, so the tag may leak information about m – and since the MAC of a fixed m is fixed, it can betray when Alice repeats a message. Fragile.
- **MAC-then-encrypt**: compute $t = \mathrm{MAC}_{k_2}(m)$, then send $\mathrm{Enc}_{k_1}(m \,\|\, t)$. Better, but the tag protects the plaintext, so Bob must *decrypt first* to check integrity – meaning he runs his decryption machinery on attacker-controlled ciphertext before he knows it is authentic. That is exactly the opening the padding-oracle attack of §1.10 exploited.
- **Encrypt-then-MAC**: compute $c = \mathrm{Enc}_{k_1}(m)$, then $t = \mathrm{MAC}_{k_2}(c)$, and send $c \,\|\, t$. Bob checks the tag on the *ciphertext* first; if it fails, he discards the message *without ever decrypting*. The attacker cannot get the decryption machinery to run on anything it forged. This order is the one modern practice settled on, and with the right primitives it is provably secure.

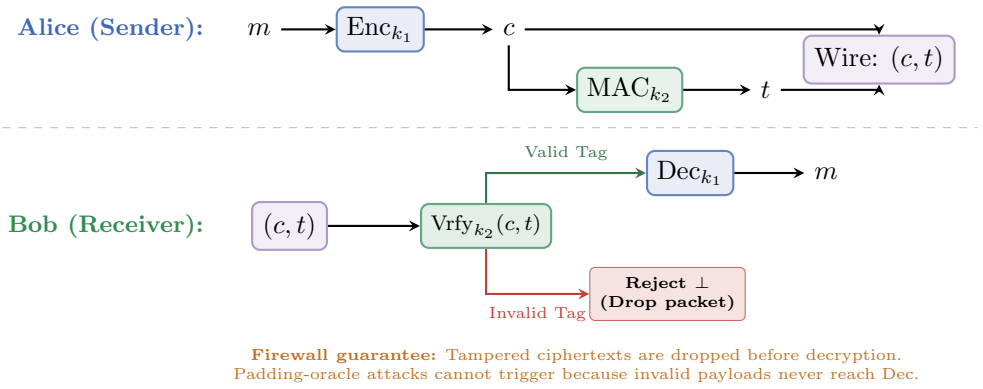


Figure 1.6: Encrypt-then-MAC (EtM). Tagging the ciphertext creates a cryptographic firewall: Bob verifies authenticity *before* handing the ciphertext to the decryption routine, cutting off padding-oracle attacks at the door.

We will prove the relevant theorems later, in Chapters 15 and 16. The general lesson is that *combining secure pieces does not automatically yield a secure whole*. Composition has rules, and ignoring them reintroduces exactly the attacks each piece was meant to stop. Modern designs sidestep the footgun by providing a single primitive, *authenticated encryption with associated data* (AEAD) – algorithms like AES-GCM and ChaCha20-Poly1305 – that delivers confidentiality and integrity together, in the proven-correct order, behind one interface. When you reach for crypto in real code, reach for an AEAD, not a bare cipher.

In the real world

Every modern TLS connection (TLS 1.3, the version behind today's HTTPS) uses exactly one category of symmetric primitive for record protection: an AEAD cipher, almost always AES-GCM or ChaCha20-Poly1305. The protocol designers removed the older, error-prone MAC-then-encrypt constructions *from the standard entirely*, because a decade of padding-

oracle and related attacks showed that leaving the composition to implementers was a reliable source of vulnerabilities. TLS 1.3 thus applies the lesson of this section directly: give people the safe composition as the only option, and the whole class of ordering bugs disappears.

1.12 What's Ahead

That completes the groundwork. In Chapter 2 we step through the history of secret writing – from Caesar and Vigenère through Enigma – to see why intuitive ciphers collapsed under frequency analysis and structural leaks. After that, Chapters 3 through 6 build the mathematics secrecy is made of (modular arithmetic, primes and Euclid's algorithm, groups, and finite fields), followed by the one-time pad and perfect secrecy, stream and block ciphers (AES), MACs and hash functions, public-key schemes (Diffie–Hellman, RSA, elliptic curves) and signatures, and finally the protocols that run the internet: TLS, WiFi, Signal, and SSH. Keep the four goals, the cast, Kerckhoffs's principle, and the three-part security definition in your pocket. Every chapter ahead is a variation on the questions of this one: secure against whom, given what, proven how?

1.13 Exercises

Exercise 1.1 (Classify the goal)**.** For each scenario, name which of the four goals (confidentiality, integrity, authentication, non-repudiation) is the *primary* one at stake, and name any secondary goals also required. (a) A software vendor publishes a download alongside a value so users can check the file was not corrupted or swapped. (b) You want to email a colleague a document so that no eavesdropper can read it. (c) A bank needs to prove in court that a customer authorized a wire transfer. (d) A login server must be sure the client connecting really holds the account password and is not an impostor.

Exercise 1.2 (Break a shift cipher by hand)**.** You intercept the ciphertext `WKHUHLVQRVHFXULWB` and know only that a shift cipher (§1.3) was used. (a) Recover the key k and the plaintext by exhaustive search over the 25 useful shifts. (Hint: the plaintext is an English sentence, spaces removed.) (b) Explain why the size of the keyspace gave you no protection here. (c) The substitution cipher has $26! \approx 2^{88}$ keys, yet is also broken easily. Name the single property both ciphers share that makes frequency analysis work, and describe what a cipher's output must look like to defeat it.

Exercise 1.3 (Deterministic encryption is doomed)**.** Using the indistinguishability game of Figure 1.4 and Example 1.1, prove that *any* deterministic encryption scheme fails the game whenever the adversary can obtain the encryption of at least one message of its choosing. State the adversary's exact strategy and compute its winning probability. Then explain in one sentence what randomization buys you.

Exercise 1.4 (Eve versus Mallory)**.** Give a concrete example of a system that is secure against Eve (a passive eavesdropper) but completely broken by Mallory (an active attacker). Describe exactly what Mallory does. Then explain in one sentence why confidentiality alone never implies integrity.

Exercise 1.5 (Refute the obscurity argument)**.** A startup advertises: "Our encryption is unbreakable because our algorithm is a closely guarded trade secret that no attacker has ever seen." Write a short rebuttal grounded in Kerckhoffs's principle. Give at least two distinct concrete failure modes their

argument ignores, and explain why a published algorithm can be *more* trustworthy than a secret one.

Exercise 1.6 (Feel the numbers)**.** A key-search machine tries 10^{12} keys per second (a trillion per second, far beyond any real device per chip). (a) How many years to exhaust a 64-bit keyspace (2^{64} keys)? (b) How many to exhaust an 80-bit keyspace? (c) How many to exhaust a 128-bit keyspace, and how does that compare to the age of the universe ($\approx 1.4 \times 10^{10}$ years)? Use $2^{10} \approx 10^3$ and one year $\approx 3.15 \times 10^7$ seconds. What do your answers say about why standards moved from 64-bit to 128-bit keys?

Exercise 1.7 (Break the reused keystream by hand)**.** Two 6-bit messages are encrypted with the *same* 6-bit keystream k by XOR. You intercept $c_1 = 101101$ and $c_2 = 110011$. (a) Compute $c_1 \oplus c_2$ and explain why the keystream k has disappeared from the result. (b) Suppose you also learn that $m_1 = 011010$. Recover both k and m_2. (c) State, in terms of the threat model, exactly which assumption the encryptor violated.

Exercise 1.8 (Rank the attack models)**.** Order these four attacker capabilities from weakest to strongest, and for each give a one-line real-world scenario in which an attacker would actually have that capability: ciphertext-only, known-plaintext, chosen-plaintext, chosen-ciphertext. Then explain why proving security against the *strongest* model is the conservative choice.

Exercise 1.9 (Why "can't recover the plaintext" is too weak)**.** Consider an encryption scheme that perfectly hides every *character* of a message except that the ciphertext always leaks the message's exact length and whether the first character is a vowel. Argue that this scheme could satisfy "the attacker cannot recover the full plaintext" yet still be unacceptable. Tie your answer to why cryptographers use *indistinguishability* (the attacker cannot tell m_0 from m_1) as the goal instead.

Exercise 1.10 (Reductions, conceptually)**.** Explain in your own words why a theorem of the form "if factoring is hard, then this scheme is secure" gives more confidence than "I tried hard to break this scheme and could not." In your explanation, identify what is being *assumed* and what is being *proven*, and state what we would learn about the world if someone exhibited an efficient attack on the scheme.

Exercise 1.11 (Hybrid reasoning)**.** HTTPS uses slow public-key cryptography to establish a shared key, then fast symmetric cryptography (AES) for the actual data. (a) Why not use public-key cryptography for the whole conversation? (b) Why not use symmetric cryptography for the whole conversation, skipping public-key entirely? (c) What problem does the public-key step solve that pure symmetric cryptography cannot, for two parties who have never met?

Exercise 1.12 (Entropy versus length)**.** A system generates "256-bit keys," but a flaw causes the generator to seed itself only from the current time in whole seconds, drawn from a known 30-day window. (a) Roughly how many distinct keys can the system actually produce? Express it as a power of two. (b) How many bits of *effective* security does the 256-bit key really provide? (c) An attacker can test 10^9 keys per second. About how long to try them all? Relate your answer to why §1.7 insists that key *length* and key *entropy* are different things.

Exercise 1.13 (Order matters)**.** Alice wants both confidentiality and integrity and considers the three combinations in §1.11: encrypt-and-MAC, MAC-then-encrypt, and encrypt-then-MAC. (a) For MAC-then-encrypt, explain precisely why Bob is forced to run decryption on attacker-supplied data before he can detect tampering, and connect this to the padding-oracle attack. (b) For encrypt-then-MAC, explain why a forged ciphertext never reaches the decryption routine. (c) State the general principle this illustrates about composing secure building blocks, and name the modern primitive (a single interface) that removes the choice from implementers.

Exercise 1.14 (Apply the mindset)**.** A messaging app claims end-to-end encryption. List five questions you would ask, drawn from §1.10, before believing the messages are actually protected against a determined adversary. For each question, briefly state what a *bad* answer would reveal about a hidden weakness.

Exercise 1.15 (Side channels versus the model)**.** A server runs a provably CPA-secure cipher, but it decrypts incoming ciphertexts and replies with either "ok" or "bad padding," and the "bad padding" reply comes back measurably faster. (a) Which attack model from §1.6 does this behavior hand the attacker, regardless of what the proof assumed? (b) Explain why the security proof can be entirely correct while the deployment is still broken. (c) Name the general defense that would close this gap.

Chapter 2

A Short History of Secret Writing

In Chapter 1, we saw that modern cryptography demands precise definitions of secrecy and computationally bounded adversaries. Before building the mathematics of those modern guarantees, we need to understand where they came from – and why intuitive attempts to hide secrets failed. For most of human history, hiding a message meant moving letters around. There were no computers, no notion of a "key space" measured in bits, and no proofs. There were clever people on one side trying to keep a secret, and equally clever people on the other side trying to read it. That contest is worth studying closely. Every cipher in this chapter was, in its day, considered strong by serious people, and every one of them was eventually torn open. The way each one fell teaches a lesson that survives into the modern schemes we build later in this book: where the secret really lives, why reusing it is fatal, and why "it looks scrambled to me" is not a security argument.

We will work the classic breaks by hand, with small numbers and real letters, because the intuition behind frequency analysis and the Kasiski test is exactly the same intuition you need to reason about modern attacks. The tools change; the way an attacker thinks does not.

2.1 The cast and the rules of the game

Recall from Chapter 1 our cast of characters and the four standard threat models. **Alice** wants to send a message to **Bob**; **Eve** sits on the wire between them and copies down everything that passes. Alice and Bob share some secret in advance; Eve does not have it. The question is always the same: can Eve recover the message anyway?

It helps to view classical ciphers in terms of these threat models, because almost every historical failure came from underestimating Eve's access.

Definition 2.1 (Attack models in historical context)**.** In Chapter 1, we classified an attacker by what information she commands. In historical cryptanalysis, these models emerge in direct sequence:

- *Ciphertext-only.* Eve sees only encrypted messages. This was the default position for ancient military couriers and classical frequency analysis.
- *Known-plaintext.* Eve sees some plaintext/ciphertext pairs (for example, she guesses that a message ends "Heil Hitler" or begins "WEATHER REPORT").
- *Chosen-plaintext.* Eve can get messages of her choosing encrypted under the unknown key.

- *Chosen-ciphertext.* Eve can get ciphertexts of her choosing decrypted (or at least learn whether they decrypt validly).

Focus

A modern scheme must survive chosen-plaintext (and usually chosen-ciphertext) attacks. Every classical cipher in this chapter dies to a ciphertext-only attack – the weakest model there is. Keep that gap in mind; it measures how far the field traveled.

There is one more principle worth keeping in mind, because the entire history of secret writing validates it: Kerckhoffs's principle.

Formal

Kerckhoffs's principle (1883). A cipher should remain secure even if everything about the system except the key is public knowledge. The Dutch linguist Auguste Kerckhoffs laid out six rules for military ciphers in the *Journal des sciences militaires*; this is the one that endured. Claude Shannon restated it crisply: "the enemy knows the system." The practical reading: do not bet your secret on the secrecy of your *method*. Methods leak, get captured, get reverse-engineered, or get sold. Only the key should be secret, and the key should be easy to change. Nearly every disaster in this chapter is a violation of this rule.

2.2 The Caesar cipher: shift and pray

The oldest cipher most people know is named for Julius Caesar, who according to Suetonius shifted each letter of his military dispatches three places down the alphabet. `A` becomes `D`, `B` becomes `E`, and so on, wrapping `X`, `Y`, `Z` around to `A`, `B`, `C`.

To do anything with this mathematically, number the letters $\texttt{A} = 0, \texttt{B} = 1, \ldots, \texttt{Z} = 25$. Encryption with shift k is then arithmetic modulo 26:

$$c \equiv m + k \pmod{26}, \qquad m \equiv c - k \pmod{26}.$$

Caesar used $k = 3$. The key is just the single number k, and there are only 26 possible shifts (one of which, $k = 0$, does nothing).

Example 2.1 (Encrypting by hand)**.** Take the plaintext `ATTACK` and $k = 3$.

plain	A	T	T	A	C	K
m	0	19	19	0	2	10
$m + 3 \bmod 26$	3	22	22	3	5	13
cipher	D	W	W	D	F	N

So `ATTACK` encrypts to `DWWDFN`. To decrypt, subtract 3: `D`→`A`, `W`→`T`, and you are back.

A shift cipher is easiest to picture as two concentric alphabet rings: line up the plaintext `A` on the inner ring with the ciphertext letter k positions around on the outer ring, and reading off any plaintext letter gives its ciphertext. Figure 2.1 shows the $k = 3$ alignment of Example 2.1.

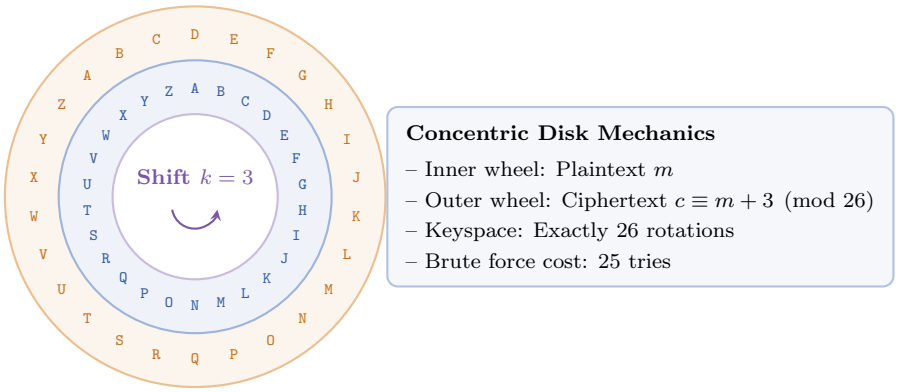


Figure 2.1: The Caesar cipher as two concentric alphabet rings. With $k = 3$, the inner (plaintext) `A` lines up against outer (ciphertext) `D`. Because only 26 distinct relative rotations exist, an adversary can exhaust every candidate key in seconds.

2.2.1 Why it fails: there is nowhere for the secret to hide

The Caesar cipher has a key space of size 26. An attacker simply tries all of them.

Attack

Brute force in 25 tries. Eve intercepts `WKHWUHDVXUHLVEXULHG`. She writes out all 25 non-trivial shifts of the first few letters and reads down the list for English:

$k = 1$: `VJGVTGCUWTG...`
$k = 2$: `UIFUSFBTVSF...`
$k = 3$: `THETREASURE...` ← readable!

Shift by $k = 3$ and the message reads `THETREASUREISBURIED`. She did not need any insight into Alice's method; she only needed to try 25 shifts.

The lesson recurs throughout cryptography: *the number of possible keys must be too large to enumerate.* Twenty-six is far too few. Even by hand a person clears all 25 shifts in under a minute. This is the first appearance of a quantity we will obsess over for the rest of the book – the size of the key space – and the first demonstration that if it is small, nothing else you do matters.

Connection

"Too large to enumerate" is a statement about counting, which is the subject of the companion combinatorics chapters. A modern 128-bit key has $2^{128} \approx 3.4 \times 10^{38}$ possibilities. If you could test a trillion keys per second on a billion machines, you would still need on the order of 10^{10} years – comfortably longer than the age of the universe. That gap, from 26 keys to 2^{128}, is what makes brute force hopeless against a modern cipher. We make this computational security model precise in Chapter 8.

2.3 General substitution: a huge key that still loses

The obvious fix for a tiny key space is to make it enormous. Instead of shifting the whole alphabet by one fixed amount, pick an arbitrary permutation: a secret one-to-one map from each plaintext letter to a ciphertext letter. This is the *monoalphabetic substitution cipher*.

A worked key might be:

plain	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R	S	T	U	V	W	X	Y	Z
cipher	Q	W	E	R	T	Y	U	I	O	P	A	S	D	F	G	H	J	K	L	Z	X	C	V	B	N	M

Now the key is the entire bottom row, and there are

$$26! = 403{,}291{,}461{,}126{,}605{,}635{,}584{,}000{,}000 \approx 4 \times 10^{26}$$

possible keys. That is about 88 bits. Brute force is hopeless – you cannot try 4×10^{26} alphabets by hand or, in 1500, by any means at all. For centuries people believed this made the cipher unbreakable. They were wrong, and the reason is the most important idea in classical cryptanalysis.

2.3.1 Frequency analysis: the key space is irrelevant if the cipher leaks

Here is the flaw. A monoalphabetic substitution maps every `E` in the plaintext to the same ciphertext letter, every `T` to the same letter, and so on. It scrambles *which* symbol stands for `E`, but it cannot hide *how often* that symbol appears. English is lopsided: `E` is about 12.7% of letters, `T` about 9.1%, `A` about 8.2%, while `J`, `Q`, `X`, `Z` are each well under 1%. The cipher carries this fingerprint straight through.

Intuition

Imagine a room where everyone swaps name tags while nobody changes height. If you know the tallest person in the room is Eve, you find Eve by looking for the tallest person, no matter what her name tag says. Letter frequencies are heights the substitution cannot change. Count the most common ciphertext symbol; it is overwhelmingly likely to be `E`. The 88-bit key did not help, because we never had to guess it – we read it off the statistics.

Al-Kindi and the birth of frequency analysis (c. 850 CE). The earliest surviving description of frequency analysis is by the Arab polymath Abu Yusuf al-Kindi, in a manuscript on deciphering encrypted messages written around 850 CE. His insight: count the symbols, line them up against the known frequencies of the language, and the substitution unravels. This was the first general cipher-breaking technique in history, and it predates the European cryptography it would later demolish by six centuries. It was also the first time anyone used *statistics* as a weapon, an idea that runs straight through to modern cryptanalysis.

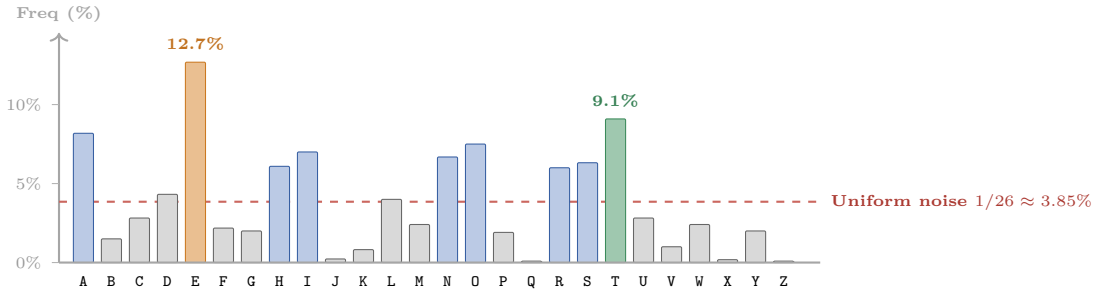


Figure 2.2: The frequency signature of written English. Monoalphabetic substitution merely permutes these bar heights across different letters, preserving the overall shape of the distribution. An attacker identifies high peaks (such as `E` and `T`) and rare letters (`Q`, `X`, `Z`) to recover the key without brute force.

2.3.2 Breaking one by hand

Let us actually do it. Eve intercepts the following ciphertext (spaces left in only so you can follow along; a real interceptor would strip them, and the attack still works):

```
ZG WT GK FGZ ZG WT ZIQZ OL ZIT JXTLZOGF
```

Step 1: count letters. Tally every symbol (ignore spaces; there are 30 letters).

```
Z : 7   G : 5   T : 4   W : 2   F : 2   I : 2   O : 2   L : 2
K : 1   Q : 1   J : 1   X : 1
```

`Z` dominates with 7 occurrences, then `G` with 5 and `T` with 4. In English, the runaway leaders are `E` and `T`, with `O` and `A` close behind.

Step 2: exploit structure, not just counts. Raw frequency is a starting hypothesis, not the whole attack. The real power comes from short words and repeated patterns. Notice the two-letter word `ZG` appears twice, and a three-letter word `ZIT` appears. The most common English two-letter words are `TO`, `OF`, `IN`, `IS`, `IT`, `BE`; the most common three-letter word is `THE`. A three-letter word whose first letter is the most common ciphertext letter is a prime suspect for `THE`. `ZIT` fits if `Z` → `T`, `I` → `H`, `T` → `E`. That is consistent with the counts: `Z` was the most common symbol (so `Z` → `T` is plausible) and `T` was third (so `T` → `E` fits a high-frequency letter). It also reads the repeated two-letter word as `ZG` = `T?`, and the only common English two-letter word starting with `T` is `TO`, forcing `G` → `O` – and `G` was the second most common symbol, exactly where `O` sits in English. Three independent checks agree. Promising.

Step 3: propagate guesses. Assume `Z` → `t`, `I` → `h`, `T` → `e`, `G` → `o`. Rewrite, lowercase for known plaintext:

```
to We oK Fot to We tIQt OL the JXeLtOoF
```

The repeated word `to We` now appears twice and `Fot` sits between them. `Fot` is almost certainly `not`, giving `F` → `n`. Then `We` is `?e`; after "to be or not to be" leaps to mind, try `W` → `b`, so `We`=`be`, and `oK` becomes `o?` = `or` with `K` → `r`. Rewrite:

```
to be or not to be tIQt OL the JXeLtOoF
```

Now `tIQt` is `t?et`-shaped: `th` was `ZI`, so `I`=`h` gives `tHQt`=`th?t`, which resolves to `that` with `Q` → `a`. The word `OL` is `?s`-shaped after we guess the next piece; `JXeLtOoF` is an eight-letter word ending the sentence after `the`, with `e` in slot 3 and `t` in slot 5 – `??e?t??n` – which resolves to `question` with `J` → `q`, `X` → `u`, `L` → `s`, `O` → `i` (and `F`=`n`, already fixed). That also reads `OL`=`is`. Fill in everything:

```
to be or not to be that is the question

TO BE OR NOT TO BE THAT IS THE QUESTION
```

We never tried even one of the 4×10^{26} keys. We recovered the substitution letter by letter, using frequencies to seed guesses and word structure to confirm or refute them. A few hundred characters of ciphertext is plenty; with a thousand, the attack is mechanical.

Why it works

Why the giant key space did not save the cipher. Security is not the same as "many keys." The substitution cipher leaks a statistic – the frequency of each letter – that is preserved exactly under the secret permutation. An attacker who can read off the answer never has to search the key space at all. The modern restatement: a cipher is broken if the ciphertext reveals *any* information about the plaintext beyond its length. Counting letters reveals an enormous amount. This is precisely the property Shannon would later forbid with his definition of perfect secrecy, and which modern ciphers achieve by making the output statistically indistinguishable from random bits.

Implement it

A toy frequency counter. This sorts ciphertext letters by how often they appear, which is the seed of the manual attack above.

```python
# LEARNING ONLY -- this is a teaching toy, never use it to protect anything.
from collections import Counter

ENGLISH = "etaoinshrdlcumwfgypbvkjxqz"  # roughly most-to-least common

def frequency_attack(ciphertext):
    letters = [c for c in ciphertext.upper() if c.isalpha()]
    counts = Counter(letters)
    # most common ciphertext letter -> guess it is 'E', and so on
    ranked = [c for c, _ in counts.most_common()]
    guess = {}
    for cipher_letter, plain_letter in zip(ranked, ENGLISH):
        guess[cipher_letter] = plain_letter
    decoded = "".join(guess.get(c, c.lower()) for c in ciphertext.upper())
    return decoded, guess

text = "ZG WT GK FGZ ZG WT ZIQZ OL ZIT JXTLZOGF"
print(frequency_attack(text))
# A first pass; you then hand-correct using word patterns, exactly as above.
```

Real attacks add bigram/trigram statistics and dictionary matching, but the core is this histogram. Preserved letter frequencies are the fatal leak: any scheme that preserves letter proportions preserves the language's statistical signature.

In the real world

Where you still meet substitution. The newspaper "cryptoquote" puzzle is exactly a monoalphabetic substitution, solvable by the method above. More seriously, naive "encryption" that maps each byte to a fixed other byte – some legacy file formats, some toy obfuscation in malware, ROT13 in Usenet – is a substitution cipher and falls to frequency analysis or simple known-plaintext. Whenever a system claims to "encrypt" but a repeated input always produces the same scrambled output, you are looking at a substitution-style leak. Modern block-cipher modes (Chapter 13) exist specifically to eliminate this determinism.

2.4 The Vigenere cipher: stacking shifts to flatten the fingerprint

If the problem with substitution is that each plaintext letter always maps to the same ciphertext letter, the natural fix is to make the mapping *change* as you go. The Vigenere cipher does this with a repeating keyword. It was described by Giovan Battista Bellaso in 1553, later misattributed to Blaise de Vigenere, and earned the nickname *le chiffre indechiffrable* – the indecipherable cipher – because it resisted frequency analysis for three centuries.

Focus

Three hundred years of false confidence. From the mid-1500s until the mid-1800s the Vigenere cipher was widely believed unbreakable. Charles Babbage broke it around 1854 but did not publish (some think at the request of British intelligence during the Crimean War). The Prussian infantry officer Friedrich Kasiski published the method independently in 1863, and it bears his name. This chapter makes the point repeatedly: a cipher being unbroken for a long time is not a proof that it is secure. It may only mean nobody clever has published yet.

2.4.1 How it works

Pick a keyword, say `KEY`. Write it repeatedly under the plaintext, and shift each plaintext letter by the corresponding key letter (treating `A`=0 as "no shift", `B`=1, etc.). Formally, with key letters $k_0, k_1, \ldots, k_{\ell-1}$ of length ℓ,

$$c_i \equiv m_i + k_{(i \bmod \ell)} \pmod{26}.$$

It is a Caesar cipher whose shift cycles through the keyword.

Example 2.2 (Encrypting with `KEY`)**.** Encrypt `ATTACKATDAWN` with keyword `KEY` ($\texttt{K} = 10, \texttt{E} = 4, \texttt{Y} =$

24).

plain	A	T	T	A	C	K	A	T	D	A	W	N
m	0	19	19	0	2	10	0	19	3	0	22	13
key	K	E	Y	K	E	Y	K	E	Y	K	E	Y
k	10	4	24	10	4	24	10	4	24	10	4	24
$m + k \bmod 26$	10	23	17	10	6	8	10	23	1	10	0	11
cipher	K	X	R	K	G	I	K	X	B	K	A	L

Result: KXRKGIKXBKAL.

Consider the three T's in the plaintext (positions 2, 3, 8). The first became X, the second R, the third X again – because the key letter under each was E, Y, E. The same plaintext letter no longer maps to a single ciphertext letter; it splits according to where the key cycle catches it. (The four A's, by contrast, all happen to fall on a key-K position and so all become K – a reminder that the flattening is statistical, not guaranteed letter by letter.) The frequency fingerprint is blurred. Run a frequency count on a long Vigenere ciphertext and the histogram is far flatter than English – which is exactly why al-Kindi's attack stalls.

Intuition

Vigenere is really ℓ independent Caesar ciphers interleaved: positions $0, \ell, 2\ell, \ldots$ all use key letter k_0; positions $1, \ell+1, 2\ell+1, \ldots$ all use k_1; and so on. If you knew where one of these "columns" lived, you would be back to a single Caesar cipher, which you can crack in 25 tries. So the problem reduces to this: *find the key length* ℓ. Once you know ℓ, the cipher collapses into ℓ easy pieces.

Figure 2.3 makes the "ℓ interleaved Caesars" picture concrete on the ciphertext KXRKGIKXBKAL from Example 2.2, whose key KEY has length $\ell = 3$.

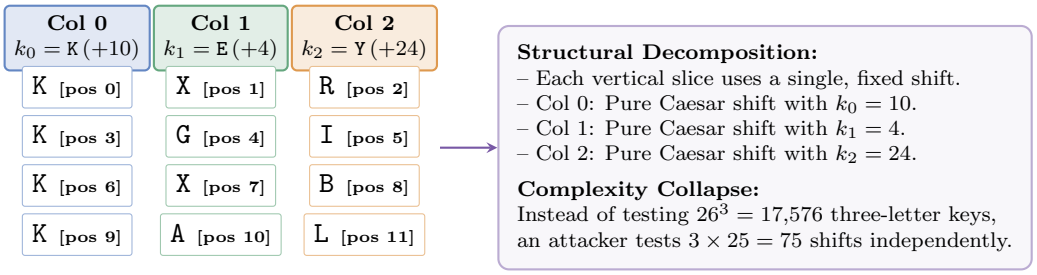


Figure 2.3: A Vigenere ciphertext sliced into $\ell = 3$ columns according to position index modulo 3. Because every character in column j is transformed by the exact same key letter k_j, finding the period ℓ decomposes the polyalphabetic cipher into ℓ independent, trivial monoalphabetic Caesar shifts.

2.4.2 Kasiski and Babbage: finding the key length

The Vigenere cipher hides the key, but it cannot hide a coincidence. Suppose a common word like THE appears twice in the plaintext, and by luck both occurrences line up against the *same* part of the keyword. Then both encrypt to the *same* ciphertext triple. The gap between those two identical triples in the ciphertext must be a whole number of keyword-lengths – it is a multiple of ℓ.

Why it works

Why repeated patterns leak the key length. If plaintext position i and position j hold the same letters *and* are encrypted with the same key letters, the ciphertexts match. Same key letters means $i \equiv j \pmod{\ell}$, i.e. the distance $j - i$ is a multiple of ℓ. So every repeated ciphertext chunk hands you a number that ℓ divides. Collect several such distances; the key length is a common divisor of all of them – usually their greatest common divisor, or a small factor of it. This is the Kasiski test.

Attack

Kasiski test, worked. Suppose Eve, hunting for repeated trigrams in a long ciphertext, finds the trigram `WMV` at positions 5, 21, and 53 (counting from 0), and the trigram `GTH` at positions 9 and 25.
Distances between repeats:

$$\begin{aligned} \texttt{WMV}: &\quad 21 - 5 = 16, \quad 53 - 21 = 32, \quad 53 - 5 = 48, \\ \texttt{GTH}: &\quad 25 - 9 = 16. \end{aligned}$$

Now take the greatest common divisor of the collected distances:

$$\gcd(16, 32, 48, 16) = 16.$$

The key length divides 16, so $\ell \in \{1, 2, 4, 8, 16\}$. A length of 1 is just a Caesar cipher (ruled out, the histogram was flat), so the strong candidates are $\ell = 8$ or $\ell = 16$. Eve tries the smallest plausible value first (Figure 2.4).

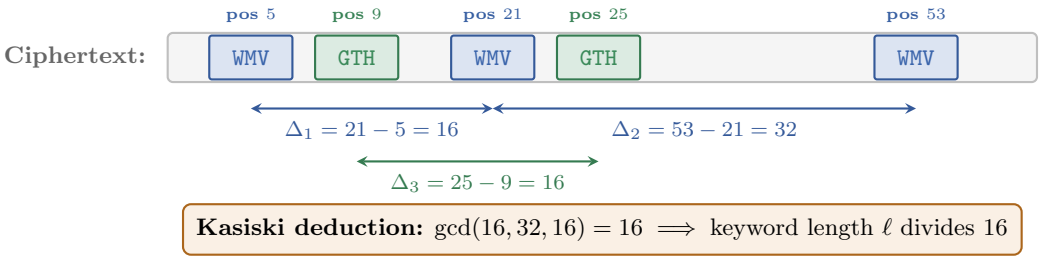


Figure 2.4: The Kasiski test. When identical plaintext trigrams happen to align with the same phase of the repeating keyword, they produce identical ciphertext trigrams (`WMV` and `GTH`). The spacing between repeats must be an integer multiple of the period ℓ, so the greatest common divisor of those spacings exposes ℓ.

Once Eve commits to a guess $\ell = 8$, she splits the ciphertext into 8 columns by position modulo 8. Each column is a pure Caesar cipher, and she breaks each one in turn – either by brute-forcing its 26 shifts and reading for English, or, better, by frequency analysis on that column alone. The most common letter in column j is probably the encryption of `E`, which pins down k_j.

Example 2.3 (Solving one column)**.** Suppose column 0 (positions $0, 8, 16, \ldots$) has these letter counts, with `R` the clear winner. Guess that `R` is the encryption of `E`. Then

$$k_0 \equiv \texttt{R} - \texttt{E} = 17 - 4 = 13 \pmod{26},$$

so $k_0 = 13 = \texttt{N}$. The first keyword letter is `N`. Repeat the same "most-common-letter-is-`E`" deduction on the other seven columns and the whole keyword falls out – say `NIGHTOWL`. If a recovered letter looks wrong (the second-most-common letter might really be `E`, with `T` the winner instead), Eve simply tries the top two or three candidates per column and keeps the keyword that yields readable English.

2.4.3 The Friedman test: counting coincidences

There is a second, quantitative way to estimate the key length, due to William Friedman in 1920, that does not rely on spotting repeats by eye. It is worth seeing because it is the first appearance of a statistic we will reuse constantly: the *index of coincidence*.

Definition 2.2 (Index of coincidence)**.** Given a text of length n with letter counts $n_A, n_B, \ldots, n_Z$, the index of coincidence is the probability that two letters drawn at random (without replacement) are equal:

$$\mathrm{IC} \;=\; \frac{\sum_{x=A}^{Z} n_x(n_x-1)}{n(n-1)}.$$

For text drawn uniformly at random over 26 letters, $\mathrm{IC} \approx 1/26 \approx 0.0385$. For English, the lopsided frequencies push it up to about $\mathrm{IC} \approx 0.0667$. A monoalphabetic substitution *preserves* the English value (it only renames letters), so it sits near 0.066 – a dead giveaway. A Vigenere ciphertext with a long key looks much closer to random, near 0.038.

Why it works

Why the index of coincidence finds the key length. If you write the ciphertext into ℓ columns and ℓ is the true key length, each column is a single Caesar shift of English and therefore has English-like statistics, with $\mathrm{IC} \approx 0.066$. If your guessed ℓ is wrong, the columns mix several different shifts and look random, $\mathrm{IC} \approx 0.038$. So you sweep $\ell = 1, 2, 3, \ldots$, compute the average column IC for each, and watch for the value that jumps up toward 0.066. That ℓ is your key length – no eyeballing of repeated trigrams required. Friedman even gave a direct estimate:

$$\ell \;\approx\; \frac{0.027\,n}{(n-1)\,\mathrm{IC} - 0.038\,n + 0.066}.$$

Connection

The index of coincidence is a collision probability: the chance that two independently chosen items land in the same bucket. That same quantity drives the birthday paradox and hash collisions, covered in the companion probability chapters and revisited when we study hash functions. The Friedman test asks "how surprised am I by how many coincidences I see?" – a question you answer with exactly the combinatorics used to bound a birthday attack.

Implement it

Index of coincidence and a key-length sweep.

```
# LEARNING ONLY -- toy code for understanding, not for protecting real data.
def index_of_coincidence(text):
```

```
    text = [c for c in text.upper() if c.isalpha()]
    n = len(text)
    if n < 2:
        return 0.0
    from collections import Counter
    counts = Counter(text)
    return sum(v * (v - 1) for v in counts.values()) / (n * (n - 1))

def guess_key_length(ciphertext, max_len=20):
    letters = [c for c in ciphertext.upper() if c.isalpha()]
    for ell in range(1, max_len + 1):
        columns = "".join(letters[i::ell]) for i in range(ell)]
        avg_ic = sum(index_of_coincidence(col) for col in columns) / ell
        print(f"key length {ell:2d}: avg IC = {avg_ic:.4f}")
        # English-like (~0.066) flags the true length; random-like is ~0.038
```

Run this on a Vigenere ciphertext and the true key length is the row where the average index of coincidence spikes toward 0.066. Then break each column as a Caesar cipher.

Common trap

"I'll just use a really long key." Lengthening the Vigenere keyword raises the bar but does not change the verdict. If the key is shorter than the message and repeats, Kasiski and Friedman eventually find the period given enough ciphertext. The *only* way a Vigenere key is truly safe is if it is as long as the message, never repeats, and is random – at which point it stops being Vigenere and becomes the one-time pad (Chapter 9). The same lesson applies throughout modern crypto: **a repeating keystream is a broken keystream.** This holds whether the repeated value is a Vigenere keyword, a stream-cipher nonce, or an IV.

In the real world

The same bug, a century later. The exact failure mode of a repeating Vigenere key reappears in modern systems whenever a stream-cipher keystream gets reused. The 2001 break of WiFi's WEP exploited IV reuse in RC4. The "two-time pad" mistake – encrypting two messages with the same keystream – lets an attacker XOR the ciphertexts together and cancel the key, leaving $m_1 \oplus m_2$, which is then separated by the same language-statistics reasoning we used on Vigenere columns. Microsoft's PPTP and several TLS missteps hit variants of this. If you remember one thing from this section: never reuse a keystream, and treat any cipher whose keystream repeats as already broken.

2.5 Rotor machines and Enigma: mechanizing the substitution

By the early twentieth century, cryptographers understood that a fixed substitution was fatal and that you wanted the substitution to change with every letter. Doing that by hand with a long keyword is slow and error-prone. The breakthrough was mechanical: build a machine whose internal wiring *is* a substitution, and make it step to a new substitution after every keypress. These are *rotor machines*, and the most famous is the German Enigma.

History

Enigma. Patented by Arthur Scherbius around 1918 and adopted by the German military through the 1920s and 30s, Enigma was the primary cipher of the Wehrmacht, Luftwaffe, and Kriegsmarine in World War II. Its operators believed the 10^{23} rotor-and-plugboard configurations made it unbreakable. They were wrong: Polish mathematicians Marian Rejewski, Jerzy Różycki, and Henryk Zygalski broke it in 1932, and after Poland shared its methods and replica machines with Britain and France in July 1939, Alan Turing, Gordon Welchman, and thousands of others at Bletchley Park industrialized the attack. Purpose-built machines (the Polish *bomba*, the British *Bombe*) turned cryptanalysis into an engineering discipline and shortened the war by years. Afterward, Britain kept Bletchley's work classified until the 1970s, obscuring Poland's breakthrough while selling captured rotor machines to former allies without revealing they could be read.

2.5.1 What the machine does

A rotor is a disk with 26 contacts on each face, wired so that current entering at one letter exits at some scrambled other letter – a fixed substitution baked into copper. Stack three rotors in series and you compose three substitutions. After each keypress, the rightmost rotor turns one position, so the overall substitution changes every letter. When the right rotor completes a full turn it kicks the middle rotor one step (like an odometer), and the middle eventually kicks the left. Enigma added two more elements:

- A *reflector* at the far end that sent the current back through the rotors a second time. This made encryption and decryption the same operation (press the key, read the lamp) – a great convenience.
- A *plugboard* (*Steckerbrett*) that swapped up to 10 pairs of letters before and after the rotor stack, multiplying the key space enormously.

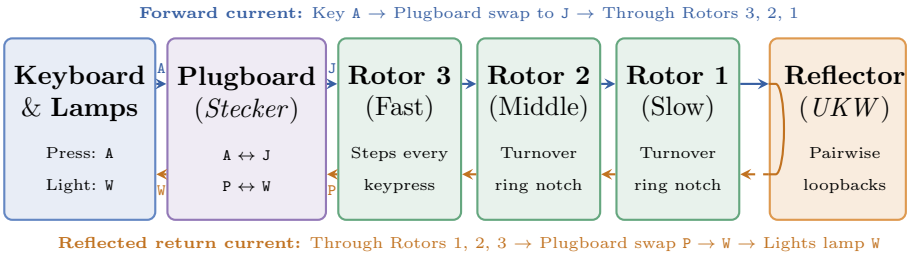


Figure 2.5: The Enigma electro-mechanical signal path. Current leaves the pressed key, traverses the plugboard and three rotors, bounces off the reflector, and returns back through the rotors along a completely distinct electrical circuit to illuminate the lamp. Because the reflector connects contacts only in disjoint pairs, no electrical signal can ever return to the key that initiated it.

The daily key was: which rotors in which order, their starting positions, the ring settings, and the plugboard pairings. Counting it all up, the Enigma key space is around 10^{23} – roughly 77 bits. By the brute-force logic of Section 2.2, that should be safe. It was not, and the reasons show how real systems fail.

2.5.2 The flaws that sank it

Attack

No letter ever encrypts to itself. The reflector wiring guarantees that the output letter is always different from the input letter. This sounds harmless, but it was a serious weakness. It let codebreakers *slide a guessed plaintext (a "crib") along the ciphertext* and instantly discard any alignment where the crib and ciphertext shared a letter in the same position. A single self-mapping anywhere kills that alignment. With common cribs – weather reports always contained `WETTER`, messages often ended `HEILHITLER` – this pruned the search enormously before a single rotor was tried.

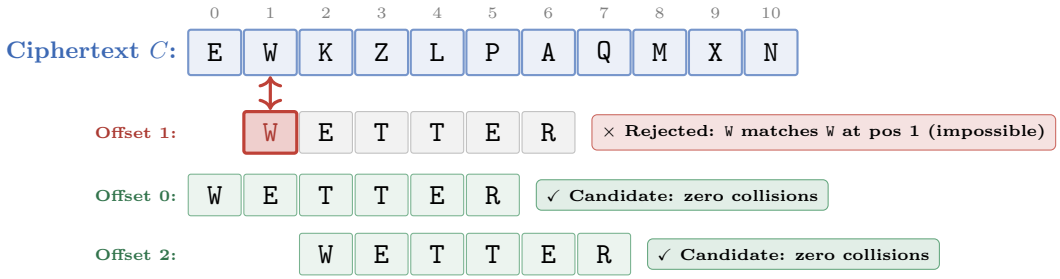


Figure 2.6: Crib-dragging against Enigma ciphertext. Because Enigma's electrical reflector prevents any letter from ever encrypting to itself, testing a suspected crib (such as `WETTER`) against the intercepted stream is instantaneous: any offset that aligns identical letters (like the `W` collision at offset 1) is ruled out immediately with zero rotor testing.

Attack

Operator habits and key reuse. Procedure required each message to carry a per-message rotor start, encrypted under the day key. Early procedure had the operator type this three-letter *message key twice* (e.g. `QWE QWE`) so the receiver could check it. Typing it twice created a fixed relationship between ciphertext positions 1&4, 2&5, 3&6 – exactly the kind of repeated structure we exploited in Vigenere. Rejewski turned this into a system of permutation equations and solved for the wiring. Beyond that, operators reused favorite settings (`CIL` for a girlfriend's name, `BER` for Berlin), chose non-random plugboard pairings, and sent stereotyped messages. Each of these habits gave the codebreakers something to exploit.

Why it works

Why "no self-encryption" is a leak, in our language. A secure cipher's output should look like a uniformly random string, independent of the plaintext except for length. Enigma's ciphertext was *not* uniform: the event "ciphertext letter equals plaintext letter" had probability exactly 0, whereas a random cipher would show it about 1/26 of the time. That is information about the plaintext leaking into the ciphertext – forbidden by the security definitions we build in later chapters. The Bombe was, in effect, a machine for exploiting that single missing 1/26, chaining crib-driven deductions through the rotors and using contradictions (a letter forced to two different plug partners) to reject wrong rotor settings at electrical speed.

Intuition

The Bombe did not try all 10^{23} keys. Given a crib – a guessed plaintext aligned under ciphertext – it set up a *logical chain*: if plug-partner of `A` is `T`, then by following the rotor wiring this forces the partner of `T` to be `G`, which forces ... and if the chain ever loops back and contradicts itself, that rotor setting is impossible. The Bombe mechanized this deduction across all rotor positions, stopping only at settings with no contradiction. The plugboard, which had made the key space huge, barely mattered once a good crib was in hand – because the deductions *ignored* the plug swaps until the very end. A huge key space bought the Germans little against an attack that never searched it.

Focus

The Polish head start. It is easy to picture Enigma as a Bletchley story, but the decisive early work was Polish. Marian Rejewski, handed the doubled-message-key flaw and some intelligence on the wiring, reconstructed Enigma's internal connections in 1932 using group theory – specifically, the cycle structure of permutations, which is invariant under the unknown plugboard and so let him peel the plugboard away from the rotor problem. Poland built electromechanical *bomba* machines and read German traffic for years. In July 1939, weeks before the invasion, Poland handed its results and replica machines to France and Britain. The British work built on that foundation. The lesson for a working cryptographer: the math that breaks your system may already exist in someone's unpublished work.

In the real world

Why Enigma still matters to you. Enigma's downfall was not a weak core algorithm in isolation – it was the *system*: stereotyped messages giving cribs, a structural quirk (no self-encryption), key-management procedures that reused and exposed message keys, and human operators who chose predictable settings. Every modern breach you will read about follows the same template. Heartbleed, the WEP break, the Sony PS3 ECDSA nonce reuse, countless implementation CVEs – the cipher math was usually fine; the system around it leaked. When you deploy crypto, your threat surface is the protocol, the randomness source, the key lifecycle, and the human, not just the cipher. Enigma shows clearly that you cannot reason about a cipher without reasoning about how people use it.

2.5.3 A tiny rotor by hand

To see the mechanism, build a one-rotor toy on a 5-letter alphabet `A B C D E` $= 0, 1, 2, 3, 4$. Let the rotor be the fixed substitution

$$S: \quad \texttt{A} \to \texttt{C},\ \texttt{B} \to \texttt{E},\ \texttt{C} \to \texttt{A},\ \texttt{D} \to \texttt{B},\ \texttt{E} \to \texttt{D},$$

and let the rotor "step" by adding 1 (mod 5) to the input position before applying S and subtracting it after – a stripped-down version of how a rotor's rotation shifts its whole map. Encrypting letter m_i at step i (starting $i = 0$):

$$c_i = \big(S\big((m_i + i) \bmod 5\big) - i\big) \bmod 5,$$

where we read S as a function on $\{0, 1, 2, 3, 4\}$: $S(0) = 2, S(1) = 4, S(2) = 0, S(3) = 1, S(4) = 3$.

Encrypt BAD $= (1, 0, 3)$:

$$
\begin{aligned}
i = 0: &\ S((1+0) \bmod 5) - 0 = S(1) = 4 = \texttt{E},\\
i = 1: &\ S((0+1) \bmod 5) - 1 = S(1) - 1 = 4 - 1 = 3 = \texttt{D},\\
i = 2: &\ S((3+2) \bmod 5) - 2 = S(0) - 2 = 2 - 2 = 0 = \texttt{A}.
\end{aligned}
$$

So BAD $\to$ EDA. The two different plaintext letters here all pass through the *same* wiring S, yet because the rotor stepped, identical inputs at different positions would land differently – the property hand-cranked Vigenere also had, now produced by a machine. Watch, too, that with this S a letter *can* map to itself at some positions; the real Enigma's reflector is what forced the never-itself property that doomed it.

Implement it

The toy rotor, in code, so you can watch the substitution change at each step.

```
# LEARNING ONLY -- a 5-letter cartoon of a rotor, not a real cipher.
S = [2, 4, 0, 1, 3]          # the fixed wiring (a permutation of 0..4)
S_inv = [S.index(j) for j in range(5)]

def enc(plain):
    out = []
    for i, ch in enumerate(plain):
        m = ord(ch) - ord('A')
        c = (S[(m + i) % 5] - i) % 5
        out.append(chr(c + ord('A')))
    return "".join(out)

def dec(cipher):
    out = []
    for i, ch in enumerate(cipher):
        c = ord(ch) - ord('A')
        m = (S_inv[(c + i) % 5] - i) % 5
        out.append(chr(m + ord('A')))
    return "".join(out)

print(enc("BAD"))        # -> EDA
print(dec(enc("BAD")))   # -> BAD
```

Add more rotors (compose several permutations, each stepping at a different rate) and a reflector, and you have the skeleton of Enigma. Add a plugboard (a fixed swap applied before and after) and you have its shape. None of this is secure – it is here so the machine stops being a black box.

2.6 What three thousand years of failure teaches

Taken together, these ciphers show a clear pattern. Every cipher in this chapter was built by capable people, trusted by serious institutions, and broken by attackers who did not behave as the designers expected. The recurring lessons are worth stating plainly, because everything that follows builds on them.

Focus

Five lessons that outlived their ciphers.

1. *The key space must be too large to search* – but a large key space guarantees nothing on its own (Section 2.3, Section 2.5). Caesar lost to brute force; substitution and Enigma had enormous key spaces and lost anyway, because the attacker never searched.
2. *Ciphertext must not leak statistics about plaintext.* Letter frequencies sank substitution; the missing 1/26 self-map sank Enigma; a flatter-but-not-flat histogram and repeated patterns sank Vigenere. Modern security definitions formalize exactly this: ciphertext should be indistinguishable from random.
3. *Reuse is the enemy.* A repeating Vigenere keyword, a doubled Enigma message key, a reused keystream – all the same mistake. Keys and keystreams must not repeat.
4. *Kerckhoffs was right: assume the enemy knows the system.* Every cipher here was eventually understood by the attacker; only key secrecy ever mattered, and key *management* – generation, distribution, rotation, operator discipline – was where systems actually died.
5. *Amateurs lose.* "It looks scrambled and nobody has broken it" is not security. Vigenere went three centuries unbroken and was still broken. You earn confidence from analysis and proof, not from the absence of a published attack.

Why this matters

This history also shows a broader change in method. The early ciphers were defended by *obscurity and effort* – surely no one will try all the shifts, surely no one will count the letters, surely no one will notice the doubled key. Each defense was an appeal to the attacker's laziness or ignorance. The modern field replaces that with something sturdier: we state precisely what the attacker can do, we define exactly what "broken" means, and we prove – or reduce to a hard mathematical problem – that breaking the cipher is as hard as solving that problem. The rest of this book builds that foundation. Still, keep the lessons of Caesar, Vigenere, and Enigma in mind: every one of them looked "visibly fine" to its builders.

Connection

The thread from here forward: substitution's frequency leak motivates Shannon's *perfect secrecy* and the one-time pad (Chapter 9); Vigenère's repeating keystream motivates *stream ciphers* and the absolute rule against keystream reuse (Chapter 10); Enigma's mechanized, ever-changing substitution is the ancestor of the *block cipher* and its round structure (Chapter 11); and the recurring "large key space, broken anyway" is why we define security against polynomial-time adversaries rather than counting keys. Each modern construction is, in part, a direct answer to one of the failures in this chapter.

Every cipher here also had something in common: Caesar shifted letters around a ring of 26 positions ($c \equiv m + k \pmod{26}$), Vigenère cycled through shifts, and Enigma stepped contacts around a 26-letter dial. In Chapter 3 we put this clockwork arithmetic on solid mathematical footing – congruence classes, modular multiplication, and modular inverses – the arithmetic that every modern cipher uses.

Exercises

Exercise 2.1 (Caesar by hand)**.** Encrypt the plaintext `CRYPTO` with a Caesar shift of $k = 7$. Then decrypt the ciphertext `ROVVY` (also $k = 7$) back to plaintext. Show the numeric steps modulo 26.

Exercise 2.2 (Brute-forcing Caesar)**.** You intercept `IBWFGVO`. Knowing only that it is a Caesar cipher of an English word, find the key and the plaintext by trying shifts. How many shifts did you need to test in the worst case, and why is 26 a fatal key-space size?

Exercise 2.3 (Frequency attack)**.** The following is a monoalphabetic substitution of an English sentence: `ZIT DTF QKT QSS OF ZIT ZTFZ`. Count the letters, identify the likely encryption of `E` and `T`, use the short words and the repeated three-letter word `ZIT` and the doubled-letter word `QSS`, and recover the plaintext. State your full substitution map.

Exercise 2.4 (Why 26! does not help)**.** The monoalphabetic key space has size $26! \approx 4 \times 10^{26}$, about 88 bits. Explain in two or three sentences why this enormous number provides essentially no security, and connect your explanation to the notion that ciphertext must not leak statistics about plaintext.

Exercise 2.5 (Vigenere encryption)**.** Encrypt `MEETATDAWN` with the Vigenere keyword `LEMON`. Write the key letters under the plaintext, give the numeric shifts, and produce the ciphertext. Then verify by decrypting your own output.

Exercise 2.6 (Kasiski test)**.** In a Vigenere ciphertext you find the trigram `XQR` repeated at positions 4, 22, and 40, and the trigram `PLM` repeated at positions 7 and 31. Compute all pairwise distances, take their greatest common divisor, and list the candidate key lengths. Which would you try first, and why?

Exercise 2.7 (Index of coincidence)**.** A ciphertext of length $n = 200$ has letter counts that yield $\sum_x n_x(n_x - 1) = 2600$. Compute the index of coincidence. Is this text closer to English (≈ 0.066) or to random (≈ 0.038)? What does that suggest about whether it is a monoalphabetic substitution or a Vigenere cipher with a long key?

Exercise 2.8 (Two-time pad / keystream reuse)**.** Alice encrypts two English messages m_1 and m_2 with the *same* Vigenere keyword (or equivalently the same keystream), producing c_1 and c_2. Show that $c_1 - c_2 \equiv m_1 - m_2 \pmod{26}$ position by position, so the key cancels. Explain how an attacker recovers m_1 and m_2 from $m_1 - m_2$ using language statistics, and name a modern system that suffered this exact failure.

Exercise 2.9 (Enigma's fatal quirk)**.** The Enigma reflector guaranteed that no letter ever encrypts to itself. Suppose you suspect a ciphertext segment `LXGPQ` is the encryption of the crib `ANGRI` (a fragment of `ANGRIFF`). Check whether this alignment is possible under Enigma, and if not, explain how this single rule lets a codebreaker discard crib positions in bulk before testing any rotor setting.

Exercise 2.10 (Toy rotor)**.** Using the five-letter toy rotor of Section 2.5.3 with $S(0) = 2, S(1) = 4, S(2) = 0, S(3) = 1, S(4) = 3$ and the stepping rule $c_i = (S((m_i + i) \bmod 5) - i) \bmod 5$, encrypt the plaintext `CAB`. Then decrypt your result with the inverse rule to confirm you recover `CAB`.

Exercise 2.11 (Design a defense)**.** For each of the three breaks in this chapter – frequency analysis on substitution, the Kasiski test on Vigenere, and crib-dragging on Enigma – name the single property of the cipher that made the attack possible, and state the modern design principle (from the five lessons in Section 2.6) that eliminates it. Keep each answer to one or two sentences.

Exercise 2.12 (Prove a small fact about shifts)**.** Show that composing two Caesar ciphers, first a shift by a then a shift by b, is the same as a single Caesar shift by $(a+b) \bmod 26$. Conclude that the set of Caesar shifts forms a group under composition, and explain why this means "double Caesar" adds no security at all.

Chapter 3

Modular Arithmetic

In Chapter 2, we saw classical ciphers repeatedly shifting letters around a 26-letter alphabet ring: Caesar encrypted by computing $c \equiv m + k \pmod{26}$, and Vigenère cycled through shifts on the same circular dial. That intuition — adding numbers along a finite circle that wraps around — is the foundation of almost every cipher, key-exchange protocol, and signature scheme in modern cryptography. RSA, Diffie–Hellman, AES, and elliptic curves all rest on one simple idea: do arithmetic, but keep only the remainder.

This chapter builds that machinery from the ground up. We start with circular clock arithmetic, work tiny examples entirely by hand, and end with the core algorithm — fast modular exponentiation — that makes public-key cryptography practical. Nothing here is conceptually hard, but it needs to become automatic, because every later chapter relies on it.

3.1 The Clock Already Taught You This

If it is 10 o'clock and you wait 5 hours, it is 3 o'clock — not 15 o'clock. You computed $10 + 5 = 15$, noticed that 15 is past 12, and subtracted 12 to land back on the dial. You have been doing modular arithmetic since you learned to read a clock.

Intuition

Modular arithmetic is ordinary arithmetic on a finite circle of numbers instead of an infinite line. Pick a *modulus* n — the number of positions on the circle. Count $0, 1, 2, \ldots, n-1$, and then wrap back to 0. Every integer, no matter how large or how negative, lands on exactly one of those n positions. "Modulo n" just means: tell me *where on the circle*, and forget *how many times around*.

A clock has modulus 12 (or 24, if you are military or European). A week has modulus 7: three days after Saturday is Tuesday, because Sat = 6, and $6 + 3 = 9$, and 9 wraps to 2 = Tue. A byte has modulus 256: add 1 to 255 and an unsigned 8-bit register rolls over to 0. That rollover, usually treated as a bug, is exactly the behavior this chapter studies.

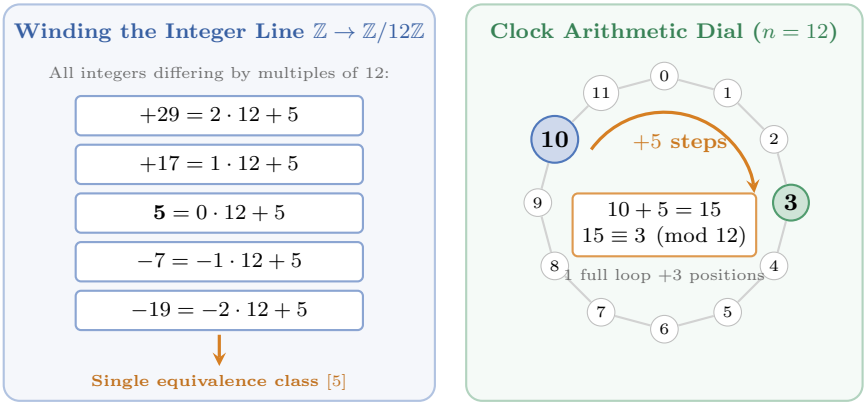


Figure 3.1: The geometry of modular arithmetic. Left: The infinite line of integers wraps infinitely around the modulus $n = 12$, collapsing all numbers that differ by multiples of 12 into a single residue class [5]. Right: Arithmetic on the finite ring $\mathbb{Z}/12\mathbb{Z}$ simply steps around the circle; adding 5 to 10 rolls past 0 to land on 3.

3.2 The Integers Modulo n

Let us make this precise. Fix a positive integer n, the modulus.

Definition 3.1 (Division with remainder)**.** For any integer a and any positive integer n there is a *unique* pair of integers q (the quotient) and r (the remainder) with

$$a = qn + r, \qquad 0 \le r < n.$$

We write $a \bmod n$ for that remainder r.

The constraint $0 \le r < n$ is what makes q and r unique — it forces the remainder onto exactly one position of the circle. So $17 \bmod 12 = 5$ because $17 = 1 \cdot 12 + 5$. And $-1 \bmod 12 = 11$, because $-1 = (-1) \cdot 12 + 11$ and we are required to pick the remainder in $\{0, \ldots, 11\}$, not the negative one your calculator might hand you.

Common trap

Many programming languages compute the remainder with a sign that follows the *dividend*, so `-1 % 12` returns `-1` in C, but `11` in Python. Mathematicians always mean the non-negative remainder $0 \le r < n$. When you implement crypto in a language with C-style `%`, normalise with `((a % n) + n) % n` or you will eventually leak a sign bit into a place it does not belong.

3.2.1 Congruence: the equality of modular arithmetic

Two integers that land on the same circle position are, for our purposes, the same. We give that relationship a name and a symbol.

Definition 3.2 (Congruence)**.** Integers a and b are *congruent modulo* n, written

$$a \equiv b \pmod{n},$$

if n divides $a - b$. Equivalently, a and b have the same remainder when divided by n.

So $17 \equiv 5 \pmod{12}$ and $17 \equiv 29 \pmod{12}$ and $17 \equiv -7 \pmod{12}$: all of $5, 17, 29, -7$ sit on position 5 of the 12-clock, because any two of them differ by a multiple of 12. The symbol $\equiv$ is read "is congruent to," and the $\pmod{n}$ tag tells you which circle you are standing on. Note the difference from the operator $a \bmod n$, which returns a single number in $\{0, \dots, n-1\}$: the relation $\equiv$ compares two numbers, while mod *reduces* one.

Intuition

Think of $a \equiv b \pmod{n}$ as "a and b are interchangeable, as long as we only ever care about remainders mod n." Everything we build will respect this interchangeability, which is exactly why we are allowed to reduce huge numbers down to small ones at every step — the small representative behaves identically to the giant one.

3.2.2 Congruence is an equivalence relation

Congruence behaves like equality, and we should check that, because we are about to treat it like equality constantly.

Proposition 3.1 (Congruence is an equivalence relation)**.** *For all integers a, b, c and a fixed modulus n:*

- ***Reflexive:*** $a \equiv a \pmod{n}$.
- ***Symmetric:*** *if* $a \equiv b$ *then* $b \equiv a \pmod{n}$.
- ***Transitive:*** *if* $a \equiv b$ *and* $b \equiv c$ *then* $a \equiv c \pmod{n}$.

Proof. Reflexivity: $n \mid (a-a) = 0$. Symmetry: if $n \mid (a-b)$ then $n \mid -(a-b) = (b-a)$. Transitivity: if $n \mid (a-b)$ and $n \mid (b-c)$, then n divides their sum $(a-b)+(b-c) = a-c$. □

Because congruence partitions the integers into n classes (one per circle position), we call each class a *residue class*. The set of all n classes is written $\mathbb{Z}/n\mathbb{Z}$, read "$\mathbb{Z}$ mod n," and we usually represent each class by its non-negative remainder:

$$\mathbb{Z}/n\mathbb{Z} = \{0, 1, 2, \dots, n-1\}.$$

When we "compute in $\mathbb{Z}/n\mathbb{Z}$," we do arithmetic and then immediately reduce the answer back into that set. The reason we are allowed to do this — and to reduce *whenever we like*, even halfway through a calculation — is the next result.

3.3 Addition, Subtraction, Multiplication All Survive Wrap-around

The single most useful fact in this chapter is that congruence is compatible with the arithmetic operations. If you replace any number by a congruent one, sums and products do not notice.

Theorem 3.1 (Congruences add and multiply)**.** *If* $a \equiv a' \pmod{n}$ *and* $b \equiv b' \pmod{n}$*, then*

$$a+b \equiv a'+b' \pmod{n}, \qquad a-b \equiv a'-b' \pmod{n}, \qquad ab \equiv a'b' \pmod{n}.$$

Proof. By hypothesis $n \mid (a - a')$ and $n \mid (b - b')$. Write $a - a' = kn$ and $b - b' = \ell n$.

For addition, $(a + b) - (a' + b') = (a - a') + (b - b') = (k + \ell)n$, a multiple of n. Subtraction is identical with a minus sign.

For multiplication, the trick is to add and subtract a bridging term:

$$ab - a'b' = ab - a'b + a'b - a'b' = (a - a')\,b + a'\,(b - b') = kn\,b + a'\,\ell n = n\,(kb + a'\ell),$$

again a multiple of n. □

Why it works

Theorem 3.1 justifies every modular-reduction shortcut in this book. It says you may reduce mod n *as early and as often as you want.* Want $123 \cdot 456 \bmod 10$? You do not need the full product 56088. Reduce first: $123 \equiv 3$, $456 \equiv 6$, so the answer is $3 \cdot 6 = 18 \equiv 8 \pmod{10}$. Keeping numbers small at every step is the only reason a 2048-bit RSA exponentiation fits in memory.

Example 3.1 (Reduce early, reduce often)**.** Compute $(38 + 27) \cdot 19 \bmod 7$ two ways.

The naive way: $38 + 27 = 65$, then $65 \cdot 19 = 1235$, then $1235 = 176 \cdot 7 + 3$, so the answer is 3.

Reducing early: $38 \equiv 3$, $27 \equiv 6$, $19 \equiv 5 \pmod 7$. So $(3 + 6) \cdot 5 = 9 \cdot 5$. Reduce again: $9 \equiv 2$, giving $2 \cdot 5 = 10 \equiv 3 \pmod 7$. Same answer, and the largest number we ever wrote down was 10 instead of 1235.

3.3.1 Addition and multiplication tables for a tiny modulus

The full operation tables of a small ring are a good way to build intuition. Here are the complete addition and multiplication tables for $\mathbb{Z}/6\mathbb{Z}$. Read row a, column b; the entry is $a+b$ (or ab) reduced mod 6.

Addition mod 6

+	0	1	2	3	4	5
0	0	1	2	3	4	5
1	1	2	3	4	5	0
2	2	3	4	5	0	1
3	3	4	5	0	1	2
4	4	5	0	1	2	3
5	5	0	1	2	3	4

Multiplication mod 6

×	0	1	2	3	4	5
0	0	0	0	0	0	0
1	0	1	2	3	4	5
2	0	2	4	0	2	4
3	0	3	0	3	0	3
4	0	4	2	0	4	2
5	0	5	4	3	2	1

The addition table is a perfect Latin square: every row and column is a permutation of $\{0, \ldots, 5\}$. This happens for a structural reason — adding a fixed number is just a rotation of the circle, which can never collide.

The multiplication table is more revealing, because its rows split cleanly into two behaviors. Look at the rows for 1 and 5: each is a permutation of $\{0, \ldots, 5\}$, hitting every value. Now look at the rows for 2, 3, 4: they repeat themselves and never produce a 1. The numbers 1 and 5 are the elements you can "divide by"; the others produce collisions (Figure 3.2).

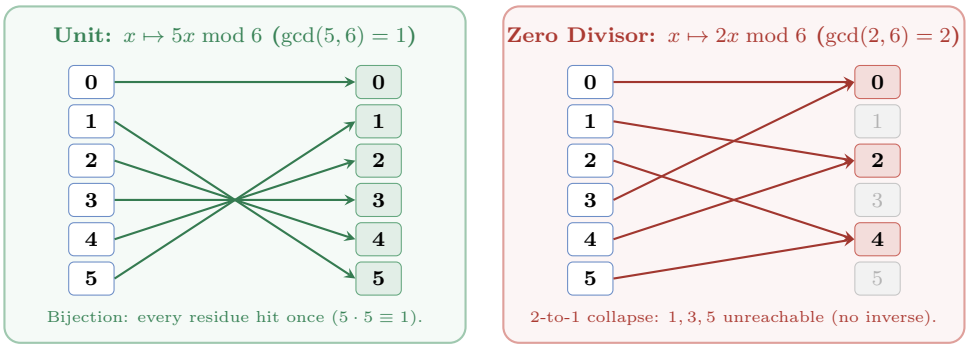


Figure 3.2: Multiplying by a unit versus a zero divisor in $\mathbb{Z}/6\mathbb{Z}$. Because $\gcd(5,6) = 1$, multiplication by 5 permutes all six residues cleanly and hits 1, so 5 is invertible. Because $\gcd(2,6) = 2$, multiplication by 2 folds pairs of inputs together onto $\{0, 2, 4\}$, missing 1 entirely and destroying unique decryption.

3.4 Where Division Goes Wrong

On the integers you cannot always divide and stay an integer ($7/2$ is not an integer), so you learn early to be careful. Modular arithmetic has its own, sharper version of this caution, and getting it wrong is a classic source of broken cryptography.

The cancellation law you rely on over the rationals — "if $ax = ay$ then $x = y$" — can *fail* mod n. Look at the multiplication table for $\mathbb{Z}/6\mathbb{Z}$: $2 \cdot 1 = 2$ and $2 \cdot 4 = 8 \equiv 2$. So $2 \cdot 1 \equiv 2 \cdot 4 \pmod 6$, yet $1 \not\equiv 4$. We multiplied two different things by 2 and got the same answer. You cannot "cancel the 2."

Common trap

$a \equiv b \pmod n$ does *not* let you cancel a common factor freely. From $2 \cdot 2 \equiv 2 \cdot 8 \pmod{12}$ (both sides are 4, since $16 \equiv 4$) you cannot conclude $2 \equiv 8 \pmod{12}$ — and indeed $2 \not\equiv 8$. Cancellation is only legal when the factor you cancel is *invertible* mod n, which is exactly what the next section pins down.

3.4.1 Zero divisors

The cause is the *zero divisor*: a nonzero element whose product with some other nonzero element is 0. In $\mathbb{Z}/6\mathbb{Z}$ we have $2 \cdot 3 = 6 \equiv 0$, even though neither 2 nor 3 is zero. Zero divisors are exactly the elements that ruin cancellation, and they appear precisely when n is composite and the element shares a factor with n.

This is why, over and over in cryptography, we will insist on a *prime* modulus p, or on working only with numbers *coprime* to n. Under those conditions, zero divisors vanish and division works again.

3.5 Modular Inverses and Division

To divide by a means to multiply by the thing that undoes a. Over the rationals, "the thing that undoes 7" is $1/7$, because $7 \cdot (1/7) = 1$. Modular arithmetic copies this definition exactly.

Definition 3.3 (Modular inverse)**.** An integer a has a *multiplicative inverse* modulo n if there is an integer x with

$$ax \equiv 1 \pmod{n}.$$

When it exists, we write the inverse as $a^{-1} \bmod n$. It is unique modulo n.

For example, $5^{-1} \bmod 6 = 5$, because $5 \cdot 5 = 25 = 4 \cdot 6 + 1 \equiv 1 \pmod 6$. The number 5 is its own inverse on the 6-clock. By contrast, 2 has *no* inverse mod 6: scan the row for 2 in the table above — it produces $0, 2, 4, 0, 2, 4$ and never a 1. There is simply no x with $2x \equiv 1 \pmod 6$.

When does an inverse exist? The complete answer is clean and is one of the most-used facts in this book.

Theorem 3.2 (Existence of inverses)**.** *The integer a has a multiplicative inverse modulo n if and only if* $\gcd(a, n) = 1$ *— that is, if and only if a and n are* coprime *(share no common factor other than 1).*

Proof. ($\Leftarrow$) Suppose $\gcd(a, n) = 1$. Bézout's identity (proved in Chapter 4) says there exist integers x, y with $ax + ny = 1$. Reducing this mod n kills the ny term: $ax \equiv 1 \pmod n$. So x is the inverse.

($\Rightarrow$) Conversely, suppose $ax \equiv 1 \pmod n$, so $ax = 1 + kn$ for some integer k, i.e. $ax - kn = 1$. Any common divisor d of a and n divides the left side, hence divides 1, so $d = 1$. Thus $\gcd(a, n) = 1$. □

Why it works

This theorem is why cryptographers favor prime moduli. If p is prime, then *every* nonzero residue $1, 2, \ldots, p - 1$ is coprime to p, so every nonzero element has an inverse. Division by any nonzero number works perfectly. That makes $\mathbb{Z}/p\mathbb{Z}$ a *field*, which is why Diffie–Hellman, ElGamal, and DSA all live mod a large prime. We will give this its proper name (a finite field) in Chapter 6; for now, just register that "prime modulus ⇒ division always works."

3.5.1 The group of units

The coprime residues are important enough to get their own name and symbol.

Definition 3.4 (Units mod n)**.** The *units* modulo n are the residues coprime to n. They form a set

$$(\mathbb{Z}/n\mathbb{Z})^* = \{\, a \in \mathbb{Z}/n\mathbb{Z} : \gcd(a, n) = 1 \,\},$$

and under multiplication mod n they form a group: closed, with identity 1, and every element invertible.

For $n = 6$, the units are $\{1, 5\}$ — exactly the two rows of the multiplication table that were permutations. For a prime p, the units are everything nonzero: $(\mathbb{Z}/p\mathbb{Z})^* = \{1, 2, \ldots, p-1\}$, which has $p-1$ elements. The size of $(\mathbb{Z}/n\mathbb{Z})^*$ is written $\varphi(n)$ (Euler's totient), and it will star in the next chapter and again in RSA. For now, the one number to remember: $|(\mathbb{Z}/p\mathbb{Z})^*| = p - 1$ for prime p.

3.5.2 Computing an inverse by hand

You can always find an inverse for small n by brute force — just try every residue — but the principled tool is the *extended Euclidean algorithm*, which produces the Bézout coefficients x, y in $ax + ny = \gcd(a, n)$. We develop that algorithm in full in Chapter 4; here is the idea on a concrete case, to show that computing an inverse is purely mechanical.

Example 3.2 (Inverting 17 modulo 43)**.** We want x with $17x \equiv 1 \pmod{43}$. First run the ordinary Euclidean algorithm to confirm $\gcd(17, 43) = 1$, recording each quotient:

$$\begin{aligned} 43 &= 2 \cdot 17 + 9 \\ 17 &= 1 \cdot 9 + 8 \\ 9 &= 1 \cdot 8 + 1 \\ 8 &= 8 \cdot 1 + 0. \end{aligned}$$

The last nonzero remainder is 1, so they are coprime and an inverse exists. Now back-substitute to write 1 as a combination of 17 and 43:

$$\begin{aligned} 1 &= 9 - 1 \cdot 8 \\ &= 9 - 1 \cdot (17 - 1 \cdot 9) = 2 \cdot 9 - 1 \cdot 17 \\ &= 2 \cdot (43 - 2 \cdot 17) - 1 \cdot 17 = 2 \cdot 43 - 5 \cdot 17. \end{aligned}$$

So $-5 \cdot 17 + 2 \cdot 43 = 1$, which means $17 \cdot (-5) \equiv 1 \pmod{43}$. Normalise the negative: $-5 \equiv 38 \pmod{43}$. Check: $17 \cdot 38 = 646 = 15 \cdot 43 + 1$. Indeed $646 \equiv 1$. So $17^{-1} \bmod 43 = 38$.

Example 3.3 (A second inverse, and a sanity check)**.** Find $7^{-1} \bmod 26$ — the multiplier Bob will need to break the affine cipher later. The Euclidean descent is short:

$$\begin{aligned} 26 &= 3 \cdot 7 + 5 \\ 7 &= 1 \cdot 5 + 2 \\ 5 &= 2 \cdot 2 + 1 \\ 2 &= 2 \cdot 1 + 0. \end{aligned}$$

Back-substitute from the last nonzero remainder:

$$\begin{aligned} 1 &= 5 - 2 \cdot 2 \\ &= 5 - 2 \cdot (7 - 1 \cdot 5) = 3 \cdot 5 - 2 \cdot 7 \\ &= 3 \cdot (26 - 3 \cdot 7) - 2 \cdot 7 = 3 \cdot 26 - 11 \cdot 7. \end{aligned}$$

So $-11 \cdot 7 \equiv 1 \pmod{26}$, and $-11 \equiv 15 \pmod{26}$. Check *every* inverse you compute by multiplying it back. $7 \cdot 15 = 105 = 4 \cdot 26 + 1 \equiv 1$. Confirmed, $7^{-1} \bmod 26 = 15$. If that final product is not 1, you have an arithmetic slip or a non-coprime input — never ship an inverse you have not multiplied back.

Common trap

A widespread implementation bug: code calls an `inverse(a, n)` routine, gets back *some* number, and uses it without checking $\gcd(a, n) = 1$. When a and n share a factor, a naive

extended-Euclid routine returns a value satisfying $ax \equiv \gcd(a, n) \pmod{n}$, *not* $\equiv 1$ — silently wrong. Downstream the "decryption" produces garbage or, worse, a value that happens to pass a weak check. Always either guarantee coprimality by construction (prime modulus, or a drawn from the units) or test that $a \cdot x \equiv 1$ before trusting the result.

Connection

The extended Euclidean algorithm runs in $O(\log n)$ division steps — it is one of the oldest and fastest algorithms known. The companion *algorithms* book analyses why the number of steps is bounded by the logarithm of the inputs (the worst case is consecutive Fibonacci numbers, which shrink the slowest). That logarithmic cost is what lets us compute inverses on 2048-bit numbers in microseconds, which RSA decryption and signature verification both rely on.

In the real world

Modular inverses appear throughout cryptography. RSA's private exponent d is the inverse of the public exponent e modulo $\varphi(n)$. Every ECDSA and DSA signature divides by the nonce k, i.e. multiplies by k^{-1} modulo the group order; if that inverse step is implemented with a non-constant-time algorithm, the timing leaks bits of k and the long-term key can be recovered (a real, repeatedly-exploited bug class, covered in Chapter 37). The unglamorous "find the inverse" step matters.

3.6 Modular Exponentiation

Raising to a power mod n — compute $a^k \bmod n$ — is the single operation that public-key cryptography is built on. RSA encryption is $m^e \bmod n$. Diffie–Hellman is $g^x \bmod p$. Get comfortable with it.

Because of Theorem 3.1, exponentiation respects congruence too: if $a \equiv a' \pmod{n}$ then $a^k \equiv (a')^k \pmod{n}$ for any non-negative integer k (just multiply the base congruence by itself k times). So once again we may reduce the base mod n first, and reduce after every multiplication, keeping all intermediate numbers below n^2.

Example 3.4 (Powers of 3 mod 7, by hand)**.** Multiply by 3 and reduce each time:

$$3^1 = 3, \quad 3^2 = 9 \equiv 2, \quad 3^3 \equiv 2 \cdot 3 = 6, \quad 3^4 \equiv 6 \cdot 3 = 18 \equiv 4,$$

$$3^5 \equiv 4 \cdot 3 = 12 \equiv 5, \quad 3^6 \equiv 5 \cdot 3 = 15 \equiv 1.$$

The powers cycle: $3, 2, 6, 4, 5, 1$, and then $3^7 \equiv 3$ again, repeating with period 6. Notice we hit every nonzero residue mod 7 before returning to 1 — so 3 is a *generator* of $(\mathbb{Z}/7\mathbb{Z})^*$, exactly the property Diffie–Hellman mod 7 needs. Notice too that $3^6 \equiv 1$, with the exponent $6 = p - 1$: that is Fermat's little theorem peeking through, which the next chapter proves.

3.6.1 Order, and why the powers cycle

The fact that the powers of 3 mod 7 returned to 1 after exactly six steps is a structural feature that cryptography exploits. The smallest positive k with $a^k \equiv 1 \pmod{n}$ is called the *order* of a, written

$\text{ord}_n(a)$. For a unit, this k always exists (the powers of a unit cannot run forever without repeating in a finite set, and the first repeat must be a return to 1). Once $a^k \equiv 1$, the sequence repeats with period k.

Example 3.5 (Orders mod 7, by inspection)**.** We computed the cycle of 3: $3, 2, 6, 4, 5, 1$, length 6, so $\text{ord}_7(3) = 6$. Compare 2: $2^1 = 2$, $2^2 = 4$, $2^3 = 8 \equiv 1$ — order 3, a short cycle hitting only $\{1, 2, 4\}$. And $6 \equiv -1$ has $6^2 = 36 \equiv 1$, order 2. The orders $1, 2, 3, 6$ that appear are exactly the divisors of $6 = p - 1$; no element has order 4 or 5. That every order divides $p - 1$ is Lagrange's theorem in miniature, and it is why $a^{p-1} \equiv 1$ (Fermat) holds for *every* nonzero a, not just the generators.

An element whose order is the full $p - 1$, like 3 mod 7, is a *generator* (or *primitive root*): its powers sweep out every nonzero residue. Diffie–Hellman needs such a generator so that g^x ranges over the whole group as x varies; an element of small order would confine the shared secret to a tiny, brute-forceable subset. We will return to choosing generators and to the subtle subgroup pitfalls in Chapter 23.

3.6.2 Why you cannot just multiply k times

Computing $g^x \bmod p$ where x is a 256-bit number by doing $x - 1$ multiplications would take roughly 2^{256} multiplications — more operations than there are atoms in the observable universe, by a wide margin. The naive loop is hopeless. We need exponentiation whose cost grows with the *number of bits* in the exponent, not the exponent itself.

3.6.3 Square-and-multiply

The trick is to use the binary expansion of the exponent and the algebraic fact $a^{2k} = (a^k)^2$ and $a^{2k+1} = (a^k)^2 \cdot a$. Squaring doubles the exponent for the price of one multiplication, so we can climb to a huge exponent in a number of steps equal to the bit-length.

Intuition

Think of building the exponent in binary, from the most significant bit down. Start with 1. For each bit of the exponent, **square** (which shifts your current exponent left by one bit, i.e. doubles it), and if the current bit is a 1, **multiply** by the base (which adds 1 to the exponent). Reading the exponent's bits left to right and applying "square, then maybe multiply" reconstructs exactly the exponent you want.

Example 3.6 ($7^{13} \bmod 11$ by square-and-multiply)**.** The exponent is $13 = 1101_2$. Read its bits left to right: $1, 1, 0, 1$. Start the accumulator at $r = 1$.

bit	action	accumulator (before reduce)	reduced mod 11
1	square, multiply	$1^2 \cdot 7 = 7$	7
1	square, multiply	$7^2 \cdot 7 = 343$	$343 \equiv 2$
0	square	$2^2 = 4$	4
1	square, multiply	$4^2 \cdot 7 = 112$	$112 \equiv 2$

So $7^{13} \equiv 2 \pmod{11}$. We used 4 squarings and 3 multiplications — 7 modular multiplications — instead of 12. Let us sanity-check against the direct route: $7^2 = 49 \equiv 5$, $7^4 \equiv 5^2 = 25 \equiv 3$, $7^8 \equiv 3^2 = 9$. Then $7^{13} = 7^8 \cdot 7^4 \cdot 7^1 \equiv 9 \cdot 3 \cdot 7 = 189 \equiv 2 \pmod{11}$. Agreed.

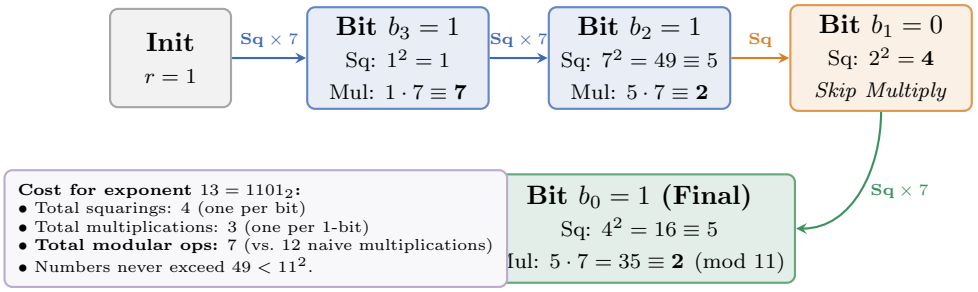


Figure 3.3: Square-and-multiply bit-register pipeline for $7^{13} \equiv 2 \pmod{11}$. Scanning the binary exponent $13 = 1101_2$ from left to right, every bit triggers a squaring step, while 1-bits trigger a subsequent multiplication by the base. Reducing modulo 11 after every step ensures intermediate values remain strictly bounded by n^2.

3.6.4 The cost, precisely

If the exponent k has $b = \lfloor \log_2 k \rfloor + 1$ bits, square-and-multiply performs at most b squarings and at most b multiplications — so at most $2b = O(\log k)$ modular multiplications. Each modular multiplication of numbers below n costs $O((\log n)^2)$ bit operations with schoolbook multiplication. The total is therefore

$$O\big((\log k)\,(\log n)^2\big)$$

bit operations. For RSA with a 2048-bit modulus and a 2048-bit exponent, that is on the order of $2^{11} \cdot (2^{11})^2 \approx 2^{33}$ bit operations — a few billion, a small fraction of a second on commodity hardware. Compare the naive loop's 2^{2048} multiplications. The gap between "logarithmic" and "linear" in the exponent is what makes public-key crypto usable.

Connection

This is the same exponential-vs-logarithm story the algorithms book tells about fast exponentiation in general (it works in *any* monoid: integers, matrices, group elements). There, the recurrence $T(k) = T(k/2) + O(1)$ multiplications gives $T(k) = O(\log k)$ by the same logic as binary search. Schoolbook $O((\log n)^2)$ multiplication can be improved to $O(\log n \log\log n)$ with FFT-based methods (Schönhage–Strassen), which real big-integer libraries switch to above a threshold — see the algorithms book's chapter on fast multiplication.

3.6.5 Toy implementation

Here is square-and-multiply in a few lines of Python, written to mirror the example above. It is short on purpose; the point is to see the algorithm, not to ship it.

Implement it

Learning code only — do not use in production. Real libraries use constant-time exponentiation (Montgomery ladders, fixed-window methods, blinding) to avoid leaking the exponent through timing or power. The version below branches on the secret bit, which is exactly the kind of leak Chapter 37 shows how to exploit. Use `pow(a, k, n)` from the standard library, which is hardened, for anything real.

```
def mod_pow(a, k, n):
    """Compute a**k mod n by square-and-multiply (teaching version)."""
    a = a % n                  # reduce the base first
    r = 1
    for bit in bin(k)[2:]:     # exponent bits, most significant first
        r = (r * r) % n        # SQUARE every step
        if bit == '1':         # MULTIPLY on a 1-bit
            r = (r * a) % n
    return r

# Trace the chapter's example:
assert mod_pow(7, 13, 11) == 2
assert mod_pow(3, 6, 7) == 1           # Fermat: 3^(7-1) = 1 mod 7
# Python's own (hardened) version agrees:
assert mod_pow(7, 13, 11) == pow(7, 13, 11)
```

Attack

The naive version leaks the exponent. The `if bit == '1'` branch does an extra multiplication only when the secret bit is 1. An attacker who can measure how long each iteration takes (timing), or watch the power trace of a smart card, sees a longer-or-shorter pattern that spells out the bits of k directly. For RSA, k is the private exponent d — recovering it is total key compromise. The defence, the *Montgomery ladder*, does the *same* sequence of one square and one multiply on every bit regardless of its value, so the trace carries no secret-dependent structure. We return to this in Chapter 37; for now, internalise that "correct math" and "safe implementation" are two different bars, and exponentiation must clear both.

3.7 Solving Linear Congruences

With inverses in hand we can solve equations. The modular analogue of $ax = b$ is the linear congruence

$$ax \equiv b \pmod{n},$$

and the recipe is exactly what you would hope: if a is invertible, multiply both sides by a^{-1}.

Example 3.7 (A unique solution)**.** Solve $5x \equiv 3 \pmod 7$. Here $\gcd(5, 7) = 1$, so 5 is invertible. Its inverse: $5 \cdot 3 = 15 \equiv 1$, so $5^{-1} \equiv 3 \pmod 7$. Multiply both sides by 3: $x \equiv 3 \cdot 3 = 9 \equiv 2 \pmod 7$. Check: $5 \cdot 2 = 10 \equiv 3$. Unique solution $x \equiv 2$.

When a is *not* coprime to n, things get more complicated, and this case is worth seeing because careless modular reasoning often goes wrong here.

Theorem 3.3 (Linear congruence solvability)**.** *Let $d = \gcd(a, n)$. The congruence $ax \equiv b \pmod{n}$ has a solution if and only if $d \mid b$. When it does, there are exactly d solutions modulo n.*

Example 3.8 (No solution, and many solutions)**.** Take $a = 4$, $n = 6$, so $d = \gcd(4, 6) = 2$.

$4x \equiv 3 \pmod 6$ has *no* solution: $4x$ is always even, 3 is odd, and $d = 2 \nmid 3$. Indeed the row for 4 in the mod-6 table is $0, 4, 2, 0, 4, 2$ — a 3 never appears.

$4x \equiv 2 \pmod 6$ has $d = 2$ solutions: scanning the same row, $4 \cdot 2 = 8 \equiv 2$ and $4 \cdot 5 = 20 \equiv 2$, so $x \equiv 2$ and $x \equiv 5$ both work. Two solutions, as promised.

Common trap

Beginners "cancel" a coefficient that is not coprime to the modulus and get nonsense. From $4x \equiv 2 \pmod 6$ you cannot just write $2x \equiv 1 \pmod 6$ (that one has no solution at all, since $2x$ is even). The legal cancellation divides *the modulus too*: $4x \equiv 2 \pmod 6 \Rightarrow 2x \equiv 1 \pmod 3$, which gives $x \equiv 2 \pmod 3$, i.e. $x \in \{2, 5\} \pmod 6$. Cancel a factor, divide the modulus by its gcd with that factor.

3.8 A Worked Mini-Cipher: Affine Encryption

Let us apply everything we have built to a real (if weak) historical cipher. The *affine cipher* encrypts each letter as a linear function mod 26. It is breakable in seconds — we will break it — but it is a clean exercise in modular inverses and linear congruences.

Intuition

The setup. Alice and Bob map letters to numbers $A = 0, B = 1, \ldots, Z = 25$, so they work in $\mathbb{Z}/26\mathbb{Z}$. The key is a pair (a, b) with $\gcd(a, 26) = 1$. To encrypt a letter x, Alice computes

$$\mathrm{Enc}_{a,b}(x) = (ax + b) \bmod 26.$$

Bob decrypts by inverting the affine map: $\mathrm{Dec}_{a,b}(y) = a^{-1}(y - b) \bmod 26$. Eve sees only ciphertext. Figure 3.4 shows the whole exchange on the worked example below.

Why must $\gcd(a, 26) = 1$? Because decryption multiplies by a^{-1}, which exists *only* when a is coprime to 26 (Theorem 3.2). Pick $a = 13$ and the map $13x+b$ collapses many letters onto the same ciphertext — the row for 13 mod 26 takes only two distinct values — and decryption is impossible. The legal multipliers are the units $(\mathbb{Z}/26\mathbb{Z})^*$, of which there are $\varphi(26) = 12$.

Example 3.9 (Encrypt and decrypt by hand)**.** Take key $(a, b) = (5, 8)$. Note $\gcd(5, 26) = 1$, good. Encrypt the letter `H`. Its number is $x = 7$:

$$\mathrm{Enc}(7) = (5 \cdot 7 + 8) \bmod 26 = (35 + 8) \bmod 26 = 43 \bmod 26 = 17 = \texttt{R}.$$

Now Bob decrypts `R` $= 17$. He needs $5^{-1} \bmod 26$. Since $5 \cdot 21 = 105 = 4 \cdot 26 + 1 \equiv 1$, we have $5^{-1} = 21$. Then

$$\mathrm{Dec}(17) = 21\,(17 - 8) \bmod 26 = 21 \cdot 9 \bmod 26 = 189 \bmod 26 = 7 = \texttt{H}.$$

Back to `H`, as it must be. The decryption inverse 21 is doing the "divide by 5" that modular inverses made legal.

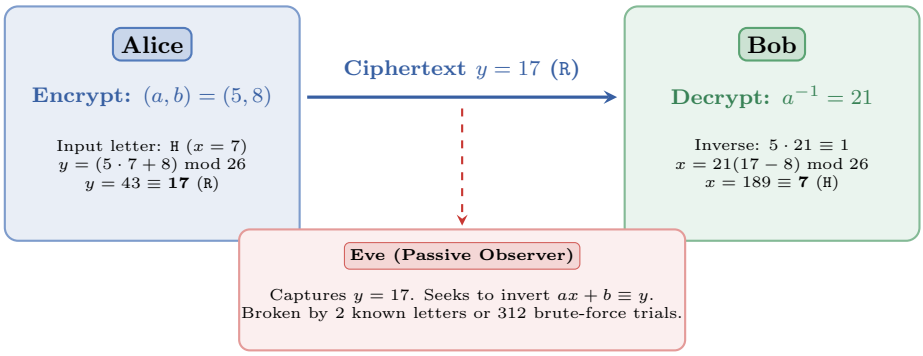


Figure 3.4: Affine cipher message flow with key $(a, b) = (5, 8)$. Alice computes $y \equiv 5x + 8 \pmod{26}$, mapping H to R. Bob inverts the transformation by multiplying $(y - 8)$ by $5^{-1} \equiv 21 \pmod{26}$, cleanly recovering the original plaintext $x = 7$.

Attack

Breaking the affine cipher. The key space is tiny: 12 valid choices of a times 26 of b gives only 312 keys. Eve can try all of them in a blink (brute force). Even without that, two known plaintext–ciphertext pairs pin the key exactly. Suppose Eve guesses that E= 4 encrypts to R= 17 and T= 19 encrypts to H= 7 (perhaps from frequency analysis). She gets two linear congruences:

$$4a + b \equiv 17, \qquad 19a + b \equiv 7 \pmod{26}.$$

Subtract the first from the second: $15a \equiv -10 \equiv 16 \pmod{26}$. Now $\gcd(15, 26) = 1$ and $15^{-1} \equiv 7 \pmod{26}$ (since $15 \cdot 7 = 105 \equiv 1$), so $a \equiv 7 \cdot 16 = 112 \equiv 8 \pmod{26}$. But $\gcd(8, 26) = 2 \neq 1$ — an illegal multiplier — so this guessed pairing is inconsistent, and Eve discards it and tries another. (That consistency check, that a recovered a must be a unit, is itself a useful filter.) The general lesson: a key space small enough to enumerate offers no protection, and linear structure leaks the key to anyone with a couple of matched samples. Modern ciphers are designed to eliminate exactly this linearity.

History

Affine and shift ciphers (the affine cipher with $a = 1$ is the Caesar cipher) are millennia old, and they were genuinely useful when the attacker was a person with a quill. What killed them was not better math but *frequency analysis*, formalised by the 9th-century polymath al-Kindi in Baghdad: because each plaintext letter always maps to the same ciphertext letter, the statistics of the language pass straight through (E is the commonest English letter, so the commonest ciphertext letter is almost certainly its image). The modern language of modular arithmetic we used to analyze them arrived much later: in 1801, at twenty-four, Carl Friedrich Gauss published *Disquisitiones Arithmeticae*, introducing the congruence symbol $\equiv$ and treating remainders modulo n as self-contained algebraic systems rather than ad hoc remainder tricks. Nearly two centuries later, Gauss's notation became standard in public-key cryptography.

3.9 Negative Numbers, and a Useful Symmetric View

So far we have written residues as $\{0, 1, \ldots, n-1\}$. Sometimes it is cleaner to use the *balanced* representatives centred on zero, $\{-\lfloor n/2 \rfloor, \ldots, -1, 0, 1, \ldots\}$. Both name the same residue classes; you just pick whichever representative is smaller in absolute value.

Mod 7, the residue 6 equals -1, and 5 equals -2. This makes some computations trivial: $6^2 \bmod 7$ is easier as $(-1)^2 = 1$, and $5^3 \bmod 7$ is easier as $(-2)^3 = -8 \equiv -1 \equiv 6$.

Walking question

What is $10^{200} \bmod 11$? Brute force is hopeless, but $10 \equiv -1 \pmod{11}$, so $10^{200} \equiv (-1)^{200} = 1 \pmod{11}$. The balanced view turned a 201-digit number into a parity check. (This is the exact mechanism behind the classic "casting out elevens" divisibility rule.)

3.10 Why This Underlies Cryptography

Modular arithmetic gives a *finite* set with rich algebraic structure: you can add, subtract, multiply, and — when you stick to units or a prime modulus — divide and exponentiate, all without numbers ever growing without bound. Finiteness is what lets a computer handle the operations exactly, with no rounding, while the structure is what lets cryptographers build problems that are easy one way and apparently hopeless to reverse.

Why this matters

The recurring pattern of public-key cryptography is a *one-way street built from modular exponentiation*. Computing $y = g^x \bmod p$ from x is cheap: square-and-multiply, $O(\log x)$ multiplications. Recovering x from y — the *discrete logarithm* — has no known fast algorithm for well-chosen p. The asymmetry between the easy forward map and the hard inverse map is the raw material of Diffie–Hellman, ElGamal, DSA, and (in multiplicative-inverse form) RSA. Everything in those schemes is the arithmetic of this chapter, scaled up to numbers hundreds of digits long. You now have the core arithmetic; the later chapters are about *which* modular computations are hard to undo.

In the real world

When your browser opens an `https://` connection, the TLS handshake almost always performs a Diffie–Hellman key exchange — today over an elliptic curve, but the classic "finite field" variant is still modular exponentiation $g^x \bmod p$ with p a 2048- or 3072-bit prime, computed by exactly the square-and-multiply you traced above. Your WiFi's WPA3 handshake (SAE) does modular exponentiation. SSH key exchange does it. Signal's X3DH does the elliptic-curve cousin. Every one of those, billions of times a day, is the clock arithmetic from the first page of this chapter, running on enormous numbers. Once this chapter is familiar, the arithmetic inside these protocols is no longer a black box.

We have seen that a residue a has a modular inverse modulo n precisely when $\gcd(a, n) = 1$. But computing greatest common divisors and modular inverses for thousand-bit numbers requires

systematic algorithms. In Chapter 4, we turn to number theory: primes, Bézout's identity, the Euclidean algorithm, and the Chinese Remainder Theorem.

3.11 Exercises

Exercise 3.1 (Reduce by hand)**.** Compute each, showing your reductions, and keep every intermediate number below the modulus squared: (a) $(123+77)\cdot 19 \bmod 5$; (b) $2^{10} \bmod 11$; (c) $(-17) \bmod 9$ as a value in $\{0,\dots,8\}$; (d) $99\cdot 98 \bmod 100$ using balanced representatives.

Exercise 3.2 (Tables)**.** Write out the full multiplication table of $\mathbb{Z}/8\mathbb{Z}$. Circle every unit (every row that is a permutation). Confirm that the units are exactly the residues coprime to 8, and that there are $\varphi(8)=4$ of them. Which nonzero elements are zero divisors, and what do they multiply to give 0?

Exercise 3.3 (Inverses)**.** Using the extended Euclidean algorithm (back-substitution), find: (a) $7^{-1} \bmod 26$; (b) $11^{-1} \bmod 30$; (c) $a^{-1} \bmod 17$ for every $a \in \{2,\dots,16\}$ (you may use the fact that $\mathbb{Z}/17\mathbb{Z}$ is a field, so all exist). Verify one of your answers by direct multiplication.

Exercise 3.4 (When inverses fail)**.** Show that 6 has no inverse modulo 9, two ways: (a) by computing $\gcd(6,9)$ and citing Theorem 3.2; (b) by exhibiting a zero divisor, i.e. a nonzero x with $6x \equiv 0 \pmod 9$, and explaining why that rules out an inverse.

Exercise 3.5 (Square-and-multiply)**.** Compute $5^{117} \bmod 19$ by square-and-multiply. Write 117 in binary, draw the bit-by-bit table as in Example 3.6, and count how many modular multiplications you used. Then check your answer using $5^{18} \equiv 1 \pmod{19}$ (Fermat) to reduce the exponent first.

Exercise 3.6 (Orders and generators)**.** Work in $(\mathbb{Z}/11\mathbb{Z})^*$, which has 10 elements. (a) Compute the order of 2 by listing its powers mod 11 until you reach 1, and decide whether 2 is a generator. (b) Do the same for 3. (c) Using Example 3.5's rule that every order divides $p-1$, list the only possible orders an element of $(\mathbb{Z}/11\mathbb{Z})^*$ can have. (d) Why would an element of order 2 be a catastrophic choice of base for Diffie–Hellman mod 11?

Exercise 3.7 (Linear congruences)**.** For each, decide whether a solution exists, and if so list *all* solutions modulo n: (a) $9x \equiv 6 \pmod{12}$; (b) $14x \equiv 5 \pmod{21}$; (c) $7x \equiv 4 \pmod{15}$. For each, state $d=\gcd(a,n)$ and reconcile your count of solutions with Theorem 3.3.

Exercise 3.8 (Break an affine cipher)**.** You intercept ciphertext and learn two pairings: plaintext `A`$=0$ encrypts to `N`$=13$, and `B`$=1$ encrypts to `Q`$=16$, under an affine cipher mod 26 with key (a,b). Recover a and b by solving the resulting linear congruences. Verify that your a is a unit mod 26, then decrypt the ciphertext letter `D`$=3$.

Exercise 3.9 (Negative tricks)**.** Use balanced representatives to evaluate, with almost no arithmetic: (a) $12^{1000} \bmod 13$; (b) for which n is $16^n \equiv 1 \pmod{17}$, and for which is it $\equiv 16$? (c) $31^2 \bmod 37$ using $31 \equiv -6 \pmod{37}$, without multiplying 31 by 31.

Exercise 3.10 (Cost, concretely)**.** Suppose the exponent k has exactly b bits, of which w are 1. Express the exact number of squarings and multiplications square-and-multiply performs (process the leading 1-bit as a setup, not a multiply, to get the tightest count). For a 2048-bit exponent with about half its bits set, estimate the total modular multiplications, and compare to the naive $k-1$ multiplications. State the gap as a power of two.

Exercise 3.11 (Prove cancellation)**.** Prove: if $\gcd(c, n) = 1$ and $ca \equiv cb \pmod{n}$, then $a \equiv b \pmod{n}$. (Multiply by c^{-1}, and justify each step from the definitions in this chapter.) Then give an explicit counterexample showing the conclusion can fail when $\gcd(c, n) > 1$.

Exercise 3.12 (A timing leak, on paper)**.** Re-read the toy `mod_pow`. Suppose each modular multiplication takes 1 microsecond and an iteration that skips the multiply step takes correspondingly less. If an attacker can time the total run for a fixed base and modulus across many exponents, explain precisely what they learn about a secret exponent, and why the Montgomery-ladder fix (one square *and* one multiply every iteration, regardless of the bit) removes the leak. Estimate the timing difference between an all-zeros and an all-ones 256-bit exponent.

Chapter 4

Number Theory: Primes, GCD, and Euclid

In Chapter 3, we learned to calculate on a finite clock: adding, multiplying, and inverting residues modulo n. Almost every public-key system in use today — RSA, Diffie–Hellman, elliptic curves, the signatures that protect your bank login — rests on a small pile of facts about the integers that were already old when Euclid wrote them down around 300 BCE. The integers look like the most boring objects in mathematics: you can add them, multiply them, and that is about it. But hidden inside that plain arithmetic is an asymmetry: multiplying two large primes is fast and easy; tearing the product back apart into its factors is, as far as anyone knows, ruinously slow. Modern cryptography lives inside that gap.

This chapter builds that machinery from scratch. We start with divisibility and primes, prove that every integer factors into primes in one way, and meet the Euclidean algorithm — a procedure that is both very old and very efficient. Extended Euclid hands us modular inverses, which is how RSA decryption keys get computed. Then come the two theorems that make exponentiation modulo a number behave predictably: Fermat's little theorem and its generalization, Euler's theorem, both proved from scratch. The Euler totient counts the units; the Chinese Remainder Theorem lets us split a hard modular problem into easy independent pieces (and speeds up RSA by a factor of four); primitive roots give us the cyclic structure that Diffie–Hellman needs. Everything is done with tiny numbers you can check on paper, because working small examples by hand is the best way to understand the abstractions.

4.1 Divisibility

We work inside $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$, the integers.

Definition 4.1 (Divisibility)**.** For integers a and b, we say a *divides* b, written $a \mid b$, if there exists an integer k with $b = ak$. We call a a *divisor* (or *factor*) of b, and b a *multiple* of a. If no such k exists we write $a \nmid b$.

So $3 \mid 12$ because $12 = 3 \cdot 4$, and $7 \nmid 12$ because no integer multiple of 7 lands on 12. Notice $a \mid 0$ for every a (take $k = 0$), and $0 \mid b$ only when $b = 0$. The relation $a \mid b$ says nothing about size beyond this: $-3 \mid 12$ just as much as 3 does.

A few facts fall straight out of the definition, and we will lean on them constantly.

Proposition 4.1 (Basic divisibility rules)**.** *Let a, b, c be integers.*

1. *If $a \mid b$ and $b \mid c$ then $a \mid c$ (transitivity).*
2. *If $a \mid b$ and $a \mid c$ then $a \mid (bx + cy)$ for all integers x, y.*
3. *If $a \mid b$ and $b \neq 0$ then $|a| \leq |b|$.*

Proof. (1) Write $b = ak$ and $c = bm$. Then $c = a(km)$, so $a \mid c$. (2) Write $b = ak$ and $c = a\ell$. Then $bx + cy = a(kx + \ell y)$, an integer multiple of a. (3) Write $b = ak$ with $k \neq 0$ (since $b \neq 0$). Then $|b| = |a||k| \geq |a|$ because $|k| \geq 1$. □

Part (2) is the one we will use most. The set of all integer combinations $bx + cy$ is exactly the set of multiples of the greatest common divisor of b and c, a fact we will prove and then use over and over.

Division with remainder

You cannot always divide one integer evenly by another, but you can always divide *with a remainder*, and that remainder is unique. This is the single most important computational fact in the chapter.

Theorem 4.1 (Division algorithm)**.** *For integers a and b with $b > 0$, there exist* unique *integers q (the quotient) and r (the remainder) such that*

$$a = bq + r, \qquad 0 \leq r < b.$$

Proof. Existence. Consider the set $S = \{a - bq : q \in \mathbb{Z},\ a - bq \geq 0\}$. It is nonempty: taking $q = -|a|$ gives $a - bq = a + b|a| \geq a + |a| \geq 0$. By the well-ordering principle, S has a least element $r = a - bq$ for some q. We have $r \geq 0$ by construction. If $r \geq b$, then $r - b = a - b(q+1) \geq 0$ would be a smaller element of S, contradicting minimality. So $0 \leq r < b$.

Uniqueness. Suppose $a = bq_1 + r_1 = bq_2 + r_2$ with both remainders in $[0, b)$. Subtracting, $b(q_1 - q_2) = r_2 - r_1$, so $b \mid (r_2 - r_1)$. But $|r_2 - r_1| < b$, and the only multiple of b with absolute value below b is 0. Hence $r_1 = r_2$, and then $q_1 = q_2$. □

This r is what your programming language writes as `a % b` (when $b > 0$ and the language follows the mathematical convention). It is the bridge to modular arithmetic: $a \equiv r \pmod{b}$.

Intuition

Lay out a number line and mark the multiples of b: $\ldots, -b, 0, b, 2b, \ldots$. Every integer a sits in exactly one of the half-open gaps $[bq, bq + b)$. The remainder r is how far a is past the multiple of b on its left. There is one gap per integer, so q and r are forced — that is the uniqueness.

4.2 Primes and the Fundamental Theorem

Definition 4.2 (Prime and composite)**.** An integer $p > 1$ is *prime* if its only positive divisors are 1 and p. An integer $n > 1$ that is not prime is *composite*: it factors as $n = ab$ with $1 < a, b < n$. By convention 1 is neither prime nor composite.

The first primes are $2, 3, 5, 7, 11, 13, 17, 19, 23, 29, \ldots$. Two is the only even prime, which makes it the perennial special case in proofs and code.

History

Euclid's *Elements*, Book IX, Proposition 20 (around 300 BCE), states: "Prime numbers are more than any assigned multitude of prime numbers." That is, there are infinitely many primes. His argument is the one every mathematician learns first, and it has not been improved in 2300 years. Suppose you had a finite list $p_1, \ldots, p_k$ of all the primes. Form $N = p_1 p_2 \cdots p_k + 1$. Dividing N by any p_i leaves remainder 1, so no p_i divides N. But $N > 1$ has *some* prime factor (we prove this below), and that factor is not on the list — contradiction. In cryptography, that endless supply guarantees that millions of servers and browsers can independently draw large prime numbers without running out or colliding; the great open problem is proving unconditionally that factoring their product is hard.

Every integer has a prime factor

Lemma 4.1. *Every integer $n > 1$ has at least one prime divisor.*

Proof. Let d be the smallest divisor of n that is greater than 1 (such divisors exist because n itself is one, and the well-ordering principle gives a least one). If d were composite, $d = ab$ with $1 < a < d$, then a would be a divisor of n smaller than d but still above 1, contradicting minimality. So d is prime. □

Euclid's lemma

The key result — the one that makes factorization *unique* — is easy to state and a little subtle to prove. It says primes cannot be split across a product.

Lemma 4.2 (Euclid's lemma)**.** *If p is prime and $p \mid ab$, then $p \mid a$ or $p \mid b$.*

We will prove it cleanly once we have Bezout's identity from the Euclidean algorithm (Section 4.3), because the natural proof uses the fact that $\gcd(p, a) = 1$ whenever $p \nmid a$. For now, accept it and watch what it buys us. Note the primality is essential: $6 \mid 4 \cdot 9 = 36$, yet $6 \nmid 4$ and $6 \nmid 9$. A composite number can hide inside a product by lending one factor to each side.

Theorem 4.2 (Fundamental Theorem of Arithmetic)**.** *Every integer $n > 1$ can be written as a product of primes,*

$$n = p_1^{e_1} p_2^{e_2} \cdots p_k^{e_k},$$

and this factorization is unique up to the order of the factors.

Proof. Existence is strong induction. If n is prime, it is its own factorization. Otherwise $n = ab$ with $1 < a, b < n$; by induction both a and b are products of primes, and concatenating those products factors n.

Uniqueness. Suppose n has two prime factorizations,

$$p_1 p_2 \cdots p_s = q_1 q_2 \cdots q_t,$$

listing primes with repetition. The prime p_1 divides the left side, hence the right side, so by Euclid's lemma (applied repeatedly) $p_1 \mid q_j$ for some j. Since q_j is prime its only divisors are 1 and q_j, forcing $p_1 = q_j$. Cancel that common prime from both sides and repeat. Each step removes one prime from each list; the lists must run out together, so $s = t$ and the multisets of primes coincide. □

Focus

Uniqueness is neither obvious nor automatic — it follows directly from Euclid's lemma. There are number systems that look like the integers but where factorization is *not* unique. In the ring $\mathbb{Z}[\sqrt{-5}]$ the number 6 factors as $2 \cdot 3$ and also as $(1 + \sqrt{-5})(1 - \sqrt{-5})$, with all four factors "prime" in the sense of having no smaller factorization. Euclid's lemma fails there. Cryptography relies on unique factorization in the integers every time it assumes an RSA modulus has exactly one factorization $n = pq$.

4.3 Greatest Common Divisor and the Euclidean Algorithm

Definition 4.3 (GCD)**.** The *greatest common divisor* $\gcd(a, b)$ of two integers, not both zero, is the largest positive integer that divides both. When $\gcd(a, b) = 1$ we call a and b *coprime* (or relatively prime): they share no prime factor.

You could compute $\gcd(a, b)$ by factoring both numbers and multiplying the shared prime powers — $\gcd(60, 84) = \gcd(2^2 \cdot 3 \cdot 5,\ 2^2 \cdot 3 \cdot 7) = 2^2 \cdot 3 = 12$. But factoring is slow. Euclid found a way to get the gcd *without* factoring at all, and it is fast.

The algorithm

The algorithm rests on one observation.

Lemma 4.3. *For $b > 0$, $\gcd(a, b) = \gcd(b, a \bmod b)$.*

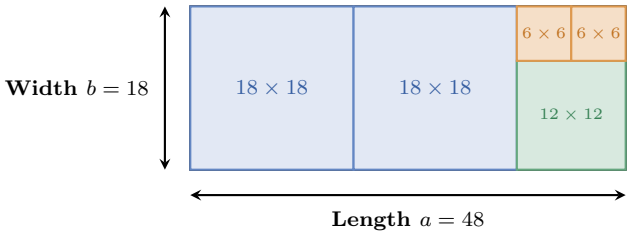
Proof. Write $a = bq + r$ with $r = a \bmod b$. Any common divisor of a and b divides $a - bq = r$ (by Proposition 4.1, part 2), so it is a common divisor of b and r. Conversely any common divisor of b and r divides $bq + r = a$, so it is a common divisor of a and b. The two pairs have identical sets of common divisors, hence the same greatest one. □

So replace (a, b) by $(b, a \bmod b)$ and repeat. The second coordinate strictly shrinks and stays non-negative, so it must hit 0. When it does, $\gcd(g, 0) = g$, and that g is the answer.

Example 4.1 ($\gcd(1071, 462)$ by hand)**.**

$$\begin{aligned} 1071 &= 2 \cdot 462 + 147 \\ 462 &= 3 \cdot 147 + 21 \\ 147 &= 7 \cdot 21 + 0. \end{aligned}$$

The last nonzero remainder is 21, so $\gcd(1071, 462) = 21$ after three divisions. Factoring $1071 = 3 \cdot 357 = 3 \cdot 3 \cdot 7 \cdot 17$ would have taken longer and given the same answer.



Geometric Euclidean Algorithm:
– **Step 1:** Cut two 18×18 squares $\Longrightarrow$ rem. 12×18
– **Step 2:** Cut one 12×12 square $\Longrightarrow$ rem. 6×12
– **Step 3:** Cut two 6×6 squares $\Longrightarrow$ rem. 0
Conclusion: Side 6 tiles all $\Longrightarrow$ $\gcd(48, 18) = 6$.

Figure 4.1: The Euclidean algorithm as geometric rectangle tiling for $\gcd(48, 18) = 6$. At each step, the largest possible square is cut out from the remaining rectangle. The smallest square that tiles the final remainder without gaps perfectly tiles all preceding squares, revealing the greatest common divisor.

Connection

How fast is this? The remainders shrink quickly — in fact each two steps at least halve the smaller number, so the number of divisions is $O(\log b)$. The slowest possible inputs are *consecutive Fibonacci numbers*: $\gcd(F_{n+1}, F_n)$ takes exactly $n - 1$ divisions, because each quotient is forced to be 1. This is Lame's theorem (1844), historically the first time anyone bounded an algorithm's running time in terms of input size. The companion algorithms book works out the $O(\log b)$ bound and the Fibonacci worst case in detail; the upshot for us is that gcd of two thousand-bit numbers finishes in a few thousand machine operations, instantly.

Bezout's identity and the extended algorithm

The Euclidean algorithm gives more than the gcd. By unwinding the divisions we can write the gcd as an integer combination of the inputs.

Theorem 4.3 (Bezout's identity)**.** *For integers a, b not both zero, there exist integers x, y with*

$$ax + by = \gcd(a, b).$$

In fact, $\gcd(a, b)$ *is the* smallest *positive value taken by $ax + by$ over all integers x, y.*

Proof. Let d be the smallest positive integer of the form $ax + by$ (the set of positive such values is nonempty — it contains $|a|$ when $a \neq 0$ — so well-ordering gives a least element). We show $d = \gcd(a, b)$.

First, d divides both a and b. Divide $a = dq + r$ with $0 \leq r < d$. Then

$$r = a - dq = a - q(ax + by) = a(1 - qx) + b(-qy),$$

which is again of the form $ax' + by'$. Since $0 \leq r < d$ and d is the smallest *positive* such value, we must have $r = 0$, i.e. $d \mid a$. The same argument gives $d \mid b$. So d is a common divisor.

Second, d is the greatest. Any common divisor c of a and b divides $ax + by = d$, so $c \leq d$. Hence $d = \gcd(a, b)$. □

The coefficients x, y are called *Bezout coefficients*, and we compute them by running Euclid forward while carrying the combinations along. That is the *extended Euclidean algorithm*. Continuing

Example 4.1, we back-substitute:

$$\begin{aligned} 21 &= 462 - 3 \cdot 147 \\ &= 462 - 3 \cdot (1071 - 2 \cdot 462) \\ &= 7 \cdot 462 - 3 \cdot 1071. \end{aligned}$$

Check: $7 \cdot 462 - 3 \cdot 1071 = 3234 - 3213 = 21$. So $x = -3$, $y = 7$.

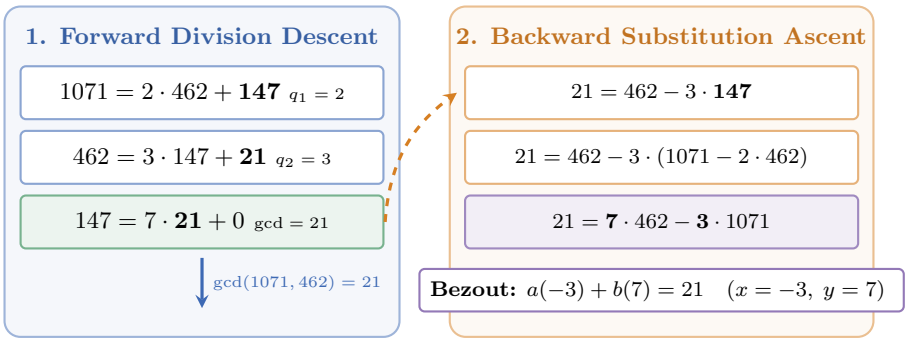


Figure 4.2: The Extended Euclidean algorithm ladder for $\gcd(1071, 462) = 21$. Left: Forward divisions isolate the remainder at each step until reaching the greatest common divisor 21. Right: Backward substitution unwinds the ladder, replacing intermediate remainders until 21 is expressed directly in terms of the initial inputs $1071(-3) + 462(7) = 21$.

Why it works

Now Euclid's lemma (Lemma 4.2) drops out for free. Suppose p is prime, $p \mid ab$, and $p \nmid a$. Then $\gcd(p, a) = 1$ — the only divisors of p are 1 and p, and p is ruled out — so Bezout gives integers x, y with $px + ay = 1$. Multiply by b:

$$pbx + aby = b.$$

Now $p \mid pbx$ directly (since p is a factor), and $p \mid aby$ because $p \mid ab$. So p divides the left side, hence $p \mid b$. That is exactly the claim: if a prime divides a product and misses one factor, it must hit the other. Unique factorization rests on this two-line argument.

Computing modular inverses

Cryptography needs the extended Euclidean algorithm to compute modular inverses. In modular arithmetic the analogue of "dividing by a" is multiplying by the *inverse* of a modulo n: an integer a^{-1} with $a \cdot a^{-1} \equiv 1 \pmod{n}$.

Theorem 4.4 (Existence of inverses)**.** *An integer a has a multiplicative inverse modulo n if and only if $\gcd(a, n) = 1$. When it exists the inverse is unique modulo n.*

Proof. If $\gcd(a, n) = 1$, Bezout gives $ax + ny = 1$, so $ax \equiv 1 \pmod{n}$ and x is the inverse. Conversely if $ax \equiv 1 \pmod{n}$ then $ax - 1 = ny$ for some y, i.e. $ax - ny = 1$; any common divisor of a and n divides the left side and so divides 1, forcing $\gcd(a, n) = 1$.

Uniqueness: if $ax \equiv 1$ and $ax' \equiv 1$, multiply the first by x' to get $x' \equiv x'ax \equiv x \pmod{n}$. □

So computing an inverse is exactly running extended Euclid and reading off x.

Example 4.2 (Inverse of 17 modulo 43)**.** Run Euclid on $(43, 17)$:

$$\begin{aligned} 43 &= 2 \cdot 17 + 9 \\ 17 &= 1 \cdot 9 + 8 \\ 9 &= 1 \cdot 8 + 1 \\ 8 &= 8 \cdot 1 + 0. \end{aligned}$$

The gcd is 1, so the inverse exists. Back-substitute:

$$\begin{aligned} 1 &= 9 - 1 \cdot 8 \\ &= 9 - (17 - 9) = 2 \cdot 9 - 17 \\ &= 2(43 - 2 \cdot 17) - 17 = 2 \cdot 43 - 5 \cdot 17. \end{aligned}$$

So $-5 \cdot 17 \equiv 1 \pmod{43}$, meaning $17^{-1} \equiv -5 \equiv 38 \pmod{43}$. Check: $17 \cdot 38 = 646 = 15 \cdot 43 + 1$. Correct.

Implement it

Extended Euclid in Python. The iterative form below tracks the Bezout coefficients in lockstep with the remainders, so it never recurses and never overflows for big integers.

```python
def egcd(a, b):
    # returns (g, x, y) with a*x + b*y = g = gcd(a, b)
    old_r, r = a, b
    old_s, s = 1, 0
    old_t, t = 0, 1
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t
    return old_r, old_s, old_t      # g, x, y

def inverse(a, n):
    g, x, _ = egcd(a % n, n)
    if g != 1:
        raise ValueError("no inverse: gcd(a, n) != 1")
    return x % n                    # normalize into [0, n)

print(inverse(17, 43))              # -> 38
```

Learning only. This is a clear, correct reference — but it is not constant-time. The number of loop iterations and the sizes of the intermediates depend on the secret input, which leaks timing. Production libraries (for instance when computing an RSA private exponent) use constant-time inversion or blinding. Use your platform's vetted big-integer routines, never a hand-rolled loop, for real keys.

In the real world

RSA key generation *is* an extended-Euclid computation. You pick a public exponent e (commonly 65537) and must produce the private exponent d with $ed \equiv 1 \pmod{\lambda(n)}$, where $\lambda(n)$ is computed from the secret primes. That d comes straight out of the extended Euclidean algorithm applied to e and $\lambda(n)$. If $\gcd(e, \lambda(n)) \neq 1$ the inverse fails to exist and the key generator must pick different primes — which is exactly why $e = 65537$ is convenient: it is prime, so it is coprime to $\lambda(n)$ unless $\lambda(n)$ happens to be a multiple of 65537, a vanishingly rare event you simply reject and retry.

4.4 Congruences and the Units of $\mathbb{Z}/n\mathbb{Z}$

Recall $a \equiv b \pmod{n}$ means $n \mid (a - b)$. This relation partitions $\mathbb{Z}$ into n *residue classes*, and arithmetic respects them: if $a \equiv a'$ and $b \equiv b'$ then $a + b \equiv a' + b'$ and $ab \equiv a'b' \pmod{n}$. The set of classes is written $\mathbb{Z}/n\mathbb{Z}$, and we usually name them by their least nonnegative representatives $\{0, 1, \ldots, n-1\}$.

Not every class has a multiplicative inverse. By Theorem 4.4, the class of a is invertible exactly when $\gcd(a, n) = 1$. These invertible classes are the *units*, and they form a group under multiplication:

$$(\mathbb{Z}/n\mathbb{Z})^* = \{a \in \{1, \ldots, n-1\} : \gcd(a, n) = 1\}.$$

Example 4.3 (Units modulo 12)**.** The classes coprime to $12 = 2^2 \cdot 3$ are $\{1, 5, 7, 11\}$. So $|(\mathbb{Z}/12\mathbb{Z})^*| = 4$. Every one of these is its own inverse: $5 \cdot 5 = 25 \equiv 1$, $7 \cdot 7 = 49 \equiv 1$, $11 \cdot 11 = 121 \equiv 1 \pmod{12}$. The non-units $\{2, 3, 4, 6, 8, 9, 10\}$ have no inverse; for instance $2x \equiv 1 \pmod{12}$ is unsolvable because $2x$ is always even.

Euler's totient

Definition 4.4 (Euler totient)**.** $\varphi(n)$ is the number of integers in $\{1, \ldots, n\}$ that are coprime to n. Equivalently $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^*|$, the size of the unit group.

For a prime p, every one of $1, 2, \ldots, p-1$ is coprime to p, so $\varphi(p) = p - 1$. For a prime power p^k, the only integers in $\{1, \ldots, p^k\}$ sharing a factor with p^k are the multiples of p, of which there are p^{k-1}. Hence

$$\varphi(p^k) = p^k - p^{k-1} = p^{k-1}(p - 1).$$

The totient is *multiplicative*: if $\gcd(m, n) = 1$ then $\varphi(mn) = \varphi(m)\varphi(n)$. We prove this cleanly with the Chinese Remainder Theorem in Section 4.6. Combining the two facts gives the general formula from the prime factorization $n = p_1^{e_1} \cdots p_k^{e_k}$:

$$\varphi(n) = \prod_{i=1}^{k} p_i^{e_i - 1}(p_i - 1) = n \prod_{p \mid n} \left(1 - \frac{1}{p}\right).$$

Example 4.4 (φ of an RSA-style modulus)**.** Take the textbook toy $n = 61 \cdot 53 = 3233$ with $p = 61$, $q = 53$ both prime. Then

$$\varphi(3233) = (61 - 1)(53 - 1) = 60 \cdot 52 = 3120.$$

Knowing the two factors makes $\varphi(n)$ a one-line computation. Without them, computing $\varphi(n)$ is as hard as factoring n — and that hardness is what keeps the RSA private key private. Anyone who can find $\varphi(3233) = 3120$ can recover the prime factors: from $p+q = n-\varphi(n)+1 = 3233-3120+1 = 114$ and $pq = 3233$, solve the quadratic $t^2 - 114t + 3233 = 0$ to get $t = 61, 53$.

Common trap

The formula $\varphi(pq) = (p-1)(q-1)$ requires p and q to be *distinct* primes. If a buggy key generator accidentally picks $p = q$, then $n = p^2$ and $\varphi(n) = p(p-1)$, not $(p-1)^2$; worse, n is now a perfect square and immediately factored by taking its integer square root. Real generators explicitly check $p \neq q$. The same caution applies to the modern RSA convention of using $\lambda(n) = \operatorname{lcm}(p-1, q-1)$ in place of $\varphi(n)$ — it is a divisor of $\varphi(n)$, it still works, and it yields a smaller private exponent.

4.5 Fermat's Little Theorem and Euler's Theorem

We now reach the two results that make modular exponentiation predictable. They are the basis of RSA and of the standard primality tests.

Theorem 4.5 (Fermat's little theorem)**.** *If p is prime and $p \nmid a$, then*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Equivalently, for every *integer a, $a^p \equiv a \pmod{p}$.*

Proof. Consider the $p-1$ nonzero residues $1, 2, \ldots, p-1$ and multiply each by a:

$$a \cdot 1,\ a \cdot 2,\ \ldots,\ a \cdot (p-1) \pmod{p}.$$

I claim these are the same $p-1$ residues, just shuffled. None is 0, since $p \nmid a$ and $p \nmid k$ for $1 \le k \le p-1$, so $p \nmid ak$. And they are distinct: if $ai \equiv aj$ then, multiplying by a^{-1} (which exists because $\gcd(a,p) = 1$), $i \equiv j$. So the list $\{a, 2a, \ldots, (p-1)a\}$ is a permutation of $\{1, 2, \ldots, p-1\}$ modulo p.

Multiply all of them together. The two products are congruent because the sets match:

$$(a \cdot 1)(a \cdot 2) \cdots (a \cdot (p-1)) \equiv 1 \cdot 2 \cdots (p-1) \pmod{p},$$

that is, $a^{p-1}\,(p-1)! \equiv (p-1)! \pmod{p}$. Since $(p-1)!$ is a product of numbers all coprime to p, it is invertible; cancel it to get $a^{p-1} \equiv 1 \pmod{p}$. □

Example 4.5 (Fermat at $p = 7$)**.** Take $a = 3$, $p = 7$. The theorem predicts $3^6 \equiv 1 \pmod 7$. Compute the powers: $3^1 = 3$, $3^2 = 9 \equiv 2$, $3^3 \equiv 6$, $3^4 \equiv 18 \equiv 4$, $3^5 \equiv 12 \equiv 5$, $3^6 \equiv 15 \equiv 1 \pmod 7$. It lands on 1 exactly at the exponent $p-1 = 6$, and notice the powers cycled through *all* of $\{1,2,3,4,5,6\}$ on the way — 3 is a primitive root mod 7, which we return to in Section 4.7.

Euler generalized this from prime moduli to all moduli by replacing $p-1$ with $\varphi(n)$.

Theorem 4.6 (Euler's theorem)**.** *If $\gcd(a,n) = 1$, then*

$$a^{\varphi(n)} \equiv 1 \pmod{n}.$$

Proof. The argument is Fermat's, run over the unit group instead of all nonzero residues. Let $u_1, u_2, \ldots, u_{\varphi(n)}$ be the units of $\mathbb{Z}/n\mathbb{Z}$ (the residues coprime to n). Multiply each by a. Because a is a unit, the map $u \mapsto au$ sends units to units (a product of units is a unit) and is injective ($au_i \equiv au_j \Rightarrow u_i \equiv u_j$ after multiplying by a^{-1}). An injective map of a finite set to itself is a bijection, so $\{au_1, \ldots, au_{\varphi(n)}\}$ is just the units permuted. Multiply everything:

$$\prod_i (au_i) \equiv \prod_i u_i \pmod{n} \quad \Longrightarrow \quad a^{\varphi(n)} \prod_i u_i \equiv \prod_i u_i \pmod{n}.$$

The product $\prod_i u_i$ is a unit (units are closed under multiplication), so cancel it: $a^{\varphi(n)} \equiv 1 \pmod{n}$. □

Fermat is the special case $n = p$, where $\varphi(p) = p - 1$.

Example 4.6 (Euler at $n = 10$)**.** Here $\varphi(10) = \varphi(2)\varphi(5) = 1 \cdot 4 = 4$. Take $a = 3$: $3^4 = 81 \equiv 1 \pmod{10}$. So the last digit of every power of 3 cycles with period dividing 4: $3, 9, 7, 1, 3, 9, 7, 1, \ldots$ — which is why 3^{2026} ends in the same digit as $3^{2026 \bmod 4} = 3^2 = 9$.

Why it works

Euler's theorem is what makes RSA decryption correct. With $n = pq$ and $ed \equiv 1 \pmod{\varphi(n)}$, write $ed = 1 + k\varphi(n)$. For a message m coprime to n,

$$m^{ed} = m^{1+k\varphi(n)} = m \cdot \left(m^{\varphi(n)}\right)^k \equiv m \cdot 1^k = m \pmod{n}$$

by Euler's theorem. Encrypting with e and decrypting with d are exact inverses. (The leftover case where m shares a factor with n is handled by the Chinese Remainder Theorem in the next section — the identity $m^{ed} \equiv m$ holds for *all* m, not just the units, which is a nice subtlety we will nail down.)

Attack

Fermat's theorem is also the basis of the cheapest *primality* screen — and of a classic trap. The contrapositive says: if $a^{n-1} \not\equiv 1 \pmod{n}$ for some a with $\gcd(a, n) = 1$, then n is definitely composite. This is the *Fermat test*. But the converse fails. There are composite n that satisfy $a^{n-1} \equiv 1 \pmod{n}$ for *every* a coprime to n — the *Carmichael numbers*, the smallest being $561 = 3 \cdot 11 \cdot 17$. A Fermat test will swear 561 is prime for every base it cannot detect through a shared factor. Carmichael numbers are infinite in supply, so the Fermat test alone is not safe for cryptographic prime generation. The fix is the Miller–Rabin test, which inspects intermediate square roots of 1 to catch Carmichael numbers; we develop it in full in Appendix A. The converse of a theorem does not hold automatically, and assuming it can create a security hole.

4.6 The Chinese Remainder Theorem

Suppose you want a number that leaves remainder 2 when divided by 3, remainder 3 when divided by 5, and remainder 2 when divided by 7. The Chinese Remainder Theorem (CRT) says there is

exactly one such number modulo $3 \cdot 5 \cdot 7 = 105$, and it hands you a recipe to find it. The result is ancient — it appears in Sunzi's *Mathematical Classic*, roughly the 3rd–5th century CE, in essentially the puzzle just stated.

Theorem 4.7 (CRT, two-modulus form)**.** *Let m and n be coprime. Then for any targets a, b the system*

$$x \equiv a \pmod{m}, \qquad x \equiv b \pmod{n}$$

has a unique solution modulo mn. The map

$$\mathbb{Z}/mn\mathbb{Z} \longrightarrow \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}, \qquad x \longmapsto (x \bmod m,\ x \bmod n)$$

is a bijection that respects addition and multiplication.

Proof. Existence and the formula. Since $\gcd(m, n) = 1$, Bezout gives $Mm + Nn = 1$ for some integers M, N. Note $Nn \equiv 1 \pmod{m}$ and $\equiv 0 \pmod{n}$; symmetrically $Mm \equiv 0 \pmod{m}$ and $\equiv 1 \pmod{n}$. Set

$$x = a \cdot (Nn) + b \cdot (Mm).$$

Modulo m the second term vanishes and $Nn \equiv 1$, so $x \equiv a$. Modulo n the first term vanishes and $Mm \equiv 1$, so $x \equiv b$. Done.

Uniqueness. If x and x' both solve the system, then $m \mid (x - x')$ and $n \mid (x - x')$. Since $\gcd(m, n) = 1$, their product divides the difference (any common multiple of coprime numbers is a multiple of their product), so $mn \mid (x - x')$, i.e. $x \equiv x' \pmod{mn}$.

The map is clearly a ring homomorphism (reduction mod m and mod n both respect $+$ and $\times$), it is injective by uniqueness, and both sides have mn elements, so it is a bijection. □

Example 4.7 (Sunzi's puzzle, worked)**.** Solve $x \equiv 2 \pmod{3}$, $x \equiv 3 \pmod{5}$, $x \equiv 2 \pmod{7}$. Do it pair by pair.

First combine mod 3 and mod 5 into mod 15. We need $x \equiv 2 \pmod{3}$ and $x \equiv 3 \pmod{5}$. Bezout for $(3, 5)$: $2 \cdot 3 - 1 \cdot 5 = 1$, so $-1 \cdot 5 \equiv 1 \pmod{3}$ and $2 \cdot 3 \equiv 1 \pmod{5}$. Then

$$x = 2 \cdot (-5) + 3 \cdot (6) = -10 + 18 = 8, \qquad x \equiv 8 \pmod{15}.$$

Check: $8 = 2 \cdot 3 + 2$ and $8 = 1 \cdot 5 + 3$. Good.

Now combine $x \equiv 8 \pmod{15}$ with $x \equiv 2 \pmod{7}$. Bezout for $(15, 7)$: $1 = 15 - 2 \cdot 7$, so $15 \equiv 1 \pmod{7}$ and $-2 \cdot 7 \equiv 1 \pmod{15}$. Then

$$x = 8 \cdot (-14) + 2 \cdot (15) = -112 + 30 = -82 \equiv -82 + 105 = 23 \pmod{105}.$$

So $x \equiv 23 \pmod{105}$. Verify all three: $23 = 7 \cdot 3 + 2$, $23 = 4 \cdot 5 + 3$, $23 = 3 \cdot 7 + 2$. All correct.

Intuition

The CRT says a residue modulo mn amounts to a *pair* of residues, one mod m and one mod n — and you can add or multiply the pairs coordinatewise. A single odometer counting to mn is indistinguishable from two independent odometers of lengths m and n spinning in parallel, *provided their lengths are coprime* so they never resynchronize early. Coprimality is what guarantees all mn pairs actually occur.

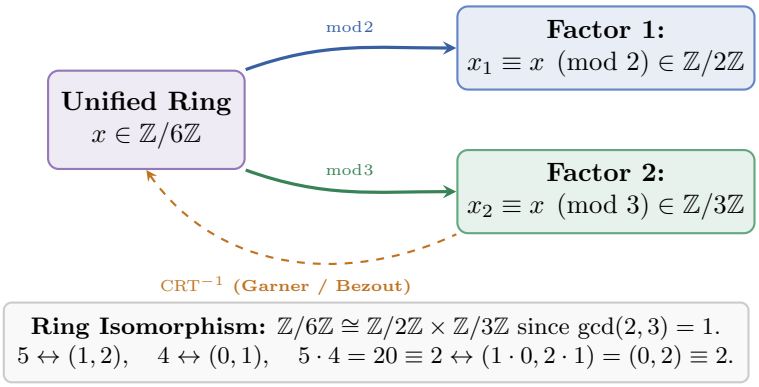


Figure 4.3: The Chinese Remainder Theorem as a ring isomorphism. Because 2 and 3 are coprime, computing modulo 6 is structurally identical to running two independent, parallel computations mod 2 and mod 3. Operations can be evaluated in the smaller, faster sub-rings and recombined at the end.

Totient is multiplicative

Corollary 4.1. *If* $\gcd(m, n) = 1$ *then* $\varphi(mn) = \varphi(m)\,\varphi(n)$.

Proof. The CRT bijection $\mathbb{Z}/mn\mathbb{Z} \to \mathbb{Z}/m\mathbb{Z} \times \mathbb{Z}/n\mathbb{Z}$ respects multiplication, so it carries units to units: x is invertible mod mn iff both coordinates are invertible. Hence it restricts to a bijection between $(\mathbb{Z}/mn\mathbb{Z})^*$ and $(\mathbb{Z}/m\mathbb{Z})^* \times (\mathbb{Z}/n\mathbb{Z})^*$. Counting both sides, $\varphi(mn) = \varphi(m)\varphi(n)$. □

This is the missing piece that completes the totient formula of Section 4.4: a general n splits into coprime prime powers, and φ multiplies across them.

In the real world

CRT makes RSA decryption about four times faster, and every serious RSA implementation uses it. The private key stores p and q alongside d. To compute $m = c^d \bmod n$, instead of one exponentiation modulo the $2k$-bit number n, you do two exponentiations modulo the k-bit numbers p and q:

$$m_p = c^{d \bmod (p-1)} \bmod p, \qquad m_q = c^{d \bmod (q-1)} \bmod q,$$

then glue m_p, m_q back together with the CRT formula to recover $m \bmod n$. Each half-size exponentiation is roughly four times cheaper (modular multiplication scales quadratically in the operand size), and the gluing is negligible. The catch: a fault during one of the two halves leaks the factorization. If m_p is computed correctly but m_q is corrupted, then the faulty signature s' satisfies $\gcd(s'^e - m,\ n) = p$ exactly, handing an attacker the secret prime. This *Bellcore fault attack* is why CRT-RSA implementations verify the result before releasing it.

Implement it

A general CRT solver over many coprime moduli, building the answer one congruence at a time.

```python
def crt(residues, moduli):
    # solve x = residues[i] (mod moduli[i]); moduli pairwise coprime
    x, M = 0, 1
    for a, m in zip(residues, moduli):
        # current x solves the first batch mod M; fold in (a mod m)
        g, p, q = egcd(M, m)        # M*p + m*q = 1  (g == 1 required)
        if g != 1:
            raise ValueError("moduli not coprime")
        x = (a * M * p + x * m * q) % (M * m)
        M *= m
    return x % M

print(crt([2, 3, 2], [3, 5, 7]))   # -> 23
```

Learning only. No constant-time guarantees and no fault checking. Real CRT-RSA code recombines m_p and m_q with the precomputed inverse q^{-1} mod p (Garner's formula) and then re-encrypts to verify, precisely to block the Bellcore fault attack above.

4.7 Primitive Roots

Euler's theorem tells us $a^{\varphi(n)} \equiv 1$, but $\varphi(n)$ might not be the *first* exponent that returns to 1. The first one has a name.

Definition 4.5 (Multiplicative order)**.** For a unit a modulo n, the *order* $\mathrm{ord}_n(a)$ is the smallest positive integer k with $a^k \equiv 1 \pmod{n}$.

Lemma 4.4. *$a^m \equiv 1 \pmod{n}$ if and only if $\mathrm{ord}_n(a) \mid m$. In particular $\mathrm{ord}_n(a) \mid \varphi(n)$.*

Proof. Let $d = \mathrm{ord}_n(a)$. Divide $m = dq + r$ with $0 \le r < d$. Then $a^m = (a^d)^q a^r \equiv a^r \pmod{n}$, so $a^m \equiv 1$ iff $a^r \equiv 1$. By minimality of d, $0 \le r < d$ forces $r = 0$, i.e. $d \mid m$. The "in particular" follows from Euler's theorem, which gives $a^{\varphi(n)} \equiv 1$. □

Definition 4.6 (Primitive root)**.** A unit g modulo n is a *primitive root* if $\mathrm{ord}_n(g) = \varphi(n)$ — the largest order possible. Then the powers $g^0, g^1, \ldots, g^{\varphi(n)-1}$ run through *every* unit exactly once: g *generates* $(\mathbb{Z}/n\mathbb{Z})^*$, and the group is *cyclic*.

Example 4.8 (Primitive root mod 7)**.** We saw in Example 4.5 that the powers of 3 mod 7 are $3, 2, 6, 4, 5, 1$ — all six units before returning to 1. So $\mathrm{ord}_7(3) = 6 = \varphi(7)$ and 3 is a primitive root. Compare 2: its powers are $2, 4, 1, 2, 4, 1, \ldots$, order 3, a proper divisor of 6. So 2 is not a primitive root mod 7. The orders of the six units are $\mathrm{ord}(1) = 1$, $\mathrm{ord}(6) = 2$, $\mathrm{ord}(2) = \mathrm{ord}(4) = 3$, $\mathrm{ord}(3) = \mathrm{ord}(5) = 6$ — and every order divides 6, as Lemma 4.4 promised.

Not every modulus has a primitive root. The ones that do are exactly $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p — a theorem of Gauss we will not prove here. The case we care about most is clean.

Theorem 4.8 (Primitive roots mod a prime)**.** *For every prime p, the group $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic: a primitive root exists, and there are exactly $\varphi(p-1)$ of them.*

The proof uses the fact that a degree-d polynomial over $\mathbb{Z}/p\mathbb{Z}$ has at most d roots (a field property we develop in Chapter 6), so we defer the full argument. The consequence for cryptography is this: once a generator g is fixed, every nonzero residue is g^k for a unique exponent $k \in \{0, \ldots, p-2\}$, and that exponent is the *discrete logarithm* $\operatorname{dlog}_g(x)$.

Intuition

A primitive root turns the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ into a clock with $p-1$ positions. Multiplying two units corresponds to *adding* their discrete logs and wrapping around. So multiplication mod p has the exact same structure as addition mod $p-1$ — once you know the dictionary $x \leftrightarrow \operatorname{dlog}_g(x)$. The catch, and the basis of Diffie-Hellman, is that going from x to g^x is fast while going back from g^x to x (computing the discrete log) appears to be exponentially hard for well-chosen p.

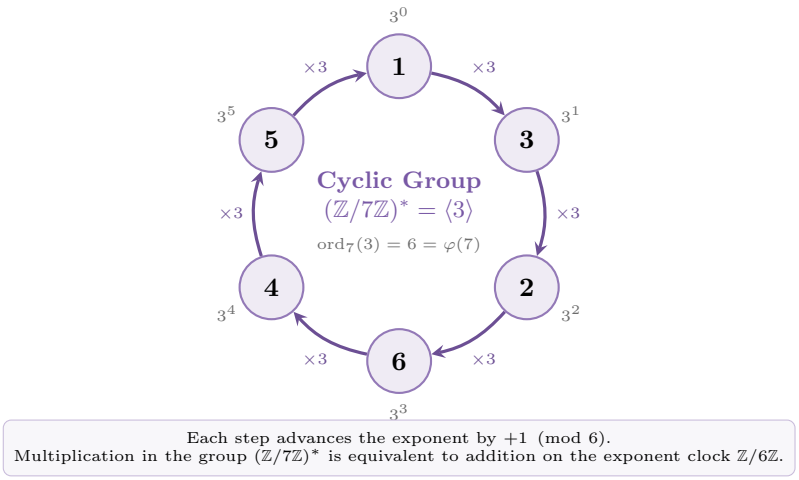


Figure 4.4: The cyclic orbit of the primitive root $g = 3$ modulo 7. The powers 3^k mod 7 cycle through every non-zero residue before returning to 1. The Diffie-Hellman discrete logarithm problem is posed in this cyclic group.

Example 4.9 (Counting primitive roots mod 11)**.** Here $\varphi(11) = 10$, so the count of primitive roots is $\varphi(10) = \varphi(2)\varphi(5) = 1 \cdot 4 = 4$. Test $g = 2$: its order must divide 10, so it is $1, 2, 5$, or 10. Check the proper divisors — $2^2 = 4 \neq 1$, $2^5 = 32 \equiv 10 \neq 1 \pmod{11}$ — neither hits 1, so $\operatorname{ord}_{11}(2) = 10$ and 2 is a primitive root. The four primitive roots are the powers 2^k with $\gcd(k, 10) = 1$, namely $k \in \{1, 3, 7, 9\}$: $2^1 = 2$, $2^3 = 8$, $2^7 \equiv 7$, $2^9 \equiv 6 \pmod{11}$. So $\{2, 6, 7, 8\}$.

Connection

Why test only the divisors of $\varphi(n)$, not all exponents up to $\varphi(n)$? Because Lemma 4.4 pins the order to a divisor of $\varphi(n)$. To confirm g is a primitive root it suffices to check $g^{\varphi(n)/q} \not\equiv 1$ for each prime q dividing $\varphi(n)$ — if none of those drop to 1, no proper divisor can be the order. This is the standard fast primitive-root test, and it needs the factorization of $\varphi(n)$. The probability side of the story (a random residue is a primitive root with probability $\varphi(p-1)/(p-1)$, which the prime-number theorem keeps bounded away from zero) lives in the companion probability book.

In the real world

Diffie-Hellman key exchange lives in exactly this cyclic structure. Alice and Bob publicly agree on a large prime p and a generator g of (a large prime-order subgroup of) $(\mathbb{Z}/p\mathbb{Z})^*$. Alice picks a secret a and sends $g^a \bmod p$; Bob picks secret b and sends $g^b \bmod p$. Each raises the other's value to their own secret, and both arrive at $g^{ab} \bmod p$ — the shared key. Eve sees g, g^a, g^b but to find g^{ab} she would seemingly need a discrete logarithm, which is infeasible for a well-chosen p of a couple thousand bits. The protocol turns that asymmetry — exponentiation is easy, its inverse is hard — into a two-message exchange. Elliptic-curve Diffie-Hellman replaces $(\mathbb{Z}/p\mathbb{Z})^*$ with the points on a curve but reuses the same cyclic-group structure.

4.8 Putting It Together: A Tiny RSA by Hand

We now combine the tools of this chapter in one worked example. We build a complete RSA keypair with $p = 61$, $q = 53$ and run a message through it (Figure 4.6), computing each piece with the methods above.

Key generation.

1. Modulus: $n = pq = 61 \cdot 53 = 3233$.
2. Totient: $\varphi(n) = (p-1)(q-1) = 60 \cdot 52 = 3120$ (Example 4.4).
3. Public exponent: choose $e = 17$. It is coprime to 3120: since $3120 = 2^4 \cdot 3 \cdot 5 \cdot 13$ has no factor of 17, $\gcd(17, 3120) = 1$, confirmed by extended Euclid below.
4. Private exponent: $d = e^{-1} \bmod \varphi(n) = 17^{-1} \bmod 3120$. Run extended Euclid:

$$\begin{aligned} 3120 &= 183 \cdot 17 + 9 \\ 17 &= 1 \cdot 9 + 8 \\ 9 &= 1 \cdot 8 + 1. \end{aligned}$$

   Back-substitute: $1 = 9 - 8 = 9 - (17 - 9) = 2 \cdot 9 - 17 = 2(3120 - 183 \cdot 17) - 17 = 2 \cdot 3120 - 367 \cdot 17$. So $-367 \cdot 17 \equiv 1 \pmod{3120}$, giving $d \equiv -367 \equiv 2753 \pmod{3120}$. Check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$. Correct.

The public key is $(n, e) = (3233, 17)$; the private key is $d = 2753$ (with p, q kept for the CRT speedup).

Encrypting. Let the message be $m = 65$ (say, the letter `A` in ASCII). The ciphertext is

$$c = m^e \bmod n = 65^{17} \bmod 3233.$$

Compute by square-and-multiply, reducing mod 3233 at every step. Powers of 65: $65^2 = 4225 \equiv 992$; $65^4 \equiv 992^2 = 984064 \equiv 984064 - 304 \cdot 3233 = 1232$; $65^8 \equiv 1232^2 = 1517824 \equiv 1517824 - 469 \cdot 3233 = 1547$; $65^{16} \equiv 1547^2 = 2393209 \equiv 2393209 - 740 \cdot 3233 = 789$. Since $17 = 16 + 1$,

$$c = 65^{16} \cdot 65 \equiv 789 \cdot 65 = 51285 \equiv 51285 - 15 \cdot 3233 = 2790 \pmod{3233}.$$

So $c = 2790$.

Decrypting with the CRT speedup. We must recover $m = c^d \bmod n = 2790^{2753} \bmod 3233$. Done directly this is a 2753-exponent slog, so use the CRT route — exactly what real RSA does.

Reduce the private exponent modulo each factor's totient:

$$d_p = d \bmod (p-1) = 2753 \bmod 60 = 53, \qquad d_q = d \bmod (q-1) = 2753 \bmod 52 = 49.$$

Reduce the ciphertext into each prime: $2790 \bmod 61 = 45$ and $2790 \bmod 53 = 34$. Then by Fermat the two half-size exponentiations are

$$m_p = 45^{53} \bmod 61 = 4, \qquad m_q = 34^{49} \bmod 53 = 12.$$

Glue them with Garner's form of the CRT. The formula needs the inverse *of p modulo q*:

$$m = m_p + p \cdot \big[(m_q - m_p)\,(p^{-1} \bmod q) \bmod q\big].$$

A tempting mistake is to grab the wrong inverse. Garner wants $p^{-1} \bmod q$, i.e. $61^{-1} \bmod 53$; since $61 \equiv 8 \pmod{53}$ this is $8^{-1} \bmod 53$. If you carelessly used some other value such as 12, you would get $(12-4)\cdot 12 = 96 \equiv 43 \pmod{53}$ and $m = 4 + 61 \cdot 43 = 2627$ — a legal-looking residue (it is below $n = 3233$) that is silently *wrong*, because the round trip must return $m = 65$. The genuine inverse is $8^{-1} \equiv 20 \pmod{53}$ (check $8 \cdot 20 = 160 = 3 \cdot 53 + 1$). With it,

$$m = 4 + 61 \cdot \big[(12-4)\cdot 20 \bmod 53\big] = 4 + 61 \cdot \big[160 \bmod 53\big] = 4 + 61 \cdot 1 = 65.$$

The round trip closes, and Euler's theorem guaranteed it would (Figure 4.5).

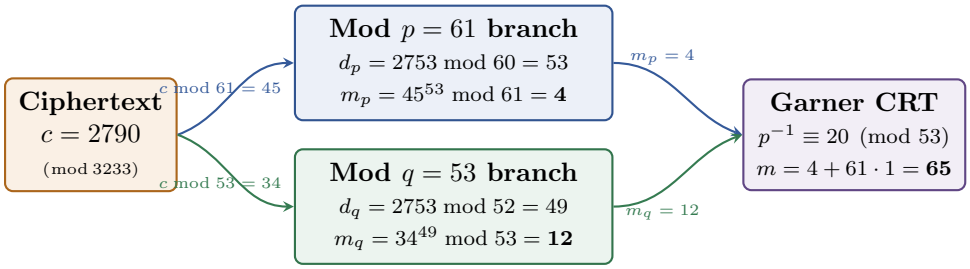


Figure 4.5: CRT-accelerated RSA decryption. Instead of raising $c = 2790$ to the full exponent $d = 2753$ modulo $n = 3233$, the receiver reduces into the secret factors $p = 61$ and $q = 53$, performs two much smaller exponentiations in parallel, and recombines $(m_p, m_q) = (4, 12)$ via Garner's formula to recover $m = 65$.

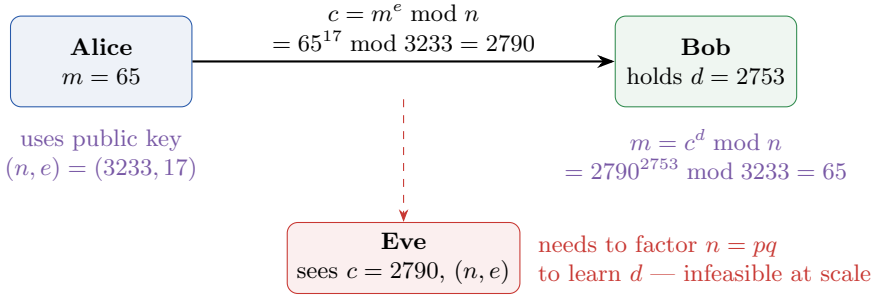


Figure 4.6: The toy RSA round trip. Encryption with the public exponent and decryption with the private exponent are exact inverses (Euler's theorem); Eve, lacking the factorization of n, cannot reconstruct d.

Walking question

The lesson of that near-miss: Garner recombination is exactly where careless CRT-RSA code goes wrong, and the bug is silent — you get *a* residue, just not the right one. The fix is to use the genuine inverse of p mod q. Work the whole decryption yourself: recompute $45^{53} \bmod 61$ and $34^{49} \bmod 53$ by square-and-multiply (the chains are short, five or six squarings each), confirm $m_p = 4$, $m_q = 12$, and verify Garner with the coefficient 20 lands on 65. If it does, you have personally executed every theorem in this chapter — division with remainder, extended Euclid for d, Fermat for the subgroup exponents, and CRT for the recombination — on real numbers.

We have seen how integer arithmetic, prime factors, and modular inverses are the computations behind RSA. To understand why these operations work, and to construct the number systems used by AES and elliptic curves, we need more general tools. Chapter 5 moves from individual integers to the algebraic structures they form: groups, rings, and fields.

4.9 Exercises

Exercise 4.1 (Warm-up gcd and Bezout)**.** Compute $\gcd(252, 198)$ with the Euclidean algorithm, showing each division. Then run extended Euclid to find integers x, y with $252x+198y = \gcd(252, 198)$, and verify your answer by direct substitution.

Exercise 4.2 (Modular inverse)**.** Compute $11^{-1} \bmod 26$ (the inverse that "undoes" an affine cipher with multiplier 11 on the 26-letter alphabet). Show the extended-Euclid steps and check that the product is $\equiv 1$. Then explain why $13^{-1} \bmod 26$ does *not* exist.

Exercise 4.3 (Fermat in action)**.** Use Fermat's little theorem to compute $7^{222} \bmod 11$ without a calculator, by first reducing the exponent modulo 10. Then compute $2^{1000} \bmod 13$ the same way.

Exercise 4.4 (Totient practice)**.** Compute $\varphi(360)$, $\varphi(1000)$, and $\varphi(p^2q)$ for distinct primes p, q (give a formula for the last). For which $n \leq 30$ is $\varphi(n)$ odd? Prove your characterization.

Exercise 4.5 (CRT by hand)**.** Find the smallest positive integer x with $x \equiv 1 \pmod 4$, $x \equiv 2 \pmod 5$, and $x \equiv 3 \pmod 7$. Show the pairwise combining steps, and state the full set of solutions modulo 140.

Exercise 4.6 (Order and primitive roots)**.** For the prime $p = 13$: (a) compute the order of 2, 3, and 5 modulo 13; (b) determine which of them are primitive roots; (c) using a primitive root you found, list the discrete logarithm of every nonzero residue. How many primitive roots does 13 have, and does your count match $\varphi(12)$?

Exercise 4.7 (Break a toy RSA)**.** An adversary intercepts the public key $(n, e) = (3233, 17)$ and, by factoring the small modulus, recovers $p = 61$, $q = 53$. Reconstruct the private exponent d from scratch with extended Euclid, then decrypt the ciphertext $c = 2790$ to recover the plaintext. (You have just performed the entire RSA break that large primes are designed to prevent.)

Exercise 4.8 (Recover the factors from the totient)**.** You are told that $n = 8051$ is a product of two primes and that $\varphi(n) = 7872$. Without trial division, recover p and q by setting up and solving a quadratic in $p+q$ and pq, as in Example 4.4. Explain in one sentence why this shows that computing $\varphi(n)$ is as hard as factoring.

Exercise 4.9 (Carmichael number)**.** Verify that $561 = 3 \cdot 11 \cdot 17$ is a Fermat pseudoprime to base 2 — that is, $2^{560} \equiv 1 \pmod{561}$ — by computing 2^{560} modulo 3, 11, and 17 separately (Fermat makes each trivial) and recombining with CRT. Why does this make 561 dangerous to a naive primality test?

Exercise 4.10 (Euclid's lemma, used in anger)**.** Prove that $\sqrt{2}$ is irrational using the Fundamental Theorem of Arithmetic: suppose $\sqrt{2} = a/b$ in lowest terms, square it, and derive a contradiction from the parity of the prime factorization of $2b^2 = a^2$. Identify exactly where the proof needs Euclid's lemma (or unique factorization).

Exercise 4.11 (Prove a divisibility identity)**.** Prove that for all integers $n \geq 1$, $6 \mid n^3 - n$. (Hint: factor $n^3 - n = (n-1)n(n+1)$ and argue about three consecutive integers, or use Fermat's little theorem at the primes 2 and 3 together with CRT.)

Exercise 4.12 (Finish the worked decryption)**.** Complete the CRT recombination from Section 4.8 cleanly. Recompute $m_p = 45^{53} \bmod 61$ and $m_q = 34^{49} \bmod 53$ with square-and-multiply, then apply Garner's formula $m = m_p + p \cdot \big[(m_q - m_p)\,(p^{-1} \bmod q) \bmod q\big]$ with the correct coefficient $p^{-1} \equiv 20 \pmod{53}$, and confirm you recover $m = 65$. Explain precisely why the value 2627 appeared in the text and what single number was wrong.

Chapter 5

Groups, Rings, and Fields

By now you have spent two chapters (Chapters 3 and 4) living inside the integers modulo n: adding, multiplying, finding inverses, hunting for primes. You may have noticed that the same moves keep coming back. Addition mod n behaves a lot like ordinary addition. The nonzero residues mod a prime behave a lot like ordinary multiplication. Inverses appear exactly when a gcd is 1. These patterns come from three *algebraic structures* that crypto is built on: groups, rings, and fields.

This chapter is the bridge between "I can compute mod n" and "I understand why Diffie–Hellman, RSA, and elliptic curves work." We are going to name the structures, state their rules precisely, and – this is the part most courses rush – build the intuition for *why* a cryptographer cares. The single most important payoff is Lagrange's theorem, which tells you that the order of an element must divide the size of the group. That one fact underlies RSA's correctness, the security parameters of Diffie–Hellman, and the choice of curve parameters in ECC. We will get there carefully, with tiny numbers you can check on your fingers.

5.1 Why crypto needs algebra at all

Cryptography is the art of doing arithmetic that is *easy one way and hard the other*. You want a function the legitimate parties can evaluate quickly and the adversary cannot invert. Plain integer arithmetic is hopeless for this: addition, multiplication, and even exponentiation over $\mathbb{Z}$ are all easy to undo.

The trick is to do arithmetic inside a *finite* world that wraps around. When you work modulo n, the operation $g \mapsto g^x \bmod n$ scrambles the order of elements so thoroughly that recovering x from g^x – the discrete logarithm problem – has no known fast algorithm for well-chosen n. The wrapping-around is what creates the asymmetry between "forward" (cheap) and "backward" (apparently astronomically expensive).

But the moment you commit to a finite, wrap-around arithmetic, you need to know which of your familiar rules survive. Can you always divide? When does an inverse exist? How many times can you square an element before it cycles back to where it started? Those questions have clean answers, and the answers are exactly the theorems about groups, rings, and fields. Algebra is the vocabulary in which "this is secure" becomes a statement you can actually prove.

Intuition

Think of a structure as a *game with rules*. A group is the minimal game where you can combine two things, undo any move, and never fall off the board. A ring adds a second operation so you have both "add" and "multiply." A field is the luxury edition where you can also *divide* by anything nonzero. Crypto schemes are recipes written in this game; the security proofs are arguments about what the rules do and do not let an attacker do.

5.2 Groups

5.2.1 The definition, and why each axiom earns its place

Definition 5.1 (Group)**.** A *group* is a set G together with a binary operation $* : G \times G \to G$ satisfying four axioms:

1. **Closure.** For all $a, b \in G$, the result $a * b$ is again in G.
2. **Associativity.** For all $a, b, c \in G$, $(a * b) * c = a * (b * c)$.
3. **Identity.** There is an element $e \in G$ with $e * a = a * e = a$ for every a.
4. **Inverses.** For every $a \in G$ there is some $a^{-1} \in G$ with $a * a^{-1} = a^{-1} * a = e$.

If in addition $a * b = b * a$ for all a, b, the group is *abelian* (commutative). Every group in this book is finite and abelian unless we say otherwise.

Each axiom pulls its weight. Closure says the operation never throws you out of the set – you can compose moves forever and stay inside the game. Associativity is what lets you write $a * b * c * d$ without parentheses, which is what makes *repeated* operation (and hence exponentiation) well defined. Identity gives you a "do nothing" move, the anchor that "g^0" or "$0 \cdot P$" refers to. Inverses are the main reason groups matter to crypto: every action is reversible, so encryption with a group operation can always be undone by the right party.

Focus

Notice what is *not* required: there is no second operation, no notion of "bigger" or "smaller," no division. The definition is deliberately minimal. Because the axioms are so few, an enormous range of objects qualify – integers under addition, nonzero residues under multiplication, points on an elliptic curve, symmetries of a cube – and any theorem proved from the four axioms applies to all of them at once.

5.2.2 The two groups you already know

You have met two groups already without the name.

Additive group $\mathbb{Z}/n\mathbb{Z}$. Take the residues $\{0, 1, \ldots, n-1\}$ with the operation "add, then reduce mod n." The identity is 0; the inverse of a is $n - a$ (and 0 is its own inverse). Closure and associativity come straight from ordinary addition. This group always has exactly n elements, for every $n \geq 1$.

Multiplicative group $(\mathbb{Z}/n\mathbb{Z})^*$. Take the residues that are *coprime* to n, with operation "multiply, then reduce mod n." The identity is 1. The catch – and the reason we throw away the non-coprime residues – is inverses. Recall from the previous chapter that a has a multiplicative inverse mod n if and only if $\gcd(a, n) = 1$. So if we want *every* element to be invertible (axiom 4), we must keep only the coprime ones. The size of this group is Euler's totient $\varphi(n)$.

Example 5.1 ($(\mathbb{Z}/15\mathbb{Z})^*$ by hand)**.** The residues coprime to $15 = 3 \cdot 5$ are $\{1, 2, 4, 7, 8, 11, 13, 14\}$, so $\varphi(15) = 8$. Check closure on a couple of products: $2{\cdot}8 = 16 \equiv 1$, $4{\cdot}4 = 16 \equiv 1$, $7{\cdot}13 = 91 = 6{\cdot}15{+}1 \equiv 1$. Each of those pairs is mutually inverse. Notice $5 \notin (\mathbb{Z}/15\mathbb{Z})^*$ because $\gcd(5, 15) = 5 \neq 1$: indeed $5 \cdot 3 = 15 \equiv 0$, and an element that can multiply to 0 can never have an inverse.

Common trap

A common slip: thinking $(\mathbb{Z}/n\mathbb{Z})^*$ is "all of $\{1, \ldots, n-1\}$." That is only true when n is prime. For composite n you must drop every residue sharing a factor with n. Forgetting this is the source of countless "why is my inverse code returning garbage" bugs – the modular inverse simply does not exist for those inputs.

5.2.3 Group notation: additive vs. multiplicative

The same abstract group can be written two ways, and crypto uses both. When we write the operation as "+", repeating it k times is written $k \cdot a = a + a + \cdots + a$, the identity is 0, and the inverse is $-a$. When we write the operation as "$\cdot$" (or just juxtaposition), repeating it k times is written $a^k = a \cdot a \cdots a$, the identity is 1, and the inverse is a^{-1}. Elliptic-curve people use additive notation (so the secret-key operation is "multiply a point by a scalar"); the $(\mathbb{Z}/p\mathbb{Z})^*$ world uses multiplicative notation (so the operation is "raise to a power"). Both notations describe the exact same group operations.

Walking question

Quick check before moving on: in the additive group $\mathbb{Z}/6\mathbb{Z}$, what is the inverse of 4? You need x with $4 + x \equiv 0 \pmod 6$, so $x = 2$. In the multiplicative group $(\mathbb{Z}/7\mathbb{Z})^*$, what is the inverse of 4? You need $4x \equiv 1 \pmod 7$; testing, $4 \cdot 2 = 8 \equiv 1$, so the inverse is 2. Same word "inverse," two different operations.

5.3 Order of an element, and cyclic groups

5.3.1 Order of an element

Pick a group element g and start applying the operation to itself: $g, g^2, g^3, \ldots$ (in multiplicative notation). Because the group is finite, this list cannot go on forever without repeating. The first time it lands back on the identity is the element's *order*.

Definition 5.2 (Order of an element)**.** The *order* of $g \in G$, written $\mathrm{ord}(g)$, is the smallest positive integer k such that $g^k = e$. (In additive notation: the smallest $k > 0$ with $k \cdot g = 0$.)

Example 5.2 (Orders in $(\mathbb{Z}/7\mathbb{Z})^*$)**.** Here 7 is prime so $(\mathbb{Z}/7\mathbb{Z})^* = \{1, 2, 3, 4, 5, 6\}$, six elements. Let us trace the powers of a few:

g	g^1	g^2	g^3	g^4	g^5	g^6
2	2	4	1	2	4	1
3	3	2	6	4	5	1
6	6	1	6	1	6	1

Reading off the first return to 1: $\mathrm{ord}(2) = 3$, $\mathrm{ord}(3) = 6$, $\mathrm{ord}(6) = 2$. Also $\mathrm{ord}(1) = 1$ since $1^1 = 1$. Notice every order we found – $1, 2, 3, 6$ – divides 6, the size of the group. This illustrates Lagrange's theorem in miniature: an element's orbit length must cleanly divide the total order of the group.

Intuition

The powers of g form a *cycle*: a closed loop that returns to the identity and then repeats. The order is the length of that loop. A short loop means g "can't reach much" by exponentiation; a long loop means g sweeps through many elements before coming home. For crypto we usually want long loops – an element whose powers cover as much ground as possible, so that g^x is a good hiding place for the secret x.

5.3.2 Cyclic groups and generators

When a single element's loop covers the *entire* group, that element is a generator and the group is cyclic.

Definition 5.3 (Cyclic group, generator)**.** A group G is *cyclic* if there exists $g \in G$ – a *generator* – such that every element of G equals g^k for some integer k. Equivalently, $\mathrm{ord}(g) = |G|$. We write $G = \langle g \rangle$.

In Example 5.2, the element 3 has order $6 = |(\mathbb{Z}/7\mathbb{Z})^*|$, so its powers $3, 2, 6, 4, 5, 1$ run through all six elements. Thus $(\mathbb{Z}/7\mathbb{Z})^*$ is cyclic with generator 3 (Figure 5.1). The element 2, by contrast, has order 3 and generates only the smaller set $\{1, 2, 4\}$ – a perfectly good group in its own right, sitting inside the big one.

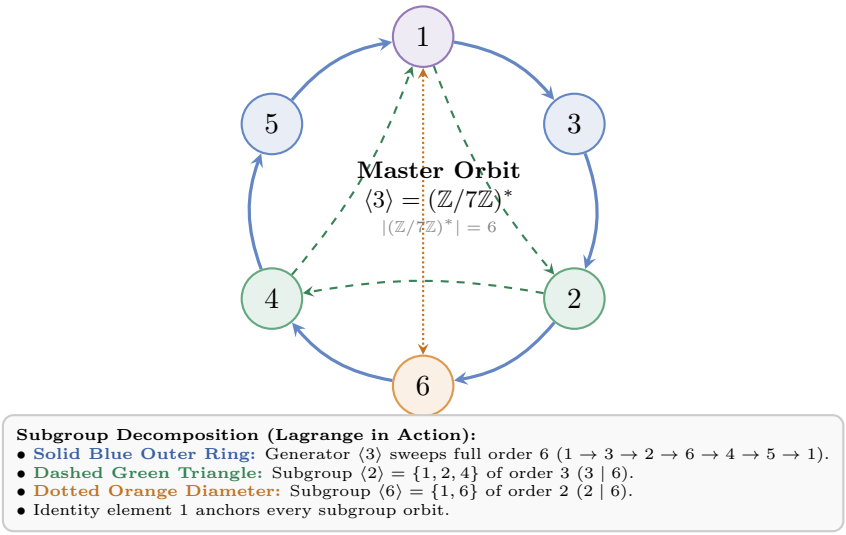


Figure 5.1: The cyclic group $(\mathbb{Z}/7\mathbb{Z})^* = \langle 3 \rangle$ and its internal subgroup orbits. The generator 3 visits all six elements. Inside this orbit live the order-3 subgroup $\langle 2 \rangle = \{1, 2, 4\}$ and the order-2 subgroup $\langle 6 \rangle = \{1, 6\}$. Every subgroup orbit length divides the master group size $6 = \varphi(7)$.

Example 5.3 ($\mathbb{Z}/6\mathbb{Z}$ is cyclic too)**.** In additive notation, "powers of g" means multiples. Take $g = 1$ in $\mathbb{Z}/6\mathbb{Z}$: $1, 2, 3, 4, 5, 0$ – all six residues, so 1 generates. Take $g = 5$: $5, 4, 3, 2, 1, 0$ – also all six, so 5 generates too. But $g = 2$ gives $2, 4, 0, 2, 4, 0, \ldots$, only $\{0, 2, 4\}$: order 3, not a generator. The generators of $\mathbb{Z}/6\mathbb{Z}$ are exactly 1 and 5, the residues coprime to 6. (That is a general fact: k generates $\mathbb{Z}/n\mathbb{Z}$ iff $\gcd(k, n) = 1$.)

Why it works

Why cyclic groups suit cryptography. In a cyclic group $\langle g \rangle$ of size q, every element is g^x for a unique $x \in \{0, 1, \ldots, q-1\}$. The map $x \mapsto g^x$ is therefore a bijection between exponents and group elements. Going *forward* ($x \mapsto g^x$) is a handful of multiplications via fast exponentiation. Going *backward* ($g^x \mapsto x$, the discrete logarithm) has no known efficient algorithm in well-chosen groups. That gap – a bijection that is easy one way and hard the other – is precisely a one-way function with structure, and structure is what lets two parties combine their secrets (Diffie–Hellman) without ever revealing them.

History

The word *group* comes from Évariste Galois, who in 1832 – the night before a fatal duel at age twenty – studied permutations of polynomial roots to characterize which equations can be solved by radicals; rejected by the French Academy during his lifetime, his manuscripts were published by Joseph Liouville in 1846 and founded abstract algebra. A century and a half later, Whitfield Diffie and Martin Hellman's 1976 paper "New Directions in Cryptography" put cyclic groups at the heart of public-key cryptography. They worked in $(\mathbb{Z}/p\mathbb{Z})^*$ for a large prime p. Decades of effort have made the discrete log *somewhat* easier in $(\mathbb{Z}/p\mathbb{Z})^*$ (index-calculus methods), which is why modern systems migrated to elliptic-curve groups, where those methods do not apply and the same security needs far smaller numbers. The underlying algebra – a cyclic group with a hard discrete log – is identical; only the group changed.

5.3.3 Not every group is cyclic

$(\mathbb{Z}/n\mathbb{Z})^*$ is cyclic for some n and not for others. It *is* cyclic exactly when $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p. The smallest cautionary example is $(\mathbb{Z}/8\mathbb{Z})^* = \{1, 3, 5, 7\}$.

Example 5.4 ($(\mathbb{Z}/8\mathbb{Z})^*$ has no generator)**.** Square every element mod 8: $1^2 = 1$, $3^2 = 9 \equiv 1$, $5^2 = 25 \equiv 1$, $7^2 = 49 \equiv 1$. *Every* element squares to 1, so every non-identity element has order 2. No element has order $4 = |(\mathbb{Z}/8\mathbb{Z})^*|$, hence no generator exists. This group is not cyclic; structurally it is the "Klein four-group," two independent order-2 switches.

This matters in practice. When a protocol assumes a generator exists and you hand it a non-cyclic group, the math quietly breaks. Picking the modulus so that $(\mathbb{Z}/n\mathbb{Z})^*$ is cyclic (or, more precisely, working inside a prime-order cyclic *subgroup*) is part of doing crypto correctly, which we make precise after Lagrange.

5.4 Subgroups and Lagrange's theorem

5.4.1 Subgroups

A *subgroup* is a subset that is itself a group under the same operation.

Definition 5.4 (Subgroup)**.** A nonempty subset $H \subseteq G$ is a *subgroup* (written $H \leq G$) if it is closed under the operation and under inverses: $a, b \in H \Rightarrow a * b \in H$ and $a^{-1} \in H$. (These two conditions force $e \in H$ and inherit associativity from G.)

The cleanest source of subgroups is a single element. The set of all powers of g,

$$\langle g \rangle = \{e, g, g^2, \ldots, g^{\mathrm{ord}(g)-1}\},$$

is always a subgroup – the *cyclic subgroup generated by* g. Its size is exactly $\mathrm{ord}(g)$. We saw this concretely: in $(\mathbb{Z}/7\mathbb{Z})^*$, $\langle 2 \rangle = \{1, 2, 4\}$ has $3 = \mathrm{ord}(2)$ elements.

5.4.2 Lagrange's theorem

Lagrange's theorem is the central result of this chapter.

Theorem 5.1 (Lagrange)**.** *If G is a finite group and $H \leq G$ is a subgroup, then $|H|$ divides $|G|$.*

Why it works

Why Lagrange is true (the coset argument). For a fixed $a \in G$, define the *coset* $aH = \{a * h : h \in H\}$. Two facts do all the work.
Every coset has the same size as H. The map $h \mapsto a * h$ from H to aH is reversible – multiply by a^{-1} to undo it – so it is a bijection. Hence $|aH| = |H|$.
Distinct cosets do not overlap. Suppose aH and bH share an element. Then $ah_1 = bh_2$ for some $h_1, h_2 \in H$, so $b^{-1}a = h_2 h_1^{-1} \in H$. Multiplying H on the left by an element of H just reshuffles H, which forces $aH = bH$. So any two cosets are either identical or disjoint.

Now every element a lives in its own coset aH (since $e \in H$ gives $a = a * e \in aH$), so the cosets *tile* G completely: they partition it into disjoint blocks, each of size $|H|$. If there are m blocks, then $|G| = m \cdot |H|$, and therefore $|H|$ divides $|G|$. The number m is called the *index* $[G : H]$.

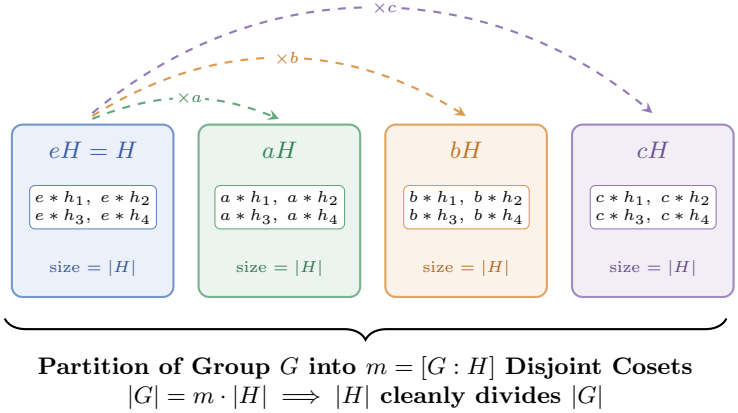


Figure 5.2: Visual proof of Lagrange's theorem by coset tiling. The subgroup H is mapped bijectively to each disjoint coset aH by left multiplication. Because every element of G belongs to exactly one coset and every coset has identical size $|H|$, the subgroup size $|H|$ must divide the group size $|G|$.

5.4.3 The consequences crypto actually uses

Lagrange by itself is a counting statement. Its corollaries are what you reach for daily.

Corollary 5.1 (Order divides group size)**.** *For any g in a finite group G,* $\mathrm{ord}(g)$ *divides* $|G|$.

This is immediate: $\mathrm{ord}(g) = |\langle g \rangle|$ and $\langle g \rangle \leq G$, so apply Lagrange. It explains the pattern from Example 5.2: in a group of 6 elements, the only possible orders are $1, 2, 3, 6$ – the divisors of 6. You will never find an element of order 4 or 5 there, no matter how hard you look.

Corollary 5.2 (Identity at the group size)**.** *For any g in a finite group G of order $n = |G|$, we have $g^n = e$.*

Write $k = \mathrm{ord}(g)$, which divides n by the previous corollary, say $n = km$. Then $g^n = (g^k)^m = e^m = e$. Raising *any* element to the size of the group sends it home to the identity.

This last corollary has two familiar special cases. Specialize $G = (\mathbb{Z}/n\mathbb{Z})^*$, whose size is $\varphi(n)$, and Corollary 5.2 becomes Euler's theorem; specialize further to a prime modulus and it becomes Fermat's little theorem.

Corollary 5.3 (Euler and Fermat, for free)**.** *If* $\gcd(a, n) = 1$ *then* $a^{\varphi(n)} \equiv 1 \pmod{n}$ (Euler)*. If p is prime and $p \nmid a$ then* $a^{p-1} \equiv 1 \pmod{p}$ (Fermat).

Intuition

You may have seen Euler's and Fermat's theorems proved by clever direct manipulation in the previous chapter. Here they fell out of a single structural idea – "order divides group size" – with no manipulation at all. That is the main benefit of the algebraic viewpoint: a theorem that looked like a number-theory trick turns out to be an instance of something that holds in *every* finite group, elliptic curves included.

Example 5.5 (Euler in action)**.** Take $n = 15$, where $\varphi(15) = 8$ (Example 5.1). Euler promises $a^8 \equiv 1$ for any a coprime to 15. Check $a = 2$: $2^4 = 16 \equiv 1$, and indeed $2^8 = (2^4)^2 \equiv 1$. Here $\mathrm{ord}(2) = 4$, which divides 8 – Lagrange again. So you do not even need the full exponent 8; the true order 4 already does the job, but 8 is guaranteed to work for *every* element at once.

Connection

The cost of computing $g^x \bmod n$ is the subject of the companion algorithms book's chapter on *modular exponentiation*: square-and-multiply does it in about $\log_2 x$ modular multiplications, so even a 2048-bit exponent costs only a few thousand multiplies. Lagrange tells you something orthogonal and equally practical: you may first reduce the exponent modulo $\mathrm{ord}(g)$ (or modulo $\varphi(n)$) before you start, because $g^x = g^{x \bmod \mathrm{ord}(g)}$. The two facts together are why RSA decryption can use a private exponent reduced mod $\varphi(n)$ rather than something astronomically large.

5.5 From one operation to two: rings

A group gives you one operation. Ordinary arithmetic has two – you add *and* multiply – and they interact through distributivity. The structure that captures "two compatible operations" is a ring.

Definition 5.5 (Commutative ring with identity)**.** A *ring* is a set R with two operations $+$ and $\cdot$ such that:

- $(R, +)$ is an abelian group, with additive identity 0 and negatives $-a$;
- multiplication is associative, commutative, and has an identity 1;
- the two are linked by *distributivity*: $a \cdot (b + c) = a \cdot b + a \cdot c$.

The flagship example is $\mathbb{Z}/n\mathbb{Z}$ *with both operations*: add mod n, multiply mod n. Every $\mathbb{Z}/n\mathbb{Z}$ is a ring. So is the set of integers $\mathbb{Z}$, and the set of polynomials with coefficients in any ring – a fact we rely on in Chapter 6, where we build $\mathbb{F}_{2^8}$ (the byte field inside AES) out of polynomials mod 2.

Focus

What a ring *lacks* is the interesting part. Multiplication is *not* required to have inverses. In $\mathbb{Z}/n\mathbb{Z}$ you can always add, subtract, and multiply, but you cannot always divide. The elements you *can* divide by are exactly the *units*: those with a multiplicative inverse, i.e. the coprime residues. Collect the units of a ring and they form a group under multiplication – and for $\mathbb{Z}/n\mathbb{Z}$ that group is precisely $(\mathbb{Z}/n\mathbb{Z})^*$ from Section 5.2.

5.5.1 Zero divisors: the wrinkle that composite moduli introduce

In ordinary integers, if $a \cdot b = 0$ then $a = 0$ or $b = 0$. In a general ring that can fail, and the failure is exactly what makes composite moduli awkward.

Definition 5.6 (Zero divisor)**.** A nonzero $a \in R$ is a *zero divisor* if there is a nonzero b with $a \cdot b = 0$.

Example 5.6 (Zero divisors mod 12)**.** In $\mathbb{Z}/12\mathbb{Z}$, $3 \cdot 4 = 12 \equiv 0$, yet neither 3 nor 4 is zero. Both are zero divisors. They are also exactly the residues sharing a factor with 12 – the non-units. A zero divisor can never be a unit: if $ab = 0$ with $b \neq 0$ and a also had an inverse a^{-1}, then $b = a^{-1}(ab) = a^{-1} \cdot 0 = 0$, a contradiction. So in $\mathbb{Z}/n\mathbb{Z}$ each nonzero residue is *either* a unit *or* a zero divisor, decided by gcd with n.

Common trap

Zero divisors break the cancellation law. Over the rationals, $ax = ay$ with $a \neq 0$ lets you cancel to $x = y$. Over $\mathbb{Z}/12\mathbb{Z}$ it does not: $3 \cdot 2 = 6 = 3 \cdot 6$, but $2 \neq 6$. Any algorithm that "divides through" by a value must first check that the value is a unit. This is a real, recurring implementation bug – and it is the main reason cryptographers prefer prime moduli, where the next section shows there are no zero divisors at all.

5.6 Fields: rings with division

A field is a ring in which division by anything nonzero is allowed. It is the structure where all four arithmetic operations behave the way your school algebra led you to expect.

Definition 5.7 (Field)**.** A *field* is a commutative ring with $1 \neq 0$ in which *every* nonzero element has a multiplicative inverse. Equivalently: $(F, +)$ is an abelian group and $(F \setminus \{0\}, \cdot)$ is also an abelian group.

The familiar infinite fields are the rationals $\mathbb{Q}$, the reals $\mathbb{R}$, and the complex numbers $\mathbb{C}$. But crypto runs on *finite* fields, and the most important family is straightforward to describe.

Theorem 5.2 (Prime fields)**.** *$\mathbb{Z}/p\mathbb{Z}$ is a field if and only if p is prime. When p is prime we write it $\mathbb{F}_p$.*

Why it works

Why primality is exactly the dividing line. If p is prime, then every residue $a \in \{1, \ldots, p-1\}$ satisfies $\gcd(a, p) = 1$, so every nonzero element is a unit and has an inverse – that is the definition of a field. If p is composite, write $p = ab$ with $1 < a, b < p$; then a and b are zero divisors (Example 5.6 showed the pattern), and a zero divisor cannot have an inverse, so $\mathbb{Z}/p\mathbb{Z}$ falls short of being a field. Primality is exactly the condition under which "divide by anything nonzero" becomes legal.

Example 5.7 ($\mathbb{F}_5$ in full)**.** The field $\mathbb{F}_5 = \{0, 1, 2, 3, 4\}$. Its multiplication table on nonzero elements reveals why division works:

·	1	2	3	4
1	1	2	3	4
2	2	4	1	3
3	3	1	4	2
4	4	3	2	1

Finding Inverses:
To find 3^{-1}, locate 3 in the left column, scan right to find the **1**, and look up: it sits in column 2, so $3^{-1} = 2$.
Because $p = 5$ is prime, every row is a permutation containing exactly one 1.

Figure 5.3: The multiplication table for the nonzero elements of $\mathbb{F}_5$. The appearance of a 1 in every row and column confirms that every nonzero element has a unique multiplicative inverse. This structure defines a field.

Read off inverses by finding the 1 in each row: $1^{-1} = 1$, $2^{-1} = 3$, $3^{-1} = 2$, $4^{-1} = 4$. Every nonzero element has one, as a field demands. And the nonzero elements $\{1, 2, 3, 4\}$ form the multiplicative group $(\mathbb{Z}/5\mathbb{Z})^* = \mathbb{F}_5^*$; tracing powers of 2 gives $2, 4, 3, 1$, order 4, so 2 generates – the multiplicative group of a finite field is always cyclic, a fact we exploit constantly.

Formal

The cyclic structure of $\mathbb{F}_p^*$ is worth stating on its own because so much depends on it: *for every prime p, the multiplicative group $\mathbb{F}_p^*$ is cyclic of order $p - 1$.* A generator of it is called a *primitive root* mod p. Its existence is what guarantees Diffie–Hellman has a generator to stand on. We prove this carefully in Chapter 6; for now, treat it as a reliable promise that prime moduli always hand you a generator.

Focus

Are there finite fields whose size is *not* prime? Yes – but only when the size is a prime *power* p^k. There is exactly one field of each prime-power size, written $\mathbb{F}_{p^k}$, and none of any other size (no field of 6 or 10 elements exists). The p^k fields are not built as $\mathbb{Z}/p^k\mathbb{Z}$ – that ring has zero divisors – but from polynomials, which is the subject of the next chapter and the reason AES can treat a byte as an element of $\mathbb{F}_{2^8}$.

5.7 A worked tour: the structures inside $\mathbb{Z}/n\mathbb{Z}$

Here are all three structures for a single modulus, $n = 9$.

As a **ring**, $\mathbb{Z}/9\mathbb{Z} = \{0, 1, \ldots, 8\}$ with addition and multiplication mod 9. It is not a field: $9 = 3 \cdot 3$ is a prime power but composite, and $3 \cdot 3 = 9 \equiv 0$ makes 3 a zero divisor.

Its **additive group** $(\mathbb{Z}/9\mathbb{Z}, +)$ has 9 elements and is cyclic, generated by 1 (or by any residue coprime to 9, namely $1, 2, 4, 5, 7, 8$).

Its **group of units** is $(\mathbb{Z}/9\mathbb{Z})^* = \{1, 2, 4, 5, 7, 8\}$, the residues coprime to 9, so $\varphi(9) = 6$. Is it cyclic? Trace powers of 2 mod 9:

$$2,\ 4,\ 8,\ 16 \equiv 7,\ 14 \equiv 5,\ 10 \equiv 1.$$

That is $2, 4, 8, 7, 5, 1$ – all six units, so $\mathrm{ord}(2) = 6$ and 2 is a generator. $(\mathbb{Z}/9\mathbb{Z})^*$ is cyclic. (This

matches the rule from Section 5.3: $9 = 3^2$ is a prime power, so its units form a cyclic group.) Contrast $(\mathbb{Z}/8\mathbb{Z})^*$ from Example 5.4, which was *not* cyclic – the modulus matters.

Walking question

Verify Lagrange inside $(\mathbb{Z}/9\mathbb{Z})^*$. Its size is 6, so element orders must divide 6, i.e. lie in $\{1, 2, 3, 6\}$. Check ord(8): $8 \equiv -1$, and $(-1)^2 = 1$, so ord(8) = 2. Check ord(4): $4, 16 \equiv 7, 28 \equiv 1$, so ord(4) = 3. Both 2 and 3 divide 6, as required. The subgroup $\langle 4 \rangle = \{1, 4, 7\}$ has size 3, which divides 6 – two whole cosets tile the six units. Everything lines up with Theorem 5.1.

5.8 Toy code: groups by hand

The fastest way to make these definitions concrete is to compute orders and hunt for generators in code. The snippet below works in any $(\mathbb{Z}/n\mathbb{Z})^*$.

Implement it

Learning only – never use this in real cryptography. It is plain, variable-time, and tiny on purpose. Real systems use vetted libraries with constant-time arithmetic and carefully chosen, large parameters.

```python
from math import gcd

def units(n):
    "The group (Z/nZ)* : residues coprime to n."
    return [a for a in range(1, n) if gcd(a, n) == 1]

def order(a, n):
    "Smallest k > 0 with a^k = 1 mod n. Assumes gcd(a, n) = 1."
    k, x = 1, a % n
    while x != 1:
        x = (x * a) % n
        k += 1
    return k

def generators(n):
    "Elements whose order equals |(Z/nZ)*| = phi(n); empty if non-cyclic."
    U = units(n)
    phi = len(U)
    return [a for a in U if order(a, n) == phi]

# Check Lagrange: every order divides phi(n).
n = 9
U = units(n)                        # [1, 2, 4, 5, 7, 8]
phi = len(U)                        # 6
orders = {a: order(a, n) for a in U}
assert all(phi % orders[a] == 0 for a in U)   # Lagrange holds
print("phi =", phi)
print("orders =", orders)           # {1:1, 2:6, 4:3, 5:6, 7:3, 8:2}
print("generators =", generators(n))          # [2, 5]
```

```
30
31 # A non-cyclic group has no generator at all:
32 print("generators of 8:", generators(8))      # []  (Z/8Z)* is not cyclic
```

The assertion is Lagrange's theorem turned into a runtime check: no matter which n you plug in, every element's order divides $\varphi(n)$. The empty generator list for $n = 8$ is Example 5.4 reappearing as data.

5.9 A pitfall: small subgroups and where Lagrange bites back

Lagrange is usually your friend, but an attacker can turn the same theorem against a careless implementation. The danger is the *small subgroup*.

Attack

Small-subgroup confinement. Diffie–Hellman in $(\mathbb{Z}/p\mathbb{Z})^*$ is supposed to live in a large prime-order subgroup. But $(\mathbb{Z}/p\mathbb{Z})^*$ has size $p-1$, which is even, so by Lagrange it *always* contains a subgroup of order 2 – namely $\{1, p-1\}$, since $(p-1)^2 \equiv 1$. More generally, every divisor of $p-1$ gives a subgroup of that size.

Now suppose Alice computes a shared secret as $K = Y^a \bmod p$, where Y is a value Bob sent. If Alice does not validate Y, Eve (impersonating Bob) can send a Y of small order – say order 2, so $Y \in \{1, p-1\}$. Then $K = Y^a$ can take only 2 possible values regardless of Alice's secret a, and which one it is leaks the parity of a (Figure 5.4). Repeat with subgroups of order $3, 5, \ldots$ and Eve peels off a modulo each small factor, then reassembles a by the Chinese Remainder Theorem. The exponent's secrecy collapses one small subgroup at a time.

The fix follows from Lagrange. Choose $p = 2q + 1$ with q prime (a *safe prime*), so the only subgroup sizes are $1, 2, q, 2q$ – no small factors to confine into – and work in the order-q subgroup. Then *validate* every received element: reject anything not in that subgroup, e.g. by checking $Y^q \equiv 1$ and $Y \neq 1$. Elliptic-curve deployments do the analogous "point validation" and choose curves of nearly prime order (small *cofactor*) for exactly the same reason.

In the real world

This is not a textbook hypothetical. The order-q subgroup of a safe-prime $(\mathbb{Z}/p\mathbb{Z})^*$ is the setting of classic Diffie–Hellman as standardized for TLS and IPsec. RFC 7919 ships fixed, vetted safe-prime groups ("ffdhe2048", "ffdhe3072", ...) precisely so implementers stop rolling their own parameters and stop tripping over small subgroups. Modern TLS 1.3 leans instead on elliptic-curve groups such as Curve25519, whose designer Daniel Bernstein chose the curve so that the cofactor is a tiny 8 and the order-validation problem largely takes care of itself. In every case the design rule is the same sentence: *know the order of your group, and make sure every element you accept lives in the big prime-order piece of it.*

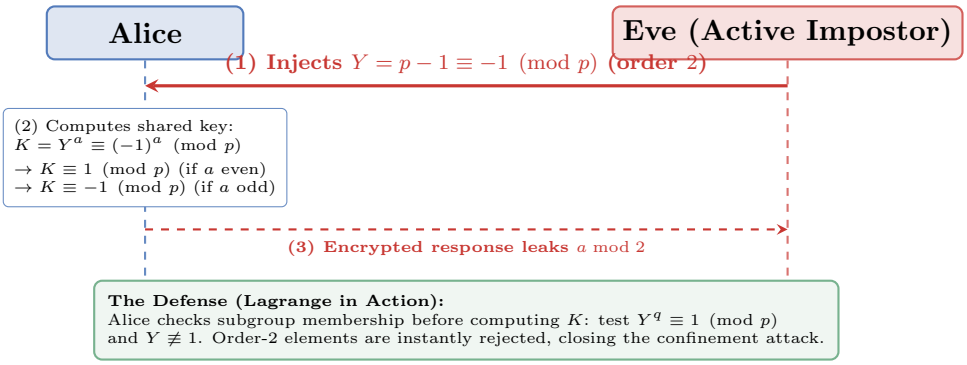


Figure 5.4: Small-subgroup confinement attack on unvalidated Diffie-Hellman. Eve sends an element $Y = p - 1$ of order 2. Alice's exponentiation collapses K into a 1-bit value revealing a mod 2. Group membership validation blocks the attack by checking that received elements live strictly in the large prime-order subgroup.

5.10 How the three structures power the rest of the book

The rest of the book builds on these three structures as follows.

Groups give you the discrete-log world: a cyclic group with a hard-to-invert exponentiation map. That single idea drives Diffie–Hellman key exchange (Chapter 18), ElGamal and DSA (Chapter 20), and elliptic-curve cryptography (Chapter 21), where the group is points on a curve instead of residues but the algebra is the same.

Rings give you $\mathbb{Z}/n\mathbb{Z}$ with both operations, which is the natural home of RSA (Chapter 19): encryption is $m \mapsto m^e \bmod n$ for a *composite* $n = pq$, and the correctness proof is Euler's theorem – our Corollary 5.3 – applied in the ring $\mathbb{Z}/n\mathbb{Z}$. The composite modulus, with its zero divisors, is essential: factoring n is what is hard.

Fields give you exact, reversible arithmetic with no zero divisors. Prime fields $\mathbb{F}_p$ underlie elliptic curves (the curve lives over a field). Prime-power fields $\mathbb{F}_{2^k}$ underlie the byte-level mixing in AES and the carry-less arithmetic in GCM authentication. The next chapter builds those fields from the ring of polynomials – yet another instance of "take a ring, quotient by something irreducible, and a field pops out," the exact same move that turned $\mathbb{Z}$ into $\mathbb{F}_p$ by quotienting by a prime.

Intuition

If you remember one sentence from this chapter, make it this: *a group lets you combine and undo, a ring adds a second operation, and a field lets you divide; crypto picks whichever structure makes the easy direction cheap and the hard direction hopeless.* Everything else – generators, orders, Lagrange, zero divisors – is bookkeeping in service of that choice.

5.11 Exercises

Exercise 5.1 (Group or not?)**.** For each, decide whether it is a group; if not, name the failing axiom. (a) The even integers under addition. (b) The odd integers under addition. (c) $\{1, 2, 3, 4\}$

under multiplication mod 5. (d) $\{0, 1, 2, 3\}$ under multiplication mod 4. (e) The set $(\mathbb{Z}/n\mathbb{Z})^*$ under multiplication mod n, for general n.

Exercise 5.2 (Orders by hand)**.** Work entirely mod 11. (a) Compute the order of every element of $(\mathbb{Z}/11\mathbb{Z})^*$. (b) List all generators. (c) Confirm that each order you found divides 10, and explain which theorem guarantees this in advance.

Exercise 5.3 (Cyclic or not)**.** Determine whether $(\mathbb{Z}/n\mathbb{Z})^*$ is cyclic for $n = 7, 8, 9, 10, 12, 16$. For each cyclic case exhibit a generator; for each non-cyclic case, show that every element's order is strictly less than $\varphi(n)$ and identify the largest order that occurs.

Exercise 5.4 (Subgroups and Lagrange)**.** In $(\mathbb{Z}/13\mathbb{Z})^*$ (which has 12 elements): (a) Find the cyclic subgroup generated by 3 and state its size. (b) Find a subgroup of size 4 and one of size 6. (c) Explain why no subgroup of size 5 can exist, citing Lagrange.

Exercise 5.5 (Euler from Lagrange)**.** Using only Corollary 5.2 ("$g^{|G|} = e$"), prove Euler's theorem: if $\gcd(a, n) = 1$ then $a^{\varphi(n)} \equiv 1 \pmod{n}$. Then specialize to deduce Fermat's little theorem. State precisely which group you are applying the corollary to and why a belongs to it.

Exercise 5.6 (Zero divisors and cancellation)**.** (a) List all zero divisors in $\mathbb{Z}/18\mathbb{Z}$. (b) Show $\gcd(a, 18) > 1$ exactly characterizes them. (c) Exhibit a concrete failure of cancellation: find $a \neq 0$ and $x \neq y$ in $\mathbb{Z}/18\mathbb{Z}$ with $ax \equiv ay$. (d) Explain why the same cannot happen in $\mathbb{F}_{19}$.

Exercise 5.7 (Fields and non-fields)**.** (a) Prove that $\mathbb{Z}/n\mathbb{Z}$ is a field iff n is prime, filling in every step of Theorem 5.2. (b) Build the full multiplication table of $\mathbb{F}_7$ and read off the inverse of each nonzero element. (c) Explain in one sentence why no field of order 6 exists.

Exercise 5.8 (Break a careless Diffie–Hellman)**.** Let $p = 23$, so $p - 1 = 22 = 2 \cdot 11$. (a) Find an element of order 2 and an element of order 11 in $(\mathbb{Z}/23\mathbb{Z})^*$. (b) Suppose Alice computes $K = Y^a \bmod 23$ without validating Y, and Eve sends the order-2 element as Y. Show that K reveals $a \bmod 2$. (c) State the safe-prime fix and the validation check that defeats this attack, connecting it to the small-subgroup discussion in Section 5.9.

Exercise 5.9 (Generators of the additive group)**.** Prove that k generates the additive group $\mathbb{Z}/n\mathbb{Z}$ if and only if $\gcd(k, n) = 1$. (Hint: the multiples of k are $\{0, k, 2k, \dots\} \bmod n$; relate the number of distinct values to ord of k and to $\gcd(k, n)$.)

Exercise 5.10 (Count the generators)**.** A cyclic group of order n has exactly $\varphi(n)$ generators. (a) Verify this for $(\mathbb{Z}/7\mathbb{Z})^*$ ($n = 6$) by listing its generators and counting. (b) Verify for $(\mathbb{Z}/9\mathbb{Z})^*$ ($n = 6$). (c) Use the formula to predict, without enumeration, how many primitive roots $(\mathbb{Z}/p\mathbb{Z})^*$ has when $p = 2q + 1$ is a safe prime; evaluate it for $p = 23$.

Chapter 6

Finite Fields

In Chapter 5 we saw that a field is a place where you can add, subtract, multiply, and divide, and where the usual algebra you learned in school keeps working: zero times anything is zero, $a(b+c) = ab + ac$, and every nonzero element has a reciprocal. The rational numbers $\mathbb{Q}$, the reals $\mathbb{R}$, and the complex numbers $\mathbb{C}$ are all fields. They are also all infinite, and a computer cannot store an arbitrary real number. Cryptography runs on machines that manipulate fixed-size chunks of bits, so we need fields with only finitely many elements: *finite fields*.

This chapter builds the two kinds of finite field that nearly all of modern cryptography stands on. The first, $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ for a prime p, you have already met — it is arithmetic modulo a prime, and it is where Diffie–Hellman, RSA's exponent group, and elliptic curves live. The second, $\mathbb{F}_{2^n}$, is stranger and more useful: a field whose elements are bit-strings, whose addition is XOR, and whose multiplication mixes bits in a way that is cheap in hardware and algebraically rich. AES lives in $\mathbb{F}_{2^8}$, and by the end of this chapter you will have built that exact field by hand, multiplied two bytes in it, and understood why the AES designers chose it.

The main results of the chapter: every finite field has a prime-power number of elements $q = p^n$, there is essentially only one field of each size, and we can construct it concretely using only polynomials and long division. Once you see that, $\mathbb{F}_{2^8}$ stops looking like an opaque recipe in the AES specification and becomes something you can compute with on paper.

6.1 What a Field Is, Quickly

We will lean on the previous chapter's definitions, but let us restate the one that matters so the chapter stands on its own.

Definition 6.1 (Field)**.** A *field* is a set $\mathbb{F}$ with two operations, addition $+$ and multiplication $\cdot$, such that:

- $(\mathbb{F}, +)$ is an abelian group with identity 0 (you can add and subtract; addition commutes);
- $(\mathbb{F} \setminus \{0\}, \cdot)$ is an abelian group with identity 1 (every nonzero element has a multiplicative inverse; you can multiply and divide);
- multiplication distributes over addition: $a \cdot (b + c) = a \cdot b + a \cdot c$.

The key clause is "every nonzero element has an inverse." That is what separates a field from a mere ring like $\mathbb{Z}/n\mathbb{Z}$ for composite n. In $\mathbb{Z}/6\mathbb{Z}$ the element 2 has no inverse: there is no y with $2y \equiv 1 \pmod 6$, because 2 and 6 share the factor 2. So $\mathbb{Z}/6\mathbb{Z}$ is a ring but not a field. The moment the modulus is prime, *every* nonzero element is coprime to it, every nonzero element gets an inverse, and we have a field.

Intuition

A field is the smallest setting in which "solve $ay = b$ for y" always has a unique answer when $a \neq 0$: just multiply both sides by a^{-1}. Without inverses, no division, and almost every cryptographic construction that needs to "undo" an operation would fall apart. AES decryption, Shamir secret sharing, Reed–Solomon codes, and polynomial commitment schemes all rely on being able to invert and divide inside a finite field.

6.1.1 The first family: $\mathbb{F}_p$

Theorem 6.1 (Prime moduli give fields)**.** *For a prime p, the ring $\mathbb{Z}/p\mathbb{Z} = \{0, 1, \ldots, p-1\}$ with addition and multiplication mod p is a field. We write it $\mathbb{F}_p$ (or $\mathrm{GF}(p)$, "Galois field of order p").*

Proof. $\mathbb{Z}/p\mathbb{Z}$ is already a commutative ring with 1. We only need multiplicative inverses for nonzero elements. Take $a \in \{1, \ldots, p-1\}$. Since p is prime and $0 < a < p$, we have $\gcd(a, p) = 1$. By Bézout's identity (the extended Euclidean algorithm from Chapter 4) there are integers u, v with $ua + vp = 1$. Reducing mod p gives $ua \equiv 1 \pmod p$, so $u \bmod p$ is the inverse of a. Every nonzero element is invertible, so $\mathbb{F}_p$ is a field. □

Example 6.1 (Inverses in $\mathbb{F}_7$)**.** In $\mathbb{F}_7$, what is 3^{-1}? We want $3y \equiv 1 \pmod 7$. Trying $y = 5$: $3 \cdot 5 = 15 = 14 + 1 \equiv 1$. So $3^{-1} = 5$. The full inverse table for $\mathbb{F}_7$:

$$1^{-1} = 1, \quad 2^{-1} = 4, \quad 3^{-1} = 5, \quad 4^{-1} = 2, \quad 5^{-1} = 3, \quad 6^{-1} = 6.$$

Notice $6 = -1$ is its own inverse, as is 1. Everything else pairs up. Division is now legal: $\frac{2}{3}$ in $\mathbb{F}_7$ means $2 \cdot 3^{-1} = 2 \cdot 5 = 10 \equiv 3$.

Connection

Computing a^{-1} in $\mathbb{F}_p$ is exactly the extended Euclidean algorithm, which runs in $O(\log p)$ steps — see the algorithms companion for the cost analysis. An alternative uses Fermat's little theorem: $a^{p-1} \equiv 1$, so $a^{-1} \equiv a^{p-2} \pmod p$, computed by fast modular exponentiation in $O(\log p)$ multiplications. The Euclidean route is usually faster; the Fermat route is constant-time-friendlier, which matters for side-channel resistance (see Chapter 37).

6.2 Why Prime Powers, and Nothing Else

So $\mathbb{F}_p$ exists for every prime. Can we have a field with, say, 6 elements? Or 4? The answer is one of the cleanest theorems in algebra.

Theorem 6.2 (Classification of finite fields)**.** *A finite field has exactly $q = p^n$ elements for some prime p and integer $n \geq 1$. Conversely, for every prime power p^n there is a finite field with that many elements, and any two finite fields with the same number of elements are isomorphic — structurally identical up to relabeling their elements.*

We will not prove the whole theorem, but the "why p^n" half is short, so we give it here.

Why it works

Let $\mathbb{F}$ be a finite field. Look at the elements you get by repeatedly adding 1 to itself: $1, 1+1, 1+1+1, \ldots$ Because $\mathbb{F}$ is finite, this sequence must eventually repeat, and the first time you return to 0 defines a number p with $\underbrace{1+1+\cdots+1}_{p} = 0$. This p is called the *characteristic*. It must be prime: if $p = ab$ with $1 < a, b < p$, then $(\underbrace{1+\cdots+1}_{a})(\underbrace{1+\cdots+1}_{b}) = 0$, a product of two nonzero elements equalling zero — impossible in a field, where nonzero times nonzero is nonzero (otherwise one of them would be a zero-divisor and could not have an inverse).
Now the copy of $\{0, 1, \ldots, p-1\}$ sitting inside $\mathbb{F}$ is a sub-field isomorphic to $\mathbb{F}_p$. The whole field $\mathbb{F}$ is a vector space over this $\mathbb{F}_p$: you can add field elements and scale them by elements of $\mathbb{F}_p$, and all the vector-space axioms hold. A finite-dimensional vector space over $\mathbb{F}_p$ of dimension n has exactly p^n elements — choose each of the n coordinates from p options. Hence $|\mathbb{F}| = p^n$.

That argument also tells us *how* to build $\mathbb{F}_{p^n}$: realize it as an n-dimensional vector space over $\mathbb{F}_p$ with a compatible multiplication. The trick for the multiplication is polynomials.

Intuition

Why can't there be a field with 6 elements? Because $6 = 2 \cdot 3$ is not a prime power. There is a field of size $4 = 2^2$, a field of size $8 = 2^3$, a field of size $9 = 3^2$, but none of size 6, 10, 12, or 15. The gaps are exactly the non-prime-powers, and the classification theorem says no construction can fill them.

6.3 Polynomials over a Field

To build $\mathbb{F}_{p^n}$ we copy the construction of $\mathbb{F}_p$ itself, one level up. Recall how $\mathbb{F}_p$ was made: take the integers $\mathbb{Z}$, pick a prime p, and work modulo p. The primality of p was what made the quotient a field.

We will do the same with *polynomials* in place of integers. Fix a base field $\mathbb{F}_p$ and consider polynomials in one variable x with coefficients in $\mathbb{F}_p$:

$$\mathbb{F}_p[x] = \{\, a_d x^d + \cdots + a_1 x + a_0 : a_i \in \mathbb{F}_p \,\}.$$

You add them coefficient-by-coefficient and multiply them the usual way, reducing coefficients mod p. This set $\mathbb{F}_p[x]$ behaves very much like $\mathbb{Z}$:

- There is a notion of *degree* (the highest power with nonzero coefficient), playing the role that size/magnitude plays for integers.

- There is *division with remainder*: given f and a nonzero g, there are unique q (quotient) and r (remainder) with $f = qg + r$ and $\deg r < \deg g$. This is just polynomial long division.
- There are *irreducible* polynomials — ones that do not factor into lower-degree pieces over $\mathbb{F}_p$. These are the "primes" of $\mathbb{F}_p[x]$.

Intuition

The dictionary to keep in your head:

$$\mathbb{Z} \leftrightarrow \mathbb{F}_p[x], \qquad \text{prime} \leftrightarrow \text{irreducible polynomial}, \qquad \mathbb{Z}/p\mathbb{Z} \leftrightarrow \mathbb{F}_p[x]/(m(x)).$$

Working modulo a prime integer gave a field; working modulo an irreducible polynomial will give a field too. The degree of the polynomial controls how big the field is.

6.3.1 Irreducibility with small numbers

Over $\mathbb{F}_2$, the coefficients are just 0 and 1, so a polynomial is just a bit-string telling you which powers of x are present. For example $x^3 + x + 1$ is the bit-string 1011.

A polynomial of degree ≥ 1 is reducible over $\mathbb{F}_2$ if it factors. The quickest first test: a polynomial has the factor x iff its constant term is 0, and it has the factor $(x + 1)$ iff it has an even number of terms (iff $x = 1$ is a root, since over $\mathbb{F}_2$ a root at 1 means the coefficients sum to 0).

Example 6.2 (Cataloguing low-degree irreducibles over $\mathbb{F}_2$)**.** *Degree 1:* x and $x + 1$ are both irreducible (degree-1 polynomials always are).

Degree 2: the candidates are $x^2, x^2 + 1, x^2 + x, x^2 + x + 1$. The first three have a root in $\mathbb{F}_2$ (x^2 and $x^2 + x$ have root 0; $x^2 + 1 = (x + 1)^2$ has root 1). Only $x^2 + x + 1$ has no root in $\mathbb{F}_2$ — check: plug in 0 gives 1, plug in 1 gives $1 + 1 + 1 = 1$. So $x^2 + x + 1$ is the unique irreducible quadratic over $\mathbb{F}_2$, and it builds $\mathbb{F}_4$.

Degree 3: a cubic over $\mathbb{F}_2$ is reducible iff it has a root (a cubic that factors must have a linear factor). Testing the 8 cubics, exactly two have no root: $x^3 + x + 1$ and $x^3 + x^2 + 1$. Either one builds $\mathbb{F}_8$.

Walking question

Is $x^4 + x + 1$ irreducible over $\mathbb{F}_2$? It has no root: $f(0) = 1$, $f(1) = 1 + 1 + 1 = 1$. So it has no linear factor. But a degree-4 polynomial could still factor as two irreducible quadratics. The only irreducible quadratic over $\mathbb{F}_2$ is $x^2 + x + 1$, and $(x^2 + x + 1)^2 = x^4 + x^2 + 1 \neq x^4 + x + 1$. No linear factor, not the square of the lone irreducible quadratic — so $x^4 + x + 1$ is irreducible. (This polynomial builds $\mathbb{F}_{16}$ and shows up in CRC checksums.)

6.4 Building $\mathbb{F}_4$ by Hand

Let us construct the smallest non-prime finite field completely, because once you have seen $\mathbb{F}_4$ in full, $\mathbb{F}_{2^8}$ is just the same idea with bigger bit-strings.

Take $\mathbb{F}_2$ and the irreducible polynomial $m(x) = x^2 + x + 1$. Define

$$\mathbb{F}_4 = \mathbb{F}_2[x]/(m(x)),$$

the polynomials over $\mathbb{F}_2$ taken modulo $m(x)$. Because m has degree 2, every remainder has degree < 2, so the elements are

$$\mathbb{F}_4 = \{\, 0,\ 1,\ x,\ x+1 \,\},$$

exactly four of them — 2^2, as promised. Often x is written α and people call α "a root of m."

Addition is XOR of the coefficient vectors (since we are over $\mathbb{F}_2$):

$$(x) + (x+1) = 1, \qquad (x+1) + (x+1) = 0, \qquad 1 + 1 = 0.$$

Multiplication multiplies the polynomials and then reduces mod $m(x)$. The key relation comes from $m(x) = 0$ in the quotient:

$$x^2 + x + 1 = 0 \quad \Longrightarrow \quad x^2 = x + 1$$

(remember $-1 = +1$ over $\mathbb{F}_2$). So whenever an x^2 appears, replace it with $x + 1$. Watch:

$$x \cdot x = x^2 = x + 1,$$

$$x \cdot (x+1) = x^2 + x = (x+1) + x = 1,$$

$$(x+1)(x+1) = x^2 + 1 = (x+1) + 1 = x.$$

$\cdot$	0	1	x	x+1
0	0	0	0	0
1	0	1	x	x+1
x	0	x	x+1	1
x+1	0	x+1	1	x

Read off the inverses from the table: $x^{-1} = x + 1$ (their product is 1), and $(x+1)^{-1} = x$. Every nonzero element has an inverse, so this is a genuine field — and it has 4 elements, the field you cannot build out of $\mathbb{Z}/4\mathbb{Z}$ (which has the zero-divisor $2 \cdot 2 = 0$).

Common trap

$\mathbb{F}_4$ is *not* $\mathbb{Z}/4\mathbb{Z}$. In $\mathbb{Z}/4\mathbb{Z}$, $1 + 1 = 2$ and $2 + 2 = 0$; the additive group is cyclic of order 4. In $\mathbb{F}_4$, every element added to itself is 0 (characteristic 2); the additive group is $\mathbb{Z}_2 \times \mathbb{Z}_2$, not $\mathbb{Z}_4$. They have the same size and are both rings, but only $\mathbb{F}_4$ is a field. Mixing them up is one of the most common beginner errors with GF(2^n).

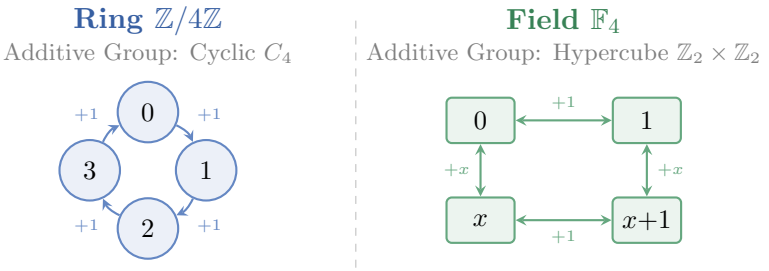


Figure 6.1: Additive structure of $\mathbb{Z}/4\mathbb{Z}$ vs $\mathbb{F}_4$. The integer ring $\mathbb{Z}/4\mathbb{Z}$ forms a single cycle of 4 elements under addition, meaning $1+1=2\neq 0$. The Galois field $\mathbb{F}_4$ forms a 2D hypercube of degree-1 polynomials over $\mathbb{F}_2$; every element is its own additive inverse, so $x+x=0$ and $1+1=0$. The two additive groups are therefore not isomorphic.

History

These fields are named for Évariste Galois (1811–1832), who published "Sur la théorie des nombres" in 1830 introducing finite fields of prime-power order p^n by adjoining roots of irreducible polynomials modulo p. In 1893, the American mathematician Eliakim Hastings Moore proved that all finite fields of the same order p^n are isomorphic and coined the term *Galois field*. The notation $\mathrm{GF}(q)$ is standard in engineering, while mathematicians write $\mathbb{F}_q$: for the same object, AES documents say $\mathrm{GF}(2^8)$ and algebra texts say $\mathbb{F}_{256}$. In modern computing, $\mathrm{GF}(2^8)$ is used in the AES S-box and $\mathrm{GF}(2^{128})$ in the GHASH authenticator of AES-GCM, though quasi-polynomial discrete-logarithm algorithms in characteristic 2 have pushed public-key cryptography away from binary fields toward prime-order fields.

6.5 The Field AES Uses: $\mathbb{F}_{2^8}$

A byte is eight bits, and the natural setting for byte-oriented cryptography is a field of $256=2^8$ elements. We identify a byte $b_7b_6\cdots b_1b_0$ with the polynomial

$$b_7x^7+b_6x^6+\cdots+b_1x+b_0\in\mathbb{F}_2[x],$$

a polynomial of degree at most 7. For instance the byte `0x57` $=0101\,0111$ corresponds to $x^6+x^4+x^2+x+1$.

To make these 256 bit-strings into a field, we pick an irreducible polynomial of degree 8 over $\mathbb{F}_2$ and reduce modulo it. AES fixes the choice

$$\boxed{m(x)=x^8+x^4+x^3+x+1} \qquad (\texttt{0x11B} \text{ in hex}),$$

sometimes called the Rijndael polynomial. So

$$\mathbb{F}_{2^8}=\mathbb{F}_2[x]/(m(x)),$$

with elements the 256 bytes, addition = XOR, and multiplication = polynomial multiplication followed by reduction mod $m(x)$.

Why it works

Why is $x^8 + x^4 + x^3 + x + 1$ irreducible, and why this one? Irreducibility you can verify by trial division: it has no root in $\mathbb{F}_2$ ($f(0) = 1$ and $f(1) = 1$), and you check it shares no factor with the irreducible polynomials of degree $2, 3, 4$. The AES designers needed *some* degree-8 irreducible; there are 30 of them. They picked one with few terms so that reduction is cheap: a smaller polynomial means fewer XORs when you reduce. The field $\mathbb{F}_{2^8}$ you get is the same regardless of which irreducible you choose (all fields of size 256 are isomorphic by Theorem 6.2); only the bit-level recipe for multiplication changes.

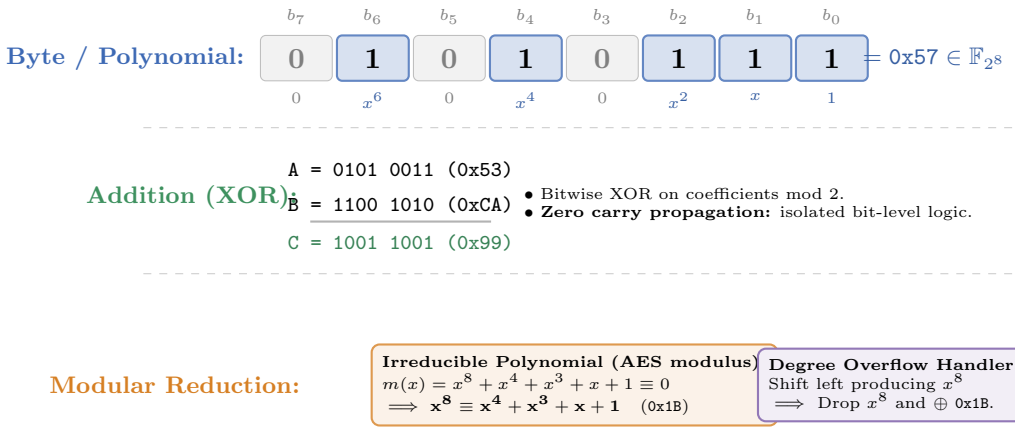


Figure 6.2: The structure of $\mathbb{F}_{2^8}$ (AES byte field). Bytes are interpreted directly as degree-7 polynomials over $\mathbb{F}_2$. Addition is purely carry-less bitwise XOR. Multiplication is polynomial multiplication reduced modulo the irreducible Rijndael polynomial $m(x) = x^8 + x^4 + x^3 + x + 1$, where any overflow degree x^8 is replaced by $x^4 + x^3 + x + 1$ (`0x1B`).

6.5.1 Addition is free

Adding two elements of $\mathbb{F}_{2^8}$ adds the polynomials coefficient-wise mod 2, which is bitwise XOR. There is no carry, no reduction needed (the degree never grows). So

$$\texttt{0x53} \oplus \texttt{0xCA} = \texttt{0x99}.$$

In hardware this is eight independent XOR gates. Subtraction is identical to addition, since $-1 = 1$.

In the real world

This is exactly why GF(2^n) is popular with hardware designers and why AES is fast. Addition is XOR (no carry propagation, the slow part of ordinary integer addition), and AES's `MixColumns` step is a small matrix multiply over $\mathbb{F}_{2^8}$ that becomes a handful of XORs and table lookups. The Galois-counter mode (GCM) used in TLS computes its authentication tag entirely with multiplication in $\mathbb{F}_{2^{128}}$ — same construction, a degree-128 irreducible instead of degree-8 — precisely because field multiplication of bit-strings is easy to pipeline in hardware (the `PCLMULQDQ` carry-less-multiply instruction exists for exactly this).

6.5.2 Multiplication, worked by hand

Multiplication is where the polynomial quotient structure comes into play. The recipe:

1. Multiply the two polynomials over $\mathbb{F}_2$ (a carry-less multiply — partial products combined with XOR, not addition).
2. If the result has degree ≥ 8, reduce it modulo $m(x)$ by polynomial long division, keeping the remainder.

Let us compute `0x57` $\cdot$ `0x83` in $\mathbb{F}_{2^8}$, the worked example from the original Rijndael documentation.

`0x57` $= x^6 + x^4 + x^2 + x + 1$ and `0x83` $= x^7 + x + 1$.

Step 1: carry-less polynomial product. Distribute every term of the first factor across x^7+x+1 and XOR like terms (any power that appears an even number of times cancels mod 2):

$$\begin{aligned}(x^6 + x^4 + x^2 + x + 1) \cdot x^7 &= x^{13} + x^{11} + x^9 + x^8 + x^7,\\ (x^6 + x^4 + x^2 + x + 1) \cdot x &= x^7 + x^5 + x^3 + x^2 + x,\\ (x^6 + x^4 + x^2 + x + 1) \cdot 1 &= x^6 + x^4 + x^2 + x + 1.\end{aligned}$$

XOR the three rows. The two x^7 terms cancel, the two x^2 terms cancel, and the two x terms cancel, leaving

$$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1.$$

This has degree 13, so we must reduce.

Step 2: reduce mod $m(x) = x^8 + x^4 + x^3 + x + 1$**.** The reduction rule comes from $m(x) = 0$, i.e.

$$x^8 = x^4 + x^3 + x + 1.$$

Knock down each high power using $x^k = x^{k-8}\,(x^4 + x^3 + x + 1)$:

$$\begin{aligned}x^{13} &= x^5(x^4{+}x^3{+}x{+}1) = x^9 + x^8 + x^6 + x^5,\\ x^{11} &= x^3(x^4{+}x^3{+}x{+}1) = x^7 + x^6 + x^4 + x^3,\\ x^9 &= x\,(x^4{+}x^3{+}x{+}1) = x^5 + x^4 + x^2 + x,\\ x^8 &= x^4 + x^3 + x + 1.\end{aligned}$$

Substitute these for the degree-≥ 8 terms in the dividend and XOR everything together (the new x^9 and x^8 produced on the right then get reduced in turn). Carrying the bookkeeping through, all powers above 7 cancel and the surviving remainder is

$$x^7 + x^6 + 1 \;=\; \texttt{0xC1}.$$

So in $\mathbb{F}_{2^8}$,

$$\texttt{0x57} \cdot \texttt{0x83} = \texttt{0xC1}.$$

Walking question

The cleanest way to do Step 2 on paper is schoolbook long division, except "subtract" is XOR. Write the degree-13 dividend. Its leading term is x^{13}; to cancel it, XOR in $x^5 \cdot m(x)$ (which starts x^{13}). That kills x^{13} and flips some lower bits. Whatever is now the leading term, cancel

it with the appropriate shift $x^j \cdot m(x)$. Repeat until the leading term has degree < 8; what remains is the answer. Each step is one XOR of a shifted copy of the 9-bit pattern `1 0001 1011` (= `0x11B`).

6.5.3 The doubling trick: `xtime`

Full multiplication is fiddly, but multiplication by the single element x (the byte `0x02`) is so simple it has a name: `xtime`. Multiplying a polynomial by x shifts every coefficient up by one degree — a left shift of the byte. If the top bit b_7 was 0, the result still fits in 8 bits and we are done. If b_7 was 1, the shift produces an x^8 term, which we immediately reduce using $x^8 = x^4 + x^3 + x + 1$, i.e. we XOR in `0x1B`.

$$\texttt{xtime}(b) = \begin{cases} b \ll 1, & \text{if top bit of } b \text{ is } 0, \\ (b \ll 1) \oplus \texttt{0x1B}, & \text{if top bit of } b \text{ is } 1. \end{cases}$$

Example 6.3 (`xtime` in action)**.** `xtime(0x57)`: `0x57` = 0101 0111, top bit 0, so just shift: 1010 1110 = `0xAE`.

`xtime(0xAE)`: `0xAE` = 1010 1110, top bit 1. Shift to 1 0101 1100, drop to a byte `0x5C`, then XOR `0x1B`: `0x5C` $\oplus$ `0x1B` = `0x47`. So `0x57` $\cdot$ `0x04` = `0x47`, since multiplying by `0x04` = x^2 is two doublings.

Why does this matter? Because *any* multiplication in $\mathbb{F}_{2^8}$ can be built from `xtime` and XOR. To multiply b by an arbitrary constant c, write c in binary; each set bit of c corresponds to a power x^k, and $b \cdot x^k$ is k repeated `xtime`s. XOR together the contributions for the set bits. This is "Russian-peasant" multiplication, carry-less. Figure 6.3 traces the doubling chain for b = `0x57`, the one the next example reuses.

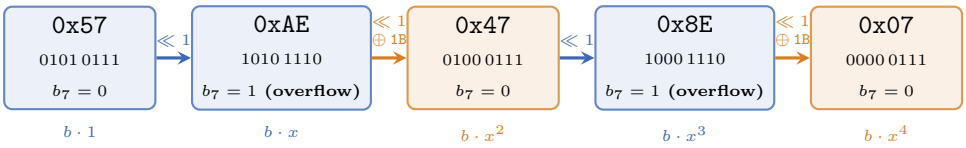


Figure 6.3: The `xtime` doubling chain for b = `0x57`. Each step shifts the byte left by one position. Whenever the high bit b_7 is 1 (overflowing into x^8), the operation reduces modulo the Rijndael polynomial by XORing with `0x1B`. Any field multiplication reduces to selecting and XORing appropriate terms from this chain.

Example 6.4 (`0x57` $\cdot$ `0x13` via `xtime`)**.** `0x13` = $x^4 + x + 1$, so we need $b \cdot x^4$, $b \cdot x$, and $b \cdot 1$ for b = `0x57`, then XOR them.

$$\begin{aligned} b \cdot 1 &= \texttt{0x57}, \\ b \cdot x &= \texttt{xtime(0x57)} = \texttt{0xAE}, \\ b \cdot x^2 &= \texttt{xtime(0xAE)} = \texttt{0x47}, \\ b \cdot x^3 &= \texttt{xtime(0x47)} = \texttt{0x8E}, \\ b \cdot x^4 &= \texttt{xtime(0x8E)} = \texttt{0x07}. \end{aligned}$$

Now XOR the pieces for the set bits of `0x13` (x^4, x^1, x^0):

$$\texttt{0x07} \oplus \texttt{0xAE} \oplus \texttt{0x57} = \texttt{0xFE}.$$

So `0x57` · `0x13` = `0xFE`. The small constants `0x01`, `0x02`, `0x03` are exactly the ones AES's `MixColumns` multiplies by, which is why the operation is so cheap.

Implement it

A complete $\mathbb{F}_{2^8}$ multiplier in a dozen lines. **For learning only** — this is a textbook reference, not a production routine. The branch on the high bit makes it variable-time, which leaks the operands through timing; real AES implementations use constant-time tables or hardware `AES-NI`/`PCLMULQDQ` instructions.

```
def gf_mul(a, b):
    """Multiply two bytes in GF(2^8) with the AES polynomial 0x11B."""
    p = 0
    for _ in range(8):
        if b & 1:           # current low bit of b set -> add a
            p ^= a          # addition is XOR
        b >>= 1
        hi = a & 0x80       # remember top bit before shifting
        a = (a << 1) & 0xFF
        if hi:              # overflow past x^7 -> reduce mod 0x11B
            a ^= 0x1B
    return p

def xtime(b):
    """Multiply by x (the element 0x02)."""
    b <<= 1
    if b & 0x100:
        b ^= 0x11B
    return b & 0xFF

assert gf_mul(0x57, 0x83) == 0xC1   # the worked example above
assert gf_mul(0x57, 0x13) == 0xFE
assert xtime(0x57) == 0xAE
```

The outer loop walks the bits of b; `xtime`'s reduction (XOR `0x1B`) happens inline each time `a` would overflow past degree 7. The asserts reproduce the hand computations, so you can check the paper work against the code.

6.5.4 Finding inverses in $\mathbb{F}_{2^8}$

AES's `SubBytes` step (the S-box) is built from the multiplicative inverse in $\mathbb{F}_{2^8}$: each nonzero byte is replaced by its inverse (and 0 maps to 0), followed by an affine bit transformation. So we need to invert field elements. Two routes, mirroring the $\mathbb{F}_p$ case:

- **Extended Euclid on polynomials.** Run the extended Euclidean algorithm on the byte's polynomial and $m(x)$; the cofactor is the inverse. This is the polynomial analogue of Theorem 6.1's proof.
- **Fermat-style exponentiation.** The nonzero elements form a group of order 255, so $a^{255} = 1$ for every nonzero a, hence $a^{-1} = a^{254}$. Compute a^{254} by repeated squaring and multiplying in $\mathbb{F}_{2^8}$.

Example 6.5 (An inverse)**.** In $\mathbb{F}_{2^8}$ with the AES polynomial, $\texttt{0x53}^{-1} = \texttt{0xCA}$. You can confirm it by multiplying: running the long-division recipe on $\texttt{0x53} \cdot \texttt{0xCA}$ collapses to the single byte 1. Try it once by hand and you will appreciate why implementations precompute the answer.

Common trap

Do not eyeball field inverses, and do not confuse a *factor* in a product with an *inverse*. The only reliable way to get a^{-1} in $\mathrm{GF}(2^8)$ is to run extended Euclid or compute a^{254} — or to look it up in the precomputed 256-byte inverse table that every real AES implementation stores. The S-box you see in the AES spec is precisely that table (inverse, then an affine map) baked in, so the field arithmetic happens once, offline, and the cipher itself just indexes.

Implement it

Compute the inverse via a^{254}. **Learning only.**

```python
def gf_inv(a):
    """Multiplicative inverse in GF(2^8): a^254, since |group| = 255."""
    if a == 0:
        return 0          # AES defines inv(0) = 0 by convention
    result, base, e = 1, a, 254
    while e:              # square-and-multiply for exponent 254
        if e & 1:
            result = gf_mul(result, base)
        base = gf_mul(base, base)
        e >>= 1
    return result

assert gf_inv(0x53) == 0xCA           # 0x53 * 0xCA == 1 in GF(2^8)
assert gf_mul(0x53, gf_inv(0x53)) == 1
```

The asserts pin down the actual inverse value, which is the only trustworthy way to know it.

6.6 The Multiplicative Group Is Cyclic

The nonzero elements of any finite field form a group under multiplication, written $\mathbb{F}_q^*$, of size $q-1$. The most important structural fact about finite fields for cryptography is the following.

Theorem 6.3 (The multiplicative group is cyclic)**.** *For any finite field $\mathbb{F}_q$, the group $\mathbb{F}_q^*$ is cyclic of order $q-1$. That is, there exists a* generator *(or* primitive element*) g such that every nonzero element is a power of g:*

$$\mathbb{F}_q^* = \{\, g^0, g^1, g^2, \ldots, g^{q-2} \,\}.$$

Intuition

"Cyclic" means one element, raised to successive powers, cycles through *all* the nonzero elements before returning to 1. It is the multiplicative analogue of a clock: a single generator's powers tick through every position. This is what makes discrete logarithms meaningful — if the group were a tangled product of small cycles, "the exponent that produces h" might not

be well defined or might be easy to find.

Example 6.6 (A generator of $\mathbb{F}_7^*$)**.** $\mathbb{F}_7^* = \{1, 2, 3, 4, 5, 6\}$, order 6. Try $g = 3$:

$$3^1 = 3,\ 3^2 = 2,\ 3^3 = 6,\ 3^4 = 4,\ 3^5 = 5,\ 3^6 = 1.$$

The powers of 3 run through all six nonzero elements, so 3 is a generator. By contrast 2 is not: $2^1 = 2, 2^2 = 4, 2^3 = 1$, then it repeats — 2 has order 3, only reaching half the group. The orders of elements always divide $q - 1 = 6$, here landing on $1, 2, 3, 6$.

Example 6.7 (A generator of $\mathbb{F}_8^*$)**.** Build $\mathbb{F}_8 = \mathbb{F}_2[x]/(x^3 + x + 1)$, so $x^3 = x + 1$. Is x a generator? Its order must divide 7, which is prime, so the order is 1 or 7; since $x \neq 1$, the order is 7 and x generates everything:

$$x^1 = x, \quad x^2, \quad x^3 = x + 1, \quad x^4 = x^2 + x, \quad x^5 = x^2 + x + 1,$$

$$x^6 = x^2 + 1, \quad x^7 = 1.$$

All seven nonzero elements appear exactly once. Whenever $q-1$ is prime (here 7), *every* non-identity element is automatically a generator. Figure 6.4 lays the seven powers out as a clock: stepping once around multiplies by x, and after seven steps you are back at 1.

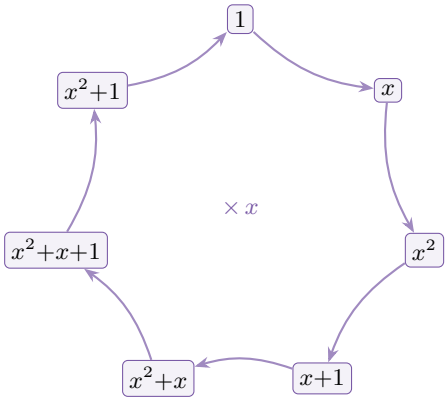


Figure 6.4: The cyclic group $\mathbb{F}_8^*$ generated by x. Each arrow is multiplication by the generator x; seven steps return to 1. Every nonzero element sits on the cycle exactly once, which is what "cyclic" means.

Why it works

Why is $\mathbb{F}_q^*$ always cyclic? The clean argument: in a field, the polynomial $t^d - 1$ has at most d roots (a degree-d polynomial over a field cannot have more roots than its degree — itself a consequence of the division algorithm). So for each divisor d of $q - 1$, at most d elements satisfy $t^d = 1$. A finite abelian group with this "at most d solutions to $t^d = 1$" property must be cyclic. The step that fails for non-fields like $\mathbb{Z}/8\mathbb{Z}$ is exactly the bound on roots: $t^2 - 1$ has four roots in $\mathbb{Z}/8\mathbb{Z}$ $(1, 3, 5, 7)$, more than its degree, which is why $(\mathbb{Z}/8\mathbb{Z})^*$ is not cyclic.

Connection

The order of an element a in $\mathbb{F}_q^*$ always divides $q-1$ (Lagrange's theorem from Chapter 5). In $\mathbb{F}_{2^8}$, $q-1 = 255 = 3 \cdot 5 \cdot 17$, so element orders are divisors of 255. The element $x =$ `0x02` in the AES field happens *not* to be a generator (its order is 51, not 255); a primitive element there is `0x03` $= x+1$, whose powers enumerate all 255 nonzero bytes. Chapters 18 and 20 work inside this cyclic structure, where the difficulty of *reversing* $g \mapsto g^k$ provides the security.

6.6.1 Why AES lives in $\mathbb{F}_{2^8}$, summarized

The AES designers wanted a byte-to-byte nonlinear map (the S-box) that resists linear and differential cryptanalysis, plus a column-mixing step that diffuses changes across bytes. A field gives them exactly the algebra to make these both *invertible* (decryption must undo them) and *nonlinear in the bit sense.*

- **Bytes are field elements.** $2^8 = 256$ matches the byte, so every byte is a field element and no value is wasted.
- **Addition is XOR.** Free in hardware, no carries.
- **The inverse map $a \mapsto a^{-1}$ is highly nonlinear** over $\mathbb{F}_2$, which is precisely what frustrates linear cryptanalysis — yet it is directly invertible, so decryption stays clean.
- `MixColumns` is multiplication by a fixed polynomial in $\mathbb{F}_{2^8}[y]/(y^4+1)$ with coefficients `0x01`, `0x02`, `0x03` — the cheap constants we saw — giving strong diffusion for almost no cost.

Attack

What if you "simplified" AES by replacing $\mathbb{F}_{2^8}$ multiplication with ordinary multiplication mod 256? Then $\mathbb{Z}/256\mathbb{Z}$ has zero-divisors ($2^7 \cdot 2 = 0$) and most elements have no inverse, so `MixColumns` would not be reversible — decryption breaks outright. Worse, the inverse map $a \mapsto a^{-1}$ would not exist for half the bytes, gutting the S-box. And were you to pick a *reducible* degree-8 polynomial instead of an irreducible one, the quotient $\mathbb{F}_2[x]/(m)$ would again have zero-divisors (the factors of m multiply to 0), and the same collapse follows. The irreducibility of $x^8 + x^4 + x^3 + x + 1$ is what guarantees every step of AES can be undone.

6.7 The Frobenius Map and Square Roots

One more structural fact will be useful later for square roots, point counting on elliptic curves, and certain attacks. Over a field of characteristic p, raising to the p-th power is a field homomorphism.

Theorem 6.4 (Freshman's dream / Frobenius)**.** *In a field of characteristic p, for all a, b,*

$$(a+b)^p = a^p + b^p.$$

The map $\phi(a) = a^p$ is a field automorphism of $\mathbb{F}_{p^n}$ (the Frobenius *map), fixing exactly the subfield $\mathbb{F}_p$.*

Why it works

Expand $(a+b)^p$ by the binomial theorem. Every middle coefficient $\binom{p}{k}$ for $0 < k < p$ is divisible by p (because p is prime and appears in the numerator $p!$ but not the denominator $k!(p-k)!$). In characteristic p, those terms vanish, leaving $a^p + b^p$. The "mistake" students make over $\mathbb{R}$ becomes a theorem over $\mathbb{F}_p$. In characteristic 2 it reads $(a+b)^2 = a^2 + b^2$ — squaring is XOR-linear, which is why squaring in $\mathbb{F}_{2^n}$ is just a bit-permutation (spread the bits out, then reduce) and cheaper than a general multiply.

Example 6.8 (Squaring is linear in $\mathbb{F}_8$)**.** Using $\mathbb{F}_8 = \mathbb{F}_2[x]/(x^3 + x + 1)$: $(x+1)^2 = x^2 + 1$ directly, with no cross term $2x$ (it vanishes). And $(x^2+x)^2 = x^4 + x^2$; reduce $x^4 = x \cdot x^3 = x(x+1) = x^2 + x$, giving $x^2 + x + x^2 = x$. The Frobenius map ϕ sends $x \mapsto x^2$, and applying it three times returns to the start (since $\mathbb{F}_8$ has $p^n = 2^3$ and $\phi^n = \mathrm{id}$).

Connection

In characteristic 2 the squaring map $a \mapsto a^2$ is a bijection, so every element has a *unique* square root, given by $\sqrt{a} = a^{2^{n-1}}$ in $\mathbb{F}_{2^n}$ (because $(a^{2^{n-1}})^2 = a^{2^n} = a$, using $a^{2^n} = a$ for all field elements). Square roots over $\mathbb{F}_p$ for odd p are harder — that is the Tonelli– Shanks algorithm, which the elliptic-curve and Rabin-cryptosystem chapters need to recover a y-coordinate from y^2.

6.8 Summary

- A **finite field** exists for exactly the prime-power sizes $q = p^n$, and is unique up to isomorphism (Theorem 6.2).
- $\mathbb{F}_p = \mathbb{Z}/p\mathbb{Z}$ is a field because every nonzero residue is coprime to p and so invertible (Theorem 6.1).
- $\mathbb{F}_{p^n}$ is built as $\mathbb{F}_p[x]/(m(x))$ for an **irreducible** polynomial m of degree n — the polynomial analogue of working modulo a prime. Irreducibility is what forces a field rather than a ring with zero-divisors.
- In $\mathbb{F}_{2^n}$, elements are bit-strings, addition is XOR, and multiplication is carry-less polynomial multiply followed by reduction mod m. **AES uses** $\mathbb{F}_{2^8}$ with $m(x) = x^8 + x^4 + x^3 + x + 1$; multiplication reduces to `xtime` (shift, conditional XOR `0x1B`) plus XOR.
- The **multiplicative group** $\mathbb{F}_q^*$ is cyclic of order $q-1$ (Theorem 6.3); its generators drive discrete-log cryptography, and the existence of inverses is what lets AES decrypt.
- The **Frobenius** map $a \mapsto a^p$ is an automorphism; squaring in characteristic 2 is linear, cheap, and gives unique square roots.

Every later building block — AES, GCM, Diffie–Hellman, elliptic curves, secret sharing — does its arithmetic in a finite field.

Before we can define modern security we need one more mathematical tool: probability. Chapter 7 covers probability and information theory (randomness, entropy, and the birthday bound), which we need for Shannon's definition of perfect secrecy and the symmetric ciphers of Part II.

Exercises

Exercise 6.1 (Inverses in $\mathbb{F}_{11}$)**.** Compute the multiplicative inverse of each of $2, 3, 5, 7$ in $\mathbb{F}_{11}$ using the extended Euclidean algorithm. Verify each by multiplying back to 1. Which element of $\mathbb{F}_{11}^*$, besides 1, is its own inverse?

Exercise 6.2 (No field of size 6)**.** Prove directly that there is no field with exactly 6 elements. (Hint: what must the characteristic be, and what does Theorem 6.2's "why p^n" argument force?)

Exercise 6.3 (Build $\mathbb{F}_9$)**.** The polynomial x^2+1 is irreducible over $\mathbb{F}_3$ (it has no root in $\{0,1,2\}$ — check). Construct $\mathbb{F}_9 = \mathbb{F}_3[x]/(x^2+1)$, list its 9 elements, and compute $(x+1)(x+2)$ and $(2x+1)^{-1}$ inside it.

Exercise 6.4 (Irreducible hunting)**.** Find all monic irreducible polynomials of degree 4 over $\mathbb{F}_2$. There are 16 monic degree-4 polynomials; remove the ones with a root in $\mathbb{F}_2$ and the one that is the square of the irreducible quadratic x^2+x+1. How many irreducibles remain, and which are they?

Exercise 6.5 (Multiply in the AES field)**.** Compute the following products in $\mathbb{F}_{2^8}$ with $m(x) = x^8 + x^4 + x^3 + x + 1$, showing the carry-less product and the reduction: (a) $\texttt{0x02} \cdot \texttt{0x87}$; (b) $\texttt{0x03} \cdot \texttt{0x6E}$; (c) $\texttt{0x57} \cdot \texttt{0x13}$ (you should get `0xFE`). Check each against the `gf_mul` routine.

Exercise 6.6 (`xtime` chain)**.** Starting from `0x01`, apply `xtime` repeatedly. How many steps until you return to `0x01`? What does that number tell you about the order of the element $x = \texttt{0x02}$ in $\mathbb{F}_{2^8}^*$, and is x a generator? (The chapter states its order; confirm it.)

Exercise 6.7 (Generator search)**.** In $\mathbb{F}_{2^8}$, verify that $\texttt{0x03} = x+1$ is a primitive element by checking that its order is 255 and not a proper divisor of 255. It suffices to check $\texttt{0x03}^{255/r} \neq 1$ for $r \in \{3, 5, 17\}$ — explain why those three checks are enough.

Exercise 6.8 (Break a broken cipher)**.** Suppose a student implements "AES" but replaces the irreducible $x^8+x^4+x^3+x+1$ with the *reducible* polynomial $x^8+1 = (x+1)^8$ over $\mathbb{F}_2$. Exhibit two nonzero bytes whose product is 0 in this broken "field," and explain in one sentence why `MixColumns` can no longer be inverted.

Exercise 6.9 (Frobenius cycles)**.** In $\mathbb{F}_{16} = \mathbb{F}_2[x]/(x^4+x+1)$, compute the Frobenius orbit of x under $\phi(a) = a^2$: that is, x, x^2, x^4, x^8, x^{16}. Confirm $x^{16} = x$ and that the orbit has length $4 = \deg m$. What do the orbit lengths tell you about the subfields of $\mathbb{F}_{16}$?

Exercise 6.10 (Square roots via Frobenius)**.** Using $\sqrt{a} = a^{2^{n-1}}$ in $\mathbb{F}_{2^n}$, find the square root of $\texttt{0x53}$ in the AES field $\mathbb{F}_{2^8}$ (so compute $\texttt{0x53}^{128}$), and verify by squaring your answer. Why does this exponent formula give the square root?

Exercise 6.11 (Counting irreducibles)**.** The number of monic irreducible polynomials of degree n over $\mathbb{F}_2$ is $\frac{1}{n}\sum_{d \mid n} \mu(n/d)\, 2^d$ (the necklace / Möbius formula). Use it to confirm there are 30 irreducibles of degree 8 over $\mathbb{F}_2$ — the pool the AES designers chose $m(x)$ from — and write down the count for degree 128 (the field GCM uses) as a clean expression.

Chapter 7

Probability and Information for Cryptography

The previous chapters built our algebraic foundations: groups, rings, and finite fields define the deterministic rules of modular arithmetic. But deterministic rules alone cannot keep a secret – secrecy is the art of making the right things unpredictable. A key an attacker cannot guess, a ciphertext that looks like noise, a nonce that never repeats: every one of these is a statement about probability.

This chapter builds the probability tools we lean on for the rest of the book: sample spaces, uniform keys, independence, *entropy* (average unpredictability), *min-entropy* (worst-case unpredictability), the birthday paradox, and *statistical distance*. None of this requires measure theory; everything lives on finite sets, where probability is counting done carefully.

7.1 Why probability is the native language of secrecy

Picture the standard cast. Alice wants to send Bob a message. Eve sits on the wire and sees everything that crosses it. The central question is: can Eve learn something she should not?

Eve almost never has zero information and almost never has perfect information. She has *partial* information, and partial information is exactly what probability measures. When we say "a 128-bit key is secure," we are not claiming Eve *cannot* guess it – she can, with probability 2^{-128} per guess. We are saying that probability is so small that even a planet-sized computer running for the age of the universe will not stumble onto it. Security is quantitative, and the quantity is a probability.

Intuition

A secret is only as good as it is unpredictable. "Unpredictable" can be measured with a number. A coin you have already seen land heads has zero unpredictability. A freshly flipped fair coin has exactly one bit of it. The entire job of a key-generation routine is to manufacture unpredictability and hand it to you in a usable form.

7.1.1 Sample spaces, events, and uniform distributions

A finite probability space is a finite set Ω (the *sample space*) of possible outcomes, together with a function Pr assigning each outcome $\omega \in \Omega$ a probability $\Pr[\omega] \geq 0$, such that $\sum_{\omega \in \Omega} \Pr[\omega] = 1$. An *event* $A \subseteq \Omega$ is any set of outcomes, and $\Pr[A] = \sum_{\omega \in A} \Pr[\omega]$.

The distribution we use more than any other is the *uniform* one. If $|\Omega| = N$, the uniform distribution puts $\Pr[\omega] = 1/N$ on every outcome. For an event A,

$$\Pr[A] = \frac{|A|}{|\Omega|} = \frac{\text{favorable outcomes}}{\text{total outcomes}}.$$

This is the "count and divide" rule you already know, and it is the basis of key generation: a uniformly random k-bit key is a uniform draw from $\Omega = \{0,1\}^k$, a set of size 2^k.

Example 7.1 (A four-bit key)**.** Let the key space be $\Omega = \{0,1\}^4$, so $N = 2^4 = 16$ keys, from 0000 to 1111. Under the uniform distribution each key has probability 1/16. Eve guesses one key. The event "Eve guesses right" has exactly one favorable outcome, so $\Pr[\text{correct}] = 1/16 = 0.0625$. The event "the key starts with a 1" contains the 8 keys $1000, \ldots, 1111$, so its probability is $8/16 = 1/2$. We will scale these toy numbers up to 128 and 256 bits later, but the arithmetic never changes – only the exponents do.

We write $k \xleftarrow{\$} \{0,1\}^\ell$ to mean "sample k uniformly at random from the set of ℓ-bit strings." That dollar-sign arrow is one of the most important symbols in applied cryptography. Almost every security proof in this book begins by assuming some value was drawn this way, and almost every real-world break begins with the discovery that it was *not*.

7.1.2 Conditional probability and Bayes

Eve's knowledge changes as she observes ciphertexts. The tool for "probability of A given that I now know B" is conditional probability:

$$\Pr[A \mid B] = \frac{\Pr[A \cap B]}{\Pr[B]}, \qquad \text{defined when } \Pr[B] > 0.$$

Read it as: restrict the universe to the world where B happened, then ask what fraction of that smaller world also has A. Rearranging gives the multiplication rule $\Pr[A \cap B] = \Pr[A \mid B]\,\Pr[B]$, and swapping the roles of A and B gives *Bayes' rule*:

$$\Pr[A \mid B] = \frac{\Pr[B \mid A]\,\Pr[A]}{\Pr[B]}.$$

Bayes is how an attacker updates a guess about a key in light of leaked data, and it also gives the formal meaning of "a ciphertext reveals nothing": the message should be just as likely after seeing the ciphertext as before. We make that precise when we reach perfect secrecy in Chapter 9.

Example 7.2 (A side channel updates a guess)**.** Return to the four-bit key of Example 7.1, uniform over 16 values. A noisy power-analysis probe leaks one bit about the key's Hamming weight (the number of 1s). The probe reports "the weight is even" and is correct with probability 0.8, wrong with probability 0.2. Of the 16 keys, exactly 8 have even weight $(0000, 0011, \ldots, 1111)$ and 8 have odd weight, so before the leak each class has prior probability 1/2.

Let A be the event "the true key has even weight" and B the event "the probe says even." We want $\Pr[A \mid B]$. The likelihoods are $\Pr[B \mid A] = 0.8$ (probe right on an even key) and $\Pr[B \mid \bar{A}] = 0.2$ (probe wrong on an odd key). The denominator is $\Pr[B] = 0.8 \cdot \frac{1}{2} + 0.2 \cdot \frac{1}{2} = 0.5$. Bayes gives

$$\Pr[A \mid B] = \frac{0.8 \cdot 0.5}{0.5} = 0.8.$$

The leak moved Eve from 1/2 to 0.8 confidence that the weight is even. Conditioned on that, her best guess targets one of the 8 even-weight keys, and a single guess there is correct with probability $0.8 \cdot \frac{1}{8} = 0.1$ – up from the $1/16 = 0.0625$ she had with no leak (Figure 7.1). One noisy bit nearly doubled her one-shot odds. Side-channel attacks work this way: each leaked measurement is a Bayesian update, and the updates accumulate.

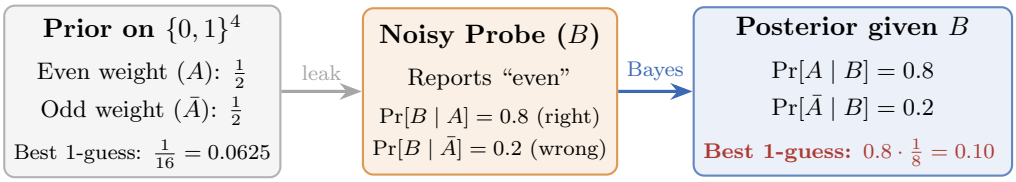


Figure 7.1: Bayesian update from a single noisy side-channel measurement (Example 7.2). Even with a 20% error rate, reporting "even weight" concentrates 80% of the posterior mass onto the 8 even-weight keys, raising Eve's single-guess success probability from 0.0625 to 0.10.

7.1.3 Independence

Two events A and B are *independent* if

$$\Pr[A \cap B] = \Pr[A] \cdot \Pr[B],$$

equivalently $\Pr[A \mid B] = \Pr[A]$: learning B tells you nothing about A. Independence is what we demand of the bits coming out of a good random source – each new bit should be a fresh fair coin, unaffected by all the bits before it. When that assumption fails, keys become guessable.

Attack

[When "random" bits are not independent] In 2012 two research teams scanned millions of RSA public keys collected from the public Internet and computed pairwise GCDs. Roughly 0.2% of TLS and SSH keys shared a prime factor with some other key, which let anyone factor both and recover the private keys outright. The cause was a broken independence assumption: embedded devices (routers, firewalls) generated key material moments after boot, before their entropy pools had collected enough unpredictable events. Different devices ended up drawing from nearly the same impoverished state, so their "random" primes were correlated rather than independent. The mathematics was sound, but the probability model the implementers assumed – fresh independent randomness – did not hold in the field. We revisit entropy starvation in Chapter 17.

A handy consequence of independence: if you draw $k_1, k_2, \ldots, k_n$ as independent uniform bits, then the whole string $k_1 k_2 \cdots k_n$ is uniform over $\{0,1\}^n$. Independence is precisely what turns "n good coins" into "one uniform n-bit key." Lose independence and you lose the size of your key space, even if every individual bit still looks balanced.

7.2 Random variables and expectation

A *random variable* X is a function from the sample space to numbers, $X : \Omega \to \mathbb{R}$. If you roll a die, "the number showing" is a random variable; "1 if the roll is even, else 0" is another. The *expectation* (mean) is the probability-weighted average of its values:

$$\mathbb{E}[X] = \sum_{\omega \in \Omega} \Pr[\omega]\, X(\omega) = \sum_{x} x \cdot \Pr[X = x].$$

Expectation is *linear*: $\mathbb{E}[X + Y] = \mathbb{E}[X] + \mathbb{E}[Y]$ always, with no independence required. That one fact makes a surprising number of counting arguments trivial, and we use it below to count expected collisions.

Example 7.3 (Expected number of guesses)**.** Eve attacks the 4-bit key of Example 7.1 by trying keys one at a time in a random order until she hits the right one. Let X be the number of guesses she needs. Each position $1, 2, \ldots, 16$ is equally likely to hold the true key, so $\Pr[X = i] = 1/16$ for each i, and

$$\mathbb{E}[X] = \sum_{i=1}^{16} i \cdot \tfrac{1}{16} = \tfrac{1}{16} \cdot \tfrac{16 \cdot 17}{2} = \tfrac{17}{2} = 8.5.$$

On average she needs about half the key space, which is the familiar "brute force costs $2^{\ell-1}$ on average" rule. For a 128-bit key that is 2^{127} expected guesses, which is why brute force is hopeless.

7.2.1 The indicator trick

A random variable that is 1 when some event happens and 0 otherwise is called an *indicator*, written $\mathbf{1}_A$. Its expectation is just the probability of the event: $\mathbb{E}[\mathbf{1}_A] = 1 \cdot \Pr[A] + 0 \cdot \Pr[\bar{A}] = \Pr[A]$. Combine that with linearity of expectation and you can count the expected number of times *any* of a pile of events fires, even when the events overlap or are correlated. We use this argument for the birthday bound below.

7.3 Entropy: measuring unpredictability on average

We need a single number that says "how hard is this secret to guess." Claude Shannon gave us one in 1948, and it remains the right tool for reasoning about average-case unpredictability.

History

Claude Shannon at Bell Labs founded information theory in 1948 with "A Mathematical Theory of Communication" and modern cryptography a year later with "Communication Theory of Secrecy Systems." Using entropy to measure average uncertainty, he proved that the one-time pad achieves perfect secrecy against an eavesdropper with unlimited computing time, and proved a sobering converse: any cipher that is unconditionally unbreakable needs a key at least as long as the message. He borrowed the word "entropy" from thermodynamics on the advice of John von Neumann, who reportedly told him it would win every argument because nobody really knows what entropy is. What Shannon left open was whether short, reusable keys could still protect messages against an adversary restricted to realistic, polynomial-time

computation.

7.3.1 The definition and where it comes from

Let X be a random variable taking value x with probability $p_x = \Pr[X = x]$. The *Shannon entropy* of X, measured in bits, is

$$H(X) = -\sum_x p_x \log_2 p_x = \sum_x p_x \log_2 \frac{1}{p_x},$$

with the convention $0 \log_2 0 = 0$ (an impossible outcome contributes nothing). The quantity $\log_2(1/p_x)$ is the *surprise* of outcome x: rare outcomes (p_x small) are very surprising (large $\log_2(1/p_x)$), certain outcomes ($p_x = 1$) carry zero surprise. Entropy is the *average surprise* you experience when you learn the value of X.

Intuition

Think of entropy as the average number of yes/no questions an optimal interrogator needs to pin down X. If X is one of 8 equally likely values, a smart questioner splits the candidates in half each time – "is it in the top half?" – and nails it in 3 questions: $H(X) = \log_2 8 = 3$ bits. If one value is overwhelmingly likely, the questioner usually guesses it first and is done in roughly one question, so entropy is low. Each yes/no question corresponds to one bit.

7.3.2 Worked entropies on tiny distributions

Example 7.4 (Fair versus biased coin)**.** A fair coin has $p_{\mathrm{H}} = p_{\mathrm{T}} = 1/2$:

$$H = -\tfrac{1}{2}\log_2 \tfrac{1}{2} - \tfrac{1}{2}\log_2 \tfrac{1}{2} = \tfrac{1}{2} + \tfrac{1}{2} = 1 \text{ bit}.$$

Exactly one bit, as it must be. Now bias the coin so heads comes up with probability 0.9:

$$H = -0.9 \log_2 0.9 - 0.1 \log_2 0.1 \approx 0.9(0.152) + 0.1(3.32) \approx 0.137 + 0.332 = 0.469 \text{ bits}.$$

The biased coin still produces "a bit" of data per flip, but it carries less than half a bit of genuine unpredictability. If you built a key out of such coins, an attacker who knows the bias would guess each bit right 90% of the time. Many weak "random" tokens have failed because of this gap between raw bit-length and real entropy.

Example 7.5 (A loaded four-sided die)**.** Let X take four values with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Then

$$H(X) = \tfrac{1}{2}\log_2 2 + \tfrac{1}{4}\log_2 4 + \tfrac{1}{8}\log_2 8 + \tfrac{1}{8}\log_2 8 = \tfrac{1}{2}(1) + \tfrac{1}{4}(2) + \tfrac{1}{8}(3) + \tfrac{1}{8}(3) = \tfrac{7}{4} = 1.75 \text{ bits}.$$

A uniform four-sided die would have entropy $\log_2 4 = 2$ bits, so the loading costs us a quarter-bit. The most likely outcome ($p = 1/2$) contributes its small surprise of 1 bit, while the rare outcomes contribute more surprise each but are down-weighted by how seldom they occur.

7.3.3 The properties that matter for crypto

Three facts about Shannon entropy are used throughout the book.

Theorem 7.1 (Entropy is maximized by the uniform distribution)**.** *For any random variable X over a set of size N, $0 \le H(X) \le \log_2 N$. The lower bound is met exactly when X is constant (no uncertainty), and the upper bound is met exactly when X is uniform.*

Why it works

Why the uniform distribution maximizes entropy. Suppose two outcomes had unequal probabilities, say p and q with $p > q$. Nudge a sliver of probability from the likelier one to the rarer one, making them more equal. A short calculus check on $-p\log_2 p - q\log_2 q$ shows this *increases* the total – surprise grows when you flatten the peaks. The only configuration with no peak left to flatten is the flat one, where every outcome has probability $1/N$. So among all distributions on N values, uniform is the most uncertain, and its entropy is $\log_2 N$. This is exactly why we want keys uniform: any deviation strictly lowers entropy and hands the attacker an edge, however small.

The second fact: entropy of independent sources *adds*. If X and Y are independent, $H(X,Y) = H(X) + H(Y)$. Sixteen independent fair coins give 16 bits of entropy, which is why "ℓ independent uniform bits" equals "ℓ bits of entropy." The third fact, the data-processing inequality: no deterministic function can manufacture entropy, $H(f(X)) \le H(X)$. You cannot post-process your way to more unpredictability than you started with – a hash of a low-entropy password is still low-entropy, which is exactly what password crackers exploit.

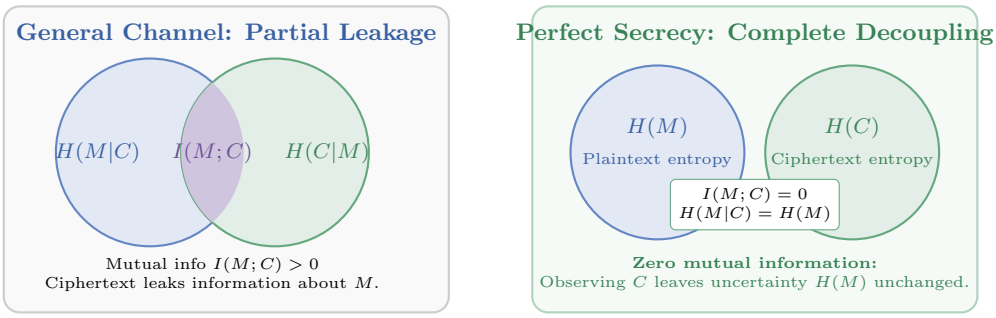


Figure 7.2: Information-theoretic view of cryptographic secrecy. Left: In a leaky or classical cipher, observing ciphertext C reduces uncertainty about message M by the mutual information $I(M;C) > 0$. Right: Under Shannon's condition of perfect secrecy, M and C are statistically independent ($I(M;C) = 0$), so $H(M|C) = H(M)$ and seeing the ciphertext provides zero information about the plaintext.

Connection

Entropy, expectation, and the logarithm all live in the companion probability volume. The $\log_2(1/p)$ "surprise" quantity is the information content of an event; averaging it against the distribution is just an expectation, $H(X) = \mathbb{E}[\log_2(1/p_X)]$. If the manipulations in the worked examples felt fast, the chapters on discrete random variables and expectation there walk the same algebra at a slower pace, including the calculus argument behind Theorem 7.1.

7.4 Min-entropy: the paranoid measure

Shannon entropy is an *average*, and an average can mislead in exactly the situation cryptographers care about: the attacker who makes the single best guess. Suppose a key is 0 with probability $1/2$ and otherwise uniform over the remaining $2^{128}-1$ values. Its Shannon entropy is enormous, 65 bits (the $1/2$ spike contributes a half-bit, and the flat tail contributes $\frac{1}{2}\log_2\big(2(2^{128}-1)\big) \approx 64.5$). But an attacker who simply guesses "0" wins half the time on the first try. By Shannon entropy the key looks strong, yet in practice it is very weak. We need a measure that reports the worst case, not the average.

Definition 7.1 (Min-entropy)**.** The *min-entropy* of X is

$$H_\infty(X) = -\log_2\Big(\max_x \Pr[X=x]\Big) = \min_x \log_2 \frac{1}{p_x}.$$

In words: find the single most likely outcome, call its probability $p_{\max}$, and report $-\log_2 p_{\max}$. Because the best single guess succeeds with probability exactly $p_{\max} = 2^{-H_\infty(X)}$, min-entropy directly bounds the attacker's one-shot success:

$$\Pr[\text{best single guess is correct}] = 2^{-H_\infty(X)}.$$

That clean operational meaning is why min-entropy, not Shannon entropy, is the quantity randomness extractors and key-derivation standards are written against.

Example 7.6 (The two measures disagree)**.** Take the loaded die of Example 7.5 with probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Its Shannon entropy was 1.75 bits. Its min-entropy uses only the peak, $p_{\max} = 1/2$:

$$H_\infty(X) = -\log_2 \tfrac{1}{2} = 1 \text{ bit}.$$

That is three-quarters of a bit lower. The most likely outcome happens half the time, so a one-guess attacker wins 50% of the time, exactly as 2^{-1} predicts. For a uniform distribution the two measures coincide – when every $p_x = 1/N$, both equal $\log_2 N$ – which is one more reason to use uniform keys. The measures only diverge when the distribution has a peak, and the peak is precisely what an attacker aims at (Figure 7.3).

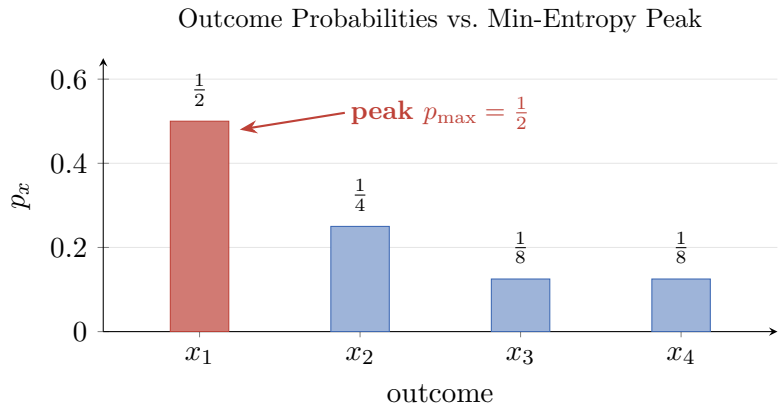


Figure 7.3: The loaded die of Example 7.6. Shannon entropy (1.75 bits) averages the surprise across all four bars; min-entropy (1 bit) looks only at the tallest bar, the one an attacker guesses first.

Formal

In general $0 \le H_\infty(X) \le H(X) \le \log_2 N$. Min-entropy is always the most pessimistic of the entropy family (it is the limit of the Renyi entropies H_α as $\alpha \to \infty$, hence the ∞ subscript). When someone in cryptography says "this source has k bits of entropy" and means it as a security claim, they should mean min-entropy. If a standard says a seed needs "256 bits of entropy," read it as min-entropy unless told otherwise – that is the reading that makes the security argument go through.

In the real world

NIST Special Publication 800-90B, the U.S. standard for entropy sources feeding random-number generators, is built entirely around min-entropy. A hardware noise source must be *estimated* for its min-entropy, and the estimate must survive a battery of statistical tests designed to catch the source overstating itself. Then a *conditioning* step (often based on a hash or block cipher) compresses many weak samples into fewer, fuller-entropy bits. The reason the standard fixates on min-entropy rather than Shannon entropy is the one-shot-guess bound above: a generator that is usually unpredictable but occasionally spits out a likely value is exactly the failure mode that causes trouble in deployment, and only min-entropy captures it.

Common trap

Having many bytes does not mean having much entropy. A megabyte of the digits of π has eight million bits of length and essentially zero entropy – it is fully determined. Length is what you store; entropy is what the attacker has to guess. Always ask of a "random" value: what distribution did it come from, and what is the *largest* probability any single value gets? That second question is the min-entropy question, and its answer determines whether a key is safe.

7.4.1 Guessing entropy: how many tries on average

Min-entropy answers "how often does the *first* guess win?" Sometimes you want the dual question: "if the attacker guesses values in the smartest order, how many guesses until success on average?" That number is the *guessing entropy* (or guesswork), and it is simply the expected value of the guess index.

Order the outcomes from most to least likely, $p_1 \ge p_2 \ge \cdots \ge p_M$. An optimal attacker tries them in exactly that order, so the value with probability p_i is found on guess number i. The expected number of guesses is

$$G(X) = \sum_{i=1}^{M} i \cdot p_i.$$

Example 7.7 (Three ways to score the loaded die)**.** Take yet again the die with sorted probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{8}, \frac{1}{8}$. Its guessing entropy is

$$G(X) = 1 \cdot \tfrac{1}{2} + 2 \cdot \tfrac{1}{4} + 3 \cdot \tfrac{1}{8} + 4 \cdot \tfrac{1}{8} = \tfrac{1}{2} + \tfrac{1}{2} + \tfrac{3}{8} + \tfrac{1}{2} = \tfrac{15}{8} = 1.875.$$

A uniform draw over 4 values would need $G = (1+2+3+4)/4 = 2.5$ guesses on average. So we now have three numbers describing the same distribution: $H = 1.75$ bits (average surprise), $H_\infty = 1$ bit

(worst-case one-shot), and $G = 1.875$ guesses (average optimal guesswork). They measure genuinely different things, and a security claim has to name which one it means.

Why it works

Why guesswork is not just 2^H. A tempting shortcut says "ℓ bits of entropy means about 2^ℓ guesses." That is right for the uniform distribution and badly wrong for a peaked one. Massey proved a lower bound $G(X) \geq \frac{1}{4} \cdot 2^{H(X)} + 1$ when $H(X) \geq 2$, but no matching upper bound exists: you can build distributions with large Shannon entropy whose *first* guess still wins almost always, dragging the average guesswork far below 2^H. The biased-key Example 7.6 pattern – one fat spike on a sea of tiny probabilities – is exactly such a case. When you care about the attacker who batches many guesses (an offline password cracker), guesswork is the right scoreboard; when you care about the single best shot (a one-time token), min-entropy is. Reach for the measure whose operational meaning matches the threat.

7.5 The birthday paradox and collision security

In a room of just 23 people, the chance that two of them share a birthday is already better than even. Most people guess you would need closer to 183 (half of 365). The intuition fails because we instinctively count the wrong thing – people instead of *pairs*. This "paradox" is the most important counting argument in symmetric cryptography, because it fixes the security level of every hash function and every randomly chosen nonce.

7.5.1 Why 23 is enough

The trick is to compute the probability of *no* shared birthday and subtract from one. With n people and $N = 365$ equally likely birthdays, line them up and place them one at a time. The first person can have any birthday. The second must dodge the first: probability $\frac{364}{365}$. The third must dodge two: $\frac{363}{365}$. In general the k-th must dodge $k-1$ taken days. The probability all n are distinct is the product

$$\Pr[\text{all distinct}] = \prod_{k=1}^{n-1} \left(1 - \frac{k}{N}\right),$$

so $\Pr[\text{some collision}] = 1 - \prod_{k=1}^{n-1} \left(1 - \frac{k}{N}\right)$. Plugging in $N = 365$:

people n	Pr[some shared birthday]
10	0.117
20	0.411
23	0.507
30	0.706
50	0.970
70	0.999

At $n = 23$ we cross one-half. The reason is the number of *pairs*, not people: with 23 people there

are $\binom{23}{2} = 253$ pairs, each a chance for a match, and 253 is already comfortably more than half of 365. We count pairs, and pairs grow like n^2.

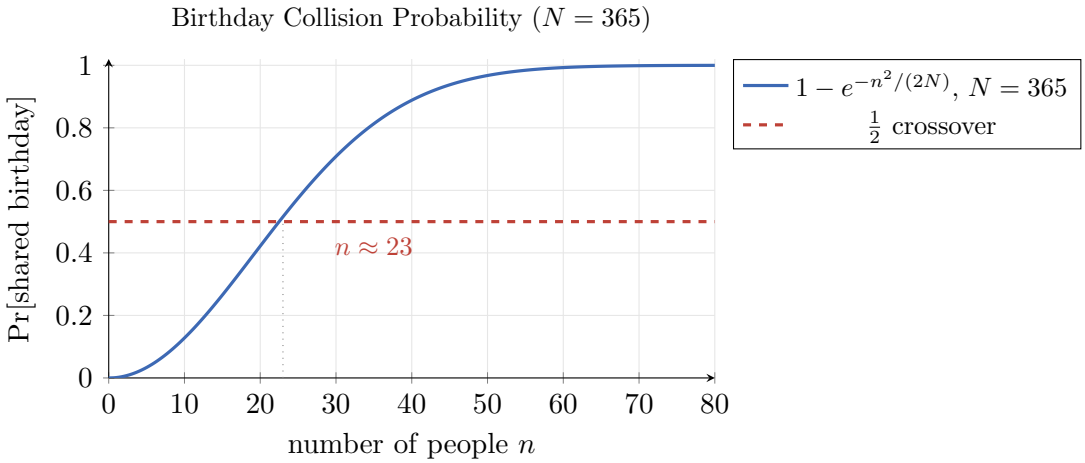


Figure 7.4: The collision probability climbs steeply and crosses $\frac{1}{2}$ near $n = 23$. The curve is the $1 - e^{-n^2/(2N)}$ approximation; the steep rise is the n^2 growth of the pair count at work.

7.5.2 The clean bound

To see the n^2 scaling, use the inequality $1 - x \le e^{-x}$ (the exponential always sits above its tangent line at zero). Then

$$\Pr[\text{all distinct}] = \prod_{k=1}^{n-1}\Big(1 - \frac{k}{N}\Big) \le \prod_{k=1}^{n-1} e^{-k/N} = \exp\Big(-\frac{1}{N}\sum_{k=1}^{n-1} k\Big) = \exp\Big(-\frac{n(n-1)}{2N}\Big).$$

Therefore the collision probability is at least

$$\Pr[\text{collision}] \;\ge\; 1 - \exp\Big(-\frac{n(n-1)}{2N}\Big) \;\approx\; 1 - \exp\Big(-\frac{n^2}{2N}\Big).$$

To find the fifty-fifty crossover, set the collision probability $1 - \exp(-n^2/(2N))$ to $\frac{1}{2}$. This means $\exp(-n^2/(2N)) = \frac{1}{2}$, so $n^2/(2N) = \ln 2 \approx 0.693$. Solving for n gives

$$\boxed{n \approx \sqrt{2 \ln 2 \cdot N} \approx 1.177\sqrt{N}.}$$

Setting the exponent to 1 gives $n \approx \sqrt{2N} \approx 1.414\sqrt{N}$, by which point the collision probability is already $1 - 1/e \approx 0.632$. For birthdays, $1.177\sqrt{365} \approx 22.5$ – matching the true discrete crossover at 23 (Figure 7.4). The key point is the square root: you need on the order of $\sqrt{N}$ samples, not N, before a collision is likely.

Why it works

The square root is what matters for cryptography. If outputs are drawn uniformly from a space of size $N = 2^n$ (an n-bit hash), then collisions become likely after roughly $\sqrt{2^n} = 2^{n/2}$ samples. The n-bit space gives you only $n/2$ bits of *collision* resistance. No cleverer hash

design removes this weakness, because it follows from counting pairs and applies to every function with a fixed-size output. It is the reason hash output lengths are chosen to be *twice* the desired security level.

7.5.3 Counting collisions with the indicator trick

There is a second, even quicker way to land on $\sqrt{N}$ that shows off the indicator trick from earlier. Throw n samples into N uniform bins. For each unordered pair $\{i, j\}$ of samples, let $\mathbf{1}_{ij}$ be the indicator that samples i and j collide. Two uniform draws land in the same bin with probability $1/N$, so $\mathbb{E}[\mathbf{1}_{ij}] = 1/N$. By linearity of expectation – no independence needed, even though the pair-events overlap – the expected number of colliding pairs is

$$\mathbb{E}[\text{collisions}] = \sum_{\{i,j\}} \mathbb{E}[\mathbf{1}_{ij}] = \binom{n}{2} \cdot \frac{1}{N} = \frac{n(n-1)}{2N}.$$

This expected count passes 1 exactly when $n(n-1) \approx 2N$, i.e. $n \approx \sqrt{2N}$ again. Same threshold, derived in two lines. When the expected number of collisions crosses one, collisions stop being rare.

7.5.4 What this dictates for hash functions

A cryptographic hash $\mathrm{H} : \{0,1\}^* \to \{0,1\}^n$ should be *collision resistant*: nobody should find $x \neq y$ with $\mathrm{H}(x) = \mathrm{H}(y)$. The birthday bound says a generic attacker – one who just hashes random inputs and waits for a repeat, knowing nothing about the hash's internals – finds a collision in about $2^{n/2}$ work. So:

- A 128-bit hash gives only $\approx 2^{64}$ collision resistance. 2^{64} operations is large but has been within reach of well-funded effort for years. Treat 128-bit digests as obsolete for collision resistance.
- SHA-256 outputs 256 bits, for $\approx 2^{128}$ collision resistance – the modern comfortable target.
- If you need 256-bit security against collisions (e.g. long-term or post-quantum margins), reach for a 512-bit digest like SHA-512.

Note the asymmetry that trips people up. *Preimage* resistance (given a digest, find any input mapping to it) costs $\approx 2^n$ – the full space, because you are aiming at one fixed target, not waiting for any pair to coincide. *Collision* resistance costs only $\approx 2^{n/2}$. The birthday bound bites collisions, not preimages, which is why a hash is always weaker against collisions than its length naively suggests.

Attack

[Birthday attacks that actually happened] The birthday bound matters in practice. MD5 produces a 128-bit digest, so its generic collision resistance is only 2^{64} – and structural weaknesses dropped real attacks far below even that. In 2008 researchers used MD5 collisions to forge a rogue certificate-authority certificate, letting them mint trusted certificates for any website. The 2012 Flame espionage malware used an MD5 collision to forge a Microsoft code-signing certificate and spread via Windows Update. SHA-1, with a 160-bit digest (2^{80} generic), fell in 2017 to the *SHAttered* attack, which produced two different PDF files with the same SHA-1 hash; in 2020 a *chosen-prefix* SHA-1 collision broke real PGP trust. In every case the under-

lying clock was the birthday bound: a digest only half as strong as its length suggests, eroded further by cryptanalysis.

Connection

The companion probability book derives the $\sqrt{N}$ scaling from first principles and proves the $1 - e^{-x}$ bound used here, and the algorithms book pairs it with the *rho* method (Pollard's cycle-finding) that turns the birthday bound into a memory-cheap $O(2^{n/2})$ collision-finding algorithm – the practical method behind the attacks above. If you want the variance of the collision count and tail bounds (how concentrated the "about $2^{n/2}$" really is), that is the second-moment material there.

Implement it

A tiny birthday-collision experiment. We hash random inputs with a deliberately short 24-bit digest and watch how many tries it takes to collide, then compare with the prediction $\approx 2^{n/2} = 2^{12} = 4096$. This is illustration code for learning, never for production – truncating SHA-256 to 24 bits is exactly the kind of thing you must never ship.

```
import hashlib, os

def short_hash(data, nbits=24):
    full = hashlib.sha256(data).digest()
    val = int.from_bytes(full, "big")
    return val & ((1 << nbits) - 1)      # keep only the low nbits

def find_collision(nbits=24):
    seen = {}                            # digest -> input that produced it
    tries = 0
    while True:
        x = os.urandom(16)
        h = short_hash(x, nbits)
        tries += 1
        if h in seen and seen[h] != x:
            return tries, seen[h], x      # collision found
        seen[h] = x

n = 24
tries, a, b = find_collision(n)
print(f"collided after {tries} hashes; birthday estimate ~ {2**(n//2)}")
# Typical run: a few thousand hashes, right around 2**12 = 4096.
```

Run it a dozen times and average: the collision count clusters near 2^{12}, not near 2^{24}. The experiment shows the square-root law directly.

In the real world

Birthday math sets concrete limits in protocols you use daily. AES in GCM mode uses a 96-bit nonce; a *random* nonce risks repetition by the birthday bound after about 2^{48} messages under one key, and a repeated GCM nonce is catastrophic – it leaks the authentication key. That

is why TLS 1.3 derives GCM nonces from a counter rather than gambling on randomness. The older 64-bit block ciphers (DES, 3DES, Blowfish) suffer the *Sweet32* attack: in CBC mode, ciphertext blocks start colliding after $\approx 2^{32}$ blocks (≈ 32 GB), and each collision leaks a relationship between plaintexts. A long-lived HTTPS or VPN session over 3DES could be partially decrypted purely from birthday collisions, which is why 64-bit block ciphers were retired from TLS.

7.6 Statistical distance and indistinguishability

The final tool is the formal way to say two distributions "look the same." Nearly every security definition in modern cryptography is built on it: a scheme is secure when the distribution an attacker sees in the real world is indistinguishable from some ideal, secret-free distribution.

7.6.1 Definition and the guessing-game meaning

Definition 7.2 (Statistical distance)**.** Let P and Q be distributions over the same finite set Ω. Their *statistical distance* (also called total variation distance) is

$$\Delta(P, Q) = \frac{1}{2} \sum_{\omega \in \Omega} |P(\omega) - Q(\omega)| = \max_{A \subseteq \Omega} \big(P(A) - Q(A)\big).$$

The two forms are equal, and each has its own interpretation. The sum form adds up, over every outcome, how much the two distributions disagree, halved so the total never exceeds 1. The max form is the operational one: it says $\Delta(P, Q)$ is the *best possible advantage* any test can achieve at telling the two distributions apart. $\Delta = 0$ means the distributions are identical and no test does better than a coin flip; $\Delta = 1$ means their supports are disjoint and one sample reveals which is which.

Intuition

Imagine a game. A referee secretly flips a fair coin: heads, she hands you a sample from P; tails, a sample from Q. You inspect the sample and shout your guess. If P and Q are identical you are reduced to guessing, winning half the time. The more they differ, the better you can do – and the very best win probability you can ever achieve is exactly $\frac{1}{2} + \frac{1}{2}\Delta(P, Q)$. Statistical distance is your *edge over random guessing*, doubled. That is what "0.01 statistical distance" buys an attacker: a 1% edge, and no more.

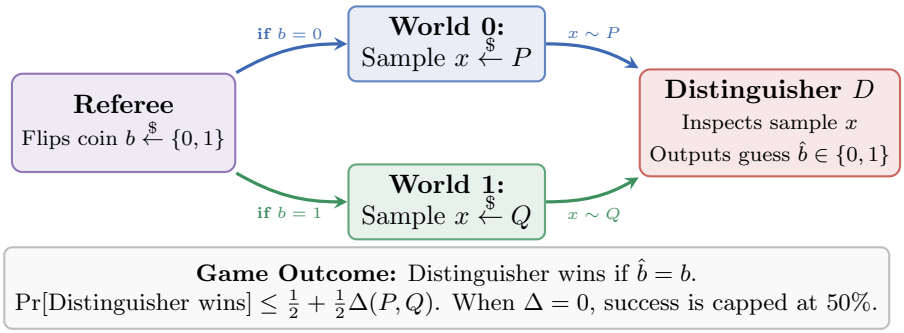


Figure 7.5: The statistical distinguishing game. A referee flips a fair bit b and provides a sample from either distribution P or Q. The distinguisher inspects the sample and guesses which world produced it. The maximum possible advantage over a blind guess is precisely half the statistical distance $\Delta(P, Q)$.

7.6.2 A worked distance

Example 7.8 (Two coins)**.** Let P be a fair coin, $P(\mathrm{H}) = P(\mathrm{T}) = \frac{1}{2}$, and Q a biased coin, $Q(\mathrm{H}) = \frac{3}{4}, Q(\mathrm{T}) = \frac{1}{4}$. Then

$$\Delta(P,Q) = \tfrac{1}{2}\Big(|\tfrac{1}{2} - \tfrac{3}{4}| + |\tfrac{1}{2} - \tfrac{1}{4}|\Big) = \tfrac{1}{2}\Big(\tfrac{1}{4} + \tfrac{1}{4}\Big) = \tfrac{1}{4}.$$

Check it against the max form: the best distinguisher guesses "Q" whenever it sees heads. It is right with probability $Q(\mathrm{H}) = \frac{3}{4}$ when the sample truly came from Q, and wrong (calls Q on a head) with probability $P(\mathrm{H}) = \frac{1}{2}$ when it came from P. The advantage is $\frac{3}{4} - \frac{1}{2} = \frac{1}{4}$, matching Δ. Best win probability $\frac{1}{2} + \frac{1}{2} \cdot \frac{1}{4} = \frac{5}{8} = 0.625$ in the referee game of Figure 7.5. So one look at a single flip lets you beat the coin flip by 12.5 percentage points – meaningful, because these distributions are crude. Cryptographic distributions are tuned so Δ is negligibly small.

Common trap

"Small distance per sample" does not mean "small distance forever." Statistical distance is not free under repetition. If a single sample of P sits at distance δ from a single sample of Q, then m independent samples sit at distance *at most* $m\delta$ – and often close to it. A pseudorandom generator whose one-block output is distance 2^{-40} from uniform is not distance 2^{-40} from uniform after a billion blocks; the gap can grow toward $2^{30} \cdot 2^{-40} = 2^{-10}$, a thousand times larger. This is why security proofs track how many queries the attacker makes and why "rekey before the bound" rules exist. A distance that is negligible per use can become a real edge once an attacker harvests enough uses.

7.6.3 From statistical to computational indistinguishability

Statistical distance is an *information-theoretic* measure: it bounds *any* distinguisher, even one with unlimited computing time. That is a high bar, and only a few schemes (the one-time pad, secret sharing) clear it. Most of cryptography settles for something weaker but sufficient: distributions that are *computationally* indistinguishable, meaning no *efficient* (polynomial-time) algorithm can tell them apart with non-negligible advantage, even though a computationally unbounded adversary could.

Formal

We say ensembles $\{P_\lambda\}$ and $\{Q_\lambda\}$ (indexed by a security parameter λ, e.g. key length) are *computationally indistinguishable*, written $P_\lambda \approx_c Q_\lambda$, if for every PPT distinguisher D,

$$|\Pr[D(P_\lambda) = 1] - \Pr[D(Q_\lambda) = 1]| \leq \mathsf{negl}(\lambda),$$

where $\mathsf{negl}(\lambda)$ shrinks faster than any inverse polynomial. Statistical indistinguishability ($\Delta \leq \mathsf{negl}$) is the special case where the bound holds against *all* distinguishers, not just efficient ones. A pseudorandom generator's defining promise is exactly this: its output is computationally indistinguishable from uniform, even though, having a short seed, it is statistically very far from uniform. We develop the computational story in Chapter 8.

Why it works

Why indistinguishability is the right notion of "secure." We cannot demand that a ciphertext literally be independent of the message – that forces one-time-pad-length keys (Shannon's theorem). Instead we demand something operationally just as good: the ciphertext distribution when encrypting message m_0 should be indistinguishable from the distribution when encrypting m_1. If no efficient attacker can tell which message was sent, the encryption leaks nothing an attacker can *use*, even if information is technically present. Every "IND" security definition in this book – IND-CPA, IND-CCA – is a statistical-distance-style game where the attacker's advantage is required to be negligible. Statistical distance is the unbounded version; the definitions in later chapters use the bounded-distinguisher version.

7.6.4 The data-processing inequality for distance

One property of statistical distance is used constantly. Post-processing cannot increase statistical distance: for any (possibly randomized) function f,

$$\Delta\big(f(P), f(Q)\big) \leq \Delta(P, Q).$$

If you cannot tell P from Q, you cannot tell apart anything you compute from them either – mashing the samples through f can only blur the difference, never sharpen it. This is what licenses the hybrid arguments that dominate security proofs: swap one component for an indistinguishable one, and the distance the whole system moves is at most the distance you introduced. Distances along a chain of swaps simply add (the triangle inequality, $\Delta(P, R) \leq \Delta(P, Q) + \Delta(Q, R)$, illustrated in Figure 7.6), so a proof with a handful of indistinguishable steps keeps the total advantage negligibly small.

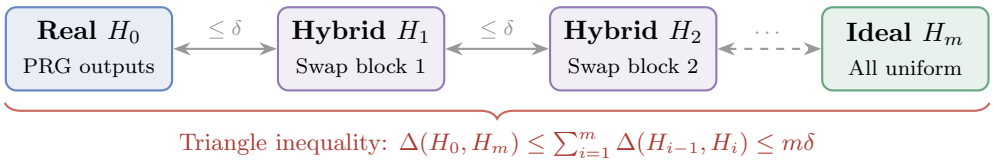


Figure 7.6: A hybrid chain across m intermediate worlds. If each single-component replacement moves the distribution by at most δ, the triangle inequality guarantees the real and ideal worlds sit at distance at most $m\delta$.

Connection

Statistical distance is the total-variation distance from the probability book's chapter on metrics between distributions, and the triangle inequality and data-processing inequality are proved there in full generality. The "advantage = edge over guessing" identity is the discrete case of the Neyman–Pearson lemma – the optimal distinguisher thresholds the likelihood ratio $P(\omega)/Q(\omega)$, which is precisely the rule "guess Q when you see heads" from Example 7.8.

7.7 Putting the pieces together

Every chapter ahead leans on something here. When we build a stream cipher, "the keystream is indistinguishable from random" is a statistical-distance claim. When we size a hash, "n-bit digest, $n/2$-bit collision security" is the birthday bound. When we specify a key generator, "256 bits of min-entropy" is Definition 7.1, not Shannon entropy. And when we prove the one-time pad perfectly secret in Chapter 9, the proof is a single line of statistical distance: the ciphertext distribution is identical for every message, so $\Delta = 0$, so no attacker – bounded or not – learns anything. Because Shannon's theorem shows that $\Delta = 0$ requires a key as long as the message, practical ciphers with short keys must bound the adversary's computation; Chapter 8 builds that language of computational hardness, PPT adversaries, and reductions.

Pause

Before moving on, make sure you can answer these without notes: (1) Why does a uniform key maximize the attacker's uncertainty? (2) What does min-entropy measure that Shannon entropy misses, and why does an attacker care? (3) Why does an n-bit hash give only $n/2$-bit collision resistance? (4) What does statistical distance 0.01 mean for an attacker trying to distinguish two distributions? If any of these is fuzzy, reread that section – the rest of the book assumes them.

7.8 Exercises

Exercise 7.1 (Uniform key arithmetic)**.** A system uses a 6-bit key, drawn uniformly. (a) How many keys are there, and what is the probability Eve guesses the key on a single try? (b) What is the expected number of guesses for a brute-force attacker trying keys in random order without repeats? (c) Eve learns that the key is even (last bit 0). After this leak, what is her single-guess success probability, and how many bits of the key's entropy has she gained?

Exercise 7.2 (A Bayesian leak)**.** A 4-bit key is uniform over 16 values. A side channel reports the parity of the key (XOR of all four bits) and is correct with probability 0.75, wrong with probability 0.25. The probe says "parity is 0." (a) Using Bayes, compute the posterior probability that the true parity is actually 0. (b) Conditioned on the report, what is Eve's best single-guess success probability, and how does it compare with her 1/16 before the leak? (c) A second, independent probe of the same accuracy also reports "parity is 0." Update again and give the new posterior. What does the trend say about repeated noisy measurements?

Exercise 7.3 (Three scores for one distribution)**.** For the distribution with probabilities 0.5, 0.25, 0.125, 0.125: (a) compute the Shannon entropy H, the min-entropy H_∞, and the guessing entropy

$G = \sum_i i\, p_i$ (outcomes sorted by decreasing probability). (b) Compute the same three for the uniform distribution on 4 values. (c) Explain in two sentences which of the three numbers you would quote to an attacker who gets exactly one guess, and which to an attacker running an offline cracker that tries candidates in the best order.

Exercise 7.4 (Entropy by hand)**.** Compute the Shannon entropy $H(X)$ and the min-entropy $H_\infty(X)$ for each distribution. (a) X uniform on 8 values. (b) Probabilities $\frac{1}{2}, \frac{1}{4}, \frac{1}{4}$. (c) Probabilities $0.7, 0.1, 0.1, 0.1$. In which case is the gap $H - H_\infty$ largest, and explain in one sentence why that distribution is the most dangerous to use as a key despite a respectable Shannon entropy.

Exercise 7.5 (Min-entropy is what bounds guessing)**.** A password is chosen so that "*password*123" has probability 0.05 and the remaining probability is spread uniformly over $10^6 - 1$ other strings. (a) Compute H_∞. (b) Compute (approximately) H. (c) An attacker gets one guess. What is the best success probability, and which entropy predicted it? (d) Argue in two sentences why quoting the Shannon entropy here would dangerously overstate security.

Exercise 7.6 (The biased-bit key)**.** A flawed generator produces each key bit independently, but each bit is 1 with probability 0.6 rather than 0.5. The key is 128 bits. (a) What is the Shannon entropy per bit, and the total over 128 bits? (b) What is the min-entropy per bit (hint: the most likely value of each bit is 1), and the total? (c) The key has 128 bits of length but how many effective bits of security against a best-guess attacker? Compare to a correct uniform 128-bit key.

Exercise 7.7 (Birthday crossover)**.** Using the approximation $\Pr[\text{collision}] \approx 1 - e^{-n^2/(2N)}$: (a) For $N = 365$, find the smallest n making the collision probability at least $\frac{1}{2}$, and compare with the exact answer 23. (b) For a 40-bit hash ($N = 2^{40}$), about how many random inputs before a collision is more likely than not? (c) State the general rule for an n-bit hash in one line.

Exercise 7.8 (Sizing a hash)**.** You are designing a system that must resist collision-finding by an attacker capable of 2^{100} hash evaluations. (a) What is the minimum digest length you should choose, and why? (b) Your colleague argues a 128-bit digest is fine "because $128 > 100$." Explain precisely where the reasoning breaks. (c) The same system needs preimage resistance against the same 2^{100} attacker – what digest length does that requirement alone demand, and why is it different?

Exercise 7.9 (Counting collisions with indicators)**.** Prove, using indicator random variables and linearity of expectation, that when n items are thrown uniformly and independently into N bins, the expected number of colliding (unordered) pairs is $\binom{n}{2}/N$. Then find the n at which this expectation first reaches 1 for $N = 2^{64}$, and connect it to the $2^{n/2}$ rule.

Exercise 7.10 (Statistical distance)**.** Let P be uniform on $\{1, 2, 3, 4\}$ and let Q assign probabilities $0.4, 0.3, 0.2, 0.1$ to those same four values. (a) Compute $\Delta(P, Q)$ from the sum form. (b) Find the optimal distinguisher and its advantage, and verify it equals Δ. (c) What is the best probability of correctly identifying the source in the referee game of this chapter?

Exercise 7.11 (Distance cannot grow)**.** Let P and Q be distributions on $\{0,1\}^8$ with $\Delta(P, Q) = 0.2$. Let f map each byte to its parity (XOR of its bits), a single output bit. (a) Argue that $\Delta(f(P), f(Q)) \le 0.2$ without computing anything. (b) Give a concrete P, Q where the inequality is strict (post-processing strictly loses distinguishing power), and one where equality holds. (c) Explain in one sentence why this inequality is what makes hybrid security proofs sound.

Exercise 7.12 (Why indistinguishability, not independence)**.** Shannon proved that perfect secrecy (ciphertext statistically independent of plaintext) forces the key to be at least as long as the message. (a) State why this makes perfect secrecy impractical for, say, encrypting a 1 GB file with a short key.

(b) Explain how computational indistinguishability sidesteps the limit while still giving meaningful security, and identify which assumption from Chapter 8 the relaxation rests on. (c) In one sentence, relate “IND-CPA security” to the statistical-distance guessing game of this chapter.

Chapter 8

Computational Hardness and One-Way Functions

In Chapter 7, we used probability and statistical distance to measure uncertainty. If we demand that a ciphertext leak zero information even to an adversary with unlimited time and computers, Shannon's theorem (which we prove next chapter) dictates an impossible price: the secret key must be as long as everything you will ever send.

Modern cryptography escapes this trap by changing the question. Instead of asking "can the adversary *ever* recover the message?" we ask "can the adversary recover the message *using a realistic amount of computation*?" In exchange for bounding the adversary's computation, we get short keys, public keys, and digital signatures. That trade rests on one bet: that certain problems are *easy to pose and hard to solve*. Multiplying two large primes is fast; recovering them from the product seems to require effort that grows wildly with their size. This chapter builds the grammar of computational security – polynomial time, PPT adversaries, negligible functions, one-way functions, trapdoors, and reductions – used in every security proof in the rest of the book.

8.1 The asymmetry we are buying

Picture the usual cast. Alice wants to send Bob something secret; Eve listens to everything that crosses the wire. In the unconditional world Eve has unlimited computing power. In the computational world Eve is merely the best-funded adversary you can imagine – a nation-state with warehouses of hardware – but still bound by the same laws of computation as everyone else. Our schemes are allowed to be breakable in principle, as long as breaking them would take Eve longer than she (or the universe) has.

For that to be a sensible bargain we need a job that is cheap for Alice and Bob and ruinously expensive for Eve. The cleanest example is a padlock. Snapping a padlock shut takes a second and no key. Opening it without the key takes a locksmith, a drill, and time. The lock is *asymmetric*: the forward direction (lock it) is easy, the backward direction (open it without the key) is hard. Cryptography relies on mathematical versions of this padlock.

Intuition

Three small concrete asymmetries, all of which we will use later:

- **Multiplication vs. factoring.** Given $p = 61$ and $q = 53$, computing $N = pq = 3233$ is instant. Given only $N = 3233$, finding the factors takes real searching. For 2048-bit N it is, as far as anyone knows, hopeless.
- **Exponentiation vs. discrete log.** Working modulo the prime $p = 23$, computing $5^6 \bmod 23 = 8$ is a handful of multiplications. Going backwards – given that $5^x \equiv 8$, find x – means searching for the exponent. With a 256-bit modulus and the right group, the backward search is the discrete logarithm problem, and it is believed hard.
- **Hashing vs. inverting a hash.** A good hash H turns "hello" into a fixed 256-bit digest in microseconds. Given the digest alone, producing *any* input that hashes to it should require trying astronomically many candidates.

In each case one direction is cheap and the other is expensive.

Everything depends on what we mean by "hard." A child can fail to factor 3233; that does not make factoring hard in any useful sense. We need a definition of hardness that (a) talks about all input sizes at once rather than one number, (b) measures effort against the size of the input, and (c) survives the fact that a lucky adversary might break one instance now and then. Building that definition carefully is the work of Sections 8.3–8.4. With it in hand we can finally say what a one-way function is and prove things about it.

History

The idea that security could rest on computational difficulty rather than perfect secrecy is due to a cluster of work in the 1970s. Whitfield Diffie and Martin Hellman at Stanford, building on independent work by Ralph Merkle, framed it openly in their 1976 paper "New Directions in Cryptography": they proposed building secrecy on functions that are easy to compute and infeasible to invert, and coined the modern notion of a *trapdoor* one-way function. Within a year Ron Rivest, Adi Shamir, and Leonard Adleman at MIT turned that abstract wish into RSA, an actual trapdoor built from factoring. Despite attempts by intelligence agencies to restrict academic publication, this conceptual move – from "unbreakable" to "infeasible to break in practice" – let cryptography leave the domain of military one-time pads and become an open science. Yet the foundational question remains open: proving that one-way functions exist would settle P $\neq$ NP.

8.2 Measuring effort: input size and asymptotics

When we say factoring "gets hard for large numbers," large compared to what? The yardstick is not the numerical value N but the number of bits it takes to *write down* N, which is about $n = \log_2 N$. A 2048-bit modulus is a number with roughly 617 decimal digits; trying to divide it by every integer up to $\sqrt{N}$ would mean about 2^{1024} trial divisions. The *input* is 2048 bits – tiny, fits in a tweet – yet the naive attack is exponential in that tiny input.

This is exactly the gap we want. We measure an algorithm's running time as a function $T(n)$ of the input length n in bits, and we care about how T grows.

Definition 8.1 (Polynomial and exponential time)**.** An algorithm runs in *polynomial time* if there is a constant c and a bound n_0 such that on every input of length $n \geq n_0$ it halts within $T(n) \leq n^c$ steps. We write $T(n) = n^{O(1)}$. An algorithm is *exponential time* if its running time grows like $2^{\Omega(n^\delta)}$ for some $\delta > 0$ – it eventually outpaces every polynomial.

Polynomial time is our stand-in for "feasible." It is a generous, slightly leaky abstraction: an algorithm that takes n^{50} steps is polynomial and yet useless in practice, while one that takes 1.000001^n is exponential and yet fine for the sizes you meet. But the abstraction is robust in a way that any fixed cutoff ("feasible means under 2^{80} steps") is not. Polynomials are closed under addition, multiplication, and composition, so if you build a feasible procedure out of feasibly many feasible pieces, the whole thing stays feasible. That closure is what makes proofs go through, and it is why the entire theory is phrased asymptotically.

Intuition

Why bits and not the number itself? Because the difficulty of a problem should be compared against how much it costs even to *state* the problem. Writing down a 2048-bit number costs 2048 bits; an attack that costs 2^{2048} is exponentially more expensive than the question. If we measured against the value N instead, trial division (about $\sqrt{N}$ steps) would look "polynomial in N" and we would fool ourselves into thinking factoring is easy. The input-length yardstick keeps us out of that trap.

Connection

The companion algorithms book develops this scaling in detail: why multiplication of n-bit numbers is polynomial (schoolbook is $O(n^2)$, and fast methods do better), why fast modular exponentiation $a^b \bmod N$ costs only $O(n)$ multiplications via repeated squaring rather than b multiplications, and why the best known factoring algorithm (the general number field sieve) runs in time roughly $\exp\big(c\, n^{1/3}(\log n)^{2/3}\big)$ – *sub*exponential, but still far above polynomial. Those cost analyses are the empirical evidence behind every hardness assumption in this chapter.

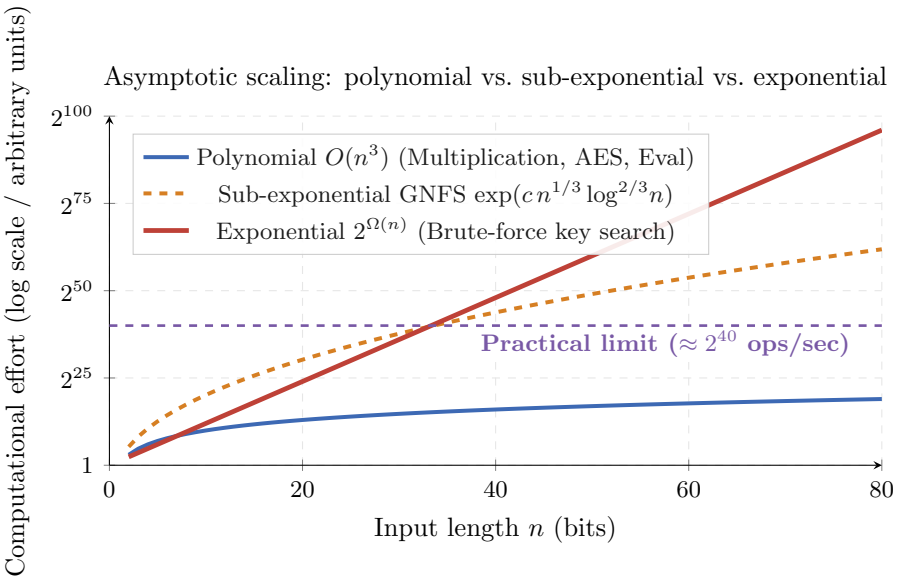


Figure 8.1: The computational hardness gap. While polynomial-time forward operations (blue) remain tiny and flat even as keys grow, exponential brute-force attacks (red) skyrocket beyond reach. Sub-exponential attacks (orange dashed, such as the General Number Field Sieve for factoring) grow faster than any polynomial, explaining why RSA requires 3072-bit keys while 256-bit elliptic curves suffice.

8.3 The adversary: probabilistic polynomial time

We do not model Eve as one fixed program but as the *best* program of her size. We model her as an algorithm, and we grant her two things beyond a plain deterministic computer. First, randomness. Real attackers flip coins – they pick random points to evaluate, guess random candidates, run randomized number-theory algorithms. Forbidding randomness would understate Eve's power and let us claim false security. So the adversary may toss fair coins for free. Second, we hold her to polynomial time, because an adversary allowed exponential time can break essentially everything (she can brute-force any key). The interesting and achievable goal is security against *efficient* adversaries. Putting these together gives the standard model of a realistic attacker.

Definition 8.2 (PPT adversary)**.** A *probabilistic polynomial-time* (PPT) algorithm is a randomized algorithm whose running time is bounded by a polynomial in the length of its input, on every input and every setting of its internal coins. A PPT *adversary* is such an algorithm whose job is to break a scheme: recover a key, distinguish two ciphertexts, forge a signature, invert a function.

Because Eve gets coins, the outcome of an attack is a random variable. "Eve breaks the scheme" is an *event with a probability*, not a yes/no fact. This is why the next ingredient – a way to talk about probabilities that are "so small they do not count" – is central to the framework.

Focus

We will attach a *security parameter*, usually called n (or λ in other texts, or k), to every scheme. Think of n as a dial: turn it up and keys get longer, the legitimate work grows

polynomially, and – if the design is sound – the adversary's success probability shrinks faster than any polynomial. Key length is roughly n; an RSA modulus of n bits is the security parameter made concrete. Every quantity below is a *function of* n: the adversary's run time, her advantage, the time to encrypt. Holding n in mind as "the knob" is the single most useful habit for reading this material.

8.4 Negligible vs. noticeable

Suppose an attack against a 256-bit key succeeds with probability 2^{-128}. That is no cause for worry: even running the attack a trillion times a second since the Big Bang, the expected number of successes is far below one. A probability that small is, for every practical and theoretical purpose, zero. We need to make "that small" precise – and make it scale with the security parameter, so that turning up n actually buys safety.

Definition 8.3 (Negligible function)**.** A function $\mathsf{negl} : \mathbb{N} \to \mathbb{R}_{\geq 0}$ is *negligible* if for every positive constant c there is a bound n_0 such that

$$\mathsf{negl}(n) < \frac{1}{n^c} \qquad \text{for all } n \geq n_0.$$

Equivalently: negl eventually drops below the reciprocal of *every* polynomial.

The definition says "smaller than $1/n^c$ for all c," which sounds abstract until you plug in examples. The functions 2^{-n}, $2^{-\sqrt{n}}$, and $n^{-\log n}$ are all negligible: each one eventually undercuts $1/n^{1000}$, $1/n^{10^6}$, and so on. By contrast $1/n^3$, $1/(1000\,n^2)$, and $1/2^{\log_2 n} = 1/n$ are *not* negligible – they are polynomially large. A function that is not negligible (more precisely, that exceeds $1/n^c$ for some fixed c infinitely often) is called *noticeable*.

Example 8.1 (Negligible or not?)**.** Sort these by plugging in the definition.

- $f(n) = 2^{-n}$: at $n = 80$ this is about 10^{-24}; it beats $1/n^c$ for every c once n is large enough. **Negligible.**
- $f(n) = 2^{-n/2}$: halving the exponent only changes the constant in front; still beats every polynomial. **Negligible.** (This is why "128-bit security" against a $2^{n/2}$ birthday attack wants $n = 256$.)
- $f(n) = 1/n^{100}$: matches the polynomial $1/n^{100}$, so it cannot stay below $1/n^{101}$. **Noticeable.**
- $f(n) = n^{-\log n}$: take logs, $\log f = -(\log n)^2$, which beats $-c \log n$ for every c. **Negligible.**
- $f(n) = 2^{-n} + 1/n^5$: a sum is negligible only if both parts are; the $1/n^5$ term dominates. **Noticeable.**

It helps to watch the crossover happen with actual numbers. The negligible function 2^{-n} starts out *larger* than the polynomial $1/n^3$ for small n and only later dives underneath it – "eventually" is doing real work in the definition.

n	2^{-n}	$1/n^3$	which is smaller?
5	3.1×10^{-2}	8.0×10^{-3}	$1/n^3$ (poly wins)
10	9.8×10^{-4}	1.0×10^{-3}	roughly equal (crossover)
20	9.5×10^{-7}	1.3×10^{-4}	2^{-n}
40	9.1×10^{-13}	1.6×10^{-5}	2^{-n}
80	8.3×10^{-25}	2.0×10^{-6}	2^{-n}

Past the crossover near $n = 10$ the exponential plunges and never resurfaces; by $n = 80$ it is twenty orders of magnitude below the polynomial, and the gap only widens. Negligibility is the claim that this crossing happens against *every* polynomial, no matter how large its degree – a higher-degree polynomial just pushes the crossover point to a larger n. We use this exact definition because it behaves well under the two kinds of composition that proofs rely on.

Why it works

Negligible × polynomial is negligible. If $\mathsf{negl}(n)$ is negligible and $p(n)$ is any polynomial, then $p(n) \cdot \mathsf{negl}(n)$ is still negligible. *Proof sketch.* Fix any constant c. We must show $p(n)\,\mathsf{negl}(n) < 1/n^c$ eventually. Since p is a polynomial, $p(n) \le n^d$ for some d and all large n. Because negl is negligible, it eventually falls below $1/n^{c+d}$. Multiply: $p(n)\,\mathsf{negl}(n) \le n^d \cdot n^{-(c+d)} = n^{-c}$, as needed. □ This is the property we lean on constantly. A PPT adversary runs polynomially many steps; if each step or each query succeeds only with negligible probability, a union bound over those steps multiplies a polynomial by a negligible, and the total stays negligible. An attacker cannot amplify a negligible edge into a real one by repeating a polynomial number of times. **Negligible + negligible is negligible.** The sum of two negligible functions is negligible (each is below $\frac{1}{2}n^{-c}$ eventually, so the sum is below n^{-c}). So a proof that strings together polynomially many negligible failure events still concludes "negligible."

These closure facts are exactly why we did *not* pick a fixed threshold like "probability under 2^{-80}." A fixed threshold is not closed under the operations proofs perform – run a 2^{-80} attack 2^{40} times and you cross it. The asymptotic definition is engineered so that the natural manipulations of a security proof (repeat a polynomial number of times, union-bound over polynomially many cases, compose two reductions) all preserve negligibility.

Common trap

Negligible is an *asymptotic* statement about a function of n, not a property of a single number. "Is 2^{-100} negligible?" is a category error – 2^{-100} is just a number. The right question is whether the *family* of success probabilities, as you scale n, eventually beats every polynomial. In engineering we do fix a concrete n ("use a 3072-bit modulus") and then talk about a concrete number like 2^{-128}; that is the asymptotic theory evaluated at one operating point. Keep the two registers straight: theory quantifies over n, deployment picks an n.

8.5 Advantage: how much better than guessing

Many attacks do not recover a key outright; they merely *distinguish*. Eve sees a ciphertext and tries to tell whether it encrypts "attack at dawn" or "attack at noon," or to tell a real cipher output from

random noise. Even an adversary who ignores the ciphertext and flips a coin wins half the time. So raw success probability overstates her skill. The meaningful quantity is how far above the trivial baseline she climbs.

Definition 8.4 (Distinguishing advantage)**.** Let $\mathcal{D}$ be a distinguisher (a PPT algorithm outputting a single bit) faced with one of two experiments, Exp_0 and Exp_1, chosen with probability $\frac{1}{2}$ each. Its *advantage* is

$$\mathsf{Adv}(\mathcal{D}) \;=\; \Big|\Pr[\mathcal{D}=1 \mid \mathsf{Exp}_1] \;-\; \Pr[\mathcal{D}=1 \mid \mathsf{Exp}_0]\Big|.$$

Equivalently, if the experiment is chosen by a hidden fair coin b and $\mathcal{D}$ outputs a guess b', then $\Pr[b'=b] = \frac{1}{2} + \frac{1}{2}\mathsf{Adv}(\mathcal{D})$.

The two forms say the same thing. Guessing blindly gives $\Pr[b'=b]=\frac{1}{2}$ and advantage 0. A perfect distinguisher gives $\Pr[b'=b]=1$ and advantage 1. Advantage is the slice of certainty Eve adds on top of coin-flipping, rescaled to live in $[0,1]$.

Example 8.2 (A tiny advantage calculation)**.** Suppose a flawed cipher leaks the last bit of the plaintext into the ciphertext one time in eight; the rest of the time the ciphertext looks uniform. Eve's strategy: if she spots the leak, output the leaked bit as her guess of b; otherwise flip a coin. With probability $\frac{1}{8}$ she sees the leak and is always right; with probability $\frac{7}{8}$ she guesses and is right half the time:

$$\Pr[b'=b] \;=\; \tfrac{1}{8}\cdot 1 \;+\; \tfrac{7}{8}\cdot\tfrac{1}{2} \;=\; \tfrac{1}{8}+\tfrac{7}{16} \;=\; \tfrac{9}{16}.$$

So $\frac{1}{2}+\frac{1}{2}\mathsf{Adv}=\frac{9}{16}$, giving $\mathsf{Adv}=\frac{1}{8}$. A non-negligible advantage of one eighth means the cipher is broken: Eve does measurably better than chance, and that is all our security definitions forbid.

Focus

What "secure" means, in one sentence. A scheme is secure (in the computational sense) when *every* PPT adversary has only *negligible* advantage, as a function of the security parameter n. Spelled out: for all PPT $\mathcal{A}$ there is a negligible negl with $\mathsf{Adv}(\mathcal{A}) \le \mathsf{negl}(n)$. Read it as a contest with three quantifiers – *for all* efficient strategies, the edge over guessing is *negligible* in n. Every formal definition in later chapters (semantic security, CPA security, unforgeability, collision resistance) is this template with a specific experiment plugged in.

8.6 One-way functions

Now we can name the central object. A one-way function is the mathematical padlock: easy to apply, hard to undo. The two "easy" and "hard" clauses are exactly the polynomial-time and negligible-advantage notions we just built.

Definition 8.5 (One-way function)**.** A function $f : \{0,1\}^* \to \{0,1\}^*$ is *one-way* if:

1. **Easy to compute.** There is a deterministic polynomial-time algorithm that, given x, outputs $f(x)$.
2. **Hard to invert.** For every PPT algorithm $\mathcal{A}$, the probability that $\mathcal{A}$ produces *some* preimage is negligible:

$$\Pr_{x \xleftarrow{\$} \{0,1\}^n}\Big[\,\mathcal{A}\big(1^n, f(x)\big) \in f^{-1}\big(f(x)\big)\,\Big] \;\le\; \mathsf{negl}(n).$$

The probability is over the random choice of x *and* over $\mathcal{A}$'s coins.

Several details in that definition deserve attention, because each one closes a loophole that would otherwise make the notion vacuous or immediately false.

Walking question

Why "some preimage" and not "the same x"? Because f need not be injective. If $f(x) = f(x')$, any inverter that outputs x' has done its job – it found something mapping to the target. Demanding the original x would be unfair when the information to distinguish x from x' is simply not present in $f(x)$. *Why is x chosen at random?* One-wayness is an *average-case* property. We are not claiming f is hard to invert on its worst input; we are claiming it is hard on a *random* input. That is a stronger, more useful demand for cryptography, where keys and messages are random. A function could be hard in the worst case yet trivial on the inputs you actually meet – useless to us. *What is that 1^n doing as an argument?* It is the security parameter written in *unary* – a string of n ones. Handing $\mathcal{A}$ a string of length n guarantees that "polynomial time" for $\mathcal{A}$ means polynomial in n, even when $f(x)$ happens to be short. Without it, an adversary could be "efficient" relative to a tiny input yet effectively be granted very little time. It is a bookkeeping device that pins down the clock.

Why it works

Why both clauses are mandatory. Drop "easy to compute" and you can declare any random-looking map one-way without being able to use it – legitimate parties could not even apply the function. Drop "hard to invert" and you have an ordinary function, no padlock at all. The notion is useful only because it combines both: useful (fast forward) *and* safe (slow backward). A constant function $f(x) = 0$ is immediately easy to compute but immediately easy to invert (output anything); the identity $f(x) = x$ is easy both ways. One-wayness lives in the narrow band where forward is cheap and backward is hopeless on average.

Intuition

A homely model: dropping a wine glass. Going from glass to shards is one function evaluation and takes a flick of the wrist. Going from shards back to glass is, thermodynamically, possible – no law forbids it – but the number of shard arrangements you would have to search through makes it hopeless in practice. One-way functions are the digital version: the forward map mixes information so thoroughly that running it backward means searching a space too large to search.

8.6.1 Candidate one-way functions

Nobody has proved that one-way functions exist. Their existence would imply P $\neq$ NP, one of the great open problems. What we have instead are *candidates* – functions that have resisted decades of expert attack and that we *believe* are one-way. Cryptography is, in this sense, an empirical science resting on unproven (but heavily stress-tested) hardness conjectures.

Example 8.3 (Multiplication / the factoring candidate)**.** Define $f(p, q) = pq$ where p, q are n-bit primes (this is the multiplication function restricted to primes). Forward: multiply, polynomial

time. Backward: given $N = pq$, recover $\{p, q\}$ – this is integer factoring. With $p = 61, q = 53$ we get $N = 3233$; inverting means factoring 3233. At 2048 bits, inverting is the problem RSA's security leans on.

Example 8.4 (Modular exponentiation / the discrete-log candidate)**.** Fix a large prime p and a generator g of $(\mathbb{Z}/p\mathbb{Z})^*$. Define $f(x) = g^x \bmod p$. Forward: fast exponentiation, $O(n)$ multiplications. Backward: given $y = g^x$, find x – the discrete logarithm. With $p = 23$, $g = 5$, and $x = 6$ we get $y = 5^6 \bmod 23 = 8$; inverting means solving $5^x \equiv 8 \pmod{23}$ for x.

Example 8.5 (A hash as a one-way function)**.** A cryptographic hash $\mathrm{H} : \{0,1\}^* \to \{0,1\}^{256}$ is designed to be *preimage resistant*: given a digest d, finding any m with $\mathrm{H}(m) = d$ should cost about 2^{256} work. Preimage resistance is precisely one-wayness for H. (Hashes ask for more – collision resistance too – which we treat in Chapter 14.)

Connection

The discrete-log toy in Example 8.4 is worth tracing by hand, because it shows the asymmetry in miniature. Forward, in $(\mathbb{Z}/23\mathbb{Z})^*$ with $g = 5$: the powers cycle as $5^1{=}5$, $5^2{=}2$, $5^3{=}10$, $5^4{=}4$, $5^5{=}20$, $5^6{=}8, \ldots$ (each step is "multiply by 5, reduce mod 23"). Backward, you are handed 8 and must find which exponent produced it – here you would scan the list. For p of a few hundred bits the list has more entries than there are atoms in the observable universe, and the best known general attacks (baby-step giant-step, Pollard's rho, index calculus) still cost far more than the polynomial forward direction. The number-theory appendix proves 5 is a generator mod 23 and explains why a generator makes the forward map a bijection onto $(\mathbb{Z}/p\mathbb{Z})^*$.

Implement it

A toy "one-way" function and a brute-force inverter, to feel the asymmetry. This is for intuition only: real one-way functions use cryptographic sizes, and a 16-bit modulus like this one is broken in microseconds.

```
# LEARNING ONLY -- toy sizes, NOT secure. Do not use for anything real.
p = 65537          # a small prime (2^16 + 1)
g = 3              # a generator of the multiplicative group mod p
def f(x):
    # forward direction: fast, O(log x) multiplications via pow()
    return pow(g, x, p)
def invert_bruteforce(y):
    # backward direction: search every exponent until one matches.
    # Cost grows with p -- here ~65k tries; at cryptographic sizes
    # this loop would outlast the universe.
    for x in range(1, p):
        if pow(g, x, p) == y:
            return x
    return None
secret = 41234
y = f(secret)                        # easy: one call
print("public value y =", y)
print("recovered x   =", invert_bruteforce(y))   # "hard": brute force
# Notice: f(secret) returned instantly. invert_bruteforce had to grind
# through up to p candidates. That gap -- tiny here -- is the entire
```

```
# bet of modern cryptography, blown up to 2048+ bits.
```

What matters here is the shape of the cost: `f` is one operation, `invert_bruteforce` is a loop over the whole space. Scale p to thousands of bits and the loop becomes physically impossible while `f` stays instant.

The brute-force loop above is the simplest possible inverter; it scans all p candidates. The real reason discrete log is only *conjectured* hard – not provably 2^n hard – is that cleverer attacks shave the exponent. The simplest of these is worth running by hand, because it shows both that the forward/backward gap is real *and* that it is smaller than naive counting suggests.

Work it yourself

Baby-step giant-step on a tiny group. We solve $5^x \equiv 9 \pmod{23}$ – recover the discrete log of 9 to base 5 in $(\mathbb{Z}/23\mathbb{Z})^*$ – in far fewer than 22 steps. The group has order $m = 22$; set $t = \lceil\sqrt{22}\rceil = 5$. Write the unknown as $x = it + j$ with $0 \le j < t$ and $0 \le i \le t$. *Baby steps.* Tabulate $5^j \bmod 23$ for $j = 0, \ldots, 4$:

$$5^0{=}1, \quad 5^1{=}5, \quad 5^2{=}2, \quad 5^3{=}10, \quad 5^4{=}4.$$

Store the pairs $\{(1,0),(5,1),(2,2),(10,3),(4,4)\}$ keyed by the power. *Giant steps.* The giant stride is $5^{-t} = 5^{-5} \bmod 23$. Since $5^5 \equiv 20$ and $20^{-1} \equiv 15 \pmod{23}$ (because $20 \cdot 15 = 300 = 13 \cdot 23 + 1$), the stride factor is 15. Now compute $9 \cdot 15^i \bmod 23$ for $i = 0, 1, 2, \ldots$ and look each result up in the baby table:

$$i{=}0: \ 9, \quad i{=}1: \ 9 \cdot 15 = 135 \equiv 20, \quad i{=}2: \ 20 \cdot 15 = 300 \equiv 1.$$

At $i = 2$ the value 1 is in the table, paired with $j = 0$. So $x = it + j = 2 \cdot 5 + 0 = 10$. *Check.* $5^{10} \bmod 23$: $5^5 \equiv 20$, so $5^{10} \equiv 20^2 = 400 \equiv 9 \pmod{23}$. Correct. We found $x = 10$ in 5 baby steps plus 3 giant steps – 8 multiplications instead of scanning 22 exponents. The win is $O(\sqrt{m})$ versus $O(m)$, the same square-root that the connection box above attributes to the birthday bound. For a 256-bit group $\sqrt{m} \approx 2^{128}$, still hopeless; but the $2\times$ in the exponent is exactly why "128-bit security" demands a 256-bit group, not a 128-bit one.

8.7 Trapdoors: a hidden shortcut

A one-way function locks the door for everyone, including the person who set it up. That is great for, say, password storage (store H(password); nobody can run it backward). But it is hopeless for sending Bob a secret: if Bob cannot invert the function either, the message is lost to him too. We need a function that is hard to invert *in general* but easy to invert for someone holding a secret key. That secret is the *trapdoor*.

Definition 8.6 (Trapdoor one-way function)**.** A *trapdoor one-way function* is a family of functions $\{f_{pk}\}$ together with a generation algorithm Gen that, on input 1^n, outputs a pair (pk, td) – a public index and a trapdoor – such that:

1. Given pk, the function f_{pk} is easy to compute.

2. Without the trapdoor, f_{pk} is one-way: for every PPT $\mathcal{A}$, $\Pr[\mathcal{A}(pk, f_{pk}(x)) \in f_{pk}^{-1}(f_{pk}(x))] \leq \mathsf{negl}(n)$.

3. **With** the trapdoor td, inversion is easy: there is a polynomial-time algorithm that, given td and $y = f_{pk}(x)$, outputs a preimage of y.

The padlock analogy extends to this case. A plain one-way function is a padlock with no key at all – you can snap it shut but nobody opens it. A trapdoor function is a padlock *with* a key: anyone can click it shut (the public direction), but only the keyholder reopens it (the trapdoor). Handing out open padlocks while keeping the key is the idea behind public-key cryptography.

Intuition

RSA, which gets all of Chapter 19, is the canonical trapdoor. The public index is (N, e) with $N = pq$; the function is $f(x) = x^e \bmod N$, easy for anyone. The trapdoor is the factorization p, q, from which one computes the secret exponent d that undoes the e-th power. Tiny instance: $p = 61, q = 53, N = 3233$. Knowing p, q lets you compute d and decrypt in a flash; knowing only $N = 3233$ leaves you factoring. Same asymmetry as before, now with a back door for the rightful owner.

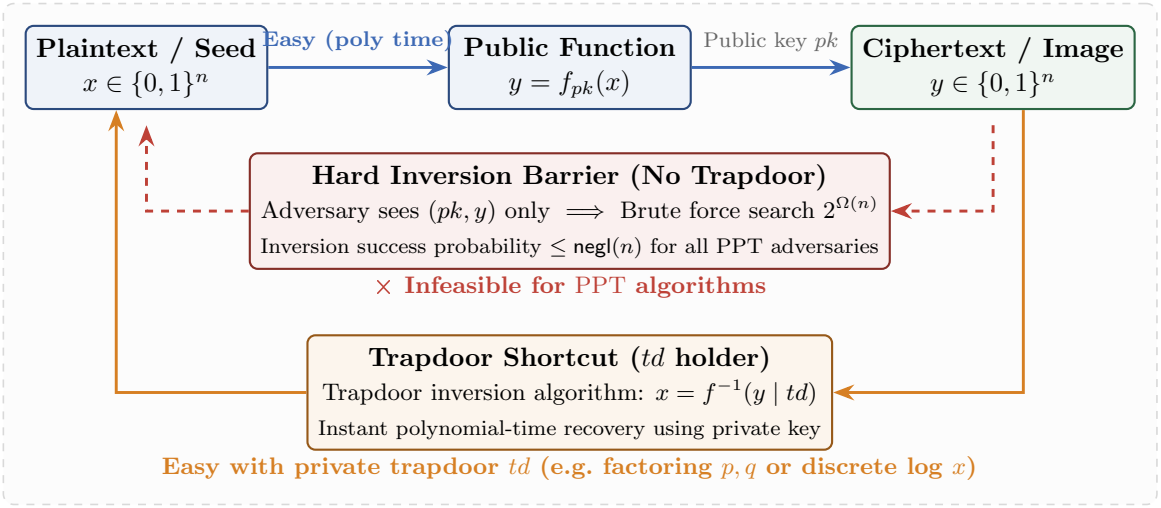


Figure 8.2: The three-way architecture of a trapdoor one-way function. Moving forward (blue) is computationally easy for anyone holding public key pk. Moving backward without the secret (red dashed) requires exhaustive searching over an exponentially large space. But holding the secret trapdoor td (orange) provides an algebraic shortcut that inverts the function in polynomial time.

Common trap

"Hard to invert without the trapdoor" must hold even though pk is *public*. The adversary sees the full description of f_{pk} – the modulus, the exponent, everything except td. Security never rests on hiding the algorithm or the public key; it rests entirely on the secrecy of td and the conjectured hardness of inverting. This is Kerckhoffs's principle in computational form: assume Eve knows the system, and make the secret the only thing she lacks.

8.8 Reductions: turning hardness into security

We have hard problems (factoring, discrete log) and we want secure schemes. The bridge between them is a *reduction*, and it is the single most important proof technique in the book. The logic runs by contraposition.

We want to prove: *if problem Y is hard, then scheme X is secure.* Directly arguing "no attack exists" is hopeless – you cannot enumerate all possible attacks. So we flip it: assume an attack exists and derive a contradiction.

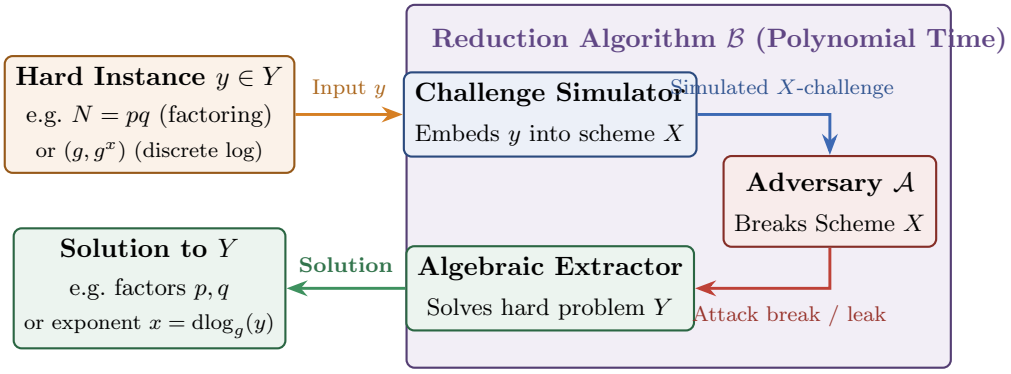
Focus

The shape of every reduction. To prove "scheme X is secure assuming Y is hard":

1. Suppose, for contradiction, that some PPT adversary $\mathcal{A}$ breaks X with non-negligible advantage $\varepsilon(n)$.
2. Build a PPT algorithm $\mathcal{B}$ – the *reduction* – that uses $\mathcal{A}$ as a subroutine (a "black box") to solve Y.
3. Show $\mathcal{B}$ solves Y with probability related to $\varepsilon(n)$, also non-negligible, in polynomial time.
4. Conclude: a PPT solver for Y exists, contradicting "Y is hard." Hence no such $\mathcal{A}$ exists, i.e. X is secure.

The slogan: *breaking X lets you solve Y*; since Y cannot be solved, X cannot be broken.

The hard part is step 2: $\mathcal{B}$ must feed $\mathcal{A}$ inputs that look exactly like a real attack scenario, then read off $\mathcal{A}$'s answer and *translate* it into a solution to Y. $\mathcal{B}$ does not need to understand *how* $\mathcal{A}$ works – it treats $\mathcal{A}$ as an oracle that, when handed a valid-looking challenge, coughs up a break with probability $\varepsilon(n)$.



Proof by Contradiction: If $\mathcal{A}$ runs in PPT with non-negligible advantage $\varepsilon(n)$, then $\mathcal{B}$ solves hard problem Y in PPT with probability $\text{poly}(\varepsilon(n))$. Since no efficient solver exists for Y, no such adversary $\mathcal{A}$ can exist: scheme X is secure.

Figure 8.3: The mechanics of a cryptographic reduction. To prove scheme X secure from hard problem Y, we build an efficient reduction $\mathcal{B}$ that wraps a hypothetical adversary $\mathcal{A}$. $\mathcal{B}$ feeds $\mathcal{A}$ a simulated environment derived from Y. When $\mathcal{A}$ breaks the scheme, $\mathcal{B}$ extracts a solution to Y, contradicting Y's hardness.

Example 8.6 (A worked reduction: square roots are as hard as factoring)**.** Let $N = pq$ with p, q

distinct odd primes, and consider the function $f(x) = x^2 \bmod N$ (the basis of the Rabin trapdoor). We show: *if you can compute square roots mod N, you can factor N.* So inverting f is at least as hard as factoring – a reduction *from* factoring *to* root-finding.

Suppose $\mathcal{A}(N, y)$ returns a square root of y whenever one exists. Build $\mathcal{B}$ to factor N:

1. Pick a random r in $\{1, \ldots, N-1\}$ with $\gcd(r, N) = 1$ (if the gcd is not 1 you already found a factor – a lucky shortcut).
2. Compute $y = r^2 \bmod N$ and hand (N, y) to $\mathcal{A}$. It returns some root s with $s^2 \equiv y \pmod N$.
3. Now $s^2 \equiv r^2$, so $N \mid (s-r)(s+r)$. Because N has two prime factors, y has *four* square roots mod N; $\mathcal{A}$ cannot tell which one $\mathcal{B}$ started from, so with probability $\frac{1}{2}$ the returned s is *not* $\pm r$. In that case $\gcd(s-r, N)$ is a nontrivial factor of N.
4. If $s = \pm r$, retry with a fresh r. Each try factors with probability $\frac{1}{2}$, so after k tries the failure probability is 2^{-k} – negligible in a polynomial number of tries.

Run it on $N = 3233 = 61 \cdot 53$. Say $r = 2$, so $y = 4$. The four roots of 4 modulo 3233 are 2, 1588, 1645, 3231 (that is, ± 2 and ± 1645, arising from the four sign choices in Figure 8.4). Suppose $\mathcal{A}$ hands back $s = 1645$, which is $\neq \pm 2$: it is congruent to $-2 \pmod{61}$ and $+2 \pmod{53}$, so $s - r = 1643$ is divisible by 53 but not by 61. Then $\gcd(s-2, 3233) = \gcd(1643, 3233) = 53$, a nontrivial factor of N. We have factored N using one call to the root-finder. *Breaking X (taking square roots) solved Y (factoring).*

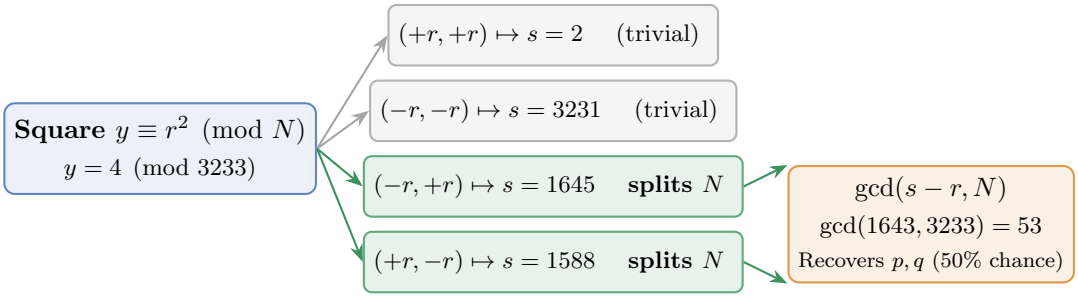


Figure 8.4: Why finding square roots modulo $N = pq$ factors N (Example 8.6). By the Chinese Remainder Theorem, every quadratic residue $y \equiv r^2 \pmod N$ has four roots corresponding to $(\pm r \bmod p,\ \pm r \bmod q)$. Because y hides which root r the reduction chose, any root-finding oracle returns a mixed-sign root $s \not\equiv \pm r \pmod N$ with probability 1/2, and $\gcd(s-r, N)$ immediately exposes a prime factor.

Why it works

What the reduction did and did not prove. It proved that root-finding mod N is *at least as hard* as factoring: a root-finder yields a factorizer of comparable efficiency. It did *not* prove factoring is hard in any absolute sense – that remains a conjecture. Reductions move hardness around; they relate problems. The foundation – "factoring is hard" – is an assumption we adopt because the world's best mathematicians have failed to refute it for half a century. A good security proof is therefore a chain: *scheme* reduces to *root-finding* reduces to *factoring*, and the last link rests on the assumption that factoring is hard.

Focus

Tightness matters. A reduction is only as useful as it is efficient. If breaking X with advantage ε yields a Y-solver with success ε/q^2 after q queries, then a real attacker with $\varepsilon = 1/2$ and $q = 2^{40}$ queries gives a Y-solver with success only 2^{-81} – weak evidence. A *tight* reduction loses little (constant factors), so breaking X would almost as easily break Y. When you read "provably secure," always ask: secure under *which* assumption, and how much does the reduction *lose*? Loose reductions still rule out whole classes of attacks, but they translate into larger recommended key sizes.

Attack

When the assumption is the weak link. A reduction proves "no efficient attack on X unless Y is easy." If Y turns out to be easier than believed, the proof is intact but the scheme is dead. Two real episodes:

- *Discrete log in weak groups.* Picking the group for a discrete-log scheme carelessly – a prime p where $p - 1$ has only small prime factors – lets the Pohlig–Hellman algorithm solve discrete log fast. The reduction "scheme is secure if discrete log is hard" is fine; the chosen instance simply made discrete log easy. The fix is to choose "safe" parameters where the assumption genuinely holds.
- *Quantum computers.* Shor's algorithm solves both factoring and discrete log in polynomial time on a large quantum computer. Every reduction to those problems survives word-for-word, yet the schemes break, because the assumption "factoring is hard" fails against a quantum adversary. This is exactly why post-quantum cryptography (Chapter 35) reaches for *different* hard problems – lattices, codes – whose hardness is believed to persist against quantum machines.

A proof of security is conditional, so check that its assumption actually holds for the parameters and adversaries you face.

8.9 Standard hardness assumptions

The chapters ahead build schemes on a short list of conjectured-hard problems. It helps to see them gathered, with the one-line statement of each and where it shows up.

Definition 8.7 (Foundational hardness assumptions)**.**

- **Factoring.** Given $N = pq$ for unknown primes p, q, find p and q. Best known classical method: the general number field sieve, sub-exponential time. *Powers:* RSA, Rabin.
- **RSA problem.** Given (N, e, y), find x with $x^e \equiv y \pmod{N}$ – invert the RSA permutation without the factorization. Believed hard; clearly no harder than factoring (factor N and you win), possibly easier (unproven). *Powers:* RSA encryption and signatures.
- **Discrete logarithm (DLP).** In a cyclic group $\mathbb{G} = \langle g \rangle$, given $y = g^x$, find x. *Powers:* Diffie–Hellman, ElGamal, DSA, and – in elliptic-curve groups – ECDH and ECDSA.
- **Computational Diffie–Hellman (CDH).** Given g^a and g^b, compute g^{ab}. No harder than DLP (solve for a, then exponentiate). *Powers:* key exchange.

- **Decisional Diffie–Hellman (DDH).** Distinguish (g^a, g^b, g^{ab}) from (g^a, g^b, g^c) for random c. A *decision* problem, strictly easier to attack than CDH in some groups, which is why group choice matters. *Powers:* the security *proofs* of ElGamal and many protocols.

These problems are related by short reductions (Figure 8.5): DDH $\leq$ CDH $\leq$ DLP, where $\leq$ means "no harder than" (an algorithm for the right-hand problem yields one for the left). Solving DLP solves CDH solves DDH; the converses are open. Cryptographers like to assume the *weakest* sufficient problem is hard, because a scheme proven secure under a weaker assumption stands on firmer ground.

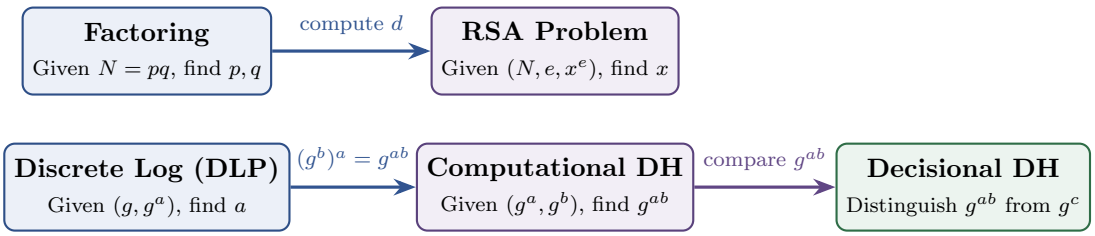


Figure 8.5: Reduction hierarchy among standard number-theoretic assumptions. Arrows point from the harder problem to the no-harder problem ($A \to B$ means an oracle for A immediately solves B).

Connection

The two cost claims underpinning these assumptions are developed in the companion books. The algorithms book derives the number field sieve's sub-exponential running time and the $O(\sqrt{|\mathbb{G}|})$ cost of generic discrete-log attacks (baby-step giant-step, Pollard's rho) – the $\sqrt{}$ being a birthday-bound phenomenon straight out of the probability book. Those two results are why a 3072-bit RSA modulus and a 256-bit elliptic-curve group are reckoned to offer comparable "128-bit" security: the elliptic-curve group has no known sub-exponential attack, so it can be far smaller for the same effort.

In the real world

Where this lands in practice. When your browser opens a TLS connection, the handshake runs an elliptic-curve Diffie–Hellman key exchange – its secrecy is the CDH/DDH assumption in an EC group, made concrete at curve `X25519`. The server's certificate is signed with RSA or ECDSA, leaning on factoring or EC discrete log. Your saved passwords sit in a database as one-way hashes (a slow, salted hash like `argon2` or `bcrypt`, chosen so that even inverting one digest is costly). WiFi's WPA3 uses a Diffie–Hellman-based handshake (Dragonfly) so that an eavesdropper who records the exchange still faces a discrete-log wall. Signal's double ratchet chains EC Diffie–Hellman steps so that compromising today's key does not unravel yesterday's messages. Every one of these is the same bet – a problem easy forward, hard backward – instantiated at a key size chosen so the backward cost dwarfs any attacker's budget. Pick the size from current guidance (a competent library defaults sensibly), not from a textbook's toy numbers.

8.10 Putting the pieces together

The argument of this chapter, end to end: we cannot afford unconditional secrecy, so we settle for security against *efficient* adversaries. "Efficient" is polynomial time in the security parameter n; the adversary is PPT, randomized and time-bounded. "Insecure" means some such adversary wins with *noticeable* probability; "secure" means every such adversary's *advantage* – her edge over guessing – is *negligible*, shrinking faster than any inverse polynomial as we turn up n. The negligible/noticeable line is drawn exactly where it is so that the manipulations of a proof (repeat polynomially often, union-bound, compose) cannot turn safe into unsafe.

On that foundation we built the central object: a *one-way function*, easy forward and hard backward on average, and its richer cousin the *trapdoor* function, hard backward for everyone except the holder of a secret – the seed of public-key cryptography. We do not know one-way functions exist, but we have well-studied candidates from number theory. Finally, the *reduction* is how we turn a hardness conjecture into a security guarantee: show that breaking the scheme would solve a problem believed hard, and you have proven the scheme secure *relative to* that belief.

Everything after this is specialization. Perfect secrecy (Chapter 9) is the unconditional baseline we are departing from. Stream and block ciphers build fast symmetric mixing. RSA instantiates a trapdoor from factoring; Diffie–Hellman and ElGamal from discrete log; elliptic curves shrink the groups. Signatures, key exchange, TLS, and Signal stack these primitives. When a later proof says "secure assuming DDH" or "a forger would invert RSA," you now hold the entire grammar of that sentence: the quantifier over PPT adversaries, the negligible advantage, the reduction doing the work.

Pause

Before the exercises, self-check. Can you (1) say why we measure time in the input's *bit-length*; (2) decide whether $n^{-\log n}$ is negligible and justify it; (3) state the two clauses of a one-way function and the loophole each closes; (4) explain what the trapdoor adds and why pk being public is fine; (5) recite the four-step shape of a reduction and what contradiction it reaches? If any of these is fuzzy, reread that section – the rest of the book assumes all five.

8.11 Exercises

Exercise 8.1 (Negligible or noticeable)**.** For each function decide whether it is negligible in n and justify against Definition 8.3: (a) $2^{-\sqrt{n}}$; (b) $\dfrac{1}{n^{10}+5n}$; (c) $\dfrac{n^3}{2^n}$; (d) $\dfrac{1}{(\log n)^n}$; (e) $2^{-n}+n^{-2}$. For the noticeable ones, exhibit a constant c for which the function exceeds $1/n^c$ infinitely often.

Exercise 8.2 (Closure properties)**.** Prove from the definition that if negl_1 and negl_2 are negligible then so is $\mathsf{negl}_1 + \mathsf{negl}_2$. Then show that for any fixed polynomial p, $p(n) \cdot \mathsf{negl}_1(n)$ is negligible. Finally, give a counterexample showing that the *quotient* $\mathsf{negl}_1(n)/\mathsf{negl}_2(n)$ need not be negligible (indeed need not even be bounded).

Exercise 8.3 (Advantage arithmetic)**.** A distinguisher $\mathcal{D}$ outputs its correct guess of the hidden bit b with probability $\Pr[b' = b] = 0.62$. (a) What is its advantage? (b) A second distinguisher achieves advantage 0.001; what is its $\Pr[b' = b]$? (c) Suppose a cipher leaks one specific plaintext bit with

probability 1/16 and otherwise looks uniform. Following Example 8.2, compute the advantage of the obvious distinguisher.

Exercise 8.4 (Trace the toy one-way function)**.** Work in $(\mathbb{Z}/23\mathbb{Z})^*$ with generator $g = 5$. (a) Compute $5^1, \ldots, 5^{11} \bmod 23$ by hand, each as "previous $\times 5 \bmod 23$." (b) Use your table to solve the discrete log $5^x \equiv 9 \pmod{23}$. (c) Roughly how many table entries would the analogous brute-force search scan for a 256-bit modulus, and why does that make the forward/backward asymmetry usable?

Exercise 8.5 (Break a bad one-way function)**.** Consider $f(x) = (3x + 7) \bmod 2^n$, claimed by a vendor to be one-way on n-bit inputs. Show it is *not* one-way by giving an explicit polynomial-time inverter. (Hint: 3 is invertible modulo 2^n because $\gcd(3, 2^n) = 1$; find $3^{-1} \bmod 2^n$ and solve for x.) State the general lesson about why *linear* functions can never be one-way.

Exercise 8.6 (A reduction by hand)**.** Carry out the factoring-from-square-roots reduction of Example 8.6 concretely with $N = 77 = 7 \cdot 11$. (a) List all four square roots of $y = 4$ modulo 77 (use the Chinese Remainder structure: a root is $\pm 2 \bmod 7$ combined with $\pm 2 \bmod 11$). (b) Pick $r = 2$ and a returned root $s \neq \pm 2$; compute $\gcd(s - 2, 77)$ and verify you recover a prime factor. (c) Explain why the reduction fails to learn anything when $\mathcal{A}$ returns $s = \pm r$, and why retrying fixes this.

Exercise 8.7 (Order the assumptions)**.** Using only the reduction idea "an algorithm for the harder problem yields one for the easier," prove the two relations CDH $\leq$ DLP and DDH $\leq$ CDH. That is: (a) show a discrete-log oracle lets you solve CDH; (b) show a CDH oracle lets you solve DDH. Conclude that a scheme proven secure under DDH is on weaker (hence safer) ground than one needing DLP to be hard.

Exercise 8.8 (Why the assumption can fail)**.** A protocol designer proves their scheme secure "assuming discrete log is hard in $(\mathbb{Z}/p\mathbb{Z})^*$" and then picks p such that $p - 1 = 2 \cdot 3 \cdot 5 \cdots$ is a product of small primes. (a) Without computing it in full, explain why Pohlig–Hellman makes discrete log in this group easy, by reducing it to discrete logs in the small prime-order subgroups. (b) Is the security *proof* wrong? Argue precisely what broke – the proof or the instantiation – and what "safe prime" choice repairs it.

Exercise 8.9 (Trapdoor, conceptually)**.** Explain in your own words, with the padlock analogy, why a *plain* one-way function is enough to store passwords safely but *not* enough to let a stranger send you a private message, whereas a *trapdoor* one-way function enables both. Then identify, for RSA's trapdoor $f(x) = x^e \bmod N$, exactly which piece of data is the public index pk and which is the trapdoor td, and why publishing pk does not help an attacker.

Exercise 8.10 (Baby-step giant-step by hand)**.** Reusing the method from the lab box, solve $5^x \equiv 6 \pmod{23}$ in $(\mathbb{Z}/23\mathbb{Z})^*$. (a) Reuse the baby table $\{5^0, \ldots, 5^4\}$ and the giant stride 15; tabulate $6 \cdot 15^i \bmod 23$ until you hit a stored power, and read off $x = it + j$. (b) Verify by computing $5^x \bmod 23$. (c) You did at most $5 + 5 = 10$ multiplications; the brute-force loop would do up to 22. State the general count, $O(\sqrt{m})$ versus $O(m)$, and explain why this forces a 256-bit group for 128-bit security.

Exercise 8.11 (Find the crossover)**.** Consider the negligible function 2^{-n} against the polynomial $1/n^5$. (a) By trial (or a few lines of code) find the smallest integer n at which $2^{-n} < 1/n^5$, and confirm the inequality never flips back. (b) Explain why raising the polynomial to $1/n^{50}$ only moves this crossover to a larger n rather than making 2^{-n} non-negligible. (c) Contrast with $1/n^5$ versus $1/n^6$: why do *two* polynomials never settle the question of negligibility between them?

Exercise 8.12 (Tightness costs key bits)**.** A scheme has a reduction showing that an adversary with advantage ε making q queries yields a factoring algorithm succeeding with probability ε/q^2. Suppose a realistic adversary has $\varepsilon = 1$ and $q = 2^{40}$. (a) With what probability does the derived factorizer succeed, and why is that "weak" evidence of security at a modulus where factoring costs about 2^{112}? (b) Argue qualitatively why a *loose* reduction pushes you toward *larger* key sizes than a tight one would, to keep the guaranteed security level fixed.

Part II

Symmetric Cryptography

Chapter 9

Perfect Secrecy and the One-Time Pad

In Chapter 8, we built the language of computational hardness and bounded adversaries. Before we rely on computational limits, we examine the unconditional baseline: there is one encryption scheme in this book that is provably, mathematically unbreakable against an adversary with infinite computing power and infinite time. It is the one-time pad, and Claude Shannon proved it secure in 1949.

If we have a perfect cipher, why does the rest of this book exist? The one-time pad's perfection comes with a price tag so steep that almost nobody can afford to pay it. This chapter defines perfect secrecy, proves the one-time pad meets that definition, and proves Shannon's theorem that any perfectly secret scheme needs a key at least as long as the message; the two-time pad attack then shows what happens when that requirement is violated.

9.1 The setting: Alice, Bob, Eve, and a shared secret

Fix the cast. Alice wants to send a message m to Bob over a channel that Eve can read in full. Alice and Bob share a secret key k that Eve does not know. Alice computes a ciphertext $c = \mathrm{Enc}_k(m)$ and sends c; Bob computes $m = \mathrm{Dec}_k(c)$ and recovers the message. Eve sees c and nothing else. This is *symmetric* or *private-key* encryption: one key, known to both legitimate parties, used to lock and to unlock.

Definition 9.1 (Private-key encryption scheme)**.** A *private-key encryption scheme* over message space $\mathcal{M}$, key space $\mathcal{K}$, and ciphertext space $\mathcal{C}$ is a triple of algorithms $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$:

- Gen outputs a key $k \in \mathcal{K}$, chosen according to some distribution (for us, almost always uniform).
- Enc takes a key k and a message $m \in \mathcal{M}$ and outputs a ciphertext $c = \mathrm{Enc}_k(m) \in \mathcal{C}$. It may be randomized.
- Dec takes a key k and a ciphertext c and outputs a message $m = \mathrm{Dec}_k(c)$.

Correctness requires that for every key k that Gen can output and every message $m \in \mathcal{M}$,

$$\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = m.$$

Correctness is the boring half: decryption undoes encryption for the legitimate user. Secrecy is the interesting half, and the rest of the chapter is about pinning down what it should mean.

Before we can talk about secrecy we have to be precise about what randomness is in play. There are two independent random things in this story, and conflating them is the source of most confusion.

The first is the key. We choose k at random when we run Gen. Write K for the random variable that is the key.

The second is the message. From Eve's point of view, the message is also uncertain – if she already knew m, there would be nothing to hide. We model her uncertainty with a random variable M over $\mathcal{M}$, with some distribution $\Pr[M = m]$. In our model, this distribution is Eve's *prior*: whatever she believed about the message before she saw any ciphertext. Maybe she knows Alice is voting in an election and $\mathcal{M} = \{\texttt{yes}, \texttt{no}\}$ with $\Pr[M = \texttt{yes}] = 0.7$. The scheme does not get to choose this distribution; the world does. A good definition of secrecy must work for *every* possible prior.

Focus

Two random variables, kept rigorously separate:

- K – the key, sampled by Gen, unknown to Eve. The scheme controls its distribution.
- M – the message, modeling Eve's prior uncertainty. The world controls its distribution.

K and M are **independent**: Alice's choice of what to say has nothing to do with the random key her crypto library happened to generate. The ciphertext $C = \mathrm{Enc}_K(M)$ is then a random variable determined by both.

9.2 What should "secret" mean? Three tries

We build up to the definition in three attempts.

Try 1: "Eve cannot recover the key." Too weak, and aimed at the wrong target. We do not care about the key for its own sake; we care about the message. A scheme could leak the whole message while keeping the key safe – imagine $\mathrm{Enc}_k(m) = m$, ignoring the key entirely. The key is perfectly hidden, yet the message is sent in plaintext.

Try 2: "Eve cannot recover the message." Better, but still too weak. Suppose Eve learns the first half of a message but not the second. She has not "recovered the message," yet she has learned plenty. Or suppose the ciphertext reveals that m is one of two possibilities and rules out a million others. Recovering nothing in full while learning almost everything is a real failure mode.

Try 3: "The ciphertext tells Eve nothing she did not already know." The right standard is that seeing c should not change Eve's beliefs about the message at all. Whatever probability she assigned to each candidate message before, she assigns exactly the same probability after. The ciphertext carries no information about the message.

That is Shannon's idea, and we can write it as a clean equation.

Definition 9.2 (Perfect secrecy)**.** An encryption scheme $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ over message space $\mathcal{M}$ is *perfectly secret* if for every distribution over M, every message $m \in \mathcal{M}$, and every ciphertext $c \in \mathcal{C}$ with $\Pr[C = c] > 0$,

$$\Pr[M = m \mid C = c] \;=\; \Pr[M = m].$$

In words: the probability the message was m, *given that Eve saw ciphertext* c, equals the probability it was m with no ciphertext at all. Conditioning on the ciphertext does nothing. In the language of

the probability tools from earlier chapters, M and C are independent random variables.

Intuition

Perfect secrecy is the statement "C and M are independent." Eve can inspect the ciphertext at length, run any computation on it, cross-reference it with anything – and her posterior belief about the message is identical to her prior. The ciphertext is correlated with the message through the key, but because the key is unknown and random, that correlation carries no usable information.

This is a strong demand. It must hold for *every* prior distribution, so an adversary who happens to have excellent side information about the message is still no better off after seeing c. And nothing in the definition mentions computing power. Eve may have unbounded computing time. Perfect secrecy is an *information-theoretic* guarantee: the information simply is not there to be extracted, regardless of how hard anyone computes.

9.2.1 An equivalent and easier-to-check form

Definition 9.2 quantifies over all priors, which is awkward to verify. Here is a reformulation that is logically equivalent and usually far easier to use.

Lemma 9.1 (Ciphertext distribution is message-independent)**.** *A scheme is perfectly secret if and only if for every pair of messages $m_0, m_1 \in \mathcal{M}$ and every ciphertext $c \in \mathcal{C}$,*

$$\Pr[\mathrm{Enc}_K(m_0) = c] \;=\; \Pr[\mathrm{Enc}_K(m_1) = c],$$

where the probability is over the choice of key $K \leftarrow \mathrm{Gen}$ (and any randomness in Enc*).*

In words: the distribution of the ciphertext is the same no matter which message you encrypt. Encrypting `ATTACK` and encrypting `RETREAT` produce ciphertexts drawn from *identical* distributions. Since the ciphertext's distribution carries no fingerprint of the message, observing a particular c cannot favor one message over another.

Why it works

Why the two forms agree. Start from the equivalent form: $\Pr[\mathrm{Enc}_K(m) = c]$ is some value $p(c)$ that does not depend on m. Apply Bayes' rule:

$$\Pr[M = m \mid C = c] = \frac{\Pr[C = c \mid M = m]\,\Pr[M = m]}{\Pr[C = c]} = \frac{p(c)\,\Pr[M = m]}{\Pr[C = c]}.$$

Now $\Pr[C = c] = \sum_{m'} \Pr[C = c \mid M = m']\Pr[M = m'] = \sum_{m'} p(c)\Pr[M = m'] = p(c)$, because the probabilities of all messages sum to 1. The $p(c)$ cancels top and bottom, leaving $\Pr[M = m \mid C = c] = \Pr[M = m]$. That is Definition 9.2. The converse runs the same algebra backward. The proof is one application of Bayes plus the observation that a message-independent likelihood factors out of the normalizer.

We now have a concrete goal: build a scheme where every message produces the same spread of ciphertexts. The one-time pad does exactly this, using XOR.

9.3 XOR: the operation that makes pads work

The one-time pad runs on bitwise exclusive-or, written $\oplus$. On single bits,

$$0 \oplus 0 = 0, \quad 0 \oplus 1 = 1, \quad 1 \oplus 0 = 1, \quad 1 \oplus 1 = 0.$$

XOR is addition modulo 2: it returns 1 exactly when the inputs differ. On bit-strings of equal length we XOR position by position. Three properties make it suitable for the pad.

- **It is its own inverse.** $b \oplus b = 0$ for any bit b, so $(x \oplus k) \oplus k = x$. XOR-ing with the same value twice returns you to the start. This single fact *is* the decryption algorithm.
- **It is associative and commutative.** You can reorder and regroup XORs freely: $a \oplus b \oplus c$ is unambiguous.
- **XOR with a uniform random bit is a uniform random bit, and it hides the input.** This is the key property that makes the one-time pad work, so let us isolate it.

Lemma 9.2 (The masking lemma)**.** *Let x be any fixed bit (or bit-string) and let K be a uniformly random bit (or bit-string of the same length), independent of x. Then $Y = x \oplus K$ is uniformly distributed, and in particular its distribution does not depend on x.*

Proof. Take the single-bit case; the multi-bit case is just this applied independently to each position. We want $\Pr[Y = 0]$ and $\Pr[Y = 1]$. If $x = 0$, then $Y = K$, so $\Pr[Y = 0] = \Pr[K = 0] = \frac{1}{2}$ and $\Pr[Y = 1] = \frac{1}{2}$. If $x = 1$, then $Y = 1 \oplus K$, which is 1 when $K = 0$ and 0 when $K = 1$; again each outcome has probability $\frac{1}{2}$. Either way Y is uniform. Since uniform is uniform regardless of x, the distribution of Y does not depend on x. □

Intuition

A fair coin XOR-ed onto your bit *is* a fair coin. The mask K is equally likely to leave your bit alone (when $K = 0$) or to flip it (when $K = 1$), and those two cases are equally likely, so the output is a coin flip no matter what you started with. Whatever you wanted to hide, the result looks like noise – and looks like *the same* noise whether your secret was a 0 or a 1. That last clause is Lemma 9.1 in miniature: the ciphertext's distribution is the same for every message. Stack n independent fair coins side by side and you can hide an n-bit message under n bits of perfect noise.

This is the main idea of the one-time pad; the construction and proof below follow from the masking lemma.

9.4 The one-time pad

Definition 9.3 (One-time pad)**.** Fix a message length n. The message, key, and ciphertext spaces are all $\mathcal{M} = \mathcal{K} = \mathcal{C} = \{0,1\}^n$. The scheme is:

- Gen: output a key $k \xleftarrow{\$} \{0,1\}^n$, i.e. n uniformly random independent bits.
- $\mathrm{Enc}_k(m) = m \oplus k$.

- $\mathrm{Dec}_k(c) = c \oplus k$.

Correctness is immediate from XOR being self-inverse:

$$\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = (m \oplus k) \oplus k = m \oplus (k \oplus k) = m \oplus 0^n = m.$$

The data flow is one row of XOR gates with the key tapped in twice – once to lock, once to unlock.

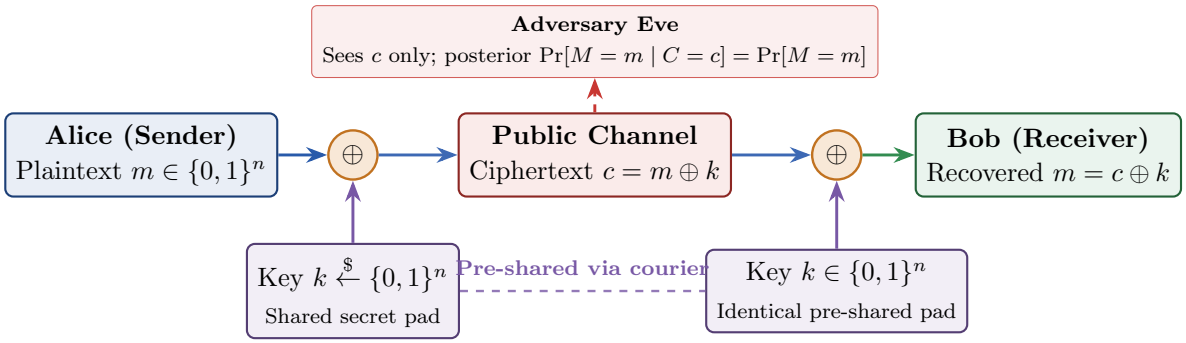


Figure 9.1: The one-time pad architecture. Alice and Bob pre-share an n-bit uniform random key k through an out-of-band secure channel. Encryption bitwise XORs the plaintext with k, creating uniform noise on the wire. Bob recovers m by XOR-ing c with the same key, since $(m \oplus k) \oplus k = m \oplus 0 = m$.

9.4.1 A tiny worked example

Let us encrypt a 5-bit message by hand. Say Alice wants to send the message $m = \texttt{10110}$ and the key generator handed her $k = \texttt{01101}$. XOR them column by column – output 1 where the bits differ:

$$\begin{array}{rccccc} m = & 1 & 0 & 1 & 1 & 0 \\ k = & 0 & 1 & 1 & 0 & 1 \\ \hline c = & 1 & 1 & 0 & 1 & 1 \end{array}$$

So $c = \texttt{11011}$ goes on the wire. Bob, holding the same $k = \texttt{01101}$, XORs it back:

$$\begin{array}{rccccc} c = & 1 & 1 & 0 & 1 & 1 \\ k = & 0 & 1 & 1 & 0 & 1 \\ \hline m = & 1 & 0 & 1 & 1 & 0 \end{array}$$

and recovers $\texttt{10110}$. Notice that the third column shows $1 \oplus 1 = 0$ in encryption and $0 \oplus 1 = 1$ in decryption: the key bit toggles the message bit on the way out and toggles it back on the way in.

The same example shows why the pad is perfectly secret. Suppose Eve intercepts $c = \texttt{11011}$ and wonders whether the message was $\texttt{10110}$. The key that would produce c from that message is $k = m \oplus c = \texttt{10110} \oplus \texttt{11011} = \texttt{01101}$ – a perfectly possible key. But suppose she guesses the message was instead $\texttt{00000}$. The key explaining *that* is $k' = \texttt{00000} \oplus \texttt{11011} = \texttt{11011}$ – also a perfectly possible key. In fact, *for every one of the* $2^5 = 32$ *candidate messages, there is exactly one key that maps it to the observed* c, and all 32 keys are equally likely. Eve cannot prefer any message over any other. The ciphertext has told her nothing.

Walking question

You intercept the 3-bit ciphertext $c = \texttt{101}$. The message space is all of $\{0,1\}^3$. For the candidate message $m = \texttt{111}$, which key would Alice have needed? And for $m = \texttt{000}$? Convince yourself each of the 8 candidate messages has its own unique explanatory key.

Answer. $k = m \oplus c$. For $\texttt{111}$: $\texttt{111} \oplus \texttt{101} = \texttt{010}$. For $\texttt{000}$: $\texttt{000} \oplus \texttt{101} = \texttt{101}$. As m ranges over all 8 strings, $k = m \oplus c$ also ranges over all 8 strings, hitting each exactly once – the map $m \mapsto m \oplus c$ is a bijection. So eight messages correspond to eight distinct, equally likely keys, and no message is favored.

9.4.2 The proof of perfect secrecy

Theorem 9.1 (Shannon, 1949)**.** *The one-time pad is perfectly secret.*

Proof. We verify the equivalent form, Lemma 9.1: for every message m and ciphertext c, the quantity $\Pr[\mathrm{Enc}_K(m) = c]$ is the same. Compute it. The encryption is $\mathrm{Enc}_K(m) = m \oplus K$, so

$$\Pr[\mathrm{Enc}_K(m) = c] = \Pr[m \oplus K = c] = \Pr[K = m \oplus c].$$

The last step XORs m onto both sides of the event. Now K is uniform on $\{0,1\}^n$, so every specific value – including the value $m \oplus c$ – has probability exactly 2^{-n}. Therefore

$$\Pr[\mathrm{Enc}_K(m) = c] = 2^{-n}$$

for every m and every c. The value 2^{-n} does not mention m at all, so it is the same for all messages. By Lemma 9.1, the scheme is perfectly secret. □

The proof is short because the masking lemma does the work: a uniform key turns the ciphertext into uniform noise whose distribution forgets the message. We can also see it through Definition 9.2 directly, which makes the claim that Eve learns nothing explicit.

Why it works

The Bayesian view. Eve sees c and updates her belief about m using Bayes:

$$\Pr[M = m \mid C = c] = \frac{\Pr[C = c \mid M = m]\ \Pr[M = m]}{\Pr[C = c]}.$$

The likelihood is $\Pr[C = c \mid M = m] = \Pr[K = m \oplus c] = 2^{-n}$, a constant. The evidence term is $\Pr[C = c] = \sum_{m'} 2^{-n} \Pr[M = m'] = 2^{-n}$. The constants cancel:

$$\Pr[M = m \mid C = c] = \frac{2^{-n}\ \Pr[M = m]}{2^{-n}} = \Pr[M = m].$$

The posterior equals the prior: whatever Eve believed before seeing c, she still believes afterward.

Common trap

A frequent misreading: "the one-time pad is secure because the key is secret." Secrecy of the key is necessary for every cipher, including breakable ones. What makes the pad *perfect* is something sharper: the key is *as long as the message* and *uniformly random*, so the set of possible keys exactly covers the set of possible messages, one key per (message, ciphertext) pair. Shorten the key, reuse the key, or skew its distribution, and the bijection breaks – some messages become more likely than others given c, and the proof collapses. The next sections show each of those three failures.

History

The construction is older than its proof. In 1882 Sacramento banker Frank Miller described a cipher using single-use random key lists for telegraph traffic, but the idea was forgotten. In 1917 Gilbert Vernam, an engineer at AT&T, patented an electromechanical cipher that XORed teletype characters against a punched paper key tape. Vernam originally used repeating loops of tape; Captain Joseph Mauborgne of the U.S. Army Signal Corps recognized that repeating tapes fell to frequency analysis, and that if the tape were truly random and never reused, the system was unbreakable.

That engineering conviction became a theorem in 1949 when Claude Shannon published *Communication Theory of Secrecy Systems*, the paper that founded cryptography as a mathematical science (a year after he founded information theory). Shannon defined perfect secrecy, proved the pad achieves it, and proved that *any* perfectly secret scheme needs a key at least as long as the message (Section 9.6). The one-time pad saw heavy real use: Soviet intelligence used pads for sensitive cables, and the Washington–Moscow "hot line" established after the Cuban Missile Crisis was protected by one-time tape.

Implement it

A toy one-time pad in Python. It shows the mechanics – and, by construction, every way to misuse it.

```python
import os

def otp_encrypt(message: bytes, key: bytes) -> bytes:
    if len(key) != len(message):
        raise ValueError("key must be exactly as long as the message")
    return bytes(m ^ k for m, k in zip(message, key))

# decryption is the SAME function: XOR is its own inverse
otp_decrypt = otp_encrypt

msg = b"ATTACK AT DAWN"
key = os.urandom(len(msg))          # fresh, uniform, message-length key
ct  = otp_encrypt(msg, key)
print(ct.hex())                      # looks like pure noise
assert otp_decrypt(ct, key) == msg   # round-trips perfectly

# The whole security argument lives in three rules this code does NOT enforce:
#   1. key from a TRUE random source (os.urandom is a CSPRNG, not perfect)
```

```
#   2. key length == message length (checked above)
#   3. NEVER reuse a key (nothing here stops you -- and that is the danger)
```

Learning only. This is a teaching toy. Real systems do not use one-time pads except in rare, specialized settings, precisely because the three rules above are nearly impossible to satisfy at scale. `os.urandom` is a cryptographic PRNG, not a source of perfect randomness, so even this code is only *computationally* secure, not perfectly secret. Never roll your own cipher for production; use a vetted authenticated-encryption library.

9.5 Why the key must be long, random, and fresh

The proof leaned on three properties of the key: it is uniformly random, it is as long as the message, and (implicitly, because we encrypt one message) it is used once. Drop any one and perfect secrecy is lost. We take them in turn, with the most consequential – reuse – getting its own attack section.

9.5.1 Random: a biased pad leaks

The masking lemma needed K *uniform*. Suppose instead an implementation's key bits are biased, say each bit is 1 with probability 0.9. Then for a single-bit message, $\Pr[\mathrm{Enc}_K(0) = 1] = \Pr[K = 1] = 0.9$ while $\Pr[\mathrm{Enc}_K(1) = 1] = \Pr[K = 0] = 0.1$. The ciphertext bit's distribution now *depends on the message*: a ciphertext 1 strongly indicates that the plaintext was 0. Lemma 9.1 fails and so does secrecy. The keystream must be drawn from the uniform distribution, which in practice means a genuine entropy source, not a predictable counter or a low-entropy seed. Chapter 17 returns to how hard this is to get right.

9.5.2 Long: a short key cannot cover the messages

What if the key is shorter than the message – say we have a 128-bit key and want to send a 1-megabyte file? The natural temptation is to stretch or repeat the key. Stretching it with an algorithm is exactly what a *stream cipher* does (next chapter), and it gives up perfect secrecy for computational security. Repeating it is the classic Vigenere blunder and is catastrophically weak. But the deepest answer is Shannon's theorem: *no* clever trick can make a short key perfectly secret over long messages. We prove that next.

9.6 Shannon's theorem: perfect secrecy forces a huge key

The next result explains why the one-time pad is impractical. Its one impractical-looking property – a key as long as everything you will ever send – is required for *any* perfectly secret scheme, not just for XOR or Vernam's tape.

Theorem 9.2 (Shannon's key-length bound)**.** *If* $(\mathrm{Gen}, \mathrm{Enc}, \mathrm{Dec})$ *is perfectly secret over message space* $\mathcal{M}$ *with key space* $\mathcal{K}$*, then* $|\mathcal{K}| \geq |\mathcal{M}|$*. In particular, the key must contain at least as many bits as the message.*

Proof. The argument is a counting argument. Assume for contradiction $|\mathcal{K}| < |\mathcal{M}|$.

Take any ciphertext c that actually occurs, i.e. $\Pr[C = c] > 0$. Consider the set of messages you could possibly decrypt c *to*, ranging over all keys:

$$\mathcal{M}(c) = \{\, \mathrm{Dec}_k(c) : k \in \mathcal{K} \,\}.$$

Each key k yields one message $\mathrm{Dec}_k(c)$, so $\mathcal{M}(c)$ has at most $|\mathcal{K}|$ elements – you cannot reach more distinct messages than you have keys. Since we assumed $|\mathcal{K}| < |\mathcal{M}|$, the set $\mathcal{M}(c)$ misses at least one message. Call a missed message m^*: *no key* decrypts c to m^* (Figure 9.2).

But that means, if the true message had been m^*, the ciphertext c could never have arisen: $\Pr[C = c \mid M = m^*] = 0$. Therefore, for an adversary who sees c,

$$\Pr[M = m^* \mid C = c] = 0.$$

Now take a prior where m^* has nonzero probability, say the uniform prior over $\mathcal{M}$, so $\Pr[M = m^*] > 0$. We have just shown $\Pr[M = m^* \mid C = c] = 0 \neq \Pr[M = m^*]$. The posterior differs from the prior, contradicting perfect secrecy (Definition 9.2). Hence $|\mathcal{K}| \geq |\mathcal{M}|$. □

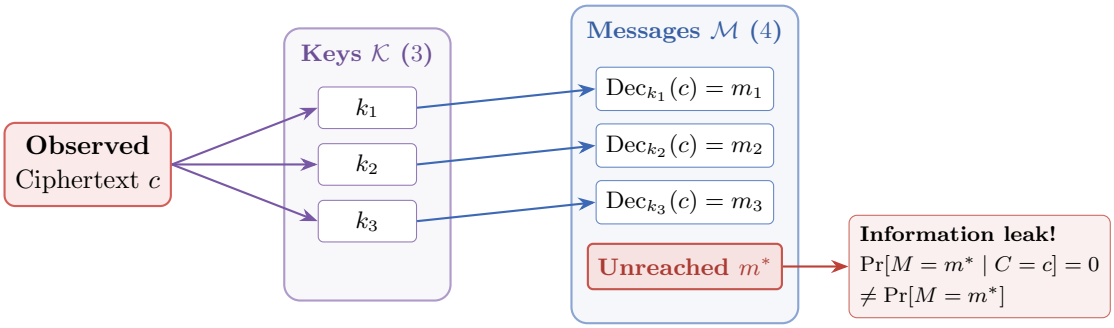


Figure 9.2: The pigeonhole argument behind Shannon's theorem (Theorem 9.2). If $|\mathcal{K}| < |\mathcal{M}|$, decrypting an observed ciphertext c under every possible key reaches at most $|\mathcal{K}|$ messages, leaving at least one message m^* with no explanatory key. Seeing c lets Eve rule out m^* with certainty, destroying perfect secrecy.

Intuition

The picture behind the proof: perfect secrecy demands that *every* message be a plausible explanation of *every* ciphertext. Otherwise some ciphertext would rule a message out, and ruling out is information. To keep every message in play for a given c, you need at least one key per message – a key that bends c back to that message. With fewer keys than messages, some message has no key pointing to it from c, and the instant Eve sees c she can cross that message off. Perfect secrecy allows no crossed-off messages, so there must be at least as many keys as messages.

Focus

To send up to 2^n different messages with perfect secrecy you need at least 2^n keys, hence an n-bit key – one fresh key bit for every message bit you will ever transmit. To protect a gigabyte you must have already shared a gigabyte of secret key, by some channel you trusted

before you needed the cipher. The cipher *transports* secrecy you already established rather than creating it, and this cost is unavoidable for any perfectly secret scheme.

Connection

This theorem is a direct consequence of Shannon's entropy bound applied to encryption. In the language of entropy (companion probability book), perfect secrecy is equivalent to $H(M \mid C) = H(M)$ – the ciphertext leaves the message's entropy untouched. Shannon's bound then follows from $H(K) \geq H(M)$: the key must carry at least as much uncertainty as the message it conceals. A short key has too little entropy to "absorb" a long message's uncertainty, and the leftover uncertainty has to show up somewhere – namely, as information leaking into the ciphertext. The key-length bound and the entropy inequality are the same fact stated in two ways.

9.7 The two-time pad: reuse is fatal

The "one" in "one-time pad" matters. If a single pad is reused on two messages, the scheme fails completely rather than degrading gracefully. This is the most common way the pad is broken in practice, and the attack is simple enough to work through bit by bit.

Attack

Setup. Alice encrypts two messages m_1 and m_2 with the *same* key k. Eve intercepts both ciphertexts:

$$c_1 = m_1 \oplus k, \qquad c_2 = m_2 \oplus k.$$

The break. Eve XORs the two ciphertexts together. The shared key cancels:

$$c_1 \oplus c_2 = (m_1 \oplus k) \oplus (m_2 \oplus k) = m_1 \oplus m_2 \oplus (k \oplus k) = m_1 \oplus m_2.$$

The key has vanished entirely (Figure 9.3). Eve now holds $m_1 \oplus m_2$ – the XOR of the two plaintexts, with no key involved. She has reduced an "unbreakable" cipher to the problem of separating two overlaid plaintexts, and that is a problem natural-language redundancy solves easily.

Why $m_1 \oplus m_2$ leaks the plaintexts. English (and any structured data) is far from random. In ASCII, every letter has its top bits fixed; space is `0x20`; XOR-ing a space onto a letter flips the case bit, so letter $\oplus$ space is just the same letter with flipped case. Eve does "crib dragging": she guesses a probable word like `"the "` appears in m_1 at some position, XORs that guess into $c_1 \oplus c_2$ at that position, and reads off the corresponding slice of m_2. If the result is intelligible text, the guess was right and she has just recovered a chunk of *both* messages at once. She slides the crib along, extends the readable region, and unzips the two messages letter by letter. With three or more reuses the job is trivial.

Example 9.1 (A reuse break in miniature)**.** Work with 6-bit "messages" and watch only the XOR structure. Say

$$m_1 = \texttt{101100}, \quad m_2 = \texttt{110010}, \quad k = \texttt{011011}.$$

Then

$$c_1 = m_1 \oplus k = \texttt{110111}, \qquad c_2 = m_2 \oplus k = \texttt{101001}.$$

Eve never learns k, but she computes

$$c_1 \oplus c_2 = \texttt{110111} \oplus \texttt{101001} = \texttt{011110}.$$

Check against the plaintexts directly: $m_1 \oplus m_2 = \texttt{101100} \oplus \texttt{110010} = \texttt{011110}$. They match – the key is gone. If Eve now learns or guesses even one of the messages, say she cribs $m_1 = \texttt{101100}$, she instantly gets the other: $m_2 = (c_1 \oplus c_2) \oplus m_1 = \texttt{011110} \oplus \texttt{101100} = \texttt{110010}$. One leaked message reveals the other, with no key recovery needed.

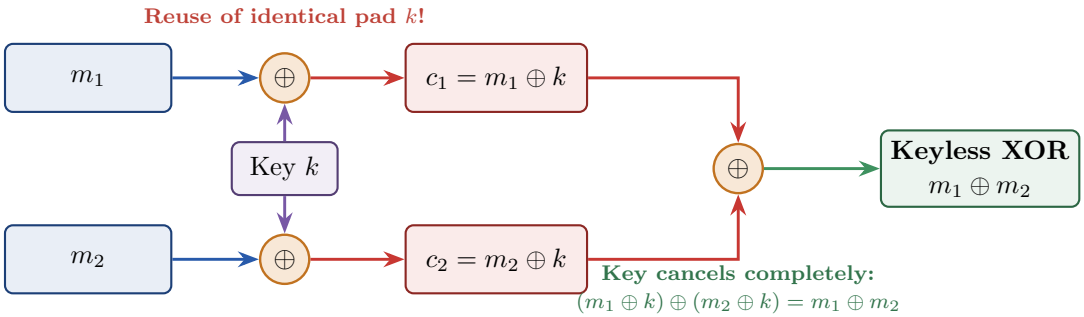


Figure 9.3: The two-time pad attack. When the same key k is used to encrypt two distinct messages m_1 and m_2, Eve XORs the two ciphertexts together. Because $k \oplus k = 0$, the secret key cancels entirely, leaving the raw bitwise sum $m_1 \oplus m_2$ exposed.

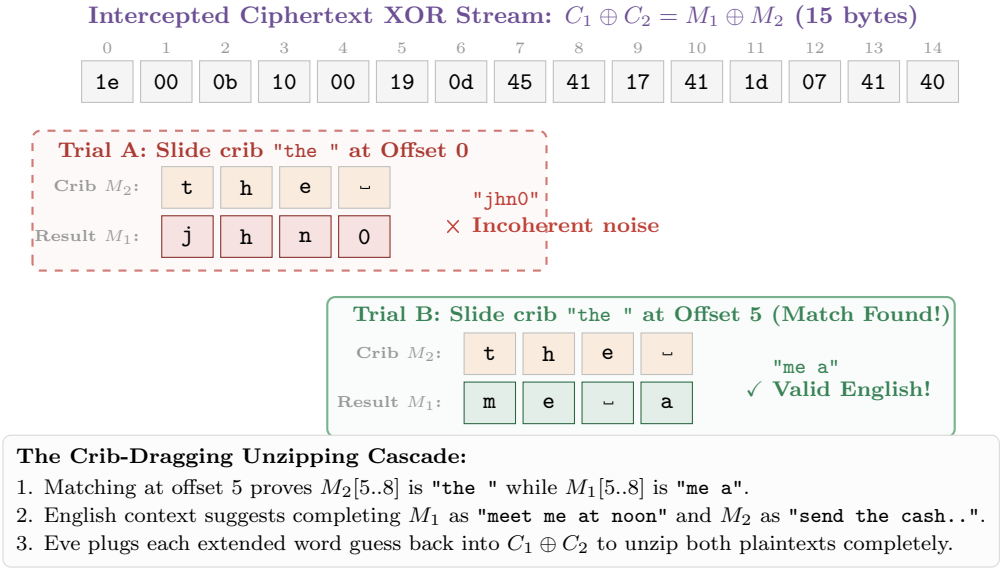


Figure 9.4: Crib dragging against a two-time pad. Eve intercepts $C_1 \oplus C_2 = M_1 \oplus M_2$. She slides a common English crib (`"the "`) along the stream and XORs it with the bytes. At wrong offsets (Trial A), the result is gibberish like `"jhn0"`. At the correct offset 5 (Trial B), valid English plaintext (`"me a"`) emerges in M_1, confirming the guess and revealing both plaintexts at that position.

Attack

[Historical case study: VENONA and pad reuse] During and after World War II, Soviet agents encrypted cables with one-time pads. Under wartime pressure, the pad-printing operation duplicated some pages of key material – the same pad used on more than one message. U.S. and British cryptanalysts (the VENONA project, 1943–1980) detected the reuse and exploited the $c_1 \oplus c_2 = m_1 \oplus m_2$ collapse above, separating overlaid plaintexts through crib dragging and frequency analysis. The decrypts exposed espionage networks, including the leak of atomic-bomb secrets. A theoretically perfect cipher was defeated not by breaking the math but by an operational shortcut that violated the "one-time" rule.

Attack

A modern case: WEP and stream-cipher reuse. The same break sank Wi-Fi's first security protocol. WEP used the RC4 stream cipher – effectively a pad generated from a key plus a 24-bit "initialization vector" (IV). The IV is sent in the clear and is supposed to make each packet's pad unique. But 24 bits is tiny: on a busy network the IV cycles in hours, and by the birthday bound (companion probability book) collisions appear much sooner. Two packets with the same IV reuse the same pad, and an eavesdropper recovers $p_1 \oplus p_2$ exactly as above. Known plaintext structure in network packets (fixed headers, predictable handshake bytes) gives the cribs for free. WEP thus suffered the two-time pad attack at scale, which is why it was abandoned. The repair, carried through the rest of this book, is the *nonce*: a per-message value guaranteed never to repeat, so a key plus nonce never produces the same keystream twice.

Common trap

Ideas such as "I will just hash the key differently each time" or "I will append a counter to the message" define new ciphers, and each needs its own security analysis. With a genuine one-time pad there is no patch for reuse: the only correct behavior is to advance to fresh, never-before-used key material for every message, exactly $|m|$ bits of it. The moment you find yourself wanting to reuse pad bytes "just this once," you have outgrown the one-time pad and need a stream or block cipher with a nonce.

9.8 Why we settle for computational security

Consider the total cost. Shannon's bound (Theorem 9.2) says a perfectly secret scheme needs a key as long as the total volume of traffic it will ever protect. That key must be uniformly random, transported to both parties over an already-secure channel before any communication, kept perfectly secret, and *never* reused under penalty of total collapse (Section 9.7). For two people sharing a suitcase of one-time tape ahead of a mission, fine. For your browser opening a TLS connection to a server it has never met, having shared no prior secret, intending to exchange megabytes – impossible. You cannot pre-share a gigabyte of perfect key with every website on Earth.

Modern cryptography therefore makes a trade. We give up *perfect* secrecy and ask instead for *computational* secrecy: the ciphertext may, in principle, contain information about the message, but extracting it requires solving a problem so hard that no realistic adversary – bounded to, say, 2^{128}

operations over the lifetime of the universe – can do it.

Focus

The two parts of the relaxation, in contrast to the perfect-secrecy bullets above:

- **Bounded adversary.** We stop worrying about a computationally unlimited Eve and only defend against a probabilistic polynomial-time (PPT) one. An unbounded adversary could break our ciphers; no physically realizable machine can.
- **Negligible advantage.** We allow the adversary a *tiny* probability of success – but tiny in the strong sense of *negligible* (negl): smaller than $1/p(n)$ for every polynomial p, as the security parameter n grows. Doubling the key size makes the break time explode while the adversary's success chance vanishes.

In exchange for these two concessions we get what the pad could not provide: short keys (a 256-bit key protecting terabytes), key reuse under proper nonce discipline, and – once we add public-key methods – secure communication with strangers we never pre-shared a secret with.

Why it works

Is the trade safe? It is conditional. Perfect secrecy needs no assumptions – it is a theorem. Computational secrecy rests on *unproven* hardness assumptions: that factoring large integers is slow, that the AES function behaves like a random permutation, that certain discrete logarithms are intractable. If one of those assumptions falls, the schemes built on it fall too. We accept this because the alternative – perfect secrecy – is unusable at scale, and because the assumptions have survived decades of the world's best cryptanalysts trying to break them. The companion algorithms book quantifies exactly what "2^{128} operations" buys you and why doubling a key length squares the attacker's work. The rest of *this* book is about building secure systems within that trade.

In the real world

Where the pad survives, and where computational security takes over.

- **One-time pad, still used:** diplomatic and intelligence links where couriers can pre-distribute key tape and the secrecy requirement is absolute and indefinite. Numbers stations – shortwave broadcasts of spoken digit groups – are widely understood to deliver one-time-pad key material to field agents. Quantum key distribution (QKD) is, in effect, a way to grow fresh shared random bits for use as a pad.
- **Computational security, everywhere else:** TLS (the `https` lock), the WPA2/WPA3 that replaced broken WEP, Signal's end-to-end messaging, SSH, disk encryption, your phone. None of these pre-share a message-length key; all of them rely on bounded adversaries and unproven hardness. They give up Shannon's perfection and in return scale to the entire Internet.

The one-time pad is the ideal that is too expensive to use; everything that follows is a practical substitute that no adversary limited to 2^{128} operations can tell apart from it.

9.9 What to remember

Perfect secrecy means the ciphertext is statistically independent of the message: posterior equals prior, for every prior, against an adversary of unlimited power (Definition 9.2). The one-time pad achieves it because a uniform key masks any message into uniform noise whose distribution forgets which message it was (Theorem 9.1), and the masking lemma (Lemma 9.2) is the fact the whole construction depends on.

The price is set by Shannon's bound (Theorem 9.2): the key must be at least as long as the message, for *any* perfectly secret scheme, no exceptions, no clever encodings. The key must also be uniformly random and used exactly once – reuse collapses two ciphertexts to $m_1 \oplus m_2$ and hands the plaintexts to anyone (Section 9.7), as the VENONA and WEP cases showed.

Because that price is unpayable for Internet-scale communication, we descend from perfect secrecy to computational secrecy: bounded adversaries, negligible advantage, short reusable keys, and security resting on hardness assumptions instead of theorems. In Chapter 10, we make our first practical trade: replacing the infinite random key tape of the one-time pad with an algorithm that stretches a short seed into a pseudorandom keystream – the stream cipher.

Exercises

Exercise 9.1 (XOR warm-up)**.** Let $m = \texttt{11010010}$ and $k = \texttt{01100101}$. Compute $c = m \oplus k$. Then verify $c \oplus k = m$. Finally, suppose Eve learns that the ciphertext is c and the message is m; show she can compute the key, and explain in one sentence why this does not violate perfect secrecy.

Exercise 9.2 (Every message is reachable)**.** Fix a one-time-pad ciphertext $c \in \{0,1\}^n$. Prove that the map $m \mapsto m \oplus c$ is a bijection on $\{0,1\}^n$, and explain how this bijection is exactly the statement that every candidate message has exactly one explanatory key. Relate this to the counting step in the proof of Shannon's bound (Theorem 9.2).

Exercise 9.3 (Posterior equals prior, by hand)**.** Let $\mathcal{M} = \{0,1\}^2$ and suppose Eve's prior is $\Pr[M = \texttt{00}] = \frac{1}{2}$, $\Pr[M = \texttt{01}] = \frac{1}{4}$, $\Pr[M = \texttt{10}] = \frac{1}{8}$, $\Pr[M = \texttt{11}] = \frac{1}{8}$. Using a one-time pad with a uniform 2-bit key, compute $\Pr[M = \texttt{00} \mid C = \texttt{10}]$ from first principles with Bayes' rule, and confirm it equals the prior $\frac{1}{2}$.

Exercise 9.4 (Break a two-time pad)**.** You intercept two ciphertexts encrypted under the *same* 8-bit pad:

$$c_1 = \texttt{10011100}, \qquad c_2 = \texttt{11001010}.$$

Compute $c_1 \oplus c_2$. You are then told (a crib) that $m_1 = \texttt{01000001}$. Recover m_2 without ever learning the key, and state the general formula you used.

Exercise 9.5 (The bias leak)**.** An implementation generates each key bit independently as 1 with probability $p \neq \frac{1}{2}$. For a one-bit message, compute $\Pr[M = 0 \mid C = 1]$ under a uniform prior $\Pr[M = 0] = \Pr[M = 1] = \frac{1}{2}$, as a function of p. Show it equals $\frac{1}{2}$ only when $p = \frac{1}{2}$, and interpret what a deviation tells the adversary.

Exercise 9.6 (A scheme that is not perfectly secret)**.** Define a "shift cipher" over $\mathcal{M} = \mathbb{Z}/26\mathbb{Z}$: Gen picks $k \xleftarrow{\$} \mathbb{Z}/26\mathbb{Z}$, $\mathrm{Enc}_k(m) = (m + k) \bmod 26$. Prove that for a *single* letter this scheme *is* perfectly secret. Then argue that for two-letter messages drawn from English (so M is non-uniform),

encrypting both letters with the same k is no longer perfectly secret, and exhibit a ciphertext-message pair whose posterior differs from its prior.

Exercise 9.7 (Shannon's bound is tight)**.** The one-time pad meets Shannon's bound with equality: $|\mathcal{K}| = |\mathcal{M}| = 2^n$. Construct a *different* perfectly secret scheme over $\mathcal{M} = \mathbb{Z}/q\mathbb{Z}$ (any modulus q, not just a power of two) with $|\mathcal{K}| = |\mathcal{M}| = q$, prove it perfectly secret, and confirm it too meets the bound with equality. (Hint: replace XOR by addition modulo q.)

Exercise 9.8 (Why pre-sharing is the catch)**.** Estimate the amount of one-time-pad key material two people would need to pre-share to protect one year of typical messaging traffic (pick and state a reasonable bytes-per-day figure). Compare this to the 32 bytes of a typical symmetric key in a computationally secure scheme. Use the contrast to explain, in a short paragraph, why TLS cannot use a one-time pad.

Exercise 9.9 (Perfect secrecy via entropy)**.** Using the entropy formulation from the companion probability book, perfect secrecy is equivalent to $H(M \mid C) = H(M)$. Assuming this, prove $H(K) \geq H(M)$, and explain how this recovers Shannon's key-length bound. (Hint: the message is a deterministic function of the key and ciphertext together, so $H(M \mid C) \leq H(K \mid C) \leq H(K)$.)

Exercise 9.10 (Malleability of the pad)**.** The one-time pad is perfectly *secret* but offers no *integrity*. Suppose Eve knows that m is an account number and that the ciphertext is $c = m \oplus k$. Show that by XOR-ing a chosen mask δ into c, Eve can flip specific bits of the decrypted message to a value of her choosing, *without knowing* k. Give a concrete 4-bit example, and explain why this means perfect secrecy does not imply security against active tampering – motivating message authentication in Chapter 15.

Chapter 10

Stream Ciphers

In Chapter 9, the one-time pad gave us a perfect cipher and immediately took it away: Shannon's bound demands a key as long as the message, used once and drawn from true randomness. Alice cannot hand Bob a terabyte of pre-shared key every time she wants to send him a terabyte of video.

This chapter covers the compromise that most internet encryption uses. We keep the XOR structure of the one-time pad, but manufacture the keystream from a short key using a deterministic algorithm – a *pseudorandom generator* – trading unconditional secrecy for security against any efficient adversary. We formalize pseudorandom generators, meet RC4 (everywhere for two decades, now broken and retired) and ChaCha20 (the modern default in TLS, WireGuard, and Linux), see how keystream reuse broke WEP, and analyze a tiny linear-feedback shift register to show why a linear keystream is insecure.

10.1 From the one-time pad to a generator

Recall the one-time pad. Plaintext m and key k are bitstrings of the same length ℓ. Encryption and decryption are both exclusive-or:

$$c = m \oplus k, \qquad m = c \oplus k.$$

The strength of the pad is that if k is uniformly random and used once, then c is uniformly random and independent of m: the ciphertext leaks nothing. The catch is that k must be as long as m.

Intuition

The idea is this: what if we did not need a *truly* random k of length ℓ? What if a key that merely *looks* random to any efficient adversary were good enough? Then we could keep a short secret seed – say 256 bits – and stretch it into a keystream of length ℓ with a public algorithm G. Alice and Bob both know the seed, so they both compute the same long string $G(\text{seed})$ and xor it in. Eve knows the algorithm G but not the seed, and if G is well built she cannot tell its output from coin flips.

The deterministic function that does the stretching is the pseudorandom generator.

Definition 10.1 (Pseudorandom generator, PRG)**.** A *pseudorandom generator* is a deterministic,

efficiently computable function

$$G : \{0,1\}^s \to \{0,1\}^{\ell(s)}, \qquad \ell(s) > s,$$

that maps a short *seed* of s bits to a longer *output* of $\ell(s)$ bits. The quantity $\ell(s) - s$ is the *stretch*.

A stream cipher is then the one-time pad with the pad replaced by PRG output.

Definition 10.2 (Stream cipher from a PRG)**.** Let $G : \{0,1\}^s \to \{0,1\}^\ell$ be a PRG. Define

$$\mathrm{Enc}_k(m) = m \oplus G(k), \qquad \mathrm{Dec}_k(c) = c \oplus G(k),$$

for $m \in \{0,1\}^\ell$ and key $k \in \{0,1\}^s$. The string $G(k)$ is called the *keystream*.

Correctness is immediate, exactly as for the pad: $\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = (m \oplus G(k)) \oplus G(k) = m$, because $x \oplus x = 0$.

Focus

The *operation* on the data is identical to the pad – a bitwise xor with a pad. The *source* of the pad changed from a true-random tape to a deterministic expansion of a short seed. The hard questions about stream ciphers all concern that substitution: how good is G, and what happens if you ever reuse $G(k)$.

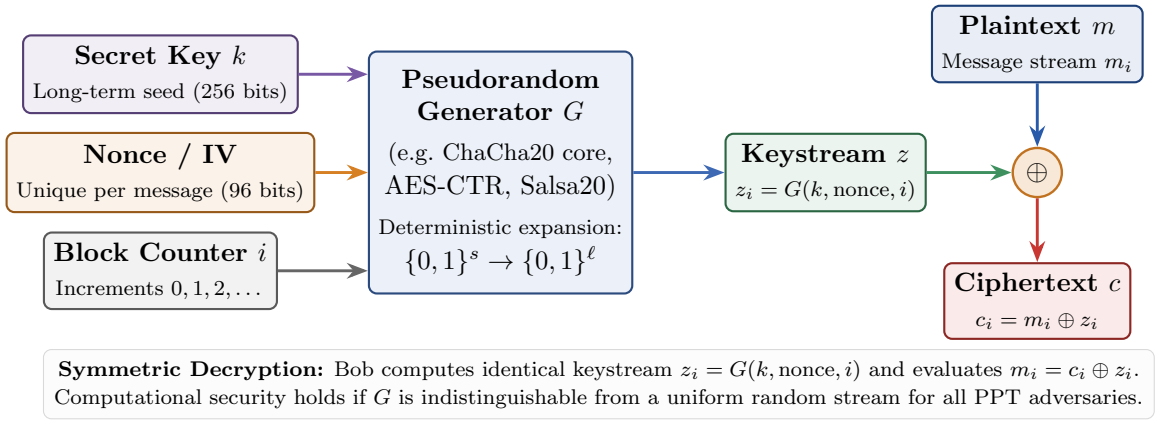


Figure 10.1: The stream cipher architecture. A short secret key k, unique nonce, and block counter are expanded by a deterministic pseudorandom generator G into an arbitrary-length keystream z. Encryption simply XORs the keystream with the plaintext stream. Decryption is identical: XOR-ing z into ciphertext c cancels the keystream and recovers m.

10.1.1 Why this cannot be perfectly secret

Before we ask what security we *do* get, let us be clear about what we lost. A stream cipher with a 256-bit key and a 1-gigabyte message cannot be perfectly secret. The reason is a counting argument that does not depend on the design of G at all.

Why it works

Shannon's theorem says perfect secrecy requires the key space to be at least as large as the message space. Here there are at most 2^{256} keys but $2^{8 \cdot 10^9}$ possible gigabyte messages. So the encryption map cannot reach every message from a given ciphertext: most messages are simply *impossible* decryptions of a fixed c. An unbounded adversary who sees c could, in principle, try all 2^{256} seeds, generate each keystream, xor, and look at which plaintexts result – only 2^{256} of the $2^{8 \cdot 10^9}$ candidates survive. That leaks information. Perfect secrecy is impossible whenever the key is shorter than the message.

So we are not aiming for perfect secrecy. We are aiming for *computational* secrecy: secure against adversaries that run in feasible time. Everything that follows rests on one assumption – that good pseudorandom generators exist – which we now state precisely.

10.2 What we demand of a PRG

The intuitive requirement is "the output looks random." Cryptography makes that precise with a game played by a distinguisher D. With probability one half we feed it a true random string of length ℓ; with probability one half we feed it $G(k)$ for a uniformly random seed k. The distinguisher prints a guess: "random" or "pseudorandom." If D can do appreciably better than a coin flip, it has *distinguished* G from random, and G has failed.

Definition 10.3 (PRG security)**.** A PRG $G : \{0,1\}^s \to \{0,1\}^\ell$ is *secure* if for every PPT distinguisher D,

$$\mathsf{Adv}_{\mathrm{prg}}(D) \;=\; \Big|\; \Pr_{k \xleftarrow{\$} \{0,1\}^s}\big[D(G(k)) = 1\big] \;-\; \Pr_{r \xleftarrow{\$} \{0,1\}^\ell}\big[D(r) = 1\big] \;\Big| \;\le\; \mathsf{negl}(s).$$

The left probability is over a random seed; the right is over a truly random string. Their difference is the distinguisher's edge. "negl" means the edge shrinks faster than any inverse polynomial as the seed grows: lengthening the seed should quickly drive any feasible attacker's edge toward zero.

Common trap

A PRG is *not* merely an algorithm whose output passes statistical tests like Diehard or the NIST suite. Those test for specific biases – a skewed bit frequency, a short period, correlated bytes. A cryptographic PRG must defeat *every* efficient test, including ones nobody has invented yet, and including tests that know the algorithm G and are searching for its fingerprint. Passing chi-squared is necessary, never sufficient. Many badly broken generators pass statistical test suites.

10.2.1 Next-bit unpredictability

There is an equivalent and more operational way to think about PRG security, due to Yao and to Blum and Micali. Instead of asking "can you tell the whole output from random," ask "given a prefix of the output, can you predict the very next bit?"

Definition 10.4 (Next-bit test)**.** G passes the *next-bit test* if for every PPT predictor A and every position i, given the first i output bits $g_1 g_2 \cdots g_i$, the probability that A correctly outputs g_{i+1} is at most $\frac{1}{2} + \mathsf{negl}(s)$.

Theorem 10.1 (Yao)**.** *A PRG is secure (Definition 10.3) if and only if it passes the next-bit test.*

The "only if" direction is easy: a successful next-bit predictor is itself a distinguisher (a random string has unpredictable next bits, so beating one half on prediction means you can tell G apart). The "if" direction – that defeating every next-bit predictor implies defeating every distinguisher – is the substantive half, proved by a *hybrid argument*.

Why it works

The hybrid argument is worth seeing once, because the same structure appears throughout cryptography. Build a ladder of distributions $H_0, H_1, \ldots, H_\ell$. Hybrid H_j outputs the first j bits from a truly random string and the remaining $\ell - j$ bits from G. Then H_0 is pure G output and H_ℓ is pure random. If some distinguisher D separates H_0 from H_ℓ with advantage ε, then because there are ℓ steps in the ladder, *some* adjacent pair H_j, H_{j+1} must be separated with advantage at least ε/ℓ. But those two hybrids differ in exactly one bit – position $j{+}1$ is real-random in one and PRG-generated in the other. A distinguisher that notices the difference is, after a little massaging, predicting that one bit better than chance. So a whole-string distinguisher with edge ε yields a next-bit predictor with edge about ε/ℓ. Contrapositive: kill all next-bit predictors and you have killed all distinguishers, up to a factor of ℓ.

Connection

The hybrid argument is a probabilistic telescoping sum: $\Pr[D(H_0)] - \Pr[D(H_\ell)] = \sum_j \big(\Pr[D(H_j)] - \Pr[D(H_{j+1})]\big)$, and if a sum of ℓ terms exceeds ε, one term exceeds ε/ℓ by pigeonhole. The companion probability volume develops this averaging-over-a-telescope pattern; it is the same move behind union bounds and the triangle inequality for statistical distance.

10.2.2 Security of the stream cipher, granted a good PRG

With a secure PRG in hand, the stream cipher inherits the right notion of secrecy for a single message: semantic security, equivalently ciphertext indistinguishability against an eavesdropper.

Theorem 10.2 (Stream-cipher security)**.** *If G is a secure PRG, then the stream cipher of Definition 10.2 is semantically secure against a passive eavesdropper for a* single *encryption.*

Why it works

The proof is a one-line reduction. Suppose an adversary $\mathcal{A}$ breaks the cipher – distinguishes $\mathrm{Enc}_k(m_0)$ from $\mathrm{Enc}_k(m_1)$ for chosen m_0, m_1. Build a PRG distinguisher D: on input a string p (either $G(k)$ or random), pick a random bit b, hand $\mathcal{A}$ the ciphertext $m_b \oplus p$, and output 1 iff $\mathcal{A}$ guesses b. If $p = G(k)$, then $\mathcal{A}$ sees a real stream-cipher ciphertext and wins with whatever edge it had. If p is truly random, then $m_b \oplus p$ is a one-time-pad ciphertext, perfectly hiding b, so $\mathcal{A}$ is guessing – edge zero. Thus D's advantage equals $\mathcal{A}$'s. If G is secure, D's advantage is negligible, so $\mathcal{A}$'s is too. The cipher is as secure as the generator.

Note the restriction in Theorem 10.2: a *single* encryption. Reuse the same keystream on a second message and the proof collapses, because the reduction relied on p padding exactly one ciphertext. The next section shows what happens when this restriction is ignored.

10.3 Keystream reuse: the two-time pad disaster

Encrypt two messages with the *same* keystream $z = G(k)$:

$$c_1 = m_1 \oplus z, \qquad c_2 = m_2 \oplus z.$$

Eve xors the two ciphertexts. The keystream cancels:

$$c_1 \oplus c_2 = (m_1 \oplus z) \oplus (m_2 \oplus z) = m_1 \oplus m_2.$$

The key is gone, and Eve holds the xor of two plaintexts. This is the same trap as reusing a one-time pad – the "two-time pad" – and it is just as fatal here, because nothing about G helps once the keystream cancels. Figure 10.2 shows the cancellation.

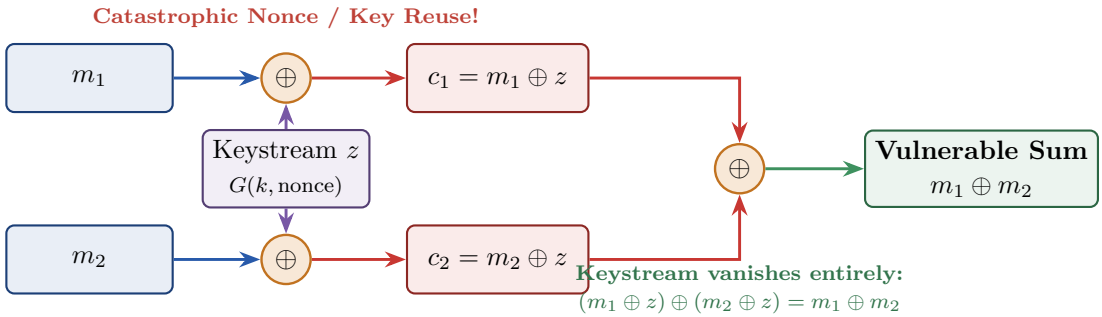


Figure 10.2: Keystream reuse. Both ciphertexts carry the exact same pseudorandom pad $z = G(k, \text{nonce})$. XOR-ing the intercepted ciphertexts deletes z ($z \oplus z = 0$) and leaves the raw plaintext sum $m_1 \oplus m_2$ exposed to crib dragging, so the cipher's computational strength no longer matters.

Intuition

$m_1 \oplus m_2$ does not look like much, but English (or HTTP, or a JPEG header) is extremely redundant. Where one message has a space character (`0x20`) and the other has a lowercase letter, the xor flips the case bit and reveals structure. Knowing or guessing a stretch of m_1 instantly hands you the same stretch of m_2, since $m_2 = (m_1 \oplus m_2) \oplus m_1$. This is the classic "crib-dragging" attack: slide a likely word like `" the "` along $m_1 \oplus m_2$; wherever it produces sensible text in the other message, you have recovered part of both messages. With three or more ciphertexts under one keystream, recovery is usually total.

Example 10.1 (A four-bit two-time pad by hand)**.** Take a toy keystream $z = 1011$ and two messages $m_1 = 1100$, $m_2 = 0110$.

$$c_1 = 1100 \oplus 1011 = 0111, \qquad c_2 = 0110 \oplus 1011 = 1101.$$

Eve never learns z, but she computes

$$c_1 \oplus c_2 = 0111 \oplus 1101 = 1010 = m_1 \oplus m_2.$$

Check: $m_1 \oplus m_2 = 1100 \oplus 0110 = 1010$. If Eve guesses $m_1 = 1100$ (maybe it is a known protocol header), she recovers $m_2 = 1010 \oplus 1100 = 0110$ exactly. The keystream's quality was irrelevant; only its *reuse* mattered.

Attack

WEP: keystream reuse at industrial scale. Wired Equivalent Privacy, the original 1999 Wi-Fi encryption, used RC4. RC4 takes a single key, so to encrypt many packets under one shared Wi-Fi password WEP prepended a 24-bit *initialization vector* (IV) to the key, sent the IV in the clear, and ran RC4 on IV $\|$ key. The keystream for a packet is therefore determined by the IV.

Twenty-four bits is only about 16.7 million IVs. On a busy access point sending thousands of packets per second, the birthday bound guarantees IV *collisions* within hours, often minutes – and many cards reset the IV to zero on reboot, colliding immediately. Two packets with the same IV use the *same keystream*: a two-time pad. Wi-Fi traffic is full of known plaintext (ARP packets, IP/LLC headers like `0xAA AA 03 00 00 00`), so the attacker xors colliding packets, subtracts the known header, and reads traffic.

A second weakness was worse. The structure of RC4's key schedule, combined with attacker-chosen IVs, let Fluhrer, Mantin, and Shamir (2001) recover the *actual WEP key* from enough collected IVs – not just decrypt individual packets. Tools like `aircrack-ng` cracked 104-bit WEP keys in minutes. WEP is the textbook case of a sound primitive (RC4 was fine in 1999) destroyed by a broken *construction* around it: a too-short IV concatenated into the key, with no separation between IV and secret. WEP was deprecated in 2004 (WPA, then WPA2).

Focus

The lesson applies well beyond Wi-Fi. **Never reuse a keystream.** For a real stream cipher this means: never feed the same (key, nonce) pair twice. The key is long-lived; the *nonce* (number used once) is what guarantees a fresh keystream per message. Get nonce management wrong – a counter that resets, a random nonce that collides, a nonce reused across reboots – and you are back to a two-time pad no matter how strong the cipher is.

10.4 RC4: simple, ubiquitous, and broken

History

RC4 ("Rivest Cipher 4") was designed by Ron Rivest at RSA Security in 1987 for fast software execution on 8- and 32-bit microprocessors, using a dynamic 256-byte permutation array indexed by two moving pointers. RSA Security kept it as a trade secret until September 1994, when its source was anonymously posted to the Cypherpunks mailing list; the leaked algorithm matched RC4's behavior, and for legal reasons the unlicensed version was called "ARCFOUR." Because it took just a few lines of C with no precomputed constant tables, it went on to encrypt a huge fraction of internet traffic – WEP, WPA-TKIP, SSL/TLS, and more. Subtle statistical biases in its keystream eventually broke it in practice, and RFC 7465 prohibited it in TLS in 2015, leaving modern software stream encryption to constant-time designs like ChaCha20.

RC4 keeps a secret internal state: a permutation S of the 256 bytes $0 \ldots 255$, plus two indices i and j. It has two phases.

Key-scheduling algorithm (KSA). Initialize S to the identity permutation $S[k] = k$, then shuffle it using the key:

$$
\begin{aligned}
&\text{for } i = 0 \text{ to } 255: \;\; S[i] \leftarrow i \\
&j \leftarrow 0 \\
&\text{for } i = 0 \text{ to } 255: \\
&\qquad j \leftarrow (j + S[i] + \text{key}[i \bmod \text{keylen}]) \bmod 256 \\
&\qquad \text{swap } S[i], S[j]
\end{aligned}
$$

Pseudorandom generation (PRGA). Emit one keystream byte per step:

$$
\begin{aligned}
&i \leftarrow 0, \;\; j \leftarrow 0 \\
&\text{each step:} \\
&\qquad i \leftarrow (i + 1) \bmod 256 \\
&\qquad j \leftarrow (j + S[i]) \bmod 256 \\
&\qquad \text{swap } S[i], S[j] \\
&\qquad \text{output } S[(S[i] + S[j]) \bmod 256]
\end{aligned}
$$

Each output byte is xored into the plaintext. That is the whole cipher.

Implement it

RC4 in a few lines of Python – learning only. This is a teaching toy. RC4 is broken; do not use it for anything real, and do not copy this into production. It is here so you can trace the bytes.

```python
def rc4(key, data):
    # Key-scheduling algorithm (KSA): shuffle S with the key.
    S = list(range(256))
    j = 0
    for i in range(256):
        j = (j + S[i] + key[i % len(key)]) % 256
        S[i], S[j] = S[j], S[i]
    # Pseudorandom generation (PRGA): emit keystream, xor into data.
    out = bytearray()
    i = j = 0
    for byte in data:
        i = (i + 1) % 256
        j = (j + S[i]) % 256
        S[i], S[j] = S[j], S[i]
        k = S[(S[i] + S[j]) % 256]   # one keystream byte
        out.append(byte ^ k)
    return bytes(out)

key = b"Key"
ct = rc4(key, b"Plaintext")
print(ct.hex())                      # bbf316e8d940af0ad3
```

```
print(rc4(key, ct))                    # b'Plaintext'  (xor is its own inverse)
```

Encryption and decryption are the same function, because the cipher only ever xors keystream into its input. The published test vector for key `"Key"`, plaintext `"Plaintext"` is ciphertext `BBF316E8D940AF0AD3`; running the code reproduces it.

10.4.1 Why RC4 is broken

RC4's downfall is that its keystream is *not* indistinguishable from random – the early bytes carry measurable biases, and the key schedule leaks key bits when an IV is glued to the key.

Attack

The biases that killed RC4.
Second-byte bias (Mantin–Shamir, 2001). The second output byte of RC4 equals 0 with probability about 2/256, twice what a uniform byte should be. This single bias means: if the same plaintext byte (say, a fixed HTTP cookie character) is encrypted under many random keys, the attacker collects the second ciphertext byte across sessions and recovers the plaintext byte by statistics alone. If the same secret is encrypted often enough, it can be recovered.
Fluhrer–Mantin–Shamir (FMS), 2001. When RC4's key is $\mathrm{IV} \,\|\, K$ with a known IV (exactly the WEP setup), certain "weak" IVs cause the first keystream byte to reveal information about K. Collect enough packets with such IVs and you solve for K byte by byte. This is the attack behind practical WEP cracking.
The full break of RC4 in TLS (AlFardan et al., 2013). Mapping out the biases across the first 256 bytes (and the long-term Fluhrer–McGrew digraph biases) let researchers recover TLS-protected secrets – like session cookies – given on the order of 2^{30} encryptions of the same plaintext, achievable by a malicious web page forcing a browser to send the cookie repeatedly. This pushed the industry to ban RC4 outright.

Common trap

A common "fix" for RC4 is to throw away the first N keystream bytes (RC4-drop$N$, e.g. $N = 768$ or 3072), which discards the worst early biases. It helps but does not save RC4: long-term biases remain, and the key-schedule weaknesses persist. The right response was not to patch RC4 but to replace it. When a primitive needs a growing list of caveats to stay safe, retire it.

10.5 ChaCha20: the modern stream cipher

In the real world

Daniel J. Bernstein designed Salsa20 in 2005 for the eSTREAM competition, then refined it into ChaCha20 in 2008 with better diffusion per round. ChaCha20 is now the standard choice wherever a stream cipher is needed: TLS 1.3 (as ChaCha20-Poly1305), the WireGuard VPN, OpenSSH, the Linux kernel's `getrandom` CSPRNG, and mobile devices where CPUs lack AES

hardware. It is fast, constant-time by construction (no secret-dependent table lookups – the very thing that exposes AES software to cache-timing attacks), and has resisted nearly two decades of cryptanalysis.

Unlike RC4, ChaCha20 is built the modern way: as a keystream generator that is really a *counter mode* over a fixed pseudorandom permutation-like core. It does not iterate a tiny secret state; instead, for each 64-byte block of keystream it runs a fresh computation on a public-structure input made of the key, a nonce, and a block counter.

10.5.1 The state and the quarter-round

ChaCha20 works on a 4×4 matrix of sixteen 32-bit words – 512 bits of state, laid out as in Figure 10.3.

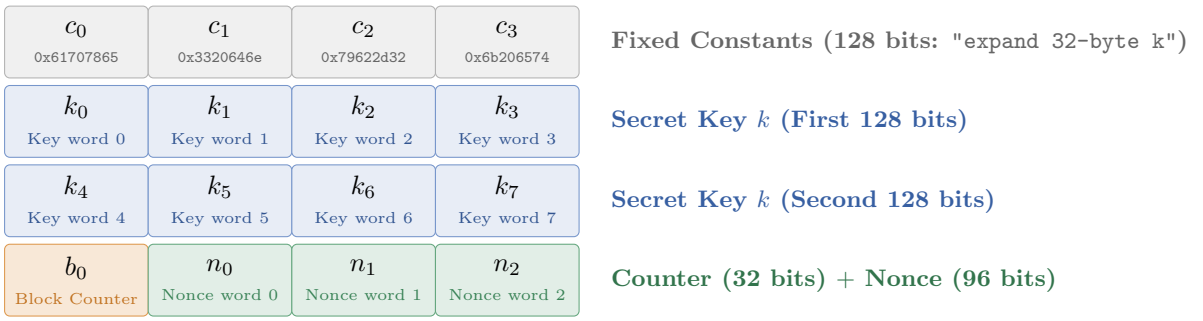


Figure 10.3: The ChaCha20 4×4 state matrix of sixteen 32-bit words (512 bits total). Row 0 holds fixed constants; rows 1 and 2 store the 256-bit secret key; row 3 contains a 32-bit incrementing block counter and a 96-bit per-message nonce. Only the key is secret; the counter ensures that every 64-byte keystream block generated under the same key and nonce is completely distinct.

Everything except the key is public and structured. Incrementing the counter b_0 between blocks gives each 64-byte block its own keystream.

The basic operation is the *quarter-round* on four words a, b, c, d, built only from addition mod 2^{32}, xor, and bitwise rotation. The "ARX" name – Add, Rotate, Xor – describes the entire primitive:

$$\begin{aligned} a \mathrel{+}= b; \quad d \mathrel{\oplus}= a; \quad d \lll 16 \\ c \mathrel{+}= d; \quad b \mathrel{\oplus}= c; \quad b \lll 12 \\ a \mathrel{+}= b; \quad d \mathrel{\oplus}= a; \quad d \lll 8 \\ c \mathrel{+}= d; \quad b \mathrel{\oplus}= c; \quad b \lll 7 \end{aligned}$$

Here $\lll r$ is a left rotation by r bits and "+" is addition modulo 2^{32}.

Intuition

Why these three operations? Addition mod 2^{32} mixes bits with *carries* that propagate from low to high – a cheap source of nonlinearity (carrying is not a linear function over $\mathbb{F}_2$). Xor combines words without carries. Rotation moves information *across* the 32-bit word so that a change in a low bit can affect high bits and vice versa. Alone, each is weak; interleaved over many rounds they produce avalanche: flip one input bit and roughly half the output bits flip,

unpredictably. No lookup tables means no data-dependent memory access, so no cache-timing side channel. That is why ChaCha is fast *and* naturally constant-time.

A full round applies the quarter-round to all four columns of the matrix; the next round applies it to the four diagonals (Figure 10.4). ChaCha20 does 20 such rounds (10 column-rounds interleaved with 10 diagonal-rounds), hence the "20." After the rounds, the result is added word-wise to the *original* input matrix (this feed-forward prevents the round function from being inverted to recover the state). Serializing the sixteen words little-endian yields 64 keystream bytes.

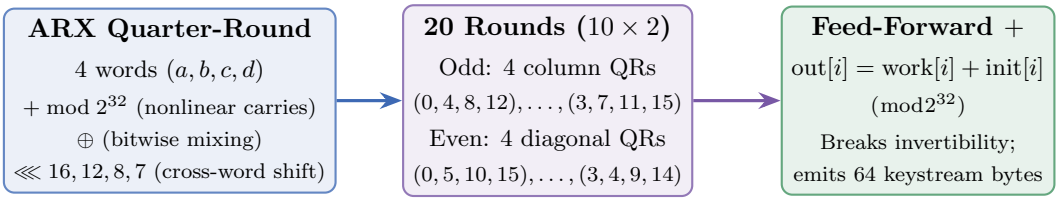


Figure 10.4: Inside the ChaCha20 block function. Each quarter-round mixes four 32-bit words using only addition modulo 2^{32}, XOR, and fixed rotations (ARX). Alternating column rounds and diagonal rounds diffuses changes across the entire 4×4 matrix, and a final word-wise addition of the initial state makes the mapping non-invertible.

10.5.2 From core to cipher

To encrypt, run the core once per 64-byte block with the counter incremented each time, concatenate the blocks into a keystream, and xor with the plaintext:

$$\text{keystream} = \text{ChaChaBlock}(k, n, 0) \,\|\, \text{ChaChaBlock}(k, n, 1) \,\|\, \cdots$$

$$c = m \oplus \text{keystream}.$$

Walking question

How long can one (key, nonce) pair safely encrypt? The 32-bit block counter addresses 2^{32} blocks of 64 bytes each, so $2^{32} \cdot 64 = 2^{38}$ bytes = 256 GiB before the counter wraps. Reach the wrap and you would repeat keystream – a two-time pad. In practice you change the nonce per message long before that. The 96-bit nonce gives 2^{96} messages per key, large enough that a properly *random* nonce essentially never collides, and a per-message *counter* nonce never collides by construction. Pick one discipline and never reuse a nonce under a fixed key.

Implement it

The ChaCha20 block function – learning only. Teaching code. It omits the constant-time care and the authentication (Poly1305) that any real deployment needs. Never ship a hand-rolled cipher; use a vetted library (libsodium, OpenSSL, your language's standard crypto).

```
import struct

def rotl(x, n):
```

```
    return ((x << n) | (x >> (32 - n))) & 0xffffffff

def quarter_round(s, a, b, c, d):
    s[a] = (s[a] + s[b]) & 0xffffffff; s[d] = rotl(s[d] ^ s[a], 16)
    s[c] = (s[c] + s[d]) & 0xffffffff; s[b] = rotl(s[b] ^ s[c], 12)
    s[a] = (s[a] + s[b]) & 0xffffffff; s[d] = rotl(s[d] ^ s[a], 8)
    s[c] = (s[c] + s[d]) & 0xffffffff; s[b] = rotl(s[b] ^ s[c], 7)

def chacha20_block(key, counter, nonce):
    const = b"expand 32-byte k"
    state = list(struct.unpack("<4I", const))
    state += list(struct.unpack("<8I", key))      # 256-bit key
    state += [counter]
    state += list(struct.unpack("<3I", nonce))    # 96-bit nonce
    work = state[:]
    for _ in range(10):                           # 10 double-rounds = 20 rounds
        # column rounds
        quarter_round(work, 0, 4, 8, 12)
        quarter_round(work, 1, 5, 9, 13)
        quarter_round(work, 2, 6, 10, 14)
        quarter_round(work, 3, 7, 11, 15)
        # diagonal rounds
        quarter_round(work, 0, 5, 10, 15)
        quarter_round(work, 1, 6, 11, 12)
        quarter_round(work, 2, 7, 8, 13)
        quarter_round(work, 3, 4, 9, 14)
    out = [(work[i] + state[i]) & 0xffffffff for i in range(16)]  # feed-forward
    return struct.pack("<16I", *out)              # 64 keystream bytes

def chacha20(key, nonce, data, counter=0):
    out = bytearray()
    for blk in range(0, len(data), 64):
        ks = chacha20_block(key, counter + blk // 64, nonce)
        chunk = data[blk:blk+64]
        out += bytes(x ^ y for x, y in zip(chunk, ks))
    return bytes(out)
```

The feed-forward step `work[i] + state[i]` is essential: without it the 20 rounds are an invertible permutation an attacker could run backward from one keystream block to recover the key. Adding back the original state breaks that invertibility.

10.5.3 Why ChaCha20 is trusted

We cannot *prove* ChaCha20 is a secure PRG – doing so would settle deep open problems in complexity. What we have is the standard cryptographic basis for confidence: a clean design that resists every known class of attack, with a large security margin.

Why it works

The confidence rests on two arguments. *Margin:* the best published cryptanalysis breaks roughly 7 of the 20 rounds (differential-linear attacks by Aumasson and others), and even those require effort far beyond brute force. The thirteen unbroken rounds that remain are a wide security margin. *Structure:* the counter-mode construction means the security question reduces to "does the core look like a random function of (key, nonce, counter)?" The ARX core has strong, well-understood diffusion – each output bit depends on every input bit after a few rounds – and has *no* secret-dependent branches or memory accesses, so it avoids the timing and cache side channels that affect table-based ciphers. The nonlinearity comes entirely from integer carries, which are cheap and analyzable.

In the real world

ChaCha20 in the wild. In TLS 1.3 the cipher suite `TLS_CHACHA20_POLY1305_SHA256` pairs ChaCha20 with the Poly1305 authenticator to give *authenticated* encryption (AEAD) – confidentiality *and* integrity in one pass. Browsers prefer it on devices without AES hardware acceleration, where it outruns AES-GCM. WireGuard uses ChaCha20-Poly1305 as its only data cipher (no negotiation, no downgrade attacks). OpenSSH offers `chacha20-poly1305@openssh.com`. The Linux kernel's `getrandom()` and `/dev/urandom` are built on a ChaCha-based CSPRNG. A raw stream cipher is never deployed alone for real traffic: it is always wrapped in a MAC, because confidentiality without integrity lets an attacker flip ciphertext bits – and xor-based stream ciphers flip the *exact* corresponding plaintext bits, silently. That malleability is the subject of authenticated encryption in Chapter 16.

Common trap

Stream-cipher ciphertext is *malleable*. Because $c = m \oplus z$, an attacker who knows or guesses part of m can flip chosen bits of c to flip the same bits of m, with no key. Encrypt "transfer \$0100" and an attacker who knows the format can flip a byte to "transfer \$9100" without ever decrypting. Encryption hides content; it does *not* prevent tampering. Always add a MAC (an AEAD mode like ChaCha20-Poly1305). "Encrypt-then-MAC," covered in Chapter 16, is the standard way to prevent this.

10.6 Linear feedback shift registers and the curse of linearity

Before software stream ciphers, hardware did the job with *linear-feedback shift registers* (LFSRs): a row of bit cells that shifts every clock tick, with the incoming bit computed as an xor of selected "tap" positions. They are small, very fast in hardware, and produce long, statistically flat sequences. Used naively, they also show clearly why *linear* keystreams are insecure.

10.6.1 A tiny LFSR by hand

Figure 10.5 shows the standard maximal-period 4-bit LFSR: seeded with $(s_1 s_2 s_3 s_4) = 1001$ and tapping $s_1 \oplus s_4$ (primitive polynomial $x^4 + x + 1$), it outputs the rightmost bit each tick, shifts right, and feeds $s_1 \oplus s_4$ in on the left ($1001 \to 0100 \to 0010 \to 0001 \to 1000 \to \cdots$), cycling through all

$2^4 - 1 = 15$ nonzero states in Panel B before repeating.

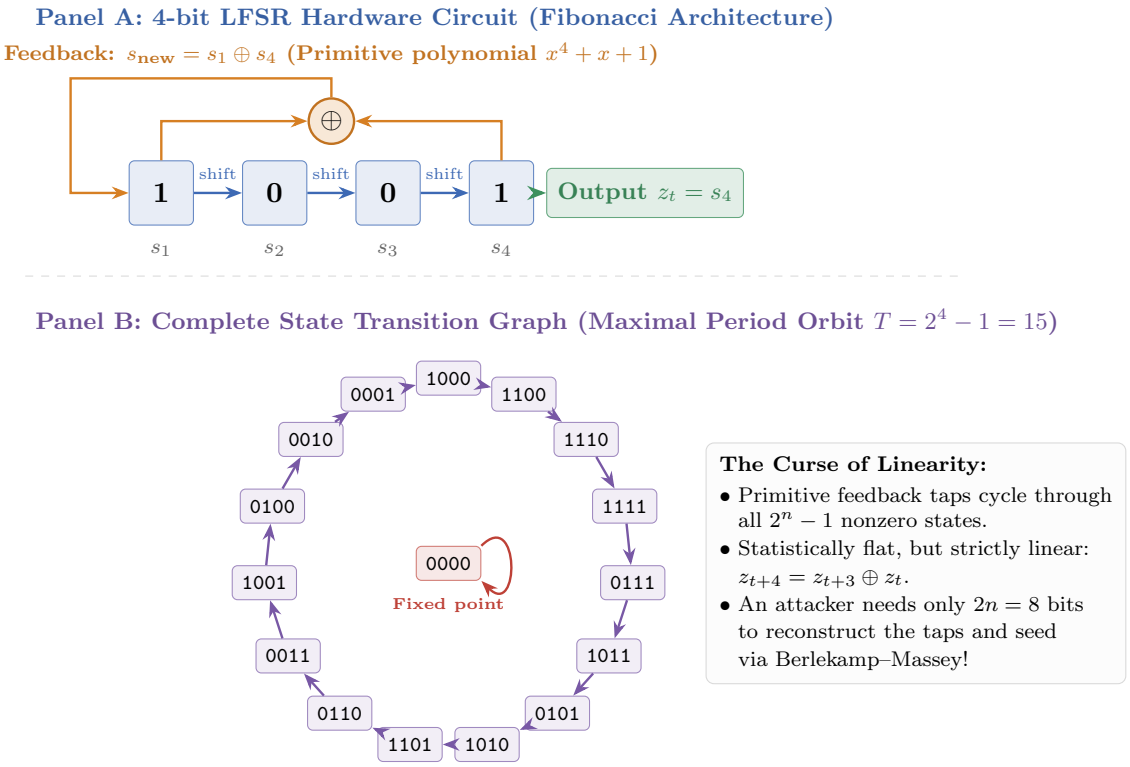


Figure 10.5: Linear Feedback Shift Registers (LFSRs). Panel A: A 4-bit Fibonacci LFSR where the rightmost bit is emitted while the input bit is linearly formed via $s_1 \oplus s_4$. Panel B: The complete state space partition. When configured with a primitive polynomial (x^4+x+1), the register traverses a single maximal-period cycle of length $2^4 - 1 = 15$ states before repeating (leaving the all-zero state as an isolated fixed point). Despite statistical flatness, the recurrence is linear over $\mathbb{F}_2$ and easily broken.

To trace a shorter cycle in full by hand, suppose instead we place taps at the two leftmost cells so the feedback bit is (old s_1) $\oplus$ (old s_2). Write the state left to right and output the rightmost bit each tick; starting from seed 1001:

tick	state $s_1s_2s_3s_4$	output (right bit)	feedback $s_1 \oplus s_2$
0	1001	1	1
1	1100	0	0
2	0110	0	1
3	1011	1	1
4	1101	1	0
5	0110	0	1
6	1011	...	...

Walking question

Trace tick 0 to 1 yourself. State 1001: output the rightmost bit, 1. Feedback $= s_1 \oplus s_2 = 1 \oplus 0 = 1$. Shift everyone right and drop the new feedback bit into the left cell: new state $= 1\,|\,100 = 1100$. Tick 1 to 2: output 0, feedback $1 \oplus 1 = 0$, new state $0\,|\,110 = 0110$. This $s_1 \oplus s_2$ tap choice does not hit the maximal period of $2^4 - 1 = 15$; a register with a *primitive* feedback polynomial (such as $s_1 \oplus s_4$ in Figure 10.5) cycles through all 15 nonzero states before repeating. The all-zero state is a fixed point and must be avoided – an LFSR that ever reaches 0000 stays stuck at 0000.

The output bit sequence here is $1, 0, 0, 1, 1, 0, \ldots$ – flat-looking and balanced. It could pass a naive statistical test, yet cryptographically it is very weak.

10.6.2 Why linearity is fatal

The register's update is a linear map over $\mathbb{F}_2$. Every output bit is a fixed $\mathbb{F}_2$-linear combination of the seed bits. That makes the keystream solvable by linear algebra once an attacker sees a short stretch of it.

Attack

Breaking an LFSR with known plaintext. Suppose Eve knows (or guesses) a stretch of plaintext, so she knows the corresponding keystream bits $z_1, z_2, \ldots$ An n-bit LFSR has n unknown seed bits. If Eve knows the taps, the state at any time is an $\mathbb{F}_2$-linear function of the seed, so each known keystream bit gives one *linear equation* in the n unknowns. Collect n equations – n known keystream bits – and solve the linear system by Gaussian elimination over $\mathbb{F}_2$. She recovers the seed and can generate the entire keystream forward and backward. If Eve does *not* know the taps either, the Berlekamp–Massey algorithm finds the shortest LFSR that produces an observed sequence, given only about $2n$ consecutive keystream bits, in $O(n^2)$ time. The number n here is the *linear complexity* of the sequence. So any sequence with low linear complexity is broken with a short known-plaintext sample – automatically, taps and seed and all.

Example 10.2 (Solving our toy LFSR)**.** Our register has $n = 4$ with taps $\{1, 2\}$. The output at each tick is the rightmost cell s_4, and the state is a known linear shift of the seed. Writing the seed (s_1, s_2, s_3, s_4) as unknowns, the first outputs read off shifted seed bits directly: $z_0 = s_4$, $z_1 = s_3$, $z_2 = s_2$, and $z_3 = s_1$ (each tick the bit that was at s_3 slides to s_4 and out, and so on). Four leaked outputs $z_0 z_1 z_2 z_3$ therefore pin down all four seed bits with no work beyond reading them off; further outputs are then determined by the recurrence. Even in a less transparent tap configuration, Gaussian elimination over $\mathbb{F}_2$ – which is just xor-ing rows – recovers the seed from n bits. Four leaked keystream bits are enough for a total break.

Why it works

The underlying reason is that a secure PRG must pass the next-bit test (Theorem 10.1), but for an LFSR the next bit is an *exact* linear function of bits you have already seen. A predictor that solves a 4×4 linear system predicts the next bit with probability 1, not $\frac{1}{2} + \mathsf{negl}$. A linear keystream is completely predictable. Any keystream you can describe with a short linear

recurrence has tiny linear complexity, and Berlekamp–Massey turns that into a full break. This is why "looks random statistically" is nowhere near "is cryptographically pseudorandom."

Connection

Berlekamp–Massey is the same algorithm used to decode BCH and Reed–Solomon error-correcting codes – it finds the minimal linear recurrence behind a sequence. The companion algorithms volume develops Gaussian elimination over $\mathbb{F}_2$ (which is bitwise xor of rows, extremely fast) and the $O(n^2)$ recurrence-finding that makes the LFSR attack run in microseconds. The linear-complexity profile of a sequence is a coding-theory invariant reused as a cryptanalytic tool.

10.6.3 Trying to rescue LFSRs

Engineers did not abandon LFSRs; they tried to break the linearity by combining several of them nonlinearly – a *nonlinear combining function* mixing the outputs of several LFSRs, or *irregular clocking* (one register's output decides whether another steps).

Attack

[Historical attacks on LFSR ciphers: GSM A5/1 and Bluetooth E0] The GSM mobile standard's voice cipher **A5/1** combines three LFSRs (of lengths 19, 22, 23 bits) with majority-vote irregular clocking. It was meant to be strong but is not. The small 64-bit state and clocking structure allow time–memory tradeoff attacks (Biryukov–Shamir, and later rainbow-table projects) that recover the session key for a GSM call from a few seconds of known keystream using precomputed tables. **E0**, the Bluetooth stream cipher, combines four LFSRs and has likewise been broken below brute force. **Trivium**, an eSTREAM finalist, is a more careful nonlinear-feedback design that remains unbroken, showing that shift-register ciphers *can* be done well, but only with serious nonlinearity and enough state.

In the real world

Where stream ciphers actually live today. Hardware-friendly LFSR-style ciphers persist in constrained radios: **A5/1** and **A5/3** in 2G GSM, **SNOW 3G** and **ZUC** as standardized confidentiality cores in 4G LTE and 5G (both are LFSR-based but with strong nonlinear filters and have held up). On the software side, the world moved to ChaCha20 (TLS, WireGuard, SSH, OS randomness) and to AES in counter-like modes (which turns a block cipher into a stream cipher – the subject of Chapter 13). All of these follow the design rule developed in this chapter: stretch a short secret into an unpredictable keystream, never reuse a (key, nonce) pair, and never rely on linearity for secrecy.

10.7 Summary

A stream cipher is the one-time pad with the truly random pad replaced by the output of a pseudorandom generator. That swap costs us perfect secrecy (the key is shorter than the message) but buys a practical cipher that is secure against any feasible adversary – provided the PRG is secure,

meaning no efficient distinguisher beats a coin flip, equivalently no efficient predictor guesses the next bit. The proof that a good PRG yields a secure cipher is a one-line reduction; the proof that next-bit unpredictability suffices is a hybrid argument.

The single most important operational rule is: **never reuse a keystream.** Reuse turns the cipher into a two-time pad whose ciphertexts xor to the xor of plaintexts, readable by crib-dragging – the failure that broke WEP through a 24-bit IV. RC4 taught the second lesson: a primitive can be fine in its day yet die to discovered biases (Mantin–Shamir, FMS, the 2013 TLS break) and must then be retired, not patched. ChaCha20 is the modern answer: an ARX counter-mode design with a wide security margin, constant-time by construction, deployed across TLS 1.3, WireGuard, SSH, and OS randomness, always wrapped in a MAC because stream ciphertext is malleable. LFSRs show that "statistically flat" is not "pseudorandom": linear keystreams fall to Gaussian elimination and Berlekamp–Massey from a handful of known bits, which is why every shift-register cipher still in use relies on genuine nonlinearity.

Instead of stretching a short key into an ongoing keystream, we can partition data into fixed-size blocks and transform each block with a keyed, reversible permutation; Chapter 11 builds block ciphers, Feistel networks, and DES.

10.8 Exercises

Exercise 10.1 (Two-time pad recovery)**.** A stream cipher reuses keystream z on two messages. You intercept (in hex) $c_1 =$ `0x3A1F` and $c_2 =$ `0x2B0E`, and you know the first message begins with the ASCII byte `'H'` (`0x48`). Compute $c_1 \oplus c_2$, then recover the first byte of m_2. (Hint: $m_2 = (m_1 \oplus m_2) \oplus m_1$.)

Exercise 10.2 (RC4 key schedule by hand)**.** Run the RC4 KSA on a register of size *8* instead of 256 (so all arithmetic is mod 8), with key bytes $[1, 2]$. Start with $S = [0, 1, 2, 3, 4, 5, 6, 7]$ and trace the eight swaps, writing S after each step. Then run one PRGA step and give the first keystream value.

Exercise 10.3 (Why dropping bytes is not enough)**.** Explain, using the Mantin–Shamir second-byte bias, why a long-lived secret (such as an HTTP cookie) sent at a *fixed* position in many RC4-encrypted streams is recoverable even though each stream uses a different key. Why does discarding the first 256 keystream bytes (RC4-drop256) reduce but not eliminate the risk?

Exercise 10.4 (ChaCha quarter-round trace)**.** Apply one ChaCha20 quarter-round to the four 32-bit words $a =$ `0x11111111`, $b =$ `0x01020304`, $c =$ `0x9b8d6f43`, $d =$ `0x01234567`. Carry out the eight operations in order (add mod 2^{32}, xor, rotate) and give the final (a, b, c, d). Then flip the lowest bit of the input b and verify that many output bits change (avalanche).

Exercise 10.5 (Nonce budget)**.** ChaCha20 has a 32-bit block counter and a 96-bit nonce. (a) How many bytes of keystream can one (key, nonce) pair produce before the counter wraps? (b) If you choose nonces uniformly at random, roughly how many messages can you send under one key before the probability of a nonce collision reaches $\frac{1}{2}$? Use the birthday approximation $n \approx 2^{(b+1)/2}$ for a b-bit nonce, and state why a collision is a security disaster.

Exercise 10.6 (Cranking an LFSR)**.** Take a 4-bit LFSR with feedback bit $= s_1 \oplus s_2$ (taps at the two leftmost cells), output the rightmost bit, seed 1011. Produce the first 8 output bits and the state at each tick. Does this seed and tap choice reach the maximal period of 15? What happens if a register ever reaches the all-zero state?

Exercise 10.7 (Solve an LFSR by linear algebra)**.** You know an unknown 3-bit LFSR has feedback bit $= s_1 \oplus s_3$, outputs the rightmost bit, and you observe the keystream bits $z_0 z_1 z_2 z_3 z_4 = 1, 1, 0, 1, 0$. Set up the linear equations over $\mathbb{F}_2$ relating these outputs to the unknown seed (s_1, s_2, s_3) and solve for the seed by xor-elimination. Then predict z_5.

Exercise 10.8 (Malleability)**.** A bank app encrypts the message `AMOUNT=00100` with a stream cipher and sends the ciphertext. The attacker knows the plaintext format and the byte offset of the amount field. Show precisely which ciphertext bytes the attacker xors, and with what values, to change the decrypted amount to `AMOUNT=90100`, without knowing the key. What single defensive mechanism stops this attack, and why does encryption alone not?

Exercise 10.9 (PRG distinguisher from a bias)**.** Suppose a candidate PRG outputs bits that are each independently 1 with probability 0.51 instead of 0.5. Describe a distinguisher that, given a long output, decides "PRG" vs "random" with advantage close to 1, and estimate how many output bits it needs. (Hint: count ones; compare to the expected number under each hypothesis, and use a concentration bound.) Conclude why even a tiny per-bit bias disqualifies a generator.

Exercise 10.10 (Hybrid argument)**.** Let G be a PRG with output length ℓ. Suppose a distinguisher D separates $G(k)$ from a random ℓ-bit string with advantage ε. Using the hybrids H_j (first j bits random, last $\ell - j$ bits from G), prove there exists an index j such that D separates H_j from H_{j+1} with advantage at least ε/ℓ, and explain how this yields a next-bit predictor with the same advantage.

Chapter 11

Block Ciphers, Feistel Networks, and DES

In Chapter 10, stream ciphers softened the one-time pad's cost by stretching a short key into a long pseudorandom keystream. But a stream cipher is still essentially a pad: reuse the keystream, or flip a ciphertext bit to flip the corresponding plaintext bit, and the underlying XOR structure is exposed.

This chapter is about a different and, in practice, more central object: the *block cipher*. Rather than producing a keystream to XOR onto data, a block cipher defines, for each key, a reversible scrambling of a fixed-size block of bits — a *keyed pseudorandom permutation* of, say, all 2^{64} possible 64-bit strings. We cover the pseudorandom permutation model, Shannon's principles of *confusion* and *diffusion*, the *Feistel construction* that builds an invertible permutation from a function that need not be invertible at all, and the design and cryptanalysis of the Data Encryption Standard (DES).

11.1 What a Block Cipher Is

Definition 11.1 (Block cipher)**.** A *block cipher* with block length n and key length ℓ is a function

$$E : \{0,1\}^{\ell} \times \{0,1\}^{n} \longrightarrow \{0,1\}^{n}$$

such that for every fixed key $k \in \{0,1\}^{\ell}$, the map $E_k(\cdot) = E(k,\cdot)$ is a *bijection* (a permutation) of $\{0,1\}^{n}$. Because E_k is a bijection it has an inverse $D_k = E_k^{-1}$, the decryption map, and $D_k(E_k(x)) = x$ for all x.

The key word in that definition is "permutation." For a fixed key, E_k takes the 2^n possible input blocks and shuffles them among themselves. Nothing is lost and nothing collides: two distinct plaintexts always land on two distinct ciphertexts. That is forced on us — if E_k mapped two plaintexts to the same ciphertext, decryption could not recover which one we meant. Any encryption that can be undone is a permutation.

Intuition

A block cipher is a giant lookup table you never write down. Imagine, for a 64-bit block, a table with 2^{64} rows: row x holds the ciphertext $E_k(x)$. Every key k selects a different table. To encrypt is to look up a row; to decrypt is to find which row holds your ciphertext. The design problem is to make a table that *behaves* like a random shuffle of all 2^{64} strings, while being computable from a 16-byte key with a few dozen cheap operations and no actual 2^{64}-row table anywhere in memory.

11.1.1 Why "pseudorandom permutation" is the right goal

How many permutations of $\{0,1\}^n$ are there? Exactly $(2^n)!$ — the number of ways to order 2^n items. For $n = 64$ that is $(2^{64})!$, a number whose number of digits already dwarfs the number of atoms in the universe. To name one specific permutation you would need about $\log_2\big((2^n)!\big) \approx n \cdot 2^n$ bits; for $n = 64$ that is on the order of 2^{70} bits, an impossibly large key.

A block cipher cannot afford that. Its key is short — 128 or 256 bits — so it can only reach a vanishingly thin sliver of all possible permutations: at most 2^ℓ of the $(2^n)!$ of them. The security goal is that this thin sliver should be *indistinguishable* from the whole. An adversary who is handed a black box — either E_k for a secret random key k, or a permutation chosen uniformly at random from all $(2^n)!$ of them — and who may query it on inputs of their choice, should not be able to tell which world they are in.

Definition 11.2 (PRP advantage)**.** Let E be a block cipher with block length n. For an adversary Adv define

$$\mathsf{Adv}^{\mathrm{prp}}_E(\mathsf{Adv}) \;=\; \Big|\; \Pr_{k \xleftarrow{\$} \{0,1\}^\ell}\big[\mathsf{Adv}^{E_k(\cdot)} = 1\big] \;-\; \Pr_{\pi \xleftarrow{\$} \mathrm{Perm}(n)}\big[\mathsf{Adv}^{\pi(\cdot)} = 1\big] \;\Big|,$$

where $\mathrm{Perm}(n)$ is the set of all permutations of $\{0,1\}^n$. We say E is a *secure pseudorandom permutation* (PRP) if for every efficient Adv this advantage is negligible.

A *strong* PRP is the same but the adversary also gets to query the inverse box (E_k^{-1} or π^{-1}). Strong PRPs are what you want when an attacker can submit ciphertexts for decryption, which happens constantly in real protocols.

Why it works

Why does "looks like a random permutation" deliver security? Because a truly random permutation leaks nothing about its inputs. If π is a uniformly random permutation and you have not yet queried x, then $\pi(x)$ is uniform over all the block values you have not yet seen as outputs — it is as good as a fresh coin flip. So an attacker watching ciphertexts of a random permutation learns nothing beyond which plaintexts were equal (equal plaintexts give equal ciphertexts; that is the one unavoidable leak). If E_k is a good PRP, the same is true of it up to a negligible slip. Every standard mode of operation — CTR, CBC, GCM, all of Chapter 13 — is proved secure by *first* assuming the underlying cipher is a (strong) PRP, then arguing the mode on top. The block cipher supplies the hardness, and the mode only has to use it correctly.

Connection

The PRP definition is the permutation cousin of the pseudorandom *generator* and pseudorandom *function* ideas from Chapter 8. The leap from PRF to PRP is small but real: a random *function* can collide (two inputs to one output), a random *permutation* cannot. The PRP/PRF switching lemma quantifies the gap: any adversary making q queries can distinguish a random permutation from a random function with advantage at most $q^2/2^{n+1}$ — a birthday bound. That $q^2/2^n$ term, straight out of the birthday calculation in the probability companion, is exactly why a 64-bit block becomes unsafe after about 2^{32} blocks, and why modern ciphers moved to 128-bit blocks.

11.2 Confusion and Diffusion

In 1949 Claude Shannon, in the same paper that gave us perfect secrecy, named the two properties a good cipher must have. They have guided every block-cipher design since.

Confusion makes the relationship between the key and the ciphertext as complicated as possible. Each bit of ciphertext should depend on many bits of the key in a tangled, nonlinear way, so that knowing some plaintext/ciphertext pairs gives you no clean equation to solve for the key. Confusion defeats algebraic attacks: it stops you from writing the cipher as a tidy linear system over $\mathbb{F}_2$.

Diffusion spreads the influence of each plaintext bit across the whole ciphertext. Change one bit of the input, and about half the output bits should flip — and they should be unpredictable bits. Diffusion destroys the statistics of the plaintext: a message with lots of repeated letters comes out looking like noise, because each plaintext bit has been smeared across the entire block.

Intuition

Picture kneading dough with a drop of food coloring. *Diffusion* is the stretching and folding that spreads the single drop until every part of the dough is faintly tinted — one input change touching everything. *Confusion* is making the relationship between "how much you knead" (the key schedule) and "where the color ends up" so chaotic that you cannot reverse-engineer the kneading pattern from the final swirl. Real ciphers achieve diffusion with bit permutations and mixing layers, and confusion with *S-boxes*: small nonlinear lookup tables. Most block ciphers are built by alternating the two, round after round.

11.2.1 The avalanche criterion

Diffusion has a crisp test. A cipher satisfies the *strict avalanche criterion* if flipping any single input bit causes each output bit to flip with probability 1/2. Equivalently: complement one bit of the plaintext, re-encrypt, and the two ciphertexts should differ in about half their bits, with the differing positions looking random. If they differed in only one or two positions, a single plaintext bit barely moved — diffusion failed, and an attacker could track plaintext structure straight through to the ciphertext.

Focus

A one-round cipher cannot avalanche: a single S-box touches only a few output bits, so a one-bit input change reaches only those. Avalanche is a *multi-round* phenomenon. Round 1 spreads the change to a handful of bits; the permutation layer scatters them to different S-boxes; round 2 each of those S-boxes amplifies again; after a few rounds the change has reached every output bit. DES needs about 5 of its 16 rounds before full avalanche kicks in. The extra rounds beyond that are the safety margin against cryptanalysis.

11.3 The Feistel Construction

Block-cipher design faces a structural problem: the cipher as a whole must be a bijection (an invertible permutation) so that decryption is uniquely possible, yet security demands that the transformation be highly nonlinear and complex. If we try to assemble a permutation purely out of small functions that are each individually invertible, we restrict our design choices, because invertible operations over small bit vectors tend to have rigid algebraic structure that linear or differential attacks can exploit.

Horst Feistel, working at IBM in the early 1970s, found a way around this problem. The Feistel construction converts *any* function f whatsoever — whether it is non-invertible, non-injective, or many-to-one — into a strictly invertible permutation of twice the input width. Inversion never evaluates the inverse of f; it only evaluates f in the forward direction. Invertibility comes entirely from the algebraic properties of XOR.

11.3.1 One Feistel round

Split the n-bit block into two halves L and R of $w = n/2$ bits each. A single round, parameterized by a round subkey k_i and a round mixing function $f : \{0,1\}^{\ell_k} \times \{0,1\}^w \to \{0,1\}^w$, maps the pair (L_{i-1}, R_{i-1}) to (L_i, R_i) by the recurrence:

$$L_i = R_{i-1}, \tag{11.1}$$

$$R_i = L_{i-1} \oplus f(k_i, R_{i-1}). \tag{11.2}$$

The right half passes through untouched to become the next left half. Simultaneously, that same right half is fed into the round function f alongside the round key k_i, and the resulting mask is XORed onto the left half to produce the new right half.

Why it works

[Why a Feistel round always inverts] Suppose an observer is handed the output pair (L_i, R_i) and the round key k_i, and wishes to recover the inputs (L_{i-1}, R_{i-1}).
Look at the forward equations (11.1) and (11.2). Equation (11.1) gives R_{i-1} directly:

$$R_{i-1} = L_i.$$

With R_{i-1} in hand, the observer can immediately evaluate the round function $f(k_i, R_{i-1})$ in the standard forward direction — no inverse function f^{-1} is ever evaluated or even required to exist.

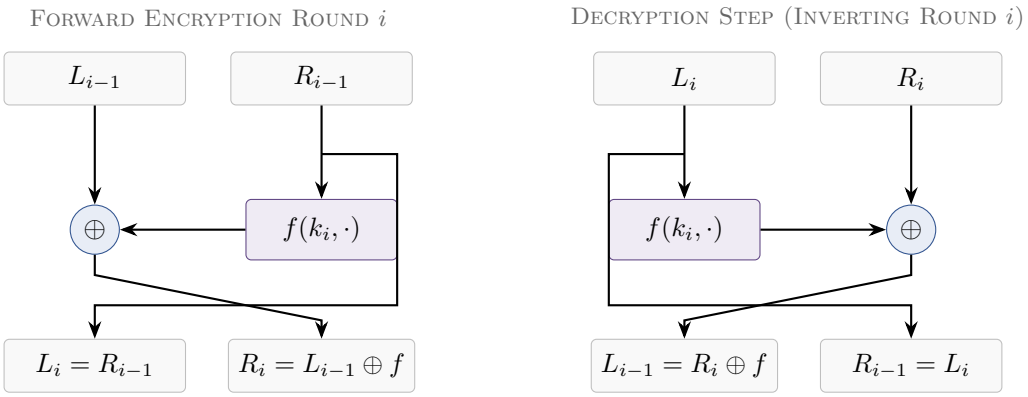


Figure 11.1: The Feistel round architecture and its step-by-step inversion. Decryption uses the identical function f in the forward direction; the input half L_i supplies the exact mask to cancel $f(k_i, R_{i-1})$ via XOR.

Next, examine Equation (11.2): $R_i = L_{i-1} \oplus f(k_i, R_{i-1})$. Taking the bitwise XOR of both sides with $f(k_i, R_{i-1})$ and using the associativity of XOR and the fundamental identity $y \oplus y = 0$:

$$\begin{aligned} R_i \ \oplus \ f(k_i, R_{i-1}) &= \big(L_{i-1} \ \oplus \ f(k_i, R_{i-1})\big) \ \oplus \ f(k_i, R_{i-1}) \\ &= L_{i-1} \ \oplus \ \big(f(k_i, R_{i-1}) \ \oplus \ f(k_i, R_{i-1})\big) \\ &= L_{i-1} \ \oplus \ 0 \\ &= L_{i-1}. \end{aligned}$$

Because $R_{i-1} = L_i$, substituting gives the explicit inverse:

$$L_{i-1} = R_i \ \oplus \ f(k_i, L_i).$$

Both halves (L_{i-1}, R_{i-1}) are recovered using only forward evaluations of f and XOR addition. The internal structure of f can be as heavily nonlinear, non-surjective, or non-injective as desired. To decrypt an entire r-round Feistel cipher, you run the exact same circuit in reverse, applying the round subkeys in backwards order: $k_r, k_{r-1}, \ldots, k_1$.

Formal

Let $\rho_k(L, R) = (R, \ L \oplus f(k, R))$ be one Feistel round (before the final swap). Its inverse is $\rho_k^{-1}(A, B) = (B \oplus f(k, A), \ A)$. Check: $\rho_k^{-1}(\rho_k(L, R)) = \rho_k^{-1}(R, L \oplus f(k, R)) = \big((L \oplus f(k, R)) \oplus f(k, R), \ R\big) = (L, R)$, using $x \oplus y \oplus y = x$. So each round is a bijection, and a composition of bijections is a bijection. An r-round Feistel network is therefore a permutation *regardless of* f — which is precisely Definition 11.1's requirement.

11.3.2 How secure is a Feistel network? The Luby–Rackoff theorem

The construction guarantees invertibility for free, but does it give you a *secure* PRP? That depends on f and on how many rounds you run. Michael Luby and Charles Rackoff answered this in 1988.

Theorem 11.1 (Luby–Rackoff, informal)**.** *If the round function f is a secure pseudorandom function*

and the round keys are independent, then:

- ***Three rounds** give a secure PRP (secure against chosen-plaintext attacks).*
- ***Four rounds** give a secure* strong *PRP (secure even when the adversary may also query the decryption box).*

This result reduces "build a secure block cipher" to "build a secure pseudorandom function on half-blocks," and the Feistel structure turns the function into a permutation that is still secure. One and two rounds are insecure — two rounds let an attacker who controls inputs detect structure — but three rounds suffice.

Common trap

Real ciphers like DES use *far* more than four rounds (DES uses 16), and for good reason. The Luby–Rackoff theorem assumes f is an *ideal* pseudorandom function with a fully independent random key each round. The DES round function is a fixed, public, decidedly non-ideal function, and its round keys come from a simple key schedule, not independent randomness. The extra rounds are the price of using a real, cheap, structured f instead of the theorem's idealized one. Never read "three rounds suffice" as engineering advice — it is an existence proof about idealized components.

11.4 A Toy Feistel Cipher, Worked by Hand

Let us actually run one. We build an 8-bit block cipher: $n = 8$, so each half is 4 bits. We will do 3 rounds. The point is to see every bit move, not to be secure — this cipher is breakable in seconds.

Round function. Our f takes a 4-bit round key k_i and a 4-bit half R, and returns 4 bits. Define

$$f(k_i, R) \;=\; S\big[R \oplus k_i\big] \lll 1,$$

where S is the tiny S-box below (a fixed nonlinear lookup), and $\lll 1$ means rotate the 4-bit result left by one position. The S-box maps each 4-bit value (a nibble, 0–15) to another:

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$S[x]$	E	4	D	1	2	F	B	8	3	A	6	C	5	9	0	7

(This happens to be the first row of DES's S-box S_1 — a real nonlinear table, reused here as a toy.)

Keys. Hand each round a 4-bit key: $k_1 = \texttt{1010}_2 = \text{A}$, $k_2 = \texttt{0011}_2 = 3$, $k_3 = \texttt{1100}_2 = \text{C}$.

Plaintext. Encrypt the byte $P = \texttt{0110 1011}_2$. Split into $L_0 = \texttt{0110} = 6$ and $R_0 = \texttt{1011} = \text{B}$.

Round 1. $L_1 = R_0 = \text{B}$. Compute $f(k_1, R_0)$:

$$R_0 \oplus k_1 = \texttt{1011} \oplus \texttt{1010} = \texttt{0001} = 1, \quad S[1] = 4 = \texttt{0100}, \quad \texttt{0100} \lll 1 = \texttt{1000} = 8.$$

So $R_1 = L_0 \oplus 8 = \texttt{0110} \oplus \texttt{1000} = \texttt{1110} = \text{E}$. State: $(L_1, R_1) = (\text{B}, \text{E})$.

Round 2. $L_2 = R_1 = \mathrm{E}$. Compute $f(k_2, R_1)$:

$$R_1 \oplus k_2 = \texttt{1110} \oplus \texttt{0011} = \texttt{1101} = \mathrm{D}, \quad S[\mathrm{D}] = 9 = \texttt{1001}, \quad \texttt{1001} \lll 1 = \texttt{0011} = 3.$$

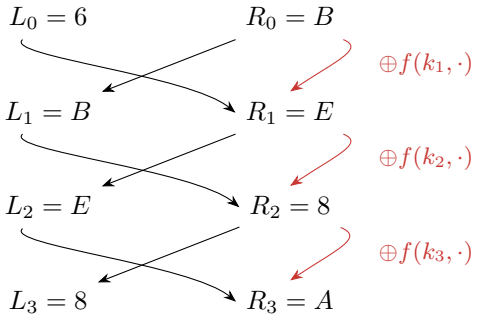
So $R_2 = L_1 \oplus 3 = \texttt{1011} \oplus \texttt{0011} = \texttt{1000} = 8$. State: $(L_2, R_2) = (\mathrm{E}, 8)$.

Round 3. $L_3 = R_2 = 8$. Compute $f(k_3, R_2)$:

$$R_2 \oplus k_3 = \texttt{1000} \oplus \texttt{1100} = \texttt{0100} = 4, \quad S[4] = 2 = \texttt{0010}, \quad \texttt{0010} \lll 1 = \texttt{0100} = 4.$$

So $R_3 = L_2 \oplus 4 = \texttt{1110} \oplus \texttt{0100} = \texttt{1010} = \mathrm{A}$. State: $(L_3, R_3) = (8, \mathrm{A})$.

Ciphertext $C = L_3 \,\|\, R_3 = \texttt{1000}\ \ \texttt{1010}_2 = \texttt{8A}_{16}$.



Walking question

Decrypt $C = \texttt{8A}$ back to P using only *forward* evaluations of f, and confirm you land on $\texttt{6B}$. Start from $(L_3, R_3) = (8, \mathrm{A})$ and undo round 3 first: recover $R_2 = L_3 = 8$, then $L_2 = R_3 \oplus f(k_3, R_2)$. You already computed $f(k_3, 8) = 4$ above, so $L_2 = \mathrm{A} \oplus 4 = \texttt{1010} \oplus \texttt{0100} = \texttt{1110} = \mathrm{E}$. Keep going. Notice you never inverted S or the rotation — decryption relies only on XOR being its own inverse.

Implement it

A complete toy Feistel cipher in Python. It mirrors the worked example exactly, so you can check the trace.

```
# TOY CIPHER -- FOR LEARNING ONLY. 8-bit block, instantly breakable.
# Never use anything in this book for real encryption.

SBOX = [0xE,0x4,0xD,0x1,0x2,0xF,0xB,0x8,
        0x3,0xA,0x6,0xC,0x5,0x9,0x0,0x7]   # first row of DES S1

def rotl4(x, n=1):                  # rotate a 4-bit nibble left
    return ((x << n) | (x >> (4 - n))) & 0xF

def f(ki, R):                       # round function: nonlinear + diffusing
    return rotl4(SBOX[(R ^ ki) & 0xF])

ROUND_KEYS = [0xA, 0x3, 0xC]        # k1, k2, k3
```

```python

def encrypt(P):
    L, R = (P >> 4) & 0xF, P & 0xF  # split byte into two nibbles
    for ki in ROUND_KEYS:
        L, R = R, L ^ f(ki, R)      # one Feistel round (with swap)
    return (L << 4) | R

def decrypt(C):
    L, R = (C >> 4) & 0xF, C & 0xF
    for ki in reversed(ROUND_KEYS): # same keys, reverse order
        R, L = L, R ^ f(ki, L)      # undo one round
    return (L << 4) | R

P = 0x6B
C = encrypt(P)
assert C == 0x8A and decrypt(C) == P
print(hex(P), "->", hex(C), "->", hex(decrypt(C)))   # 0x6b -> 0x8a -> 0x6b
```

Warning. This is a teaching toy. The block is 8 bits, so a single chosen-plaintext run over all 256 inputs rebuilds the entire code book. Real ciphers use 128-bit blocks and dozens of rounds with vetted components.

Work it yourself

Verify the avalanche idea on the toy. Encrypt $P = \texttt{6B}$ (you got $\texttt{8A}$) and then encrypt $P' = \texttt{6A}$, which differs in a single bit. How many ciphertext bits change? With only 3 rounds and a 4-bit round function you will see partial, not full, avalanche — exactly the lesson of the focus box: diffusion needs rounds. Bump `ROUND_KEYS` to 8 or 16 rounds (repeat the three keys cyclically) and watch the difference spread toward half the bits.

11.4.1 Watching the avalanche fail (and then start)

Rather than leave the avalanche claim as an exercise, we trace it here. Take $P = \texttt{6B}$, whose ciphertext we already know is $\texttt{8A} = \texttt{1000}\,\texttt{1010}_2$, and flip its lowest bit to get $P' = \texttt{6A} = \texttt{0110}\,\texttt{1010}_2$. We re-run the three rounds on P'.

Split P' into $L'_0 = \texttt{0110} = 6$ and $R'_0 = \texttt{1010} = \text{A}$ (only R_0 changed, by one bit).

Round 1. $L'_1 = R'_0 = \text{A}$. Then $R'_0 \oplus k_1 = \texttt{1010} \oplus \texttt{1010} = \texttt{0000} = 0$, $S[0] = \text{E} = \texttt{1110}$, and $\texttt{1110} \lll 1 = \texttt{1101} = \text{D}$. So $R'_1 = L'_0 \oplus \text{D} = \texttt{0110} \oplus \texttt{1101} = \texttt{1011} = \text{B}$. State (A, B) — compare the original (B, E). Already two nibbles differ, because the S-box turned a one-bit input change into a multi-bit output change.

Round 2. $L'_2 = R'_1 = \text{B}$. Then $R'_1 \oplus k_2 = \texttt{1011} \oplus \texttt{0011} = \texttt{1000} = 8$, $S[8] = 3 = \texttt{0011}$, $\texttt{0011} \lll 1 = \texttt{0110} = 6$. So $R'_2 = L'_1 \oplus 6 = \texttt{1010} \oplus \texttt{0110} = \texttt{1100} = \text{C}$. State (B, C) vs. original $(\text{E}, 8)$.

Round 3. $L'_3 = R'_2 = \mathrm{C}$. Then $R'_2 \oplus k_3 = \texttt{1100} \oplus \texttt{1100} = \texttt{0000} = 0$, $S[0] = \mathrm{E} = \texttt{1110}$, $\texttt{1110} \lll 1 = \texttt{1101} = \mathrm{D}$. So $R'_3 = L'_2 \oplus \mathrm{D} = \texttt{1011} \oplus \texttt{1101} = \texttt{0110} = 6$. Ciphertext $C' = \texttt{1100}\,\texttt{0110}_2 = \texttt{C6}_{16}$.

Now count: $C = \texttt{1000}\,\texttt{1010}$ versus $C' = \texttt{1100}\,\texttt{0110}$. XOR them: $\texttt{0100}\,\texttt{1100}$, which has *three* bits set. So one flipped input bit produced three flipped output bits out of eight. Full avalanche would be four. Three out of eight after only three rounds, from a single S-box and rotation, is already close to half, and the lab above lets you close the gap by adding rounds. Diffusion is cumulative: each extra round spreads the change further.

11.5 DES: The Data Encryption Standard

History

In the early 1970s the US National Bureau of Standards (now NIST) put out a call for a public encryption standard — something unheard of, since serious cryptography had lived inside intelligence agencies. IBM responded with a Feistel cipher descended from its earlier "Lucifer" design, led by Horst Feistel, Walter Tuchman, and Don Coppersmith. The National Security Agency reviewed it before adoption and made two changes that drew suspicion for decades: they *shrank the key* from Lucifer's 128 bits down to 56, and they *altered the S-boxes* without public explanation. The shortened key looked like a deliberate ceiling on security; the S-box tweak looked like a possible backdoor. The cipher was published as FIPS 46 in 1977 under the name DES.

In hindsight, the two changes turned out very differently. The S-box change turned out to be *protective*: in 1990 Eli Biham and Adi Shamir publicly discovered *differential cryptanalysis*, and the DES S-boxes proved to be nearly optimally hardened against it — IBM and the NSA had known about the attack 15 years early and quietly defended against it. The key-length cut, though, was the weakness it appeared to be: a 56-bit key fell in public to the EFF's Deep Crack machine in 1998, motivating the open competition that produced AES.

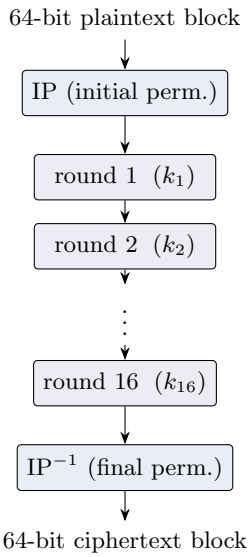
11.5.1 The shape of DES

DES is a 16-round Feistel cipher with a 64-bit block and a 56-bit key. (The key is supplied as 64 bits, but 8 of those are parity bits and carry no entropy — hence 56 effective.) The overall flow:

1. An *initial permutation* IP shuffles the 64 input bits by a fixed wiring. IP has no cryptographic value — it was an artifact of 1970s hardware — but it is part of the standard.

2. The 64 bits split into L_0, R_0 of 32 bits each, and **16 Feistel rounds** run, using round keys $k_1, \ldots, k_{16}$ derived from the master key by the *key schedule*.

3. After round 16, the halves are swapped once more and the *final permutation* IP^{-1} (the inverse of IP) is applied.



11.5.2 The DES round function f

This is the core of DES, where confusion and diffusion are produced. The round function f takes a 32-bit half R and a 48-bit round key k_i and returns 32 bits, in four steps:

1. **Expansion (E).** The 32-bit half is expanded to 48 bits by duplicating 16 of its bits according to a fixed table. This widens R to match the 48-bit round key, and by design it routes each input bit into *two* adjacent S-boxes to accelerate bit diffusion across the state.
2. **Key mixing.** XOR the 48-bit expanded half with the 48-bit round key: $E(R) \oplus k_i$. This is the only place the key enters — confusion's entry point.
3. **Substitution (S-boxes).** The 48 bits are cut into eight 6-bit chunks. Each chunk goes through its own S-box $S_1, \ldots, S_8$, a fixed table that maps 6 bits to 4 bits. Eight S-boxes output $8 \times 4 = 32$ bits. **This is the only nonlinear step in all of DES.** Everything else — expansion, XOR, permutation — is linear over $\mathbb{F}_2$. Without the S-boxes, DES would reduce to a solvable linear system; they supply all of its confusion.
4. **Permutation (P).** The 32 S-box output bits are shuffled by a fixed permutation P, so that each S-box's 4 output bits are scattered to feed *different* S-boxes in the next round. This step provides the diffusion.

11.5.3 Inside an S-box

Take S_1. It is a table with 4 rows and 16 columns, holding 4-bit values. The 6-bit input $b_1b_2b_3b_4b_5b_6$ is read as follows: the *outer* bits b_1b_6 pick the row (0–3), and the *inner* four bits $b_2b_3b_4b_5$ pick the column (0–15). The entry there is the 4-bit output.

Example 11.1 (Pushing 6 bits through S_1)**.** Feed S_1 the input $\texttt{011011}_2$.

- Row bits: $b_1b_6 = \texttt{0 1} = \texttt{01}_2 = 1$. So row 1.
- Column bits: $b_2b_3b_4b_5 = \texttt{1101}_2 = 13$. So column 13.

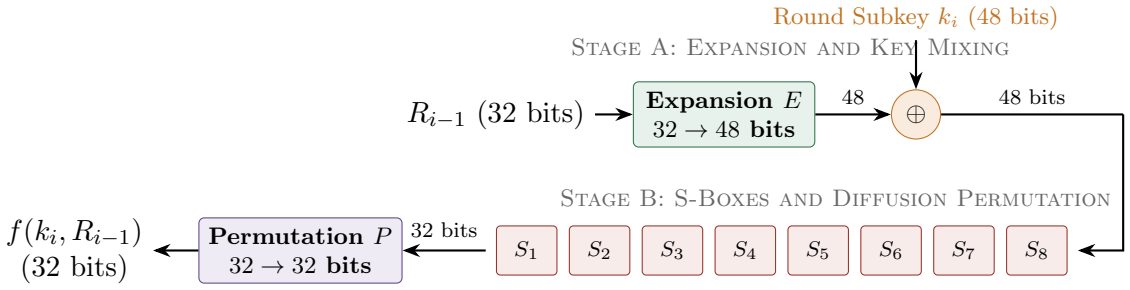


Figure 11.2: The two-stage internal pipeline of the DES round function $f(k_i, R_{i-1})$. Stage A expands the 32-bit half and XORs the 48-bit subkey. Stage B applies the eight parallel nonlinear S-boxes and the 32-bit bitwise diffusion permutation P.

Row 1, column 13 of S_1 holds the value $5 = \texttt{0101}_2$. So $S_1(\texttt{011011}) = \texttt{0101}$. The six-to-four-bit mapping is nonlinear: flipping b_3 can change the output in ways flipping b_4 does not, and this irregularity is the reason to use a lookup table rather than a formula.

Why it works

[What makes a DES S-box good] The S-boxes were not random. They were engineered against criteria, several of which only became public years later:

- **Nonlinearity.** No output bit is close to a linear (XOR) combination of input bits. This is what blocks you from writing the cipher as linear algebra, and it is the direct defense against *linear cryptanalysis* (Matsui, 1993).
- **The 4-row design.** Each row of an S-box is a permutation of $\{0, \ldots, 15\}$, and the rows differ, so the outer "control" bits genuinely reshape the mapping.
- **Differential resistance.** For any fixed input XOR-difference Δ_{in}, the output differences are spread out, never concentrated on one value. This is the property that defeats *differential cryptanalysis* — and the reason the NSA's S-box edits, far from being a backdoor, strengthened the cipher against an attack the public would not discover until 1990.

A change of even a single S-box entry tends to wreck one of these properties, which is why "random-looking" S-boxes are usually worse than carefully designed ones.

11.5.4 The key schedule

The 56-bit key is split into two 28-bit halves C_0, D_0. Before each round both halves are rotated left (by 1 or 2 positions, per a fixed schedule), and then 48 of the 56 bits are selected by a compression permutation (PC-2) to form that round's 48-bit key k_i (Figure 11.3). Because the halves keep rotating, each round sees a different slice of the key. For decryption you simply present the round keys in reverse order — the Feistel structure does the rest, exactly as in our toy.

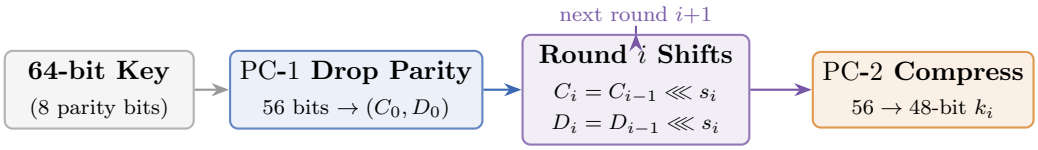


Figure 11.3: The DES key schedule. Permuted Choice 1 (PC-1) strips the 8 parity bits and splits the remaining 56 bits into two 28-bit registers (C_0, D_0). In each round $i \in \{1, \ldots, 16\}$, both halves rotate left by $s_i \in \{1, 2\}$ positions and Permuted Choice 2 (PC-2) selects 48 bits as the round subkey k_i.

11.5.5 A first taste of differential cryptanalysis

The brute-force story below is the practical killer, but it is worth seeing why the *internal* attacks — differential and linear cryptanalysis — failed to bring DES down faster, because that failure shows the S-box design working. Both attacks are statistical: they collect many plaintext/ciphertext pairs and look for a bias the cipher should not have if it were a random permutation.

Differential cryptanalysis tracks how an input *difference* propagates. Fix two plaintexts P and $P^* = P \oplus \Delta$ with a chosen XOR-difference Δ. Because XOR and the bit permutations are linear, a difference passes through them deterministically — Δ in gives a fixed Δ out, key and all, since the key XOR cancels: $(x \oplus k) \oplus (x^* \oplus k) = x \oplus x^*$. The *only* place a difference behaves probabilistically is the S-boxes.

Example 11.2 (An S-box difference is not uniform)**.** Consider one DES S-box and a fixed input difference Δ_{in} across its 6 input bits. For each of the $2^6 = 64$ input pairs $(x, x \oplus \Delta_{\text{in}})$, record the output difference $\Delta_{\text{out}} = S(x) \oplus S(x \oplus \Delta_{\text{in}})$. If the S-box behaved like a random function, the 64 pairs would scatter their output differences fairly evenly over the $2^4 = 16$ possible Δ_{out} values — about $64/16 = 4$ pairs each. The table of counts over all $(\Delta_{\text{in}}, \Delta_{\text{out}})$ is the S-box's *difference distribution table*. A cryptanalyst wants a cell with a large count: an input difference that forces a particular output difference with high probability. For the DES S-boxes the worst cell tops out at $16/64 = 1/4$, and most are at or below $4/64$. Those small, flat peaks are exactly what the NSA's edits tuned, and they are what makes the full 16-round differential characteristic so weak that you need about 2^{47} chosen plaintexts to exploit it — fewer than the 2^{56} trials of brute force, but still impractical to collect.

Focus

What matters here is the structure of the argument, not the arithmetic of one S-box. Every linear operation passes a difference through predictably; only the S-boxes obscure it. So the entire cryptanalytic strength of DES is concentrated in eight little tables, and the designer's main task is to flatten their difference distribution and their linear-approximation tables at once. Get one flat and the other spiky and you have merely traded which attack works. The published DES boxes stand out because they are good against *both* — including against an attack (differential) that would not be public for another fifteen years.

11.6 Why DES Died: 56 Bits Is Not Enough

DES's downfall was never a clever break of its internals. Differential and linear cryptanalysis both work in principle, but both need an impractical pile of chosen or known plaintexts (2^{47} chosen pairs for differential; 2^{43} known plaintexts for linear) — more than an attacker usually has. The real problem was simple arithmetic: the key space is only 2^{56}, and you can just try them all.

Attack

[Exhaustive key search] Given a single known plaintext/ciphertext pair (P, C) with $C = \mathrm{DES}_k(P)$, recover k by trying every candidate:

1. For each $\hat{k} = 0, 1, 2, \ldots, 2^{56} - 1$: compute $\mathrm{DES}_{\hat{k}}(P)$.
2. If it equals C, output $\hat{k}$ as the (probable) key.

With a 64-bit block and a 56-bit key, a single pair already pins the key down with overwhelming odds: the chance a wrong key also maps $P \mapsto C$ is about $2^{56}/2^{64} = 2^{-8}$, so two or three pairs leave essentially one survivor. The search is also *embarrassingly parallel* — split the key range across N machines and each tests $2^{56}/N$ keys with zero coordination.

Walking question

Put numbers on it. Deep Crack tested about 9×10^{10} keys per second. Half the $2^{56} \approx 7.2 \times 10^{16}$ key space is 3.6×10^{16} keys, so the expected time is $3.6 \times 10^{16} / 9 \times 10^{10} \approx 4 \times 10^5$ seconds, about *4.6 days* — and indeed Deep Crack's averages landed in the day-to-days range. Now redo the sum for AES-128: half of 2^{128} is 1.7×10^{38} keys, and even at a fanciful 10^{18} keys/second that is 1.7×10^{20} seconds, which is roughly 400 times the 4×10^{17}-second age of the universe. Same machine, same algorithm, 72 extra key bits — and the search time jumps from a long weekend to hundreds of universe-lifetimes. That is what each doubling, compounded 72 times, does.

The timeline below shows how this became a practical attack:

- **1977.** Whitfield Diffie and Martin Hellman estimate that a $20 million special-purpose machine could find a DES key in a day. At this point it was only an estimate.
- **1997.** A distributed Internet effort (DESCHALL) cracks an RSA Security challenge in 96 days by farming the key space across thousands of volunteer computers.
- **1998.** The Electronic Frontier Foundation builds *Deep Crack* for about $250,000: 1,856 custom chips, each testing many keys in parallel, in a machine the size of a few PC towers. It finds a DES key in **56 hours**. The EFF publishes the full design in a book, partly to refute the official claim that DES was still safe.
- **1999.** Deep Crack teams up with the distributed.net network and cracks a DES key in **22 hours**.

Today a few GPUs, or rented cloud time, or an FPGA cluster like COPACOBANA, find a single-DES key in hours to days for a modest budget. Single DES is dead for confidentiality.

Connection

The brute-force cost is pure counting, the kind the algorithms companion formalizes. Searching a κ-bit key is $\Theta(2^\kappa)$ cipher evaluations; each added key bit *doubles* the work. That exponential is also exactly why merely *lengthening* the key — AES-128, AES-256 — buys an astronomical margin: 2^{128} is so far beyond 2^{56} that no foreseeable classical machine touches it. The same exponential applies to attacker and defender; the defender only needs a key a few dozen bits longer.

11.6.1 The natural fix that mostly fails: Double DES

If 56 bits is too few, why not encrypt twice with two keys, $C = \mathrm{DES}_{k_2}(\mathrm{DES}_{k_1}(P))$, for a 112-bit combined key? It looks like you doubled the security. You did not — and a simple meet-in-the-middle argument shows why.

Attack

[Meet-in-the-middle on Double DES] Write the relation as $\mathrm{DES}_{k_1}(P) = \mathrm{DES}^{-1}_{k_2}(C)$. The intermediate block M must be reachable from plaintext P going forward under the true key k_1, and from ciphertext C going backward under the true key k_2:

1. **Build the lookup table.** For all 2^{56} possible candidate keys k_1, compute the forward midpoint $M = \mathrm{DES}_{k_1}(P)$ and insert the pair (M, k_1) into a hash table or sorted array indexed by M. This requires 2^{56} encryptions and stores 2^{56} entries.

2. **Probe from the ciphertext side.** For each candidate key $k_2 \in \{0,1\}^{56}$, compute the backward midpoint $M' = \mathrm{DES}^{-1}_{k_2}(C)$ and query the table for matches where $M = M'$. Each match yields a candidate key pair (k_1, k_2) satisfying $C = \mathrm{DES}_{k_2}(\mathrm{DES}_{k_1}(P))$.

3. **Filter false positives.** Test each candidate pair (k_1, k_2) against a second known plaintext/ciphertext pair (P_2, C_2) to eliminate accidental collisions.

Total computational effort: 2^{56} forward encryptions plus 2^{56} backward decryptions, giving $2^{56} + 2^{56} = 2^{57}$ operations — far below the expected 2^{112} brute-force bound. The adversary trades space for time: Double DES provides only 57 bits of security against a known-plaintext attack with access to large storage.

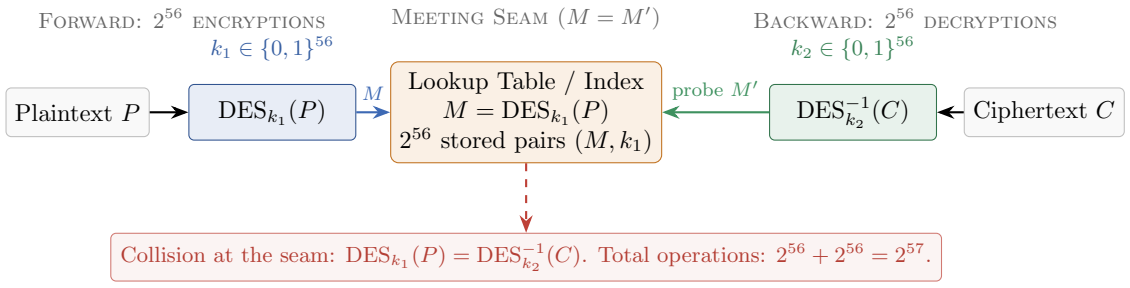


Figure 11.4: The meet-in-the-middle attack on Double DES. Instead of searching all 2^{112} key pairs sequentially, the attacker computes from both ends toward the middle state M, meeting at a collision in a table of size 2^{56}.

Walking question

Where did the "one extra bit" go? Count it. Naive double-key brute force is $2^{56} \times 2^{56} = 2^{112}$ encryptions. Meet-in-the-middle does 2^{56} forward encryptions to fill the table, then 2^{56} backward decryptions to scan it: $2^{56} + 2^{56} = 2^{57}$ operations. Compared to single DES's 2^{56}, that is a single doubling — one bit. The catch is the table: 2^{56} entries of (say) 8 bytes of intermediate plus a key index is on the order of 2^{59} bytes, roughly 5×10^{17} bytes — hundreds of petabytes. Thus the attack trades time against memory, with storage as the main practical bottleneck. Time–memory trade-offs (Hellman tables, rainbow tables) let you slide along a curve — spend more time to use less memory — but the product stays large. The general lesson: "compose it twice" almost never doubles security, because a meet-in-the-middle seam runs right down the middle of any two-stage construction.

Why it works

[Why Triple DES survives where Double DES fails] The repair is **Triple DES (3DES)**: $C = \mathrm{DES}_{k_3}(\mathrm{DES}^{-1}_{k_2}(\mathrm{DES}_{k_1}(P)))$, the famous EDE (encrypt–decrypt–encrypt) order. Why the middle decryption? So that setting $k_1 = k_2 = k_3$ makes 3DES collapse to single DES, giving backward compatibility with legacy hardware. With three independent keys the combined key is 168 bits, but meet-in-the-middle still bites: split it as "first stage" versus "last two stages" and you get an attack costing about 2^{112} time with 2^{56} memory. So 3DES delivers roughly **112 bits** of security, not 168 — comfortably out of brute-force range, which is why it lingered in banking and payment systems (EMV chip cards, some PIN-block formats) long after single DES was abandoned. Its weakness today is the small 64-bit *block*, not the key: the birthday bound (the $q^2/2^n$ term from the connection box in Section 11.1) makes 3DES leak after about 2^{32} blocks under one key — the basis of the 2016 SWEET32 attack. NIST has deprecated 3DES; AES is the modern answer.

In the real world

Where these ciphers actually run:

- **TLS / HTTPS.** Modern TLS 1.3 uses AES (and ChaCha20, a stream cipher) only — DES and 3DES were removed. But TLS 1.0/1.1 offered 3DES cipher suites for years, and SWEET32 is precisely why those 64-bit-block suites were finally purged from browsers and servers around 2016.
- **Banking and payments.** The EMV chip-card and ATM ecosystem leaned on 3DES for PIN encryption and message authentication well into the 2010s, with a long, careful migration to AES because replacing deployed hardware is slow and expensive.
- **Legacy and embedded.** Old VPNs (IPsec with 3DES), Kerberos's original single-DES types (long since deprecated), set-top boxes, and SIM/telecom kit all carried DES-family ciphers. You still meet them in protocol negotiations, which is why knowing the structure — and the weaknesses — matters when you audit a real system.

The takeaway for a working cryptographer: never reach for DES or 3DES in a new design. Use AES (next chapter). But you must understand the Feistel/DES machinery, because later ciphers were designed in response to it, and because legacy DES will stay in deployed systems for a long time.

11.7 What to Carry Forward

A block cipher is a keyed permutation that should look like a random one — a (strong) pseudorandom permutation. Security against everything built on top (the modes of Chapter 13) reduces to that single property. You manufacture a good permutation by alternating *confusion* (nonlinear S-boxes, which tangle the key into the output) with *diffusion* (permutations and expansions, which smear each input bit across the whole block), round after round, until one flipped input bit avalanches into half the output.

The *Feistel construction* makes the permutation invertible for free: split, mix one half into the other with an arbitrary function f, swap, repeat. XOR being its own inverse means f never has to be inverted, so f can be as nonlinear as you like. Luby–Rackoff proves three or four Feistel rounds over a strong pseudorandom function already yield a (strong) PRP; real ciphers add many more rounds to compensate for using cheap, fixed, structured components instead of idealized ones.

DES put this into silicon: 16 Feistel rounds, eight carefully hardened S-boxes carrying all the nonlinearity, a 64-bit block, and a 56-bit key. Its internal design was strong enough to resist differential and linear cryptanalysis decades ahead of their public discovery. But 56 bits was always too few, and exhaustive search, which is cheap and parallelizes well, broke it by 1998. Double DES fails to help because meet-in-the-middle trades memory for time; Triple DES buys real headroom (about 112 bits) but is hobbled by a 64-bit block and is now deprecated in favor of AES. The next chapter builds AES, which abandons the Feistel structure for a substitution–permutation network with a 128-bit block and keys long enough to retire brute force for good.

Exercises

Exercise 11.1. A toy block cipher has block length $n = 3$ bits. (a) How many distinct permutations of $\{0,1\}^3$ exist? (b) If the key is 4 bits, what fraction of all permutations can the cipher possibly realize? (c) Explain in one or two sentences why this gap is unavoidable for any real cipher, and why it is not, by itself, a security flaw.

Exercise 11.2. Using the toy Feistel cipher of Section 11.4 (the same S-box, rotation, and round keys $k_1 = \mathrm{A}, k_2 = 3, k_3 = \mathrm{C}$), encrypt the plaintext byte $P = \texttt{1100 0101}_2$ by hand. Show (L_i, R_i) after each round and give the final ciphertext in hex.

Exercise 11.3. Take the ciphertext $C = \texttt{8A}_{16}$ from the worked example and decrypt it fully by hand, recovering $\texttt{6B}_{16}$. Write out all three round-undo steps, and at each step state the value of $f(k_i, \cdot)$ you reuse. Confirm you never invert the S-box.

Exercise 11.4. Evaluate the DES S-box S_1 (using the row/column convention of Example 11.1) on the 6-bit inputs `000000`, `111111`, and `101010`. For each, state which row and column you used. You may take S_1's row 0 to be $(\mathrm{E}, 4, \mathrm{D}, 1, 2, \mathrm{F}, \mathrm{B}, 8, 3, \mathrm{A}, 6, \mathrm{C}, 5, 9, 0, 7)$; look up the other rows from a DES reference table.

Exercise 11.5. Modify the Python toy cipher to run 16 rounds (repeat the three round keys cyclically). Encrypt $P = \texttt{0x6B}$ and $P' = \texttt{0x6A}$ (one bit different) and count how many of the 8 ciphertext bits differ. Repeat for several plaintext pairs differing in one bit and report the average. How close to 4 (half) do you get, and what does that say about diffusion versus round count?

Exercise 11.6. Prove from the round equations that one Feistel round $\rho_k(L, R) = (R,\ L \oplus f(k, R))$ is a bijection on $\{0,1\}^n$ for *every* function f, by exhibiting its inverse explicitly and verifying $\rho_k^{-1} \circ \rho_k = \mathrm{id}$. Where, exactly, does the proof use that f need not be invertible?

Exercise 11.7. A machine tests 10^{12} DES keys per second. (a) How long, on average, to find a 56-bit key by exhaustive search (expect to scan half the space)? (b) The same machine attacks AES-128. How long on average? Express the answer in a way that makes the gap concrete (compare to the age of the universe, about 4×10^{17} seconds).

Exercise 11.8. Explain why the meet-in-the-middle attack of Section 11.6.1 reduces Double DES to roughly 2^{57} time rather than 2^{112}. (a) What resource does it consume heavily in exchange? (b) State the rough time and memory cost of the analogous attack on Triple DES, and conclude how many bits of security 3DES actually offers.

Exercise 11.9. 3DES has a 168-bit key but is considered to offer about 112 bits of security and is nonetheless deprecated. (a) Why 112 and not 168? (b) Separately from the key length, name the *block-size* weakness that motivated deprecation, identify the attack (SWEET32), and estimate after roughly how many blocks under one key a 64-bit-block cipher starts leaking, citing the birthday bound $q^2/2^n$.

Exercise 11.10. Differential cryptanalysis (Example 11.2) leans on the S-box *difference distribution table*. Take the toy S-box of Section 11.4 (the 4-bit table S) and fix the input difference $\Delta_{\text{in}} = \texttt{0001}_2$. (a) For all 16 inputs x, compute $\Delta_{\text{out}} = S[x] \oplus S[x \oplus \texttt{0001}]$ and tally how often each Δ_{out} value appears. (b) If the S-box were a random function, roughly how many of the 16 pairs would you expect per output difference? (c) Does any output difference occur noticeably more often than that baseline, and what would such a peak mean for an attacker?

Exercise 11.11. Reproduce the worked avalanche trace of Section 11.4: encrypt $P' = \texttt{6A}$ by hand and confirm the ciphertext is `C6`. Then XOR it against the original `8A` and verify three of the eight bits differ. Now flip a *different* single input bit of `6B` (your choice) and repeat; do you get three again, more, or fewer? What does the spread of these counts say about why one trial is not enough to judge diffusion?

Exercise 11.12. Suppose someone replaces all eight DES S-boxes with *linear* functions over $\mathbb{F}_2$ (each output bit an XOR of some input bits). Argue that the entire cipher then becomes an affine map over $\mathbb{F}_2$, and sketch why this lets an attacker recover the key from a handful of known plaintext/ciphertext pairs by solving a linear system. What does this tell you about which DES component is essential for security?

Chapter 12

The Advanced Encryption Standard (AES)

In Chapter 11, DES and the Feistel architecture established the template for modern block ciphers, while showing why a 56-bit key and 64-bit block had to be replaced. When you open a web page over HTTPS, connect to a VPN, unlock an encrypted disk, or send a message on Signal today, the bytes leaving your machine are almost always scrambled by one replacement: the Advanced Encryption Standard (AES). Unlike DES, AES was designed in the open as a substitution–permutation network (SPN) on a 128-bit block, its every constant justified in public, and its mathematics is finite-field algebra over $\mathbb{F}_{256}$ that you can do by hand.

This chapter takes AES apart layer by layer: the four operations of a round (`SubBytes`, `ShiftRows`, `MixColumns`, `AddRoundKey`), a single round traced on real bytes, the S-box derived from the multiplicative inverse in $\mathbb{F}_{256}$, the key schedule, and the Wide Trail argument against differential and linear attacks.

12.1 Where AES came from

DES, the cipher of the 1970s and 1980s, was dying. Its 56-bit key was small enough that by the late 1990s a purpose-built machine (the Electronic Frontier Foundation's "Deep Crack") could brute-force it in days. Triple-DES patched the key size but tripled an already slow algorithm. The world needed a replacement, and the U.S. National Institute of Standards and Technology (NIST) did something unusual for a cryptographic standard: it held an open, international competition.

History

In January 1997 NIST announced a call for a new block cipher to replace DES. The requirements: a 128-bit block, support for 128-, 192-, and 256-bit keys, royalty-free worldwide, and fast in both software and hardware. Fifteen submissions came from teams around the globe. Over three years cryptographers publicly attacked every candidate, and NIST held conferences to compare security, speed on everything from 8-bit smart cards to servers, and resistance to side-channel leakage. Five finalists emerged: MARS (IBM), RC6 (RSA Labs), Serpent (Anderson, Biham, Knudsen), Twofish (Schneier et al.), and Rijndael (Joan Daemen

and Vincent Rijmen, two Belgians). Serpent was widely judged the most conservative (with 32 rounds and a huge security margin, though slower in software), while Rijndael used the Wide Trail strategy in 10 to 14 rounds to achieve provable bounds against differential and linear cryptanalysis with high speed across platforms. In October 2000 NIST chose Rijndael, and in 2001 it became FIPS 197: the Advanced Encryption Standard. The name "Rijndael" is a portmanteau of the designers' surnames; "AES" is the standardized profile of it (fixed 128-bit block, three key sizes).

It is worth dwelling on what the open process bought us. DES's S-boxes looked arbitrary, and for years people suspected a backdoor; only in the 1990s was it revealed that IBM and the NSA had *strengthened* them against differential cryptanalysis, a technique the public would not rediscover until 1990. AES took no such leap of faith. Every constant — the irreducible polynomial, the S-box affine map, the MixColumns matrix, the round constants — is published with a design rationale tied to measurable security goals. You can recompute all of them yourself. That transparency is a big part of why the cryptographic community trusts AES.

Focus

The setting. AES is a *block cipher*: a keyed, invertible permutation on 128-bit blocks. Fix a key k of 128, 192, or 256 bits and AES becomes a bijection $\mathrm{Enc}_k : \{0,1\}^{128} \to \{0,1\}^{128}$ with inverse Dec_k. By itself it encrypts exactly one 128-bit block. To encrypt a message of any length — and to do so securely — you wrap it in a *mode of operation* (CTR, CBC, GCM), the subject of the next chapter. Here we study the permutation itself: what Enc_k does to one block.

12.2 The state: bytes in a 4×4 grid

AES works on bytes, not bits. A 128-bit block is 16 bytes, and AES arranges them in a 4×4 matrix called the **state**. The byte order is *column-major*: the first four input bytes fill the first column top to bottom, the next four fill the second column, and so on.

If the input block is the byte sequence $b_0, b_1, \ldots, b_{15}$, the state is

$$\begin{bmatrix} b_0 & b_4 & b_8 & b_{12} \\ b_1 & b_5 & b_9 & b_{13} \\ b_2 & b_6 & b_{10} & b_{14} \\ b_3 & b_7 & b_{11} & b_{15} \end{bmatrix}.$$

Every operation in AES is described as an action on this grid. `SubBytes` replaces each cell independently. `ShiftRows` permutes within rows. `MixColumns` mixes within columns. `AddRoundKey` XORs in key material cell by cell. The grid shape matters for the diffusion argument (Section 12.11): ShiftRows moves bytes *between* columns and MixColumns spreads *within* a column, so the two together touch the whole block in a couple of rounds.

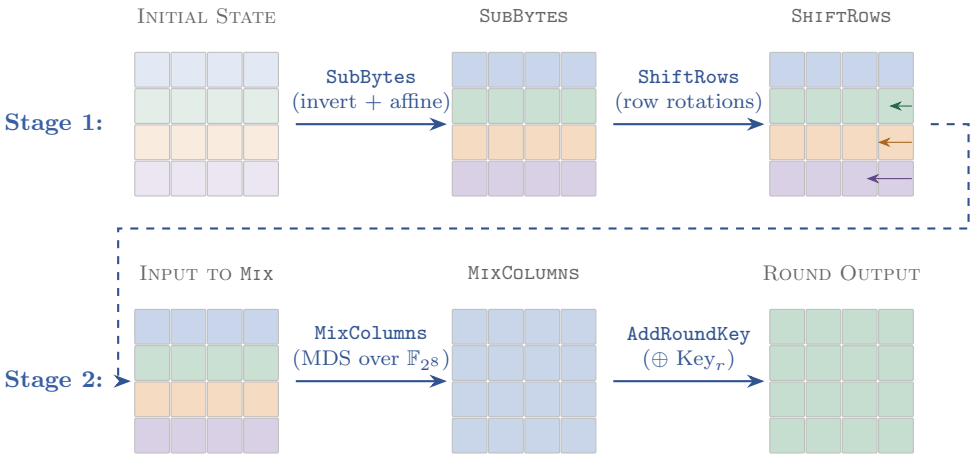


Figure 12.1: The four-layer state transformation across one round of AES. `SubBytes` applies nonlinear inversion and an affine map byte-by-byte. `ShiftRows` shifts rows cyclically to move data across columns. `MixColumns` mixes bytes within each column by matrix multiplication over $\mathbb{F}_{2^8}$. Finally, `AddRoundKey` injects the round subkey via XOR.

Intuition

Picture the state as a 4×4 checkerboard of bytes. AES alternates two kinds of stirring. One kind scrambles each square in place (confusion). The other kind slides squares around and blends neighbors so that a single changed input square quickly contaminates the whole board (diffusion). Repeat enough times and the output looks like noise, with no usable statistical handle for an attacker. The rest of the chapter makes each of these steps precise.

12.2.1 Number of rounds

AES is a *substitution-permutation network* (SPN): there is no Feistel split into halves as in DES. Instead every byte is transformed in every round. The number of rounds depends on the key size:

Key size	Rounds N_r	Round-key words
AES-128	10	44
AES-192	12	52
AES-256	14	60

The high-level structure for AES-128 is:

1. `AddRoundKey` with round key 0 (a "whitening" step).
2. Rounds 1 through 9, each: `SubBytes` $\to$ `ShiftRows` $\to$ `MixColumns` $\to$ `AddRoundKey`.
3. Round 10 (final): `SubBytes` $\to$ `ShiftRows` $\to$ `AddRoundKey`, with **no** MixColumns.

The final round drops MixColumns for a clean reason we will see: it makes decryption symmetric in structure with encryption and adds no security (the last MixColumns could be undone by the attacker by linear algebra without the key, so it would buy nothing).

12.3 The field $\mathbb{F}_{256}$: arithmetic on bytes

To understand `SubBytes` and `MixColumns` you need to think of a byte as an element of the finite field $\mathbb{F}_{256} = \mathbb{F}_{2^8} = \mathrm{GF}(2^8)$. We met finite fields in Chapter 6; here is the working summary AES needs.

A byte $b_7 b_6 \cdots b_1 b_0$ is read as the polynomial

$$b_7 x^7 + b_6 x^6 + \cdots + b_1 x + b_0$$

with coefficients in $\mathbb{F}_2 = \{0, 1\}$. **Addition** of two bytes is just polynomial addition mod 2, i.e. bitwise XOR. There is no carry; addition and subtraction are the same operation: `0x57` $\oplus$ `0x83` = `0xD4`.

Multiplication is polynomial multiplication followed by reduction modulo a fixed irreducible polynomial of degree 8. AES fixes

$$m(x) = x^8 + x^4 + x^3 + x + 1, \qquad \text{in hex, } \texttt{0x11B}.$$

This $m(x)$ has no factors over $\mathbb{F}_2$ (it is irreducible), which is exactly what makes $\mathbb{F}_2[x]/(m(x))$ a field: every nonzero byte has a multiplicative inverse. That is the property `SubBytes` exploits.

Walking question

Multiply `0x57` by `0x83` in $\mathbb{F}_{256}$.
As polynomials, `0x57` $= x^6 + x^4 + x^2 + x + 1$ and `0x83` $= x^7 + x + 1$. Multiply them out (over $\mathbb{F}_2$, so combine like terms mod 2):

$$(x^6 + x^4 + x^2 + x + 1)(x^7 + x + 1).$$

Expanding term by term and cancelling pairs gives

$$x^{13} + x^{11} + x^9 + x^8 + x^6 + x^5 + x^4 + x^3 + 1.$$

This has degree $13 \geq 8$, so reduce modulo $m(x) = x^8 + x^4 + x^3 + x + 1$. Each time a power x^{8+i} appears, replace x^8 by $x^4 + x^3 + x + 1$ (since $x^8 \equiv x^4 + x^3 + x + 1 \pmod{m}$) and keep folding down. Carrying out the reduction yields $x^7 + x^6 + 1 =$ `0xC1`. So `0x57` $\cdot$ `0x83` = `0xC1`. (This is the canonical example from the AES specification; it is worth doing once fully by hand.)

In practice nobody multiplies two arbitrary bytes the long way. The trick AES relies on is multiplication by x, called `xtime`, which is a left shift plus a conditional XOR with `0x1B` (the low byte of $m(x)$):

$$\texttt{xtime}(b) = \begin{cases} b \ll 1 & \text{if the top bit of } b \text{ is 0,} \\ (b \ll 1) \oplus \texttt{0x1B} & \text{if the top bit of } b \text{ is 1.} \end{cases}$$

Multiplying by 2 is `xtime`; multiplying by 3 is $\texttt{xtime}(b) \oplus b$; larger constants are built from these. MixColumns only ever multiplies by 1, 2, and 3, so `xtime` is all the field multiplication you need for encryption.

Connection

The companion algebra book builds $\mathbb{F}_{2^n}$ from scratch: why $\mathbb{F}_2[x]/(m(x))$ is a field exactly when m is irreducible, how to find irreducible polynomials, and why all fields of size 2^8 are isomorphic (so AES's particular choice of $m(x)$ is a convention, not a deep necessity — a different irreducible polynomial would give an equally good but incompatible cipher). The multiplicative group $\mathbb{F}_{256}^*$ is cyclic of order 255; that cyclic structure is what guarantees inverses exist for the S-box.

12.4 SubBytes: the only nonlinear layer

`SubBytes` replaces every byte of the state by another byte, using a single fixed lookup table S of 256 entries (the **S-box**). It is the sole source of nonlinearity in AES. Everything else — ShiftRows, MixColumns, AddRoundKey — is linear (or affine) over $\mathbb{F}_2$, meaning it commutes with XOR. If AES had no nonlinear step, the whole cipher would be one big affine map, and an attacker could solve for the key with linear algebra. The S-box is what stops that.

The S-box is built in two stages.

Stage 1: the multiplicative inverse in $\mathbb{F}_{256}$. Map each byte a to its inverse a^{-1} in the field, with the convention that `0x00` maps to itself (it has no inverse, so we just fix the missing point). This inverse map is highly nonlinear — it is, in a precise sense, about as far from linear as a byte permutation can be, which is exactly why it resists differential and linear cryptanalysis (Section 12.11).

Stage 2: an affine transformation over $\mathbb{F}_2$. The pure inverse map has one cosmetic weakness: it has fixed points (e.g. $0 \mapsto 0$, $1 \mapsto 1$) and a clean algebraic description that an attacker might exploit. So AES post-composes it with an invertible affine map. Writing the byte after inversion as bits $(y_0, \dots, y_7)$, each output bit s_i is

$$s_i = y_i \oplus y_{(i+4) \bmod 8} \oplus y_{(i+5) \bmod 8} \oplus y_{(i+6) \bmod 8} \oplus y_{(i+7) \bmod 8} \oplus c_i,$$

where the constant byte is $c = \texttt{0x63} = (c_0, \dots, c_7) = (1,1,0,0,0,1,1,0)$. In matrix form, over $\mathbb{F}_2$,

$$\begin{bmatrix} s_0 \\ s_1 \\ s_2 \\ s_3 \\ s_4 \\ s_5 \\ s_6 \\ s_7 \end{bmatrix} = \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} y_0 \\ y_1 \\ y_2 \\ y_3 \\ y_4 \\ y_5 \\ y_6 \\ y_7 \end{bmatrix} \oplus \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

The matrix is a *circulant*: each row is the previous one rotated, which is why the formula above is just "XOR five rotated copies."

Why it works

Why split SubBytes into inversion-then-affine? The inverse map $a \mapsto a^{-1}$ gives the cryptographic strength: it has the best known resistance to differential attacks (its differential uniformity is 4, near the theoretical minimum of 2) and to linear attacks. But it is too "algebraic" on its own — it is described by a simple equation over $\mathbb{F}_{256}$, and it fixes 0 and 1. The affine layer is cheap insurance: it breaks the algebraic simplicity over the field (the affine map is $\mathbb{F}_2$-linear but *not* $\mathbb{F}_{256}$-linear, so the composition has no clean field description), removes fixed points, and ensures $S(a) \neq a$ and $S(a) \neq \bar{a}$ for all a. Because the affine map is invertible and linear over $\mathbb{F}_2$, it does not weaken the differential or linear properties that the inverse map provides — it simply destroys polynomial tractability.

Walking question

Compute $S(\texttt{0x53})$ from scratch.
First invert `0x53` in $\mathbb{F}_{256}$. You can check (by the extended Euclidean algorithm on polynomials, or a log table) that $\texttt{0x53}^{-1} = \texttt{0xCA}$. Now apply the affine map to $y = \texttt{0xCA} = (y_0, \ldots, y_7) = (0, 1, 0, 1, 0, 0, 1, 1)$ (bit 0 is the least significant). Computing each $s_i = y_i \oplus y_{i+4} \oplus y_{i+5} \oplus y_{i+6} \oplus y_{i+7} \oplus c_i$ (indices mod 8) gives the byte `0xED`. And indeed, looking up `0x53` in the published S-box table returns `0xED`. The standard AES S-box lookup table simply records the output of this two-stage calculation evaluated across all 256 byte values and stored for fast execution.

Implement it

A toy S-box generator in Python. It builds the field inverse with a log/antilog table over generator 3, then applies the affine map. **For learning only** — real code uses the precomputed table (or hardware) and must avoid the data-dependent memory lookups shown here, which leak timing (see Chapter 37).

```
def xtime(b):                  # multiply by x (i.e. by 2) in GF(2^8)
    b <<= 1
    if b & 0x100:              # overflow past degree 7?
        b ^= 0x11B             # reduce mod x^8+x^4+x^3+x+1
    return b & 0xFF

def gmul(a, b):                # full field multiply
    p = 0
    for _ in range(8):
        if b & 1:
            p ^= a
        a = xtime(a)
        b >>= 1
    return p

# build log / antilog tables using generator g = 3
antilog = [0] * 256
x = 1
for i in range(255):
    antilog[i] = x
    x = gmul(x, 3)             # 3 is a generator of GF(2^8)*
```

```
log = [0] * 256
for i in range(255):
    log[antilog[i]] = i

def inv(a):                    # multiplicative inverse, 0 -> 0
    if a == 0:
        return 0
    return antilog[(255 - log[a]) % 255]

def rotl8(v, n):
    return ((v << n) | (v >> (8 - n))) & 0xFF

def sbox(a):
    y = inv(a)
    # affine: s = y XOR (y<<<1) XOR (y<<<2) XOR (y<<<3) XOR (y<<<4) XOR 0x63
    s = y ^ rotl8(y, 1) ^ rotl8(y, 2) ^ rotl8(y, 3) ^ rotl8(y, 4) ^ 0x63
    return s & 0xFF

assert sbox(0x53) == 0xED
assert sbox(0x00) == 0x63      # 0 -> 0 under inverse, then +0x63
print([hex(sbox(i)) for i in range(8)])
```

12.5 ShiftRows: cheap cross-column movement

`SubBytes` treats each byte independently — it never mixes bytes together. If that were the whole cipher, byte 0 of the ciphertext would depend only on byte 0 of the plaintext, and AES would be 16 tiny independent ciphers. We need to spread information across positions. `ShiftRows` is the first, cheapest spreading step: it cyclically rotates the rows of the state.

$$\begin{bmatrix} a_0 & a_4 & a_8 & a_{12} \\ a_1 & a_5 & a_9 & a_{13} \\ a_2 & a_6 & a_{10} & a_{14} \\ a_3 & a_7 & a_{11} & a_{15} \end{bmatrix} \xrightarrow{\texttt{ShiftRows}} \begin{bmatrix} a_0 & a_4 & a_8 & a_{12} \\ a_5 & a_9 & a_{13} & a_1 \\ a_{10} & a_{14} & a_2 & a_6 \\ a_{15} & a_3 & a_7 & a_{11} \end{bmatrix}$$

Row 0 is left unchanged. Row 1 is rotated left by one byte, row 2 by two, row 3 by three. It is a pure permutation of byte positions — no arithmetic, no key, just relabeling. On its own it does nothing cryptographically; its job is to set up MixColumns. Because MixColumns mixes *within* a column, you need something to move bytes *between* columns first, or the four columns would evolve as four independent sub-ciphers forever. ShiftRows guarantees that after one round the four bytes that were in one column are spread across all four columns, so the next MixColumns blends them with bytes from elsewhere.

Intuition

ShiftRows and MixColumns work together. MixColumns mixes up-and-down inside a column; ShiftRows mixes left-and-right across columns. Together, each output byte after two rounds depends on *every* input byte. Neither alone achieves this: MixColumns keeps each column

isolated, ShiftRows keeps each byte's value untouched. Together they give AES full diffusion in two rounds.

12.6 MixColumns: diffusion inside a column

`MixColumns` is the heavy diffusion step. It treats each column of the state as a vector of four bytes and multiplies it by a fixed 4×4 matrix over $\mathbb{F}_{256}$:

$$\begin{bmatrix} a'_0 \\ a'_1 \\ a'_2 \\ a'_3 \end{bmatrix} = \begin{bmatrix} \texttt{02} & \texttt{03} & \texttt{01} & \texttt{01} \\ \texttt{01} & \texttt{02} & \texttt{03} & \texttt{01} \\ \texttt{01} & \texttt{01} & \texttt{02} & \texttt{03} \\ \texttt{03} & \texttt{01} & \texttt{01} & \texttt{02} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ a_2 \\ a_3 \end{bmatrix},$$

where all matrix operations are performed over the finite field $\mathbb{F}_{2^8}$. Recall how matrix-vector multiplication operates: the i-th element of the resulting vector is the inner product of the i-th row of the matrix with the input column vector:

$$a'_i = \bigoplus_{j=0}^{3} (M_{i,j} \cdot a_j).$$

In $\mathbb{F}_{2^8}$, the field addition $\bigoplus$ is bitwise XOR ($\oplus$), and field multiplication $\cdot$ is polynomial multiplication modulo $m(x) = x^8 + x^4 + x^3 + x + 1$. Because every entry in the matrix is either `01`, `02`, or `03`, computing each product requires no general polynomial long division:

- $\texttt{01} \cdot a = a$ (the multiplicative identity).
- $\texttt{02} \cdot a = \texttt{xtime}(a)$: shift byte a left by 1 bit, and if its high bit before shifting was 1, XOR with the reduction byte `1B`.
- $\texttt{03} \cdot a = (\texttt{02} \oplus \texttt{01}) \cdot a = \texttt{xtime}(a) \oplus a$.

Concretely, the first output byte is:

$$a'_0 = (\texttt{02} \cdot a_0) \oplus (\texttt{03} \cdot a_1) \oplus a_2 \oplus a_3,$$

and the remaining three rows are cyclic rotations of this exact coefficient pattern (Figure 12.2). The matrix is circulant, derived from multiplying by the fixed polynomial $c(x) = \texttt{03}\,x^3 + \texttt{01}\,x^2 + \texttt{01}\,x + \texttt{02}$ modulo $x^4 + 1$ over $\mathbb{F}_{2^8}$.

Walking question

Mix one column. Take the column $(\texttt{D4}, \texttt{BF}, \texttt{5D}, \texttt{30})$, a value that appears in the FIPS-197 worked example. Compute the first output byte:

$$a'_0 = (\texttt{02} \cdot \texttt{D4}) \oplus (\texttt{03} \cdot \texttt{BF}) \oplus \texttt{5D} \oplus \texttt{30}.$$

Now $\texttt{02} \cdot \texttt{D4} = \texttt{xtime}(\texttt{D4})$: the top bit of $\texttt{D4} = 11010100$ is set, so shift left to `1A8` and XOR `1B`, giving $\texttt{A8} \oplus \texttt{1B} = \texttt{B3}$. And $\texttt{03} \cdot \texttt{BF} = \texttt{xtime}(\texttt{BF}) \oplus \texttt{BF}$; $\texttt{xtime}(\texttt{BF})$: top bit of $\texttt{BF} = 10111111$

is set, shift to `17E`, XOR `1B` gives `65`, then $\texttt{65} \oplus \texttt{BF} = \texttt{DA}$. So

$$a'_0 = \texttt{B3} \oplus \texttt{DA} \oplus \texttt{5D} \oplus \texttt{30} = \texttt{04}.$$

Working out all four rows the same way gives the output column $(\texttt{04}, \texttt{66}, \texttt{81}, \texttt{E5})$. The matrix has spread each input byte into all four outputs.

Why it works

Why *these* constants $(\texttt{02}, \texttt{03}, \texttt{01}, \texttt{01})$? Two reasons.

Maximal diffusion. The MixColumns matrix is the generating matrix of a *maximum-distance-separable* (MDS) code. The practical consequence is the "branch number" is 5, the largest possible for a 4×4 matrix: if you change d input bytes of a column ($1 \le d \le 4$), then the number of changed input bytes plus changed output bytes is at least 5. So flipping even a single input byte forces all four output bytes to change. This is the strongest possible single-column avalanche.

Cheapness. The constants $\texttt{01}, \texttt{02}, \texttt{03}$ are the smallest values that achieve MDS, and they need only `xtime` and XOR — no general multiplier. A designer who wanted MDS but used bigger constants would pay for it in every implementation forever. Daemen and Rijmen searched for the cheapest MDS matrix; this is it.

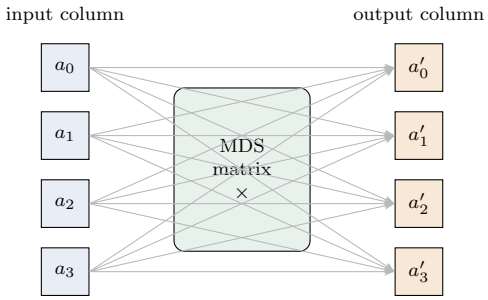


Figure 12.2: `MixColumns` multiplies each 4-byte column by a fixed MDS matrix over $\mathbb{F}_{256}$. Because the branch number is 5, every input byte influences every output byte (all sixteen arrows are live), so changing one input byte changes all four outputs.

Connection

The branch-number argument is coding theory: the rows of the MixColumns matrix, appended to the identity, generate a $[8, 4, 5]$ MDS code over $\mathbb{F}_{256}$, whose minimum distance 5 *is* the branch number. The companion algebra book's treatment of Reed–Solomon and MDS codes is exactly the machinery that proves an attacker cannot find a low-weight differential through MixColumns, which links error-correcting codes to provable resistance to differential cryptanalysis.

12.7 AddRoundKey: where the key finally enters

The three layers so far are fixed and public — they do not depend on the key at all. `AddRoundKey` is the only step that uses secret material, and it could not be simpler: XOR the 128-bit round key into the 128-bit state, byte for byte (equivalently, column for column).

$$\text{state} \leftarrow \text{state} \oplus \text{RoundKey}_r.$$

That is the whole operation. Its own inverse is itself (XOR twice with the same key returns the original). It looks trivial, but it is essential: without it, Enc_k would be a fixed public permutation with no secret, and anyone could decrypt. With it, the attacker faces a different permutation for every key, and the nonlinear SubBytes layers wrapped around the key additions make it computationally infeasible to peel the keys back off.

Common trap

A naive reaction is "AddRoundKey is just XOR, that is weak, like a one-time pad with a reused key." Two things save it. First, the key material is fresh per round (the key schedule generates a new round key each time). Second, and more importantly, AddRoundKey is sandwiched between *nonlinear* SubBytes layers. A plain XOR with a repeated key is breakable because the cipher is linear; AES is not. The key XOR is the simplest possible way to inject the secret, and the nonlinearity around it does the protecting.

12.8 The key schedule

AES-128 needs eleven 128-bit round keys (one for the initial whitening plus one per round). The **key schedule** (or key expansion) stretches the 128-bit cipher key into these $11 \times 16 = 176$ bytes. It is organized as a sequence of 4-byte *words* $W_0, W_1, \ldots, W_{43}$. The first four words are just the cipher key; each later word is derived from earlier ones.

For AES-128:

- W_0, W_1, W_2, W_3 = the four words of the cipher key.
- For $i = 4, \ldots, 43$:

$$W_i = \begin{cases} W_{i-4} \oplus \big(\texttt{SubWord}(\texttt{RotWord}(W_{i-1})) \oplus \texttt{Rcon}_{i/4}\big) & \text{if } 4 \mid i, \\ W_{i-4} \oplus W_{i-1} & \text{otherwise.} \end{cases}$$

Here `RotWord` rotates a word's four bytes left by one ($[a_0, a_1, a_2, a_3] \to [a_1, a_2, a_3, a_0]$), `SubWord` applies the S-box to each of the four bytes, and the **round constant** $\texttt{Rcon}_j = (\text{rc}_j, 0, 0, 0)$ has $\text{rc}_1 = \texttt{01}$ and $\text{rc}_j = \texttt{xtime}(\text{rc}_{j-1})$, i.e. the powers of x in $\mathbb{F}_{256}$:

$$\texttt{01},\ \texttt{02},\ \texttt{04},\ \texttt{08},\ \texttt{10},\ \texttt{20},\ \texttt{40},\ \texttt{80},\ \texttt{1B},\ \texttt{36}.$$

Round key r is the concatenation of words $W_{4r}, W_{4r+1}, W_{4r+2}, W_{4r+3}$, as shown in Figure 12.3.

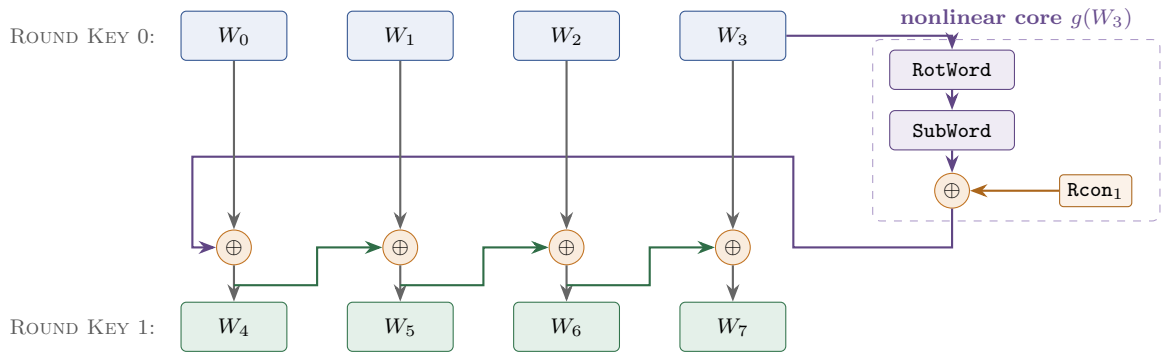


Figure 12.3: One step of the AES-128 key schedule expanding (W_0, W_1, W_2, W_3) into (W_4, W_5, W_6, W_7). Every fourth word $(4 \mid i)$ passes the previous word through the nonlinear core g (`RotWord`, `SubWord`, and $\oplus$ `Rcon`$_{i/4}$) before XORing with W_{i-4}; the remaining three words simply XOR W_{i-1} with W_{i-4}.

Why it works

Every step in the key schedule serves a distinct security purpose.

Nonlinearity (`SubWord`). Without the S-box in the schedule, the round keys would be a linear function of the cipher key, and an attacker who learned one round key could solve for the others. SubWord makes the schedule nonlinear so round keys cannot be linearly related.

Symmetry breaking (`Rcon`). The round constants differ every round (they are distinct powers of x). Without them, the schedule would treat each round identically, creating symmetries an attacker could exploit (related-key and slide attacks thrive on round-to-round symmetry). Distinct `Rcon` values make every round key structurally different.

Rotation (`RotWord`). Rotating before substitution diffuses byte positions across the word, so a change in one key byte spreads to others.

Walking question

Expand the all-zero key, first step. Let the cipher key be sixteen zero bytes, so $W_0 = W_1 = W_2 = W_3 =$ `00000000`. Compute W_4. Since $4 \mid 4$: `RotWord`$(W_3)=$ `00000000`, `SubWord` of that applies $S($`00`$) =$ `63` to each byte giving `63636363`, then XOR `Rcon`$_1 =$ `01000000` gives `62636363`. Finally $W_4 = W_0 \oplus$ `62636363` $=$ `62636363`. Then $W_5 = W_1 \oplus W_4 =$ `62636363`, $W_6 = W_2 \oplus W_5 =$ `62636363`, $W_7 = W_3 \oplus W_6 =$ `62636363`. So round key 1 is `62636363` repeated four times. The S-box and Rcon have already injected structure into what was an all-zero key.

Common trap

The key schedule is the part of AES with the smallest security margin, and it is where most academic "breaks" live. The known *related-key* attacks on AES-256 (Biryukov–Khovratovich, 2009) exploit the simpler key schedule of the 256-bit variant: with 14 rounds but a schedule that does relatively little mixing per round key, an attacker who can query the cipher under keys with chosen *differences* can recover the key faster than brute force. This is mostly of theoretical concern — related-key access is a strong and unusual assumption, and no attack

threatens AES under a single fixed unknown key — but it shows that the key schedule needs as much care as the round function. If you ever design a cipher, do not skimp on key mixing.

12.9 A full round, traced on bytes

Let us run one complete round on a concrete state. We will not do all ten rounds by hand (that is the FIPS-197 appendix's job and runs for pages), but one round shows how the four steps fit together. Start with the post-whitening state used in the standard's example:

$$\text{state} = \begin{bmatrix} \texttt{19} & \texttt{A0} & \texttt{9A} & \texttt{E9} \\ \texttt{3D} & \texttt{F4} & \texttt{C6} & \texttt{F8} \\ \texttt{E3} & \texttt{E2} & \texttt{8D} & \texttt{48} \\ \texttt{BE} & \texttt{2B} & \texttt{2A} & \texttt{08} \end{bmatrix}.$$

Step 1, SubBytes. Replace each byte by its S-box value. For example $S(\texttt{19}) = \texttt{D4}$, $S(\texttt{A0}) = \texttt{E0}$, $S(\texttt{3D}) = \texttt{27}$, and so on. The state becomes

$$\begin{bmatrix} \texttt{D4} & \texttt{E0} & \texttt{B8} & \texttt{1E} \\ \texttt{27} & \texttt{BF} & \texttt{B4} & \texttt{41} \\ \texttt{11} & \texttt{98} & \texttt{5D} & \texttt{52} \\ \texttt{AE} & \texttt{F1} & \texttt{E5} & \texttt{30} \end{bmatrix}.$$

Step 2, ShiftRows. Rotate row 1 left by 1, row 2 by 2, row 3 by 3:

$$\begin{bmatrix} \texttt{D4} & \texttt{E0} & \texttt{B8} & \texttt{1E} \\ \texttt{BF} & \texttt{B4} & \texttt{41} & \texttt{27} \\ \texttt{5D} & \texttt{52} & \texttt{11} & \texttt{98} \\ \texttt{30} & \texttt{AE} & \texttt{F1} & \texttt{E5} \end{bmatrix}.$$

Step 3, MixColumns. Apply the matrix to each column. The first column is $(\texttt{D4}, \texttt{BF}, \texttt{5D}, \texttt{30})$ — exactly the column we mixed by hand earlier — giving $(\texttt{04}, \texttt{66}, \texttt{81}, \texttt{E5})$. Mixing all four columns yields

$$\begin{bmatrix} \texttt{04} & \texttt{E0} & \texttt{48} & \texttt{28} \\ \texttt{66} & \texttt{CB} & \texttt{F8} & \texttt{06} \\ \texttt{81} & \texttt{19} & \texttt{D3} & \texttt{26} \\ \texttt{E5} & \texttt{9A} & \texttt{7A} & \texttt{4C} \end{bmatrix}.$$

Step 4, AddRoundKey. XOR in round key 1, which for this example is

$$\text{RoundKey}_1 = \begin{bmatrix} \texttt{A0} & \texttt{88} & \texttt{23} & \texttt{2A} \\ \texttt{FA} & \texttt{54} & \texttt{A3} & \texttt{6C} \\ \texttt{FE} & \texttt{2C} & \texttt{39} & \texttt{76} \\ \texttt{17} & \texttt{B1} & \texttt{39} & \texttt{05} \end{bmatrix}.$$

XORing cell by cell (e.g. $\texttt{04} \oplus \texttt{A0} = \texttt{A4}$) gives the state that enters round 2:

$$\begin{bmatrix} \texttt{A4} & \texttt{68} & \texttt{6B} & \texttt{02} \\ \texttt{9C} & \texttt{9F} & \texttt{5B} & \texttt{6A} \\ \texttt{7F} & \texttt{35} & \texttt{EA} & \texttt{50} \\ \texttt{F2} & \texttt{2B} & \texttt{43} & \texttt{49} \end{bmatrix}.$$

That is one round. Nine more like it (the last without MixColumns) and the block is encrypted. Notice the avalanche already building: the single input byte `19` in the top-left, after SubBytes, ShiftRows, and MixColumns, has influenced an entire column of outputs, and the next round's ShiftRows will spread those across all four columns. Figure 12.4 summarizes the four layers in order, and Figure 12.5 shows the ShiftRows rotation that carries bytes between columns.

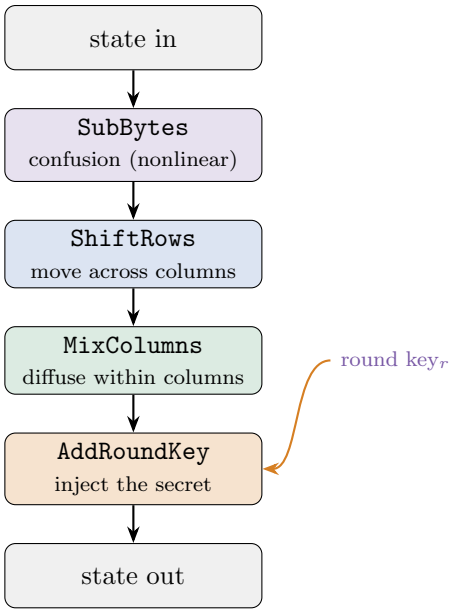


Figure 12.4: One AES round. The first three layers are fixed and public; only `AddRoundKey` uses the secret. The final round omits `MixColumns`.

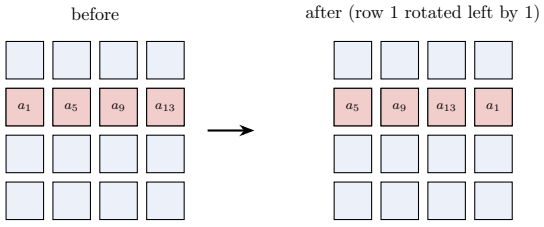


Figure 12.5: `ShiftRows` on row 1: a left cyclic rotation by one byte. Rows 2 and 3 rotate by two and three. This is what carries bytes between columns so the next `MixColumns` blends them.

12.10 Decryption: inverting each layer

Because every layer is invertible, AES decryption simply applies the inverses in reverse order. `InvSubBytes` uses the inverse S-box (apply the inverse affine map, then field-invert). `InvShiftRows` rotates rows right instead of left. `InvMixColumns` multiplies by the inverse of the MixColumns matrix (whose entries are `0E`, `0B`, `0D`, `09` — deliberately larger, which is why naive decryption is a touch slower than encryption). `AddRoundKey` is its own inverse.

With a small reordering trick (the "equivalent inverse cipher"), decryption can be written with the *same* sequence of operations as encryption, just with modified round keys, so hardware can share circuitry. This symmetry is partly why the final round drops MixColumns — it makes the structure line up.

Focus

AES is not a stream cipher and not its own inverse. Encryption and decryption use *different* routines (inverse S-box, inverse matrix, reversed key order). This differs from a Feistel cipher like DES, where decryption is encryption with the round keys reversed and no separate inverse round function is needed. The SPN structure trades that convenience for the ability to transform every byte every round.

12.11 Why AES is secure

"Secure" for a block cipher means it behaves like a *pseudorandom permutation* (PRP): without the key, no efficient adversary can distinguish Enc_k (for a random secret k) from a truly random permutation of 128-bit blocks, even given many chosen plaintext/ciphertext pairs. We cannot prove AES is a PRP from any standard hardness assumption — no one can prove that of any practical cipher — but we can show it resists every known general attack with a comfortable margin, and that is the working standard of trust.

Why it works

Confusion and diffusion in AES. Claude Shannon's two design goals are realized by specific layers.
Confusion means the relationship between key and ciphertext is complicated and nonlinear. SubBytes provides it: the only nonlinear step, with the best available resistance to algebraic attacks.
Diffusion means each plaintext (and key) bit influences many ciphertext bits, so statistics smear out. ShiftRows + MixColumns provide it, as traced in Figure 12.6. The *wide-trail strategy* of Daemen and Rijmen quantifies this: the MixColumns branch number is 5, and over four rounds any differential or linear trail must activate at least 25 S-boxes. Each active S-box contributes a factor bounding the trail's probability (at most 2^{-6} for differentials), so any four-round trail has probability at most $(2^{-6})^{25} = 2^{-150}$, far below the 2^{-128} threshold an attacker would need. Ten rounds leave a large security margin.

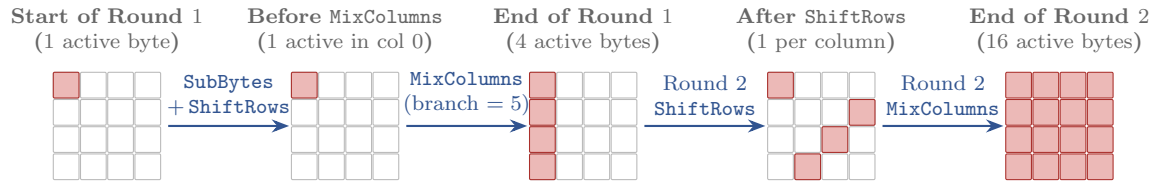


Figure 12.6: Two-round full diffusion via the wide-trail strategy. A single perturbed byte (red) spreads to all four bytes of its column after Round 1's `MixColumns` (branch number 5). Round 2's `ShiftRows` distributes those four active bytes into four distinct columns, and Round 2's `MixColumns` expands each into a full column, activating all 16 state bytes in just two rounds.

Attack

What attacks exist, and why they fall short.
Brute force on AES-128 needs 2^{128} trial decryptions — physically impossible (the energy alone exceeds what is available).
Differential cryptanalysis tracks how input differences propagate. The wide-trail bound above rules it out: no usable four-round differential exists, let alone ten-round.
Linear cryptanalysis looks for biased linear approximations; the same 25-active-S-box bound makes any biased relation vanish below detectability.
Biclique attack (Bogdanov, Khovratovich, Rechberger, 2011) is the best known key-recovery on full AES-128: it costs about $2^{126.1}$ — a factor of roughly 4 better than brute force. After two decades of intense scrutiny, that is the best known result against the full cipher.
Related-key attacks on AES-192/256 (cost around $2^{99.5}$ for AES-256 in the related-key model) are real but require the attacker to control key differences, an assumption that almost never holds in practice.
The lesson: do not expect to break AES's math. The real attacks on AES deployments are *implementation* attacks — below.

Attack

Cache-timing: the practical break. The original software AES used a 256-entry table for SubBytes (and combined tables for the round). Table lookups take different amounts of time depending on whether the indexed entry is in the CPU cache, and the index depends on secret state. In 2005 Daniel Bernstein, and independently Osvik–Shamir–Tromer, showed you can recover an AES key by measuring timing or by priming and probing the cache from another process on the same machine — no break of the math required, just a leak through the implementation. The fix that won: the **AES-NI** instruction set (Intel 2010, now everywhere), which executes a whole round in hardware in constant time, with no data-dependent table lookups. If you must do AES in software without AES-NI, use a *bitsliced* implementation that has no secret-indexed memory access. **Never** ship the textbook table-lookup version in production.

Connection

The distinguishing game — "tell Enc_k apart from a random permutation" — is the PRP security definition built in Chapters 8 and 11. The 2^{-150} trail probabilities are union bounds over many possible trails; the companion probability book's treatment of union bounds and tail

estimates is the exact tool. And the 2^{64} "birthday" limit on how many blocks you may safely encrypt under one key in a chaining mode (next chapter) comes straight from the birthday bound on a 128-bit block size.

12.12 Performance and where AES runs

AES was designed to be fast in software *and* cheap in hardware. The four layers are byte operations and XORs; the classic software trick (Daemen and Rijmen's "T-tables") folds SubBytes, ShiftRows, and MixColumns into four 1024-byte lookup tables so a round is sixteen table reads and a handful of XORs. With AES-NI, a single `aesenc` instruction does an entire round; modern cores encrypt at several gigabytes per second per core, and the constant-time hardware sidesteps the cache-timing problem entirely.

In the real world

AES in the wild.
TLS / HTTPS. The dominant cipher suite is `AES-GCM` (AES in counter mode with a Galois MAC), giving authenticated encryption. On a phone without strong AES hardware, TLS 1.3 may instead pick ChaCha20-Poly1305, which is faster in pure software — but on anything with AES-NI, AES-GCM wins.
WiFi (WPA2 / WPA3). The CCMP cipher is AES in CCM mode (counter mode plus CBC-MAC). When you join a protected network, AES is encrypting your frames.
Disk encryption. BitLocker, FileVault, LUKS, and Android/iOS file-based encryption all use AES (in XTS mode for full-disk, where ciphertext stealing and a tweak handle fixed-size sectors).
Signal / iMessage / Telegram. The message payload encryption uses AES (typically AES-CBC or AES-CTR with a separate HMAC, or AES-GCM).
VPNs and SSH. IPsec, WireGuard's fallback paths, OpenVPN, and SSH all offer AES-GCM or AES-CTR transport encryption.
In every case AES encrypts the bulk data; the asymmetric crypto (RSA, ECDH) only sets up the AES key.

Why it works

[Post-quantum margins and key sizes] AES-128 offers 128-bit security against classical attackers, far beyond the reach of brute force. Grover's quantum search algorithm would, in principle, halve the effective key strength to 64 bits of quantum search — which is why guidance for post-quantum margins often recommends AES-256 (leaving 128 bits of strength even against an idealized quantum brute-forcer). Unlike RSA and elliptic curves, which Shor's algorithm breaks outright, AES survives the quantum era essentially intact by just using a longer key. This is one more reason AES is expected to stay in use for decades.

12.13 Putting the whole cipher together

For reference, here is the complete AES-128 encryption in pseudocode, every piece of which we have now justified:

```
# AES-128 encryption, structural pseudocode (LEARNING ONLY).
# Real use: call a vetted library (which uses AES-NI / constant time).
def aes128_encrypt(plaintext16, key16):
    state = to_state(plaintext16)          # 4x4 column-major bytes
    rk = key_expansion(key16)              # 11 round keys, words W0..W43

    add_round_key(state, rk[0])            # initial whitening
    for r in range(1, 10):                 # rounds 1..9
        sub_bytes(state)                   # nonlinear: S-box each byte
        shift_rows(state)                  # rotate rows 0,1,2,3 by 0,1,2,3
        mix_columns(state)                 # MDS mix within each column
        add_round_key(state, rk[r])        # XOR round key
    sub_bytes(state)                       # round 10: NO MixColumns
    shift_rows(state)
    add_round_key(state, rk[10])
    return from_state(state)
```

Each line maps to a design argument: SubBytes provides confusion, ShiftRows and MixColumns provide diffusion, AddRoundKey injects the secret, the loop count sets the security margin, and omitting the final MixColumns keeps decryption symmetric at no security cost.

AES scrambles a single 16-byte block; to encrypt files, packets, and streams of arbitrary length without leaking patterns across blocks, Chapter 13 builds block-cipher modes of operation (ECB, CBC, CTR, CFB, and OFB).

Pause

Before the exercises, a quick self-check. Can you (1) state the four round operations and which one is nonlinear; (2) explain why ShiftRows and MixColumns are useless apart but achieve full diffusion together; (3) derive an S-box entry from the field inverse plus the affine map; (4) say why AES-128 uses ten rounds rather than, say, four; and (5) name the real attack class that threatens AES deployments (timing/cache side channels, not the math)? If any of these is shaky, reread the matching section before moving on.

12.14 Exercises

Exercise 12.1. Compute $\texttt{xtime}(\texttt{0x80})$, $\texttt{xtime}(\texttt{0x7F})$, and $\texttt{0x02} \cdot \texttt{0xFF}$ in $\mathbb{F}_{256}$ with the AES polynomial. Show the shift-and-conditional-XOR for each.

Exercise 12.2. Verify by hand that $\texttt{0x53} \cdot \texttt{0xCA} = \texttt{0x01}$ in $\mathbb{F}_{256}$, confirming that $\texttt{0xCA}$ is the multiplicative inverse of $\texttt{0x53}$ used in the worked S-box example. (Multiply the two polynomials and reduce mod $m(x)$.)

Exercise 12.3. Using the affine formula $s_i = y_i \oplus y_{i+4} \oplus y_{i+5} \oplus y_{i+6} \oplus y_{i+7} \oplus c_i$ with $c = \texttt{0x63}$, compute $S(\texttt{0x00})$ and $S(\texttt{0x01})$ from their field inverses ($0^{-1} = 0$ by convention, $1^{-1} = 1$). Confirm the famous values $S(\texttt{0x00}) = \texttt{0x63}$ and $S(\texttt{0x01}) = \texttt{0x7C}$.

Exercise 12.4. Diffusion. Take any single column of four bytes, flip exactly one bit of one byte, and run both versions through MixColumns. Show that all four output bytes change. Explain how this demonstrates the branch number being greater than 1, and why ShiftRows is needed for the change to spread to other columns in the next round.

Exercise 12.5. Run `ShiftRows` on the state whose rows are $(\texttt{00}, \texttt{01}, \texttt{02}, \texttt{03})$, $(\texttt{10}, \texttt{11}, \texttt{12}, \texttt{13})$, $(\texttt{20}, \texttt{21}, \texttt{22}, \texttt{23})$, $(\texttt{30}, \texttt{31}, \texttt{32}, \texttt{33})$. Write the resulting state.

Exercise 12.6. Key schedule. Continue the all-zero-key expansion from the worked example: compute words W_8 through W_{11} (so round key 2). You will need $\texttt{Rcon}_2 = \texttt{02000000}$ and the S-box value $S(\texttt{0x63}) = \texttt{0xFB}$. Show each step.

Exercise 12.7. Explain why AES omits `MixColumns` in its final round. Argue that if the final MixColumns were present, an attacker could strip it off without knowing the key, so it would add no security. (Hint: MixColumns is a fixed, public, invertible linear map.)

Exercise 12.8. Counting active S-boxes. The wide-trail argument claims any four-round differential trail activates at least 25 S-boxes. Sketch the argument: one round of MixColumns has branch number 5, so a single active column in one round forces at least 5 active S-boxes across two consecutive rounds; combine across the 4×4 structure over four rounds to reach 25. Then estimate the trail probability bound using 2^{-6} per active S-box.

Exercise 12.9. Break a toy SPN. Build a one-round "AES" with *no* SubBytes (only ShiftRows, MixColumns, AddRoundKey). Show that the entire map from plaintext to ciphertext is affine over $\mathbb{F}_2$, and describe how an attacker recovers the round key from a few known plaintext/ciphertext pairs by solving a linear system. This is precisely why the nonlinear SubBytes layer cannot be removed.

Exercise 12.10. Side channel. Explain why a table-lookup implementation of SubBytes can leak the key through cache timing, while an AES-NI hardware round or a bitsliced software implementation does not. What property of the memory-access pattern is the difference?

Exercise 12.11. AES-256 uses 14 rounds and a slightly different key schedule than AES-128. Given that the best single-key attack on AES-128 (biclique) is about 2^{126} and related-key attacks on AES-256 reach about $2^{99.5}$, argue why "more rounds and a longer key" did not straightforwardly make AES-256 stronger than AES-128 in the related-key model, and why this is nonetheless not a practical concern for most deployments.

Exercise 12.12. Implementation. Using the toy S-box generator from this chapter as a start, write functions `shift_rows`, `mix_columns`, and `add_round_key`, plus a `key_expansion`, and reproduce the single-round trace in Section 12.9 (state in $\rightarrow$ state out). Then encrypt the FIPS-197 test vector (plaintext `00112233445566778899aabbccddeeff`, key `000102030405060708090a0b0c0d0e0f`) and check you get ciphertext `69c4e0d86a7b0430d8cdb78070b4c55a`. State clearly in a comment that this code is for learning, not production.

Chapter 13

Modes of Operation

In Chapter 12, AES gave us a fast, secure permutation for a single 128-bit block (16 bytes, about the length of "`attack at dawn!!`"). Real messages are not 16 bytes long – an email is kilobytes, a disk image is gigabytes – so the first engineering question after building a block cipher is how to encrypt a *long* message with a primitive that only eats one block at a time.

The naive answer – chop the message into blocks and encrypt each one independently (ECB mode) – leaks structure so loudly that you can often read the plaintext straight off the ciphertext. The fix is a *mode of operation*: a recipe for chaining the block cipher's invocations with feedback, counters, and fresh randomness (CBC, CTR, CFB, and OFB) so that the ciphertext reveals nothing beyond message length, while showing why CTR mode became the basis of modern authenticated encryption.

13.1 The block cipher as a tool, and what it is not

Let $E\colon \{0,1\}^k \times \{0,1\}^n \to \{0,1\}^n$ be a block cipher. For a fixed key K the map $E_K(\cdot) = E(K,\cdot)$ is a permutation of $\{0,1\}^n$: it is a bijection, so $E_K^{-1} = D_K$ exists and decryption recovers the block. For AES, $n = 128$; for the older DES, $n = 64$.

The security goal we proved (or assumed) for the block cipher itself is that E_K is a *pseudorandom permutation* (PRP): to anyone who does not know K, the function E_K is computationally indistinguishable from a permutation drawn uniformly at random from all $2^n!$ permutations of n-bit strings. That is a statement about *one* block. It says nothing about what happens when you call E_K many times on related inputs. A mode of operation takes this per-block guarantee and lifts it to a per-message guarantee.

Focus

A block cipher is a building block, not an encryption scheme. "Encrypt this file with AES" is an underspecified instruction – it is like saying "build this house with bricks" without saying how the bricks are stacked. The mode of operation specifies how the cipher is applied across blocks, and a bad mode makes the scheme insecure no matter how strong the cipher is.

13.1.1 The security target: IND-CPA

What does "secure" mean for a scheme that encrypts long messages? We carry over the notion from earlier chapters: *indistinguishability under a chosen-plaintext attack* (IND-CPA). Picture the game between a challenger and an adversary Adv.

1. The challenger picks a key $K \xleftarrow{\$} \{0,1\}^k$ and a secret bit $b \xleftarrow{\$} \{0,1\}$.

2. Adv may submit any messages it likes and receive their encryptions (this is the "chosen plaintext" power – it models the reality that attackers can often induce a victim to encrypt data they choose).

3. At some point Adv submits two equal-length messages m_0, m_1. The challenger returns $\mathrm{Enc}_K(m_b)$.

4. Adv may keep querying, then outputs a guess b'. It wins if $b' = b$.

The scheme is IND-CPA secure if every efficient Adv wins with probability at most $\frac{1}{2} + \mathsf{negl}$. The intuition is exactly what you want from encryption: even an attacker who can get arbitrary plaintexts encrypted, and who picks the two candidate messages itself, cannot tell which of the two was encrypted any better than flipping a coin.

Why it works

IND-CPA has an important consequence: **encryption must be randomized (or stateful)**. Suppose Enc_K were a deterministic function. Then Adv asks for $\mathrm{Enc}_K(m_0)$ during the query phase, then submits m_0, m_1 as its challenge. If the challenge ciphertext equals the one it already saw, $b = 0$; otherwise $b = 1$. It wins with probability 1. A deterministic encryption scheme *cannot* be IND-CPA secure. This observation alone rules out ECB and explains why every good mode injects either a random IV or a counter.

13.2 ECB: the mode you must never use

Electronic Codebook (ECB) mode is the naive recipe. Split the message into blocks $m_1, m_2, \ldots, m_\ell$, each n bits, and encrypt each one independently with the same key:

$$c_i = E_K(m_i), \qquad i = 1, \ldots, \ell.$$

Decryption is just $m_i = D_K(c_i)$. The name comes from the idea that E_K acts like a giant lookup table – a "codebook" – mapping each possible block to its ciphertext.

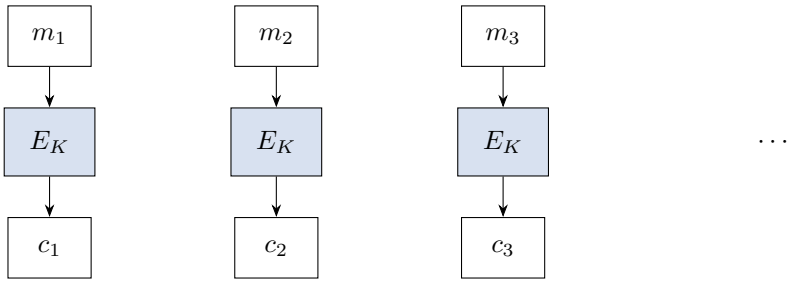


Figure 13.1: ECB mode: each block goes through E_K with no interaction. Identical plaintext blocks produce identical ciphertext blocks.

The structural failure is directly visible in Figure 13.1: there is no communication or chaining between columns. Each block is encrypted in total isolation. Because E_K is deterministic, **equal plaintext blocks always produce equal ciphertext blocks**. Statistical patterns and identical blocks in the plaintext survive into the ciphertext without alteration.

Intuition

Think of ECB as a substitution cipher whose alphabet is the set of all n-bit blocks instead of the 26 letters. In Chapter 2 we watched frequency analysis break substitution ciphers. ECB is the same idea scaled up: the "alphabet" is astronomically large, so you cannot do classical letter-frequency analysis, but *any repetition in the plaintext at block granularity becomes a visible repetition in the ciphertext.* For structured data – bitmaps, fixed-format records, repeated headers – that is plenty.

13.2.1 A tiny ECB trace

Let us make this concrete with an absurdly small toy cipher so we can do it by hand. Take a block size of $n = 4$ bits, and let our "cipher" be a fixed permutation E_K of the 16 possible nibbles given by this table (this is just an arbitrary bijection standing in for AES):

x	0	1	2	3	4	5	6	7	8	9	A	B	C	D	E	F
$E_K(x)$	6	4	C	5	0	7	2	E	1	F	3	D	8	A	9	B

Our blocks are single hex digits (nibbles). Take the plaintext stream of nibbles $m = (\texttt{A}, \texttt{1}, \texttt{A}, \texttt{1}, \texttt{C}, \texttt{A})$. ECB gives:

$$c = \big(E_K(\texttt{A}),\, E_K(\texttt{1}),\, E_K(\texttt{A}),\, E_K(\texttt{1}),\, E_K(\texttt{C}),\, E_K(\texttt{A})\big) = (\texttt{3}, \texttt{4}, \texttt{3}, \texttt{4}, \texttt{8}, \texttt{3}).$$

Look at the ciphertext: `3 4 3 4 8 3`. The pattern `A,1` repeated in the plaintext shows up as `3,4` repeated in the ciphertext, and every `A` maps to `3`. An eavesdropper who never learns the key still sees exactly where the plaintext repeated itself. With a 4-bit block this is unavoidable; with a 128-bit block it is just as unavoidable whenever the plaintext contains 16-byte-aligned repeats.

13.2.2 The penguin

Attack

The canonical demonstration is the "ECB penguin." Take a bitmap image of Tux the Linux penguin, encrypt the pixel data with AES in ECB mode, and re-interpret the ciphertext as an image. Large regions of the picture are a solid color – say, the white background, where many consecutive 16-byte blocks are byte-for-byte identical. ECB maps each of those identical blocks to the same ciphertext block, so the encrypted image still has those regions as a single uniform (different) color. The result is an image in which the penguin's outline, beak, belly, and feet are perfectly legible. Each block was encrypted correctly, yet the image content leaked anyway.

The lesson generalizes far past images. Encrypt a database of fixed-width records in ECB and you reveal which records share field values. Encrypt a protocol with repeated headers and you reveal the message boundaries and repeats. ECB does not hide *structure*; it only hides the contents of a single isolated block.

Why does the penguin work mathematically? Because ECB is *deterministic* and *stateless across blocks*, it fails the IND-CPA game in one move. Submit $m_0 = (P, P)$ – the same block twice – and $m_1 = (P, Q)$ with $P \neq Q$. The challenge ciphertext for m_0 has its two halves equal; for m_1 they differ. The adversary just checks whether $c_1 = c_2$ and wins with probability 1. The attack needs no key recovery and no number theory; the repeated structure gives the answer directly.

Common trap

"But my data is high-entropy / already compressed / random-looking, so ECB is fine." This is the single most common rationalization, and it is a trap. First, you rarely control the plaintext as tightly as you think – protocol framing, padding, and fixed file-signature bytes sneak repeats in. Second, a mode is a long-lived decision, and the next engineer will feed it structured data. Third, ECB still fails IND-CPA in principle, which means a security proof for the surrounding protocol simply does not exist. There is no scenario in which choosing ECB is the right call over CTR or CBC. Treat its appearance in a code review as a bug.

13.3 CBC: chaining blocks with feedback

The fix is to make each block's encryption depend on everything before it, and to inject fresh randomness at the start so the same message encrypts differently each time. Cipher Block Chaining (CBC) does both. Choose a random n-bit *initialization vector* IV, then:

$$c_0 = IV, \qquad c_i = E_K(m_i \oplus c_{i-1}), \quad i = 1, \ldots, \ell.$$

Each plaintext block is XORed with the previous ciphertext block before encryption, as Figure 13.2 shows. The IV plays the role of a "zeroth ciphertext block." The ciphertext you transmit is $(IV, c_1, \ldots, c_\ell)$ – the IV travels in the clear, which is fine; it is not secret, it only needs to be unpredictable.

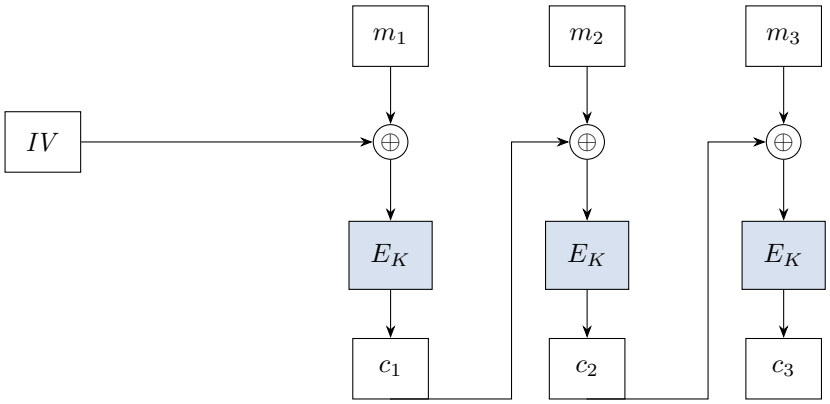


Figure 13.2: CBC encryption. Each plaintext block is XORed with the previous ciphertext block (the IV seeds the first). The chaining removes the ECB pattern leak.

Decryption inverts the chain. Since $c_i = E_K(m_i \oplus c_{i-1})$, apply D_K and XOR back:

$$m_i = D_K(c_i) \oplus c_{i-1}, \qquad c_0 = IV.$$

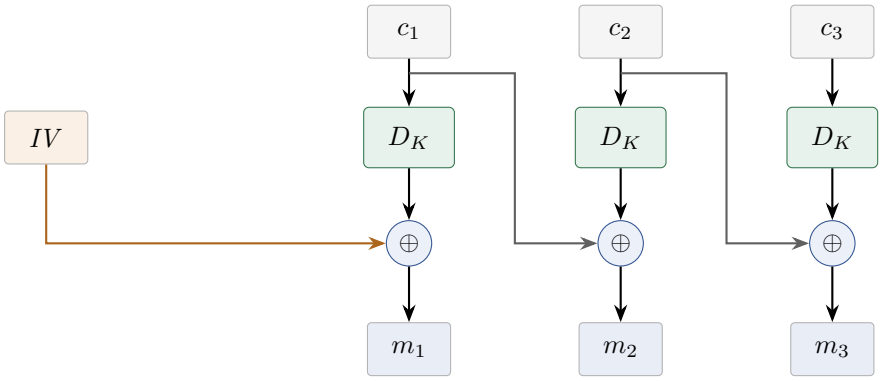


Figure 13.3: CBC decryption. Unlike encryption, CBC decryption is fully parallelizable across cores because every ciphertext block c_i is already available and can be decrypted independently by D_K before XORing with c_{i-1}.

Intuition

The chaining means a single plaintext block's ciphertext depends on that block *and every block before it* (through the cascade of XORs). So two identical plaintext blocks at different positions almost never produce the same ciphertext: the previous ciphertext block they get XORed with is different. The encrypted penguin image looks like noise. And because the IV is fresh per message, encrypting the same file twice yields two unrelated ciphertexts – exactly the randomization IND-CPA demands.

13.3.1 A tiny CBC trace

Reuse the 4-bit toy permutation E_K from the ECB section. Encrypt the nibble stream $m = (\texttt{A}, \texttt{1}, \texttt{A})$, and pick the random IV to be $\texttt{D}$ ($= 1101_2$). XOR is bitwise.

- $c_1 = E_K(m_1 \oplus IV) = E_K(\texttt{A} \oplus \texttt{D})$. Now $\texttt{A} = 1010$, $\texttt{D} = 1101$, so $\texttt{A} \oplus \texttt{D} = 0111 = \texttt{7}$. Then $E_K(\texttt{7}) = \texttt{E}$. So $c_1 = \texttt{E}$.
- $c_2 = E_K(m_2 \oplus c_1) = E_K(\texttt{1} \oplus \texttt{E})$. $\texttt{1} = 0001$, $\texttt{E} = 1110$, XOR $= 1111 = \texttt{F}$. $E_K(\texttt{F}) = \texttt{B}$. So $c_2 = \texttt{B}$.
- $c_3 = E_K(m_3 \oplus c_2) = E_K(\texttt{A} \oplus \texttt{B})$. $\texttt{A} = 1010$, $\texttt{B} = 1011$, XOR $= 0001 = \texttt{1}$. $E_K(\texttt{1}) = \texttt{4}$. So $c_3 = \texttt{4}$.

Ciphertext: $(IV, c_1, c_2, c_3) = (\texttt{D}, \texttt{E}, \texttt{B}, \texttt{4})$. Notice that the two identical plaintext blocks $m_1 = m_3 = \texttt{A}$ produced *different* ciphertext blocks E and 4. The ECB pattern is gone.

Let us also verify decryption of the last block to confirm the inverse formula. We need D_K, the inverse of the table: since $E_K(\texttt{1}) = \texttt{4}$, we have $D_K(\texttt{4}) = \texttt{1}$. Then $m_3 = D_K(c_3) \oplus c_2 = \texttt{1} \oplus \texttt{B} = 0001 \oplus 1011 = 1010 = \texttt{A}$. Recovered.

13.3.2 Why the IV must be unpredictable

The IV's job is to randomize the first block. But CBC is fussier than "the IV must be random." It must be *unpredictable to the adversary at the moment of encryption*. Here is the attack when it is not.

Attack

Predictable-IV attack on CBC (the BEAST family). Suppose the attacker can predict the IV that will be used for the next message – for instance, an implementation that sets the next IV to the last ciphertext block of the previous message (an old SSL/TLS 1.0 habit). The attacker also has chosen-plaintext power: it can ask the victim to encrypt blocks of its choice. CBC's first block is $c_1 = E_K(m_1 \oplus IV)$. The attacker wants to test a guess g for some secret block s that was encrypted earlier: it saw the ciphertext block $c^\star = E_K(s \oplus IV^\star)$ under a known earlier IV $IV^\star$. It now gets to choose the plaintext for a fresh encryption whose IV it can predict to be IV'. It submits the plaintext block

$$m_1 = g \oplus IV^\star \oplus IV'.$$

Then the cipher computes $c_1 = E_K(m_1 \oplus IV') = E_K(g \oplus IV^\star)$. If the guess was right, $g = s$, so $c_1 = E_K(s \oplus IV^\star) = c^\star$ – an exact match with the earlier ciphertext block. The attacker confirms or refutes its guess for a secret block. Combined with byte-at-a-time alignment tricks, this recovers secrets like session cookies. This is the heart of the 2011 BEAST attack on TLS 1.0, which is exactly why TLS 1.1+ switched to fresh *random* per-record IVs.

Why it works

The IND-CPA proof for CBC requires the IV to be a fresh uniform random value, independent of everything the adversary has seen, chosen *after* the adversary commits its plaintext. A predictable IV breaks that independence, so the proof no longer applies, and the attack above shows that the gap matters in practice. The rule to memorize: **CBC needs IVs that are unpredictable and unique. Generate them from a CSPRNG, never from a counter, a timestamp, or the previous ciphertext.**

13.3.3 CBC malleability and the bit-flipping attack

A fundamental lesson of applied cryptography is that confidentiality does not imply integrity. CBC mode guarantees that an eavesdropper cannot read the plaintext, but it provides zero protection against active tampering. An attacker who modifies ciphertext in transit can induce deterministic, predictable changes in the decrypted plaintext.

Recall the CBC decryption equation for block i:

$$m_i = D_K(c_i) \oplus c_{i-1}.$$

Suppose an active attacker intercepts the ciphertext stream and replaces block c_1 with a modified block $c_1' = c_1 \oplus \Delta$, where Δ is an arbitrary bitmask chosen by the attacker. When the recipient decrypts:

1. **Block 1 is pseudo-randomly scrambled:** The recipient computes $m_1' = D_K(c_1 \oplus \Delta) \oplus IV$. Because D_K is a secure pseudorandom permutation, flipping even a single bit of its input causes approximately half of its output bits to flip unpredictably. Plaintext block m_1' is completely garbled.
2. **Block 2 is modified with algebraic precision:** The recipient computes:

$$m_2' = D_K(c_2) \oplus c_1' = D_K(c_2) \oplus (c_1 \oplus \Delta) = \big(D_K(c_2) \oplus c_1\big) \oplus \Delta = m_2 \oplus \Delta.$$

Notice the cancellation: because c_1' enters the second block only as a feedforward XOR mask, the block cipher D_K never touches Δ. The exact difference Δ passes straight into the plaintext m_2!

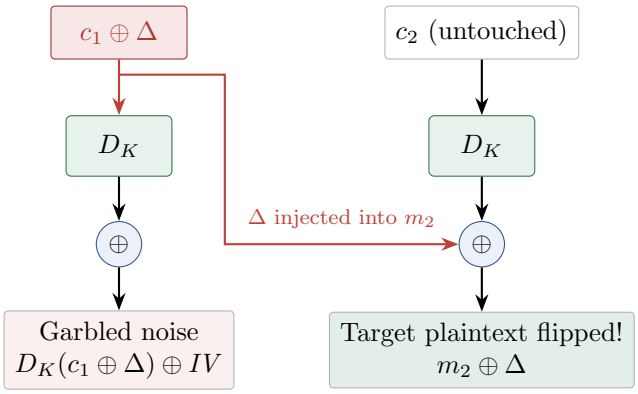


Figure 13.4: The CBC bit-flipping attack. Modifying ciphertext block c_1 by XORing Δ destroys the contents of m_1, but injects the exact bitmask Δ into m_2 without requiring knowledge of key K.

If an attacker knows that block m_2 contains a predictable permission string such as `"role=user;"`, they can craft Δ so that XORing it flips `"user"` to `"root"`. The victim server may discard or ignore the corrupted block m_1 (for instance, treating it as an unimportant transaction ID or header) and process the tampered block m_2 with elevated privileges. Confidentiality alone cannot prevent forgery.

13.3.4 Sequential, and the padding problem

CBC has two practical annoyances. First, encryption is inherently *sequential*: c_i needs c_{i-1}, so you cannot parallelize the encrypt direction across cores (decryption, happily, *is* parallelizable, since

$m_i = D_K(c_i) \oplus c_{i-1}$ needs only ciphertext blocks you already have). Second, CBC needs the message length to be a whole number of blocks. Real messages are not, so you must *pad*.

The standard is PKCS#7 padding: if you need to add p bytes to reach a block boundary (with $1 \le p \le n/8$), append p bytes each equal to the value p. To reach a 16-byte boundary when 5 bytes are missing you append `05 05 05 05 05`. If the message already ends on a boundary you add a *full* block of padding (all bytes `10` for a 16-byte block) so that decryption can always unambiguously strip it. On decrypt you read the last byte p, check that the final p bytes all equal p, and remove them.

That "check that the final p bytes all equal p" step led to a decade of practical attacks.

13.4 The padding-oracle attack (a preview)

We will treat authenticated encryption properly in Chapter 16; here is the cautionary preview that motivates it. CBC by itself provides *confidentiality* but not *integrity*: an attacker can flip bits in the ciphertext and CBC will dutifully decrypt to some other plaintext without complaint. If the surrounding system then *tells the attacker* whether the decrypted plaintext had valid padding, the attacker can decrypt arbitrary ciphertext without the key.

Attack

The padding oracle. A padding oracle is any service that, given a ciphertext, reveals whether decryption produced valid PKCS#7 padding – via an explicit error, a different HTTP status, or even a timing difference. Suppose the attacker wants to decrypt a target ciphertext block c_i. Recall CBC decryption: $m_i = D_K(c_i) \oplus c_{i-1}$. Let $I_i = D_K(c_i)$ be the (unknown) intermediate value, so $m_i = I_i \oplus c_{i-1}$.

The attacker crafts a fake two-block ciphertext (c', c_i) where c' is a block it controls, and submits it. The oracle decrypts the last block as $m' = D_K(c_i) \oplus c' = I_i \oplus c'$. The attacker now tweaks the last byte of c' through all 256 values until the oracle says "padding valid." Valid padding almost always means the last byte of m' is `01`, i.e. the last byte of $I_i \oplus c'$ is `01`. That pins down the last byte of I_i: it equals (last byte of c') $\oplus$ `01`. Knowing I_i, the real plaintext byte is recovered as (that byte of I_i) $\oplus$ (corresponding byte of the real c_{i-1}).

Then the attacker moves to the second-to-last byte: it sets the last byte of c' to force the last plaintext byte to `02`, and brute-forces the second-to-last byte to find padding `02 02`. Byte by byte, 256 tries each, it peels off the entire block – never knowing the key, as traced in Figure 13.5. This is Vaudenay's 2002 padding-oracle attack. Its descendants (Lucky Thirteen, POODLE) repeatedly broke real TLS deployments.

Intuition

Make the last-byte step concrete with one number. Say the unknown intermediate value's last byte is $I =$ `0x7A` (the attacker does not know this yet). The attacker sweeps the last byte of its controlled block c' through `0x00`, `0x01`, ... The oracle returns "valid" exactly when the decrypted last byte $I \oplus c'_{\text{last}}$ equals `0x01`, i.e. when $c'_{\text{last}} =$ `0x7A` $\oplus$ `0x01` $=$ `0x7B`. Seeing that the guess `0x7B` is the one that validates, the attacker computes $I =$ `0x7B` $\oplus$ `0x01` $=$ `0x7A` and then the true plaintext byte `0x7A` $\oplus$ (real c_{i-1} byte). No key, one byte recovered in at most 256 tries.

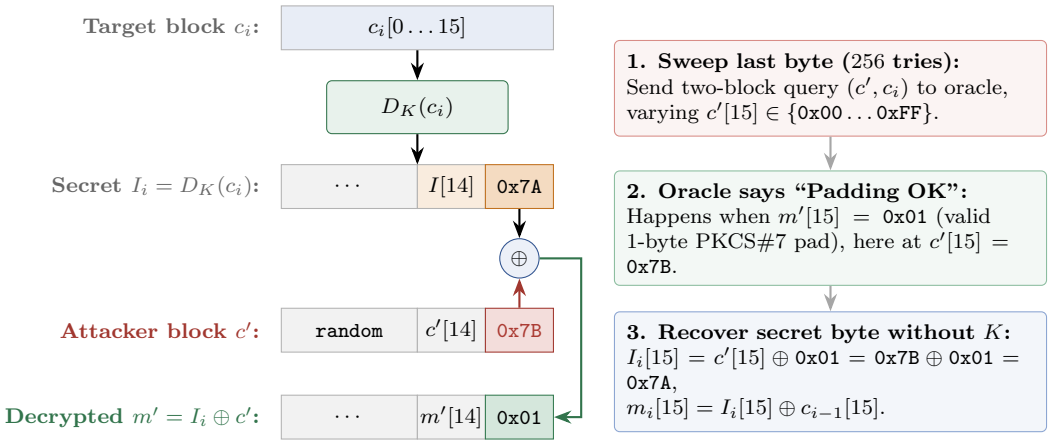


Figure 13.5: Vaudenay's CBC padding-oracle attack recovering the last byte of block m_i. By submitting (c', c_i) and sweeping $c'[15]$ until the server reports valid PKCS#7 padding ($m'[15] = 0x01$), the attacker learns the secret intermediate byte $I_i[15] = c'[15] \oplus 0x01$ and XORs it with the real $c_{i-1}[15]$ to recover $m_i[15]$.

Why it works

The lesson: **encryption without integrity is dangerous**, because a decryption process that reacts differently to malformed input becomes an oracle. The structural fix is to *authenticate* the ciphertext – attach a MAC computed over $(IV, c_1, \ldots, c_\ell)$ and verify it (in constant time) *before* you even attempt to decrypt or unpad. If the MAC fails, you reject immediately and the decryptor never reveals anything about padding. This "encrypt-then-MAC" discipline, and the AEAD constructions that bake it in, are the subject of Chapter 16. For now, remember: a bare mode of operation is a confidentiality tool, not a complete encryption scheme.

13.5 CTR: turn the block cipher into a stream cipher

Counter (CTR) mode takes a different and simpler route. Instead of feeding plaintext through the cipher, it uses the cipher to manufacture a pseudorandom *keystream*, then XORs that keystream into the plaintext – exactly like a one-time pad, exactly like a stream cipher.

Pick a nonce N and define a sequence of counter blocks $T_1, T_2, \ldots$, where T_i encodes the nonce together with the block index i (a common layout: the top bits are the nonce, the low bits are a counter that increments per block). The keystream is $E_K(T_i)$, and:

$$c_i = m_i \oplus E_K(T_i), \qquad m_i = c_i \oplus E_K(T_i).$$

Figure 13.6 shows the data flow. Encryption and decryption are the *same* operation – both just XOR with $E_K(T_i)$ – because XOR is its own inverse. You never need D_K at all; CTR only ever calls E_K in the forward direction.

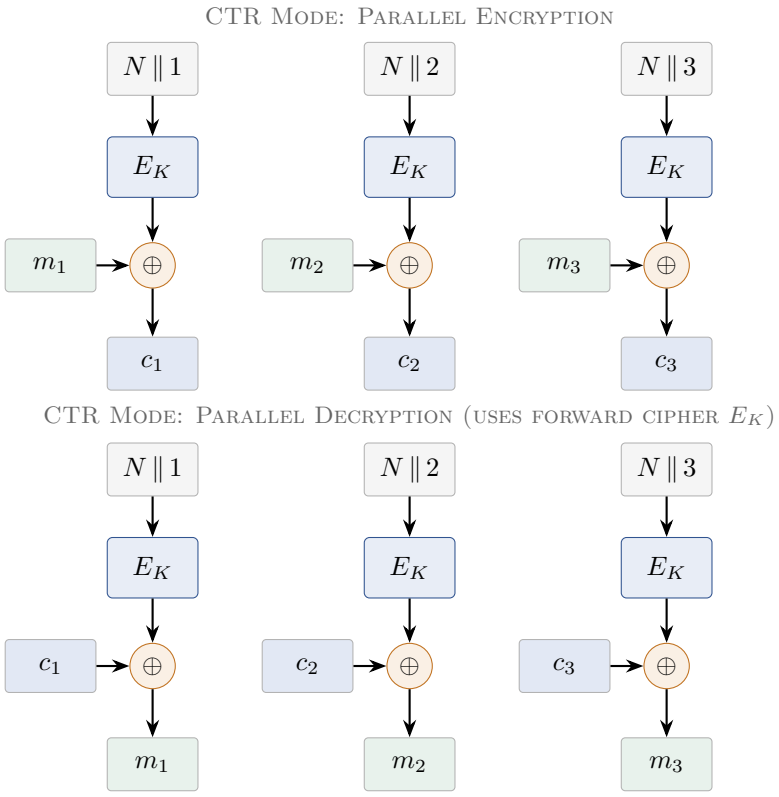


Figure 13.6: CTR mode encryption and decryption. Both directions evaluate E_K in the forward direction on independent counter blocks $N \| i$, producing identical pseudorandom pad blocks that are XORed with plaintext or ciphertext. Processing is entirely parallel with full random access.

Intuition

CTR is the most "one-time-pad-like" of all the modes. The block cipher's only job is to be a key-and-counter-driven random number generator: feed it the counters $N\|1, N\|2, \ldots$ and it emits pad blocks that look uniformly random to anyone without the key (that is precisely the PRP/PRF guarantee). Then encryption is the one-time pad: $c = m \oplus \text{pad}$. Everything you learned about the one-time pad – including its central rule, never reuse the pad – carries over directly.

13.5.1 A tiny CTR trace

Toy permutation E_K again. Let the nonce-and-counter blocks simply be the nibbles $\texttt{9}, \texttt{A}, \texttt{B}$ (pretend the nonce plus counter packs into a nibble for the demo). Keystream:

$$E_K(\texttt{9}) = \texttt{F}, \quad E_K(\texttt{A}) = \texttt{3}, \quad E_K(\texttt{B}) = \texttt{D}.$$

Encrypt $m = (\texttt{A}, \texttt{1}, \texttt{A})$:

$$\begin{aligned} c_1 &= m_1 \oplus \texttt{F} = \texttt{A} \oplus \texttt{F} = 1010 \oplus 1111 = 0101 = \texttt{5}, \\ c_2 &= m_2 \oplus \texttt{3} = \texttt{1} \oplus \texttt{3} = 0001 \oplus 0011 = 0010 = \texttt{2}, \\ c_3 &= m_3 \oplus \texttt{D} = \texttt{A} \oplus \texttt{D} = 1010 \oplus 1101 = 0111 = \texttt{7}. \end{aligned}$$

Ciphertext $(\texttt{5}, \texttt{2}, \texttt{7})$. To decrypt, regenerate the *same* keystream $(\texttt{F}, \texttt{3}, \texttt{D})$ and XOR: $\texttt{5} \oplus \texttt{F} = 0101 \oplus 1111 = 1010 = \texttt{A}$, and so on – back to the plaintext. Encryption and decryption are the same computation.

13.5.2 CTR's strengths and its nonce-uniqueness rule

Focus

What CTR buys you:

- **Parallelism.** Every $E_K(T_i)$ is independent, so you can compute the whole keystream across all cores at once, in either direction. CBC cannot do this on encrypt.
- **Random access / seekability.** To decrypt block 4242 you compute $E_K(N\|4242)$ directly – no need to process blocks 1 through 4241. This is why full-disk and database encryption love CTR-derived constructions.
- **No padding.** The keystream is truncated to the message length, so CTR handles messages of any byte length with no PKCS#7 padding, hence no padding oracle.
- **Decrypt uses only E_K.** Smaller implementations: you never ship D_K.

Common trap

The critical rule: **never reuse a (key, counter) pair.** If two plaintext blocks m and m' are ever XORed with the *same* keystream block $E_K(T)$, then

$$c \oplus c' = (m \oplus E_K(T)) \oplus (m' \oplus E_K(T)) = m \oplus m'.$$

The keystream cancels and the attacker gets the XOR of two plaintexts – the classic two-time-pad disaster, fully readable with a little language modeling. This happens if you reuse a nonce N under the same key, or if the per-message counter ranges overlap. With a 96-bit random nonce you are safe up to roughly the birthday bound; with a small counter you must guarantee uniqueness by construction. Nonce reuse in CTR does not degrade security gradually; it breaks confidentiality for every block where it occurs.

13.5.3 A tiny two-time-pad example

It is worth seeing the nonce-reuse break on numbers rather than as an algebraic identity, because the complete cancellation often surprises people. Stay with the toy permutation and the keystream $(\mathtt{F}, \mathtt{3}, \mathtt{D})$ from the previous trace, generated from nonce-and-counter blocks $\mathtt{9}, \mathtt{A}, \mathtt{B}$. Now suppose a careless implementer encrypts *two* messages under the *same* nonce, so both reuse that identical keystream. Message one is $m = (\mathtt{A}, \mathtt{1}, \mathtt{A})$ as before, giving $c = (\mathtt{5}, \mathtt{2}, \mathtt{7})$. Message two is $m' = (\mathtt{C}, \mathtt{1}, \mathtt{4})$:

$$
\begin{aligned}
c'_1 &= \mathtt{C} \oplus \mathtt{F} = 1100 \oplus 1111 = 0011 = \mathtt{3},\\
c'_2 &= \mathtt{1} \oplus \mathtt{3} = 0001 \oplus 0011 = 0010 = \mathtt{2},\\
c'_3 &= \mathtt{4} \oplus \mathtt{D} = 0100 \oplus 1101 = 1001 = \mathtt{9}.
\end{aligned}
$$

The attacker has $c = (\mathtt{5}, \mathtt{2}, \mathtt{7})$ and $c' = (\mathtt{3}, \mathtt{2}, \mathtt{9})$ and never sees the key or the keystream. It XORs them position-by-position:

$$
c \oplus c' = (\mathtt{5} \oplus \mathtt{3},\ \mathtt{2} \oplus \mathtt{2},\ \mathtt{7} \oplus \mathtt{9}) = (\mathtt{6},\ \mathtt{0},\ \mathtt{E}).
$$

Check against the plaintexts directly: $m \oplus m' = (\mathtt{A} \oplus \mathtt{C},\ \mathtt{1} \oplus \mathtt{1},\ \mathtt{A} \oplus \mathtt{4}) = (\mathtt{6}, \mathtt{0}, \mathtt{E})$ – identical. The keystream cancelled completely, and the second block being $\mathtt{0}$ tells the attacker outright that $m_2 = m'_2$. On real English text, that $m \oplus m'$ is enough to "crib-drag" both messages into the clear. A single reused nonce is enough to break confidentiality.

Connection

"How big can the nonce be before collisions get likely?" is a birthday-bound question, handled in the companion probability book. If you draw nonces uniformly from a space of size 2^t, the chance of a collision crosses $1/2$ at roughly $2^{t/2}$ messages. For a 96-bit nonce that is about 2^{48} messages under one key – comfortable, but not infinite, which is exactly why standards cap how much data a single key may protect and then force a rekey. The cost of computing each $E_K(T_i)$, and why modular-counter increments are cheap, is the arithmetic-cost material in the algorithms companion.

13.6 CFB and OFB: the older stream-cipher modes

Before CTR became dominant, two other modes turned a block cipher into a stream-like cipher using *feedback* rather than a counter. They are worth knowing because you will meet them in legacy protocols (PGP's CFB variant, older link encryptors), and because they sharpen the intuition about what feedback does.

Output Feedback (OFB). Generate the keystream by repeatedly encrypting, starting from the IV, feeding the cipher's *output* back as its next input:

$$z_0 = IV, \quad z_i = E_K(z_{i-1}), \quad c_i = m_i \oplus z_i.$$

The keystream $z_1, z_2, \ldots$ depends only on the key and IV, never on the plaintext (Figure 13.8) – so, like CTR, it is just a pseudorandom pad XORed in, and decryption is identical to encryption. OFB is essentially CTR with a chained feedback register instead of a counter. Its drawback: keystream generation is strictly sequential (you must compute z_{i-1} to get z_i), and a short cycle in the feedback would repeat the pad, so CTR's clean counter is simply better.

Cipher Feedback (CFB). Here the *ciphertext* feeds back into the cipher:

$$c_0 = IV, \qquad c_i = m_i \oplus E_K(c_{i-1}).$$

Decryption: $m_i = c_i \oplus E_K(c_{i-1})$ – note again only E_K is used, never D_K. CFB resembles CBC turned inside out (Figure 13.7): CBC encrypts $m_i \oplus c_{i-1}$, while CFB XORs m_i with $E_K(c_{i-1})$.

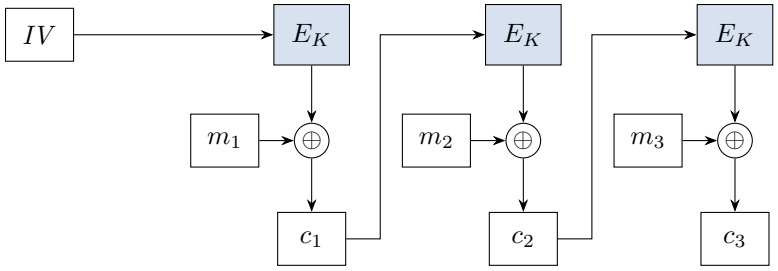


Figure 13.7: CFB: each ciphertext block feeds back as the cipher's next input (the IV seeds the first). Unlike OFB, the feedback carries the ciphertext, so the keystream depends on the message.

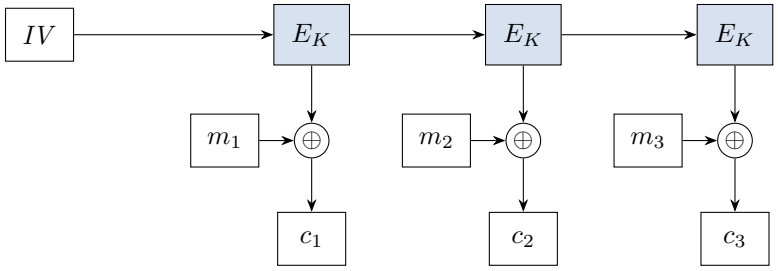


Figure 13.8: OFB: the cipher's output is chained as its own next input, producing a plaintext-independent keystream that is XORed in.

Remark 13.1. A subtle but practical difference is *error propagation*. In OFB and CTR, a single flipped *ciphertext* bit flips exactly the one corresponding plaintext bit on decryption (XOR is bit-local) – handy for noisy channels, dangerous because it makes silent bit-flipping easy without integrity protection. In CBC, a flipped ciphertext bit garbles the entire block it lands in *and* flips the single aligned bit in the next block. None of these modes *detect* the tampering – that is the MAC's job – but the propagation pattern is exactly what the padding-oracle and bit-flipping attacks exploit.

13.7 Putting the modes side by side

	ECB	CBC	CFB	OFB	CTR
IND-CPA secure?	no	yes†	yes†	yes†	yes†
Needs IV/nonce	no	yes (unpred.)	yes (unpred.)	yes (unique)	yes (unique)
Uses D_K to decrypt	yes	yes	no	no	no
Parallel encrypt	yes	no	no	no	yes
Parallel decrypt	yes	yes	yes	no	yes
Random access	yes	no	no	no	yes
Needs padding	yes	yes	no	no	no

† Secure only with a correctly generated IV/nonce *and* only for confidentiality – none of these modes provides integrity on its own.

Intuition

The table summarizes the trade-offs. ECB is the outlier: deterministic, hence broken. The middle three (CBC, CFB, OFB) are the classic feedback modes, each making the cipher randomized through feedback, each with awkward edges (sequential encryption, padding, IV fragility). CTR is the modern choice: parallel both ways, seekable, no padding, decrypt without D_K – a clean pseudorandom pad. That is why today's authenticated ciphers are built on it.

13.8 Toy code: ECB versus CBC versus CTR

Implement it

The snippet below implements all three modes over a real block cipher (AES via PyCryptodome's raw single-block primitive) so you can watch ECB leak and the others not. It is deliberately minimal – no MAC, no constant-time anything.

Learning only. Do not use this in production. It has no integrity protection (so it is wide open to bit-flipping and padding-oracle attacks), the CBC IV handling is simplistic, and there is no rekeying or nonce-management discipline. For real work call a vetted library's AEAD interface (AES-GCM, ChaCha20-Poly1305) and never assemble a mode by hand.

```
from Crypto.Cipher import AES   # raw 16-byte block primitive
import os

BS = 16  # AES block size in bytes

def blocks(data):
    return [data[i:i+BS] for i in range(0, len(data), BS)]

def xor(a, b):
```

```
    return bytes(x ^ y for x, y in zip(a, b))

def ecb_encrypt(key, msg):            # NEVER use ECB for real data
    e = AES.new(key, AES.MODE_ECB)
    return b"".join(e.encrypt(b) for b in blocks(msg))   # msg must be block-aligned

def cbc_encrypt(key, msg):            # msg must be block-aligned (pad in real code)
    e = AES.new(key, AES.MODE_ECB)    # we drive the single-block primitive ourselves
    iv = os.urandom(BS)               # unpredictable IV from the OS CSPRNG
    prev, out = iv, []
    for m in blocks(msg):
        c = e.encrypt(xor(m, prev))  # c_i = E_K(m_i XOR c_{i-1})
        out.append(c)
        prev = c
    return iv + b"".join(out)

def ctr_keystream(key, nonce, nblocks):
    e = AES.new(key, AES.MODE_ECB)
    ks = []
    for i in range(nblocks):
        ctr = nonce + i.to_bytes(BS - len(nonce), "big")  # nonce || counter
        ks.append(e.encrypt(ctr))     # keystream block = E_K(nonce || i)
    return b"".join(ks)

def ctr_encrypt(key, nonce, msg):     # nonce MUST be unique per key -- never reuse
    ks = ctr_keystream(key, nonce, (len(msg) + BS - 1) // BS)
    return xor(msg, ks[:len(msg)])    # decryption is the identical function

key = os.urandom(16)
# Two identical 16-byte blocks reveal ECB's pattern leak:
plain = b"YELLOW SUBMARINE" * 2
print("ecb:", ecb_encrypt(key, plain).hex())   # two halves IDENTICAL -> leak
print("cbc:", cbc_encrypt(key, plain).hex())   # halves differ; fresh each run
print("ctr:", ctr_encrypt(key, os.urandom(12), plain).hex())
```

Run it: the ECB output's two 16-byte halves are byte-for-byte equal, while CBC and CTR show no such repeat and change on every run. That visible difference is exactly what an IND-CPA adversary exploits against ECB.

13.9 Where the modes actually live

In the real world

TLS. TLS 1.0's CBC record protection, with its predictable IV, gave us BEAST; its MAC-then-encrypt-then-pad ordering gave us Lucky Thirteen and POODLE. TLS 1.2 patched around these, but TLS 1.3 removed the problem at the root: it permits *only* AEAD ciphers, and the two mandatory ones – AES-GCM and ChaCha20-Poly1305 – are both CTR-style keystream generators wrapped in an authenticator. The mode-of-operation history of TLS is essentially the history of this chapter's attacks.

Disk and database encryption. Full-disk encryption needs random access to any sector without decrypting the whole disk, which rules out CBC chaining and points straight at counter-based and tweakable modes (XTS, a relative of CTR built for storage). Sector-level encryption also cannot store a per-sector nonce, which is why storage uses specialized tweakable constructions rather than plain CTR.
WiFi. WPA2's CCMP is AES in *Counter mode* for confidentiality combined with CBC-MAC for integrity – CTR doing the encrypting, exactly as in this chapter. WPA3 keeps CTR-style AEAD (GCMP). The catastrophic WEP, by contrast, used RC4 with reused IVs – the exact same keystream-reuse failure as CTR nonce reuse, happening in a stream cipher.
Signal, SSH, WireGuard. Modern secure messaging and tunnels standardize on ChaCha20-Poly1305 (a CTR-like stream cipher plus a MAC) or AES-GCM. The common pattern is: CTR to make the keystream, an authenticator bolted on, never a bare mode. The next chapters build exactly that.

History

The modes were standardized for DES by the National Bureau of Standards in FIPS 81 (1980): ECB, CBC, CFB, and OFB. CTR was described by Diffie and Hellman in 1979 but left out of the original standard; it was only standardized by NIST in SP 800-38A in 2001 alongside AES. Because of that omission, early protocols like SSL/TLS and IPsec settled on CBC with a bolted-on MAC. The series of attacks that followed – Serge Vaudenay's 2002 PKCS#7 padding-oracle attack, BEAST in 2011, and Lucky Thirteen and POODLE in 2013–14 – convinced the field that confidentiality and integrity must be designed *together* in Authenticated Encryption with Associated Data (AEAD), and motivated misuse-resistant modes that survive accidental nonce repetition.

13.10 Summary

A block cipher encrypts one fixed-size block; a mode of operation extends it to messages of any length. ECB encrypts each block independently and is broken by construction – deterministic, pattern-leaking, the penguin. CBC chains each block through the previous ciphertext block and seeds the chain with an unpredictable IV; it is secure for confidentiality but sequential on encrypt, needs padding, and its IV and padding behavior have repeatedly been exploited. CTR turns the cipher into a one-time-pad keystream generator indexed by a nonce-and-counter; it is parallel, seekable, padding-free, and decrypts without the inverse cipher – at the cost of one absolute rule, never reuse a (key, counter) pair. CFB and OFB are the older feedback-based stream modes, now largely supplanted by CTR. And *none* of these modes provides integrity: a bare mode is a confidentiality primitive, and the padding-oracle attack shows how damaging that is when an attacker can tamper with ciphertext and watch the system react.

The resolution – authenticate, then (or while) you encrypt, with CTR as the underlying encryption – is modern AEAD. Before we can build authenticated encryption in Chapter 16, we need the primitives that detect tampering and verify message integrity: Chapter 14 introduces cryptographic hash functions (SHA-2, SHA-3, collision resistance, and the Merkle–Damgård construction), and Chapter 15 builds message authentication codes (MACs).

13.11 Exercises

Exercise 13.1. Using the 4-bit toy permutation E_K from the chapter (the 16-entry table), encrypt the nibble stream $(2, 6, 2, 6)$ in ECB mode. Identify the repeated ciphertext blocks and explain in one sentence what an eavesdropper learns.

Exercise 13.2. With the same toy E_K and IV $= 3$, encrypt the nibble stream $(2, 6, 2, 6)$ in CBC mode, showing each XOR and each table lookup. Confirm that the two identical plaintext blocks no longer produce identical ciphertext blocks.

Exercise 13.3. Prove formally that ECB is not IND-CPA secure by describing an adversary that wins the IND-CPA game with probability 1 using a single challenge query. State exactly which two messages it submits and how it decides its guess.

Exercise 13.4. In CTR mode, suppose an implementer reuses the same nonce N (under the same key) to encrypt two messages m and m' of equal length, producing c and c'. Show that an attacker who sees c and c' can compute $m \oplus m'$, and explain why this typically lets the attacker recover both messages if they are natural-language text.

Exercise 13.5. You are handed a CBC decryption service that returns "ok" if the PKCS#7 padding of the decrypted message is valid and "bad" otherwise. Describe step by step how to recover the last byte of an unknown plaintext block, then sketch how the attack extends to the whole block. Approximately how many oracle queries does recovering one 16-byte block require in the worst case?

Exercise 13.6. Explain why CBC encryption cannot be parallelized across blocks but CBC *decryption* can. Then explain why CTR mode is parallelizable in both directions. Reference the exact recurrence relations.

Exercise 13.7. PKCS#7 padding adds a full block of padding when the message length is already a multiple of the block size. Give a concrete example showing why omitting this rule – i.e. adding no padding when the message is already block-aligned – makes unpadding ambiguous, so the decryptor cannot always tell padding from data.

Exercise 13.8. Recall OFB's keystream recurrence $z_i = E_K(z_{i-1})$ with $z_0 = IV$. Argue that if the sequence $z_0, z_1, z_2, \ldots$ ever revisits a previous value, the keystream becomes periodic from that point, and explain why that is a confidentiality problem. Why does CTR's counter-based keystream not suffer this issue?

Exercise 13.9. Consider CBC with a *predictable* IV: before encrypting message j, the attacker can compute the IV that will be used. Construct the chosen-plaintext query that lets the attacker test whether an earlier secret first block equals a guessed value g, and explain why a freshly random per-message IV defeats the attack. (This is the BEAST setup.)

Exercise 13.10. A bare CTR-mode ciphertext is delivered to a receiver who decrypts and acts on the plaintext with no integrity check. The attacker knows that bytes 5–8 of the plaintext encode a dollar amount and knows the original value. Show how the attacker can change that amount to any chosen value by XORing a known mask into the corresponding ciphertext bytes, *without* knowing the key. What single countermeasure stops this entire class of attack?

Exercise 13.11. Modes leak the message length (CTR and CFB/OFB leak it to the byte; CBC and ECB to the block). Describe a realistic scenario – for instance, encrypted web traffic or voice – where leaking only the length still reveals sensitive information, and name one mitigation (hint: think about deliberately wasteful encoding).

Chapter 14

Hash Functions

In Chapter 13 we saw how block cipher modes encrypt messages of arbitrary length, hiding their content from an eavesdropper without guaranteeing that no one has flipped bits along the way. A cryptographic hash function takes a message of any length and crushes it down to a short, fixed-length string of bits – its *digest*. SHA-256 turns the complete works of Shakespeare, or a single byte, or this entire book, into exactly 256 bits. That much is easy: any number of cheap functions can map long inputs to short outputs. What makes a hash function *cryptographic* is a set of promises about how hard it is to find inputs that behave in inconvenient ways – inputs that collide, or inputs that hit a target you picked in advance.

Those promises are what let a 32-byte digest stand in for a multi-gigabyte file with no loss of trust. They are what let a website store something derived from your password without storing the password. They are what let a blockchain commit to a billion transactions with one number. Hash functions are the most-used primitive in all of cryptography, and nearly every later chapter depends on them: MACs (Chapter 15), signatures (Chapter 22), key derivation, commitments (Chapter 32), and the Merkle trees that hold blockchains together (Chapter 36) all sit on top of a good hash.

This chapter builds the idea from the security properties up. We will pin down what "hard to break" means, derive the birthday bound that governs collision-finding, dissect the two construction styles (Merkle–Damgard and the sponge), break a Merkle–Damgard hash with a length-extension attack, and end with fingerprints, commitments, and memory-hard password hashing.

14.1 What a hash function is, and what it must promise

Formally, a hash function is a function

$$\mathrm{H} : \{0,1\}^* \to \{0,1\}^n,$$

where $\{0,1\}^*$ is the set of all finite bit strings and n is the fixed output length (256 for SHA-256, 512 for SHA-512). The input is sometimes called the *message* or *preimage*; the output is the *digest*, *hash*, or *fingerprint*.

Because the domain is infinite and the range is finite, collisions *must* exist: there are infinitely many messages and only 2^n possible digests, so by the pigeonhole principle countless pairs of distinct

messages share a digest. Cryptography never claims collisions do not exist. It claims you cannot *find* one with any feasible amount of work.

There are three security properties, in increasing order of how much they buy you.

Definition 14.1 (Preimage resistance)**.** H is *preimage resistant* ("one-way") if, given a target digest $y = \mathrm{H}(x)$ for a randomly chosen x, no efficient adversary can find *any* message x' with $\mathrm{H}(x') = y$, except with negligible probability. For an ideal n-bit hash the best attack is brute force, costing about 2^n evaluations.

Definition 14.2 (Second-preimage resistance)**.** H is *second-preimage resistant* if, given a specific message x, no efficient adversary can find a *different* message $x' \neq x$ with $\mathrm{H}(x') = \mathrm{H}(x)$, except with negligible probability. The ideal cost is again about 2^n.

Definition 14.3 (Collision resistance)**.** H is *collision resistant* if no efficient adversary can find *any* pair $x \neq x'$ with $\mathrm{H}(x) = \mathrm{H}(x')$, except with negligible probability. Here the adversary picks both messages and has total freedom. As we will prove in Section 14.2, the ideal cost is only about $2^{n/2}$.

Intuition

A clean way to keep the three straight is to ask *who chooses what*.

- **Preimage**: the world hands you an *output* y; you must produce *some* input mapping to it. ("Here is a hash. What message made it?")
- **Second-preimage**: the world hands you an *input* x; you must produce a *different* input with the same hash. ("Here is a contract. Forge a different one that hashes the same.")
- **Collision**: the world hands you *nothing*; you produce *any* two inputs that hash alike. ("Just find any clash at all.")

More freedom for the attacker means an easier attack, which is why collision resistance is the strongest property and the first to fall.

Figure 14.1 contrasts the three games side by side: solid nodes are fixed by the challenger before the adversary starts, and dashed red nodes are what the adversary is free to choose.

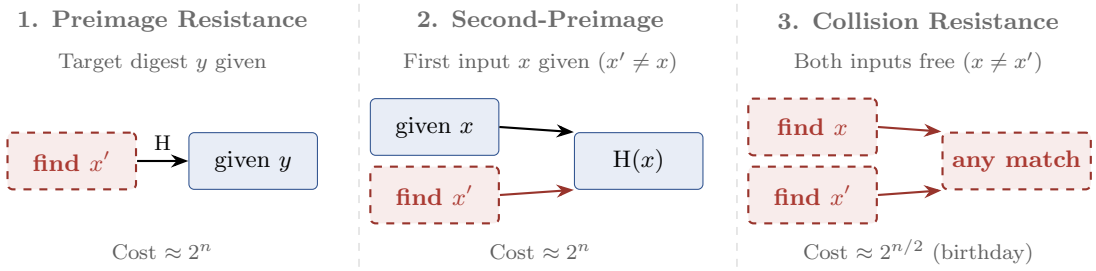


Figure 14.1: The three hash security properties compared by what the challenger fixes (solid blue boxes) versus what the adversary chooses (dashed red boxes). Collision resistance gives the adversary control of both inputs, dropping the generic work from 2^n to $2^{n/2}$.

These properties are not independent. Collision resistance is the strongest of the three in the following precise sense.

Proposition 14.1 (Collision resistance implies second-preimage resistance)**.** *If* H *is collision resistant then it is second-preimage resistant.*

Why it works

The argument is a *reduction*: we show that breaking the weaker property would break the stronger one, so the stronger one is at least as hard.
Suppose an adversary $\mathcal{A}$ finds second preimages. Build a collision finder $\mathcal{B}$: sample a random message x yourself, hand it to $\mathcal{A}$, and receive $x' \neq x$ with $\mathrm{H}(x') = \mathrm{H}(x)$. Then (x, x') is a collision. So if second preimages are easy to find, collisions are too – contrapositively, if collisions are hard, so are second preimages. The reduction loses nothing here; it is tight.

The relationship between collision resistance and *preimage* resistance is subtler. For a random function with output much shorter than its input they tend to come together, but one can hand-craft contrived functions that are collision resistant yet not one-way, or vice versa. In practice we demand all three from any hash we are willing to call cryptographic, and we design and test for all three.

Common trap

A non-cryptographic hash – CRC32, the hash behind a Java `HashMap`, FNV, MurmurHash – has *none* of these properties and is trivial to collide on purposе. These functions exist to spread inputs evenly for speed, not to resist an adversary. Using `hashCode()` or a CRC anywhere a security decision depends on the output is a classic, fatal mistake. "Hash function" means two completely different things to a systems engineer and to a cryptographer.

14.1.1 The random-oracle ideal

When cryptographers reason about a hash informally, they often pretend it is a *random oracle*: an idealized black box that, for each new input, flips n fair coins to produce the output, and thereafter returns that same output whenever the input repeats. A random oracle has every property we could want – it is one-way, collision resistant, and its outputs look independent and uniform – because there is simply no structure to exploit; the only way to learn $\mathrm{H}(x)$ is to ask.

Real hash functions are deterministic public algorithms, not idealized black boxes, so the random-oracle *model* is a heuristic, and proofs "in the random-oracle model" are weaker than proofs in the standard model. But the heuristic is enormously useful for intuition and for the security arguments behind RSA-OAEP, Fiat–Shamir signatures, and much else. We will lean on it whenever it sharpens the picture, and flag the places where the gap between the ideal and reality has actually bitten people.

14.2 The birthday bound: why n-bit hashes give $n/2$-bit collision security

Brute-forcing a preimage of an n-bit hash takes about 2^n tries: you are searching for one specific output among 2^n. Collisions are far cheaper, and the reason is the same mathematics behind the famous party trick that 23 people suffice for an even chance of two sharing a birthday.

Intuition

You are not looking for a message that hashes to one *fixed* target. You are looking for *any* two of your messages that hash to the *same* value – and the more messages you have, the more *pairs* you have. With q messages there are $\binom{q}{2} \approx q^2/2$ pairs, each of which collides with probability about 2^{-n}. The expected number of colliding pairs crosses 1 when $q^2/2 \approx 2^n$, i.e. when $q \approx 2^{(n+1)/2} \approx 2^{n/2}$. Collisions appear after only the *square root* of the naive number of tries.

Let us do the probability carefully.

Theorem 14.1 (Birthday bound)**.** *Draw q outputs independently and uniformly from a set of size $N = 2^n$. The probability that at least two coincide satisfies*

$$p(q) \;=\; 1 - \prod_{j=1}^{q-1}\Bigl(1 - \frac{j}{N}\Bigr) \;\approx\; 1 - e^{-q(q-1)/(2N)}.$$

Consequently $p(q) \geq \frac{1}{2}$ once $q \approx 1.177\sqrt{N} = 1.177 \cdot 2^{n/2}$.

Why it works

Number the draws $1, 2, \ldots, q$. Given that the first j draws are all distinct, draw $j+1$ avoids those j earlier values with probability $1 - j/N$. Multiplying these conditional probabilities for $j = 1, \ldots, q-1$, the chance that *all* q draws are distinct is

$$\prod_{j=1}^{q-1}\Bigl(1 - \frac{j}{N}\Bigr).$$

Now use $1 - x \leq e^{-x}$, valid for all real x:

$$\prod_{j=1}^{q-1}\Bigl(1 - \frac{j}{N}\Bigr) \leq \prod_{j=1}^{q-1} e^{-j/N} = \exp\Bigl(-\frac{1}{N}\sum_{j=1}^{q-1} j\Bigr) = \exp\Bigl(-\frac{q(q-1)}{2N}\Bigr).$$

The collision probability is one minus that. Setting the exponent equal to $\ln 2$ gives $q(q-1) \approx 2N \ln 2$, hence $q \approx \sqrt{2 \ln 2}\,\sqrt{N} \approx 1.177\sqrt{N}$.

Connection

This is exactly the birthday-collision calculation from the companion probability book, with the 365 days of the year replaced by the $N = 2^n$ possible digests. There we found $\approx 1.177\sqrt{365} \approx 22.5$, so 23 people. Here the same constant rides on top of a $2^{n/2}$ that swamps everything. The cube-root variant – three-way collisions appearing at $N^{1/3}$ – and the memoryless cycle-finding methods (Pollard's rho, Floyd's tortoise-and-hare, van Oorschot–Wiener parallel collision search) that find collisions in $\approx 2^{n/2}$ time but tiny memory are all developed in the algorithms companion.

Example 14.1 (A toy 16-bit hash)**.** Take $n = 16$, so $N = 65536$. A preimage costs about $2^{16} = 65{,}536$ evaluations. A collision costs about $1.177\sqrt{65536} = 1.177 \cdot 256 \approx 301$ evaluations – you would expect a collision after hashing roughly 300 random short strings and watching for a repeat. With

$q = 300$ the exponent is $-300 \cdot 299/(2 \cdot 65536) \approx -0.684$, giving $p \approx 1 - e^{-0.684} \approx 0.495$, almost exactly even odds. The square-root gap between 65,536 and 301 is why output length matters so much.

Focus

Design consequence. To resist collision attacks at the "b-bit security level" (meaning 2^b work) you need an output of $n = 2b$ bits. That is why a 128-bit security target demands a 256-bit hash. It is why 128-bit-output hashes like MD5 (2^{64} collision work) and 160-bit SHA-1 (2^{80} nominal) are no longer collision-safe: 2^{64} is now a weekend on rented hardware, and SHA-1 collisions have actually been computed (see Section 14.10). Preimage attacks on these same functions remain near 2^n and are still out of reach; it is always *collisions* that fall first.

Walking question

A protocol truncates SHA-256 to its first 80 bits to save space in a fingerprint. What is the collision security level, and is that wise in 2026?

Answer. Collision work is $2^{80/2} = 2^{40}$, about a trillion evaluations – a few minutes on a GPU. If the fingerprint is only ever compared to a value a legitimate party computed (a second-preimage setting), security is closer to 2^{80}, which is borderline. If an attacker can choose *both* sides of the comparison (a collision setting – e.g. getting two documents to share a fingerprint), 2^{40} is catastrophic. Always ask which property your application actually relies on.

14.3 Building a hash from a small component: Merkle–Damgard

We can compress a *fixed* pair of blocks with a carefully designed mixing function. How do we hash a message of *arbitrary* length? The classic answer, used by MD5, SHA-1, and the SHA-2 family, is the *Merkle–Damgard* construction. It iterates a fixed-input *compression function* across the message, carrying a running state called the *chaining value*.

In 1989, Ralph Merkle and Ivan Damgard independently proved the reduction theorem that makes this design work: if the underlying compression function is collision resistant, then the full iterated hash is collision resistant as well. That result reduced the problem of hashing arbitrary-length messages to the simpler task of building a single fixed-input compression function. Nearly every widely deployed hash from 1990 to 2010 – MD4, MD5, SHA-0, SHA-1, SHA-256, and SHA-512 – adopted this Merkle–Damgard architecture. The structure's one notorious architectural wart, the length-extension vulnerability, is examined in Section 14.4.

The compression function is

$$f : \{0,1\}^n \times \{0,1\}^b \to \{0,1\}^n,$$

taking an n-bit chaining value and a b-bit message block to a new n-bit chaining value. To hash a message M:

1. **Pad** M so its length is a multiple of the block size b. The padding includes the message's original bit-length ("Merkle–Damgard strengthening"), which is what makes the security proof go through. Split the padded message into blocks $M_1, M_2, \ldots, M_k$.

2. **Initialize** the chaining value to a fixed constant $H_0 = IV$ baked into the standard.
3. **Absorb** each block: $H_i = f(H_{i-1}, M_i)$ for $i = 1, \ldots, k$.
4. **Output** H_k (possibly after a final transform).

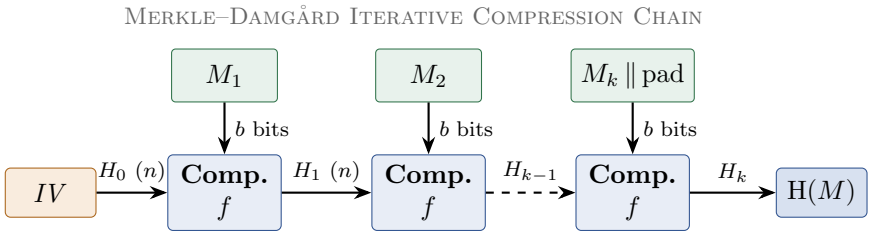


Figure 14.2: The Merkle–Damgård iterative domain-extension structure. Fixed-size compression function f takes an n-bit chaining state and a b-bit message block to produce the next chaining state. The final state H_k is the digest.

Theorem 14.2 (Merkle–Damgard collision resistance)**.** *If the compression function f is collision resistant, then the Merkle–Damgard hash* H *built from it (with length-strengthening padding) is collision resistant.*

Why it works

Suppose someone hands us two distinct messages $M \neq M'$ with $\mathrm{H}(M) = \mathrm{H}(M')$. We will extract a collision in f, contradicting its collision resistance.
Walk the two chains *backwards* from the equal final outputs. The last compression step gives

$$f(H_{k-1}, M_k) = H_k = H'_{k'} = f(H'_{k'-1}, M'_{k'}).$$

If the two inputs to f differ – that is, $(H_{k-1}, M_k) \neq (H'_{k'-1}, M'_{k'})$ – we have found a collision in f and we stop. Otherwise the inputs are equal, so $H_{k-1} = H'_{k'-1}$ and the last blocks match; step back one position and repeat the argument.
If the messages have different lengths, the length-strengthening padding forces the final blocks to differ (they encode different lengths), so the walk produces a collision at the last step. If the lengths are equal but the messages differ, they must differ in some block; the first block (scanning from the end) where they differ yields distinct inputs to f with equal outputs. Either way we manufacture an f-collision. So H-collisions imply f-collisions, and the theorem follows by contrapositive.

Implement it

A toy Merkle–Damgard hash with an 8-bit chaining value, just to watch the machinery move. **This is for learning only – its 8-bit state means a collision after ≈ 16 inputs, and the "compression function" has no cryptographic strength. Never use anything like this for real.**

```
def compress(h, block):          # h: int in 0..255, block: int in 0..255
    # A deliberately weak toy mixer. NOT cryptographic.
    x = (h ^ block) & 0xFF
    x = ((x << 3) | (x >> 5)) & 0xFF      # rotate left 3
```

```
    x = (x * 167 + 13) & 0xFF              # mix
    return x ^ ((h + block) & 0xFF)

def md_hash(message: bytes) -> int:
    h = 0x6A                               # the IV
    msg = message + bytes([len(message) & 0xFF])   # length strengthening
    for byte in msg:                       # each byte is one "block"
        h = compress(h, byte)
    return h

print(hex(md_hash(b"hello")))              # e.g. 0x..
print(hex(md_hash(b"world")))
# Find a collision by brute force -- with only 256 outputs it is instant:
seen = {}
i = 0
while True:
    d = md_hash(str(i).encode())
    if d in seen:
        print("collision:", seen[d], "and", i, "both ->", hex(d))
        break
    seen[d] = i
    i += 1
```

Run it and you will see a collision pop out after a few hundred tries at most – exactly the birthday bound for $n = 8$ ($\approx 1.177 \cdot 2^4 \approx 19$, with variance pushing the typical first hit somewhat higher). Scaling the state to 256 bits is what turns this toy into something nobody can collide.

14.4 Length extension: the Merkle–Damgard wart

Merkle–Damgard has a structural leak that is easy to miss and has caused real breaches. The final output H_k *is* the internal chaining value – there is nothing hiding it. So if you know $\mathrm{H}(M)$ and the *length* of M, you can resume the iteration from H_k and compute $\mathrm{H}(M \,\|\, \mathrm{pad} \,\|\, S)$ for a suffix S of your choosing, *without knowing M itself.*

Attack

The length-extension attack, and how it breaks a homemade MAC.
A tempting but broken way to authenticate a message with a secret key k is the "prefix MAC":

$$t = \mathrm{H}(k \,\|\, M).$$

The idea is that only someone who knows k can produce a valid tag t for M. With a Merkle–Damgard hash this is false.
Suppose an attacker sees a legitimate pair (M, t) where $t = \mathrm{H}(k \,\|\, M)$, and knows (or guesses) the length of k. The tag t is exactly the chaining value after absorbing $k \,\|\, M \,\|\, \mathrm{pad}_1$, where pad_1 is the padding the hash appended. The attacker now:

1. sets the internal state of a fresh hash object to t;
2. feeds in any chosen suffix S;

3. reads off $t' = \mathrm{H}(k \,\|\, M \,\|\, \mathrm{pad}_1 \,\|\, S)$.

The result is a *valid tag* for the new message $M' = M \,\|\, \mathrm{pad}_1 \,\|\, S$ – a message the attacker partly chose – even though they never learned k. The notorious instance: a web request signed by appending `&admin=true` after the legitimate padding still verifies.

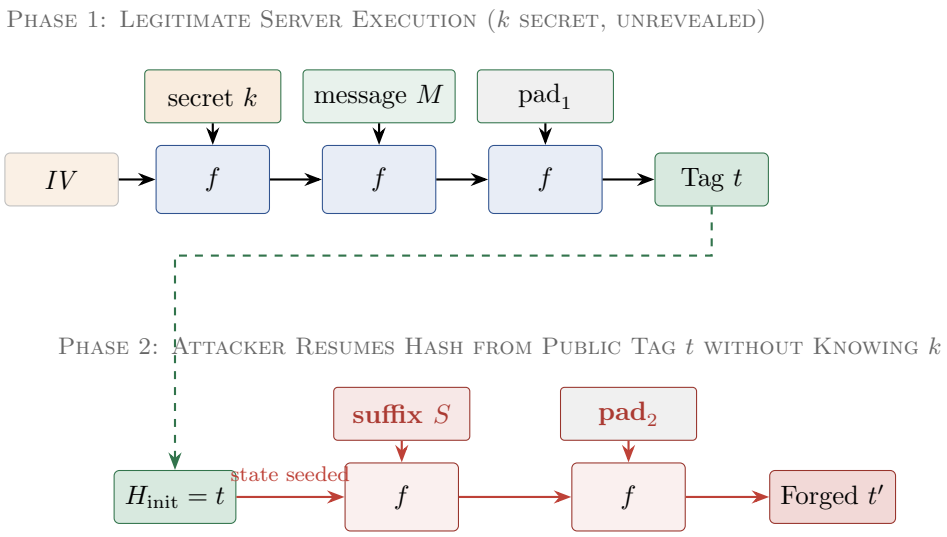


Figure 14.3: Length extension against a prefix MAC $t = \mathrm{H}(k \,\|\, M)$. The published tag t *is* the chaining value after the server absorbs $k \,\|\, M \,\|\, \mathrm{pad}_1$. The attacker feeds that value straight back into f with a chosen suffix S, producing a valid tag t' for $k \,\|\, M \,\|\, \mathrm{pad}_1 \,\|\, S$ without ever seeing k.

Walking question

Why can the attacker resume without k or M (Figure 14.3)? Because the only thing the next compression step needs is the *current chaining value*, and that value is the tag t sitting in plain view. Merkle–Damgard never "closes off" the state. The attacker does need to reconstruct pad_1, which encodes the length of $k \,\|\, M$ – hence they must know or brute-force the key length. Keys are short, so that is cheap.

There are several clean fixes, and you should recognize all of them:

- **HMAC** (Chapter 15) nests two hash calls, $\mathrm{H}(k_{\text{out}} \,\|\, \mathrm{H}(k_{\text{in}} \,\|\, M))$. The outer hash hides the inner chaining value, so resuming buys the attacker nothing. This is the right way to key a Merkle–Damgard hash, and the reason you should never roll your own prefix MAC.
- **Truncation / output transform.** SHA-512/256 runs SHA-512 internally (512-bit state) but outputs only 256 bits, so the full chaining value is never revealed. SHA-224 and SHA-384 are similar.
- **A different construction.** Sponge-based hashes (SHA-3, next section) are immune to length extension by design, because their output is a *squeeze* of a larger hidden state.

Implement it

Length extension is so mechanical that standard libraries reproduce it in a few lines. We show it here so you can see how little the attacker needs. **Learning only – the lesson is "do not build prefix MACs," not "go forge tags."**

```
# Pseudocode against a Merkle-Damgard hash H with block-aligned padding.
# Attacker knows: message M, tag t = H(secret || M), and len(secret).
def forge(t, M, secret_len, suffix):
    glue = md_padding(secret_len + len(M))   # the pad H would have appended
    forged_message = M + glue + suffix       # what the new tag authenticates
    # Resume the hash from the leaked internal state == t:
    h = HashState.from_digest(t)             # set chaining value = t
    h.set_length(secret_len + len(M) + len(glue))
    h.update(suffix)
    forged_tag = h.digest()                  # == H(secret || M || glue || suffix)
    return forged_message, forged_tag
```

Real tools (`hashpump`, the `hlextend` library) automate exactly this. The defense is never to expose a raw $\mathrm{H}(k \,\|\, M)$ as an authenticator.

14.5 The sponge construction and SHA-3

In 2007 NIST opened a public competition for a new hash standard, partly hedging against the possibility that the MD-family weaknesses behind the MD5 and SHA-1 breaks might eventually reach SHA-2. The winner, announced in 2012 and standardized as SHA-3 in 2015, was *Keccak*, built on a completely different idea: the *sponge*.

A sponge maintains a wide internal state of $b = r + c$ bits, conceptually split into a *rate* part of r bits (the part that touches the message) and a *capacity* part of c bits (the secret reservoir that never directly meets the input or output). A fixed permutation π – for Keccak this is the 1600-bit permutation Keccak-f – stirs the whole state. Hashing proceeds in two phases.

Absorbing. Pad the message and cut it into r-bit blocks. For each block, XOR it into the rate part of the state, then apply π to the full b-bit state. The capacity part is never touched by the message directly; it only changes through π.

Squeezing. To produce output, read off the first r bits of the state; if more output is needed, apply π and read another r bits, repeating until enough bits are out. Truncate to the desired digest length.

Figure 14.4 traces both phases across the split state: message blocks enter and output blocks exit through the top r-bit rate lane, while the bottom c-bit capacity lane remains sealed inside the permutation chain.

Why it works

Why the capacity buys security, and why length extension dies.
The collision and (second-)preimage resistance of a sponge are governed by the *capacity* c, not

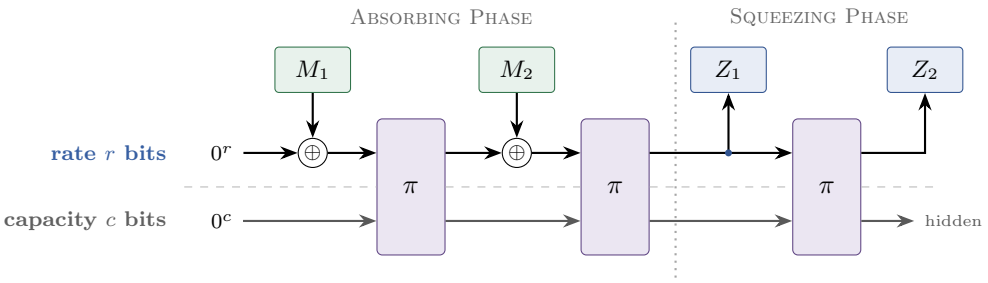


Figure 14.4: The sponge construction. Message blocks M_i are XORed only into the r-bit rate portion of the state before each full-state permutation π; output blocks Z_j are squeezed only from the rate portion. The c-bit capacity never touches inputs or outputs directly.

the output length alone: a well-built sponge offers roughly $\min(2^{c/2}, 2^{n/2})$ collision resistance and $\min(2^{c/2}, 2^n)$ preimage resistance, where n is the digest length. SHA3-256 uses $c = 512$, comfortably above its 2^{128} collision target.

Length extension cannot work because the digest is only the *rate* portion that gets squeezed out – the c-bit capacity is never revealed. An attacker who wants to resume the computation would need the full b-bit internal state, but the output exposes at most r of those bits per squeeze and *none* of the capacity. There is no leaked chaining value to continue from. This immunity is a clean, structural win over Merkle–Damgard, and it is why protocols can safely use raw $\text{SHA3}(k \,\|\, M)$ as a MAC (KMAC formalizes exactly this).

Focus

The SHA-3 family. All members share the 1600-bit Keccak-f permutation and differ only in rate/capacity and output length:

Function	Output n	Capacity c	Rate r	Collision sec.
SHA3-224	224	448	1152	2^{112}
SHA3-256	256	512	1088	2^{128}
SHA3-384	384	768	832	2^{192}
SHA3-512	512	1024	576	2^{256}
SHAKE128	any	256	1344	2^{128} (XOF)
SHAKE256	any	512	1088	2^{256} (XOF)

SHAKE128 and SHAKE256 are *extendable-output functions* (XOFs): the squeeze phase can run as long as you like, so a single call produces a stream of any length. XOFs are heavily used in post-quantum schemes (Chapter 35) for deterministic randomness expansion – Kyber and Dilithium both lean on SHAKE.

Connection

The Keccak-f permutation organizes the 1600-bit state as a $5 \times 5 \times 64$ array of bits and applies five sub-steps per round $(\theta, \rho, \pi, \chi, \iota)$ for 24 rounds, all over $\mathbb{F}_2$. The diffusion analysis – how a one-bit change spreads to touch the whole state – is linear-algebra-over-$\mathbb{F}_2$ in spirit, the same finite-field machinery from Chapter 6 that underlies AES's MixColumns.

14.6 SHA-2 in practice

Despite SHA-3's elegance, the dominant standard in deployed cryptography in 2026 is still *SHA-2*, specifically SHA-256 and SHA-512. SHA-2 is a Merkle–Damgard hash whose compression function is itself built from a block-cipher-like round function (a Davies–Meyer construction: $H_i = E_{M_i}(H_{i-1}) \oplus H_{i-1}$, where the feed-forward XOR is what makes a permutation one-way).

Focus

SHA-256 at a glance.

- Output: 256 bits. Block: 512 bits. State: eight 32-bit words $a, b, \ldots, h$ (256 bits total).
- Padding: append a `1` bit, then `0`s, then the 64-bit message length, to reach a multiple of 512 bits.
- 64 rounds per block, each mixing in one 32-bit "message schedule" word W_t and one round constant K_t (the K_t are fractional parts of cube roots of the first 64 primes – a "nothing-up-my-sleeve" choice so no one suspects a planted backdoor).
- Operations per round: modular addition mod 2^{32}, bitwise XOR/AND, and fixed rotations – the "ARX-like" style, fast in both hardware and software.

SHA-512 is the same design widened to 64-bit words, an 80-round schedule, a 1024-bit block, and a higher security margin across the board. On 64-bit CPUs SHA-512 is often *faster per byte* than SHA-256 because it digests twice the data per block.

In the real world

Where SHA-2 actually runs.

- **TLS** (Chapter 25): certificate signatures, the transcript hash in the handshake, and HKDF key derivation all use SHA-256 or SHA-384. The cipher-suite name `TLS_AES_128_GCM_SHA256` names the hash explicitly.
- **Bitcoin** (Chapter 36): block hashing and proof-of-work are double SHA-256, SHA256(SHA256(x)) – the double application was Satoshi's reflex defense against length extension.
- **Git** stores every object under its SHA-1 name and is migrating to SHA-256; **package managers** (apt, npm, pip) ship SHA-256 digests so you can verify a download was not tampered with.
- **Signatures** (Chapter 22): RSA, ECDSA, and EdDSA all sign H(M), never M directly. The hash is what lets a fixed-size signature cover an arbitrarily long message, and signature unforgeability depends on the collision resistance of the hash.

Modern x86 and ARM cores include SHA extensions (`SHA256RNDS2`, ARMv8 `SHA256H`) that hash at several gigabytes per second per core.

Common trap

Do not confuse fast with safe for passwords. SHA-256's speed is a virtue for fingerprints and signatures and a *liability* for password storage: an attacker can try billions of password

guesses per second per GPU against a bare SHA-256. Password hashing needs a deliberately *slow*, memory-hard function – bcrypt, scrypt, Argon2 – as Section 14.9 explains. "Hash the password with SHA-256" is one of the most common and most dangerous half-measures in the wild.

14.7 Application 1: fingerprints and integrity

The simplest use of a hash is as a *fingerprint*: a short value that stands in for a large object so you can detect any change. Publish H(file) over a trusted channel; anyone who downloads the file over an untrusted channel recomputes the hash and compares. A single flipped bit anywhere in the file changes about half the digest bits (the *avalanche* property), so tampering is caught with overwhelming probability.

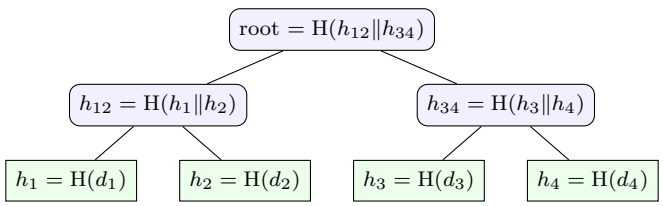
This is the backbone of software-update integrity, content-addressed storage (Git, IPFS, Docker image layers – each object is named by its hash, so the name *is* a verifiable identity), and deduplication. The property that does the work is *second-preimage* resistance: given the published file, an attacker must not be able to craft a *different* file with the same fingerprint.

Attack

When fingerprints rely on the wrong property: the SHA-1 collision and "evil twins."
If an attacker controls *both* the original and the tampered file – say, a software author who later wants to repudiate, or an attacker who submits a benign file for review then swaps in a malicious twin with the same hash – then *collision* resistance is what matters, and a 128- or 160-bit hash is too weak. In 2017 the SHAttered attack produced two distinct PDF files with identical SHA-1 digests at a cost of $\approx 2^{63}$ compressions. In 2019–2020 *chosen-prefix* SHA-1 collisions (where the two files can begin with arbitrary attacker-chosen prefixes) became practical, breaking SHA-1's use in PGP web-of-trust certificates. The lesson: a fingerprint is only as trustworthy as the hash's collision resistance whenever the adversary can influence the original, and SHA-1 and MD5 are finished for that role.

14.7.1 Merkle trees: fingerprinting a collection

To fingerprint not one file but a whole set – and to let someone verify a single element without downloading the rest – hash the elements pairwise up a binary tree. Each leaf is the hash of a data block; each internal node is the hash of its two children's hashes; the single *root* hash commits to the entire collection.



To prove block d_3 is in the tree under a known root, you reveal d_3 and the *authentication path*: the sibling hashes h_4 and h_{12}. The verifier recomputes $h_3 = \mathrm{H}(d_3)$, then $h_{34} = \mathrm{H}(h_3 \,\|\, h_4)$, then $\mathrm{H}(h_{12} \,\|\, h_{34})$ and checks it equals the root. A proof for one element among N has only $\log_2 N$ hashes. Collision resistance of H means a cheater cannot present a different d_3' that recomputes to the same root.

In the real world

Merkle trees are everywhere: Bitcoin and Ethereum commit each block's transactions to a Merkle root in the block header (so a light client verifies one payment with a $\log N$ proof); Certificate Transparency logs are Merkle trees of every issued TLS certificate; Git's tree objects, BitTorrent v2, ZFS and Btrfs block checksums, and the `rsync`/Dropbox style sync protocols all use the same structure. Chapter 36 develops the blockchain version in full.

14.8 Application 2: commitments

A *commitment scheme* is a cryptographic envelope. You want to lock in a value now and reveal it later, in a way that (a) you cannot change after committing, and (b) reveals nothing in the meantime. Think of sealing a prediction in an envelope and handing it over: *binding* (you cannot swap the contents) and *hiding* (no one can peek).

A hash gives a simple commitment. To commit to a message m, choose a random *nonce* r (say 256 bits) and publish

$$c = \mathrm{H}(m \,\|\, r).$$

To open, reveal m and r; the verifier recomputes $\mathrm{H}(m \,\|\, r)$ and checks it equals c.

Why it works

Why it binds and why it hides.

- **Binding** rests on collision resistance. To open the same c to a different message $m' \neq m$, you would need (m', r') with $\mathrm{H}(m' \,\|\, r') = \mathrm{H}(m \,\|\, r)$ – a collision. Find one and you have broken the hash.
- **Hiding** rests on one-wayness, with help from the random r. Without r, committing to a low-entropy value ("yes"/"no", or a chess move from a small set) would let anyone just hash all possibilities and match c. The nonce r injects enough entropy that the digest reveals nothing guessable about m – in the random-oracle model, c is uniform and independent of m until r is revealed.

The random nonce is not optional. A commitment to a value from a small or guessable set, computed as $\mathrm{H}(m)$ with no nonce, is immediately broken by brute force over the possibilities – the same mistake as unsalted password hashing.

In the real world

Hash commitments power coin-flipping and fair lotteries (each party commits to a random value, then both open and XOR), sealed-bid auctions, the "commit–reveal" pattern in smart contracts, and the Fiat–Shamir transform that turns interactive zero-knowledge proofs into

non-interactive signatures (Chapters 22 and 31). When you cannot afford the algebraic, additively-homomorphic Pedersen commitments of Chapter 32, a salted hash commitment is the cheap default.

14.9 Password Hashing, Memory-Hard Key Derivation, and Phishing-Resistant Authentication

Up to this point in the chapter, speed has been an unmixed advantage. We praised Merkle–Damgård and sponge designs that stream multiple gigabytes per second through vector registers. On modern x86 and ARM silicon, SHA-256 compresses a 64-byte block in under 200 nanoseconds, while BLAKE3 saturates memory buses at three gigabytes per second per core. For verifying operating system images, signing code, and aggregating transactions into Merkle roots, this high throughput is essential.

For storing human passwords, however, the same speed is a serious weakness.

Consider the physical asymmetry between the legitimate user and the attacker. A human user logging into an account types their password, submits the form, and waits. If the server takes two microseconds or two hundred milliseconds to verify the credential, the human perceives no difference: two hundred milliseconds feels instantaneous. And the server performs that calculation once per login session.

Now place a compromised database dump of password hashes into the hands of an adversary. The attacker does not execute one hash; they execute trillions of guesses against every compromised row. If the verification algorithm requires only 200 nanoseconds, a commodity desktop equipped with eight modern GPUs evaluates more than 10^{11} candidate passwords every second.

The problem is that people choose predictable passwords. A cryptographic key chosen uniformly from $\{0,1\}^{256}$ contains 256 bits of Shannon entropy; testing 10^{11} keys per second across 2^{256} possibilities would require 3×10^{58} years. But passwords are created by humans, not physical entropy sources. Empirical studies of billions of leaked credentials reveal that human passwords have low entropy, clustering between 25 and 40 bits. Even an eight-character password drawn from the full set of 95 printable ASCII symbols yields only $95^8 \approx 6.6 \times 10^{15} \approx 2^{52.6}$ combinations. Real users rarely sample uniformly: they select dictionary words, familiar names, calendar dates, and predictable substitutions (`P@ssw0rd123`). Against a realistic dictionary of $10^9 \approx 2^{30}$ common passwords, a cluster running at 10^{11} hashes per second exhausts the entire search space in ten milliseconds.

Faster hashing helps an offline password cracker far more than it helps a login server. To protect human credentials, we do not want a hash that is fast. We need a function that is deliberately slow and whose cost is tied to hardware resources that cannot be parallelized away.

14.9.1 Why raw hashes and unsalted hashes fail

The naive approach to password authentication stores the direct digest of the password:

$$h = \mathrm{H}(P).$$

When the user logs in with candidate password P', the server checks whether $\mathrm{H}(P') == h$. If an attacker steals the database, this scheme collapses under two independent failure modes:

1. **Precomputation and Rainbow Tables.** Because the hash function is deterministic and input-independent, an attacker does not need to perform brute force after stealing the database. The attacker selects a dictionary $\mathcal{D}$ of candidate passwords and computes $\mathrm{H}(w)$ for every $w \in \mathcal{D}$ in advance. In 2003, Philippe Oechslin introduced *rainbow tables*, a space-time tradeoff that stores reduction chains of the form:

$$P_0 \xrightarrow{\mathrm{H}} H_0 \xrightarrow{R_1} P_1 \xrightarrow{\mathrm{H}} H_1 \xrightarrow{R_2} \cdots \xrightarrow{R_L} P_L,$$

where each $R_i : \{0,1\}^n \to \Sigma^k$ is a reduction function mapping a hash digest back to a candidate password string. By storing only the endpoints (P_0, P_L), the attacker compresses millions of precomputed hashes into a few gigabytes of disk storage. Looking up a stolen digest takes milliseconds. Worse, a single precomputed table cracks matching passwords across every leaked database worldwide.

2. **Leakage of Password Reuse.** If Alice and Bob choose the same password P, their database entries contain identical hashes $\mathrm{H}(P)$. An attacker holding a database dump spots shared credentials at a glance, enabling cross-account tracking without cracking the underlying string.

To defeat precomputed tables, the server generates a fresh, cryptographically secure 128-bit random value $s \xleftarrow{\$} \{0,1\}^{128}$ (the *salt*) for each user upon registration. The server stores the salt in plaintext alongside the salted hash:

$$(s, h), \quad \text{where } h = \mathrm{H}(s \,\|\, P).$$

The salt is public and requires no secrecy. Its purpose is to make each user's hash input unique. Because s is drawn from a space of 2^{128} possibilities, an attacker cannot use a global precomputed table: that table would have to be computed for $\mathrm{H}(s \,\|\, P)$ across all 2^{128} salts, requiring $|\mathcal{D}| \cdot 2^{128}$ evaluations. In addition, if Alice and Bob choose the identical password, their independent salts $s_{\text{Alice}} \neq s_{\text{Bob}}$ ensure that $\mathrm{H}(s_{\text{Alice}} \,\|\, P) \neq \mathrm{H}(s_{\text{Bob}} \,\|\, P)$, concealing reuse.

Common trap

The salt arithmetic trap. Salting completely destroys global precomputation across multiple users, but it provides zero protection against targeted attacks on a single stolen row. Consider an attacker focusing on Alice's compromised record $(s_{\text{Alice}}, h_{\text{Alice}})$. The salt s_{Alice} is known. To test a candidate password w, the attacker computes $\mathrm{H}(s_{\text{Alice}} \,\|\, w)$ and compares it to h_{Alice}. If H is SHA-256, an attacker running hardware capable of 10^{11} evaluations per second tests candidate passwords against Alice's record at that exact speed. Salting forces the attacker to do dedicated work per user, but if each test costs only a few nanoseconds, a 35-bit password falls in under a second.

14.9.2 Iterative CPU-hard stretching and its silicon limits

To protect a single salted record against brute-force search, the verification function must consume measurable time. This is the concept of *key stretching*: artificially inflating the number of sequential operations required to evaluate the hash.

In 2000, RSA Laboratories standardized **PBKDF2** (Password-Based Key Derivation Function 2, RFC 2898). PBKDF2 iterates a pseudorandom function (typically HMAC-SHA-256) across a user-specified iteration count c:

$$U_1 = \mathrm{HMAC}(P, s \,\|\, \mathrm{INT}(1)), \quad U_2 = \mathrm{HMAC}(P, U_1), \quad \ldots, \quad U_c = \mathrm{HMAC}(P, U_{c-1}),$$

and computes the stretched output as the XOR sum:

$$T = U_1 \oplus U_2 \oplus \cdots \oplus U_c.$$

If a server selects $c = 600{,}000$, computing T takes approximately 100 milliseconds on a server CPU. An attacker attempting 10^9 dictionary guesses must now evaluate 6×10^{14} HMAC operations, multiplying their cracking duration by a factor of 600,000.

Yet PBKDF2 fails against modern adversaries armed with custom ASICs and GPUs. The reason is the economics of chip area.

A general-purpose CPU is engineered to execute complex, branch-heavy instructions on unpredictable data. More than 90% of the physical silicon area on an x86 or ARM CPU die is dedicated to out-of-order execution pipelines, branch predictors, instruction decoders, register renaming tables, and multi-megabyte SRAM caches (L1, L2, and L3). The actual 32-bit arithmetic logic unit (ALU) that performs the additions and bitwise shifts of SHA-256 occupies a minuscule fraction of the chip.

In contrast, computing SHA-256 requires almost no silicon state. The entire working memory of SHA-256 consists of eight 32-bit words – just 32 bytes (256 bits) of register flip-flops. The round logic consists of simple combinational gates (AND, XOR, bit rotations, and modular 32-bit adders). An adversary building an Application-Specific Integrated Circuit (ASIC) discards all branch predictors, instruction caches, and memory buses. The ASIC designer packs tens of thousands of fully unrolled, pipelined SHA-256 cores onto a single 250 W silicon die.

Because PBKDF2 is purely sequential and requires only 32 bytes of working memory, each tiny ASIC core runs completely in registers without touching external RAM. While a high-end 16-core CPU executes 32 concurrent threads, a single custom ASIC executes 10^4 independent candidate streams in parallel at gigahertz clock frequencies. Both defender and attacker must compute c iterations per guess, but an iteration on custom silicon costs fractions of a nanojoule. CPU-bound iteration scales linearly for defender and attacker alike, allowing custom hardware to overcome the iteration count.

14.9.3 Memory-hard key derivation: scrypt and Argon2

To negate the advantage of custom silicon, cryptographers identified the one physical component that an attacker cannot miniaturize cheaply: **memory** (RAM) and memory bandwidth.

An SRAM memory bit requires a six-transistor cell (6T SRAM). Placing 64 megabytes of SRAM directly onto an ASIC die consumes roughly $60\,\text{mm}^2$ of expensive silicon, crowding out compute logic. Dynamic RAM (DRAM) is denser, but DRAM must reside on separate memory dies connected through physical copper pins or through-silicon vias (TSVs). Those physical interfaces are strictly constrained by pin count, thermal dissipation, and bus bandwidth limits (high-bandwidth memory rarely exceeds 1 to 2 TB/s per package).

In 2009, Colin Percival introduced the concept of *memory-hard functions* in the design of **scrypt**. Percival formalized the defensive metric through **Area-Time (AT) complexity**. The true cost of designing and running custom hardware is proportional to the product of the silicon die area A and the execution time T (AT or AT^2). If an algorithm requires 64 MiB of working memory per instance, an ASIC designer can no longer place 10^4 parallel cores on a chip: the physical memory required would demand an impossibly large die. If the cores share an external DRAM bus, thousands of cores contend for memory channels and stall on memory latency.

Memory-hard functions also enforce a severe *time-memory tradeoff* (TMTO): if an attacker attempts

to save silicon area by storing only a fraction $1/M$ of the intermediate states, they must repeatedly recompute the discarded blocks, causing total execution time T to explode.

In 2015, the Password Hashing Competition (PHC) selected **Argon2**, designed by Alex Biryukov, Daniel Dinu, and Dmitry Khovratovich, as the new standard (RFC 9106). Argon2 turns memory hardness into a concrete design:

- **Memory Matrix Layout.** Argon2 configures memory as a 2D matrix of p parallel lanes (threads) and $q = m/p$ columns, where each matrix cell is a 1 KiB block (1024 bytes, structured as an 8×16 array of 64-bit words). A standard configuration allocating $m = 64$ MiB across $p = 4$ lanes establishes a matrix of $4 \times 16{,}384$ blocks.
- **Four Slices.** Computation across all lanes proceeds through four vertical synchronization slices (Slice 0 through Slice 3). Within a slice, the p lanes compute blocks in parallel; at slice boundaries, lanes synchronize.
- **Block Mixing Function G.** The core permutation G operates on two 1 KiB blocks using the round function of BLAKE2b. It evaluates eight 64-bit SIMD rounds, vectorizing efficiently on CPU vector units (AVX2, AVX-512, ARM Neon) while using only 64-bit additions, rotations, and XORs.

Argon2 resolves a subtle tension between memory hardness and side-channel security through three distinct variants:

Argon2d (Data-Dependent Addressing). To compute block $B_{i,j}$, the algorithm takes the 64-bit integer stored in the first word of the previous block $B_{i,j-1}$ and uses it to compute a pseudorandom reference index (i', j'). Block $B_{i,j}$ is then computed as $G(B_{i,j-1}, B_{i',j'})$. Because the memory address depends directly on the data, an attacker who drops blocks cannot predict which block will be referenced next, forcing full memory retention. However, because memory lookup addresses depend on password data, an attacker sharing physical CPU hardware or a cloud hypervisor can monitor CPU cache misses (cache-timing attacks like Flush+Reload) to deduce memory addresses and recover password bits.

Argon2i (Data-Independent Addressing). The reference address (i', j') is generated by applying G to a deterministic sequence counter, the lane index, and the slice index, without reading any password-dependent block contents. Because memory access patterns are independent of secret data, Argon2i is immune to cache-timing side channels. However, because the access sequence is predictable in advance, an attacker can use graph-pebble algorithms to construct time-memory tradeoffs with lower recomputation penalties.

Argon2id (The Hybrid Default). Argon2id combines both approaches. During the first pass, Slices 0 and 1 compute reference addresses using data-independent addressing (Argon2i mode), overwriting memory and shielding the early password-dependent state against cache-timing leaks. Slices 2 and 3 of the first pass, and all subsequent passes $2, \ldots, t$, switch to data-dependent addressing (Argon2d mode), locking in maximal Area-Time hardness against ASICs and GPUs.

Figure 14.5 illustrates this contrast: while a SHA-256 ASIC core requires only 32 bytes of state on $< 0.05\,\text{mm}^2$ of silicon, Argon2id's 64 MiB memory matrix saturates external memory buses and ties the attacker to the limits of DRAM.

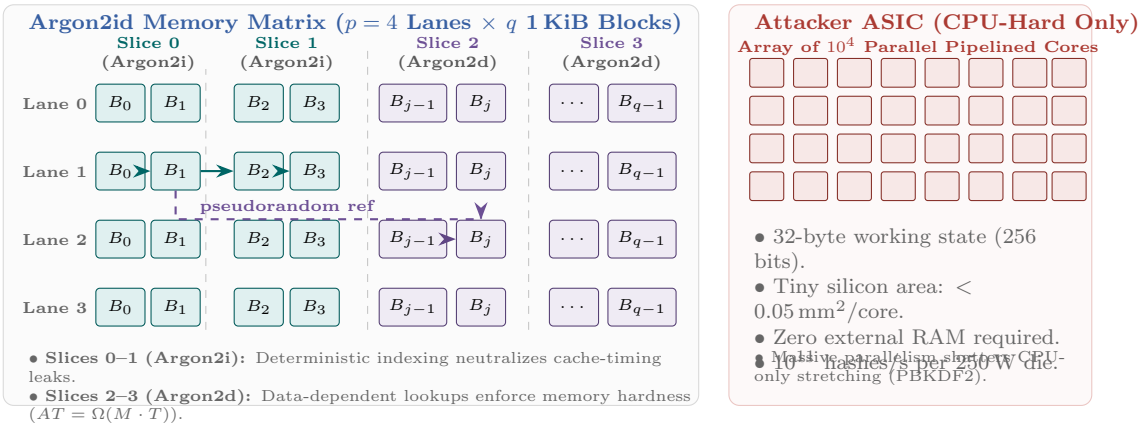


Figure 14.5: Memory-hard functions (Argon2id) force computation across a large 2D matrix (p lanes $\times$ q blocks) of 1 KiB units. Slices 0–1 use data-independent addressing to neutralize cache-timing side channels, while Slices 2–3 use data-dependent addressing to enforce high Area-Time complexity, defeating massively parallel ASIC cores that hold only 32 bytes of state.

Implement it

Computing Argon2id with standard parameters. In production systems, invoke a vetted C binding such as `argon2-cffi` or `libsodium`.

```python
import os, hmac
from argon2.low_level import hash_secret_raw, Type

def register_user(password: str) -> tuple[bytes, bytes]:
    # 16-byte random salt per user defeats rainbow tables:
    salt = os.urandom(16)
    # 64 MiB memory, 3 passes, 4 parallel lanes:
    digest = hash_secret_raw(
        secret=password.encode("utf-8"),
        salt=salt,
        time_cost=3,
        memory_cost=64 * 1024,    # 64 MiB in KiB
        parallelism=4,
        hash_len=32,
        type=Type.ID              # Argon2id hybrid
    )
    return salt, digest           # salt is stored openly next to the digest

def verify_user(password: str, salt: bytes, stored_digest: bytes) -> bool:
    candidate = hash_secret_raw(
        secret=password.encode("utf-8"),
        salt=salt,
        time_cost=3,
        memory_cost=64 * 1024,
        parallelism=4,
        hash_len=32,
        type=Type.ID
    )
```

```
    # Mandatory constant-time comparison to prevent timing side channels:
    return hmac.compare_digest(candidate, stored_digest)
```

Focus

Salt vs. pepper. A *salt* is unique per user, random, and stored publicly in the database table next to the hash digest. Its sole job is to destroy precomputation and rainbow tables. A *pepper* is a single secret key shared across all users, held in an isolated hardware security module (HSM) or secret-management service, and injected into the hash computation. If an adversary dumps the SQL database but lacks access to the application server or HSM, the pepper prevents offline dictionary attacks entirely. A pepper provides defense-in-depth, but it does not replace per-user salts: omitting salts would still expose password reuse across accounts.

14.9.4 Origin-bound phishing-resistant authentication: WebAuthn and passkeys

Argon2id forces attackers to expend vast memory and silicon die area per guess, raising the computational barrier to offline password cracking. Yet even if a server provisions a gigabyte of RAM per hash, password authentication has a basic weakness: **a password is a shared secret entered into an untrusted client interface**.

If an attacker registers a lookalike domain name (such as `g00gle.com`), sends a convincing email lure, and presents a replica of a legitimate login portal, the human user willingly types their plaintext password into the form.

The attacker's reverse proxy intercepts the plaintext string. The proxy immediately forwards that password to the genuine service and authenticates as the user. Traditional multi-factor authentication (MFA) mechanisms – including SMS one-time passcodes and six-digit time-based codes (TOTP, RFC 6238) – fail to defeat this attack. Automated reverse-proxy frameworks (such as Evilginx) intercept and relay the user's six-digit OTP in real time, obtain the authenticated session cookies from the legitimate server, and compromise the account. Human vigilance cannot solve this problem: users cannot reliably detect microscopic typographic substitutions in URL address bars under stress.

The **Web Authentication** standard (**WebAuthn**), formulated by the W3C and the FIDO Alliance (and popularized under the commercial moniker *Passkeys*), resolves the vulnerability by discarding shared secrets entirely.

Instead of authenticating through a shared secret string transmitted across the network, WebAuthn authenticates via an **origin-bound asymmetric digital signature** executed by local security hardware – such as a Secure Enclave in an iPhone or Mac, a Trusted Platform Module (TPM) on a PC, or an external hardware security key (FIDO2 token).

The protocol divides authentication into two phases:

1. **Registration.** When a user registers with a relying party (such as `https://accounts.google.com`), the browser requests credential creation from the hardware authenticator via the WebAuthn API. The browser explicitly declares the canonical Relying Party Identifier:

$$\text{RP ID} = \texttt{"accounts.google.com"}.$$

The authenticator creates an asymmetric keypair (pk, sk) (e.g., Ed25519 or ECDSA over NIST P-256). The private key sk is sealed within tamper-resistant silicon and cannot be extracted under any software instruction. The public key pk is returned to the server alongside a credential identifier. The server stores only pk. The server stores no password, no hash digest, and no secret. If the server database is breached, the attacker acquires only public keys.

2. **Authentication (Challenge-Response).** During login, the server generates a fresh 256-bit cryptographically random challenge nonce $c \xleftarrow{\$} \{0,1\}^{256}$ and transmits it to the client. The browser constructs a structured UTF-8 JSON object called `clientDataJSON`:

$$\begin{aligned}\texttt{clientDataJSON} = \{&\texttt{"type"} : \texttt{"webauthn.get"},\\ &\texttt{"challenge"} : c,\\ &\texttt{"origin"} : \texttt{"https://accounts.google.com"}\}.\end{aligned}$$

The browser computes the SHA-256 digest of this structure:

$$h_{\text{client}} = \text{SHA-256}(\texttt{clientDataJSON}).$$

The browser passes h_{client} and the requested RP ID to the hardware authenticator. After verifying local user presence through biometric confirmation (TouchID/FaceID) or a PIN, the authenticator signs the concatenation:

$$\sigma = \text{Sign}_{sk}(\text{authenticatorData} \,\|\, h_{\text{client}}).$$

The browser transmits σ and `clientDataJSON` back to the server. The server computes SHA-256(`clientDataJSON`), verifies that the origin field in `clientDataJSON` strictly matches the expected origin `https://accounts.google.com`, verifies that the challenge matches c, and checks that $\text{Vrfy}_{pk}(\sigma, \dots)$ accepts.

Figure 14.6 shows why this construction defeats phishing.

Suppose an attacker tricks a user into visiting `https://evil-login.com`. The phishing server connects to `accounts.google.com`, obtains a valid challenge c, and relays c to the victim's browser. The victim agrees to authenticate.

When the malicious webpage invokes the WebAuthn API, the browser itself determines the origin. The webpage cannot spoof this value: the browser user agent reads the active, verified address bar origin:

$$\texttt{origin} = \texttt{"https://evil-login.com"}.$$

The browser passes RP ID `"evil-login.com"` to the authenticator and constructs `clientDataJSON` containing that domain string.

Two separate mechanisms stop the attack:

1. **Credential Isolation.** The hardware authenticator looks up keys scoped to the RP ID `"evil-login.com"`. Because the authenticator isolated the user's Google keypair strictly to `"accounts.google.com"`, no matching private key exists for the phishing domain, and the authenticator refuses to sign.

2. **Cryptographic Origin Binding.** Even if the authenticator held a key registered under `evil-login.com`, the signature σ commits to the digest of the client data:

$$h_{\text{client}} = \text{SHA-256}(\dots \texttt{"https://evil-login.com"} \dots).$$

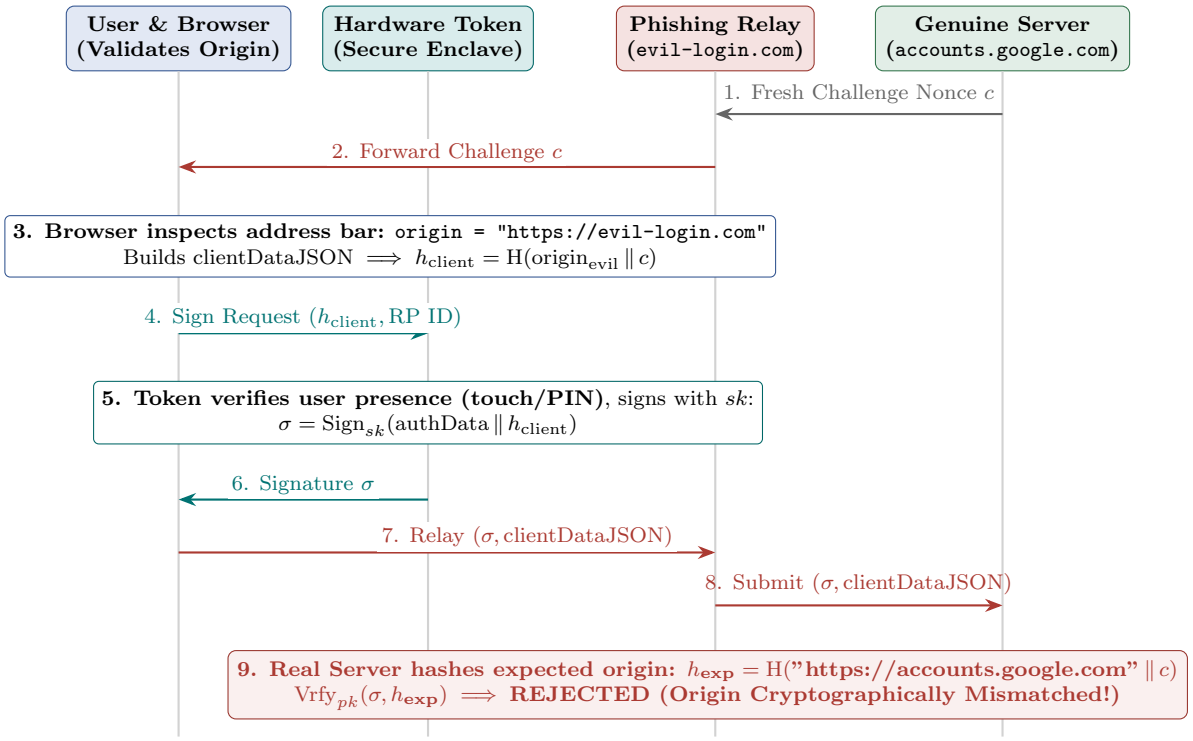


Figure 14.6: WebAuthn / FIDO2 origin-bound authentication defeating a real-time phishing relay. The browser embeds the verified address-bar origin (`evil-login.com`) into the challenge digest h_{client}, causing signature verification on the genuine server (`accounts.google.com`) to fail by cryptographic origin mismatch.

When the phishing relay transmits σ to `accounts.google.com`, the genuine server evaluates:

$$h_{\text{expected}} = \text{SHA-256}(\ldots \texttt{"https://accounts.google.com"} \ldots).$$

Because the hash function is second-preimage resistant, $h_{\text{client}} \neq h_{\text{expected}}$, and signature verification $\text{Vrfy}_{pk}(\sigma, \ldots)$ fails.

Phishing resistance therefore comes from binding the origin into the signed data, not from the user's ability to spot a fake domain.

14.10 The real-world status of named hashes

Hash	Output	Status (2026)	Notes
MD5	128	**broken**	collisions trivial since 2004; never use
SHA-1	160	**broken**	chosen-prefix collisions practical; retired
SHA-256	256	strong	the default modern standard
SHA-512	512	strong	faster on 64-bit CPUs; higher margin
SHA-512/256	256	strong	length-extension-safe, 256-bit output
SHA3-256	256	strong	sponge; length-extension-immune
SHAKE128/256	any	strong	XOFs; used across post-quantum schemes
BLAKE2/3	var	strong	very fast; BLAKE3 is tree-based and parallel

History

The history of broken hashes is a cautionary one. Ron Rivest designed MD4 (1990) and MD5 (1991) as fast software hashes, while the NSA and NIST published SHA-0 (1993) and SHA-1 (1995). Weaknesses in MD4 were found almost immediately. At Crypto 2004, Xiaoyun Wang and co-authors presented differential attacks that found MD5 collisions in minutes and showed SHA-1 breakable in theory in 2005. Despite Wang's attack, certificate authorities kept issuing MD5 certificates until Sotirov et al. forged a rogue root CA certificate in 2008. A full SHA-1 collision was computed in 2017 (SHAttered, Stevens et al., roughly 2^{63} operations), followed by chosen-prefix collisions in 2020.

Every one of those broken hashes was Merkle–Damgard with a short output and an under-analyzed compression function. That drove NIST's public SHA-3 competition (2007–2012), won by Keccak (Guido Bertoni, Joan Daemen, Michaël Peeters, and Gilles Van Assche), which introduced the sponge construction as a backup independent of Merkle–Damgard. SHA-2 (2001) has stood for over two decades, and the pattern across all of them – collisions first, preimages much later or never, output length and compression-function strength deciding the timeline – is what the birthday bound and the MD security proof predict.

Why this matters

Hash functions are the most general-purpose tool in cryptography. A single primitive that is one-way, collision resistant, and behaves like a random oracle lets you fingerprint data, commit to secrets, key a MAC, build a signature, store a password, name an object by its content, and anchor a blockchain – often all in the same system. That is why an attack on a widely

used hash is front-page news in the field, and why so much care goes into output length, construction style, and the slow, public scrutiny that earns a hash the right to be trusted.

Connection

An unkeyed hash proves that a file matches its digest, but anyone who intercepts a message M can replace it with M' and recompute $\mathrm{H}(M')$. In Chapter 15 we mix a shared secret key into the hash to build Message Authentication Codes (MACs) and HMAC, turning unkeyed digests into tamper-evident seals.

14.11 Exercises

Exercise 14.1 (Counting pairs)**.** A hash has $n = 32$ output bits. Roughly how many random inputs must you hash before you expect a collision? How many before you expect to hit one *specific* target digest by chance? Explain the 2^{16} versus 2^{32} gap in one sentence using the birthday bound.

Exercise 14.2 (Birthday probability)**.** Using $p(q) \approx 1 - e^{-q(q-1)/(2N)}$ with $N = 2^{40}$, find the smallest q (to the nearest power of two) for which the collision probability exceeds $1/2$. Compare to the rule of thumb $q \approx 1.177\sqrt{N}$.

Exercise 14.3 (Which property?)**.** For each application, state which of the three resistance properties it most directly relies on, and give a one-line attack if that property fails: (a) a software-download checksum published by the vendor; (b) a sealed-bid auction using $\mathrm{H}(\text{bid} \,\|\, r)$; (c) deduplicating a backup store by content hash; (d) a signature over $\mathrm{H}(\text{contract})$ where the signer wrote the contract.

Exercise 14.4 (Reduction)**.** Prove that if H is collision resistant then the function $\mathrm{H}'(x) = \mathrm{H}(\mathrm{H}(x))$ is also collision resistant. Where exactly does your reduction use collision resistance of H?

Exercise 14.5 (Length extension by hand)**.** A naive server authenticates messages with $t = \mathrm{H}(k \,\|\, M)$ for a Merkle–Damgard hash and an 8-byte secret k. Given a captured pair $(M = \texttt{"amount=10"}, t)$, describe precisely the bytes of the forged message you can authenticate without knowing k, and the one piece of information about k you must know or guess. Then explain in one sentence why HMAC blocks your attack.

Exercise 14.6 (Toy collision)**.** Run the toy Merkle–Damgard hash from Section 14.3 (8-bit state). Empirically, after how many random inputs does the first collision typically appear? Average over 20 runs and compare to the birthday prediction $1.177 \cdot 2^4$. Why is the empirical mean a bit higher than that point estimate?

Exercise 14.7 (Sponge security)**.** SHA3-256 has capacity $c = 512$ and output $n = 256$. State its collision and preimage security levels and explain, in terms of the rate/capacity split, why an attacker cannot mount a length-extension attack against $\text{SHA3-256}(k \,\|\, M)$.

Exercise 14.8 (Salt arithmetic)**.** A breached database stores $(\text{salt}, \text{SHA256}(\text{salt} \,\|\, \text{password}))$ for 10 million users with unique 16-byte salts. An attacker has a wordlist of 10^9 candidate passwords and a rig doing 10^{10} SHA-256/s. (a) How long to crack *one* target account? (b) Why does the per-user salt mean the total work to crack all 10 million accounts is essentially 10 million times that, rather than amortized? (c) How would switching to Argon2 with a 0.5-second cost per guess change part (a)?

Exercise 14.9 (Pepper)**.** Explain why a secret pepper stored outside the database renders a database-only breach uncrackable, but does *not* replace per-user salts. Construct a scenario where using only a pepper (no salt) leaks password reuse across users.

Exercise 14.10 (Merkle proof)**.** A Merkle tree has 8 leaves $d_1, \ldots, d_8$. Write out the exact authentication path (the list of sibling hashes, in order) needed to prove d_5 is in the tree, and the sequence of hash computations the verifier performs to reach the root. How does the proof size grow with the number of leaves?

Exercise 14.11 (Break a homemade commitment)**.** A protocol commits to a single decision bit $b \in \{0, 1\}$ by publishing $c = \mathrm{H}(b)$ with no nonce. Show that this commitment is not hiding by giving the attacker's exact procedure. Then show that adding a 256-bit random nonce r and publishing $\mathrm{H}(b \,\|\, r)$ restores hiding, citing the property of H you rely on.

Chapter 15

Message Authentication Codes and HMAC

In Chapter 14, hash functions gave us a tamper-evident fingerprint for any file, but an unkeyed hash cannot prove *who* computed it: anyone can alter a message and hash the new version. Up to now we have focused on keeping Eve from *reading* a message, yet confidentiality is only half of what security means in practice, and on many days the less important half.

Picture Alice wiring money. She sends her bank the message

`"Pay Bob $100"`.

Suppose Eve cannot read it – it is encrypted under a strong cipher. Has Eve been defeated? Not at all. If Eve can *change* the ciphertext as it flies past, she may be able to turn it into an encryption of `"Pay Eve $9999"`, or simply replay last week's `"Pay Eve $100"` ten thousand times. Encryption said nothing about whether the message arrived intact, or whether it really came from Alice. That second guarantee – this message is exactly what the sender sent, and it really was that sender – is *integrity* and *authenticity*. The tool that delivers it is the *message authentication code*, or MAC.

This chapter builds MACs from the ground up: what they promise, what "secure" means for them, how to construct them from block ciphers (CBC-MAC) and from hash functions (HMAC), why the famous length-extension weakness of plain hashing forces HMAC's nested shape, how to combine a MAC with encryption without shooting yourself in the foot, and the small but deadly matter of comparing tags in constant time.

15.1 Confidentiality Is Not Integrity

The two goals are genuinely different, and it helps to nail down the distinction before building anything.

- **Confidentiality** hides the *content*. After seeing the ciphertext, Eve learns nothing useful about the plaintext. This is what encryption gives you.
- **Integrity** guarantees the message was not *altered* in transit. If even one bit flips, the receiver notices and rejects.

- **Authenticity** guarantees the message came from who it claims to come from – someone holding the secret key.

In the symmetric-key world, integrity and authenticity come bundled together: a MAC delivers both. If a tag verifies under the shared key, the receiver knows simultaneously that the bits are unchanged and that whoever produced them held the key. (In the public-key world these split apart again into digital signatures, Chapter 22.)

Intuition

A MAC is a *tamper-evident seal* for digital data. Think of a wax seal on a letter, or the foil under a bottle cap. Anyone can see the message; only the holder of the secret signet ring can stamp a valid seal; and a stamp made for one letter cannot be peeled off and stuck onto another. The MAC is that seal, computed with a key only Alice and Bob share. Eve can rewrite the letter all she likes, but she cannot forge the stamp, so Bob throws her version away.

A point that surprises newcomers: *encryption alone provides no integrity.* It is tempting to reason "the ciphertext is gibberish to Eve, so she cannot meaningfully modify it." That reasoning is wrong, and the next box shows exactly how wrong.

Attack

Malleability of stream ciphers. A stream cipher (Chapter 10) encrypts by XOR: $c = m \oplus k$, where k is the keystream. Suppose Alice encrypts the 11-byte ASCII message `"Pay Eve $10"` and Eve, who cannot read the keystream, nonetheless *knows the format* of these wire-transfer messages. She wants to change the amount.
The last two bytes of plaintext are `'1'` (0x31) and `'0'` (0x30). Eve wants `'9'` (0x39) and `'9'` (0x39). She does not touch the key; she XORs into the ciphertext bytes the difference between what is there and what she wants:

$$\Delta = \texttt{0x31} \oplus \texttt{0x39} = \texttt{0x08}, \qquad \Delta' = \texttt{0x30} \oplus \texttt{0x39} = \texttt{0x09}.$$

She replaces c_{10} with $c_{10} \oplus \texttt{0x08}$ and c_{11} with $c_{11} \oplus \texttt{0x09}$. When Bob decrypts:

$$(c_{10} \oplus \texttt{0x08}) \oplus k_{10} = (m_{10} \oplus k_{10} \oplus \texttt{0x08}) \oplus k_{10} = m_{10} \oplus \texttt{0x08} = \texttt{'1'} \oplus \texttt{0x08} = \texttt{'9'}.$$

The keystream cancels regardless of its value. Eve never learned a single keystream byte, never decrypted anything, yet flipped the payment from \$10 to \$99. Confidentiality was perfect, but integrity was nonexistent. *This is why we need MACs.*

15.2 What a MAC Is, Formally

Definition 15.1 (Message authentication code)**.** A MAC is a triple of algorithms $(\mathrm{Gen}, \mathrm{MAC}, \mathrm{Vrfy})$:

- $\mathrm{Gen}(1^n)$ outputs a secret key k, typically $k \xleftarrow{\$} \{0,1\}^n$.
- $\mathrm{MAC}_k(m)$ takes the key and a message $m \in \{0,1\}^*$ and outputs a *tag* t. We write $t \leftarrow \mathrm{MAC}_k(m)$.

- $\mathrm{Vrfy}_k(m, t)$ outputs 1 (accept) or 0 (reject).

Correctness: for every key and message, $\mathrm{Vrfy}_k(m, \mathrm{MAC}_k(m)) = 1$.

When MAC is deterministic – which the constructions in this chapter all are – verification is the obvious thing: recompute the tag and compare. $\mathrm{Vrfy}_k(m, t)$ recomputes $t' = \mathrm{MAC}_k(m)$ and accepts iff $t' = t$. (We will see in Section 15.8 that "compare" hides a trap.)

Alice and Bob share k ahead of time. To send m, Alice transmits (m, t) with $t = \mathrm{MAC}_k(m)$. Bob runs $\mathrm{Vrfy}_k(m, t)$; if it returns 1 he trusts the message, otherwise he discards it. The tag travels *in the clear* alongside the message – a MAC is not encryption and does not try to hide anything. Its only job is to be unforgeable.

Focus

A MAC tag is short – 128 or 256 bits is typical – no matter how long the message is. That is the same compression you saw in hash functions (Chapter 14): a fingerprint. The difference is the key. A hash $\mathrm{H}(m)$ is public; anyone can compute it, so anyone can forge a "valid" hash for a forged message. A MAC $\mathrm{MAC}_k(m)$ folds in a secret, so only key-holders can produce valid tags. A MAC is, very roughly, a *keyed* fingerprint.

15.3 What "Secure" Means: Unforgeability

Now define the threat. We must be careful, because the obvious-sounding definitions are too weak.

The right notion is **existential unforgeability under an adaptive chosen-message attack**, abbreviated EUF-CMA. Each part of the name has a specific meaning:

- *Chosen-message attack:* the adversary may ask for tags on any messages she likes. We model this by giving her a *MAC oracle* – a black box that, on input m, returns $\mathrm{MAC}_k(m)$ – without revealing k. "Adaptive" means she picks each query after seeing previous answers.
- *Existential forgery:* she wins by producing *any* new valid message–tag pair (m^*, t^*) – "existential" because it can be any message she manages to forge, not a specific target. The only restriction: m^* must be a message she never queried.

Formal

The forgery game $\mathsf{MacForge}_{\mathsf{Adv},\Pi}(n)$. For a MAC scheme $\Pi = (\mathrm{Gen}, \mathrm{MAC}, \mathrm{Vrfy})$ and adversary Adv:

1. Run $k \leftarrow \mathrm{Gen}(1^n)$.
2. Adv is given oracle access to $\mathrm{MAC}_k(\cdot)$. Let Q be the set of messages she queries.
3. Adv outputs (m^*, t^*).
4. The game outputs 1 (adversary wins) iff $\mathrm{Vrfy}_k(m^*, t^*) = 1$ *and* $m^* \notin Q$.

Π is **EUF-CMA secure** if for every PPT adversary Adv,

$$\Pr\big[\mathsf{MacForge}_{\mathsf{Adv},\Pi}(n) = 1\big] \;\le\; \mathsf{negl}(n).$$

Figure 15.1 summarizes the interaction: the adversary may query the MAC oracle adaptively on as many chosen messages $m_1, \ldots, m_q$ as she likes, and wins if she can output a valid tag t^* on any fresh message $m^* \notin \{m_1, \ldots, m_q\}$.

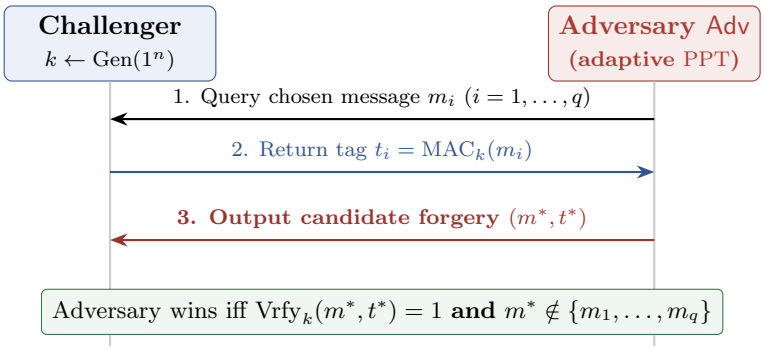


Figure 15.1: The EUF-CMA security game $\mathsf{MacForge}_{\mathsf{Adv},\Pi}(n)$. Even after collecting valid tags on adaptively chosen messages $m_1, \ldots, m_q$, no efficient adversary can forge a valid tag t^* on a new message m^*.

Why such a strong definition? Because real attackers have exactly these capabilities.

Why it works

Why adaptive chosen-message? In practice the attacker often *can* get the system to MAC messages of her choosing. A login server returns a MAC'd session cookie for any username she registers; a payment terminal MACs whatever transaction she types in. If a MAC were secure only against attackers who passively eavesdrop, it would fail the moment the attacker could nudge the system into signing things. We assume the worst: she gets tags on everything she asks for.

Why existential (any new message)? A MAC's job is to make *every* forgery hard, not just forgeries of "important-looking" messages. If an attacker could forge a tag for even one nonsense message she never queried, the scheme is broken – because that nonsense might be a control command, a length field, or a building block for a bigger attack. Demanding that no new valid pair exists is the cleanest, strongest bar, and it is the one cryptographers hold MACs to.

Common trap

EUF-CMA does *not* promise protection against **replay**. If Alice sends (m, t) and Eve records it, Eve can resend the same pair later; the tag still verifies because m is unchanged. The MAC did its job – the message is authentic and intact – it is just *old*. Defeating replay is a separate, protocol-level concern: include a counter, a timestamp, or a nonce *inside* the MAC'd message, so each authenticated message is unique. TLS, IPsec, and Signal all do exactly this. Never assume "it has a MAC" means "it cannot be replayed."

15.3.1 A pseudorandom function is already a MAC

We start with a simple theoretical fact. Recall a pseudorandom function (PRF) is a keyed function F_k that no efficient adversary can distinguish from a truly random function, even after querying it

on chosen inputs. If you have a good PRF, you already have a good MAC:

$$\mathrm{MAC}_k(m) = F_k(m), \qquad \mathrm{Vrfy}_k(m, t) = [\, F_k(m) \stackrel{?}{=} t \,].$$

Why it works

Suppose F_k were a *truly* random function with ℓ-bit outputs. The adversary queries it on messages $m_1, \ldots, m_q$ and learns those outputs. Now she must guess $t^* = F(m^*)$ for a *fresh* m^*. But a random function's value on a point she has never seen is uniformly random and independent of everything she has seen. Her best guess is a blind one: she wins with probability exactly $2^{-\ell}$, negligible for ℓ like 128.

A real PRF is computationally indistinguishable from this ideal, so the adversary's forgery probability against F_k can exceed $2^{-\ell}$ only by a negligible amount – otherwise her forgeries would themselves be a distinguisher. Hence *any secure PRF is a secure fixed-length MAC.* AES, viewed as a PRF, is a MAC for 128-bit messages. The engineering problem is extending this to messages of *arbitrary* length without losing security – which is what CBC-MAC and HMAC solve.

15.4 CBC-MAC: A MAC From a Block Cipher

We have a block cipher E_k (think AES) that maps one block to one block and behaves like a PRF. How do we authenticate a message of many blocks? The idea: chain the blocks through the cipher exactly like CBC-mode encryption (Chapter 13), but throw away every intermediate ciphertext and keep only the last block. That final block depends on every input block, so it is a fingerprint of the whole message.

Focus

Raw CBC-MAC. Split the message into blocks $m_1, m_2, \ldots, m_\ell$ (each one cipher-block wide). Set $t_0 = 0$ (the all-zero block – note: *no random IV*, unlike CBC encryption). For $i = 1, \ldots, \ell$:

$$t_i = E_k\big(t_{i-1} \oplus m_i\big).$$

Output the tag $t = t_\ell$.

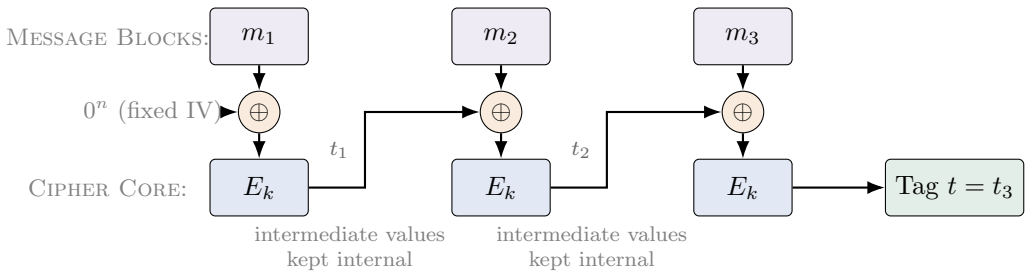


Figure 15.2: Raw CBC-MAC chaining over three message blocks. All intermediate cipher outputs are discarded; only the final block t_ℓ is emitted as the authentication tag.

Theorem 15.1 (Security of raw CBC-MAC, informal)**.** *If E_k is a secure PRF and we restrict ourselves to messages of a single* fixed *length ℓ blocks, then raw CBC-MAC is an EUF-CMA-secure MAC.*

The "fixed length" condition is essential. If messages may vary in length, raw CBC-MAC can be forged.

15.4.1 Working it by hand with a 4-bit toy cipher

Let us trace CBC-MAC on numbers small enough to do in your head. Take a block size of 4 bits, so blocks are integers 0–15. We need a "cipher" E_k; for illustration use the fixed permutation of the nibbles

$$E(x) = (7x + 3) \bmod 16,$$

which is a bijection on $\{0, \dots, 15\}$ because $\gcd(7, 16) = 1$. (A real cipher is far from affine; this is only to make the chaining visible.) Authenticate the two-block message $m_1 = 5$, $m_2 = 9$.

Walking question

$t_0 = 0$.
Block 1: $t_0 \oplus m_1 = 0 \oplus 5 = 5$. Then $t_1 = E(5) = (35 + 3) \bmod 16 = 38 \bmod 16 = 6$.
Block 2: $t_1 \oplus m_2 = 6 \oplus 9$. In binary $0110 \oplus 1001 = 1111 = 15$. Then $t_2 = E(15) = (105 + 3) \bmod 16 = 108 \bmod 16 = 12$.
Tag $= t_2 = 12$.

Flip a single bit of the message and watch the tag change completely. Take $m_1 = 4$ instead of 5 (one bit different), $m_2 = 9$ unchanged. Now $t_0 \oplus m_1 = 4$, $t_1 = E(4) = 31 \bmod 16 = 15$; then $15 \oplus 9 = 0110 = 6$, $t_2 = E(6) = 45 \bmod 16 = 13$. The tag jumped from 12 to 13, and with a real cipher the change would be unpredictable and total. That avalanche is what makes the tag a fingerprint.

15.4.2 The length-extension forgery on raw CBC-MAC

Attack

Forging variable-length CBC-MAC. Suppose the system uses raw CBC-MAC on messages of *any* length, with the fixed 0 start. The attacker requests the tag on a one-block message m and gets back

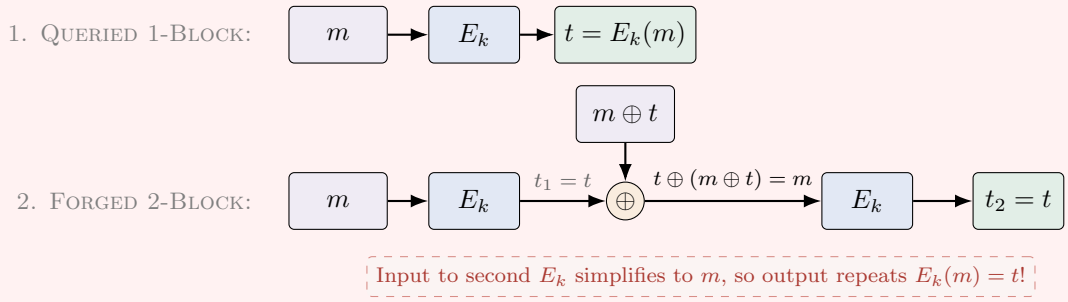
$$t = E_k(m).$$

Now she claims a valid tag for the *two-block* message $M = m \,\|\, (m \oplus t)$ – and the tag is the very same t she already holds. Check it by running CBC-MAC on M:

$$t_1 = E_k(0 \oplus m) = E_k(m) = t,$$

$$t_2 = E_k(t_1 \oplus (m \oplus t)) = E_k(t \oplus m \oplus t) = E_k(m) = t.$$

The second block's XOR cancels t_1 back down to m, so $t_2 = t$. She has produced a valid tag for a message she never queried. **Existential forgery: the scheme is completely broken for variable lengths.**



The root cause: with a fixed start and no length binding, the attacker can splice the known output of one computation into the input of the next. This is the same length-extension pattern that breaks naive secret-prefix hash MACs in Section 15.5.

15.4.3 Fixing CBC-MAC

There are two standard repairs, both of which break the splicing trick by making the computation depend on the length.

- **Prepend the length.** Make the *first* block encode the message length in blocks, then run CBC-MAC over $\langle \ell \rangle \,\|\, m_1 \,\|\, \cdots \,\|\, m_\ell$. Now a message of a different length starts from a different first block, so spliced tags do not line up. (Appending the length does *not* work – the attacker controls the last block and can compensate.)
- **Encrypt the final block under a second key (EMAC / ECBC-MAC).** Compute raw CBC-MAC to get t_ℓ, then output $t = E_{k_2}(t_\ell)$ with an independent key k_2. The extra encryption hides the intermediate chaining value the attacker needs, so she can no longer use a returned tag as an internal state. This is the basis of standardized schemes like CMAC (NIST SP 800-38B), which folds the two keys into one and adds clean handling of partial final blocks.

In the real world

You meet CBC-MAC's descendants more often than the raw version. **CMAC** (also called OMAC) is the modern, length-safe block-cipher MAC; it underlies the integrity check in some IPsec and storage-encryption deployments. **CCM mode** (used in WPA2 Wi-Fi as AES-CCMP, Chapter 26) is literally "CTR encryption + CBC-MAC for authentication," combined carefully so the two parts share a cipher key safely. So while you should never reach for raw CBC-MAC yourself, its chaining idea still protects WPA2 Wi-Fi traffic.

15.5 Why You Cannot Just MAC With a Hash

Hash functions (Chapter 14) already compress arbitrary messages to short digests and are built to be collision-resistant. The naive instinct is to make a MAC by hashing the key together with the message:

$$\mathrm{MAC}_k(m) \stackrel{?}{=} \mathrm{H}(k \,\|\, m).$$

This is called the *secret-prefix* MAC, and for the hash functions everyone actually uses – MD5, SHA-1, SHA-256, all built on the Merkle–Damgård construction – it is **broken**. The reason is *length extension*.

Intuition

A Merkle–Damgård hash processes the message block by block, carrying a running "state." **The final state it outputs as the digest is identical to the internal state it would feed into the next block.** If you know $\mathrm{H}(x)$, you know exactly where the compression chain paused after consuming x – and you can pick up where it left off and feed in additional blocks, computing $\mathrm{H}(x \,\|\, \texttt{pad} \,\|\, y)$ for any suffix y of your choice *without knowing* x. The digest functions as a public continuation token.

Attack

Length-extension forgery on secret-prefix MAC. The server authenticates with $t = \mathrm{H}(k \,\|\, m)$ using SHA-256. An attacker observes a valid pair (m, t) for some message m – say

$$m = \texttt{"amount=100\&to=alice"},$$

and she also knows (or guesses) the key length $|k|$.
A Merkle–Damgård hash internally pads its input to a block boundary with a known `pad` (a 1 bit, zeros, then the bit-length). So t is actually the hash state after consuming

$$k \,\|\, m \,\|\, \texttt{pad}_1,$$

where $\texttt{pad}_1$ is the padding for that length. The attacker *sets her hash's internal state to* t and feeds in any suffix y she likes, for example

$$y = \texttt{"\&to=eve\&admin=true"}.$$

Out comes a valid tag t' for the extended message

$$m' = m \,\|\, \texttt{pad}_1 \,\|\, y,$$

because $t' = \mathrm{H}(k \,\|\, m')$ is exactly what the machine computes – and she produced it *without ever knowing* k. She has forged a valid tag on an extended message that the server will accept as authentic. The secret-prefix construction fails because the public digest acts as an unrestricted continuation token.

Common trap

This attack has been used in practice. The classic "Flickr API signature" break and a string of web-API authentication bugs in the 2000s were exactly length-extension attacks on $\mathrm{H}(\text{secret} \,\|\, \text{message})$. Whenever you see homegrown authentication that hashes a secret in front of the data with SHA-1 or SHA-256, assume it is forgeable. Use HMAC.

15.5.1 The length extension, worked on a toy hash

The attack can seem counterintuitive until you trace the internal state step by step. Build a cartoon Merkle–Damgård hash whose "state" is one number mod 251 and whose compression function folds in one message byte at a time:

$$s_0 = 0, \qquad s_i = C(s_{i-1}, x_i) = \big(31\, s_{i-1} + x_i\big) \bmod 251.$$

After the last byte, the hash outputs the final state as the digest – and that is the vulnerability: *the digest is the live state*, ready to absorb more bytes. To keep the arithmetic tiny we use a one-byte "padding" rule: a real Merkle–Damgård hash appends a `0x80` byte and a length encoding, so we append the single pad byte $P = 128$ and pretend that finishes the block.

Let the secret be $k = [9]$ (one byte the attacker never learns) and the visible message be $m = [2, 5]$.

Walking question

The legitimate tag. The server computes $t = \mathrm{H}(k \,\|\, m)$ over the padded stream $[9, 2, 5, P]$ with $P = 128$:

$$\begin{aligned} s_1 &= (31 \cdot 0 + 9) \bmod 251 = 9, \\ s_2 &= (31 \cdot 9 + 2) \bmod 251 = 281 \bmod 251 = 30, \\ s_3 &= (31 \cdot 30 + 5) \bmod 251 = 935 \bmod 251 = 935 - 753 = 182, \\ s_4 &= (31 \cdot 182 + 128) \bmod 251 = 5770 \bmod 251. \end{aligned}$$

For the last step, $251 \cdot 22 = 5522$ and $5770 - 5522 = 248$. So $t = s_4 = 248$. The attacker sees the pair $(m, t) = ([2, 5],\ 248)$ and knows $|k| = 1$.

Walking question

The forgery, without k. The attacker wants a tag for a message that continues past the padding. She picks a suffix byte $y = [7]$ and *sets her hash state to the published digest*, $s \leftarrow t = 248$, then resumes the same recurrence:

$$s' = (31 \cdot 248 + 7) \bmod 251 = 7695 \bmod 251.$$

Since $251 \cdot 30 = 7530$, we get $s' = 7695 - 7530 = 165$. She then appends *her own* padding byte $P = 128$ to close the block:

$$t' = (31 \cdot 165 + 128) \bmod 251 = 5243 \bmod 251.$$

Since $251 \cdot 20 = 5020$, $t' = 5243 - 5020 = 223$. She claims that $t' = 223$ is a valid tag for the forged message

$$m' = m \,\|\, [P] \,\|\, y = [2, 5, 128, 7].$$

Walking question

Does the server agree? The server, holding $k = [9]$, computes $\mathrm{H}(k \,\|\, m')$ over $[9, 2, 5, 128, 7]$ plus its own final pad $[128]$. The first four absorptions are identical to the legitimate run,

landing at $s_4 = 248$ – exactly the digest the attacker resumed from. Continuing:

$$s_5 = (31 \cdot 248 + 7) \bmod 251 = 165, \qquad s_6 = (31 \cdot 165 + 128) \bmod 251 = 223.$$

The server's tag is 223, matching the attacker's t'. The forgery verifies, and she never touched k. The leak was structural: publishing s_4 gave her the state from which to resume the computation.

HMAC removes exactly this leak. If this same toy hash were wrapped as $\mathrm{H}\big((k_0 \oplus \texttt{opad}) \,\|\, \mathrm{H}((k_0 \oplus \texttt{ipad}) \,\|\, m)\big)$, the value the attacker observes is the *outer* state, while the resumable inner state s_4 is never published. With no state to resume from, the forgery in the second step above fails.

15.5.2 Does secret-suffix help? Partly, and not enough

You might try $\mathrm{H}(m \,\|\, k)$ instead. Length extension no longer applies (the secret is at the end). But now the security leans entirely on collision resistance of H: if an attacker finds any two messages $m_1 \neq m_2$ with $\mathrm{H}(m_1) = \mathrm{H}(m_2)$ – which the birthday bound (Chapters 7 and 14) makes cheaper than you would hope, and which is feasible for MD5 and SHA-1 today – then $\mathrm{H}(m_1 \,\|\, k) = \mathrm{H}(m_2 \,\|\, k)$, so the tag for m_1 is also valid for m_2. That is a forgery. We want a construction that survives even when the underlying hash has weaknesses. That is HMAC.

15.6 HMAC: The Right Way to MAC With a Hash

HMAC (Hash-based MAC) wraps an ordinary hash in a specific nested pattern that kills length extension and provably reduces security to mild assumptions about the hash's compression function. It was designed by Bellare, Canetti, and Krawczyk in 1996, standardized as RFC 2104 and FIPS 198, and it is everywhere.

History

Before HMAC, every protocol designer improvised their own "hash the key with the message" scheme, such as $\mathrm{H}(k \,\|\, m)$ or $\mathrm{H}(m \,\|\, k)$, and most of them were breakable by length extension or collisions, as above. In 1996, Mihir Bellare, Ran Canetti, and Hugo Krawczyk introduced HMAC (RFC 2104, FIPS 198). Their contribution was twofold: a *fixed, simple* recipe that anyone could implement on top of an unmodified hash library, and a *proof* that the recipe is a secure MAC as long as the hash's underlying compression function is a decent PRF – a far weaker assumption than full collision resistance. When Xiaoyun Wang broke MD5 collisions in 2004, many feared HMAC-MD5 was dead, until Bellare proved in 2006 that collision resistance is unnecessary for HMAC security. That is why HMAC-MD5 and HMAC-SHA-1 stayed trustworthy for authentication years after their underlying hashes' collision resistance fell apart.

Focus

The HMAC construction. Let H be a hash with block size B bytes (for SHA-256, $B = 64$) and output size L bytes. Define two fixed pad constants, each B bytes:

$$\texttt{ipad} = \texttt{0x36} \text{ repeated}, \qquad \texttt{opad} = \texttt{0x5c} \text{ repeated}.$$

Process the key k into a B-byte block k_0: if k is longer than B bytes, replace it with $\mathrm{H}(k)$; then right-pad with zero bytes to length B. Then

$$\mathrm{HMAC}_k(m) \;=\; \mathrm{H}\Big((k_0 \oplus \texttt{opad}) \;\|\; \mathrm{H}\big((k_0 \oplus \texttt{ipad}) \,\|\, m\big)\Big).$$

Read that from the inside out. The *inner* hash $\mathrm{H}\big((k_0 \oplus \texttt{ipad}) \,\|\, m\big)$ is essentially a secret-prefix MAC of m – and we just said that is length-extension-vulnerable. The *outer* hash then hashes the key again over that inner digest. The outer hash is what fixes this: it takes the leaky inner digest and runs it through one more keyed compression, so what the attacker sees is no longer a usable "resume point."

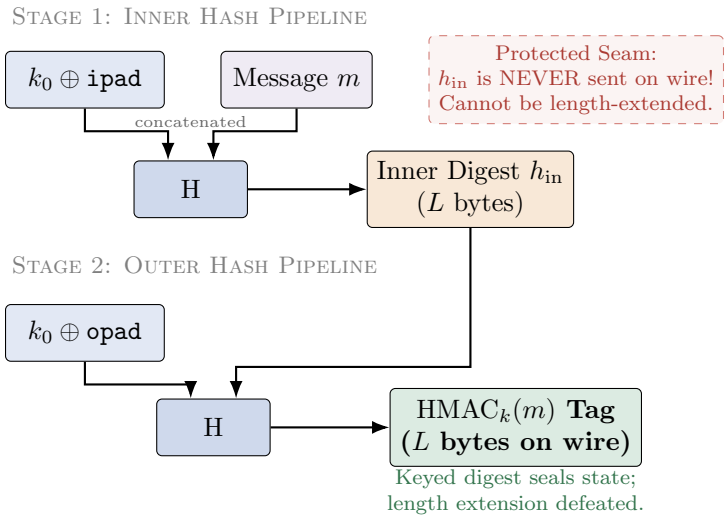


Figure 15.3: The nested HMAC pipeline. The inner hash processes the padded key $k_0 \oplus \texttt{ipad}$ followed by arbitrary message blocks. Its intermediate digest h_{in} is absorbed by the outer hash together with $k_0 \oplus \texttt{opad}$, hiding the inner continuation state from any eavesdropper.

Why it works

Why the nesting defeats length extension. The dangerous object is the inner digest $h = \mathrm{H}\big((k_0 \oplus \texttt{ipad}) \,\|\, m\big)$, because an attacker who saw *it* could extend it. But the attacker never sees h. What she sees is the *outer* value $\mathrm{H}\big((k_0 \oplus \texttt{opad}) \,\|\, h\big)$. To length-extend *that*, she would need to know its preimage state – but that requires knowing h and the secret $k_0 \oplus \texttt{opad}$, which she does not. The inner digest stays hidden inside the outer hash.

Why the security proof needs so little. Bellare–Canetti–Krawczyk show HMAC is a secure MAC provided the hash's *compression function* is a PRF when keyed through its chaining input – and provided the inner hash is weakly collision-resistant for the queried

messages. Both are much milder than demanding the full hash be collision-resistant. That gap is exactly why HMAC-SHA-1 remained safe for MACs after SHA-1 collisions became practical: those collisions attack the full hash, not the keyed compression function HMAC relies on.

Why two different pads? `ipad` and `opad` differ in many bit positions (0x36 vs 0x5c), so $k_0 \oplus$ `ipad` and $k_0 \oplus$ `opad` behave like two effectively independent keys derived from one k. The proof wants the inner and outer hashes keyed differently; the two constants buy that for free, with no extra key material.

15.6.1 A tiny HMAC by hand

The real HMAC uses 64-byte blocks; that is too big to trace on paper. So shrink everything to a cartoon "hash" over single bytes to expose the *shape*, not the cryptography. Let block size $B = 1$ byte, and let the toy hash be $h(x) = \left(\sum_i x_i \cdot 31^i\right) \bmod 251$ over the byte sequence x (251 is prime). Take pads `ipad` $=$ `0x36` $= 54$, `opad` $=$ `0x5c` $= 92$, key $k = 10$ (already one byte, so $k_0 = 10$), message $m = [7]$ (one byte).

Walking question

$k_0 \oplus$ `ipad` $= 10 \oplus 54$. In binary $00001010 \oplus 00110110 = 00111100 = 60$.
Inner input: $[60,\ 7]$. Inner digest:

$$h = (60 \cdot 31^0 + 7 \cdot 31^1) \bmod 251 = (60 + 217) \bmod 251 = 277 \bmod 251 = 26.$$

$k_0 \oplus$ `opad` $= 10 \oplus 92$: $00001010 \oplus 01011100 = 01010110 = 86$.
Outer input: $[86,\ 26]$. Tag:

$$\text{HMAC} = (86 + 26 \cdot 31) \bmod 251 = (86 + 806) \bmod 251 = 892 \bmod 251 = 892 - 753 = 139.$$

So $\text{HMAC}_{10}([7]) = 139$ under our toy hash. The numbers are meaningless cryptographically – what matters is the sequence of steps: XOR the key with `ipad`, hash with the message, XOR the key with `opad`, hash again over the inner result. Change k, the inner key 60 changes, the inner digest changes, the tag changes. Change m, the inner digest changes, the tag changes. An attacker who lacks k cannot reproduce either keyed step.

Implement it

A spelled-out HMAC over a real hash is short. This builds HMAC on Python's `hashlib` from the block-level operations, so you can see every step. **Learning only – never ship this. Use `hmac.new(key, msg, hashlib.sha256)` from the standard library, which is constant-time-verified and audited.**

```
import hashlib

def hmac_sha256(key: bytes, msg: bytes) -> bytes:
    B = 64                          # SHA-256 block size in bytes
    if len(key) > B:                # long keys get hashed down first
        key = hashlib.sha256(key).digest()
```

```
    k0 = key + b"\x00" * (B - len(key))   # right-pad to B bytes

    ipad = bytes(b ^ 0x36 for b in k0)
    opad = bytes(b ^ 0x5c for b in k0)

    inner = hashlib.sha256(ipad + msg).digest()
    return hashlib.sha256(opad + inner).digest()

# Sanity check against the standard library
import hmac
key, msg = b"my secret key", b"transfer 100 to bob"
mine = hmac_sha256(key, msg)
ref  = hmac.new(key, msg, hashlib.sha256).digest()
assert mine == ref, "mismatch!"
print(mine.hex())
```

Notice there is no length field anywhere in our code – HMAC needs none, because the nesting, not a length prefix, is what stops extension.

Connection

The forgery probability for a PRF-based MAC bottoms out at $2^{-\ell}$ for an ℓ-bit tag (Section 15.3.1), which is just the chance of guessing one fixed random value – a direct application of the uniform-and-independent reasoning from the probability companion. The collision-based attacks on secret-suffix MACs rely on the *birthday bound*: a hash with L-bit output starts colliding after about $2^{L/2}$ messages, so its collision-based forgery resistance is only $2^{L/2}$, not 2^{L}. HMAC's proof sidesteps this by not requiring full collision resistance, which is why its effective security tracks the 2^{ℓ} guessing bound rather than the weaker birthday bound. The cost of one HMAC is two compression-function passes plus the message, i.e. essentially the cost of one hash – see the modular-exponentiation and hashing cost analyses in the algorithms companion.

15.7 Combining Encryption and Authentication

Most real traffic needs *both* confidentiality and integrity. You have a cipher and you have a MAC; how do you bolt them together? There are three obvious orders (Figure 15.4), and the choice matters: two of them have led to real, exploited vulnerabilities. Let Enc be a CPA-secure cipher with key k_E and MAC a secure MAC with an *independent* key k_M.

- **Encrypt-and-MAC (E&M):** send $c = \mathrm{Enc}_{k_E}(m)$ and $t = \mathrm{MAC}_{k_M}(m)$. The tag is computed over the *plaintext* and sent alongside the ciphertext.
- **MAC-then-Encrypt (MtE):** compute $t = \mathrm{MAC}_{k_M}(m)$, then send $c = \mathrm{Enc}_{k_E}(m \,\|\, t)$. The tag is buried inside the ciphertext.
- **Encrypt-then-MAC (EtM):** compute $c = \mathrm{Enc}_{k_E}(m)$, then $t = \mathrm{MAC}_{k_M}(c)$, and send (c, t). The tag is computed over the *ciphertext*.

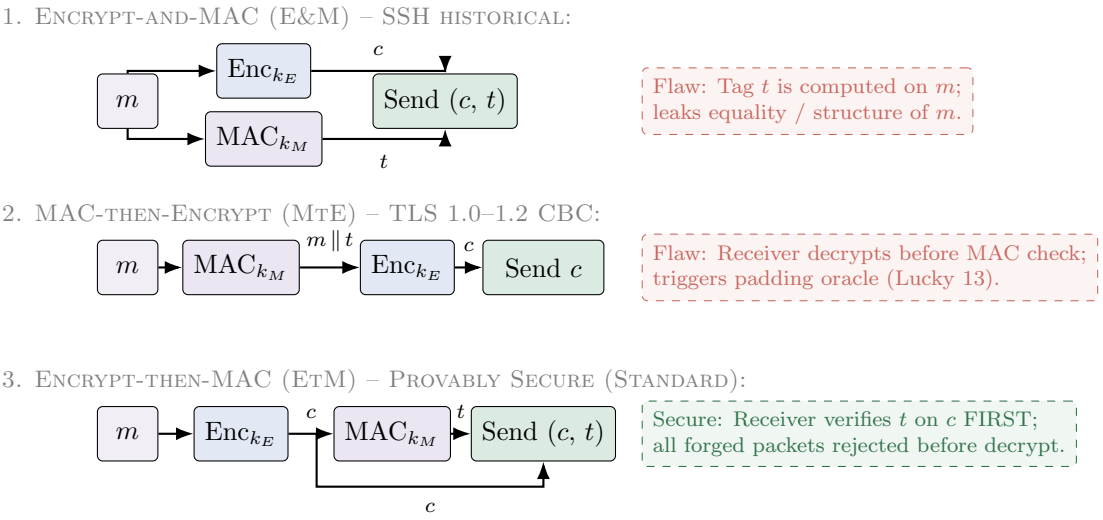


Figure 15.4: The three paradigms for combining symmetric encryption and authentication. Only Encrypt-then-MAC (bottom) guarantees integrity of ciphertexts: invalid packets are rejected by the MAC check before any decryption or padding logic ever runs.

Focus

The conclusion, which the rest of this section justifies: **Encrypt-then-MAC is the one to use.** It is the only one of the three that is provably secure for *any* CPA-secure cipher and *any* secure MAC, with no extra assumptions. The other two can be secure with specific primitives, but they are fragile and have failed catastrophically in deployed protocols.

15.7.1 Why Encrypt-then-MAC wins

Why it works

With EtM, the receiver **checks the MAC on the ciphertext *before decrypting anything***. If the tag does not verify, he discards the packet and never touches the decryption logic. This has two consequences:

1. *Integrity of ciphertexts.* An attacker cannot produce any (c, t) that verifies unless c is one the sender actually transmitted – forging t for a tampered c means forging the MAC, which is hard. So every ciphertext the decryptor ever processes is a genuine one.

2. *No decryption oracle.* Because invalid ciphertexts are rejected at the MAC check, the attacker learns nothing from how decryption *would* have behaved – no padding errors, no parse failures, no timing differences in the cipher. This is precisely what kills the padding-oracle attacks below.

The clean theorem: EtM applied to a CPA-secure cipher and an EUF-CMA MAC yields an *authenticated encryption* scheme secure against chosen-ciphertext attacks. The MAC's strength carries the whole construction, regardless of the cipher's quirks.

15.7.2 Why the other orders are dangerous

Attack

MAC-then-Encrypt and the padding oracle. In MtE, the receiver must *decrypt first* to recover $m \,\|\, t$, then check the tag. With CBC-mode encryption, decryption involves removing padding (Chapter 13) *before* the MAC is even examined. That ordering opens a side channel. A *padding oracle* is any observable difference – an error message, a connection reset, a timing gap – between "decryption produced valid padding" and "invalid padding." Because MtE decrypts and unpads before MACing, a tampered ciphertext can trigger the padding check, and the attacker can watch which response she gets. By submitting carefully mauled ciphertexts and observing valid-vs-invalid padding one byte at a time, she decrypts the entire message *without the key*, at a cost of roughly 128 tries per byte.
This is not hypothetical. **TLS used MAC-then-Encrypt with CBC**, and the result was a series of attacks: the original 2002 Vaudenay padding oracle, then Lucky 13 (2013), which read the padding-check *timing* even after implementers tried to hide the explicit error. Each one let an active attacker recover plaintext such as session cookies. The fixes were painful retrofits. EtM would have made all of them impossible, because the MAC over the ciphertext would have rejected every mauled packet before any padding logic ran.

Attack

Encrypt-and-MAC leaks through the tag. E&M computes $t = \mathrm{MAC}_{k_M}(m)$ over the *plaintext* and sends it in the clear. But a MAC is only required to be *unforgeable*; it is **not** required to hide its input. A perfectly good MAC may be deterministic, so equal plaintexts produce equal tags. An eavesdropper who sees two sessions with the same tag learns the two underlying plaintexts were identical – a confidentiality leak the cipher was supposed to prevent. Worse, some MACs reveal partial structure of the message. So E&M can break CPA-security outright, depending on the MAC. (**SSH** historically used a variant of E&M and had to add countermeasures.) The tag, sitting in the clear over the plaintext, is the leak.

Common trap

Two non-negotiable rules whenever you compose encryption and a MAC:

1. **Use independent keys** $k_E \neq k_M$. Reusing one key across the cipher and the MAC can let the two interact in unexpected, exploitable ways; the security proofs all assume separate keys (derive them from a master key with a KDF or HKDF).

2. **Prefer Encrypt-then-MAC**, and verify the tag in constant time *before* decrypting. Reject early; never let an unauthenticated ciphertext reach your decryption or parsing code.

Better still, do not hand-compose at all: use a vetted *authenticated encryption* mode (AES-GCM, ChaCha20-Poly1305 – Chapter 16) that bakes EtM-style security in and is hard to misuse.

In the real world

Modern protocols converged on the lesson. **TLS 1.3** (2018) threw out CBC-with-MtE entirely and mandates AEAD ciphers (AES-GCM, ChaCha20-Poly1305) that authenticate the ciphertext. **IPsec** ESP uses Encrypt-then-MAC (or AEAD), and **HMAC-SHA-256** is a standard integrity choice there. **Signal** authenticates ciphertexts with HMAC-SHA-256 in its message encryption. The 2014 "Encrypt-then-MAC for TLS" extension (RFC 7366) was a direct retrofit of EtM onto TLS 1.2 to escape Lucky 13. Getting this order right is a large part of why TLS is more robust today than it was a decade ago.

15.8 Verifying Tags in Constant Time

One last trap is small in code but serious in its consequences. Verification compares the received tag t against the recomputed tag t'. The natural way to compare two byte strings – and the way nearly every programming language compares them by default – stops at the *first* differing byte:

```
def insecure_verify(t_received, t_computed):
    return t_received == t_computed   # short-circuits on first mismatch
```

That early exit is a *timing side channel*. Comparing tags that differ in byte 0 returns slightly faster than tags that match through byte 5. An attacker who can measure response time can recover the correct tag one byte at a time.

Attack

Timing a tag, byte by byte. The attacker wants to forge a tag for a chosen message m. She submits (m, g) for a guess g and times the rejection.

1. Fix all bytes of g to zero, then sweep the first byte g_0 through all 256 values. For the correct g_0, the comparison proceeds to byte 1 before failing – measurably *slower*. She keeps the value that took longest.
2. With byte 0 locked in, sweep byte 1 the same way, finding the value that pushes the comparison to byte 2.
3. Repeat down the tag. Each byte costs about 256 timed queries (averaged over noise), so a 32-byte tag falls in roughly $32 \times 256 \approx 8000$ probes instead of the 2^{256} a blind search would need.

The result: an unforgeable MAC with a correct construction is defeated by the `==` operator. The flaw is in the *comparison*, not the cryptography. Real instances of this class – KeyCzar (2009), various API-signature checks – have been found and fixed.

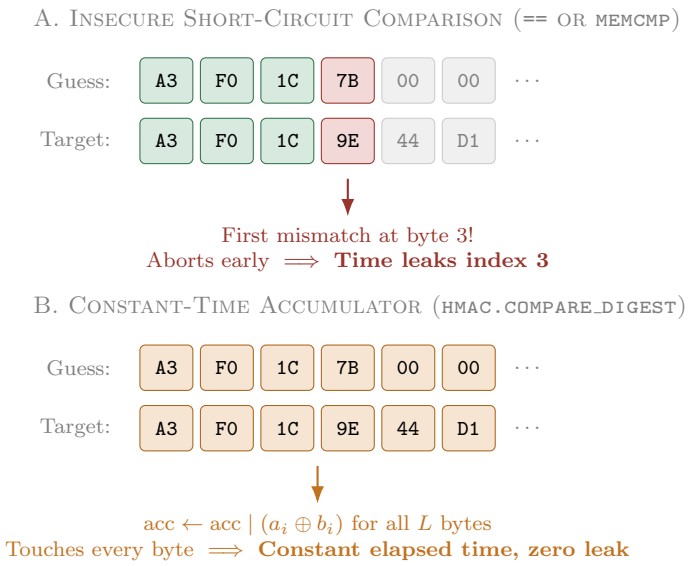


Figure 15.5: Comparison of short-circuit vs constant-time tag verification. A short-circuit comparison aborts on the first byte mismatch, giving an attacker an observable timing difference that reveals how many leading bytes matched. Constant-time verification scans every byte unconditionally.

The defense is a *constant-time comparison*: always examine every byte, and let the running time depend only on the *length* of the tags, never on *where* they differ. Accumulate the differences with XOR and OR, then test the accumulator once at the end.

Implement it

Learning only – in production call `hmac.compare_digest` (Python) or `CRYPTO_memcmp` (OpenSSL), which are written and audited for this.

```python
def constant_time_equal(a: bytes, b: bytes) -> bool:
    # Assumes a and b have the same, non-secret length.
    if len(a) != len(b):
        return False
    acc = 0
    for x, y in zip(a, b):
        acc |= x ^ y          # any differing bit sets a 1 in acc, forever
    return acc == 0           # one branch, after touching every byte

# The real thing:
import hmac
assert hmac.compare_digest(b"abc", b"abc")
```

The loop runs the same number of iterations and does the same work whether the tags match or differ in byte 0 or byte 31. There is exactly one branch, at the very end, on the fully-mixed accumulator – no information about *where* a mismatch occurred leaks out.

Connection

This is a first taste of *side-channel attacks* (Chapter 37): the secret leaks not through the math but through *how long* or *how much power* the computation takes. The discipline it teaches – make running time independent of secret data – recurs throughout real crypto engineering: constant-time modular exponentiation for RSA, constant-time table lookups in AES software, constant-time conditional swaps in elliptic-curve ladders. The probability companion's view of an attacker accumulating a statistical signal over many noisy timing samples is exactly the model that makes the byte-by-byte attack work despite measurement noise.

15.9 Putting It Together

A MAC is the symmetric-key tool for integrity and authenticity: a keyed fingerprint $\mathrm{MAC}_k(m)$ that travels in the clear and lets a key-holder detect any tampering. The bar it must clear is EUF-CMA – no efficient adversary, even one who collects tags on messages of her choosing, can produce a valid tag for a fresh message except with negligible probability. Any secure PRF is already such a MAC for fixed-length inputs; the work is extending to arbitrary lengths.

Two constructions do the extending. CBC-MAC chains a block cipher and is secure for fixed-length messages, but raw CBC-MAC is forgeable across lengths by splicing, so real systems use length-prefixed or final-encrypted variants like CMAC. HMAC nests a hash twice – $\mathrm{H}((k_0 \oplus \texttt{opad}) \,\|\, \mathrm{H}((k_0 \oplus \texttt{ipad}) \,\|\, m))$ – which defeats the length-extension attack that destroys the naive $\mathrm{H}(k \,\|\, m)$ MAC, and which provably needs only mild assumptions on the hash. When you need secrecy too, compose with *Encrypt-then-MAC* and independent keys, never MAC-then-Encrypt (padding oracles, Lucky 13) or Encrypt-and-MAC (tag leakage). And whatever you build, compare tags in constant time, or the `==` operator will hand an attacker your tag one byte at a time. The next chapter folds all of this into a single primitive – authenticated encryption – so you never have to compose by hand again.

Exercises

Exercise 15.1 (Malleability, concretely)**.** Alice encrypts the 8-byte ASCII message `"send $05"` with a stream cipher, producing ciphertext c. Eve knows the format and wants the bank to read `"send $98"`. The last two plaintext bytes are `'0'` (0x30) and `'5'` (0x35); she wants `'9'` (0x39) and `'8'` (0x38). Compute the two XOR masks Eve must apply to the last two ciphertext bytes, and explain in one sentence why she needs no knowledge of the keystream.

Exercise 15.2 (Trace CBC-MAC)**.** Using the toy cipher $E(x) = (7x+3) \bmod 16$ from Section 15.4.1 and start value $t_0 = 0$, compute the raw CBC-MAC tag of the three-block message $m_1 = 2$, $m_2 = 11$, $m_3 = 4$. Show t_1, t_2, t_3.

Exercise 15.3 (Break variable-length CBC-MAC)**.** You can query a raw CBC-MAC oracle (fixed 0 start, the cipher from Section 15.4.1, no length binding). Query the one-block message $m = 6$ to get its tag t. Using the splicing attack of Section 15.4.2, write down a two-block message you have never queried whose tag is also t, and verify by tracing CBC-MAC on it.

Exercise 15.4 (Length extension on paper)**.** Explain, in terms of Merkle–Damgård internal state, why an attacker who knows $\mathrm{H}(k \,\|\, m)$ and $|k|$ can compute a valid secret-prefix tag for a longer message but *cannot* do so against HMAC. Identify the single structural feature of HMAC that blocks her.

Exercise 15.5 (Toy HMAC)**.** With the toy hash $h(x) = \left(\sum_i x_i \cdot 31^i\right) \bmod 251$, block size $B = 1$, `ipad` $= 54$, `opad` $= 92$ from Section 15.6.1, compute $\mathrm{HMAC}_k(m)$ for key $k = 20$ and one-byte message $m = [3]$. Show the inner key, the inner digest, the outer key, and the final tag.

Exercise 15.6 (Why two pads?)**.** Suppose someone defines a "simplified HMAC" that uses the *same* pad for inner and outer: $\mathrm{H}((k_0 \oplus \mathtt{p}) \,\|\, \mathrm{H}((k_0 \oplus \mathtt{p}) \,\|\, m))$. Argue why having the inner and outer hashes keyed by the *same* value is undesirable, and what property of `ipad` $\neq$ `opad` the standard relies on.

Exercise 15.7 (Order matters)**.** For each of MAC-then-Encrypt, Encrypt-and-MAC, and Encrypt-then-MAC, state in one sentence the concrete failure mode discussed in this chapter (padding oracle / tag leakage / none), and name the real protocol associated with each lesson.

Exercise 15.8 (Replay is not forgery)**.** Eve records a valid pair (m, t) from a session and resends it an hour later; the receiver accepts it again. Explain why this does *not* contradict EUF-CMA security, and describe a concrete change to what gets MAC'd that defeats the replay.

Exercise 15.9 (Timing attack budget)**.** A server uses a non-constant-time `==` to compare 16-byte HMAC tags. Estimate the number of timed queries an attacker needs to recover a full valid tag for a chosen message via the byte-by-byte attack of Section 15.8, and compare it to the 2^{128} cost of a blind forgery. By what factor does the timing leak reduce the work?

Exercise 15.10 (Constant-time proof obligation)**.** The `constant_time_equal` routine in Section 15.8 early-returns `False` when the two inputs have different lengths. Explain why this length check does *not* leak secret information in the HMAC verification setting (hint: who controls the tag length, and is it secret?), and give one situation where leaking a length *would* be dangerous.

Exercise 15.11 (PRF implies MAC)**.** Let F_k be a secure PRF with 128-bit outputs, used directly as a fixed-length MAC $\mathrm{MAC}_k(m) = F_k(m)$. An adversary makes q oracle queries on distinct messages, then outputs a forgery (m^*, t^*) with m^* fresh. Bound her success probability, and explain why it does not grow with q (beyond the PRF's own distinguishing advantage).

Chapter 16

Authenticated Encryption

Up to now we have built two kinds of machine and kept them in separate rooms. One kind hides a message: stream ciphers, block ciphers, the modes that turn a block cipher into something that encrypts arbitrary data. The other kind detects tampering: hash functions, and the keyed message authentication codes built on top of them. Encryption protects *confidentiality*; a MAC protects *integrity* and *data-origin authentication*. The natural next question is the one a working engineer actually faces: I have a channel, I have a key, I want to send a message that nobody can read and nobody can secretly alter. Which machine do I use?

The answer, almost always, is "both, and you must combine them in exactly the right way, or you get neither." That last clause is what this chapter is about. Combining encryption and authentication looks like it should be trivial – encrypt, then also MAC, ship both – but it is not. There are three obvious ways to glue the two primitives together, two of them are broken in general, and the broken ones were the popular choices for years. Real protocols suffered years of practical exploits from getting this wrong. The modern resolution is a single primitive, *authenticated encryption with associated data* (AEAD), that does both jobs in one pass with one key and one security definition, and refuses to hand you a fragile pile of parts to assemble yourself.

We will start with why you almost never want confidentiality without integrity, watch a concrete attack on "encryption alone," state the AEAD definition and the associated-data idea, then dissect the two AEAD schemes that carry essentially all of today's secure traffic: AES-GCM (counter mode stapled to a polynomial hash called GHASH) and ChaCha20-Poly1305. GCM fails badly if a nonce is ever reused: a single repeat exposes the hash key and allows forgeries. We will carry out that attack with small numbers so you can see exactly why it happens. By the end you should reach for an AEAD by reflex, know which one and why, and never again write "encrypt then maybe checksum" in a real system.

16.1 Why Confidentiality Without Integrity Is a Trap

Beginners carry a stubborn intuition: if Eve cannot *read* my ciphertext, surely she cannot *change* it into anything meaningful either. The ciphertext looks like random noise; how could an attacker who cannot decode it nudge it toward a chosen plaintext? The intuition is wrong, and the reason lies in the structure of the fast, simple ciphers we have already built.

Recall the one-time pad and every stream cipher built in its image (Chapter 10). Encryption is $c = m \oplus k$ where k is the keystream. This is simple and perfectly confidential when k is used once. But look at what an attacker who never learns k can still do. Suppose Eve knows – or guesses – that the plaintext at some position is a particular value, or she just wants to flip specific bits. She picks a difference Δ and replaces c with $c' = c \oplus \Delta$. Bob decrypts:

$$m' = c' \oplus k = (m \oplus k \oplus \Delta) \oplus k = m \oplus \Delta.$$

The keystream cancels exactly as it did for the legitimate plaintext, and Bob recovers $m \oplus \Delta$ – the original message with Eve's chosen bits flipped. Eve changed the plaintext in a controlled way without knowing a single bit of the key or the message. This property is called *malleability*, and it is fatal whenever integrity matters.

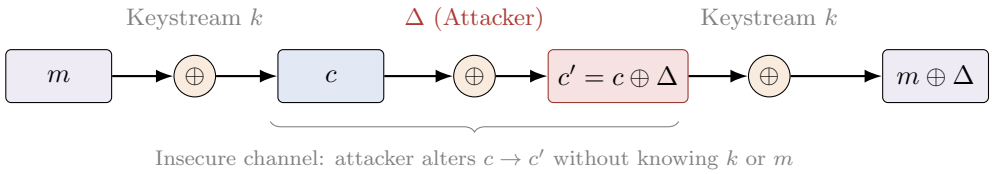


Figure 16.1: Stream-cipher malleability. The attacker XORs a chosen mask Δ into the ciphertext; because keystream k cancels on decryption ($(m \oplus k \oplus \Delta) \oplus k = m \oplus \Delta$), the recipient recovers a predictably altered message.

Intuition

Encryption scrambles the *content* of a message; it says nothing about whether the message you received is the one that was sent. Picture a sealed envelope written in a code only Bob can read. Confidentiality means Eve cannot read what is inside. But nothing about the seal stops Eve from steaming it open, scribbling over it (even blindly), and resealing – or from swapping it for a different sealed envelope entirely. Hiding the contents does nothing to detect changes to them. You need a second mechanism whose only job is to reject the message when a bit changes.

Attack

Flipping a transfer amount under encryption. A toy banking gateway sends instructions as fixed-format ASCII encrypted with a stream cipher: `PAY USER 0001 AMOUNT 0000100`. Suppose Eve knows the format (Kerckhoffs: she knows everything but the key) and knows the byte offset of the amount field. She does not know the keystream and cannot read the ciphertext, but she does not need to – she is just instantiating Figure 16.1 with a Δ aimed at one byte. The ASCII for the digit `1` is `0x31`; for `9` it is `0x39`. Their XOR difference is `0x31` $\oplus$ `0x39` = `0x08`. Eve XORs the ciphertext byte at the leading-amount position with `0x08`. When Bob decrypts, that one plaintext byte changes from `1` to `9`: the instruction now reads `AMOUNT 9000100`. Eve has multiplied the transfer ninety-fold without breaking the encryption at all. She broke *integrity*, which the cipher never promised to protect.

This is not a quirk of stream ciphers. CBC mode has its own malleability: flipping a bit in ciphertext block C_i flips the same bit in plaintext block P_{i+1} (while turning block P_i into garbage), because CBC decryption XORs the previous ciphertext block into the decrypted block. CTR mode is exactly

as malleable as a stream cipher, for the same reason – it *is* a stream cipher. The lesson generalizes: **encryption schemes that achieve confidentiality say nothing about integrity, and most of them are actively malleable.** If you want Bob to reject a tampered ciphertext, you must add a mechanism that checks integrity. That mechanism is a MAC, and combining it correctly with encryption is the subject of authenticated encryption.

Common trap

"It's encrypted, so it can't be forged" is one of the most expensive false beliefs in applied cryptography. WEP (early WiFi) encrypted with RC4 and "protected integrity" with a plain CRC32 checksum – a linear, unkeyed function. Because both RC4's keystream XOR and CRC32 are linear, an attacker could flip ciphertext bits *and* fix up the checksum to match, forging valid-looking packets without the key. WEP provided confidentiality but no real integrity. We will return to WEP's failure in Chapter 26, but note the mistake now: a checksum is not a MAC, and encryption is not integrity.

16.2 What "Secure" Means Here: AE and AEAD

To combine encryption and authentication we first need to say precisely what we want the combination to achieve. We already have a confidentiality goal from earlier chapters – indistinguishability under chosen-plaintext attack (IND-CPA), the property that ciphertexts leak nothing about plaintexts even to an adversary who can get things encrypted. Integrity gets its own goal.

Definition 16.1 (Ciphertext integrity, INT-CTXT)**.** A symmetric encryption scheme $(\mathsf{Gen}, \mathsf{Enc}, \mathsf{Dec})$ has *ciphertext integrity* if no PPT adversary, allowed to request encryptions of any messages it likes under the secret key, can produce a *new* ciphertext (one not returned by an encryption query) that Dec accepts as valid rather than rejecting. Formally, the adversary wins if it outputs a ciphertext c^* with $\mathsf{Dec}_k(c^*) \neq \perp$ and c^* was never an output of the encryption oracle; the scheme is INT-CTXT secure if the winning probability is negl.

The $\perp$ symbol ("bottom") is the decryption routine's way of saying "reject – this is not a valid ciphertext." A scheme with ciphertext integrity is one where the *only* ciphertexts Bob will accept are ones Alice actually produced. Eve can copy them, but she cannot manufacture a fresh one that decrypts to anything at all.

Definition 16.2 (Authenticated encryption)**.** A scheme is an *authenticated encryption* (AE) scheme if it is both IND-CPA secure (confidentiality) and INT-CTXT secure (integrity).

Why is this pairing the right target? Because it implies something stronger that you might have asked for instead: security against chosen-*ciphertext* attacks (IND-CCA), where the adversary may even submit ciphertexts of its choice to a decryption oracle. The following is one of the cleanest results in symmetric crypto.

Theorem 16.1 (AE implies CCA security)**.** *If a scheme is IND-CPA secure and INT-CTXT secure, then it is IND-CCA secure.*

Why it works

The intuition is short. In a chosen-ciphertext attack the adversary's extra power is the decryption oracle: it can hand over ciphertexts and learn their plaintexts. But INT-CTXT says the adversary cannot construct *any* new ciphertext that decrypts to something other than $\perp$. So every ciphertext it submits to the decryption oracle is either one it already got from encryption (telling it nothing new) or a forgery that the oracle rejects with $\perp$ (also telling it nothing). The decryption oracle is therefore useless – integrity has neutralized it – and we are left with a plain IND-CPA adversary, which by assumption cannot win. Integrity turns CCA back into CPA. This is exactly why we chase authenticated encryption rather than confidentiality alone: it is the property that makes a channel safe against an attacker who can both read and *inject*, which is every real network.

Now the practical refinement. Real messages are not just a blob of plaintext to be hidden. A network packet has a header – routing addresses, sequence numbers, version tags, packet types – that must travel *in the clear* (routers need to read it) but must still be *authenticated* (an attacker must not be able to alter it undetected). A TLS record has a content type and a length; a packet has a sequence number you want bound to the payload so it cannot be replayed or reordered. We need a scheme that encrypts some data while authenticating both that data *and* additional unencrypted data, all under one tag.

Definition 16.3 (AEAD)**.** An *authenticated encryption with associated data* (AEAD) scheme is a pair of algorithms

$$\mathrm{Enc}_k(N, A, M) \to C, \qquad \mathrm{Dec}_k(N, A, C) \to M \text{ or } \perp,$$

where N is a *nonce* (a number used once), A is the *associated data* (authenticated but not encrypted), and M is the plaintext (authenticated and encrypted). Decryption returns the plaintext only if the nonce, associated data, and ciphertext are all exactly what encryption produced together; if any of them was altered, it returns $\perp$. Security requires IND-CPA confidentiality of M and INT-CTXT integrity covering N, A, and C jointly.

Intuition

Think of AEAD as sealing a shipping crate. The plaintext M is the cargo, sealed inside where no one can see it. The associated data A is the shipping label glued to the outside: everyone can read it (the courier needs the address), but the seal is engineered so that peeling off or editing the label breaks the seal. The nonce N is the unique tracking number stamped on this specific crate so two crates can never be confused or one replayed as another. A single operation with one key and one tag gives all three guarantees: the cargo is hidden, the label cannot be altered, and the crate is unique.

Figure 16.2 shows how an AEAD packet looks on the wire: the nonce N and associated data A travel in cleartext so routers and endpoints can parse the header, the payload C is encrypted, and the single tag T binds all three fields together.

Focus

The nonce is not a key and not a secret. It can be a counter, it can be sent in the clear, an attacker is allowed to see it. The single rule – and it is an absolute rule for the schemes in this chapter – is that a given key must **never** encrypt two different messages under the **same**

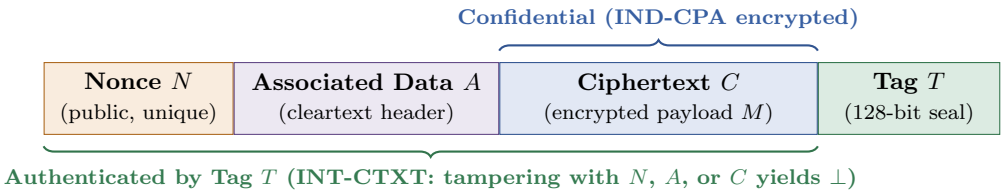


Figure 16.2: Anatomy of an AEAD record on the wire. Only the payload M is encrypted into C, while the authentication tag T covers the nonce N, cleartext associated data A, and ciphertext C jointly.

nonce. "Number used once" means once per key, forever. We will spend a full section on what happens when this rule is broken for GCM, because security does not degrade gradually: a single reuse breaks it completely.

16.3 Three Ways to Glue, Two of Them Broken

Before the named schemes, settle the generic question: given a secure encryption scheme and a secure MAC *with independent keys* k_e and k_m, how do you combine them? There are exactly three natural orderings, and they are not equally good. This is the famous "generic composition" analysis of Bellare and Namprempre.

Encrypt-and-MAC (E&M). Compute $c = \mathrm{Enc}_{k_e}(m)$ and $t = \mathrm{MAC}_{k_m}(m)$, send (c, t). The tag is computed over the *plaintext*.

MAC-then-Encrypt (MtE). Compute $t = \mathrm{MAC}_{k_m}(m)$, then $c = \mathrm{Enc}_{k_e}(m \,\|\, t)$, send c. The tag is hidden inside the ciphertext.

Encrypt-then-MAC (EtM). Compute $c = \mathrm{Enc}_{k_e}(m)$, then $t = \mathrm{MAC}_{k_m}(c)$, send (c, t). The tag is computed over the *ciphertext*.

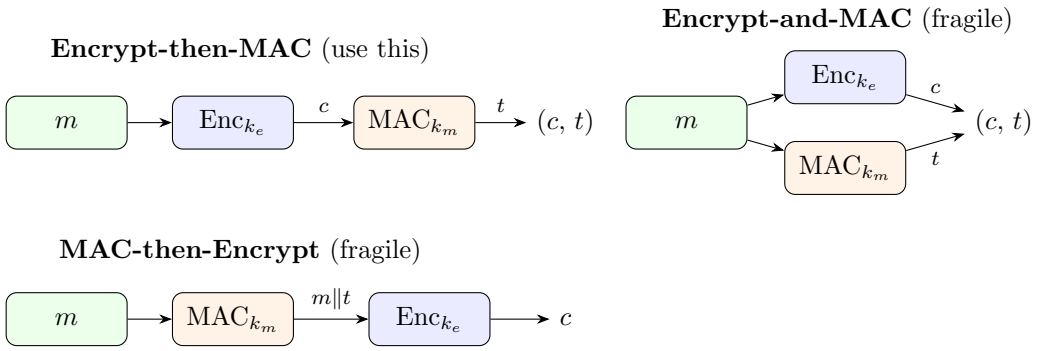


Figure 16.3: The three generic compositions. Only Encrypt-then-MAC is secure for *any* secure cipher and MAC; the other two depend on extra properties and have broken real systems.

We give the verdict for each composition, followed by the reasoning.

Encrypt-then-MAC is the one that always works. If Enc is IND-CPA secure and MAC is a strongly unforgeable MAC, then EtM is an authenticated encryption scheme – no matter what the underlying cipher and MAC are, as long as each is individually secure. The MAC covers the ciphertext c. Before Bob decrypts anything, he checks $\text{MAC}_{k_m}(c) \stackrel{?}{=} t$. By unforgeability, the only c values that pass are ones Alice actually sent, so Bob's decryption routine never even runs on an attacker-chosen ciphertext. INT-CTXT falls out directly: a forged ciphertext fails the MAC check and is rejected as $\perp$. Confidentiality is untouched because the tag is computed on c, which IND-CPA already says leaks nothing about m. Verify before you decrypt, and an attacker's injected ciphertext is rejected before decryption ever runs.

Encrypt-and-MAC can leak plaintext. The tag $t = \text{MAC}_{k_m}(m)$ is sent in the clear and is a deterministic function of the plaintext. A standard MAC makes no promise to hide its input – HMAC of a message is the same every time for the same message. So if Alice sends the same plaintext twice, the tags are identical, and Eve learns "these two messages are equal" without breaking the encryption. That violates IND-CPA. E&M can also fail to give ciphertext integrity, because the MAC says nothing about c.

MAC-then-Encrypt is secure only for *some* ciphers, and the exceptions failed badly in practice. Here the tag is inside the ciphertext, so Bob must *decrypt first* to even see the tag, then check it. Decrypting attacker-controlled bytes before authenticating them is the root mistake. If the decryption step leaks any information about *why* a ciphertext was malformed – a padding error versus a MAC error, or merely a timing difference between them – the attacker gets an oracle. This is precisely the padding-oracle family of attacks (POODLE, Lucky 13, the original Vaudenay attack), which broke MtE-with-CBC in SSL/TLS for years.

Attack

The padding oracle, in one paragraph. TLS up through 1.2 used MAC-then-Encrypt with CBC-mode AES. To decrypt, Bob removes CBC padding, *then* checks the MAC. If the padding is malformed he can reject before even computing the MAC – and if that rejection is observably different (a distinct error, or just faster), the attacker learns "this ciphertext had valid padding" for any ciphertext he submits. Vaudenay showed that a valid-padding oracle is enough to decrypt the entire message byte by byte, no key required: you tamper with a ciphertext block, watch whether the padding check passes, and each pass reveals one byte of intermediate state. Lucky 13 (2013) revived the attack through a *timing* side channel even after implementations tried to hide the explicit error. The root cause is structural: MtE forces you to process unauthenticated ciphertext, and any leak from that processing is an oracle. Encrypt-then-MAC has no such step – a bad ciphertext never reaches the decryptor.

Common trap

Do not separate encryption and authentication if you can avoid it. Even Encrypt-then-MAC, the safe composition, asks you to manage two keys, two primitives, the right order, constant-time tag comparison, and "verify before decrypt" – and one slip reintroduces a padding oracle or a key-reuse bug. The industry's hard-won conclusion is to not hand engineers a do-it-yourself kit at all. Use a single AEAD primitive that bakes the correct composition in, exposes one function, and makes the dangerous orderings unrepresentable. That is what GCM and ChaCha20-Poly1305 are. Reach for them; do not reinvent EtM by hand unless you have a very specific reason and a cryptographer reviewing it.

The generic-composition question was settled theoretically by Mihir Bellare and Chanathip Namprempre in 2000, and independently informed by Hugo Krawczyk's analysis the same year ("The order of encryption and authentication for protecting communications"). Their conclusion – Encrypt-then-MAC is the robust choice – was already known when SSL, IPsec, and SSH each picked a *different* ordering. SSL/TLS chose MAC-then-Encrypt; IPsec chose Encrypt-then-MAC; the original SSH chose Encrypt-and-MAC. The protocol that picked the theoretically sound option, IPsec, was the one spared the padding-oracle decade. Real-world exploit history matched the theoretical boundary almost to the letter.

16.4 AES-GCM: Counter Mode Meets a Polynomial Hash

Galois/Counter Mode is the AEAD that secures the majority of TLS connections on the planet, and it is the default in IPsec, SSH, and disk encryption. It is fast because both halves parallelize and AES has hardware support (the `AES-NI` and `PCLMULQDQ` instructions), and it is an AEAD by construction rather than by hand-assembly. It combines two parts: CTR mode for confidentiality, and a keyed polynomial hash called GHASH for integrity. We build each part, then show how they fit together.

16.4.1 The confidentiality half: CTR mode

Counter mode turns a block cipher into a stream cipher. Pick a per-message counter block, encrypt successive counter values with AES to produce a keystream, XOR it onto the plaintext. In GCM the counter is derived from the 96-bit nonce N: the first counter block is $J_0 = N \,\|\, 0^{31}1$ (the nonce followed by the 32-bit integer 1), and the blocks that encrypt data use $J_0{+}1, J_0{+}2, \ldots$, incrementing the low 32 bits. Block i of plaintext is encrypted as

$$C_i = P_i \oplus \mathrm{AES}_k(J_0 + i).$$

Counter block J_0 itself is reserved – it encrypts the authentication tag, not data. CTR is IND-CPA secure as long as no counter block ever repeats under the same key, which is exactly why the nonce must be unique: a repeated nonce makes J_0 and all the $J_0 + i$ repeat, reusing keystream. This is the root of the nonce-reuse attack in the next section.

16.4.2 The integrity half: GHASH over $\mathbb{F}_{2^{128}}$

GHASH is a *universal hash* evaluated in the finite field $\mathbb{F}_{2^{128}}$. Treat every 128-bit block as an element of $\mathbb{F}_{2^{128}}$ – the field of polynomials over $\mathbb{F}_2$ modulo the irreducible $x^{128} + x^7 + x^2 + x + 1$ (Chapter 6 built exactly these objects). Field addition is XOR; field multiplication is carryless polynomial multiplication reduced by that modulus. Let $H = \mathrm{AES}_k(0^{128})$ be the *hash key*: AES applied to the all-zero block. GHASH takes the associated data A and ciphertext C, splits the concatenation (with a final block encoding the bit-lengths of A and C) into blocks $X_1, X_2, \ldots, X_m$, and evaluates

$$\mathrm{GHASH}_H(X_1, \ldots, X_m) = X_1 H^m \oplus X_2 H^{m-1} \oplus \cdots \oplus X_{m-1} H^2 \oplus X_m H,$$

all arithmetic in $\mathbb{F}_{2^{128}}$. It is a polynomial in the secret point H whose coefficients are the data blocks. The authentication tag is this hash, masked by one block of keystream:

$$T = \mathrm{GHASH}_H(A, C) \oplus \mathrm{AES}_k(J_0).$$

Why it works

Why is a polynomial in a secret point a good integrity check? GHASH is what cryptographers call an *ε-almost-universal* hash: for any two distinct inputs, the probability (over the random hash key H) that they collide is tiny – at most about $m/2^{128}$ for m-block messages. The mechanism is the root bound for polynomials over a field. If an attacker changes the data from blocks X to blocks X', the two GHASH outputs differ by $(X_1 - X'_1)H^m \oplus \cdots \oplus (X_m - X'_m)H$, which is a nonzero polynomial in H of degree at most m. A nonzero degree-m polynomial over a field has at most m roots, so at most m values of H make the difference vanish. The attacker does not know H (it is locked behind AES of zero), so the chance a blind tampering lands on one of those $\leq m$ bad points out of 2^{128} is negligible. The keystream mask $\mathrm{AES}_k(J_0)$ then hides the raw hash value so the attacker cannot even see GHASH outputs to learn about H. Integrity rests on one fact: you cannot guess a root of a polynomial whose secret evaluation point you cannot see.

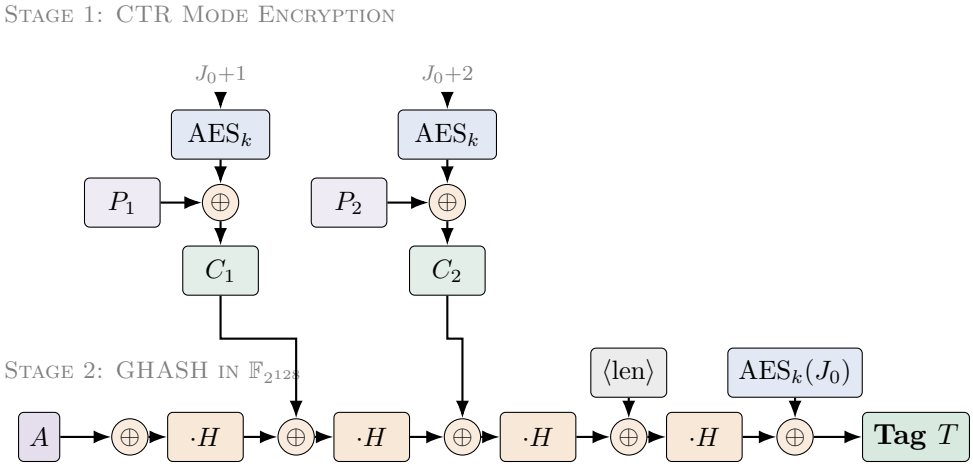


Figure 16.4: The structure of AES-GCM. Stage 1 (top) runs CTR mode to encrypt plaintext blocks P_i into C_i. Stage 2 (bottom) feeds associated data A, ciphertext blocks C_i, and the length block through a Horner-rule polynomial evaluation in $\mathbb{F}_{2^{128}}$ with hash key $H = \mathrm{AES}_k(0^{128})$, masking the final field accumulator with $\mathrm{AES}_k(J_0)$ to produce tag T.

16.4.3 A worked GHASH in a toy field

The real field $\mathbb{F}_{2^{128}}$ is too big to trace by hand, so we shrink to $\mathbb{F}_{2^4}$, the field of 4-bit blocks modulo the irreducible polynomial $x^4 + x + 1$. Elements are nibbles; we write them as 0–`f` in hex and remember each is a polynomial: `b` $= 1011_2 = x^3 + x + 1$. Addition is XOR. Multiplication is polynomial multiply, then reduce by $x^4 = x + 1$ (the relation that the modulus gives us, since $x^4 + x + 1 = 0$ means $x^4 = x + 1$ in this field).

Let the hash key be $H =$ `2` $= x$. Suppose after splitting we have just two blocks to authenticate, $X_1 =$ `3` and $X_2 =$ `1`, so we want

$$\mathrm{GHASH}_H(X_1, X_2) = X_1 H^2 \oplus X_2 H.$$

Compute the powers of $H = x$:

$$H = x, \qquad H^2 = x^2 = \texttt{4}.$$

Now the two terms.

$$X_1 H^2 = \mathtt{3} \cdot \mathtt{4} = (x+1)(x^2) = x^3 + x^2 = \mathtt{c}.$$

That product has degree $3 < 4$, so no reduction is needed. The second term:

$$X_2 H = \mathtt{1} \cdot \mathtt{2} = (1)(x) = x = \mathtt{2}.$$

XOR them: $\mathtt{c} \oplus \mathtt{2} = 1100_2 \oplus 0010_2 = 1110_2 = \mathtt{e}$. So $\mathrm{GHASH}_x(\mathtt{3}, \mathtt{1}) = \mathtt{e}$.

Now see the universal-hash property at work. Suppose an attacker flips X_2 from $\mathtt{1}$ to $\mathtt{1} \oplus \mathtt{1} = \mathtt{0}$, a difference of $\Delta = \mathtt{1}$ in the last block. The hash changes by $\Delta \cdot H = \mathtt{1} \cdot \mathtt{2} = \mathtt{2}$, so the new hash is $\mathtt{e} \oplus \mathtt{2} = \mathtt{c}$. For the attacker's forgery to pass, the tag must not change – which requires $\Delta \cdot H = 0$, i.e. $H = 0$. But $H = x \neq 0$. To force *any* specific tag change the attacker would need to know H and solve for the right Δ, and H is hidden behind AES. In the full field the same algebra holds with 2^{128} possible H values and the chance of a blind hit is $\leq m/2^{128}$.

Work it yourself

Reduction practice. In $\mathbb{F}_{2^4}$ with $x^4 = x+1$, compute H^4 for $H = x$. You get $x^4 = x+1 = \mathtt{3}$. Continue: $H^5 = x \cdot \mathtt{3} = x^2 + x = \mathtt{6}$, $H^6 = x \cdot \mathtt{6} = x^3 + x^2 = \mathtt{c}$, $H^7 = x \cdot \mathtt{c} = x^4 + x^3 = (x+1) + x^3 = x^3 + x + 1 = \mathtt{b}$. Notice the powers of x cycle through nonzero elements – x is a generator of the multiplicative group $\mathbb{F}_{2^4}^*$ of order 15. Verify $H^{15} = \mathtt{1}$ by extending the table. GHASH relies on this same field arithmetic to evaluate its polynomial.

Implement it

A deliberately tiny GHASH and GCM-style tag over $\mathbb{F}_{2^4}$, to make the algebra runnable. This is a learning toy: a 4-bit field gives *zero* real security, multiplication is not constant-time, and nothing here resists a real attacker. Never use this for anything but understanding.

```
# Toy GHASH in GF(2^4), modulus x^4 + x + 1.  LEARNING ONLY -- not secure.
def gf_mul(a, b):
    """Carryless multiply of two nibbles, reduced mod x^4 + x + 1."""
    p = 0
    for _ in range(4):
        if b & 1:
            p ^= a
        b >>= 1
        carry = a & 0x8          # bit x^3; shifting it makes x^4
        a = (a << 1) & 0xF
        if carry:
            a ^= 0b0011          # x^4 = x + 1  ->  XOR with 0011
    return p

def ghash(H, blocks):
    """Horner evaluation: (((X1*H) ^ X2)*H ^ X3)*H ... all in GF(2^4)."""
    y = 0
    for X in blocks:
        y = gf_mul(y ^ X, H)
    return y

def gcm_tag(H, mask, A_blocks, C_blocks, lenblock):
    # Authenticate associated data, ciphertext, and a length block, then mask.
```

```
    data = A_blocks + C_blocks + [lenblock]
    return ghash(H, data) ^ mask

H = 0x2                                  # hash key = x  (= AES_k(0) in the real
    scheme)
print(hex(ghash(H, [0x3, 0x1])))         # -> 0xe, matching the hand computation

# Flipping the last block changes the tag predictably by Delta*H:
print(hex(ghash(H, [0x3, 0x0])))         # -> 0xc  ( = 0xe XOR (0x1 * H) )
```

The Horner form `y = (y ^ X)*H` is the standard streaming way to evaluate the GHASH polynomial one block at a time; expand it for two blocks and you recover $X_1H^2 \oplus X_2H$ exactly.

16.5 The Nonce-Reuse Catastrophe in GCM

GCM has one weak point, and it is the most important thing to know when deploying it. Reuse a nonce – encrypt two different messages with the same key and the same N – and you lose the confidentiality of *both* messages; worse, you leak the hash key H, which lets the attacker forge *arbitrary* messages from then on. One repeated nonce can destroy the integrity of every message under that key.

16.5.1 Why a repeat is so much worse than for a plain stream cipher

You already know half of the damage. CTR mode is a stream cipher; reusing the nonce reuses the keystream blocks $\mathrm{AES}_k(J_0 + i)$. If $C = P \oplus S$ and $C' = P' \oplus S$ with the same keystream S, then $C \oplus C' = P \oplus P'$ – the XOR of two plaintexts, with the keystream cancelled, the classic two-time-pad break that leaks the messages. This is bad, but no worse than any other stream-cipher misuse.

The second half is specific to GCM: the attacker recovers the *authentication key* H. The two tags are

$$T = \mathrm{GHASH}_H(A, C) \oplus \mathrm{AES}_k(J_0), \qquad T' = \mathrm{GHASH}_H(A', C') \oplus \mathrm{AES}_k(J_0),$$

and because the nonce repeated, J_0 repeated, so the masking block $\mathrm{AES}_k(J_0)$ is *identical* in both. XOR the tags and the mask cancels:

$$T \oplus T' = \mathrm{GHASH}_H(A, C) \oplus \mathrm{GHASH}_H(A', C').$$

The right side is a known polynomial in the unknown H – its coefficients are the (known) differences of the ciphertext and associated-data blocks, and it equals the (known) value $T \oplus T'$. The attacker now has a single polynomial equation $\Phi(H) = 0$ over $\mathbb{F}_{2^{128}}$ whose roots can be found efficiently (polynomial root-finding over a finite field is a solved problem). A degree-m polynomial has at most m roots; with two distinct message lengths the attacker gets a short list of candidate H values and, with a third reused-nonce message or a single forgery attempt, pins down the real one.

Once H is known, the masking is the only secret left in the tag, and the attacker can do affine algebra on tags: GHASH is linear in its block inputs over $\mathbb{F}_{2^{128}}$, so knowing H lets the attacker compute, for any tampered ciphertext, exactly what tag it must carry to verify – relative to a known good tag. Integrity is gone for the whole key, not just the two reused messages.

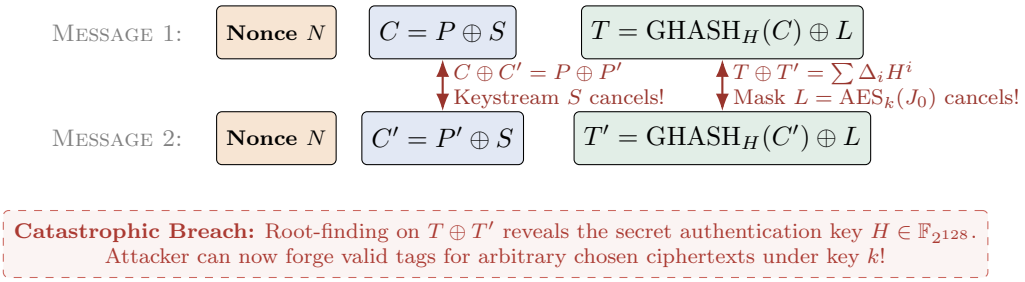


Figure 16.5: The GCM nonce-reuse catastrophe. Reusing a nonce under the same key reuses both the CTR keystream S and the tag mask $L = \mathrm{AES}_k(J_0)$. XORing the ciphertexts leaks plaintext XOR; XORing the tags eliminates the mask, leaving a polynomial equation whose roots reveal the hash key H.

Attack

Recovering H from a reused nonce, in the toy field. Work in $\mathbb{F}_{2^4}$ again, H unknown, modulus $x^4 + x + 1$. Say two one-block-ciphertext messages were sent under the same nonce (so J_0, hence the mask $\mathrm{AES}_k(J_0) =: L$, repeats), with no associated data and a length block ℓ that is the same for both, so it cancels. Their tags are

$$T = (C_1 H^2 \oplus \ell H) \oplus L, \qquad T' = (C_1' H^2 \oplus \ell H) \oplus L.$$

XOR: $T \oplus T' = (C_1 \oplus C_1')\, H^2$. Suppose the attacker observes ciphertext blocks $C_1 = \mathtt{5}$, $C_1' = \mathtt{6}$ and tags whose XOR is $T \oplus T' = \mathtt{c}$. Then

$$(C_1 \oplus C_1')\, H^2 = (\mathtt{5} \oplus \mathtt{6})\, H^2 = \mathtt{3}\, H^2 = \mathtt{c}.$$

Solve for H^2: divide by $\mathtt{3} = x{+}1$. The inverse of $\mathtt{3}$ in $\mathbb{F}_{2^4}$ is $\mathtt{e}$ (check: $\mathtt{3}{\cdot}\mathtt{e} = (x{+}1)(x^3{+}x^2{+}x) = x^4 + x = \mathtt{1}$ after $x^4 = x + 1$), so $H^2 = \mathtt{c} \cdot \mathtt{e}$. Compute $\mathtt{c} \cdot \mathtt{e} = (x^3 + x^2)(x^3 + x^2 + x)$; reducing gives $\mathtt{4} = x^2$. Finally $H^2 = x^2$ gives $H = x = \mathtt{2}$ (the square root in characteristic two is unique). The attacker has recovered the secret hash key using nothing but two tags and two ciphertexts. In $\mathbb{F}_{2^{128}}$ the arithmetic is bigger but not harder – the same single equation, the same root-finding – and the attacker again recovers H.

Common trap

The 96-bit nonce is small, and random nonces collide. GCM nonces are only 96 bits. If you generate them *randomly* per message under one key, the birthday bound says a collision becomes likely after about 2^{48} messages – and *one* collision is the catastrophe above. 2^{48} sounds large until you are a busy server: it is a real limit, and the safe deployment is a **counter** nonce (message number, never repeating) rather than a random one, with a hard cap of 2^{32} messages per key as a comfortable margin. If you cannot guarantee unique nonces – say, in a distributed system with no shared counter – do not use plain GCM; use a nonce-misuse-resistant mode like AES-GCM-SIV, which degrades gracefully (a reused nonce leaks only whether two plaintexts were equal, and never the key).

Connection

The "2^{48} messages" figure is the birthday bound from the probability companion: drawing 96-bit values uniformly at random, the expected number of draws before a repeat is about $\sqrt{2^{96}} = 2^{48}$. The same $\sqrt{N}$ law governs hash collisions (Chapter 14) and the meet-in-the-middle attacks on double encryption. Here it sets a hard ceiling on how many messages a single GCM key may protect with random nonces, and it is why "just randomize the nonce" is not a safe substitute for "never repeat the nonce."

History

GCM was designed by David McGrew and John Viega in 2004, building on the authenticated-encryption definitions of Mihir Bellare, Phillip Rogaway, and Hugo Krawczyk around 2000, and was standardized by NIST in SP 800-38D (2007). Its nonce-reuse fragility is not a flaw in the mathematics, but it has caused real failures. A 2016 study found hundreds of HTTPS servers reusing GCM nonces because of faulty random-number generators, VM snapshots, or broken nonce-construction code, each one a server whose TLS sessions could be decrypted and forged. The lesson stuck: later work, including the CAESAR competition (2014–2019), the nonce-misuse-resistant AES-GCM-SIV mode (RFC 8452), and the strict nonce rules in TLS 1.3, treats unique nonces as a property the protocol must *enforce* rather than something the implementer must remember.

16.6 ChaCha20-Poly1305: The Software-Friendly AEAD

AES-GCM is the fastest choice when you have AES hardware. On a device without it – many phones a decade ago, embedded chips, anything where a software AES is slow and, worse, leaks timing through cache-dependent table lookups – you want an AEAD built from operations that are fast and constant-time in plain software. That is ChaCha20-Poly1305, designed largely by Daniel J. Bernstein and standardized in RFC 8439. Google deployed it to protect Android and Chrome traffic precisely because it runs fast and side-channel-resistant without special instructions.

The shape mirrors GCM: a stream cipher for confidentiality, a one-time universal-hash MAC for integrity, combined the right way.

ChaCha20 is a stream cipher built from a 512-bit state (sixteen 32-bit words: constants, the 256-bit key, a 32-bit counter, a 96-bit nonce) churned by twenty rounds of the ARX operations *add, rotate, XOR*. No lookup tables, so no cache-timing leak; just register arithmetic that is the same speed regardless of data. Each block of keystream is the state run through the rounds and added back to itself, then XORed onto plaintext exactly like CTR mode.

Poly1305 is the MAC, a polynomial evaluation hash like GHASH but over the prime field $\mathbb{F}_p$ with $p = 2^{130} - 5$ instead of a binary field. The message is chopped into 128-bit chunks $c_1, \ldots, c_q$ (each with a high bit set to disambiguate length), and with a secret evaluation point r and a secret additive blinder s the tag is

$$\text{Poly1305}_{r,s}(M) = \Big(\big(c_1 r^q + c_2 r^{q-1} + \cdots + c_q r \big) \bmod p \Big) + s \pmod{2^{128}}.$$

Same idea as GHASH – the data are coefficients of a polynomial in a secret point – with prime-field arithmetic that is friendly to constant-time software and a $\bmod 2^{128}$ final addition. The key material

(r, s) is *derived freshly for each message* from ChaCha20 itself: running ChaCha20 with counter 0 on the message's nonce produces a 64-byte block; the first 32 bytes are taken as (r, s) (with specific bits of r "clamped" to zero for security and speed), while counters $1, 2, \ldots$ encrypt the plaintext. Because (r, s) is refreshed per nonce, Poly1305 functions as a fresh one-time MAC for each packet, and the whole construction achieves Encrypt-then-MAC: the tag is computed over the ciphertext and associated data, not the plaintext.

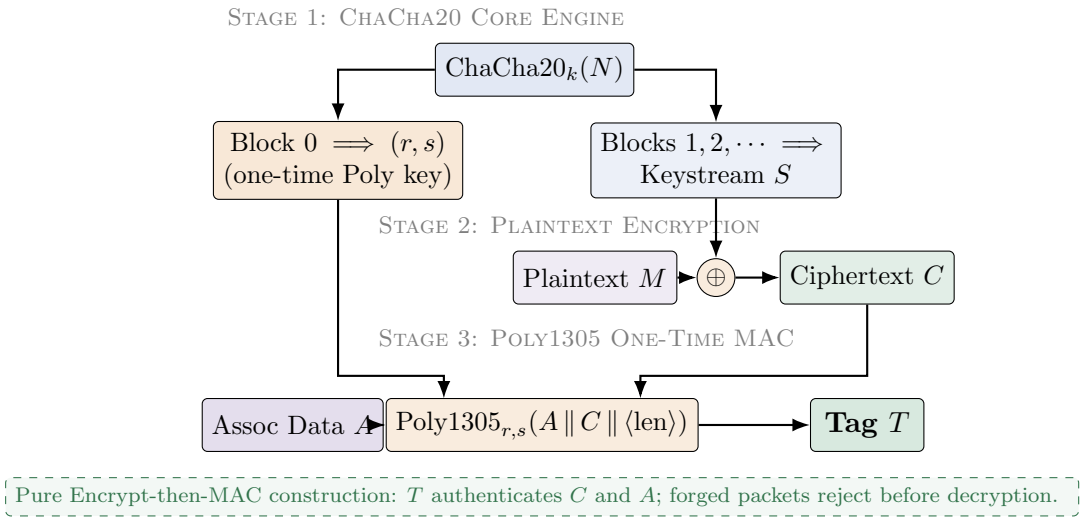


Figure 16.6: The ChaCha20-Poly1305 authenticated encryption architecture. ChaCha20 block 0 derives the one-time evaluation point r and blinder s; subsequent blocks form the keystream that encrypts plaintext M. Poly1305 computes an unforgeable tag over the associated data and ciphertext in $\mathbb{F}_{2^{130}-5}$.

Why it works

The security argument is the same shape as GCM's, with a friendlier failure mode for one specific mistake. Poly1305 is an ε-almost-universal hash: tampering changes the polynomial by a nonzero polynomial of degree $\leq q$ in the secret r, which vanishes for at most q values of r out of $\approx 2^{128}$, so a blind forgery succeeds with probability about $q/2^{128}$. The additive secret s hides the raw polynomial value. Confidentiality is ChaCha20's IND-CPA security as a stream cipher. And because the construction is Encrypt-then-MAC over the ciphertext, the AE-implies-CCA theorem (Theorem 16.1) applies: a forged ciphertext fails Poly1305 and is rejected before any decryption that could leak information.

Work it yourself

A worked Poly1305-style tag in a toy prime field. The real $p = 2^{130} - 5$ is unwieldy by hand, so shrink to the prime $p = 17$ and pretend each "block" is a single residue (the real scheme uses 128-bit blocks with a set high bit, but the polynomial structure is identical). Take the secret evaluation point $r = 5$ and the secret blinder $s = 9$, and authenticate two blocks $c_1 = 3$, $c_2 = 7$. The polynomial is $c_1 r^2 + c_2 r \pmod p$, then add s:

$$c_1 r^2 + c_2 r = 3 \cdot 25 + 7 \cdot 5 = 75 + 35 = 110 \equiv 110 - 6 \cdot 17 = 8 \pmod{17}.$$

Add the blinder: tag $= (8 + 9) \bmod 17 = 17 \bmod 17 = 0$. Now the forgery test: flip c_2 from 7 to $7 + \Delta$. The polynomial value moves by $\Delta \cdot r = 5\Delta \pmod{17}$, a nonzero shift for every $\Delta \neq 0$ because $r \neq 0$ and 17 is prime (no zero divisors). An attacker who does not know r cannot predict that shift, and s hides the raw value so observed tags reveal nothing about r. In $\mathbb{F}_{2^{130}-5}$ the only change is scale: a degree-q polynomial, at most q roots, a $\leq q/2^{128}$ chance of a blind hit – the exact same bound as GHASH, reached through a prime field instead of a binary one.

Focus

ChaCha20-Poly1305 is *not* nonce-misuse resistant – reuse a nonce and you reuse keystream (two-time pad) *and* reuse the one-time Poly1305 key (r, s), which similarly leaks r and enables forgery. The same rule holds: unique nonce per key, always. What ChaCha20-Poly1305 buys you over GCM is not nonce robustness but *software safety*: constant-time by design, fast without hardware, no carryless-multiply instruction needed, and a cleaner implementation surface with fewer ways to introduce a timing leak.

Implement it

The right way to "implement" an AEAD is to call a vetted library. Here is what correct usage looks like with PyCA `cryptography`; the point is the *interface*, not a reimplementation. Note that the library refuses to let you skip the tag check: a tampered ciphertext raises an exception rather than returning garbage plaintext. This is real library code, but treat the surrounding nonce handling as the lesson – never reuse a nonce under a key.

```
import os
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305, AESGCM

key   = ChaCha20Poly1305.generate_key()      # 256-bit key
aead  = ChaCha20Poly1305(key)

nonce = os.urandom(12)                        # 96-bit nonce: UNIQUE per key!
ad    = b"v1|type=transfer|hdr"               # associated data: authenticated, not
    hidden
msg   = b"PAY USER 0001 AMOUNT 0000100"

ct = aead.encrypt(nonce, msg, ad)             # ciphertext WITH appended tag
pt = aead.decrypt(nonce, ct, ad)              # returns msg, or RAISES on any
    tampering
assert pt == msg

# Integrity in action: flip one ciphertext byte (the stream-cipher attack from
# the start of the chapter) and decryption now FAILS instead of silently lying.
tampered = bytearray(ct); tampered[20] ^= 0x08
try:
    aead.decrypt(nonce, bytes(tampered), ad)
    print("BUG: forgery accepted")
except Exception:
    print("rejected forged ciphertext")      # <-- this branch runs

```

```
# AES-GCM is a drop-in with the SAME interface and the SAME nonce rule:
gkey = AESGCM.generate_key(bit_length=256)
AESGCM(gkey).encrypt(os.urandom(12), msg, ad)
```

The malleability attack that multiplied a bank transfer ninety-fold at the start of this chapter is exactly the `tampered[20] ^= 0x08` line – and under AEAD it no longer works, because the altered ciphertext fails the tag check and `decrypt` refuses to return anything.

16.7 Choosing and Using AEAD in the Real World

In the real world

Where AEAD lives. TLS 1.3 (2018) removed every non-AEAD cipher: the only record-protection algorithms left are AES-GCM, ChaCha20-Poly1305, and AES-CCM. The MAC-then-Encrypt CBC suites behind a decade of padding-oracle attacks were removed. In TLS the record header (content type, version, length) is the associated data, bound to the encrypted payload, and the nonce is built from a per-connection counter so it never repeats. The same primitives appear everywhere: WPA3 and modern WPA2 use AES-GCM/CCM (a clean break from WEP's broken CRC); SSH offers `aes128-gcm` and `chacha20-poly1305`; IPsec uses AES-GCM; the Signal protocol and WireGuard VPN use ChaCha20-Poly1305; QUIC uses AEAD for every packet, with the packet number as the nonce. If you transmit a byte over a modern secure channel, an AEAD almost certainly protected it.

A short decision guide for the practitioner:

- **Default to an AEAD, never raw encryption.** If your library offers `encrypt(key, nonce, plaintext, associated_data)`, use it. If you find yourself manually composing a cipher and a MAC, stop: you are likely to introduce a bug.
- **AES-GCM** when AES hardware is present (essentially all modern servers and laptops, via AES-NI). Fastest there, and the most widely supported.
- **ChaCha20-Poly1305** when you lack AES hardware or want guaranteed constant-time software – mobile, embedded, or any context where a cache-timing leak worries you. Often the better default for portable code.
- **Nonces: enforce uniqueness structurally.** Prefer a counter you control over a random nonce. If a counter is impossible across a distributed fleet, use a misuse-resistant mode (AES-GCM-SIV) rather than gambling on the 2^{48} birthday bound.
- **Put everything that must not change into the associated data** – versions, sequence numbers, recipient identities, packet types. Anything an attacker could profitably alter, but that you cannot or need not encrypt, belongs in A so the tag covers it.
- **Never act on plaintext before the tag verifies.** A correct AEAD library does this for you (decryption returns $\perp$ or raises). If you ever see code that decrypts, uses the result, and checks integrity afterward, it is broken.

Why this matters

This chapter traces how the field matured. Early systems treated confidentiality and integrity as separate add-ons, glued primitives together by intuition, and paid for it with a decade of padding oracles and forged WiFi packets. The fix was a cleaner *abstraction* rather than a cleverer cipher: one primitive, one key, one nonce, one tag, with the dangerous choices made unrepresentable. AEAD is what it looks like when cryptographers stop asking "how do I encrypt?" and start asking "what is the smallest interface that an ordinary engineer cannot misuse?" That second question – design so the easy path is the safe path – is a central idea in applied cryptography, and authenticated encryption is one of its clearest successes.

Connection

Every security guarantee in authenticated encryption rests on an unstated premise: that keys are chosen uniformly at random and nonces never repeat. If those numbers come from a broken pseudorandom generator or a low-entropy seed, GCM collapses regardless of how strong AES is. Chapter 17 explores where cryptographic keys and unpredictable nonces come from: physical entropy and Cryptographically Secure Pseudorandom Number Generators (CSPRNGs).

Exercises

Exercise 16.1. A message is encrypted with a stream cipher as $c = m \oplus k$. You do not know k or m, but you know m is ASCII and that byte 5 of the plaintext is the digit `1` (`0x31`), and you want Bob to decrypt it as `8` (`0x38`). Which byte of the ciphertext do you modify, and by what XOR mask? Explain why this works without knowing the key, and name the security property the stream cipher fails to provide.

Exercise 16.2. State the three generic compositions of an encryption scheme and a MAC. For each of Encrypt-and-MAC and MAC-then-Encrypt, give a concrete reason it can fail to be a secure authenticated-encryption scheme (one confidentiality failure, one integrity/oracle failure). Then prove informally that Encrypt-then-MAC achieves ciphertext integrity, assuming the MAC is strongly unforgeable.

Exercise 16.3. Prove Theorem 16.1 in detail: show that IND-CPA plus INT-CTXT implies IND-CCA. Structure the argument as a reduction – assume a CCA adversary with non-negligible advantage and build either an INT-CTXT forger or an IND-CPA distinguisher. Where exactly does the decryption oracle become useless?

Exercise 16.4. Work in $\mathbb{F}_{2^4}$ with modulus x^4+x+1 (so $x^4 = x+1$). Let $H = \mathtt{3}$. Compute GHASH_H of the two-block input $(X_1, X_2) = (\mathtt{6}, \mathtt{a})$, i.e. evaluate $X_1 H^2 \oplus X_2 H$, showing the reduction step. Then determine the new GHASH value if an attacker flips X_2 to $\mathtt{a} \oplus \mathtt{4} = \mathtt{e}$, and confirm the change equals $\mathtt{4} \cdot H$.

Exercise 16.5. In the toy field $\mathbb{F}_{2^4}$, two one-block GCM-style messages are sent under the *same* nonce (so the tag mask L is identical). You observe ciphertext blocks $C_1 = \mathtt{7}$, $C_1' = \mathtt{2}$ and tags whose XOR is $T \oplus T' = \mathtt{5}$ (assume no associated data and that the length block is the same for both, so it cancels). Recover the hash key H. (Hint: $T \oplus T' = (C_1 \oplus C_1')\, H^2$; you will need the inverse of $C_1 \oplus C_1'$ in the field and a square root in characteristic two.)

Exercise 16.6. Explain precisely why reusing a nonce in AES-GCM is strictly worse than reusing a one-time-pad key. Your answer should identify (a) the confidentiality loss common to both, and (b) the integrity loss unique to GCM, including why the masking block $\mathrm{AES}_k(J_0)$ cancels and what equation in H the attacker is left with.

Exercise 16.7. A server generates GCM nonces by sampling 96 random bits per message under a fixed key. Using the birthday bound, estimate how many messages it can send before a nonce collision becomes likely, and explain why a *single* collision is catastrophic rather than merely a small leak. Propose two concrete fixes and state the trade-off of each.

Exercise 16.8. In ChaCha20-Poly1305, the one-time MAC key (r, s) is derived from ChaCha20 block 0 under the message's nonce, and the plaintext is encrypted with blocks $1, 2, \ldots$. Explain what goes wrong if an implementer mistakenly uses block 0 both as the MAC key and as the first keystream block. Which security property breaks, and how could an attacker exploit it?

Exercise 16.9. You are designing a packet format. Each packet has a cleartext header (`version`, `sequence_number`, `type`) that routers must read, and an encrypted payload. Show how to use an AEAD so that (i) the payload is confidential, (ii) the header cannot be altered undetected, and (iii) an attacker cannot replay or reorder packets. State exactly what goes in the nonce, the associated data, and the plaintext, and justify each choice.

Exercise 16.10. (Break a homemade scheme.) An engineer "saves a key" by using the *same* AES key for CTR-mode encryption and for a CBC-MAC over the plaintext, sending (c, t) with $c = \mathrm{CTR}_k(m)$ and $t = \text{CBC-MAC}_k(m)$. Identify at least two distinct problems: one from key reuse across two primitives, and one from the Encrypt-and-MAC structure leaking information. Then describe how a correct AEAD avoids both with a single key and a single pass.

Exercise 16.11. Work in the toy prime field $\mathbb{F}_{17}$ with Poly1305-style structure: secret point $r = 4$, blinder $s = 6$. Compute the tag for the two blocks $c_1 = 2$, $c_2 = 5$, evaluating $c_1 r^2 + c_2 r \pmod{17}$ and then adding $s \pmod{17}$. Next, show that if an attacker flips c_2 by Δ the tag shifts by $r\Delta \bmod 17$, and explain why no nonzero Δ can leave the tag unchanged in a prime field. How would knowing r (say, from a reused nonce) let the attacker forge the tag for a chosen c_2?

Exercise 16.12. (Design / research.) AES-GCM-SIV is a "nonce-misuse-resistant" AEAD: if a nonce repeats, it leaks only whether two plaintexts were identical, and never the authentication key. Read the high-level idea (a synthetic IV derived from the message itself) and explain why this makes nonce reuse a graceful degradation rather than a cliff. What does SIV give up compared to plain GCM, and in what deployment would you prefer it?

Chapter 17

Randomness and CSPRNGs

In Chapter 16 we saw how a single repeated nonce destroys AES-GCM, which points to a question we have put off for sixteen chapters: where do fresh random bits actually come from? Every secret in this book starts as a coin flip – an AES key is 128 coin flips; a nonce, an initialization vector, a password salt, or an ephemeral key is a request for bits no adversary can predict – and producing them on a deterministic machine is a genuine engineering problem whose failure means broken plaintexts and exposed keys.

In this chapter we stop taking random bits for granted. We will separate *true* physical randomness from *pseudo* randomness stretched from a small seed, define what makes a generator *cryptographically* secure, look at OS entropy pipelines and process forking, and examine three well-known randomness failures: the Debian OpenSSL key collapse, the `Dual_EC_DRBG` backdoor, and repeated-nonce signatures.

17.1 Why Cryptography Lives or Dies on Randomness

Recall the one-time pad: it achieves perfect secrecy, but *only* when the key is uniformly random and never reused. Without the randomness, the same algorithm gives no security at all. The same is true across cryptography. The algorithm is public (Kerckhoffs); the only thing standing between Eve and the plaintext is a quantity she cannot guess. If she *can* guess it – because it came from a predictable source – the public algorithm becomes a public decryption procedure that she runs as easily as Bob does.

Intuition

A cipher is a lockbox whose combination is your key. Kerckhoffs says the lockbox design is printed on the side; security rests entirely on the combination being unguessable. Now suppose the factory sets every combination by reading the wall clock at manufacturing time. The boxes are still "random" in the loose sense that you cannot easily predict any one of them in advance – but an attacker who knows the trick just tries the few thousand plausible timestamps and opens them all. The combination space did not shrink because the math changed; it shrank because the *source* of the combination had far less unpredictability than its length suggested. In cryptography what matters is how far an adversary can narrow the value down, not how

long it is.

The quantity that matters is *how many guesses the best attacker needs*, not how many bits long the value is. We measure that with min-entropy. If a random variable X takes value x with probability p_x, its min-entropy is

$$H_\infty(X) \;=\; -\log_2 \max_x p_x.$$

The max picks out the *most likely* value – the attacker's first guess. If the most likely outcome has probability 2^{-128}, then $H_\infty = 128$ and even the optimal attacker needs about 2^{128} tries; the value is worth its full length. But if some outcome has probability 2^{-20} – say a flawed generator only ever produces one of a million distinct keys – then $H_\infty = 20$ no matter how many bits the key occupies in memory. A "256-bit key" with 20 bits of min-entropy offers only 20 bits of real security.

Focus

Min-entropy, not Shannon entropy, is the right measure for cryptography. Shannon entropy $H = -\sum_x p_x \log_2 p_x$ averages over all outcomes and answers "how many bits to encode this on average." An attacker does not care about the average; she guesses the *single most likely* value first. A source that emits one fixed value 99.9% of the time and a uniform 127-bit string the rest has Shannon entropy of about 0.14 bits – but more to the point a min-entropy of only $-\log_2(0.999) \approx 0.0014$ bits. You crack it on the first guess almost always. Always size a key by its worst case, which is min-entropy.

Walking question

Suppose a website generates "random" 6-digit two-factor codes by seeding a generator with the current Unix time in seconds and reading one code out. Two users who request a code in the same second get correlated codes. Even ignoring that, how big is the real keyspace an attacker faces over a one-day window? (Hint: it is not 10^6. Count seconds.) Keep the count in mind; it is the difference between 10^6 guesses and roughly 86,400.

Connection

Min-entropy, Shannon entropy, and the collision/Renyi entropies are all instances of the Renyi entropy family H_α developed in the companion probability book. Min-entropy is H_∞ (the $\alpha \to \infty$ limit), Shannon is H_1, and the collision entropy H_2 governs birthday bounds. The ordering $H_\infty \le H_2 \le H_1$ is exactly why min-entropy is the conservative choice: it is the smallest of the three, so sizing a key by it never overstates your security.

17.2 True Randomness versus Pseudorandomness

There are two fundamentally different ways to get bits that look random, and a real system uses them in series. Confusing the two is one of the most common conceptual errors in applied cryptography.

True (physical) randomness comes from a physical process believed to be genuinely unpredictable – thermal noise across a resistor, jitter between independent clock oscillators, the timing of radioactive decay, the precise arrival times of network packets and keystrokes, shot noise in a reverse-biased diode. The defining feature is that the unpredictability is a property of the *world*, not of any algorithm.

There is no formula an attacker could run to reproduce it, because it was never computed. The downside: physical sources are slow, biased, and hard to test. A noisy diode might give you a few kilobits per second of raw entropy, heavily skewed toward 0 or 1, that you then have to clean up.

Pseudorandomness comes from a deterministic algorithm: feed in a short *seed* and it stretches it into an arbitrarily long stream of bits that *look* random to anyone who does not know the seed. The same seed always produces the same stream – that determinism is a feature (reproducibility, speed) and a danger (predictability if the seed leaks). A pseudorandom generator manufactures no new entropy. If the seed has 128 bits of min-entropy, the gigabyte of output it produces still has only 128 bits of min-entropy; the generator has spread that unpredictability thin, not created more.

Intuition

Think of true randomness as mining raw ore and pseudorandomness as stamping coins. The mine (your diode, your interrupt timings) produces a trickle of irregular, impure metal – real value, but slow and ugly. The mint (your CSPRNG) takes a lump of that metal as a seed and stamps out millions of identical-looking coins fast and cheaply. The mint creates no new gold; it only spreads the gold from the seed across all the coins. Spend the seed's gold twice – reuse or leak the seed – and every coin you ever stamped from it is counterfeit. So a real system uses both: gather enough true entropy *once* to seed the mint, then let the mint do the bulk production.

Formal

A *pseudorandom generator* (PRG) is a deterministic, efficiently computable function

$$G : \{0,1\}^n \;\to\; \{0,1\}^{\ell(n)}, \qquad \ell(n) > n,$$

that maps an n-bit seed to a longer $\ell(n)$-bit output (it "stretches" the seed). The output is not random – it ranges over at most 2^n of the $2^{\ell(n)}$ possible strings, a vanishing fraction. The goal is to make that tiny sparse subset *look* indistinguishable from the full uniform distribution to any efficient observer. We make "look indistinguishable" precise in the next section.

Common trap

"It passes the statistical tests, so it is secure." Statistical test batteries (Diehard, TestU01, NIST SP 800-22) check whether output is *distinguishable from random by generic statistics* – bias, runs, correlations. Passing them is necessary but nowhere near sufficient for cryptography. A linear congruential generator, $x_{i+1} = (ax_i + c) \bmod m$, can pass many such tests, yet given a handful of consecutive outputs you can solve for a, c, m and predict every future and past value with linear algebra. The Mersenne Twister, default in Python's `random` and many languages, passes essentially every statistical test and is still completely predictable: observe 624 consecutive 32-bit outputs and you can reconstruct its entire internal state and clone it. Statistical randomness is a property of the *output*; cryptographic randomness is a property of the output *relative to an adversary who analyzes the generator.*

Example 17.1 (Breaking an LCG with two outputs)**.** Let us verify that claim on a concrete example. Take an LCG $x_{i+1} = (ax_i + c) \bmod m$ with $m = 2^{31}$ and *unknown* multiplier a and increment c. Say we observe three consecutive outputs $x_0 = 5$, $x_1 = 33$, $x_2 = 117$. Two unknowns, so two equations

suffice:

$$33 \equiv 5a + c, \qquad 117 \equiv 33a + c \pmod{2^{31}}.$$

Subtract the first from the second to kill c:

$$117 - 33 \equiv (33 - 5)\, a \implies 84 \equiv 28\, a \pmod{2^{31}}.$$

Now $28 = 4 \cdot 7$ and $84 = 4 \cdot 21$, and $\gcd(4, 2^{31}) = 4$ divides 84, so we can divide the congruence by 4: $21 \equiv 7a \pmod{2^{29}}$. The inverse of 7 modulo 2^{29} exists (7 is odd), and $7^{-1} \equiv 383479223 \pmod{2^{29}}$, giving $a \equiv 21 \cdot 383479223 \equiv 3 \pmod{2^{29}}$, hence $a = 3$. Back-substitute: $c = 33 - 5 \cdot 3 = 18$. Predict the next value: $x_3 = (3 \cdot 117 + 18) \bmod 2^{31} = 369$. We recovered the entire generator and all of its future and past output from *three* samples and a couple of modular operations. Better statistical behavior does not help: the single affine update that makes an LCG fast is also what lets two equations reveal its parameters. Cryptographic security is about resisting an adversary who solves equations, not one who runs chi-squared tests.

17.3 What Makes a Generator *Cryptographically* Secure

An ordinary PRG (good enough for a physics simulation) and a *cryptographically secure* PRG (CSPRNG) are held to different standards. The simulation only needs output that is unbiased and uncorrelated enough that statistical estimates converge. A CSPRNG must defeat an active, intelligent adversary who knows the algorithm, sees part of the output, and is trying to predict the rest. Two extra properties separate the two.

Definition 17.1 (Next-bit unpredictability)**.** A generator is *next-bit unpredictable* if no efficient (PPT) algorithm, given the first i output bits, can guess bit $i+1$ correctly with probability noticeably better than $1/2$. Formally, for every PPT predictor A and every i,

$$\Pr\big[\, A(b_1 b_2 \cdots b_i) = b_{i+1} \,\big] \;\le\; \tfrac{1}{2} + \mathsf{negl}(n).$$

Definition 17.2 (Indistinguishability / CSPRNG)**.** A PRG G is a *CSPRNG* if its output on a uniform seed is computationally indistinguishable from a truly uniform string of the same length: for every PPT distinguisher D,

$$\Big|\, \Pr_{s \xleftarrow{\$} \{0,1\}^n}\big[D(G(s)) = 1\big] \;-\; \Pr_{r \xleftarrow{\$} \{0,1\}^\ell}\big[D(r) = 1\big] \,\Big| \;\le\; \mathsf{negl}(n).$$

D is handed a string that is either real G output or true coin flips, and must decide which. If nobody efficient can do better than a coin flip, the generator passes.

These two definitions look different but are equivalent, a clean and famous result.

Theorem 17.1 (Yao, 1982)**.** *A PRG is next-bit unpredictable if and only if it is indistinguishable from uniform.*

Why it works

Why next-bit unpredictability already gives you indistinguishability. One direction is intuitive, and it is the reason "predict the next bit" is the right informal test. Suppose

some distinguisher D *can* tell G's output from random with advantage ε. Build a next-bit predictor from it via a *hybrid argument*. Consider strings that are "real for the first i bits, then random for the rest," as i slides from 0 (fully random) to ℓ (fully real). D behaves differently at the two ends by ε, so by averaging there must be some position i where swapping bit i from random to real shifts D's output by at least ε/ℓ. That single sensitive position is a leak: it means bit i is *correlated* with the earlier real bits in a way D detects. Run D, and when it leans toward "real," bet that bit i matched the prediction; when it leans "random," bet the opposite. That betting rule predicts bit i from bits $1 \dots i{-}1$ with advantage ε/ℓ over chance. So "no distinguisher" and "no predictor" stand or fall together: any statistical edge an adversary finds can be sharpened into a prediction, and any prediction is a statistical edge. This is why we can design CSPRNGs by asking only the local question – "can you guess the next bit?" – and get the global guarantee for free.

For a cryptographic generator we demand more than even these, because a CSPRNG holds long-lived internal state and the attacker may eventually compromise it. Two state-compromise properties:

Backtracking resistance (forward secrecy). An adversary who learns the generator's internal state at time t *cannot* recover bits it output *before* t. Past output stays secret even after a state leak. This is achieved by updating the state through a one-way step so old states cannot be reconstructed from new ones.

Prediction resistance / recovery (backward secrecy). After a state compromise, if the generator is *reseeded* with fresh entropy, its future output becomes unpredictable again – the attacker who saw the old state is locked back out. Without reseeding, a state compromise is permanent: a deterministic generator with known state has no secrets left.

Intuition

Backtracking resistance is a one-way ratchet on the state: each tick grinds the previous state through a hash-like step you cannot run backward, so capturing today's state tells you nothing about last week's output. Prediction resistance is a re-locking mechanism: stir in fresh physical entropy and the attacker who photographed your state is suddenly behind a new lock they never saw. A good CSPRNG has both – it forgets its past on every step and can lock out an intruder on every reseed.

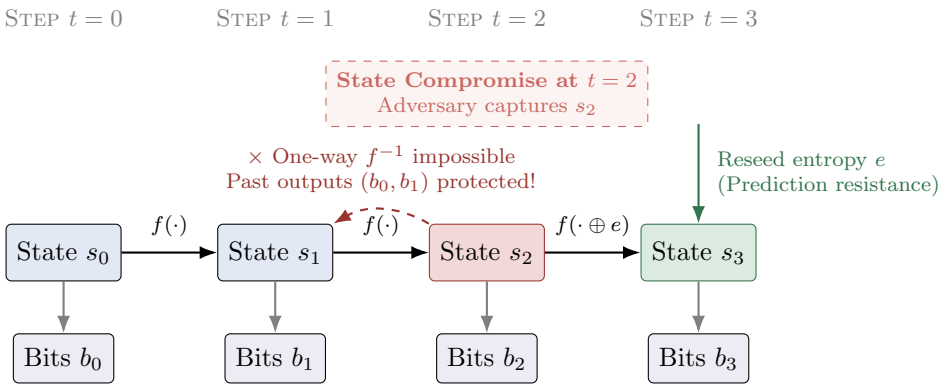


Figure 17.1: CSPRNG state evolution and compromise properties. The one-way transition f prevents inverting back from a leaked state s_2 to recover past states (s_1, s_0), providing backtracking resistance (forward secrecy). Folding in fresh entropy e at s_3 locks out the attacker moving forward, providing prediction resistance.

Example 17.2 (A CSPRNG you already have: AES in counter mode)**.** You can turn any secure block cipher into a CSPRNG. Take a random 128-bit key k (the seed) and a counter `ctr` starting at 0, and output

$$G(k) \;=\; \mathrm{Enc}_k(0) \,\|\, \mathrm{Enc}_k(1) \,\|\, \mathrm{Enc}_k(2) \,\|\, \cdots$$

Each block is the cipher applied to a fresh counter value. If AES is a secure pseudorandom permutation – which is exactly the assumption Chapter 12 rests on – then its outputs on distinct inputs are indistinguishable from independent random blocks, so G is a CSPRNG. This construction is used in practice: it is essentially the core of the CTR_DRBG construction in NIST SP 800-90A, the most widely deployed CSPRNG family on the planet. Backtracking resistance is added by periodically replacing k with fresh cipher output and discarding the old key.

17.4 Where Entropy Actually Comes From

A CSPRNG is a deterministic mint; it needs a seed of real, physical, unpredictable bits to stamp. Where does a computer – a machine engineered to be perfectly deterministic and repeatable – find unpredictability? It collects it from the few places where physical processes affect the machine's behavior.

Hardware noise sources. Modern CPUs ship a dedicated entropy source. Intel's `RDRAND/RDSEED` and AMD's equivalent sample thermal noise in a metastable latch (a circuit deliberately balanced on a knife edge so noise tips it 0 or 1). This is the fastest true-random source available, but it is opaque silicon you cannot audit – which, as we will see, makes some people nervous.

Interrupt and device timing. The operating system records the precise cycle counter at each hardware interrupt: disk seeks completing, network packets arriving, keys pressed, mouse moved. The low bits of these timestamps are influenced by chaotic physical timing (head turbulence, thermal drift, human muscle jitter) and are hard to predict even for code running on the same machine.

Uninitialized and chaotic state. Historically, things like the contents of uninitialized memory or fine-grained scheduling races were tapped. These are weak and discouraged now – they can

be far more predictable than they look, especially right after boot. (One such trick caused the Debian disaster of §17.6.1.)

The raw bits from any of these are *biased* and *correlated* – a diode might emit 0 sixty percent of the time, interrupt timestamps share high bits. You cannot use them directly as a key. You run them through an *entropy extractor* (in practice a cryptographic hash or a keyed construction) that compresses many biased input bits into fewer, nearly uniform output bits.

Why it works

Why hashing concentrates entropy. Suppose you collect 1000 raw bits from a biased, correlated source, but you can argue they jointly contain at least 256 bits of min-entropy – meaning no particular 1000-bit pattern has probability above 2^{-256}. Feed them into SHA-256. The output is 256 bits. The *leftover hash lemma* (and in practice the random-oracle heuristic) tells us that hashing a source with k bits of min-entropy down to somewhat fewer than k bits yields output statistically close to uniform. Intuitively, a good hash spreads its input chaotically across all 2^{256} outputs, so the input's unpredictability – however lumpily distributed across the raw bits – gets stirred into near-uniform unpredictability across the digest. The catch is the bookkeeping: you must *conservatively estimate* how much min-entropy the raw input really carries, because if you over-credit it (assume 256 when it is really 40), the polished output looks uniform but is only 40 bits strong. Entropy estimation is the hardest and most error-prone part of the pipeline.

Connection

The leftover hash lemma is a probability result: if a source X has min-entropy $\geq k$ and H is drawn from a universal hash family with output length $m \leq k - 2\log_2(1/\varepsilon)$, then $(H, H(X))$ is within statistical distance ε of uniform. The $2\log_2(1/\varepsilon)$ gap is the "entropy tax" you pay for a provable guarantee – you must throw away a couple dozen bits to push the statistical distance down to cryptographic levels. The companion probability book derives this from the collision-entropy (H_2) bound; it is the same birthday-style counting that governs hash collisions.

Example 17.3 (Estimating min-entropy from a biased coin)**.** Suppose your only entropy source is a coin that lands heads with probability $p = 0.6$, flips independent. How many flips to seed a 128-bit key? The min-entropy *per flip* is governed by the more likely outcome, heads:

$$H_\infty(\text{one flip}) = -\log_2(0.6) \approx 0.737 \text{ bits.}$$

For n independent flips the min-entropy adds: $0.737\,n$ bits. To reach 128 bits you need

$$n \;\geq\; \frac{128}{0.737} \approx 174 \text{ flips.}$$

Notice you need *more* than 128 flips even though each flip is one bit – bias costs you. A naive engineer who assumed "128 flips give 128 bits" would seed with only $128 \times 0.737 \approx 94$ bits of real strength, a 34-bit shortfall that turns a 2^{128} search into a 2^{94} one. And 0.737 is the *optimistic* estimate that assumes perfect independence; real sources are correlated, so you discount further.

Example 17.4 (Sizing an extractor with the leftover-hash tax)**.** Now run those biased flips through an extractor and see what the bookkeeping costs. From Example 17.3, 174 flips of the $p = 0.6$ coin

carry about 128 bits of min-entropy. We want output within statistical distance $\varepsilon = 2^{-64}$ of uniform – a comfortable cryptographic margin. The leftover hash lemma caps the safe output length at

$$m \;\le\; k - 2\log_2\frac{1}{\varepsilon} \;=\; 128 - 2\cdot 64 \;=\; 0 \text{ bits.}$$

Zero usable bits: at this ε the entropy tax eats the entire seed. To extract a 128-bit key you must instead collect enough input that $k \ge m + 2\log_2(1/\varepsilon) = 128 + 128 = 256$ bits of min-entropy, i.e.

$$n \;\ge\; \frac{256}{0.737} \approx 348 \text{ flips.}$$

So provable extraction of a 128-bit key from this source needs roughly 348 flips, not 174. In practice systems lean on the random-oracle heuristic (treat SHA-256 as ideal) to avoid paying the full $2\log_2(1/\varepsilon)$ tax, but the lesson stands: a provable guarantee costs entropy, and the gap between "looks uniform" and "provably uniform" is real bits you must harvest.

17.5 Seeding, Reseeding, and the OS CSPRNG

Put the pieces together and you get the architecture every modern operating system uses. There is exactly one true-entropy collection problem, solved once (well) by the kernel, and then a fast CSPRNG that everyone draws from.

1. **Collect.** The kernel continuously harvests raw bits from interrupt timings, hardware RNG instructions, and device events into an *entropy pool*, tracking a conservative estimate of how much min-entropy the pool holds.

2. **Seed.** When enough min-entropy has accumulated (Linux waits for a threshold before declaring the pool "initialized"), the pool is hashed down to seed a CSPRNG – on Linux, a ChaCha20-based generator.

3. **Stretch.** Applications call `getrandom()` / read `/dev/urandom` / call `BCryptGenRandom`, and the CSPRNG stamps out as many bits as asked, fast.

4. **Reseed.** The kernel periodically folds fresh harvested entropy back into the CSPRNG state, providing recovery (prediction resistance) in case the state ever leaked.

In the real world

What to actually call. On Linux/modern Unix, call `getrandom()` (or read `/dev/urandom` once it is seeded); in Python use `secrets` or `os.urandom`; in Go use `crypto/rand`; in Java `SecureRandom`; in the browser `crypto.getRandomValues`; on Windows `BCryptGenRandom`. These all funnel to the OS CSPRNG. You should essentially *never* build your own generator or seed one yourself – the OS has already solved entropy collection, reseeding, and fork safety better than you will. The one caveat: at boot, before the pool is initialized, `/dev/urandom` historically returned low-entropy output without complaint. The modern fix is `getrandom()` with no flags, which *blocks* until the pool is seeded. Use it.

Common trap

`/dev/random` versus `/dev/urandom`: the myth. Folklore says `/dev/random` is "more secure" because it blocks when entropy is "low." On modern Linux this is wrong and harmful. Once the CSPRNG is seeded with enough min-entropy (a few hundred bits, once), it produces cryptographically strong output forever – entropy is not a fuel that gets "used up" as you read bits, because a CSPRNG is computational, not information-theoretic. The old blocking behavior of `/dev/random` caused real outages (servers hanging on key generation waiting for "entropy" that was a fiction) without adding security. The correct mental model: you need a sufficient seed *once*; after that, read as much as you want from the unblocking interface. The only genuine concern is the very early boot window before that first seed exists.

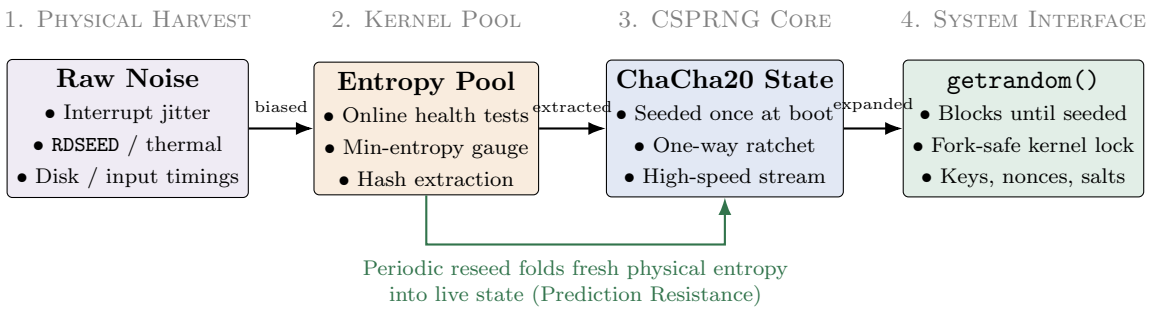


Figure 17.2: The operating system randomness pipeline. Physical noise is continually gathered into an entropy pool where online health tests and min-entropy accounting monitor source viability. Once initialized, the pool seeds a deterministic CSPRNG (such as ChaCha20 or AES-CTR), which emits unlimited high-speed cryptographic bits via `getrandom()`. Periodic reseeding restores security after a state leak.

17.5.1 Fork Safety: When One Becomes Two

A CSPRNG is deterministic: same state, same output. Usually that is fine – the state keeps advancing. But `fork()` clones a process *including its memory*, and that is exactly where determinism becomes a problem. If an application keeps a userspace CSPRNG (its own copy of the state, seeded once at startup) and then forks, parent and child now hold *identical* generator state. They will produce the *same* "random" stream – the same session keys, the same nonces – independently, each unaware of the other.

Attack

Fork-induced nonce reuse. Picture a web server that seeds an in-process CSPRNG at startup, then forks worker processes to handle connections. Every worker inherits the identical seed. Worker A and worker B each generate what they think is a fresh random nonce for AES-GCM – and produce the *same* nonce for different messages under the same key. A repeated GCM nonce is catastrophic: it leaks the XOR of the two plaintexts and, worse, lets an attacker recover the GHASH authentication key and forge tags. One careless fork turns a correct AEAD scheme into a broken one. The same bug sank some early TLS deployments and is a recurring cause of duplicate ECDSA nonces (see §17.6.3).

Why it works

Why the OS interface is fork-safe and a userspace cache is not. `getrandom()` and `/dev/urandom` go to the kernel, which has *one* CSPRNG state shared across all processes; the kernel serializes access, so parent and child reading after a fork get different bytes – there is no per-process copy to clone. A userspace library that caches generator state in process memory loses this for free. The fixes are: (1) just call the OS each time (simplest and recommended); (2) detect forks by comparing the cached PID against `getpid()` and reseed on mismatch – fragile, because PIDs wrap around and recycle; or (3) on modern Linux, register the seed buffer with `MADV_WIPEONFORK` so the child's copy is zeroed at fork, forcing a reseed before use. OpenSSL, after being hit by this bug, now mixes the PID and a fork-generation counter into its DRBG precisely to survive this.

Implement it

A toy showing the two failures and how the OS interface dodges them. **Learning only – never ship a hand-rolled generator; call your platform CSPRNG.**

```python
import os, time, hashlib

# --- TOY userspace CSPRNG (insecure illustration) -------------------
class ToyDRBG:
    def __init__(self, seed: bytes):
        self.state = hashlib.sha256(seed).digest()
    def random_bytes(self, n: int) -> bytes:
        out = b""
        while len(out) < n:
            # one-way step: hashing gives backtracking resistance
            self.state = hashlib.sha256(b"S" + self.state).digest()
            out += hashlib.sha256(b"O" + self.state).digest()
        return out[:n]

# PITFALL 1: seeding from a predictable source (the clock).
bad = ToyDRBG(seed=str(int(time.time())).encode())   # ~seconds of entropy
print("clock-seeded (guessable):", bad.random_bytes(8).hex())

# PITFALL 2: fork inherits identical state -> identical output.
g = ToyDRBG(seed=os.urandom(32))      # good seed, but cached in memory
pid = os.fork()
if pid == 0:                          # child
    print("child :", g.random_bytes(8).hex())
    os._exit(0)
else:                                 # parent
    os.wait()
    print("parent:", g.random_bytes(8).hex())  # SAME as child -- bug!

# THE FIX: ask the OS every time. Different in parent and child,
# correctly seeded at boot, no state to clone.
print("os.urandom:", os.urandom(8).hex())
```

Run it: the two `ToyDRBG` lines after the fork print *identical* hex (shared cloned state), while `os.urandom` differs. The clock-seeded line is reproducible by anyone who guesses the second

you ran it. Both bugs vanish the moment you defer to `os.urandom` / `getrandom()`.

17.5.2 Health Tests: Catching a Source That Dies

A physical entropy source is a piece of hardware, and hardware fails – a noise diode degrades, a ring oscillator locks to a fixed value, RDSEED returns all-ones because a power rail sagged. The danger is that a *stuck* source still produces output; it just produces *predictable* output. A CSPRNG dutifully stretches that into key material, and nothing downstream looks wrong. So a real entropy pipeline runs *online health tests* on the raw source, before extraction, to catch a source that has died.

Repetition Count Test. Flag if the same raw sample value repeats more than a cutoff number of times in a row – the signature of a stuck output. NIST SP 800-90B sets the cutoff from the claimed per-sample min-entropy H: roughly $C = 1 + \lceil 20/H \rceil$ for a false-positive rate near 2^{-20}. A source claiming $H = 1$ bit/sample trips after about 21 identical samples.

Adaptive Proportion Test. Over a sliding window (e.g. 512 or 1024 samples), count how often the window's first value reappears. If one value dominates beyond a binomial cutoff, the source has collapsed toward a biased or frozen state even though it is not perfectly stuck.

Why it works

Why test the raw source and not the CSPRNG output. You cannot health-test the output of a good CSPRNG – by design it looks uniform even when seeded from a totally dead source, so a statistical test on it tells you nothing about the seed. A failure is visible only *upstream*, on the raw physical samples before they are whitened. This is why the architecture separates "estimate and test the source" from "stretch with the CSPRNG": the health test guards the one stage where degradation is still detectable. Put the test after the hash and you are testing your hash function, not your diode.

17.6 Famous RNG Failures

The ideas of min-entropy and reseeding are easier to appreciate after seeing real keys recovered from broken generators. We look at three cases, each with a distinct lesson: a bug that shrank the keyspace, a backdoor baked into a standard, and a misuse that leaks the secret key directly.

17.6.1 Debian OpenSSL (2008): The 15-Bit Keyspace

Attack

What happened. In 2006 a Debian maintainer, chasing warnings from the Valgrind/Purify memory-analysis tools, commented out a line in OpenSSL's seeding code that mixed *uninitialized* buffer contents into the entropy pool. The tools complained because reading uninitialized memory is normally a bug; here it was deliberate entropy gathering. The edit removed essentially all the unpredictability from OpenSSL's seeding on Debian and its derivatives (Ubuntu,

etc.) for nearly two years. The PRG's only remaining real input was the process ID.
The damage. A Linux PID is at most 32,768 values. So every key OpenSSL generated on an affected system – SSH host and user keys, TLS server keys, OpenVPN keys, DSA/RSA keypairs – came from one of only about 2^{15} possible seeds (times a couple of architecture and key-size variants). Researchers simply *pre-generated every possible key.* To attack a server you did not break any math; you looked its public key up in a table of 32,768 candidates and read off the matching private key. Worse, any DSA or ECDSA *signature* ever produced on an affected machine leaked the long-term private key, because the per-signature nonce was equally low-entropy (the same flaw that powers §17.6.3). Affected keys had to be revoked and regenerated worldwide.

Intuition

Nothing was wrong with the mathematics: RSA was as hard as ever and AES was unbroken. The keys were weak because the seed – the one input that must be unpredictable – had about 15 bits of entropy instead of 128+. A 2048-bit RSA key with 15 bits of seed entropy is, to an attacker with the table, a 15-bit key. This incident illustrates the main point of this chapter: *your cryptography is only as strong as the unpredictability of its random seed.*

Common trap

The meta-lesson: do not silence the tool by gutting the security. The change looked like a clean fix for a memory-safety warning. The author had no way to know that the "uninitialized read" was an essential source of entropy. The takeaway is twofold: security-critical code needs comments explaining clearly *why* unusual-looking lines exist, and changes to crypto libraries need review by people who understand the crypto, not just the C. "It still compiles and the warning is gone" is not a passing test for a CSPRNG.

17.6.2 Dual_EC_DRBG (c. 2006–2013): A Standardized Backdoor

History

`Dual_EC_DRBG` was one of four CSPRNGs in NIST SP 800-90A (2006), alongside Hash_DRBG, HMAC_DRBG, and CTR_DRBG. It generates pseudorandom bits using elliptic-curve point multiplication over two fixed public points P and Q on a NIST curve. It was always the odd one out: orders of magnitude slower than its siblings, with a measurable output bias. Nobody could explain why such a slow, slightly broken generator was in the standard at all. At the Crypto 2007 rump session, Dan Shumow and Niels Ferguson showed that the relationship between P and Q could hide a *backdoor*: whoever chose the points could hold a secret scalar d with $Q = d \cdot P$ that turns one observed output into the generator's internal state. In 2013, leaks revealed that the NSA had pushed the design into NIST and paid RSA Security $10 million to make it the default in its BSAFE library. NIST withdrew `Dual_EC_DRBG` in 2014.

Attack

How the backdoor works (the shape of it). The generator's state s advances and emits bits derived from two EC points. Output block r is computed essentially as $r = x\big(s \cdot Q\big)$ – the x-coordinate of the point $s \cdot Q$ – truncated by a few bits. P and Q are fixed public points, but suppose they were chosen so that $Q = d \cdot P$ for a secret scalar d known only to the designer. The state update uses P; the output uses Q. Someone who knows d can take an observed output r, recover the full point $s \cdot Q$ (only a few truncated bits to brute-force, since the curve equation pins down the candidates), and then – because $Q = dP$ – compute $d^{-1}(s \cdot Q) = s \cdot P$, which is precisely the value used to derive the *next* state. With the next state in hand, the backdoor holder predicts all future output. A single generator output therefore gives the holder of d every later key and nonce.

Figure 17.3 traces the asymmetry: normal advancement moves left-to-right from state s_t through $s_t \cdot P$ to the next state s_{t+1}, while anyone holding the secret trapdoor d^{-1} (where $Q = dP$) can invert the published output $x(s_t \cdot Q)$ upward into $s_t \cdot P$ and compute s_{t+1} directly.

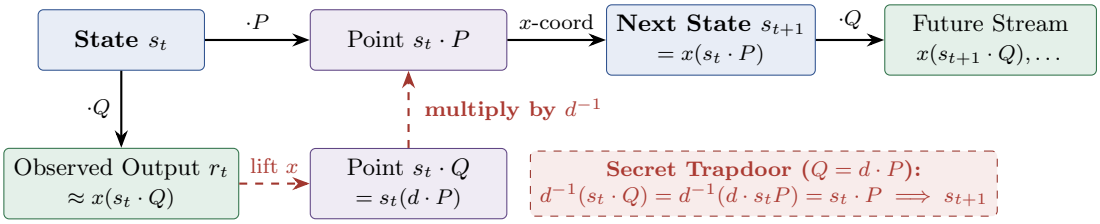


Figure 17.3: The `Dual_EC_DRBG` kleptographic backdoor. An observer who knows the secret discrete logarithm d relating the standard points $Q = d \cdot P$ lifts one observed output r_t to the curve point $s_t \cdot Q$ and multiplies by d^{-1} to recover $s_t \cdot P$, exposing the next internal state s_{t+1} and all subsequent outputs.

Why it works

Why this case is the most troubling. Debian was a bug – a mistake anyone could have made and anyone could find by reading the diff. `Dual_EC` was, by all available evidence, a deliberate, *mathematically invisible* backdoor in a published international standard, exploitable only by whoever generated the constants P and Q. To everyone else it looked like an ordinary (if slow) CSPRNG; you could audit the algorithm line by line and find nothing, because the secret was not in the code but in the *provenance of two numbers*. This is the origin of the modern demand for *nothing-up-my-sleeve* constants: cryptographic parameters must be derived from public, arbitrary values (digits of π, hashes of fixed strings) so that no designer could have chosen them to hide a trapdoor. When you cannot explain *where a constant came from*, treat it as hostile.

Focus

The practical takeaway is not to avoid elliptic curves – ECC is excellent and Chapter 21 is devoted to it. Instead, *prefer simple, fast, auditable generators with transparently derived constants*, and be suspicious of any primitive that is mysteriously slow, mysteriously biased, or mysteriously favored. CTR_DRBG (AES) and the ChaCha20-based and HMAC_DRBG

constructions show none of these warning signs; use them.

17.6.3 Repeated Nonces in Digital Signatures: The Secret Key Falls Out

The previous two failures weakened a generator. This one shows what a weak generator *does* to a real protocol – and it is the most direct private-key leak in the book. Digital signatures such as DSA and ECDSA (which we study in Chapter 22) need a fresh secret random value, the *nonce* k, for every signature. Reuse k once, or even make it slightly predictable, and your long-term private key is recoverable from public signatures using only the modular arithmetic of Chapters 3 and 4.

To sign a message hash h modulo a prime n with long-term private key $x \in \{1, \ldots, n-1\}$, the signer picks a per-signature secret nonce $k \in \{1, \ldots, n-1\}$, derives a public commitment r deterministically from k, and computes

$$s \equiv k^{-1}\,(h + x\,r) \pmod{n}.$$

The signature is (r, s), and r, s, h are public. The secret is x; k must also stay secret *and* be fresh.

Attack

Two signatures, one nonce, and the key is yours. Suppose Alice signs two different messages with hashes $h_1 \neq h_2$ but – through a broken RNG, a fork, or a fixed "random" value – reuses the *same* secret nonce k. Because the commitment r depends solely on k, both signatures share the same r, which is the visible tell that this happened. Write the two s equations:

$$s_1 \equiv k^{-1}(h_1 + xr), \qquad s_2 \equiv k^{-1}(h_2 + xr) \pmod{n}.$$

Subtract: $s_1 - s_2 \equiv k^{-1}(h_1 - h_2) \pmod{n}$, so

$$\boxed{k \equiv \frac{h_1 - h_2}{s_1 - s_2} \bmod n}.$$

Now k is known. Plug it back into $s_1 \equiv k^{-1}(h_1 + xr) \pmod{n}$ and solve for the private key:

$$\boxed{x \equiv \frac{s_1\,k - h_1}{r} \bmod n}.$$

Two public signatures with a repeated nonce, a handful of modular operations, and Alice's long-term private key is exposed (Figure 17.4). The attack needs no lattices, factoring, or brute force, only two linear equations in two unknowns.

Example 17.5 (Working it with tiny numbers)**.** Work in modular arithmetic modulo the prime $n = 23$ (where we invert mod 23 using the Euclidean algorithm from Chapter 4). Let the signer's secret key be $x = 7$ and the reused secret nonce be $k = 4$, with public commitment $r = 9$. Sign two message hashes $h_1 = 5$ and $h_2 = 11$. We need k^{-1} mod 23: since $4 \cdot 6 = 24 \equiv 1$, we have $k^{-1} = 6$.

$$s_1 = 6\,(5 + 7 \cdot 9) = 6\,(5 + 63) = 6 \cdot 68 = 408 \equiv 408 - 17 \cdot 23 = 17 \pmod{23},$$

$$s_2 = 6\,(11 + 63) = 6 \cdot 74 = 444 \equiv 444 - 19 \cdot 23 = 7 \pmod{23}.$$

The attacker sees $(r, s_1, h_1) = (9, 17, 5)$ and $(r, s_2, h_2) = (9, 7, 11)$ – same r, so a reused nonce. Recover k:

$$k = \frac{h_1 - h_2}{s_1 - s_2} = \frac{5 - 11}{17 - 7} = \frac{-6}{10} \pmod{23}.$$

Now $-6 \equiv 17$ and we need $10^{-1} \bmod 23$: $10 \cdot 7 = 70 \equiv 1$, so $10^{-1} = 7$. Then $k = 17 \cdot 7 = 119 \equiv 119 - 115 = 4 \pmod{23}$ – correct, $k = 4$. Recover the private key:

$$x = \frac{s_1 k - h_1}{r} = \frac{17 \cdot 4 - 5}{9} = \frac{68 - 5}{9} = \frac{63}{9} \pmod{23}.$$

Now $63 \equiv 63 - 46 = 17$ and $9^{-1} \bmod 23$: $9 \cdot 18 = 162 \equiv 1$, so $9^{-1} = 18$. Then $x = 17 \cdot 18 = 306 \equiv 306 - 299 = 7 \pmod{23}$ – the master private key, recovered exactly. Trace it once by hand; the whole attack is this short.

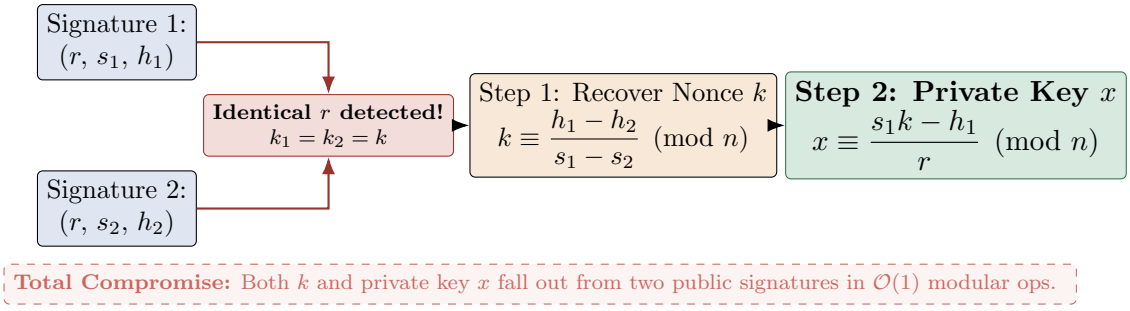


Figure 17.4: The repeated-nonce signature private key recovery. Because the commitment component r depends solely on the nonce k, two signatures with the same r immediately announce a repeated nonce. Subtracting the two signature equations cancels the private key x and reveals k; back-substituting k then extracts the signer's master private key x using only public values and basic modular arithmetic.

Attack

This is not hypothetical. The Sony PlayStation 3 (2010) signed all its software with ECDSA using a *constant* nonce – the same k for every signature. The `fail0verflow` group applied exactly the equations above to two official Sony signatures and extracted Sony's master private signing key, allowing anyone to sign code the console would trust. Android's Java `SecureRandom` had a 2013 bug that produced repeated ECDSA nonces, draining Bitcoin wallets whose keys signed transactions on affected devices – the blockchain is public, so every duplicate-r pair was sitting in the open ledger waiting to be mined for private keys. The same flaw underlay the signature leaks in the Debian incident.

Why it works

The fix: make the nonce deterministic. If a fresh random k is so dangerous to get wrong, why gamble on the RNG at all? RFC 6979 specifies *deterministic* (EC)DSA: derive k as an HMAC of the message hash keyed by the private key, $k = \mathrm{HMAC}_x(h)$, run through a generation loop. This k is unpredictable to anyone without x (HMAC is a PRF), is automatically distinct for distinct messages, and requires *no entropy at signing time*, so a broken or forked RNG cannot cause a repeat. Modern signing libraries default to it. EdDSA (Ed25519) bakes

the same idea into its design: the nonce is a hash of the message and a secret key half, deterministic by construction. The lesson generalizes: when you can replace a fragile randomness requirement with a deterministic, key-bound derivation, do it.

Connection

The recovery in Example 17.5 needed only that n be prime so inverses exist mod n – the extended Euclidean algorithm computes each $k^{-1}, 10^{-1}, 9^{-1}$ in a few steps. Even *partial* nonce leakage is fatal: if only the top few bits of each k are predictable (a biased RNG), an attacker collecting a few hundred signatures recovers x via lattice reduction (the "hidden number problem," solved with LLL). Predictability does not have to be total to be lethal; a slow leak across many signatures is enough.

17.7 Getting Randomness Right on a Real System

The checklist below pulls the chapter together. The good news is that the correct path is also the easy one: lean on the OS, and resist the urge to be clever.

Use the OS CSPRNG, period. Rely directly on platform APIs like `getrandom()`, `os.urandom`, `secrets`, `crypto/rand`, `SecureRandom`, or `BCryptGenRandom`. The kernel handles collection, extraction, reseeding, and fork safety. Do not reimplement it.

Never seed a CSPRNG from time, PID, or counters. Those are the Debian and clock-seeding failures. The seed must come from physical entropy via the OS.

Never use non-cryptographic PRNGs for secrets. Functions like `rand()`, `random()`, Mersenne Twister, or `java.util.Random` are fast and statistically fine, but *completely* predictable to an adversary. Keys, nonces, IVs, salts, and session tokens all demand a vetted CSPRNG.

Handle the boot window. Before the pool is seeded, randomness is weak. Use the blocking `getrandom()` (no `GRND_NONBLOCK`); in VMs and embedded devices, supply a seed via the hypervisor (virtio-rng) or a saved seed file, because freshly booted headless boxes have famously little entropy.

Be fork-aware. If you must cache generator state in userspace, reseed after `fork()`. Simpler: call the OS each time and let the kernel handle it.

Make nonces robust. Prefer deterministic nonces (RFC 6979, Ed25519) for signatures; for AEAD, use a counter or a sufficiently long random nonce (XChaCha20's 192-bit nonce makes random collisions negligible) – never a short random nonce generated by a possibly-broken RNG.

Mistrust opaque primitives. Avoid primitives like `Dual_EC_DRBG`. Prefer generators with transparent, nothing-up-my-sleeve constants. If you query hardware instructions like `RDRAND`, mix the output with OS entropy rather than trusting opaque silicon alone.

In the real world

In the protocols you know. TLS 1.3 draws the client and server random fields, the ephemeral ECDHE private keys, and session-ticket secrets from the OS CSPRNG; a predictable client-random or ECDHE key would let an eavesdropper derive the session keys. WiFi's WPA3 SAE and the older WPA2 four-way handshake need unpredictable nonces (the *A*nonce/*S*nonce) – the KRACK family of attacks abused nonce *reuse* forced by handshake replays, the same nonce-reuse theme from §17.6.3. Signal generates fresh ephemeral Diffie-Hellman keys for its Double Ratchet on every message; a compromised RNG there would break forward secrecy. In every case, the protocol's careful security proof silently assumes a uniform random tape, and the OS CSPRNG is what supplies it.

Focus

The chapter can be summarized in one sentence: *collect real physical entropy once, seed a good CSPRNG with it, then stretch – and let the operating system do all three for you.* Every disaster in §17.6 came from violating some part of that sentence – a seed without entropy (Debian), a CSPRNG you cannot trust (Dual_EC), or output reused where it had to be fresh (nonce reuse). If the random bits are weak, the mathematics in the rest of the book protects nothing.

Connection

This chapter brings Part II to a close. Every tool we have built so far – stream ciphers, block ciphers, hash functions, MACs, authenticated encryption, and CSPRNGs – belongs to *symmetric* cryptography, where Alice and Bob already share a secret key. In Part III, starting with Chapter 18, we ask how two strangers can establish a shared secret over a public channel monitored by an eavesdropper.

17.8 Exercises

Exercise 17.1 (Min-entropy versus length)**.** A token generator produces 128-bit values, but internally it picks a random English word from a 50,000-word dictionary and hashes it with SHA-256 to get the token. The output is 128 bits. What is the min-entropy of a token? How many guesses does the best attacker need, and how does that compare to the 2^{128} the length suggests? State the general lesson in one sentence.

Exercise 17.2 (Biased-source seeding)**.** A hardware source emits independent bits, each 1 with probability $p = 0.75$. (a) Compute the min-entropy per bit. (b) How many bits must you collect to seed a 256-bit key with full strength? (c) Compare to the Shannon entropy per bit $-(p \log_2 p + (1-p)\log_2(1-p))$, explain why the two numbers differ, and say which one you must use.

Exercise 17.3 (Predicting an LCG)**.** A linear congruential generator outputs $x_{i+1} = (ax_i + c) \bmod m$ with unknown a, c and known $m = 2^{31}$. You observe consecutive outputs $x_0 = 5$, $x_1 = 33$, $x_2 = 117$. Set up the two equations $x_1 \equiv ax_0 + c$ and $x_2 \equiv ax_1 + c \pmod m$, solve for a and c, and predict x_3. Why does this show an LCG is unusable for cryptography even though its output may pass statistical tests?

Exercise 17.4 (Break a reused signature nonce)**.** Work in modular arithmetic modulo the prime $n = 29$. A signer with unknown private key x reuses an ephemeral nonce k, producing commitment $r = 13$ on both signatures, with message hashes and signature components $(h_1, s_1) = (7, 21)$ and $(h_2, s_2) = (18, 5)$. Recover k and then the master key x using the modular elimination formulas of §17.6.3. Show each modular inverse mod 29 you compute using the Euclidean algorithm from Chapter 4.

Exercise 17.5 (The Debian keyspace)**.** On the affected Debian systems the effective seed was essentially the process ID (max 32,768) plus a small number of architecture/key-size variants. Estimate the total number of distinct RSA-2048 keys an attacker would need to pre-generate to cover, say, 4 architectures and 2 key sizes. Roughly how long would building such a table take if generating one key costs 0.1 s? What does this tell you about the relationship between key length and seed entropy?

Exercise 17.6 (Fork detection)**.** Sketch (in pseudocode) a userspace CSPRNG wrapper that stays safe across `fork()` by caching the PID and reseeding when `getpid()` changes. Then identify a concrete scenario where PID-comparison fails to detect a fork (hint: PIDs are finite and get recycled), and explain why "just call the OS each time" avoids the problem entirely.

Exercise 17.7 (Why deterministic nonces are safe)**.** RFC 6979 sets $k = \mathrm{HMAC}_x(h)$ where h is the message hash and x the private key. (a) Argue why an attacker who does not know x cannot predict k. (b) Argue why two *different* messages cannot accidentally share a nonce. (c) Argue why this construction needs zero entropy at signing time, and why that is exactly what makes it immune to the failures in §17.6.3. (d) Could an attacker who learns one nonce k recover x? Relate your answer to the recovery formula.

Exercise 17.8 (Nothing-up-my-sleeve)**.** The `Dual_EC_DRBG` backdoor lived in the relationship $Q = dP$ between two "random-looking" public points. (a) Explain why publishing d would have defused the backdoor. (b) Propose a procedure for generating P and Q that would convince a skeptic no trapdoor could be hidden, using only public, arbitrary inputs. (c) Why does deriving constants from, say, the binary digits of $\sqrt{2}$ help, and what residual assumption are you still making about the designer's choices?

Exercise 17.9 (State compromise)**.** Define backtracking resistance and prediction resistance precisely. (a) For the toy hash-chain DRBG in the §17.5.1 code (`state = SHA256("S" + state)`), argue informally why it has backtracking resistance – learning the current state does not reveal past output. (b) Show it has *no* prediction resistance without reseeding: an attacker who learns the state predicts all future output. (c) What minimal change restores prediction resistance?

Exercise 17.10 (The leftover-hash tax)**.** A source delivers 200 bits of min-entropy. (a) Using $m \le k - 2\log_2(1/\varepsilon)$, what is the longest output you can extract that is within $\varepsilon = 2^{-80}$ of uniform? (b) You need a 128-bit key at that ε; how much source min-entropy must you collect? (c) Why does the random-oracle heuristic let practical systems ignore most of this tax, and what do they give up in exchange? Relate your answer to Example 17.4.

Exercise 17.11 (A stuck source)**.** A hardware noise source claims $H = 0.5$ bits of min-entropy per raw sample. (a) Using the Repetition Count cutoff $C = 1 + \lceil 20/H \rceil$, after how many identical consecutive samples does the test fire? (b) Explain why running this test on the *raw* samples catches a frozen diode but running an equivalent test on the post-SHA-256 CSPRNG output would not. (c) A reviewer proposes "just run NIST SP 800-22 on `/dev/urandom` nightly to confirm the RNG is healthy." Explain why this provides essentially no assurance about the entropy source.

Exercise 17.12 (Birthday bound on random nonces)**.** You generate random 96-bit nonces for AES-GCM under a fixed key. (a) Using the birthday approximation, roughly how many nonces can you generate before the probability of any collision reaches 2^{-32}? (b) Why is a collision catastrophic for GCM specifically? (c) How does switching to XChaCha20-Poly1305's 192-bit nonce change the calculation, and why does that make random nonce generation safe again?

Part III

Public-Key Cryptography

Chapter 18

The Public-Key Idea and Diffie-Hellman

In Chapter 17 we saw how to harvest physical entropy and stretch it into unpredictable random keys. Yet throughout Parts I and II, every system we constructed rested on an unstated assumption: Alice and Bob already share a secret. The one-time pad needs a shared pad. AES needs a shared key. HMAC needs a shared MAC key. Authenticated encryption needs both. Every one of these schemes is a machine that turns a small shared secret into confidentiality and integrity for a large conversation. None of them tells you how the shared secret got there in the first place.

This gap was the central practical problem of twentieth-century cryptography, and for most of that century the only known answer was depressing: meet in person, or trust a courier. If you want to talk securely with n people, you need n separate keys, each one established by some out-of-band procedure. A bank with a million customers needs a million keys, each handed over face to face. Scale that to a planet of strangers buying things from each other online, and the courier model collapses entirely. You cannot fly a briefcase to every website you have never visited before.

This chapter is about the idea that broke the problem open – the idea that two people who have never met, communicating entirely over a wire that an eavesdropper reads in full, can nonetheless agree on a secret that the eavesdropper cannot compute. That sounds impossible: it is like two people shouting numbers across a crowded room and ending up with a secret nobody else in the room knows. Yet it can be done, it is not even hard, and the mathematics behind it is the same modular arithmetic we have been building since Chapter 3. The construction is Diffie–Hellman key exchange, and modern secure communication depends on it. When your browser negotiates a TLS connection, when two phones set up a Signal session, when an SSH client greets a server it has never spoken to, a descendant of Diffie–Hellman runs in the first few milliseconds.

We will build it the way we build everything: the intuition first, with a physical analogy and tiny numbers you can check by hand; then the precise protocol; then the hardness assumptions that make it secure and the careful distinction between two of them; then the attack that Diffie–Hellman does *not* stop, which is the reason the rest of public-key cryptography exists; and finally a preview of forward secrecy, the property that makes the modern, ephemeral version of the protocol so valuable.

18.1 The key-distribution problem

As before, Alice wants to talk to Bob, and Eve sits on the wire and reads every bit that passes. The new wrinkle is that Alice and Bob have *never met*. They share no key, no pad, no password. They have a public channel and nothing else.

With only the tools of the previous chapters, this is hopeless. Symmetric encryption is symmetric precisely because the same key locks and unlocks; if Alice has to send that key to Bob, she has to send it over the wire, and Eve reads it. Encrypting the key under another key only pushes the problem back a step: how did they agree on *that* key? The recursion never bottoms out. Information-theoretically, if every bit Alice sends is visible to Eve, then anything Bob can compute from those bits, Eve can compute too. Bob has no advantage over Eve. They are looking at the same transcript.

Intuition

The key-distribution problem in one sentence: Bob and Eve see exactly the same messages, so how can Bob end up knowing something Eve does not?
The resolution, which will take the rest of the section to make precise, is that Bob has one thing Eve lacks – not a secret message on the wire, but a secret he *chose* and never sent. Both Alice and Bob keep a private number to themselves and broadcast only a scrambled version of it. The key property is that their two private numbers can be combined into a shared value, but only by someone holding at least one of them. Eve holds neither.

Before the 1970s, the standard count made the problem look unwinnable. A network of n parties who all want to talk pairwise needs a distinct key for each pair, which is $\binom{n}{2} = n(n-1)/2$ keys. For $n = 1000$ that is just under half a million keys, every one of them established by some trusted, out-of-band method and stored securely forever. Add one new member and you must distribute n fresh keys to the existing crowd. Militaries and embassies lived with this – they had couriers and diplomatic pouches – but it does not scale to commerce, and it certainly does not scale to the open internet.

History

The breakthrough has a tangled history. In 1974 Ralph Merkle conceived "puzzles" that gave a quadratic advantage over an eavesdropper, and in 1976 Whitfield Diffie and Martin Hellman published "New Directions in Cryptography," which laid out the public-key concept and gave the key-exchange protocol that now bears their names (Hellman has long insisted the method should be called Diffie–Hellman–Merkle). The paper opens with a line that reads, in hindsight, like a manifesto: "We stand today on the brink of a revolution in cryptography." They were right, though open publication sparked a decade-long fight with the NSA over academic publication and export controls.
What no one outside the British signals intelligence agency GCHQ knew was that James Ellis, Clifford Cocks, and Malcolm Williamson had discovered "non-secret encryption" and a Diffie–Hellman-like exchange between 1969 and 1973. Their work was classified and stayed secret until 1997. So the idea was invented twice: once in the open, where it changed the world, and once in secret, where it sat in a vault. The open invention is the one that matters historically, because cryptography is only useful when people can actually use it.

18.2 The intuition: mixing paint

The cleanest analogy for Diffie–Hellman uses paint, and it is worth internalizing before any algebra, because the algebra is just this picture with numbers.

Suppose mixing two colors of paint is easy but *un*-mixing – recovering the original colors from a blend – is hard. Diffie–Hellman is built on that asymmetry.

1. Alice and Bob publicly agree on a common starting color, say a particular yellow. Everyone, including Eve, sees this yellow.

2. Alice picks a secret color, say red, and keeps it private. She mixes yellow + red into an orange blend and sends the orange to Bob over the public channel. Eve sees the orange but cannot un-mix it to recover Alice's red.

3. Bob picks a secret color, say blue, keeps it private, mixes yellow + blue into a green blend, and sends the green to Alice. Eve sees the green but cannot extract Bob's blue.

4. Alice stirs her secret red into the received green, while Bob stirs his secret blue into the received orange. Both arrive at the same three-color blend: *yellow + red + blue*.

Alice and Bob now share that final blend. Eve, watching the whole exchange, holds the public yellow, the orange, and the green. To reach the shared blend she would need to add either Alice's red or Bob's blue to something – but those secret colors were never transmitted, only ever mixed in. Un-mixing is hard by assumption, so Eve is stuck with two mixtures she cannot pull apart.

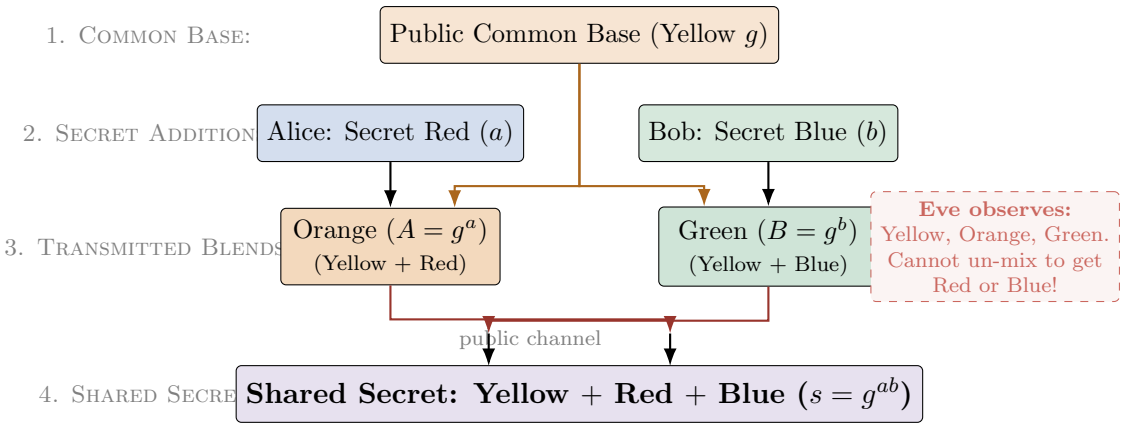


Figure 18.1: Diffie–Hellman represented as paint mixing. Both parties begin with a common public color (yellow g). Each party mixes in a private secret color and transmits the resulting mixture over the open wire. Adding their own private color to the received blend brings both parties to the identical shared three-color mixture (g^{ab}), while an eavesdropper cannot un-mix either public blend.

The remaining task is to find a numerical operation that behaves like paint: easy to mix, hard to un-mix, and one where the order of mixing does not matter, so that Alice's "green then red" equals Bob's "orange then blue." Modular exponentiation is exactly that operation, and the order-independence is just the law of exponents.

18.3 Diffie–Hellman key exchange

We work in a cyclic group, which for now you can read as "the nonzero residues mod a prime, multiplied." Fix a large prime p and a generator g of the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$. Recall from Chapter 5 that a generator is an element whose powers $g^1, g^2, g^3, \ldots$ cycle through every nonzero residue mod p before repeating; the group has order $p-1$, so $g^{p-1} \equiv 1 \pmod{p}$ and no smaller positive exponent does that. Both p and g are *public*. Eve knows them. They can even be baked into a standard and shared by millions of connections.

"Mixing the public color with a secret" becomes "raising g to a secret exponent." Here is the protocol.

Focus

Diffie–Hellman key exchange. Public parameters: a prime p and a generator g of $(\mathbb{Z}/p\mathbb{Z})^*$.

1. Alice samples a secret exponent $a \xleftarrow{\$} \{1, \ldots, p-2\}$, computes $A = g^a \bmod p$, and sends A to Bob.
2. Bob samples a secret exponent $b \xleftarrow{\$} \{1, \ldots, p-2\}$, computes $B = g^b \bmod p$, and sends B to Alice.
3. Alice computes $s = B^a \bmod p$. Bob computes $s = A^b \bmod p$.

Both arrive at the same value $s = g^{ab} \bmod p$, the shared secret.

The correctness is one line of exponent algebra:

$$B^a = (g^b)^a = g^{ba} = g^{ab} = (g^a)^b = A^b \pmod{p}.$$

Alice's secret a plays the role of her red paint, Bob's secret b is his blue, g is the public yellow, and g^a, g^b are the one-color blends sent across. The shared g^{ab} is the three-color result, and the commutativity $g^{ba} = g^{ab}$ is precisely the order-independence the paint story needed. Eve sees p, g, $A = g^a$, and $B = g^b$. To finish, she would need g^{ab}, and the entire security of the scheme is the claim that recovering g^{ab} from g^a and g^b is hard. We will name that claim shortly.

Why it works

Why can Eve not just read off a from $A = g^a \bmod p$? Because inverting modular exponentiation – finding the exponent a given g, p, and $g^a \bmod p$ – is the *discrete logarithm problem*, and for well-chosen p it is believed to be computationally infeasible. Computing $g^a \bmod p$ takes a few thousand multiplications via square-and-multiply (Chapter 3 and the companion algorithms book). Going the other way, with the best known general algorithms, takes time that grows sub-exponentially in the size of p – astronomically slow once p has a couple thousand bits. This is the same one-way asymmetry as "mixing is easy, un-mixing is hard."

18.3.1 A worked example with tiny numbers

Real Diffie–Hellman uses primes thousands of bits long. To follow each step, we shrink everything to numbers you can verify on paper.

Take $p = 23$ and $g = 5$. First, a sanity check that 5 generates $(\mathbb{Z}/23\mathbb{Z})^*$. The group has order $p - 1 = 22$. The order of 5 must divide 22, so it is one of $1, 2, 11, 22$. Squaring up: $5^2 = 25 \equiv 2$,

$5^4 \equiv 4$, $5^8 \equiv 16 \pmod{23}$. Then $5^{11} = 5^8 \cdot 5^2 \cdot 5^1 \equiv 16 \cdot 2 \cdot 5 = 160 \equiv 160 - 6 \cdot 23 = 22 \equiv -1 \pmod{23}$. Since $5^{11} \equiv -1 \neq 1$ and $5^2 \equiv 2 \neq 1$, the order is not $1, 2$, or 11; it must be 22. So 5 is a generator. Good.

Now run the protocol.

- Alice picks $a = 6$. She computes $A = 5^6 \bmod 23$. Using $5^4 \equiv 4$ and $5^2 \equiv 2$, we get $5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8$. Alice sends $A = 8$.

- Bob picks $b = 15$. He computes $B = 5^{15} \bmod 23$. Using $5^8 \equiv 16$, $5^4 \equiv 4$, $5^2 \equiv 2$, $5^1 \equiv 5$: $5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5^1 \equiv 16 \cdot 4 \cdot 2 \cdot 5$. Reduce as you go: $16 \cdot 4 = 64 \equiv 64 - 2 \cdot 23 = 18$; $18 \cdot 2 = 36 \equiv 13$; $13 \cdot 5 = 65 \equiv 65 - 2 \cdot 23 = 19$. Bob sends $B = 19$.

- Alice computes $s = B^a = 19^6 \bmod 23$. Note $19 \equiv -4$, so $19^6 \equiv (-4)^6 = 4^6$. Now $4^3 = 64 \equiv 18$, so $4^6 = (4^3)^2 = 18^2 = 324 \equiv 324 - 14 \cdot 23 = 2$. So $s = 2$.

- Bob computes $s = A^b = 8^{15} \bmod 23$. Since $8 = 5^6$ we expect $g^{ab} = 5^{90}$, and $90 \bmod 22 = 90 - 4 \cdot 22 = 2$, so $s = 5^2 = 2$. Direct check: $8^2 = 64 \equiv 18$, $8^4 = 18^2 = 324 \equiv 2$, $8^8 = 2^2 = 4$, and $8^{15} = 8^8 \cdot 8^4 \cdot 8^2 \cdot 8^1 \equiv 4 \cdot 2 \cdot 18 \cdot 8$. Reduce: $4 \cdot 2 = 8$; $8 \cdot 18 = 144 \equiv 144 - 6 \cdot 23 = 6$; $6 \cdot 8 = 48 \equiv 2$. So $s = 2$.

Both parties hold $s = 2 = g^{ab} \bmod 23$, computed by two different routes. Eve, watching the wire, knows $p = 23$, $g = 5$, $A = 8$, $B = 19$. To find s she must solve a discrete log – find a with $5^a \equiv 8$, or b with $5^b \equiv 19$. With $p = 23$ she can just try all 22 exponents, and indeed she would find $a = 6$ in a moment. That is the point of a toy: the security vanishes at small sizes. Replace 23 with a 2048-bit prime and the exhaustive search becomes a search over 2^{2047} candidates, which no machine that will ever exist can finish.

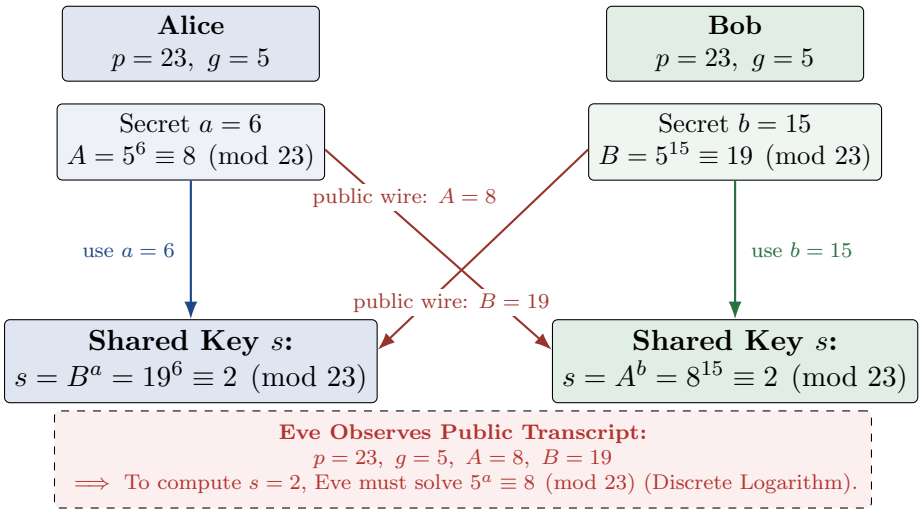


Figure 18.2: The concrete Diffie–Hellman exchange with toy numbers $p = 23, g = 5$. Alice and Bob exchange public powers $A = 8$ and $B = 19$ across the insecure channel. Each computes the shared secret $s = 2$ using their private exponent. Eve must solve the discrete logarithm problem to reconstruct s.

Implement it

A toy Diffie–Hellman in Python. This is for **learning only** – it uses a microscopic prime, samples exponents with a non-cryptographic RNG, and derives a key by naively hashing the group element. Never use anything like this for real traffic; reach for a vetted library (`cryptography`, `libsodium`) that uses standardized parameters and constant-time arithmetic.

```python
import random, hashlib

p = 23          # toy prime -- a real one is >= 2048 bits
g = 5           # generator of (Z/23Z)*

def keypair():
    x = random.randrange(2, p - 1)   # secret exponent (use os.urandom in reality!)
    X = pow(g, x, p)                 # public value g^x mod p, via fast modexp
    return x, X

a, A = keypair()                     # Alice
b, B = keypair()                     # Bob

# each side raises the OTHER's public value to its OWN secret
s_alice = pow(B, a, p)
s_bob   = pow(A, b, p)
assert s_alice == s_bob              # both get g^(ab) mod p

# turn the shared group element into a symmetric key
shared = s_alice
key = hashlib.sha256(str(shared).encode()).digest()
print("shared group element:", shared)
print("derived key (hex):", key.hex())
```

Note the symmetry of `pow(B, a, p)` and `pow(A, b, p)`: each party exponentiates the value it received by the secret it kept. Apart from choosing the parameters, that is the entire protocol.

Common trap

Three pitfalls already visible in the toy code, all fatal in practice.

Tiny or weak parameters. A small p, or a p where $p-1$ has only small prime factors, lets attackers solve the discrete log fast (the Pohlig–Hellman attack, Chapter 20, decomposes the problem across the small factors of $p-1$). Real deployments use large *safe primes* where $p = 2q+1$ for a prime q.

Bad randomness. `random.randrange` is a Mersenne Twister, not cryptographic; an attacker who can predict it recovers your secret exponent and the whole exchange falls. Use the operating system CSPRNG (Chapter 17).

Using the raw group element as a key. $g^{ab} \bmod p$ is an integer in a structured set, not a uniform bitstring. You must run it through a key-derivation function (a hash or HKDF) before using it as an AES or MAC key. The toy hashes it; production code uses HKDF with a salt and context label.

18.4 What "secure" means here: three hardness assumptions

So far "hard to un-mix" has been an informal analogy. To build a security proof on top of it, we need to state that hardness assumption precisely. Three related problems underlie Diffie–Hellman, and the differences between them matter. We state each in a cyclic group $\mathbb{G} = \langle g \rangle$ of order q – for the protocol above, $\mathbb{G} = (\mathbb{Z}/p\mathbb{Z})^*$ and $q = p - 1$, though we will see later that prime-order subgroups and elliptic-curve groups are better choices.

Definition 18.1 (Discrete logarithm problem, DLP)**.** Given g and $h = g^x$ in $\mathbb{G}$, find $x \in \{0, 1, \ldots, q-1\}$. The DLP is *hard* in $\mathbb{G}$ if every PPT algorithm solves it with only negligible probability.

Definition 18.2 (Computational Diffie–Hellman, CDH)**.** Given g, g^a, and g^b in $\mathbb{G}$ (with a, b uniform and unknown), compute g^{ab}. The CDH assumption says every PPT algorithm succeeds with only negligible probability.

Definition 18.3 (Decisional Diffie–Hellman, DDH)**.** Given g, g^a, g^b, and a fourth element z that is *either* g^{ab} *or* a uniformly random group element g^c (each with probability $1/2$), decide which. The DDH assumption says no PPT algorithm distinguishes the two cases with advantage better than negligible:

$$\left|\Pr[\,\mathcal{A}(g, g^a, g^b, g^{ab}) = 1\,] - \Pr[\,\mathcal{A}(g, g^a, g^b, g^c) = 1\,]\right| \le \mathsf{negl}.$$

These line up on a ladder of strength. If you can solve DLP you can solve CDH: recover a from g^a, then raise g^b to the a to get g^{ab}. And if you can solve CDH you can solve DDH: compute g^{ab} yourself and check whether z equals it. So

$$\text{DLP hard} \impliedby \text{CDH hard} \impliedby \text{DDH hard},$$

reading "$X \Leftarrow Y$" as "Y being hard is at least as strong an assumption as X being hard." DDH is the strongest assumption (easiest to break), DLP the weakest (hardest to break). Whether the implications reverse – whether breaking DLP is the *only* way to break CDH, for instance – is not known. In some groups they might come apart.

Why it works

Why do we need DDH and not just CDH? Because the goal of key exchange is not merely that Eve cannot *compute* the whole shared secret, but that the shared secret *looks random* to her – so that we can safely feed it into a hash and get a key with no exploitable structure. CDH only promises Eve cannot produce g^{ab} outright. DDH promises something stronger: g^{ab} is indistinguishable from a random group element even to an Eve who only wants to learn one bit about it. That indistinguishability is what lets us treat the derived key as good as random in a security proof. In short, CDH says Eve cannot compute the secret; DDH says she cannot even distinguish it from random, and security proofs need the latter.

Common trap

DDH is genuinely *false* in the full group $(\mathbb{Z}/p\mathbb{Z})^*$, and the reason is a short attack. The map $x \mapsto x^{(p-1)/2} \bmod p$ – the Legendre symbol – reveals whether x is a quadratic residue (a perfect square mod p), outputting $+1$ or -1. The QR-ness of g^{ab} is a deterministic function of the QR-ness of g^a and g^b, which Eve can read off directly. So Eve computes the residue character that the true g^{ab} must have and checks whether z matches it; a random g^c fails the check half

the time. That single leaked bit breaks DDH.
The fix is to work in a *prime-order subgroup*: pick a safe prime $p = 2q+1$ and run the protocol in the order-q subgroup of quadratic residues, where every element is a QR and the Legendre attack says nothing. This is exactly why standardized finite-field DH parameters use safe primes and restrict to the prime-order subgroup, and why elliptic-curve groups (Chapter 21), which have prime order by construction, are now preferred – DDH is believed to hold there cleanly.

Connection

The cost asymmetry that makes all of this work is quantified in the companion algorithms book. Computing $g^a \bmod p$ by square-and-multiply costs $O(\log a)$ modular multiplications – a few thousand for a 2048-bit exponent. The best known classical attack on finite-field discrete log, the general number field sieve, runs in time roughly $\exp\big(c\,(\ln p)^{1/3}(\ln \ln p)^{2/3}\big)$: sub-exponential, but still wildly out of reach for a 2048-bit prime. On elliptic curves even that sieve does not apply, and the best attacks are the generic square-root methods (Pollard's rho), costing about $\sqrt{q}$ group operations – which is why a 256-bit elliptic curve buys security comparable to a 3072-bit finite-field prime. The probability and number-theory chapters of this book supply the group theory; the algorithms book supplies the running-time bounds.

18.4.1 From shared secret to usable key

The shared value $s = g^{ab}$ is an element of $\mathbb{G}$, not a key. Three things have to happen before it can encrypt anything. First, it must be encoded as a fixed-length byte string. Second, even granting DDH, s is uniform over $\mathbb{G}$, not over all bitstrings of its length – a 2048-bit prime leaves gaps in the 2048-bit space. Third, you usually want several keys from one exchange (one for each direction, perhaps separate keys for encryption and MAC). The standard remedy for all three is a key-derivation function, almost always HKDF (Chapter 15), used as

$$k_1 \,\|\, k_2 \,\|\, \cdots = \mathrm{HKDF}\big(s,\ \text{salt},\ \text{context}\big),$$

where the context label binds the keys to this protocol and these parties. Feeding s straight into AES is a real-world bug; the KDF step is required.

18.5 The man-in-the-middle problem

Plain Diffie–Hellman has a serious weakness, and the certificates and signatures of later chapters exist largely to deal with it. Plain Diffie–Hellman defeats a *passive* eavesdropper who only reads the wire. It does nothing against an *active* attacker who can intercept and replace messages. The reason is simple: nothing in the protocol ties the value A to Alice or B to Bob. They are just numbers. Eve can substitute her own.

Attack

Man-in-the-middle on unauthenticated Diffie–Hellman.

Eve sits between Alice and Bob and relays their connection, but tampers. She runs two separate exchanges, pretending to be Bob toward Alice and Alice toward Bob.

1. Alice sends $A = g^a$, intending it for Bob. Eve intercepts it. Bob never sees it.
2. Eve picks her own secret e, computes $E = g^e$, and sends E to Bob as if it came from Alice. She also sends E to Alice as if it came from Bob.
3. Bob, seeing E, computes his shared secret $s_B = E^b = g^{eb}$, believing he shares it with Alice. Bob sends $B = g^b$ back toward Alice; Eve intercepts that too.
4. Alice, seeing E, computes $s_A = E^a = g^{ea}$, believing she shares it with Bob.
5. Eve computes both: $s_A = A^e = g^{ae}$ and $s_B = B^e = g^{be}$.

Now Eve shares one key with Alice (g^{ae}) and a different key with Bob (g^{be}). When Alice encrypts a message under s_A and sends it "to Bob," Eve decrypts it with s_A, reads it, re-encrypts it under s_B, and forwards it to Bob. Neither Alice nor Bob notices anything wrong: each completed a perfect Diffie–Hellman exchange and holds a working key. They are simply both talking to Eve. Confidentiality is gone, and so is integrity – Eve can alter messages at will before re-encrypting.

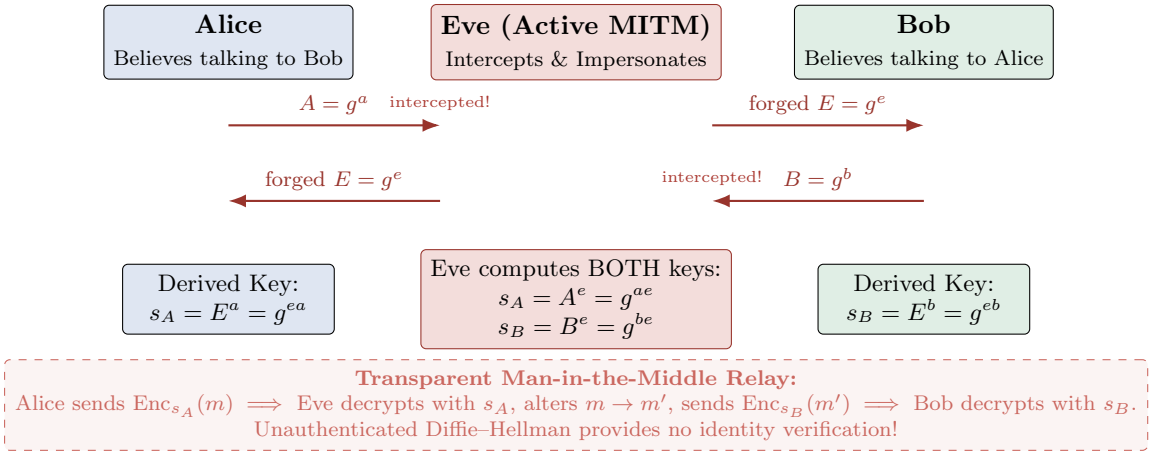


Figure 18.3: The man-in-the-middle attack on unauthenticated Diffie–Hellman. Eve intercepts public values A and B, substituting her own value $E = g^e$. She completes independent exchanges with Alice and Bob, establishing distinct shared keys s_A and s_B. Eve transparently decrypts, reads, alters, and re-encrypts all forwarded traffic.

The lesson is that key exchange and *authentication* are different problems, and Diffie–Hellman solves only the first. The exchange establishes a shared secret with *someone*; it gives no assurance about *who* that someone is. To pin down identity you need an external anchor: something Alice already knows about Bob (a public key she trusts) or a trusted third party who vouches for Bob's public value. That is the job of digital signatures (Chapter 22), certificates, and the public-key infrastructure (Chapter 24). In a real TLS handshake the server signs its Diffie–Hellman value with the private key matching the certificate your browser validates, so a substituted E would carry an invalid signature and the connection would abort. Without that signature, Diffie–Hellman alone is wide open to the attack above.

Intuition

A useful slogan: *Diffie–Hellman gets you a secure pipe to somebody; authentication tells you who is on the other end.* Both are necessary. Encrypting to the wrong party with perfect secrecy is no comfort at all.

18.6 Ephemeral Diffie–Hellman and forward secrecy

One more property explains why Diffie–Hellman – rather than, say, encrypting a fresh key under the server's RSA public key – is at the center of modern TLS. It concerns what happens *after* a conversation, when the secrets get stolen.

Distinguish two ways to use the protocol. In *static* Diffie–Hellman, a party reuses a long-term secret exponent across many sessions: the server's b is fixed, baked into its key material, used for every client. In *ephemeral* Diffie–Hellman (written DHE for finite fields, ECDHE for elliptic curves), each party generates a *fresh* random exponent for every single session and throws it away the moment the session key is derived.

The payoff is *forward secrecy* (also called perfect forward secrecy): the compromise of long-term secrets in the future does not expose past sessions. Picture an adversary who records all of Alice and Bob's encrypted traffic for years, patiently storing ciphertext she cannot yet read. Later she breaks into Bob's server and steals every long-term private key it holds. What can she decrypt retroactively?

With ephemeral Diffie–Hellman: nothing (Figure 18.4). The session key for a past conversation was g^{ab} for that session's a and b, both of which were deleted right after use. The long-term key she stole was used only to *sign* the ephemeral values for authentication, never to compute the shared secret. The recorded ciphertext stays opaque forever.

With static key transport – the old TLS mode where the client encrypts the session key under the server's long-term RSA public key – the same break is catastrophic. The session key was protected solely by the server's long-term private key, so stealing that key later lets the adversary decrypt every recorded session that ever used it. Years of recorded traffic become readable at once.

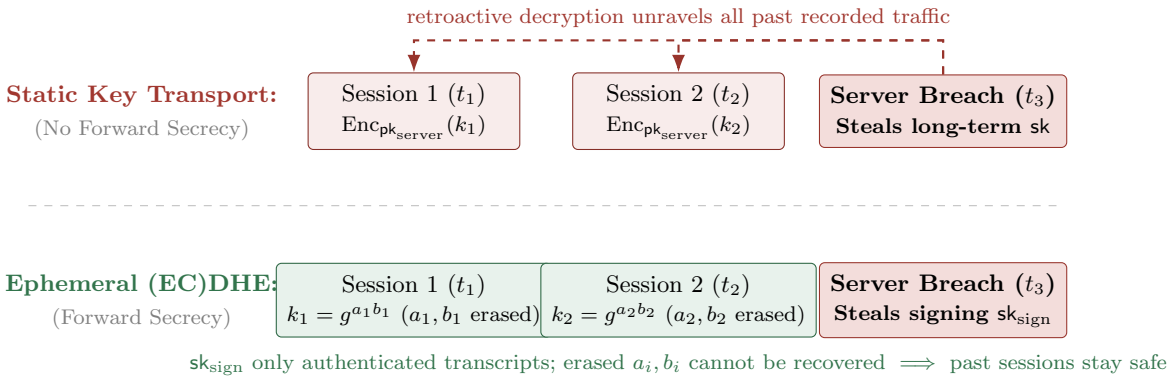


Figure 18.4: Why ephemeral Diffie–Hellman provides forward secrecy. Under static key transport (top), stealing the server's long-term decryption key sk at time t_3 lets an eavesdropper retroactively decrypt every recorded session (t_1, t_2). Under ephemeral (EC)DHE (bottom), each session derives $k_i = g^{a_i b_i}$ from one-time exponents that are immediately erased; stealing the server's long-term signing key $\mathsf{sk}_{\mathrm{sign}}$ at t_3 reveals nothing about past session keys.

Why it works

Forward secrecy works because of a deliberate separation of duties. The long-term key authenticates – it answers "who are you?" – and only that. The ephemeral exchange establishes confidentiality, using secrets that exist for one session and are then destroyed. Because the confidentiality secret is never derivable from any stored long-term value, there is nothing left for a future thief to use against an old recording. You cannot un-delete a and b; they are gone. The security of last week's call no longer depends on a key that is still sitting on a server waiting to be stolen.

In the real world

Ephemeral Diffie–Hellman is everywhere in the protocols you use daily.

TLS 1.3 (Chapter 25) removed static RSA key transport entirely. Every TLS 1.3 handshake uses ephemeral (EC)DHE, so forward secrecy is mandatory, not optional. The client offers a fresh ECDHE public value in its very first message; the server replies with its own and signs the transcript with its certificate key.

Signal (Chapter 27) goes further. Its X3DH handshake combines several Diffie–Hellman exchanges – some ephemeral, some using medium-term "prekeys" – and the Double Ratchet then performs a fresh ECDH on (almost) every message, continually replacing keys. This gives both forward secrecy and *post-compromise security*: even if a device is briefly compromised, future messages heal back to secure once the attacker loses access.

SSH (Chapter 28) and **IPsec/IKE** (Chapter 30) both run ephemeral Diffie–Hellman to set up each session, authenticated by host keys or certificates.

The standardized parameters matter too. Finite-field DHE uses the safe primes of RFC 7919; elliptic-curve ECDHE overwhelmingly uses Curve25519, whose careful design makes the implementation pitfalls of this chapter (small subgroups, bad parameters, non-constant-time arithmetic) hard to trip over.

Connection

The phrase "perfect forward secrecy" predates its TLS fame; it comes from 1990s work on authenticated key exchange (the STS protocol of Diffie, van Oorschot, and Wiener, and later the IKE design). For a long time forward secrecy was treated as a nice-to-have that cost extra computation, and many servers ran static RSA for speed. The 2013 Snowden disclosures, which revealed bulk recording of internet traffic, quickly changed this view: "record now, decrypt later" stopped being hypothetical. Within a few years ephemeral Diffie–Hellman went from optional to default to, in TLS 1.3, the only choice. The same "harvest now, decrypt later" worry now drives post-quantum migration (Chapter 35).

18.7 Looking ahead

Diffie–Hellman gave us the first piece of public-key cryptography: two strangers agreeing on a secret over an open line. But it left two threads dangling that the next chapters pick up. The man-in-the-middle attack showed that key exchange without authentication fails against an active attacker, which points to digital signatures and certificates. And the discrete-log assumption that secured the exchange turns out to be the same hard problem behind ElGamal encryption and the DSA/ECDSA signature schemes – so the structure here recurs. The very next chapter takes a different hard problem, integer factoring, and builds RSA: the other foundational public-key system, which can encrypt and sign directly rather than only exchange keys. With both DLP-based and factoring-based cryptography in hand, the machinery of TLS, Signal, and SSH can be built from parts you understand.

18.8 Exercises

Exercise 18.1. With $p = 23$ and $g = 5$ as in the chapter, suppose Alice's secret is $a = 4$ and Bob's is $b = 3$. Compute $A = g^a \bmod 23$, $B = g^b \bmod 23$, and the shared secret $s = g^{ab} \bmod 23$ two ways (as B^a and as A^b), checking that they agree.

Exercise 18.2. Take $p = 11$. Show that $g = 2$ is a generator of $(\mathbb{Z}/11\mathbb{Z})^*$ by listing the powers $2^1, 2^2, \ldots, 2^{10} \bmod 11$ and confirming they hit all of $\{1, \ldots, 10\}$. Then show that $g = 3$ is *not* a generator by finding the order of 3 mod 11.

Exercise 18.3. (Breaking a toy.) Eve observes a Diffie–Hellman exchange with $p = 23$, $g = 5$, $A = 4$, and $B = 16$. By trying exponents (or by any method you like), recover Alice's secret a and Bob's secret b, then compute the shared secret s. How many exponentiations did your search take in the worst case, and how does that count scale with the size of p?

Exercise 18.4. Prove the reduction CDH $\leq$ DLP: given a PPT algorithm that solves the discrete logarithm problem in $\mathbb{G}$, construct one that solves CDH in $\mathbb{G}$ with the same success probability. Then prove DDH $\leq$ CDH similarly. State clearly why neither reduction directly runs in the other direction.

Exercise 18.5. (The Legendre attack on DDH in $(\mathbb{Z}/p\mathbb{Z})^*$.) Let $p = 23$, so $g = 5$ is a generator and the quadratic residues mod 23 are the even powers of 5. Recall that x is a QR iff $x^{11} \equiv 1 \pmod{23}$. Given a DDH challenge (g^a, g^b, z), explain how to compute the QR character that a genuine g^{ab} must

have from the characters of g^a and g^b, and how matching z against it distinguishes a real g^{ab} from a random group element with probability $3/4$ over a single challenge. Why does restricting to the subgroup of quadratic residues kill this attack?

Exercise 18.6. Carry out the man-in-the-middle attack of Section 18.5 concretely with $p = 23$, $g = 5$. Let Alice choose $a = 6$, Bob choose $b = 15$, and Eve choose $e = 9$. Compute the public value E Eve sends to both, the key $s_A = g^{ae}$ Eve shares with Alice, and the key $s_B = g^{be}$ Eve shares with Bob. Verify that Alice and Bob each derive a key that matches one of Eve's.

Exercise 18.7. Explain in one or two paragraphs why hashing the shared group element through a KDF is necessary even when DDH holds. Give a specific example of something that could go wrong if you used the raw integer $g^{ab} \bmod p$ directly as an AES-128 key (consider its length and its distribution).

Exercise 18.8. A server runs *static* Diffie–Hellman: it uses the same secret exponent b for every client, publishing $B = g^b$ as a long-lived public value. An attacker records a year of encrypted sessions and later steals b. Show that the attacker can now recompute the shared secret of every recorded session. Then explain precisely which step of the ephemeral variant prevents this, and why deleting the ephemeral exponents is the essential step.

Exercise 18.9. (Small-subgroup pitfall, preview of Chapter 21.) Suppose $p = 23$ so $(\mathbb{Z}/p\mathbb{Z})^*$ has order $22 = 2 \cdot 11$. The element $h = 22 \equiv -1$ has order 2. Imagine Bob fails to validate Alice's incoming value and Eve sends $A = 22$ instead of a legitimate public value. Show that Bob's computed shared secret $A^b \bmod 23$ can then take only one of two values depending on the parity of b, leaking a bit of b. What general validation rule on incoming group elements prevents this class of attack?

Exercise 18.10. Estimate, using the cost figures from Section 18.4, how the work of the legitimate parties compares to the work of an attacker for a 2048-bit prime. Roughly how many modular multiplications does Alice perform to compute $g^a \bmod p$? Contrast that with the (sub-exponential) cost of the number field sieve, and explain in words why this gap is what makes the scheme usable.

Chapter 19

RSA

In Chapter 18 Alice and Bob agreed on a shared secret across a public wire using Diffie–Hellman. That is an interactive key-*agreement* protocol: both parties send a message, and the shared secret comes out of the exchange. RSA does something that looks even more surprising the first time you see it. Alice publishes a single number, her *public key*, and then *anyone in the world* can encrypt a message that only Alice can read. No round trip, no prior contact. Bob can write to Alice the way you drop a letter through a mail slot: the slot is public, the key to the box is not.

That asymmetry — public to lock, private to unlock — is what defines public-key cryptography, and RSA was the first concrete scheme that delivered it for both encryption and signatures. It is built almost entirely out of the number theory you already met in Chapters 3 and 4: modular arithmetic, Euler's theorem, and the difficulty of factoring a large integer. This chapter takes you from those ingredients to a working (toy) implementation, through a full numeric example with $p = 37$ and $q = 43$, and then shows you why the "textbook" version you just built is dangerously broken and what real systems do instead.

19.1 The setup: a one-way operation with a trapdoor

Recall the shape of a public-key encryption scheme. We need three algorithms:

- Gen: produces a key pair (pk, sk) — a public key everyone may see and a secret key only the owner holds.
- $\mathrm{Enc}_{pk}(m)$: anyone with pk can turn a message m into a ciphertext c.
- $\mathrm{Dec}_{sk}(c)$: only the holder of sk can recover m from c.

Figure 19.1 shows the asymmetry we are after. For this to be useful, encryption must be easy for everyone but inverting it must be hard for everyone *except* the holder of sk. The technical name for the object we want is a *trapdoor one-way permutation*: a function that is easy to compute, hard to invert, but easy to invert if you know one extra piece of information (the trapdoor).

Intuition

Think of a padlock you can snap shut with a push — no key needed to lock it. Snapping it open again requires the key. The "snap shut" is encryption with the public key; the "open" is decryption with the private key. Diffie-Hellman gave us a one-way function (exponentiation in a group) with no trapdoor; that is why DH needs two messages. RSA's contribution is the *trapdoor*: building the one-way function so that a secret piece of information makes inverting it easy.

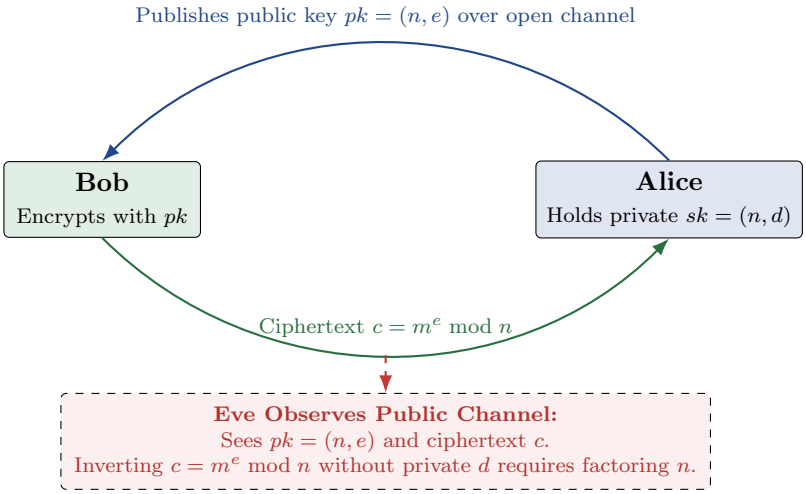


Figure 19.1: The asymmetric public-key trapdoor flow. Alice publishes public key $pk = (n, e)$, allowing anyone to encrypt messages. Decryption requires private trapdoor $d = e^{-1} \pmod{\varphi(n)}$, which only Alice can compute because only Alice knows the prime factors of n.

History

The scheme is named for Ron Rivest, Adi Shamir, and Leonard Adleman, who published it at MIT in 1977, a year after Diffie and Hellman described the idea of public-key cryptography without giving a full encryption construction. The story goes that the working version came to Rivest after a Passover dinner; Shamir and Adleman had spent months poking holes in his earlier proposals, and the surviving idea — using Euler's theorem over a composite modulus $n = pq$ — was the one nobody could break. Open publication triggered a long political fight with the U.S. government over academic prior restraint and export controls that classified RSA software as a munition under ITAR. Much later, in 1997, it emerged that Clifford Cocks at Britain's GCHQ had discovered essentially the same algorithm in 1973, where it stayed classified for over two decades.

19.2 The number theory you need

Everything rests on one fact from Chapter 4, Euler's theorem, plus its specialization, Fermat's little theorem. Let us restate them so the chapter stands on its own.

For a positive integer n, Euler's totient $\varphi(n)$ counts the integers in $\{1, 2, \ldots, n\}$ that are coprime to

n. If $n = pq$ for distinct primes p, q, then

$$\varphi(n) = (p-1)(q-1),$$

because the only numbers below n *not* coprime to n are the multiples of p (there are $q-1$ of them below n) and the multiples of q (there are $p-1$), and these two sets do not overlap. So $\varphi(pq) = pq - 1 - (q-1) - (p-1) = (p-1)(q-1)$.

Theorem 19.1 (Euler)**.** *If* $\gcd(a, n) = 1$ *then* $a^{\varphi(n)} \equiv 1 \pmod{n}$.

The reason is purely group-theoretic and worth keeping in your head: the units modulo n, written $(\mathbb{Z}/n\mathbb{Z})^*$, form a group of size $\varphi(n)$ under multiplication. By Lagrange's theorem the order of any element divides the size of the group, so raising any unit to the $\varphi(n)$ power lands you back at the identity, 1. Fermat's little theorem is the case $n = p$ prime, where $\varphi(p) = p - 1$:

$$a^{p-1} \equiv 1 \pmod{p} \quad \text{whenever } p \nmid a.$$

Connection

The cost of actually computing $a^e \bmod n$ is the subject of the modular-exponentiation section in the companion algorithms book. The trick is square-and-multiply: an exponent with k bits costs about k squarings and at most k multiplications, each modulo n. So exponentiating a 2048-bit number takes a few thousand big-integer multiplications — fast. The asymmetry that makes RSA work is that *forward* exponentiation is cheap while *inverting* it (finding e-th roots mod n without the trapdoor) appears to require factoring n, for which no comparably fast algorithm is known.

19.3 Key generation

Here is the whole construction. To make a key pair:

1. Choose two large distinct primes p and q (in practice each around 1024 bits, so n is 2048 bits).
2. Compute the *modulus* $n = pq$ and the totient $\varphi(n) = (p-1)(q-1)$.
3. Choose a public exponent e with $1 < e < \varphi(n)$ and $\gcd(e, \varphi(n)) = 1$. A common choice is $e = 65537$.
4. Compute the private exponent $d \equiv e^{-1} \pmod{\varphi(n)}$, the modular inverse of e modulo $\varphi(n)$. This exists precisely because $\gcd(e, \varphi(n)) = 1$, and you find it with the extended Euclidean algorithm.

The public key is $pk = (n, e)$. The private key is $sk = (n, d)$. The primes p, q and the totient $\varphi(n)$ must be destroyed or kept as secret as d itself — anyone who learns p and q can recompute d.

Focus

The defining relation is $ed \equiv 1 \pmod{\varphi(n)}$. Read it as: e and d are inverse operations *in the exponent*. Encryption raises to the e, decryption raises to the d, and because the exponents multiply to "1 mod $\varphi(n)$", doing both in sequence is the identity on the message. The next

section makes that precise.

19.4 Encryption, decryption, and why it works

A message is encoded as an integer m with $0 \le m < n$. Then:

$$\mathrm{Enc}_{pk}(m) = m^e \bmod n = c, \qquad \mathrm{Dec}_{sk}(c) = c^d \bmod n.$$

That is it. Both operations are a single modular exponentiation.

Theorem 19.2 (Correctness of RSA)**.** *For every m with $0 \le m < n$ we have $(m^e)^d \equiv m \pmod n$.*

Why it works

Why decryption undoes encryption. We need $m^{ed} \equiv m \pmod n$. By construction $ed \equiv 1 \pmod{\varphi(n)}$, so $ed = 1 + k\varphi(n)$ for some integer $k \ge 0$. Hence

$$m^{ed} = m^{1+k\varphi(n)} = m \cdot \left(m^{\varphi(n)}\right)^k.$$

If $\gcd(m, n) = 1$, Euler's theorem (19.1) says $m^{\varphi(n)} \equiv 1 \pmod n$, so the bracket is $1^k = 1$ and $m^{ed} \equiv m$. Done — for units.
But what if m shares a factor with n, say $p \mid m$? Then Euler's theorem does not apply directly. We patch this with the Chinese Remainder Theorem. Work modulo p and modulo q separately.
Modulo q: since $p \mid m$ and $q \ne p$, we have $\gcd(m, q) = 1$, so Fermat gives $m^{q-1} \equiv 1 \pmod q$. Now $ed - 1 = k(p-1)(q-1)$ is a multiple of $q - 1$, so $m^{ed-1} \equiv 1 \pmod q$ and therefore $m^{ed} \equiv m \pmod q$.
Modulo p: here $p \mid m$, so $m \equiv 0 \pmod p$, and likewise $m^{ed} \equiv 0 \equiv m \pmod p$ (any positive power of 0 is 0).
So $m^{ed} \equiv m$ both mod p and mod q. Since p and q are distinct primes, CRT says these two congruences pin down m^{ed} uniquely modulo $pq = n$, giving $m^{ed} \equiv m \pmod n$ for *every* m, units or not.

So RSA decryption is exact for all messages in range, not just the coprime ones. (In practice m with $\gcd(m, n) > 1$ would mean Bob accidentally encoded one of Alice's secret primes, an event of vanishing probability — and if it happened, $\gcd(m, n)$ would hand the attacker a factor of n.)

19.4.1 A miniature toy trace: $p = 5$, $q = 11$, $n = 55$

Before tracing multi-digit numbers, look at the arithmetic on numbers small enough to compute in your head:

1. **Primes:** Choose $p = 5$ and $q = 11$.
2. **Modulus and totient:** $n = 5 \times 11 = 55$, and $\varphi(n) = (5-1)(11-1) = 4 \times 10 = 40$.
3. **Public exponent:** Pick $e = 3$. We check $\gcd(3, 40) = 1$.

4. **Private exponent:** Solve $3d \equiv 1 \pmod{40}$. Since $3 \times 27 = 81 = 2 \times 40 + 1$, we get $d = 27$.

5. **Keys:** Public key $pk = (n = 55, e = 3)$, private key $sk = (n = 55, d = 27)$.

6. **Encrypt message $m = 9$:**

$$c = m^e \bmod n = 9^3 \bmod 55 = 729 \bmod 55 = 14 \quad (\text{since } 55 \times 13 = 715).$$

7. **Decrypt ciphertext $c = 14$:**

$$m = c^d \bmod n = 14^{27} \bmod 55.$$

Modulo 5: $14 \equiv -1$, so $(-1)^{27} = -1 \equiv 4 \pmod 5$. Modulo 11: $14 \equiv 3$, and by Fermat $3^{10} \equiv 1$, so $3^{27} = (3^{10})^2 \cdot 3^7 \equiv 2187 \equiv 9 \pmod{11}$. Combining $m \equiv 4 \pmod 5$ and $m \equiv 9 \pmod{11}$ yields $m = 9$.

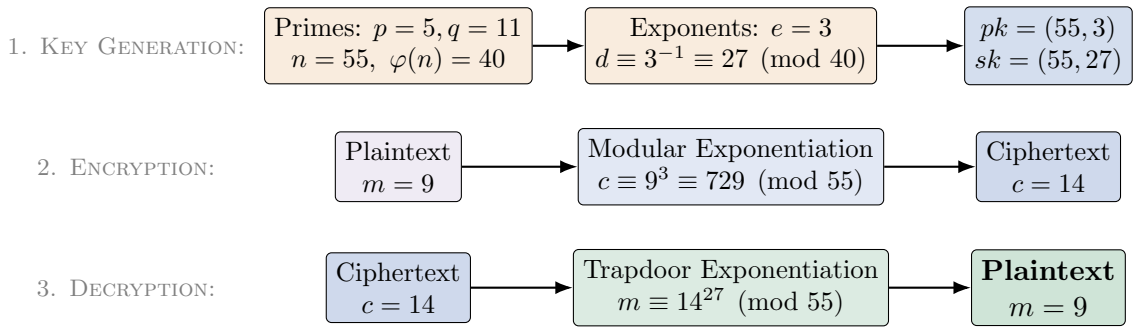


Figure 19.2: The end-to-end RSA lifecycle demonstrated with toy parameters ($p = 5, q = 11$). Stage 1 derives public exponent $e = 3$ and private exponent $d = 27$. Stage 2 encrypts $m = 9 \to c = 14$. Stage 3 inverts $c = 14 \to m = 9$ using trapdoor d.

19.5 A full worked example: $p = 37$, $q = 43$

Let us run the entire scheme with tiny primes so every number is checkable by hand.

Key generation.

- $p = 37$, $q = 43$.
- $n = pq = 37 \cdot 43 = 1591$.
- $\varphi(n) = (p-1)(q-1) = 36 \cdot 42 = 1512$.
- Choose $e = 11$. Check $\gcd(11, 1512) = 1$: since $1512 = 11 \cdot 137 + 5$, then $11 = 5 \cdot 2 + 1$, so the gcd is 1. Good, 11 is a valid public exponent.
- Compute $d = 11^{-1} \bmod 1512$ with the extended Euclidean algorithm. Running it back: $1 = 11 - 5 \cdot 2 = 11 - 2(1512 - 11 \cdot 137) = 275 \cdot 11 - 2 \cdot 1512$. So $275 \cdot 11 \equiv 1 \pmod{1512}$, i.e. $d = 275$.

Check: $11 \cdot 275 = 3025$, and $3025 = 2 \cdot 1512 + 1 = 3024 + 1$. Indeed $ed \equiv 1 \pmod{\varphi(n)}$.

So $pk = (1591, 11)$ and $sk = (1591, 275)$.

Encrypt. Suppose Bob's message is $m = 42$. He computes

$$c = 42^{11} \bmod 1591.$$

Do it with square-and-multiply. Build up powers of 42 modulo 1591:

$$42^1 \equiv 42, \qquad 42^2 \equiv 1764 \equiv 173,$$
$$42^4 \equiv 173^2 = 29929 \equiv 29929 - 18 \cdot 1591 = 1291 \equiv -300,$$
$$42^8 \equiv (-300)^2 = 90000 \equiv 90000 - 56 \cdot 1591 = 904.$$

Now $11 = 8+2+1$, so $42^{11} = 42^8 \cdot 42^2 \cdot 42^1 \equiv 904 \cdot 173 \cdot 42$. First, $904 \cdot 173 = 156392 = 98 \cdot 1591 + 474 \equiv 474$. Then $474 \cdot 42 = 19908 = 12 \cdot 1591 + 816 \equiv 816$. Thus

$$c = 816.$$

Decrypt. Alice computes $m = 816^{275} \bmod 1591$. Doing the full square-and-multiply by hand is tedious (the exponent $275 = 100010011_2$), but the structure is identical. The end result, which you can verify with the toy code below, is $816^{275} \equiv 42 \pmod{1591}$ — the original message.

Walking question

Want to sanity-check decryption without grinding all eight squarings of the full exponent? Use CRT, exactly as in the correctness proof. The decryptor knows $p = 37$ and $q = 43$, so it precomputes the reduced exponents

$$d_p = d \bmod (p-1) = 275 \bmod 36 = 23, \qquad d_q = d \bmod (q-1) = 275 \bmod 42 = 23,$$

and the per-prime ciphertext residues $c_p = 816 \bmod 37 = 2$ and $c_q = 816 \bmod 43 = 42 \equiv -1$. Then it computes the two small exponentiations

$$m_p = c_p^{d_p} \bmod 37 = 2^{23} \bmod 37 = 5, \qquad m_q = c_q^{d_q} \bmod 43 = (-1)^{23} \bmod 43 = -1 \equiv 42.$$

Finally it CRT-combines $\{5 \bmod 37,\ 42 \bmod 43\}$. Writing $m = m_q + q \cdot t = 42 + 43t$ and forcing $42 + 43t \equiv 5 \pmod{37}$ gives $5 + 6t \equiv 5 \pmod{37}$, so $6t \equiv 0 \pmod{37}$, yielding $t = 0$. Thus

$$m = 42 + 43 \cdot 0 = 42.$$

The original message, recovered through two exponentiations on half-width numbers instead of one on the full modulus (Figure 19.3). Real implementations decrypt this way for a roughly fourfold speedup — it is called *RSA-CRT*.

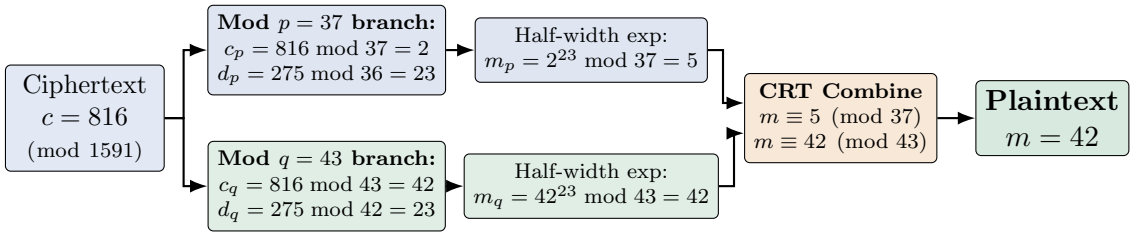


Figure 19.3: RSA-CRT decryption pipeline for $n = 37 \times 43 = 1591$ and $d = 275$. Instead of computing $816^{275} \bmod 1591$ directly, the decryptor reduces the ciphertext and exponent modulo p and q, performs two fast half-width exponentiations in parallel, and recombines $(m_p, m_q) = (5, 42)$ via the Chinese Remainder Theorem.

Pause

The spot-check $m_q = (-1)^{23} \bmod 43 = 42$ was instant because reducing mod 43 turns 816 into -1. The general lesson is that by-hand RSA arithmetic is error-prone — it is easy to drop a factor in a long squaring chain — so verify against the toy code below, and use Fermat or Euler to collapse exponents whenever a prime modulus lets you. The machine is the source of truth; the hand computation is for building intuition about *why* the answer comes out, not for trusting the digits.

Implement it

A complete textbook-RSA in a few lines of Python. **Learning only — never deploy this. It is missing padding, constant-time arithmetic, and safe key generation, and is insecure in every real sense.**

```python
from math import gcd

def egcd(a, b):
    if b == 0:
        return (a, 1, 0)
    g, x, y = egcd(b, a % b)
    return (g, y, x - (a // b) * y)

def inv_mod(a, m):
    g, x, _ = egcd(a % m, m)
    if g != 1:
        raise ValueError("no inverse")
    return x % m

def keygen(p, q, e=65537):
    n = p * q
    phi = (p - 1) * (q - 1)
    if gcd(e, phi) != 1:
        raise ValueError("bad e")
    d = inv_mod(e, phi)
    return (n, e), (n, d)        # public, private

def encrypt(pk, m):
    n, e = pk
    return pow(m, e, n)          # pow does square-and-multiply

def decrypt(sk, c):
    n, d = sk
    return pow(c, d, n)

# Reproduce the chapter's worked example:
pub, priv = keygen(37, 43, e=11)
print(pub, priv)                 # (1591, 11) (1591, 275)
c = encrypt(pub, 42)
print(c)                         # 816
print(decrypt(priv, c))          # 42
```

The encryptor and decryptor are one line each — but the absence of padding is the danger

the next section describes.

19.6 The factoring assumption

Why can't Eve, who sees $pk = (n, e)$ and ciphertext $c = m^e \bmod n$, just recover m? She would need to compute an e-th root modulo n. The obvious route is to learn d, which means learning $\varphi(n) = (p-1)(q-1)$; and since $\varphi(n) = n - (p+q) + 1$, knowing φ together with $n = pq$ pins down $p+q$ and pq, hence p and q. Learning $\varphi(n)$ is equivalent to factoring n. So if factoring is hard, the direct attack is out.

Definition 19.1 (RSA assumption)**.** Sample two random large primes p, q, set $n = pq$, pick a valid e, and pick a uniformly random target $y \in (\mathbb{Z}/n\mathbb{Z})^*$. The *RSA problem* is to find x with $x^e \equiv y \pmod{n}$. The RSA assumption says no PPT algorithm solves this with non-negligible probability.

Definition 19.2 (Factoring assumption)**.** Given $n = pq$ for random large primes p, q, no PPT algorithm outputs p or q with non-negligible probability.

Factoring breaks RSA: if you can factor n you recover $\varphi(n)$, then d, then everything. The converse — whether breaking RSA (computing e-th roots) requires factoring — is *not* known to hold. Breaking RSA could conceivably be easier than factoring; there is no proof either way. So the RSA assumption is formally stronger (a stronger thing to assume) than the factoring assumption. In practice we lean on both, and decades of cryptanalysis have left the best general-purpose factoring algorithm, the general number field sieve, running in time

$$\exp\Big(\big(\tfrac{64}{9}\big)^{1/3}(\ln n)^{1/3}(\ln\ln n)^{2/3}\Big),$$

which is sub-exponential but still wildly impractical for a well-chosen 2048-bit n.

In the real world

The RSA Factoring Challenge. The RSA Factoring Challenge (1991–2007) put cash prizes on factoring specific moduli. RSA-129 (a 426-bit number from a 1977 *Scientific American* column by Martin Gardner, estimated by Rivest to need 40 quadrillion years) fell in 1994 to a distributed effort across the early internet. RSA-768 (768 bits) was factored in 2009 after about two years of computation. RSA-250 (829 bits) fell in 2020. No 1024-bit or larger challenge modulus has been publicly factored, which is why 2048 bits is today's floor and 3072+ is recommended for long-lived keys.

Connection

The sub-exponential running time of the number field sieve is why RSA keys must be so much larger than symmetric keys for equivalent security: a 128-bit AES key and roughly a 3072-bit RSA modulus sit at a comparable security level. The companion algorithms book derives where the $\exp((\ln n)^{1/3}\dots)$ shape comes from — it is the cost of finding enough "smooth" relations to build a congruence of squares. Shor's quantum algorithm, in Chapter 35, factors in polynomial time and would break RSA completely; this is the main motivation for post-quantum cryptography.

19.7 Why textbook RSA is broken

Everything so far defines *textbook RSA*: encrypt as $c = m^e \bmod n$, full stop. It is correct, and the factoring assumption protects the key. Yet as an encryption scheme it fails badly, because correctness and key-secrecy are not the same as message-secrecy. The fatal flaw is that textbook RSA is *deterministic*: the same m always maps to the same c. A deterministic public-key scheme can never be semantically secure — Eve encrypts her guesses herself and compares.

Attack

Several ways textbook RSA leaks.
Guess-and-check. Suppose Bob is sending one of two known messages, "BUY" or "SELL". Eve has pk. She computes $\mathrm{Enc}_{pk}(\mathrm{BUY})$ and $\mathrm{Enc}_{pk}(\mathrm{SELL})$ herself and compares to the intercepted c. She reads the plaintext without touching the private key. Any low-entropy message space (a PIN, a vote, a yes/no) falls instantly.
Small-message / small-e attack. If $e = 3$ and the message is small enough that $m^3 < n$, then $c = m^3$ with *no* modular reduction, and Eve just takes an ordinary integer cube root of c over $\mathbb{Z}$. No factoring needed. More generally, short messages (like a 128-bit AES key being transported) are vulnerable whenever m^e does not wrap around the modulus.
Håstad's broadcast attack. Send the *same* m to three recipients with $e = 3$ and distinct moduli n_1, n_2, n_3. Eve collects $c_i = m^3 \bmod n_i$. By CRT she reconstructs $m^3 \bmod n_1 n_2 n_3$; but $m^3 < n_1 n_2 n_3$, so this congruence *is* m^3 as an integer, and she takes the cube root. Three ciphertexts suffice to recover m fully, with no factoring.
Malleability. RSA's homomorphic structure, $\mathrm{Enc}(m_1) \cdot \mathrm{Enc}(m_2) = \mathrm{Enc}(m_1 m_2) \bmod n$, lets Eve maul ciphertexts. Given $c = \mathrm{Enc}(m)$ she can produce $\mathrm{Enc}(2m) = c \cdot 2^e \bmod n$ without knowing m. In a chosen-ciphertext setting this is devastating: ask the server to decrypt the harmless-looking $c \cdot 2^e$, get back $2m$, divide by 2.

Common trap

The lesson: **never encrypt a raw message with $m^e \bmod n$.** Determinism alone sinks the scheme regardless of key size. The fix is to inject randomness and structure into m before exponentiating — a padding scheme — so that (a) the same plaintext encrypts to different ciphertexts each time, and (b) malformed or mauled ciphertexts are rejected on decryption.

19.8 Padding done right: OAEP

The modern fix for RSA encryption is *Optimal Asymmetric Encryption Padding* (OAEP), designed by Bellare and Rogaway in 1994. The idea: before exponentiating, run the message through a randomized, reversible transformation built from two hash functions used as "random oracles," so that the integer you actually raise to the e looks uniformly random and any tampering scrambles it beyond recognition.

Let n be k bits and fix two mask-generating hash functions G and H. To encrypt a message M (first extended with a fixed amount of zero bytes and a label hash to a block M' of n_1 bits), choose a fresh random seed r of n_0 bits and compute a two-round Feistel-like mixing:

$$X = M' \oplus G(r), \qquad Y = r \oplus H(X),$$

then set the padded integer to $m = X \,\|\, Y$ and finally $c = m^e \bmod n$. Decryption exponentiates to get $m = X \,\|\, Y$ back, recovers $r = Y \oplus H(X)$, then $M' = X \oplus G(r)$, and checks that the fixed padding bytes are exactly where they should be — if not, it rejects. Figure 19.4 traces the mixing.

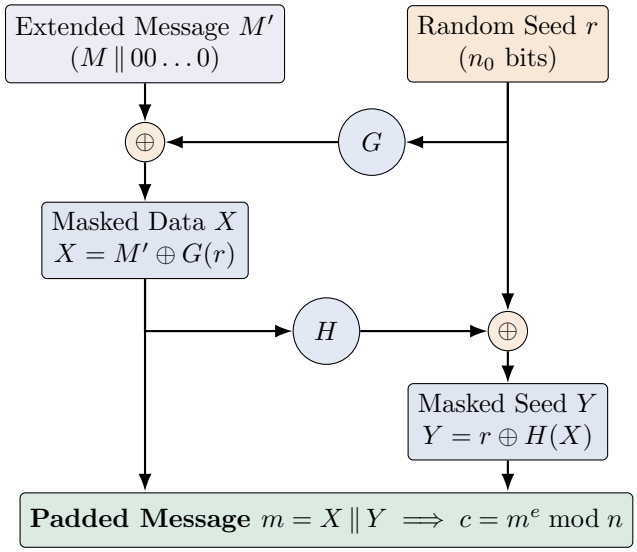


Figure 19.4: The RSA-OAEP two-round asymmetric Feistel structure. Fresh random seed r masks the padded message M' via hash function G; the resulting block X masks r via hash function H. Every bit of the padded block $m = X \,\|\, Y$ depends nonlinearly on every bit of M' and r, preventing malleability and broadcast attacks.

Why it works

Why OAEP fixes the leaks. The random seed r makes encryption probabilistic: encrypt the same M twice and the two ciphertexts differ, killing the guess-and-check and broadcast attacks. The Feistel mixing means every bit of m depends on every bit of M and r through the hash functions, so m is spread across the full width of the modulus — no more small-message cube roots. And the integrity check on the fixed padding bytes makes the scheme reject mauled ciphertexts: flip anything in c and decryption yields a random-looking m whose padding almost surely fails the check, defeating malleability. In the random-oracle model, RSA-OAEP is provably IND-CCA2 secure under the RSA assumption.

Attack

The padding that came *before* OAEP, called PKCS#1 v1.5, was not so lucky. In 1998 Daniel Bleichenbacher showed that a server which merely reveals *whether* a decrypted ciphertext has valid v1.5 padding (a one-bit "padding oracle") leaks enough to recover an entire RSA-encrypted message in roughly a million adaptively chosen queries. The attack walks the unknown plaintext into a shrinking interval, one valid-padding response at a time. It plagued TLS implementations for two decades: ROBOT (2017) found the same flaw still exploitable in major TLS stacks because they never fully removed v1.5 key transport. Padding is therefore part of the security of the scheme rather than an optional detail, and getting its error handling subtly wrong reopens the hole.

19.9 Implementation mistakes that have sunk real systems

Even with correct padding, RSA has a long list of subtle pitfalls at key-generation time. Each of these has broken deployed systems.

Attack

Small public exponent e on unpadded messages. As shown, $e = 3$ with a small or unpadded m invites cube-root and Håstad attacks. The community standardized on $e = 65537 = 2^{16}+1$: it is large enough to dodge those low-exponent pitfalls, yet has only two 1-bits, so encryption is fast. (A small e is safe *with* OAEP; the danger is small e plus no/structured padding.)

Attack

Small or leaked private exponent d. Tempting to pick a small d to speed up decryption — but Wiener's attack (1990) recovers d in polynomial time using continued fractions whenever $d < \frac{1}{3}n^{1/4}$. Boneh and Durfee pushed the bound to $d < n^{0.292}$. Never shrink d; speed up decryption with CRT instead.

Attack

Shared modulus. A naive organization issues many users key pairs (n, e_i) over the *same* modulus n, thinking the distinct exponents keep them separate. This fails in two ways. First, any keyholder knows a d_i with $e_i d_i \equiv 1 \pmod{\varphi(n)}$, and from *any* valid (e, d) pair one can factor n in expected polynomial time — so user A can compute everyone else's private key. Second, if the same message m is sent to two users with coprime exponents e_1, e_2, an outsider with $c_1 = m^{e_1}$ and $c_2 = m^{e_2}$ finds a, b with $ae_1 + be_2 = 1$ (extended Euclid) and computes $c_1^a c_2^b = m^{ae_1+be_2} = m \bmod n$. Common-modulus is never acceptable.

Attack

Low-entropy primes (the GCD harvest). If two different RSA moduli accidentally share a prime factor — $n_1 = p\,q_1$ and $n_2 = p\,q_2$ — then $\gcd(n_1, n_2) = p$ instantly factors both, with a cheap Euclid computation and no number sieve at all. In 2012 two independent teams (Lenstra et al. "Ron was wrong, Whit is right," and Heninger et al.) scanned millions of public TLS and SSH keys from the internet and computed pairwise GCDs. Hundreds of thousands of keys shared primes and were immediately factored. The root cause was embedded devices (routers, firewalls) generating keys at first boot before their entropy pools had filled, so their "random" primes collided. The fix is simple: never generate keys without a properly seeded CSPRNG (Chapter 17).

Concretely, suppose a batch of devices emits $n_1 = 10403$ and $n_2 = 10807$. Neither factor is obvious by inspection, and factoring a single one in isolation means trial division up to its square root. But one Euclidean GCD reveals the shared prime in microseconds: $\gcd(10403, 10807) = 101$, whence $n_1 = 101 \cdot 103$ and $n_2 = 101 \cdot 107$ by a single division each. Both keys are now fully factored. Scale this to a corpus of N moduli: a batch-GCD algorithm computes *all* pairwise gcds in roughly $O(N \log^2 N)$ time using a product tree, so a few hours of compute over tens of millions of internet keys is enough to harvest every colliding pair —

which is exactly what those 2012 scans did.

Common trap

A few more sharp edges worth memorizing: p and q must be of similar size but not *too* close (if $|p-q|$ is small, Fermat's factorization $n = (\frac{p+q}{2})^2 - (\frac{p-q}{2})^2$ cracks n fast); they must be genuinely random, not from a weak prime-generation library (the 2017 ROCA flaw factored Infineon-generated keys due to structured primes); and RSA decryption must run in constant time, or timing and cache differences leak d bit by bit (Chapter 37).

19.10 The other half: RSA signatures

The same trapdoor that encrypts also signs, with the roles of the two exponents swapped. To *sign* a message, the holder of the private key applies d; to *verify*, anyone applies the public e. Where encryption is "public to lock, private to unlock," signing is "private to stamp, public to check."

Textbook RSA signatures look symmetric to encryption:

$$\mathrm{Sign}_{sk}(m) = m^d \bmod n = \sigma, \qquad \mathrm{Vrfy}_{pk}(m,\sigma):\ \text{accept iff } \sigma^e \equiv m \pmod{n}.$$

Correctness is the very same identity $(m^d)^e \equiv m^{ed} \equiv m \pmod{n}$ from Theorem 19.2; it does not matter which exponent you apply first.

Why it works

Why this authenticates. Only the key owner knows d, so only she can produce a σ that survives the e-th power check. A forger who sees $pk = (n, e)$ and wants to forge a signature on a chosen m must compute an e-th root of m modulo n — precisely the RSA problem of Definition 19.1. Verification is public because e is public. Encryption and signing use the same arithmetic with the exponents applied in opposite order: encryption hides a message, while a signature binds it to the key owner.

But textbook RSA signatures are broken for the same family of reasons as textbook encryption, plus some special to signing.

Attack

Forgeries against raw RSA signatures.
Existential forgery. Pick any σ at random and publish the "message" $m = \sigma^e \bmod n$. The pair (m, σ) verifies by construction. You do not control m, but for many protocols a forged signature on *some* value is already damage.
Multiplicative forgery. Given valid signatures $\sigma_1 = m_1^d$ and $\sigma_2 = m_2^d$, the product $\sigma_1\sigma_2 = (m_1 m_2)^d \bmod n$ is a valid signature on $m_1 m_2$. An attacker who can get two innocuous messages signed can forge a signature on their product without the key.
Blinding. To get a signature on a message m the signer would refuse, an attacker picks random r, asks for a signature on the disguised $m' = r^e m \bmod n$ (which looks random), receives $\sigma' = (m')^d = r\,m^d$, and divides out: $\sigma = \sigma'/r \bmod n$ is a valid signature on the original m.

Common trap

As with encryption, the fix is structured, randomized padding before exponentiating — you never sign a raw m. Modern RSA signing means **RSA-PSS** (Probabilistic Signature Scheme), which hashes the message, mixes in a random salt, and formats a full-width block before applying d. The hash destroys the multiplicative structure (you would need a hash collision to exploit it), the salt makes signatures probabilistic, and the fixed format makes existential forgeries fail verification. PSS has a tight security proof in the random-oracle model under the RSA assumption. The older deterministic PKCS#1 v1.5 signature padding is still widely deployed and not catastrophically broken for signing the way it is for encryption, but PSS is the modern default. Chapter 22 develops signatures in full.

19.11 RSA in the real world

In the real world

Where RSA actually shows up. For most of the internet's history, TLS used RSA for two jobs: the server's certificate carried an RSA public key, and the client encrypted a random pre-master secret to it (RSA key transport). That pattern is now deprecated — precisely because of Bleichenbacher/ROBOT and because it offers no forward secrecy — and TLS 1.3 dropped RSA key exchange entirely in favor of ephemeral Diffie-Hellman over elliptic curves. But RSA is far from gone: TLS 1.3 still allows RSA *signatures* (RSA-PSS) for the server to authenticate itself, and the global web PKI is still dominated by RSA certificate keys. SSH host keys and user keys are frequently RSA (`ssh-rsa`, now with SHA-2 signatures). PGP/GnuPG email encryption and signing has long used RSA. Code-signing, document signing, and smart cards lean on it heavily. So in practice you will meet RSA mostly as a *signature* and *certificate* algorithm today, with encryption duties increasingly handed to ECDH for the key exchange and AES-GCM for the data.

Focus

A practical rule of thumb you can carry out of this chapter: RSA encryption in the wild should always mean *RSA-OAEP*, RSA signatures should always mean *RSA-PSS* (Chapter 22), the modulus should be at least 2048 bits (3072+ for keys meant to last past 2030), the public exponent should be 65537, and the primes must come from a vetted, well-seeded generator. Anything that encrypts with raw $m^e \bmod n$, uses PKCS#1 v1.5, or hand-rolls prime generation should be treated as broken until proven otherwise.

Connection

RSA builds its trapdoor from the hardness of factoring $n = pq$: knowing the factors gives $\varphi(n)$ and lets you invert exponents. In Chapter 18 we met a second hard problem, computing discrete logarithms in a cyclic group. Chapter 20 turns that discrete logarithm problem into a full public-key encryption scheme: ElGamal.

19.12 Exercises

Exercise 19.1. With $p = 11$, $q = 23$, compute n and $\varphi(n)$. Show that $e = 3$ is a valid public exponent, and find the corresponding private exponent d using the extended Euclidean algorithm.

Exercise 19.2. Using your key from the previous exercise, encrypt the message $m = 7$ by computing $7^3 \bmod n$ with square-and-multiply, then decrypt the result and confirm you recover 7.

Exercise 19.3. Prove that for RSA, $\mathrm{Enc}_{pk}(m_1) \cdot \mathrm{Enc}_{pk}(m_2) \equiv \mathrm{Enc}_{pk}(m_1 m_2) \pmod{n}$. Explain in one sentence why this homomorphic property makes textbook RSA malleable, and why OAEP destroys it.

Exercise 19.4. (Common modulus.) Two users share modulus $n = 1591$ with public exponents $e_1 = 11$ and $e_2 = 5$ (check that each is coprime to $\varphi(1591) = 1512$, so both are valid exponents), and the same message m is sent to both, yielding c_1 and c_2. Show $\gcd(e_1, e_2) = 1$, find integers a, b with $ae_1 + be_2 = 1$, and write the formula that recovers m from c_1, c_2. (One of a, b is negative; how do you handle a negative exponent modulo n? Hint: replace c_1^a by $(c_1^{-1})^{|a|}$.)

Exercise 19.5. (Håstad in miniature.) A message $m = 10$ is sent with $e = 3$ to three recipients with moduli $n_1 = 21$, $n_2 = 143$, $n_3 = 323$ (each a product of two small distinct primes, and the three are pairwise coprime — a requirement for CRT). Compute the three ciphertexts $c_i = m^3 \bmod n_i$, CRT-combine them to recover m^3 modulo $n_1 n_2 n_3$, and take the integer cube root to get m. Verify $m^3 < n_1 n_2 n_3$. (You should find $c_1 = 13$, $c_2 = 142$, $c_3 = 31$, and the combined value 1000.)

Exercise 19.6. Suppose you are told $n = 1591$ and $\varphi(n) = 1512$. Recover p and q without trial division, by setting up and solving the quadratic whose roots are p and q. (Hint: you know $p + q = n - \varphi(n) + 1$ and $pq = n$.)

Exercise 19.7. Two RSA moduli $n_1 = 5{,}459$ and $n_2 = 5{,}777$ were generated by a device with a broken random source and happen to share a prime. Compute $\gcd(n_1, n_2)$ to find the shared prime, then factor both moduli completely. Explain why this attack is exponentially cheaper than factoring either modulus from scratch.

Exercise 19.8. Explain precisely why a deterministic public-key encryption scheme can never satisfy IND-CPA security. Frame your answer as a winning strategy for the IND-CPA adversary against textbook RSA.

Exercise 19.9. (Why $\gcd(m, n) > 1$ never matters in practice but is interesting in theory.) Show that if Bob ever encrypts a message m with $\gcd(m, n) = p$, then anyone who sees m can factor n. Estimate the probability that a uniformly random $m \in \{1, \ldots, n-1\}$ has $\gcd(m, n) > 1$ for a 2048-bit modulus, and argue it is negligible.

Exercise 19.10. (Wiener's setup.) Look up the statement of Wiener's attack and explain, in your own words, why choosing a small private exponent d to speed up decryption is dangerous, and why using RSA-CRT achieves the same speedup safely. You need not reproduce the continued-fraction machinery; argue at the level of which quantities leak.

Exercise 19.11. (RSA-CRT by hand.) Using $pk = (1591, 11)$, $sk = (1591, 275)$ from the worked example, decrypt the ciphertext $c = 816$ *via CRT*: reduce c modulo 37 and 43, reduce d modulo 36 and 42, perform the two small exponentiations, and CRT-combine to recover m. Check that you get 42, and count how many modular multiplications you used versus the full $816^{275} \bmod 1591$.

Exercise 19.12. (Signature forgery.) For the public key $pk = (1591, 11)$, produce an existential forgery: pick any σ you like, compute $m = \sigma^{11} \bmod 1591$, and confirm that (m, σ) passes textbook verification. Then, given that σ_1 signs m_1 and σ_2 signs m_2, write the signature on $m_1 m_2 \bmod 1591$ in terms of σ_1, σ_2. Explain in one sentence how hashing the message before signing kills both forgeries.

Exercise 19.13. (Blinding.) A signer will stamp anything that "looks random" but refuses to sign the specific value $m^\star$. Describe precisely the messages an attacker submits, and the post-processing on the returned signatures, to obtain a valid signature on $m^\star$ anyway. Which property of textbook RSA makes this possible, and why does RSA-PSS prevent it?

Chapter 20

The Discrete Logarithm Problem and ElGamal

In Chapter 19 we built public-key encryption around the hardness of factoring, and in Chapter 18 we met a second one-way function: given a generator g of some group and an exponent x, computing g^x is cheap – a few hundred multiplications even for numbers thousands of bits long – while going backwards to recover x from g^x appears to cost an astronomical amount of work. That backwards problem has a name, the *discrete logarithm problem*, and almost everything in public-key cryptography that is not built on factoring is built on it instead: Diffie–Hellman key exchange, the ElGamal encryption scheme we develop here, the DSA and Schnorr signatures of Chapter 22, and – with the group swapped out for points on an elliptic curve – the ECC ecosystem of Chapter 21 that protects your phone calls and web traffic today.

This chapter pins down the discrete logarithm problem in a general group and in $(\mathbb{Z}/p\mathbb{Z})^*$, studies the classic baby-step giant-step and Pollard's rho attacks that set key sizes, and builds ElGamal encryption on top of the Decisional Diffie–Hellman assumption.

20.1 The discrete logarithm, precisely

Fix a finite cyclic group $\mathbb{G}$ of order n, written multiplicatively, and a generator g. That g generates $\mathbb{G}$ means every element of $\mathbb{G}$ equals g^k for some integer k, and the map

$$k \longmapsto g^k, \qquad \mathbb{Z}/n\mathbb{Z} \longrightarrow \mathbb{G},$$

is a bijection (a group isomorphism, in fact, since $g^{a+b} = g^a g^b$). The *discrete logarithm* of an element $h \in \mathbb{G}$ to base g is the exponent that produces it.

Definition 20.1 (Discrete logarithm)**.** Let $\mathbb{G} = \langle g \rangle$ be cyclic of order n. For $h \in \mathbb{G}$, the discrete logarithm $\operatorname{dlog}_g h$ is the unique residue $x \in \mathbb{Z}/n\mathbb{Z}$ with $g^x = h$. The *discrete logarithm problem* (DLP) is: given $\mathbb{G}$, g, and h, compute x.

The word "logarithm" is exactly the right analogy. Over the real numbers, $\log_b y$ is the exponent you raise b to in order to get y, and it is easy because the real exponential is smooth and monotone – you can binary-search it. The word "discrete" is the warning. In a finite group the powers $g^0, g^1, g^2, \ldots$

do not march steadily upward; they hop around the group with no useful order. There is nothing to binary-search. Knowing g^x is just under h tells you nothing about whether x is close to $\operatorname{dlog}_g h$, because "just under" is meaningless once you reduce mod p.

Intuition

Think of a clock with $p-1$ positions and a hand that, each tick, jumps forward by a multiplicative step. Telling someone the current position is easy if you know how many ticks have passed. But if I only show you where the hand is pointing, and the step size scrambles the positions, you have no shortcut: you seem forced to try tick counts one by one until the hand lands where I said. Going forward is a direct computation; going backward is a search.

20.1.1 The concrete case: $(\mathbb{Z}/p\mathbb{Z})^*$

The primary group for classical discrete-log cryptography is $(\mathbb{Z}/p\mathbb{Z})^*$, the nonzero residues mod a prime p under multiplication. This group is cyclic of order $p-1$ (a fact proved in Chapter 6; every finite subgroup of the multiplicative group of a field is cyclic), so it has generators. A generator of $(\mathbb{Z}/p\mathbb{Z})^*$ is traditionally called a *primitive root* mod p.

Example 20.1 (A primitive root mod 7)**.** Take $p = 7$, so $(\mathbb{Z}/7\mathbb{Z})^* = \{1, 2, 3, 4, 5, 6\}$ has order 6. Try $g = 3$ and list its powers:

$$3^0 = 1, \quad 3^1 = 3, \quad 3^2 = 2, \quad 3^3 = 6, \quad 3^4 = 4, \quad 3^5 = 5 \pmod 7,$$

and $3^6 = 1$, closing the loop. The powers $1, 3, 2, 6, 4, 5$ run through every nonzero residue, so 3 is a primitive root. Now read the table backwards. We have $3^4 = 4$, so $\operatorname{dlog}_3 4 = 4$. And $3^5 = 5$, so $\operatorname{dlog}_3 5 = 5$. There is no pattern relating the value 4 to its log 4, or 5 to its log 5; the coincidence is an artifact of this tiny example. The sequence of logs $0, 1, 2, 3, 4, 5$ corresponds to values $1, 3, 2, 6, 4, 5$ – thoroughly shuffled.

Even at $p = 7$ you can see the difficulty in miniature. If I tell you $g = 3$ and $h = 6$ and ask for $\operatorname{dlog}_3 6$, you cannot reason "6 is close to 7 so the exponent is large." You have to walk the table: $3, 2, 6$ – found it at exponent 3. With p a 2048-bit prime the table has about 2^{2048} rows and you cannot walk it in the lifetime of the universe.

Focus

Two flavors of "hard." The DLP is the *search* problem: recover the exponent. Cryptography often needs a weaker-looking but sufficient assumption, the *Computational Diffie-Hellman* (CDH) problem: given g, g^a, g^b, compute g^{ab} – without necessarily finding a or b. And ElGamal's proof will rest on a still weaker assumption, *Decisional Diffie-Hellman* (DDH): merely *distinguish* g^{ab} from a random group element. These three form a hierarchy, DDH $\leq$ CDH $\leq$ DLP: an algorithm for the harder one solves the easier ones. We will use each where it fits.

20.1.2 Why we believe it is hard

There is no proof that the discrete log problem is hard – such a proof would resolve P versus NP-style questions far beyond present mathematics. What we have is decades of unsuccessful attacks. The

DLP in $(\mathbb{Z}/p\mathbb{Z})^*$ has been studied since the 1970s, attacked by the same number-field sieve machinery that attacks factoring, and the best known algorithms remain *subexponential* but not polynomial: roughly

$$L_p[1/3, c] \;=\; \exp\Big((c + o(1))\,(\ln p)^{1/3}(\ln \ln p)^{2/3}\Big)$$

operations, the same shape as the cost of factoring an integer of similar size. This is why DLP-mod- p keys and RSA keys are comparable in length: 2048-bit and 3072-bit primes are the current mainstream. On elliptic-curve groups, by contrast, no subexponential attack is known, the cost stays fully exponential, and that is exactly why a 256-bit elliptic-curve key matches a 3072-bit mod-p key in strength. We will see the elliptic-curve story in the next chapter; here we develop everything over $(\mathbb{Z}/p\mathbb{Z})^*$ where the arithmetic is concrete.

History

The hardness of discrete logs is the core ingredient of the 1976 Diffie–Hellman paper, *New Directions in Cryptography* – the paper that opened public-key cryptography. Whitfield Diffie and Martin Hellman did not have a public-key *encryption* scheme; they had a key-*exchange* protocol. It was Taher ElGamal, in his 1985 paper "A Public Key Cryptosystem and a Signature Scheme Based on Discrete Logarithms," who turned the Diffie–Hellman idea into a full encryption scheme and a signature scheme. Because ElGamal did not patent his algorithm, NIST based its 1994 Digital Signature Algorithm (DSA) on an ElGamal variant, triggering a licensing dispute with Public Key Partners (which held the Diffie–Hellman and Merkle patents). ElGamal later became chief scientist at Netscape and had a direct hand in SSL, the ancestor of TLS, so the ideas in this chapter fed directly into the padlock in your browser.

20.2 Attacking the DLP: generic algorithms

Before trusting a group we should know the best attacks on it. Two algorithms work in *any* cyclic group, using only the group operation – they do not exploit the special structure of $(\mathbb{Z}/p\mathbb{Z})^*$. These are the *generic* attacks, and they set a floor that no DLP-based scheme can beat. A theorem of Shoup says that a purely generic algorithm needs about $\sqrt{n}$ group operations in a group of order n, and both algorithms below hit that bound.

The main consequence: **the security of a DLP group is governed by the largest prime factor of its order, and the cost of breaking it is about the square root of that prime.** If n has a small prime factor, the problem partially collapses (we will see why via Pohlig-Hellman in the exercises). So we want n to have a large prime factor – ideally $n = 2q$ with q a large prime, the "safe prime" setup.

20.2.1 Baby-step giant-step

Daniel Shanks's baby-step giant-step (BSGS) is a time-memory trade-off. We want x with $g^x = h$, where $0 \le x < n$. Write $m = \lceil \sqrt{n} \rceil$ and split x in base m:

$$x \;=\; i \cdot m + j, \qquad 0 \le i < m, \quad 0 \le j < m.$$

Such a decomposition always exists because i, j together range over $0 \dots m^2 - 1 \geq n - 1$. Now rearrange the equation $g^{im+j} = h$:

$$g^j \;=\; h \cdot (g^{-m})^i.$$

The left side depends only on j (the "baby steps," small increments of g); the right side only on i (the "giant steps," big leaps of size m). The algorithm:

1. **Baby steps.** Compute $g^0, g^1, \dots, g^{m-1}$ and store each in a hash table keyed by the group element, with the exponent j as the value. Cost: m multiplications, m table entries.

2. **Giant steps.** Compute $u = g^{-m}$ once. Then for $i = 0, 1, 2, \dots$ form $h \cdot u^i$ and look it up in the table. The moment $h \cdot u^i$ equals some stored g^j, you have $g^j = h \cdot g^{-im}$, hence $x = im + j$.

Total work is $O(\sqrt{n})$ group operations and $O(\sqrt{n})$ storage. That square root is huge: for $n \approx 2^{256}$ you would need 2^{128} operations and 2^{128} words of memory, both hopelessly out of reach. But it is also a large improvement over the 2^{256} of brute force, and it is why the relevant prime needs to be twice the bit-length of your security target.

Example 20.2 (BSGS by hand, $p = 29$)**.** Let $p = 29$, so $n = 28$, and take the primitive root $g = 2$. We want $\operatorname{dlog}_2 17$. Set $m = \lceil \sqrt{28} \rceil = 6$.

Baby steps – powers $2^j \bmod 29$ for $j = 0 \dots 5$:

$$2^0 = 1, \quad 2^1 = 2, \quad 2^2 = 4, \quad 2^3 = 8, \quad 2^4 = 16, \quad 2^5 = 3.$$

Store the table $\{1{:}0,\ 2{:}1,\ 4{:}2,\ 8{:}3,\ 16{:}4,\ 3{:}5\}$.

Giant step size. We need $g^{-m} = 2^{-6} \bmod 29$. First $2^6 = 64 \equiv 6$, and $6^{-1} \bmod 29 = 5$ (since $6 \cdot 5 = 30 \equiv 1$). So $u = 2^{-6} \equiv 5$.

Giant steps – compute $17 \cdot 5^i \bmod 29$ and check the table:

$$\begin{aligned}
&i = 0 : 17 \cdot 1 = 17 \quad \text{(not in table)} \\
&i = 1 : 17 \cdot 5 = 85 \equiv 27 \quad \text{(no)} \\
&i = 2 : 27 \cdot 5 = 135 \equiv 19 \quad \text{(no)} \\
&i = 3 : 19 \cdot 5 = 95 \equiv 8 \quad (\textbf{hit!}\ 8 \mapsto j = 3).
\end{aligned}$$

Thus $i = 3$, $j = 3$, and $x = im + j = 3 \cdot 6 + 3 = 21$ (Figure 20.1). Check: $2^{21} \bmod 29$. We have $2^5 = 3$, $2^{10} = 9$, $2^{20} = 81 \equiv 81 - 58 = 23$, and $2^{21} = 46 \equiv 17$. Correct, $\operatorname{dlog}_2 17 = 21$.

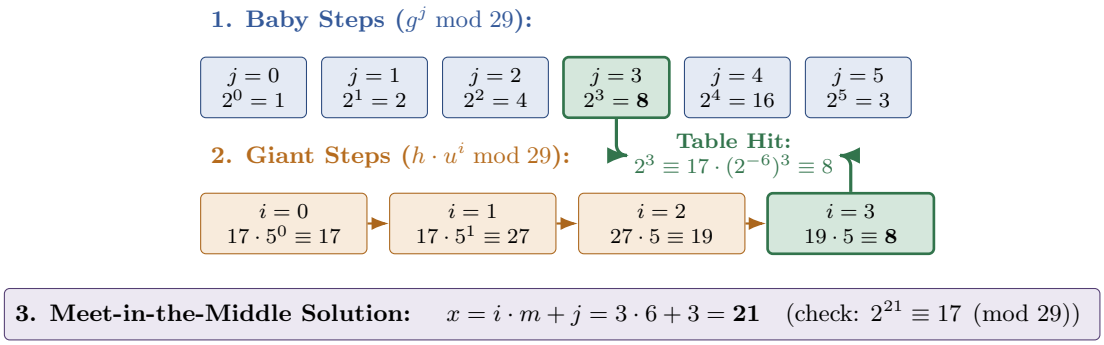


Figure 20.1: Baby-step giant-step (BSGS) meet-in-the-middle search for $\operatorname{dlog}_2 17 \bmod 29$ with stride $m = \lceil\sqrt{28}\rceil = 6$ and giant-step multiplier $u = 2^{-6} \equiv 5 \pmod{29}$. Six baby steps 2^j are stored in a hash table; the fourth giant step ($i = 3$) lands on 8, colliding with $j = 3$ to recover $x = 3\cdot 6 + 3 = 21$ in $O(\sqrt{n})$ operations.

Connection

BSGS is the same idea as the meet-in-the-middle attack on double encryption from Chapter 11, and the same idea as a two-sided search in the algorithms companion. You trade an exponential-time one-sided search for two square-root-sized searches that "meet" through a hash table. The cost analysis – $O(\sqrt{n})$ time, $O(\sqrt{n})$ space, balanced when both halves are $\sqrt{n}$ – is a textbook time-memory trade-off; see the discussion of meet-in-the-middle in the algorithms book.

20.2.2 Pollard's rho: square-root time, constant memory

BSGS has a weakness: it needs $\sqrt{n}$ *memory*, and at cryptographic sizes memory is the binding constraint long before time is. Pollard's rho method achieves the same $O(\sqrt{n})$ *time* with only a constant amount of memory, by replacing the stored table with a pseudorandom walk and the birthday paradox.

The idea: define a deterministic but "random-looking" function f on the group that, at each step, multiplies the current element by either g, by h, or squares it, while carefully tracking the exponents of g and h that produced the current element. So at every point of the walk we know integers a, b with

$$x_k \;=\; g^{a_k} h^{b_k}.$$

The walk wanders through the group. Because the group is finite, the sequence $x_0, x_1, x_2, \ldots$ must eventually repeat, and once it repeats it cycles forever – the trajectory looks like the Greek letter ρ, a tail leading into a loop (Figure 20.2), which is where the method gets its name. By the birthday bound (Chapter 14), a collision $x_i = x_j$ with $i \neq j$ shows up after only about $\sqrt{\pi n/2}$ steps. A collision means

$$g^{a_i} h^{b_i} = g^{a_j} h^{b_j} \implies g^{a_i - a_j} = h^{b_j - b_i} \implies g^{a_i - a_j} = g^{x(b_j - b_i)},$$

so, reading exponents modulo n,

$$a_i - a_j \equiv x\,(b_j - b_i) \pmod{n}, \qquad \text{hence} \qquad x \equiv (a_i - a_j)\,(b_j - b_i)^{-1} \pmod{n},$$

provided $b_j - b_i$ is invertible mod n (true whenever n is prime and $b_j \neq b_i$). We solved for the discrete log without ever building a table.

To detect the collision without storing the whole walk, Pollard borrows Floyd's tortoise-and-hare: run one pointer at single speed and another at double speed; they are guaranteed to meet inside the cycle, and you keep only two group elements in memory at a time.

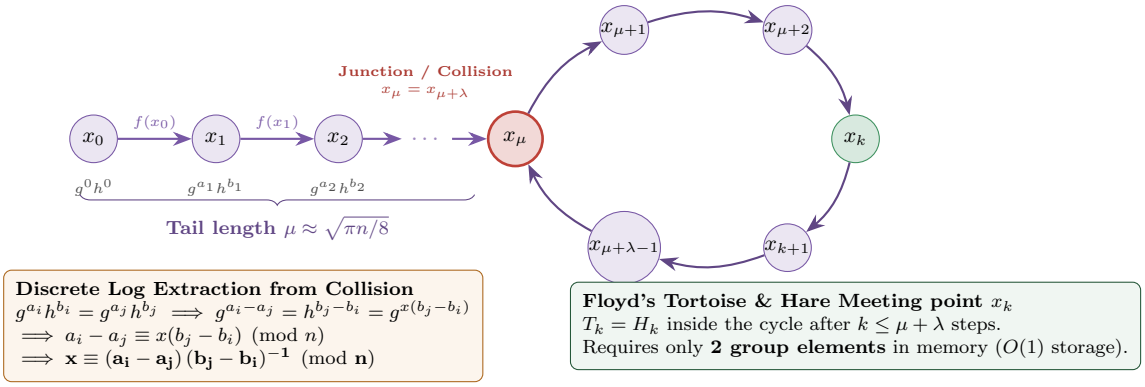


Figure 20.2: The ρ shape of Pollard's walk for discrete logarithms. A deterministic pseudo-random walk traces a tail $x_0, x_1, \ldots$ until it enters a cycle of length λ at junction x_μ. Because each group element is recorded as $x_k = g^{a_k}h^{b_k}$, the first detected collision yields a linear relation in the exponents that directly solves for $\operatorname{dlog}_g h$. Floyd's cycle-finding pointers (tortoise and hare) guarantee collision detection with $O(1)$ storage.

Why it works

Why $\sqrt{n}$ and not less, generically. Both BSGS and rho are special cases of a general result: in a black-box group where the only thing you can do is multiply and test equality, any algorithm needs $\Omega(\sqrt{n})$ operations to find a discrete log (Shoup, 1997). The intuition is the birthday bound running in reverse – you cannot learn the hidden exponent until two of your probe elements collide, and random collisions in a set of size n take $\sqrt{n}$ probes. So $\sqrt{n}$ is a lower bound for any attack that ignores the internal structure of the group. The subexponential attacks on $(\mathbb{Z}/p\mathbb{Z})^*$ beat this bound precisely by exploiting that internal structure (index calculus), which is why elliptic curves – where no such structure is known – keep the bound and let you use much smaller keys.

Example 20.3 (Extracting x from a rho collision)**.** The bookkeeping part of rho is what is worth doing by hand, and it goes cleanest in a prime-order group where every nonzero difference is invertible. Work in a subgroup of prime order $q = 11$, generator g, target $h = g^x$. The walk reports a collision between two points carrying the labels $x_i = g^8h^2$ and $x_j = g^3h^9$. Since these are the same group element,

$$g^8h^2 = g^3h^9 \implies g^{8-3} = h^{9-2} \implies g^5 = g^{7x},$$

so $5 \equiv 7x \pmod{11}$. Invert 7: since $7 \cdot 8 = 56 \equiv 1 \pmod{11}$, we have $7^{-1} \equiv 8$, and $x \equiv 5 \cdot 8 = 40 \equiv 7 \pmod{11}$. The walk never needed a table – two collided labels and one modular inverse pinned the log down. (If q were composite and the relevant difference shared a factor with it, the inverse would not exist and we would restart the walk; this is exactly why prime order is so convenient.)

Implement it

A compact Pollard rho for discrete logs in $(\mathbb{Z}/p\mathbb{Z})^*$, with the exponent bookkeeping spelled out. **Learning only – this has no constant-time discipline, no validation, and is meant to make the algorithm legible, never to attack or protect anything real.**

```python
from math import gcd

def pollard_rho_dlog(g, h, p):
    n = p - 1                       # order of the group Z_p^*
    def step(x, a, b):
        # partition the group into 3 zones by x mod 3
        if   x % 3 == 0:            # zone 0: multiply by g
            return (x*g) % p, (a+1) % n, b
        elif x % 3 == 1:            # zone 1: multiply by h
            return (x*h) % p, a, (b+1) % n
        else:                       # zone 2: square
            return (x*x) % p, (2*a) % n, (2*b) % n
    # tortoise (one step) and hare (two steps)
    x, a, b = 1, 0, 0
    X, A, B = 1, 0, 0
    for _ in range(n):
        x, a, b = step(x, a, b)
        X, A, B = step(*step(X, A, B))
        if x == X:                  # collision found
            r = (b - B) % n         # need to solve  r * dlog = (A - a)  (mod n)
            if r == 0:
                return None         # rare degenerate case; restart with new walk
            t = (A - a) % n
            d = gcd(r, n)           # n is composite here, so d may exceed 1
            if t % d != 0:
                return None         # no solution this run; restart with new walk
            # d candidate residues mod n; exponentiate to pick the right one
            base = (t // d) * pow(r // d, -1, n // d) % (n // d)
            for k in range(d):
                cand = (base + k * (n // d)) % n
                if pow(g, cand, p) == h % p:
                    return cand
            return None
    return None

# Demo (toy parameters): dlog of 17 base 2 mod 29 prints 21.
print(pollard_rho_dlog(2, 17, 29))
```

The single modular inverse of the prime-order Example 20.3 becomes a short `gcd` check and candidate loop here because $n = p - 1$ is even (hence composite): the collision difference r can share a factor with n, leaving a handful of candidate logs that one exponentiation each resolves. In a prime-order subgroup that whole branch collapses to a single inverse – one more reason cryptographic deployments live in prime-order subgroups. The cost is $O(\sqrt{n})$ multiplications and a few integers of memory. For n around 2^{256} that is 2^{128} multiplications – the same cost as BSGS, but now with no storage problem, which is why rho (and its parallel "van Oorschot-Wiener" variant) is the practical generic attack of record, and the one used to set elliptic-curve key sizes.

20.3 From hard problem to encryption: ElGamal

We now have a problem we believe is hard. ElGamal turns it into public-key encryption. The scheme is Diffie-Hellman with one extra step.

Intuition

Recall Diffie-Hellman. Alice and Bob agree on g in a public group. Alice picks secret a and publishes $A = g^a$; Bob picks secret b and publishes $B = g^b$. Each computes the shared secret $g^{ab} = A^b = B^a$. Eve sees g, g^a, g^b and – believing CDH – cannot get g^{ab}.
ElGamal's insight: Alice does not need to be online. Her published $A = g^a$ *is* her permanent public key. Whenever Bob wants to send Alice a secret message M, he runs his own half of a *fresh* Diffie-Hellman against A: he picks a one-time random y, forms his half $c_1 = g^y$, computes the shared secret $s = A^y = g^{ay}$ on the spot, and uses s as a one-time mask to hide M, sending $c_2 = M \cdot s$. Alice, holding a, reconstructs the same secret from Bob's half: $s = c_1^a = g^{ya}$, divides it out, and reads M. ElGamal is Diffie-Hellman with the second party replaced by a fresh random exponent for every message.

20.3.1 The scheme

Definition 20.2 (ElGamal encryption)**.** Fix a cyclic group $\mathbb{G} = \langle g \rangle$ of prime order q in which DDH is believed hard, and in which messages are encoded as group elements.

- **Key generation** Gen**:** choose $a \xleftarrow{\$} \{1, \ldots, q-1\}$, set $A = g^a$. Public key $pk = (\mathbb{G}, g, A)$; secret key $sk = a$.
- **Encryption** $\mathrm{Enc}_{pk}(M)$**:** choose a fresh $y \xleftarrow{\$} \{1, \ldots, q-1\}$, output the ciphertext

$$(c_1, c_2) \;=\; \big(\, g^y,\;\; M \cdot A^y \,\big).$$

- **Decryption** $\mathrm{Dec}_{sk}(c_1, c_2)$**:** compute $s = c_1^a$ and recover $M = c_2 \cdot s^{-1}$.

Why it works

Correctness. The shared mask computed by each side agrees:

$$c_1^a \;=\; (g^y)^a \;=\; g^{ay} \;=\; (g^a)^y \;=\; A^y.$$

So Dec computes $c_2 \cdot (c_1^a)^{-1} = (M \cdot A^y) \cdot (A^y)^{-1} = M$. The blinding factor A^y is exactly the Diffie-Hellman secret between the long-term key $A = g^a$ and the one-time key $c_1 = g^y$; Bob raises A to y, Alice raises c_1 to a, and commutativity of exponents makes them meet.

Example 20.4 (ElGamal by hand, $p = 23$)**.** We use $(\mathbb{Z}/23\mathbb{Z})^*$ for arithmetic that fits in your head. Its order is 22, which is not prime, so this toy is not a secure group (more on that below), but the mechanics are identical. Take $g = 5$; one checks 5 is a primitive root mod 23.

Keygen. Alice picks $a = 6$. Her public key is $A = 5^6 \bmod 23$. Compute: $5^2 = 25 \equiv 2$, $5^3 = 10$, $5^6 = (5^3)^2 = 100 \equiv 8$. So $A = 8$.

Encrypt $M = 10$. Bob picks $y = 3$. Then

$$c_1 = g^y = 5^3 = 10, \qquad c_2 = M \cdot A^y = 10 \cdot 8^3 \bmod 23.$$

Now $8^2 = 64 \equiv 18$, $8^3 = 8 \cdot 18 = 144 \equiv 144 - 138 = 6$. So $c_2 = 10 \cdot 6 = 60 \equiv 60 - 46 = 14$. Ciphertext $(c_1, c_2) = (10, 14)$.

Decrypt. Alice computes $s = c_1^a = 10^6 \bmod 23$. Step it: $10^2 = 100 \equiv 8$, $10^3 = 10 \cdot 8 = 80 \equiv 80 - 69 = 11$, $10^6 = (10^3)^2 = 121 \equiv 121 - 115 = 6$. So $s = 6$, matching $A^y = 6$ as it must. Invert: $6^{-1} \bmod 23$. Since $6 \cdot 4 = 24 \equiv 1$, we get $6^{-1} = 4$. Finally $M = c_2 \cdot s^{-1} = 14 \cdot 4 = 56 \equiv 56 - 46 = 10$. Recovered. Figure 20.3 shows the same exchange schematically.

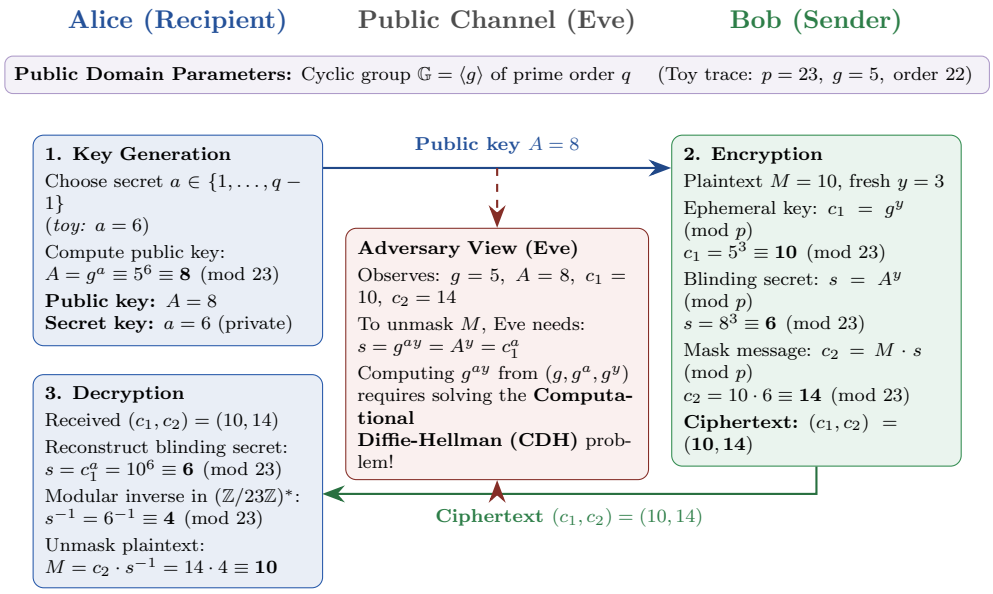


Figure 20.3: End-to-end flow of ElGamal encryption. Alice's long-term public key $A = g^a$ stays fixed. For each message, Bob generates an ephemeral key pair $(y, c_1 = g^y)$ and folds the message M into the one-time Diffie-Hellman secret $s = A^y = g^{ay}$. Alice raises Bob's ephemeral public value c_1 to her private key a to reproduce the identical blinding secret s, inverting it to recover M. Eavesdropper Eve, observing (g, g^a, g^y), faces the Computational Diffie-Hellman problem.

20.3.2 Why it is secure

Focus

Threat model. The attacker is a probabilistic polynomial-time eavesdropper who sees the public key and a ciphertext. The goal we promise to deny her is *indistinguishability under chosen-plaintext attack* (IND-CPA): she picks two messages M_0, M_1, we encrypt one of them at random, and she cannot guess which better than a coin flip. We do *not* promise IND-CCA security here – ElGamal as stated is malleable, as the homomorphic property below makes clear, and a chosen- ciphertext attacker breaks it. Real deployments either add a MAC/KEM-DEM wrapper or use a CCA-secure variant.

Definition 20.3 (Decisional Diffie-Hellman)**.** DDH is hard in $\mathbb{G} = \langle g \rangle$ of order q if no PPT distinguisher can tell apart the two distributions, over random $a, b, c \in \{0, \ldots, q-1\}$,

$$\big(g^a,\, g^b,\, g^{ab}\big) \qquad \text{vs.} \qquad \big(g^a,\, g^b,\, g^c\big),$$

with non-negligible advantage. In words: a real Diffie-Hellman triple is indistinguishable from one where the third coordinate is a random group element.

Theorem 20.1 (ElGamal is IND-CPA under DDH)**.** *If DDH is hard in $\mathbb{G}$, then ElGamal over $\mathbb{G}$ is IND-CPA secure.*

Why it works

Proof sketch. A ciphertext is $(g^y,\ M \cdot A^y)$ with $A = g^a$. Rewrite it as $(g^y,\ M \cdot g^{ay})$. The second coordinate hides M behind the mask g^{ay}, and the attacker's view is the triple (g^a, g^y, g^{ay}) *plus* the message folded into the third slot.
Now run a hybrid. Replace the genuine mask g^{ay} with a *uniformly random* group element g^c. By DDH, no PPT attacker notices the swap – the two worlds are computationally indistinguishable. But in the swapped world the ciphertext is $(g^y,\ M \cdot g^c)$ with g^c uniform and independent of M. A uniform group element times M is again uniform and carries *zero* information about M – it is a one-time pad in the group. So in the swapped world the attacker's advantage is exactly zero, and DDH bounds how much advantage she could have had in the real world:

$$\mathsf{Adv}^{\text{ind-cpa}}_{\text{ElGamal}}(\mathcal{A}) \;\le\; \mathsf{Adv}^{\text{ddh}}_{\mathbb{G}}(\mathcal{B}).$$

Encryption is secure precisely because the Diffie-Hellman secret g^{ay} is computationally indistinguishable from uniform to anyone without a or y, making it an effective one-time pad over the group.

Common trap

DDH must really hold – and in $(\mathbb{Z}/p\mathbb{Z})^*$ it often does not! DDH is *false* in the full group $(\mathbb{Z}/p\mathbb{Z})^*$, because the Legendre symbol (quadratic residue test) leaks one bit of g^{ab} that an attacker can predict from g^a and g^b. Concretely, whether g^{ab} is a quadratic residue is determined by whether a and b are even, which the QR-ness of g^a, g^b reveals. A distinguisher that just compares Legendre symbols wins. **Fix:** work in the prime-order subgroup of quadratic residues – the order-q subgroup of $(\mathbb{Z}/p\mathbb{Z})^*$ where $p = 2q+1$ – in which every element is a QR and the leak vanishes. This is why "ElGamal over $(\mathbb{Z}/p\mathbb{Z})^*$" done right means "ElGamal in the prime-order subgroup," not the whole group.

20.3.3 The homomorphic property

ElGamal has a structural feature that is sometimes a weakness and sometimes exactly what is wanted: multiplying two ciphertexts encrypts the product of the plaintexts. Take two encryptions of M and M' under the same key, with independent randomness y, y':

$$(c_1, c_2) = (g^y,\ MA^y), \qquad (c_1', c_2') = (g^{y'},\ M'A^{y'}).$$

Multiply componentwise:

$$(c_1 c_1',\ c_2 c_2') \;=\; \big(g^{y+y'},\ (MM')\, A^{y+y'}\big),$$

which is a valid ElGamal encryption of MM' under the same key, with randomness $y + y'$. The scheme is *multiplicatively homomorphic*.

Intuition

You can compute on the data without ever decrypting it. A tallying server can multiply together ciphertexts to combine encrypted values, and only the keyholder, at the end, learns the combined result – never the individual inputs. This idea leads to homomorphic encryption (Chapter 33) and of cryptographic voting. By tweaking the scheme to encrypt g^M instead of M ("exponential ElGamal"), multiplication of ciphertexts becomes *addition* of the messages in the exponent, which is exactly what you want for summing votes – at the price of having to solve a small discrete log at the end to read the tally off $g^{\sum M_i}$.

Attack

Malleability: the flip side. The same property breaks chosen-ciphertext security. Suppose Eve intercepts a ciphertext (c_1, c_2) for an unknown price M and wants to change it to encrypt $2M$. She does not need the key: she sends $(c_1,\ 2c_2)$, and the recipient decrypts $2c_2 \cdot s^{-1} = 2M$. She mauled the plaintext blind. Worse, against a decryption oracle she can recover M entirely: ask the oracle to decrypt $(c_1,\ r \cdot c_2)$ for a known random r; it returns rM; divide by r to get M. This is why textbook ElGamal is never used raw for confidentiality. Production systems wrap it: hash the Diffie-Hellman secret into a symmetric key and encrypt-then-MAC the payload (the KEM/DEM pattern of ECIES), which removes malleability by binding the ciphertext together under authentication (Figure 20.4).

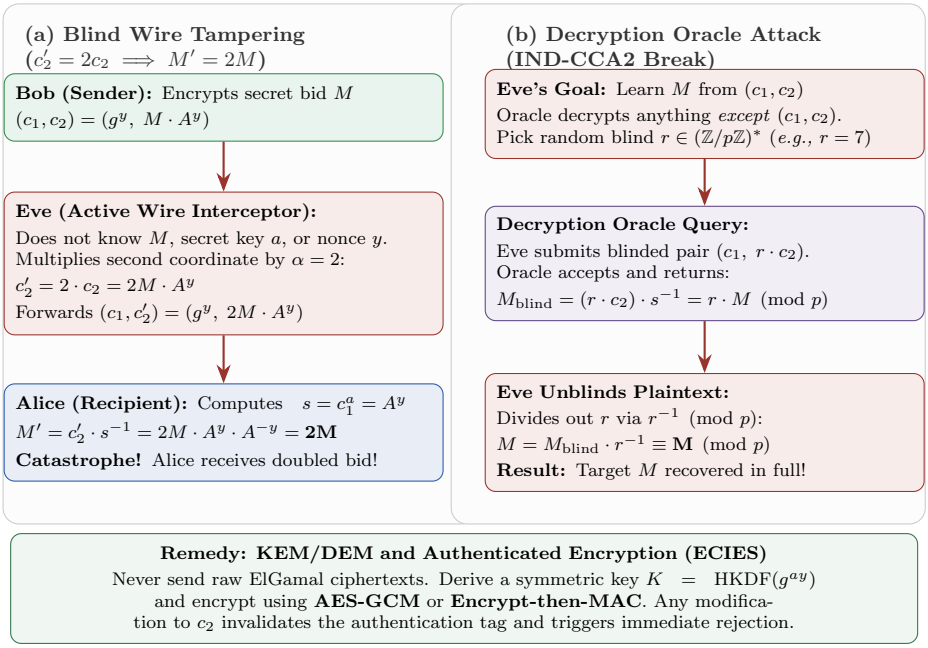


Figure 20.4: The malleability of ElGamal and why textbook encryption fails chosen-ciphertext security. (a) Multiplicative homomorphism allows an active wire attacker to scale plaintexts blind: multiplying c_2 by 2 doubles the decrypted plaintext without triggering an error. (b) Given access to a decryption oracle, Eve bypasses challenge-ciphertext filters by blinding c_2 with random factor r, querying the oracle on (c_1, rc_2), and multiplying the returned value by $r^{-1} \bmod p$ to recover M. Hardened systems use hybrid KEM/DEM schemes (ECIES) to bind the Diffie-Hellman secret to an authenticated symmetric ciphertext.

20.4 Choosing safe groups and parameters

Getting ElGamal (or Diffie-Hellman) right is mostly about the group. Three rules.

1. Use a prime-order subgroup. Work in a subgroup of prime order q, not in $(\mathbb{Z}/p\mathbb{Z})^*$ whole. The standard construction: pick a *safe prime* $p = 2q + 1$ with q also prime. The quadratic residues mod p then form a subgroup of order q, and you take g to be a generator of *that* subgroup. Prime order pays off in several ways at once: every non-identity element is a generator, there are no small subgroups to fall into, the Pohlig-Hellman attack (which would split the DLP across the prime factors of the order) has nothing to bite on, and DDH's Legendre-symbol leak disappears.

Focus

Why prime order kills small-subgroup attacks. If the order n factors as $n = q_1 q_2 \cdots$, Pohlig-Hellman solves the DLP separately modulo each q_i – in time $\sqrt{q_{\max}}$, governed by the *largest* factor – and reassembles by CRT. A composite order with all-small factors is catastrophic: order $2^{40} \cdot$ (small primes) would be breakable on a laptop regardless of how big p is. Prime order q means there is exactly one prime factor and it is the whole thing, so $\sqrt{q}$ is the true cost. This is the discrete-log analogue of "a chain is as strong as its weakest link."

2. Size for the best attack, on both fronts. Two attacks set the sizes. The generic attack (rho)

costs $\sqrt{q}$, so q must be at least 256 bits to reach the 2^{128} generic-security level. The index-calculus attack costs $L_p[1/3, \cdot]$ in the size of p, so p must be far larger – about 3072 bits for the same 128-bit target, and the older 1024-bit primes are now considered broken by well-resourced adversaries. You size p against index calculus and q against rho; the safe-prime construction $p = 2q + 1$ couples them, which is why it forces a large q along with a large p.

3. Fresh, uniform randomness for y every time. The per-message exponent y must be freshly sampled and never reused.

Attack

Nonce reuse breaks ElGamal completely. Suppose Bob lazily reuses the same y for two messages M_1, M_2. The ciphertexts are $(g^y, M_1 A^y)$ and $(g^y, M_2 A^y)$ – same c_1, so an eavesdropper instantly spots the reuse, and the ratio of second components is

$$\frac{M_1 A^y}{M_2 A^y} = \frac{M_1}{M_2},$$

leaking the ratio of the two plaintexts with the key entirely irrelevant. If she ever learns M_1 (a known header, a guessed field), she gets M_2 for free. This is the same lesson as IV reuse in stream ciphers and nonce reuse in GCM, and it is a recurring theme: the randomness in a randomized scheme is essential to its security. The signature cousin of ElGamal (DSA) is even less forgiving – reusing y there leaks the *secret key*, the bug that broke code signing on the Sony PlayStation 3.

Connection

The cost of the *forward* direction – computing g^y, A^y, c_1^a – is square-and-multiply modular exponentiation, $O(\log q)$ multiplications, each on numbers of size $\log p$. That is the algorithm analyzed in the modular-exponentiation section of the algorithms companion. The asymmetry that makes ElGamal possible is exactly the gap between this $O(\log q)$ forward cost and the $O(\sqrt{q})$ (or subexponential, via index calculus) backward cost. Public-key cryptography depends on this gap.

In the real world

Where this actually runs. Pure mod-p ElGamal is rare in modern transport security – it is slow and the ciphertext doubles the message size – but its structure appears in many deployed systems:

- **GnuPG / OpenPGP** has long offered ElGamal as an encryption key type (paired with DSA signatures), encrypting a random session key that then encrypts the actual email or file under a fast symmetric cipher – the hybrid pattern.
- **ECIES**, the Elliptic Curve Integrated Encryption Scheme, is ElGamal moved onto an elliptic-curve group and hardened: derive a symmetric key from the Diffie-Hellman secret with a KDF, then encrypt-then-MAC. It is the standard way to do public-key encryption to a recipient on a curve, used in messaging and in cryptocurrency wallets.
- **TLS** does not encrypt with ElGamal, but its ephemeral key exchange (DHE and ECDHE) is the Diffie-Hellman half of this story, run fresh per session for forward secrecy – the subject of Chapter 23.

- **Cryptographic voting** (Helios and relatives) uses *exponential* ElGamal exactly for its additive homomorphism: encrypted ballots are multiplied to tally without decrypting individual votes.

20.5 Relationship to Diffie-Hellman

The three hardness problems and the two schemes relate as follows:

- **Diffie-Hellman key exchange** is an interactive protocol: both parties contribute a fresh ephemeral exponent, they jointly compute g^{ab}, and they get a shared symmetric key. Nothing is "encrypted"; a key is *agreed*. Its security rests on CDH (to hide the key) and on DDH (to argue the key looks uniform).
- **ElGamal encryption** is non-interactive: one party (the recipient) has a long-lived key g^a; the sender runs a one-sided Diffie-Hellman with a fresh y and folds the message into the resulting secret. Security rests on DDH (Theorem 20.1).

So ElGamal *is* Diffie-Hellman, frozen into a public-key encryption scheme by making one side's contribution permanent and the other side's per-message. The shared secret g^{ay} plays in ElGamal exactly the role g^{ab} plays in DH. And the same hardness assumptions underwrite both: break the DLP and you break everything; break only CDH and you can read DH-keyed traffic and forge ElGamal masks; break only DDH and you cannot necessarily recover keys but you *can* distinguish ciphertexts, which is enough to break the IND-CPA guarantee.

Focus

Summary. Forward $(g, x \mapsto g^x)$ is cheap; backward $(g, g^x \mapsto x)$ is the DLP and is hard. Diffie-Hellman builds a shared secret in that gap; ElGamal freezes one half of the exchange into a public key and masks a message with the secret. All of these schemes – along with their elliptic-curve counterparts in Chapter 21 – rest on the same discrete logarithm problem.

20.6 Capabilities and Limitations of ElGamal

We have built a complete public-key encryption scheme out of the discrete logarithm problem. This section summarizes what the scheme provides, where it falls short, and how later chapters address its limitations.

Why it works

Diffie and Hellman gave us interactive key agreement, and RSA gave us trapdoor permutations based on factoring, but ElGamal showed how to freeze one half of Diffie–Hellman into a permanent public key to obtain randomized encryption directly from the Decisional Diffie–Hellman assumption. Unlike textbook RSA, which is deterministic and leaks when two plaintexts are equal, ElGamal's fresh nonce y ensures that encrypting the same message twice yields completely independent ciphertexts. At the same time, its clean multiplicative

structure $(c_1c_1', c_2c_2') = \mathrm{Enc}(MM')$ opened the door to computing directly on encrypted data, zero-knowledge proofs, and verifiable electronic voting systems such as Helios.

Common trap

These strengths come with real structural costs in $(\mathbb{Z}/p\mathbb{Z})^*$. Every ElGamal ciphertext carries two group elements (c_1, c_2), doubling the wire size so that a 3072-bit message requires 6144 bits of ciphertext. Worse, because ordinary integers admit prime factorizations into small primes, index-calculus algorithms solve discrete logs in $(\mathbb{Z}/p\mathbb{Z})^*$ in subexponential time $L_p[1/3, c]$, forcing p to be at least 3072 bits long for 128-bit security. And because an active attacker can multiply (c_1, c_2) by known factors to scale or blind the underlying plaintext, raw ElGamal is malleable under chosen-ciphertext attacks and must be wrapped in a hybrid KEM/DEM construction (such as ECIES) in practice – while remaining, like RSA, vulnerable to Shor's quantum algorithm.

Connection

The limitations of ElGamal over $(\mathbb{Z}/p\mathbb{Z})^*$ motivate several later chapters. In Chapter 21, we transfer the discrete logarithm problem to elliptic curves: because curves lack the multiplicative factor bases that power index calculus, generic square-root attacks are the best known, allowing us to drop key sizes from unwieldy 3072-bit integers down to compact 256-bit curve points at identical security. Chapter 22 turns ElGamal's ephemeral exponent into non-repudiable digital signatures via DSA and Schnorr. Later, Chapter 33 extends ElGamal's homomorphic property into fully homomorphic schemes that evaluate arbitrary computation on encrypted data, and Chapter 35 constructs post-quantum primitives over high-dimensional lattices to withstand Shor's algorithm.

20.7 Exercises

Exercise 20.1 (Hand-cranking discrete logs)**.** Work in $(\mathbb{Z}/13\mathbb{Z})^*$ with $g = 2$. (a) Verify 2 is a primitive root by listing all powers $2^0, \ldots, 2^{11} \bmod 13$. (b) Read off $\mathrm{dlog}_2 6$, $\mathrm{dlog}_2 7$, and $\mathrm{dlog}_2 11$ from your table. (c) Confirm the identity $\mathrm{dlog}_2(6 \cdot 7) \equiv \mathrm{dlog}_2 6 + \mathrm{dlog}_2 7 \pmod{12}$, and explain in one line why this additivity must hold.

Exercise 20.2 (Baby-step giant-step)**.** Let $p = 41$, $g = 6$ (a primitive root mod 41). Use baby-step giant-step with $m = \lceil\sqrt{40}\rceil = 7$ to compute $\mathrm{dlog}_6 29$. Show the baby-step table, the giant-step value $g^{-m} \bmod 41$, and the iteration that produces the hit. Verify your answer by direct exponentiation.

Exercise 20.3 (Pollard rho extraction)**.** In a group of prime order $q = 101$ with generator g, a rho walk reports a collision between $x_i = g^{12}h^7$ and $x_j = g^{83}h^4$, where $h = g^x$ is the unknown target. Solve the linear congruence for $x \bmod 101$. (You will need $3^{-1} \bmod 101$; find it.) Why was it essential that the order be prime for this step to go through cleanly?

Exercise 20.4 (ElGamal round trip)**.** Use $(\mathbb{Z}/23\mathbb{Z})^*$, $g = 5$, secret key $a = 9$. (a) Compute Alice's public key A. (b) Encrypt $M = 15$ using one-time randomness $y = 4$, giving (c_1, c_2). (c) Decrypt your ciphertext as Alice and confirm you recover 15. Show every modular reduction.

Exercise 20.5 (Homomorphic combination)**.** Continuing in $(\mathbb{Z}/23\mathbb{Z})^*$ with A from the previous exercise: encrypt $M_1 = 3$ with $y_1 = 2$ and $M_2 = 4$ with $y_2 = 5$. Multiply the two ciphertexts componentwise, decrypt the product ciphertext, and verify it yields $M_1 M_2 = 12 \bmod 23$. Then state what randomness the product ciphertext "uses," and confirm it is $y_1 + y_2$.

Exercise 20.6 (Nonce reuse)**.** Bob encrypts two messages to Alice under the same public key and accidentally reuses the same y, producing (c_1, c_2) and (c_1, c_2'). (a) How can an eavesdropper detect the reuse from the ciphertexts alone? (b) Show that the eavesdropper learns M/M' without the secret key. (c) If a known header tells her $M = 1000$ and she computes $M/M' = 8$, what is M'? (d) Contrast with DSA, where reusing the per-signature nonce leaks something far worse – name what.

Exercise 20.7 (The DDH leak in the full group)**.** Let p be an odd prime and g a primitive root, so g is a quadratic non-residue. (a) Show that g^x is a quadratic residue if and only if x is even. (b) Given g^a and g^b, explain how to predict the QR-ness of g^{ab}, and use this to build a DDH distinguisher with advantage $1/4$ in the full group $(\mathbb{Z}/p\mathbb{Z})^*$. (c) Explain why restricting to the order-q subgroup of quadratic residues (with $p = 2q + 1$) shuts this distinguisher down.

Exercise 20.8 (Pohlig-Hellman, why prime order matters)**.** Suppose a group has *smooth* order $n = 2 \cdot 3 \cdot 5 \cdot 7 = 210$. Sketch how Pohlig-Hellman reduces one discrete log in this group to four small discrete logs (mod $2, 3, 5, 7$), each solvable by inspection, recombined by the Chinese Remainder Theorem. Estimate the total work and compare it to $\sqrt{210}$. What does this say about ever using a group whose order has only small prime factors?

Exercise 20.9 (Malleability into a CCA break)**.** You are given an ElGamal decryption oracle (it decrypts anything except the exact challenge ciphertext (c_1, c_2)) and the challenge ciphertext itself, encrypting unknown M. Design a single oracle query that recovers M, using the multiplicative homomorphism. Why does encrypt-then-MAC over the ciphertext defeat your attack?

Exercise 20.10 (Sizing a real parameter set)**.** You want 128-bit security for ElGamal over $(\mathbb{Z}/p\mathbb{Z})^*$ with a safe prime $p = 2q + 1$. (a) Using the generic $\sqrt{q}$ bound, what is the minimum bit-length of q? (b) Given that index calculus on p costs roughly $L_p[1/3]$, mainstream guidance puts p at about 3072 bits for this level – explain why p must be *so much* larger than q. (c) An elliptic curve achieves the same 128-bit security with a 256-bit field. In one or two sentences, what structural difference lets the curve get away with the smaller parameter?

Exercise 20.11 (Prove the homomorphism)**.** Prove from the definitions that ElGamal is multiplicatively homomorphic: if (c_1, c_2) encrypts M with randomness y and (c_1', c_2') encrypts M' with randomness y', then $(c_1 c_1',\ c_2 c_2')$ is a valid encryption of MM', and identify its randomness. Then prove the "exponential ElGamal" claim: if instead you encrypt g^M and $g^{M'}$, the product ciphertext decrypts to $g^{M+M'}$, so reading off $M + M'$ requires one discrete log.

Chapter 21

Elliptic Curve Cryptography

By now you have met two great families of public-key cryptography. RSA leans on the hardness of factoring; Diffie–Hellman and ElGamal lean on the discrete logarithm problem in a multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$. Both work. Both are deployed. And both are, by modern standards, fat: a comfortable RSA key is 3072 bits, a comfortable finite-field Diffie–Hellman prime is the same. The reason is the same in both cases. The best known attacks — the number field sieve for factoring, index calculus for discrete logs — run in *sub-exponential* time. They are far better than brute force, so you have to pad the key length to stay ahead of them.

Elliptic-curve cryptography is the answer to a simple question: is there a group where the discrete logarithm problem is *harder* — where the clever index-calculus machinery does not apply, so the only attacks left are the generic ones that work in any group? If such a group exists, you can shrink the keys dramatically and get the same security. The answer is yes, and the group lives on an elliptic curve. A 256-bit elliptic-curve key matches a 3072-bit RSA key. That factor-of-ten saving is why your phone, your browser, and Signal almost all run on curves today.

This chapter builds the curve group from the ground up: the geometry, the algebra over a finite field, the worked arithmetic on tiny numbers, the elliptic-curve discrete logarithm problem (ECDLP) and why it resists the sub-exponential attacks, then ECDH, Curve25519, and the engineering reasons ECC won.

21.1 From a multiplicative group to a curve

Recall the shape of Diffie–Hellman. Fix a cyclic group $\mathbb{G}$ of prime order q with a generator g. Alice picks a secret a and publishes g^a; Bob picks b and publishes g^b; the shared secret is $g^{ab} = (g^a)^b = (g^b)^a$. Security rests on the discrete log problem: given g and g^a, recover a.

Nothing in that description mentions $(\mathbb{Z}/p\mathbb{Z})^*$ specifically. Diffie–Hellman needs *any* finite cyclic group in which:

- the group operation is cheap to compute,
- exponentiation (repeated operation) is cheap via square-and-multiply, but
- the discrete logarithm is hard.

The multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ satisfies the first two. The trouble is the third: $(\mathbb{Z}/p\mathbb{Z})^*$ has too much structure. Its elements are integers, and the index-calculus attack exploits the fact that small integers factor into small primes, building linear relations among discrete logs and solving the whole thing in sub-exponential time.

Intuition

Index calculus exploits multiplicative relations among small integers. In $(\mathbb{Z}/p\mathbb{Z})^*$ the elements can be lifted to ordinary integers $1, 2, 3, \ldots$, where $6 = 2 \cdot 3$ links the discrete logarithm of 6 to those of 2 and 3. Collect enough such relations and you can solve a linear system for all the small logs, then recover any target logarithm. Points on an elliptic curve have no analogous notion of "factoring into smaller points." There is no smooth subset from which to build relations.

So we go hunting for a different group — one whose elements are not integers and admit no factoring story, but that still has a clean, computable group law. Elliptic curves provide exactly that. The elements are points (x, y) on a curve, and the group operation is a geometric construction: draw a line, find where it hits the curve.

History

Elliptic curves are old mathematics, studied since the nineteenth century in connection with elliptic integrals (which compute arc lengths of ellipses — hence the name; the curves themselves are cubic equations, not ellipses). Their cryptographic use is young. In 1985 Neal Koblitz (University of Washington) and Victor Miller (IBM) independently proposed using the elliptic-curve group for discrete-log cryptography. For two decades ECC was treated as exotic and patent-encumbered; RSA dominated. The combination of expiring patents, the rise of mobile devices that cannot spare the cycles for 3072-bit RSA, and the 2013 disclosures that revealed a likely backdoor in the NIST-standardized random generator `Dual_EC_DRBG` finally tipped the balance. That incident poisoned trust in standards whose constants could not be explained, making the rigid, criteria-driven design of Daniel J. Bernstein's Curve25519 (2005) a major selling point; the IETF adopted it as a first-class option, and it now secures most of the web.

21.2 The curve and its points

Over a field K whose characteristic is not 2 or 3 (so we can divide by 2 and 3), an elliptic curve in *short Weierstrass form* is the set of solutions to

$$y^2 = x^3 + ax + b, \qquad a, b \in K, \tag{21.1}$$

together with one extra point "at infinity," written $\mathcal{O}$, which we will explain shortly. We require the cubic on the right to have no repeated roots, which is equivalent to the *nonsingularity* condition

$$4a^3 + 27b^2 \neq 0. \tag{21.2}$$

If the discriminant vanishes the curve develops a cusp or self-crossing, the group law breaks, and the cryptography collapses. Always check it.

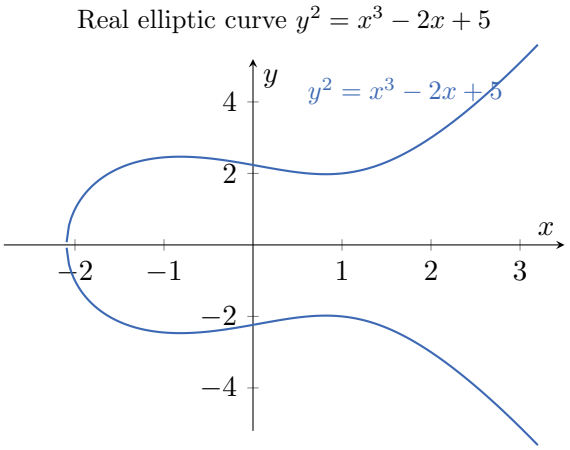


Figure 21.1: A real elliptic curve. It is symmetric about the x-axis: if (x, y) is on the curve so is $(x, -y)$, because only y^2 appears. This symmetry is what gives every point an inverse.

Over the real numbers the curve is a smooth one-dimensional shape, sometimes in one piece and sometimes in two (an oval plus an unbounded branch). It is the symmetry across the x-axis that matters: replacing y by $-y$ leaves the equation untouched, so the curve is its own mirror image.

21.2.1 The point at infinity

The extra point $\mathcal{O}$ is a genuine point of the group, needed to make the group law total. Picture it as sitting infinitely far up (and down) the y-axis, the place where all vertical lines meet. Every vertical line "passes through $\mathcal{O}$." It will be the identity element of our group — the analog of 1 in $(\mathbb{Z}/p\mathbb{Z})^*$, or of the all-zeros bit in $\oplus$.

Focus

A clean way to make $\mathcal{O}$ rigorous is *projective coordinates*. Write points as triples $[X : Y : Z]$ with $(X, Y, Z) \neq (0, 0, 0)$, where $[X : Y : Z]$ and $[\lambda X : \lambda Y : \lambda Z]$ are the same point for any $\lambda \neq 0$. The affine point (x, y) is $[x : y : 1]$, and the curve becomes $Y^2Z = X^3 + aXZ^2 + bZ^3$. Setting $Z = 0$ forces $X^3 = 0$, so $X = 0$, leaving the single point $[0 : 1 : 0]$. That is $\mathcal{O}$. You can carry on with the affine picture and just remember "vertical lines hit $\mathcal{O}$"; the projective view is there if you want the bookkeeping to be airtight.

21.3 The group law: geometry first

The following geometric rule turns the set of points into a group; the algebraic formulas later in the chapter are derived from it.

Focus

Chord-and-tangent addition. To add two points P and Q on the curve: draw the straight line through P and Q. Because the curve is a cubic, that line meets it in exactly three points

(counted with multiplicity); call the third one R'. Now reflect R' across the x-axis to get R. Define $P + Q = R$.

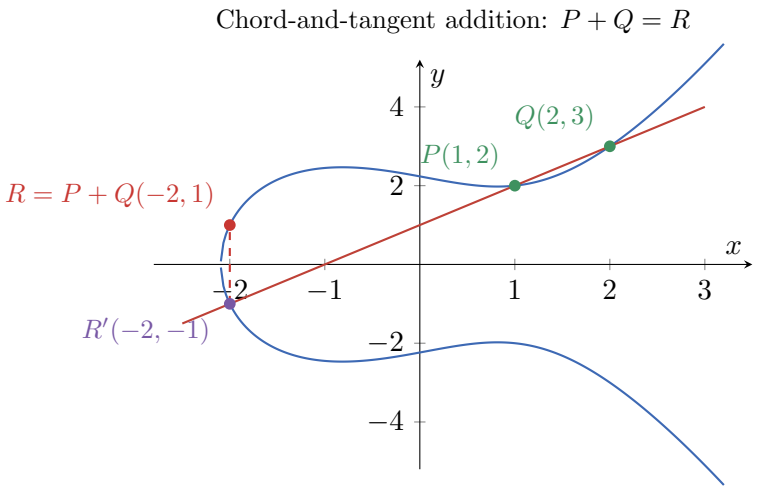


Figure 21.2: Adding two points on $y^2 = x^3 - 2x + 5$. The secant chord through $P(1, 2)$ and $Q(2, 3)$ meets the curve at a third point $R'(-2, -1)$; reflecting across the x-axis yields $R = P + Q = (-2, 1)$.

Three special cases complete the definition.

- **Adding $\mathcal{O}$.** For any point P, $P + \mathcal{O} = P$. The line through P and "infinity" is the vertical line at P; it meets the curve again at the reflection $(x_P, -y_P)$, and reflecting *that* back returns P. So $\mathcal{O}$ is the identity.

- **Inverses.** The reflection $-P = (x_P, -y_P)$ is the inverse of P. The line through P and $-P$ is vertical, so it "meets the curve at $\mathcal{O}$," and reflecting $\mathcal{O}$ gives $\mathcal{O}$. Hence $P + (-P) = \mathcal{O}$.

- **Doubling.** To compute $P + P$ when $P = Q$, use the *tangent* line at P (the limiting position of the chord as $Q \to P$). It meets the curve at one other point R'; reflect to get $2P$.

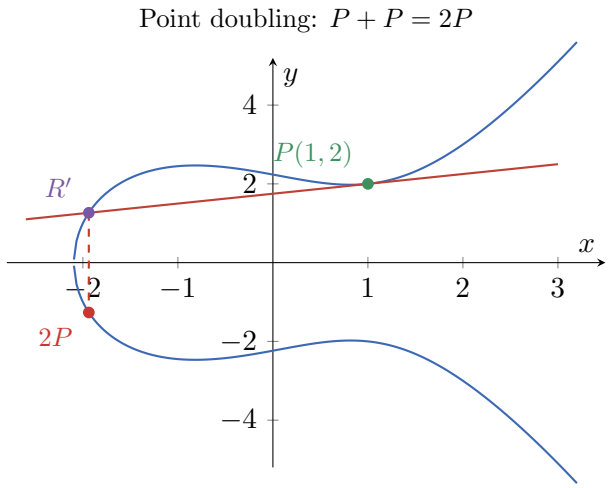


Figure 21.3: Doubling a point on $y^2 = x^3 - 2x + 5$. The tangent line at $P(1,2)$ meets the curve once more at $R'(-1.9375, 1.2656)$; reflecting across the horizontal axis gives $2P = P + P = (-1.9375, -1.2656)$.

Why it works

Why does this geometric recipe form a *group*? Closure and the existence of an identity and inverses are visible from the construction. Commutativity follows directly: the line through P and Q is identical to the line through Q and P, so $P + Q = Q + P$.

The one genuinely intricate property is **associativity**: $(P + Q) + S = P + (Q + S)$. Geometrically the two sides evaluate along completely different secant lines and tangents, yet they always land on the exact same final reflected point. A formal proof uses the theory of divisors or Bézout's theorem applied to cubics passing through eight shared points. Once associativity is established, the points of the curve form an abelian group with identity $\mathcal{O}$. That this algebraic structure carries over identically from continuous real space to finite fields $\mathbb{F}_p$ is what makes elliptic curves the premier setting for discrete-log cryptography.

21.4 The group law: algebra

Geometry gives us visual intuition; real cryptographic software needs explicit formulas. We translate "draw the line, find the third intersection, reflect" into coordinate algebra. The arithmetic works in any field K whose characteristic is not 2 or 3 — over $\mathbb{R}$ for geometric intuition, and over finite field $\mathbb{F}_p$ for real cryptography. The coordinate formulas are identical; only the arithmetic rules change.

Let $P = (x_1, y_1)$ and $Q = (x_2, y_2)$ be points with neither equal to $\mathcal{O}$.

Case 1: $P \neq \pm Q$ (a genuine chord)

The line through them has slope

$$m = \frac{y_2 - y_1}{x_2 - x_1}.$$

Substituting the line equation $y = m(x - x_1) + y_1$ into the curve $y^2 = x^3 + ax + b$ produces a single cubic in x:

$$(m(x - x_1) + y_1)^2 = x^3 + ax + b \implies x^3 - m^2x^2 + (a + 2m^2x_1 - 2my_1)x + \cdots = 0.$$

The roots of this cubic are the x-coordinates of the three intersection points: x_1, x_2, and the unknown third intersection x_3. By Vieta's formulas, the sum of the roots of any cubic $x^3 - c_2x^2 + c_1x - c_0 = 0$ equals the coefficient of x^2, which here is m^2:

$$x_1 + x_2 + x_3 = m^2 \implies x_3 = m^2 - x_1 - x_2.$$

The y-coordinate on the chord is $y = m(x_3 - x_1) + y_1$. Reflecting across the horizontal axis to obtain $P + Q$ simply negates this value:

$$x_3 = m^2 - x_1 - x_2, \qquad y_3 = m(x_1 - x_3) - y_1. \tag{21.3}$$

Case 2: $P = Q$ (the tangent, "doubling")

When $P = Q$, the secant chord becomes the tangent line at (x_1, y_1). Differentiating $y^2 = x^3 + ax + b$ implicitly gives $2y\,dy = (3x^2 + a)\,dx$, so the tangent slope is

$$m = \frac{3x_1^2 + a}{2y_1}.$$

The tangent line intersects the curve at x_1 with multiplicity two (a tangency is a double root). Vieta's sum of roots becomes $x_1 + x_1 + x_3 = m^2$, giving:

$$x_3 = m^2 - 2x_1, \qquad y_3 = m(x_1 - x_3) - y_1. \tag{21.4}$$

Case 3: $x_1 = x_2$ but $y_1 \neq y_2$, or $y_1 = 0$

If $x_1 = x_2$ and $y_1 = -y_2$, the two points are vertical reflections of each other: $Q = -P$. The line between them is strictly vertical, meeting the curve at the point at infinity: $P + (-P) = \mathcal{O}$. If $P = Q$ and $y_1 = 0$, the tangent line is vertical, giving $2P = \mathcal{O}$.

Formal

Summarizing for $P, Q \neq \mathcal{O}$ with $P \neq -Q$:

$$m = \begin{cases} \dfrac{y_2 - y_1}{x_2 - x_1} & P \neq Q, \\[2ex] \dfrac{3x_1^2 + a}{2y_1} & P = Q, \end{cases} \qquad \begin{aligned} x_3 &= m^2 - x_1 - x_2, \\ y_3 &= m(x_1 - x_3) - y_1. \end{aligned}$$

When computing over $\mathbb{F}_p$, every addition, subtraction, multiplication, and squaring is evaluated modulo p. Division by D is computed as multiplication by the modular multiplicative inverse $D^{-1} \pmod p$, obtained via the extended Euclidean algorithm.

21.4.1 Projective coordinates: eliminating modular inversions

The affine formulas above have a hidden performance cost: computing the slope m requires dividing by $(x_2 - x_1)$ or $2y_1$. Over a 256-bit prime field $\mathbb{F}_p$, a modular inversion is by far the most computationally expensive field operation. While a field multiplication takes a handful of CPU cycles, computing a modular inverse via the extended Euclidean algorithm (or via Fermat's Little Theorem $z^{-1} \equiv z^{p-2} \pmod{p}$) requires hundreds of operations — roughly 256 field multiplications for a 256-bit prime!

If an implementation performed scalar multiplication kP using naive affine coordinates, each bit of the scalar would demand one or two modular inversions. For a 256-bit scalar, that would mean approximately 384 inversions, choking handshake throughput.

Intuition

The projective insight. Instead of evaluating divisions immediately at every step, carry the denominators forward in an auxiliary coordinate Z. Defer all divisions until the very end of the computation, so hundreds of expensive inversions are replaced by slightly more cheap multiplications, ending with just *one* single inversion.

Two projective coordinate systems are widely used in practice:

- **Standard Projective Coordinates:** Represent the affine point (x, y) as a triple $[X : Y : Z]$ with $Z \neq 0$, where $x = X/Z$ and $y = Y/Z$. The curve equation becomes the homogeneous cubic

$$Y^2 Z = X^3 + aXZ^2 + bZ^3.$$

  The identity element $\mathcal{O}$ is represented unambiguously by $[0 : 1 : 0]$.

- **Jacobian Projective Coordinates:** Assign different weights to the coordinates: $x = X/Z^2$ and $y = Y/Z^3$. Substituting into the Weierstrass equation and clearing denominators gives:

$$Y^2 = X^3 + aXZ^4 + bZ^6.$$

  Here $\mathcal{O}$ is $[1 : 1 : 0]$ (or $[\lambda^2 : \lambda^3 : 0]$ for any $\lambda \neq 0$). Jacobian coordinates are favored for Weierstrass curves because point doubling can be computed in just 4 field multiplications and 4 field squarings ($4M + 4S$), and addition of two distinct points takes $12M + 4S$ — with **zero** modular inversions.

During a 256-bit scalar multiplication, the processor executes roughly 256 doublings and 128 additions entirely in Jacobian coordinates. At the very end of the double-and-add loop, the algorithm converts the final projective triple $[X : Y : Z]$ back to affine coordinates by computing a single modular inverse $Z^{-1} \pmod{p}$:

$$x = X \cdot (Z^{-1})^2 \pmod{p}, \qquad y = Y \cdot (Z^{-1})^3 \pmod{p}.$$

This replaces 384 costly inversions with one single inversion, accelerating curve operations by more than an order of magnitude.

21.5 Curves over a finite field

Everything above used a field only through $+, -, \times, \div$. Swap $\mathbb{R}$ for the finite field $\mathbb{F}_p$ (the integers mod a prime p) and the definitions carry over verbatim. The curve

$$E(\mathbb{F}_p) = \{(x, y) \in \mathbb{F}_p \times \mathbb{F}_p : y^2 \equiv x^3 + ax + b \pmod{p}\} \cup \{\mathcal{O}\}$$

is now a *finite* set of points, and the chord-and-tangent law makes it a finite abelian group. There is no picture of a smooth curve anymore — just a scatter of dots (Figure 21.4) — but the algebra is unchanged.

Example 21.1 (A tiny curve by hand)**.** Take $p = 13$ and the curve

$$E : y^2 = x^3 + 3x + 8 \pmod{13}.$$

Check nonsingularity: $4a^3 + 27b^2 = 4 \cdot 27 + 27 \cdot 64 = 108 + 1728 = 1836$, and $1836 = 141 \cdot 13 + 3$, so $\equiv 3 \not\equiv 0 \pmod{13}$. Good.

To list the points, for each $x \in \{0, \ldots, 12\}$ compute the right-hand side $f(x) = x^3 + 3x + 8 \bmod 13$, then ask whether it is a quadratic residue (a square) mod 13. The nonzero squares mod 13 are

$$1^2{=}1,\ 2^2{=}4,\ 3^2{=}9,\ 4^2{=}3,\ 5^2{=}12,\ 6^2{=}10,$$

i.e. $\{1, 3, 4, 9, 10, 12\}$, and each such value has two square roots $\pm y$. Tabulating:

x	$x^3 + 3x + 8 \bmod 13$	square?	y values	
0	8	no	—	
1	12	yes	$5, 8$	$(5^2 = 25 \equiv 12)$
2	9	yes	$3, 10$	
3	5	no	—	
4	6	no	—	
5	5	no	—	
6	8	no	—	
7	8	no	—	
8	11	no	—	
9	10	yes	$6, 7$	
10	11	no	—	
11	7	no	—	
12	4	yes	$2, 11$	

Counting: each "yes" row gives two points, and we add $\mathcal{O}$. There are 4 yes-rows, so 8 affine points plus $\mathcal{O}$, for a total of

$$\#E(\mathbb{F}_{13}) = 9.$$

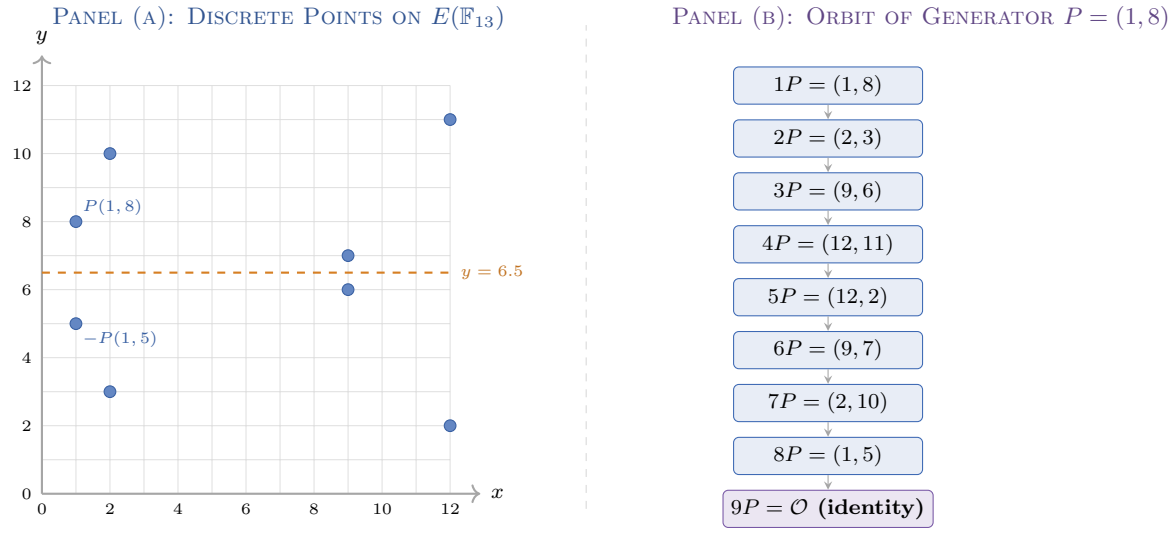


Figure 21.4: Panel (a): The 8 affine points of $E : y^2 = x^3 + 3x + 8$ over $\mathbb{F}_{13}$ (the ninth element is $\mathcal{O}$, off the plot). Reduction modulo p scatters points into a discrete point cloud, preserving the mirror symmetry about $y = 6.5$: each point (x, y) pairs with its inverse $(x, 13 - y)$. Panel (b): Repeated scalar multiplication by generator $P = (1, 8)$ visits every point in a pseudorandom cycle before terminating at $\mathcal{O}$.

Walking question

Add two points by hand. On the curve of Example 21.1, take $P = (1, 8)$ and $Q = (2, 3)$. Slope:

$$m = \frac{3 - 8}{2 - 1} = \frac{-5}{1} = -5 \equiv 8 \pmod{13}.$$

Then

$$x_3 = m^2 - x_1 - x_2 = 64 - 1 - 2 = 61 \equiv 61 - 52 = 9 \pmod{13},$$

$$y_3 = m(x_1 - x_3) - y_1 = 8(1 - 9) - 8 = 8(-8) - 8 = -64 - 8 = -72 \equiv -72 + 78 = 6 \pmod{13}.$$

So $P + Q = (9, 6)$. Sanity check that it is on the curve: $x^3 + 3x + 8 = 729 + 27 + 8 = 764$, and $764 = 58 \cdot 13 + 10$, so $\equiv 10$; meanwhile $y^2 = 36 \equiv 10 \pmod{13}$. They match. The point is on the curve.

Walking question

Double a point by hand. Take $P = (1, 8)$ and compute $2P$. Tangent slope:

$$m = \frac{3x_1^2 + a}{2y_1} = \frac{3 \cdot 1 + 3}{2 \cdot 8} = \frac{6}{16} = \frac{6}{3} \pmod{13},$$

since $16 \equiv 3$. Now $6/3$: we need 3^{-1} mod 13. Because $3 \cdot 9 = 27 \equiv 1$, we have $3^{-1} = 9$, so $m = 6 \cdot 9 = 54 \equiv 2 \pmod{13}$. Then

$$x_3 = m^2 - 2x_1 = 4 - 2 = 2,$$

$$y_3 = m(x_1 - x_3) - y_1 = 2(1 - 2) - 8 = -2 - 8 = -10 \equiv 3 \pmod{13}.$$

So $2P = (2, 3) = Q$. Interesting — on this curve $Q = 2P$, which means $P + Q = P + 2P = 3P$. Our earlier result $(9, 6)$ is therefore $3P$.

Work it yourself

Keep going from $P = (1, 8)$. You found $2P = (2, 3)$ and $3P = (9, 6)$. Compute $4P$ by adding P to $3P$, then $5P, 6P, \ldots$ until you return to $\mathcal{O}$. You will find that P generates a cyclic subgroup, and its order divides 9. (Spoiler: this P has order 9, so $\{\mathcal{O}, P, 2P, \ldots, 8P\}$ is the *entire* group — P is a generator.) Tracing the full cycle by hand once makes the cyclic structure concrete.

21.5.1 How many points? Hasse's theorem

The group $E(\mathbb{F}_p)$ had 9 points when $p = 13$. Is that a coincidence, or is there a pattern? Naively you might guess about p points: for each of the p values of x, the right-hand side $f(x)$ is a square (two y's), zero (one y), or a non-square (no y), and "square or not" is roughly a coin flip, so you expect roughly p affine points plus $\mathcal{O}$. That guess is right on average, and the deviation is tightly controlled.

Theorem 21.1 (Hasse, 1933)**.** *For an elliptic curve E over $\mathbb{F}_p$,*

$$\#E(\mathbb{F}_p) = p + 1 - t, \qquad |t| \le 2\sqrt{p}.$$

The integer t is called the trace of Frobenius.

So the number of points sits in the window $[\,p + 1 - 2\sqrt{p},\ p + 1 + 2\sqrt{p}\,]$, of width $4\sqrt{p}$ centered at $p + 1$. For $p = 13$ the window is $[14 - 7.2,\ 14 + 7.2] = [6.8, 21.2]$, and indeed 9 lands inside. For a 256-bit prime the group order is a 256-bit number known to within a relative error of about 2^{-127} — essentially p itself.

Why it works

Hasse's theorem is the quantitative reason ECC "just works." It guarantees the group is big (roughly p points), so the discrete-log search space is huge, and it lets a designer compute the exact order with a polynomial-time algorithm (Schoof's algorithm counts points without enumerating them). Knowing the exact order matters because secure ECC wants the order to be prime, or a small cofactor times a large prime. If the order factors into many small pieces, the Pohlig–Hellman attack (Section 21.6) chops the discrete log into easy sub-problems and the curve is dead.

21.6 Scalar multiplication and the ECDLP

In a multiplicative group we wrote the repeated operation as exponentiation g^a. On a curve the operation is written additively, so "repeating" means *scalar multiplication*:

$$kP = \underbrace{P + P + \cdots + P}_{k \text{ times}}.$$

This is the exact analog of g^k. And just as g^k is computed by square-and-multiply, kP is computed by *double-and-add*: write k in binary and process the bits.

Example 21.2 (Double-and-add)**.** To compute $13P$, note $13 = 1101_2$. Starting from $\mathcal{O}$ and reading bits left to right, double at each step and add P where the bit is 1:

$$\mathcal{O} \xrightarrow{1} P \xrightarrow{1} 2P{+}P{=}3P \xrightarrow{0} 6P \xrightarrow{1} 12P{+}P{=}13P.$$

Four doublings and two additions, instead of twelve additions. For a 256-bit scalar this is about 256 doublings and 128 additions — a few hundred curve operations — versus 2^{256} for the naive method.

Definition 21.1 (Elliptic-Curve Discrete Logarithm Problem)**.** Let E be an elliptic curve over $\mathbb{F}_p$ and let $P \in E(\mathbb{F}_p)$ be a point of large prime order n. Given P and $Q = kP$ for an unknown integer $k \in \{0, \ldots, n-1\}$, the **ECDLP** is to recover k.

Computing kP from k is easy (double-and-add). Going backward — recovering k from Q — is believed to be hard. ECC's security rests on that asymmetry, exactly as classic Diffie–Hellman's rests on the discrete log problem in $(\mathbb{Z}/p\mathbb{Z})^*$.

Intuition

Think of scalar multiplication as a deterministic but wild walk around the curve. Starting from P and repeatedly stepping, you visit $2P, 3P, 4P, \ldots$ in an order that looks random as a function of the coordinates. Given a destination Q, there is no shortcut that reads off the number of steps; you appear to have to walk. The best generic walk is a clever cycle-finding trick (Pollard's rho) that needs about $\sqrt{n}$ steps, not n — but $\sqrt{n}$ is still astronomically large when n is 256 bits.

21.6.1 Why ECC keys are small: generic versus special attacks

The following comparison is the central argument of this chapter. In *any* finite cyclic group of prime order n, the discrete log can be solved by generic algorithms — baby-step giant-step or Pollard's rho — in time about $\sqrt{n}$. These attacks treat the group as a black box; they work the same on $(\mathbb{Z}/p\mathbb{Z})^*$, on a curve, on anything. The $\sqrt{n}$ bound is a hard floor: Shoup proved that no generic algorithm can do better.

The difference between $(\mathbb{Z}/p\mathbb{Z})^*$ and an elliptic curve is what happens *beyond* the generic attacks.

- In $(\mathbb{Z}/p\mathbb{Z})^*$, the elements are integers, and **index calculus** exploits their factorization structure to solve discrete logs in *sub-exponential* time $L_p[1/3] = \exp\big(c\,(\ln p)^{1/3}(\ln \ln p)^{2/3}\big)$. This is far faster than $\sqrt{n}$. To stay secure you must inflate p until even sub-exponential time is infeasible — hence 3072-bit primes.
- On a well-chosen elliptic curve, **no index-calculus analog is known**. There is no factoring story for points, no smooth subset, nothing to build relations from. The best known attack is the generic $\sqrt{n}$. So n only has to be big enough to make $\sqrt{n}$ infeasible.

Why it works

"Infeasible" means roughly 2^{128} operations — the same security level AES-128 targets. For the generic attack to cost 2^{128} we need $\sqrt{n} \approx 2^{128}$, i.e. $n \approx 2^{256}$. That is why a curve over a 256-bit field gives 128-bit security with a 256-bit key.
For $(\mathbb{Z}/p\mathbb{Z})^*$ to reach 128-bit security against sub-exponential index calculus, you solve $L_p[1/3] \approx 2^{128}$, which lands around $p \approx 2^{3072}$. Same security, 12× the key length and far more arithmetic per operation. The table below compares the two across security levels.

Security level	Symmetric (AES)	RSA / finite-field DH	Elliptic curve
80	80	1024	160
112	112	2048	224
128	128	3072	256
192	192	7680	384
256	256	15360	512

(All entries in bits; these are the long-standing NIST equivalences.) The right column grows linearly with the security level while the RSA column explodes. That gap is why a 521-bit curve handles top-secret-grade traffic while the RSA equivalent needs 15360 bits and is painfully slow.

Connection

The $\sqrt{n}$ cost of Pollard's rho comes directly from the *birthday bound*. The algorithm generates a pseudorandom sequence of points and waits for a collision; by the birthday paradox a collision appears after about $\sqrt{n}$ steps, and that collision yields a linear equation modulo n that reveals k. The companion probability book derives the $\sqrt{(\text{space size})}$ birthday bound in general; here the "space" is the group of size n. The same mathematics here breaks a discrete log.

21.6.2 When the security collapses: pitfalls

ECC is only as strong as the curve and the implementation. Several traps have bitten real systems.

Attack

Pohlig–Hellman: composite group order. If the point's order n factors as $n = \prod n_i$ with small n_i, the ECDLP splits (by the Chinese remainder theorem) into independent subproblems, one modulo each n_i. Each is solvable in $\sqrt{n_i}$ time, and you reassemble k from the pieces. A curve whose order is, say, $2^3 \cdot 3 \cdot$ (500-bit prime) is only as strong as that 500-bit prime — fine — but a curve whose order is a product of many medium primes is broken. *Fix:* choose the point's order to be a large prime n, with at most a tiny cofactor $h = \#E/n$ (typically $h \in \{1, 2, 4, 8\}$).

Attack

Anomalous and supersingular curves. A handful of special curve families have an "extra" structure that defeats the generic floor. *Anomalous* curves, where $\#E(\mathbb{F}_p) = p$ exactly, admit

the Smart–Satoh–Araki attack: a map to the additive group $\mathbb{F}_p$ solves the discrete log in *linear* time. *Supersingular* curves admit the MOV/Frey–Rück attack, which uses a pairing to push the ECDLP into a small extension field $\mathbb{F}_{p^k}$ where index calculus applies. Standard curves are screened to avoid both. *Fix:* use a vetted curve, never one you rolled yourself.

Attack

Invalid-curve and small-subgroup attacks. If your code receives a point from the network and does *not* verify it actually lies on the agreed curve, an attacker can send a point on a different, weak curve (or a low-order point). The arithmetic formulas only use a (not b), so they happily compute on the wrong curve, and the attacker reads off your secret modulo a small number. Repeated, this recovers the whole key. This broke real TLS and Bluetooth implementations. *Fix:* always check that a received point satisfies the curve equation and is not $\mathcal{O}$ or low-order; or use a curve (like X25519) whose design neutralizes the attack.

Common trap

The lesson common to all of these is: **do not invent your own curve, and do not skip point validation.** The mathematics of secure curve selection (point counting, cofactor analysis, twist security, avoiding anomalous/supersingular families) is specialist work. Use NIST P-256, Curve25519, or another standardized curve, and validate every point you receive.

21.7 Elliptic-Curve Diffie–Hellman (ECDH)

With a hard ECDLP in hand, Diffie–Hellman transplants onto the curve with zero conceptual change. Replace the cyclic group $(\mathbb{Z}/p\mathbb{Z})^*$ by the curve subgroup $\langle P\rangle$ of prime order n, replace exponentiation g^a by scalar multiplication aP, and you have **ECDH**.

Focus

The ECDH protocol. Public parameters: a curve $E/\mathbb{F}_p$ and a base point P of prime order n.

1. Alice picks a secret scalar $a \xleftarrow{\$} \{1,\dots,n-1\}$ and sends $A = aP$.
2. Bob picks a secret scalar $b \xleftarrow{\$} \{1,\dots,n-1\}$ and sends $B = bP$.
3. Alice computes $S = aB = a(bP) = abP$.
4. Bob computes $S = bA = b(aP) = abP$.

Both arrive at the same point $S = abP$. They derive a symmetric key by hashing S (specifically its x-coordinate): $k = \mathrm{H}(x_S)$.

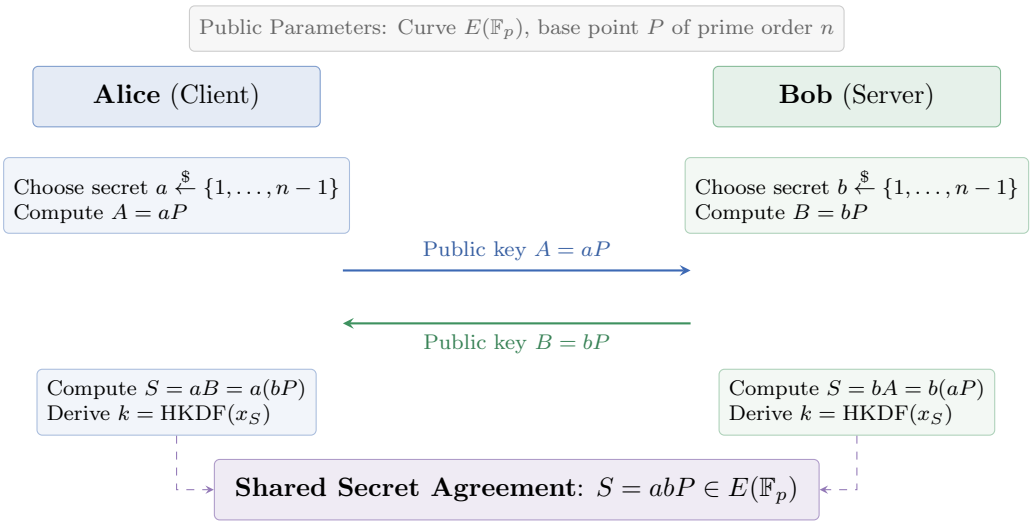


Figure 21.5: Elliptic-Curve Diffie–Hellman (ECDH) protocol flow. Each party generates an ephemeral scalar, multiplies the generator P to obtain their public point, exchanges points across the channel, and multiplies the received point by their own private scalar. Both arrive at the identical shared curve point $S = abP$.

Why it works

Eve sees P, $A = aP$, and $B = bP$. To get $S = abP$ she must either recover a from A or b from B — both ECDLP instances — or solve the *Computational Diffie–Hellman* problem on the curve directly: produce abP from aP and bP without either scalar. No method is known to do this faster than solving an ECDLP, which on a good 256-bit curve costs about 2^{128} operations. So ECDH inherits exactly the security of the underlying ECDLP. As with classic Diffie–Hellman, plain ECDH gives *no* authentication — an active attacker can man-in-the-middle it — so it is always paired with signatures or a pre-shared identity (the topic of Chapter 23).

Example 21.3 (ECDH on the toy curve)**.** Reuse $E : y^2 = x^3 + 3x + 8$ over $\mathbb{F}_{13}$ with base point $P = (1, 8)$ of order 9. Say Alice's secret is $a = 2$ and Bob's is $b = 3$.

- Alice sends $A = 2P = (2, 3)$ (computed earlier).
- Bob sends $B = 3P = (9, 6)$.
- Alice computes $aB = 2 \cdot (3P) = 6P = (9, 7)$.
- Bob computes $bA = 3 \cdot (2P) = 6P = (9, 7)$.

Both get $S = 6P = (9, 7)$, agreeing as required. (In a real system n is a 256-bit prime, not 9, so the secret scalars are astronomically large — but the mechanism is identical.)

Implement it

A toy ECDH in Python, written for clarity, not safety. The arithmetic mirrors the formulas of Section 21.4 exactly.

```python
# TOY elliptic-curve Diffie-Hellman -- FOR LEARNING ONLY.
# No constant-time arithmetic, no point validation, no side-channel
# protection, tiny insecure parameters. NEVER use this for anything real.

p, a, b = 13, 3, 8                  # curve y^2 = x^3 + 3x + 8 over F_13
O = None                            # the point at infinity (identity)

def inv(z):                         # modular inverse via Fermat (p prime)
    return pow(z % p, p - 2, p)

def add(P, Q):
    if P is O: return Q
    if Q is O: return P
    x1, y1 = P; x2, y2 = Q
    if x1 == x2 and (y1 + y2) % p == 0:
        return O                    # P == -Q  ->  infinity
    if P == Q:
        m = (3 * x1 * x1 + a) * inv(2 * y1) % p     # tangent slope
    else:
        m = (y2 - y1) * inv(x2 - x1) % p            # chord slope
    x3 = (m * m - x1 - x2) % p
    y3 = (m * (x1 - x3) - y1) % p
    return (x3, y3)

def scalar_mul(k, P):               # double-and-add
    R, base = O, P
    while k:
        if k & 1: R = add(R, base)
        base = add(base, base)
        k >>= 1
    return R

P = (1, 8)                          # base point, order 9 on this curve
a_sec, b_sec = 2, 3                 # Alice's and Bob's secret scalars
A = scalar_mul(a_sec, P)            # Alice's public point
B = scalar_mul(b_sec, P)            # Bob's public point
print("Alice:", scalar_mul(a_sec, B))   # a*(bP)
print("Bob:  ", scalar_mul(b_sec, A))   # b*(aP)  -- same point
```

Running this prints the same shared point for both parties. The warning in the comments matters: a real ECDH must use constant-time scalar multiplication (to resist the timing attacks of Chapter 37), must validate incoming points (to resist the invalid-curve attack above), and must use a vetted curve with a cryptographic field size.

21.8 Curve25519 and X25519

The most widely deployed curve today is not one of the original NIST curves but **Curve25519**, designed by Daniel J. Bernstein in 2005. It is the curve behind the **X25519** key-exchange function used in TLS 1.3, the Signal protocol, SSH, WireGuard, and many other protocols. The reasons it displaced the NIST curves are mostly matters of practical cryptographic engineering.

Curve25519 is a *Montgomery curve*,

$$E : y^2 = x^3 + 486662\,x^2 + x \quad \text{over} \quad \mathbb{F}_p, \qquad p = 2^{255} - 19,$$

a slightly different normal form from short Weierstrass but with the same group law in spirit. The prime $p = 2^{255} - 19$ is chosen to be just under a power of two, which makes reduction modulo p extremely fast on real hardware. The curve order is $8n$ where n is a large prime — cofactor 8 — and the design swallows that cofactor cleanly.

Why this matters

X25519 is fast, but its main advantage is that it removes *footguns*. Several historically dangerous decisions are taken out of the implementer's hands:

- **Montgomery ladder.** Scalar multiplication uses a ladder that performs the same operations regardless of the secret bits, making constant-time implementation natural — timing side channels are designed out.
- **x-coordinate only.** X25519 works with just the x-coordinate of points. This shrinks keys to 32 bytes and means every 32-byte string maps to a valid x, so there is no "is this point on the curve?" check to forget — the invalid-curve attack is structurally prevented.
- **Cofactor handling.** "Clamping" the scalar (forcing certain bits) guarantees the result lands in the large prime-order subgroup, neutralizing small-subgroup attacks.
- **Rigid design.** The constants are derived from clear criteria with little wiggle room, reducing the worry that a curve was secretly chosen to be weak — a concern that dogged the NIST curves after 2013.

Focus

Using X25519 (the right way). As an application programmer you never touch the group law. The interface is two functions:

- Generate a 32-byte random secret a; compute the public key $A = \texttt{X25519}(a, \text{basepoint})$.
- On receiving the peer's public key B, compute the shared secret $S = \texttt{X25519}(a, B)$, then run S through a key-derivation function (HKDF) to get your symmetric keys.

That is the whole API. The library handles clamping, the ladder, and reduction. This is the model you should follow everywhere in cryptography: use a vetted, narrow, hard-to-misuse interface, and never reach for the primitives directly.

21.8.1 Edwards curves and complete addition formulas

Every short Weierstrass curve carries a subtle engineering hazard: *exceptional cases*. Look back at the addition formulas of Section 21.4. If $P \neq Q$, you compute a chord slope; if $P = Q$, you compute a tangent slope; if $P = -Q$, you return $\mathcal{O}$; and if either input is $\mathcal{O}$, you return the other input. In software, every "if" statement whose condition depends on secret data is an invitation to a timing side-channel attack: an adversary measuring CPU execution time or CPU branch prediction caches can deduce which branch was taken and slowly reconstruct the secret scalar.

In 2007, Harold Edwards introduced an alternative algebraic model that is now widely used in

cryptographic engineering. An **Edwards curve** over a field K is the set of solutions to

$$x^2 + y^2 = 1 + d\,x^2y^2, \qquad d \notin \{0, 1\}. \tag{21.5}$$

When plotted over $\mathbb{R}$ with $d < 0$ (Figure 21.6), the curve looks like a smooth, symmetrical rounded square or distorted circle centered at the origin. The identity element is the ordinary affine point $(0, 1)$ rather than a point at infinity. The inverse of (x, y) is simply $(-x, y)$ — reflection across the vertical y-axis.

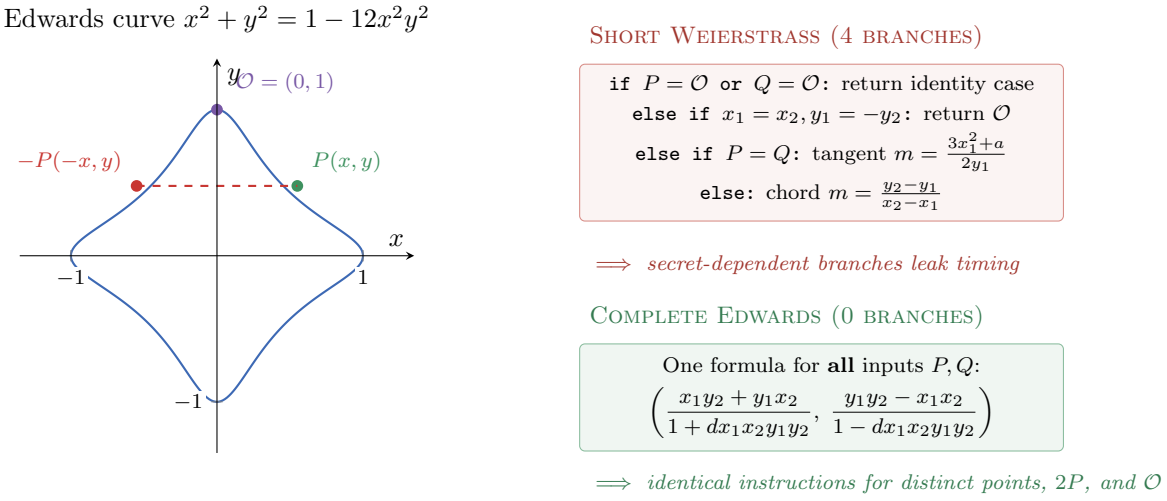


Figure 21.6: Left: An Edwards curve over $\mathbb{R}$ ($d = -12$) is a bounded, four-fold symmetric loop whose identity element is the ordinary affine point $\mathcal{O} = (0, 1)$ and whose inverse is horizontal reflection $-P = (-x, y)$. Right: Whereas short Weierstrass addition must branch on four exceptional cases, the Edwards addition law uses a single branch-free rational formula for every pair of points.

The addition law on an Edwards curve is symmetric in its two inputs:

$$(x_1, y_1) + (x_2, y_2) = \left(\frac{x_1y_2 + y_1x_2}{1 + d\,x_1x_2y_1y_2}, \frac{y_1y_2 - x_1x_2}{1 - d\,x_1x_2y_1y_2}\right). \tag{21.6}$$

Theorem 21.2 (Completeness of Edwards addition, Bernstein–Lange 2007)**.** *If d is not a quadratic residue in $\mathbb{F}_p$ (that is, d has no square root in $\mathbb{F}_p$), then the denominators $1 \pm d\,x_1x_2y_1y_2$ are **never zero** for any points $(x_1, y_1), (x_2, y_2) \in E(\mathbb{F}_p)$.*

This property is called **completeness**. Equation (21.6) works for *every* pair of points on the curve:

- It adds distinct points $P \neq Q$.
- It doubles points when $P = Q$: set $(x_2, y_2) = (x_1, y_1)$ and the formula yields the correct point doubling $2P$.
- It adds the identity $(0, 1)$: substituting $(x_2, y_2) = (0, 1)$ leaves (x_1, y_1) completely unchanged.
- It adds inverses: adding $(x, y) + (-x, y)$ evaluates directly to $(0, 1) = \mathcal{O}$.

There are no exceptional points, no separate doubling code, and no special branch paths. An implementation runs the exact same arithmetic instructions for every operation, so point addition has no secret-dependent branches to leak through timing.

Connection

Two coordinate models for Curve25519. Generalizing the equation slightly to $ax^2+y^2 = 1+dx^2y^2$ yields *Twisted Edwards curves*. The Montgomery curve Curve25519 used in X25519 is *birationally equivalent* to the Twisted Edwards curve

$$-x^2 + y^2 = 1 - \frac{121665}{121666}\,x^2y^2 \pmod{2^{255}-19}.$$

Every point on Curve25519 maps to a point on this Edwards curve through a simple rational transformation. Montgomery form is the fastest choice for Diffie–Hellman key exchange using only x-coordinates, while Twisted Edwards form is preferred for signatures: its complete addition formulas power **Ed25519**, the high-speed signature scheme we examine in Chapter 22.

In the real world

Where curves run the internet.

- **TLS 1.3** (HTTPS): the key exchange is ECDH, and X25519 is the default group in essentially every modern browser and server. When you load a web page, an elliptic-curve Diffie–Hellman almost certainly established the session key.
- **Signal / WhatsApp**: the X3DH handshake and the Double Ratchet both run on X25519, and identity signatures use Ed25519 (the signature scheme on the same curve, covered in Chapter 22).
- **SSH**: `curve25519-sha256` is the default key exchange in OpenSSH.
- **WireGuard** (modern VPNs): X25519 is the *only* key-exchange primitive — the protocol is deliberately built around a single hard-to-misuse curve.
- **Cryptocurrencies**: Bitcoin and Ethereum sign transactions with ECDSA on the curve `secp256k1`.

The common thread: small keys, fast operations, and side-channel-resistant designs make curves the right fit for phones, embedded devices, and high-volume servers alike.

21.9 Why ECC dominates: the summary

Elliptic-curve cryptography has four advantages over RSA and finite-field Diffie–Hellman.

1. **Small keys for equal security.** 256-bit ECC matches 3072-bit RSA. Less bandwidth per handshake, less storage per key, smaller certificates.
2. **Fast operations.** Scalar multiplication on a 256-bit curve is cheaper than a 3072-bit modular exponentiation, so handshakes are faster and servers carry more connections per core.
3. **Hard problem, no shortcut.** The ECDLP resists the sub-exponential index-calculus attacks that force RSA and finite-field DH to inflate their keys; the best attack on a good curve is generic $\sqrt{n}$.
4. **Misuse-resistant modern designs.** Curves like Curve25519 engineer away timing leaks, invalid-point attacks, and small-subgroup attacks at the level of the primitive, not the appli-

cation.

Reading note

One caveat: ECC, like RSA and finite-field DH, falls to a large-scale quantum computer. Shor's algorithm solves the ECDLP (and factoring, and finite-field discrete log) in polynomial time on a quantum machine. The smaller key does not help — in fact a 256-bit curve needs *fewer* logical qubits to break than a 3072-bit RSA modulus. Chapter 35 takes up what replaces curves when that day arrives; current practice is to run X25519 *alongside* a post-quantum scheme in a hybrid handshake, so the session is safe if either one holds.

What to remember

An elliptic curve over a finite field is a finite abelian group whose elements are points (x, y) satisfying $y^2 = x^3 + ax + b$, plus a point at infinity $\mathcal{O}$ serving as the identity. The group operation is chord-and-tangent addition, captured by explicit formulas built from one modular inversion. Repeated addition is scalar multiplication kP, computed by double-and-add; reversing it is the ECDLP. Because no index-calculus attack is known on a well-chosen curve, the ECDLP is harder per bit than discrete logs in $(\mathbb{Z}/p\mathbb{Z})^*$, so 256-bit curves deliver 128-bit security where RSA needs 3072 bits. ECDH transplants Diffie–Hellman onto the curve unchanged, and X25519 packages it in a fast, hard-to-misuse interface that now secures most of the internet's key exchanges.

Agreeing on a shared secret is not enough, however: without authentication, an adversary in the middle can impersonate either party. In Chapter 22 we turn to digital signatures, showing how RSA and elliptic curves bind messages and keys to verifiable identities.

21.10 Exercises

Exercise 21.1 (Point counting by hand)**.** Let $E : y^2 = x^3 + 2x + 3$ over $\mathbb{F}_7$. First verify nonsingularity via $4a^3 + 27b^2 \pmod 7$. Then, for each $x \in \{0, \dots, 6\}$, compute $f(x) = x^3 + 2x + 3 \bmod 7$, decide whether it is a quadratic residue (the nonzero squares mod 7 are $\{1, 2, 4\}$), and list all points. How many points does $E(\mathbb{F}_7)$ have, and does your count satisfy Hasse's bound $|\#E - 8| \le 2\sqrt{7} \approx 5.29$?

Exercise 21.2 (Adding and doubling)**.** On the curve $E : y^2 = x^3 + 3x + 8$ over $\mathbb{F}_{13}$, take $P = (1, 8)$. Using the algebraic formulas, compute $4P$ (add P to $3P = (9, 6)$), then $5P, \dots$ until you reach $\mathcal{O}$. Confirm that $9P = \mathcal{O}$, proving P has order 9. Show all modular-inverse computations. (You should find $4P = (12, 11)$ and $5P = (12, 2)$.)

Exercise 21.3 (A full group)**.** Continuing the previous exercise, write out the complete cyclic group $\langle P \rangle = \{\mathcal{O}, P, 2P, \dots, 8P\}$ with coordinates (here P generates all nine points). Verify that $4P + 5P = \mathcal{O}$ and that $-P = 8P$ by checking that $8P = (x_P, -y_P)$.

Exercise 21.4 (Toy ECDH)**.** Using the curve and base point of Exercise 21.2, let Alice's secret be $a = 3$ and Bob's be $b = 4$. Compute the public points $A = 3P$ and $B = 4P$, then the shared secret from each side ($3B$ and $4A$), and confirm they agree. What goes wrong with this "key exchange" for real use, given that P has order only 9?

Exercise 21.5 (Why no eavesdropper shortcut)**.** Explain, in terms of the ECDLP and the Computational Diffie–Hellman problem, exactly which computational problem Eve must solve to recover the ECDH shared secret from the transcript (P, aP, bP). Why does knowing aP and bP not let her "multiply" to get abP directly?

Exercise 21.6 (Key-size reasoning)**.** The best generic discrete-log attack runs in time about $\sqrt{n}$ where n is the subgroup order. (a) What size n (in bits) gives 128-bit security? (b) Explain why $(\mathbb{Z}/p\mathbb{Z})^*$ needs a roughly 3072-bit prime for the same 128-bit security, naming the attack responsible. (c) A junior engineer proposes a 128-bit curve for "128-bit security." What is the actual security level of a curve with a 128-bit group order, and why?

Exercise 21.7 (Pohlig–Hellman intuition)**.** Suppose a point P has order $n = 2^4 \cdot 3 \cdot 5 \cdot 7 \cdot$(a 30-bit prime). Roughly how much work is the ECDLP in $\langle P \rangle$, and which factor dominates? State the design rule this motivates for choosing the group order, and identify which attack class it defends against.

Exercise 21.8 (Invalid-curve attack)**.** The addition formulas of Section 21.4 never use the coefficient b. Explain how an attacker exploits this when a server fails to check that a received point lies on the agreed curve, and why working "on the wrong curve" can leak the server's secret scalar modulo small numbers. State the two-line fix.

Exercise 21.9 (Discriminant matters)**.** Show that the curve $y^2 = x^3$ (so $a = b = 0$) fails the nonsingularity test, and find the single point where it is singular. Explain geometrically why the chord-and-tangent construction breaks there (hint: what does the curve look like near the origin?).

Exercise 21.10 (Prove an inverse formula)**.** Prove from the group law that for any non-identity point $P = (x, y)$ on $y^2 = x^3 + ax + b$, the inverse is exactly $-P = (x, -y)$. (Show that $P + (x, -y)$ falls into the vertical-line case and therefore equals $\mathcal{O}$.) Then prove $-\mathcal{O} = \mathcal{O}$.

Exercise 21.11 (Why X25519 needs no point check)**.** X25519 operates on only the x-coordinate of points and accepts any 32-byte string as input. Argue informally why this design choice removes the invalid-curve attack of Exercise 21.8 as a class — that is, why there is no "off-curve" input an attacker can supply that the implementation must reject. (Hint: think about the quadratic twist and how clamping confines the result to the safe subgroup.)

Chapter 22

Digital Signatures

In Chapter 21, we saw how elliptic curves make Diffie–Hellman fast and compact, while leaving open how either party verifies who is on the other end of the connection. In the symmetric setting, we solved message authenticity with a MAC (Chapter 15), which proves that a message came from *somebody who holds the shared key*. That is enough when Alice and Bob already share a secret and trust each other. But it has two limits that turn out to matter enormously in the open world. First, the receiver can also *produce* valid tags – the same key both signs and verifies – so Bob cannot prove to a third party that Alice, and not Bob himself, made the message. Second, before any of this works, Alice and Bob must already share a secret, which is exactly the bootstrapping problem public-key cryptography was invented to solve.

A *digital signature* fixes both. Alice signs with a *private* key that only she holds; anyone with her *public* key can verify. The verifier cannot forge – verification and signing use different keys – so a valid signature is evidence, checkable by the whole world, that the holder of Alice's private key endorsed this exact message. That is *public verifiability*, and it brings with it *non-repudiation*: Alice cannot later credibly deny having signed, because nobody else could have produced the signature.

Signatures underpin most practical uses of public-key cryptography. Every time your browser trusts a certificate, every software update your phone installs, every block added to a cryptocurrency ledger, every SSH host key you accept – a signature was verified. This chapter builds them from the ground up: what they promise and what they do *not*; why "textbook" RSA signatures are dangerous and why we always hash-then-sign; the discrete-log signatures DSA, ECDSA, and EdDSA/Ed25519; and one of the most damaging implementation bugs in the field's history – nonce reuse in ECDSA, the bug that broke code signing on the Sony PlayStation 3 and emptied Bitcoin wallets. Small numerical examples run throughout.

22.1 Signatures Versus MACs

Both a MAC and a signature attach a short tag to a message so the receiver can detect tampering and confirm origin. The difference is *who can make a valid tag* and *who can check it*.

- **MAC:** one secret key k does everything. To make a tag you need k; to verify a tag you need k. Sender and verifier are symmetric – either can forge against the other.

- **Signature:** a key *pair* (pk, sk). The private sk makes signatures; the public pk only verifies them. The signer and the verifier are asymmetric – the verifier provably cannot forge.

This asymmetry between signer and verifier gives signatures three capabilities that symmetric MACs lack.

- **Public verifiability.** Anyone holding pk – which is not secret and can be published on a billboard – can check the signature. A MAC tag is checkable only by the holder of k.
- **No shared-secret setup.** The verifier needs only Alice's public key, which she can hand out freely. No secret channel is required before the first verified message.
- **Non-repudiation.** Because only Alice could have produced a signature that verifies under her pk, she cannot plausibly deny it later. A MAC has no such property: since Bob also holds k, a tag "from Alice" could just as easily have been forged by Bob, so it proves nothing to a judge.

Intuition

A MAC is a wax seal pressed with a signet ring that *both* correspondents own. If you find a sealed letter, you know one of the two ring-holders made it – but you cannot tell which, and you cannot show anyone else, because you own the ring too and might have faked it.
A signature is more like a handwritten signature backed by a worldwide notary: Alice alone can produce her mark (the private key), but *everyone* carries a reference copy to check it against (the public key). The asymmetry – easy to verify, impossible to forge – is what lets a stranger, or a court, rely on it.

Focus

Non-repudiation is a feature *and* a liability. Signatures are exactly the wrong tool for a casual chat where you might later want deniability. This is why Signal's messaging (Chapter 27) deliberately uses shared-key MACs for per-message authentication and reserves signatures for the long-term identity step – so that a leaked transcript proves nothing about who said what. "Deniable authentication" is a real design goal, and signatures destroy it.

22.1.1 The syntax of a signature scheme

Definition 22.1 (Signature scheme)**.** A *signature scheme* is a triple of algorithms $(\mathrm{Gen}, \mathrm{Sign}, \mathrm{Vrfy})$:

- $\mathrm{Gen}(1^n) \to (pk, sk)$: a randomized key generator producing a public verification key pk and a private signing key sk.
- $\mathrm{Sign}_{sk}(m) \to \sigma$: takes the private key and a message, returns a signature σ. May be randomized.
- $\mathrm{Vrfy}_{pk}(m, \sigma) \to \{0, 1\}$: takes the public key, a message, and a candidate signature, returns 1 (accept) or 0 (reject).

Correctness: for every key pair from Gen and every message m, $\mathrm{Vrfy}_{pk}(m, \mathrm{Sign}_{sk}(m)) = 1$.

22.1.2 What "secure" means: existential unforgeability

The security target mirrors the MAC definition, with one big change: the attacker is now *given the public key*. She can verify on her own all day long; she does not even need an oracle to verify. What she must not be able to do is produce a *new* valid message–signature pair.

Formal

EUF-CMA (existential unforgeability under chosen-message attack). Run Gen to get (pk, sk) and hand pk to the adversary Adv. Give Adv a signing oracle: she submits messages $m_1, m_2, \ldots$ of her choosing and receives $\sigma_i = \mathrm{Sign}_{sk}(m_i)$. Eventually Adv outputs a pair (m^*, σ^*). She *wins* if

$$\mathrm{Vrfy}_{pk}(m^*, \sigma^*) = 1 \quad \text{and} \quad m^* \notin \{m_1, m_2, \ldots\}.$$

The scheme is EUF-CMA secure if every PPT adversary wins with only negligible probability:

$$\Pr[\mathsf{Adv} \text{ wins}] \le \mathsf{negl}(n).$$

"Existential" means the forgery can be on *any* message Adv likes – even meaningless gibberish counts as a break. "Chosen-message" means she may first collect signatures on messages she picks. This is a deliberately paranoid bar: we demand that even after seeing signatures on a million chosen documents, she cannot forge on a single fresh one. A scheme that clears EUF-CMA is safe against far weaker real-world attackers.

Common trap

A subtle but important strengthening is *strong* unforgeability (sEUF-CMA): Adv wins even if m^* *was* queried, as long as σ^* is a *new* signature on it. Plain EUF-CMA permits "signature malleability" – producing a different valid signature on an already-signed message. That sounds harmless, but it has caused real damage: Bitcoin's pre-2017 "transaction malleability" bug let attackers alter a transaction's signature (and thus its ID) without invalidating it, which was used to confuse exchanges. We will see ECDSA's malleability concretely in Section 22.4.

22.2 RSA Signatures

RSA (Chapter 19) gives us a trapdoor permutation on $(\mathbb{Z}/N\mathbb{Z})^*$: raising to the public exponent e is easy for everyone, and the inverse – raising to the private exponent d – is easy only for the key-holder. Encryption used e to scramble and d to unscramble. Signing flips the roles: the signer applies the *private* operation, and verification applies the *public* one. Anyone can run the public map forward; only the key-holder can run it backward; so only the key-holder can produce a value whose public image is what it should be.

History

Diffie and Hellman named the goal — a public-key signature whose forgery is as hard as inverting a one-way function — in their 1976 paper, and Rivest, Shamir, and Adleman's 1978 paper gave the first construction alongside RSA encryption: "a message can be *signed* by decipher-

ing it with the secret deciphering key." Discrete-log signatures followed with Taher ElGamal (1985) and Claus Schnorr (1989). Because Schnorr's signature was patent-encumbered until 2008, NIST's 1991 federal standard, the *Digital Signature Algorithm* (DSA, in FIPS 186), was deliberately designed to sidestep that patent; it was also mildly controversial because some suspected NIST and the NSA of pushing a signature-only scheme they understood especially well. Once Schnorr's patent expired, a 2011 paper by Bernstein, Duif, Lange, Schwabe, and Yang ("High-speed high-security signatures") paired Schnorr's clean equation with edwards25519 to create EdDSA and Ed25519 (RFC 8032), which won rapid adoption across SSH, TLS 1.3, and Signal.

22.2.1 Textbook RSA, and why it is broken

The naive ("textbook") scheme is the direct dual of textbook RSA encryption. Keys: $N = pq$, public exponent e, private exponent d with $ed \equiv 1 \pmod{\varphi(N)}$.

$$\mathrm{Sign}_{sk}(m) = m^d \bmod N, \qquad \mathrm{Vrfy}_{pk}(m, \sigma) : \text{ accept iff } \sigma^e \equiv m \pmod{N}.$$

Correctness is the same Euler-theorem calculation that makes RSA decryption work: $\sigma^e = (m^d)^e = m^{ed} = m^{1+k\varphi(N)} \equiv m \pmod{N}$.

Let us trace it with the textbook tiny primes.

Example 22.1 (Textbook RSA signing by hand)**.** Take $p = 61$, $q = 53$, so $N = 3233$ and $\varphi(N) = 60 \cdot 52 = 3120$. Pick $e = 17$; then $d = 2753$ since $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$.

Sign the "message" $m = 100$. The signature is

$$\sigma = 100^{2753} \bmod 3233 = 1391.$$

(Compute by square-and-multiply; the algorithms companion covers the cost.) To verify, the public side computes

$$\sigma^e = 1391^{17} \bmod 3233 = 100 = m. \checkmark$$

The verifier used only the public $(N, e) = (3233, 17)$ and recovered m, confirming the signer knew d.

The scheme is correct but badly insecure. Three attacks break it.

Attack

1. Forge on a random "message". The forger does not need to choose the message first. Pick any σ you like, compute $m = \sigma^e \bmod N$, and announce "σ is my signature on m." It verifies by construction. The message m is garbage you did not control – but EUF-CMA says forging on *any* message is a win, and in a system where messages carry meaning (a bank balance, a serial number) a garbage message can still be damaging or can be massaged into something useful.

2. Multiplicative forgery (the deadly one). Textbook RSA is *homomorphic*: $(m_1 m_2)^d \equiv m_1^d \cdot m_2^d \pmod{N}$. So if Eve gets Alice to sign m_1 and m_2, she gets a free signature on $m_1 m_2 \bmod N$ that Alice never produced:

$$\sigma_1 = m_1^d, \quad \sigma_2 = m_2^d \implies \sigma_1\sigma_2 \bmod N = (m_1 m_2)^d = \mathrm{Sign}(m_1 m_2 \bmod N).$$

Worse, to forge on a *target* m^*, Eve writes $m^* = m_1 \cdot m_2 \bmod N$ for messages m_1, m_2 she chooses, asks Alice to sign those two (innocuous-looking) values, and multiplies the results. This is a textbook chosen-message break.

3. No binding to one message. Nothing ties σ to a single m; the structure is pure algebra, so algebraic relations among messages carry straight over to signatures.

Example 22.2 (Multiplicative forgery with tiny numbers)**.** Continue with $N = 3233$, $e = 17$, $d = 2753$. Eve wants a signature on $m^* = 4$ but Alice refuses to sign 4 directly. Eve factors the target: $4 = 2 \cdot 2 \bmod 3233$. She asks Alice to sign $m_1 = 2$:

$$\sigma_1 = 2^{2753} \bmod 3233 = 1027.$$

Then $\sigma^* = \sigma_1^2 \bmod 3233 = 1027^2 \bmod 3233 = 1054729 \bmod 3233 = 771$. Check: $771^{17} \bmod 3233 = 4 = m^*$. Eve forged a signature on 4 from a signature on 2. No knowledge of d required.

22.2.2 Hash-then-sign: the fix

Every attack above exploited *algebraic structure* in the message – the forger picked m's that multiply nicely or read m off backward from σ. The fix is to destroy that structure before signing: run the message through a cryptographic hash H first, then sign the digest (Figure 22.1).

$$\mathrm{Sign}_{sk}(m) = \mathrm{H}(m)^d \bmod N, \qquad \mathrm{Vrfy}_{pk}(m, \sigma): \text{ accept iff } \sigma^e \equiv \mathrm{H}(m) \pmod N.$$

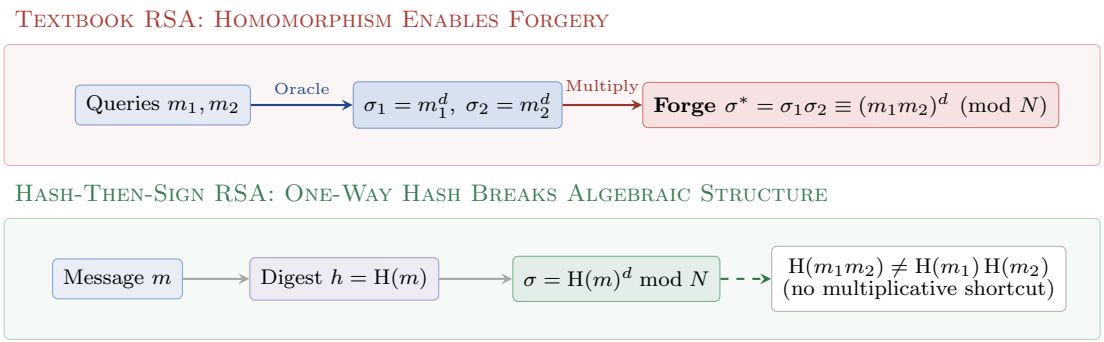


Figure 22.1: Why we always hash before signing. Top: In textbook RSA, the signing map $m \mapsto m^d \bmod N$ is multiplicatively homomorphic, allowing an attacker to multiply two valid signatures into a forgery on $m_1 m_2$. Bottom: Applying a cryptographic hash H before raising to d destroys the multiplicative relation between messages and digests.

Why it works

Hashing breaks all three attacks at once.

- **Against the random-σ forgery:** given a random σ, the forger gets a random target $t = \sigma^e \bmod N$. To pass verification she now needs a message m with $\mathrm{H}(m) = t$. That is a *preimage* on the hash – infeasible for SHA-256. She can pick σ or pick m, but not both consistently.

- **Against multiplicative forgery:** the homomorphism still holds at the level of integers, but now the forger needs three messages m_1, m_2, m^* with $\mathrm{H}(m^*) \equiv \mathrm{H}(m_1) \cdot \mathrm{H}(m_2) \pmod N$. Hash outputs are effectively random in $(\mathbb{Z}/N\mathbb{Z})^*$; finding such a multiplicative relation among hash digests is as hard as a fresh search and provides no leverage.
- **Bonus – arbitrary length:** RSA can only sign numbers below N. Hashing first compresses any message, however long, to a fixed-size digest.

Common trap

Hash-then-sign with a *bare* hash padded with zeros is still subtle. The deployed standard, **RSASSA-PKCS1-v1_5**, sandwiches the digest in a rigid byte structure `00 01 FF FF ... FF 00 || ASN.1(hashID) || digest`; **RSA-PSS** adds randomized salt and a mask, with a tight security proof in the random-oracle model. The infamous *Bleichenbacher signature-forgery* attack (2006) hit careless implementations that parsed the PKCS#1 padding loosely: if a verifier did not check that the `FF` padding ran all the way to the right edge, an attacker could, for small $e = 3$, construct a perfect cube whose low bits encoded a valid-*looking* structure, forging without the private key. The lesson: verification must check the padding *exactly*, byte for byte. Prefer PSS for new designs.

In the real world

RSA signatures with $e = 65537$ remain ubiquitous. The majority of TLS certificates in the field (Chapter 24) are RSA-signed; code-signing for Windows drivers and many firmware images uses RSA; DNSSEC, JWT (`RS256`), and PDF signing all rely on it. New systems increasingly prefer the smaller, faster Ed25519 (Section 22.6), but RSA's two-decade install base means it will be verified for a long time yet. When you see `RSASSA-PSS` or `sha256WithRSAEncryption` in a certificate, this is the machinery behind it.

Implement it

A toy RSA signature, hash-then-sign, in Python. **For learning only – never use this for anything real.** It uses tiny keys, a truncated hash reduced mod a small N, no PKCS#1/PSS padding, and Python's non-constant-time big-int arithmetic. Every one of those is a fatal flaw in production.

```python
import hashlib

# Toy keypair (textbook tiny primes -- NOT secure)
p, q = 61, 53
N = p * q                       # 3233
phi = (p - 1) * (q - 1)         # 3120
e = 17
d = pow(e, -1, phi)             # 2753

def H(msg: bytes) -> int:
    # Hash then reduce into Z_N. Real RSA uses full-width PSS padding.
    digest = hashlib.sha256(msg).digest()
    return int.from_bytes(digest, "big") % N

```

```python
def sign(msg: bytes) -> int:
    return pow(H(msg), d, N)              # private op: ^d

def verify(msg: bytes, sigma: int) -> bool:
    return pow(sigma, e, N) == H(msg)     # public op: ^e

s = sign(b"Pay Bob 100")
print(verify(b"Pay Bob 100", s))          # True
print(verify(b"Pay Eve 9999", s))         # False -- different hash
```

Tampering with the message changes the digest, so the recomputed $\mathrm{H}(m)$ no longer matches σ^e, and verification fails. These few lines show integrity and authenticity at work, but the warnings above still apply.

22.3 Discrete-Log Signatures: DSA

RSA signatures rest on the hardness of factoring. A second family rests on *discrete logarithms* (Chapter 20): given g and g^x in a cyclic group, recovering x is hard. The Digital Signature Algorithm (DSA), standardized by NIST in 1991, brought discrete-log signatures into federal standards and established the foundation for modern elliptic-curve variants. Because its signatures are substantially shorter than RSA's, the structure is worth understanding in detail.

22.3.1 Setup and the algorithm

DSA works in a subgroup of $(\mathbb{Z}/p\mathbb{Z})^*$. Public parameters: a large prime p, a prime q dividing $p-1$, and a generator g of the order-q subgroup (so $g^q \equiv 1 \pmod p$). The private key is $x \xleftarrow{\$} \{1, \ldots, q-1\}$; the public key is $y = g^x \bmod p$.

To sign a message m with digest $h = \mathrm{H}(m) \bmod q$:

1. Choose a fresh random *nonce* $k \xleftarrow{\$} \{1, \ldots, q-1\}$.
2. Compute $r = (g^k \bmod p) \bmod q$. If $r = 0$, restart with a new k.
3. Compute $s = k^{-1}(h + xr) \bmod q$. If $s = 0$, restart.
4. The signature is (r, s).

To verify (r, s) on m (with $h = \mathrm{H}(m) \bmod q$), reject unless $0 < r < q$ and $0 < s < q$, then:

$$w = s^{-1} \bmod q, \quad u_1 = h\,w \bmod q, \quad u_2 = r\,w \bmod q, \quad v = \left(g^{u_1} y^{u_2} \bmod p\right) \bmod q.$$

Accept iff $v = r$.

Why it works

Why verification works. From step 3, $s = k^{-1}(h + xr) \bmod q$, so $k \equiv s^{-1}(h + xr) \equiv w(h + xr) \equiv wh + wxr \equiv u_1 + xu_2 \pmod q$. Now raise g to both sides, working in the order-q

subgroup where exponents only matter mod q:

$$g^k \equiv g^{u_1+xu_2} = g^{u_1}(g^x)^{u_2} = g^{u_1}y^{u_2} \pmod{p}.$$

Reducing both sides mod q gives $r = v$. The signer's secret x appears on the right only through the public $y = g^x$, so the verifier never needs x – exactly the asymmetry we want.

Example 22.3 (DSA by hand with tiny parameters)**.** Let $p = 23$, $q = 11$ (note $11 \mid 22$). We need g of order 11. Take $h_0 = 2$ and set $g = h_0^{(p-1)/q} = 2^2 = 4 \bmod 23$. Check: $4^{11} \bmod 23 = 1$, and $4 \neq 1$, so $\mathrm{ord}(g) = 11$. Good.

Private key $x = 3$; public key $y = g^x = 4^3 = 64 \bmod 23 = 18$.

Sign a message whose digest is $h = 9$. Pick nonce $k = 7$ (with $k^{-1} \bmod 11 = 8$, since $7 \cdot 8 = 56 = 5 \cdot 11 + 1$).

$$r = (g^k \bmod p) \bmod q = (4^7 \bmod 23) \bmod 11.$$
$$4^7 = 16384, \quad 16384 \bmod 23 = 8, \quad r = 8 \bmod 11 = 8.$$
$$s = k^{-1}(h + xr) \bmod q = 8\,(9 + 3 \cdot 8) \bmod 11 = 8 \cdot 33 \bmod 11 = 8 \cdot 0 = 0.$$

Here $33 \equiv 0 \pmod{11}$ gives $s = 0$, which DSA forbids – so we restart. Take $k = 4$ instead (with $k^{-1} \bmod 11 = 3$, since $4 \cdot 3 = 12 \equiv 1$).

$$r = (4^4 \bmod 23) \bmod 11 = (256 \bmod 23) \bmod 11 = 3 \bmod 11 = 3.$$
$$s = 3\,(9 + 3 \cdot 3) \bmod 11 = 3 \cdot 18 \bmod 11 = 54 \bmod 11 = 10.$$

Signature: $(r, s) = (3, 10)$.

Verify. $w = s^{-1} = 10^{-1} \bmod 11 = 10$ (since $10 \cdot 10 = 100 = 9 \cdot 11 + 1$).

$$u_1 = hw = 9 \cdot 10 = 90 \bmod 11 = 2,$$
$$u_2 = rw = 3 \cdot 10 = 30 \bmod 11 = 8,$$
$$v = (g^{u_1}y^{u_2} \bmod 23) \bmod 11 = (4^2 \cdot 18^8 \bmod 23) \bmod 11.$$

Now $4^2 = 16$. And $18^2 = 324 \bmod 23 = 2$, so $18^4 = 2^2 = 4$, $18^8 = 4^2 = 16 \bmod 23$. Thus $g^{u_1}y^{u_2} = 16 \cdot 16 = 256 \bmod 23 = 3$, and $v = 3 \bmod 11 = 3 = r$. Accept. ✓

Common trap

DSA's security depends directly on how the nonce k is generated, and flawed nonce generation is the most common way real implementations fail. It must be *uniformly random*, *secret*, and *unique per signature*. Reusing it, or letting it be predictable or biased, leaks the private key outright. Section 22.5 is devoted to exactly this, because nonce failures have broken real, shipped systems.

22.4 ECDSA: DSA on an Elliptic Curve

ECDSA is DSA with the multiplicative group $(\mathbb{Z}/p\mathbb{Z})^*$ replaced by the additive group of points on an elliptic curve (Chapter 21). The benefit is the usual one for elliptic curves: equivalent security at

far smaller key and signature sizes, because the best discrete-log attack on a well-chosen curve is the generic $\sqrt{|\text{group}|}$ square-root attack. A 256-bit curve gives roughly 128-bit security; matching that with DSA over $(\mathbb{Z}/p\mathbb{Z})^*$ needs a 3072-bit p.

Public parameters: a curve E over $\mathbb{F}_p$, a base point G of large prime order n. Private key $d \xleftarrow{\$} \{1, \ldots, n-1\}$; public key $Q = dG$ (scalar multiplication, the elliptic-curve analogue of g^x).

Sign m with $h = \mathrm{H}(m) \bmod n$:

1. Pick a fresh random nonce $k \xleftarrow{\$} \{1, \ldots, n-1\}$.
2. Compute the point $(x_1, y_1) = kG$; set $r = x_1 \bmod n$. If $r = 0$, restart.
3. Compute $s = k^{-1}(h + d\,r) \bmod n$. If $s = 0$, restart.
4. Signature is (r, s).

Verify (r, s) on m: reject unless $0 < r, s < n$; then

$$w = s^{-1} \bmod n, \quad u_1 = hw \bmod n, \quad u_2 = rw \bmod n, \quad (x_1, y_1) = u_1 G + u_2 Q.$$

Accept iff $x_1 \bmod n = r$.

The correctness argument is identical in shape to DSA's: $k \equiv w(h + dr) \equiv u_1 + du_2 \pmod n$, so $kG = u_1 G + du_2 G = u_1 G + u_2 Q$, and the x-coordinates match.

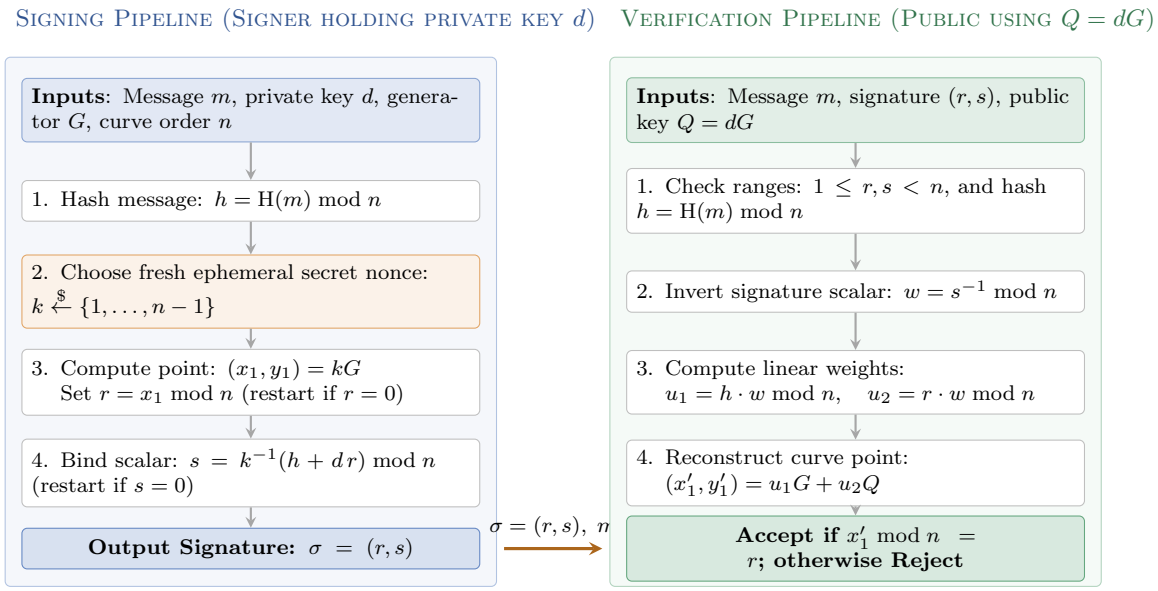


Figure 22.2: The ECDSA signature generation and verification pipeline. The signer binds the message hash h and private key d to an ephemeral point kG. The verifier uses the public key $Q = dG$ to reconstruct the ephemeral point and confirms its x-coordinate matches r.

Example 22.4 (ECDSA on a tiny curve)**.** Use the curve $E : y^2 = x^3 + 2x + 2$ over $\mathbb{F}_{17}$, a standard small teaching curve. The point $G = (5, 1)$ has order $n = 19$. (You can verify $19G = \mathcal{O}$; the group has 19 points.)

Take private key $d = 7$. Then $Q = 7G$. Doubling and adding on this curve (Chapter 21's point arithmetic) gives the multiples; in particular $7G = (0, 6)$, so $Q = (0, 6)$.

Sign a message with digest $h = 11$. Pick nonce $k = 3$. Compute $kG = 3G$. On this curve $3G = (10, 6)$, so $r = x_1 \bmod n = 10 \bmod 19 = 10$.

Now $k^{-1} \bmod 19$: $3 \cdot 13 = 39 = 2 \cdot 19 + 1$, so $k^{-1} = 13$.

$$s = k^{-1}(h + dr) \bmod n = 13\,(11 + 7 \cdot 10) \bmod 19 = 13 \cdot 81 \bmod 19.$$

$81 \bmod 19 = 81 - 76 = 5$, so $s = 13 \cdot 5 = 65 \bmod 19 = 65 - 57 = 8$. Signature $(r, s) = (10, 8)$.

Verify. $w = s^{-1} = 8^{-1} \bmod 19 = 12$ (since $8 \cdot 12 = 96 = 5 \cdot 19 + 1$).

$$u_1 = hw = 11 \cdot 12 = 132 \bmod 19 = 132 - 114 = 18, \quad u_2 = rw = 10 \cdot 12 = 120 \bmod 19 = 6.$$

Compute $u_1G + u_2Q = 18G + 6Q$. Since $Q = 7G$, this is $18G + 42G = 60G$. And $60 \bmod 19 = 60 - 57 = 3$, so the result is $3G = (10, 6)$, with $x_1 = 10$. Then $x_1 \bmod n = 10 = r$. Accept. ✓

The shortcut $Q = 7G \Rightarrow u_2Q = 42G$ is just for hand-checking; a real verifier does the two scalar multiplications directly without knowing d.

Common trap

ECDSA is malleable. If (r, s) is a valid signature, so is $(r,\ n - s)$: negating s corresponds to negating the nonce point's y-coordinate, which leaves $r = x_1 \bmod n$ unchanged. Both verify. This is the source of Bitcoin's transaction-malleability headaches; the fix (BIP 62 / "low-s" rule) is to canonicalize by requiring $s \le n/2$, rejecting the high-s twin. EdDSA avoids the whole issue by being deterministic and strongly unforgeable.

In the real world

ECDSA over the NIST curve P-256 (and increasingly P-384) is the default signature in modern TLS certificates and in the TLS handshake's `CertificateVerify`. It signs Apple and Android app packages, secures Bluetooth pairing, and – over the Koblitz curve secp256k1 – authorizes every Bitcoin and Ethereum transaction. Its compactness (a 64-byte signature for 128-bit security) is why it displaced RSA in size-constrained settings. Its sharp edge is the nonce, which the next section demonstrates concretely.

22.5 The Catastrophic Nonce Attack

DSA and ECDSA share a fragility so severe it deserves its own section: the security of the *long-term private key* rests entirely on the per-signature nonce k being random, secret, and never repeated. Violate any of those and the key falls out with grade-school algebra. We show it for ECDSA; DSA is identical with G replaced by g and n by q.

22.5.1 Reusing one nonce reveals the key

Suppose Alice signs two *different* messages with digests $h_1 \neq h_2$ using the *same* nonce k. Then both signatures share the same r, because r depends only on k: $r = x\text{-coord}(kG) \bmod n$. The two s values are

$$s_1 = k^{-1}(h_1 + dr) \bmod n, \qquad s_2 = k^{-1}(h_2 + dr) \bmod n.$$

Subtract. The dr term cancels:

$$s_1 - s_2 \equiv k^{-1}(h_1 - h_2) \pmod{n} \implies k \equiv \frac{h_1 - h_2}{s_1 - s_2} \pmod{n}.$$

That recovers the nonce. With k in hand, rearrange either signature equation for the private key:

$$s_1 k \equiv h_1 + dr \pmod{n} \implies d \equiv \frac{s_1 k - h_1}{r} \pmod{n}.$$

Two signatures, a subtraction, and two modular inverses recover the entire private key from public data. An attacker who merely *observes* two same-nonce signatures (they are public!) computes d and can forge anything forever.

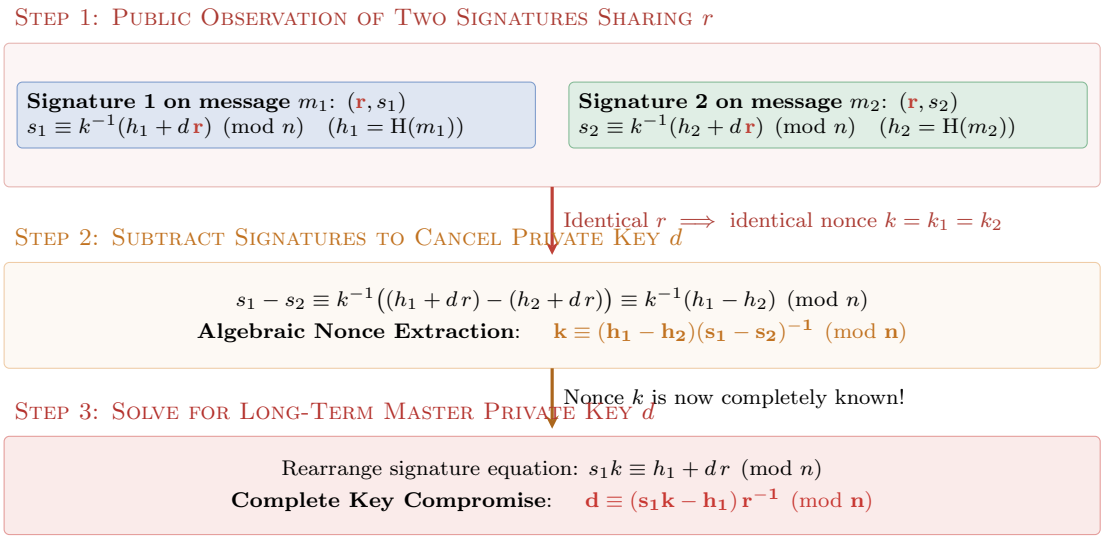


Figure 22.3: The ECDSA repeated-nonce key extraction attack (the Sony PlayStation 3 disaster). Because $r = (kG)_x$ depends solely on the nonce k, reusing k across two distinct messages produces the same r. Subtracting the signature equations eliminates the secret key d, allowing the attacker to solve directly for k, and then immediately extract d.

Example 22.5 (Extracting the key from a reused nonce)**.** Use the tiny ECDSA setup of Example 22.4: curve over $\mathbb{F}_{17}$, $n = 19$, true secret $d = 7$ (which the attacker does not know). Alice carelessly reuses $k = 3$ (so $r = 10$ in both) on two messages:

$$h_1 = 11: \ s_1 = 8 \text{ (from Example 22.4)}, \qquad h_2 = 5: \ s_2 = k^{-1}(h_2 + dr) = 13(5 + 70) \bmod 19.$$

$5 + 70 = 75 \equiv 75 - 57 = 18 \pmod{19}$, so $s_2 = 13 \cdot 18 = 234 \bmod 19 = 234 - 228 = 6$.

Eve sees $(r, s_1) = (10, 8)$ and $(r, s_2) = (10, 6)$ with the same r – the red flag. She recovers:

$$k \equiv \frac{h_1 - h_2}{s_1 - s_2} = \frac{11 - 5}{8 - 6} = \frac{6}{2} = 3 \pmod{19}.$$

($2^{-1} \bmod 19 = 10$, so $6 \cdot 10 = 60 \bmod 19 = 3$. Indeed $k = 3$.) Then

$$d \equiv \frac{s_1 k - h_1}{r} = \frac{8 \cdot 3 - 11}{10} = \frac{13}{10} \bmod 19.$$

$10^{-1} \bmod 19 = 2$ (since $10 \cdot 2 = 20 \equiv 1$), so $d = 13 \cdot 2 = 26 \bmod 19 = 7$. Eve has recovered the secret key $d = 7$ exactly, from two public signatures.

Attack

The PlayStation 3 break (2010). Sony signed PS3 firmware and games with ECDSA. The whole chain of trust rested on those signatures: only Sony-signed code would run. At the Chaos Communication Congress (27C3), the group `fail0verflow` revealed that Sony's signing implementation used a *constant* nonce k – the same value for every signature, rather than a fresh random one each time. Every pair of signed binaries therefore shared r, and the two-signature subtraction above handed over Sony's master private key.
With the private key, anyone could sign arbitrary code that the PS3 would accept as genuine Sony software – a complete, unrecoverable break of the platform's root of trust. There was no patch: you cannot un-leak a private key, and the public key was burned into millions of consoles. It is the standard real-world example of how ECDSA's security depends entirely on generating the nonce correctly.

Attack

Android Bitcoin wallet theft (2013). A flaw in the `SecureRandom` of certain Android versions caused some Bitcoin wallet apps to generate *repeated* ECDSA nonces across transactions. Because every Bitcoin signature is published forever on the blockchain, anyone scanning the chain for two signatures sharing an r value could apply Example 22.5's arithmetic, extract the wallet's private key, and sweep its funds. Real coins were stolen. The combination of permanently public signatures and a faulty RNG is especially dangerous: anyone can search the entire history for repeated nonces, and stolen coins cannot be recovered.

Connection

You do not even need a full *repeat* to lose the key – a slight *bias* is enough. If each nonce leaks just a few bits (say its top byte is always zero, or it comes from a too-short range), then each signature gives a noisy linear equation in d, and gathering a few hundred such equations turns key recovery into a *lattice* problem (the hidden number problem) that LLL-style lattice reduction solves. The probability companion's treatment of accumulating a faint statistical signal across many samples applies directly: each biased nonce is a weak measurement, and enough weak measurements pin down the secret. "Random enough" is therefore a precise, quantitative requirement.

22.5.2 The fix: deterministic nonces (RFC 6979)

The cleanest defense removes randomness from the nonce entirely. Instead of sampling k from an RNG you may not trust, *derive* it deterministically from the private key and the message:

$$k = \text{HMAC-derived}(d, \mathrm{H}(m)).$$

This is RFC 6979. Because k depends on the message, two different messages get different (and unpredictable, since d is secret) nonces, so reuse is structurally impossible. Because it depends on d, an outsider cannot predict it. And because it needs no RNG at signing time, a broken `SecureRandom` can no longer sink you. The same private key and message always yield the same signature – which is fine, and even desirable for reproducibility and testing.

Why it works

Determinism sounds paradoxical – did we not say the nonce must be "random"? The real requirement is weaker and more precise: each nonce must be *unpredictable to the attacker* and *distinct across distinct messages*. A keyed PRF (HMAC) of the message satisfies both: distinct messages give distinct, pseudorandom outputs, and without the key the outputs look uniform. Determinism just means we obtain that unpredictability from a hash of secret data rather than from a hardware RNG – trading a fragile dependency for a robust one. EdDSA, next, bakes this in by design.

22.6 EdDSA and Ed25519

EdDSA (Edwards-curve Digital Signature Algorithm), and its instantiation *Ed25519*, is the modern signature you should reach for first. It pairs Schnorr's clean signature equation with Bernstein's edwards25519 curve, standardized in RFC 8032. The design philosophy is strict misuse-resistance: no per-signature randomness to mishandle, no point-validation footguns, and complete, constant-time curve arithmetic. Today, SSH, TLS 1.3, Signal, and modern operating systems use it as their default.

22.6.1 How Ed25519 signs

Parameters: the twisted Edwards curve edwards25519 over $\mathbb{F}_{2^{255}-19}$, base point B of prime order $\ell \approx 2^{252}$, and the hash H = SHA-512.

Keys. The private key is a 32-byte seed. Hash it with SHA-512 to get 64 bytes; the first 32 (clamped) form the scalar a, the second 32 form a secret prefix pre used for nonce derivation. The public key is $A = aB$.

Sign message M:

1. $r = H(\text{pre} \,\|\, M) \bmod \ell$ – the nonce, derived deterministically from a secret prefix and the message (no RNG!).
2. $R = rB$.
3. $k = H(R \,\|\, A \,\|\, M) \bmod \ell$ – the challenge, binding R, the public key, and the message.
4. $S = (r + k\,a) \bmod \ell$.
5. Signature is (R, S), a fixed 64 bytes.

Verify (R, S) on M: compute $k = H(R \,\|\, A \,\|\, M) \bmod \ell$ and accept iff

$$S\,B = R + k\,A.$$

Why it works

Why verification works. By definition $S = r + ka \pmod{\ell}$, and scalars on this curve act mod ℓ. So

$$SB = (r + ka)B = rB + k(aB) = R + kA.$$

The signer's secret scalar a enters the right side only through the public $A = aB$, so the verifier checks the equation with public data alone (Figure 22.4). This is Schnorr's structure: a commitment R, a challenge k that hashes R together with the message, and a response S that ties them to the secret. Hashing R *into* the challenge is what makes Schnorr/EdDSA cleanly provable and non-malleable, unlike (EC)DSA's bolted-together $s = k^{-1}(h + dr)$.

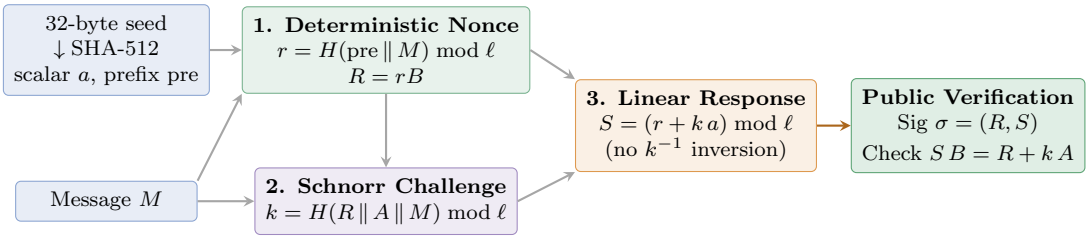


Figure 22.4: The Ed25519 signing and verification pipeline. Hashing the secret prefix pre with the message M produces a deterministic nonce r without an RNG; hashing the commitment $R = rB$ and public key $A = aB$ into the challenge k prevents malleability; and the linear response $S = r+ka \bmod \ell$ eliminates modular inversion.

Focus

What Ed25519 fixed, point by point:

- **Deterministic nonce** ($r = H(\text{pre} \,\|\, M)$): the PS3 and Bitcoin disasters are structurally impossible – there is no RNG to fail and no way to repeat r across distinct messages.
- **No modular inversion** in signing: $S = r + ka$ has no k^{-1}, so no division-by-zero edge cases and easier constant-time code.
- **Complete addition formulas** on the Edwards curve: every point addition uses one formula with no exceptional cases, removing a class of implementation bugs and timing leaks.
- **Strong unforgeability and non-malleability**: the challenge hashes R, so the $(r, n-s)$ trick that plagues ECDSA does not apply.
- **Fast**: tens of thousands of signatures and verifications per second on a laptop core; batch verification is faster still.

In the real world

Ed25519 is now the default host-key and user-key type in OpenSSH (`ssh-keygen -t ed25519`); it authenticates the identity keys in Signal's X3DH handshake (Chapter 27); it signs Git commits and tags, Debian and Arch package metadata, certificate-transparency logs, and many cryptocurrency and DNSSEC deployments; and it is a standard signature algorithm in TLS 1.3. If you are choosing a signature scheme for a new system in the absence of a hard interoperability or post-quantum constraint, Ed25519 is the conservative default.

Implement it

A schematic of Ed25519 signing, to show the *shape* of the algorithm. **For learning only – do not use.** This sketch omits clamping details, point encoding/decoding, cofactor handling, and constant-time discipline; a real implementation (use `libsodium` or `cryptography`) gets all of those exactly right.

```python
import hashlib

# Assume curve helpers exist: B (base point), L (subgroup order),
# point_mul(scalar, P), point_add(P, Q), encode(P) -> bytes.
def H(b: bytes) -> int:
    return int.from_bytes(hashlib.sha512(b).digest(), "little")

def sign(A_enc: bytes, M: bytes, a: int, prefix: bytes):
    r = H(prefix + M) % L                  # deterministic nonce -- no RNG
    R = point_mul(r, B)
    R_enc = encode(R)
    k = H(R_enc + A_enc + M) % L           # challenge binds R, A, M
    S = (r + k * a) % L                    # response: NO inversion
    return R_enc + S.to_bytes(32, "little")

def verify(A, A_enc: bytes, M: bytes, sig: bytes) -> bool:
    R_enc, S = sig[:32], int.from_bytes(sig[32:], "little")
    R = decode(R_enc)
    k = H(R_enc + A_enc + M) % L
    # accept iff  S*B == R + k*A
    return point_mul(S, B) == point_add(R, point_mul(k, A))
```

The two lines worth memorizing: the nonce r comes from hashing secret prefix plus message (never an RNG), and verification checks $SB = R + kA$ with no inversion in sight.

22.7 What Signatures Guarantee – and What They Do Not

It is as important to know the limits as the powers. A valid signature is a precise but *narrow* statement, and most real-world signature failures come from expecting it to say more than it does.

A valid signature σ on m under pk guarantees:

- **Authenticity:** someone holding the private key matching pk produced σ for this exact m.
- **Integrity:** m has not changed since it was signed; one flipped bit makes verification fail.
- **Non-repudiation:** the signer cannot later credibly deny it (modulo key compromise).

It does *not* guarantee:

- **That pk belongs to who you think.** A signature ties a message to a *key*, not to a *person*. Binding a key to an identity is the job of *certificates* and PKI (Chapter 24). Verify a perfect signature under the wrong public key and you have authenticated an impostor.
- **Confidentiality.** Signatures protect integrity and origin, not secrecy. The message is typically sent in the clear alongside σ. If you need both, sign *and* encrypt – and be careful how you

combine them (Chapter 16).

- **Freshness.** A signature on "Pay Bob $100" is equally valid today and replayed next year. Signatures say nothing about *when* or *how many times*; defeating replay needs a nonce, timestamp, or counter *inside* the signed message (the same lesson as MACs, Chapter 15).
- **That the signer read or meant the message.** The key endorsed these bytes; it did not promise the human understood them. "Blind signing" of opaque hashes has cost people real money in hardware-wallet phishing.
- **Anything after key compromise.** If sk leaks (the PS3, the Android wallets), every past and future signature is suspect. This motivates short-lived keys, revocation (Chapter 24), and key-rotation hygiene.

Common trap

Sign-then-encrypt vs. encrypt-then-sign, and identity confusion. Naively signing a plaintext and then encrypting it (Enc(Sign(m))) lets a recipient *strip* your signature and re-encrypt the still-signed message to a third party, who is fooled into thinking you sent it to *them*. Naively encrypting then signing the ciphertext lets an attacker strip your signature and re-sign the ciphertext as their own. The robust fix is to bind identities *inside* the signed/encrypted content – e.g. sign "`from Alice to Bob:` m" – so the message is meaningless out of context. This "surreptitious forwarding" problem famously bit early PGP/email designs.

Intuition

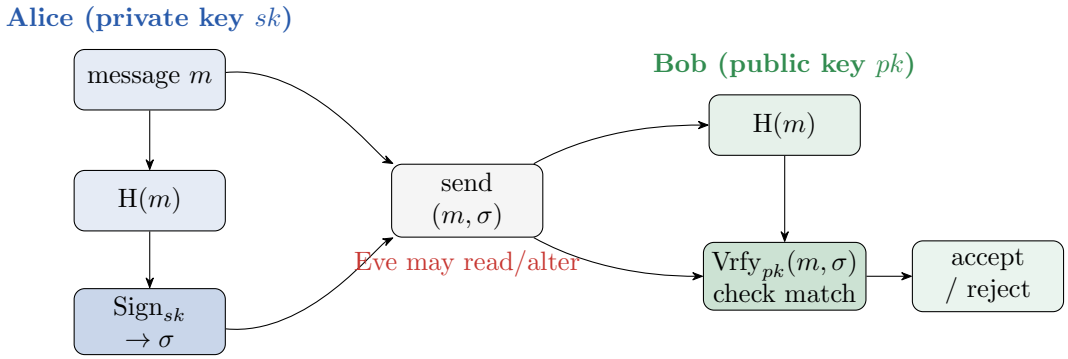
A *certificate* is a signature whose *message* is "public key pk belongs to `example.com`", signed by a certificate authority. Signatures are the building block; PKI (Chapter 24) uses them to connect a key to a name you can trust. Everything in TLS, code-signing, and the web's trust model is built by stacking signatures: a chain of "this key vouches for that key" all the way down to a root you decided to trust in advance.

22.8 Putting the Family Together

The three live signature families trade off along the same axes: underlying hard problem, key/signature size, speed, and how much rope they give a careless implementer.

Scheme	Hard problem	Sig size (128-bit)	Nonce?	Main risk
RSA-PSS	Factoring	256–384 B	none	big keys, padding bugs
DSA	DLP in $(\mathbb{Z}/p\mathbb{Z})^*$	64 B	critical	nonce reuse/bias
ECDSA	EC-DLP	64 B	critical	nonce reuse, malleable
Ed25519	EC-DLP (Schnorr)	64 B	built-in	few; misuse-resistant

The message flow is the same for all of them, and it is worth fixing in a single picture: the signer applies a private operation to a hash of the message; the verifier applies the matching public operation and checks consistency.



Hash to fix the message size and kill algebraic structure; apply the private operation; ship message and signature together; verify with the public key. The schemes differ only in what the "private operation" is – a d-th power mod N, a discrete-log signature equation, or a Schnorr response on a curve – and in how much care the nonce demands.

With signatures in hand, we can authenticate who is speaking across an untrusted network. In Chapter 23, we combine ephemeral Diffie–Hellman with digital signatures to build authenticated key exchanges with forward secrecy, so that past sessions stay secret even if a long-term signing key is later compromised.

22.9 Exercises

Exercise 22.1 (MAC vs. signature, in court)**.** Alice sends Bob a money order authenticated with a shared-key MAC. Bob takes the tag to a judge as proof Alice authorized the payment. Explain precisely why the MAC tag is *not* convincing evidence, and why a digital signature would be. Which exact property is doing the work?

Exercise 22.2 (Textbook RSA forgery)**.** With $N = 3233$, $e = 17$, $d = 2753$ (Example 22.1), an attacker has obtained Alice's textbook-RSA signatures on $m_1 = 3$ and $m_2 = 5$. Produce, with no knowledge of d, a valid signature on $m^* = 15 \bmod 3233$. Show the arithmetic and verify your forgery passes $\sigma^e \equiv m^* \pmod N$.

Exercise 22.3 (Why hashing stops the forgery)**.** Repeat the previous exercise but for *hash-then-sign* RSA. Explain why the multiplicative trick now fails, and state precisely what computational problem the attacker would have to solve instead (name it, and say on which object – message or digest).

Exercise 22.4 (DSA verification by hand)**.** Using the DSA parameters of Example 22.3 ($p = 23$, $q = 11$, $g = 4$, $y = 18$), verify the signature $(r, s) = (3, 10)$ on a message with digest $h = 9$ by computing w, u_1, u_2, and v independently, and confirming $v = r$.

Exercise 22.5 (Sign with ECDSA)**.** On the curve of Example 22.4 ($\mathbb{F}_{17}$, $G = (5, 1)$, $n = 19$, $d = 7$, $Q = (0, 6)$), sign a message with digest $h = 4$ using nonce $k = 5$. You are given $5G = (9, 16)$. Compute r, then s, and write the signature. (You will need $5^{-1} \bmod 19$.)

Exercise 22.6 (Recover the key from reuse)**.** Continuing the previous setup, suppose Alice also signed a message with digest $h' = 13$ using the *same* nonce $k = 5$, producing s'. Without using the known value $d = 7$, set up the two-signature equations, solve for k, then for d, and confirm you recover $d = 7$. (Compute s' first from the signing equation, then pretend you only know r, s, s', h, h'.)

Exercise 22.7 (Spot the bad nonce)**.** You are auditing a batch of ECDSA signatures from one signer. Describe the single cheap check that detects same-nonce reuse across a corpus of signatures, why it works, and what an attacker who runs the same check can then do. Relate this to how the 2013 Android-wallet thefts were carried out on the public blockchain.

Exercise 22.8 (ECDSA malleability)**.** Show that if (r, s) verifies under public key Q for message digest h, then $(r, n - s)$ also verifies. (Hint: trace what happens to $w = s^{-1}$, then to u_1, u_2, and finally to the recovered point's x-coordinate.) State the "low-s" canonicalization rule that removes the ambiguity.

Exercise 22.9 (Why no inversion in EdDSA)**.** EdDSA's response is $S = r + ka \bmod \ell$, with no k^{-1}, whereas ECDSA's is $s = k^{-1}(h + dr) \bmod n$. Give two concrete engineering advantages of avoiding the modular inverse (think edge cases and side channels), and explain why hashing R into the challenge k makes EdDSA strongly unforgeable where ECDSA is only malleably unforgeable.

Exercise 22.10 (The narrow promise)**.** For each scenario, say whether a correctly verifying signature defends against it, and if not, name the mechanism that does: (a) an attacker flips a byte of the message in transit; (b) an attacker replays last week's signed "unlock door" command; (c) an attacker presents a valid signature under a key that is not actually the bank's; (d) an eavesdropper reads the signed message's contents.

Exercise 22.11 (Deterministic but not random)**.** RFC 6979 derives k deterministically from the private key and the message. Argue that this satisfies the real security requirement on the nonce even though k is no longer "random," and identify exactly which property of an HMAC-based derivation guarantees that two distinct messages cannot collide on the same k in practice.

Exercise 22.12 (Surreptitious forwarding)**.** Alice computes $\mathrm{Enc}_{pk_B}(\mathrm{Sign}_{sk_A}(m))$ and sends it to Bob, intending "m, from Alice, for Bob." Show how Bob can make Carol believe Alice sent m to *Carol*. Then propose a concrete change to what gets signed that defeats the attack, and explain why it works.

Chapter 23

Key Exchange and Forward Secrecy

In Chapter 18, we saw how Diffie and Hellman let two strangers agree on a shared secret over a public channel, and in Chapter 22 we saw how digital signatures prove who authored a message. On its own, however, an unauthenticated Diffie–Hellman handshake has a fatal weakness: an active adversary, Eve, can sit between Alice and Bob and run a separate handshake with each of them, impersonating Bob to Alice and Alice to Bob.

This chapter is about turning a raw key-agreement primitive into something you can actually deploy. We need three things the bare handshake lacks. First, *authentication*: each party must know who is on the other end, so that the secret is shared with Bob and not with Eve-pretending-to-be-Bob. Second, *forward secrecy*: if Bob's long-term key is stolen next year, the messages he sent this morning must stay unreadable. Third, a disciplined way to turn the one shared group element the handshake produces into the several independent keys a real protocol needs — that is the job of a *key derivation function*. Along the way we will worry about replay and downgrade, two failure modes that have sunk real protocols. By the end you will have hand-assembled the core of what TLS 1.3 and the Signal protocol actually do.

23.1 The threat we are defending against

Let us fix the cast and the rules before we build anything. Alice and Bob want to establish a shared secret key over a network. Eve sits on the wire. We grant Eve quite a lot of power, because real adversaries have it:

- She sees every byte that crosses the network (eavesdropping).
- She can drop, reorder, replay, and modify messages, and inject her own (an *active*, or *man-in-the-middle*, adversary).
- She knows the protocol completely. The only things she does not know are the secret keys and the random coins the parties flip.

Against this adversary we want a protocol with two output guarantees.

Definition 23.1 (Informal goals of authenticated key exchange)**.** A *key-exchange protocol* run between two parties produces a session key k. We say the protocol is a secure *authenticated key exchange* (AKE) if:

1. **Secrecy.** An adversary who watches (and tampers with) the protocol cannot distinguish the real session key k from a uniformly random string of the same length. Concretely, k should look like fresh randomness to anyone but the two participants.
2. **Authentication.** When Alice finishes a run believing she shares k with Bob, then either she really does share k with the genuine Bob, or Bob has no matching session at all (he never accepted). She is never fooled into sharing a key with Eve while thinking it is Bob.

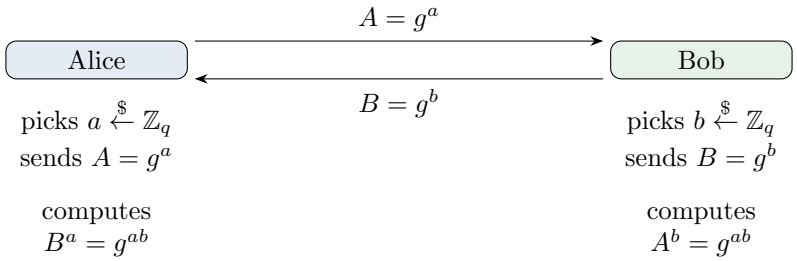
The secrecy notion above is the "real-or-random" game you have met for encryption, lifted to keys. We will make it sharper when we tie security to the Decisional Diffie-Hellman assumption.

Intuition

Secrecy means that an outsider learns nothing useful about the key you end up with. Authentication means that you know who else holds it. The plain Diffie-Hellman handshake delivers the first but not the second, and Eve exploits exactly that gap.

23.2 Recalling the bare handshake, and watching it fail

Fix a cyclic group $\mathbb{G}$ of prime order q with generator g. (Concretely think of $(\mathbb{Z}/p\mathbb{Z})^*$ for a large prime p, or an elliptic-curve group; the algebra is identical.) The textbook Diffie-Hellman exchange is one round:



Both sides arrive at g^{ab}. Eve sees g^a and g^b and would have to compute g^{ab} from them — the *Computational Diffie-Hellman* problem, believed hard. So against a *passive* Eve the shared point is secret. Good.

Now make Eve active. She does not need to solve any hard problem; she just plays two handshakes at once.

Attack

[Man-in-the-middle on unauthenticated DH] Eve intercepts Alice's $A = g^a$ and never lets it reach Bob. She picks her own $e \xleftarrow{\$} \mathbb{Z}_q$ and:

- sends $E = g^e$ to Bob, pretending to be Alice;
- sends $E = g^e$ to Alice, pretending to be Bob (she can reuse e or pick a fresh one — say e' toward Alice).

Now Alice computes $k_A = E^a = g^{ae}$ and thinks she shares it with Bob. Bob computes

$k_B = E^b = g^{be}$ and thinks he shares it with Alice. Eve knows e, so she computes *both*: $k_A = A^e$ and $k_B = B^e$ (Figure 23.1). She decrypts everything Alice sends under k_A, re-encrypts it under k_B, and forwards it to Bob. Neither party sees anything wrong. Eve is a transparent relay who reads and rewrites the whole conversation.

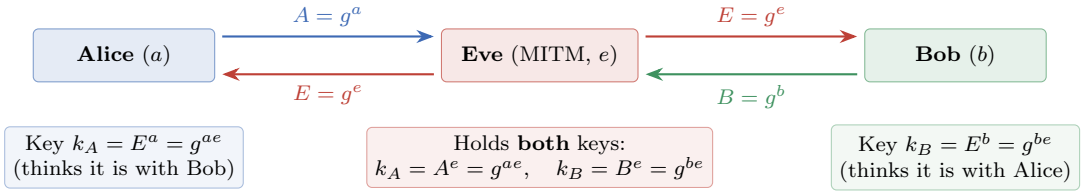


Figure 23.1: Active man-in-the-middle attack on unauthenticated Diffie–Hellman. By intercepting both public shares and substituting her own $E = g^e$, Eve splits the session into two separate Diffie–Hellman agreements ($k_A = g^{ae}$ with Alice and $k_B = g^{be}$ with Bob) and relays traffic between them.

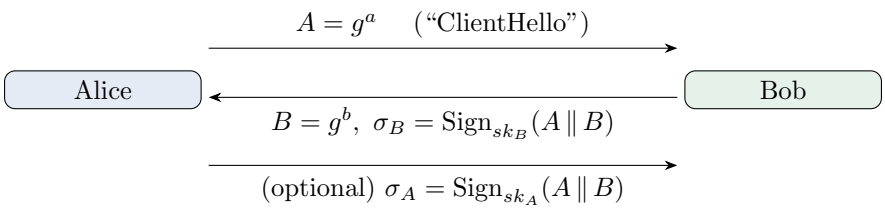
The lesson is blunt: **a key is only as trustworthy as your knowledge of who holds the other copy.** Diffie-Hellman gives you a shared secret with *somebody*. Turning "somebody" into "Bob" is the entire problem of authentication, and it cannot come from the group algebra alone. It must come from something Alice already knows about Bob: a long-term key, bound to his identity, that Eve cannot forge.

23.3 Authenticating the handshake

We have two long-term-key tools that can pin an identity: digital signatures and long-term Diffie-Hellman keys. They lead to two families of AKE, and modern protocols use both. Let us build the signature-based one first because it is the most transparent, then the implicitly-authenticated one because it is what elliptic-curve protocols and Signal actually run.

23.3.1 Signed ephemeral Diffie-Hellman

Give Bob a long-term signature key pair (sk_B, pk_B), and suppose Alice already holds an authentic copy of pk_B — through a certificate, a pinned key, or a trust-on-first-use record. (Where pk_B comes from is the job of Chapter 24; here we just assume Alice has the right one.) The idea is to keep the Diffie-Hellman values *ephemeral* — fresh per session — but have each party *sign* the ephemeral value they send.



1. Alice picks $a \xleftarrow{\$} \mathbb{Z}_q$ and sends $A = g^a$.

2. Bob picks $b \xleftarrow{\$} \mathbb{Z}_q$, and signs the *transcript* so far: $\sigma_B = \mathrm{Sign}_{sk_B}(A \,\|\, B)$. He sends (B, σ_B).
3. Alice checks $\mathrm{Vrfy}_{pk_B}(A \,\|\, B, \sigma_B)$. If it verifies, she knows the genuine Bob saw *her* A and produced *this* B. She computes $k = B^a = g^{ab}$.
4. Bob computes $k = A^b = g^{ab}$. (If Alice also has a long-term key and Bob needs to authenticate her — "mutual authentication" — she signs the transcript too in step 3.)

Why does signing the transcript, rather than just signing B, matter so much? Because the signature must *bind Bob's ephemeral to Alice's ephemeral*. If Bob signed only B, Eve could record an old (B, σ_B) pair from a previous session and replay it into a new one — the signature would still verify. By covering $A \,\|\, B$, the signature is bound to this specific run. Alice put a fresh, random A on the wire; a valid signature over $A \,\|\, B$ proves Bob acted *after* seeing that fresh A, so the message cannot be a recording.

Why it works

[Why MITM dies here] Return to Eve from box 23.2. To impersonate Bob to Alice, Eve must send some B' and a signature σ' with $\mathrm{Vrfy}_{pk_B}(A \,\|\, B', \sigma')$ accepting. But producing a valid signature under pk_B on a message of her choosing is exactly an existential forgery — the thing a secure signature scheme makes infeasible. Eve does not have sk_B. She is stuck. She can still relay the genuine messages unchanged, but then she has merely forwarded a real Alice–Bob exchange and learned nothing: the resulting g^{ab} is secret from her by Computational Diffie-Hellman. The signature converts "a key with somebody" into "a key with whoever owns sk_B," and the certificate says that owner is Bob.

Common trap

[Sign the transcript, not just your own share] A recurring real-world bug is signing too little. If a party signs only its own public value, or signs a message that omits the peer's nonce, an attacker can splice signed pieces from different sessions together. The discipline "everyone signs a hash of the full transcript they have seen so far" is what TLS 1.3's `CertificateVerify` does, and it is the safe default. We will see the matching idea — a final MAC over the whole transcript — in the downgrade discussion.

23.3.2 The foundation: ephemeral keys

Notice that a and b are freshly chosen for this session and *thrown away afterward*. The long-term keys (sk_B, and sk_A if used) sign, but they never touch the shared secret g^{ab}. That separation — long-term keys for *authentication*, ephemeral keys for *secrecy* — is what provides forward secrecy, the subject of the next section.

History

The "station-to-station" (STS) protocol of Diffie, van Oorschot, and Wiener (1992) is essentially the signed-ephemeral-DH scheme above, with a refinement: each party signs the DH values and then MACs that signature under the new session key, which proves they actually computed the same key ("key confirmation"). Early authenticated handshakes later drew

scrutiny when Hugo Krawczyk showed that subtle identity-misbinding bugs arise unless signatures and MACs bind the peers' identities and the full transcript; his SIGMA protocol family (2003) fixed this and became the basis for IKE (the IPsec key-exchange protocol) and TLS 1.3. The handshake protecting your bank login today descends from the 1976 Diffie–Hellman idea through STS and SIGMA.

23.3.3 Implicit authentication: long-term DH keys

Signatures are not the only way. Suppose instead Bob's long-term key *is* a Diffie-Hellman key: a secret b_{lt} with public value $B_{\mathrm{lt}} = g^{b_{\mathrm{lt}}}$ that Alice trusts. Alice similarly has a_{lt}, $A_{\mathrm{lt}} = g^{a_{\mathrm{lt}}}$. Then Alice and Bob can agree on a key with no signatures at all, because only the true owners of the long-term secrets can compute the right shared value.

The cleanest illustration is the *triple Diffie-Hellman* pattern used by Signal. Each party has a long-term "identity" key and a fresh ephemeral key:

$$\text{Alice: } (a_{\mathrm{lt}},\, a_{\mathrm{eph}}), \qquad \text{Bob: } (b_{\mathrm{lt}},\, b_{\mathrm{eph}}),$$

with the corresponding public values exchanged. They form the shared secret by hashing together several DH outputs:

$$k = \mathrm{H}\big(\, g^{a_{\mathrm{lt}}\, b_{\mathrm{eph}}} \,\|\, g^{a_{\mathrm{eph}}\, b_{\mathrm{lt}}} \,\|\, g^{a_{\mathrm{eph}}\, b_{\mathrm{eph}}} \,\big).$$

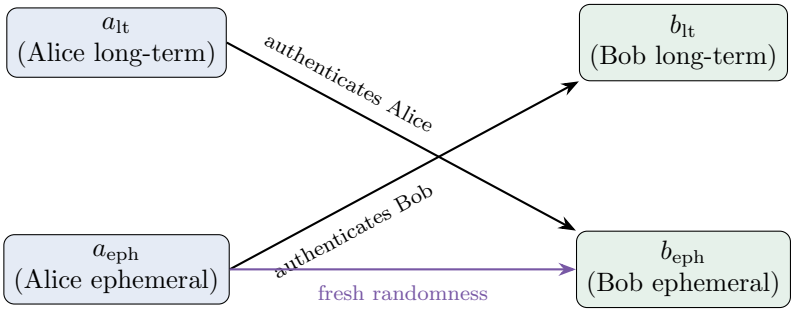


Figure 23.2: The three Diffie-Hellman cross-products of triple DH. Each long-term-against-ephemeral arrow pins one party's identity; the ephemeral-against-ephemeral arrow injects per-session freshness for forward secrecy.

Read each factor as a claim. The first mixes Alice's *long-term* key with Bob's *ephemeral* — only the real Alice (who knows a_{lt}) and the sender of b_{eph} can form it, so it authenticates Alice. The second mixes Bob's long-term with Alice's ephemeral — it authenticates Bob. The third mixes the two ephemerals — it injects fresh, throwaway randomness so that the session secret depends on per-session coins (Figure 23.2). "Implicit" means there is no separate signature to verify: a party who can compute the same k has, by that fact alone, proven it holds the right long-term secret.

Intuition

A signature is a notarized letter: "I, Bob, hereby endorse this handshake," verifiable by anyone with Bob's public key. Implicit authentication builds identity directly into the key calculation:

nobody sends a separate signature, because neither side can compute the matching session key unless it holds the private half of the expected long-term Diffie–Hellman key. Signal prefers the implicit style partly because it leaves no signature on the wire — a small but real privacy gain, since a signature is a non-repudiable record that you said a particular thing.

23.4 Forward secrecy

Forward secrecy is designed for the following scenario. Eve cannot break your handshake today, so she does the patient thing: she records every encrypted session you have, byte for byte, and stores them. Months later she compromises your server — a stolen laptop, a subpoena, a Heartbleed-style memory leak — and walks away with your *long-term private key*. Whether this is a disaster depends on one question:

Does possessing the long-term key today let Eve decrypt the traffic she recorded yesterday?

Definition 23.2 (Perfect forward secrecy)**.** A key-exchange protocol has *(perfect) forward secrecy* if the compromise of long-term secret keys does not compromise the session keys of *past* sessions. Equivalently: an adversary who records a session and *later* learns all the long-term private keys still cannot recover that session's key.

The word "perfect" here is historical and a bit grandiose; people increasingly just say "forward secrecy." The property is entirely about a *time ordering*: secrets created and destroyed before the compromise stay safe.

23.4.1 Why ephemeral DH gives it, and static encryption does not

Picture two ways Alice could send Bob a session key.

Way 1 — static key transport (no forward secrecy). Alice picks a random session key k, encrypts it under Bob's long-term public key, and sends $\mathrm{Enc}_{pk_B}(k)$. This is essentially old TLS with RSA key transport. Eve records the ciphertext. Years later she steals sk_B. She computes $\mathrm{Dec}_{sk_B}(\mathrm{Enc}_{pk_B}(k)) = k$ and decrypts the whole recorded session. The long-term key is a *master key to the past*. One key compromise exposes every session ever recorded under it. This is a disaster, and it is why this mode was removed from TLS 1.3.

Way 2 — ephemeral DH (forward secrecy). Alice and Bob run the signed ephemeral handshake. The session key is g^{ab}, derived from the ephemeral secrets a and b, which both parties *erase* as soon as the handshake finishes. The long-term key only *signed*; it never touched g^{ab}. Now Eve steals sk_B later. What can she do with it? She can forge signatures from *now on* — she can impersonate Bob in *future* handshakes. But for the recorded past session she has only g^a, g^b, and a signature. To recover g^{ab} from g^a and g^b is, again, Computational Diffie-Hellman — and the stolen signing key is no help at all, because it has nothing to do with the DH secret. Once the ephemeral exponents a and b are erased, recorded past sessions remain unreadable.

Why it works

[The one-sentence reason] Forward secrecy works because the session secret is computed from *ephemeral* values that are destroyed after use, while the long-term key only *authenticates* and never enters the secret. Compromising the authenticator cannot retroactively reconstruct secrets it never saw.

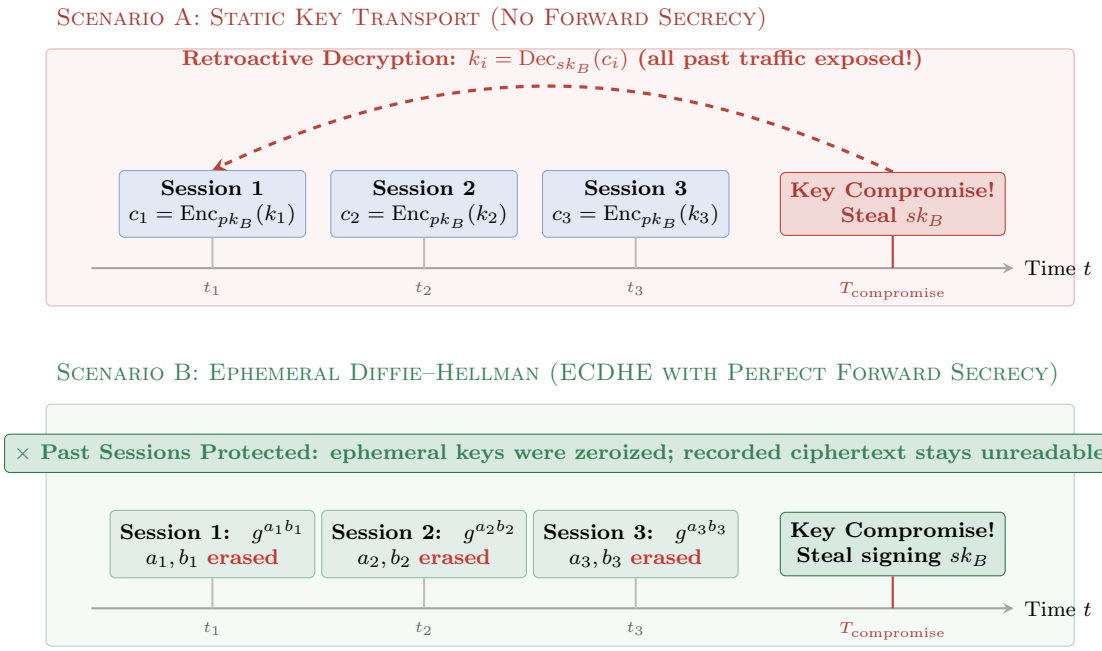


Figure 23.3: Timeline comparison of session key security upon long-term key compromise. In static key transport (Scenario A), the long-term secret key acts as a master key to all past recorded traffic. In ephemeral Diffie–Hellman (Scenario B), past session keys are derived from transient exponents zeroized after the handshake; compromise of the long-term signing key allows only future impersonation, never retroactive decryption of the past.

Common trap

["Ephemeral" is only as good as your delete] Forward secrecy is a property of the *whole system*, not just the math. If your server keeps the ephemeral secret b in memory, or worse logs it, or reuses one "ephemeral" value across thousands of connections to save CPU, then stealing it resurrects all those sessions. There is a real-world cautionary tale here: some servers cached a single DH ephemeral for the lifetime of the process (sometimes called a static-ephemeral or "semi-static" key) to avoid recomputing g^b. That quietly destroys forward secrecy for every session sharing the cached value. Ephemeral means *generated fresh and wiped promptly.* If you cannot guarantee the wipe, you do not have the property.

In the real world

This is why every modern TLS deployment insists on "(EC)DHE" cipher suites — the trailing `E` is for *ephemeral.* TLS 1.3 went further and made ephemeral (elliptic-curve) Diffie-Hellman the

only key-establishment mechanism: static RSA key transport was deleted from the protocol outright, precisely because it lacked forward secrecy and made mass retroactive decryption a one-key-theft-away proposition. If you operate a server, "do we have forward secrecy" reduces to "are we negotiating an ephemeral DH suite and actually discarding the ephemerals."

23.4.2 A worked tiny example

Numbers make the time-ordering concrete. Work in $(\mathbb{Z}/p\mathbb{Z})^*$ with the small prime $p = 23$. The element $g = 5$ generates the group (5 has order 22 modulo 23, so it is a generator). All arithmetic is mod 23.

The session. Alice picks ephemeral $a = 6$, sends $A = 5^6 \bmod 23$. Compute: $5^2 = 25 \equiv 2$, $5^3 \equiv 10$, $5^6 = (5^3)^2 \equiv 10^2 = 100 \equiv 8$. So $A = 8$.

Bob picks ephemeral $b = 15$, sends $B = 5^{15} \bmod 23$. Compute by squaring: $5^1 = 5$, $5^2 = 2$, $5^4 = 2^2 = 4$, $5^8 = 4^2 = 16$. Then $5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5^1 = 16 \cdot 4 \cdot 2 \cdot 5$. Step it out: $16 \cdot 4 = 64 \equiv 18$; $18 \cdot 2 = 36 \equiv 13$; $13 \cdot 5 = 65 \equiv 19$. So $B = 19$.

Shared secret. Alice computes $B^a = 19^6 \bmod 23$ and Bob computes $A^b = 8^{15} \bmod 23$; both equal $g^{ab} = 5^{90} \bmod 23$. Since the group has order 22, reduce the exponent: $90 = 4 \cdot 22 + 2$, so $5^{90} \equiv 5^2 \equiv 2 \pmod{23}$. The session secret is 2. (You can check 19^6 directly: $19 \equiv -4$, $(-4)^6 = 4096$, and $4096 = 178 \cdot 23 + 2$, so $19^6 \equiv 2$. Agreement.)

Now the theft. Suppose tomorrow Eve steals Bob's long-term *signing* key. From the recorded wire she has $A = 8$ and $B = 19$. The signing key lets her forge signatures but tells her nothing about $a = 6$ or $b = 15$, which Alice and Bob deleted. To get the secret 2 she must solve $5^x \equiv 8$ (find a) or $5^y \equiv 19$ (find b) — a discrete log. With $p = 23$ she could, of course, by brute force; that is the price of toy numbers. With a real 2048-bit or 256-bit-curve group, that search is infeasible. The structural point survives the small numbers: the stolen long-term key is no shortcut to the recorded session's secret.

23.5 From one group element to many keys: KDFs

The handshake hands you a single shared secret — a group element like our 2, or in practice a 256-bit string. But a real session needs *several* independent keys: one to encrypt client-to-server, one for server-to-client, maybe separate MAC keys, IVs, a key to confirm the handshake. You must not just slice the raw DH output into chunks and use them directly. Two reasons.

First, the raw shared secret is not a uniform random string. A Diffie-Hellman value g^{ab} in $(\mathbb{Z}/p\mathbb{Z})^*$ is an element of a structured set — it is biased, it has algebraic relations, its bits are not independent. Cryptographic keys are supposed to be (indistinguishable from) uniform. We need to "iron out" the structure.

Second, you want *domain separation*: keys for different purposes must be provably unrelated, so that a flaw in one use cannot leak another. Reusing one key for two jobs is a classic footgun.

The tool for both is a *key derivation function* (KDF). The standard, HKDF, splits the job into two stages with the slogan *extract-then-expand*.

Definition 23.3 (HKDF, extract-then-expand)**.** Built from an HMAC keyed by a hash H:

- **Extract.** From the raw, non-uniform input keying material IKM (here IKM $= g^{ab}$) and an optional non-secret *salt*, produce a fixed-length, uniform-looking pseudorandom key:

$$\text{PRK} \;=\; \text{HMAC}(\text{salt},\ \text{IKM}).$$

  This is the "ironing" step. HMAC, used as a randomness extractor, concentrates whatever entropy is in IKM into a full-strength uniform key PRK.

- **Expand.** From PRK and a per-key *info* label, stretch out as many key bytes as you need, one HMAC block at a time:

$$T_i = \text{HMAC}\big(\text{PRK},\ T_{i-1} \,\|\, \text{info} \,\|\, i\big), \quad T_0 = \text{empty},$$

  and concatenate $T_1 \,\|\, T_2 \,\|\, \cdots$ to the requested length. The *info* string is where domain separation lives: feed `"c hs traffic"` for one key and `"s hs traffic"` for another, and the two outputs are unrelated.

Intuition

Extract takes the raw DH element, whose entropy is unevenly distributed, and condenses it into a uniform key (PRK). *Expand* takes PRK and a label (`"enc"`, `"mac"`, `"iv"`) and produces an independent key for each label (Figure 23.4). Knowing one output key tells you nothing about the others, so a single handshake secret yields as many keys as the protocol needs.

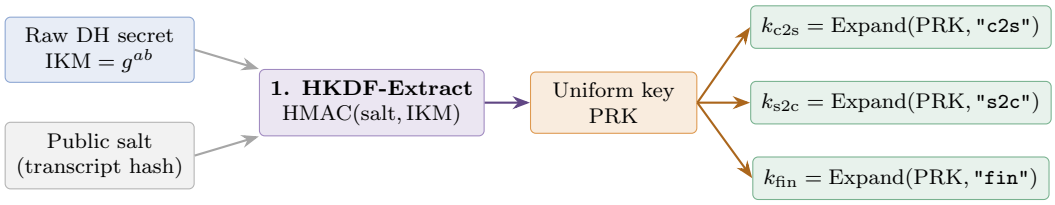


Figure 23.4: The two-stage HKDF pipeline. *Extract* compresses the structured, non-uniform Diffie–Hellman group element g^{ab} into a uniform pseudorandom key PRK; *Expand* fans PRK out into multiple domain-separated session keys using distinct context labels.

Connection

Why two stages instead of just "hash the DH output"? It is the same distinction the probability companion draws between an *extractor* (turn a weak, biased source into near-uniform bits — a min-entropy and leftover-hash argument) and a *pseudorandom generator* (stretch a short uniform seed into a long pseudorandom stream). Extract handles the non-uniform *source*; expand handles the *length*. Conflating them is how subtle biases sneak from the group into your keys. The HMAC indistinguishability that makes expand safe is the same PRF property developed in Chapter 15.

Implement it

[Toy HKDF over a real hash — learning only] This shows the extract-then-expand shape. It is a teaching toy: no constant-time care, no memory wiping, and it leans on the standard library for HMAC. **Do not use this in production — use your platform's vetted HKDF.**

```
import hmac, hashlib

def _hmac(key, msg):
    return hmac.new(key, msg, hashlib.sha256).digest()

def hkdf_extract(salt, ikm):
    # salt acts as the HMAC key; if absent, use a zero block
    if not salt:
        salt = b"\x00" * hashlib.sha256().digest_size
    return _hmac(salt, ikm)            # PRK: one uniform pseudorandom key

def hkdf_expand(prk, info, length):
    out, t, counter = b"", b"", 1
    while len(out) < length:
        # T_i = HMAC(PRK, T_{i-1} || info || i)
        t = _hmac(prk, t + info + bytes([counter]))
        out += t
        counter += 1
    return out[:length]

def hkdf(salt, ikm, info, length):
    return hkdf_expand(hkdf_extract(salt, ikm), info, length)

# One shared DH secret -> several independent keys via different labels
shared = b"raw-diffie-hellman-output-not-uniform"
prk_salt = b"handshake transcript hash goes here"
enc_key = hkdf(prk_salt, shared, b"c2s enc", 32)   # client->server
dec_key = hkdf(prk_salt, shared, b"s2c enc", 32)   # server->client
mac_key = hkdf(prk_salt, shared, b"c2s mac", 32)
print(enc_key.hex())
print(dec_key.hex())     # unrelated to enc_key, by domain separation
```

The three keys come from one shared secret yet are computationally independent: flipping the `info` label is all it takes. In TLS 1.3 the "salt" is literally a running hash of the handshake transcript, which is how the derived keys also become bound to everything said so far — a point we exploit next.

23.6 Replay and downgrade

Two active attacks deserve their own treatment because they bypass secrecy entirely and attack the *protocol logic*.

23.6.1 Replay

A *replay* attack records a valid protocol message and re-sends it later. The classic damage: Eve records Alice's encrypted "transfer $100 to Eve" message and replays it ten times. Each copy is a perfectly valid ciphertext; encryption alone does not notice it is a duplicate.

The defense in key exchange is *freshness*: weave an unpredictable, single-use value into each session so that a recording from one run is invalid in another. Ephemeral DH already does this for you — the secret is g^{ab} with fresh a, b, so a captured handshake cannot be replayed to produce a usable key in a new session, because the peer chose a new ephemeral. More generally, protocols add explicit *nonces* (numbers used once) or per-session *counters*, and require every authenticated message to cover them.

Common trap

[The 0-RTT replay hole in TLS 1.3] TLS 1.3 offers a "0-RTT" (zero round-trip) mode where a returning client can send application data in its very first flight, encrypted under a key derived from a secret remembered from a previous connection. This saves a round trip, but at a cost: because that first-flight data is *not* bound to a fresh server ephemeral (the server has not spoken yet), it is *replayable*. Eve can capture the 0-RTT data and re-send it to the server multiple times. The protocol therefore restricts 0-RTT to *idempotent* requests (a `GET` you can safely repeat), never a "transfer money" `POST`, and servers maintain anti-replay caches. Without freshness, replay becomes possible; the trade-off is explicit, and the application must respect it.

23.6.2 Downgrade

A *downgrade* attack does not break strong crypto; it tricks the parties into *negotiating weak crypto*. Real protocols support many cipher suites and versions for backward compatibility. Negotiation usually starts with the client advertising what it supports in cleartext — before any key exists to protect that list. So Eve, sitting in the middle, edits the list.

Attack

[Strip the strong options] Alice's ClientHello offers, say, modern ephemeral-ECDH suites *and*, for an ancient peer, an export-grade 512-bit DH suite. Eve intercepts it and deletes every strong option, leaving only the weak one. Bob, seeing only the weak choice, agrees to it. Now the session uses 512-bit DH, which Eve can break with modest precomputation — and she reads everything. Neither party intended weak crypto; the negotiation itself was unauthenticated. This is the shape of the real Logjam and FREAK attacks (2015), which forced browsers and servers to drop export-grade suites entirely.

The defense is *transcript authentication*: at the end of the handshake, once a key exists, each party computes a MAC (or signature) over a hash of *every handshake message it sent and received*, and the peer checks it.

Why it works

[Why a final transcript MAC stops downgrade] If Eve edited the negotiation, Alice's recorded transcript and Bob's recorded transcript *disagree* — Alice "sent" a strong-suites list, Bob "received" a stripped one. When each side MACs its own transcript hash under the freshly derived key and sends it for the other to verify, the two MACs cover different inputs and *will not match*. The handshake aborts. Eve cannot fix this by forging a matching MAC, because the MAC key is the session secret she does not have. So any mid-handshake tampering, including a downgrade edit, is detected the moment the parties cross-check transcripts. This "Finished" message is exactly how TLS seals its handshake, and it is the same key-confirmation idea that the station-to-station protocol introduced.

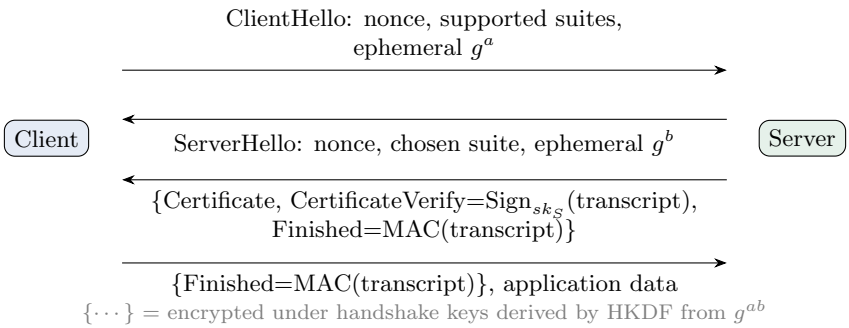
Intuition

In a downgrade, Eve edits the list of options Bob sees so that only a weak one remains. The transcript MAC has Alice and Bob compare, at the end of the handshake, what each of them actually sent and received. If the records differ, somebody tampered, and they abort. The negotiation can happen in the clear; it just has to be *ratified* under the key once the key exists.

23.7 Putting it together: the shape of TLS 1.3 and Signal

We now have every piece. Let us see the two protocols that assemble them, because they make different choices for different reasons.

23.7.1 TLS 1.3 at a glance



Trace the parts you built:

- The exchanged g^a, g^b are *ephemeral* ECDH — forward secrecy by construction (Definition 23.2).
- `CertificateVerify` is the server *signing the transcript* with its long-term key — signed ephemeral DH, authenticating the server (and the certificate ties pk_S to the hostname, as we see in Chapter 24).
- The handshake and application keys come from g^{ab} through HKDF, with the transcript hash as salt — many keys from one secret, all bound to the transcript.

- The two `Finished` MACs are the *transcript authentication* that kills downgrade and confirms both sides derived the same key.
- Nonces in the Hellos provide *freshness* against replay.

Every defense in this chapter appears exactly once, because TLS 1.3 was designed by stripping the protocol down to precisely these guarantees and removing everything (static RSA, renegotiation, weak suites) that violated them.

23.7.2 Signal at a glance

Signal solves a harder problem: the recipient is usually *offline* when the first message arrives, so there is no live round trip. Its X3DH ("extended triple Diffie-Hellman") handshake lets Alice start a session against a bundle of Bob's public keys that he published to a server in advance.

Bob pre-publishes an *identity key* (long-term), a *signed prekey* (medium-term, signed by his identity key so Alice can trust it), and a batch of one-time *prekeys*. Alice fetches a bundle, generates her own ephemeral, and computes the multi-DH secret from the earlier section — mixing her ephemeral and identity keys against Bob's signed prekey, identity key, and a one-time prekey:

$$k = \mathrm{H}\left(\mathrm{DH}(a_{\mathrm{id}},\, B_{\mathrm{spk}}) \,\|\, \mathrm{DH}(a_{\mathrm{eph}},\, B_{\mathrm{id}}) \,\|\, \mathrm{DH}(a_{\mathrm{eph}},\, B_{\mathrm{spk}}) \,\|\, \mathrm{DH}(a_{\mathrm{eph}},\, B_{\mathrm{otp}})\right).$$

Read the pieces as before: identity-against-something terms supply *implicit authentication* of each party; the ephemeral terms supply *forward secrecy*; the one-time prekey B_{otp}, consumed and deleted after a single use, adds replay resistance and an extra layer of forward secrecy for that very first message. The signed prekey carries the *signature-based* authentication (Bob's identity key signs it). Then HKDF turns k into the initial chain keys, and the "double ratchet" takes over from there to refresh keys per message — pushing forward secrecy down to the granularity of individual texts.

In the real world

TLS authenticates with a *signature* over the transcript because a web server is online and wants any client to verify it against a public certificate chain. Signal authenticates *implicitly* with long-term DH keys because it wants asynchronous, no-round-trip session setup and prefers to leave no signed, non-repudiable record of who said what. Both reach the same destination — an authenticated, forward-secret session key fed through HKDF — by the routes that fit their setting. WireGuard's handshake (the modern VPN) is a third point in the same design space: it uses the implicit, DH-only "Noise" framework, no certificates at all, with both parties' static keys pre-known.

Focus

[The whole chapter on one page] A deployable key exchange is ephemeral Diffie-Hellman (for *forward secrecy*) plus an *authenticator* bound to the transcript (a signature, as in TLS, or implicit long-term-DH terms, as in Signal) — which kills the *man-in-the-middle* — plus *nonces* (against replay) and a final *transcript MAC* (against downgrade) — with the one shared secret fanned out into many independent keys by *HKDF*. Lose the ephemerals' freshness and you lose forward secrecy; lose the transcript binding and you lose authentication or downgrade protection. Every real handshake is some arrangement of exactly these moving parts.

Throughout this chapter, we assumed Alice already had an authentic copy of Bob's public key pk_B. In Chapter 24, we tackle how she gets one: certificates and Public-Key Infrastructure (PKI), which bind public keys to real-world names across the internet.

23.8 Exercises

Exercise 23.1. In the man-in-the-middle attack of box 23.2, Eve must actively relay messages for the whole session. Suppose she goes offline halfway through, after Alice and Bob have each computed their keys $k_A = g^{ae}$ and $k_B = g^{be}$. Describe exactly what breaks for Alice and Bob, and explain why this means a MITM on unauthenticated DH must stay present for the entire conversation.

Exercise 23.2. Work the toy handshake in $(\mathbb{Z}/23\mathbb{Z})^*$ with $g = 5$ for new ephemerals $a = 9$ and $b = 4$. Compute $A = g^a$, $B = g^b$, and the shared secret g^{ab}, reducing the exponent modulo the group order 22. Verify Alice's and Bob's computations agree.

Exercise 23.3. Explain precisely why a server that caches one "ephemeral" DH value g^b and reuses it for an hour of connections *still authenticates correctly* but *loses forward secrecy*. What does an attacker who steals that cached b recover, and over what set of sessions?

Exercise 23.4. Consider the signed handshake but with Bob signing only B, not $A \,\|\, B$. Give a concrete replay attack in which Eve causes Alice to accept a session using a B that Bob produced in an *earlier, unrelated* session. Then show that signing $A \,\|\, B$ defeats your attack.

Exercise 23.5. Prove a small lemma about HKDF's domain separation under the assumption that HMAC keyed by PRK is a pseudorandom function. Argue that for distinct labels $\text{info}_1 \neq \text{info}_2$, an adversary given the expanded output for info_1 cannot distinguish the output for info_2 from random. (Hint: reduce to the PRF security of HMAC; the labels become distinct inputs.)

Exercise 23.6. A protocol designer proposes to skip the Extract stage of HKDF and feed the raw Diffie-Hellman element $g^{ab} \in (\mathbb{Z}/p\mathbb{Z})^*$ directly into Expand as the PRK. Give two distinct reasons this is dangerous, one about the *non-uniformity* of g^{ab} and one about *small-subgroup* structure (what if g^{ab} lands in a small subgroup?). Relate each to a property HKDF's Extract is supposed to provide.

Exercise 23.7. Reconstruct the LOGJAM-style downgrade at the protocol level. Alice offers suites {ECDHE-strong, DHE-export-512}; Eve wants the export suite chosen. Write the message Eve forwards to Bob, the message Bob returns, and then show exactly where the final transcript-MAC check (if present) would have caught her — and why it does not if the handshake omits transcript authentication.

Exercise 23.8. In TLS 1.3 0-RTT, explain why data in the client's first flight is replayable while data sent *after* the server's ServerHello is not. Identify the specific ingredient (from this chapter) that the later data has and the 0-RTT data lacks. Then classify which of these HTTP requests are safe to send as 0-RTT: `GET /balance`, `POST /transfer`, `GET /logo.png`, `POST /search`.

Exercise 23.9. In Signal's X3DH secret, identify which of the four DH terms would still be computable by an attacker who has stolen *Bob's long-term identity private key but none of his ephemeral/prekey secrets*, and which remain out of reach. Use this to argue that X3DH gives forward secrecy for the initial message with respect to identity-key compromise.

Exercise 23.10. Mutual authentication: modify the signed-ephemeral-DH protocol so that Bob also verifies Alice. State exactly what Alice signs and what Bob checks, and explain why both signatures must cover the *same* transcript value rather than each party signing only its own ephemeral.

Part IV

Protocols and Applications

Chapter 24

Certificates and Public-Key Infrastructure

In Chapter 23, we saw how ephemeral Diffie–Hellman and digital signatures combine to create forward-secret, authenticated sessions. But the entire construction rested on one assumption that we did not examine: that when Alice grabs Bob's public verification key, it genuinely belongs to Bob. Diffie–Hellman lets two strangers agree on a shared secret over a wire that Eve is reading. RSA and elliptic-curve signatures let anyone verify that a message came from the holder of a particular private key. But none of that tells you *whose* key you are holding. A public key is just a large number — nothing inside the number itself tells you who generated it.

This is the binding problem, and it is the subject of this chapter. If Eve can convince you that her key is Bob's, then every guarantee fails: she decrypts what you meant for Bob, she signs in his name, she sits in the middle of your "secure" channel and relays a doctored copy of everything. No cryptography is broken; what fails is your belief about whose key you hold.

We will build up the standard answer the Internet actually uses — X.509 certificates, certificate authorities, chains of trust — starting from the smallest possible version with tiny numbers, then layer on the machinery that keeps it from falling over in practice: revocation, the real attacks that have happened, Certificate Transparency, and the automation (Let's Encrypt and ACME) that made HTTPS nearly universal. We close with the main alternative, PGP's web of trust, and why it lost.

24.1 The binding problem

Here is a concrete version of the danger.

Alice wants to send a credit-card number to her bank's website, `bank.example`. The bank has an RSA key pair (pk_B, sk_B). The plan: Alice fetches pk_B, encrypts under it (or, more realistically, uses it to authenticate a Diffie–Hellman exchange), and only the bank, holding sk_B, can read the result. This works only if Alice gets the real pk_B.

Eve runs the coffee-shop WiFi. When Alice's laptop asks the network for `bank.example`, Eve answers. She hands Alice her own key pk_E and claims it is the bank's. Alice, seeing only a number, has no way to tell. She encrypts to pk_E, Eve decrypts with sk_E, reads the card number, re-encrypts to the

real pk_B, and forwards it on. Both Alice and the bank see a working session. Eve sees everything.

Intuition

Public-key crypto solves *secrecy* and *authenticity of messages* given that you already know the right key. It does not, by itself, solve *identity*. "This message was signed by the holder of key K" is a mathematical fact. "Key K belongs to Bob's Bank" is a fact about the world, and the world is not provable by exponentiation. Some out-of-band authority has to vouch for the mapping

$$\text{name} \longmapsto \text{public key}.$$

A certificate is just that vouching, written down and signed.

24.1.1 Three ways to bind a name to a key

There are essentially three architectures for solving the binding problem, and every real system is one of them or a blend.

1. **Trust on first use (TOFU).** The first time you talk to a party, you accept whatever key they present and remember it. On every later connection you check that the key has not changed. This is how SSH treats host keys: the famous prompt asking you to confirm a fingerprint, then silence forever after (or a loud warning if the key ever changes). Cheap, no central authority — but the first connection is wide open, and key rotation is painful.

2. **A trusted introducer (the CA model).** A small set of authorities you have decided to trust will sign statements of the form "key K belongs to `bank.example`." You ship with their public keys baked in. Anyone can then prove their key by showing you such a signed statement. This is the web's model, and most of this chapter.

3. **A web of trust.** There are no special authorities. Instead, *everyone* can vouch for everyone else, and you decide whom to believe by tracing chains of personal endorsements. This is PGP. We get to it at the end.

History

Loren Kohnfelder invented the digital certificate in his 1978 MIT bachelor's thesis under Leonard Adleman, proposing that a trusted authority sign a record binding a user's name to a public key. A decade later, both the phrase "public-key infrastructure" and the X.509 certificate format grew out of the 1988 X.500 directory project, an ambitious telecom effort (CCITT, now ITU-T) to build a single global directory of everything and everyone. X.500 itself never took over the world, but its certificate format, X.509, was adopted far beyond it and became the standard for TLS, S/MIME, code signing, and smart cards — so the data structure your browser checks a hundred times a day was designed for a 1980s phone-company directory that does not exist. The version we use, X.509v3 (1999), bolted on the *extensions* mechanism that made it flexible enough to survive.

24.2 A certificate is a signed statement

Without the formatting, a certificate is one sentence with a signature under it:

"The public key pk belongs to the entity named N, and this is true until time T. — signed, the CA."

Formally, let the certificate authority (CA) have its own signature key pair (pk_{CA}, sk_{CA}). A certificate for subject N with key pk_N is

$$\text{cert} = \big(\underbrace{N \,\|\, pk_N \,\|\, T \,\|\, \ldots}_{\text{the "to be signed" body, call it } m} , \sigma \big), \qquad \sigma = \text{Sign}(sk_{CA}, m).$$

To check it, you compute $\text{Vrfy}(pk_{CA}, m, \sigma)$. If it passes, and you trust the CA, you now believe pk_N is N's key. You no longer have to know Bob's key in advance. You have to know *one* key, pk_{CA}, and from it you can authenticate the keys of millions of websites you have never visited. The CA's key is the trust anchor from which all the other keys are authenticated.

Why it works

Why is forging a certificate hard? Because forging σ means forging a signature under sk_{CA}, which by the unforgeability of the signature scheme (Chapter 22) is infeasible without sk_{CA}. Eve can write any body m she likes — "pk_E belongs to `bank.example`" — but she cannot produce a σ that verifies under pk_{CA}. The security of the entire web's identity layer reduces to: (1) the signature scheme is unforgeable, and (2) the CAs guard their private keys and only sign true statements. Point (1) is math we can prove. Point (2) is an operational, human, and political problem, and it is where every real-world disaster has come from.

24.2.1 A toy certificate with tiny numbers

Let us actually sign one. We will use textbook RSA signatures (Chapter 19) with small numbers so you can follow every step by hand. Real keys are thousands of bits; ours will be tens.

Let the CA pick primes $p = 61$, $q = 53$, so $n = pq = 3233$ and $\phi(n) = 60 \cdot 52 = 3120$. Choose public exponent $e = 17$ (it is coprime to 3120). The private exponent d satisfies $ed \equiv 1 \pmod{3120}$; running the extended Euclidean algorithm gives $d = 2753$ (check: $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1$). So

$$pk_{CA} = (n, e) = (3233, 17), \qquad sk_{CA} = d = 2753.$$

Now suppose the body of Bob's certificate hashes down to the value $m = 100$. (A real cert hashes a long DER blob with SHA-256; we just assert the hash is 100.) The CA signs by raising to d:

$$\sigma = m^d \bmod n = 100^{2753} \bmod 3233.$$

Computing this by square-and-multiply gives $\sigma = 1391$. The certificate Bob now carries is the pair $(m = 100,\ \sigma = 1391)$ — plus, in reality, the human-readable body that hashes to 100.

A browser that already trusts $pk_{CA} = (3233, 17)$ verifies by raising σ to e:

$$\sigma^e \bmod n = 1391^{17} \bmod 3233 = 100 = m. \checkmark$$

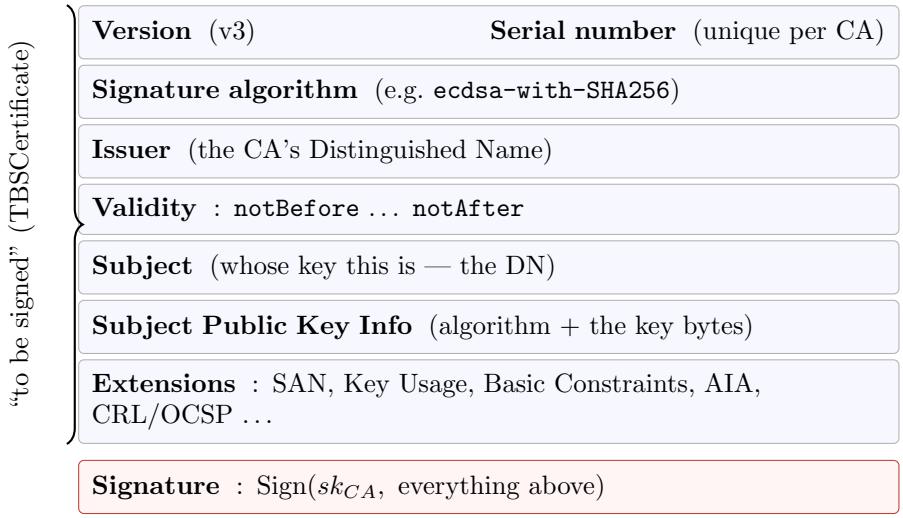
It recomputes the hash of the body, gets 100, sees it matches $\sigma^e \bmod n$, and accepts. Eve, who knows (n, e) but not d, cannot find a σ' with $(\sigma')^{17} \equiv 100$ without inverting RSA.

Common trap

Textbook RSA signatures like the toy above are dangerously malleable — given signatures on m_1 and m_2 you get one on $m_1 m_2 \bmod n$ for free, and you can forge on chosen values by working backwards. Never sign a raw message this way. Real certificates use a padded, randomized scheme such as RSASSA-PSS, or sign with ECDSA / Ed25519. The point of the toy is only to show the *shape*: CA signs the body, verifier checks with the CA's public key.

24.3 Inside an X.509 certificate

The sentence "*pk* belongs to *N* until *T*" has to be written in a format every implementation agrees on, byte for byte, or signatures will not verify. That format is X.509, encoded in DER (Distinguished Encoding Rules, a canonical form of ASN.1). You rarely read the raw bytes; tools show you the fields. Here is the anatomy.



A few fields deserve a closer look because attacks have turned on them.

Subject and the Common Name trap. Historically the entity's name lived in the Subject's *Common Name* (CN), a free-text field meant for human-readable names like "Bob's Bank, Inc." Early browsers also matched the website's hostname against the CN. That was a mistake, and we will see the null-byte attack it enabled. Modern certificates put hostnames in the **Subject Alternative Name** (SAN) extension, a structured list of DNS names like `bank.example` and `www.bank.example`. Browsers since roughly 2017 ignore the CN entirely and match *only* the SAN.

Basic Constraints: `cA = TRUE/FALSE`. This single bit says whether the certificate is allowed to sign *other* certificates. A leaf certificate for a website has `cA = FALSE`; it can authenticate a TLS server but cannot mint child certs. A CA certificate has `cA = TRUE`. Forgetting to check this bit was the cause of a well-known attack (Section 24.6).

Key Usage / Extended Key Usage. Restricts what the key may do: sign certificates, do TLS server auth, sign code, encrypt email. A key issued "for TLS server authentication" should be rejected if presented for code signing.

AIA, CRL Distribution Points, OCSP. Pointers telling the verifier where to fetch the issuer's certificate and where to check whether this certificate has been revoked (Section 24.5).

Implement it

You do not need a library to peek inside a real certificate — OpenSSL ships everywhere. The following is *for learning only*; do not parse security-critical certificates with hand-rolled code, use a vetted library.

```
# Fetch and pretty-print the certificate a live server presents:
openssl s_client -connect example.com:443 -servername example.com </dev/null 2>/dev/
    null \
  | openssl x509 -noout -text

# Look for, in the output:
#   Issuer:  (which CA signed it)
#   Subject: (whose key it is)
#   Validity Not Before / Not After
#   X509v3 Subject Alternative Name:  DNS:example.com, DNS:www.example.com
#   X509v3 Basic Constraints: CA:FALSE
#   X509v3 Extended Key Usage: TLS Web Server Authentication
```

The `-text` view is the human rendering of the DER bytes that were actually signed.

24.4 Chains of trust

If a single CA had to sign every certificate on Earth with its most precious key, that key would be online, busy, and one breach away from catastrophe. So the system uses *delegation*, and the delegation is itself just more certificates.

At the top sits a **root CA**. Its certificate is self-signed: the issuer and subject are the same, and it signs itself with its own key. A self-signed certificate proves nothing cryptographically — anyone can self-sign — so roots are not trusted because of their signature. They are trusted because their public keys are *shipped in your browser and operating system*, vetted by a root program (Mozilla's, Apple's, Microsoft's, Google's) under heavy audit. The root store is where trust actually originates; everything else is derived from it.

The root rarely signs leaf certificates directly. Instead it signs one or more **intermediate CA** certificates (with `cA = TRUE`), and the intermediates, kept online, sign the millions of **leaf** certificates for actual websites. The root key can then live offline in a vault, used only a few times a year.

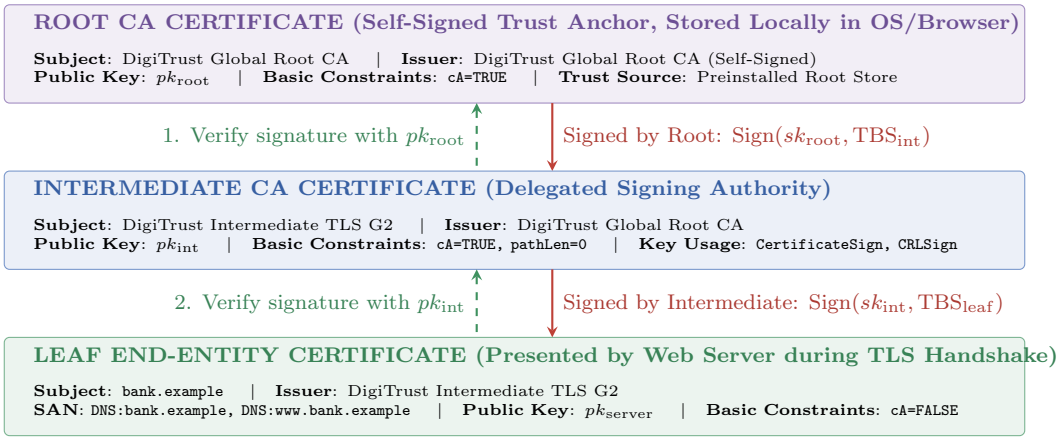


Figure 24.1: The hierarchical X.509 certificate chain of trust. The root CA resides in the client's preinstalled trust store and delegates signing authority to an intermediate CA (`cA=TRUE`). The intermediate signs the leaf server certificate (`cA=FALSE`). The client verifies signatures bottom-to-top until anchoring at the trusted root.

When you connect, the server sends its leaf certificate *and* the intermediate(s) — the chain. Your client validates the chain bottom to top:

1. Verify the leaf's signature using the intermediate's public key.
2. Verify the intermediate's signature using the root's public key.
3. Check that the root is in your trust store.
4. For *every* link, check: validity dates cover "now"; the issuer field of each cert equals the subject of the one above it; the signing certs have `cA = TRUE`; key-usage and name constraints are respected; the certificate is not revoked.
5. Finally, check that the hostname you asked for matches a SAN entry in the leaf.

Formal

A chain $C_0, C_1, \ldots, C_k$ (leaf to root) is *valid for name* N *at time* t iff:

- Vrfy $\big(pk_{i+1},\ \mathrm{TBS}(C_i),\ \sigma(C_i)\big) = 1$ for all $i = 0, \ldots, k-1$, where pk_{i+1} is the public key in C_{i+1};
- C_k (or its key) is a trust anchor;
- $t \in [\texttt{notBefore}_i, \texttt{notAfter}_i]$ for every i;
- every C_i with $i \geq 1$ has `BasicConstraints.cA` = TRUE and a `keyCertSign` usage;
- path-length and name constraints in the CA certs are satisfied;
- N matches a SAN of the leaf C_0; and
- no C_i is revoked at time t.

Drop *any one* of these checks and you have a real, exploited vulnerability in the historical record. The `cA` check is the famous one.

24.4.1 Tracing a tiny chain by hand

Reuse the toy RSA CA above as the *root*: $pk_{root} = (3233, 17)$, $sk_{root} = 2753$. The root signs an intermediate. Suppose the intermediate has its own key pair and the hash of its certificate body is $m_{int} = 920$. The root signs:

$$\sigma_{int} = 920^{2753} \bmod 3233 = 995.$$

(You can verify $995^{17} \bmod 3233 = 920$.) Now the intermediate uses *its* key to sign Bob's leaf. To keep the arithmetic in one modulus, pretend the intermediate reuses the same (n, e, d) — a contrivance, never do this — and signs Bob's leaf body $m_{leaf} = 100$, producing $\sigma_{leaf} = 1391$ as before.

A client holding only pk_{root} now reasons:

- $\sigma_{leaf}^{17} \bmod 3233 = 1391^{17} \bmod 3233 = 100 = m_{leaf}$, and the intermediate's cert says its key is the one to use — leaf checks out *under the intermediate's key.*
- $\sigma_{int}^{17} \bmod 3233 = 995^{17} \bmod 3233 = 920 = m_{int}$ — intermediate checks out under the root's key.
- The root's key $(3233, 17)$ is in the trust store. Chain complete.

The client never had Bob's key, nor the intermediate's, in advance. One preinstalled anchor authenticated both, in sequence.

Walking question

Why not let the leaf certificate be signed directly by the offline root, skipping intermediates? You could, and a few systems do. But then the root's private key must come out of the vault for every issuance — millions of times — multiplying the chance it leaks. Worse, if it ever did leak, you would have to replace the root, and replacing a root means pushing a software update to every browser and OS on Earth and waiting years for the old one to age out. Intermediates limit the damage: if an intermediate's key is compromised, the root revokes that one intermediate and issues a new one, no client update required. The root key stays offline.

24.5 Revocation: taking it back

A certificate says "valid until T," but life intervenes before T. A server's private key is stolen. An employee with signing access is fired. A CA discovers it issued a certificate by mistake. The certificate is still a perfectly valid signed statement — the math still checks out — yet we need every client in the world to stop believing it, *now*, before its natural expiry.

This is revocation, and it is the genuinely hard, never-fully-solved part of PKI. The difficulty is structural: validation is offline and local (you can check a signature with no network), but revocation is a fact about the present moment that lives at the CA. To learn it you must, somehow, ask.

24.5.1 Certificate Revocation Lists (CRLs)

The first design: each CA publishes a signed list of the serial numbers of all certificates it has revoked and not yet expired. A client downloads the CRL, checks the CA's signature on it, and refuses any

certificate whose serial appears.

Intuition

A CRL is a blacklist with a timestamp and a signature, like a stack of bounced-check notices the bank mails out. It is simple and verifiable offline once you have it. The trouble is freshness versus size. To be current you must re-download often; but a busy CA's CRL can grow to many megabytes, because it lists *every* unexpired revoked cert. Fetching a multi-megabyte list before every TLS handshake is a non-starter, so clients cache CRLs for hours or days — which means there is a window where a revoked, stolen certificate still works. If you check revocation once a day, an attacker may have up to a day to use a stolen certificate.

24.5.2 OCSP: ask about one certificate

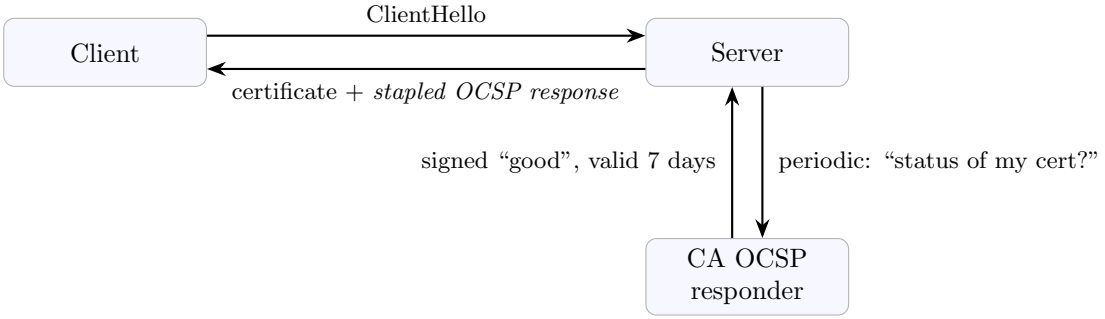
The Online Certificate Status Protocol fixes the size problem by inverting the query. Instead of downloading every revocation, the client asks a CA-run *responder* about the single certificate in front of it: "serial 4F:A3:. . . — good, revoked, or unknown?" The responder returns a small, signed, timestamped answer.

OCSP responses are smaller and fresher, but OCSP introduces three new problems:

1. **Latency.** An extra network round-trip to the CA, on the critical path of your page load.
2. **Privacy.** The OCSP responder learns every site you visit, because you phone home to it for each one. The CA becomes a surveillance choke point.
3. **Soft-fail.** If the responder is down or slow, what should the browser do? If it *hard-fails* (refuses to connect), then anyone who can DoS the responder can take websites offline — and an attacker holding a stolen cert simply blocks the OCSP query to keep their cert "unrevoked." So browsers historically *soft-fail*: if the check times out, proceed anyway. But soft-fail makes OCSP nearly useless against an active network attacker, who just drops the OCSP packets. The very adversary you most want to stop is the one who can defeat the check.

24.5.3 OCSP stapling: bring your own proof

The clever fix to latency and privacy: have the *server* fetch a fresh, signed OCSP response from the CA periodically, and "staple" it into the TLS handshake. Now the client gets the freshness proof from the server it is already talking to — no extra round-trip, no phoning the CA, no leaking your browsing to the CA.



Stapling solves latency and privacy. It does not by itself solve soft-fail: a network attacker can strip the staple, and the client falls back to "proceed anyway." The intended cure is the **Must-Staple** flag, an extension baked into the certificate that says "a valid stapled response is *required*; reject me if it is missing." Then stripping the staple kills the connection instead of bypassing the check. Adoption of Must-Staple has been thin, which tells you how genuinely awkward revocation remains.

In the real world

Real browsers have largely given up on classic per-connection revocation checking and moved to *pushed* revocation. Chrome ships **CRLSets**: Google crawls CAs' CRLs, compresses the high-value revocations into a compact set, and pushes it to Chrome through the normal update channel, so the check is local and instant with no network call at handshake time. Firefox does the analogous thing with **CRLite**, which uses a cascade of Bloom filters to compress the status of *every* known WebPKI certificate into a few megabytes that fit on the client and answer with no false negatives. The lesson the industry absorbed: online revocation checks on the handshake critical path are too slow, too leaky, and too easy to suppress, so push a fresh compressed snapshot to the client instead and check locally. Short-lived certificates (Section 24.8) attack the same problem from the other end: if a cert lives only 90 days — soon perhaps days — the revocation window shrinks to where it barely matters.

Connection

CRLite's Bloom-filter cascade is a direct application of the probabilistic data structures from the companion algorithms book. A single Bloom filter has one-sided error: it never says "absent" for something present, but it sometimes false-positives. A *cascade* chains filters so that each layer catches the previous layer's false positives over a *known, finite* universe (all issued certificates, learned from Certificate Transparency logs), driving the error to zero while keeping the structure tiny. The birthday-bound and false-positive-rate analyses you met there are exactly the tools used to size each layer.

24.6 When the trust breaks: CA mis-issuance

The CA model has a serious weakness: **any** CA in your trust store can vouch for **any** name. There are hundreds of trusted CAs and intermediates, spanning many jurisdictions. Your browser trusts a certificate for `bank.example` signed by a CA in any country, whether or not the bank has ever heard of that CA. The system is only as trustworthy as its *weakest* CA — a structural vulnerability of the WebPKI that has been exploited repeatedly.

Attack

The null-prefix / CN confusion attack. Early TLS clients matched the hostname against the certificate's Common Name, which was a length-prefixed string. Many clients, though, were written in C and treated it as a NUL-terminated string. Moxie Marlinspike's 2009 attack exploited the gap. He requested a certificate for a name he legitimately controlled but crafted as:

`www.bank.example` \0 `.attacker.example`

A CA, validating control of `attacker.example`, might approve the registration. But a C-based client reading the CN stopped at the embedded NUL and saw exactly `www.bank.example`. The result was a single signed certificate that the victim's browser accepted for the bank. The fixes: stop matching on CN, match only on structured SAN entries, and forbid embedded NULs. This is why the CN field is now ignored.

Attack

The missing `Basic Constraints` check. Recall that only certificates with `cA = TRUE` may sign other certificates. In the mid-2000s, several widely deployed TLS stacks (including a famous Internet Explorer / CryptoAPI bug) *forgot to check that bit*. The consequences are severe (Figure 24.2). An attacker buys an ordinary *leaf* certificate for `attacker.example` — `cA = FALSE`, perfectly legitimate. Because the buggy client never checks `cA`, the attacker can then use that leaf's key to sign a brand-new certificate for `bank.example`, and the buggy client will happily build a chain: bank-leaf ← attacker-leaf ← real-CA ← root. Every signature verifies. The single skipped boolean turns any $10 certificate into a universal forging key. This is the canonical illustration of why *every* field in *every* link of the chain must be checked.

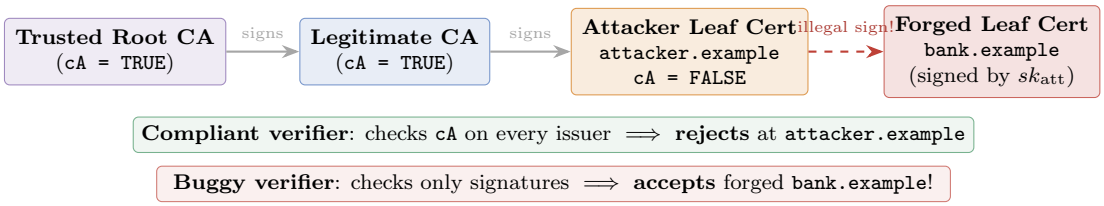


Figure 24.2: Exploiting a missing `BasicConstraints` (`cA`) check. The attacker obtains a legitimate leaf certificate for `attacker.example` (`cA=FALSE`) and uses its private key to sign a forged certificate for `bank.example`. A verifier that checks signatures without enforcing `cA=TRUE` on every intermediate link accepts the forged chain.

Attack

DigiNotar, 2011 — a CA fully compromised. The Dutch CA DigiNotar was breached by an attacker who used its signing infrastructure to issue *hundreds* of fraudulent certificates, including a wildcard `*.google.com`. That certificate was then used in a man-in-the-middle attack against an estimated 300,000 Iranian Gmail users, who saw a green padlock the whole time. The certificates were mathematically flawless; the CA itself had been subverted. The fallout: browser vendors yanked DigiNotar from every trust store within days, and the company — a functioning business with a long history — was bankrupt within weeks. The episode proved that a single compromised CA endangers the entire web, and it is the direct ancestor of Certificate Transparency.

Why it works

Why does one bad CA endanger everyone, when there are hundreds of good ones? Because trust in the CA model is a logical *OR*, not an *AND*. A certificate is accepted if *any* trusted CA signed it. You do not get the security of the strongest CA, or the average; you get the

security of the weakest. Adding more CAs makes things *worse*, never better, from a pure-trust standpoint, because it adds more keys that can each forge for your bank. Every structural fix since — name constraints, CAA records, Certificate Transparency, pinning — is an attempt to claw back some of that lost ground without throwing out the whole model.

24.6.1 Partial fixes before transparency

Public-key pinning (HPKP) let a site tell browsers "in future, only accept these specific keys for me," converting the trust-any-CA OR back into a narrow whitelist. It worked in theory and was a footgun in practice: a site that pinned the wrong key, or lost it, could lock itself out of its own users for the pin's lifetime ("HPKP suicide"). Browsers deprecated it.

CAA records (Certification Authority Authorization) let a domain publish, in DNS, which CAs are permitted to issue for it: `bank.example. CAA 0 issue "goodca.example"`. CAs are required to check CAA before issuing and refuse if they are not listed. It does not stop a malicious CA — a compromised CA can ignore CAA — but it stops *accidental* mis-issuance by well-run CAs, which is most of it. CAA is widely deployed and cheap.

Name constraints let a root limit an intermediate to a set of domains ("this intermediate may only issue under `*.gov.uk`"). When honored, they bound the blast radius of a delegated CA. The catch is the same as everywhere: they only help if every client enforces them.

24.7 Certificate Transparency

A central problem with DigiNotar was that the forgery stayed *invisible*. Google was MITM'd against its own users and had no way to know a `*.google.com` certificate existed. Certificate Transparency (CT) addresses exactly this: rather than trying to prevent mis-issuance, it makes mis-issuance *undeniable and detectable*.

The idea: every certificate a CA issues must be recorded in a public, append-only, cryptographically verifiable **log**. If your certificate is not in a log, browsers reject it. Now anyone — Google, your bank's security team, an independent monitor — can watch the logs and notice "a certificate for `google.com` was issued by a CA in another country that Google never asked for." The forgery cannot hide.

Intuition

CT does not make CAs trustworthy; it makes them *accountable*, much as a camera in a vault does not stop a thief but does put the theft on record. The design rests on one cryptographic primitive that turns "append-only public ledger" from a promise into a proof: the Merkle tree.

24.7.1 Merkle trees: proving membership and append-only-ness

A CT log is a growing list of certificates. We need two things from it: (1) a client should be able to verify "my certificate is in this log" cheaply, without downloading the whole log; and (2) the log

must not be able to *rewrite history* — remove or alter a certificate it already logged. Merkle trees give both.

Hash every certificate to a leaf. Hash pairs of leaves together, then pairs of those, up to a single **root hash** (the "tree head"). The log periodically signs the current root hash and timestamp; that signed head is its public commitment to "these exact certificates, in this order, are what I have logged."

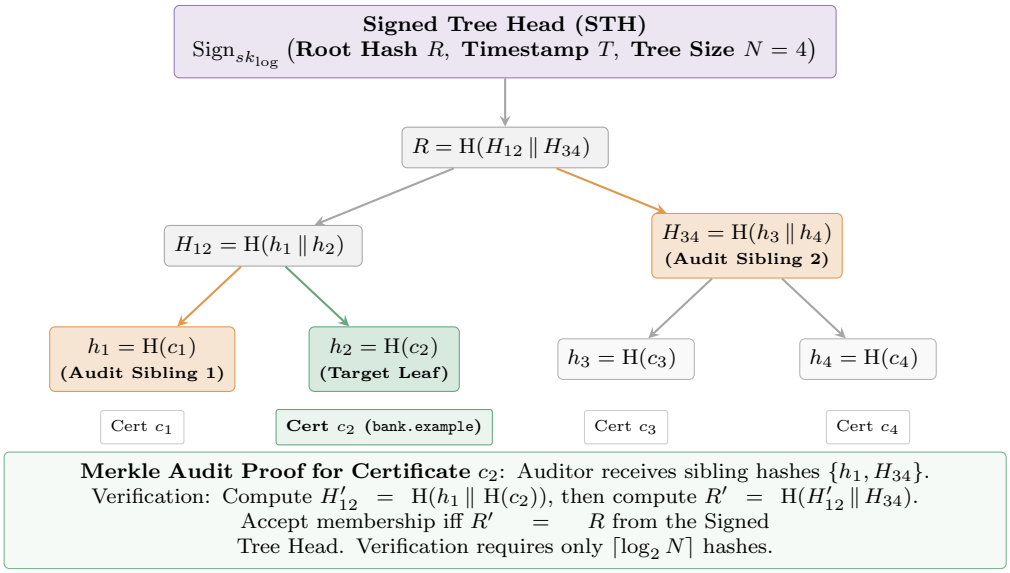


Figure 24.3: Certificate Transparency Merkle tree audit proof. To verify that certificate c_2 is recorded in the public append-only log, the log provides the Merkle audit path of sibling hashes (orange). The auditor recomputes the root hash along the path and compares it with the Signed Tree Head (STH).

To prove certificate c_2 is in the log, the log gives you the **audit path**: the sibling hashes you need to recompute the root, here h_1 and H_{34} (shaded). You compute

$$H\big(H(h_1 \,\|\, H(c_2)) \,\|\, H_{34}\big)$$

and check it equals the signed root. For a log of N certificates the path has only $\lceil \log_2 N \rceil$ hashes — about 30 even for a billion certificates. You verified membership while downloading almost nothing.

Why it works

Why can't the log lie? Because the root hash is a commitment under a collision-resistant hash. To show you a fake audit path proving c_2 is present when it is not, the log would need a second preimage or collision against H — infeasible. And to *remove* a certificate it already committed to, it would have to produce a new, consistent root that drops c_2 but matches the old signed head, which again requires breaking H. CT adds a second proof, the **consistency proof**: given two signed tree heads at sizes $m < n$, the log can prove the size-n tree is the size-m tree with $n - m$ entries *appended*, never edited. Auditors demand consistency proofs over time; a log caught failing one is distrusted. So "append-only" stops being a policy you hope the operator follows and becomes a property you can check.

Connection

This is the same Merkle-tree construction that underlies Git commit history, BitTorrent and IPFS content addressing, and blockchain blocks. The companion algorithms book develops Merkle trees as an authenticated data structure; here it carries a security reduction: "the log cannot cheat" reduces to "the hash is collision-resistant," exactly the property analyzed via birthday bounds in the probability volume.

24.7.2 How CT plugs into TLS

When a CA logs a certificate, the log returns a **Signed Certificate Timestamp** (SCT): a signed promise "I will include this certificate in my tree within a bounded time" (the Maximum Merge Delay). Browsers require a leaf certificate to come with SCTs from several independent logs — delivered in the certificate itself, in the TLS handshake, or via OCSP stapling — or they reject it. Since Chrome and Apple made CT mandatory (2018 onward), essentially every publicly trusted certificate is logged. That is why, today, you can search a service like `crt.sh` and see every certificate ever issued for a domain. Defenders use exactly that to catch mis-issuance early.

In the real world

CT has caught real mis-issuance in the wild. Monitors have flagged CAs issuing certificates with invalid fields, certificates for domains the requester did not control, and internal-hostname leaks where companies accidentally logged the names of private servers. Symantec's CA business was effectively ended after CT-driven scrutiny surfaced a pattern of improper issuance, leading Google to distrust its certificates — a multi-billion-dollar business unwound because the logs made the misbehavior auditable. CT replaced trust in CAs' self-reporting with public verification, and the WebPKI is safer for it.

24.8 Let's Encrypt and ACME: automating trust

For two decades, getting a certificate meant paying a CA, emailing documents, and pasting blobs into web forms — a manual, error-prone, expensive process. As a result, most of the web ran on unencrypted HTTP, because HTTPS was a hassle. Let's Encrypt (launched 2015, a nonprofit CA) set out to make certificates free and fully automatic, and in doing so moved the web from mostly-HTTP to mostly-HTTPS in a few years.

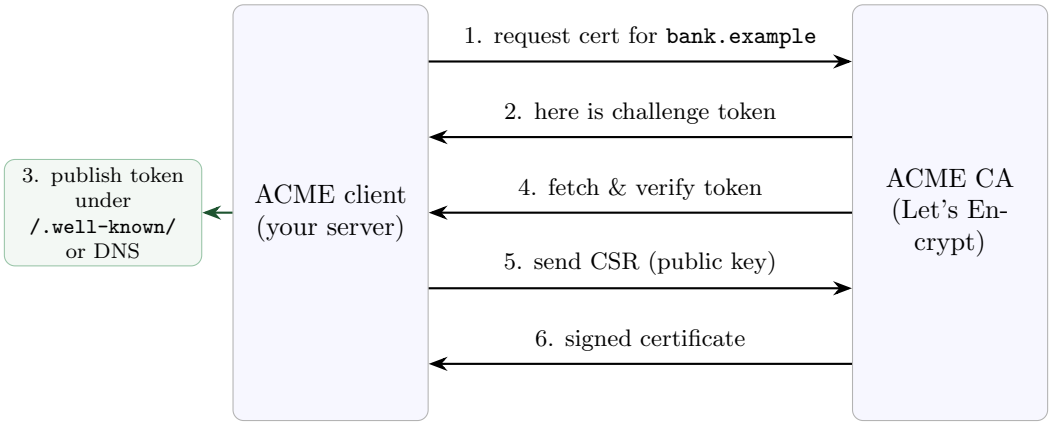
The underlying protocol is **ACME** (Automatic Certificate Management Environment), which a client speaks to the CA to prove domain control and obtain a certificate with no human in the loop.

24.8.1 Domain validation: proving you control the name

A CA must not hand `bank.example`'s certificate to just anyone who asks. For *domain-validated* certificates, the only thing the CA verifies is: *do you control this domain right now?* ACME proves it with a challenge–response. The CA sends a random token; the client must publish a value derived from it where only the domain's controller could put it.

1. **HTTP-01**: serve the file `http://bank.example/.well-known/acme-challenge/<token>` containing token $\|$ thumbprint(pk_{account}). The CA fetches it. Only someone running the web server for that name could place it.
2. **DNS-01**: publish a TXT record `_acme-challenge.bank.example` with the hash of that same value. Only someone controlling the domain's DNS could. (DNS-01 can prove control of an entire wildcard `*.bank.example`.)
3. **TLS-ALPN-01**: answer a special TLS handshake on port 443 with a certificate carrying the challenge — proving control of the TLS endpoint itself.

Because the response is cryptographically bound to the client's *account key* via the thumbprint, a network eavesdropper who sees the token cannot complete the challenge with their own key. The CA, satisfied, signs the certificate. The whole exchange takes seconds.



Common trap

Domain validation proves *control of the name*, nothing more. A DV certificate for `bank-of-america-login.example` certifies only that whoever holds it controls that exact domain — not that they are a bank, not that they are trustworthy. The padlock means "the connection is encrypted to the entity that controls this name," never "this site is safe." Phishers get free DV certificates for lookalike domains constantly. Higher-assurance *Organization-Validated* and *Extended-Validation* certificates add vetting of the legal entity, but browsers have quietly dropped the special EV address-bar treatment because users never reliably noticed it. Identity-of-the-business is a problem certificates only partly solve.

Implement it

Issuing a real certificate now takes a single command, which is what ACME was designed to achieve. *This is real, not toy* — but read your CA's rate limits before scripting it, and never paste private keys into untrusted machines.

```
# Using certbot (the reference ACME client) with the HTTP-01 challenge.
# It generates a keypair, proves control of the domain, fetches the cert,
# and sets up a cron job to renew automatically before the 90-day expiry.
sudo certbot certonly --standalone -d bank.example -d www.bank.example

```

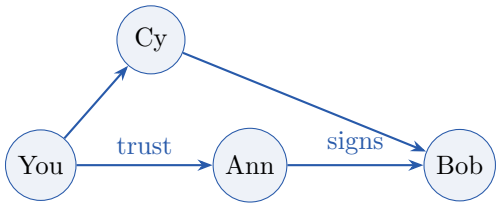
```
# Simplified, the account-key-bound challenge response is:
#   key_authorization = token || "." || base64url(SHA256(account_public_key_JWK))
# served at  http://bank.example/.well-known/acme-challenge/<token>
```

The reason Let's Encrypt issues *short-lived* (90-day, trending toward days) certificates is that automation makes frequent renewal painless — and short lifetimes shrink the revocation window, the very problem Section 24.5 could not fully solve. Besides making certificates cheaper, automation made short lifetimes, and the safer security posture they bring, practical.

24.9 The alternative: web of trust

The CA model centralizes trust in a few hundred authorities. PGP (Pretty Good Privacy, Phil Zimmermann, 1991) took the opposite approach: *no* central authorities, and let people vouch for each other directly. This is the **web of trust**.

In PGP, anyone can *sign* anyone else's public key, asserting "I have checked that this key belongs to this person." If you meet Bob in person and verify his key's fingerprint, you sign it. You upload the signature. Now anyone who trusts *you* can use your signature as evidence about Bob. Trust propagates through chains of personal endorsement.



You reach Bob's key through Ann (or Cy). No authority involved — just transitive personal endorsement.

The user assigns *trust levels* to keys: a key is considered valid if it is signed by one fully trusted introducer, or several marginally trusted ones. The math is the same signature-verification we have used throughout; what changes is the trust *topology* — a sprawling social graph instead of a shallow tree rooted at preinstalled CAs.

Zimmermann released PGP as free software in 1991, partly in response to a proposed US law mandating crypto backdoors. Because strong crypto was then classified as a munition under export law, the US government opened a criminal investigation into Zimmermann for "exporting" PGP when it spread overseas. His response became famous: he published the entire PGP source code as a *book*, which was protected speech under the First Amendment and could be exported freely, then re-scanned abroad. The case was dropped in 1996. The web of trust was, from the start, as much a political statement — trust should not require permission from authorities — as a technical design.

Why it works

The web of trust's strength is also its weakness. **Strength**: no single point of failure, no DigiNotar that can forge for everyone, no authority to compel or breach. Your trust decisions are yours. **Weakness**: it does not scale to strangers. To talk securely to a website you have never met, run by people you will never meet, there is no chain of personal endorsements to follow — and building one would require a key-signing social graph far denser than any that exists. The CA model scales precisely because it does *not* require you to know anyone; one preinstalled anchor authenticates the entire web. That is why the web runs on CAs and the web of trust stayed a niche for activists, developers, and the privacy-conscious.

In the real world

The web of trust did not die, but the world moved past its hardest parts. Modern secure messengers — Signal, WhatsApp, iMessage — bind keys to identities not through CAs *or* a global signing graph, but through *safety numbers* you can optionally verify out-of-band (scan a QR code when you meet) plus key-change notifications (TOFU with alerts). It is the SSH host-key model applied to people: accept on first contact, shout if the key ever changes. For software, *key transparency* systems (used by WhatsApp, Apple's iMessage Contact Key Verification, and Keybase before it) put the name-to-key mapping into a Merkle-tree append-only log — the same CT machinery from Section 24.7, now applied to people instead of websites. The binding problem does not go away, and the answers increasingly converge on "make the mapping public and auditable."

24.10 Putting it together: what your browser does

When you type `https://bank.example` and press Enter, the identity machinery of this entire chapter fires in milliseconds:

1. The server sends its leaf certificate and intermediate(s) in the TLS handshake.
2. Your browser builds a chain from the leaf up to a root in its trust store, verifying each signature with the key above it.
3. It checks every link: validity dates, issuer/subject matching, `cA=TRUE` on the signers, key-usage flags, name constraints.
4. It checks that `bank.example` matches a SAN in the leaf.
5. It checks revocation — typically against a pushed set (CRLSet/CRLite) rather than a live OCSP call.
6. It checks that the leaf carries valid **SCTs** proving it was logged in Certificate Transparency.
7. Only if *all* of that passes does the padlock appear and the encrypted session proceed — and the certificate's key is what authenticates the Diffie–Hellman exchange that follows.

The cryptography you learned in earlier chapters — signatures, hashes, key exchange — is the easy, provable part. The hard part, the part that has actually failed in the wild, is binding those keys to the right names and revoking the bindings when they go bad. PKI is the messy but necessary

infrastructure that lets provably secure primitives operate in a world of fallible authorities, stolen keys, and adversaries on the wire.

Pause

The chapter in one sentence: *a certificate is a signed claim that a key belongs to a name, a chain reduces trusting millions of keys to trusting a few preinstalled anchors, and every catastrophe in the history of the WebPKI came not from broken math but from a CA vouching for something it should not have — which is why the modern fixes (CT, CAA, short-lived certs, key transparency) are all about making those claims public, auditable, and short-lived.*

With symmetric ciphers, hashes, MACs, signatures, key exchange, and certificates in place, we have all the pieces of a real-world secure channel. In Chapter 25, we see how Transport Layer Security (TLS) combines them to verify a server's certificate, agree on forward-secret session keys, and encrypt web traffic.

Exercises

Exercise 24.1 (Sign and verify a toy certificate)**.** Using the toy CA with $n = 3233$, $e = 17$, $d = 2753$, suppose Bob's certificate body hashes to $m = 65$. Compute the CA's signature $\sigma = m^d \bmod n$ by square-and-multiply, then verify by computing $\sigma^e \bmod n$ and checking it equals 65. Show your intermediate squarings.

Exercise 24.2 (Why the chain order matters)**.** A client receives a chain (leaf, intermediate, root) but validates it *top-down*, checking the root is trusted first and then descending, and forgets to verify that each certificate's issuer field equals the subject of the one above it. Describe a concrete forgery: how can an attacker who owns one legitimately-signed leaf splice in a certificate of their choosing? Which check in Section 24.4 stops it?

Exercise 24.3 (The `cA` bit, exploited)**.** Spell out, step by step, the attack from Section 24.6 in which a client fails to check `BasicConstraints`. Start from the attacker buying a normal leaf certificate for `attacker.example` and end with the client accepting a forged certificate for `bank.example`. At which exact step would a correct `cA` check abort the chain?

Exercise 24.4 (Merkle audit path)**.** A CT log has eight certificates $c_1, \ldots, c_8$ at the leaves of a balanced Merkle tree (leaf i is $h_i = H(c_i)$, parents are hashes of concatenated children). Write the exact audit path (the list of sibling hashes, in order from leaf to root) needed to prove that c_5 is in the log, and write the sequence of hash computations a verifier performs to recover the root.

Exercise 24.5 (Revocation tradeoffs)**.** Compare CRLs, plain OCSP, and OCSP stapling along four axes: freshness, client privacy, latency added to the handshake, and behaviour under an active network attacker who can drop packets. For each mechanism, identify the single property that makes it inadequate *on its own*, and explain why "Must-Staple" is needed to close the gap.

Exercise 24.6 (Soft-fail is self-defeating)**.** Explain precisely why an active man-in-the-middle holding a *stolen but revoked* certificate is undisturbed by a browser that does OCSP checking with soft-fail. Then argue why hard-fail OCSP is not a usable fix on its own, and how short-lived certificates and pushed revocation sets (CRLSet/CRLite) sidestep the dilemma.

Exercise 24.7 (ACME challenge binding)**.** In the HTTP-01 challenge, the published value is

$$\text{token} \,\|\, \text{“.”} \,\|\, \text{base64url}\big(\,\text{H}(\text{account public key})\big),$$

not just the token. Construct the attack that would succeed if the response were *only* the bare token (think about a passive eavesdropper on the network, or a shared-hosting neighbour), and explain how binding to the account key defeats it.

Exercise 24.8 (DV is not identity)**.** A user sees a valid padlock on `https://secure-bank-login.example` and concludes the site is their bank. Exactly what does the certificate prove, and what does it not prove? Explain why a DV certificate for this domain is trivial for a phisher to obtain, and what additional mechanism (if any) could have warned the user.

Exercise 24.9 (Trust is an OR)**.** Argue formally why, in the CA model, adding a new trusted CA can only *decrease* or hold constant the security of the name-to-key binding for an existing domain, never increase it. Then explain how CAA records and name constraints reintroduce an *AND*-like restriction, and why CT helps even though it prevents nothing.

Exercise 24.10 (Web of trust at scale)**.** Estimate, with a back-of-envelope model, how many pairwise key-signings would be needed for a PGP-style web of trust to give every pair of N Internet users a short verification path (say, length ≤ 3). Compare with the CA model's requirement (each user trusts a constant number of roots). Use this to explain, quantitatively, why the web of trust never scaled to the public web. (Hint: think about the diameter and density of random graphs from the probability companion.)

Chapter 25

TLS: Securing the Web

Every chapter so far has built a single component. AES is a permutation on 128-bit blocks. GCM turns it into authenticated encryption. Diffie–Hellman lets two strangers agree on a secret over a monitored channel, and elliptic curves make that agreement small and fast. Digital signatures prove authorship, and certificates (Chapter 24) bind a public key to a domain name you can read. TLS is the protocol that assembles all of these pieces to secure the web.

When your browser shows a padlock and the address bar reads `https://`, you are speaking **TLS**: Transport Layer Security. It is the protocol underneath HTTPS, but also underneath secure email (SMTP over TLS), encrypted DNS, most VPNs, the control channel of countless mobile apps, and the back-end traffic of essentially every cloud service. Trillions of TLS handshakes happen every day. If you study one applied protocol in depth, make it this one: in TLS, a single mismatched assumption between two components has repeatedly become a real attack with a logo and a CVE number.

We will focus on **TLS 1.3**, standardized in 2018 as RFC 8446. It is the version you should deploy, it is dramatically cleaner than its predecessors, and its design choices read like a catalogue of lessons learned the hard way from two decades of attacks on TLS 1.0 through 1.2. We will trace a full handshake with small numbers, watch the key schedule derive a dozen keys from one shared secret, see how the record layer carries encrypted application data, and understand exactly which property each message protects. Then we will revisit the famous disasters — BEAST, POODLE, Heartbleed, and a few cousins — because the surest way to appreciate why TLS 1.3 looks the way it does is to see what the old designs let an attacker do.

25.1 What TLS provides, and against whom

Before any protocol, pin down the threat model. TLS assumes the worst plausible network: the **Dolev–Yao** attacker. Eve is not merely listening. She sits on the wire and can read, drop, reorder, replay, and inject any packet she likes. She controls the routers, the coffee-shop access point, possibly the ISP. The only thing she cannot do is break the cryptographic primitives or read memory inside Alice's and Bob's machines. This is the realistic model for the public Internet, and TLS is built to survive it.

Against that attacker, a completed TLS connection between a client (Alice's browser) and a server

(Bob's web server) promises three things.

- **Confidentiality.** Everything after the handshake — URLs, cookies, form data, page contents — is encrypted. Eve sees ciphertext and traffic sizes, not plaintext.
- **Integrity.** Eve cannot alter a single byte of the conversation without detection. Every record is authenticated; a flipped bit makes decryption fail loudly.
- **Authentication.** Alice's browser cryptographically verifies that it is really talking to the server that owns `example.com`, not to Eve impersonating it. This relies on certificates and the PKI from the previous chapter. (By default only the server is authenticated; client authentication is optional and used in enterprise and mutual-TLS settings.)

Focus

What TLS does *not* promise. It does not hide *that* you talked to a server, nor how much, nor when. The server's identity leaks through the SNI field (see later) and through the IP address itself. TLS protects the *contents and integrity* of a channel to an *authenticated* endpoint. It says nothing about whether that endpoint is trustworthy — a phishing site can get a perfectly valid certificate for `paypa1.com`. Anonymity is the job of Tor; endpoint trust is the job of the user and the browser's UI.

There is a fourth property TLS 1.3 takes seriously enough that it deserves a name of its own: **forward secrecy**. If, a year from now, Eve breaks into Bob's server and steals his long-term private key, she should *not* be able to decrypt the traffic she recorded today. We met this idea in Chapter 23; TLS 1.3 makes it mandatory. We will see exactly how.

Intuition

The handshake is the short, expensive phase in which client and server, communicating over a network full of eavesdroppers and impostors, agree on keys nobody else can compute and check the server's identity. The record protocol is the long, cheap phase that follows, in which every message is encrypted and authenticated under those keys. Eve can watch records go by and can destroy them, but she cannot read them, forge them, or impersonate either endpoint.

25.2 The shape of a handshake

A TLS connection has two phases. The **handshake** runs first: it establishes shared keys and authenticates the server, using public-key cryptography. Then the **record protocol** takes over: it carries the actual application data under fast symmetric authenticated encryption. The handshake is the expensive part (a few public-key operations and one network round trip); the record protocol is cheap and runs for the life of the connection.

The headline achievement of TLS 1.3 is that the handshake takes a single round trip — "1-RTT" — before the client can send encrypted application data. TLS 1.2 needed two. The trick is that the client *guesses* which key-exchange group the server will accept and sends its key share immediately, in its very first message. If it guesses right (it almost always does), the server can finish the key agreement in one reply. Figure 25.1 contrasts the legacy TLS 1.2 two-round-trip flow with the modern TLS 1.3 one-round-trip protocol. We will then walk each message in detail.

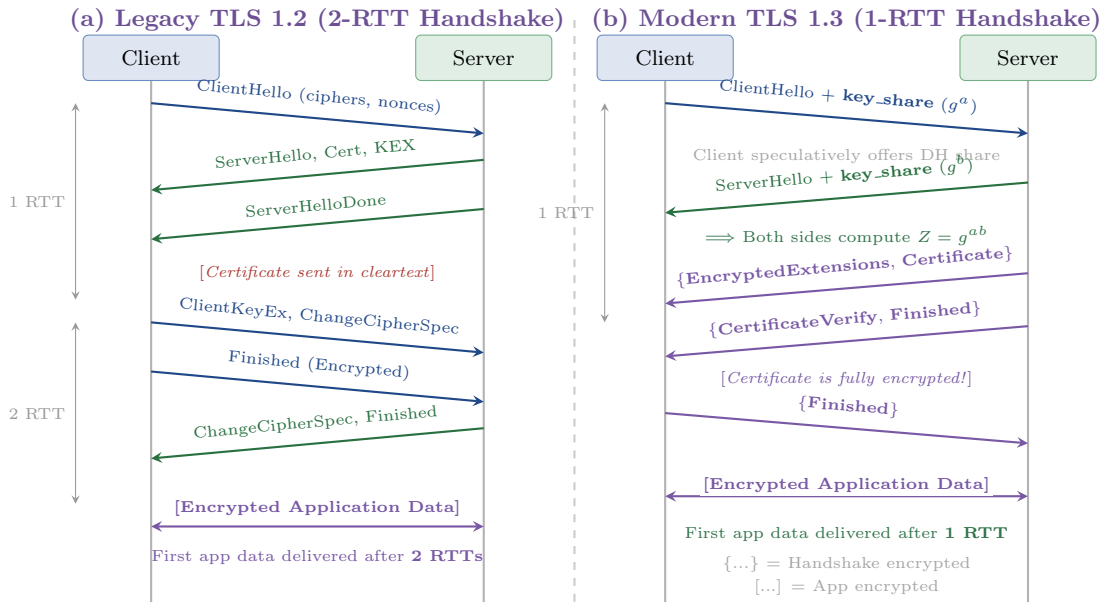


Figure 25.1: Comparison of TLS handshakes. In legacy TLS 1.2 (a), public-key parameters and certificates are negotiated sequentially in cleartext over two round trips before application data can flow. In TLS 1.3 (b), the client speculatively transmits its Diffie–Hellman key share in the first packet, collapsing connection latency to 1 RTT while encrypting the server's certificate and extensions.

The notation matters. Messages in the clear are sent before any key exists. Messages in braces $\{\cdots\}$ are encrypted under *handshake* keys — a key the two parties can compute the instant the ServerHello arrives, even though the server has not yet proven who it is. Messages in brackets $[\cdots]$ ride under the final *application* keys. That TLS 1.3 encrypts most of the handshake itself (the certificate, the extensions) is a privacy win over 1.2, where the server's certificate flew in plaintext for any eavesdropper to log.

25.3 Step 1: key exchange (the ClientHello)

The client opens with a `ClientHello`. Stripped to essentials it carries:

- a random nonce r_C (32 bytes), to make every handshake unique;
- a list of *cipher suites* it supports (e.g. `TLS_AES_128_GCM_SHA256`);
- a list of *supported groups* for key exchange (e.g. X25519, `secp256r1`, or finite-field groups);
- one or more *key shares*: for each group it is betting on, an actual ephemeral public value. For an elliptic-curve group this is a point; for a classic finite-field group it is $g^a \bmod p$.
- the *server name* it wants to reach (the SNI extension), so a server hosting many sites knows which certificate to present.

The key share is the most important field. The client picks a fresh random secret a, computes its public share, and ships it immediately. Because a is fresh and thrown away when the connection ends, this is **ephemeral** Diffie–Hellman — the source of forward secrecy. TLS 1.3 *only* allows ephemeral

(EC)DHE key exchange; the old static-RSA key transport, where the client encrypted a secret under the server's long-term RSA key, is gone, precisely because it had no forward secrecy.

25.3.1 A worked exchange with tiny numbers

Real TLS uses X25519 or 256-bit curves, with numbers hundreds of bits long. To *see* the arithmetic, shrink it to a finite-field group you can do by hand. Take the prime $p = 23$ and generator $g = 5$ (a generator of $(\mathbb{Z}/23\mathbb{Z})^*$).

The client picks secret $a = 6$ and sends

$$A = g^a \bmod p = 5^6 \bmod 23.$$

Compute $5^2 = 25 \equiv 2$, so $5^4 \equiv 2^2 = 4$, and $5^6 = 5^4 \cdot 5^2 \equiv 4 \cdot 2 = 8$. The client sends $A = 8$.

The server picks secret $b = 15$ and sends

$$B = g^b \bmod p = 5^{15} \bmod 23.$$

Build it up: $5^4 \equiv 4$, $5^8 \equiv 4^2 = 16$, $5^{15} = 5^8 \cdot 5^4 \cdot 5^2 \cdot 5^1 \equiv 16 \cdot 4 \cdot 2 \cdot 5$. Now $16 \cdot 4 = 64 \equiv 64 - 2 \cdot 23 = 18$; $18 \cdot 2 = 36 \equiv 13$; $13 \cdot 5 = 65 \equiv 65 - 2 \cdot 23 = 19$. So $B = 19$.

Each side now raises the other's share to its own secret:

$$\text{client: } B^a = 19^6 \bmod 23, \qquad \text{server: } A^b = 8^{15} \bmod 23.$$

The client: $19 \equiv -4$, so $19^2 \equiv 16$, $19^4 \equiv 16^2 = 256 \equiv 256 - 11 \cdot 23 = 3$, $19^6 = 19^4 \cdot 19^2 \equiv 3 \cdot 16 = 48 \equiv 2$. The server: $8^2 = 64 \equiv 18$, $8^4 \equiv 18^2 = 324 \equiv 324 - 14 \cdot 23 = 2$, $8^8 \equiv 2^2 = 4$, $8^{15} = 8^8 \cdot 8^4 \cdot 8^2 \cdot 8 \equiv 4 \cdot 2 \cdot 18 \cdot 8$. Then $4 \cdot 2 = 8$; $8 \cdot 18 = 144 \equiv 144 - 6 \cdot 23 = 6$; $6 \cdot 8 = 48 \equiv 2$. Both get the same shared secret

$$Z = g^{ab} \bmod p = 2.$$

Eve, who saw $p = 23$, $g = 5$, $A = 8$, $B = 19$, must solve the discrete log to recover a or b. With $p = 23$ she can; with a 256-bit curve she cannot, and the security of the exchange rests on that difference.

Connection

The cost of computing $g^a \bmod p$ is the modular-exponentiation cost from the algorithms companion: $O(\log a)$ multiplications by square-and-multiply, which is why we built up 5^{15} from $5^8, 5^4, 5^2$ rather than multiplying fifteen times. The security rests on the discrete-log assumption (Chapter 18) or, for X25519, the elliptic-curve discrete-log problem (Chapter 21). The shared secret Z is *not* used directly as a key — raw g^{ab} has algebraic structure and is not uniform. It is run through a key-derivation function, the subject of Section 25.5, exactly as the "hash the DH output" advice in Chapter 23 demanded.

25.4 Step 2: the ServerHello and the handshake key

The server receives the ClientHello, picks one of the offered groups, generates its own ephemeral secret b, and replies with a `ServerHello` containing its own random nonce r_S, the chosen cipher

suite, and its key share B. The instant this message lands, both parties hold the same shared secret $Z = g^{ab}$ and the same transcript of the two hellos.

From here, *everything* is encrypted. The server immediately derives the handshake traffic keys (next section) and sends the rest of its messages under them. This is a sharp break from TLS 1.2, where the certificate was plaintext.

Common trap

A subtle point that trips people up: at this moment the client has a confidential, integrity-protected channel to *somebody*, but it does not yet know *who*. The Diffie–Hellman exchange authenticates nothing; an active Eve could have run her own DH with the client. Confidentiality without authentication is a trap — it gives a secure channel to the attacker. Authentication is the job of the next two messages (Certificate, CertificateVerify). Until those verify, the client must treat the peer as unproven.

25.5 Step 3: the key schedule

We have a shared secret Z. We need many keys: one to encrypt the server's handshake messages, one for the client's, one for application data in each direction, plus keys to compute the Finished MACs. TLS 1.3 manufactures all of them from Z with a single tool: **HKDF**, the HMAC-based key-derivation function from Chapter 23 (built on HMAC from Chapter 15).

HKDF has two halves. *Extract* takes a salt and some input keying material and squeezes out a uniform pseudorandom key:

$$\text{PRK} = \text{HKDF-Extract}(\text{salt}, \text{IKM}) = \text{HMAC}(\text{salt}, \text{IKM}).$$

Expand stretches that key into as many labelled output keys as you want, binding each to a label and to the handshake transcript:

$$\text{HKDF-Expand-Label}(\text{secret}, \text{label}, \text{context}, L).$$

The label (a short ASCII string like `"c hs traffic"`) guarantees that two keys derived for different purposes are independent even though they came from the same secret. The context is usually a hash of the handshake transcript so far, so the keys are cryptographically bound to *exactly* the messages that have been exchanged.

TLS 1.3 arranges these derivations in a **schedule**: a chain of Extract and Expand steps, each feeding the next (Figure 25.2).

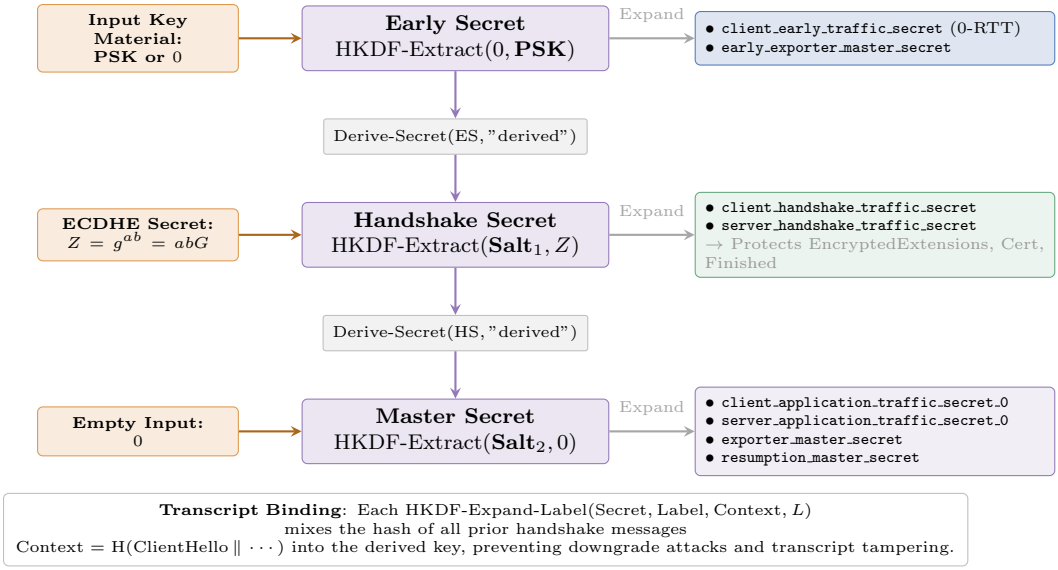


Figure 25.2: The TLS 1.3 HKDF key derivation schedule. Entropy from the optional pre-shared key (PSK) and ephemeral Diffie–Hellman exchange (Z) is absorbed in stages via `HKDF-Extract`. Intermediate secrets are expanded into cryptographically isolated traffic keys for the handshake, application data, and future session resumption.

The **Early Secret** is extracted from a pre-shared key (PSK) if one exists, or from a string of zeros if not. The **Handshake Secret** mixes in the actual Diffie–Hellman output Z; from it the two handshake traffic keys are expanded, and the server starts using its key right away. After the handshake completes, the **Master Secret** (extracted with the DH input already consumed, so the input is zeros) yields the application traffic keys that protect real data, plus a resumption secret for next time.

Why it works

Why a chain instead of one Expand from Z? There are three reasons, each learned from older protocols. **(1) Domain separation:** distinct labels make the handshake keys provably independent from the application keys, so compromising one phase does not leak another. **(2) Transcript binding:** because each key is expanded with the hash of the messages so far, an attacker who tampers with any handshake message produces different keys on the two sides, and the Finished check (below) fails. **(3) PSK and DH composition:** the Extract steps let TLS fold in a pre-shared key *and* fresh DH cleanly, so resumed connections still get forward secrecy from the new g^{ab}. TLS 1.2 derived everything in one ad-hoc PRF call and paid for it with attacks like Triple Handshake that this structure closes.

Implement it

A toy of the TLS 1.3 key schedule using Python's `hmac` and `hashlib`. It mirrors the real structure but is radically simplified — no real labels, no transcript encoding, fixed lengths. **Learning only: never use this to secure anything.**

```
1 import hmac, hashlib
```

```

def hkdf_extract(salt, ikm):
    return hmac.new(salt, ikm, hashlib.sha256).digest()

def hkdf_expand_label(secret, label, context, length=32):
    # Real TLS encodes (length, "tls13 "+label, context) into the info field.
    info = b"tls13 " + label + b"|" + context
    out, t, i = b"", b"", 1
    while len(out) < length:
        t = hmac.new(secret, t + info + bytes([i]), hashlib.sha256).digest()
        out += t
        i += 1
    return out[:length]

def derive_secret(secret, label, transcript_hash):
    return hkdf_expand_label(secret, label, transcript_hash)

zeros     = b"\x00" * 32
dh_shared = b"...the g^{ab} bytes go here..."             # from the (EC)DHE step
th = hashlib.sha256(b"ClientHello||ServerHello").digest() # toy transcript hash

early_secret     = hkdf_extract(b"\x00", zeros)           # no PSK -> zeros
handshake_secret = hkdf_extract(
    derive_secret(early_secret, b"derived", b""), dh_shared)
c_hs_traffic     = derive_secret(handshake_secret, b"c hs traffic", th)
s_hs_traffic     = derive_secret(handshake_secret, b"s hs traffic", th)
master_secret    = hkdf_extract(
    derive_secret(handshake_secret, b"derived", b""), zeros)
# Application keys come from master_secret once the transcript is final.
print("client handshake key:", c_hs_traffic.hex())
```

25.6 Step 4: authentication (Certificate and CertificateVerify)

Now the server proves it is who the client wanted. It sends, all under the handshake key:

- `EncryptedExtensions` — assorted negotiated parameters that do not need to be public.
- `Certificate` — the server's X.509 certificate chain: its own certificate (binding its public key to its domain name) followed by any intermediate CA certificates up toward a root the client trusts. This is the PKI machinery from Chapter 24.
- `CertificateVerify` — the authenticating step. The server takes the entire handshake transcript hashed so far, signs it with the *private key corresponding to the public key in its certificate*, and sends the signature.

The CertificateVerify message binds the server's identity to this connection. The certificate alone proves nothing: anyone can copy Bob's certificate, since it is public. What an attacker cannot do is produce a valid signature over *this particular transcript* — which includes the client's fresh random and the ephemeral key shares — without Bob's private key. The signature is fresh, bound to this handshake, and unforgeable.

Why it works

Why signing the transcript authenticates the key exchange. The transcript the server signs contains the ServerHello, hence the server's key share B, hence (together with the client's A) the shared secret Z. So the signature simultaneously says "I am the holder of the certified private key" *and* "the key share you are about to derive your keys from is mine, not an attacker's." This is the textbook *signed Diffie–Hellman* pattern from Chapter 23: ephemeral DH gives forward secrecy, a signature over the DH values gives authentication, and binding them in one transcript stops a man-in-the-middle from splicing her own key share in. If active Eve had injected her share, the transcript Bob signs would not match the one Alice computed, and verification — or the Finished check — fails.

The client verifies the certificate chain (validity dates, name match against the SNI it requested, a path to a trusted root, revocation status), then verifies the CertificateVerify signature against the public key in that certificate. Only if both pass does the client believe it is talking to the real server.

25.7 Step 5: Finished (the MAC that seals the handshake)

The server sends one more handshake message: `Finished`. It is a MAC — specifically HMAC under a key derived from the handshake secret — computed over the *entire transcript up to this point*. The client, holding the same key and transcript, recomputes the MAC and checks it. Then the client sends its own `Finished`, a MAC over the transcript including the server's Finished, and the server checks that.

These two MACs protect the integrity of the entire handshake, for a reason worth spelling out.

Why it works

Why Finished defeats tampering and downgrades. Every handshake message — the cipher suite list, the supported groups, the key shares, the certificate — feeds into the transcript hash. The Finished MAC is computed over that hash with a key that an attacker cannot derive (it depends on g^{ab}). So if Eve altered *any* earlier message — swapped a strong cipher suite for a weak one, stripped a group, flipped a bit in a key share — the two sides' transcripts diverge, their Finished MACs do not match, and the handshake aborts. In particular this is TLS 1.3's **downgrade protection**: even though the negotiation happens in the clear, its result is authenticated after the fact. Eve cannot trick Alice and Bob into using weaker crypto than they both support, because the moment she edits the negotiation she breaks the Finished check. The lesson was learned the hard way: TLS 1.2's negotiation was not fully authenticated, and attacks like FREAK and Logjam forced servers down to deliberately weakened "export-grade" keys precisely by tampering with the unprotected handshake.

Focus

Putting the pieces together. Each message contributes specific properties:

- ClientHello / ServerHello key shares ⇒ a *shared secret* (confidentiality, forward secrecy).
- Certificate + CertificateVerify ⇒ the server is *who it claims* (authentication).
- Finished (both) ⇒ *nobody tampered* with the negotiation (integrity, downgrade protec-

tion).

Confidentiality, authentication, integrity — the three promises of the first section, each pinned to a specific message, each built from a primitive you already understand.

25.8 The record protocol and AEAD

With the handshake done, the connection switches to the **record protocol**. Application data is split into records (up to 2^{14} bytes of plaintext each), and each record is sealed with an **AEAD** cipher — authenticated encryption with associated data — from Chapter 16. TLS 1.3 permits only AEAD suites: AES-GCM, AES-CCM, or ChaCha20-Poly1305. No more encrypt-then-separately-MAC bolt-ons, no unauthenticated CBC; the cipher provides confidentiality and integrity in one operation.

An AEAD seal takes four inputs: the traffic key k, a nonce, the plaintext, and some associated data that is authenticated but not encrypted. It returns ciphertext plus an authentication tag:

$$\text{Seal}_k(\text{nonce}, \text{AD}, \text{plaintext}) \to (\text{ciphertext}, \text{tag}).$$

Decryption recomputes the tag and refuses to output plaintext if it does not match. A single flipped bit anywhere in the record makes the tag check fail and the connection tears down. That is the integrity guarantee, per record.

25.8.1 Nonces and the sequence number

GCM and ChaCha20-Poly1305 are catastrophically insecure if you ever reuse a (key, nonce) pair — repeat a nonce under AES-GCM and you leak the XOR of two plaintexts and, worse, the authentication subkey, letting an attacker forge. TLS 1.3 avoids the problem without transmitting nonces at all. Instead each side keeps a 64-bit **sequence number** that starts at zero and increments by one per record. The per-record nonce is that counter XORed into a fixed random `iv` derived from the traffic secret in the key schedule.

Focus

Nonce = static_iv ⊕ (sequence number). Because the sequence number never repeats within a connection and the `iv` is unique per direction per key, no (key, nonce) pair ever recurs. The counter is implicit — never sent — so it also costs zero bytes on the wire. Tiny example: with the last two bytes of the `static_iv` equal to `A5C3` and record numbers $0, 1, 2$, the nonces end in `A5C3`, `A5C2`, `A5C1` — distinct by construction. When the sequence number would overflow, TLS forces a key update (re-keying) rather than risk a repeat.

25.8.2 What the record header authenticates

In TLS 1.3 the record's outer header is tiny: a content-type byte (set to `application_data` even for handshake records, to hide what is inside), a legacy version field, and the length. The real content type is appended to the plaintext *before* encryption and recovered after decryption, so an eavesdropper cannot tell handshake records from data records from alerts. The outer header is fed

in as the AEAD associated data, so its length field is authenticated even though it travels in the clear: an attacker cannot lie about the record's length without breaking the tag (Figure 25.3).

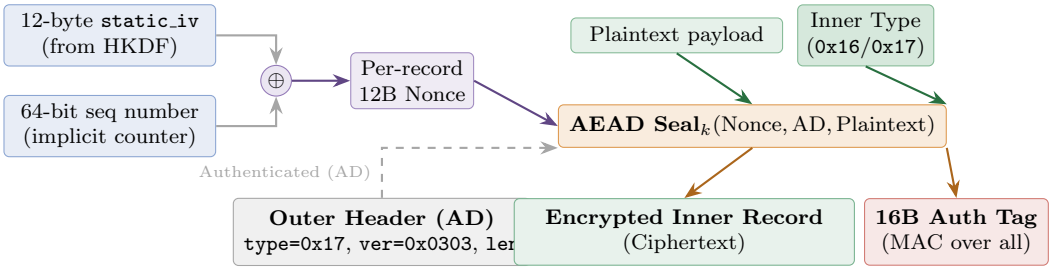


Figure 25.3: TLS 1.3 record-layer AEAD sealing. The 64-bit sequence number is XORed with the static IV to form a unique 12-byte nonce without transmitting a counter on the wire. The true content type is hidden inside the encrypted payload, while the cleartext outer header is authenticated as Associated Data (AD).

In the real world

This is the traffic under your padlock. When you load a page over HTTPS, the TLS handshake runs once (often resumed in well under a millisecond on repeat visits), and then every HTTP request and response — including HTTP/2 and HTTP/3 framing — rides inside AEAD-sealed TLS records. On a modern CPU with AES-NI, AES-GCM runs at gigabytes per second, so the record layer is essentially free; the handshake's public-key math is the only part you can feel, and even that is one round trip. ChaCha20-Poly1305 exists for the same reason but on phones and embedded chips without AES hardware, where its software speed and constant-time design (no secret-dependent table lookups) win. Google deployed it across Android and Chrome for exactly that mobile-battery, side-channel-safety combination.

25.9 Forward secrecy in TLS 1.3

We have mentioned forward secrecy several times; here is the precise statement. A protocol has forward secrecy if compromise of long-term secrets in the future does not compromise past session keys. In TLS 1.3 the long-term secret is the server's certificate private key. The session keys come from the *ephemeral* Diffie–Hellman secret $Z = g^{ab}$, where a and b are random, used once, and erased when the connection closes.

Intuition

Suppose Eve records gigabytes of your encrypted TLS traffic today and files it away. Five years from now she steals Bob's server, including its certificate private key. Can she decrypt the old recordings? **No.** The certificate key was used only to *sign* the handshake transcript, never to derive any encryption key. The encryption keys came from g^{ab}, and a and b were thrown away the moment those connections ended. To get Z she would need a or b, and the only thing on the wire was g^a and g^b — recovering the exponents is the discrete-log problem she cannot solve. The stolen signing key lets her impersonate Bob *going forward*, but it cannot retroactively open a single old session.

History

This is the property that static-RSA TLS lacked. TLS grew out of Netscape's Secure Sockets Layer (SSL 2.0 in 1995 and SSL 3.0 in 1996, designed by Paul Kocher, Taher ElGamal, and colleagues) and was standardized by the IETF as TLS 1.0 in 1999. In TLS up to 1.2, a common mode was *RSA key transport*: the client picked the session secret and encrypted it under the server's long-term RSA public key from the certificate, making the session key recoverable by anyone holding that RSA private key — *forever*. The 2013 Snowden disclosures made the risk concrete: an adversary who recorded traffic in bulk and later obtained a server's private key (by subpoena, theft, or coercion) could decrypt years of archived sessions retroactively. Under editor Eric Rescorla, TLS 1.3 (2018, RFC 8446) finished the push toward ephemeral (EC)DHE by *removing static RSA key transport from the protocol entirely*, over objections from enterprises whose monitoring middleboxes relied on passive decryption with a copy of the server's key. In TLS 1.3, forward secrecy is mandatory rather than an option you might forget to enable.

25.10 Resumption and 0-RTT (and its one sharp edge)

Re-running a full handshake (with its signature and certificate verification) for every connection to a site you just visited is wasteful. TLS 1.3 lets the server issue a **session ticket** after a handshake: a pre-shared key (PSK) the client can present next time to skip the public-key authentication. A resumed handshake is faster and still mixes in fresh DH for forward secrecy (the Handshake Secret extracts both the PSK and the new g^{ab}).

TLS 1.3 also offers **0-RTT**: using a resumption PSK, the client can send encrypted application data *in its very first flight*, before any round trip, shaving the last latency off a repeat visit. It is a genuine speedup, but it comes with a serious caveat.

Common trap

0-RTT data has no replay protection. The early data is encrypted under a key derived from the PSK alone, with no fresh server contribution yet. Eve cannot read it, but she *can* record the 0-RTT flight and replay it later — the server has no per-connection nonce from itself to reject the duplicate. If that early request is a non-idempotent action ("transfer $100," "post this comment"), a replay executes it twice. The rule: *only ever put idempotent, safe requests in 0-RTT* (a GET, a cacheable read), never a state-changing one. Application servers must enforce this; many simply forbid 0-RTT for anything but GETs. It is a textbook case of a performance feature trading away a security property, and you have to know which.

Figure 25.4 diagrams why this replay vulnerability occurs in practice.

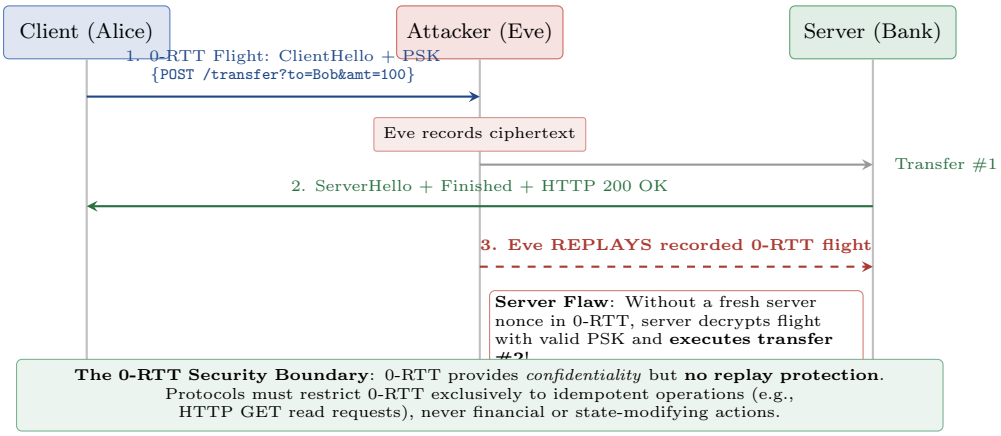


Figure 25.4: The 0-RTT early data replay vulnerability in TLS 1.3. Because 0-RTT data is encrypted before receiving a fresh random nonce from the server, an active eavesdropper can capture and retransmit the flight. Unless application servers enforce idempotency or anti-replay caches, state-altering requests are duplicated.

25.11 Famous TLS attacks

The design of TLS 1.3 was shaped directly by two decades of attacks on its predecessors. Each of the following attacks broke a real deployment, and each maps to a concrete design decision in 1.3 that eliminated the vulnerability class. Studying them shows where protocol abstractions tend to leak.

Attack

BEAST (2011) — a predictable IV in CBC mode. TLS 1.0 used CBC mode with a fatal shortcut: the IV for each record was the *last ciphertext block of the previous record*, so it was predictable to an attacker watching the wire. Recall CBC encrypts block i as $C_i = \mathrm{Enc}_k(P_i \oplus C_{i-1})$. If Eve can predict C_{i-1} *and* influence the plaintext P_i (she can, via malicious JavaScript making the browser send chosen data alongside a secret cookie), she can test guesses of a secret byte: choose P_i so that $P_i \oplus C_{i-1}$ equals a value she wants to probe, and watch whether the resulting ciphertext block collides with one she has seen for a known plaintext. Byte by byte she recovers the cookie. *Fix:* TLS 1.1 made the IV random and unpredictable per record; TLS 1.3 abandons CBC entirely for AEAD, where this whole class of chosen-plaintext games against the mode does not exist.

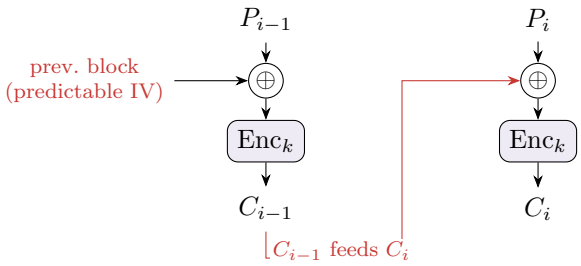


Figure 25.5: CBC chaining as BEAST exploits it: each ciphertext block is the IV for the next, so an attacker who has seen C_{i-1} knows the value XORed into the chosen P_i and can probe a secret byte through ciphertext collisions.

Concretely (Figure 25.5): because Eve knows C_{i-1} and chooses P_i, she controls $P_i \oplus C_{i-1}$ exactly. A random IV unknown to her before she commits P_i destroys that control, which is why TLS 1.1 patched it and TLS 1.3 dropped the mode.

Attack

POODLE (2014) — padding you cannot trust. SSL 3.0's CBC padding was *unauthenticated* and its contents unspecified except for the last byte. Under MAC-then-encrypt, the receiver decrypts, strips padding, then checks the MAC — so the padding is processed before authentication. Eve, as an active man-in-the-middle running a padding-oracle attack, replaces the last ciphertext block with an earlier block she wants to decrypt; with probability 1/256 the mangled padding happens to be accepted, and that acceptance leaks one plaintext byte. By forcing many connections (again via injected JavaScript) she peels off a secret byte at a time. The deeper flaw was **MAC-then-encrypt**: integrity was checked too late, after the attacker-controlled padding had already influenced behaviour. *Fix:* stop using SSL 3.0 (POODLE is also why downgrade protection matters — the attack often began by forcing a downgrade to SSL 3.0), and structurally, move to AEAD and to encrypt-then-MAC ordering, both of which TLS 1.3 mandates. The padding-oracle lineage runs through Vaudenay's 2002 attack, Lucky Thirteen's timing variant, and POODLE — all variants of the same flaw.

Attack

Heartbleed (2014) — not a protocol flaw at all. Heartbleed was a buffer over-read in OpenSSL's implementation of the TLS heartbeat extension. The heartbeat request said "echo back these N bytes," and the code trusted N without checking it against the actual payload length. An attacker sent a heartbeat claiming a 1-byte payload but a length of 64KB; the server dutifully copied 64KB of *adjacent process memory* into the reply — which could contain other users' requests, session cookies, or, devastatingly, the server's private key. No cryptography was broken; the math was perfect. A single missing bounds check in C exposed the long-term private keys of a large fraction of the web overnight, forcing mass certificate revocation and re-issuance. *Lesson:* the protocol is only as strong as the code that implements it. This is the main motivation for Chapters 37–39 — memory safety, bounds checks, and minimizing the attack surface are cryptographic concerns even when the cryptography is flawless. (It is also a sharp reminder of why forward secrecy matters: servers that used ephemeral key exchange at least did not retroactively expose past sessions when their keys leaked.)

Attack

The downgrade family — FREAK and Logjam (2015). For 1990s export regulations, TLS retained support for deliberately weak "export-grade" RSA (512-bit) and DH (512-bit) parameters. FREAK and Logjam were active attacks that tampered with the (unauthenticated) negotiation in TLS 1.2 to coax a client and server into using these weak parameters, after which the attacker factored the 512-bit RSA modulus or solved the 512-bit discrete log offline — feasible with modest compute, especially since many servers shared the *same* DH group, so one big precomputation cracked many sites (Logjam). *Fix:* TLS 1.3 removes export ciphers and all the legacy weak groups, fixes a short list of strong groups, and — the structural cure — authenticates the entire negotiation via the Finished MAC, so an attacker who edits the cipher/group choice is caught.

Focus

The pattern across all of them. Predictable IVs (BEAST), authenticate-too-late padding (POODLE, Lucky Thirteen), unauthenticated negotiation (FREAK, Logjam), and implementation memory bugs (Heartbleed). TLS 1.3's answers: random per-record nonces from a counter, AEAD only (encrypt-and-authenticate together), a fully transcript-bound Finished MAC, and a smaller, simpler surface with fewer options to get wrong. Much of the 1.3 specification is a direct response to one of these.

25.12 How the pieces fit together

Connection

TLS uses nearly every primitive in this book. The (EC)DHE key exchange is Diffie–Hellman over the elliptic curves of Chapter 21, giving the shared secret and forward secrecy. The key schedule is HKDF (Chapter 23), built from the HMAC of Chapter 15. The server's CertificateVerify is a digital signature (ECDSA or RSA-PSS or Ed25519) from Chapter 22, verified against a certificate chain validated by the PKI of Chapter 24. The Finished messages are again HMAC. The record layer is AEAD — AES-GCM or ChaCha20-Poly1305 — from Chapter 16, with AES itself from Chapter 12. The random nonces and ephemeral secrets come from the CSPRNG of Chapter 17. Every one of them is required, which is why a vulnerability in any single primitive (a weak RNG, a broken hash, a signature forgery) becomes a TLS break.

Reading note

If you want to watch a real handshake, run `openssl s_client -connect example.com:443 -tls1_3` and read the messages, or capture port 443 in Wireshark and expand the TLS layer. You will see ClientHello with its key share and SNI in the clear, ServerHello, then a wall of encrypted handshake records, then application data. Everything in this chapter is sitting right there on the wire.

TLS secures a byte stream once TCP is already moving packets between endpoints, but what protects the local radio hop before a single TCP packet reaches the router? In Chapter 26, we drop down to

the link layer to see how WEP, WPA2, and WPA3 encrypt and authenticate WiFi frames broadcast through open air.

25.13 Exercises

Exercise 25.1. Trace a tiny handshake's key exchange. Using $p = 23$, $g = 5$, let the client pick $a = 4$ and the server pick $b = 9$. Compute the public shares A and B and the shared secret $Z = g^{ab} \bmod 23$ two ways (as B^a and as A^b), checking they agree. Then explain precisely what Eve sees on the wire and what hard problem she would have to solve to recover Z.

Exercise 25.2. Why hash the DH output. The shared secret $Z = g^{ab}$ is an element of $(\mathbb{Z}/p\mathbb{Z})^*$, not a uniform bit string. Give one concrete reason raw Z is unsuitable as an AES key, and explain what the HKDF-Extract step fixes. (Hint: think about the distribution of Z and about which subgroup of $(\mathbb{Z}/p\mathbb{Z})^*$ it can land in.)

Exercise 25.3. The role of CertificateVerify. Suppose a server sent its Certificate but *not* the CertificateVerify message. Describe a concrete man-in-the-middle attack an active Eve could mount, and pinpoint which security property (confidentiality, integrity, or authentication) collapses. Then explain why the certificate by itself is not enough.

Exercise 25.4. Finished as downgrade defense. An attacker edits the ClientHello to remove all strong cipher suites, leaving only a weak one, hoping the server will pick it. Trace through the handshake and identify the exact step at which this tampering is detected. Why does it matter that the Finished MAC is computed over the *entire* transcript rather than just the final messages?

Exercise 25.5. Forward secrecy by contrast. In old RSA-key-transport TLS, the client encrypts the session secret under the server's long-term RSA public key. Show that an adversary who records the traffic and *later* obtains the server's RSA private key can decrypt the session. Then explain, step by step, why the same adversary fails against ephemeral ECDHE even after stealing the server's signing key.

Exercise 25.6. Nonce arithmetic. A TLS 1.3 connection uses a 12-byte `static_iv` = `0x01020304 05060708090A0B0C`. Give the AEAD nonces for record sequence numbers 0, 1, and 258 (the nonce is `static_iv` $\oplus$seq, with the sequence number right-aligned in the 12 bytes). Argue that within a single connection no (key, nonce) pair can ever repeat, and explain why that is essential for AES-GCM specifically.

Exercise 25.7. Break BEAST on paper. In CBC mode $C_i = \mathrm{Enc}_k(P_i \oplus C_{i-1})$. Assume Eve knows C_{i-1} (the previous record's last block, used as the next IV in TLS 1.0) and can choose the next plaintext block P_i. She wants to test whether an unknown secret block S equals a guess G. Show how she picks P_i so that a ciphertext-block collision reveals the answer, and state precisely what about TLS 1.0's IV choice made this possible. What does TLS 1.3 do that removes the attack entirely?

Exercise 25.8. 0-RTT replay. A client uses 0-RTT to send the HTTP request `POST /transfer?amount=100&to=eve` as early data. Describe how an active attacker exploits the lack of replay protection, what damage results, and the general rule a server should follow about which requests may be accepted as 0-RTT data. Contrast this with why the main 1-RTT data is not replayable.

Exercise 25.9. Heartbleed was not crypto. Explain why Heartbleed could leak a server's private key even though every cryptographic algorithm in TLS was implemented correctly. Identify the class of bug, the role of the unchecked length field, and the general principle it teaches about the relationship between protocol security and implementation security.

Exercise 25.10. Trace the whole machine. For a single TLS 1.3 connection, list every cryptographic primitive used and the exact purpose each serves: the key exchange, the KDF, the signature, the certificate validation, the record cipher, and the randomness source. For each, name one earlier chapter it came from and state what breaks in the handshake if that one primitive is broken.

Chapter 26

Securing WiFi: WPA2 and WPA3

In Chapter 25, we saw how TLS secures a byte stream once TCP already connects a client to a server. Before a single TCP packet reaches the router, however, your device has to broadcast its frames across open air. Every time a laptop or phone wakes, sees a familiar network name, and – without anyone typing a password – rejoins it and starts streaming bytes, it runs a link-layer handshake first. That routine reconnection uses one of the most widely deployed cryptographic protocols on the planet, run billions of times a day, by coffee shops and airports and the cheap router behind your television. It is also a protocol that has been broken, in public and embarrassingly, more than once. WEP was badly broken. WPA2 stood for thirteen years until KRACK forced a global patch. WPA3 is the current standard, and it has had its own vulnerabilities.

This chapter is about the radio link: the single hop between your device and the access point. That hop is special. Unlike a TCP connection across the Internet, the WiFi link is broadcast. Every frame you send is, physically, audible to anyone within range who cares to listen, whether or not they are "on" the network. That changes the threat model in concrete ways, and it is why WiFi encryption matters even when the site you are visiting already uses TLS.

We will build the WPA2 four-way handshake from the ground up – where the keys come from, what each of the four messages does, and why the design is shaped the way it is. Then we will break it: WEP first (a clinic in how *not* to use a stream cipher), then KRACK against WPA2 (a clinic in how a single reused nonce can unravel a careful protocol). Finally WPA3 and its Dragonfly/SAE handshake, which replaces the weak link – the password – with a password-authenticated key exchange. Throughout, we keep asking the practical question that actually decides whether you should worry: what can an attacker on the same network, or just within radio range, really do?

26.1 The shape of the problem

Intuition

Alice has a laptop. Bob runs the access point (AP) – the box that bridges the WiFi radio to the wired Internet. Eve is sitting two tables over with a $30 USB WiFi adapter in monitor mode, recording every frame in the air. The network is named `CoffeeHouse` and is protected by a password that Alice typed in once, months ago.

Three things must happen for the link to be secure against Eve:

1. **Confidentiality.** Eve must not be able to read the contents of Alice's frames.
2. **Integrity.** Eve must not be able to modify a frame – flip a bit in a packet, redirect a DNS lookup – without detection.
3. **Authentication.** Alice must be talking to the real Bob, not to Eve impersonating the AP; and Bob must be talking to a device that knows the password.

The list does *not* include hiding *that* Alice is using the network, or hiding which AP she joins, or hiding how much she sends. WiFi headers are sent in the clear. The link layer protects payloads, not metadata.

The shared secret everyone starts from is the network password, which in WPA2 "Personal" mode is called the *passphrase* – the thing on the sticker under the router or written on the cafe chalkboard. In enterprise mode (WPA2- Enterprise, used on corporate and university networks) the password is replaced by an 802.1X/EAP exchange against a RADIUS server, and each user gets a fresh key from that exchange. We focus on Personal mode, because it is where the interesting cryptography lives and where the famous attacks land. The enterprise key hierarchy is the same below the top; only the source of the master key differs.

Focus

WiFi security uses many overlapping names, so here they are in one place.

- **WEP** (1997): the original, fatally broken scheme. RC4 plus a 24-bit IV.
- **WPA** (2003): a stopgap. Same RC4 hardware, but wrapped in TKIP to fix WEP's worst sins. Also broken now; treat as dead.
- **WPA2** (2004): the long-lived standard. AES in CCM mode ("CCMP") for confidentiality and integrity, with the four-way handshake to derive keys. This is what "WiFi encryption" meant for over a decade.
- **WPA3** (2018): replaces the password-based handshake with SAE ("Dragonfly"), a password-authenticated key exchange, and adds forward secrecy. CCMP becomes GCMP-256 in the high-security tier.
- **IEEE 802.11i**: the standards document that defines WPA2's cryptography. "RSN" (Robust Security Network) is the same thing in the spec's language.

26.2 WEP, and how to break a stream cipher by reusing it

Before WPA2 there was WEP, Wired Equivalent Privacy, and it is worth dissecting because every mistake in it is a textbook lesson. WEP took RC4 – a fine stream cipher for its day – and used it in just about the worst way one can.

Recall how a stream cipher works. It stretches a key into a long pseudorandom keystream Z, and encrypts by XOR: $c = m \oplus Z$. The basic rule of stream ciphers is: *never reuse a keystream.* If two messages m_1 and m_2 are encrypted under the same keystream Z, then

$$c_1 \oplus c_2 \;=\; (m_1 \oplus Z) \oplus (m_2 \oplus Z) \;=\; m_1 \oplus m_2,$$

and the keystream cancels. The attacker now has the XOR of two plaintexts, which – with English text, known headers, or any structure – is often enough to recover both.

So the entire security of a stream-cipher system reduces to: *is the keystream ever reused?* WEP guaranteed that it would be.

Why it works

WEP's per-frame key was $K = \text{IV} \,\|\, k$, where k is the long-term shared WEP key (the password, essentially) and IV is a 24-bit initialization vector sent in the clear with each frame. RC4 was keyed with K to produce the keystream for that frame.
The IV is 24 bits, so there are only $2^{24} \approx 16.7$ million possible values. On a busy network sending, say, 1000 frames per second, you exhaust the entire IV space in about $2^{24}/1000 \approx 16{,}777$ seconds, under five hours. After that, IVs *must* repeat, and every repeat is a keystream reuse, because the long-term k never changes. By the birthday bound (see the companion probability volume), collisions start appearing far sooner – after roughly $\sqrt{2^{24}} = 2^{12} = 4096$ frames you expect your first repeated IV, which is a few seconds of traffic.

Attack

WEP is broken three ways over, and any one would be fatal.
Keystream reuse. As above, repeated IVs give $m_1 \oplus m_2$. Many WiFi frames have predictable structure (ARP packets are a fixed size with known fields; IP headers are guessable), so the attacker often knows or can guess m_1, recovering m_2 outright – and recovering the keystream for that IV, which then decrypts every future frame using the same IV.
No real integrity. WEP "protects" integrity with a CRC-32 checksum, which is *linear*: $\text{CRC}(x \oplus \delta) = \text{CRC}(x) \oplus \text{CRC}(\delta)$. Because encryption is also linear (XOR), an attacker can flip any bit of the ciphertext and compute exactly how to fix the checksum so the frame still verifies. CRC is an error-detecting code designed for random errors, not a MAC, so it gives no protection against a deliberate attacker.
The RC4 key-schedule weakness (FMS). In 2001, Fluhrer, Mantin, and Shamir showed that RC4's key schedule leaks information about the key when the first few keystream bytes are known and the key has a particular structure – exactly the structure WEP's $\text{IV} \,\|\, k$ provides, with the attacker-visible IV as the variable prefix. Each captured frame leaks a tiny bias about k. Collect enough "weak IV" frames and you solve for k directly. Later tools (KoreK, then the PTW attack of 2007) drove the requirement down to tens of thousands of frames – a minute or two of traffic, less if you inject packets to force more. The recovered key is the *actual* WEP key, decrypting everything past and future.

RC4's biases were a real flaw, but WEP would have fallen to keystream reuse and the CRC malleability even with a perfect cipher. A cipher is only as good as the construction around it. WEP got the IV size wrong, got the integrity primitive wrong (a checksum where a MAC was needed), and rekeyed never. WPA2 fixes all three: a long never-repeating counter, a real MAC, and fresh session keys per association.

History

WEP shipped in 1997 inside the original 802.11 standard, designed by a committee optimizing for cheap hardware and an export regime that capped key sizes. The 24- bit IV and 40-bit

key were partly artifacts of 1990s U.S. crypto export limits. By 2001 the FMS paper was public; by the mid-2000s, point-and-click tools (`aircrack-ng`) cracked WEP in minutes. The 2007 breach of TJX (millions of credit cards) was traced to a store WiFi network still running WEP. The standard formally deprecated it, and modern devices have removed WEP support entirely. If you ever see WEP offered, treat the network as plaintext.

26.3 WPA2: the key hierarchy

WPA2 starts from the same place – a shared passphrase – but never uses it directly to encrypt anything. Instead it derives a hierarchy of keys, with the passphrase at the root and short-lived per-frame protection at the leaves. The guiding principle: the long-term secret should touch the air as little as possible, and every session should use independent keys so that breaking one tells you nothing about another.

26.3.1 From passphrase to PMK

The passphrase is first stretched into a 256-bit *Pairwise Master Key* (PMK). In WPA2-Personal the stretching function is PBKDF2 (Password-Based Key Derivation Function 2) with HMAC-SHA1, 4096 iterations, salted by the network name (SSID):

$$\text{PMK} \;=\; \text{PBKDF2}\big(\text{passphrase},\ \text{SSID},\ 4096,\ 256\ \text{bits}\big).$$

Why it works

Two design choices in that line matter.
The **4096 iterations** make each password guess expensive. An attacker who captures a handshake and wants to brute-force the passphrase must run PBKDF2 – 4096 HMAC-SHA1 calls – per candidate. Without it, a GPU could test billions of guesses per second; with it, the rate drops by three to four orders of magnitude. This is the same key-stretching idea you saw for password hashing. It does not make a weak passphrase safe, only slower to crack.
The **SSID as salt** means the same passphrase on two networks with different names yields different PMKs. This blocks precomputation across networks – almost. The catch is that the salt is the *network name*, not a random value, and popular names (`linksys`, `xfinitywifi`, the default names of common routers) are predictable. Attackers precompute giant tables ("rainbow tables") of PMKs for common SSID-plus-passphrase pairs. If your SSID is a default and your passphrase is in a wordlist, the PMK is effectively already computed.

Connection

PBKDF2's cost is exactly the modular-exponentiation-versus-hashing tradeoff the algorithms volume analyzes for password storage: you deliberately pick a slow function so the legitimate party pays once at login but the attacker pays per guess. The birthday-bound reasoning behind WEP's IV collisions, and behind the nonce-reuse risks we will meet in GCMP, is the same $\sqrt{N}$ collision law derived in the probability volume's chapter on the birthday problem.

26.3.2 From PMK to PTK: the four-way handshake

The PMK is a long-term per-network secret – everyone who knows the passphrase can compute the same PMK. (Note that in WPA2-Personal, your neighbor on the same cafe network has the *same PMK as you*. We will return to what that means for attacks.) The PMK is never used to encrypt traffic directly. Instead, each time a device associates, a fresh *Pairwise Transient Key* (PTK) is derived for that session, mixing in random nonces from both sides so that every association gets independent keys.

The PTK is computed by a PRF (in WPA2, an HMAC-SHA1-based key-derivation function) from five inputs:

$$\mathrm{PTK} \;=\; \mathrm{PRF}\big(\mathrm{PMK};\; \mathrm{Anonce} \,\|\, \mathrm{Snonce} \,\|\, \mathrm{AA} \,\|\, \mathrm{SA}\big),$$

where Anonce is a random nonce chosen by the AP (the authenticator), Snonce is a random nonce chosen by the station (your device), AA is the AP's MAC address, and SA is the station's MAC address. The two MAC addresses are sorted into a fixed order, as are the two nonces, so both sides feed the PRF identical input and get identical output.

The PTK is then sliced into several sub-keys:

- **KCK** (Key Confirmation Key, 128 bits): used to MAC the handshake messages themselves, proving each side derived the same PTK.
- **KEK** (Key Encryption Key, 128 bits): used to encrypt the group key when it is handed to the station.
- **TK** (Temporal Key, 128 bits): the actual key that protects data frames with AES-CCMP.

Intuition

Why nonces from *both* sides? If only the AP contributed a nonce, then two sessions where the AP happened to reuse a nonce would derive the same PTK – and a device that always sends the same Snonce would help that along. Mixing two independent random values means the PTK is fresh as long as *either* side picks a fresh nonce, and the MAC addresses bind the key to this specific pair of devices. The handshake is a Diffie-Hellman in spirit – both parties contribute randomness – except the shared secret is the pre-existing PMK rather than a fresh DH exchange. That distinction causes WPA2's main weakness: there is no forward secrecy, because the PMK is fixed. Compromise the passphrase later and you can recompute every PTK from recorded nonces, decrypting traffic you captured years ago. WPA3 fixes exactly this.

Figure 26.1 details the four-way handshake message ladder. It is called "four-way" because four EAPOL (Extensible Authentication Protocol over LAN) frames cross the link.

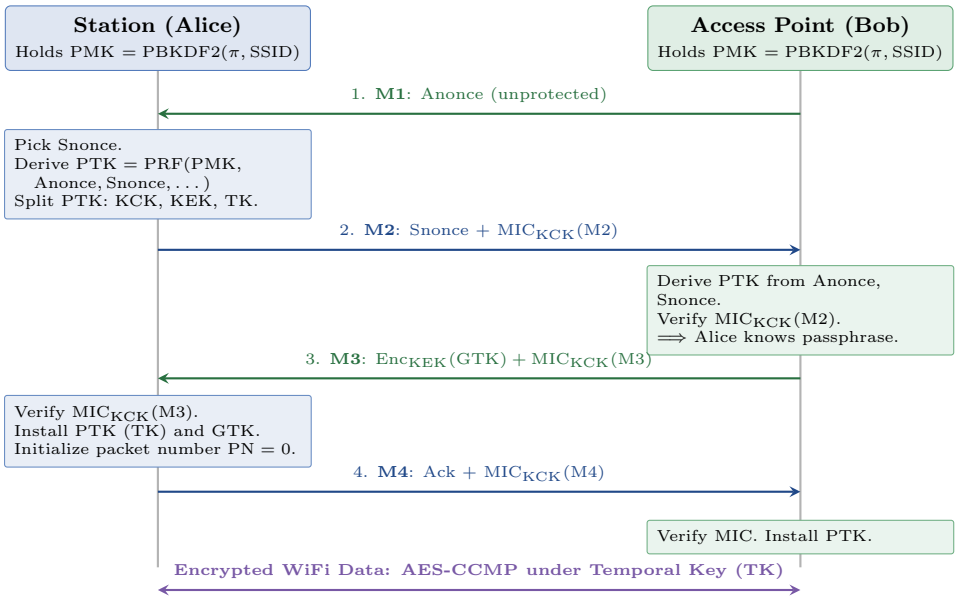


Figure 26.1: The WPA2 four-way handshake. Alice and Bob mutually authenticate their possession of the Pairwise Master Key (PMK) by exchanging nonces, deriving the Pairwise Temporal Key (PTK), and verifying Message Integrity Codes (MICs) before installing the data encryption key (TK).

M1 The AP sends its random **Anonce** to the station. This message is unauthenticated – the station cannot yet tell a real AP from a forgery, because no key exists yet. (Remember this for KRACK.)

M2 The station picks its **Snonce**, now has both nonces and both MACs, and computes the PTK. It sends Snonce back, with a **MIC** (Message Integrity Code) computed under the KCK. The AP can now also compute the PTK; if the MIC verifies, the AP knows the station derived the same PTK, hence knows the PMK, hence knows the passphrase.

M3 The AP sends the **GTK** (Group Temporal Key, the key for broadcast/multicast frames), encrypted under the KEK, with its own MIC under the KCK. The station verifies the MIC: now *it* knows the AP knew the PMK. Mutual authentication is complete. The station installs the PTK for use.

M4 The station sends an acknowledgment (with a MIC). On receipt the AP installs the PTK. Both sides switch to encrypted data frames.

Focus

The four-way handshake authenticates by *proof of knowledge of the PMK*, never by sending the PMK or passphrase. Each side proves it can compute the correct MIC under a key (KCK) that only a holder of the PMK could derive, given the public nonces. This is a challenge-response: the nonces are the challenges, the MICs are the responses. Nothing secret crosses the air – which is exactly why an eavesdropper who records the whole handshake *still* cannot read the traffic, unless they also know the passphrase.

26.3.3 Protecting data: AES-CCMP

Once the TK is installed, each data frame is protected with CCMP: AES in **Counter mode** for confidentiality, with **CBC-MAC** for integrity – the "CCM" authenticated-encryption mode. This is the piece WEP got so wrong, and WPA2 gets right.

Why it works

CCMP solves WEP's three failures cleanly:

- **No keystream reuse.** The AES-CTR counter includes a 48-bit *packet number* (PN) that increments on every frame and never repeats for the life of the TK. 2^{48} frames is astronomically more than any session will send, and a fresh TK each association resets it. The PN is also checked for monotonic increase, blocking replay.
- **Real integrity.** CBC-MAC under AES is a genuine MAC, not a linear checksum. An attacker who flips a ciphertext bit cannot repair the MIC without the key. Forgery probability is roughly 2^{-64} per try.
- **Authenticated headers.** The MIC covers not just the payload but the frame header fields that matter (addresses, QoS), so an attacker cannot redirect a frame by editing its header.

The PN fixes the problem with WEP's 24-bit IV: it is long enough never to repeat, included in the authenticated data, and checked for replay.

Connection

CCM is the counter-plus-CBC-MAC authenticated encryption mode studied in Chapter 16. Counter mode turns the block cipher AES into a stream cipher (so the no-reuse rule applies, hence the careful PN), while CBC-MAC supplies the integrity that CTR mode alone lacks. WPA3's high tier swaps CCM for GCM (GCMP-256), the same AES-GCM you meet in TLS 1.3 – faster, but with an even sharper nonce- reuse cliff, which is why the 256-bit version and careful PN handling matter.

26.4 A worked miniature handshake

Let us trace the key derivation with tiny stand-in numbers, so the data flow is concrete. Real WPA2 uses 256-bit PMKs and HMAC-SHA1; we will use a toy hash so you can follow the arithmetic. This is a *model*, not the real algorithm – the point is the shape of the dependency graph, not the bytes.

Example 26.1. Let our toy "PRF" be $\mathrm{prf}(K, x) = (K \cdot x + 7) \bmod 101$, working mod the prime 101. Suppose:

- Passphrase-derived PMK $= 42$ (in reality, 256 bits from PBKDF2).
- AP nonce Anonce $= 13$, station nonce Snonce $= 28$.
- We combine nonces by addition for the toy: $x = \text{Anonce} + \text{Snonce} = 41$.

Then the toy PTK is

$$\text{PTK} = \mathrm{prf}(42, 41) = (42 \cdot 41 + 7) \bmod 101 = (1722 + 7) \bmod 101 = 1729 \bmod 101.$$

Now $1729 = 17 \cdot 101 + 12$, so PTK = 12.

Slice it (toy-style) into a confirmation key and a data key: say KCK = PTK mod 10 = 2 and TK = $\lfloor \text{PTK}/10 \rfloor = 1$ together with the nonces. The station's M2 MIC is $\text{prf}(\text{KCK}, \text{Snonce}) = (2 \cdot 28 + 7) \bmod 101 = 63$. The AP recomputes the same PTK (it has PMK and both nonces), derives the same KCK = 2, recomputes $\text{prf}(2, 28) = 63$, and sees the MIC matches – so the station proved knowledge of the PMK without revealing it.

Trace what an eavesdropper sees: Anonce = 13, Snonce = 28, the MIC = 63, the MAC addresses. To forge M2 or read traffic, Eve needs the PTK, which needs the PMK = 42, which she does not have. Her only avenue is to guess the passphrase offline: pick a candidate, run PBKDF2 to a candidate PMK, derive a candidate PTK and KCK, and check whether $\text{prf}(\text{KCK}, 28)$ equals the captured MIC 63. That check is the entire WPA2-Personal offline attack, and the next section shows how it works in practice.

Common trap

Do not read Example 26.1 as the real PRF. Real WPA2 uses HMAC-SHA1 over a precisely ordered concatenation of sorted MAC addresses and sorted nonces, expanded to produce 384 or 512 bits of PTK. The toy linear function $Kx + 7$ is invertible and would let an attacker solve for K from one (input, output) pair – a catastrophe a real PRF prevents. The toy is for tracing data flow only.

26.5 The offline dictionary attack on WPA2-Personal

WPA2-Personal's authentication boils down to: "prove you know the PMK by producing the right MIC." Everything an attacker needs to *check* a passphrase guess is in the four-way handshake, which is sent in the clear. So WPA2-Personal is vulnerable to an *offline* dictionary attack, and there is no fixing it within the protocol – it is inherent to deriving keys from a low- entropy password without a PAKE. (WPA3 is precisely the fix.)

Attack

Capture-and-crack against WPA2-Personal.
Step 1: capture a handshake. Eve puts her adapter in monitor mode and waits for any device to associate, recording M1 through M4. She does not need to wait long – she can *force* it. A WiFi **deauthentication** frame is a management frame that, in plain WPA2, is unauthenticated: the AP can tell a client "you are disconnected," and clients obey. Eve spoofs a deauth from the AP's MAC to Alice. Alice drops off and immediately reconnects, performing a fresh four-way handshake that Eve records.
Step 2: crack offline. From the handshake Eve has Anonce, Snonce, both MACs, and the M2 MIC. For each candidate passphrase p in a wordlist:

$$\text{PMK} \leftarrow \text{PBKDF2}(p,\ \text{SSID},\ 4096), \quad \text{PTK} \leftarrow \text{PRF}(\text{PMK}, \ldots),$$
$$\text{check MIC}(\text{KCK}, \text{M2}) \stackrel{?}{=} \text{captured}.$$

A match means p is the passphrase. This is entirely offline – no further contact with the network, no rate limiting, as many guesses as Eve's hardware allows.

What decides success. The PBKDF2 cost (4096 iterations) is the only brake, and modern GPUs push WPA2 candidate rates into the hundreds of thousands to millions per second per card. A passphrase from a wordlist, a common pattern (`Summer2024!`), a phone number, or anything under roughly 10–12 random characters of entropy is realistically crackable. A long random passphrase (say 16+ random characters, or a five-word diceware phrase) is not. Security here depends entirely on the entropy of the human-chosen password.

Focus

This is the single most important practical fact about WPA2-Personal: **the encryption is only as strong as the passphrase**, because anyone within radio range can capture a handshake and grind it offline forever. AES-128 contributes nothing if the passphrase is `password123`. Use long, random passphrases, and prefer WPA3 (which defeats this attack entirely) where available.

Implement it

A learning-only sketch of the WPA2-Personal offline check. It computes the PMK and the MIC over a captured handshake and tests passphrases from a wordlist. It is deliberately incomplete and slow; real tools like `hashcat` do this on GPUs. **Use this only to understand the attack on networks you own. Cracking others' WiFi is illegal in most jurisdictions.**

```
import hashlib, hmac

def pmk(passphrase, ssid):
    # PBKDF2-HMAC-SHA1, 4096 iters, 32-byte PMK -- the WPA2-Personal stretch.
    return hashlib.pbkdf2_hmac("sha1", passphrase.encode(),
                               ssid.encode(), 4096, 32)

def prf512(pmk, A, B):
    # 802.11i PRF: HMAC-SHA1 expansion to 64 bytes. "A" is a label string,
    # "B" = min(macA,macB)||max(macA,macB)||min(n1,n2)||max(n1,n2).
    out = b""
    i = 0
    while len(out) < 64:
        out += hmac.new(pmk, A + b"\x00" + B + bytes([i]),
                        hashlib.sha1).digest()
        i += 1
    return out[:64]  # PTK = KCK(16) || KEK(16) || TK(16) || extra

def ptk(pmk_bytes, ap_mac, sta_mac, anonce, snonce):
    B = min(ap_mac, sta_mac) + max(ap_mac, sta_mac) \
        + min(anonce, snonce) + max(anonce, snonce)
    return prf512(pmk_bytes, b"Pairwise key expansion", B)

def mic_matches(passphrase, ssid, ap_mac, sta_mac,
                anonce, snonce, eapol_m2, captured_mic):
    p = ptk(pmk(passphrase, ssid), ap_mac, sta_mac, anonce, snonce)
    kck = p[:16]
    # WPA2 (CCMP) uses HMAC-SHA1 for the MIC, truncated to 16 bytes.
    mic = hmac.new(kck, eapol_m2, hashlib.sha1).digest()[:16]
```

```
    return mic == captured_mic

# for guess in open("wordlist.txt"):
#     if mic_matches(guess.strip(), ssid, ap_mac, sta_mac,
#                    anonce, snonce, eapol_m2, captured_mic):
#         print("passphrase:", guess); break
```

The real attack also handles the EAPOL frame's MIC field being zeroed before the computation, and the choice of HMAC-SHA1 versus AES-CMAC depending on the cipher suite. Apart from those details, the check is the one shown above.

26.6 KRACK: breaking WPA2 without the password

For thirteen years, the four-way handshake's cryptography held: if your passphrase was strong, your traffic was safe. Then in 2017 Mathy Vanhoef published KRACK – the Key Reinstallation Attack – which broke WPA2 *without knowing the passphrase at all*. KRACK is notable because the flaw was not in the cipher or the key derivation. It was in the *state machine* – specifically, in what happens when a handshake message is retransmitted.

Intuition

Recall message M3: the AP sends the GTK and a MIC, and on receiving it the station *installs the PTK*, which resets the CCMP packet number (the nonce counter) to its starting value and begins encrypting data frames. Message M4 acknowledges this.
But networks are lossy. M4 might not arrive. The standard says: if the AP does not get M4, it *retransmits M3*. And the standard says the station, on receiving a retransmitted M3, should *reinstall* the same PTK – and in doing so, **reset the packet number counter back to its starting value.**
So reinstalling the key resets the nonce, and resetting the nonce in a counter-mode cipher reuses keystream. KRACK forces exactly this reset, on demand, by replaying M3.

Attack

KRACK against the four-way handshake.
Eve sits as a man-in-the-middle at the radio layer, relaying frames between Alice and the AP (she can do this with a cloned AP on a different channel – a "channel- based MitM" – without knowing any key, because management and handshake frames are unprotected).

1. The handshake proceeds normally up to M3. Eve *blocks* M4 from reaching the AP (she is in the middle).
2. The AP, not hearing M4, retransmits M3. Eve forwards this retransmitted M3 to Alice.
3. Alice, following the standard, **reinstalls** the PTK and **resets her packet number to its initial value** – even though she has already sent data frames with that key.
4. Now Alice encrypts her next data frame with the *same* PN (nonce) she used right after the first install. Two plaintexts, one keystream: $c_1 \oplus c_2 = m_1 \oplus m_2$. The keystream is recovered for those PNs, and with known plaintext (TCP/IP headers, retransmissions

Eve can force) Eve decrypts the frames.

On some implementations the damage was worse than keystream reuse. Android 6.0 and certain `wpa_supplicant` versions, on reinstall, set the key to *all zeros*, so Eve did not even need the password-derived keystream – the traffic was encrypted under a known, all-zero key. That turned KRACK from "decrypt some frames" into "decrypt all captured frames without searching for the passphrase."

Figure 26.2 illustrates the channel-based man-in-the-middle sequence that triggers this nonce reset and keystream reuse.

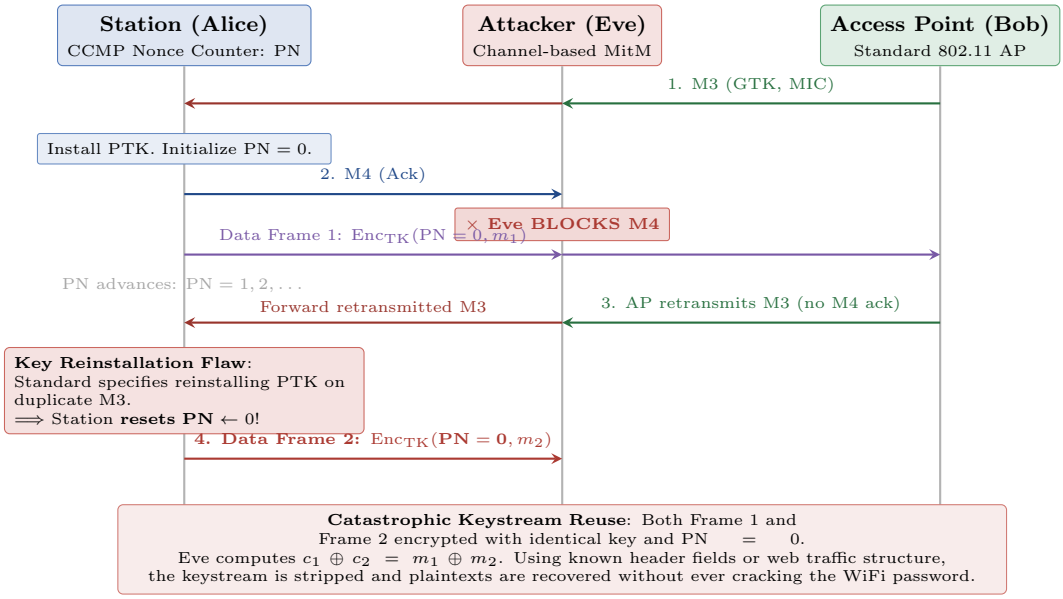


Figure 26.2: The KRACK (Key Reinstallation Attack) protocol flaw. By blocking Alice's Message 4 acknowledgment, the attacker induces the Access Point to retransmit Message 3. Compliant 802.11 clients reinstall the temporal key and reset the packet number counter (PN $\leftarrow$ 0), causing catastrophic nonce reuse in CCMP/GCMP.

Why it works

Why did such a careful protocol have this hole? Because the cryptographic *messages* were authenticated (the MICs are correct, the PTK is right – Eve never forges anything), but the *state machine around them* did not treat "reinstall the same key" as the dangerous operation it is. The protocol proof for the four-way handshake, done years earlier, modeled the key agreement – and it was correct! The flaw lived in a layer the proof did not cover: the retransmission and reinstall behavior mandated by the standard's prose, not its key-exchange core.

The fix is small and was deployed via software patches in late 2017: do not reset the packet number on a key reinstall, or refuse to reinstall an already-installed key. No new key material, no protocol redesign – just "never let a nonce go backwards." KRACK did not retire WPA2; a patched WPA2 stack is not vulnerable. But it forced an Internet-wide update and showed how completely counter-mode AEAD depends on nonce discipline.

Connection

KRACK is the WiFi instance of a general law you will see again in TLS, IPsec, and QUIC: *nonce reuse in an AEAD mode is catastrophic*. Chapter 16 shows that reusing a GCM nonce leaks not only plaintext (via keystream cancelling) but the authentication key itself, enabling forgeries. CCM is gentler – nonce reuse there "only" leaks plaintext, not the MAC key – but the lesson is identical: the counter must never repeat or reverse. KRACK is what happens when a state machine quietly violates that invariant.

26.7 WPA3 and the SAE/Dragonfly handshake

WPA2-Personal has one structural limitation that no patch can fix: the offline dictionary attack. Because authentication boils down to proving knowledge of the PMK, and the PMK comes deterministically from the passphrase, anyone who captures a handshake can test candidate passphrases offline forever. That weakness is built into deriving keys directly from a password.

WPA3-Personal closes this avenue by replacing the four-way handshake's front end with a **Password-Authenticated Key Exchange** (PAKE), specifically **SAE** – Simultaneous Authentication of Equals, also called Dragonfly. A PAKE lets two parties who share a (possibly weak) password agree on a strong, random shared key such that:

- An eavesdropper learns *nothing* that helps an offline guess.
- An active attacker gets at most *one* password guess per live interaction – they must talk to the AP for each guess, and the AP can rate-limit. Offline grinding is impossible.
- The resulting session key has **forward secrecy**: it is a fresh Diffie-Hellman secret, so compromising the password *later* does not decrypt traffic recorded earlier.

Intuition

A PAKE folds the password into a Diffie-Hellman exchange so that the DH messages look like random group elements to anyone who does not know the password, and so that the exchange only completes if both sides used the same password. You get the best of both: the strong randomness and forward secrecy of Diffie-Hellman, plus authentication from the shared password – without ever sending anything an eavesdropper can test offline. The phrase "of equals" means there is no client/server asymmetry: AP and station run identical roles, either can initiate, and either can be the first to commit.

26.7.1 How SAE works

SAE runs in a cyclic group $\mathbb{G}$ of prime order q – a large prime-field group or, in practice, an elliptic curve like NIST P-256 – with generator g. Both sides share the password π. The exchange has two phases: *commit* and *confirm*.

Setup – hashing the password to the group. First both sides derive a group element from the password, the "password element" $P = \mathrm{H2C}(\pi, \ldots)$ (hash-to-curve / hash-to-group), mixing in the two MAC addresses so P is specific to this pair of devices. The key property: P is a uniform-looking

group element, and recovering π from P is as hard as the discrete log – you cannot "read off" the password from P.

Commit phase. Each party (say Alice) picks two random scalars $r_A, m_A \in \mathbb{Z}_q$, sets her private scalar $s_A = (r_A + m_A) \bmod q$, and computes the "element"

$$E_A = -m_A \cdot P \qquad \text{(in the group, } P^{-m_A} \text{ multiplicatively)},$$

then sends the pair $(s_A,\ E_A)$. Bob does the same, sending $(s_B,\ E_B)$.

Deriving the shared secret. Alice computes

$$K = r_A \cdot \big(s_B \cdot P + E_B\big).$$

Substituting $s_B \cdot P + E_B = (r_B + m_B)P - m_B P = r_B \cdot P$, this is $K = r_A \cdot r_B \cdot P$. Bob, symmetrically, computes $K = r_B \cdot r_A \cdot P$ – the *same* element. That shared $K = r_A r_B P$ is a Diffie-Hellman secret in the group generated by P: each side contributed a fresh random scalar (r_A, r_B), giving forward secrecy, and P ties the whole thing to the password.

Confirm phase. Each side hashes K (plus the transcript) into a key κ and sends a confirmation MAC. If both used the same password, the same P, hence the same K, the confirmations verify and the exchange succeeds, producing the PMK for the (unchanged) four-way handshake that follows. If the passwords differ, P differs, K differs, and confirmation fails – with no information leaked about *which* password was wrong.

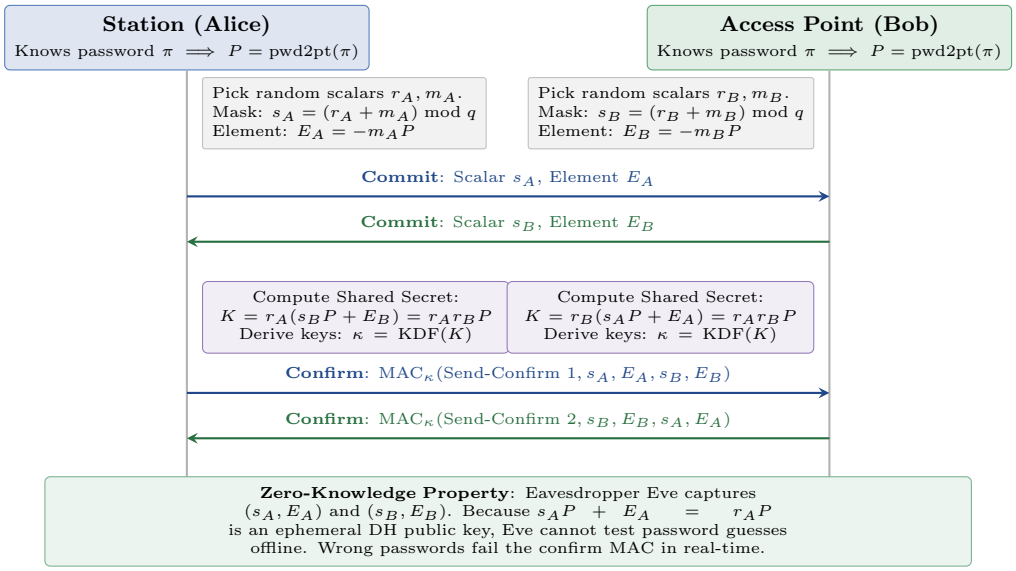


Figure 26.3: The WPA3 Simultaneous Authentication of Equals (SAE / Dragonfly) protocol. Both parties hash the passphrase to a curve point P and perform a blinded Diffie–Hellman exchange. Offline dictionary attacks are mathematically impossible because the public wire transcripts contain no verifier or MIC for password testing.

Why it works

Why can't Eve grind offline? Everything she sees on the wire – s_A, E_A, s_B, E_B, the confirmation MACs – is a function of the random scalars r, m and the password element P. To

test a password guess π', she would compute the corresponding P' and try to reproduce the transcript, but the transcript also depends on the secret scalars r_A, r_B, m_A, m_B, which she does not have and which are freshly random each run. There is no equation she can write down that involves only the password and public values – the discrete-log hardness of the group blocks it. So a password guess can only be tested by *running* the protocol against a live party who knows the real P: one guess per online attempt, rate-limitable. This is what a PAKE is designed to guarantee, and it is why WPA3 defeats the capture-and-crack attack that dooms weak WPA2 passphrases.

Example 26.2. A toy SAE in $(\mathbb{Z}/23\mathbb{Z})^*$, a cyclic group of order $q = 22$ with generator $g = 5$. Suppose the password element is $P = g^7 \bmod 23$. Compute $5^7 = 78125$; since $5^{22} \equiv 1$, reduce: $5^2 = 25 \equiv 2$, $5^4 \equiv 4$, $5^7 = 5^4 \cdot 5^2 \cdot 5 \equiv 4 \cdot 2 \cdot 5 = 40 \equiv 17 \pmod{23}$. So $P = 17$.

Alice picks $r_A = 3$, $m_A = 5$, so $s_A = 8$ and $E_A = P^{-m_A} = 17^{-5}$. Bob picks $r_B = 4$, $m_B = 6$, so $s_B = 10$ and $E_B = 17^{-6}$.

Alice's shared secret (writing the group multiplicatively):

$$K = \left(P^{s_B} \cdot E_B\right)^{r_A} = \left(P^{r_B + m_B} \cdot P^{-m_B}\right)^{r_A} = P^{r_B r_A} = 17^{12} \bmod 23.$$

Compute $17 \equiv -6$, so $17^2 \equiv 36 \equiv 13$; $17^4 \equiv 13^2 = 169 \equiv 8$; $17^8 \equiv 64 \equiv 18$; thus $17^{12} = 17^8 \cdot 17^4 \equiv 18 \cdot 8 = 144 \equiv 6 \pmod{23}$. So $K = 6$. Bob computes $K = P^{r_A r_B} = 17^{12} = 6$ too. They agree on $K = 6$ without either sending a password-derived value an eavesdropper could test.

Common trap

Example 26.2 uses a 23-element group so the arithmetic fits on a page; the discrete log there is trivial to brute-force. Real SAE uses P-256 or a 3072-bit prime field, where discrete log is infeasible. Also, real hash-to-curve (deriving P from the password) is the subtle part – and is exactly where the first WPA3 attack landed, next.

26.7.2 Dragonblood: early attacks on WPA3

Attack

Dragonblood (Vanhoef and Ronen, 2019). WPA3's first year brought side- channel attacks on the hash-to-group step. The original SAE used a "hunting and pecking" loop to map the password to a curve point: try $\mathrm{H}(\pi, \text{counter})$, check if it is a valid x-coordinate, increment the counter if not. The number of iterations, and the cache-access pattern inside, *depended on the password*. By measuring timing or cache behavior of an AP, an attacker learned how many iterations a given password needed – a partial fingerprint – and could prune the password space, restoring a form of offline attack. There were also downgrade attacks forcing WPA3-capable devices back to WPA2 (and thus back to the crackable handshake), and denial-of-service via the computationally heavy SAE commit.
The fixes: a constant-time hash-to-curve (the "hash-to-element" / SSWU method, borrowing from the broader hash-to-curve standardization effort) that runs in fixed time regardless of password, plus transition-mode hardening against downgrade. WPA3 with these fixes – "WPA3 with H2E" – is the current recommendation.

The Dragonblood story carries the same lesson as KRACK: the core cryptographic idea (a PAKE for WiFi) was sound, and the flaw lay in the *implementation around it* – a password-dependent loop that leaked through a side channel. In cryptography, constant-time code is a correctness requirement.

26.8 The practical attacker: deauth, evil twins, and what's actually at risk

Now consider the operational question: you are sitting in a cafe on `CoffeeHouse`-WiFi (WPA2-Personal). Who can do what?

Focus

What an attacker within radio range (not on the network) can do.

- **See all metadata.** Frame headers – your device's MAC address (unless MAC randomization is on), the AP you joined, the sizes and timing of your frames, which networks your device probes for – are all in the clear. Traffic analysis is wide open.
- **Deauthenticate you.** In plain WPA2, management frames are unauthenticated, so a spoofed deauth knocks you off at will (denial of service, or to force a handshake for cracking, or to push you toward an evil twin). **802.11w / Protected Management Frames (PMF)** fixes this by MAC-ing management frames; PMF is *mandatory* in WPA3 and optional in WPA2. With PMF on, deauth spoofing stops working.
- **Capture a handshake and crack a weak passphrase** offline, as in the attack box above. Strong passphrase or WPA3: no.

What an attacker on the same network (knows the passphrase) can do.

- In **WPA2-Personal**, everyone shares the same PMK. But each device's PTK is fresh (different nonces), so your neighbor cannot *directly* decrypt your unicast frames from a passive capture – almost. The catch: if they capture your four-way handshake (its nonces are public) and they know the PMK (they do – same passphrase!), they can derive *your* PTK and decrypt your traffic. So on WPA2-Personal, **anyone who knows the WiFi password and can capture your handshake can read your link-layer traffic.** The cafe's other customers are not isolated from you. (Forcing a deauth gives them your handshake on demand.)
- They can run ARP spoofing / become a man-in-the-middle for your local traffic, redirect DNS, etc. – standard LAN attacks.
- **WPA3** and WPA2-Enterprise close this: SAE gives each device an independent PMK that other password-holders cannot derive, so same-network users *cannot* decrypt each other's traffic.

Attack

The evil twin. Eve sets up her own AP broadcasting the SSID `CoffeeHouse` – same name, stronger signal (she is closer, or boosts power). She deauths Alice from the real AP. Alice's device, seeing a familiar network name with a strong signal, may reconnect to Eve's AP. If Eve's AP is open (no password) or Eve knows/guessed the passphrase, Alice now routes all

her traffic through Eve.
What saves Alice: WPA2/WPA3 *mutual* authentication. In the four-way handshake, the AP must prove it knows the PMK (message M3's MIC). Eve's evil twin, *without the passphrase*, cannot produce a valid M3, so a WPA2 client will not complete the handshake – the evil twin fails against a password-protected network it cannot authenticate to. This is why evil-twin attacks target **open** networks, captive portals, or networks whose passphrase the attacker already knows, and why they often pair with phishing ("enter the WiFi password to continue") to get the missing secret. WPA3's mandatory PMF and SAE raise the bar further by killing the deauth that bootstraps the attack.

Intuition

The reassuring takeaway: even on a hostile WiFi network, your *application* security can hold. WiFi protects the link; TLS protects the connection end to end. When you load `https://bank.com`, the TLS handshake authenticates the bank with a certificate that Eve's evil twin cannot forge, and encrypts everything above the link layer. An attacker who fully owns the WiFi sees TLS ciphertext and SNI/metadata – annoying, but not your password or your balance. This is the layered-defense principle: WiFi crypto and TLS guard different layers, and you want both. The danger zone is unencrypted application traffic (plain HTTP, some IoT protocols, DNS without DoH) on a hostile link – there, link-layer compromise is full compromise.

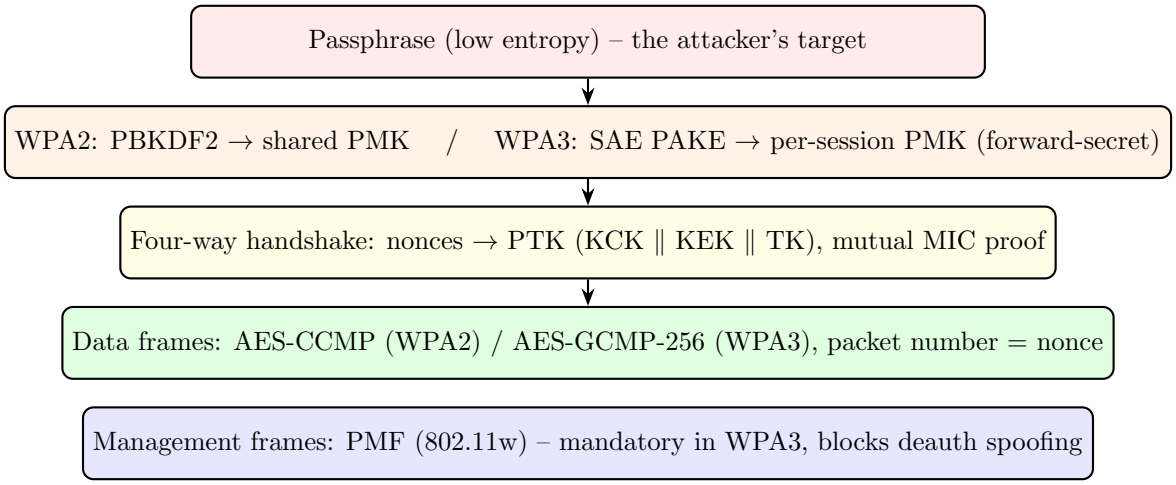
In the real world

In deployment, what to actually do.

- **Home / small office.** Use WPA3-Personal if all your devices support it; otherwise WPA2/WPA3 "transition mode" with a *long random* passphrase (the passphrase is your real security on WPA2). Enable PMF. Never WEP or WPA (TKIP); disable them on the router.
- **Enterprise / campus.** WPA2/WPA3-Enterprise with 802.1X/EAP-TLS or PEAP against RADIUS. Each user authenticates individually, gets an independent PMK, and there is no shared passphrase to crack or to let users decrypt each other. Validate the RADIUS server certificate on clients, or evil-twin RADIUS impersonation re-opens the door.
- **Public WiFi (you, the user).** Assume the link is hostile. Rely on TLS/HTTPS everywhere, prefer a VPN, keep your OS patched (KRACK and Dragonblood fixes ship in updates), and enable MAC randomization to blunt tracking. "WPA2 with a password" on a cafe network does *not* hide your traffic from other patrons who know that same password.
- **Open networks (Opportunistic Wireless Encryption, OWE).** WPA3 adds OWE / "Enhanced Open," which runs an unauthenticated Diffie-Hellman on open networks so at least passive eavesdropping is blocked even with no password. It does not stop an active evil twin, but it means open coffee-shop WiFi no longer has to be plaintext.

26.9 Putting the layers together

Taken together, the stack has several layers. At the root is a password, low entropy and human-chosen – the weakest link, and the one every attack aims at. WPA2 stretches it (PBKDF2) and proves knowledge of it (four-way handshake MICs), but cannot escape that a captured handshake makes the password grindable offline. WPA3 breaks that chain with a PAKE (SAE), so the password is no longer offline-testable and each session gets a forward-secret key. On top of either sits the data-frame protection – AES-CCMP (WPA2) or AES-GCMP (WPA3) – whose entire safety rests on a never-repeating packet number, the invariant KRACK showed is easy to violate by accident.



Every famous WiFi break maps onto one of these layers. WEP failed at the data layer (keystream reuse, linear checksum) and the key layer (FMS). The offline dictionary attack lives at the password layer and is intrinsic to WPA2-Personal. KRACK struck the handshake/data boundary by resetting a nonce. Dragonblood hit WPA3's password-to-group mapping through a side channel. The protocols got better by hardening each layer in turn. The same lesson appears at every layer: a sound cryptographic core can still be undone by the engineering around it – a checksum that should be a MAC, a counter that resets, a loop whose length leaks the secret. Cryptographic engineering means getting all of these details right at once, even on a $30 router.

WiFi protects the radio hop to the router and TLS protects the connection to a server, yet the company running that server still sees every plaintext message it relays. In Chapter 27, we turn to end-to-end encrypted messaging and the Signal protocol, where X3DH and the Double Ratchet keep messages secret even from the server and when recipients are offline.

26.10 Exercises

Exercise 26.1. (WEP keystream reuse.) Two WEP frames are captured with the same IV. Their ciphertexts are c_1 and c_2. You know that m_1 is an ARP request whose first 8 bytes are the constant `AA AA 03 00 00 00 08 06`. Show exactly how to recover the first 8 bytes of m_2, and explain why no part of the WEP key is needed.

Exercise 26.2. (CRC malleability.) WEP's integrity check is CRC-32, which is linear: $\mathrm{CRC}(x \oplus \delta) = \mathrm{CRC}(x) \oplus \mathrm{CRC}(\delta)$. Encryption is $c = (m \,\|\, \mathrm{CRC}(m)) \oplus Z$. Given a ciphertext c for an unknown

m, describe how to produce a valid ciphertext c' that decrypts to $m \oplus \delta$ for a chosen δ, *without* knowing m or the keystream Z. Why does this break integrity?

Exercise 26.3. (PBKDF2 cost.) A GPU rig tests 2×10^6 WPA2 passphrase candidates per second. (a) How long to exhaust all 8-character lowercase-letter passphrases (26^8)? (b) All 8-character passphrases over a 72-character alphabet (72^8)? (c) A five-word diceware passphrase (7776^5 possibilities)? Comment on which are realistically crackable.

Exercise 26.4. (Toy handshake.) Using the toy PRF $\mathrm{prf}(K, x) = (Kx + 7) \bmod 101$ from Example 26.1, let PMK $= 30$, Anonce $= 20$, Snonce $= 15$ (combine by addition). Compute the toy PTK, then KCK $=$ PTK mod 10, then the M2 MIC $\mathrm{prf}(\mathrm{KCK}, \mathrm{Snonce})$. Now show that, because this toy PRF is linear, an attacker who sees one (input, output) pair under KCK can solve for KCK – and explain in one sentence why HMAC-SHA1 does not have this defect.

Exercise 26.5. (KRACK invariant.) State the single invariant whose violation enables KRACK. Then explain why the official patch – "do not reset the packet number on key reinstall" – restores it, and why this required no change to the key- derivation math or the cipher.

Exercise 26.6. (Nonce reuse, two modes.) CCMP uses AES-CCM; WPA3's high tier uses AES-GCM. Reusing a nonce is bad in both, but worse in one. Explain what an attacker gains from nonce reuse in CTR-based confidentiality (both modes), and what *additional* catastrophe GCM suffers that CCM does not. (Hint: GCM's authenticator is a polynomial evaluation whose key the nonce reuse can expose.)

Exercise 26.7. (SAE arithmetic.) Work a toy SAE in $(\mathbb{Z}/23\mathbb{Z})^*$ ($g = 5$, $q = 22$) with password element $P = g^9 \bmod 23$. Let Alice use $r_A = 5, m_A = 4$ and Bob use $r_B = 6, m_B = 3$. Compute s_A, s_B, the shared secret $K = P^{r_A r_B} \bmod 23$, and verify both sides reach the same K. Then argue informally why an eavesdropper who sees s_A, s_B, E_A, E_B cannot recover P (and hence the password) when the group is large.

Exercise 26.8. (Evil twin reasoning.) Explain precisely why a WPA2-Personal evil twin *without* the passphrase cannot complete the four-way handshake with a victim client, identifying which message it fails to produce and why. Then explain why an *open* (passwordless) network gives a victim no such protection, and what OWE (Enhanced Open) does and does not fix.

Exercise 26.9. (Same-network decryption.) On a WPA2-Personal cafe network, you and a stranger both know the passphrase. The stranger captures your four-way handshake. Walk through exactly what they can derive (PMK, then PTK, then TK) and conclude whether they can decrypt your unicast traffic. Then explain why the same attack fails on WPA3-Personal even though, in a sense, "everyone still knows the password."

Exercise 26.10. (Forward secrecy.) Define forward secrecy. Explain why WPA2-Personal lacks it (what an attacker who records traffic today and learns the passphrase next year can do), and why WPA3's SAE provides it. Identify the specific ephemeral values in SAE that make recorded sessions undecryptable even after password compromise.

Exercise 26.11. (Layered defense.) You connect to a fully attacker-controlled WiFi AP (the attacker owns the link entirely). For each of the following, state whether the attacker can read or tamper with the content, and why: (a) an `https://` page load to a site with a valid certificate; (b) a plain `http://` page load; (c) a DNS lookup over plain UDP; (d) a DNS lookup over DoH (DNS-over-HTTPS). Use the result to justify the advice "assume public WiFi is hostile and rely on TLS."

Chapter 27

Secure Messaging and the Signal Protocol

In Chapter 26, we saw how WPA2 and WPA3 protect the local radio link while TLS (Chapter 25) secures connections to a server, yet the company running that server still sees every plaintext message. Worse, every protocol so far in this series has assumed both parties were online and willing to talk. TLS opens with a handshake: Alice and Bob exchange messages, derive a shared key, and start encrypting – but they do it together, in real time, both sitting at their keyboards. Diffie-Hellman key exchange (Chapter 18) is the same: a live conversation. That assumption is fine for a web browser talking to a server, but it does not work for a chat app.

When Alice messages Bob on a phone, Bob is asleep. His screen is dark, his radio is off, he is in a tunnel. Alice wants to fire off "running 10 min late" and have it waiting when Bob wakes up, encrypted end-to-end so that not even the messaging company can read it. There is no handshake, because there is no Bob to handshake with. And Alice will send a hundred more messages over the next week, each of which must stay secret even if Bob's phone is stolen on Tuesday and Alice's is compromised on Friday.

This is the *asynchronous, long-lived, end-to-end* setting, and none of the tools so far handle it. The Signal Protocol – born from Trevor Perrin and Moxie Marlinspike's work around 2013, descended from Off-the-Record Messaging – is the design that solved it. WhatsApp's two billion users, Google's RCS, Facebook Messenger's secret conversations, and Signal itself all run it essentially unchanged. This chapter builds it from the ground up: the goals (forward secrecy and post-compromise security), the X3DH key agreement that bootstraps a session against an offline partner, and the Double Ratchet that keeps the session secure message after message, year after year. We trace tiny numeric examples, draw the ratchet, break a naive design to see why the real one is shaped the way it is, and finish with the metadata problem and Signal's deniability property.

27.1 What End-to-End Really Promises

Start with the threat model, because "end-to-end encrypted" is a phrase marketers abuse and engineers must pin down.

The players are familiar. Alice and Bob want to chat. Eve sits on the network. But there is

a new, important character: the *server* – the messaging company's infrastructure, which relays every message because Alice and Bob are rarely online at the same time. In ordinary transport encryption (TLS to the server), the server decrypts everything; it is a trusted middleman. **End-to-end encryption (E2EE)** removes that trust. The server becomes just another adversary – an active one that sees all ciphertext and all timing, can drop or reorder or replay messages, and can lie about who is who – but it must never see plaintext.

Intuition

Think of the postal service. Transport encryption is an armored truck: your letter is safe *on the road*, but the post office opens it at the sorting facility, reads it, reseals it, and sends it on. End-to-end encryption is a letter you wrote in a private code only your friend knows. The post office still carries it, still sees the envelope (who, when, how big), but the contents are gibberish to them. E2EE is about distrusting the carrier, not just the road.

What should E2EE guarantee? Four properties, in rough order of how hard they are to achieve:

- **Confidentiality and integrity (the baseline).** Only Alice and Bob read the messages; tampering is detected. This is authenticated encryption (Chapter 16) with a key the server never learns.
- **Forward secrecy.** If Bob's phone is seized *today* and all its keys extracted, the attacker still cannot decrypt the messages Alice sent *last month*, even though she recorded every ciphertext. Past traffic stays secret despite a future total compromise.
- **Post-compromise security (future secrecy / self-healing).** If an attacker steals Bob's keys today and reads a few messages, the session *recovers* on its own: once the attacker loses active access to the device and Alice and Bob exchange a couple more messages, the attacker is locked out again with no manual key reset.
- **Authentication and deniability.** Bob is sure he is talking to Alice and not the server impersonating her – yet, paradoxically, neither can later *prove to a third party* what the other said.

Focus

Forward secrecy and post-compromise security point in opposite directions in time, and it is worth saying them side by side. Forward secrecy protects the *past* from a *future* break: "you cannot reach back." Post-compromise security protects the *future* from a *past* break: "you cannot stay in." A protocol with both treats every key as disposable – old keys are deleted so they cannot be stolen, and new keys are mixed in so a stolen key goes stale. The Double Ratchet is the machine that does both, automatically, on every message.

Connection

Forward secrecy already appeared in Chapter 23 in the TLS setting: use an *ephemeral* Diffie-Hellman key for each session and delete it afterward, so the long-term certificate key cannot decrypt recorded traffic. Signal pushes the same idea to its limit. Instead of one ephemeral key per session, it derives a fresh key per *message*, and it does so without a live handshake. The underlying primitives – Diffie-Hellman on the elliptic curve Curve25519 (Chapter 21) and HKDF key derivation built on HMAC (Chapter 15) – are ones you have already met. What

is new is how they are combined.

27.2 Why the Old Tools Do Not Fit

Before building Signal, let us watch the obvious approaches fail, because each failure motivates a piece of the real design.

Attempt 1: Just use a shared long-term key. Alice and Bob agree on a key k once (say by scanning a QR code in person) and AES-GCM every message under it forever. Confidential, authenticated, asynchronous – and it has *no* forward secrecy. Steal k on Tuesday and you decrypt every message ever sent and every message that will ever be sent. A single stolen key exposes the whole conversation.

Attempt 2: Ephemeral Diffie-Hellman per message. Borrow the TLS trick: for each message, Alice and Bob run a fresh DH exchange, derive a one-time key, encrypt, then delete the ephemeral private keys. Now forward secrecy holds – old keys are gone. But DH needs *both* parties to contribute a fresh public value, live. Bob is asleep, so nobody is there to send the second half of the handshake.

Attempt 3: Bob pre-publishes DH values. Have Bob upload a stack of one-time public keys to the server in advance. When Alice wants to talk, she grabs one, does half a DH with her own ephemeral key, and sends. Bob, when he wakes, finishes the DH. This is the seed of the real idea – and it is exactly what X3DH does. But two problems remain. First, the stack of pre-keys runs out (Bob is offline for a month, a thousand people message him). Second, and deeper: a single DH at the start gives you forward secrecy *up to the start*, but once the session is running, what protects message #500 if the session key leaks at message #499? You need the key to keep *moving*. That continuous motion is the ratchet.

Common trap

A tempting non-fix: "rotate the key by hashing it." Set $k_{i+1} = \mathrm{H}(k_i)$ and encrypt message i under k_i. This gives forward secrecy – H is one-way, so leaking k_{500} does not reveal k_{499} (you cannot run the hash backward). But it gives *no* post-compromise security. If the attacker steals k_{500}, she computes $k_{501}, k_{502}, \ldots$ forever by hashing forward. The chain has no fresh randomness entering it, so a single compromise is permanent. This is precisely why the Double Ratchet has *two* ratchets: a hash chain (cheap, forward-secret) *and* a Diffie-Hellman ratchet (expensive, self-healing) that periodically injects new entropy the attacker cannot predict.

27.3 X3DH: Agreeing on a Key With Someone Who Is Asleep

The Extended Triple Diffie-Hellman key agreement, **X3DH**, is how a Signal session begins. Its job: let Alice derive a shared secret with Bob and send her first encrypted message, all while Bob is offline, with three security goals baked in – forward secrecy from the very first message, cryptographic proof to Alice that the keys really are Bob's, and deniability so neither can prove to an outsider that the conversation happened.

27.3.1 The keys each party holds

Every Signal user has a small zoo of keys. All are Curve25519 key pairs; write X-uppercase for a public key and the matching lowercase for its private scalar, so $A = a \cdot G$ on the curve (Chapter 21). The four kinds:

- **Identity key** $IK = (ik, IK_{\text{pub}})$. Long-term, created once, the cryptographic name of the user. This is what you verify when you "check the safety number" with a friend.
- **Signed pre-key** $SPK = (spk, SPK_{\text{pub}})$. Medium-term (rotated weekly or so). Bob signs SPK_{pub} with his identity key, so Alice can check it is genuinely Bob's.
- **One-time pre-keys** $OPK_1, OPK_2, \ldots$. A batch of single-use public keys Bob uploads in advance; each is used by at most one incoming session and then deleted.
- **Ephemeral key** $EK = (ek, EK_{\text{pub}})$. Generated fresh by *Alice* for this one session start.

Bob, while online at some quiet moment, uploads to the server a *pre-key bundle*: IK^B_{pub}, SPK^B_{pub} with its signature, and a pile of one-time pre-keys OPK^B. The server stores them and hands them out on request. Bob can now go offline for a month.

27.3.2 Alice's side: three (or four) Diffie-Hellmans

When Alice wants to start talking to Bob, she fetches one pre-key bundle from the server. The server gives her IK^B_{pub}, the signed SPK^B_{pub}, and *one* fresh one-time pre-key OPK^B_{pub} (which the server then deletes from Bob's pile). Alice first verifies Bob's signature on SPK^B_{pub} using IK^B_{pub}; if it fails she aborts – the server tried to feed her a fake pre-key. Then she generates her ephemeral pair $EK = (ek, EK_{\text{pub}})$ and computes *four* Diffie-Hellman shared values:

$$
\begin{aligned}
DH_1 &= \mathrm{DH}(ik_A,\ SPK^B_{\text{pub}}) && \text{Alice identity } \times \text{ Bob signed pre-key} \\
DH_2 &= \mathrm{DH}(ek_A,\ IK^B_{\text{pub}}) && \text{Alice ephemeral } \times \text{ Bob identity} \\
DH_3 &= \mathrm{DH}(ek_A,\ SPK^B_{\text{pub}}) && \text{Alice ephemeral } \times \text{ Bob signed pre-key} \\
DH_4 &= \mathrm{DH}(ek_A,\ OPK^B_{\text{pub}}) && \text{Alice ephemeral } \times \text{ Bob one-time pre-key}
\end{aligned}
$$

Here $\mathrm{DH}(x, Y)$ means the Curve25519 shared point $x \cdot Y$ (or rather its x-coordinate, X25519's output). The shared secret is the HKDF of all four concatenated:

$$SK = \mathrm{HKDF}\big(DH_1 \,\|\, DH_2 \,\|\, DH_3 \,\|\, DH_4\big).$$

(DH_4 is omitted if Bob's one-time pre-keys ran out; the protocol still works, with slightly weaker guarantees for that session.)

Alice now holds SK. She deletes ek_A – the ephemeral private key – and sends Bob a first message containing: her identity public key IK^A_{pub}, her ephemeral public key EK_{pub}, which one-time pre-key she used, and her first ciphertext encrypted under a key derived from SK.

Why it works

Why four DHs and not one? Each DH binds in a different security property, and you need all of them:

- DH_1 ($ik_A \times SPK^B$) mixes in *Alice's* long-term identity, so the secret depends on who Alice really is. Combined with DH_2, it *authenticates both parties* – an attacker who does not know ik_A cannot compute DH_1, and one who does not know ik_B cannot compute DH_2.
- DH_2 ($ek_A \times IK^B$) mixes in *Bob's* long-term identity, so Alice knows the secret is shared with the real Bob and nobody else.
- DH_3 ($ek_A \times SPK^B$) and DH_4 ($ek_A \times OPK^B$) mix Alice's *ephemeral* key with Bob's *medium-* and *short-lived* keys. These provide **forward secrecy**: once ek_A, spk^B, and opk^B are deleted, nobody – not even Alice and Bob – can recompute SK from the long-term identity keys alone.

The crossed structure matters. DH_1 uses Alice's long-term and Bob's medium-term; DH_2 flips it. Authentication comes from the long-term keys appearing; forward secrecy comes from the ephemeral keys appearing; and because they are *mixed* into one HKDF, an attacker must break *every* DH to recover SK.

27.3.3 Bob's side: he catches up later

When Bob comes online, the server hands him Alice's first message. Bob now has everything he needs to compute the *same* four DH values from his side, because Diffie-Hellman is symmetric: $\mathrm{DH}(ik_A, SPK^B_{\mathrm{pub}}) = ik_A \cdot spk_B \cdot G = \mathrm{DH}(spk_B, IK^A_{\mathrm{pub}})$. Concretely Bob computes

$$DH_1 = \mathrm{DH}(spk_B,\ IK^A_{\mathrm{pub}}), \qquad DH_2 = \mathrm{DH}(ik_B,\ EK_{\mathrm{pub}}),$$
$$DH_3 = \mathrm{DH}(spk_B,\ EK_{\mathrm{pub}}), \qquad DH_4 = \mathrm{DH}(opk_B,\ EK_{\mathrm{pub}}),$$

feeds them through the same HKDF, and recovers the identical SK. He decrypts Alice's first message, deletes the one-time pre-key opk_B he just used (so it can never be reused), and the session is open. Bob did *nothing* ahead of time except upload a bundle; he ran zero live round trips. That is what makes the protocol asynchronous.

Walking question

Convince yourself the four values match. DH_3 is $\mathrm{DH}(ek_A, SPK^B_{\mathrm{pub}})$ for Alice and $\mathrm{DH}(spk_B, EK_{\mathrm{pub}})$ for Bob. Alice computes $ek_A \cdot (spk_B \cdot G)$; Bob computes $spk_B \cdot (ek_A \cdot G)$. Scalar multiplication on the curve is associative and commutative in the scalars, so both equal $(ek_A \cdot spk_B) \cdot G$. Same point, same x-coordinate, same DH output. The other three are identical in form. This is just ordinary Diffie-Hellman (Chapter 18) run four times with four pairings of the available keys.

The whole exchange has only three arrows, and not one of them needs Bob awake when Alice acts. Figure 27.1 shows the full message flow.

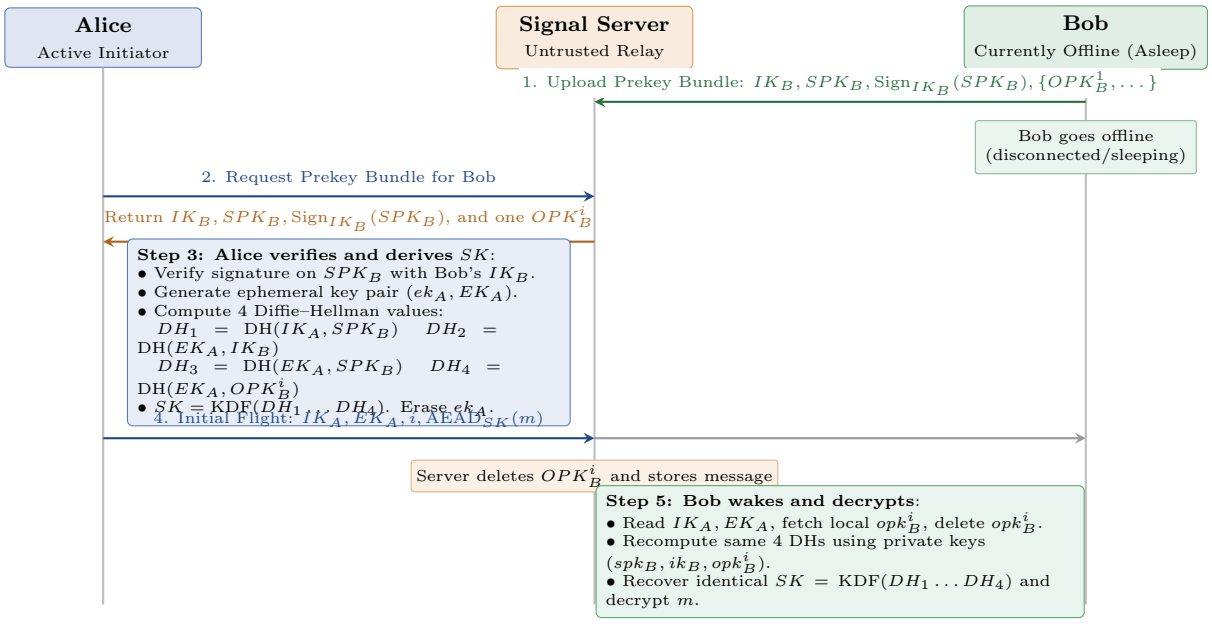


Figure 27.1: The Signal X3DH (Extended Triple Diffie–Hellman) asynchronous key agreement protocol. Bob registers a signed pre-key bundle with the untrusted server and goes offline. Alice fetches Bob's bundle, derives the shared secret key SK via four Diffie–Hellman exchanges, and sends her initial encrypted message. Bob recovers SK and decrypts whenever he reconnects.

27.3.4 A tiny worked example

Curve25519 uses a 255-bit prime, far too big to trace by hand, so we shrink to a toy group: ordinary Diffie-Hellman modulo the prime $p = 23$ with generator $g = 5$. (This is multiplicative-group DH, not elliptic, but the algebra of "four DHs combine into one secret" is identical, and small numbers let us check every step by hand.) All private keys are exponents mod 22; public keys are $g^{\text{priv}} \bmod 23$.

Assign private keys: Alice identity $ik_A = 6$, Alice ephemeral $ek_A = 9$; Bob identity $ik_B = 4$, signed pre-key $spk_B = 7$, one-time pre-key $opk_B = 3$. The public keys are

$$IK^A = 5^6 = 8, \qquad EK = 5^9 = 11,$$
$$IK^B = 5^4 = 4, \quad SPK^B = 5^7 = 17, \quad OPK^B = 5^3 = 10 \pmod{23}.$$

(Check one: $5^3 = 125 = 5 \cdot 23 + 10$, so $5^3 \equiv 10$. And $5^4 = 5 \cdot 10 = 50 \equiv 4$.) Now the four DH values. A DH between private x and public g^y is $(g^y)^x = g^{xy}$.

$$DH_1 = (SPK^B)^{ik_A} = 17^6 \bmod 23 = g^{7 \cdot 6} = g^{42} = g^{42 \bmod 22} = g^{20},$$
$$DH_2 = (IK^B)^{ek_A} = 4^9 = g^{4 \cdot 9} = g^{36} = g^{36 \bmod 22} = g^{14},$$
$$DH_3 = (SPK^B)^{ek_A} = 17^9 = g^{7 \cdot 9} = g^{63} = g^{63 \bmod 22} = g^{19},$$
$$DH_4 = (OPK^B)^{ek_A} = 10^9 = g^{3 \cdot 9} = g^{27} = g^{27 \bmod 22} = g^5.$$

Reducing the exponents mod $\text{ord}(g) = 22$ and evaluating powers of 5 mod 23: $g^{20} = 5^{20} \equiv 12$, $g^{14} = 5^{14} \equiv 13$, $g^{19} = 5^{19} \equiv 7$, and $g^5 = 5^5 \equiv 20 \pmod{23}$. So

$$SK = \text{HKDF}(12 \,\|\, 13 \,\|\, 7 \,\|\, 20).$$

Now do Bob's side and watch it match. Bob computes $DH_3 = (EK)^{spk_B} = 11^7 = g^{9 \cdot 7} = g^{63} \equiv g^{19} \equiv 7$ – the same value Alice got. Every term agrees, so Bob's HKDF input is the identical byte string, and he derives the identical SK. The session secret is established without Bob ever being online at the same time as Alice.

Common trap

Toy numbers like $p = 23$ are for understanding only. Real X3DH uses X25519, where the private keys are 32-byte scalars and the discrete log is computationally hopeless. With $p = 23$ an attacker reads the public keys $8, 11, 4, 17, 10$ off the wire and brute-forces every exponent in a fraction of a second. Never build anything on a group small enough to trace by hand.

History

X3DH descends from work by Trevor Perrin and Moxie Marlinspike at Open Whisper Systems (2013–2016) and from the Off-the-Record Messaging (OTR) protocol of Borisov, Goldberg, and Brewer (2004), which first married forward secrecy and deniability for instant messaging. OTR, though, was *synchronous* – both parties had to be online. The leap to asynchronous operation, so a message could be sent to an offline recipient with forward secrecy intact, is what made Signal usable as a real phone app. The X3DH and Double Ratchet specifications were published in 2016 (and formally analyzed by Cohn-Gordon et al. in 2017); the same year, WhatsApp finished rolling Signal's protocol out to over a billion users, making it the most widely deployed end-to-end key agreement in use.

27.4 The Double Ratchet: Keys That Never Stop Moving

X3DH gives Alice and Bob one shared secret SK. If they just used SK to key AES-GCM for every message, they would be back at Attempt 1, where one stolen key exposes everything. The **Double Ratchet** takes SK as a seed and from it derives a fresh key for *every single message*, in a way that gives both forward secrecy and post-compromise security. It is called "double" because it combines two distinct ratcheting mechanisms.

Intuition

A ratchet is a gear that turns only one way – a socket wrench, a zip tie, a car's parking pawl. Once it clicks forward it cannot click back. The cryptographic analogue: a key-derivation step you can run forward to get the next key but cannot reverse to recover the previous one. Run it on every message and the past is sealed off behind you. The Double Ratchet has a cheap one-way gear (a hash chain) for per-message keys and an asymmetric gear (Diffie-Hellman) that, in addition to turning forward, mixes in fresh randomness so a captured key goes stale. One gives forward secrecy; the other adds self-healing.

27.4.1 Ratchet one: the symmetric-key (hash) ratchet

The simpler gear is a **KDF chain**. You hold a 32-byte *chain key* CK_i. To advance, you apply a keyed derivation – in Signal, HMAC with two fixed constants – to split CK_i into the next chain key

and a message key:

$$CK_{i+1} = \text{HMAC}(CK_i, \texttt{0x02}), \qquad MK_i = \text{HMAC}(CK_i, \texttt{0x01}).$$

The message key MK_i encrypts message i (via AES + HMAC authenticated encryption). The chain key CK_{i+1} is carried forward to make the next pair. Then you *delete* CK_i and MK_i.

Why it works

Why this gives forward secrecy. HMAC is a pseudorandom function (Chapter 15); from its output you cannot recover its input. So given CK_{i+1}, an attacker cannot compute CK_i – the chain runs only forward. After Alice encrypts message i, she deletes CK_i and MK_i, keeping only CK_{i+1}. If her phone is seized after message i, the attacker finds CK_{i+1} and can derive $MK_{i+1}, MK_{i+2}, \ldots$ (future messages on this chain) – but $MK_i, MK_{i-1}, \ldots$ are unrecoverable, because the chain keys that produced them are gone and cannot be run backward. Every message Alice already sent stays secret.

Common trap

The symmetric ratchet alone does *not* self-heal – it has the exact weakness of the "hash the key forward" non-fix from Section 27.2. If the attacker grabs CK_{i+1}, she derives every future message key on that chain forever. Forward secrecy: yes. Post-compromise security: no. That is why a second ratchet exists. The symmetric ratchet handles messages sent in a row cheaply; the DH ratchet, next, periodically recovers from a past compromise.

27.4.2 Ratchet two: the Diffie-Hellman ratchet

Here is the self-healing gear. Alice and Bob each maintain a current *ratchet* DH key pair. Every message header carries the sender's current ratchet *public* key. The rule: *whenever you receive a new ratchet public key from the other side, you generate a brand-new ratchet key pair of your own and perform a fresh Diffie-Hellman*, mixing the result into the chain.

Concretely, the DH output feeds a *root chain*. There is a long-lived *root key* RK. When a new DH value DH_{out} arrives, you ratchet the root:

$$(RK', CK') = \text{HKDF}\big(RK,\ DH_{\text{out}}\big),$$

producing a new root key RK' and a new *sending* (or receiving) chain key CK'. That CK' seeds a fresh symmetric ratchet. So the picture is a tree of chains: the root chain steps forward each time the conversation changes direction and injects new DH randomness; each step spits out a chain key that drives a symmetric ratchet of per-message keys.

Why it works

Why this gives post-compromise security. Suppose the attacker steals all of Bob's state right after message i – including his current ratchet private key and chain keys. She can read messages for a while. But the next time Alice sends *her* current ratchet public key and Bob replies, Bob generates a *fresh* ratchet key pair whose private half the attacker never saw. The new DH output DH_{out} depends on this fresh private key, so the attacker cannot compute it. The root key ratchets to an RK' she cannot derive, the chain keys downstream of it are

unknown to her, and she is locked out. The session healed itself with no human intervention – just the normal back-and-forth of conversation injecting entropy she could not predict. One full DH round trip after the compromise ends, secrecy is restored.

27.4.3 Putting both gears together

The two ratchets interlock (Figure 27.2). Read the diagram top to bottom for the root chain, left to right for each symmetric chain.

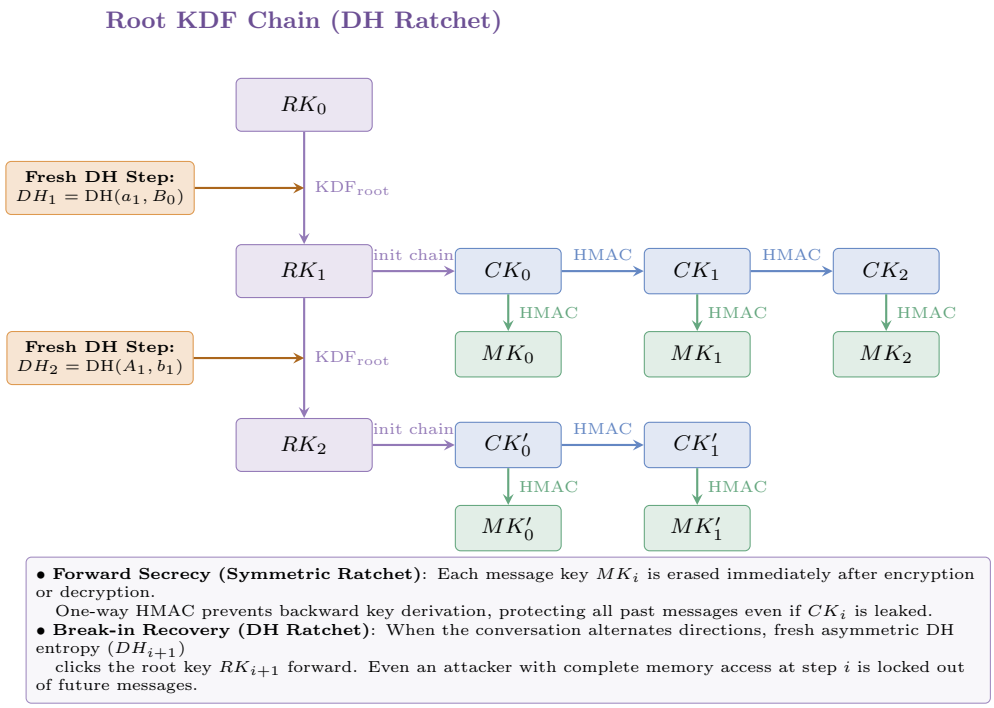


Figure 27.2: The Signal Double Ratchet algorithm. The vertical root chain executes an asymmetric Diffie–Hellman ratchet whenever conversational turns alternate, injecting new entropy to self-heal from state compromise. The horizontal symmetric KDF chains advance per-message via HMAC, yielding ephemeral keys MK_i with forward secrecy.

The *vertical* root chain is the DH ratchet: each new DH value clicks it forward and self-heals. The *horizontal* symmetric chains are the cheap per-message ratchets that give forward secrecy within a run of messages. A run of messages in one direction walks one horizontal chain; the moment the conversation turns around and a fresh ratchet key gets exchanged, the root steps down and a new horizontal chain begins.

Focus

When does each gear turn?

- Every message you *send in a row* (same direction, no reply yet) advances the *symmetric* ratchet only: cheap HMAC steps, new message key each time.

- Every time the conversation *changes direction* – you receive after sending, or send after receiving – you do a *DH ratchet* step: generate a new ratchet key pair, DH against the partner's latest ratchet public key, ratchet the root, and start a fresh symmetric chain.

So a rapid-fire monologue is cheap (HMAC only), and a back-and-forth dialogue triggers frequent DH steps – exactly when you most want the self-healing, because that is when both parties are active and entropy is flowing.

27.4.4 Tracing the ratchet with tiny numbers

Let us walk a few steps. To keep it by hand, model the symmetric ratchet's HMAC as a toy function $f(k, c) = (3k + c) \bmod 97$ – *not* cryptographic, just a deterministic mixer that lets us see the chain move. Start a sending chain with $CK_0 = 10$. Using the constants `0x01` $= 1$ for message keys and `0x02` $= 2$ for chain keys:

$$\begin{aligned} MK_0 &= f(10, 1) = (30 + 1) \bmod 97 = 31, & CK_1 &= f(10, 2) = (30 + 2) \bmod 97 = 32, \\ MK_1 &= f(32, 1) = (96 + 1) \bmod 97 = 0, & CK_2 &= f(32, 2) = (96 + 2) \bmod 97 = 1, \\ MK_2 &= f(1, 1) = 4, & CK_3 &= f(1, 2) = 5. \end{aligned}$$

Alice encrypts message 0 under $MK_0 = 31$, deletes CK_0 and MK_0, keeps $CK_1 = 32$. Encrypts message 1 under $MK_1 = 0$, deletes, keeps $CK_2 = 1$. And so on. Now suppose her phone is seized right after message 1, leaking $CK_2 = 1$. The attacker computes $MK_2 = f(1, 1) = 4$ and reads message 2 – but to get MK_1 she would need $CK_1 = 32$, and from $CK_2 = 1$ she cannot run f backward (with a real HMAC; our toy f is invertible, which is exactly why f must be a one-way PRF in reality). Messages 0 and 1 stay sealed.

Common trap

The toy $f(k, c) = 3k + c \bmod 97$ is *linear and invertible* – from CK_2 you can solve $CK_1 = (CK_2 - 2) \cdot 3^{-1} \bmod 97$ and walk the chain backward, destroying forward secrecy. We use it only to make the arithmetic visible. The real ratchet uses HMAC-SHA256, a one-way PRF for which no such inverse exists. The lesson the toy teaches is the *structure* (split each chain key into a message key and a next chain key, delete as you go); the *security* comes entirely from the underlying function being non-invertible.

27.4.5 Tracing the DH ratchet self-heal

The symmetric trace above showed forward secrecy but, as warned, no self-healing: leak CK_2 and every future message key on that chain falls. Now watch the *root* ratchet close that exposure. We keep the toy group from Section 27.3.4 ($p = 23$, $g = 5$, exponents mod 22) for the Diffie-Hellman, and a second toy mixer $r(\rho, d) = (5\rho + 7d) \bmod 97$ standing in for the root HKDF – it eats the old root value ρ and a DH output d and emits the next root value, from which we peel a chain seed.

Suppose the session is running with root value $RK_1 = 40$ and Alice currently holds ratchet key pair $a_1 = 4$ (so her public ratchet key is $5^4 \equiv 4 \bmod 23$), while Bob's latest ratchet public key on file is $B_1 = 5^6 \equiv 8$. The attacker compromises Bob right after message 10 and learns $RK_1 = 40$, Bob's ratchet private $b_1 = 6$, and the live chain keys. She can read messages on the current chain.

Now the conversation turns around. Bob, replying, generates a *fresh* ratchet pair $b_2 = 9$ (public $5^9 \equiv 11$) – a private scalar the attacker never saw. Alice receives $B_2 = 11$ and does a DH ratchet step against it with her current private $a_1 = 4$:

$$DH_{\text{out}} = (B_2)^{a_1} = 11^4 = g^{9 \cdot 4} = g^{36} = g^{36 \bmod 22} = g^{14} = 5^{14} \equiv 13 \pmod{23}.$$

The root steps to $RK_2 = r(40, 13) = (5 \cdot 40 + 7 \cdot 13) \bmod 97 = (200 + 91) \bmod 97 = 291 \bmod 97 = 0$, and that $RK_2 = 0$ seeds a brand-new symmetric chain for the new direction. The attacker, knowing $RK_1 = 40$, still cannot reach RK_2: she would need $DH_{\text{out}} = (B_2)^{a_1}$, and although she sees both public values $B_2 = 11$ and $A_1 = 4$ on the wire, recovering the secret from them is the discrete-log problem. With the real X25519 group that is infeasible; her stolen $b_1 = 6$ is useless because Bob retired it. From message 11 onward she is locked out – the session healed itself with one DH round trip's worth of fresh entropy.

Walking question

Why could the attacker not just keep reading? She holds RK_1, Bob's *old* private b_1, and the old chain keys. But the new chain hangs off $RK_2 = r(RK_1, DH_{\text{out}})$, and $DH_{\text{out}} = a_1 \cdot b_2 \cdot G$ depends on Bob's *new* private b_2, which she never captured. She knows RK_1 – the first argument to r – yet the second argument is exactly the value she cannot compute. One unknown input to a one-way mixer is enough: RK_2 is opaque, and every chain key below it with it. The lazy variant of Exercise 27.10 that drops the old RK would still self-heal *here* (the fresh DH output is enough), but mixing RK in additionally guards against an attacker who learns a single DH_{out}; see that exercise for the case it saves.

27.4.6 Out-of-order and lost messages

Networks reorder and drop messages, so Bob may receive message 5 before message 3. Each message header carries its chain step number i. If Bob is expecting step 3 and sees step 5, he advances his receiving chain from CK_3 to CK_5, derives MK_3, MK_4 along the way, and *stores* those skipped message keys in case 3 and 4 arrive later. He decrypts 5 immediately with MK_5. When 3 and 4 finally show up, he pulls their stored keys, decrypts, and deletes them.

Common trap

Storing skipped message keys is a forward-secrecy hazard and a denial-of-service hazard. A held key is a key that has not been deleted, so it can be stolen. And an attacker could send a header claiming step number 2^{31}, trying to make Bob derive and store two billion keys, exhausting his memory. Real implementations cap the number of skipped keys (Signal uses a few thousand) and the maximum chain-skip per step, refusing messages that would blow the limit. Forward secrecy is in tension with reliability here, and the cap is a compromise between the two.

27.5 A Toy Double Ratchet in Code

The following Python sketches the symmetric ratchet and the root-chain DH ratchet using real HMAC-SHA256 but a deliberately trivial "DH" (integer multiplication mod a small prime) so the

whole thing runs and prints. It shows the *shape*: chain keys split into message keys, root steps on each DH, fresh keys per message.

Implement it

Learning toy only – do not use this for anything real. It omits authentication of headers, X3DH, the AEAD that actually encrypts payloads, replay protection, skipped-key handling, constant-time comparison, and uses a joke "DH." Real code: use the `libsignal` library, never a hand-roll.

```
import hmac, hashlib

def kdf(key, info):
    return hmac.new(key, info, hashlib.sha256).digest()

# --- symmetric (chain) ratchet ---
def chain_step(ck):
    mk = kdf(ck, b'\x01')        # message key for THIS message
    ck_next = kdf(ck, b'\x02')   # carry forward
    return ck_next, mk

# --- root ratchet: mix a DH output into the root key ---
def root_step(rk, dh_out):
    out = kdf(rk, dh_out)                  # 32 bytes
    new_rk, new_ck = out[:16], out[16:]    # split
    return new_rk, new_ck

# joke 'DH' so the demo runs (NOT secure): shared = (priv_a * pub_b) mod p
P = 0x7fffffffffffffe7
def dh(priv, pub):
    return ((priv * pub) % P).to_bytes(16, 'big')

# --- demo: Alice sends three messages on one chain ---
rk = b'root-secret-from-x3dh-0000000000'   # 32 bytes, pretend SK
a_priv = 1234567
b_pub  = 7654321
rk, ck = root_step(rk, dh(a_priv, b_pub))  # DH ratchet once

for i in range(3):
    ck, mk = chain_step(ck)
    print(f"msg {i}: message key = {mk.hex()[:16]}...")
    # encrypt(plaintext, mk) would go here; then delete mk, old ck

# A new ratchet key from Bob would trigger another root_step(rk, dh(...)),
# starting a fresh chain -- that step is what heals a past compromise.
```

Run it and you will see three distinct message keys from one chain key, each unrecoverable from the next. Swap the joke `dh` for X25519, add a real AEAD, and handle headers, and you are approaching the real protocol's structure – but please do not; use `libsignal`.

27.6 Sealed Sender and the Metadata Problem

Content is now safe end-to-end. But the server still sees the *envelope*: who messaged whom, when, how often, how large. That **metadata** is enormously revealing – arguably more than content. "A journalist messaged a known source at 2am, then messaged a lawyer" tells a story without a single plaintext byte. As one former intelligence director put it bluntly, governments "kill people based on metadata."

Signal's design minimizes the metadata the server holds. It keeps no contact graph, no message logs, no group membership it can read; subpoenas have repeatedly produced little more than the account creation date and last-connection date, because that is all there is. One concrete mechanism is worth detailing.

Focus

Sealed sender. Normally a message to Bob must tell the server "this is for Bob, from Alice," so the server learns the sender of every message. Sealed sender hides the *sender*. Alice encrypts a little certificate of her identity *inside* the end-to-end ciphertext, so only Bob – after decrypting – learns who sent it. To the server, the message arrives with the recipient (it has to, for delivery) but *no* authenticated sender. The server cannot build a who-talks-to-whom graph.
To stop anyone from spamming Bob with anonymous sealed-sender messages, Bob hands out short-lived *delivery tokens* to his contacts; the server checks a token (without learning who holds it) before accepting an anonymous message. Authenticated to Bob, anonymous to the server.

Common trap

Sealed sender hides the sender's identity from the server but *not* traffic-analysis signals. The server still sees Bob's IP receiving a packet of a certain size at a certain time, and Alice's IP sending one moments earlier. With timing correlation, a global passive adversary can often re-link the two. Sealed sender raises the cost and defeats casual logging; it is not anonymity against an adversary watching the whole network. Real anonymity needs onion routing (Tor) on top, which Signal does not provide by default. Do not oversell metadata protection.

In the real world

Who runs the Signal Protocol. The Double Ratchet and X3DH (or its successor, PQXDH, which adds a post-quantum KEM alongside X25519 – see Chapter 35) are not academic. They secure: Signal itself; **WhatsApp** (every chat, 2 billion users, since 2016); **Google Messages** RCS end-to-end mode; **Facebook Messenger** and **Instagram** DMs (now default E2EE); and **Skype**'s private conversations. The protocol is also a formal-verification success story: researchers (Cohn-Gordon, Cremers, Dowling, Garratt, Stebila, 2017) produced a machine-checked analysis of its key-secrecy and post-compromise guarantees, a rare thing for a deployed protocol. This combination of wide deployment *and* formal analysis is why it is treated as the reference design for secure messaging.

27.7 Authentication, Deniability, and the Safety Number

Two subtle properties round out the design: how Alice knows she is really talking to Bob, and why – despite that – neither can prove to a court what the other said.

27.7.1 Verifying you have the right Bob

X3DH authenticates the session against Bob's *identity key*, but how does Alice know that identity key truly belongs to her friend and not to the server impersonating him? The server hands out the bundle, after all – a malicious server could substitute its own identity key, run a person-in-the-middle, and relay. This is the same trust-the-directory problem that certificates solve for TLS (Chapter 24), but Signal has no certificate authorities. Instead it uses **safety numbers**: a short fingerprint derived from *both* parties' identity keys, displayed as a 60-digit number (or a QR code). Alice and Bob compare it out of band – read it aloud on a phone call, scan the QR in person. If the numbers match, no person-in-the-middle sits between them, because the attacker's substituted key would produce a different fingerprint.

Attack

The unverified-key window. Most users never compare safety numbers. Until they do, a malicious or compromised server *could* have substituted identity keys at session setup and be reading everything – a classic person-in-the-middle. Signal mitigates this with a visible warning: "Your safety number with Bob has changed." If the server swaps Bob's key mid-conversation, Alice's app flags it, prompting the out-of-band check. The attack is detectable but not prevented by cryptography alone; it relies on users acting on the warning. This is the residual trust in any E2EE system without a verified public-key infrastructure – the cryptography is sound, but key *distribution* remains the weak point.

27.7.2 Deniability: authentic now, deniable later

This property surprises many people. When Bob receives Alice's message, he is cryptographically convinced it came from Alice – the X3DH secret could only be computed by someone holding Alice's identity private key. Yet Bob *cannot* take that ciphertext to a journalist and prove Alice wrote it. How can it be both authenticated and deniable?

Why it works

Why deniability holds. Authentication in X3DH comes from a *shared* Diffie-Hellman secret, not a *signature*. A digital signature (Chapter 22) is non-repudiable on purpose: only Alice's private key produces it, so it proves *to anyone* that Alice signed. A shared DH secret is different – it can be computed by *either* party. The MAC that authenticates a Signal message is keyed by a secret that Bob also knows, because he is a participant in the DH. So Bob could have forged *any* message "from Alice" to himself: he had the shared key, and a valid MAC under a key both parties hold proves only that *one of the two of them* made it. To an outside judge, a transcript Bob presents is indistinguishable from one Bob fabricated. Bob is convinced in the moment (he knows *he* did not forge it), but he cannot transfer that conviction

to a third party.

Intuition

The split is between *interactive* and *transferable* authentication. A wax seal you watch someone press convinces you, standing there; a photo of the seal convinces no one, because photos are forgeable. Signal gives Bob the standing-there conviction – the live cryptographic check – without giving him a transferable artifact. This is deliberate. A whistleblower wants their source's app to confirm the source is genuine, yet wants no court-admissible proof tying the source to the words. Off-the-Record Messaging introduced this goal, and Signal inherits it.

Connection

Deniability is the mirror image of digital signatures (Chapter 22). Signatures buy *non-repudiation*: the signer cannot deny it, which is what you want for contracts, software updates, and certificates. Messaging wants the opposite. The same DH-versus-signature distinction explains why TLS certificates use signatures (a CA must be held accountable for what it vouches) while Signal authenticates with shared secrets (no participant should be held accountable to outsiders for private words).

27.8 Putting It All Together

Trace one full session from cold start, naming each property as it is earned:

1. **Setup (offline).** Bob uploads a pre-key bundle: identity key, signed pre-key, one-time pre-keys. He goes offline.
2. **Session start (X3DH).** Alice fetches the bundle, verifies the signature on the signed pre-key, runs four Diffie-Hellmans, and derives SK. *Authentication* (her identity and Bob's are both mixed in) and initial *forward secrecy* (ephemeral keys deleted) are earned here. She sends her first message asynchronously.
3. **Ratchet init.** SK seeds the root key. The first DH ratchet step produces the first sending chain.
4. **Sending a run.** Each message advances the *symmetric* ratchet: a fresh message key per message, old keys deleted – *forward secrecy* per message.
5. **Turn-taking.** When the conversation changes direction, a new ratchet key is exchanged and a *DH ratchet* step mixes fresh entropy into the root – *post-compromise security*, the self-healing.
6. **Metadata minimization.** Sealed sender hides who sent what from the server; Signal stores almost nothing.
7. **Verification.** Out-of-band safety-number comparison closes the person-in-the-middle window. Throughout, authentication rests on shared DH secrets, so the transcript stays *deniable*.

Every guarantee from Section 27.1 is delivered by a specific, nameable mechanism. This direct mapping from goal to mechanism, together with asynchronous operation and machine-checked proofs, is why most other secure messengers have adopted the Signal Protocol's design.

Why this matters

Signal requires very little machinery. Two familiar primitives – Diffie-Hellman key agreement and a hash-based KDF – are arranged in a loop to achieve guarantees neither provides alone: per-message forward secrecy from the one-way hash chain, post-compromise recovery from fresh Diffie-Hellman shares, asynchronous setup from pre-published keys, and deniability from shared-secret authentication. Nothing here relies on a new hardness assumption or an exotic primitive; all four properties come from the order in which standard operations run and when old keys are erased.

Signal solves end-to-end messaging between people whose phones are only intermittently online. In Chapter 28, we turn to a live, interactive setting — an engineer logging into a remote server — and see how the Secure Shell (SSH) protocol uses host keys, trust-on-first-use (TOFU), and forwarding tunnels to protect administrative access.

27.9 Exercises

Exercise 27.1 (X3DH by hand)**.** Using the toy group of Section 27.3.4 (DH mod $p = 23$, $g = 5$, $\mathrm{ord}(g) = 22$), let Alice have $ik_A = 5$, $ek_A = 8$ and Bob have $ik_B = 9$, $spk_B = 2$, $opk_B = 6$. Compute all five public keys, then the four DH values $DH_1, \ldots, DH_4$ as powers of g, and reduce each to an element of $(\mathbb{Z}/23\mathbb{Z})^*$. Verify that Bob, computing from his side, gets the same four values.

Exercise 27.2 (Forward secrecy of the symmetric ratchet)**.** Using the toy mixer $f(k, c) = (3k + c) \bmod 97$ with $CK_0 = 20$, compute MK_0, MK_1, MK_2 and CK_1, CK_2, CK_3. Then show that this toy f *breaks* forward secrecy by exhibiting the formula that recovers CK_1 from CK_2. Explain in one sentence what property a real ratchet function must have that f lacks.

Exercise 27.3 (Why both ratchets)**.** Describe a concrete attack scenario in which a messenger with *only* the symmetric ratchet (no DH ratchet) leaks all future messages after a single compromise, and explain precisely which step of the Double Ratchet would have stopped it. Then describe a scenario where a messenger with *only* a DH ratchet (no per-message symmetric chain) is inefficient, and say why.

Exercise 27.4 (Post-compromise timing)**.** An attacker fully compromises Bob's device (all keys) immediately after Bob sends message 10. Alice and Bob then exchange messages 11 (Alice), 12 (Bob), 13 (Alice). For each of messages 11, 12, 13, state whether the attacker can decrypt it, and justify your answer in terms of which ratchet steps inject a private key the attacker never saw.

Exercise 27.5 (Skipped-key denial of service)**.** Bob caps stored skipped message keys at $N = 2000$. Describe how an attacker who can inject one forged message header could try to exhaust Bob's memory, what field of the header she manipulates, and exactly what check Bob must perform to refuse the message. Why can Bob not simply remove the cap?

Exercise 27.6 (Deniability versus signatures)**.** Explain why authenticating a Signal message with a shared Diffie-Hellman secret yields deniability, whereas authenticating it with a digital signature (Chapter 22) would destroy deniability. In your answer, state precisely what a third party can and cannot conclude from a MAC tag whose key both Alice and Bob know.

Exercise 27.7 (Person-in-the-middle on setup)**.** A malicious server substitutes its own identity key for Bob's in the pre-key bundle it gives Alice, and relays a separately negotiated session to the

real Bob. Describe the full person-in-the-middle the server must run to read Alice-to-Bob traffic, which keys it must substitute on each leg, and the exact user action that detects it. Why does the cryptography alone not prevent this attack?

Exercise 27.8 (Sealed sender limits)**.** Sealed sender removes the authenticated sender from what the server sees. Construct a scenario in which a global passive adversary, watching all network links, re-identifies the sender of a sealed-sender message anyway. What property must hold of message timing and sizes for this correlation to fail, and what additional system (not part of Signal by default) would be needed?

Exercise 27.9 (Designing a group ratchet)**.** The Double Ratchet is defined for two parties. Sketch the difficulty in extending it to a group of n members so that forward secrecy and post-compromise security still hold, and explain why naively running $\binom{n}{2}$ pairwise Double Ratchets is correct but expensive. (This is the problem the "Sender Keys" design and the newer Messaging Layer Security standard, MLS, attack – look up how MLS uses a tree of keys to get $O(\log n)$ updates.)

Exercise 27.10 (Root-chain HKDF)**.** In the DH ratchet, the root step computes $(RK', CK') = \mathrm{HKDF}(RK, DH_{\mathrm{out}})$. Suppose a lazy implementer instead set $RK' = DH_{\mathrm{out}}$ directly, discarding the old root key. Show that an attacker who learns one DH_{out} (say by compromising one ephemeral key) can then compute all subsequent root keys if she also observes the DH public values on the wire. Which security property is lost, and why does mixing the *old* RK into the HKDF prevent it?

Chapter 28

SSH and Secure Remote Access

In Chapter 27, we saw how secure messaging protects asynchronous conversations between people across untrusted relays. Remote system administration poses a related problem: protecting a live, interactive session between an engineer and a machine across an untrusted network. SSH, the *Secure Shell*, brings together nearly every primitive we have built so far – Diffie-Hellman, RSA, elliptic curves, authenticated encryption, hashes, signatures, and certificates – to answer one practical question: *how do I get a trustworthy command line on a machine that is across the room, across the building, or across an ocean, over a network I do not control?*

When you type `ssh alice@server.example.com` and a moment later you are looking at a shell prompt on a machine three thousand miles away, an enormous amount of cryptography has just happened in under a second. A fresh key was negotiated over a hostile network. The remote machine proved it is the one you meant to reach. You proved you are allowed in. And every keystroke from that point forward is encrypted and authenticated, immune to a passive eavesdropper and – if you set it up right – to an active attacker sitting in the middle. This chapter takes that second apart.

We will meet Alice (you, at your laptop), the server she wants to reach, and Eve, who controls the network in between – she can read every packet, drop packets, inject packets, and impersonate either endpoint. Our job is to understand exactly what SSH does to defeat her, where the math comes from, and the fragile moment where SSH quietly hands the security decision back to a human and how often that human gets it wrong.

28.1 What problem SSH solves

History

Before SSH there was `telnet`, `rlogin`, `rsh`, and `rcp`. All of them sent your password, and then your entire session, in cleartext. On a shared Ethernet segment or a compromised router, anyone running a packet sniffer harvested credentials for free. In 1995 Tatu Ylonen, a researcher at Helsinki University of Technology, watched exactly such a password-sniffing attack run against his network and wrote SSH-1 as a direct replacement. It spread quickly. SSH-1 had real cryptographic flaws (including a CRC-32 integrity check, an unkeyed polynomial checksum that enabled plaintext-insertion attacks), so the protocol was redesigned from scratch as

SSH-2, standardized by the IETF in RFCs 4250–4254 (2006). Today "SSH" means SSH-2; SSH-1 is dead and should be refused. The dominant implementation, OpenSSH, grew out of the OpenBSD project and now ships on essentially every server on Earth. The primary open challenge today is migrating the installed base to post-quantum hybrid key exchange (combining X25519 with ML-KEM) before store-now-decrypt-later adversaries record traffic for future quantum decryption.

The threat model is worth stating precisely, because SSH's whole design follows from it.

Focus

Threat model. The network between Alice and the server is fully controlled by Eve (a *Dolev-Yao* attacker). She can:

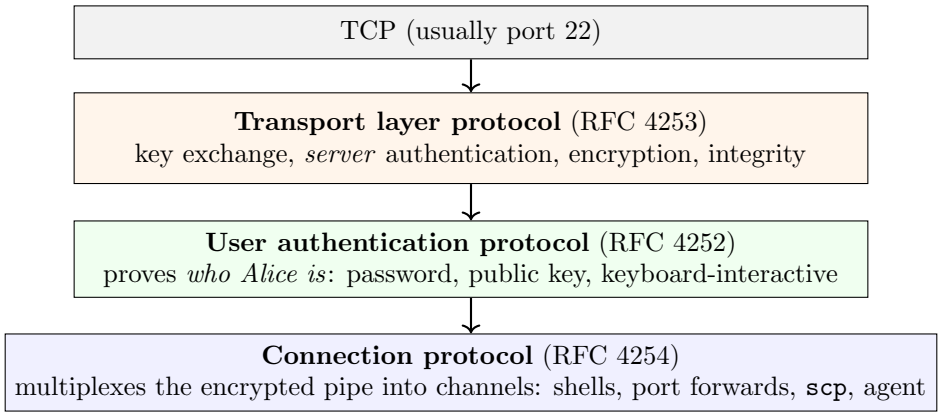
- read every byte that crosses the wire (passive eavesdropping);
- modify, delete, reorder, or replay packets;
- insert packets of her own, including pretending to be the server to Alice and pretending to be Alice to the server (an active man-in-the-middle).

What Eve *cannot* do: break the underlying primitives (factor RSA moduli, compute discrete logs in the chosen group, find AES keys, find hash collisions in a strong hash), and she cannot read the private keys stored on Alice's laptop or the server. The security goals SSH must meet against this Eve are: *confidentiality* (she learns nothing about the session contents), *integrity* (she cannot alter the session undetected), *server authentication* (Alice reaches the machine she intended, not Eve's impostor), and *client authentication* (only authorized users get a shell).

There is an asymmetry in "server authentication." For TLS on the web, a certificate authority vouches for `example.com` so your browser can verify a server it has never seen before. SSH usually has no such authority. That single missing piece – how Alice knows the server is the real server the very first time she connects – is the most interesting and most abused part of the whole system, and we will spend a full section on it.

28.2 The three layers of SSH

SSH-2 is deliberately built as three stacked protocols, each with one job. Keeping them separate is what lets you reason about the system at all.



Read it bottom to top, which is the order events happen:

1. **Transport layer.** Runs first, on the raw TCP connection. It performs the key exchange, authenticates the *server* (via its host key), and from then on encrypts and MAC-protects everything above it. After this layer finishes you have a secure pipe – but the server still has no idea who you are.
2. **User authentication.** Runs *inside* the secure pipe the transport layer built. Now that nobody can eavesdrop, Alice can safely send a password or, far better, prove possession of a private key. This is where *client* authentication lives.
3. **Connection protocol.** Once you are authenticated, this layer slices the single encrypted pipe into multiple logical *channels*. One channel carries your interactive shell; another might forward a TCP port; another tunnels your SSH agent. They share the one key-exchanged, encrypted connection.

Intuition

The layering buys a clean mental model: "first build a private tunnel and check who's at the far end (transport), then walk through it and show your ID (auth), then do as many things as you like through the tunnel (connection)." Each layer assumes the one below it already did its job. The most common SSH security failures happen at the boundary between layers one and two – specifically, Alice accepts the wrong server in layer one and then types her password into Eve's prompt in layer two. An encrypted tunnel without server authentication only gives you a private connection to an impostor.

28.3 The transport layer: key exchange and host keys

28.3.1 Identification and algorithm negotiation

The moment the TCP connection opens, both sides send a banner line, in cleartext, like:

```
SSH-2.0-OpenSSH_9.6
```

The `2.0` pins the protocol version (a `1.x` here would let an old client fall back to broken SSH-1; modern servers simply do not offer it). Then each side sends a `SSH_MSG_KEXINIT` message listing, in

preference order, the algorithms it supports for: key exchange, host-key signature, symmetric cipher (each direction), MAC (each direction), and compression. The two ordered lists are reconciled by a deterministic rule – the chosen algorithm is the client's most-preferred one that the server also supports – so both ends independently arrive at the same choice without extra round trips.

These `KEXINIT` messages are sent before any key exists, so they are unencrypted, which means Eve can read them. That is fine; the algorithm names are not secret. But she could *tamper* with them to force a weak choice (a downgrade attack). SSH defends against this by folding the exact bytes of both `KEXINIT` messages into the key-exchange hash, as we are about to see. If Eve flips a bit, the two sides compute different hashes and the host-key signature fails to verify. The negotiation is unencrypted but not unauthenticated.

28.3.2 The key exchange itself

The classic SSH key exchange is Diffie-Hellman; modern OpenSSH defaults to *Curve25519* (the elliptic-curve DH from Chapter 21) but the structure is identical, so let us write it with plain DH over a prime field $\mathbb{Z}/p\mathbb{Z}$ and a generator g, and you can mentally swap in the curve. The server owns a long-term *host key pair* (pk_H, sk_H) – this is the key that names the server's identity, and remembering it across connections is the subject of the next section.

Formal

SSH Diffie-Hellman key exchange. Let V_C, V_S be the client and server identification strings, and I_C, I_S the two `KEXINIT` payloads.

1. Client picks $x \xleftarrow{\$} \{2, \dots, p-2\}$, computes $e = g^x \bmod p$, sends e.
2. Server picks $y \xleftarrow{\$} \{2, \dots, p-2\}$, computes $f = g^y \bmod p$ and the shared value $K = e^y \bmod p$.
3. Server forms the *exchange hash*

$$H = \mathrm{H}(V_C \,\|\, V_S \,\|\, I_C \,\|\, I_S \,\|\, pk_H \,\|\, e \,\|\, f \,\|\, K)\,,$$

   signs it: $s = \mathrm{Sign}_{sk_H}(H)$, and sends $(pk_H,\ f,\ s)$.
4. Client computes $K = f^x \bmod p$, reconstructs the *same* H from the values it already has, and verifies $\mathrm{Vrfy}_{pk_H}(H, s)$. If the signature checks out, the server is authenticated.

The shared secret is K because $e^y = g^{xy} = f^x \pmod p$. The session key material is derived from H and K by hashing, described below. The exchange hash H doubles as the *session identifier* for the life of the connection.

Why it works

Why is this man-in-the-middle proof? Plain Diffie-Hellman is *not* – Eve can run one DH with the client and another with the server, sitting in the middle and reading everything. The single ingredient that fixes this is the server's signature over H. Because H commits to f (the server's DH share) *and* to pk_H *and* to both `KEXINIT` payloads, a valid signature under the host key proves: "the entity that owns sk_H chose this DH share f, in the context of these negotiated algorithms." If Eve substitutes her own $f' = g^{y'}$ to set up her half of the tunnel, she would

have to produce a signature on the corresponding H' under sk_H – which she does not have. Her only option is to present her *own* host key pk'_H and sign with it. That signature verifies, and the protocol cannot stop her; the defense moves to the human: *does Alice recognize that host key?* That is trust-on-first-use, and it is the subject of the next section.

Connection

The security of K against a passive Eve is exactly the *computational Diffie-Hellman* assumption from Chapter 18: given g^x and g^y she cannot compute g^{xy}. With Curve25519 the group is the curve's point group of prime order $\approx 2^{252}$, and the best known attack is generic discrete log at cost $\approx 2^{126}$ (Pollard rho, from the algorithms companion). Folding K through a hash to make keys means even the *decisional* DH leakage is laundered into uniform-looking bits. Why hash rather than use K directly? Raw DH outputs are not uniform – $g^{xy} \bmod p$ lives in a subgroup and has detectable bias – and a single K must yield several independent keys; hashing fixes both, exactly as the KDF discussion in Chapter 23 argued.

28.3.3 From one secret to six keys

A session needs more than one key. Each direction (client-to-server and server-to-client) needs an encryption key, a MAC key (when a separate MAC is used), and an initialization vector. SSH derives all of them from the pair (K, H) by repeated hashing, where the letters `A`–`F` are literal ASCII tags:

$$\begin{aligned}
\text{IV}_{C\to S} &= \text{H}(K \,\|\, H \,\|\, \texttt{"A"} \,\|\, \mathit{session_id}) \\
\text{IV}_{S\to C} &= \text{H}(K \,\|\, H \,\|\, \texttt{"B"} \,\|\, \mathit{session_id}) \\
\text{Enckey}_{C\to S} &= \text{H}(K \,\|\, H \,\|\, \texttt{"C"} \,\|\, \mathit{session_id}) \\
\text{Enckey}_{S\to C} &= \text{H}(K \,\|\, H \,\|\, \texttt{"D"} \,\|\, \mathit{session_id}) \\
\text{MACkey}_{C\to S} &= \text{H}(K \,\|\, H \,\|\, \texttt{"E"} \,\|\, \mathit{session_id}) \\
\text{MACkey}_{S\to C} &= \text{H}(K \,\|\, H \,\|\, \texttt{"F"} \,\|\, \mathit{session_id})
\end{aligned}$$

On the very first key exchange, $\mathit{session_id} = H$. If a key is longer than one hash output, SSH extends it by hashing $K \,\|\, H$ concatenated with all key bytes produced so far – a simple counter-free expansion. The distinct tags `A`–`F` guarantee the six outputs are independent even though they all derive from the same (K, H): flipping one tag byte changes the hash input completely. Figure 28.1 shows how the shared DH secret K and the exchange hash H fan out into the six directional keys and IVs.

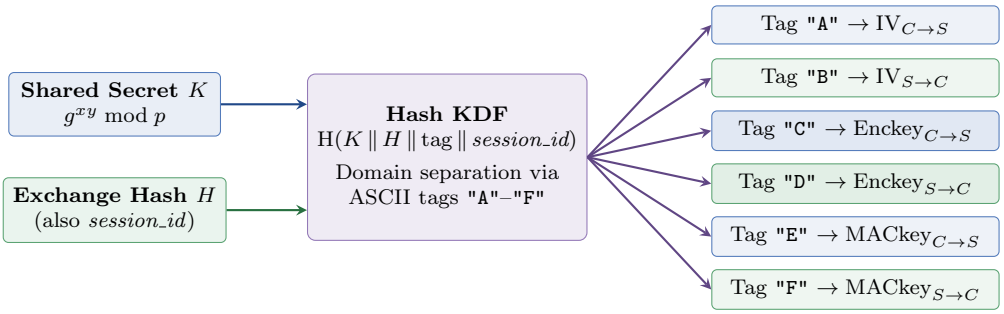


Figure 28.1: SSH key derivation pipeline. A single Diffie-Hellman shared secret K and exchange hash H are domain-separated by single-byte ASCII tags (`"A"` through `"F"`) to produce six independent cryptographic keys and IVs – three for client-to-server traffic and three for server-to-client traffic.

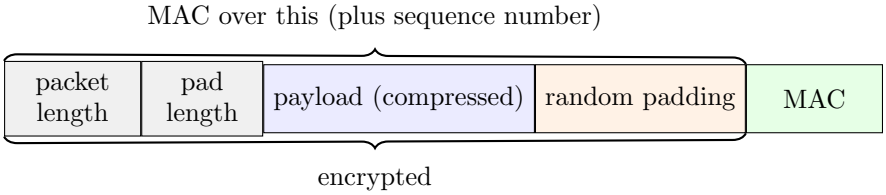
Common trap

Using *different* keys per direction is a real requirement. If the client-to-server and server-to-client streams shared a key and IV, an attacker could reflect a packet back at its sender and, with some cipher modes, learn or forge content. Separate keys per direction also stop trivial replay of a captured packet into the opposite stream. This per-direction, per-purpose key separation is a pattern you will see in TLS, IPsec, and Signal – never reuse a key for two jobs.

Once both sides have the six keys, the client sends `SSH_MSG_NEWKEYS`, and from the next packet on, every SSH packet is encrypted and integrity-protected. SSH also *rekeys* periodically (after roughly a gigabyte of traffic or an hour, by default) by re-running the key exchange inside the existing encrypted channel, which limits how much data ever rides on a single key and gives a measure of forward secrecy across the rekey boundary.

28.3.4 The Binary Packet Protocol: encrypt and MAC

Every byte above the transport layer is wrapped in a *binary packet*:



The padding (4–255 random bytes) rounds the packet up to a cipher-block multiple and hides the exact payload length. The MAC is computed over an implicit 32-bit *packet sequence number* concatenated with the unencrypted packet – the sequence number is never transmitted, it is just a counter both sides keep, and including it is what makes each packet's MAC unique and prevents replay and reordering. If Eve drops or duplicates a packet, the sequence numbers desynchronize and the next MAC fails.

Attack

Encrypt-and-MAC and the 2009 plaintext-recovery attack. Classic SSH used *Encrypt-and-MAC*: it MACs the plaintext and encrypts separately, and – worse – the *packet length field itself was encrypted with CBC mode*, so the receiver had to decrypt the first block just to learn how many more bytes to read. Albrecht, Paterson, and Watson (2009) showed this leaks: by injecting a captured ciphertext block and watching whether the connection produced a length error or a MAC error, and how many bytes it waited for, an attacker could recover 14 to 32 bits of an underlying plaintext block per connection. The fixes that shipped: prefer *CTR* mode (no block-boundary length dependence), prefer dedicated AEAD constructions, and add Encrypt-then-MAC variants (`*-etm@openssh.com`). Today OpenSSH defaults to `chacha20-poly1305@openssh.com` (an AEAD that even encrypts the length field with a separate key) or AES-GCM. The lesson generalizes Chapter 16: *Encrypt-and-MAC is the fragile composition; prefer Encrypt-then-MAC or a real AEAD.*

Walking question

Trace a tiny exchange. Take the toy group $\mathbb{Z}/23\mathbb{Z}$ with generator $g = 5$. Suppose the client draws $x = 6$ and the server draws $y = 15$.

- Client sends $e = 5^6 \bmod 23$. Compute: $5^2 = 25 \equiv 2$, $5^3 \equiv 10$, $5^6 = (5^3)^2 = 100 \equiv 100 - 4 \cdot 23 = 8$. So $e = 8$.
- Server sends $f = 5^{15} \bmod 23$. Compute: $5^{15} = (5^3)^5 = 10^5 \bmod 23$. $10^2 = 100 \equiv 8$, $10^4 \equiv 8^2 = 64 \equiv 64 - 46 = 18$, $10^5 \equiv 18 \cdot 10 = 180 \equiv 180 - 7 \cdot 23 = 19$. So $f = 19$.
- Server's secret $K = e^y = 8^{15} \bmod 23$. Client's secret $K = f^x = 19^6 \bmod 23$. Check they match: $8^{15} = (g^x)^y = g^{xy} = g^{90}$, and $19^6 = (g^y)^x = g^{90}$ too. Reduce g^{90}: the order of 5 mod 23 divides 22; $5^{22} \equiv 1$, so $g^{90} = g^{90 \bmod 22} = g^2 = 25 \equiv 2$. Both sides get $K = 2$.

Now the server hashes $H = \mathrm{H}(\ldots \| pk_H \| 8 \| 19 \| 2)$ and signs it. Eve, watching $e = 8$ and $f = 19$ on the wire, faces the discrete-log problem $5^x \equiv 8$, which for these toy numbers she can brute force ($x = 6$) but for a 252-bit curve she cannot. The signature over H is what stops her from swapping $f = 19$ for her own value.

28.4 Host keys and trust on first use

This is where SSH differs most from TLS, and where users most often go wrong.

The server's host key authenticates the server. But where does Alice's laptop learn the *correct* host key? On the web, a certificate authority signs `example.com`'s key and your browser ships with the CA's key built in, so verification is automatic. Plain SSH has no CA. The first time Alice connects to a new server, her client has nothing to check the host key against. So OpenSSH does the only thing it can: it shows her the key's fingerprint and asks.

```
The authenticity of host 'server.example.com (203.0.113.7)' can't be established.
ED25519 key fingerprint is SHA256:Zm9vYmFyYmF6cXV4cXV1eGNvcmdlZ3JhdWx0.
Are you sure you want to continue connecting (yes/no/[fingerprint])?
```

If Alice types `yes`, the client stores the host key in `~/.ssh/known_hosts` and *from then on* verifies

every future connection against that stored copy. This is *Trust On First Use* (TOFU), also called *key continuity*.

Intuition

TOFU trades away first-contact security to gain a strong property afterward. The very first connection is vulnerable: if Eve is in the middle right then, she presents her own host key, Alice (who has nothing to compare against) clicks `yes`, and Eve owns the session. But *every connection after that* is protected, because the real key is now pinned. Eve cannot quietly take over later – if she swaps the key, Alice's client refuses loudly. TOFU bets that the attacker is not present at the precise instant of first contact, and that the window of one connection is small. For most situations that bet pays off; for high-value targets it does not, which is why the alternatives below exist.

Common trap

The fingerprint prompt is a real security decision, and clicking `yes` reflexively defeats its purpose. The fingerprint is a hash of the host key (SHA-256 of the public key in current OpenSSH). To use TOFU correctly, Alice must obtain the genuine fingerprint through an *independent* channel – a provisioning system, a printed runbook, a phone call to the admin, a value baked into the cloud instance metadata – and compare it to what the prompt shows. Nobody does this for their personal servers. Almost everybody should do this for production infrastructure.

Attack

The host key change warning, and why it is scary. If a host's key ever differs from the pinned `known_hosts` entry, OpenSSH refuses to connect with a wall of warning text:

```
WARNING: REMOTE HOST IDENTIFICATION HAS CHANGED!
IT IS POSSIBLE THAT SOMEONE IS DOING SOMETHING NASTY!
Someone could be eavesdropping on you right now (man-in-the-middle attack)!
```

This means one of two things. Benign: the server was rebuilt or reinstalled and legitimately has a new key, or its IP was recycled. Malicious: an active man-in-the-middle is presenting a substitute host key right now. The protocol cannot tell these apart – it just sees a key it did not expect. The dangerous human reaction is to run `ssh-keygen -R hostname` to delete the old entry and reconnect, accepting the new key. If the cause was actually an attacker, Alice just hand-deleted her only evidence and trusted Eve. The correct response is to verify the new fingerprint out of band *before* deleting anything.

28.4.1 Escaping TOFU: certificates and SSHFP

Two mechanisms remove the first-use gamble.

SSH certificates. OpenSSH supports its own (non-X.509) certificate format. An organization creates one *host CA* key pair, keeps the private part offline, and signs each server's host key, producing a host certificate. Every laptop is configured once with a single line in `known_hosts`:

```
@cert-authority *.example.com ssh-ed25519 AAAA...
```

Now when any server in `example.com` presents a host certificate signed by that CA, the client verifies the signature and connects – *no prompt, no TOFU, no per-host pinning.* Clients trust new servers automatically the day they are provisioned. The same machinery signs *user* keys: a user CA issues short-lived certificates (valid for, say, eight hours) tying a public key to a username and an expiry, so the server trusts one CA instead of managing thousands of `authorized_keys` files. This is how large fleets (Facebook, Netflix, and others have written about it publicly) run SSH at scale.

Why it works

SSH certificates convert key continuity into a one-time root of trust, exactly the CA model from Chapter 24 but minus X.509's complexity. The security argument is the signature: a host cert is $\mathrm{Sign}_{sk_{CA}}(pk_H \,\|\, \mathit{principals} \,\|\, \mathit{validity})$. A client holding pk_{CA} verifies it offline. Eve cannot forge a cert without sk_{CA}, which lives offline. Short-lived *user* certs are forward-secure in a practical sense: a stolen user key is worthless once the cert expires, so there is no giant `authorized_keys` file to audit and no revocation list to distribute for the common case. Revocation, when needed, is a published `KRL` (key revocation list) the servers load.

SSHFP DNS records. You can publish a host key's fingerprint in DNS as an `SSHFP` record. With DNSSEC signing that record, a client configured with `VerifyHostKeyDNS=yes` authenticates the host key against signed DNS instead of asking the user. The trust then rests on the DNSSEC chain rather than on a private CA – useful when you control DNS but do not want to run an SSH CA. Figure 28.2 contrasts the vulnerable first-contact window of TOFU against the offline root of trust provided by an SSH host CA.

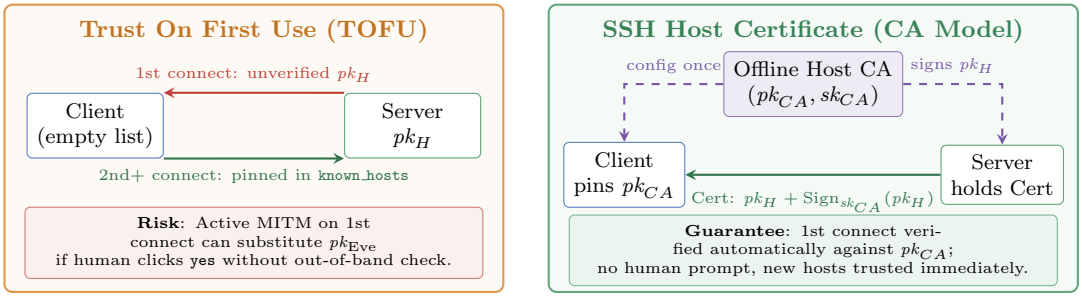


Figure 28.2: Comparing Trust On First Use (TOFU) with SSH host certificates. TOFU leaves the very first connection exposed to human error at the fingerprint prompt, whereas an offline Host CA signs each server's host key so clients pinning pk_{CA} verify even first contact cryptographically.

28.5 User authentication: passwords versus public keys

Once the encrypted, server-authenticated tunnel exists, Alice proves who she is. SSH offers several methods; the two that matter are *password* and *public key.*

28.5.1 Password authentication

The client sends Alice's password to the server through the encrypted tunnel. The server checks it against its user database (hashed with a slow password hash, as Chapter 14 insists). Because the tunnel is encrypted, Eve on the wire learns nothing. So what is wrong with it?

Common trap

Password auth has three weaknesses that public keys do not:

1. **The password reaches the server in usable form.** A malicious or compromised server – or an attacker who fooled Alice past a host-key warning – receives Alice's actual password and can replay it elsewhere (people reuse passwords). With public keys, the server never sees anything reusable.
2. **Online guessing.** The server is exposed to brute-force and credential-stuffing attempts. Internet-facing `sshd` on port 22 sees thousands of login attempts an hour from botnets. Public keys with no password fallback make guessing pointless: there is nothing to guess in a 2^{256} keyspace.
3. **Phishing of the secret.** Because the secret is a human-typed string, it can be socially engineered or keylogged. A private key never leaves the laptop.

The standard hardening is `PasswordAuthentication no` in `sshd_config`, allowing only public keys.

28.5.2 Public-key authentication

This is the right way to use SSH. Alice generates a key pair once. The public key goes into `~/.ssh/authorized_keys` on every server she may log into; the private key never leaves her laptop (ideally never leaves a hardware token). To log in, she proves she holds the private key by *signing a challenge that is bound to this specific session.*

Formal

SSH public-key authentication (publickey method). Let (pk_U, sk_U) be Alice's user key pair and $session_id = H$ the transport-layer exchange hash.

1. Client sends a `SSH_MSG_USERAUTH_REQUEST` naming the username, the service, the method `publickey`, the algorithm, and pk_U, plus a *signature* σ.
2. The signed data is

$$M = session_id \,\|\, \texttt{USERAUTH_REQUEST} \,\|\, username \,\|\, service \\ \|\, \texttt{"publickey"} \,\|\, true \,\|\, alg \,\|\, pk_U,$$

   and $\sigma = \mathrm{Sign}_{sk_U}(M)$.
3. Server checks that pk_U appears in this user's `authorized_keys`, then verifies $\mathrm{Vrfy}_{pk_U}(M, \sigma)$. Both must pass.

Why it works

Two design points make this secure and they are easy to miss. First, the signature covers $session_id = H$, which is unique to this connection (it contains both fresh DH shares). So σ cannot be replayed onto a different connection – a captured login signature is worthless against tomorrow's session, whose H differs. This is *channel binding*: the auth proof is welded to the exact encrypted tunnel it was made in. Second, the server only ever sees a *signature*, never the

private key, so even a fully malicious server learns nothing it can reuse to impersonate Alice elsewhere. Compare password auth, where a malicious server harvests the reusable secret directly. Channel binding plus a non-reusable proof is precisely why public-key auth resists the malicious-server and the man-in-the-middle cases that password auth does not.

Connection

Modern SSH user keys are `ssh-ed25519` – the EdDSA signature scheme over Curve25519 from Chapter 22. A key is 32 bytes; a signature is 64 bytes; verification is a couple of curve operations. Ed25519 is deterministic (no per-signature randomness to leak, sidestepping the nonce-reuse catastrophe that broke ECDSA in that chapter) and fast. RSA keys still work but should be at least 3072 bits, and the old `ssh-dss` (1024-bit DSA) is disabled by default for good reason. Prefer Ed25519; fall back to `ecdsa` or RSA-3072 only when an old server forces it.

Implement it

A stripped-down sketch of the publickey challenge-response, using Ed25519, to show the *shape* of the protocol – not a real SSH implementation.

```
# LEARNING ONLY. Not real SSH. No transport encryption here,
# no packet format, no algorithm negotiation. Do not use.
from nacl.signing import SigningKey, VerifyKey
import os

# Alice generates her user key pair once.
sk = SigningKey.generate()
pk = sk.verify_key                      # goes into authorized_keys

# --- transport layer already ran; both sides share session_id H ---
session_id = os.urandom(32)             # stand-in for the exchange hash

def build_signed_blob(session_id, username, pk):
    # Mirrors the M that real SSH signs: bind to THIS session.
    return (session_id
            + b"USERAUTH_REQUEST"
            + username.encode()
            + b"ssh-connection"
            + b"publickey"
            + b"\x01"                    # "true": this is a real signature
            + b"ssh-ed25519"
            + bytes(pk))

# Client side: sign the session-bound blob.
M   = build_signed_blob(session_id, "alice", pk)
sig = sk.sign(M).signature

# Server side: pk must be in authorized_keys AND the signature must verify.
authorized_keys = {bytes(pk)}           # the server's allowlist
def authenticate(username, offered_pk, sig, session_id):
    if bytes(offered_pk) not in authorized_keys:
```

```
        return False                    # unknown key: reject
    M = build_signed_blob(session_id, username, offered_pk)
    try:
        VerifyKey(bytes(offered_pk)).verify(M, sig)   # raises on bad sig
        return True
    except Exception:
        return False

print(authenticate("alice", pk, sig, session_id))     # True

# Replay onto a DIFFERENT session fails, because M changes with session_id.
print(authenticate("alice", pk, sig, os.urandom(32))) # False
```

The last two lines show the key property: the same signature authenticates only the session it was made for. Real SSH adds the encrypted transport, the binary packet protocol, and careful canonical encoding of every field. Use OpenSSH, never code like this.

28.5.3 Protecting the private key

A private key sitting in `~/.ssh/id_ed25519` is a plaintext credential – anyone who reads that file becomes Alice. Two layers of defense:

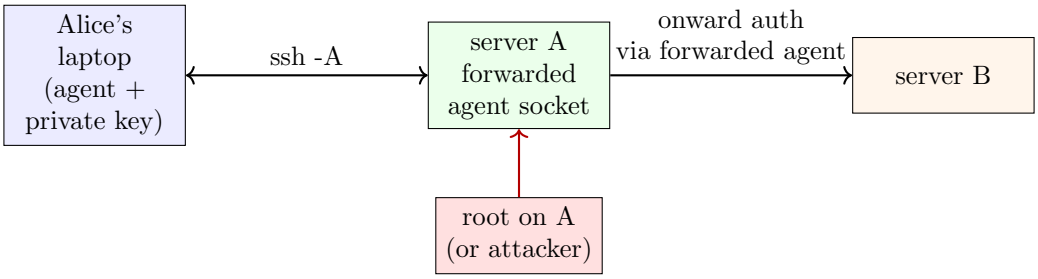
- **Passphrase encryption.** `ssh-keygen` encrypts the private key file under a passphrase, using a slow KDF (OpenSSH's own `bcrypt`-based KDF with a configurable round count) so that a stolen file resists offline cracking. A stolen *encrypted* key still needs the passphrase.
- **Hardware-backed keys.** An Ed25519 key can be generated on a FIDO2/U2F security token (`ssh-keygen -t ed25519-sk`) or stored in a TPM/Secure Enclave. The private key never exists in the laptop's memory; signing happens on the token, often gated by a physical touch. Now even malware with full disk access cannot exfiltrate the key.

28.6 The agent, and the risk of forwarding it

Typing a passphrase on every `ssh`, `scp`, and `git push` is intolerable, so the `ssh-agent` holds decrypted private keys in memory and performs signatures on request. Your terminals talk to the agent over a local Unix socket; you unlock once. The agent never hands out the key – it only ever returns *signatures*, which is the right design.

28.6.1 Agent forwarding

Agent forwarding is where this becomes risky. Alice is logged into server `A` and from there wants to `git pull` from a repository or `ssh` onward to server `B`, both of which require her key. *Agent forwarding* (`ssh -A`) makes the agent socket on her laptop reachable from `A`: processes on `A` can ask Alice's laptop agent to sign things.



Attack

Forwarded agents are a hijackable signing oracle. Whoever can read the forwarded agent socket on `A` – which includes `root` on `A`, and anyone who has compromised `A` – can ask *Alice's* agent to sign *anything* for as long as Alice's session is open. They cannot steal the key, but they do not need to: they can use it. While Alice is connected to `A`, the attacker silently authenticates to `B`, to `C`, to her GitHub, to every machine her key opens, all with valid signatures that look exactly like Alice. The moment you `ssh -A` into a box you do not fully trust, you have lent your identity to its administrator. This has been the pivot point in real breaches.

Why it works

Use safer mechanisms instead of blanket forwarding:

- **`ProxyJump` (`ssh -J bastion target`).** The connection to `target` is tunneled *through* the bastion, but the bastion only sees ciphertext – the authentication to `target` happens end-to-end from your laptop. The bastion never gets access to your agent. This is the correct replacement for "forward my agent through the jump host."
- **Confirmation on use (`ssh-add -c`).** The agent prompts you (a desktop dialog) before every signature, so a hijacker on `A` cannot sign silently.
- **FIDO2 keys (`-sk`).** Each signature requires a physical touch, so even a forwarded agent cannot be abused without you tapping the token.
- **Per-host scoping.** If you must forward, scope it to one trusted destination, never `ForwardAgent yes` globally in `~/.ssh/config`.

28.7 Port forwarding and the connection layer

After authentication, the connection protocol multiplexes *channels* over the one encrypted pipe. Beyond the interactive shell, its most heavily used feature is *port forwarding*, which tunnels arbitrary TCP through SSH.

- **Local forward** (`ssh -L 5432:db.internal:5432 bastion`): your laptop's port 5432 is tunneled, through the SSH connection to `bastion`, onward to `db.internal:5432`. You reach an internal database as if it were local, with all traffic encrypted to the bastion.
- **Remote forward** (`ssh -R 8080:localhost:3000 server`): a port on the *server* forwards back to a service on your laptop – handy for exposing a local dev server, dangerous if it

exposes you.

- **Dynamic forward** (`ssh -D 1080 server`): SSH acts as a local SOCKS proxy, sending your browser's traffic out through `server`. A poor man's VPN.

Figure 28.3 illustrates how local port forwarding encapsulates arbitrary TCP streams into encrypted SSH channels.

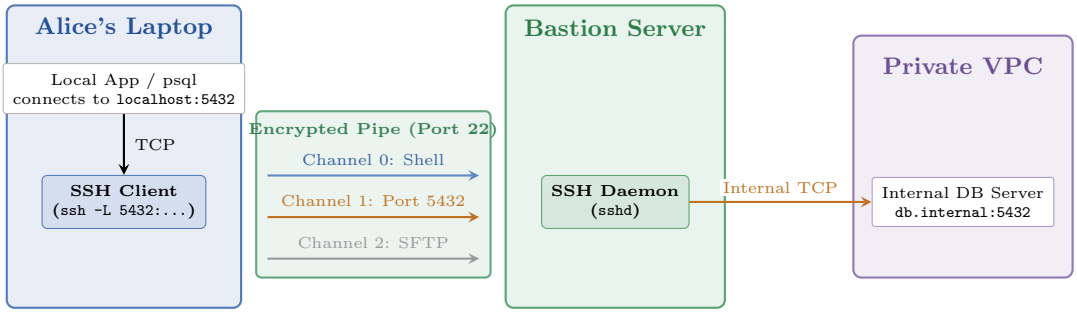


Figure 28.3: Port forwarding and channel multiplexing in SSH. A single encrypted and authenticated TCP connection carries multiple logical channels simultaneously (interactive terminal, file transfer, and forwarded network ports), securely tunneling traffic to internal services through a bastion host.

In the real world

Where SSH shows up. It is the universal remote-administration tool for Linux and Unix servers – effectively every cloud instance you will ever touch is reached over SSH. `scp` and `sftp` ride the same protocol for file transfer. `git` over SSH (`git@github.com`) is how most developers push code; the `git@` login uses your public key and a forced command. Configuration-management tools (Ansible) drive fleets entirely over SSH. Bastion/jump hosts gate access to private networks. The pattern is always the same: one well-analyzed, server-authenticated, public-key-authenticated encrypted channel, reused for shells, files, ports, and version control. When someone says "connect to the prod box," this chapter describes the cryptography involved.

28.8 Common misconfigurations

The protocol is sound; deployments leak through their settings. The high-value items in `sshd_config`:

Attack

The misconfiguration checklist.

- `PermitRootLogin yes` – lets attackers brute-force the one account that exists on every box and grants full control. Set `prohibit-password` (key-only root) or `no`.
- `PasswordAuthentication yes` on an internet-facing host – exposes every account to online guessing and credential stuffing. Set `no`; require keys.

- Weak or legacy algorithms left enabled – SSH-1 (never), `ssh-dss`, `arcfour`, CBC-mode ciphers, `hmac-md5`, small DH groups. Pin a modern `KexAlgorithms`, `Ciphers`, and `MACs` list.
- World-readable or group-writable `authorized_keys`, or a writable `~/.ssh` – SSH *silently refuses* keys with loose permissions, and a writable `authorized_keys` lets a local attacker add their own key. Lock to `600`/`700`.
- Stale `authorized_keys` entries – an employee leaves, their key stays, and nobody audits. This is the case SSH certificates with short expiry were invented to kill.
- `ForwardAgent yes` globally – discussed above; turns every box into an agent-hijack opportunity.
- No `MaxAuthTries`/rate limiting and `sshd` on the default port wide open – pair key-only auth with `fail2ban` or a firewall to cut log noise and slow probing. (Key-only auth is the real defense; rate limiting is hygiene.)
- Reused or backed-up *host* keys across many servers – if one server's host private key leaks, every server sharing it can be impersonated. Each host gets its own key.

Intuition

SSH's cryptography is strong, so attackers go after the seams. The seams are (1) the human at the TOFU prompt, (2) password auth as a guessable fallback, (3) the forwarded agent as a borrowed identity, and (4) stale or over-broad authorization that outlives its purpose. A hardened SSH deployment closes all four: certificate-based host trust (no prompt to fumble), key-only auth (nothing to guess), `ProxyJump` instead of agent forwarding (no borrowed identity), and short-lived certificates (authorization that expires on its own). None of those fixes change the cryptography; they address the points where the protocol hands control back to people.

28.9 Putting it together

A full `ssh alice@server` from cold start runs the message flow in Figure 28.4, in order:

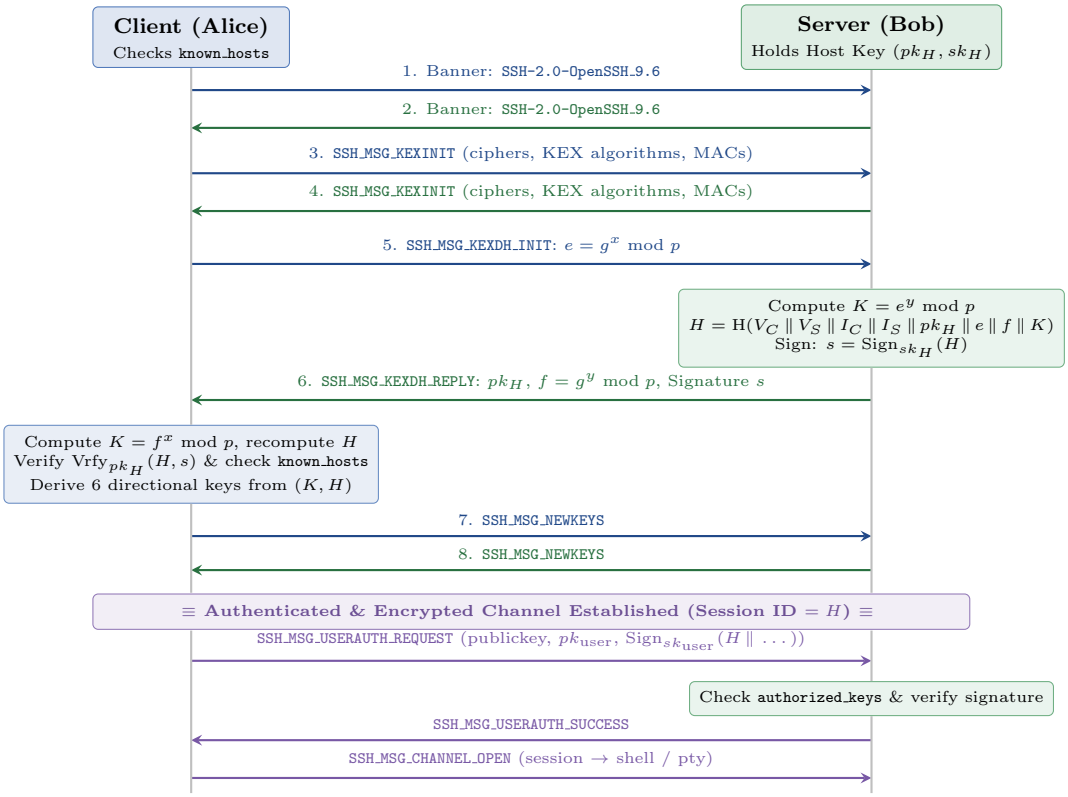


Figure 28.4: The full SSH-2 protocol sequence. Unauthenticated banners and algorithm lists are folded into the exchange hash H and signed by the server's host key. Once keys are activated via `NEWKEYS`, user public-key authentication and multiplexed channels operate inside the secure tunnel.

In words:

1. TCP connects to port 22; both sides exchange `SSH-2.0` banners.
2. `KEXINIT` negotiates key exchange, host-key signature, cipher, MAC, compression – client preference wins among mutual support.
3. Diffie-Hellman (Curve25519) runs; the server signs the exchange hash H with its host key. The client verifies the signature and checks pk_H against `known_hosts` (TOFU or certificate). *Server authenticated.*
4. Both sides derive six keys from (K, H), send `NEWKEYS`; the channel is now encrypted and integrity-protected.
5. User authentication runs inside the tunnel: Alice signs a session-bound blob with her private key; the server checks `authorized_keys` and the signature. *Client authenticated.*
6. The connection protocol opens a session channel; the server allocates a pseudo-terminal and spawns Alice's shell.
7. Every keystroke and every byte of output rides the binary packet protocol – AEAD-encrypted, sequence-numbered, MAC-checked. After about a gigabyte or an hour, the key exchange reruns to rekey.

In summary, SSH uses Diffie-Hellman for the secret, a host-key signature to authenticate the server, key continuity (or a certificate) to decide whether to trust that host key, a session-bound user signature for the client, and an AEAD binary packet protocol carrying the rest. Every piece is something you have already met in earlier chapters; SSH combines them into a tool you will use daily as a working engineer.

SSH secures point-to-point connections over IP networks where endpoints are computers with rich operating systems and filesystems. In the next chapter, we look at cellular networks, where tiny tamper-resistant SIM cards, radio towers, and resource-constrained handsets must authenticate and encrypt traffic at line rate from 2G to 5G.

28.10 Exercises

Exercise 28.1. In the toy group $\mathbb{Z}/23\mathbb{Z}$ with $g = 5$, the client picks $x = 9$ and the server picks $y = 4$. Compute the client's public value $e = g^x$, the server's $f = g^y$, and the shared secret K two ways (e^y and f^x) to confirm they agree. Use $5^{22} \equiv 1$ to reduce exponents mod 22.

Exercise 28.2. Explain precisely why a passive Eve who records e and f in the previous exercise cannot recover K when the group is a 252-bit elliptic curve, but can when the group is $\mathbb{Z}/23\mathbb{Z}$. Name the underlying hardness assumption and the best known attack cost.

Exercise 28.3. The six SSH session keys are derived as $\mathrm{H}(K \,\|\, H \,\|\, t \,\|\, \mathit{session_id})$ for tag bytes $t \in \{\texttt{A}, \ldots, \texttt{F}\}$. Argue why an attacker who somehow learns $\mathrm{Enckey}_{C\to S}$ (tag `C`) still learns nothing about $\mathrm{MACkey}_{C\to S}$ (tag `E`). Which property of H are you relying on?

Exercise 28.4. A junior admin reflexively runs `ssh-keygen -R server` and reconnects whenever the "REMOTE HOST IDENTIFICATION HAS CHANGED" warning appears. Describe a concrete attack this habit enables, and the out-of-band step that would have caught it. Why can the SSH protocol itself not distinguish the benign from the malicious cause?

Exercise 28.5. Show that the SSH `publickey` signature cannot be replayed from one connection onto another. Identify exactly which field in the signed blob M guarantees this, and trace what would have to be equal between two connections for a replay to succeed.

Exercise 28.6. Alice runs `ssh -A` into a shared bastion she does not administer, then opens a long-lived session and walks away. The bastion's root user is malicious. Describe step by step what root can do during that session, what root *cannot* do, and how `ProxyJump` or `ssh-add -c` would each have prevented or limited the damage.

Exercise 28.7. Classic SSH used Encrypt-and-MAC with the CBC-mode packet-length field encrypted. Sketch why the receiver's behavior (waiting for N more bytes versus emitting a MAC error) leaks plaintext bits when an attacker injects a captured ciphertext block. State the two countermeasures OpenSSH adopted and why each removes the leak.

Exercise 28.8. Your organization has 4,000 servers and 300 engineers. Compare two trust models: (a) every engineer's public key in every server's `authorized_keys`, with TOFU host keys; (b) one host CA and one user CA issuing eight-hour user certificates. For each model, state how a new server is trusted by laptops, how a departing engineer is deprovisioned, and the worst-case blast radius of a single leaked private key. Which model scales, and why?

Exercise 28.9. Give three distinct reasons public-key authentication resists a malicious or compromised server that password authentication does not. For each, point to the specific mechanism (what the server does or does not receive) that makes the difference.

Exercise 28.10. The transport-layer exchange hash is $H = \mathrm{H}(V_C \,\|\, V_S \,\|\, I_C \,\|\, I_S \,\|\, pk_H \,\|\, e \,\|\, f \,\|\, K)$, signed by the host key. Suppose Eve tampers with the client's `KEXINIT` (I_C) on the wire to force a weaker cipher. Trace, step by step, where the two sides' computations diverge and at which exact check the attack is detected.

Chapter 29

Cryptography in Telecoms: GSM, LTE, and 5G

In the previous chapter, SSH protected remote command lines between general-purpose computers using public-key authentication and ephemeral Diffie–Hellman key exchanges. Cellular telephony grew up in a harsher world. The first digital mobile standard, GSM, was designed in the late 1980s for handsets with the compute budget of a pocket calculator, a radio link that drops packets for fun, and a regulatory environment where strong cryptography was treated as a munition. The cryptographers who built it worked under tight constraints, and the cipher they shipped reflects each of them.

This chapter follows mobile security across three generations. We start with GSM (2G): how a SIM card proves who you are, how the A5 family of stream ciphers scrambles the air interface, and why both were broken so thoroughly that you can decrypt a GSM call today with a laptop and a rainbow table. Then we meet the Authentication and Key Agreement protocol – AKA – introduced in 3G and carried forward, with steady repairs, into LTE (4G) and 5G. One attacker recurs throughout: the *IMSI catcher*, a fake base station that lifts your identity out of the air and, in the GSM era, your conversation too. Each generation closes one more door on that attacker. 5G finally shuts the last obvious one by encrypting your permanent identity before it ever leaves the phone.

The reason telecom crypto deserves its own chapter is that it is the largest deployed cryptosystem on Earth – billions of SIM cards, every one a tamper-resistant token holding a symmetric key – and because its history is the cleanest available case study in how real-world constraints, export law, and backward compatibility shape a design, sometimes fatally.

29.1 The setting: who is who, and what they trust

Before any cipher, fix the cast. A mobile network has more named parties than our usual Alice and Bob, and the trust relationships between them determine the security.

- The **subscriber**, carrying a **handset** (the phone, called *Mobile Equipment* or ME in the specs) into which a **SIM** card is inserted. The SIM is a small tamper-resistant smartcard. It is the cryptographic root of the subscriber's identity.
- The **home network** – the operator you pay. Deep inside it sits a database, historically the

Authentication Centre (AuC), that holds the master secret key for every SIM it ever issued.

- The **visited network** – the operator whose tower you are actually camped on right now, which may not be your home network (this is roaming). The tower itself is the *base station* (BTS in GSM, eNodeB in LTE, gNodeB in 5G).
- **Eve**, who owns a radio. In the cellular threat model Eve is unusually capable, because she can do more than listen. She can transmit. A radio that transmits the right beacons can impersonate a base station, and a phone has historically had no way to tell a real tower from a fake one.

Intuition

The single fact that organizes all of telecom security is this: **the SIM and the home network share a secret symmetric key K_i, and nobody else ever learns it – not even the visited network you are roaming on.** The SIM is manufactured with K_i burned in; the same K_i is recorded in the operator's AuC. From the day the card is printed, those two endpoints share a 128-bit secret that never travels over any wire or any radio. Everything else – authentication, the per-call encryption key, identity protection – is bootstrapped from that one pre-shared symmetric key.

This is a very different starting point from TLS. TLS strangers establish a key from nothing using public-key cryptography. Cellular subscribers are not strangers to their operator; they were provisioned. So telecom can lean on cheap, fast symmetric primitives – no modular exponentiation on a 1990 SIM – at the cost of a logistics problem (printing and protecting billions of keys) that TLS does not have.

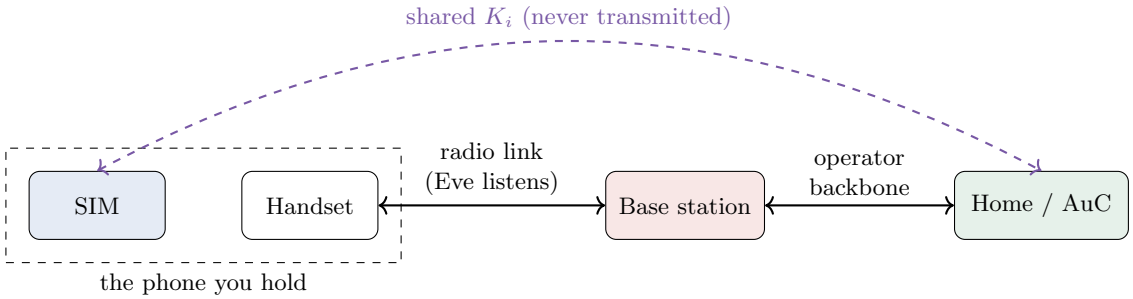


Figure 29.1: The trust map. The secret K_i lives only on the SIM and in the home network's AuC. The vulnerable link is the radio between handset and base station.

29.2 GSM authentication: a challenge and a response

When your GSM phone first contacts a tower, two things must happen before you can place a call. The network must be convinced the SIM is genuine (authentication), and both sides must derive a session key to encrypt the radio link (key agreement). GSM does both with a single challenge–response built on the shared K_i.

Here is the message flow, with the concrete names from the standard.

1. The phone announces an identity to the tower. Ideally a temporary one (the TMSI, discussed

below), but on a first contact, or when the network asks, it sends its permanent identity, the **IMSI** (International Mobile Subscriber Identity – the number that uniquely names your SIM).

2. The visited network forwards the IMSI to the home network and asks for an *authentication triplet*. The home AuC looks up K_i, generates a fresh random 128-bit challenge **RAND**, and computes two values with secret algorithms that take (K_i, RAND) as input:

$$\text{SRES} = A3(K_i, \text{RAND}) \in \{0,1\}^{32}, \qquad K_c = A8(K_i, \text{RAND}) \in \{0,1\}^{64}.$$

SRES is a 32-bit *signed response*; K_c is the 64-bit session key for encryption. The home network ships the triplet $(\text{RAND}, \text{SRES}, K_c)$ to the visited network and keeps K_i to itself.

3. The visited network sends RAND over the air to the phone. The SIM, which also holds K_i, computes its own $A3(K_i, \text{RAND})$ and $A8(K_i, \text{RAND})$, getting SRES' and K_c.

4. The phone returns SRES'. The network checks $\text{SRES}' \stackrel{?}{=} \text{SRES}$. If they match, the SIM has proven it knows K_i without ever revealing it, and both sides now hold the same K_c to encrypt the call with the A5 cipher.

Why it works

Why is this an authentication? RAND is fresh and unpredictable, so a recorded old (RAND, SRES) pair is useless against a new challenge – this defeats replay. Producing the correct SRES requires K_i; an attacker who does not know K_i can do no better than guess a 32-bit value, succeeding with probability 2^{-32}. And K_i never crosses the radio, so passive eavesdropping on the exchange does not leak it (assuming $A3$ does not).
Why is the key agreement "free"? $K_c = A8(K_i, \text{RAND})$ is a deterministic function of the shared secret and the public nonce, so both endpoints compute the identical K_c without transmitting it. RAND is public; K_i is shared; therefore K_c is shared. The eavesdropper sees RAND but, lacking K_i, cannot compute K_c. This is the symmetric-key analogue of a key exchange: not Diffie-Hellman, just "run a PRF on a common secret and a common nonce."

In practice operators implement $A3$ and $A8$ together as a single algorithm called **COMP128**, which takes (K_i, RAND) and emits 128 bits: the first 32 are SRES, and 64 of the rest become K_c (with, as we will see, a fatal twist).

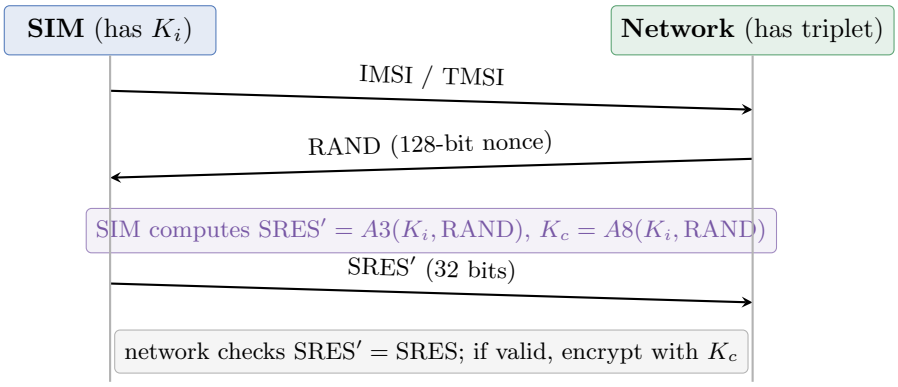


Figure 29.2: GSM challenge–response. Only RAND and the 32-bit SRES cross the air. Note the asymmetry: the phone never checks the network.

Common trap

Read figure 29.2 again and find the missing arrow. The network challenges the phone, but *the phone never challenges the network*. GSM authentication is **one-way**: the SIM proves itself to the tower, the tower proves nothing to the SIM. A phone will obediently camp on, authenticate to, and encrypt toward any device that speaks the protocol and tells it to. This single asymmetry is the root of the IMSI-catcher problem, and fixing it is the main job of 3G/4G/5G AKA.

29.2.1 A worked triplet with tiny numbers

COMP128 is too large to trace by hand, but the *structure* of challenge–response is just a keyed function evaluated on a nonce, and we can build a toy that behaves the same way. Let our toy keyed function be

$$F(K, x) = (3\,K + 7\,x + 11) \bmod 256,$$

a deliberately weak stand-in for $A3/A8$ over the ring $\mathbb{Z}/256\mathbb{Z}$. Suppose the SIM and AuC share $K_i = 100$.

The AuC picks a random challenge, say RAND $= 42$. It computes a "response" and "key" from two slices; to keep it tiny we will say

$$\text{SRES} = F(100, 42) = (300 + 294 + 11) \bmod 256 = 605 \bmod 256 = 93,$$

$$K_c = F(100, 43) = (300 + 301 + 11) \bmod 256 = 612 \bmod 256 = 100.$$

The AuC ships the triplet $(\text{RAND}, \text{SRES}, K_c) = (42, 93, 100)$ to the visited network and keeps $K_i = 100$. The tower forwards RAND $= 42$ to the phone. The SIM, holding $K_i = 100$, recomputes $F(100, 42) = 93$ and returns $\text{SRES}' = 93$. The tower checks $93 = 93$: pass. Both sides now share $K_c = 100$ as the session key, having transmitted only the public nonce 42 and the response 93. An eavesdropper who saw $(42, 93)$ but does not know K_i cannot recover $K_c = 100$.

Common trap

Our toy F is linear, so an attacker who captures one pair $(\text{RAND}, \text{SRES})$ can solve for K immediately. From $\text{SRES} \equiv 3K + 7\,\text{RAND} + 11 \pmod{256}$ we get $3K \equiv \text{SRES} - 7\,\text{RAND} - 11 \pmod{256}$. Since $\gcd(3, 256) = 1$, multiply by $3^{-1} \equiv 171 \pmod{256}$ and recover K from a *single* captured challenge–response. Real $A3/A8$ must be a one-way, nonlinear function precisely so that this cannot happen. COMP128's first version failed at exactly this kind of structural weakness – the next section shows how.

29.3 Where GSM broke: COMP128, A5/1, and A5/2

GSM's cryptography is a set of examples of what not to do, and each is worth studying because each failure has a clear cause.

29.3.1 COMP128-1: cloning the SIM

The original $A3/A8$ algorithm, COMP128-1, was secret – "security through obscurity" – but leaked and was reverse-engineered in 1998 by Briceno, Goldberg, and Wagner. Its compression function had a collision weakness: certain pairs of RAND inputs that differed only in a few bytes produced internal collisions that the attacker could detect from the outputs. By choosing on the order of 2^{17} to 2^{19} carefully crafted RAND challenges and feeding them to a SIM (you need physical access, or a fake base station that keeps issuing challenges), an attacker could solve for K_i a few bits at a time.

Attack

SIM cloning via the COMP128-1 chosen-challenge attack. The 128-bit K_i is recovered in small groups. The attacker queries the SIM with chosen RANDs designed to collide in the second round of COMP128's butterfly structure; a collision in the output narrows two key bytes at a time. Roughly 2^{18} queries – a few hours over the SIM's slow smartcard interface – and K_i falls. With K_i in hand the attacker writes it to a blank programmable SIM and now owns a perfect clone: it authenticates as you, places calls billed to you, and can decrypt your traffic. The fix was COMP128-2 and -3 (collision-resistant, still secret) and, properly, operator-specific algorithms built on a public, vetted primitive (the MILENAGE construction over AES, used in 3G+). The lesson is a familiar one: do not roll a secret cipher and hope nobody looks.

29.3.2 The A5 family: the cipher on the air

Once K_c is agreed, the call's voice frames are encrypted with a stream cipher from the A5 family. There were three relevant variants:

- **A5/1** – the "strong" version, for Western Europe and North America. A hardware-efficient stream cipher built from three linear-feedback shift registers (LFSRs).
- **A5/2** – a deliberately weakened version for export to markets where regulators forbade strong crypto. It was designed to be breakable.
- **A5/0** – no encryption at all. Plaintext over the air. Selectable by the network.

A5/1 takes the 64-bit K_c and a 22-bit publicly-known frame number (the count of which 114-bit GSM burst you are on) and produces a keystream that is xored into the voice frame, exactly the stream-cipher construction of Chapter 10: $c = m \oplus \text{keystream}(K_c, \text{frame})$. The frame number is the nonce that keeps each burst's keystream distinct so you do not get the two-time-pad disaster.

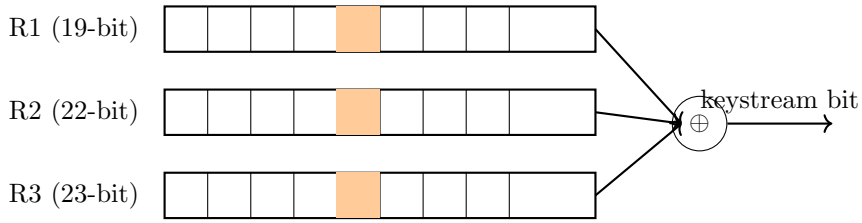


Figure 29.3: A5/1: three LFSRs of lengths 19, 22, 23 (total 64 bits = K_c). The output bit is the xor of the three top bits; majority-rule clocking decides which registers advance each step.

The clever bit in A5/1 is the irregular, majority-rule clocking: each register has a designated clocking tap, the majority value among the three taps is computed, and only registers whose clocking tap agrees with the majority step forward. This nonlinear clock control is what stops the cipher from being a trivial linear system, but it is not enough.

Attack

Breaking A5/1. A5/1 has been broken many times over, each break cheaper than the last.

- *Time–memory tradeoff (Biryukov–Shamir–Wagner, 2000; Barkan–Biham–Keller).* The 64-bit internal state is small enough to attack with precomputation. The keystream for parts of a call is partly predictable (frames carry known structure), giving known keystream. With a few seconds of known keystream you look up the state in a precomputed table.
- *The A5/1 Cracking Project (Nohl et al., 2009).* Karsten Nohl's team publicly computed about 2 TB of rainbow tables for A5/1. With those tables and a cheap software radio capturing a call, K_c is recovered in seconds to minutes on a PC. The point was political: to prove the cipher was dead and shame operators into upgrading.
- *Key length.* Even ignoring structural flaws, K_c is 64 bits – and in many deployments the 10 least-significant bits were forced to zero, leaving 54 bits of effective key. A brute force over 2^{54} is within reach of a modest GPU farm.

Attack

A5/2 is broken in real time – and it poisons A5/1. A5/2 was built to be weak, and Barkan, Biham, and Keller (2003) showed it falls to a ciphertext-only attack in well under a second: the cipher is so linear that a handful of frames let you solve for the state by linear algebra. The more serious problem is the *cross-protocol attack*. Recall K_c is derived from (K_i, RAND) *independent of which A5 variant will use it.* So an active attacker (an IMSI catcher) replays a captured RAND to the victim, but forces the phone to use A5/2, instantly recovers K_c from the A5/2 traffic, and now holds the very same K_c that the victim used for a real A5/1 call with the legitimate network. Because the weak and strong ciphers share the same key, one weak variant compromises the whole suite. The repair: A5/2 was prohibited in handsets from 2006 onward, and modern phones refuse it.

History

Why ship a cipher you know is weak? Cold-war and 1990s export controls. In the late 1980s, the GSM committee under the European Telecommunications Standards Institute (ETSI) had to navigate strict export laws: cryptography was regulated as a munition under regimes like COCOM (Coordinating Committee for Multilateral Export Controls) and the US ITAR (International Traffic in Arms Regulations), so "strong" crypto could not be exported to many countries. A5/1 was the NATO-region cipher; A5/2 was the deliberately enfeebled export cipher so that GSM phones could be sold worldwide while keeping intercept capability for the exporting states. The same political pressure explains the truncated effective key length. The hardware remained in service long after export rules were relaxed, illustrating the core hazard of mandated cryptographic weakness: a deliberate flaw built for authorized interception can eventually be exploited by anyone within radio range. Today, the open operational challenge is turning off legacy 2G and 3G networks worldwide so rogue base stations can no longer exploit downgrade paths.

29.3.3 The arithmetic of a 54-bit key

Put numbers on why K_c's key length matters, because the gap between 64 and 54 bits is large in practice. Take the deployments that forced the low 10 bits of K_c to zero, leaving 2^{54} effective keys. Suppose a brute-force rig tries 2^{30} candidate A5/1 states per second (a billion per second, a plausible figure for a small GPU cluster checking each state against a few frames of known keystream). Then

$$\frac{2^{54}}{2^{30}} = 2^{24} \text{ seconds} \approx 1.68 \times 10^{7} \text{ s} \approx 194 \text{ days}.$$

That is a single rig, brute over the whole reduced space, in well under a year – and the expected time to a hit is half that, about 97 days. Now compare the full 64-bit key:

$$\frac{2^{64}}{2^{30}} = 2^{34} \text{ seconds} \approx 1.72 \times 10^{10} \text{ s} \approx 545 \text{ years}.$$

The 10 forced bits bought the attacker a factor of $2^{10} = 1024$, collapsing five centuries into a few months. And this is the *naive* attack. The time–memory tradeoff changes the economics again: you pay a large one-time precomputation (Nohl's tables were about 2 TB) to build a lookup structure, and thereafter each captured call is cracked in seconds, not months, because you are doing table lookups instead of search. The 64-bit state was always the ceiling; once precomputation amortizes that ceiling across every future call, even the unreduced key offers little protection. The lesson is quantitative: a stream cipher whose entire secret state fits in 64 bits has no margin, and shaving 10 bits off turns "expensive" into "trivial."

Common trap

A tempting misreading: "2^{54} is still a huge number, so we are fine." The error is treating brute force as the only attack and treating one key as the only target. A surveillance operator does not brute one key from scratch each time; they precompute once and then read calls in real time forever. Security margins must be measured against the *best* known attack with its

precomputation amortized, not against the textbook 2^n search. A 64-bit symmetric cipher has not been a safe confidentiality floor for decades.

29.3.4 No integrity, and the network is never authenticated

Beyond broken ciphers, GSM has two structural holes that no amount of stronger keystream would fix. First, there is **no message integrity** on the air interface: the keystream gives confidentiality (badly) but nothing stops an active attacker from flipping ciphertext bits, since stream-cipher xor is malleable (Chapter 10). Second, and worse, the network is **never authenticated to the phone** (the missing arrow of figure 29.2). A device that broadcasts a strong signal and speaks GSM can convince your phone to attach to it, ask for your IMSI, and tell it to use A5/0 (no encryption). That device is an IMSI catcher.

29.4 IMSI catchers (stingrays): the canonical telecom attack

An **IMSI catcher** – "Stingray" is one commercial brand name that became generic – is a fake base station. It exploits two facts about GSM: the phone always attaches to the strongest compatible cell, and the phone authenticates the network not at all. Put a transmitter near the target that is louder than the real towers, and nearby phones camp on it.

Attack

What an IMSI catcher does.

1. *Identity capture.* The fake BTS sends an *identity request*. Per spec, the phone answers with its IMSI – its permanent, globally-unique identity. The operator of the catcher gets to learn "which subscribers are physically here right now," which is exactly what a surveillance device wants. (Catchers also harvest the IMEI, the hardware serial.)
2. *Downgrade to no encryption.* Because the network chooses the cipher and the phone cannot refuse, the catcher commands A5/0 – plaintext – or A5/2, which it then cracks. Now the call and SMS content are readable.
3. *Man-in-the-middle.* A more capable catcher relays between the victim and a real tower, authenticating to the real network using the victim's responses while sitting in the middle of the now-unencrypted radio link. It can intercept, and sometimes alter, calls and texts.

The first step works even against later-generation phones that fall back to 2G, which is why "downgrade to GSM" is the catcher's favorite move.

Figure 29.4 diagrams how the rogue base station exploits GSM's lack of mutual authentication.

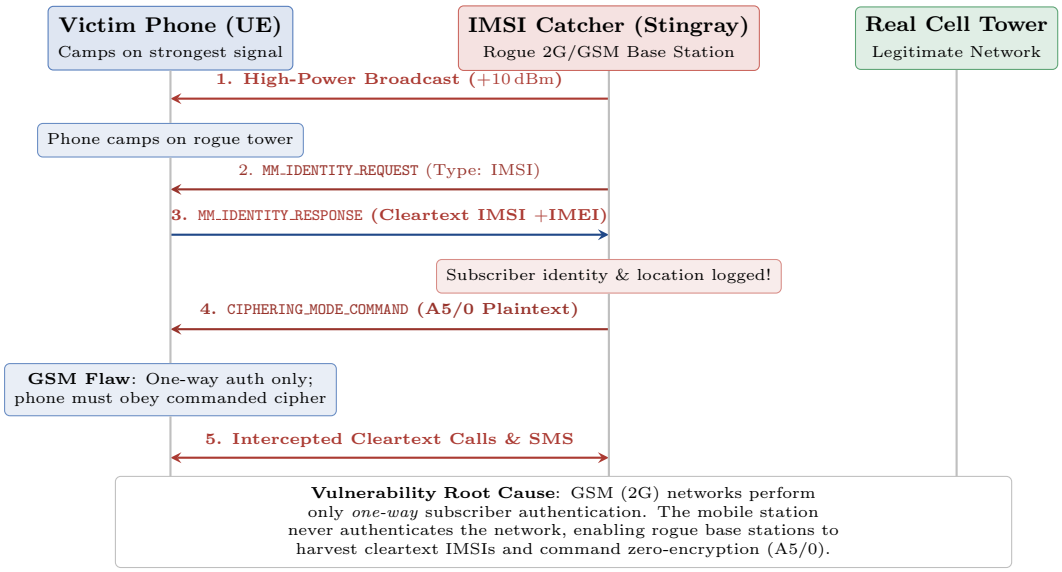


Figure 29.4: The IMSI catcher (Stingray) active interception and downgrade attack. By transmitting at higher radio power than legitimate base stations, the rogue cell forces the victim phone to camp on it. Exploiting 2G's lack of mutual authentication, it requests the unencrypted IMSI and commands plaintext mode (A5/0).

Intuition

The IMSI catcher succeeds because of two independent design choices that compound.
(1) The phone shouts its permanent name in cleartext. Identity (IMSI) is sent unencrypted, on request, before any security is set up – it has to be, on a first contact, because the network has no other way to look you up. So a passive listener, not even a transmitter, can sometimes catch IMSIs during attach. This is a *privacy/identity* leak.
(2) The phone trusts any tower. One-way authentication means the phone cannot tell friend from foe, so the catcher's commands ("use A5/0", "tell me your IMSI") are obeyed. This is a *confidentiality/MITM* leak.
Each later generation attacks one of these. 3G/4G AKA fixes (2): the network must now prove itself. 5G fixes (1): the permanent identity is encrypted with public-key crypto before it leaves the phone. We take them in order.

GSM tries to mitigate (1) with the **TMSI** (Temporary Mobile Subscriber Identity): after the first attach, the network assigns a short-lived temporary id, encrypted under K_c, and the phone uses the TMSI on subsequent contacts so the IMSI stays off the air. It is a real improvement, but it is fragile – the network can always issue an identity request and force the IMSI out, and the catcher simply does. TMSI hides the IMSI from casual passive sniffing; it does not stop an active attacker.

29.5 3G/4G AKA: making the network prove itself

The Authentication and Key Agreement protocol (AKA), introduced with UMTS (3G) and carried into LTE (4G) and 5G with refinements, keeps the SIM/home-network shared key K (now 128 bits, used with a public, vetted algorithm such as MILENAGE built on AES, or TUAK built on Keccak)

and the challenge–response shape. It adds the things GSM lacked: **mutual authentication**, **a freshness counter against replay**, and **integrity protection**, with longer keys.

The two new ingredients are a **sequence number** SQN, kept in sync between the SIM and the home network, and a **message authentication code** the network computes so the SIM can verify it. The home AuC now produces a five-element *authentication vector* (the 3G successor to the GSM triplet):

$$(\text{RAND},\ \text{XRES},\ \text{CK},\ \text{IK},\ \text{AUTN}).$$

Reading these: RAND is the random challenge as before; XRES is the expected response; CK is the 128-bit cipher key and IK the 128-bit integrity key (two keys now, confidentiality and integrity separated); and AUTN is the *authentication token* that lets the SIM check the network. The home network computes, using key K and a set of functions $f1, \ldots, f5$:

$$\text{MAC} = f1_K(\text{SQN} \,\|\, \text{RAND} \,\|\, \text{AMF}), \quad \text{XRES} = f2_K(\text{RAND}),$$

$$\text{CK} = f3_K(\text{RAND}), \quad \text{IK} = f4_K(\text{RAND}), \quad \text{AK} = f5_K(\text{RAND}),$$

$$\text{AUTN} = (\text{SQN} \oplus \text{AK}) \,\|\, \text{AMF} \,\|\, \text{MAC}.$$

AK is an *anonymity key* used only to mask SQN (so the sequence number, which could leak how often you connect, is not sent in clear). AMF is an authentication management field. The interesting object is AUTN: it bundles the (masked) sequence number with a MAC over that sequence number and the challenge.

The protocol runs like GSM with one new check at the phone:

1. Network sends RAND and AUTN to the phone.
2. The SIM, holding K, recomputes $\text{AK} = f5_K(\text{RAND})$, unmasks $\text{SQN} = (\text{SQN} \oplus \text{AK}) \oplus \text{AK}$, then recomputes $\text{MAC}^* = f1_K(\text{SQN} \,\|\, \text{RAND} \,\|\, \text{AMF})$ and checks it equals the MAC inside AUTN. **This is the network authentication GSM lacked**: only a party holding K – the genuine home network – could have produced a valid MAC.
3. The SIM also checks that SQN is in the acceptable window ahead of its stored counter. **This is the anti-replay**: a catcher replaying an old, captured (RAND, AUTN) supplies a stale SQN, the freshness check fails, and the SIM rejects it (triggering a resync).
4. Only if both checks pass does the SIM compute $\text{RES} = f2_K(\text{RAND})$ and send it back. The network verifies RES = XRES as before. Now both sides share CK and IK.

Why it works

Mutual authentication, in one line each. **Phone → network:** only the real SIM can compute $\text{RES} = f2_K(\text{RAND})$, because only it shares K. **Network → phone:** only the real home network can compute the MAC inside AUTN, because only it shares K. The MAC is keyed by the same secret K that authenticates the subscriber, so a fake base station – which does not hold K – cannot forge AUTN and is unmasked at step 2. The sequence number SQN then closes the replay gap that a pure challenge–response would still allow if the attacker could capture and resend a once-valid AUTN. CK and IK being separate keys means integrity (a real MAC on signaling) is now possible, killing GSM's bit-flipping malleability.

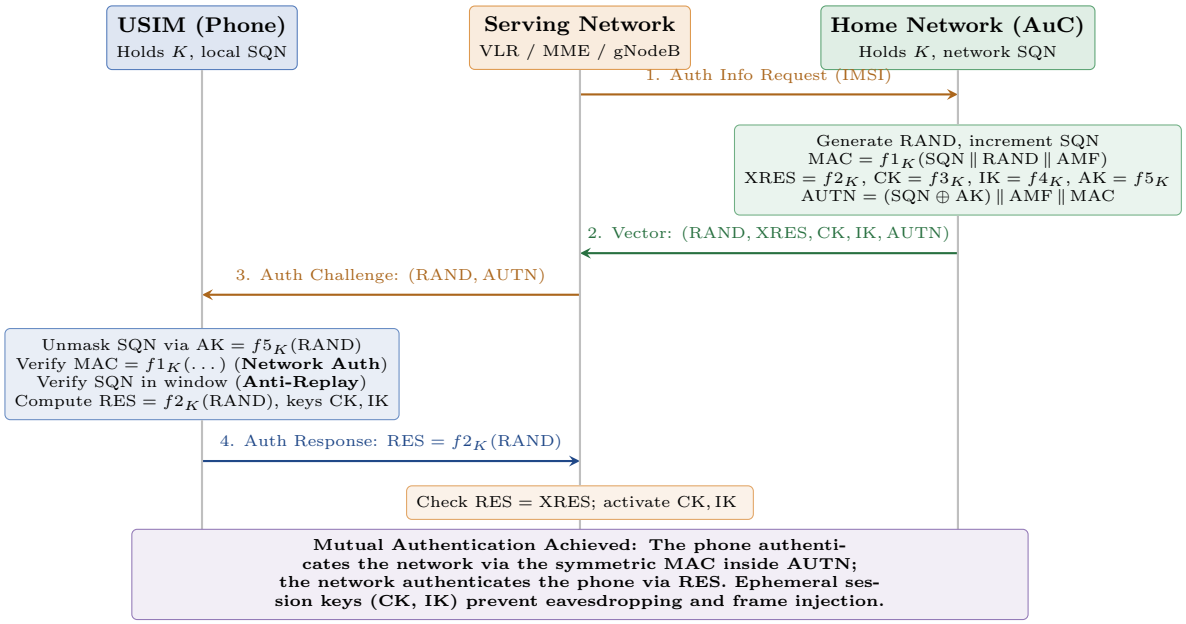


Figure 29.5: The 3G/4G Authentication and Key Agreement (AKA) protocol flow. Unlike 2G GSM, AKA introduces mutual authentication and replay protection. The home network provides an authentication vector containing AUTN, allowing the USIM to verify network identity and freshness before returning the response RES.

29.5.1 A worked SQN trace: replay, freshness, and resync

The sequence number is the part of AKA people find slipperiest, so trace it with tiny numbers (Figure 29.6). Say the SIM stores the highest accepted sequence number $\mathrm{SQN}_{\mathrm{MS}} = 40$, and the home network's counter for this subscriber is $\mathrm{SQN}_{\mathrm{HE}} = 41$ (the home end is one step ahead, about to issue the next vector). The SIM accepts any incoming SQN that is greater than its stored value but within a window $\Delta = 32$ ahead – i.e. it accepts SQN in $(40,\ 72]$, and stores the new value on success.

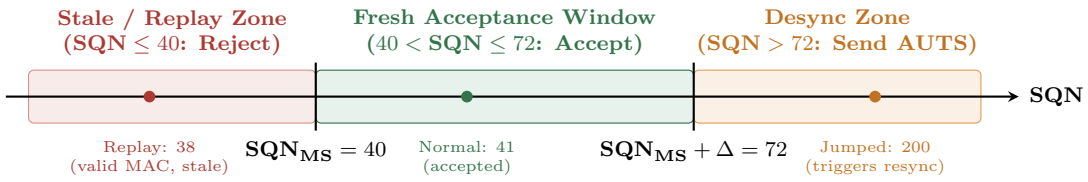


Figure 29.6: AKA sequence-number (SQN) acceptance window on the SIM. Even a genuine, validly MACed authentication vector is rejected if its sequence number lies at or below $\mathrm{SQN}_{\mathrm{MS}}$ (preventing replay), while sequence numbers beyond $\mathrm{SQN}_{\mathrm{MS}} + \Delta$ trigger an authenticated AUTS resynchronization.

Normal run. The home network builds a vector with $\mathrm{SQN} = 41$, computes $\mathrm{MAC} = f1_K(41 \| \mathrm{RAND} \| \mathrm{AMF})$, and sends AUTN. The SIM unmasks $\mathrm{SQN} = 41$, recomputes the MAC and it matches (genuine network), checks $41 \in (40, 72]$ (fresh), accepts, and advances its store to $\mathrm{SQN}_{\mathrm{MS}} = 41$. Authentication succeeds; CK, IK are derived.

Replay attempt. An IMSI catcher recorded an *earlier* valid vector that carried SQN = 38, with its genuine MAC. It replays (RAND, AUTN) to the SIM now. The MAC *verifies* – it was made by the real network, the catcher did not forge it. This is the central subtlety: **the MAC check alone does not stop replay**, because a captured-once-valid token is, by definition, validly MACed forever. What stops it is the freshness test: $38 \leq 41$, so $38 \notin (41, 73]$; the SIM rejects with a "synchronisation failure." The catcher cannot bump the SQN to a fresh value because that would require recomputing the MAC, which needs K.

Resync. Now a benign desync: suppose the home counter has jumped far ahead (other vectors were generated and never used) to $\text{SQN}_{\text{HE}} = 200$ while the SIM still stores 41, and the window is only $\Delta = 32$. The network sends SQN = 200; the SIM sees $200 \notin (41, 73]$ – too far ahead, outside the window – and cannot accept. Instead of failing silently, the SIM emits an authenticated *resync token* AUTS that carries its own current $\text{SQN}_{\text{MS}} = 41$, masked and MACed with a *separate* resync MAC $f1^*_K$. The home network reads AUTS, resets its counter to just above the SIM's, and issues a fresh vector the SIM will accept.

Why it works

Why resync cannot be turned into an attack. AUTS is itself keyed by K via $f1^*$, so a fake base station cannot forge a resync to push the SIM's counter to an attacker-chosen value – it does not hold K. And AUTS reveals only the SIM's current SQN masked by $f5^*_K(\text{RAND})$, an anonymity-key mask, so an eavesdropper does not even learn the raw counter. The resync path closes a real operational problem (counters drift; vectors get generated out of order in roaming) without handing the attacker a lever, because every message that moves a counter is authenticated by the one secret the attacker lacks. The design rule on display: never let an unauthenticated party move security-critical state.

Work it yourself

Work it yourself. With stored $\text{SQN}_{\text{MS}} = 40$ and window $\Delta = 32$, decide accept/reject for incoming SQN values 41, 40, 72, 73, and 39. (Answers: 41 accept; 40 reject (not strictly greater); 72 accept (top of window); 73 reject (just past window); 39 reject (stale).) Then argue why making Δ enormous – say 2^{28} – would weaken the anti-replay guarantee even though every accepted SQN is still "ahead."

29.5.2 What AKA still leaks: the IMSI on attach

AKA closes the man-in-the-middle and downgrade-by-impersonation doors for 3G/4G. A genuine IMSI catcher cannot force encryption off in LTE the way it could in GSM, because the eNodeB-to-phone link runs integrity-protected signaling keyed by IK, and the phone authenticates the network.

But AKA did *not* fix leak (1). On initial attach – or whenever the network claims it cannot resolve your temporary identity (the GUTI/TMSI) – the phone still sends its IMSI in cleartext, for the same chicken-and-egg reason as GSM: before any key is set up, the network must be told whom to look up. So an LTE IMSI catcher does not need to break encryption; it just sends an identity request and harvests IMSIs for tracking, then optionally pushes the phone down to 2G where the old attacks still work. This is why through 4G, IMSI catchers remained a live surveillance tool, and why "deny 2G fallback" became a real phone hardening option.

Attack

LTE catcher, residual attacks. Even with mutual authentication, several attacks survived into LTE:

- *IMSI harvesting* on attach, as above – a privacy/location attack, not a content attack.
- *Downgrade to 2G/3G*, where weaker crypto or no mutual auth applies, by jamming LTE and advertising only a 2G cell.
- *Pre-authentication message attacks.* Early LTE signaling messages (some RRC and NAS messages) were sent before security activation and were not integrity-protected, enabling denial-of-service and some tracking (the "aLTEr" and related research families, 2018–2019).

The structural cause of the first is the cleartext permanent identity. 5G targets exactly that.

29.6 5G: encrypting the identity itself

5G keeps 5G-AKA (the same AKA skeleton with mutual authentication, SQN freshness, and separate keys, now with a 256-bit-capable key hierarchy) and adds the fix that closes leak (1): the permanent identity is never sent in the clear. The permanent id is renamed **SUPI** (Subscription Permanent Identifier, the IMSI's successor). Before the phone ever transmits it, the SIM encrypts the SUPI into a **SUCI** (Subscription *Concealed* Identifier) using public-key cryptography.

Intuition

The chicken-and-egg problem – "how do I tell the network who I am before we share a key?" – is solved the way the rest of the internet solves first contact: with public-key crypto. The home network publishes a public key; the operator provisions that public key onto the SIM at manufacture (it is in the home network already, so no new trust root is needed). When the phone must reveal its identity, the SIM encrypts the SUPI to the home network's public key. Only the home network, holding the matching private key, can decrypt the SUCI back to the SUPI. A fake base station that issues an identity request now receives a SUCI it cannot read, and because the encryption is randomized with an ephemeral key share, the SUCI is *different every time*, so the catcher cannot even use it as a stable tracking handle.

The concealment uses the **ECIES** scheme (Elliptic Curve Integrated Encryption Scheme): an ephemeral ECDH key exchange against the home network's public key, a KDF to derive a symmetric key, then a symmetric AEAD over the SUPI. Curve25519 (Profile A) and NIST P-256 (Profile B) are the standardized choices – the very curves from Chapter 21, now embedded in a SIM.

Why it works

Why this kills passive and active IMSI catching of the permanent id.

- *Confidentiality of identity.* ECIES is semantically secure encryption to the home network's public key. An attacker sees SUCI but cannot recover SUPI without the home private key. IMSI harvesting – the one attack that survived all the way through 4G – stops working.

- *Unlinkability.* ECIES uses a fresh ephemeral key per encryption, so two SUCIs from the same subscriber look independent. The catcher cannot say "that is the same phone as yesterday" from the concealed identifier alone.
- *No new trust root.* The home network already shares secrets with the SIM; provisioning one more public key onto the SIM costs nothing in trust. There is no PKI/certificate problem here – the SIM trusts exactly one key, its own operator's, baked in at manufacture.

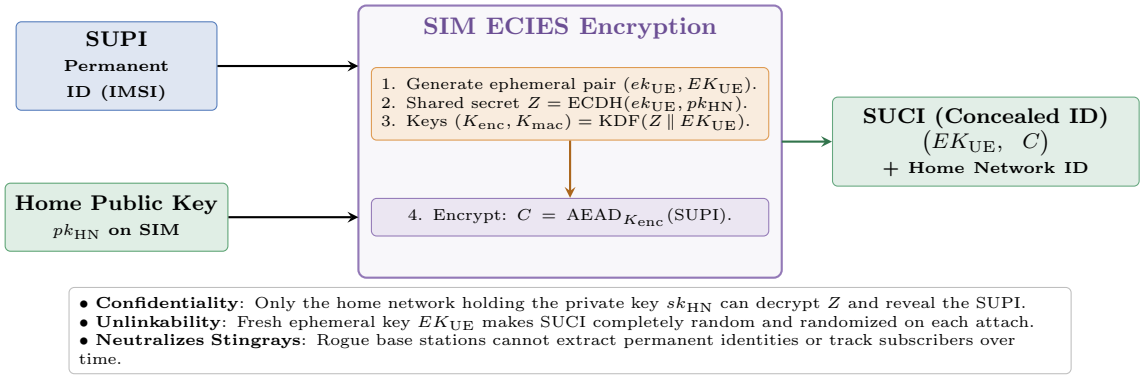


Figure 29.7: 5G Subscription Concealed Identifier (SUCI) generation via ECIES. Before transmitting its identity over the radio link, the 5G SIM encrypts the permanent SUPI under the home network's public key using ephemeral Curve25519 or P-256 Diffie–Hellman, defeating IMSI-catcher tracking completely.

29.7 A toy SUCI: ECIES identity concealment in miniature

To make the 5G identity-concealment concrete, here is a stripped-down ECIES. It shows the exact shape of what a 5G SIM does to a SUPI: ephemeral key, shared secret via ECDH, derive a symmetric key, encrypt. We use X25519 for the ECDH and a hash-based KDF, then a one-time-pad-style symmetric step to keep it tiny.

Implement it

Learning only – never deploy this. This toy omits the AEAD authentication tag, uses a hand-rolled KDF and a pad rather than a vetted AEAD, has no padding or format checks, and leaks length. Real 5G concealment uses standardized ECIES profiles with AES-GCM or similar. Use a real library and the 3GPP profiles in production.

```
# Toy 5G SUPI -> SUCI concealment with X25519 ECDH + hash KDF.
# Demonstrates the SHAPE of ECIES identity hiding. Do not deploy.
from cryptography.hazmat.primitives.asymmetric.x25519 import (
    X25519PrivateKey, X25519PublicKey)
from cryptography.hazmat.primitives import hashes

# Home network's long-term keypair (the SIM is provisioned with home_pub).
home_priv = X25519PrivateKey.generate()
```

```python
home_pub  = home_priv.public_key()

def kdf(shared, salt):                          # toy KDF: hash the ECDH secret
    h = hashes.Hash(hashes.SHA256())
    h.update(salt + shared)
    return h.finalize()

def conceal(supi: bytes, home_pub):
    # SIM side: fresh ephemeral key every time -> unlinkable SUCI.
    eph_priv = X25519PrivateKey.generate()
    eph_pub  = eph_priv.public_key()
    shared   = eph_priv.exchange(home_pub)  # ECDH against home public key
    key      = kdf(shared, b"5G-SUCI")
    ct       = bytes(a ^ b for a, b in zip(supi, key))  # toy pad (NO MAC!)
    return eph_pub.public_bytes_raw() + ct  # SUCI = eph pubkey || ciphertext

def deconceal(suci: bytes, home_priv):
    eph_raw, ct = suci[:32], suci[32:]
    eph_pub = X25519PublicKey.from_public_bytes(eph_raw)
    shared  = home_priv.exchange(eph_pub)   # same ECDH secret, other side
    key     = kdf(shared, b"5G-SUCI")
    return bytes(a ^ b for a, b in zip(ct, key))

supi = b"imsi-310150123456789"
s1 = conceal(supi, home_pub)
s2 = conceal(supi, home_pub)
print("two SUCIs differ:", s1 != s2)        # True: unlinkable
print("home recovers   :", deconceal(s1, home_priv) == supi)  # True
# A fake base station holds neither home_priv nor a stable handle: it is stuck.
```

Run it and two calls to `conceal` on the *same* SUPI give two different SUCIs (fresh ephemeral key each time), yet the home network recovers the SUPI from either. That is precisely the unlinkability-plus-recoverability that defeats the IMSI catcher's identity harvesting.

Common trap

5G concealment protects the *permanent* identity, not necessarily everything. The temporary identifier (5G-GUTI) is still used after attach, and if it is not refreshed often enough it can become a tracking handle – a known weakness researchers have flagged. And 5G non-standalone deployments still ride on a 4G core, inheriting some 4G exposures; downgrade attacks to older generations remain a concern wherever 2G/3G are still switched on. 5G closes the most glaring exposure (the cleartext IMSI), but it does not by itself make a phone untrackable, and "5G" on your status bar may indicate non-standalone (NSA) mode still anchored to LTE security.

29.8 The constraints that shaped all of this

Why did telecom crypto look the way it did, and why does it still differ from web crypto? Four constraints explain it, and none has fully gone away.

- **Tiny, cheap, battery-bound endpoints.** A 1990 SIM had on the order of kilobytes of memory and a glacial clock. Public-key operations (modular exponentiation, EC point multiplication) were out of the question on the air-interface critical path, which is why GSM and even LTE authentication are pure symmetric-key. 5G can afford one ECIES operation at attach because silicon caught up, not because the constraint vanished.
- **A brutal radio channel.** The air interface drops and corrupts bits constantly, and bandwidth is precious. Stream ciphers (xor a keystream) were chosen because they encrypt bit-by-bit with no block alignment, no ciphertext expansion, and graceful behavior under error – a corrupted bit corrupts one bit, not a whole block. The price was malleability and the catastrophic cost of any keystream reuse.
- **Backward compatibility forever.** Networks must serve a 12-year-old handset. That is why 2G fallback existed for decades, and why a 5G phone can still be dragged down to attackable GSM. Security improvements in telecom land on top of an installed base measured in billions; you cannot flag-day the world.
- **Lawful intercept and export politics.** Operators must, by law in most jurisdictions, be able to wiretap under warrant – so end-to-end encryption of call content was never a design goal of the bearer (it is added on top, by apps like Signal). And export controls actively mandated weak ciphers (A5/2) and short keys. Much of GSM's weakness came from policy rather than incompetence, and that policy left holes anyone could exploit.

In the real world

What this means for you on the ground.

- Cellular encryption protects the *radio link to the tower*, and historically did it badly. It has never protected your call *end to end* – inside the operator's network, and at the far end, your voice is in the clear to the carriers (and to lawful intercept). If you want real end-to-end secrecy for a conversation, use an application that provides it (Signal and friends) over the data bearer, and treat the cellular layer as an untrusted transport.
- IMSI catchers are real and commercially sold. On modern phones, disabling 2G ("2G off" / "allow 2G: never") removes the easiest downgrade path; some operating systems warn on null-cipher (A5/0) connections. 5G SUCI concealment finally protects your permanent identity, but only in standalone 5G and only if the GUTI is refreshed – so do not assume the 5G icon means you are untrackable.
- For engineers: the SIM is a deployed, tamper-resistant, key-holding secure element in nearly every pocket on the planet. That infrastructure is reused well beyond telephony – eSIM, embedded SIMs in IoT, and SIM-based authentication to Wi-Fi (EAP-SIM / EAP-AKA) all lean on the same K_i/AKA machinery you have just learned.

Connection

The pieces of telecom crypto are pieces you have already met. The air cipher A5/1 is an LFSR-based stream cipher – see the LFSR analysis and the two-time-pad disaster in Chapter 10; the same linearity that broke the toy keystream there is what made A5/2 fall to linear algebra. AKA's $f1$ is a MAC and the $f2$–$f5$ are PRFs; their security is the PRF/MAC story of Chapters 11 and 15. 5G's SUCI is ECIES: ephemeral ECDH (Chapters 18 and 21) plus a KDF plus an AEAD (Chapter 16), assembled exactly as the hybrid-encryption pat-

tern prescribes. The cost of that one EC point multiplication on a constrained SIM is the modular-exponentiation/EC-cost discussion in the companion algorithms book; the unlinkability argument – that fresh randomness makes two ciphertexts independent – is the semantic-security/indistinguishability argument restated for a privacy goal.

Cellular cryptography secures the first hop over the radio between a handset and a base station. To extend secure channels across arbitrary, untrusted wide-area networks – linking entire branch offices or remote workers back to an enterprise core – we turn to network-layer tunnels. In the next chapter, we examine Virtual Private Networks: IPsec, OpenVPN, and WireGuard.

29.9 Exercises

Exercise 29.1 (Recover a toy key from one challenge)**.** Using the toy keyed function $F(K, x) = (3K{+}7x{+}11) \bmod 256$ from section 29.2.1, an eavesdropper captures the single pair $(\text{RAND}, \text{SRES}) = (50, 200)$. Compute $3^{-1} \bmod 256$, then solve for K. Explain in one sentence why a real $A3$ must be nonlinear so that this attack fails.

Exercise 29.2 (Trace a GSM triplet)**.** With the same F and $K_i = 73$, let the AuC choose $\text{RAND} = 17$. Compute $\text{SRES} = F(K_i, \text{RAND})$ and $K_c = F(K_i, \text{RAND} + 1)$, write out the triplet the AuC ships to the visited network, and state which one of the three values the visited network must *not* be able to use to recover K_i. Which values does the eavesdropper see on the air?

Exercise 29.3 (Why the missing arrow matters)**.** GSM authentication is one-way. Describe, step by step, how an IMSI catcher exploits this to (a) learn a victim's IMSI and (b) read the victim's call, naming the specific protocol message it sends in each step. Then state which one AKA field would have stopped step (b) and why.

Exercise 29.4 (The A5/2 cross-protocol attack)**.** Explain why a network that supports both A5/1 and A5/2 is no stronger than one that supports only A5/2, given that K_c is computed independently of the chosen A5 variant. Your answer should make clear what an active attacker does and why the recovered K_c is useful against an A5/1 session.

Exercise 29.5 (Effective key length)**.** A5/1 takes a 64-bit K_c, but some deployments forced the lowest 10 bits to zero. (a) What is the effective key space? (b) If a GPU rig tests 2^{30} A5/1 states per second, give an order-of-magnitude time to brute-force the reduced key, and compare with the full 64-bit space. (c) Why does a precomputed time–memory tradeoff change this calculus entirely?

Exercise 29.6 (AKA replay defense)**.** In 3G/4G AKA the SIM checks both the MAC inside AUTN and that SQN lies in an acceptable window. Construct a replay attack that the MAC check alone would *not* stop, and show how the SQN window stops it. Then explain what "resync" does when the SIM's stored SQN and the network's drift apart, and why an attacker cannot abuse resync to learn K.

Exercise 29.7 (Anonymity key)**.** AUTN sends $\text{SQN} \oplus \text{AK}$ rather than SQN in the clear, where $\text{AK} = f5_K(\text{RAND})$. (a) What does an eavesdropper learn about a subscriber if SQN is sent in the clear and slowly increments per attach? (b) Show that the SIM can still recover SQN despite the masking. (c) Why is xor with $f5_K(\text{RAND})$ a safe mask even though K is reused across many different RANDs?

Exercise 29.8 (SUCI unlinkability)**.** In the toy of section 29.7, two encryptions of the same SUPI yield different SUCIs. (a) Identify the exact line of code that provides unlinkability and explain why removing it (reusing one ephemeral key) would let a passive attacker track the subscriber. (b) The toy uses an unauthenticated pad. Describe a concrete bit-flipping attack on the concealed identity and name the AEAD property that 5G's real ECIES profile uses to prevent it.

Exercise 29.9 (Stream-cipher malleability on the air)**.** GSM provides no integrity on the air interface, and A5/1 is a stream cipher $c = m \oplus \text{ks}$. (a) Show that an attacker who knows the plaintext at some bit positions can flip those plaintext bits in the decrypted output without knowing K_c. (b) Explain why AKA's separate integrity key IK and a real MAC on signaling messages closes this, and why a merely stronger A5 (confidentiality only) would not have.

Exercise 29.10 (Constraints drive design)**.** For each of the four telecom constraints in section 29.8 (tiny endpoints, brutal radio channel, eternal backward compatibility, lawful-intercept/export politics), name one specific design decision in GSM or LTE that the constraint forced, and state whether 5G relaxed, kept, or worsened that decision. Which constraint do you think will most shape 6G security, and why?

Chapter 30

VPNs and Tunnels

In the previous chapter, we secured the radio hop between cellular handsets and base stations. Once traffic leaves the base station and enters the wider internet—or when a remote worker needs to reach an internal network across untrusted links—protection has to span arbitrary IP hops. A virtual private network exposes an encrypted tunnel to the operating system as an ordinary virtual network interface. Its goal is simple to state: carry packets across a public network as if the endpoint were plugged directly into a private one, and protect them in transit. Everything subtle about VPNs lies in the gap between what that abstraction suggests and what the cryptography actually guarantees.

Earlier chapters developed the core primitives: authenticated encryption (Chapter 16), Diffie–Hellman key exchange (Chapter 18), forward secrecy (Chapter 23), elliptic curves (Chapter 21), and the way TLS stitches them into a transport protocol (Chapter 25). A VPN is what you get when you point those same tools at a different target. Instead of protecting one TCP connection between a browser and a web server, a VPN protects *whole packets*—and sometimes whole networks—between two endpoints that have agreed to trust each other.

This chapter does three things. First, it nails down precisely what a tunnel is, and the threat model: what a VPN protects, what it leaks anyway, and the popular fictions sold under the word "privacy." Second, it works through the three families that matter in practice—IPsec (the old, sprawling, standards-body design), WireGuard (the modern minimal design built on the Noise protocol and Curve25519), and TLS-based VPNs (OpenVPN and friends). Third, it shows you concrete failure modes: a downgrade, a replay, a key-reuse catastrophe, and the difference between encrypting a packet and hiding that you sent it.

30.1 What a tunnel actually is

Start with the bare mechanism, no crypto yet. A packet on the Internet has a destination address in its header, and routers along the way read that address to forward it. A *tunnel* is the trick of taking a whole packet—header and all—and stuffing it inside the payload of *another* packet addressed somewhere else. The outer packet travels to that somewhere-else, which unwraps it and releases the inner packet onto a different network. This is called *encapsulation*.

Intuition

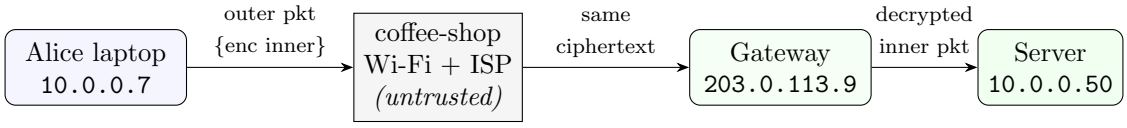
Think of mailing a sealed envelope inside another envelope. The inner envelope says "To Bob, Apartment 4, Internal Mailroom." The post office never sees that; it only reads the outer envelope, "To the Mailroom Building, 5th Avenue." The mailroom opens the outer envelope and drops the inner one into Bob's slot. The public postal system routed a letter to an address it could not itself deliver to, by wrapping it in one it could.
A VPN is exactly this, plus the inner envelope is encrypted and sealed so the postal workers cannot read it or forge it.

Concretely, suppose Alice's laptop has private address `10.0.0.7` on her company's network, and the company gateway sits at public address `203.0.113.9`. Alice is at a coffee shop with public address `198.51.100.4`. When Alice's machine wants to reach the internal server `10.0.0.50`, it builds an inner IP packet:

```
src=10.0.0.7  dst=10.0.0.50  [TCP payload]
```

The VPN software encrypts that entire inner packet, then wraps it in an outer packet the coffee-shop router can actually deliver:

```
src=198.51.100.4  dst=203.0.113.9  [ENCRYPTED inner packet]
```



Two design dimensions fall out of this picture, and they recur in every VPN technology:

- **Tunnel mode vs. transport mode.** In *tunnel mode* you encapsulate the entire inner IP packet (header included), as above; the endpoints can be gateways protecting whole subnets. In *transport mode* you keep the original IP header and only protect the payload; this is for two hosts talking directly, with no address translation. Tunnel mode is what people mean by "VPN"; transport mode is a host-to-host special case.
- **Site-to-site vs. remote-access.** A *site-to-site* VPN joins two networks (the New York office and the London office) so machines on each see the other as local. A *remote-access* VPN connects one roaming device (Alice's laptop) into a network. The crypto is the same; the routing and address assignment differ.

Focus

A VPN is two ideas glued together: (1) *encapsulation*, a pure networking trick that re-routes packets through a chosen endpoint, and (2) *authenticated encryption* of the encapsulated payload, the cryptographic trick that makes the channel private and tamper-evident. Without the crypto you still have a tunnel—an unprotected one, like GRE or plain IP-in-IP. Without the encapsulation you have ordinary transport security like TLS on a single connection. A VPN needs both.

30.2 The threat model, stated precisely

Before any protocol, pin down who Eve is and what she can do. This is where most public confusion about VPNs lives, so we will be precise.

The channel adversary. The classic VPN adversary is a Dolev–Yao network attacker who controls the path between Alice and the gateway: she sees every outer packet, can drop, reorder, replay, and inject packets, and can read and rewrite anything not cryptographically protected. This is the coffee-shop Wi-Fi snoop, the malicious ISP, the airport router, the nation-state tapping a backbone link. Against *this* adversary a good VPN delivers a lot.

Formal

A VPN tunnel should provide, for the encapsulated traffic, the same guarantees as a secure channel:

- **Confidentiality.** Eve learns nothing about inner-packet contents beyond length and timing.
- **Integrity / authenticity.** Eve cannot forge or modify an inner packet that the receiver will accept; every accepted packet was sent by the holder of the key.
- **Replay resistance.** Eve cannot resubmit a previously valid packet and have it accepted as fresh.
- **Forward secrecy** (for good designs). Compromise of long-term keys today does not decrypt traffic Eve recorded yesterday.

We will hold every scheme below up to this list.

Now the part the marketing leaves out. A VPN moves the trust boundary; it does not abolish it.

Common trap

What a VPN does *not* protect.

- **The endpoints.** Everything is plaintext at Alice's laptop and at the gateway. Malware on either end sees it all. The gateway operator (your employer, or a commercial "VPN provider") sees every site you visit, because the tunnel *terminates there* and your traffic exits onto the real Internet from that point. You did not become anonymous; you swapped "my ISP can see me" for "my VPN provider can see me."
- **Traffic metadata.** Eve still sees that Alice's IP is sending a steady stream of encrypted packets to `203.0.113.9`, how many, how big, and when. Packet *lengths* and *timing* leak: a VPN does not hide that you are on a video call, and often not even which site you loaded (see traffic analysis below).
- **What the server logs.** The destination web server sees the gateway's IP, not Alice's—good for hiding her home IP—but it still sets cookies, fingerprints her browser, and logs her login. "The website cannot identify me because I used a VPN" is false the moment she signs in.
- **DNS, unless you are careful.** If name resolution leaks outside the tunnel (the infamous *DNS leak*), Eve learns every hostname even though the traffic is encrypted.

- **The existence of the tunnel.** Encryption hides contents, not the fact that you are running a VPN. A censor can often detect and block VPN traffic by its fingerprint even without breaking it.

In the real world

The one-line summary for the public: a VPN *relocates* your traffic's exit point and *protects the first hop*. That is genuinely useful—on hostile Wi-Fi, against a snooping local network, to reach a corporate intranet, or to make your traffic appear to originate elsewhere. It is not anonymity (use Tor for that, and understand its limits too), and "no-logs" is a legal and operational claim about a company, not a cryptographic guarantee you can verify. Several providers advertising "military-grade encryption" and "total anonymity" have, when subpoenaed, produced exactly the logs they swore they did not keep.

History

The first widely deployed tunneling protocol, the Point-to-Point Tunneling Protocol (PPTP), was published in 1996 by an industry consortium led by Gurdeep Singh-Pall at Microsoft. Designed to let telecommuters dial into enterprise local networks over commodity dial-up ISPs, PPTP wrapped PPP frames inside Generic Routing Encapsulation (GRE) packets, authenticating users with MS-CHAPv2 and encrypting payloads using MPPE with the RC4 stream cipher. Its security was soon shown to be badly flawed. Cryptanalysts, notably Bruce Schneier and Mudge in 1998, showed that MS-CHAPv2 password hashes could be recovered offline, and Moxie Marlinspike and David Hulton in 2012 demonstrated that MS-CHAPv2 reduces to a single DES key crackable in under a day. Worse, MPPE reset RC4 state insecurely across packets, producing keystream reuse. PPTP's failures pushed the IETF toward standardizing IPsec. Today, the open operational frontier is transitioning enterprise VPN fleets from sprawling, complex IKE suites to lean, formally verified protocols like WireGuard and post-quantum hybrids.

30.3 IPsec: the standards-body design

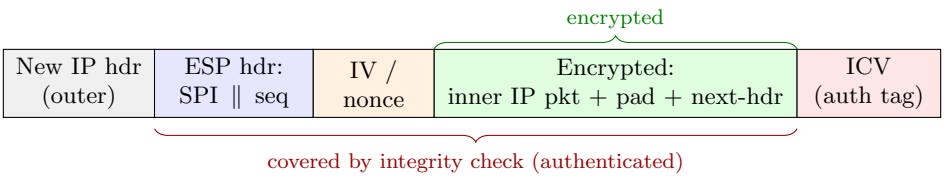
IPsec is the IETF's answer to "encrypt IP itself." It is not one protocol but a suite, and it is notoriously sprawling—negotiable algorithms, two key-exchange protocol versions, multiple modes. We will extract the usable core.

IPsec has two data-plane protocols and a control-plane protocol:

- **AH (Authentication Header)** authenticates a packet but does not encrypt it. Almost nobody uses AH today; it conflicts badly with NAT (it authenticates parts of the outer header that NAT rewrites). We mention it and move on.
- **ESP (Encapsulating Security Payload)** is the primary payload carrier: it encrypts *and* authenticates the payload. This is what carries your data.
- **IKE (Internet Key Exchange)** is the handshake that authenticates the peers and derives the keys ESP uses. IKEv2 (RFC 7296) is the modern version; use it, not IKEv1.

30.3.1 ESP: the data plane

A modern ESP packet in tunnel mode, using an AEAD cipher (AES-GCM is the standard choice), looks like this:



Two unencrypted header fields carry the state the receiver needs:

- The **SPI (Security Parameter Index)** is a 32-bit tag that names *which* security association—which set of keys and algorithms—this packet belongs to. The receiver may hold many tunnels at once; the SPI tells it which keys to grab. It is a lookup index, not a secret.
- The **sequence number** is a monotonically increasing 32-bit (or 64-bit, extended) counter. The sender increments it per packet; the receiver tracks a sliding window of recently seen numbers and rejects duplicates or anything too old. This is the anti-replay mechanism.

Why it works

Why ESP-with-AEAD is secure (as a channel). ESP with AES-GCM is, packet by packet, just authenticated encryption with associated data. The plaintext is the inner IP packet plus padding plus a "next header" byte; the associated data is the ESP header (SPI and sequence number); the nonce is derived from the sequence number. Under the standard AEAD security definition, an adversary without the key cannot (a) distinguish the ciphertext of two equal-length inner packets, giving confidentiality, nor (b) produce any new (header, ciphertext, tag) triple the receiver accepts except with negligible probability, giving integrity. The sequence number, fed into the integrity check and checked against the replay window, upgrades per-packet integrity to *stream* integrity: Eve cannot replay, reorder undetectably, or splice.

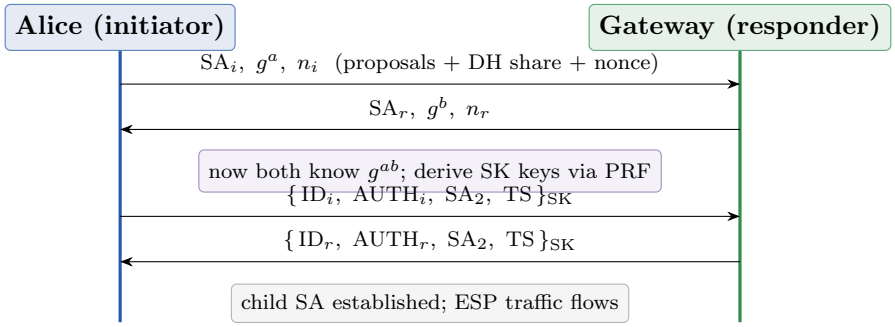
The single most important rule: **the GCM nonce must never repeat under one key.** Because the nonce is built from the per-packet sequence number and the sequence number strictly increases, it never repeats *within* a security association—which is exactly why IKE must rekey before the counter wraps. Repeat a GCM nonce and you leak the XOR of two plaintexts and, far worse, you leak the GHASH authentication key, letting Eve forge arbitrary packets. We will see this failure concretely below.

30.3.2 IKE: the control plane

ESP needs keys. IKE is a Diffie–Hellman handshake that authenticates both peers and produces a shared secret, from which all the ESP keys are derived. IKEv2 runs in two exchanges:

1. **IKE_SA_INIT.** The peers do an (anonymous) Diffie–Hellman exchange and negotiate algorithms. After this exchange they share a secret but have not yet proven who they are.

2. **IKE_AUTH.** Under the encryption keys from step 1, each peer proves its identity—by a pre-shared key (PSK), an RSA/ECDSA signature, or EAP—and they set up the first ESP security association.



The shared secret g^{ab} is fed into a PRF (an HMAC, typically) together with the nonces to derive a key tree: SK_d (for deriving child SA keys), $\mathrm{SK}_{ai}, \mathrm{SK}_{ar}$ (integrity, each direction), $\mathrm{SK}_{ei}, \mathrm{SK}_{er}$ (encryption, each direction), and $\mathrm{SK}_{pi}, \mathrm{SK}_{pr}$ (for the AUTH payload). Because the DH exponents a, b are ephemeral and discarded after the handshake, IKEv2 gives **forward secrecy**: recording the traffic and later stealing the PSK or signing key does not recover g^{ab}.

The AUTH payload is where identity binds to the handshake. Each side signs (or MACs, under the PSK) a transcript of the messages it sent and the other's nonce. This is the same trick TLS uses: by signing the transcript, a peer proves both "I am who I claim" and "I saw exactly these handshake messages," which defeats a man-in-the-middle who tried to substitute her own DH share.

Implement it

The IKEv2 key schedule, with toy numbers. The whole tree hangs off one seed value $\mathrm{SKEYSEED} = \mathrm{PRF}(n_i \,\|\, n_r,\ g^{ab})$—the nonces key the PRF, the DH secret is the message. From SKEYSEED, IKEv2 runs PRF+, a counter-mode expansion that chains the PRF to produce as many key bytes as the negotiated algorithms need:

$$\mathrm{PRF{+}}(K, S) = T_1 \,\|\, T_2 \,\|\, T_3 \,\|\, \cdots, \quad T_1 = \mathrm{PRF}(K, S \,\|\, \texttt{0x01}),\ T_{j+1} = \mathrm{PRF}(K,\ T_j \,\|\, S \,\|\, (j{+}1)).$$

The output stream is sliced, in fixed order, into $\mathrm{SK}_d \,\|\, \mathrm{SK}_{ai} \,\|\, \mathrm{SK}_{ar} \,\|\, \mathrm{SK}_{ei} \,\|\, \mathrm{SK}_{er} \,\|\, \mathrm{SK}_{pi} \,\|\, \mathrm{SK}_{pr}$. Suppose (absurdly small, for illustration) the PRF were a function returning one byte and we needed three one-byte keys $\mathrm{SK}_d, \mathrm{SK}_{ai}, \mathrm{SK}_{ar}$. With SKEYSEED $= K$ and seed $S = n_i \,\|\, n_r \,\|\, \mathrm{SPI}_i \,\|\, \mathrm{SPI}_r$:

- $T_1 = \mathrm{PRF}(K, S \,\|\, \texttt{01})$ becomes SK_d,
- $T_2 = \mathrm{PRF}(K, T_1 \,\|\, S \,\|\, \texttt{02})$ becomes SK_{ai},
- $T_3 = \mathrm{PRF}(K, T_2 \,\|\, S \,\|\, \texttt{03})$ becomes SK_{ar}.

The chaining (T_2 depends on T_1) means you cannot compute a later key without the earlier ones, and every key depends on both nonces, both SPIs, and the DH secret. Two properties fall out for free: distinct directions get distinct keys (so a packet captured in one direction can never be replayed into the other—they verify under different SK_a), and re-running the same handshake with fresh nonces yields a completely independent key tree even if g^{ab} somehow repeated.

30.3.3 Rekeying, dead peers, and roaming

A long-lived tunnel cannot ride one key forever. Two clocks force a rekey. The first is the AEAD data limit: as the ESP sequence number climbs toward wrap, the GCM nonce would eventually repeat (see the nonce-reuse catastrophe below), so IKEv2 negotiates a *child SA* rekey—a fresh Diffie–Hellman (CREATE_CHILD_SA with a new DH group) that installs new ESP keys and retires the old SPI. The second is simply policy: rekey every hour, or every few gigabytes, whichever comes first. Because each rekey can carry its own ephemeral DH, forward secrecy is renewed continuously—an attacker who compromises today's session keys learns nothing about the segment of traffic before the last rekey.

Two operational pieces round out IKEv2 in the field:

- **Dead Peer Detection (DPD).** Either side periodically sends an encrypted, acknowledged "are you there?" (an informational exchange). Silence past a threshold tears down the SA, freeing state and prompting a reconnect. Without it, a gateway leaks memory to half-open tunnels whose clients vanished.
- **MOBIKE (RFC 4555).** The mobility extension lets an established SA survive a change of the client's outer IP address. When Alice's phone hops from Wi-Fi to LTE, MOBIKE updates the SA's endpoint address *without* a new handshake—the keys are unchanged, only the routing of the outer packet moves. This is why IKEv2 is the default "always-on" phone VPN: the tunnel does not drop every time the radio changes.

Why it works

Why the man-in-the-middle fails. Suppose Mallory sits between Alice and the gateway and swaps in her own DH shares, so she shares $g^{am'}$ with Alice and g^{mb} with the gateway. She can decrypt and re-encrypt the encrypted handshake messages. But IKE_AUTH requires each party to authenticate a transcript that *includes the DH share the other party actually sent*. The gateway signs over the value it received; Alice expects a signature over the value *she* sent. Mallory cannot produce the gateway's signature over Alice's original share (she lacks the gateway's signing key), and she cannot make Alice's share equal the one she forwarded without breaking DH. The AUTH check fails on at least one side and the tunnel never forms. Authenticating the key exchange transcript is the critical barrier; an unauthenticated Diffie–Hellman exchange is wide open to interception.

Attack

Aggressive mode and offline PSK cracking (IKEv1). The older IKEv1 had a "main mode" and a faster "aggressive mode." Aggressive mode with a pre-shared key sends a hash of the PSK *before* the channel is encrypted, and—worse—an attacker can simply *ask* a misconfigured responder for it by initiating a handshake. The captured hash $H = \mathrm{PRF}(\mathrm{PSK},\ g^a \,\|\, g^b \,\|\, \cdots)$ is then a target for an offline dictionary attack: guess a PSK, recompute H, compare. Tools like `ike-scan` plus `psk-crack` automate this. The lessons: never use IKEv1 aggressive mode with a guessable PSK; prefer certificate authentication; and remember that "it is encrypted" says nothing if the secret is a human-chosen password exposed to offline guessing.

Attack

Logjam and weak DH groups. IKE negotiates a Diffie–Hellman group. Several IPsec deployments offered 768- or 1024-bit groups, and what made this devastating in practice was that millions of servers shared the *same* standardized Oakley prime. The Logjam work (2015) showed that the expensive number-field-sieve precomputation for discrete log can be done *once per prime*, after which individual exchanges fall quickly. A 1024-bit standard group is within reach of a nation-state budget; the authors argued this plausibly explained bulk IPsec decryption in leaked intelligence slides. Fix: use 2048-bit groups or, better, elliptic-curve groups (Curve25519, P-256), and do not all share one prime. This is the same Logjam that hit TLS—the lesson transfers wholesale because the math is identical.

30.4 WireGuard: the minimal modern design

Where IPsec offers many negotiable options, WireGuard (Jason Donenfeld, 2016) is deliberately minimal. It fits in a few thousand lines of kernel code, has *no* algorithm negotiation, and is built directly on the Noise protocol framework. The design philosophy is: pick good primitives, hard-code them, and make misconfiguration nearly impossible (Figure 30.1).

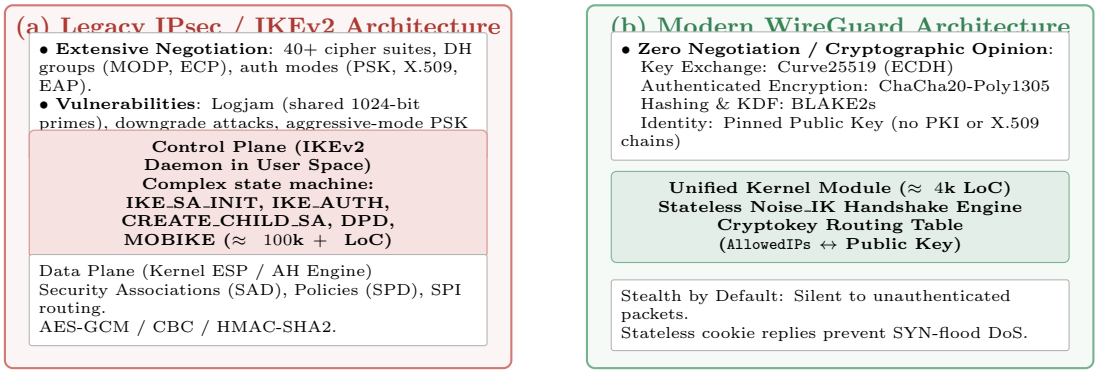


Figure 30.1: Architectural comparison between IPsec/IKEv2 (a) and WireGuard (b). IPsec delegates complex negotiation and certificate validation across massive user-space daemons, creating a vast attack surface. WireGuard enforces cryptographic minimalism, embedding a simple 1-RTT Noise pattern directly in the kernel.

The fixed cipher suite, no negotiation:

- **Curve25519** for the Diffie–Hellman key exchange (Chapter 21).
- **ChaCha20-Poly1305** for authenticated encryption of data packets (Chapter 16).
- **BLAKE2s** for hashing and as the HMAC core in key derivation.
- **HKDF** (over BLAKE2s) to derive session keys.

Intuition

Algorithm negotiation is a recurring source of protocol vulnerabilities. Every "which cipher shall we use?" message creates a potential downgrade vector for an attacker and invites misconfiguration (as we saw earlier with PPTP and Logjam—both are negotiation failures). WireGuard removes the question entirely. There is exactly one cipher suite. If it is ever broken, you ship a new protocol version, not a new negotiated option. This is "cryptographic agility considered harmful," and for a tunnel it is the right call.

30.4.1 Identity is a public key

In WireGuard there are no certificates, no usernames, no PKI. A peer *is* its Curve25519 static public key. You configure your own private key and a list of peers, each pinned to its public key and the IP ranges (`AllowedIPs`) it is permitted to use. This is the SSH `authorized_keys` model, applied to a network tunnel: trust is a list of public keys you typed in, not a chain you validate.

30.4.2 The handshake: Noise IK

WireGuard's handshake is the **Noise_IK** pattern. The names encode the situation: I = the initiator's static key is *Immediately* transmitted (encrypted), K = the responder's static key is *Known* to the initiator in advance (you configured it). Because the responder's key is known up front, the initiator can encrypt to it from the very first message—no round trip wasted establishing who the responder is.

A Noise handshake threads every key through a running hash and a chaining key. Let us write the core with small notation. Each party has a static keypair $(s, S = g^s)$ and generates an ephemeral keypair $(e, E = g^e)$ per handshake (g is the Curve25519 base point; g^x means scalar multiplication). Write $\mathrm{DH}(x, Y) = Y^x$ for the shared point. The handshake performs a sequence of DH operations and mixes each result into a chaining key C via HKDF:

Figure 30.2 illustrates the Noise_IK message ladder, showing the sequence of DH operations and how each result is folded into the chaining key.

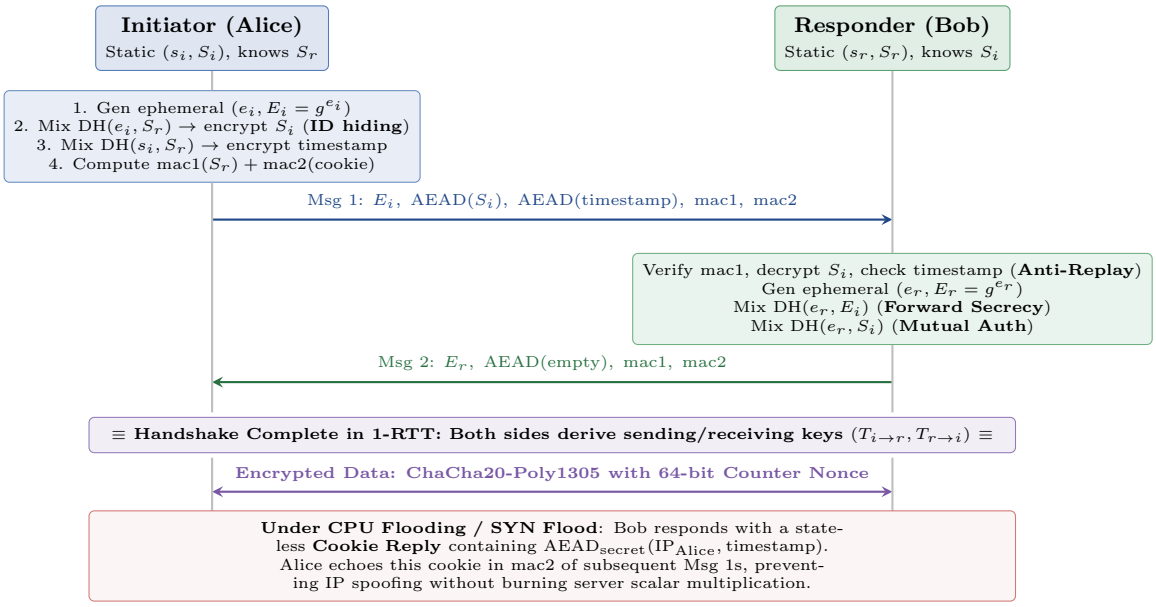


Figure 30.2: The WireGuard Noise_IK 1-RTT handshake protocol ladder. By leveraging the known static public key of the responder (S_r), the initiator immediately encrypts its identity and timestamp in the first packet. The protocol achieves mutual authentication, identity hiding, forward secrecy, and anti-DoS cookie defense without certificates or signatures.

The chaining key absorbs four DH outputs across the two messages: ephemeral–static, static–static, ephemeral–ephemeral, and ephemeral–static the other way. The mix of *both* ephemerals gives forward secrecy; the mix of *both* statics gives mutual authentication. Because each side proves it can compute DH with the other's static key, identity is bound to the handshake without a single signature.

Why it works

Why mixing DH outputs authenticates without signatures. The final session key is a hash of all four DH results. The responder can compute DH(s_r, S_i) = DH(s_i, S_r) only if it holds s_r; the initiator includes that term by computing DH(s_i, S_r). If Mallory does not know s_r, she cannot compute the static–static term, so she derives a different key and her data packets fail Poly1305 verification. The ephemeral–ephemeral term DH(e_i, e_r) requires at least one fresh secret she does not have, giving forward secrecy: even if she later steals s_i and s_r, the ephemerals e_i, e_r were discarded, so recorded sessions stay closed. It is the same logic as authenticated DH in IKE, but accomplished purely by *which keys you DH together*, with no transcript signature at all.

Why it works

Identity hiding and the stealth design. The initiator's static public key S_i is encrypted in message 1 (under a key derived from DH(e_i, S_r)), so a passive eavesdropper does not learn who is connecting—only the responder, who can decrypt it, does. In addition, WireGuard does not reply to unauthenticated packets: if the first DH-derived MAC does not check out, the server is silent, as if no service were listening. This makes a WireGuard endpoint hard to discover by scanning—an attacker who does not already know a valid peer key gets no

response at all.

Walking question

Trace which of the four DH outputs each party can compute, and *when*, to see that the key is bound to both identities. Label the secrets: the initiator holds e_i, s_i and knows S_r; the responder holds e_r, s_r and (after decrypting msg1) learns S_i.

- $\mathrm{DH}(e_i, S_r) = \mathrm{DH}(s_r, E_i)$: the initiator computes it from e_i, S_r at send time; the responder computes the equal value from s_r, E_i on receipt. This is the key that *encrypts* S_i in msg1, so only the holder of s_r can unwrap the initiator's identity—identity hiding.
- $\mathrm{DH}(s_i, S_r) = \mathrm{DH}(s_r, S_i)$: the static–static term. The responder can match it only with s_r; the initiator only with s_i. Folding it into the chaining key means a correct final key proves *both* long-term identities—mutual authentication.
- $\mathrm{DH}(e_r, E_i) = \mathrm{DH}(e_i, E_r)$: the ephemeral–ephemeral term. Neither long-term key appears, so even total static-key compromise later cannot reconstruct it—forward secrecy.
- $\mathrm{DH}(e_r, S_i) = \mathrm{DH}(s_i, E_r)$: binds the responder's fresh ephemeral to the initiator's identity, completing the responder's confidence that it is talking to S_i and not a replay.

A man-in-the-middle who lacks s_r is stuck at the very first term: she cannot decrypt S_i, and even if she guessed it she cannot produce the static–static term, so her derived key diverges and every subsequent message fails Poly1305. No signature was used anywhere; *which keys you DH together* carries the entire authentication argument.

Attack

Handshake floods and the cookie reply. A naive responder that ran a Curve25519 scalar multiply for every msg1 it received would be a denial-of-service target: an attacker spraying forged msg1 packets from spoofed source addresses could pin the CPU doing DH for garbage. WireGuard defends with two MACs on each handshake packet. mac1 is keyed by the responder's *public* key, so only someone who knows whom they are dialing can form a packet worth examining—this silences internet background scans cheaply. Under active load the responder switches on mac2: it replies with a *cookie* (an encrypted token bound to the sender's source IP), and refuses to do the expensive DH until a subsequent packet echoes a valid mac2 computed from that cookie. Because the cookie is tied to the source address, an attacker spoofing addresses cannot collect valid cookies, and the responder spends real CPU only on peers that have proven they can receive at their claimed address. It is the same anti-DoS idea as a TCP SYN cookie, lifted into the handshake's MAC structure.

30.4.3 The data plane and the counter nonce

After the handshake each direction has a symmetric key T and a 64-bit counter N starting at zero. A data packet is

$$\texttt{type} \,\|\, \texttt{receiver_index} \,\|\, N \,\|\, \underbrace{\mathrm{ChaCha20Poly1305}_T(N,\ \text{inner packet})}_{\text{ciphertext} \,\|\, \text{16-byte tag}}.$$

The nonce for ChaCha20-Poly1305 is simply the counter N, incremented once per packet and never reset within a session. No randomness, no negotiation: the counter cannot repeat because it only goes up, and WireGuard rekeys (a fresh handshake every two minutes of use, by default) long before a 64-bit counter could wrap. The receiver uses a sliding window, exactly like IPsec's anti-replay, to reject duplicates and stale packets while tolerating reordering.

Connection

The counter-nonce discipline here applies a result from Chapter 16: ChaCha20-Poly1305 (like AES-GCM) is secure *only* as a nonce-respecting AEAD. Reuse a (key, nonce) pair and Poly1305's one-time-MAC structure collapses—the polynomial evaluation key becomes recoverable, and forgeries follow. WireGuard's "counter that only increases, plus frequent rekeying" is the engineering that guarantees the theorem's hypothesis. The cost of a single DH and one HKDF every two minutes is the price of never having to think about nonce reuse again. See also the modular-exponentiation cost discussion in the companion algorithms book: a Curve25519 scalar multiply is a few hundred microseconds, cheap enough to rekey constantly.

Implement it

A toy sketch of the WireGuard *data-plane* transform (NOT the handshake), to make the counter-nonce idea concrete. This is for learning only—it skips the handshake, key derivation, replay window, framing, and timing-attack hardening that a real implementation needs. Never use it for anything real.

```
# LEARNING TOY -- not secure, not real WireGuard. Do not deploy.
from cryptography.hazmat.primitives.ciphers.aead import ChaCha20Poly1305
import struct

class ToyTunnel:
    def __init__(self, send_key, recv_key):
        self.s = ChaCha20Poly1305(send_key)    # one key per direction
        self.r = ChaCha20Poly1305(recv_key)
        self.send_ctr = 0
        self.recv_seen = set()                 # toy replay check

    def seal(self, inner_packet):
        n = self.send_ctr
        self.send_ctr += 1                     # counter ONLY goes up
        nonce = struct.pack('<Q', n) + b'\x00\x00\x00\x00'  # 12-byte nonce
        ct = self.s.encrypt(nonce, inner_packet, None)
        return struct.pack('<Q', n) + ct       # send counter in the clear

    def open(self, wire):
        n = struct.unpack('<Q', wire[:8])[0]
        if n in self.recv_seen:                # reject replays
            raise ValueError("replay!")
        nonce = struct.pack('<Q', n) + b'\x00\x00\x00\x00'
        pt = self.r.decrypt(nonce, wire[8:], None)  # raises if tag bad
        self.recv_seen.add(n)
        return pt
```

The key line is `self.send_ctr += 1`: the nonce is a monotonic counter, so a (key, nonce) pair

never repeats. The real protocol replaces `recv_seen` (which grows forever) with a fixed-size sliding bitmap window, and replaces the two hard-coded keys with the output of the Noise handshake.

In the real world

WireGuard is now in the mainline Linux kernel (since 5.6), and consumer "VPN apps" increasingly use it (sometimes under their own branding, e.g. NordLynx, Tailscale, Mullvad). Its statelessness simplifies mobility: a roaming phone that changes networks simply sends from its new IP address, and the server updates the peer's endpoint on the next authenticated packet—no reconnection handshake required. The flip side: because a peer's endpoint is whatever address last sent a valid packet, and there is no built-in dynamic IP assignment or DNS push, products layer their own control plane on top (Tailscale's coordination server, for instance, distributes keys and `AllowedIPs` so users never hand-edit configs).

30.5 A worked nonce-reuse catastrophe

The single most important crypto rule in any of these tunnels is "never reuse a nonce under one key." Let us show, with tiny numbers, exactly why a VPN that violates it is not merely weakened but *forgeable*.

Both ChaCha20-Poly1305 and AES-GCM are stream-cipher-plus-polynomial-MAC constructions. The cipher produces a keystream Z depending only on $(\text{key}, \text{nonce})$; the ciphertext is $C = P \oplus Z$. The tag is a polynomial in the ciphertext evaluated at a secret point.

Step 1: two plaintexts under one nonce leak their XOR. Suppose a buggy VPN encrypts two packets P_1, P_2 under the same key and same nonce. Then

$$C_1 \oplus C_2 = (P_1 \oplus Z) \oplus (P_2 \oplus Z) = P_1 \oplus P_2.$$

The keystream cancels. With tiny 1-byte plaintexts: say $P_1 = \texttt{0x48}$ ('H'), $P_2 = \texttt{0x69}$ ('i'), and the keystream byte is $Z = \texttt{0xAC}$. Then $C_1 = \texttt{0x48} \oplus \texttt{0xAC} = \texttt{0xE4}$ and $C_2 = \texttt{0x69} \oplus \texttt{0xAC} = \texttt{0xC5}$. Eve never learns Z, but $C_1 \oplus C_2 = \texttt{0xE4} \oplus \texttt{0xC5} = \texttt{0x21} = \texttt{0x48} \oplus \texttt{0x69}$. Knowing one plaintext (packets are full of known headers) hands her the other.

Step 2: nonce reuse leaks the *authentication* key, which is the real disaster. Poly1305 computes a tag as the evaluation of a polynomial whose coefficients are the ciphertext blocks, at a secret point r, in a prime field $\mathbb{F}_p$ with $p = 2^{130} - 5$:

$$\text{tag} = \Big(\sum_i c_i \, r^{\,k-i} \Big) + s \pmod{p},$$

where r and the one-time pad s are derived from $(\text{key}, \text{nonce})$. The essential fact: r and s are *fixed by the nonce*. If two messages share a nonce, they share the same r and s. Subtracting their two tag equations *cancels* the unknown s and leaves a polynomial equation in the single unknown r:

$$\text{tag}_1 - \text{tag}_2 = \Big(\textstyle\sum c_i^{(1)} r^{k-i} - \sum c_i^{(2)} r^{k-i} \Big) \pmod{p}.$$

Everything but r is known (Eve sees the ciphertexts and tags). She solves this polynomial for its roots over $\mathbb{F}_p$ (a handful of candidates), recovering r. With r known she can compute s for that

nonce from either equation, and now she can produce a *valid tag for any ciphertext she likes*. The tunnel's integrity is gone: she forges packets at will. Figure 30.3 traces this two-stage collapse from a repeated (K, N) pair to full packet forgery.

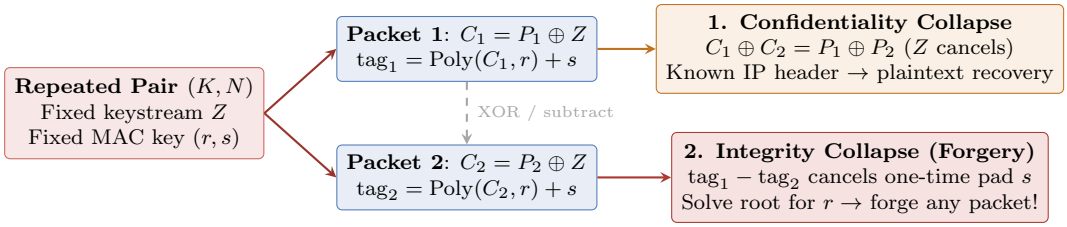


Figure 30.3: The two-stage catastrophe of reusing a nonce N under the same AEAD key K (ChaCha20-Poly1305 or AES-GCM). First, XORing the two ciphertexts cancels the keystream Z, exposing $P_1 \oplus P_2$. Second, subtracting the two authentication tags cancels the one-time mask s, leaving a polynomial equation in the secret evaluation key r whose roots allow universal packet forgery.

Intuition

A miniature of the same algebra over $\mathbb{F}_{17}$. Imagine a one-block "tag" $\text{tag} = c \cdot r + s \pmod{17}$. One observation is useless: one equation, two unknowns (r, s). But two messages reusing the nonce share r, s:

$$\text{tag}_1 = c_1 r + s, \qquad \text{tag}_2 = c_2 r + s \pmod{17}.$$

Subtract: $\text{tag}_1 - \text{tag}_2 = (c_1 - c_2)\, r$, so $r = (\text{tag}_1 - \text{tag}_2)(c_1 - c_2)^{-1}$. Concretely with $c_1 = 5, c_2 = 2, \text{tag}_1 = 14, \text{tag}_2 = 8$: $r = (14 - 8) \cdot (5 - 2)^{-1} = 6 \cdot 3^{-1}$. Since $3 \cdot 6 = 18 \equiv 1$, $3^{-1} = 6$, so $r = 6 \cdot 6 = 36 \equiv 2 \pmod{17}$. Then $s = \text{tag}_1 - c_1 r = 14 - 10 = 4$. Eve now has $(r, s) = (2, 4)$ and forges the tag for *any* c: for $c = 9$, $\text{tag} = 9 \cdot 2 + 4 = 22 \equiv 5$. Two messages under one reused nonce were enough to recover the MAC key. This is why every tunnel in this chapter enforces nonce uniqueness strictly.

Attack

This is not hypothetical. The 2016 "Nonce-Disrespecting Adversaries" survey scanned the Internet for IPsec and TLS endpoints that repeated AES-GCM nonces (usually from a broken random nonce generator or a counter that reset on reboot) and found real, exploitable hosts whose GHASH authentication key could be recovered exactly as above. The mitigations are structural: derive the nonce from a counter that physically cannot repeat (WireGuard, modern ESP), and rekey before it wraps. "Use a random nonce" is the dangerous option—a 96-bit random nonce collides by the birthday bound after about 2^{48} packets, which a busy gateway reaches in hours.

Connection

That 2^{48}-packet figure is the birthday bound from the probability companion: drawing 96-bit nonces uniformly, the chance of a collision crosses $1/2$ near $\sqrt{2^{96}} = 2^{48}$ draws. It is the same calculation that sets safe data limits for a single key and that underlies hash-collision attacks. A tunnel's rekey interval is, in effect, a budget set by the birthday bound.

30.6 TLS-based VPNs

The third family wraps the tunnel inside TLS instead of inventing its own record layer. OpenVPN is the canonical example; "SSL VPN" web portals and protocols like AnyConnect are cousins.

The idea: open a TLS connection from client to gateway (Chapter 25 gives you the handshake and record layer for free), then run the encapsulated IP packets *through* that connection. TLS already provides authenticated encryption, forward secrecy (with ephemeral DH), certificate-based authentication, and a battle-tested handshake; the VPN just has to move packets across it.

OpenVPN actually uses *two* channels: a TLS *control* channel that does the handshake and key negotiation, and a *data* channel that carries the tunneled packets under keys derived from the TLS session. Historically the data channel could run over UDP (lower latency, no head-of-line blocking) with its own packet format, or over TCP.

Common trap

TCP-in-TCP meltdown. Running OpenVPN over TCP, and then carrying a TCP connection inside the tunnel, stacks one reliable-retransmit loop inside another. When the outer TCP slows down and retransmits, the inner TCP *also* thinks packets were lost and retransmits, and the two timers fight each other—throughput collapses under loss. The standard advice is to tunnel over UDP and let the inner TCP do its own congestion control. This is a networking pitfall, not a crypto one, but it is the first thing that bites people who reach for "TLS VPN = TCP" by reflex.

Intuition

Why would anyone wrap a VPN in TLS when WireGuard exists? **Censorship resistance and firewall traversal.** TLS over port 443 looks exactly like ordinary HTTPS web traffic. A restrictive network that blocks WireGuard's recognizable handshake or IKE's UDP ports often cannot block a connection that is indistinguishable from someone reading the news over HTTPS—blocking it would block the whole web. This is why "stealth" VPN modes almost always mean "make it look like TLS to a web server." The price is the overhead and complexity of TLS, and the TCP-in-TCP trap above when UDP is also blocked.

In the real world

The split in practice: **WireGuard** when you control both ends and want speed and simplicity (site-to-site, Tailscale-style mesh, a personal VPN to your home). **IPsec/IKEv2** where you need interoperability with enterprise gear, OS-native clients (it is built into Windows, macOS, iOS), and certificate-based auth at scale; IKEv2's quick reconnection (MOBIKE) makes it the default "always-on" phone VPN. **TLS-based (OpenVPN)** where you must punch through hostile firewalls by masquerading as HTTPS, or need its mature ecosystem of clients. None is strictly best; the threat model and the network you must cross decide.

30.7 What a VPN leaks even when the crypto is perfect

Suppose every primitive is flawless: AEAD with fresh nonces, authenticated forward-secret key exchange, no implementation bugs. Eve *still* sees the outer packets—their sizes, counts, and timing—and that side channel is rich.

Attack

Website fingerprinting. Loading a web page generates a characteristic burst pattern: a small request, then a sequence of responses whose sizes reflect the page's resources. Even fully encrypted and padded into the tunnel, the *sequence of packet sizes and inter-arrival times* forms a fingerprint. A classifier trained on a closed set of sites can identify which of them you visited through the VPN with accuracy that has been reported above 90% in lab conditions. The tunnel hid the bytes; it did not hide the shape of the traffic. Defenses (constant-rate padding, traffic shaping) cost bandwidth and latency, which is why most VPNs do not deploy them—and why VPN $\neq$ anonymity.

Attack

The DNS leak and the kill-switch gap. Two classic operational failures. (1) If the OS resolves hostnames using a configured DNS server *outside* the tunnel, every site name you visit travels in cleartext to that resolver even though the page traffic is encrypted—Eve reads your browsing history from the DNS alone. Fix: route DNS through the tunnel and use the gateway's resolver. (2) If the VPN drops (handshake re-key fails, Wi-Fi flaps) and traffic silently falls back to the raw interface, you leak your real IP and plaintext until it reconnects. The fix is a *kill switch*: a firewall rule that forbids any traffic except through the tunnel, so a dropped VPN means no connectivity rather than a silent leak. These are configuration bugs, not crypto bugs, and they account for most real-world VPN privacy failures.

Walking question

You are on an airport network. You connect a correctly configured WireGuard tunnel to your home server and browse. Walk through what the airport's network operator can and cannot learn.
Cannot: the contents of any page, the hostnames you visit (DNS goes through the tunnel), which inner services you reach, or modify/inject anything (Poly1305 rejects tampering).
Can: that your device is running WireGuard (the packet format and port are recognizable), the IP of your home server, the total volume and timing of your traffic, when you are active, and—via website fingerprinting on packet sizes/timing—a guess at which sites you loaded from a known set. If you log into your bank inside the tunnel, the bank still knows it is you; the tunnel only hides you from the airport, not from the destination.

30.8 Putting the families side by side

	IPsec / IKEv2	**WireGuard**	**TLS VPN (OpenVPN)**
Key exchange	IKEv2, neg. DH groups	Noise_IK, Curve25519 only	TLS handshake (ECDHE)
Data AEAD	ESP, AES-GCM (neg.)	ChaCha20-Poly1305 only	TLS record / data channel
Auth model	PSK, X.509 certs, EAP	pinned public keys	X.509 certs (+ optional tls-auth)
Negotiation	extensive (attack surface)	none	cipher suites (TLS)
Forward secrecy	yes (ephemeral DH)	yes (ephemeral DH)	yes (ECDHE)
Nonce discipline	seq. number, then rekey	counter, rekey ∼2 min	TLS sequence numbers
Firewall stealth	poor (known ports)	poor (known format)	strong (looks like HTTPS)
Code size	very large	tiny (∼4k LoC)	large (OpenSSL + glue)

Connection

The store-now-decrypt-later clock. Every design here rests on Diffie–Hellman—X25519, P-256, or a finite-field group. A large quantum computer running Shor's algorithm breaks all of them, recovering g^{ab} from the public shares. The forward secrecy that protects against *stolen keys* does *not* protect against this: an adversary recording ciphertext today can decrypt it once the machine exists, because the DH transcript needed to recover the session key is right there in the recording. This is why both IPsec and WireGuard are gaining *hybrid* key exchange: run the classical DH and a post-quantum KEM (e.g. ML-KEM/Kyber) side by side and feed *both* shared secrets into the key-derivation PRF. The session stays secure as long as *either* primitive holds, so a quantum break of DH alone no longer exposes the traffic. WireGuard's fixed-suite philosophy makes this a version bump, not a negotiation; IKEv2 already has an RFC (8784/9370) for mixing in a post-quantum or pre-shared secret. The migration is driven entirely by the recording-today threat, not by any quantum computer existing yet.

Pause

Three ideas to carry out of this chapter. (1) A VPN = encapsulation (a routing trick) + authenticated encryption (the crypto), and it moves the trust boundary to the gateway rather than erasing it. (2) Every family solves the *same* cryptographic problem—authenticated, forward-secret key exchange feeding a nonce-respecting AEAD—and their differences are about negotiation, identity, and how they cross networks, not about secrecy guarantees. (3) The reliable failures are at the seams: weak or negotiated parameters (Logjam), nonce reuse (forgeable

AEAD), and leaks outside the tunnel (DNS, kill-switch, traffic analysis). The cipher is rarely the weak point; the plumbing around it is.

With VPNs, we conclude our tour of network-layer security: taking symmetric ciphers, key exchanges, and signatures to build encrypted tunnels across untrusted infrastructure. In Part V, we step into advanced cryptography, where the goal shifts from transporting private bytes across a wire to proving statements and computing over hidden data. In the next chapter, we explore zero-knowledge proofs: convincing a verifier that a statement is true without revealing anything beyond the fact that it is true.

30.9 Exercises

Exercise 30.1 (Encapsulation by hand)**.** Alice's laptop (`10.0.0.7`) on a coffee-shop network (her public IP `198.51.100.4`) tunnels to a gateway at `203.0.113.9` to reach internal server `10.0.0.50`. Write out the source and destination addresses of (a) the inner packet and (b) the outer packet, for a request *and* for the reply. Which addresses can the coffee-shop router see, and which are hidden inside the ciphertext?

Exercise 30.2 (Threat-model true/false)**.** For a correctly configured remote-access VPN, mark each claim true or false and justify in one sentence: (a) "My ISP cannot see which websites I visit." (b) "The websites I visit cannot tell who I am." (c) "My VPN provider cannot see my traffic." (d) "An attacker on the same Wi-Fi cannot modify my packets." (e) "Nobody can tell I am using a VPN."

Exercise 30.3 (Recover the Poly1305-style key)**.** A toy authenticator computes $\text{tag} = c_1 r + s \pmod{23}$ for a single ciphertext block c_1, with secret (r, s) fixed by the nonce. Two messages reuse the nonce: $(c_1, \text{tag}) = (4, 19)$ and $(c_1, \text{tag}) = (9, 7)$. Recover r and s, then forge a valid tag for $c_1 = 15$. (Hint: subtract the equations; you will need an inverse mod 23.)

Exercise 30.4 (Why the counter, not randomness)**.** WireGuard uses a strictly increasing 64-bit counter as its ChaCha20-Poly1305 nonce, while a naive design might draw a random 96-bit nonce per packet. (a) Using the birthday bound, estimate after how many packets a uniform 96-bit nonce has a noticeable chance of repeating. (b) Explain why a 64-bit counter that never resets and is rekeyed every two minutes is strictly safer than the random scheme. (c) What event, in the random scheme, makes nonce reuse near-certain after a reboot?

Exercise 30.5 (Authenticated DH defeats Mallory)**.** In IKEv2, Mallory sits in the middle and substitutes her own DH share toward each side. Explain precisely which message lets the legitimate peers detect her, and what secret Mallory would need to forge it. Then explain why an *unauthenticated* Diffie–Hellman tunnel (no AUTH payload, no signature) gives Mallory a working man-in-the-middle.

Exercise 30.6 (Noise IK without signatures)**.** WireGuard's handshake mixes four Diffie–Hellman outputs into the session key: $\text{DH}(e_i, S_r)$, $\text{DH}(s_i, S_r)$, $\text{DH}(e_r, E_i)$, $\text{DH}(e_r, S_i)$. For each, state whether it primarily contributes *forward secrecy*, *authentication of the initiator*, *authentication of the responder*, or *identity hiding*, and explain why. Then argue why no digital signature is needed.

Exercise 30.7 (Logjam arithmetic)**.** A weak IPsec deployment uses a fixed 1024-bit Diffie–Hellman prime shared across thousands of endpoints. Explain why the number-field-sieve cost structure (an expensive per-*prime* precomputation, then a cheap per-*connection* solve) makes a shared prime far more dangerous than a unique prime per endpoint of the same size. What two concrete fixes remove the problem?

Exercise 30.8 (Detect the nonce-reuse host)**.** You are auditing a gateway that encrypts ESP packets with AES-GCM using a 96-bit nonce drawn from a pseudorandom generator that is *not* reseeded after reboot. (a) Describe an observation you could make purely from captured ciphertext that reveals two packets reused a nonce. (b) Once you have such a pair, sketch the steps to recover the GHASH authentication key and forge a packet. (c) Name the structural fix.

Exercise 30.9 (TCP-in-TCP)**.** Explain, in terms of retransmission timers and congestion control, why tunneling a TCP connection over an OpenVPN-on-TCP link degrades badly under packet loss, while OpenVPN-on-UDP does not. Is this a cryptographic weakness? Why or why not?

Exercise 30.10 (Design a kill switch)**.** Write (in prose or pseudo-firewall rules) a kill-switch policy that guarantees no application traffic ever leaves the machine except through the VPN tunnel interface, while still permitting the handshake packets needed to (re)establish the tunnel to the gateway's public IP. Explain how your rules behave at three moments: before the tunnel is up, while it is up, and at the instant it drops.

Exercise 30.11 (Traffic-analysis budget)**.** A privacy tool wants to defeat website fingerprinting by padding all traffic to a constant rate of R bytes per second regardless of activity. (a) Why does this defeat size/timing fingerprinting? (b) Express the bandwidth overhead for a user whose genuine average rate is $r < R$. (c) Explain why most commercial VPNs do not do this, and what that implies about equating "VPN" with "anonymity."

Exercise 30.12 (The IKEv2 key tree)**.** IKEv2 derives all keys from $\mathrm{SKEYSEED} = \mathrm{PRF}(n_i \,\|\, n_r,\ g^{ab})$ and then expands with PRF+, slicing the stream in the fixed order $\mathrm{SK}_d, \mathrm{SK}_{ai}, \mathrm{SK}_{ar}, \mathrm{SK}_{ei}, \mathrm{SK}_{er}, \ldots$. (a) Why does feeding the nonces into the PRF (rather than only g^{ab}) matter, given that a repeated DH value is at least conceivable? (b) Why are the integrity keys for the two directions, SK_{ai} and SK_{ar}, deliberately different? (c) The expansion is a *chain*: T_{j+1} depends on T_j. What property would be lost if each T_j were instead computed independently as $\mathrm{PRF}(K, S \,\|\, j)$?

Exercise 30.13 (Rekeying and forward secrecy)**.** A site-to-site IPsec tunnel rekeys its child SA every hour using CREATE_CHILD_SA with a fresh ephemeral Diffie–Hellman exchange. An attacker records all ciphertext and, at 14:30, fully compromises the gateway and extracts its current session keys. (a) Which recorded traffic can the attacker now decrypt, and which stays protected? (b) How would the answer change if the rekey carried *no* new DH and instead derived the next keys from SK_d of the current SA? (c) What does this tell you about the difference between "rekeying" and "forward secrecy"?

Exercise 30.14 (Cookie-reply DoS defense)**.** WireGuard's responder will not perform the expensive Curve25519 operation for an incoming handshake until the sender has echoed a valid mac2 cookie tied to its source IP. (a) Explain how this stops an attacker who floods msg1 packets with *spoofed* source addresses. (b) Why does the cheaper mac1 (keyed by the responder's public key) not by itself stop a flood from an attacker who *does* know the responder's public key? (c) Compare this to a TCP SYN cookie: what is the analogous "proof of return reachability"?

Exercise 30.15 (Store now, decrypt later)**.** A surveillance agency records, but cannot yet break, the Diffie–Hellman handshakes of a heavily used WireGuard deployment. (a) Explain why classical forward secrecy does *not* protect this traffic against a future quantum computer, even though the ephemeral keys were discarded. (b) A hybrid handshake mixes both an X25519 secret and an ML-KEM secret into the key-derivation PRF. Why does the session stay secure if *only one* of the two is later broken? (c) Why is WireGuard's no-negotiation design an advantage rather than an obstacle when rolling out such a change?

Part V

Advanced Topics

Chapter 31

Zero-Knowledge Proofs

In the previous chapter, we used encryption and signatures to protect data in transit through network tunnels. Here we take on a very different cryptographic task: proving that a statement is true without revealing a single scrap of information about why.

The central guarantee of a zero-knowledge proof sounds self-contradictory the first time you hear it stated plainly: *I can convince you that I know a secret, and at the end of the conversation you will be certain that I know it, yet you will have learned nothing whatsoever about the secret itself.* Not a hint. Not a single bit. If you tried afterwards to convince a third party of what you saw, you would fail, because the transcript of our conversation could have been manufactured by anyone, knowledge or no knowledge.

That sentence packs three separate promises, and the whole chapter is really an unpacking of them. The proof must *work* when the prover is telling the truth (completeness). It must *fail* when the prover is lying (soundness). And it must *leak nothing* beyond the bare fact that the statement is true (zero-knowledge). Each of these is a precise mathematical condition, and the genius of the field, born in a 1985 paper by Goldwasser, Micali, and Rackoff, was figuring out how to state the third one rigorously at all.

We will build the idea from a classic thought experiment (a ring-shaped cave with a locked door), sharpen it into definitions, then construct a real protocol you could ship: Schnorr's identification scheme, the foundation behind a great deal of modern authentication. We turn it non-interactive with the Fiat-Shamir transform, which is how zero-knowledge becomes a *signature*. Commitments show up as the universal glue. And we close with a glimpse of zk-SNARKs, the technology that lets a blockchain verify a computation it never witnessed.

31.1 The cave: intuition before formalism

The foundational thought experiment is Jean-Jacques Quisquater's "Ali Baba cave." It maps directly to every formal definition in the modern theory.

Picture a circular cave with a single entrance corridor. The path splits into two branches: a left passage (Path A) and a right passage (Path B). Deep inside, where the two passages meet, a locked door blocks the way. The door opens only when someone enters a secret passphrase. Peggy (the

prover) claims she knows the secret passphrase. Victor (the verifier) wants proof, but Peggy refuses to reveal the phrase itself, as speaking it aloud would surrender the secret permanently.

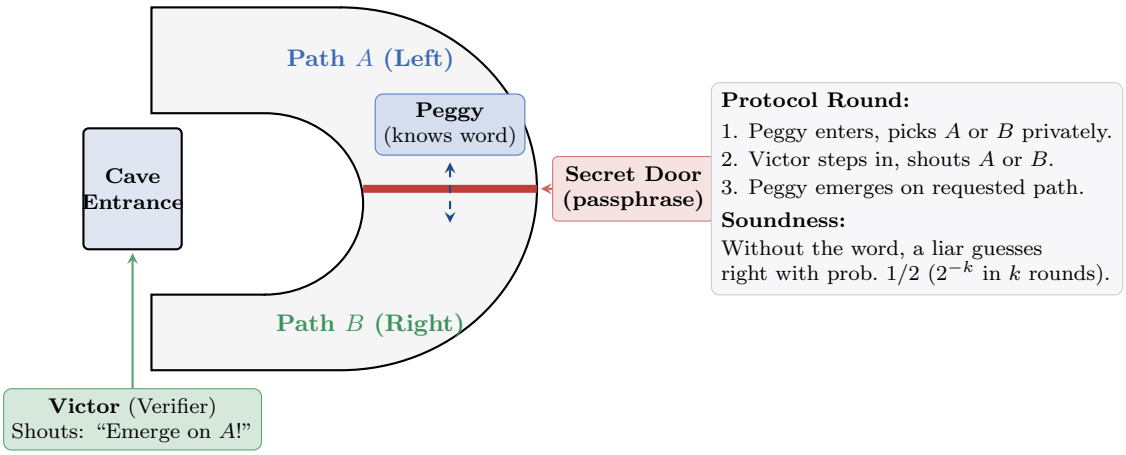


Figure 31.1: The Ali Baba cave thought experiment. Peggy proves she knows the secret passphrase to the locked door by emerging from whichever passage Victor calls out, without ever revealing the passphrase.

They play the game in rounds. Victor stands outside the entrance so he cannot see which path Peggy takes. Peggy walks in and privately chooses Path A or Path B. Once Peggy is inside, Victor steps to the cave entrance and shouts his challenge: he requests that she emerge from a specific branch — say, "Path A!" If Peggy initially chose Path A, she simply turns around and walks out. But if she entered Path B, her only way to emerge from Path A is to open the locked door with the passphrase and walk through. A prover who knows the passphrase can always open the door and satisfy the request, regardless of which path Victor calls.

Intuition

Suppose Peggy does not know the secret word. She can still attempt to deceive Victor by guessing which path he will shout and waiting on that side. If she guesses correctly, she walks out and appears truthful. However, because Victor chooses his challenge uniformly at random and cannot see her initial choice, her probability of guessing right is exactly $1/2$ per round. Running the game for 40 independent rounds reduces a dishonest prover's survival probability to $2^{-40} \approx 9 \times 10^{-13}$. Repeated challenges transform an ambiguous single round into near-certain verification.

Consider the information Victor observes across the conversation. He sees a sequence of challenges and Peggy emerging from the requested passages. He never hears the passphrase, and he never sees the door open.

In fact, Victor could fabricate an identical-looking video recording on his own without Peggy's help: film an actor walking into a random side, shout a challenge, and keep only the takes where the random choice matched the challenge. Discard the failed takes, splice the successful ones together, and the resulting film is indistinguishable from a genuine demonstration. Because an identical transcript can be generated without any knowledge of the secret word, the transcript itself conveys zero information. This thought experiment is formalized as the *simulator*.

Focus

Three core requirements define any zero-knowledge proof:

- **Completeness**: an honest prover holding the secret witness always convinces the verifier.
- **Soundness**: no cheating prover can convince the verifier, except with negligible probability.
- **Zero-knowledge**: the interaction reveals nothing beyond the validity of the statement; any verifier could simulate an indistinguishable transcript on their own.

History

Shafi Goldwasser, Silvio Micali, and Charles Rackoff introduced interactive proofs and zero-knowledge in "The Knowledge Complexity of Interactive Proof-Systems" (STOC 1985), a paper rejected three times before publication — reviewers struggled with the idea that a proof could convey *less* than the fact it established. The trio later shared the 2012 ACM Turing Award, in part for this work. In 1989, Jean-Jacques Quisquater and a long list of co-authors wrote the Ali Baba cave story in "How to Explain Zero-Knowledge Protocols to Your Children" (CRYPTO 1989), which remains the gentlest on-ramp ever written for a cryptographic primitive. Today, zero-knowledge powers privacy-preserving cryptocurrencies and succinct blockchain rollups, with active research focused on transparent, post-quantum proof systems that need no trusted setup.

31.2 Interactive proofs

Let us replace the cave with mathematics. We fix a language L: a set of true statements. For example, L could be "all graphs that have a Hamiltonian cycle," or "all numbers y that are squares modulo n," or in the cases we care about most, "all public keys for which the prover knows the matching private key." A statement x is true precisely when $x \in L$.

An *interactive proof system* for L is a pair of algorithms, the prover P and the verifier V, who exchange messages. V runs in probabilistic polynomial time — it is meant to be cheap and realistic. P may be computationally unbounded in the classical theory, though in the protocols we build P is also efficient *provided it holds a secret witness* w. At the end V outputs accept or reject.

Definition 31.1 (Interactive proof, informal)**.** (P, V) is an interactive proof for L if:

- **Completeness:** for every $x \in L$, the prover makes V accept with probability at least $1 - \mathsf{negl}$ (often exactly 1).
- **Soundness:** for every $x \notin L$ and *every* cheating prover strategy P^* (however clever), V accepts with probability at most some small s, the *soundness error*. Repetition drives s toward 0.

The asymmetry is deliberate. Completeness protects the truthful prover from a false rejection; soundness protects the verifier from *every conceivable* lying prover, including ones we have not imagined. A protocol that only stops the liars we thought of is worthless. This is the same "quantify over all adversaries" stance you have seen throughout the book.

Walking question

Why allow V to be *probabilistic*? Because a deterministic verifier in a single-prover setting buys you nothing beyond an ordinary written proof (the class NP). Randomness — the unpredictable challenge "Left or right?" — is exactly what stops the prover from preparing one canned answer in advance. The interaction plus the coins is where the extra power lives; the complexity class of all such interactive proofs, IP, turns out to equal PSPACE, a celebrated 1990 result of Shamir. For cryptography we never need that much muscle, but it tells you the framework is genuinely more expressive than static proofs.

Proof of knowledge: a stronger flavor

For authentication we usually want something sharper than "the statement is true." We want "the prover actually *knows* a witness." Authentication requires that the prover actually holds the private key corresponding to the claimed identity. This is captured by a *proof of knowledge*, and the tool that defines it is the *knowledge extractor*.

Definition 31.2 (Proof of knowledge, informal)**.** A protocol is a proof of knowledge for a relation $R = \{(x, w)\}$ if there is an efficient *extractor* E with the following power: given the ability to run any successful prover P^* and to *rewind* it — replay it from a chosen point with a different challenge — E outputs a valid witness w with $(x, w) \in R$. Informally: anything that can convince V must possess w, because we could squeeze w out of it by replaying.

Rewinding is a thought experiment, not something a real verifier does in a live session. Its purpose is to argue: *the only way to answer two different challenges about the same first message is to know the secret.* We will see this concretely in Schnorr's protocol, where two accepting transcripts that share a first message let you solve for the private key with a single subtraction.

31.3 Commitments: the cryptographic envelope

Before the headline protocol we need a building block that appears in almost every zero-knowledge construction: the *commitment scheme*, which functions as a sealed, tamper-evident envelope. (We introduce the core mechanics here to power our proofs; Chapter 32 explores the full algebraic theory of commitments and secret sharing in depth.)

You **commit** to a value m by computing $c = \mathsf{Commit}(m, r)$ with fresh randomness r, and you hand c to the other party. Later you **open** it by revealing (m, r), and they check $c = \mathsf{Commit}(m, r)$. Two properties make the envelope useful:

- **Hiding:** c reveals nothing about m. The sealed envelope is opaque.
- **Binding:** you cannot find a second $(m', r') \neq (m, r)$ with the same c. Once sealed, the contents are fixed; you cannot swap the letter.

Intuition

Commitments let you say "I have decided on my answer" *before* you learn the other party's challenge, without revealing the answer yet. That ordering — lock in your move, then see the challenge, then reveal — is what stops a cheater from adapting their answer to the challenge after the fact. The cave achieves the same ordering physically: Peggy commits to a passage by walking down it before Victor shouts.

A clean number-theoretic commitment is *Pedersen's*. Fix a cyclic group $\mathbb{G}$ of prime order q with two generators g and h whose discrete-log relation nobody knows. To commit to $m \in \mathbb{Z}_q$, pick $r \xleftarrow{\$} \mathbb{Z}_q$ and set

$$c \;=\; g^m\, h^r.$$

Hiding is *perfect*: because h^r is uniform over the group, $g^m h^r$ is uniform too, independent of m. Binding rests on the discrete-log assumption: if you could open c as both m and m', you would have $g^m h^r = g^{m'} h^{r'}$, hence $h^{r-r'} = g^{m'-m}$, which hands you $\log_g h = (m' - m)(r - r')^{-1} \bmod q$ — the very discrete log we assumed is hard. A simpler, hash-based commitment is $c = \mathrm{H}(m \,\|\, r)$, binding because H is collision-resistant and hiding because H behaves like a random function for unknown r.

Common trap

You cannot have *both* perfect hiding and perfect binding at once. If c truly carries zero information about m (perfect hiding), then in principle *some* (m', r') also maps to c for every m', so binding can only be computational. Pedersen is perfectly hiding and computationally binding; hash-based commitments are typically computationally hiding and (heuristically) binding. Pick the side you cannot afford to lose. For a coin flip you want binding; for a sealed-bid auction you may prize hiding.

Connection

Commitments are close cousins of hash functions (Chapter 14) and of the discrete-log assumption (Chapters 18 and 20). The companion algorithms book covers the cost of modular exponentiation by square-and-multiply, which is what makes $g^m h^r$ cheap to compute; the probability book's birthday bound is what governs how collision-resistance — hence binding — degrades as the output shrinks.

31.4 Schnorr's identification protocol

Now the centerpiece. Schnorr's scheme lets Peggy prove she knows the discrete logarithm of her public key without revealing it. It is short, elegant, and underlies real signature standards (EdDSA among them). We work in a cyclic group $\mathbb{G} = \langle g \rangle$ of prime order q. Peggy's secret is $x \xleftarrow{\$} \mathbb{Z}_q$; her public key is $y = g^x$. The statement she proves: "I know x such that $g^x = y$."

The protocol follows a three-move exchange known as a Σ-*protocol*, named after the geometric shape of its message flow: commitment, challenge, response.

1. **Commitment.** Peggy draws a fresh random nonce $k \xleftarrow{\$} \mathbb{Z}_q$ and sends $a = g^k$. She commits to an ephemeral exponent before seeing any challenge.

2. **Challenge.** Victor draws a challenge $e \xleftarrow{\$} \mathbb{Z}_q$ at random and sends e.
3. **Response.** Peggy computes $s = k + e\,x \bmod q$ and sends s.
4. **Verify.** Victor accepts if and only if $g^s = a \cdot y^e$.

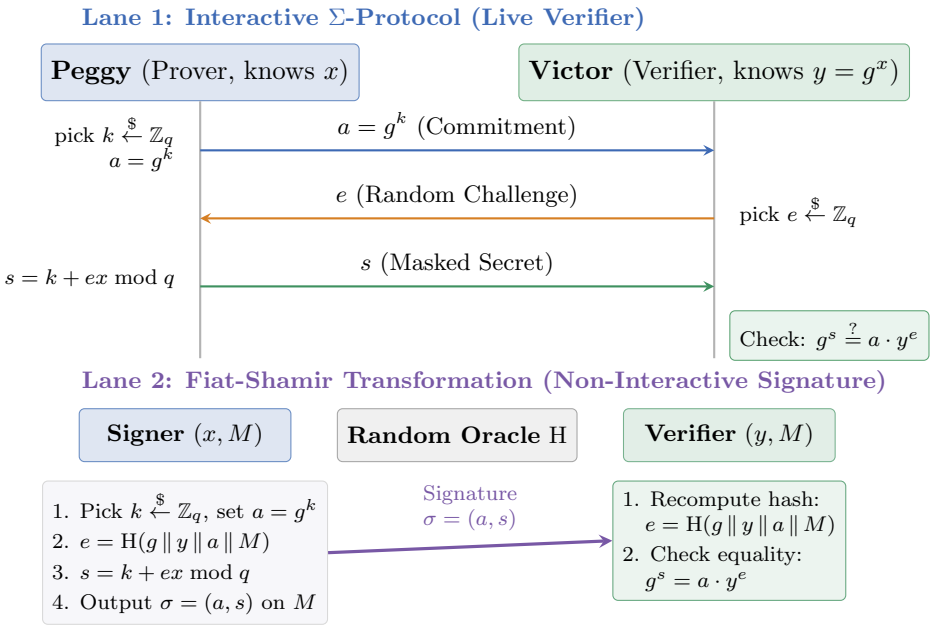


Figure 31.2: Schnorr identification protocol and its Fiat-Shamir transformation. In the interactive protocol (Lane 1), Victor's random challenge stops Peggy from preparing a canned response in advance. In the Fiat-Shamir transform (Lane 2), a cryptographic hash function replaces Victor, binding the challenge e to the commitment a and message M to produce a standalone signature.

Why verification works (completeness)

If Peggy follows the protocol, the check is just algebra:

$$g^s \;=\; g^{\,k+ex} \;=\; g^k\,(g^x)^e \;=\; a \cdot y^e.$$

So a rule-following Peggy always passes. Completeness holds with probability 1.

A worked example with tiny numbers

Let us trace it concretely in a group small enough to do by hand. Take the multiplicative group modulo the prime $p = 23$. Its order is $22 = 2 \cdot 11$. The element $g = 2$ has order 11 (you can check $2^{11} \equiv 1 \pmod{23}$ and no smaller power is 1), so g generates a subgroup $\mathbb{G}$ of prime order $q = 11$. We do all exponent arithmetic mod $q = 11$ and all group arithmetic mod $p = 23$.

Peggy's secret: $x = 7$. Her public key: $y = g^x = 2^7 = 128 \equiv 13 \pmod{23}$. So $y = 13$.

1. Peggy draws nonce $k = 4$. She sends $a = 2^4 = 16 \pmod{23}$.
2. Victor draws challenge $e = 6$.

3. Peggy responds $s = k + ex = 4 + 6 \cdot 7 = 46 \equiv 2 \pmod{11}$. She sends $s = 2$.
4. Victor checks. Left side: $g^s = 2^2 = 4$. Right side: $a \cdot y^e = 16 \cdot 13^6 \pmod{23}$. Compute $13^2 = 169 \equiv 8$, $13^3 = 13 \cdot 8 = 104 \equiv 12$, $13^6 = (13^3)^2 = 12^2 = 144 \equiv 6 \pmod{23}$. So the right side is $16 \cdot 6 = 96 \equiv 4 \pmod{23}$.

Both sides equal 4. Victor accepts, and at no point did $x = 7$ cross the wire; he only saw $(a, e, s) = (16, 6, 2)$.

Work it yourself

Re-run the example yourself with $k = 9$ and $e = 3$. You should get $s = 9 + 21 = 30 \equiv 8 \pmod{11}$, $a = 2^9 = 512 \equiv 6 \pmod{23}$, and both sides of the check equal to $2^8 = 256 \equiv 3 \pmod{23}$. Confirm the right side: $y^3 = 13^3 \equiv 12$, times $a = 6$ gives $72 \equiv 3 \pmod{23}$. It matches — a different transcript, same accept.

Why a cheat fails (soundness via extraction)

Here is the elegant argument. Suppose a prover, knowing only y but not x, can answer *two different challenges* $e \neq e'$ for the *same* first message a, producing valid responses s and s'. Both pass verification:

$$g^s = a\, y^e, \qquad g^{s'} = a\, y^{e'}.$$

Divide one by the other: $g^{\,s-s'} = y^{\,e-e'}$. Since $y = g^x$, this says $g^{\,s-s'} = g^{\,x(e-e')}$, so in the exponent (mod q):

$$s - s' \equiv x\,(e - e') \pmod{q} \quad \Longrightarrow \quad x \;=\; (s - s')\,(e - e')^{-1} \bmod q.$$

The inverse exists because q is prime and $e \neq e'$. **Anyone who can answer two challenges on one commitment can compute x outright.** That is the knowledge extractor of Definition 31.2 made fully concrete: rewind the prover to just after it sent a, feed a fresh challenge, and subtract (Figure 31.3, left panel).

Why it works

This is why Schnorr is a proof of knowledge. A prover who does not know x cannot consistently answer two challenges on the same a — if it could, the extraction above would compute x from the prover's own outputs, contradicting the hardness of discrete log. So with challenge space of size q, the best a cheater can do is guess the single challenge it can answer and pray Victor sends exactly that one: soundness error $1/q$. With a 128-bit prime q, that is 2^{-128} in one shot. (With a tiny challenge set, say a single bit $e \in \{0, 1\}$, the error is $1/2$ per round, and you repeat — exactly the cave.)

Why it leaks nothing (zero-knowledge via simulation)

Completeness and soundness are about correctness. Zero-knowledge is the delicate one, and we prove it by building the simulator from the cave: a machine that produces transcripts *indistinguishable* from real ones *without knowing* x.

For *semi-trusted-verifier* zero-knowledge (the verifier picks e at random, exactly as the protocol specifies, rather than adversarially) the simulator is startlingly simple. To fake a transcript (a, e, s):

1. Pick the response $s \xleftarrow{\$} \mathbb{Z}_q$ first.
2. Pick the challenge $e \xleftarrow{\$} \mathbb{Z}_q$.
3. *Solve for* the commitment that makes the check pass: $a = g^s\, y^{-e}$.

By construction $g^s = a\, y^e$, so the fake transcript verifies. And its distribution is identical to a real run: in a real run k is uniform so a is uniform and $s = k + ex$ is uniform; in the simulation s is uniform and e is uniform, which forces a to be uniform too. Same distribution, no secret used. Figure 31.3 contrasts the two thought-experiment machines side by side: the knowledge extractor that forks a prover to recover x, and the simulator that runs the steps backward to forge a transcript without x.

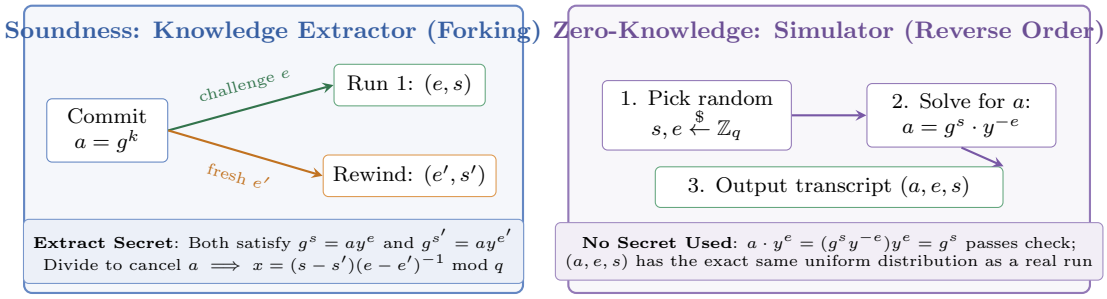


Figure 31.3: The two core proof techniques of zero-knowledge protocols. Left: the *knowledge extractor* rewinds a prover to the same commitment a with two distinct challenges $e \neq e'$, canceling the nonce k to solve for the witness x. Right: the *simulator* chooses the response s and challenge e first and solves backwards for $a = g^s y^{-e}$, producing an indistinguishable valid transcript without knowing x.

Intuition

The simulator cheats by working the protocol *out of order*. The real prover commits, then is challenged, then must respond — and to respond it needs x. The simulator, free to choose e and s itself, works in reverse order: it chooses the response and the challenge first, then computes the unique commitment a that makes the verification equation hold. A real verifier cannot do this because it does not get to choose a after seeing e and s. But the resulting *transcript* looks identical: because this simulated transcript is indistinguishable from a real one, the transcript conveys no information about x.

Formal

Zero-knowledge against a rule-following verifier (the standard literature name is HVZK). A Sigma protocol has this property if there is a PPT simulator S that, on input the statement y (no witness), outputs transcripts (a, e, s) whose distribution equals (or is computationally indistinguishable from) that of real protocol runs in which the verifier draws e uniformly as specified. For Schnorr the two distributions are *identical*, so the simulation is *perfect*. Upgrading to security against a *malicious* verifier who picks e adversarially needs more care — one route is to have the verifier first commit to e before seeing a; another is the Fiat-Shamir transform of the next section, which removes the verifier's freedom entirely by deriving e from a hash.

Common trap

Never reuse the nonce k. If Peggy ever signs or proves twice with the same $a = g^k$ under two different challenges $e_1 \neq e_2$, an eavesdropper runs exactly the extractor above on the two public transcripts and recovers x: $x = (s_1 - s_2)(e_1 - e_2)^{-1}$. The same flaw leaked the Sony PlayStation 3 ECDSA signing key in 2010 (a fixed nonce) and has burned countless wallets. The nonce must be fresh, uniform, and secret every single time. Deterministic nonce generation (RFC 6979, or hashing the message with the key) is the modern defense.

31.5 Fiat-Shamir: from conversation to signature

Interaction is inconvenient. A signature, a login token, a blockchain proof — these should be one-shot artifacts you can post and walk away from, with no live verifier to volley challenges. The *Fiat-Shamir transform* (Amos Fiat and Adi Shamir, 1986) removes the verifier from the loop with a single brilliant substitution:

Replace the verifier's random challenge e with $e = \mathrm{H}$(everything so far).

The prover can compute the hash herself, so she no longer needs anyone to talk to. But because a good hash is unpredictable, she still cannot choose a *after* seeing e — the hash of a *is* e, so fixing a fixes e. The unpredictability that the live verifier used to supply is now supplied by the hash function. To prevent the soundness argument from collapsing, we model H as a random oracle.

For Schnorr, the transformed, non-interactive protocol on statement $y = g^x$ is:

1. Pick $k \xleftarrow{\$} \mathbb{Z}_q$, set $a = g^k$.
2. Compute $e = \mathrm{H}(g \,\|\, y \,\|\, a)$. *(For a signature on message M, hash M in too: $e = \mathrm{H}(g \,\|\, y \,\|\, a \,\|\, M)$.)*
3. Compute $s = k + ex \bmod q$.
4. Output the proof or signature $\sigma = (a, s)$ (equivalently (e, s)).

A verifier recomputes $e = \mathrm{H}(g \,\|\, y \,\|\, a \,\|\, M)$ and checks $g^s = a\, y^e$. That is the **Schnorr signature scheme** — and EdDSA (Ed25519), used in SSH, TLS 1.3, Signal, and many cryptocurrencies, is precisely this idea over a specific elliptic curve.

Why it works

Why hashing the message in gives a secure signature. Binding e to M means a forger who wants a signature on a chosen M is forced into the same trap as a cheating prover: to produce a valid (a, s) for that M they would need to answer the challenge $e = \mathrm{H}(\ldots \,\|\, M)$ on a commitment a, which by the soundness/extraction argument requires knowing x. In the random oracle model one proves this rigorously via a "forking lemma": run the forger, rewind it to the point it queried the oracle on a, answer with a fresh e', and obtain two transcripts (a, e, s), (a, e', s') — then extract x. A forger thus implies a discrete-log solver.

Common trap

Always hash the statement, not just the commitment. A notorious class of bugs ("weak Fiat-Shamir") comes from hashing too little — e.g. computing $e = \mathrm{H}(a)$ but forgetting to bind y (the public key or statement) and any public parameters. An attacker who can choose the statement *after* seeing the challenge can then forge. This exact omission produced real CVEs in production proof systems (the 2022 "Frozen Heart" vulnerabilities affecting several SNARK and Bulletproof libraries). Rule: the hash input must commit to *the full statement and all public values*, not just the prover's first message.

Implement it

A self-contained Schnorr identification plus Fiat-Shamir signature over a small prime-order subgroup. **Learning only — do not use for anything real.** The group is microscopic, the randomness is Python's non-cryptographic PRNG, and there is no constant-time arithmetic or input validation.

```
import hashlib, random

# Toy prime-order group: subgroup of Z_p^* of order q, generator g.
p, q, g = 2267, 103, 354      # 354 has order 103 modulo 2267 (toy params)

def H(*parts):                # Fiat-Shamir challenge in [0, q)
    data = "|".join(str(x) for x in parts).encode()
    return int.from_bytes(hashlib.sha256(data).digest(), "big") % q

# --- key generation ---
x = random.randrange(1, q)    # secret key
y = pow(g, x, p)              # public key  y = g^x mod p

# --- interactive Schnorr proof of knowledge of x ---
def prove_interactive(x):
    k = random.randrange(1, q)
    a = pow(g, k, p)                  # 1. commitment
    e = random.randrange(0, q)        # 2. verifier's challenge
    s = (k + e * x) % q               # 3. response
    return a, e, s

def verify_interactive(y, a, e, s):
    return pow(g, s, p) == (a * pow(y, e, p)) % p

a, e, s = prove_interactive(x)
assert verify_interactive(y, a, e, s)

# --- non-interactive signature on a message via Fiat-Shamir ---
def sign(x, msg):
    k = random.randrange(1, q)        # MUST be fresh & secret each time
    a = pow(g, k, p)
    e = H(g, y, a, msg)               # challenge from the hash, binds y and msg
    s = (k + e * x) % q
    return a, s

```

```python
def verify(y, msg, sig):
    a, s = sig
    e = H(g, y, a, msg)
    return pow(g, s, p) == (a * pow(y, e, p)) % p

sig = sign(x, "pay Bob 5 coins")
assert verify(y, "pay Bob 5 coins", sig)
assert not verify(y, "pay Bob 500 coins", sig)   # message is bound

# --- the nonce-reuse catastrophe, demonstrated ---
def sign_fixed_nonce(x, msg, k):       # DON'T: reuse k across messages
    a = pow(g, k, p)
    e = H(g, y, a, msg)
    return a, (k + e * x) % q, e
a1, s1, e1 = sign_fixed_nonce(x, "msg one", k=42)
a2, s2, e2 = sign_fixed_nonce(x, "msg two", k=42)   # same k -> same a
# Two transcripts, one commitment: recover the secret by subtraction.
x_recovered = ((s1 - s2) * pow(e1 - e2, -1, q)) % q
assert x_recovered == x                # secret key fully leaked
print("recovered secret:", x_recovered, "== real:", x)
```

The last block is the lesson that matters most: a single repeated nonce turns a secure scheme into a public broadcast of your private key.

31.6 A second example: graph 3-coloring

Schnorr proves an algebraic fact. But zero-knowledge is far more general — a classic theorem says *every* statement in NP has a zero-knowledge proof. The bridge is graph 3-coloring, an NP-complete problem, whose protocol illustrates how commitments enforce soundness and zero-knowledge.

A proper 3-coloring of a graph paints each vertex red, green, or blue so that no edge joins two same-colored vertices. Peggy knows such a coloring and wants to convince Victor the graph is 3-colorable without revealing which colors go where.

1. Peggy draws a uniform random permutation $\pi \in S_3$ of the color set $\{\text{red}, \text{green}, \text{blue}\}$, permutes the graph coloring, and generates a cryptographic commitment $c_v = \text{Commit}(\pi(\text{color}(v)), r_v)$ for every vertex v. She hands all sealed commitments to Victor.

2. Victor chooses an edge $e = (u, v) \in E$ uniformly at random and requests its openings.

3. Peggy opens only the two commitments c_u and c_v. Victor verifies that the opened colors are valid, that the commitments match, and that $\pi(\text{color}(u)) \neq \pi(\text{color}(v))$.

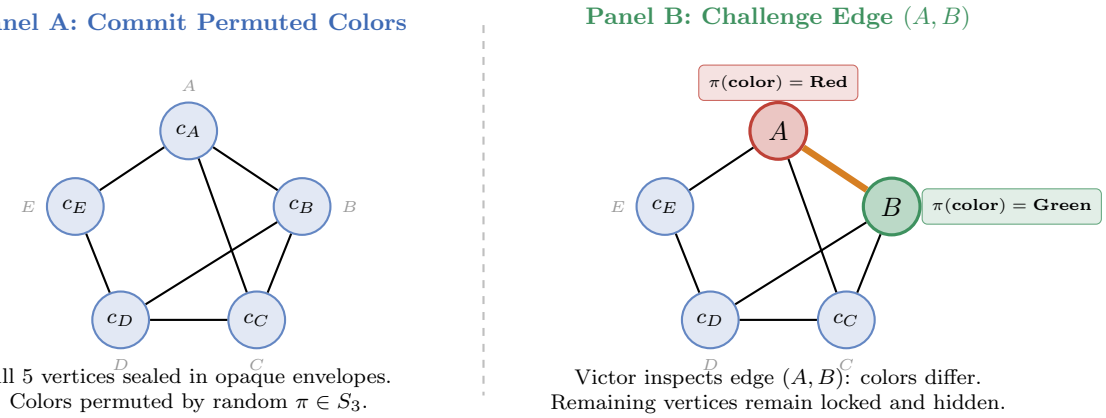


Figure 31.4: Zero-knowledge proof for Graph 3-Coloring. Peggy commits to permuted colors on all vertices (Panel A). Victor queries a single edge (A, B) (Panel B). Peggy opens only those two envelopes, revealing two distinct colors while all other vertices remain sealed.

Why it works

Completeness: a real coloring has differently-colored endpoints on every edge, so Peggy always passes. *Soundness:* if the graph is *not* 3-colorable, then under *any* commitment Peggy makes, at least one edge has matching endpoints. Victor hits that bad edge with probability at least $1/|E|$, so each round catches a fraud with probability $\geq 1/|E|$; repeating $|E| \cdot 40$ rounds drives the soundness error below 2^{-40}. *Zero-knowledge:* Victor learns only that *two specific vertices have different colors* — which, because Peggy re-permutes the colors every round, is just "two random distinct colors," carrying no information about the global coloring. The simulator fakes a round by committing to two random distinct colors on the queried edge and garbage elsewhere; since commitments are hiding, Victor cannot tell the difference.

The binding property of the commitments is what nails Peggy down: she must fix all colors *before* Victor names the edge, so she cannot recolor on the fly to dodge a bad edge. The hiding property is what protects her secret. These twin roles—binding to prevent cheating and hiding to protect knowledge—recur throughout zero-knowledge protocols.

31.7 A glimpse of zk-SNARKs

Schnorr proves knowledge of one discrete log. Real applications want to prove sprawling statements — "I ran this 10,000-step program correctly," "this transaction is valid under rules nobody should see my inputs to" — and they want the proof to be *tiny* and *instant to check*, even if the computation it attests to was enormous. That is the promise of a **zk-SNARK**: *Zero-Knowledge Succinct Non-interactive ARgument of Knowledge.* Unpack the acronym and you have the whole spec.

- **Succinct:** the proof is small (often a few hundred bytes) and verification is fast (milliseconds), *independent of how big the computation was.* This is the headline feature — you can verify in moments a computation that took an hour to run.
- **Non-interactive:** one message, no back-and-forth (Fiat-Shamir or a structured reference string makes this happen).

- **ARgument:** soundness holds against *computationally bounded* provers only — a small but real weakening from a "proof," the price of succinctness.
- **of Knowledge:** as with Schnorr, a valid proof implies the prover actually holds a witness.
- **Zero-Knowledge:** the proof leaks nothing about the witness.

The machinery underneath is heavier than anything else in this chapter, so we only sketch the pipeline. First the computation is written as an *arithmetic circuit* over a finite field, then flattened into a *rank-1 constraint system* (R1CS): a list of equations of the form $(\mathbf{a} \cdot \mathbf{z})(\mathbf{b} \cdot \mathbf{z}) = (\mathbf{c} \cdot \mathbf{z})$, where $\mathbf{z}$ holds all the wire values including the secret witness. Those constraints are then encoded as polynomials (a QAP, quadratic arithmetic program), and the prover's job becomes "demonstrate that a certain polynomial identity holds" — something that can be checked at a *single random point* thanks to the fact that a nonzero low-degree polynomial has few roots. *Polynomial commitments* (KZG commitments, built on elliptic-curve pairings, or hash-based ones in STARKs) let the prover commit to those polynomials and reveal just one evaluation, which is where "succinct" comes from (Figure 31.5).

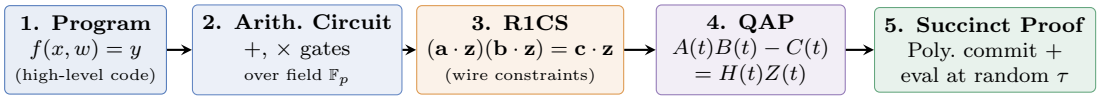


Figure 31.5: The zk-SNARK compilation and proving pipeline. An arbitrary bounded computation is flattened into an arithmetic circuit over $\mathbb{F}_p$, expressed as a Rank-1 Constraint System (R1CS), interpolated into a Quadratic Arithmetic Program (QAP) polynomial identity, and checked at a single random point via a polynomial commitment.

Intuition

The unifying trick is the same one that made Schnorr's soundness work, scaled up. To prove "these two giant polynomials are equal," you do *not* send the polynomials. You let the verifier pick a random point and you reveal both values there. If the polynomials differ, they agree at only a handful of points out of the whole field, so a random point catches the lie with overwhelming probability (Schwartz-Zippel). Commitments stop the prover from choosing the polynomial after seeing the point. Random challenge plus commitment plus "few roots" equals soundness — the exact recipe from the cave, just with polynomials instead of passages.

Common trap

The trusted-setup landmine. Many SNARKs (Groth16, the classic zk-SNARK) require a one-time *trusted setup* that produces public parameters from secret randomness — the "toxic waste." Whoever knows that randomness can forge proofs of false statements at will. Real deployments run elaborate multi-party "ceremonies" (Zcash's "Powers of Tau" had thousands of participants) so that the secret is safe as long as *one* participant deleted their share. STARKs and Bulletproofs avoid trusted setup entirely, at the cost of larger proofs. When you evaluate a SNARK system, ask first: who held the toxic waste, and do I trust that it was destroyed?

In the real world

Where zero-knowledge actually ships.

- **Authentication.** Schnorr-style proofs of knowledge let you log in by proving you hold a private key without sending it; this is the spirit of challenge-response in SSH public-key auth and of FIDO2/WebAuthn hardware keys.
- **Signatures.** EdDSA (Ed25519) is Fiat-Shamir-transformed Schnorr, used in TLS 1.3, OpenSSH, Signal, and the Bitcoin Taproot upgrade.
- **Private cryptocurrency.** Zcash uses zk-SNARKs so that a payment can be verified valid — inputs equal outputs, no double-spend — while the sender, receiver, and amount stay encrypted.
- **Blockchain scaling (zk-rollups).** Systems like zkSync, StarkNet, and Polygon zkEVM execute thousands of transactions off-chain and post a single succinct proof that all of them were valid, so the main chain verifies a batch in one cheap check instead of re-running every transaction.
- **Privacy-preserving identity.** "Prove you are over 18" or "prove your credential is in the valid set" without revealing your birthdate or identity — the core of anonymous-credential and selective-disclosure systems.

31.8 Putting the pieces together

A zero-knowledge proof is an interactive protocol governed by three core guarantees. *Completeness* is verified by algebra: the rule-following prover's response always satisfies the check. *Soundness* is verified by extraction: two answers to two challenges on one commitment yield the secret, so anyone who can pass must possess it. *Zero-knowledge* is verified by simulation: a machine with no secret produces identical-looking transcripts, so the transcripts cannot carry the secret. The Sigma-protocol triple of *commit, challenge, respond* is the skeleton; commitments supply the binding that stops cheating and the hiding that protects knowledge; Fiat-Shamir freezes the live conversation into a postable artifact by replacing the challenge with a hash; and zk-SNARKs scale the same logic — random challenge, commitment, few-roots soundness — up to arbitrary computations checked in milliseconds.

Pause

Before the exercises, make sure you can answer these in your own words: (1) In Schnorr, why does answering two challenges on the same a reveal x? (2) Why is reusing the nonce k catastrophic, and what is the one-line attack? (3) What does the simulator do, and why does its existence prove the protocol leaks nothing? (4) In Fiat-Shamir, what supplies the unpredictability that the live verifier used to provide, and why must the message be hashed in?

Throughout this chapter, commitments served as cryptographic envelopes that bind a prover's choices before a challenge is revealed. In the next chapter, we unpack commitments in their own right alongside secret sharing — examining information-theoretic hiding, Shamir's polynomial secret sharing, and how splitting secrets among distributed parties lays the foundation for multiparty computation.

Exercises

Exercise 31.1 (Trace Schnorr)**.** Work modulo $p = 23$ with $g = 2$ (order $q = 11$) and secret $x = 5$, so $y = 2^5 \equiv 9 \pmod{23}$. Peggy picks nonce $k = 8$ and Victor sends challenge $e = 4$. Compute a, the response s, and verify that $g^s \equiv a\, y^4 \pmod{23}$ on both sides. State the numeric value both sides share.

Exercise 31.2 (Break a reused nonce)**.** A careless prover (params $g = 2$, $q = 11$, $p = 23$) publishes two Schnorr signatures that reuse the same nonce, hence the same $a = 6$: first $(e_1, s_1) = (3, 2)$, second $(e_2, s_2) = (7, 0)$. Recover the secret x using $x = (s_1 - s_2)(e_1 - e_2)^{-1} \bmod 11$. Then verify your x by checking that the public key $y = g^x \bmod 23$ makes both transcripts pass $g^s \equiv a\, y^e \pmod{23}$.

Exercise 31.3 (Build the simulator)**.** For Schnorr with public key y, you are handed challenge e in advance. Describe, with formulas, how to produce an accepting transcript (a, e, s) without knowing x. Prove that your faked a is uniformly distributed over $\mathbb{G}$, and explain why this implies the protocol reveals nothing about x to a verifier who draws e as specified.

Exercise 31.4 (Soundness error and repetition)**.** A Sigma protocol uses a single-bit challenge $e \in \{0, 1\}$, giving soundness error $1/2$ per round. (a) How many independent rounds are needed to push the total soundness error below 2^{-80}? (b) If instead the challenge is drawn uniformly from $\mathbb{Z}_q$ with q a 256-bit prime, what is the soundness error of a *single* round, and why is repetition then unnecessary?

Exercise 31.5 (Pedersen binding implies discrete log)**.** Recall the Pedersen commitment $c = g^m h^r$ in a group of prime order q. Suppose an adversary outputs two distinct openings $(m, r) \neq (m', r')$ of the same c. Show step by step how to compute $\log_g h \bmod q$ from these openings. Why does this prove that breaking binding is at least as hard as discrete log?

Exercise 31.6 (Why permute the colors?)**.** In the graph 3-coloring protocol, suppose Peggy *forgets* to randomly permute the three colors before committing each round, instead committing to the same fixed coloring every time. Describe concretely what an adversarial Victor learns after $O(|E|)$ rounds, and argue that the protocol is no longer zero-knowledge.

Exercise 31.7 (Weak Fiat-Shamir)**.** A flawed implementation computes the challenge as $e = \mathrm{H}(a)$, omitting the statement y from the hash. Sketch how an attacker who is free to choose the public key y *after* computing a and e can produce an accepting transcript without knowing any discrete log. (Hint: pick s and e first, set $a = g^s y^{-e}$ — but now you also get to pick y.) Explain why including y in the hash input closes the hole.

Exercise 31.8 (Completeness proof)**.** Prove the completeness of Schnorr in full generality: for any secret x, nonce k, and challenge e in $\mathbb{Z}_q$, the prescribed response $s = k + ex \bmod q$ satisfies $g^s = a\, y^e$ where $a = g^k$ and $y = g^x$. State precisely which group law you use at each step.

Exercise 31.9 (Designing a challenge space)**.** You are deploying Fiat-Shamir Schnorr and must choose the output range of the hash that produces e. Argue for the minimum bit-length of e needed so that the soundness error $1/|\text{challenge space}|$ is below 2^{-128}. Then explain why making e *larger* than q (the group order) before reducing mod q does not increase security but can introduce a subtle bias — and how to avoid that bias.

Exercise 31.10 (zk-SNARK threat model, conceptual)**.** A zk-rollup posts a succinct proof that a batch of 10,000 transactions was executed correctly, and the layer-1 chain verifies it in a few

milliseconds. (a) Which of the three properties — completeness, soundness, zero-knowledge — is the critical one for *the chain's* security, and why? (b) For a SNARK requiring trusted setup, explain in two or three sentences what the "toxic waste" is and the precise condition under which the system remains sound. (c) Name one tradeoff you accept by choosing a transparent (setup-free) STARK instead.

Chapter 32

Commitments and Secret Sharing

In the previous chapter, we encountered commitments as the cryptographic envelopes that hold a prover to their word in zero-knowledge proofs. Here we unpack commitments and secret sharing in their own right. These two primitives underpin many systems across modern cryptography—from electronic voting and sealed-bid auctions to threshold cryptocurrency custody and the multiparty computation protocols in the chapters ahead.

The first is a *commitment*: a way for Alice to lock a value into a sealed envelope, hand the envelope to Bob, and then later open it — with the guarantees that Bob cannot peek inside before she opens it, and that Alice cannot swap the contents after the fact. The second is *secret sharing*: a way to split a secret into n pieces so that any t of them reconstruct it but any $t-1$ reveal *nothing at all*. Both rely on standard mathematical tools: hash functions, the discrete logarithm problem, and polynomials over a finite field.

32.1 The sealed envelope: commitment schemes

Picture a coin flip conducted over the telephone. Alice and Bob live in different cities and want to flip a coin fairly to decide who gets a contested concert ticket. Alice says "call it" and Bob says "heads." Alice then announces "sorry, it was tails." Bob has no way to know whether Alice flipped fairly or just declared whatever beat his call. The protocol is hopeless because Alice's commitment to the outcome and her revelation of it happen at the same instant, and she controls both.

A commitment scheme fixes exactly this. Alice flips the coin privately, gets (say) heads, and sends Bob a *commitment* c — a scrambled token that is bound to "heads" but reveals nothing about it. Bob then makes his guess. Only afterward does Alice *open* the commitment, sending Bob the value and whatever extra information he needs to check that c really was a commitment to heads. Now neither party can cheat: Alice cannot change her flip after seeing Bob's guess, and Bob learns nothing from c that would let him guess better than chance. Figure 32.1 shows the three-message shape.

Intuition

In physical terms, a commitment behaves like a locked, opaque strongbox. Alice places her value inside, locks it, and hands the box to Bob (this is *commit*). Bob holds the box but

cannot see in (*hiding*). Later Alice hands him the key, he opens it, and reads the value (*open*). Because the box was sealed the whole time, Alice could not have reached in to swap the contents (*binding*). Two properties, two directions of cheating prevented.

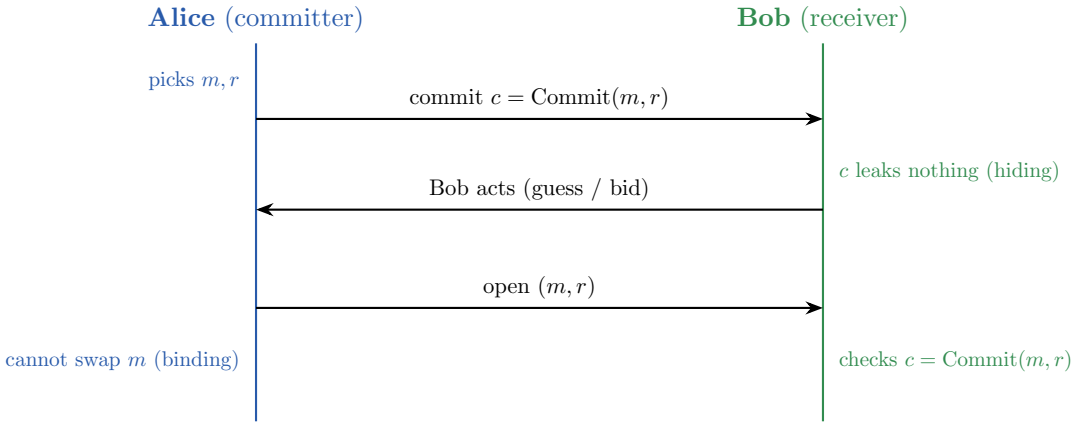


Figure 32.1: The commit–reveal flow. Alice binds herself to m with c before Bob acts; the value stays hidden until she opens. Hiding protects Bob's move; binding stops Alice from changing m afterward.

Formally a commitment scheme is a pair of algorithms. The committing party chooses a message m and a fresh random value r (the *opening* or *nonce*), and computes

$$c = \text{Commit}(m, r).$$

She publishes c. To open, she reveals (m, r), and anyone recomputes $\text{Commit}(m, r)$ and checks it equals the c they saw. The two security goals are duals of each other.

Definition 32.1 (Hiding and binding)**.** A commitment scheme is *hiding* if the commitment c leaks nothing about m: for any two messages m_0, m_1, the distributions of $\text{Commit}(m_0, r)$ and $\text{Commit}(m_1, r)$ over random r are indistinguishable. It is *binding* if the committer cannot open one commitment two ways: it is infeasible to find $(m, r) \neq (m', r')$ with $\text{Commit}(m, r) = \text{Commit}(m', r')$.

Each property can hold *perfectly* (against an unbounded adversary) or *computationally* (against a PPT adversary). A short but deep fact: you cannot have both perfectly at once.

Why it works

Why can't a commitment be both perfectly hiding *and* perfectly binding? Suppose it were perfectly hiding. Then for every commitment c and every message m, there must exist some opening r with $\text{Commit}(m, r) = c$ — otherwise c would rule out m, leaking information, contradicting perfect hiding. But if every message has a valid opening of c, an unbounded Alice can find an opening to *any* message she likes, so the scheme cannot be binding against an unbounded adversary. The contrapositive runs the same way. So one of the two properties is always merely computational. Designers choose which side to make unconditional based on the threat model: a scheme that must protect a secret *forever* wants perfect hiding; one that must prevent forgery against an unbounded prover wants perfect binding.

32.1.1 The threat model

Spell out who is trying to do what. Against *hiding*, the adversary is the receiver Bob, who holds c and wants to learn m before Alice opens. Against *binding*, the adversary is the sender Alice, who has committed to c and now wants to open it as a different value than she originally intended. A good scheme defends against both, and the security argument must say which party is bounded and which capability is assumed hard.

32.2 Building commitments from hash functions

The most direct construction uses a cryptographic hash. Let H be a hash function (think SHA-256). To commit to a message m, pick a long random nonce r — say 256 bits — and publish

$$c = \mathrm{H}(m \,\|\, r).$$

To open, reveal (m, r); the verifier recomputes $\mathrm{H}(m \,\|\, r)$ and checks it matches c.

Why it works

Binding reduces to collision resistance. If Alice could open c as both (m, r) and (m', r') with $m \neq m'$, then $\mathrm{H}(m \,\|\, r) = \mathrm{H}(m' \,\|\, r')$ is a collision in H — which we assume is infeasible to find. So binding holds as long as the hash is collision resistant.
Hiding comes from the nonce. Without r, committing to a coin flip as H("heads") would be a disaster: Bob just computes H("heads") and H("tails") himself and compares. The message space is tiny, so the commitment is directly invertible by brute force. The random r blows the effective input space up to 2^{256} possibilities, so Bob cannot enumerate. Formally, hiding here is computational and is usually argued by modeling H as a random oracle, in which case c is just a uniformly random string independent of m until someone queries the exact $m \,\|\, r$.

Common trap

The nonce is not optional, and it must be *long and unpredictable*. Two real failures: (1) committing to a low-entropy value with no nonce (a vote, a bid, a yes/no) lets the receiver brute-force the message in microseconds. (2) Reusing the same r across commitments, or deriving r from m deterministically, can leak relationships between committed messages. Always draw r fresh and uniformly. A commitment to a 1-bit secret with a 256-bit random nonce is fine; a commitment to the same bit with no nonce is worthless.

Implement it

A toy hash commitment in Python. This is for *learning only* — it skips constant-time comparison, input-length framing (so $m \,\|\, r$ is unambiguous), and the domain separation a real library would insist on.

```
import hashlib, os

def commit(message: bytes):
    r = os.urandom(32)                      # 256-bit nonce, fresh every time
    c = hashlib.sha256(message + r).digest()
```

```
    return c, r                                  # publish c, keep (message, r) to open

def verify(c, message: bytes, r: bytes) -> bool:
    return hashlib.sha256(message + r).digest() == c

# Alice flips a coin and commits.
flip = b"heads"
c, r = commit(flip)
print("commitment:", c.hex())      # Bob sees only this; learns nothing about flip

# ... Bob makes his guess ...

# Alice opens.
print("opens to:", flip, "valid?", verify(c, flip, r))   # True
# Alice cannot now claim it was b"tails": verify(c, b"tails", r) is False,
# and finding r' with sha256(b"tails"+r')==c is a SHA-256 preimage break.
```

A subtle bug lurks in `message + r`: if message lengths can vary, `b"ab"+b"c"` and `b"a"+b"bc"` collide. Real code length-prefixes each field. We will let it slide for a fixed-length flip.

The hash commitment is computationally hiding and computationally binding. That is enough for most applications, and it is very fast. But sometimes you want stronger guarantees or extra structure — in particular, the ability to do arithmetic *on commitments without opening them*. For that we turn to the discrete log.

32.3 Pedersen commitments: perfect hiding and homomorphism

Let $\mathbb{G}$ be a cyclic group of prime order q where the discrete log problem is hard — a subgroup of $(\mathbb{Z}/p\mathbb{Z})^*$, or an elliptic curve group from Chapter 21. Fix two generators g and h whose relative discrete log nobody knows: $h = g^x$ for some x that not even the person who set up the parameters has stored. (One produces h by hashing a fixed string to a group element, so that x is unknown to everyone.) To commit to a message $m \in \mathbb{Z}/q\mathbb{Z}$, pick a random $r \in \mathbb{Z}/q\mathbb{Z}$ and publish

$$c = g^m h^r.$$

To open, reveal (m, r); the verifier checks $c \stackrel{?}{=} g^m h^r$.

Why it works

Perfectly hiding. For any fixed m, as r ranges uniformly over $\mathbb{Z}/q\mathbb{Z}$, the term h^r ranges uniformly over all of $\mathbb{G}$ (because h generates the group and q is prime). So $c = g^m h^r$ is a uniformly random group element, with a distribution that does not depend on m in the slightest. Given c, *every* message m' is equally consistent: there is exactly one r' making $c = g^{m'} h^{r'}$. An unbounded Bob learns nothing. This is information-theoretic hiding — it holds forever, even against a quantum computer.

Computationally binding. Suppose Alice opens c two ways: $g^m h^r = g^{m'} h^{r'}$ with $m \neq m'$. Rearranging, $g^{m-m'} = h^{r'-r}$, so

$$h = g^{(m-m')(r'-r)^{-1} \bmod q},$$

and Alice has just computed $\text{dlog}_g h = x$, the one number nobody is supposed to know. So breaking binding is exactly as hard as solving discrete log for h relative to g. As long as that is infeasible, Alice is stuck with her commitment.

Compare the two schemes. The hash version is computationally hiding and computationally binding. The Pedersen version trades up to *perfect* hiding at the cost of making binding rest on discrete log (and being far slower, since it needs group exponentiations). By the impossibility result, you cannot get perfect on both sides, and Pedersen has chosen the hiding side. If you instead want perfect binding, use $c = g^m$ with no nonce in a group where g has prime order — now each m gives a unique c, but the scheme is only computationally hiding and only safe for high-entropy m.

32.3.1 The homomorphic bonus

Pedersen commitments have a property the hash version cannot match: they add. Multiply two commitments and the exponents add,

$$c_1 \cdot c_2 = (g^{m_1} h^{r_1})(g^{m_2} h^{r_2}) = g^{m_1+m_2}\, h^{r_1+r_2},$$

which is a valid commitment to m_1+m_2 with opening r_1+r_2. Figure 32.2 illustrates this homomorphic addition.

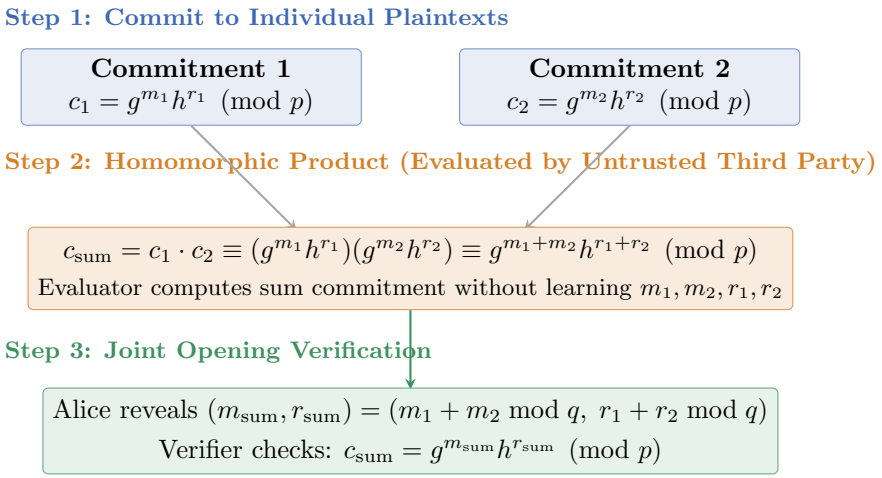


Figure 32.2: Pedersen commitment and its additive homomorphism. Because group exponents add under modular multiplication, multiplying commitments c_1 and c_2 yields a valid commitment to the sum $m_1 + m_2$ under blinding factor $r_1 + r_2$.

Walking question

Question. Suppose Alice commits to $m_1 = 7$ and $m_2 = 11$ with the same nonce r in both, instead of fresh nonces. What goes wrong?
Answer. Hiding within each commitment still holds, but the *relationship* leaks through the homomorphism. Consider $c_1/c_2 = g^{m_1-m_2} h^{r-r} = g^{m_1-m_2} = g^{-4}$. The nonces cancel, leaving a nonce-free commitment $g^{m_1-m_2}$ that is only computationally hiding — and if the difference comes from a small set, Bob brute-forces it. The lesson is the same as for hash commitments:

nonces must be fresh and independent.

In the real world

Commitments are everywhere once you look. **TLS and Signal** use the hash-commit pattern implicitly whenever a protocol transcript is hashed and bound before later steps. **Certificate Transparency** logs commit to the set of issued certificates via Merkle trees (a tree of hash commitments) so that a misissued certificate cannot be hidden. **Cryptocurrencies** use Pedersen commitments for confidential amounts (Monero, Mimblewimble) and hash commitments in commit-reveal schemes for on-chain randomness and sealed-bid auctions. **Zero-knowledge proofs** (Chapter 31) are riddled with commitments: the prover commits to its randomness, the verifier challenges, the prover opens — the commit-challenge-open shape is the skeleton of nearly every interactive proof.

32.4 Splitting a secret: the problem

Now a different scenario. The launch code for a missile silo, or the master key that decrypts a company's password vault, is too dangerous to entrust to any single person and too important to lose if one person is hit by a bus. You want to hand pieces to five officers such that any *three* of them together can reconstruct the key, but any two — even two who collude, even two with unlimited computing power — learn *absolutely nothing*. This is a (t, n) *threshold secret sharing* scheme with $t = 3$ and $n = 5$.

The naive ideas all fail. Giving each officer a third of the bits of the key? Then two officers know two-thirds of the bits and can brute-force the rest — and any officer alone already knows a third, which is a leak. Encrypting the key and splitting the ciphertext? Same problem; partial ciphertext is partial information. We want a scheme where $t - 1$ shares are *statistically independent* of the secret, so they reveal nothing in the information-theoretic sense, not merely the computational one.

Intuition

The trick rests on a fact from high-school geometry, sharpened. Two points determine a unique line. If we draw a line and the secret is its height where it crosses the y-axis, then any *one* point on the line tells you nothing about that height — infinitely many lines pass through a single point, each hitting the axis somewhere different. But *two* points pin the line down completely, and you can read off the intercept. To go from "2 reconstruct" to "t reconstruct," replace the line (degree 1) with a polynomial of degree $t - 1$: it takes exactly t points to determine it, and any $t - 1$ points leave it completely undetermined.

32.5 Shamir secret sharing

Adi Shamir published this scheme in 1979 in "How to Share a Secret," formulating threshold secret sharing via polynomial interpolation over a finite field.

History

Adi Shamir and George Blakley independently published threshold secret sharing schemes in 1979. Blakley framed secret sharing geometrically using intersecting hyperplanes in vector spaces, whereas Shamir used polynomial interpolation over finite fields. Shamir's polynomial approach became the standard because shares are compact (each share is a single field element, regardless of threshold t) and the underlying algebra is simple. The breakthrough arrived during a fertile era that immediately followed Diffie-Hellman (1976) and RSA (1977), leading to verifiable secret sharing (Paul Feldman, 1987) and secure multiparty computation (Goldreich, Micali, Wigderson and Ben-Or, Goldwasser, Wigderson, 1987–1988). An active frontier today is threshold signing without interaction (such as FROST) and verifiable secret sharing with sublinear communication for thousands of distributed nodes.

32.5.1 Setup and why a field

All arithmetic happens in a finite field $\mathbb{F}_p$ for a prime p larger than both the secret s and the number of participants n. (One can also use $\mathbb{F}_{2^k}$ for byte-aligned implementations; the algebra is identical.) We need a field — not just integers mod a composite — because reconstruction divides by differences of evaluation points, and division is only guaranteed in a field, where every nonzero element has an inverse.

Connection

This is the payoff of the finite-field chapter (Chapter 6). A field is precisely the structure where the four operations $+, -, \times, \div$ all behave, and Lagrange interpolation needs all four. Over $\mathbb{Z}/n\mathbb{Z}$ with composite n you can hit a non-invertible difference $x_i - x_j$ and the whole reconstruction collapses. Over $\mathbb{F}_p$ it never happens, as long as the evaluation points are distinct and nonzero mod p. The companion algorithms book's treatment of polynomial evaluation and the cost of modular inversion (via extended Euclid) is exactly the cost model for sharing and reconstruction here.

32.5.2 Dealing the shares

To share a secret $s \in \mathbb{F}_p$ among n players with threshold t:

1. Set the constant term to the secret: $a_0 = s$.
2. Pick the other coefficients $a_1, \ldots, a_{t-1}$ *uniformly at random* from $\mathbb{F}_p$. This defines a random polynomial of degree $t - 1$:
$$f(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{t-1} x^{t-1} \pmod{p}.$$
3. Give player i (for $i = 1, \ldots, n$) the share $(i, f(i))$. The x-coordinates are public labels; the y-values are the secret shares. In particular, *nobody receives* $f(0) = s$ itself — that point remains the unshared secret.

To reconstruct from any t shares, interpolate the unique degree-$(t-1)$ polynomial through them and evaluate at 0.

32.5.3 A worked example over $\mathbb{F}_{17}$

Let us share the secret $s = 5$ with threshold $t = 3$ among $n = 5$ players, working in $\mathbb{F}_{17}$ (so $p = 17 > n$ and $> s$). Because $t = 3$ we need a degree-2 polynomial. Set $a_0 = 5$ and pick random $a_1 = 8$, $a_2 = 3$:

$$f(x) = 5 + 8x + 3x^2 \pmod{17}.$$

Now evaluate at $x = 1, \ldots, 5$, reducing mod 17 each time:

$$\begin{aligned} f(1) &= 5 + 8 + 3 = 16, \\ f(2) &= 5 + 16 + 12 = 33 \equiv 16, \\ f(3) &= 5 + 24 + 27 = 56 \equiv 56 - 51 = 5, \\ f(4) &= 5 + 32 + 48 = 85 \equiv 85 - 85 = 0, \\ f(5) &= 5 + 40 + 75 = 120 \equiv 120 - 119 = 1. \end{aligned}$$

So the five shares are

$$(1, 16),\ (2, 16),\ (3, 5),\ (4, 0),\ (5, 1).$$

Hand one to each officer. Note $(4, 0)$: a share can perfectly well be zero, and that zero leaks nothing.

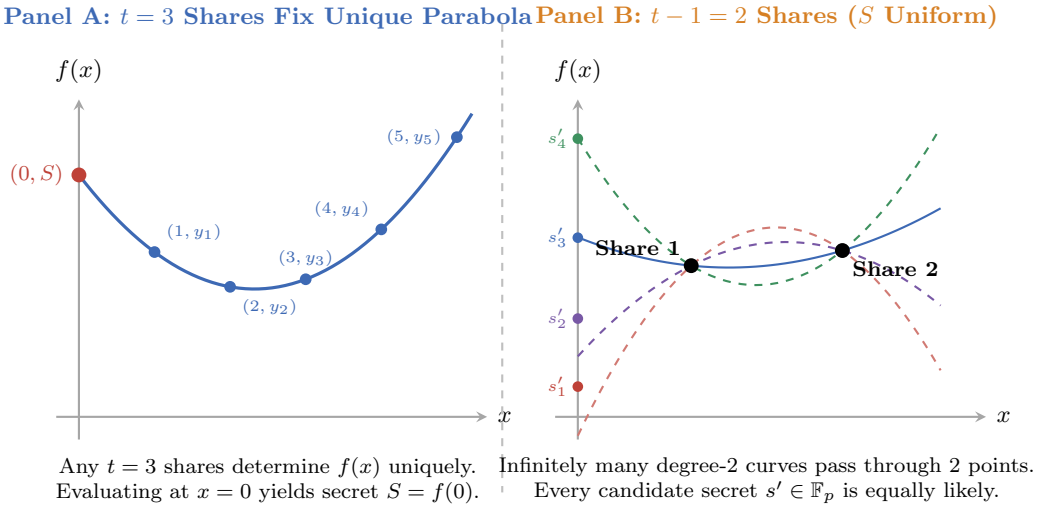


Figure 32.3: Geometry of Shamir's Secret Sharing. With threshold $t = 3$, any three points uniquely fix the degree-2 polynomial and its secret intercept $S = f(0)$ (Panel A). Any $t - 1 = 2$ shares are compatible with an infinite family of parabolas sweeping across every candidate value $s' \in \mathbb{F}_p$ with equal probability (Panel B).

The geometric intuition over $\mathbb{R}$ reflects the algebraic reality over finite fields: over $\mathbb{F}_p$, a polynomial of degree $t - 1$ has t degrees of freedom. Holding $t - 1$ shares leaves exactly one free coefficient, which establishes a strict 1-to-1 bijection between candidate polynomials and candidate secret values $s' = f(0) \in \mathbb{F}_p$.

32.5.4 Reconstruction by Lagrange interpolation

To reconstruct the secret from any t shares $(x_1, y_1), \ldots, (x_t, y_t)$, we invoke the Lagrange interpolation formula. The underlying idea is to construct t auxiliary polynomials $L_1(x), \ldots, L_t(x)$, each of degree

$t-1$, satisfying:

$$L_j(x_m) = \begin{cases} 1 & \text{if } m = j, \\ 0 & \text{if } m \neq j. \end{cases}$$

Each basis polynomial $L_j(x)$ is defined as a product of linear factors that vanish at every evaluation node except x_j:

$$L_j(x) = \prod_{m \neq j} \frac{x - x_m}{x_j - x_m}.$$

When $x = x_j$, every fraction in the product has identical numerator and denominator, yielding $L_j(x_j) = 1$. When $x = x_m$ for some $m \neq j$, the numerator includes $(x_m - x_m) = 0$, so $L_j(x_m) = 0$. The full polynomial is the linear combination $f(x) = \sum_{j=1}^{t} y_j L_j(x)$.

Because the secret is the constant term $s = f(0)$, we only need to evaluate each basis polynomial at $x = 0$:

$$\lambda_j \;=\; L_j(0) \;=\; \prod_{m \neq j} \frac{0 - x_m}{x_j - x_m} \;=\; \prod_{m \neq j} \frac{-x_m}{x_j - x_m} \pmod{p}.$$

The secret is recovered as the linear combination:

$$s = f(0) = \sum_{j=1}^{t} y_j \lambda_j \pmod{p}.$$

where the sum runs over the chosen shares. Compute each coefficient $\lambda_j = \prod_{m \neq j} \frac{-x_m}{x_j - x_m}$ for our points $x = 2, 3, 5$.

For j at $x_j = 2$:

$$\lambda_2 = \frac{(0-3)(0-5)}{(2-3)(2-5)} = \frac{(-3)(-5)}{(-1)(-3)} = \frac{15}{3} = 5.$$

For j at $x_j = 3$:

$$\lambda_3 = \frac{(0-2)(0-5)}{(3-2)(3-5)} = \frac{(-2)(-5)}{(1)(-2)} = \frac{10}{-2} = -5 \equiv 12 \pmod{17}.$$

For j at $x_j = 5$:

$$\lambda_5 = \frac{(0-2)(0-3)}{(5-2)(5-3)} = \frac{(-2)(-3)}{(3)(2)} = \frac{6}{6} = 1.$$

Now combine with the y-values $16, 5, 1$:

$$s = 16 \cdot 5 + 5 \cdot 12 + 1 \cdot 1 = 80 + 60 + 1 = 141 \equiv 141 - 8 \cdot 17 = 141 - 136 = 5 \pmod{17}.$$

We recover $s = 5$. And it works no matter *which* three shares we pick — try officers 1, 4, 5 and you will land on 5 again.

Walking question

Question. In computing λ_3 we wrote $\frac{10}{-2}$. Over the rationals that is -5; over $\mathbb{F}_{17}$ what does division even mean here, and why did -5 become 12?

Answer. Division by -2 in $\mathbb{F}_{17}$ means multiplication by the inverse of $-2 \equiv 15$. Since $15 \cdot 8 = 120 = 7 \cdot 17 + 1 \equiv 1$, we have $15^{-1} = 8$, so $10 \cdot 8 = 80 \equiv 80 - 68 = 12$. And indeed $-5 \equiv 12 \pmod{17}$. The two routes agree because every non-zero element in a finite field has

a multiplicative inverse. Over a composite modulus, if $x_j - x_m$ shared a common factor with the modulus, no multiplicative inverse would exist and interpolation would fail.

32.5.5 Why $t-1$ shares reveal nothing

This is the heart of the scheme, and it is genuinely perfect secrecy, not a computational assumption.

Theorem 32.1 (Information-theoretic security of Shamir)**.** *Given any $t-1$ shares, every candidate secret $s' \in \mathbb{F}_p$ is equally likely. The shares are statistically independent of the secret.*

Why it works

Fix any $t-1$ shares $(x_1, y_1), \ldots, (x_{t-1}, y_{t-1})$ and propose any secret value s'. Claim: there is *exactly one* degree-$(t-1)$ polynomial consistent with both the $t-1$ shares and the additional point $(0, s')$. That is t points total, and t points determine a unique degree-$(t-1)$ polynomial — so the candidate $(0, s')$ together with the held shares pins down exactly one polynomial, for *every* choice of s'.

Because the dealer chose the coefficients $a_1, \ldots, a_{t-1}$ uniformly at random, each of those p possible polynomials (one per value of s') was equally likely to have been the one used. So conditioned on the $t-1$ observed shares, the posterior distribution over the secret is uniform on $\mathbb{F}_p$: the shares told the attacker nothing. This is the same flavor of argument as the one-time pad in Chapter 9 — every plaintext stays equally consistent with what the adversary sees.

Concretely with our example: hold the two shares $(1, 16), (2, 16)$. Could the secret be 5? Yes — our actual polynomial works. Could it be 0? Also yes: there is a unique degree-2 polynomial through $(0, 0), (1, 16), (2, 16)$, namely some other f with $a_0 = 0$, equally plausible. Two shares cannot tell these apart. Only the third point breaks the tie.

Implement it

Shamir sharing and reconstruction in Python over a prime field. *Learning only*: a real library uses a constant-time field, validates share indices, authenticates shares (see verifiable sharing below), and uses a cryptographically secure RNG with care. The standard library `random` below is **not** secure — swap in `secrets` for anything real.

```
import random
P = 2**127 - 1   # a Mersenne prime field; secret must be < P

def share(secret, t, n):
    # coefficients: a0 = secret, rest uniform in F_P
    coeffs = [secret] + [random.randrange(P) for _ in range(t - 1)]
    def f(x):
        y = 0
        for a in reversed(coeffs):     # Horner's rule
            y = (y * x + a) % P
        return y
    return [(i, f(i)) for i in range(1, n + 1)]

```

```python
def reconstruct(shares):
    # Lagrange interpolation evaluated at x = 0
    secret = 0
    for j, (xj, yj) in enumerate(shares):
        num, den = 1, 1
        for m, (xm, _) in enumerate(shares):
            if m == j:
                continue
            num = (num * (-xm)) % P           # (0 - xm)
            den = (den * (xj - xm)) % P       # (xj - xm)
        lam = num * pow(den, -1, P) % P       # modular inverse via Fermat/EEA
        secret = (secret + yj * lam) % P
    return secret

shares = share(1234567, t=3, n=5)
print(reconstruct(shares[:3]))    # 1234567, any 3 of the 5 work
print(reconstruct(shares[1:4]))   # 1234567 again
```

32.5.6 Properties that make it shine

A few features fall out for free and explain why Shamir is the default in practice:

- **Minimal share size.** Each share is one field element, the same size as the secret, regardless of t. You pay no penalty for a high threshold.
- **Flexible threshold.** Choose any (t, n) with $t \le n$. New players can be added by handing out $f(n+1), f(n+2), \ldots$ without disturbing existing shares.
- **Proactive refresh.** The dealer (or the players jointly) can add a fresh random polynomial g with $g(0) = 0$ to everyone's share: $f(i) \mapsto f(i) + g(i)$. The secret $f(0) + g(0) = f(0)$ is unchanged, but old shares are now useless. This periodically "re-randomizes" shares so an attacker must compromise t players within a single epoch.
- **Weighted access.** Give an important player several shares (several x-coordinates) to give them more voting weight. A vice-president with two shares plus one officer can hit $t = 3$.

32.6 The dealer problem and verifiable secret sharing

Plain Shamir assumes a trustworthy dealer and cooperative players. Two ways that breaks down in the wild:

Attack

A cheating dealer. A malicious dealer can hand out inconsistent shares — points that do not lie on any single degree-$(t-1)$ polynomial. Then different subsets of t players reconstruct *different* secrets, and nobody can tell who got bad data. In our $\mathbb{F}_{17}$ example, suppose the dealer gives player 3 the share $(3, 9)$ instead of the correct $(3, 5)$. Then officers $\{2, 3, 5\}$ interpolate one value and officers $\{1, 2, 4\}$ interpolate another. The shares are individually plausible, so

the fraud is invisible until reconstruction — by which time you may have lost the only chance to fix it.

A cheating player at reconstruction. Even with a trustworthy dealer, a single player can submit a forged share during reconstruction and steer the recovered value to anything they like (since they control one Lagrange term). Worse, they can submit the forgery, watch the others reveal genuine shares, and then privately recover the true secret while everyone else gets garbage.

Verifiable secret sharing (VSS) closes both holes by publishing commitments that let every player check their own share against a public, binding record of the polynomial — without revealing the secret. Feldman's 1987 scheme is the classic, and it reuses the discrete-log commitment machinery from earlier in this chapter.

32.6.1 Feldman VSS

Work in a group $\mathbb{G}$ of prime order q with generator g where discrete log is hard, and take the secret and coefficients in $\mathbb{F}_q$. The dealer runs ordinary Shamir to build $f(x) = a_0 + a_1 x + \cdots + a_{t-1}x^{t-1}$ with $a_0 = s$, and distributes shares $f(i)$ privately as usual. *In addition*, the dealer broadcasts public commitments to every coefficient:

$$C_0 = g^{a_0}, \quad C_1 = g^{a_1}, \quad \ldots, \quad C_{t-1} = g^{a_{t-1}}.$$

Now any player i can verify their share $f(i)$ is consistent with the public commitments, because exponentiation turns the polynomial identity into a checkable group equation:

$$g^{f(i)} \stackrel{?}{=} \prod_{k=0}^{t-1} C_k^{i^k} = g^{a_0} \, (g^{a_1})^{i} \, (g^{a_2})^{i^2} \cdots .$$

Why it works

Why does the product equal $g^{f(i)}$? Just collect exponents:

$$\prod_{k=0}^{t-1} C_k^{i^k} = \prod_{k} (g^{a_k})^{i^k} = g^{\sum_k a_k i^k} = g^{f(i)}.$$

So a share passes the check exactly when it sits on the polynomial whose coefficients the dealer committed to. The dealer cannot hand out an off-curve share to anyone without it failing this public test — binding of the commitments (discrete log hardness) forbids opening C_k to a different a_k. And the commitments leak nothing usable about s when s is a high-entropy key: $C_0 = g^s$ is just a discrete-log-hard image of the secret, computationally hiding.

Common trap

Feldman VSS hides the secret only *computationally*, because $C_0 = g^s$ commits to s with no nonce. If the secret has low entropy, g^s can be brute-forced — exactly the no-nonce pitfall from the hash commitment. Pedersen VSS fixes this by committing to each coefficient with a paired blinding polynomial, $C_k = g^{a_k} h^{b_k}$, recovering perfect hiding of the secret at the cost of

one extra polynomial. Use Feldman when the secret is itself a high-entropy key; use Pedersen VSS when the secret might be guessable.

32.7 Distributed key generation: removing the dealer

VSS still has *one* dealer who knows the whole secret at deal time. For the highest-assurance systems — a threshold signing key that should never exist in one place, ever — you want *distributed key generation* (DKG). The idea composes the pieces we have:

1. Every one of the n players acts as a dealer for a random secret s_i of their own, running Feldman VSS to share s_i to everyone and publishing coefficient commitments.
2. Each player verifies the shares they received from every other player; complaints disqualify cheaters.
3. The group secret is defined as $s = \sum_i s_i$ over the surviving dealers, and each player's share of s is the sum of the shares they received, $\sum_i f_i(\text{me})$.

By the homomorphism of Shamir sharing (the sum of polynomials sharing s_i is a polynomial sharing $\sum s_i$), the players now hold valid shares of a secret that *no single participant ever knew* — not even the dealers, since s depends on everyone's contribution. The public key $g^s = \prod_i C_0^{(i)}$ is computable by all, while the private key s lives only as a threshold sharing. Figure 32.4 shows how Feldman VSS verifies individual shares against public coefficient commitments and how DKG sums dealer rows to eliminate the single trusted dealer.

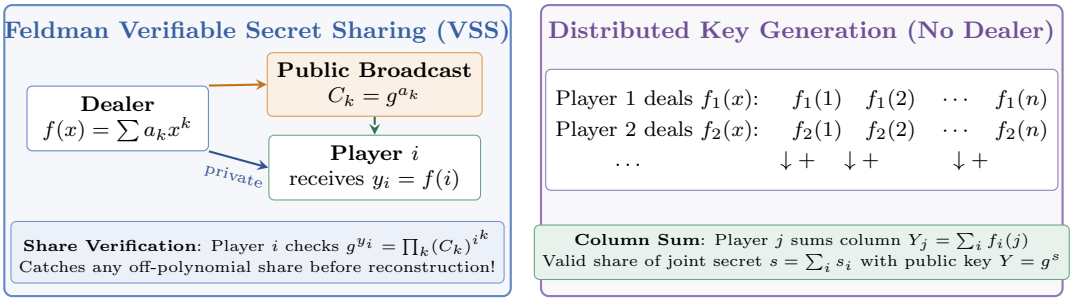


Figure 32.4: From Verifiable Secret Sharing (VSS) to Distributed Key Generation (DKG). Left: in Feldman VSS, the dealer publishes group commitments $C_k = g^{a_k}$ to the polynomial coefficients so each player i can verify $g^{y_i} = \prod C_k^{i^k}$ in the exponent. Right: in DKG, every participant acts as a VSS dealer for their own random polynomial $f_i(x)$ and sums the column of shares they receive, creating a shared secret $s = \sum s_i$ that no single party ever held.

In the real world

DKG plus threshold signatures is how modern **threshold wallets** and **validator key management** work. A custody service can require t-of-n hardware modules to cooperate to sign a transaction, so stealing fewer than t devices yields nothing and losing up to $n - t$ devices loses no funds. The same machinery secures **distributed certificate authorities**, **DNSSEC**

root key ceremonies (where physical smart cards instantiate (t, n) sharing), and **validator clusters** in proof-of-stake blockchains that sign as a group. Threshold ECDSA and threshold BLS signatures, both built on these primitives, run in production custody systems today.

32.8 Where this gets used: key escrow, multisig, distributed trust

The applications of secret sharing share a common design goal: *no single point of failure, no single point of trust.*

Key escrow and recovery. An enterprise encrypts its data with a master key. If the key is lost the data is gone; if one administrator holds it, that administrator is a liability and a target. Split the master key 3-of-5 across executives kept in separate locations. Day-to-day operations use a separate operational key; the master is reconstructed only in a genuine recovery event, and only by a quorum. The infamous government "key escrow" proposals of the 1990s (the Clipper chip) used related ideas to give law enforcement a recovery path — and the fierce debate over them is a reminder that the policy question (who should be in the quorum?) is harder than the math.

Multisignature wallets. "Multisig" in cryptocurrencies often means t-of-n *independent* signatures recorded on chain (a script checks that t distinct public keys signed). Threshold signatures via DKG are the cryptographic cousin: they produce a *single* signature under a single public key that is indistinguishable from an ordinary one, but which required t parties to cooperate. The on-chain version is transparent and simple; the threshold version is private and compact. Both deliver the same governance: a 2-of-3 wallet survives losing one key and resists one compromised key.

Distributed trust. Certificate authorities, code-signing roots, and password vaults all concentrate enormous trust in one secret. Sharing it across mutually-suspicious parties, in separate jurisdictions or organizations, means an attacker must subvert a quorum rather than a single box — and proactive refresh means they must do it fast. This is defense in depth applied to the secret itself.

Connection

Secret sharing and additive homomorphism provide the foundation for the next two chapters. In the next chapter, **homomorphic encryption** (Chapter 33) allows an untrusted server to perform additions and multiplications directly on encrypted ciphertexts without ever decrypting them. Then **multiparty computation** (Chapter 34) lets shareholders compute on a shared secret — adding and multiplying shared values — so that a function of private inputs is evaluated without anyone seeing the inputs. The homomorphism we used in DKG ($\sum$ of shares = share of $\sum$) is the addition gate of MPC; multiplication needs more work but starts here. And **zero-knowledge proofs** (Chapter 31) lean on commitments at every turn. Commitments and secret sharing serve as basic building blocks throughout multi-party and zero-knowledge protocols.

32.9 Summary

A commitment is a cryptographic sealed envelope: *hiding* keeps the receiver from peeking, *binding* keeps the sender from swapping. You cannot have both unconditionally, so you pick a side. Hash

commitments $\mathrm{H}(m \,\|\, r)$ are fast and computationally secure on both fronts — never forget the nonce r for low-entropy messages. Pedersen commitments $g^m h^r$ buy perfect hiding from the discrete log assumption and throw in an additive homomorphism that powers confidential transactions.

Shamir secret sharing splits a secret into n shares so any t reconstruct it and any $t-1$ learn nothing — perfect, information-theoretic secrecy — using a random degree-$(t-1)$ polynomial whose constant term is the secret. Reconstruction is Lagrange interpolation in a finite field. Verifiable secret sharing (Feldman, Pedersen) adds public commitments so players can detect a cheating dealer; distributed key generation removes the dealer entirely, producing a key no one ever held. These power key escrow, threshold wallets, and distributed trust — and, together with the zero-knowledge proofs of Chapter 31, they form the building blocks for secure multiparty computation in Chapter 34. First, in Chapter 33, we ask how a single untrusted server can compute directly on encrypted data without holding any secret share at all: homomorphic encryption.

32.10 Exercises

Exercise 32.1. *(Compute a hash commitment by hand-ish.)* Alice commits to the bit $b = 1$ with no nonce, publishing $c = \mathrm{H}(\text{“1”})$. Bob wants to learn b before Alice opens. Describe exactly what Bob computes and how many hash evaluations it costs him. Then redo the analysis when Alice uses a uniform 256-bit nonce, and explain why Bob's strategy now fails.

Exercise 32.2. *(Binding break is a collision.)* Suppose someone hands you an efficient algorithm that, given a hash commitment $c = \mathrm{H}(m \,\|\, r)$, outputs $(m', r') \neq (m, r)$ that also opens c. Show step by step how to turn this into a collision finder for H. Conclude that a collision-resistant hash yields a binding commitment.

Exercise 32.3. *(Pedersen perfect hiding.)* In a group of prime order q with generators g, h, show that for a fixed message m the commitment $c = g^m h^r$ is uniformly distributed over the whole group as r ranges uniformly over $\mathbb{F}_q$. Then show that for any *other* message m' there is exactly one r' with $g^{m'} h^{r'} = c$, and write r' in terms of m, m', r, and $x = \mathrm{dlog}_g h$.

Exercise 32.4. *(Equivocation reveals the trapdoor.)* Prove that if Alice can open a single Pedersen commitment c as two different messages, she can compute $\mathrm{dlog}_g h$. Give the explicit formula. Why does this mean binding is exactly as hard as discrete log?

Exercise 32.5. *(Share a secret yourself.)* Working in $\mathbb{F}_{23}$, share the secret $s = 9$ with threshold $t = 2$ and $n = 4$ using the line $f(x) = 9 + 7x \bmod 23$. List the four shares. Then take any two of them and reconstruct s via Lagrange interpolation, showing the modular inverse you use.

Exercise 32.6. *(Threshold three.)* In $\mathbb{F}_{17}$, the polynomial $f(x) = 5 + 8x + 3x^2$ from the chapter produced shares $(1, 16), (2, 16), (3, 5), (4, 0), (5, 1)$. Reconstruct s from the shares of officers $\{1, 4, 5\}$ and confirm you get 5. Then show that the two shares $\{1, 4\}$ alone are consistent with the secret being any value in $\mathbb{F}_{17}$ by exhibiting, for the candidate $s' = 0$, the unique degree-2 polynomial through $(0, 0), (1, 16), (4, 0)$.

Exercise 32.7. *(Why a field, not a ring.)* Suppose you tried to run Shamir over $\mathbb{Z}/12\mathbb{Z}$ (composite) with evaluation points $x = 1, \ldots, n$. Find two evaluation points whose difference is not invertible mod 12, and explain concretely how Lagrange reconstruction can fail. What property of $\mathbb{F}_p$ guarantees this never happens?

Exercise 32.8. *(Proactive refresh.)* Players hold Shamir shares of a secret s under a degree-2 polynomial. The group jointly generates a random degree-2 polynomial g with $g(0) = 0$ and each player adds $g(i)$ to their share. Prove the secret is unchanged, that old and new shares cannot be mixed to reconstruct, and explain in one sentence why this defends against a slow attacker compromising players over time.

Exercise 32.9. *(Catch the cheating dealer.)* In Feldman VSS over a prime-order group, the dealer publishes $C_0, \ldots, C_{t-1}$ and gives player i a share y_i. Write the exact equation player i checks. Then suppose a dealer sends player 3 a tampered share $y_3' = y_3 + 1$; show that the verification equation fails, and argue why the dealer cannot adjust the public commitments to make it pass without breaking discrete log.

Exercise 32.10. *(Homomorphic shares.)* Let f share secret s and g share secret s', both degree $t-1$ with the same evaluation points. Show that the pointwise sums $f(i) + g(i)$ are valid shares of $s + s'$ under a degree-$(t-1)$ polynomial. Explain how distributed key generation uses exactly this fact so that no single dealer knows the final key.

Exercise 32.11. *(Weighted threshold design.)* You must build a policy where the CEO alone plus any one of four VPs can recover a key, OR any three VPs together can recover it (without the CEO). Using Shamir with a suitable threshold and by handing some parties multiple share-indices, design a concrete scheme: state the threshold t and how many share-indices each role holds, and verify both authorized patterns reach t while a single VP alone does not.

Chapter 33

Homomorphic Encryption

In the previous chapter, we saw how secret sharing and Pedersen commitments allow parties to add distributed shares and private balances without opening them. Everything else we have built so far treats a ciphertext as opaque. You encrypt, you ship the bytes somewhere, and the only useful thing anyone without the key can do with them is store them and hand them back unchanged. Decrypt, compute, re-encrypt: that is the standard pattern, and it has a glaring weakness. The moment you want to *do* something with the data — add two salaries, search a database, run a spam filter, train a model — you have to decrypt first. The data sits in the clear on whatever machine does the work, and that machine, and its operator, and anyone who breaks into it, sees your secrets.

Homomorphic encryption breaks that pattern. It lets a party who does *not* hold the key take ciphertexts and combine them so that the result, when finally decrypted by the key holder, equals the answer to a computation on the original plaintexts. The cloud computes on data it cannot read. This chapter examines homomorphic encryption across two classes of constructions: partially homomorphic schemes (such as multiplication in RSA and ElGamal, or addition in Paillier), followed by fully homomorphic encryption, Gentry's bootstrapping technique, its computational costs, and practical applications.

33.1 Computing on ciphertext: the threat model

Picture the classic client-server dilemma. Alice holds sensitive personal data—medical records, a private financial audit, or genomic sequences. She wants Bob, who operates a distributed compute cluster, to run an intensive algorithm f on her data: a diagnostic risk score, an optimized portfolio model, or a gene sequence alignment. Alice cannot hand Bob her plaintext without forfeiting all confidentiality. The standard commercial answer is "encrypt in transit with TLS and trust the remote operator," but the moment the packet reaches Bob's server, the data is decrypted into memory where any insider, hypervisor compromise, or operating system exploit can inspect it.

Focus

The goal of homomorphic encryption: Alice encrypts her private inputs, transmitting only ciphertexts to Bob. Bob computes on those ciphertexts directly *without ever decrypting*, producing a ciphertext of the final result. Bob hands this ciphertext back to Alice, who

decrypts it with her private key to obtain $f(\text{data})$. Bob learns nothing about the inputs or the computed output.

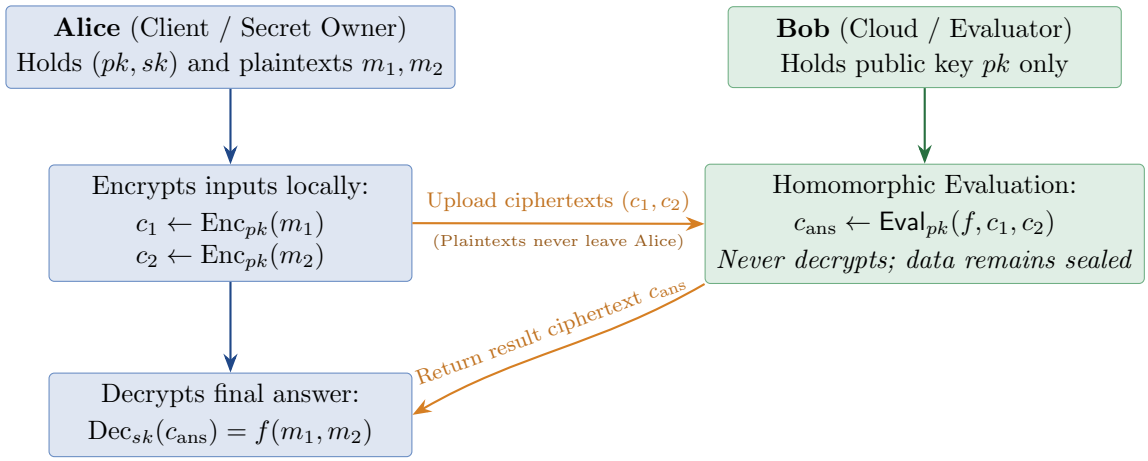


Figure 33.1: The homomorphic computing paradigm. Alice seals her inputs under her public key pk. Bob manipulates the sealed ciphertexts under an evaluation map Eval, transforming the underlying messages without ever accessing the decryption key sk. Alice recovers the computed output using sk.

Make the mathematical contract precise. A public-key encryption scheme $(\text{Gen}, \text{Enc}, \text{Dec})$ is *homomorphic* for a binary operation $\circ$ on the plaintext space $\mathcal{M}$ if there exists an efficient algorithm $\star$ operating on ciphertexts such that for all messages $m_1, m_2 \in \mathcal{M}$:

$$\text{Dec}_{sk}\big(\text{Enc}_{pk}(m_1) \star \text{Enc}_{pk}(m_2)\big) \;=\; m_1 \circ m_2.$$

The evaluator executing $\star$ needs only the public key pk, never the private key sk. If $\circ$ corresponds to multiplication, the scheme is *multiplicatively homomorphic*; if $\circ$ is addition, it is *additively homomorphic*. When a single scheme simultaneously supports both addition and multiplication over a field such as $\mathbb{F}_2$—sufficient to build any Boolean logic gate (AND, XOR) and therefore any computable function—it is *fully homomorphic*.

Intuition

Homomorphic encryption operates analogously to a sealed glovebox used to handle hazardous materials: an operator manipulates objects inside through protective barriers without opening the chamber. In homomorphic encryption, an evaluator performs operations on encrypted ciphertexts, manipulating the underlying plaintexts without possessing the decryption key.

What is the threat model? In the baseline we assume Bob is curious but procedure-following: he runs the agreed computation correctly but will happily read anything he can. Security means semantic security / IND-CPA, exactly as in Chapter 19 and Chapter 20 — ciphertexts must leak nothing about plaintexts to a PPT adversary, even one who can request encryptions of chosen messages. Note one immediate consequence: **a homomorphic scheme can never be IND-CCA secure**, because the homomorphic operation *is* a way to maul ciphertexts. If you can turn $\text{Enc}(m)$ into $\text{Enc}(2m)$, you have a decryption-oracle lever. Malleability is a feature here, not a bug, and it costs us the strongest security notion. We will return to this tension.

Connection

Everything in this chapter is built from the number theory of Chapters 3–6 and the public-key hardness assumptions of Chapters 18–21: factoring, discrete log, and — for the fully homomorphic schemes — lattice problems that also power the post-quantum cryptography of Chapter 35. The noise budget calculations later in this chapter rely on probability bounds; the companion probability book's tail-bound chapter is the right place to sharpen the intuition for why noise growth is the central enemy.

33.2 Partially homomorphic: schemes you already know

You have already built two homomorphic schemes without anyone calling them that. Let us name the structure.

33.2.1 Textbook RSA multiplies

Recall plain RSA from Chapter 19: public key (n, e), and $\mathrm{Enc}(m) = m^e \bmod n$. Multiply two ciphertexts:

$$\mathrm{Enc}(m_1) \cdot \mathrm{Enc}(m_2) \;=\; m_1^e\, m_2^e \;=\; (m_1 m_2)^e \;=\; \mathrm{Enc}(m_1 m_2) \pmod{n}.$$

The product of the ciphertexts is a valid ciphertext of the product of the plaintexts. RSA is multiplicatively homomorphic, for free, because exponentiation distributes over multiplication.

Example 33.1 (RSA homomorphism by hand)**.** Take the small RSA key $p = 11$, $q = 13$, so $n = 143$ and $\varphi(n) = 120$. Pick $e = 7$ (coprime to 120). The inverse $d = 7^{-1} \bmod 120 = 103$, since $7 \cdot 103 = 721 = 6 \cdot 120 + 1$.

Encrypt $m_1 = 5$: $5^2 = 25$, $5^4 = 625 \equiv 53$, $5^6 = 5^4 \cdot 5^2 \equiv 53 \cdot 25 = 1325 \equiv 38$, so $5^7 \equiv 38 \cdot 5 = 190 \equiv 47 \pmod{143}$. Thus $\mathrm{Enc}(5) = 47$.

Encrypt $m_2 = 4$: $4^2 = 16$, $4^4 = 256 \equiv 113$, $4^6 = 113 \cdot 16 = 1808 \equiv 92$, so $4^7 \equiv 92 \cdot 4 = 368 \equiv 82 \pmod{143}$. Thus $\mathrm{Enc}(4) = 82$.

Multiply the ciphertexts: $47 \cdot 82 = 3854 = 26 \cdot 143 + 136 \equiv 136 \pmod{143}$.

By the homomorphism this must equal $\mathrm{Enc}(20)$. Check directly: $20^2 = 400 \equiv 114$, $20^4 \equiv 114^2 = 12996 \equiv 126$, $20^6 = 126 \cdot 114 = 14364 \equiv 64$, $20^7 \equiv 64 \cdot 20 = 1280 \equiv 136 \pmod{143}$. The two agree: $\mathrm{Enc}(5) \cdot \mathrm{Enc}(4) \equiv \mathrm{Enc}(20) \pmod{143}$.

Common trap

Arithmetic mod 143 by hand is error-prone, and the moral is exactly that the *algebraic identity is the ground truth*, while your hand computation is the thing to reconcile to it. If a product of ciphertexts does not match the direct encryption of the product, recheck the small powers, not the theorem. The structural fact — $(m_1 m_2)^e = m_1^e m_2^e$ — cannot fail.

Textbook RSA is multiplicatively homomorphic, full stop. But it is also deterministic, hence *not* IND-CPA secure (equal plaintexts give equal ciphertexts), which is why nobody deploys textbook

RSA — and the homomorphism is precisely why padded RSA (OAEP) deliberately destroys it, scrambling m before exponentiation so that no algebraic relation between plaintexts survives in the ciphertexts.

33.2.2 ElGamal multiplies, with randomness

ElGamal (Chapter 20) fixes RSA's determinism and stays multiplicative. In a group $\mathbb{G}$ of prime order q with generator g, secret key x, public key $h = g^x$, encryption of $m \in \mathbb{G}$ picks fresh randomness r and outputs the pair

$$\mathrm{Enc}(m) = (g^r,\ m \cdot h^r).$$

Multiply two ciphertexts componentwise:

$$(g^{r_1}, m_1 h^{r_1}) \star (g^{r_2}, m_2 h^{r_2}) = (g^{r_1+r_2},\ m_1 m_2\, h^{r_1+r_2}) = \mathrm{Enc}(m_1 m_2;\, r_1 + r_2).$$

A clean, randomized, multiplicatively homomorphic, IND-CPA-secure scheme (under the Decisional Diffie-Hellman assumption). This is genuinely useful: it underlies the homomorphic tallying in many cryptographic voting systems, where you want to combine encrypted votes without decrypting individual ballots.

Common trap

ElGamal multiplies plaintexts, but for voting you want to *add* vote counts, not multiply them. The standard fix is *exponential ElGamal*: encode the message in the exponent, encrypting g^m instead of m. Then $\mathrm{Enc}(g^{m_1}) \star \mathrm{Enc}(g^{m_2}) = \mathrm{Enc}(g^{m_1+m_2})$, turning multiplication of ciphertexts into addition of plaintexts. The catch: to recover $m_1 + m_2$ you must take a discrete log of $g^{m_1+m_2}$, which is only feasible when the total is small (e.g. vote counts up to a few million). Cute trick, real limitation.

So we have multiplicative schemes in abundance. The harder and historically more useful direction is a clean *additive* scheme with an efficient decryption that does not choke on large sums. That is Paillier.

33.3 Paillier: additive homomorphism done right

Pascal Paillier published his cryptosystem in 1999, building on a line of higher-residue schemes (Goldwasser–Micali, Benaloh, Naccache–Stern) that encrypted a few bits at a time. Paillier's insight was to work modulo n^2 instead of n, opening an entire extra slot in the exponent where plaintexts live additively. It remains the standard additively-homomorphic scheme in deployed systems two decades later — powering private aggregation, federated analytics, and threshold ECDSA wallets — because it is simple, efficient, and IND-CPA secure under the Decisional Composite Residuosity assumption.

33.3.1 The construction

Key generation looks like RSA at first. Pick two large primes p, q, set $n = pq$. Define $\lambda = \mathrm{lcm}(p-1, q-1)$ (Carmichael's function of n, the exponent of the unit group). The plaintext space is $\mathbb{Z}/n\mathbb{Z}$

and ciphertexts live in $(\mathbb{Z}/n^2\mathbb{Z})^*$. Choose $g \in (\mathbb{Z}/n^2\mathbb{Z})^*$; the simple and standard choice is $g = n+1$, which we will use throughout because it makes everything explicit.

Encryption. To encrypt $m \in \mathbb{Z}/n\mathbb{Z}$, pick fresh random $r \xleftarrow{\$} (\mathbb{Z}/n\mathbb{Z})^*$ and output

$$c = g^m \, r^n \bmod n^2.$$

With $g = n+1$, the term $g^m = (1+n)^m$ has a clean closed form. By the binomial theorem modulo n^2, every term with n^2 or higher vanishes, leaving

$$(1+n)^m = 1 + mn + \binom{m}{2} n^2 + \cdots \equiv 1 + mn \pmod{n^2}.$$

So encryption is $c = (1+mn)\, r^n \bmod n^2$. The plaintext sits in the "mn" digit; the r^n factor is a random n-th residue that masks it (its IND-CPA security rests on the Decisional Composite Residuosity assumption: telling n-th residues mod n^2 from non-residues is hard without the factorization of n).

Decryption. Define the function $L(x) = (x-1)/n$, ordinary integer division (valid whenever $x \equiv 1 \pmod n$). Then

$$m = \frac{L\big(c^\lambda \bmod n^2\big)}{L\big(g^\lambda \bmod n^2\big)} \bmod n.$$

Why it works

Why does decryption recover m? Two facts. First, by Carmichael's theorem, $r^{n\lambda} \equiv 1 \pmod{n^2}$ for any r coprime to n (the group $(\mathbb{Z}/n^2\mathbb{Z})^*$ has exponent $n\lambda$). So raising the ciphertext to λ kills the random factor:

$$c^\lambda = (1+n)^{m\lambda} \, r^{n\lambda} \equiv (1+n)^{m\lambda} \equiv 1 + m\lambda n \pmod{n^2},$$

using the same binomial collapse as before. Thus $L(c^\lambda) = m\lambda \bmod n$. Second, $g^\lambda = (1+n)^\lambda \equiv 1 + \lambda n$, so $L(g^\lambda) = \lambda \bmod n$. Dividing, $m\lambda/\lambda \equiv m \pmod n$. The λ cancels, m survives.

33.3.2 Why it is additive

Now the payoff. Multiply two ciphertexts in $(\mathbb{Z}/n^2\mathbb{Z})^*$:

$$c_1 c_2 = (1+n)^{m_1} r_1^n \cdot (1+n)^{m_2} r_2^n = (1+n)^{m_1+m_2} \, (r_1 r_2)^n \pmod{n^2}.$$

Because $(r_1 r_2)$ is uniformly distributed in $(\mathbb{Z}/n\mathbb{Z})^*$ whenever r_1, r_2 are random, the product $c_1 c_2$ is a syntactically identical, perfectly distributed Paillier encryption of $(m_1 + m_2) \bmod n$. **Multiplying ciphertexts adds plaintexts.** Two affine operations immediately follow:

- **Add a public constant k:** multiply c by $g^k = (1+n)^k \pmod{n^2}$, yielding $\mathrm{Enc}_{pk}(m+k)$.
- **Multiply by a public scalar k:** raise the ciphertext to the k-th power:

$$c^k = \big((1+n)^m r^n\big)^k = (1+n)^{km} (r^k)^n \pmod{n^2} = \mathrm{Enc}_{pk}(km).$$

Combining addition and scalar multiplication allows any untrusted server to compute arbitrary linear combinations $\sum a_i m_i + b \pmod{n}$ over encrypted vectors. However, multiplying two encrypted values $\mathrm{Enc}(m_1) \cdot \mathrm{Enc}(m_2)$ would require evaluating $(1+n)^{m_1 m_2}$, which cannot be computed from the modular products alone; this fundamental ceiling separates partially homomorphic schemes from fully homomorphic ones.

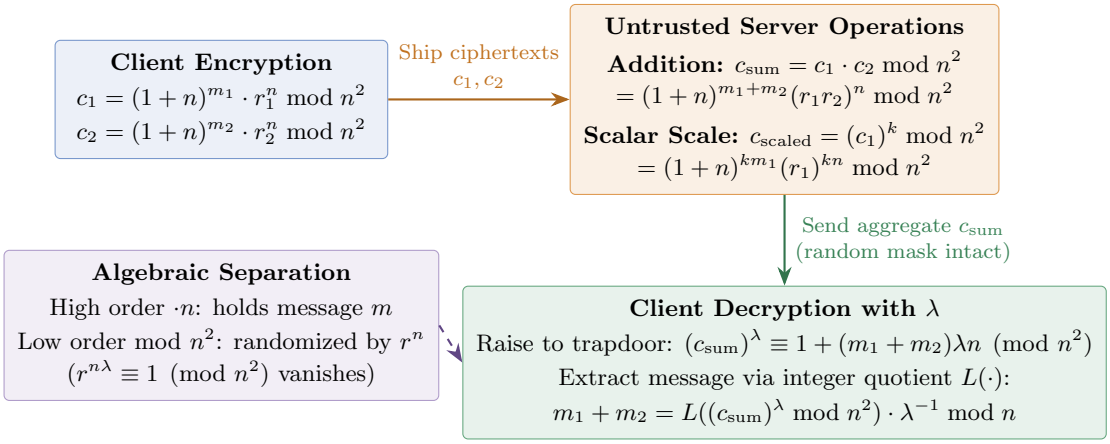


Figure 33.2: Paillier's additive homomorphism pipeline. Ciphertext multiplication in $(\mathbb{Z}/n^2\mathbb{Z})^*$ adds plaintexts in $\mathbb{Z}/n\mathbb{Z}$. Decryption applies the secret Carmichael trapdoor λ, eliminating the blinding factor $r^{n\lambda} \equiv 1 \pmod{n^2}$ and leaving the linear sum intact in the upper quotient slice.

Example 33.2 (Paillier addition by hand)**.** Take $p = 5$, $q = 7$, so $n = 35$ and $n^2 = 1225$. Then $\lambda = \mathrm{lcm}(4, 6) = 12$. Use $g = n + 1 = 36$.

Encrypt $m_1 = 3$ with $r_1 = 4$: $c_1 = (1 + 3 \cdot 35)\, 4^{35} \bmod 1225 = 106 \cdot (4^{35} \bmod 1225) \bmod 1225$. Compute $4^{35} \bmod 1225$ by repeated squaring: $4^2 = 16$, $4^4 = 256$, $4^8 = 65536 \equiv 611$, $4^{16} \equiv 611^2 = 373321 \equiv 921$, $4^{32} \equiv 921^2 = 848241 \equiv 541$. Then $4^{35} = 4^{32} \cdot 4^2 \cdot 4 \equiv 541 \cdot 16 \cdot 4 = 541 \cdot 64 = 34624 \equiv 34624 - 28 \cdot 1225 = 324$. So $c_1 = 106 \cdot 324 = 34344 \equiv 34344 - 28 \cdot 1225 = 44$.

Encrypt $m_2 = 6$ with $r_2 = 2$: $c_2 = (1 + 6 \cdot 35)\, 2^{35} \bmod 1225 = 211 \cdot (2^{35} \bmod 1225)$. Note $2^{35} = 4^{17} \cdot 2$; we have $4^{16} \equiv 921$, so $4^{17} = 921 \cdot 4 = 3684 \equiv 9$, and $2^{35} = 9 \cdot 2 = 18$. So $c_2 = 211 \cdot 18 = 3798 \equiv 3798 - 3 \cdot 1225 = 123$.

Homomorphic add: $c_1 c_2 = 44 \cdot 123 = 5412 \equiv 5412 - 4 \cdot 1225 = 512 \pmod{1225}$. This should decrypt to $m_1 + m_2 = 9$.

Decrypt $c = 512$. We need $L(c^{12} \bmod 1225)/L(g^{12} \bmod 1225) \bmod 35$. First $g^{12} = 36^{12} = (1 + 35)^{12} \equiv 1 + 12 \cdot 35 = 421 \pmod{1225}$, so $L(g^{12}) = (421 - 1)/35 = 12$. Next $c^{12} \bmod 1225$ should equal $1 + 9 \cdot 12 \cdot 35 \bmod 1225 = 1 + 3780 \bmod 1225$; since $3780 = 3 \cdot 1225 + 105$, we get $c^{12} \equiv 106$, so $L(c^{12}) = (106 - 1)/35 = 3$. Finally $m = 3/12 \bmod 35$. We need $12^{-1} \bmod 35$: $12 \cdot 3 = 36 \equiv 1$, so $12^{-1} = 3$, and $m = 3 \cdot 3 = 9 \bmod 35$. We recover $9 = 3 + 6$. The addition went through entirely under encryption.

Work it yourself

Redo Example 33.2 with $m_1 = 10$, $m_2 = 30$. Note that $m_1 + m_2 = 40 > n = 35$, so the result wraps: you should decrypt to $40 \bmod 35 = 5$. This wraparound is the analogue of integer overflow and is the reason real deployments choose n far larger than any sum that can occur.

Confirm the wrap by hand; it is the single most common "bug" beginners hit with Paillier.

33.3.3 A toy implementation

Implement it

A minimal Paillier in Python, with $g = n + 1$. **Learning only.** Real Paillier needs cryptographically strong primes, constant-time modular arithmetic, rejection of r not coprime to n, and careful encoding of signed and fixed-point numbers. Do not put this anywhere near production.

```python
from math import gcd
from random import randrange

def L(x, n):            # x is congruent to 1 mod n
    return (x - 1) // n

def keygen(p, q):       # pass in your own toy primes
    n = p * q
    lam = (p - 1) * (q - 1) // gcd(p - 1, q - 1)   # lcm
    g = n + 1
    # with g = n+1, L(g^lam mod n^2) = lam, so mu = lam^{-1} mod n
    mu = pow(lam % n, -1, n)
    return (n, g), (lam, mu)

def encrypt(pub, m):
    n, g = pub
    n2 = n * n
    while True:
        r = randrange(1, n)
        if gcd(r, n) == 1:
            break
    return (pow(g, m, n2) * pow(r, n, n2)) % n2

def decrypt(pub, priv, c):
    n, g = pub
    lam, mu = priv
    n2 = n * n
    return (L(pow(c, lam, n2), n) * mu) % n

def add_enc(pub, c1, c2):          # homomorphic addition
    n, _ = pub
    return (c1 * c2) % (n * n)

def scale_enc(pub, c, k):          # multiply plaintext by public k
    n, _ = pub
    return pow(c, k, n * n)

# Demo
pub, priv = keygen(1009, 1013)
ca = encrypt(pub, 150)             # encrypted salary A
```

```
cb = encrypt(pub, 275)              # encrypted salary B
total = add_enc(pub, ca, cb)        # server never sees plaintexts
print(decrypt(pub, priv, total))    # -> 425
print(decrypt(pub, priv, scale_enc(pub, ca, 3)))  # -> 450
```

The server runs `add_enc` and `scale_enc` knowing only `pub`. Decryption stays with whoever holds `priv`.

In the real world

Paillier is in real systems right now. Threshold-ECDSA wallets (used by custodians to split a signing key across servers so no single machine holds it) use Paillier to let parties combine secret shares additively during signing. Private-set and private-sum protocols for advertising attribution — "how many people who saw this ad later bought the product?" — aggregate encrypted counts so neither the advertiser nor the publisher sees individual records. Some smart-meter and federated-analytics pipelines sum encrypted readings across thousands of homes, decrypting only the aggregate. In every case the structure is the same: many parties contribute encrypted addends, a server multiplies the ciphertexts, exactly one party holds the key and sees only the total.

33.4 The leap to fully homomorphic encryption

Paillier adds. RSA and ElGamal multiply. For decades the open question — posed by Rivest, Adleman, and Dertouzos in 1978, the year after RSA — was whether a single scheme could do *both*, addition and multiplication, on encrypted data. That matters because addition and multiplication over $\mathbb{F}_2$ (XOR and AND) are a universal gate set: with both, you can evaluate *any* Boolean circuit, hence any computable function, on ciphertexts. A scheme with both is *fully homomorphic* (FHE), and a scheme limited to circuits of bounded depth is *somewhat homomorphic* (SHE).

For thirty-one years nobody knew if FHE existed. Plausibility arguments cut both ways: the malleability that makes homomorphism possible seemed to fight against the rigidity that makes encryption secure. Then in 2009 Craig Gentry, in his Stanford PhD thesis, gave the first construction. It reshaped the field.

History

The open problem of computing on encrypted data was framed in 1978 by Ronald Rivest, Leonard Adleman, and Michael Dertouzos shortly after RSA appeared. For three decades, cryptographers found only partial homomorphisms (RSA and ElGamal for multiplication, Goldwasser–Micali for XOR, Paillier in 1999 for addition mod n^2). Gentry's 2009 Stanford doctoral thesis resolved the thirty-year quest and is one of those rare results that everyone in the area can date precisely, like Shor's algorithm or the discovery of public-key crypto. The construction used ideal lattices and was, by his own description, "barely" feasible: evaluating a single bit operation could take many minutes, and the key sizes were enormous. What mattered was not the speed but the existence proof plus one idea — bootstrapping — that turned a depth-limited gadget into an unlimited one. Within a few years the ideal-lattice machinery was replaced by cleaner constructions by Zvika Brakerski, Craig Gentry, and Vinod

Vaikuntanathan (BGV/BFV) and Cheon–Kim–Kim–Song (CKKS) based on Learning With Errors (the same lattice assumption behind post-quantum cryptography in Chapter 35), and performance improved by many orders of magnitude. Gentry shared the 2022 ACM Prize in Computing partly for this work. The primary engineering challenge today is hardware acceleration via ASICs to make continuous real-time bootstrapping practical for deep neural-network inference.

33.4.1 Noise: the enemy and the engine

Modern FHE schemes encrypt by hiding the plaintext under a small amount of random *noise*, then relying on a lattice problem to keep the noise indistinguishable from random. The cleanest mental model is a symmetric Learning-With-Errors (LWE) encryption. Fix a modulus q and a secret vector $\mathbf{s} \in \mathbb{Z}_q^k$. To encrypt a bit $m \in \{0, 1\}$, sample a random vector $\mathbf{a} \in \mathbb{Z}_q^k$ and a *small* noise term e, and output

$$(\mathbf{a},\ b) \quad \text{where} \quad b = \langle \mathbf{a}, \mathbf{s} \rangle + e + m \cdot \lfloor q/2 \rfloor \pmod{q}.$$

The message bit is parked near 0 or near $q/2$. Decryption computes $b - \langle \mathbf{a}, \mathbf{s} \rangle = e + m\lfloor q/2 \rfloor$, and rounds: close to 0 means $m = 0$, close to $q/2$ means $m = 1$. As long as $|e| < q/4$, the rounding is correct.

Intuition

Picture the integers mod q arranged on a clock face. The plaintext bit chooses one of two "poles" — twelve o'clock for 0, six o'clock for 1. Encryption nudges the hand a little off the pole by the noise e; decryption asks "which pole is the hand nearer?" A small nudge is harmless. The danger is that arithmetic on ciphertexts makes the nudge grow, and if it ever crosses the quarter-hour line between the poles, decryption flips to the wrong answer. The primary engineering challenge in FHE is managing this noise accumulation.

Now watch what homomorphic operations do to the noise.

- **Addition** of two ciphertexts adds their noise: $e_1 + e_2$. The noise grows, but slowly (roughly $\sqrt{2}\times$ in standard deviation per addition).
- **Multiplication** of two ciphertexts *multiplies* the noise, roughly $e_1 \cdot e_2$ scaled by the message magnitude. This is the main bottleneck: noise grows multiplicatively with each multiplication, so after only a handful of multiplications in series the noise exceeds $q/4$ and decryption fails.

This is why a raw LWE/SHE scheme can evaluate only *bounded-depth* circuits. Each multiplication burns "noise budget." You start with budget $\log(q/4) - \log(\text{initial noise})$ and spend a chunk per multiplicative level. Run out, and the ciphertext becomes garbage.

Figure 33.3 illustrates this geometry. On the clock face mod q, the message bit determines the target pole, and safe decoding requires the noisy value to stay on the correct side of the $q/4$ boundary. Under repeated multiplications, the noise exponent doubles at each level, rapidly breaching the decision boundary.

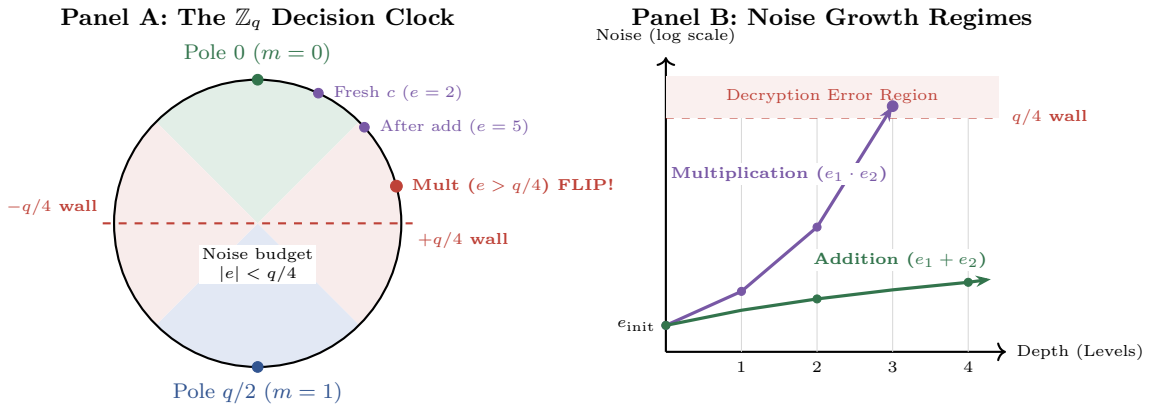


Figure 33.3: Noise dynamics in lattice-based encryption. Panel A shows the cyclic decision geometry: messages are centered at poles 0 and $q/2$; noise e rotates the state away from the pole, causing a decryption error once it crosses $\pm q/4$. Panel B contrasts the gentle additive noise drift with the explosive doubling of multiplicative depth that rapidly exhausts the noise budget.

Example 33.3 (A symmetric LWE encryption by hand)**.** Work in tiny dimensions so the arithmetic stays visible. Fix $q = 64$ (so $\lfloor q/2 \rfloor = 32$ and the decryption threshold is $q/4 = 16$), dimension $k = 3$, and secret key $\mathbf{s} = (2, 5, 1) \in \mathbb{Z}_{64}^3$.

Encrypt $m = 1$**.** Sample $\mathbf{a} = (9, 3, 14)$ and a small noise $e = -2$. Then

$$\langle \mathbf{a}, \mathbf{s} \rangle = 9 \cdot 2 + 3 \cdot 5 + 14 \cdot 1 = 18 + 15 + 14 = 47,$$

and

$$b = 47 + (-2) + 1 \cdot 32 = 77 \equiv 13 \pmod{64}.$$

The ciphertext is $\big((9, 3, 14),\ 13\big)$.

Decrypt. The key holder recomputes $b - \langle \mathbf{a}, \mathbf{s} \rangle = 13 - 47 = -34 \equiv 30 \pmod{64}$. Is 30 nearer 0 or nearer 32? It is within 2 of 32, so round to $32 \Rightarrow m = 1$. Correct, and notice the leftover $30 = 32 - 2$ is exactly $m\lfloor q/2 \rfloor + e$, exposing the noise $e = -2$.

Add two ciphertexts. Encrypt $m' = 0$ as $\mathbf{a}' = (1, 1, 8)$, $e' = 3$: $\langle \mathbf{a}', \mathbf{s} \rangle = 2 + 5 + 8 = 15$, so $b' = 15 + 3 + 0 = 18$. Adding componentwise, $\mathbf{a} + \mathbf{a}' = (10, 4, 22)$ and $b + b' = 13 + 18 = 31$. Decryption gives $31 - \langle (10, 4, 22), \mathbf{s} \rangle = 31 - (20 + 20 + 22) = 31 - 62 = -31 \equiv 33 \pmod{64}$. That rounds to $32 \Rightarrow 1 = 1 + 0$, and the noise is now $e + e' = -2 + 3 = 1$. Addition added the noises, exactly as advertised, and the sum is still comfortably under 16.

Work it yourself

Continue Example 33.3 by encrypting $m = 1$ with a deliberately large noise $e = 15$ (still under $q/4 = 16$, so a single ciphertext decrypts fine). Now add it to itself: the noise becomes $30 > 16$, and decryption flips. Compute the resulting $b - \langle \mathbf{a}, \mathbf{s} \rangle$ and confirm it lands on the wrong side of the 0/32 divide. This is the noise wall in miniature.

Connection

The noise here is the same Learning-With-Errors hardness that anchors lattice-based post-quantum KEMs and signatures in Chapter 35. The security claim is that $(\mathbf{a}, \langle \mathbf{a}, \mathbf{s} \rangle + e)$ is computationally indistinguishable from uniform random — finding $\mathbf{s}$ amounts to solving a noisy linear system, which reduces to worst-case lattice problems believed hard even for quantum computers. FHE and lattice-based post-quantum encryption thus rest on the same underlying hardness assumption.

33.4.2 Bootstrapping: refreshing noisy ciphertexts

Each multiplication increases the noise until decryption fails, limiting somewhat homomorphic schemes to bounded-depth computations. Evaluating arbitrary circuits requires resetting accumulated noise without revealing the secret key to the evaluator.

Gentry's answer is a piece of recursion that feels like a trick the first time, and like an inevitability the second. The noise is small enough that only the key holder can read past it, but the operation that removes noise *is* decryption. So: run decryption *homomorphically*.

Why it works

Suppose Alice supplies Bob with an encrypted copy of her secret key under her public key: $W = \mathrm{Enc}_{pk}(sk)$. Bob holds a noisy ciphertext c whose accumulated noise approaches the $q/4$ threshold. Bob cannot decrypt c directly without learning the plaintext and secret key. Instead, Bob treats the decryption algorithm $\mathrm{Dec}(\cdot, c)$ as a public Boolean circuit $C_c(x)$ whose inputs are the key bits $x = sk$, with the bits of c hard-coded as constants. Bob evaluates C_c homomorphically on the encrypted inputs $W = \mathrm{Enc}_{pk}(sk)$.
The output of evaluating $C_c(W)$ is an encryption of the circuit's output:

$$\mathrm{Enc}_{pk}\big(C_c(sk)\big) = \mathrm{Enc}_{pk}\big(\mathrm{Dec}_{sk}(c)\big) = \mathrm{Enc}_{pk}(m).$$

The essential property is that the noise in this new ciphertext depends solely on the depth and structure of the decryption circuit C_c itself, completely independent of how noisy c was before. As long as the noise added by evaluating C_c is smaller than the original budget, the resulting ciphertext c_{fresh} is rejuvenated with a replenished noise budget, ready for another sequence of arithmetic operations.

Intuition

A physical analogy clarifies why this works: imagine a sealed glovebox whose interior air filtration system drains power as you manipulate hazardous materials inside. You cannot open the hatch to replace the filter without releasing toxins into the room. Instead, you slip an unopened, factory-fresh secondary glovebox into the airlock, reach through the gloves, pass the delicate material into the new box, seal it, and discard the spent container. Bootstrapping is that nested-glovebox transfer: you re-encrypt the message inside the encryption itself, using an encrypted key, so the message hops into a pristine container without ever touching the open air.

For this bootstrap mechanism to succeed, the underlying encryption scheme must be capable of

evaluating its own decryption circuit plus at least one additional multiplication gate before its noise budget is exhausted. A scheme satisfying this condition is called *bootstrappable*. Gentry demonstrated that somewhat homomorphic schemes can be transformed so their decryption circuits are sufficiently shallow to evaluate homomorphically, proving that a bootstrappable SHE scheme can be extended into a fully homomorphic encryption scheme capable of unbounded computation.

33.4.3 Message flow and the cost of depth

Figure 33.4 breaks down the architecture: Panel A visualizes the inner homomorphic self-decryption circuit, while Panel B maps the end-to-end operational loop between client and cloud.

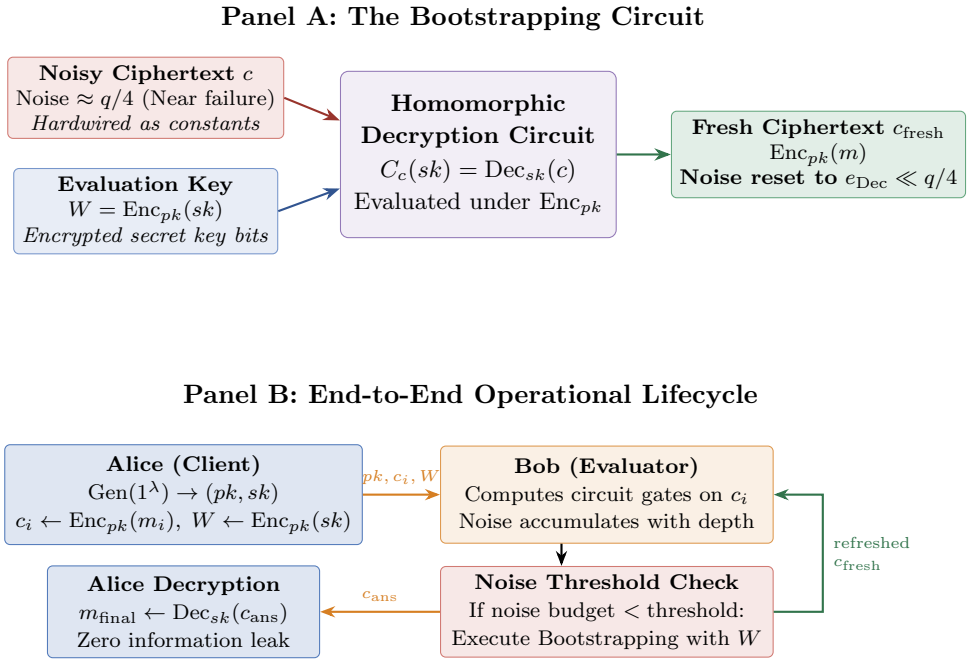


Figure 33.4: Gentry's bootstrapping architecture. Panel A shows how homomorphic evaluation of the decryption circuit consumes encrypted secret key bits W and noisy ciphertext c to output a fresh encryption of m with reset noise. Panel B traces the protocol lifecycle between client and cloud.

Example 33.4 (How many multiplications fit?)**.** Toy numbers. Take modulus $q = 2^{40}$ and a fresh ciphertext with noise bounded by $B = 2^8$. Decryption succeeds while noise $< q/4 = 2^{38}$. Suppose each multiplication squares the noise bound (the worst-case model): after one multiply, 2^{16}; after two, 2^{32}; after three, 2^{64} — already past 2^{38}. So this parameter set supports a multiplicative depth of *two* before failure. To go deeper you either raise q (bigger ciphertexts, slower arithmetic) or bootstrap to reset the noise to $\approx 2^8$ whenever it climbs toward 2^{38}. Real schemes use far larger q (hundreds to thousands of bits) and cleverer multiplication that grows noise additively in the exponent rather than by squaring, but the squaring model captures why *depth*, not total gate count, is the thing FHE charges you for.

Common trap

The cost metric people misjudge: it is the *multiplicative depth* of your circuit, not its size. A million additions are nearly free; a chain of twenty dependent multiplications is brutal. When you design an algorithm for FHE you restructure it to minimize multiplicative depth — replacing deep products with parallel ones, swapping comparisons for low-degree polynomial approximations, batching many plaintexts into one ciphertext ("SIMD slots" via the Chinese Remainder Theorem) so one homomorphic multiply does thousands of useful ones at once. Naively porting code to FHE and expecting it to be merely slow, rather than astronomically slow, is the classic beginner mistake.

33.4.4 What a real homomorphic multiply actually does

The squaring model of Example 33.4 hides two mechanical costs that dominate real FHE engineering, and you should know their names so the libraries' documentation reads as English.

When you multiply two LWE-style ciphertexts, the natural product is a ciphertext that decrypts under $\mathbf{s} \otimes \mathbf{s}$ — a *quadratic* key, not the original linear one. Left alone, every multiply doubles the ciphertext dimension, and after a few products the object is unusable. *Relinearization* (also called key-switching) fixes this: using a published *evaluation key* — encryptions of the products $s_i s_j$ under the original key — the evaluator rewrites the quadratic ciphertext back into an ordinary linear one of the original size. It is the homomorphic analogue of "simplify the expression back to standard form" after multiplying out.

The second cost is *modulus switching* (in the BGV/CKKS line): after a multiply, you rescale the ciphertext from modulus q down to a smaller q', which divides the noise by roughly q/q' at the same time. You pre-load a ladder of moduli $q_0 > q_1 > \cdots$, and each multiplicative level steps down one rung. The number of rungs is the leveled budget; when you run out of rungs you must bootstrap or stop. This is why CKKS parameters are quoted as a number of "levels" — each is one rung, one available multiply.

Why it works

Relinearization and modulus switching are why the "additive in the exponent" remark in Example 33.4 is true. With modulus switching, a multiply does not square the absolute noise; it adds a roughly fixed quantum to the noise *relative to the shrinking modulus*. So the usable depth is set by how many times you can divide q before it gets too small to hold a plaintext — a count of rungs, i.e. a sum, not a product. That linear-in-depth budget is what makes leveled FHE practical for fixed-depth circuits without ever bootstrapping.

Figure 33.5 traces a single homomorphic multiplication gate through these three stages: raw tensor multiplication, key-switching relinearization back to linear dimension, and modulus rescaling down one rung of the ladder.

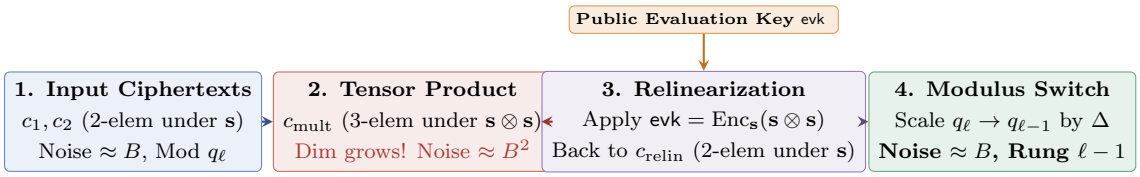


Figure 33.5: Anatomy of a real leveled FHE multiplication gate (BGV/CKKS). Multiplying two linear ciphertexts produces a degree-2 ciphertext under $\mathbf{s} \otimes \mathbf{s}$ with squared noise. Relinearization uses the public evaluation key evk to compress the ciphertext back to two components under $\mathbf{s}$, and modulus switching rescales from q_ℓ down to $q_{\ell-1}$, dividing the noise back to baseline at the cost of one modulus rung.

Common trap

Encoding bites before noise does. FHE plaintexts live in $\mathbb{Z}/t\mathbb{Z}$ (BGV/BFV) or are approximate reals (CKKS), and you must *encode* signed numbers, fixed-point fractions, and vectors into that space yourself. Forget that subtraction is addition of the negation mod t, pick a plaintext modulus t too small for your intermediate values, or mismatch the CKKS scaling factor across two operands, and you get silently wrong answers that look like noise failures but are really arithmetic-encoding bugs. Always test the plaintext arithmetic in the clear at the same modulus and scale *before* blaming the noise budget.

33.5 What it costs and where it is heading

Let us be plain about performance, because hype outran reality for years. A single homomorphic multiplication on modern LWE-based schemes (BGV, BFV, CKKS for approximate/real-number arithmetic, TFHE for fast Boolean gates) costs microseconds to milliseconds depending on the scheme and parameters. Bootstrapping is the expensive step: in TFHE it is roughly milliseconds per gate; in the BGV/CKKS family, refreshing a packed ciphertext can take seconds. Compared with native arithmetic, expect a slowdown of 10^3 to 10^6, shrinking every year. Ciphertext expansion is large too — a single bit can become kilobytes of ciphertext — which dominates bandwidth and storage.

Focus

A quick taxonomy of the deployed schemes, so the acronyms stop being noise:

- **BGV / BFV** — exact integer arithmetic mod a plaintext prime, with CRT batching for SIMD parallelism. Good for exact computations over moderate-size integers.
- **CKKS** — *approximate* arithmetic on real and complex numbers; decryption returns the answer plus a tiny rounding error. The scheme of choice for encrypted machine learning, where approximate is fine.
- **TFHE** — very fast bootstrapping per Boolean gate, so it shines on circuits with comparisons, branches, and arbitrary logic rather than bulk arithmetic.

A compact way to hold the families in mind: match the scheme to the *shape* of the computation, not the other way around.

Scheme	Plaintext	Strength	Typical use
BGV / BFV	exact integers mod t	SIMD batching, exact	encrypted statistics, counting
CKKS	approximate reals/complex	vectorized real math	encrypted ML inference
TFHE	bits, fast bootstrap	cheap arbitrary logic	comparisons, branching, decision trees

Example 33.5 (The bootstrap break-even)**.** Why "leveled when you can, bootstrap when you must" is not a slogan but arithmetic. Suppose your CKKS parameters give a leveled budget of $L = 12$ multiplicative levels per fresh ciphertext, and a single bootstrap costs the equivalent of $T = 15$ homomorphic multiplies. A circuit of multiplicative depth d:

- If $d \leq 12$, run it leveled: cost $\approx d$ multiplies, *zero* bootstraps. A depth-10 circuit costs about 10 multiplies.
- If $d = 30$, you cannot fit it in one budget. You bootstrap whenever the budget is spent: after each block of $\approx L = 12$ levels, refresh. Two bootstraps cover depth 30 (levels 1–12, refresh, 13–24, refresh, 25–30), so cost $\approx 30 + 2 \cdot 15 = 60$ multiply-equivalents — the bootstraps double the bill.

The takeaway: every level you shave off the circuit either avoids a multiply or, near a budget boundary, avoids an entire bootstrap. That is why depth-optimization (Section 33.4.3's trap) pays off super-linearly, and why fixing the depth in advance to stay leveled is the single biggest performance lever available.

In the real world

Where FHE actually shows up today:

- **Encrypted machine-learning inference.** A client encrypts an image or a medical record; the server runs a neural network homomorphically (using CKKS, with polynomial approximations of the activation functions) and returns an encrypted prediction the client alone decrypts. The server never sees the input or output. Microsoft SEAL, OpenFHE, and Zama's Concrete are the open libraries that make this buildable.
- **Private database queries (PIR).** Retrieve record i from a server without the server learning i, by encoding the query as encrypted selection coefficients and homomorphically summing.
- **Encrypted analytics and regulated data.** Hospitals and banks compute statistics across encrypted datasets that law forbids them to pool in the clear.
- **Blockchain and confidential smart contracts.** "Encrypted state machines" that execute contract logic over encrypted inputs, an active area for confidential DeFi.

The practical pattern is to use FHE for the sensitive inner kernel and ordinary crypto for the rest, and to lean on *leveled* FHE (enough noise budget for a known-depth circuit, *no* bootstrapping) whenever the computation's depth is fixed in advance — that avoids the single most expensive operation entirely.

Attack

FHE's IND-CPA security does not protect against a *cheating* server or against revealing decryptions, and both gaps bite in practice.
Malleability / no integrity. Because homomorphism is malleability, an FHE ciphertext carries

no guarantee the server computed the agreed function. A cheating server can return an encryption of *anything* it can compute, including a function chosen to leak the key. Defenses pair FHE with verifiable computation or zero-knowledge proofs (Chapter 31) so the client can check the result was computed correctly.

Reaction / decryption-leakage attacks on approximate schemes. CKKS returns an *approximate* plaintext, and the rounding error depends on the ciphertext's noise, which depends on the secret. Li and Micciancio (2021) showed that if an attacker sees the decrypted approximate results of chosen computations, the small errors leak enough to recover the secret key — a passive attack that broke deployed CKKS configurations until libraries added noise-flooding countermeasures. The lesson: "IND-CPA" assumes the adversary never sees decryptions; the moment a protocol reveals even approximate plaintexts, you have stepped outside the model and need extra defenses.

Focus

The frontier, briefly. Hardware acceleration (FPGA and ASIC bootstrapping, GPU CKKS) is closing the speed gap fast; several efforts target sub-millisecond bootstrapping in silicon. Scheme switching — moving a ciphertext between CKKS, BFV, and TFHE to use each where it is strongest — lets one pipeline do bulk arithmetic and fast logic. And FHE increasingly composes with the rest of the toolbox: with MPC (Chapter 34) to split trust across servers, with zero-knowledge proofs for integrity, with post-quantum KEMs for transport. The realistic near-term picture is not "encrypt the whole cloud" but targeted private kernels — inference on a model, a join over two encrypted tables, a private aggregate — where the privacy gain justifies the cost.

33.6 Putting the pieces together

The arc of this chapter is a ladder of capability. At the bottom, textbook RSA and ElGamal give you multiplication of encrypted values almost for free, because exponentiation respects multiplication — useful but limited, and in RSA's case not even semantically secure. Paillier climbs a rung: a clean, randomized, IND-CPA-secure scheme that *adds* encrypted values and scales them by public constants, which is exactly what private aggregation needs and why it is in production systems today. Each of these is *partially* homomorphic — one operation only.

The top rung, fully homomorphic encryption, supports both addition and multiplication, hence arbitrary circuits. Its engine is noise: a small random perturbation that hides the plaintext under a lattice assumption, grows slowly under addition and explosively under multiplication, and would cap the computation at a shallow depth — except for bootstrapping, Gentry's homomorphic self-decryption, which refreshes a noisy ciphertext into a clean one using an encrypted copy of the key. That single recursive idea is what turns a depth-limited gadget into unlimited computation on encrypted data.

What you should carry forward: the homomorphic property is the *same thing* as malleability, so these schemes trade away IND-CCA security and integrity by design — which is why real deployments bolt on verifiable computation or zero-knowledge proofs, lean on leveled (bootstrap-free) FHE when depth is known, and minimize multiplicative depth above all else. Homomorphic encryption is no longer a curiosity. It is a maturing, deployable tool for the precise problem that ordinary encryption

cannot touch: computing on data you are not allowed to see.

Homomorphic encryption lets a single untrusted evaluator compute over encrypted inputs from a single client. But what if multiple mutually distrustful parties each hold private inputs and wish to jointly evaluate a function without revealing their inputs to each other or relying on a centralized server? In the next chapter, we synthesize secret sharing, garbled circuits, and homomorphic primitives to construct secure multiparty computation.

33.7 Exercises

Exercise 33.1 (RSA homomorphism)**.** Using the toy RSA key $n = 143$, $e = 7$ from Example 33.1, encrypt $m_1 = 6$ and $m_2 = 9$, multiply the ciphertexts mod 143, and verify the product decrypts to 54 mod 143. Show every modular reduction, and recall that the algebraic identity is the ground truth against which to check your arithmetic.

Exercise 33.2 (Exponential ElGamal)**.** In a group of prime order q with generator g, exponential ElGamal encrypts m as $\mathrm{Enc}(m) = (g^r, g^m h^r)$. Show that componentwise multiplication of two ciphertexts yields an encryption of $m_1 + m_2$. Explain why recovering $m_1 + m_2$ requires a discrete-log computation, and why this is fine for tallying small vote counts but not for general arithmetic.

Exercise 33.3 (Paillier by hand)**.** Using $p = 5$, $q = 7$, $n = 35$, $g = 36$ from Example 33.2, encrypt $m_1 = 2$ (with $r_1 = 3$) and $m_2 = 8$ (with $r_2 = 6$). Homomorphically add the ciphertexts and decrypt, confirming you get 10 mod 35. Then add the public constant 20 to the encrypted result (without re-encrypting the inputs) and confirm you get 30.

Exercise 33.4 (Paillier wraparound)**.** Complete the lab following Example 33.2: with the same key, encrypt $m_1 = 10$ and $m_2 = 30$, add homomorphically, and show the decryption yields 5, not 40. Explain in one sentence why a real deployment chooses n much larger than any possible sum.

Exercise 33.5 (Scalar multiply)**.** Prove that for Paillier, $c^k \bmod n^2$ decrypts to $km \bmod n$ when c encrypts m and k is a public integer. Then show how to compute the encrypted weighted sum $a_1 m_1 + a_2 m_2 + b$ from $\mathrm{Enc}(m_1)$, $\mathrm{Enc}(m_2)$ and public constants a_1, a_2, b, using only ciphertext multiplication and exponentiation.

Exercise 33.6 (Why no IND-CCA)**.** Argue that no additively or multiplicatively homomorphic scheme can be IND-CCA secure. Concretely, given a challenge ciphertext $c^* = \mathrm{Enc}(m_b)$ in the IND-CCA game, describe how to maul it into a *different* ciphertext whose decryption (obtained from the decryption oracle) reveals m_b. Do this for both Paillier (additive) and ElGamal (multiplicative).

Exercise 33.7 (LWE decryption correctness)**.** For the symmetric LWE encryption of Section 33.4.1, with ciphertext $(\mathbf{a}, b)$ and $b = \langle \mathbf{a}, \mathbf{s} \rangle + e + m\lfloor q/2 \rfloor$, show that decryption recovers m correctly precisely when $|e| < q/4$. What goes wrong as $|e|$ approaches $q/2$?

Exercise 33.8 (Noise budget)**.** Repeat Example 33.4 with $q = 2^{60}$ and fresh noise bound $B = 2^{10}$, where each multiplication squares the noise bound and decryption needs noise $< q/4$. How many sequential multiplications can you perform before a bootstrap is required? How does the answer change if additions (which only add noise bounds) are interleaved?

Exercise 33.9 (Bootstrapping in words)**.** Explain bootstrapping to a smart friend who knows what encryption and decryption are but has never seen FHE, in under 150 words, without using the word "lattice." Your explanation must make clear (a) what problem bootstrapping solves, (b) what gadget the evaluator is given, and (c) why the evaluator never learns the secret key.

Exercise 33.10 (Circular security)**.** Bootstrapping requires publishing $\mathrm{Enc}_{pk}(sk)$. Explain why IND-CPA security does not automatically guarantee this is safe, give the name of the extra assumption required, and describe a hypothetical (contrived) scheme that is IND-CPA secure but becomes completely broken once an encryption of its own secret key is released.

Exercise 33.11 (Design for depth)**.** You must compute, under FHE, the dot product of two length-1000 encrypted vectors followed by a single sigmoid-like nonlinearity. Describe the multiplicative depth of a naive implementation versus a depth-optimized one (hint: the products in a dot product are independent; the sum is additive; approximate the nonlinearity with a low-degree polynomial). Which scheme from the taxonomy in Section 33.5 would you choose, and why?

Exercise 33.12 (LWE by hand)**.** Using the parameters of Example 33.3 ($q = 64$, $\mathbf{s} = (2, 5, 1)$), encrypt $m = 0$ with $\mathbf{a} = (7, 2, 11)$ and noise $e = 4$, and encrypt $m = 1$ with $\mathbf{a}' = (3, 6, 0)$ and noise $e' = -3$. Add the two ciphertexts, decrypt the sum, and confirm both the recovered bit and the resulting noise $e + e'$. Then state the largest $|e + e'|$ that would still decrypt correctly.

Exercise 33.13 (Relinearization, in words)**.** After multiplying two LWE-style ciphertexts, the product decrypts under a *quadratic* key $\mathbf{s} \otimes \mathbf{s}$ rather than $\mathbf{s}$. Explain in your own words (a) why the dimension grows without relinearization, (b) what the evaluation key contains, and (c) why publishing that evaluation key, like publishing $\mathrm{Enc}_{pk}(sk)$ for bootstrapping, is a key-dependent-message assumption rather than something plain IND-CPA gives you for free.

Exercise 33.14 (Bootstrap break-even)**.** Redo Example 33.5 with a leveled budget of $L = 20$ levels and a bootstrap cost of $T = 40$ multiply-equivalents. For circuit depths $d = 15$, $d = 20$, and $d = 50$, compute the number of bootstraps required and the total multiply-equivalent cost. At what depth does the first bootstrap become unavoidable, and how many multiplies would you need to remove from a depth-21 circuit to make it leveled?

Chapter 34

Secure Multiparty Computation

In the previous chapter, homomorphic encryption allowed an untrusted server to compute on encrypted data without seeing it, but it still relied on a single client who holds the decryption key. Multiparty computation breaks that mold: now the people who want to compute something together *are* the adversaries to one another. They share a goal — the answer to some function of all their inputs — but each of them would dearly like to learn everyone else's secrets along the way, and would happily lie or peek if it helped. Secure multiparty computation protocols achieve this goal: participants evaluate the shared function correctly while revealing nothing about their private inputs beyond what is implied by the output.

That is not a contradiction once you state it carefully. Secure multiparty computation (MPC) lets n parties, holding private inputs $x_1, \ldots, x_n$, jointly evaluate an agreed function $f(x_1, \ldots, x_n)$ so that everyone learns the output and *nothing else*. No trusted referee, no neutral server holding the plaintext — just the participants and the math. This capability powers real systems today: sealed-bid auctions where no auctioneer ever sees the bids, threshold signing where no single machine holds the signing key, and privacy-preserving statistics across hospitals that may not share patient records.

This chapter introduces the core problem using Yao's millionaires' problem, then constructs two primary protocol families: Yao's garbled circuits for two parties, and secret-sharing-based MPC for multiple parties. Oblivious transfer serves as the underlying primitive for both approaches. We keep the numbers tiny so you can trace every wire by hand.

34.1 The dream: a trusted party who forgets

Start with what we wish we had. Suppose there were a perfectly trustworthy outside party — call her the Oracle — with one idealized guarantee: she accepts each participant's secret input over a private channel, computes the function, announces the result to everyone, and then *instantly erases* every input she saw. If such an Oracle existed, privacy would be trivial. Each millionaire would submit their net worth privately to her, and she would announce "Alice is richer" or "Bob is richer" and nothing more.

MPC's goal is to emulate that Oracle using only messages between the parties themselves, with no Oracle in the room. This is called the *ideal/real paradigm*, and it is the gold standard for defining security here.

Intuition

The security criterion is not "the protocol leaks zero information." The output itself carries unavoidable leakage: if a sealed-bid auction announces a winning price of $100, every unsuccessful bidder learns their bid fell below $100. The foundational standard is that *the protocol leaks nothing beyond what an ideal trusted mediator would reveal.* Whatever a participant can deduce after executing the real protocol, they could already have computed from their own private input together with the announced final output alone. That strict containment condition forms the crux of the definition.

Formal

The ideal/real definition (simulation). Fix a function f and an adversary controlling some subset of corrupt parties. We compare two worlds.
In the *ideal world*, the corrupt parties hand their inputs to the Oracle, who computes f and returns the output. The adversary sees only its own inputs and the output.
In the *real world*, the parties run the actual protocol; the adversary sees every message its corrupt parties send and receive.
The protocol is *secure* if for every real-world adversary there is an ideal-world *simulator* that produces an indistinguishable view using only the ideal-world information (corrupt inputs and the output). If a simulator can fake the entire transcript of received messages knowing only the output, then those messages carried no extra information — because they could have been manufactured without ever touching the uncorrupted parties' inputs.

We will not prove full simulation security for every protocol below — the proofs are graduate-course material — but keep the simulator in mind. Every time we claim "Bob learns nothing," the underlying justification is that Bob's view could have been generated from the output alone.

34.1.1 Two adversary models

Two threat models recur throughout MPC, and they demand different machinery.

- *Passive* (also "curious-but-compliant"): corrupt parties follow the protocol exactly but try to learn extra information from what they legitimately see. For example, a participant might run the official software as specified while recording and analyzing the network transcript.
- *Malicious* ("active"): corrupt parties may deviate arbitrarily — send wrong messages, abort, lie about inputs — to break privacy or steer the output.

Passive security is much easier to achieve and is the right first target for understanding. Most of this chapter builds passive-secure protocols, then notes what malicious security costs. Real deployments need malicious security, usually bought with extra checks (commitments, zero-knowledge proofs that you followed the rules, authenticated shares).

Connection

The "follow the protocol but prove you did" idea is exactly where zero-knowledge proofs (Chapter 31) plug into MPC. A generic compiler due to Goldreich, Micali, and Wigderson upgrades any passive-secure protocol to malicious security by having every party prove in zero

knowledge, after each step, that the message they sent is consistent with their committed input and the protocol rules — without revealing the input. Commitments (Chapter 32) are also essential: they pin down inputs so nobody can adaptively change theirs after seeing others' messages.

34.2 The millionaires' problem

Andrew Yao posed the founding question in 1982. Two millionaires, Alice and Bob, want to know who is richer without revealing how much either is worth. Alice has wealth a, Bob has wealth b, both integers in some range $\{1, \dots, N\}$. They want to compute the single bit $[a \geq b]$ and learn nothing else.

History

Andrew Chi-Chih Yao's 1982 paper "Protocols for Secure Computations" introduced the Millionaires' problem and, in 1986 follow-up work, the garbled-circuit technique that solves the general two-party case when paired with 1-out-of-2 oblivious transfer (invented by Michael Rabin in 1981 and generalized by Even, Goldreich, and Lempel in 1985). Yao's framework was extended to the multi-party setting by Goldreich, Micali, and Wigderson (GMW, 1987) and Ben-Or, Goldwasser, and Wigderson (BGW, 1988), and proved its practical viability in 2008 with the Danish sugar-beet auction, the first large-scale commercial MPC deployment. The same Andrew Yao won the Turing Award in 2000, partly for this line of work. The millionaires' problem is the "hello world" of MPC: small enough to solve by hand, yet it already needs the central trick (oblivious transfer) that the general theory rests on. An active frontier today is scaling maliciously secure MPC to thousands of concurrent participants with low round latency while maintaining post-quantum security during preprocessing.

Here is a clean protocol for the comparison, in the spirit of Yao's original, that already shows the flavor. It assumes Bob has a public-key encryption scheme with the right algebraic structure. We restrict to small ranges so you can follow it; let $a, b \in \{1, \dots, 10\}$.

1. Bob picks a random large number x, computes $k = \mathrm{Enc}_{pk}(x)$ using *Alice's* public key, and sends Alice the number $k - b$ (the ciphertext interpreted as an integer, minus b).

2. Alice, who holds the secret key, computes $y_u = \mathrm{Dec}_{sk}\big((k - b) + u\big)$ for every candidate value $u = 1, 2, \dots, 10$. Exactly one of these, y_b, equals x, because $(k - b) + b = k$ decrypts to x. But Alice does not know which u is b.

3. Alice picks a random prime p of suitable size, reduces every y_u modulo p, and constructs a list: for the first a entries ($u = 1, \dots, a$) she sends $y_u \bmod p$ unchanged; for the rest ($u = a + 1, \dots, 10$) she sends $(y_u + 1) \bmod p$. She sends Bob the whole list of 10 numbers, plus p.

4. Bob looks at position b in the list and compares it against $x \bmod p$. If the b-th entry equals $x \bmod p$, then position b fell in the "unchanged" region, meaning $b \leq a$, so $a \geq b$. Otherwise $a < b$.

Walking question

Why does Bob learn only the one bit? Every number Alice sends is either $y_u \bmod p$ or $(y_u + 1) \bmod p$, and the y_u for $u \neq b$ are decryptions of values unrelated to x — effectively random mod p from Bob's view, since he does not hold the secret key. So the list looks like noise to him except at his own position b, where he can recognize $x \bmod p$. He reads off whether his entry was bumped by 1 or not. That single comparison is the bit $[b \leq a]$. He cannot tell a's exact value, only which side of b it lies on.
Why does Alice learn nothing? She never sees b. She decrypts 10 candidates and one happens to be x, but she cannot tell which, because x is random and Bob's offset $k - b$ hides b inside the ciphertext arithmetic.

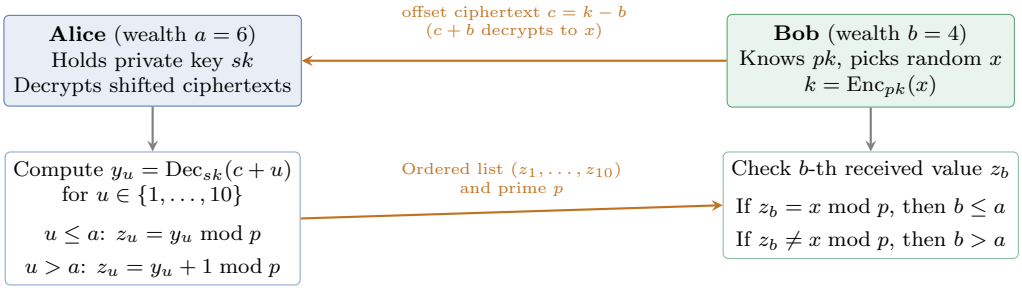


Figure 34.1: A toy protocol for the Millionaires' problem. Bob offsets a random ciphertext k by his wealth b. Alice decrypts all possible offsets u and adds 1 mod p to the results above her wealth a. Bob checks his position b: if the value is uncorrupted, he knows $b \leq a$.

Common trap

This particular protocol is delicate: it relies on the encryption being well-behaved under the integer subtraction $k - b$, which is not true for a generic modern encryption scheme. Yao's original used a number-theoretic scheme where it worked. Do not implement comparisons this way in production; the point here is to *feel* how two mutually distrustful parties can squeeze out one bit. The general, robust solution is garbled circuits, which we build next, and which compute the comparison — or any function — without such scheme-specific tricks.

34.3 Oblivious transfer: the atom of MPC

Before circuits, we need the primitive everything is made of. *Oblivious transfer* (OT) sounds mundane and turns out to be the universal building block: with OT you can build *any* secure two-party computation.

The basic flavor is *1-out-of-2 OT*. The sender holds two messages m_0 and m_1. The receiver holds a choice bit $c \in \{0, 1\}$. After the protocol:

- the receiver learns m_c (the message they chose) and nothing about m_{1-c};
- the sender learns nothing — in particular, not which message the receiver picked.

Intuition

Picture two sealed boxes on a table, m_0 and m_1. The receiver gets to walk away with exactly one box, of their choosing, and cannot peek inside the other. The sender, watching, cannot tell which box was taken. It feels paradoxical: how can the sender hand over one of two secrets without knowing which, while keeping the other hidden? Public-key math makes it possible.

34.3.1 A concrete OT from Diffie-Hellman

Here is a clean OT protocol (a simplified Bellare-Micali / Naor-Pinkas style construction) over a cyclic group $\mathbb{G} = \langle g \rangle$ of prime order q, where the Diffie-Hellman problem is hard. Recall from Chapters 18 and 20 that knowing g^x and g^y does not let you compute g^{xy}.

Setup. Public group $\mathbb{G}$, generator g, and a fixed group element C whose discrete log nobody knows (a "nothing-up-my-sleeve" constant).

Protocol.

1. Receiver (choice bit c) picks random $k \in \{0, \ldots, q-1\}$. He sets $PK_c = g^k$ and $PK_{1-c} = C/g^k = C \cdot g^{-k}$. He sends PK_0 to the sender.
2. The sender computes $PK_1 = C/PK_0$ herself. Now she has both "public keys" PK_0 and PK_1, with the property $PK_0 \cdot PK_1 = C$. She does *not* know which one the receiver can decrypt.
3. The sender picks random r_0, r_1 and encrypts: she sends
$$\big(g^{r_0},\ m_0 \oplus H(PK_0^{r_0})\big) \quad \text{and} \quad \big(g^{r_1},\ m_1 \oplus H(PK_1^{r_1})\big),$$
where H hashes a group element to a bit string.
4. The receiver knows k such that $PK_c = g^k$. For his chosen branch he computes the masking key $H\big((g^{r_c})^k\big) = H(PK_c^{r_c})$ and unmasks m_c.

Why it works

Why the receiver gets exactly one message. The receiver knows the discrete log k of PK_c only. To unmask m_{1-c} he would need $PK_{1-c}^{r_{1-c}} = (g^{r_{1-c}})^{\log PK_{1-c}}$, i.e. he would need the discrete log of $PK_{1-c} = C \cdot g^{-k}$. That requires the discrete log of C, which nobody knows. So the other message stays a sealed box — this is exactly the Diffie-Hellman hardness assumption doing the work.

Why the sender stays oblivious. The only thing the sender sees from the receiver is PK_0. Since $PK_0 = g^k$ for a uniformly random k, it is a uniformly random group element regardless of whether $c = 0$ (so $PK_0 = g^k$) or $c = 1$ (so $PK_0 = C \cdot g^{-k}$, also uniform because g^{-k} is uniform). The sender's view is identical in both cases. She cannot learn c.

Work it yourself

Trace it with tiny numbers. Work in $(\mathbb{Z}/23\mathbb{Z})^*$, restricted to the subgroup of order $q = 11$ generated by $g = 2$ (since $2^{11} \equiv 1 \pmod{23}$). Let the public constant be $C = 2^6 \bmod 23 = 18$ (pretend nobody knows the exponent 6). Suppose the receiver's choice bit is $c = 1$ and he picks $k = 4$.

He sets $PK_1 = g^k = 2^4 = 16$ and $PK_0 = C/g^k = 18 \cdot 16^{-1} \bmod 23$. Now $16^{-1} \bmod 23$: since $16 \cdot 13 = 208 = 9 \cdot 23 + 1$, the inverse is 13. So $PK_0 = 18 \cdot 13 = 234 \equiv 4 \pmod{23}$. He sends $PK_0 = 4$.
The sender computes $PK_1 = C/PK_0 = 18 \cdot 4^{-1}$. Here $4^{-1} \bmod 23 = 6$ (since $4 \cdot 6 = 24 \equiv 1$), so $PK_1 = 18 \cdot 6 = 108 \equiv 16 \pmod{23}$ — matching the receiver's $PK_1 = 16$. Check: $PK_0 \cdot PK_1 = 4 \cdot 16 = 64 \equiv 18 = C$. Good. The sender now encrypts m_0 under PK_0 and m_1 under PK_1 with fresh r_0, r_1; only the m_1 branch can be opened, because the receiver knows $k = 4$ is the log of PK_1 but has no clue about the log of $PK_0 = 4$.

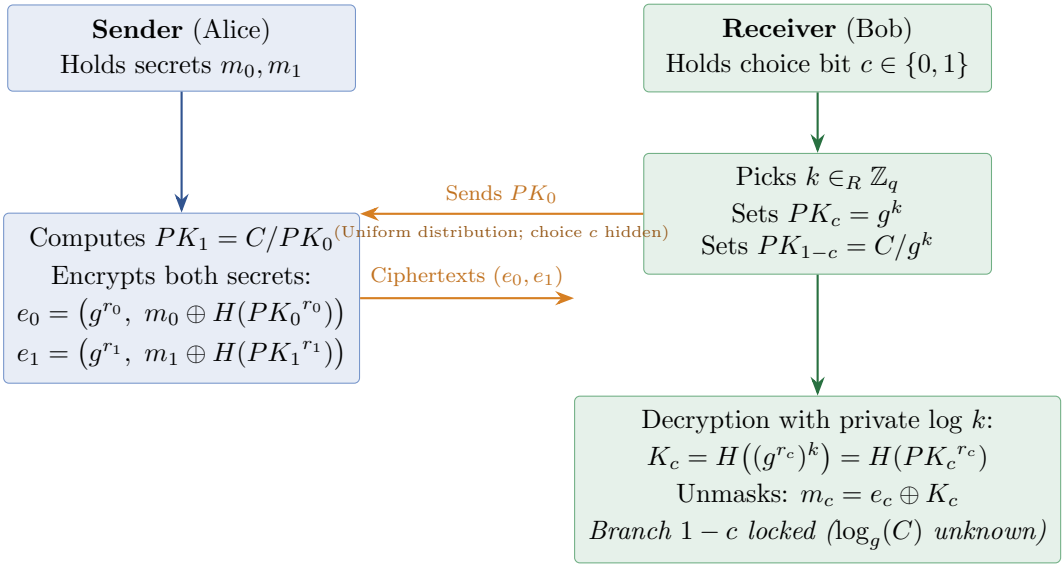


Figure 34.2: The Bellare-Micali 1-out-of-2 Oblivious Transfer protocol. The receiver creates two public keys whose product equals the public constant C. Bob knows the discrete logarithm of only his chosen branch PK_c. Alice encrypts both values; Bob recovers exactly m_c, while m_{1-c} remains protected by the computational Diffie-Hellman assumption.

Focus

OT is to MPC what AND/NOT gates are to circuits: a complete basis. Kilian proved in 1988 that OT alone is enough to build secure computation of *any* function. So once you have a working OT — from Diffie-Hellman, from RSA, from a quantum channel, whatever — you have, in principle, secure computation of everything. The rest of two-party MPC is about doing it efficiently.

34.3.2 OT extension: making it cheap

Public-key operations are expensive, and a circuit may need millions of OTs (one per input wire of one party). The breakthrough that made MPC practical is *OT extension* (Ishai-Kilian-Nissim-Petrank, 2003): perform a small fixed number of "real" public-key OTs (say 128), then use cheap symmetric cryptography (hashing) to bootstrap those into millions of OTs. It is the OT analogue of using public-key crypto only to set up a symmetric session key, then encrypting the bulk traffic with

AES. Without OT extension, garbled-circuit MPC would be a curiosity; with it, you can evaluate circuits with billions of gates.

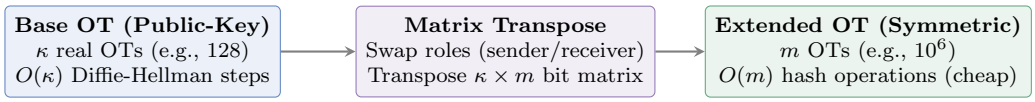


Figure 34.3: Oblivious Transfer Extension (IKNP). By performing κ expensive public-key base OTs and transposing the resulting bit matrix, parties can generate an arbitrary number m of extended OTs using only fast symmetric-key operations (hashing), enabling protocols that consume millions of OTs per second.

34.4 Yao's garbled circuits

Now the centerpiece of two-party MPC. The idea: take the function f and express it as a Boolean circuit of AND, XOR, and NOT gates. One party (the *garbler*, say Alice) "encrypts" the entire circuit — she garbles it — so that the other party (the *evaluator*, Bob) can run it on hidden inputs and read off only the final output, learning nothing about the intermediate wire values or Alice's input.

34.4.1 Garbling one wire, one gate

Every wire in the circuit carries a bit, 0 or 1. In a garbled circuit, Alice replaces each bit value on each wire with a random *label* — a fresh 128-bit key. For wire w she picks two labels: W^0 (meaning "this wire is 0") and W^1 (meaning "this wire is 1"). The labels look random; seeing one tells you nothing about whether it encodes 0 or 1.

Now consider a single AND gate with input wires a, b and output wire z. The truth table is:

a	b	$z = a \wedge b$
0	0	0
0	1	0
1	0	0
1	1	1

Alice has labels A^0, A^1 for wire a; B^0, B^1 for wire b; Z^0, Z^1 for wire z. She builds a *garbled table*: for each of the four input combinations, she encrypts the correct output label under the two corresponding input labels as keys. Using a double-key encryption $\mathrm{Enc}_{K_1,K_2}(\cdot)$ (think: encrypt under K_1, then under K_2):

$$\mathrm{Enc}_{A^0,B^0}(Z^0)$$
$$\mathrm{Enc}_{A^0,B^1}(Z^0)$$
$$\mathrm{Enc}_{A^1,B^0}(Z^0)$$
$$\mathrm{Enc}_{A^1,B^1}(Z^1)$$

Alice randomly *shuffles* these four rows (this is the "garbling" — the row order no longer leaks the input bits) and sends the scrambled table to Bob.

Why it works

Why Bob can evaluate but not snoop. Suppose Bob holds exactly one input label for each input wire — say A^1 (so wire $a = 1$, but Bob does not know that) and B^0. He tries to decrypt each of the four rows with the key pair (A^1, B^0). Three rows were encrypted under different key pairs, so they decrypt to garbage (a well-designed double encryption lets Bob detect this — for example, by a block of zero padding that only appears on correct decryption). Exactly one row, the $\mathrm{Enc}_{A^1,B^0}(Z^0)$ row, decrypts cleanly, and Bob recovers the output label Z^0.

Bob now holds Z^0, the label for "output is 0" — and indeed $1 \wedge 0 = 0$. But Z^0 is just a random 128-bit string to him; he does not know it means 0. He carries it forward as the input label to the next gate. He never learns the actual bit on any internal wire. He only ever holds one label per wire, never both, so he can never decrypt more than one row and never reconstruct the table.

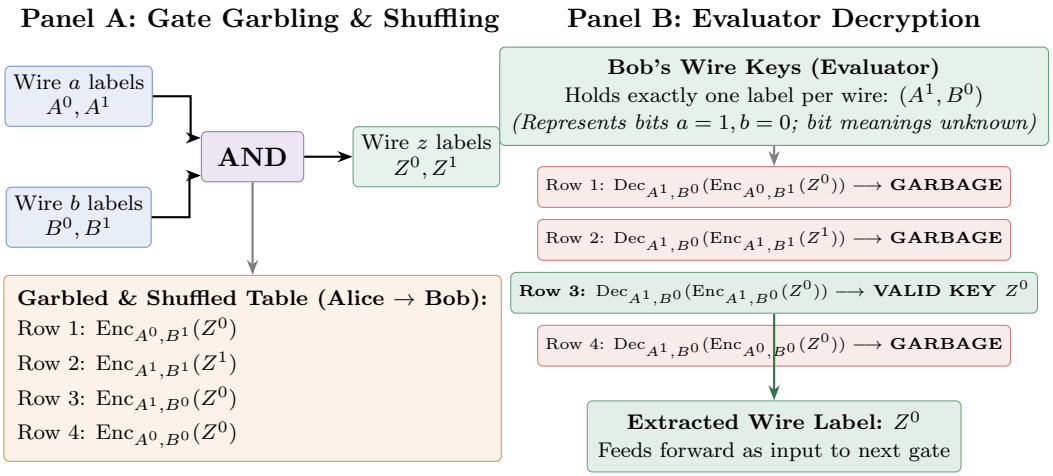


Figure 34.4: One garbled AND gate. In Panel A, Alice creates random 128-bit labels for each wire, double-encrypts the output label under the corresponding input keys, and permutes the rows. In Panel B, Bob holds key pair (A^1, B^0); exactly one row decrypts validly to reveal output label Z^0, keeping all internal wire values completely opaque.

34.4.2 Wiring up the whole circuit

A circuit is many gates chained together. Alice garbles every gate the same way, so the output label of one gate becomes an input label of the next. The remaining questions are: how does Bob get the input labels to start, and how does he read the final answer?

1. **Alice's input.** For each of Alice's input wires, she knows her bit, so she just hands Bob the corresponding label directly. If Alice's input bit on wire a is 1, she sends A^1. Bob cannot tell it means 1.

2. **Bob's input — via OT.** Bob must get the label matching *his* input bit on each of his input

wires, but Alice must not learn which bit Bob has, and Bob must not get both labels. This is exactly 1-out-of-2 oblivious transfer. For each of Bob's input wires, Alice plays OT sender with messages (B^0, B^1); Bob plays receiver with choice bit equal to his input. He gets B^c and nothing else, and Alice learns nothing about c.

3. **Evaluation.** Bob propagates labels gate by gate through the garbled tables, decrypting one row per gate, until he holds the labels on the output wires.

4. **Output decoding.** Alice provides a small decoding table mapping the two output-wire labels back to bits: "$Z^0_{\text{out}} \mapsto 0$, $Z^1_{\text{out}} \mapsto 1$." Bob looks up his final label and reads the answer. (If only Bob should learn the output, Alice sends him this table privately; if both should, Bob announces the labels and Alice decodes too.)

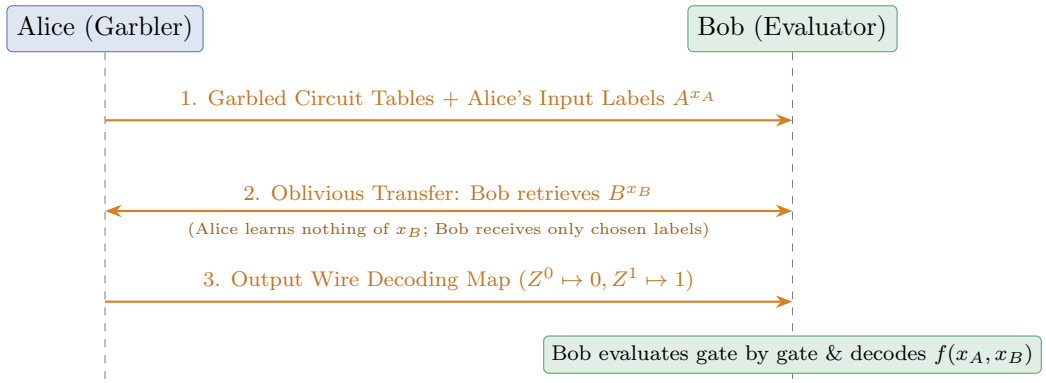


Figure 34.5: The message flow of Yao's two-party protocol. Alice garbles and transmits the circuit. Bob fetches his own wire labels via Oblivious Transfer without leaking his inputs. Bob evaluates the gates sequentially and decodes the designated output wires.

Why it works

Why the whole thing is secure (passive model). Bob's entire view is: a pile of ciphertexts (the garbled tables), one label per wire, and a decoding table. A simulator who knows only the final output can fabricate an indistinguishable view — it generates random labels, builds fake garbled tables that decrypt consistently along the single path Bob walks, and rigs the decoding table to yield the known output. Because Bob only ever sees one label per wire and the tables are encrypted, he cannot distinguish the real garbling from the simulated one. Hence he learns nothing beyond the output. Symmetrically, Alice sees only the OT transcript, from which (by OT's sender-obliviousness) she learns nothing about Bob's input.

34.4.3 The optimizations that make it real

Plain garbling sends four ciphertexts per gate. Decades of refinement shrank this dramatically, and these tricks are why garbled circuits run at line rate today.

- *Point-and-permute*: append a random "select bit" to each label so Bob knows directly which single row to decrypt, instead of trial-decrypting all four. One decryption per gate.

- *Free XOR* (Kolesnikov-Schneider, 2008): choose all labels so that $W^1 = W^0 \oplus \Delta$ for a single global secret offset Δ. Then an XOR gate needs *no* garbled table and *no* cryptographic operation at all — the output label is just the XOR of the input labels. XOR gates become free, which matters because many useful circuits (addition, comparison) are XOR-heavy.

- *Half-gates* (Zahur-Rosulek-Evans, 2015): brings each AND gate down to two ciphertexts, proven optimal for a broad class of techniques.

Implement it

A stripped-down garbler for a single AND gate, to make the labels and tables concrete. This is **toy code for learning only** — it uses a plain SHA-256 mask, no authentication beyond a toy membership check, no point-and-permute, and leaks timing. Never use it for anything real; production garbling uses AES with hardware acceleration and the optimizations above.

```python
import os, hashlib, random

def label():                        # a random 16-byte wire label
    return os.urandom(16)

def kdf(k1, k2):                    # derive a mask from two key labels
    return hashlib.sha256(k1 + k2).digest()[:16]

def xor(a, b):
    return bytes(x ^ y for x, y in zip(a, b))

# two labels per wire: index 0 means bit 0, index 1 means bit 1
A = [label(), label()]
B = [label(), label()]
Z = [label(), label()]

# AND truth table: z = a AND b
table = []
for a in (0, 1):
    for b in (0, 1):
        z = a & b
        ct = xor(Z[z], kdf(A[a], B[b]))   # "encrypt" output label
        table.append(ct)
random.shuffle(table)                      # garble: hide row order

# --- Bob evaluates with one label per wire, say a=1, b=0 ---
my_A, my_B = A[1], B[0]
recovered = None
for ct in table:
    cand = xor(ct, kdf(my_A, my_B))
    if cand in (Z[0], Z[1]):               # toy "did it decrypt?" check
        recovered = cand
print("output label == Z0 (bit 0)?", recovered == Z[0])  # True: 1 AND 0 = 0
```

Real systems replace the membership test with an authenticated decryption (a row succeeds only with valid padding or a MAC), so Bob cannot brute-force other rows.

34.5 Secret-sharing-based MPC for many parties

Garbled circuits are well suited to two parties, but they do not scale gracefully to n parties. For many participants, the dominant approach is built on *secret sharing* — Shamir's scheme from Chapter 32. The idea is direct: split every secret into shares, then compute on the shares.

34.5.1 Recap: Shamir shares are linear

Recall Shamir t-out-of-n sharing. To share a secret $s \in \mathbb{F}_p$, the dealer picks a random degree-$(t-1)$ polynomial f with $f(0) = s$, and gives party i the share $f(i)$. Any t parties can interpolate to recover s; any $t-1$ learn nothing, because $t-1$ points are consistent with every possible secret.

The key algebraic property for MPC is that *secret sharing is linear*. If f shares a and g shares b, then $f+g$ is a valid polynomial sharing $a+b$ — and each party can compute its share of the sum locally, just by adding its two shares. No communication needed.

Intuition

Think of each party as holding a single coordinate of a high-dimensional secret. Adding two secrets means adding coordinates pointwise; nobody needs to see anyone else's coordinate to do that. Scaling by a public constant is just as local: multiply your share by the constant. This is why *linear* functions of secrets are trivial in this model. The whole difficulty of MPC concentrates in one place: multiplication.

34.5.2 Addition is free, multiplication is the hard part

Let each party P_i hold a Shamir share $[x] = (x_1, \ldots, x_n)$ of secret x and $[y]$ of y, where $[x]$ means "the vector of shares of x."

Addition. To get shares of $x+y$, each party computes $x_i + y_i$ locally. Done. Shares of $x+c$ for a public constant c: by convention one designated party adds c to its share. Shares of $c \cdot x$: each party multiplies its share by c.

Multiplication. Each party can multiply locally: $x_i \cdot y_i$. But there is a problem. If f has degree $t-1$ and g has degree $t-1$, then their product $f \cdot g$ has degree $2(t-1)$. The local products $x_i \cdot y_i$ are points on this degree-$2(t-1)$ polynomial, which does share xy at 0 — but you now need $2t-1$ points to interpolate it, and after a few multiplications the degree blows past n and the secret becomes unrecoverable. Worse, $f \cdot g$ is not a uniformly random polynomial of that degree (its coefficients are correlated), so the naive product can leak. Multiplication needs a *degree-reduction* step with communication.

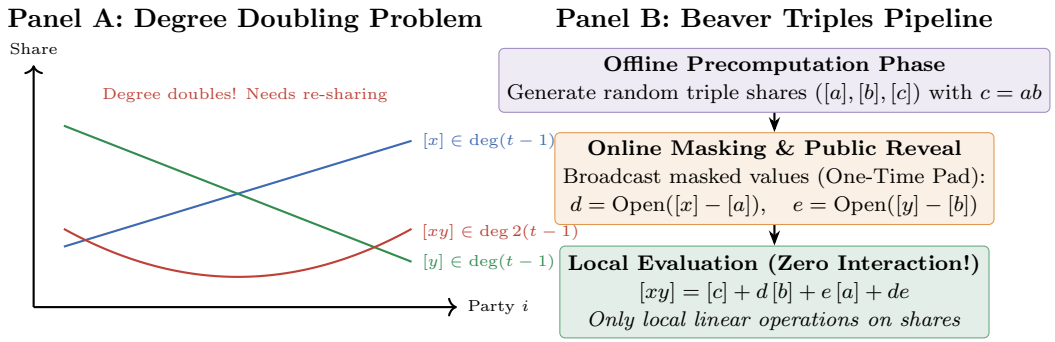


Figure 34.6: Handling multiplication in secret-sharing MPC. Panel A illustrates how pointwise multiplication doubles polynomial degree, risking security and recovery thresholds. Panel B shows Beaver's offline/online decoupling: precomputed triples convert online multiplications into one-time-pad masked openings and local linear combinations.

34.5.3 The BGW multiplication step

The classic fix (Ben-Or, Goldwasser, Wigderson, 1988) handles the multiplication with one round of communication:

1. Each party P_i computes the local product $d_i = x_i \cdot y_i$. These d_i lie on a degree-$2(t-1)$ polynomial h with $h(0) = xy$.
2. Each P_i *re-shares* its d_i: it acts as a dealer and Shamir-shares d_i with a fresh degree-$(t-1)$ polynomial, sending sub-shares to everyone.
3. Now everyone holds shares of every d_i. They take a fixed public linear combination of these (the Lagrange coefficients that would reconstruct $h(0)$) — but applied *on the shares*, which is a local linear operation. The result is a fresh degree-$(t-1)$ sharing of $\sum_i \lambda_i d_i = h(0) = xy$.

The interpolation that would normally combine the d_i into xy is a linear map, and we already know linear maps on shares are free. By pushing it through the re-shared low-degree polynomials, we get xy shared at degree $t-1$ again, re-randomized. This needs $t < n/2$ (a majority of uncorrupted parties) for the degree-$2(t-1)$ polynomial to be recoverable.

Focus

The pattern is universal: *linear operations are local and free; multiplication costs one interaction.* So the "size" of an MPC is governed by the number of multiplication gates in the circuit (its *multiplicative depth* and count), exactly as garbled circuits are governed by AND gates and free on XOR. Designing MPC-friendly functions means minimizing multiplications — which is why MPC and homomorphic encryption (the previous chapter) share so much circuit-optimization lore.

34.5.4 A worked multiplication with tiny numbers

Work over $\mathbb{F}_{11}$ (arithmetic mod 11) with $n = 3$ parties and threshold $t = 2$, so polynomials are degree 1 (lines). We multiply $x = 3$ by $y = 4$; the true answer is $12 \equiv 1 \pmod{11}$.

Share $x = 3$. Dealer picks the line $f(u) = 3 + 2u$. Shares: $f(1) = 5$, $f(2) = 7$, $f(3) = 9$. So $x_1, x_2, x_3 = 5, 7, 9$.

Share $y = 4$. Dealer picks the line $g(u) = 4 + 1u$. Shares: $g(1) = 5$, $g(2) = 6$, $g(3) = 7$. So $y_1, y_2, y_3 = 5, 6, 7$.

Local products. $d_1 = 5 \cdot 5 = 25 \equiv 3$, $d_2 = 7 \cdot 6 = 42 \equiv 9$, $d_3 = 9 \cdot 7 = 63 \equiv 8 \pmod{11}$. These three points $(1, 3), (2, 9), (3, 8)$ lie on a degree-2 parabola h with $h(0) = xy$.

Check the answer is hidden in $h(0)$. Interpolate h through the three points using Lagrange at $u = 0$. The Lagrange coefficients at 0 for nodes $1, 2, 3$ are

$$\lambda_1 = \tfrac{(0-2)(0-3)}{(1-2)(1-3)} = \tfrac{6}{2} = 3, \quad \lambda_2 = \tfrac{(0-1)(0-3)}{(2-1)(2-3)} = \tfrac{3}{-1} = -3, \quad \lambda_3 = \tfrac{(0-1)(0-2)}{(3-1)(3-2)} = \tfrac{2}{2} = 1.$$

Then $h(0) = \lambda_1 d_1 + \lambda_2 d_2 + \lambda_3 d_3 = 3 \cdot 3 + (-3) \cdot 9 + 1 \cdot 8 = 9 - 27 + 8 = -10 \equiv 1 \pmod{11}$. That is exactly $xy = 1$. The product is correctly encoded — it just sits on a degree-2 polynomial. The BGW re-sharing step would now push this same linear combination through fresh degree-1 sharings of each d_i, leaving everyone with a degree-1 sharing of 1, ready for the next gate.

34.5.5 Beaver triples: precompute the hard part

A beautiful refinement (Donald Beaver, 1991) moves the expensive multiplication *offline*. In a preprocessing phase — before inputs are even known — the parties generate shared random *triples* $([a], [b], [c])$ with $c = ab$, where a, b are random and unknown to anyone. Then, in the online phase, multiplying two shared secrets $[x] \cdot [y]$ costs only additions and two public reconstructions:

1. Parties locally compute and reveal $[d] = [x] - [a]$ and $[e] = [y] - [b]$. Since a, b are random and secret, $d = x - a$ and $e = y - b$ leak nothing about x, y (they are one-time-pad-masked).
2. Each party locally sets its share of $[xy]$ to

$$[xy] = [c] + d\,[b] + e\,[a] + d\,e.$$

Why it works

Why the triple formula gives xy. Substitute $x = d + a$ and $y = e + b$:

$$xy = (d + a)(e + b) = de + db + ea + ab = de + d\,b + e\,a + c.$$

Every term on the right is either a public value (d, e) times a share, or the precomputed $[c]$ — all linear operations on shares, hence local and free. The only communication in the online phase is revealing d and e, two field elements per multiplication. The secret-protecting masking is exactly a one-time pad: $d = x - a$ hides x because a is uniform and used once.

Focus

This offline/online split is the architecture of modern practical MPC frameworks (SPDZ and its descendants). The computationally demanding cryptographic operations—generating authenticated multiplication triples, often via homomorphic encryption or OT extension—occur in advance during an offline phase. When the real inputs arrive, the online phase is almost pure information-theoretic arithmetic and runs very fast. SPDZ also adds message authen-

tication codes on every share so that a *malicious* party who tampers with a value is caught with overwhelming probability, lifting security from the passive model to the malicious one.

34.6 An attack: when "private" inputs aren't enough

MPC's guarantee is precise and easy to over-read. It protects the inputs *during* computation. It says nothing about whether the agreed function and output are safe to reveal. This gap is where real deployments get hurt.

Attack

The function-leakage attack. Suppose three hospitals run an MPC to compute the average age of patients with a rare disease, to publish a statistic. The protocol is flawless: no hospital sees another's records. But suppose hospital A controls (or colludes with) two of the three parties and knows its own contribution exactly. From the published average and its own data, A can algebraically solve for the combined contribution of the others. If only one other patient exists, A learns that individual's exact age — a total privacy breach, with the MPC working perfectly. The leak came from the *function*, not the protocol.

Even nastier: the chosen-input attack. A malicious party can pick a *crafted* input to make the output reveal a target's secret. To learn whether Bob's salary equals \$87,000, a malicious participant in a "compute the maximum salary" MPC submits the input \$86,999. If the announced max is \$87,000, and the attacker knows everyone else is below \$86,999, the output pinpoints Bob. Again the cryptography did its job; the function composed with adversarial inputs did the leaking.

Common trap

Two defenses, neither cryptographic in the usual sense:

- *Function vetting / output privacy*: only compute functions whose output is safe to publish. This is where MPC meets *differential privacy* — add calibrated noise inside the MPC so the output cannot single out an individual. MPC hides inputs; differential privacy bounds what the output reveals. Serious privacy systems need both.
- *Input validation and collusion bounds*: enforce that inputs are well-formed (via zero-knowledge range proofs) and design for a realistic corruption threshold. A protocol that assumes most parties are uncorrupted is worthless if one entity secretly controls a majority of the "independent" parties.

The lesson: MPC protects inputs during computation, not privacy in the abstract. Always ask separately whether the *output* itself is safe to publish.

34.7 Where MPC is used in the real world

In the real world

Sealed-bid auctions. The first large-scale real MPC deployment was Denmark's 2008 sugar-beet auction (Bogetoft et al.). Thousands of farmers and the sole buyer needed a market-clearing price, but neither side trusted the other — or any third party — to see the bids, which revealed each farm's economics. Three servers ran a secret-sharing MPC; no machine ever held a plaintext bid, yet the clearing price came out correctly. This is the canonical "the auctioneer never sees the bids" application.

Threshold signing and key management. A signing key split across n machines via secret sharing, where any t can jointly produce a signature but no smaller set can, and no machine ever reconstructs the full key. Threshold ECDSA and threshold Schnorr/EdDSA protocols are MPC for the specific function "sign this message." Cryptocurrency custodians, certificate authorities, and HSM clusters use this so that compromising one server (or one insider) cannot steal the key or forge signatures. The elliptic-curve machinery from Chapter 21 is exactly what these protocols compute over, securely shared.

Privacy-preserving analytics. Google's "Private Join and Compute" and similar private-aggregation systems let parties compute aggregate statistics (ad-conversion counts, telemetry sums) across datasets without sharing the underlying records. Apple and Google's exposure-notification and private-set-intersection features use OT/MPC ideas so one party learns the intersection size or a sum without learning the other's full set.

Private set intersection (PSI). A specialized, hyper-optimized MPC: two parties learn only the elements their sets share. "Contact discovery" in messengers (find which of my contacts use the app) and "did this leaked password appear in your breach database" lookups are PSI in production, built largely from OT extension.

Connection

MPC, homomorphic encryption (previous chapter), and zero-knowledge proofs form a triad of "computing on secrets." Homomorphic encryption lets *one* party compute on another's encrypted data — great when computation is outsourced to a single untrusted server. MPC distributes the computation across the data owners themselves — great when there is no single trusted server and inputs come from many parties. ZK proves a statement about a secret without revealing it — and is the glue that makes MPC robust against malicious parties. Real systems mix them: SPDZ uses homomorphic encryption to generate Beaver triples; threshold signatures use ZK proofs to keep parties in line.

34.8 What it costs, and when to reach for it

MPC is not free. Garbled circuits send ciphertexts proportional to the number of AND gates and require a full circuit representation of f — expensive for functions with deep arithmetic. Secret-sharing MPC trades that for rounds of communication proportional to the multiplicative depth, so high-latency networks hurt. Both are orders of magnitude slower than computing f in the clear, though OT extension, free-XOR, half-gates, and the SPDZ offline/online split have pushed many useful computations into the practical range (seconds to minutes for serious circuits, microseconds

per gate).

Reading note

Rough mental model for choosing a technique:

- Two parties, function expressible as a Boolean circuit, want low round count: *Yao garbled circuits* (constant rounds).
- Many parties, arithmetic-heavy function, a majority of uncorrupted parties, can tolerate rounds: *secret-sharing MPC* (BGW/SPDZ).
- Need malicious security: add MACs (SPDZ) or ZK compilation; expect a constant-factor to order-of-magnitude slowdown.
- One untrusted compute server, not many data owners: consider *homomorphic encryption* instead (previous chapter).

The headline to carry away: any function that can be computed can be computed *securely* among mutually distrustful parties, with privacy of inputs guaranteed by oblivious transfer (for circuits) or by the linearity of secret sharing (for arithmetic). The remaining engineering is about making that theoretical possibility fast enough to use — and about the separate, non-cryptographic question of whether the output you chose to reveal was safe to reveal at all.

Throughout our study of public-key cryptography, Diffie–Hellman key exchanges, RSA, elliptic curves, and oblivious transfer, security has rested on the presumed hardness of factoring and discrete logarithms. But Shor's algorithm proved that a sufficiently large quantum computer can solve both in polynomial time, collapsing the mathematical foundation underneath today's digital infrastructure. In the next chapter, we confront this threat: post-quantum cryptography, exploring the lattice-based, hash-based, and code-based constructions engineered to survive quantum adversaries.

Exercises

Exercise 34.1. OT by hand. Use the Diffie-Hellman OT of Section 34.3 in the order-11 subgroup of $(\mathbb{Z}/23\mathbb{Z})^*$ with $g = 2$ and public constant $C = 18$. Take the receiver's choice bit $c = 0$ and $k = 5$. Compute PK_0 and PK_1, verify $PK_0 \cdot PK_1 \equiv C \pmod{23}$, and state which of the two messages the receiver can recover and why he cannot recover the other.

Exercise 34.2. Garble an OR gate. Following Section 34.4, write out the four garbled-table rows for an OR gate $z = a \vee b$ in terms of the wire labels $A^0, A^1, B^0, B^1, Z^0, Z^1$. Then, given that Bob holds A^0 and B^1, identify which row he decrypts and which output label he obtains. What bit does that label encode?

Exercise 34.3. Free XOR. Suppose all labels satisfy $W^1 = W^0 \oplus \Delta$ for a global secret Δ. Show that if the output label of an XOR gate is defined as $Z^0 = A^0 \oplus B^0$ (and $Z^1 = Z^0 \oplus \Delta$), then for *any* input bits a, b the evaluator who holds A^a and B^b can compute the correct output label $A^a \oplus B^b$ with no garbled table. Verify all four cases.

Exercise 34.4. Local addition is correct. Over $\mathbb{F}_{11}$ with $t = 2$, $n = 3$, share $x = 6$ with the line $f(u) = 6 + 3u$ and $y = 2$ with $g(u) = 2 + 5u$. Write each party's two shares, have each party add them locally, and interpolate the three sums back to $u = 0$. Confirm you recover $x + y = 8$ and that the parties needed no communication.

Exercise 34.5. Multiplication degree blow-up. Using the same shares of $x = 6$ and $y = 2$ from the previous exercise, compute the three local products $d_i = x_i y_i \bmod 11$. Interpolate the parabola through $(1, d_1), (2, d_2), (3, d_3)$ and evaluate at 0 using the Lagrange coefficients $3, -3, 1$. Confirm you get $xy = 12 \equiv 1 \pmod{11}$, and explain why the parties cannot simply use the d_i directly as shares for the next gate.

Exercise 34.6. Beaver triple arithmetic. Over $\mathbb{F}_{13}$, a precomputed triple has $a = 5$, $b = 4$, $c = ab = 20 \equiv 7$. Parties hold shares of $x = 9$ and $y = 6$. Compute the revealed values $d = x - a$ and $e = y - b$ modulo 13, then evaluate $c + db + ea + de$ modulo 13 and check it equals xy mod 13. (You may work with the secrets directly to verify the formula; the point is the algebra.)

Exercise 34.7. Break a careless deployment. Five companies run an MPC to publish the *sum* of their R&D budgets. Four of them quietly collude. Explain precisely what they learn about the fifth company's budget, and why the MPC protocol itself is not at fault. Propose one mitigation that does not abandon the MPC.

Exercise 34.8. Chosen-input attack. A salary-transparency MPC computes the *maximum* salary among participants and announces it. You are malicious and want to learn whether a specific colleague earns exactly \$120,000, and you believe everyone else earns less than \$119,000. Describe the input you submit and how you read off the answer from the announced maximum. What property of the chosen *function* enabled the attack?

Exercise 34.9. Why OT needs hardness. Argue that 1-out-of-2 oblivious transfer cannot exist with information-theoretic security against *both* parties simultaneously in the plain two-party model (no setup). Hint: consider what it would mean for the receiver to be unable to learn m_{1-c} even with unbounded computation, while the sender is also unbounded and learns nothing about c, and think about the joint distribution of transcripts.

Exercise 34.10. Round complexity. Yao's protocol is *constant round* (the message flow of Figure 34.5) while BGW-style secret-sharing MPC uses one communication round per layer of multiplication gates. For a function whose circuit is a single chain of 1000 multiplications, contrast the round counts of the two approaches, and explain why the Beaver-triple offline phase changes the practical calculus.

Chapter 35

Post-Quantum Cryptography

Every public-key scheme in this book so far rests on one of two hard problems: factoring a large integer (RSA) or computing a discrete logarithm in some group (Diffie-Hellman, ElGamal, DSA, and all of elliptic-curve cryptography). Both have served us well for half a century. Both are broken — completely, not marginally — the day someone builds a large, fault-tolerant quantum computer. Not weakened the way DES was weakened by faster machines, where you respond by adding key bits. Broken in a way that no key size can fix, because the quantum algorithm that kills them runs in time polynomial in the key length.

That is a strange situation. The machine that breaks RSA-2048 does not exist yet, and credible people disagree about whether it will exist in ten years or thirty or ever. But the data you encrypt today can be copied today and stored, and decrypted the day the machine arrives. An adversary who is patient — a nation-state archiving traffic, say — is already collecting. So the question "when will quantum computers break RSA" is the wrong one for a defender. The right question is "how long does my secret need to stay secret, and how long will migration take," and for a great deal of data the answer makes the threat present-tense.

This chapter explains why quantum computers menace exactly the schemes they menace (and, just as importantly, why they leave AES and SHA-256 mostly intact), gives you the intuition behind Shor's and Grover's algorithms without assuming you know quantum mechanics, and then builds up the replacements: lattice problems and the LWE assumption, the Kyber and Dilithium schemes that NIST standardized, and hash-based signatures that lean only on the one primitive we trust most. We will trace small numbers by hand, break a toy lattice scheme to see where the security actually lives, and close with the practical questions of migration, hybrid key exchange, and timelines.

35.1 What a quantum computer is, in just enough detail

You do not need the physics. You need a computational model precise enough to see why factoring falls and AES does not. Here it is.

A classical bit is 0 or 1. A *qubit* has a state described by two complex numbers α and β with $|\alpha|^2 + |\beta|^2 = 1$, written

$$|\psi\rangle = \alpha|0\rangle + \beta|1\rangle.$$

The notation $|0\rangle$ is just a name for a basis vector; think of $|0\rangle = \binom{1}{0}$ and $|1\rangle = \binom{0}{1}$. The qubit is a

unit vector in $\mathbb{C}^2$. When you *measure* it, you get 0 with probability $|\alpha|^2$ and 1 with probability $|\beta|^2$, and the state collapses to whatever you saw. The numbers α, β are *amplitudes*, not probabilities; the essential difference is that amplitudes can be negative (or complex), so two contributions to the same outcome can *cancel*. That cancellation—interference—provides the foundation for quantum algorithmic speedups.

With n qubits the state is a unit vector in $\mathbb{C}^{2^n}$: one amplitude for each of the 2^n classical strings. A computation is a sequence of *gates*, each a unitary matrix (a length-preserving linear map) acting on a few qubits, followed by a measurement. The catch that kills the naive "it tries all 2^n answers at once" story: you only get to read out one string, sampled according to $|\text{amplitude}|^2$. A quantum algorithm achieves a speedup only if it arranges, through interference, for the amplitudes of *wrong* answers to cancel and the amplitudes of *right* answers to reinforce, so that the single measurement at the end lands on a useful outcome with high probability.

Intuition

Picture 2^n little arrows (the amplitudes), one per possible answer. A classical randomized algorithm can only make these arrows shorter or longer — probabilities are non-negative. A quantum algorithm can also *rotate* them, so that arrows pointing in opposite directions sum to nearly zero. Designing a quantum algorithm means choosing unitary steps so that the wrong answers point in opposing directions and cancel out, while the right answers align and reinforce one another. Shor's algorithm achieves this interference pattern using the *period* of a function; Grover's uses a slower, more general sequence of reflections that gradually rotates the state vector toward the target.

History

The idea that quantum systems might compute things classical machines cannot goes back to Richard Feynman (1981/82), who pointed out that simulating quantum physics on a classical computer seems to cost exponential time, and David Deutsch (1985), who formalized the quantum Turing machine. For a decade it was a curiosity. Then in 1994 Peter Shor, at Bell Labs, found polynomial-time quantum algorithms for factoring and discrete log, and the field stopped being a curiosity overnight: the world's deployed public-key cryptography had an expiration date stamped on it. Lov Grover's search algorithm followed in 1996. To build quantum-resistant replacements, Miklós Ajtai connected average-case cryptographic hardness to worst-case lattice problems in 1996, and Oded Regev introduced Learning With Errors (LWE) in 2005. From 2016 to 2024 NIST ran an open competition to standardize post-quantum cryptography: CRYSTALS-Kyber and CRYSTALS-Dilithium ("CRYSTALS" = Cryptographic Suite for Algebraic Lattices) were selected in 2022 and standardized in 2024 as ML-KEM (FIPS 203) and ML-DSA (FIPS 204), the stateless hash-based SPHINCS+ became SLH-DSA (FIPS 205), and the lattice signature Falcon became FN-DSA.

35.2 Shor's algorithm: why factoring and discrete log fall

RSA's security rests on the gap between multiplying (easy) and factoring (believed hard classically: the best known algorithm, the general number field sieve, runs in time roughly $\exp\big(c\,(\log N)^{1/3}\,(\log\log N)^{2/3}\big)$, sub-exponential but super-polynomial). Shor's algorithm factors in time polynomial

in $\log N$. To see how, we reduce factoring to *period-finding*, and then quantum mechanics does the period-finding fast.

35.2.1 Factoring reduces to finding the order of an element

Pick a random a with $1 < a < N$ and $\gcd(a, N) = 1$ (if the gcd is not 1 you already found a factor — lucky). Consider the function

$$f(x) = a^x \bmod N.$$

Because a is a unit mod N, this function is periodic: there is a smallest $r > 0$, the *order* of a, with $a^r \equiv 1 \pmod N$, and then $f(x + r) = f(x)$ for all x. Suppose we can find r. Two pieces of luck, each with constant probability over the choice of a, finish the job:

1. r is even. Then $a^{r/2}$ is a square root of 1 mod N, so $N \mid (a^{r/2} - 1)(a^{r/2} + 1)$.
2. $a^{r/2} \not\equiv -1 \pmod N$ (and of course $a^{r/2} \not\equiv 1$, since r is the *smallest* period). Then $a^{r/2} \pm 1$ are nontrivial: neither is a multiple of N, yet their product is. So $\gcd(a^{r/2} - 1, N)$ is a nontrivial factor of N, computable in an instant by Euclid.

One can show that for N with at least two distinct odd prime factors, a random a satisfies both conditions with probability at least $1/2$. So factoring boils down to: find the order r of a random element. Everything classical here is cheap. The only hard step is finding r, and that is where the quantum computer comes in.

Example 35.1 (Factoring 15 by hand)**.** Take $N = 15$, $a = 7$. Compute powers mod 15: $7^1 = 7$, $7^2 = 49 \equiv 4$, $7^3 \equiv 28 \equiv 13$, $7^4 \equiv 91 \equiv 1$. So the order is $r = 4$, which is even. Then $a^{r/2} = 7^2 = 49 \equiv 4 \pmod{15}$, and $4 \not\equiv \pm 1$. Now

$$\gcd(4 - 1,\, 15) = \gcd(3, 15) = 3, \qquad \gcd(4 + 1, 15) = \gcd(5, 15) = 5.$$

We recovered $15 = 3 \times 5$ purely from the period $r = 4$. A quantum computer does not do the period-finding by computing the powers one at a time the way we just did — that would be no faster than classical. It uses interference, next.

35.2.2 Finding the period with a quantum Fourier transform

The quantum part has a clean shape. Prepare a uniform superposition over a large range of inputs $x \in \{0, 1, \ldots, Q - 1\}$ (with $Q = 2^q$ a power of two, far bigger than N), and in a second register compute $a^x \bmod N$ *reversibly*, entangling input and output:

$$\frac{1}{\sqrt{Q}} \sum_{x=0}^{Q-1} |x\rangle\, |a^x \bmod N\rangle.$$

Measuring the second register collapses the first to a superposition over only those x giving the observed value $a^{x_0} \bmod N$ — and those x are exactly an arithmetic progression $x_0, x_0 + r, x_0 + 2r, \ldots$ spaced by the period r. Now apply the *quantum Fourier transform* to the first register. The QFT of a spike train spaced by r is a spike train spaced by $\approx Q/r$: measuring yields a value close to a multiple of Q/r. From a measured $c \approx kQ/r$ you recover r by rounding c/Q to a nearby fraction with small denominator (continued fractions). Repeat a couple of times to nail down k and r.

Why it works

Quantum interference drives this algorithmic speedup. Each x in the progression contributes an amplitude, and the QFT causes these amplitudes to add *in phase* only at frequencies that are multiples of Q/r; at other frequencies they cancel. Classically, determining the period requires evaluating f at $\Theta(r)$ points, where r can be on the order of N. The quantum algorithm evaluates f across all inputs in a single entangled state and extracts the period frequency. The circuit size is polynomial in $\log N$: requiring several thousand logical qubits and $O((\log N)^3)$ operations for a 2048-bit modulus N. Although physical error correction requires substantial hardware overhead, the asymptotic scaling means that increasing key lengths cannot restore the security of RSA against quantum cryptanalysis.

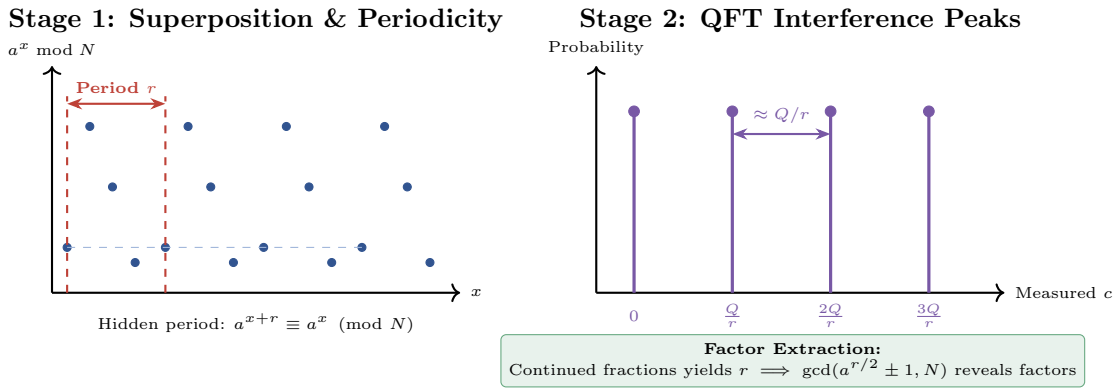


Figure 35.1: Shor's quantum period-finding architecture. In Stage 1, the quantum register evaluates $a^x \bmod N$ across an exponential superposition, establishing a periodic spike train spaced by r. In Stage 2, the Quantum Fourier Transform causes destructive interference everywhere except at discrete frequency multiples of Q/r, allowing measurement to disclose the order r and break RSA in polynomial time.

35.2.3 Discrete log dies the same way

The discrete-log problem — given g and $y = g^s$ in a cyclic group of order n, find s — is also a hidden-period problem. Define $f(x_1, x_2) = g^{x_1} y^{-x_2}$. This function is constant on the set of (x_1, x_2) with $x_1 - sx_2 \equiv 0$, i.e. it has a hidden *linear* period determined by s. The same Fourier machinery extracts s in polynomial time. Because elliptic-curve discrete log lives in a group with the same abelian structure, ECC offers *no* extra protection here: a 256-bit elliptic-curve key and a 3072-bit RSA key are both Shor-breakable, and the elliptic-curve case actually needs fewer logical qubits because the numbers are smaller. The hardness that made ECC keys short is exactly the hardness Shor dissolves.

Focus

The pattern: Shor breaks any cryptosystem whose security reduces to finding a *hidden period* or *hidden subgroup* in a *commutative* (abelian) group. Factoring, RSA, finite-field discrete log, elliptic-curve discrete log — all of these. The post-quantum schemes in this chapter are built so that their hardness does *not* have this abelian-hidden-subgroup shape. That is the

structural reason lattices, codes, and hash chains are candidates: no one knows how to phrase their hard problems as a period to be found.

35.3 Grover's algorithm and what it does to symmetric crypto

Where Shor exploits algebraic structure for an exponential speedup, Grover searches without structure and achieves only a quadratic one. Grover's algorithm searches an unstructured space of M possibilities for a marked item in $\Theta(\sqrt{M})$ quantum queries, versus $\Theta(M)$ classically. Applied to a block cipher with a k-bit key, brute force has $M = 2^k$ candidates, so Grover finds the key in about $2^{k/2}$ steps.

The mechanism, in one paragraph: start in the uniform superposition over all M keys, amplitude $1/\sqrt{M}$ each. Repeat a two-step "rotation" — (1) an oracle that flips the sign of the amplitude of the correct key, (2) a "diffusion" that reflects all amplitudes about their average. Geometrically, each repetition rotates the state a tiny angle $\approx 2/\sqrt{M}$ toward the target; after about $(\pi/4)\sqrt{M}$ rotations the state is almost entirely on the target and a measurement returns it. Too many rotations *overshoot* and the success probability falls again — another reminder that this is rotation, not accumulation.

Why it works

Why only a square-root speedup, not exponential? Because there is no structure to exploit — the search is over a black box. Each Grover step rotates by an angle proportional to $1/\sqrt{M}$, and you need to cover a constant angle, so you need $\Theta(\sqrt{M})$ steps. A theorem (the BBBV bound, 1997) proves this is optimal: no quantum algorithm searches an unstructured space of size M in fewer than $\Omega(\sqrt{M})$ queries. So $2^{k/2}$ is a hard floor for generic key search, not just the best we currently know.

The defensive consequence is mild and you already know the fix: *double the key size*. A 128-bit key offers $\approx 2^{64}$ Grover security, which is uncomfortable; a 256-bit key offers $\approx 2^{128}$, which is fine. This is why AES-256, not AES-128, is the conservative post-quantum choice, and why SHA-512 / SHA-256 hashing is judged adequate: the best quantum collision and preimage attacks give at most quadratic (and for collisions, by some analyses, only cube-root) speedups, leaving large hashes comfortable.

Common trap

A common over-reaction: "Grover halves every security level, so all symmetric crypto is in trouble." Two corrections. First, halving the *exponent* (from 2^{128} work to 2^{64}) is dramatic for 128-bit keys but readily fixed by going to 256 bits — symmetric keys are cheap. Second, Grover is deeply serial: each of the $2^{k/2}$ steps depends on the last, so unlike classical brute force you cannot parallelize it into a $2^{k/2}$ speedup across many machines. Splitting Grover across p processors only gives a $\sqrt{p}$ improvement, so the real-world cost of a Grover attack on AES-256 is astronomically beyond any plausible machine. The asymmetry is the headline: Shor is catastrophic and unfixable for RSA/ECC; Grover is a nuisance, fixed by one parameter bump.

Connection

The square-root in Grover is the same square-root you met in the companion probability book as the birthday bound, but turned inside out. There, $\sqrt{M}$ random samples suffice to find a *collision* among M values because collisions are quadratically common. Here, $\sqrt{M}$ *queries* suffice to find a single marked item because amplitude rotation accumulates as the square root of the number of steps. Both are statements that "$\sqrt{M}$ is the natural scale of M possibilities" — one probabilistic, one geometric.

Primitive	Classical security	Best quantum attack
RSA-2048 / ECC-256	sub-exponential / 2^{128}	polynomial (Shor) — broken
AES-128	2^{128}	$\approx 2^{64}$ (Grover) — weak
AES-256	2^{256}	$\approx 2^{128}$ (Grover) — safe
SHA-256 (collision)	2^{128}	$\approx 2^{85}$ or 2^{128}, model-dependent — safe
SHA-256 (preimage)	2^{256}	$\approx 2^{128}$ (Grover) — safe

Table 35.1: Quantum impact at a glance. Public-key is the emergency; symmetric is a parameter bump.

35.4 Lattices: the foundation of post-quantum public key

If period-finding is the abelian structure that Shor exploits, we want hard problems with *no* such structure. Lattices deliver, and they are the basis for most of what NIST standardized. Start with the geometry, then build the cryptography.

35.4.1 What a lattice is

Given n linearly independent vectors $\mathbf{b}_1, \ldots, \mathbf{b}_n$ in $\mathbb{R}^n$, the *lattice* they generate is the set of all integer combinations:

$$\mathcal{L} = \left\{ \sum_{i=1}^{n} z_i \mathbf{b}_i : z_i \in \mathbb{Z} \right\}.$$

It is an infinite grid of points — regular, but in general skewed. The same lattice has many bases: you can replace the $\mathbf{b}_i$ by other integer combinations and still hit exactly the same points. Some bases are "nice" (short, nearly orthogonal vectors); some are "ugly" (long, nearly parallel vectors). The gap between nice and ugly bases is where the cryptography lives.

Two hard problems:

- **Shortest Vector Problem (SVP):** find the shortest nonzero vector in $\mathcal{L}$.
- **Closest Vector Problem (CVP):** given an arbitrary target point $\mathbf{t} \in \mathbb{R}^n$ (not necessarily in the lattice), find the lattice point nearest to it.

In two or three dimensions these look easy — you can see the answer geometrically. In a few hundred dimensions, with an ugly skewed basis, they are computationally formidable: the best known classical algorithms require time exponential in n, and *no quantum speedup beyond polynomial factors is known.* There is no periodic abelian group structure for Shor's quantum Fourier transform to exploit.

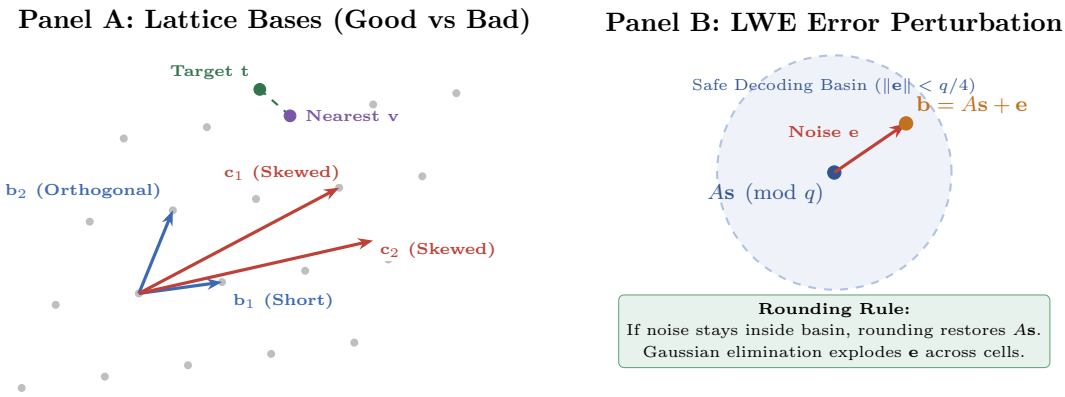


Figure 35.2: The geometric foundation of post-quantum lattice cryptography. Panel A contrasts a short, nearly orthogonal basis (secret key) with a long, skewed basis (public key), illustrating how finding the nearest lattice point becomes intractable with skewed axes. Panel B depicts the Learning With Errors (LWE) setting: a secret lattice coordinate is perturbed by a bounded noise vector $\mathbf{e}$ that conceals $\mathbf{s}$ from linear solvers while permitting clean threshold rounding during decryption.

Intuition

A good (short, orthogonal) basis is like a coordinate grid drawn on graph paper: to find the nearest lattice point to a target you just round each coordinate to the nearest integer. A bad (long, skew) basis describes the *same* grid but with axes that are nearly parallel and very long; rounding in those coordinates lands you far from the true nearest point, because a small rounding error along a long, slanted axis throws you across many cells. The public key in a lattice scheme is the bad basis; the private key is the good basis. Anyone can describe the lattice; only the key-holder can solve CVP in it.

35.4.2 Learning With Errors (LWE), the assumption that does the work

Modern lattice crypto rarely talks about bases directly. It uses an equivalent, friendlier formulation: *Learning With Errors*, introduced by Oded Regev in 2005. Fix a modulus q and a dimension n. The secret is a vector $\mathbf{s} \in \mathbb{Z}/q\mathbb{Z}^n$. The adversary is given many noisy linear equations in $\mathbf{s}$:

$$\mathbf{a}_i \xleftarrow{\$} \mathbb{Z}/q\mathbb{Z}^n, \qquad b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle + e_i \bmod q,$$

where each e_i is a small "error" drawn from a narrow distribution (say, $e_i \in \{-2, \ldots, 2\}$). The *search-LWE* problem is to recover $\mathbf{s}$ from the pairs $(\mathbf{a}_i, b_i)$. The *decision-LWE* problem is to distinguish such pairs from uniformly random pairs.

Without the errors this is trivial: n exact equations $b_i = \langle \mathbf{a}_i, \mathbf{s} \rangle$ are a linear system, and Gaussian elimination recovers $\mathbf{s}$. The errors are everything. Gaussian elimination amplifies the small errors catastrophically — each elimination step adds errors together, and after n steps the noise swamps the signal mod q. Regev proved that solving LWE on *average* is as hard as solving lattice problems (like approximate SVP) in the *worst case* — a strong theoretical guarantee that ties the security of a randomly generated key to the hardness of the hardest instance of a geometric problem.

Example 35.2 (A tiny LWE instance)**.** Let $q = 17$, $n = 3$, secret $\mathbf{s} = (4, 1, 11)$, errors in $\{-1, 0, 1\}$.

Three samples:

$$\begin{aligned}
\mathbf{a}_1 = (1,0,2): &\quad \langle \mathbf{a}_1, \mathbf{s}\rangle = 4+0+22 = 26 \equiv 9,\ e_1 = 1 \Rightarrow b_1 = 10.\\
\mathbf{a}_2 = (3,2,1): &\quad \langle \mathbf{a}_2, \mathbf{s}\rangle = 12+2+11 = 25 \equiv 8,\ e_2 = -1 \Rightarrow b_2 = 7.\\
\mathbf{a}_3 = (0,5,4): &\quad \langle \mathbf{a}_3, \mathbf{s}\rangle = 0+5+44 = 49 \equiv 15,\ e_3 = 0 \Rightarrow b_3 = 15.
\end{aligned}$$

Handed only the rows $\mathbf{a}_i$ and the values b_i, you must find $\mathbf{s}$. With $q = 17$ and $n = 3$ you could brute-force all $17^3 = 4913$ candidates and pick the one whose errors are all small — that is the attack, and it is exactly why real parameters use $n \approx 256$ or more so that q^n is astronomically large. The brute force you *can* do here is the brute force that becomes infeasible at scale.

35.4.3 Encrypting with LWE: Regev's scheme

LWE gives encryption almost for free. Stack m samples into a matrix $A \in \mathbb{Z}/q\mathbb{Z}^{m\times n}$ and a vector $\mathbf{b} = A\mathbf{s} + \mathbf{e} \bmod q$. The public key is $(A, \mathbf{b})$; the secret key is $\mathbf{s}$. To encrypt one bit $\mu \in \{0,1\}$:

1. Pick a random subset of the rows — equivalently a random vector $\mathbf{r} \in \{0,1\}^m$.
2. Send the ciphertext $(\mathbf{u}, v) = \big(A^\top \mathbf{r},\ \langle \mathbf{b}, \mathbf{r}\rangle + \mu \cdot \lfloor q/2 \rfloor\big) \bmod q$.

To decrypt, compute $v - \langle \mathbf{s}, \mathbf{u}\rangle \bmod q$. Expand:

$$v - \langle \mathbf{s}, \mathbf{u}\rangle = \langle A\mathbf{s} + \mathbf{e}, \mathbf{r}\rangle + \mu\lfloor q/2\rfloor - \langle \mathbf{s}, A^\top \mathbf{r}\rangle = \langle \mathbf{e}, \mathbf{r}\rangle + \mu\lfloor q/2\rfloor.$$

The term $\langle \mathbf{e}, \mathbf{r}\rangle$ is a sum of a few small errors, so it is small relative to $q/2$. The result is therefore close to 0 (if $\mu = 0$) or close to $q/2$ (if $\mu = 1$); rounding recovers μ. Figure 35.3 traces this flow from public-key generation through subset-sum encryption and noisy cancellation. Security: distinguishing the ciphertext from random is exactly decision-LWE, so an eavesdropper who could read μ could solve LWE.

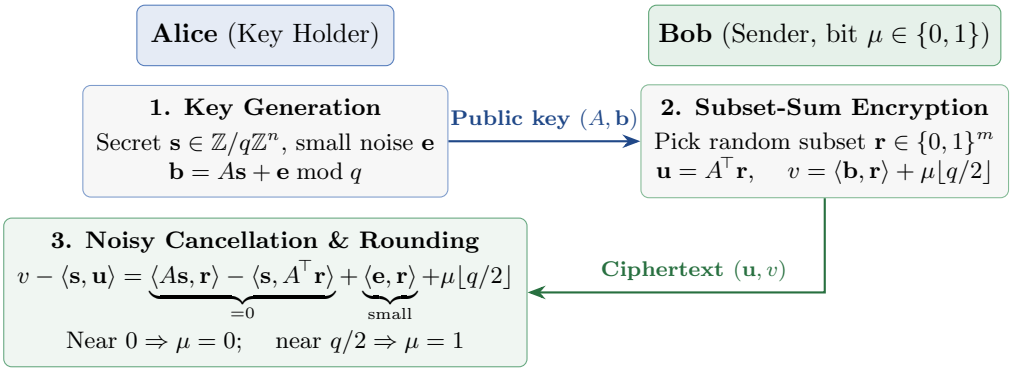


Figure 35.3: Regev's LWE encryption and decryption pipeline. Bob masks his message bit $\mu\lfloor q/2\rfloor$ with a random subset sum of Alice's noisy public equations. When Alice subtracts $\langle \mathbf{s}, \mathbf{u}\rangle$, the large matrix terms $A\mathbf{s}$ cancel identically, leaving only the small accumulated noise $\langle \mathbf{e}, \mathbf{r}\rangle$ on top of the encoded bit.

Why it works

The error is doing two opposite jobs at once, and the design is the balance between them. It must be *large enough* that the equations are hard — so an attacker cannot solve the linear system — yet *small enough* that the legitimate decryptor's noise term $\langle \mathbf{e}, \mathbf{r} \rangle$ stays well under $q/4$, leaving room to round correctly. Every lattice scheme is a budget: noise growth (which protects you) versus correctness margin (which serves you). Push the error up for security and you risk decryption failures; push it down for reliability and you weaken the assumption. Choosing q, n, and the error width is exactly negotiating that budget. Decryption-failure probability even becomes an attack surface — an adversary who can provoke and observe failures learns about $\mathbf{s}$ — which is why standardized parameters drive the failure rate down to around 2^{-130} or lower.

35.4.4 Module-LWE, Kyber, and Dilithium

Plain LWE has a practical problem: the public key is a full $m \times n$ matrix of integers mod q, which is large (megabytes). The fix is to add a controlled dose of structure, trading a little theoretical conservatism for a huge size win. Replace integers mod q with elements of a polynomial ring $R_q = \mathbb{Z}/q\mathbb{Z}[X]/(X^N + 1)$ — a single "coefficient" is now a degree-N polynomial, and multiplication mixes all N coefficients at once (this is *Ring-LWE*). One ring element does the work of an $N \times N$ structured matrix, so keys shrink dramatically. *Module-LWE* sits in between: small matrices (2×2, 3×3, 4×4) of ring elements, giving a tunable knob between size and conservatism. This is what the NIST winners use.

- **ML-KEM (Kyber)** — FIPS 203, standardized 2024 — is a *key-encapsulation mechanism*: it lets two parties agree on a shared secret, the post-quantum replacement for Diffie-Hellman / ECDH. It is Module-LWE encryption (essentially the Regev scheme above, over a module) wrapped with the Fujisaki-Okamoto transform to upgrade from "secure against eavesdroppers" to "secure against active, chosen-ciphertext attackers." Kyber-768 targets the AES-192 security level with a public key of about 1.2 KB and a ciphertext of about 1.1 KB — small enough to drop into a TLS handshake.
- **ML-DSA (Dilithium)** — FIPS 204, standardized 2024 — is a *signature* scheme over Module-LWE, the post-quantum replacement for ECDSA/RSA signatures. It uses the "Fiat-Shamir with aborts" technique: the signer runs an identification protocol, hashes the commitment to make it non-interactive, and *rejects and retries* any signature whose distribution would leak the secret key. Signatures are a couple of kilobytes — larger than ECDSA's 64 bytes, the recurring cost of going post-quantum.

Focus

Because NIST renamed the competition submissions upon standardization, you will meet both sets of names in libraries and papers: **Kyber** is **ML-KEM** (FIPS 203), **Dilithium** is **ML-DSA** (FIPS 204), **SPHINCS+** is **SLH-DSA** (FIPS 205), and **Falcon** is **FN-DSA**. Module-lattice schemes won the primary slots because they hit the best balance of key size, speed, and confidence. Still, structured lattices assume that Module-LWE is as hard as unstructured LWE — an assumption with weaker worst-case backing — which is why NIST also standardized hash-based SLH-DSA and continued evaluating code-based KEMs as a hedge.

Attack

The danger of *too much* structure is not hypothetical. In 2022 Castryck and Decru broke SIKE — a leading isogeny-based KEM that had reached round 4 of the NIST process — in about an hour on a single classical core, using a theorem (Kani's) about products of elliptic curves that exploited auxiliary structure SIKE exposed. SIKE was not a lattice scheme, but the lesson generalizes: each bit of algebraic structure you add to shrink keys is a bit an attacker might find a handle on. This is the live tension in Ring-/Module-LWE. No subexponential attack on them is known, but the extra structure means the worst-case-to-average-case guarantee is weaker than for unstructured LWE, and prudent designs (Kyber among them) keep the structure modest — small modules rather than a single huge ring — precisely to stay away from the SIKE cliff.

35.5 A toy lattice cryptosystem you can break by hand

To feel where the security lives, here is a deliberately tiny LWE encryption and the brute-force attack on it. **This is for learning only — the parameters are minuscule and instantly breakable, and you must never use anything like it to protect real data.**

Implement it

```python
# TOY LWE encryption of a single bit. LEARNING ONLY -- INSECURE.
# Real schemes (ML-KEM/Kyber) use n>=256, structured rings,
# the Fujisaki-Okamoto transform, and constant-time code.
import random, itertools

q = 97          # modulus (tiny on purpose)
n = 6           # secret dimension (tiny on purpose)
m = 12          # number of LWE samples in the public key

def dot(u, v):  return sum(a*b for a, b in zip(u, v)) % q
def small():    return random.randint(-1, 1)   # narrow error

def keygen():
    s = [random.randrange(q) for _ in range(n)]            # secret
    A = [[random.randrange(q) for _ in range(n)] for _ in range(m)]
    b = [(dot(A[i], s) + small()) % q for i in range(m)]   # noisy
    return (A, b), s                                       # pk, sk

def encrypt(pk, bit):
    A, b = pk
    r = [random.randint(0, 1) for _ in range(m)]           # subset
    u = [sum(r[i]*A[i][j] for i in range(m)) % q for j in range(n)]
    v = (sum(r[i]*b[i] for i in range(m)) + bit*(q//2)) % q
    return u, v

def decrypt(sk, ct):
    u, v = ct
    w = (v - dot(sk, u)) % q          # = <e,r> + bit*(q//2)
```

```
    if w > q//2: w -= q              # fold into [-q/2, q/2)
    return 1 if abs(w) > q//4 else 0

pk, sk = keygen()
for bit in (0, 1):
    assert decrypt(sk, encrypt(pk, bit)) == bit
print("toy LWE round-trips correctly")

# ---- the attack: brute force the secret because n,q are tiny ----
def attack(pk):
    A, b = pk
    for s in itertools.product(range(q), repeat=n):   # q**n guesses
        if all(abs(((dot(A[i], s) - b[i] + q//2) % q) - q//2) <= 1
               for i in range(m)):
            return s    # a secret consistent with small noise
# attack(pk) recovers sk because q**n (here 97^6) is small enough.
# At n=256 this loop has q**256 iterations -- utterly infeasible.
```

The attack is the lesson. With $n = 6$ and $q = 97$ there are $97^6 \approx 8.3 \times 10^{11}$ candidate secrets — a few minutes of CPU, or far less with smarter lattice reduction. Every defense in real lattice crypto is about pushing that number out of reach *while keeping the legitimate error budget intact*: bigger n, careful error distributions, and structure (rings/modules) so the keys stay small even as n climbs into the hundreds.

35.6 Hash-based signatures: security from the primitive you trust most

Lattices are efficient but young; their hardness assumptions are a couple of decades old. There is one corner of post-quantum cryptography that asks you to trust *nothing new*: hash-based signatures. Their only assumption is that a hash function is a good one-way / collision-resistant function — and Grover merely halves that, so a 256-bit hash is fine. If you believe in SHA-256 (or SHA-3), you believe in these signatures, full stop. The cost is awkwardness: state, size, and a limited number of signatures per key.

35.6.1 Lamport: signing one bit, then one message, with a hash

Leslie Lamport's 1979 one-time signature is the seed of the whole family. To sign a single bit:

- **Secret key:** two random values x_0, x_1.
- **Public key:** their hashes $y_0 = \mathrm{H}(x_0),\ y_1 = \mathrm{H}(x_1)$, published.
- **Sign bit b:** reveal x_b.
- **Verify:** check $\mathrm{H}(x_b) = y_b$.

Revealing the preimage of a hash that only you knew is the signature; preimage resistance is what stops a forger. To sign an ℓ-bit message digest, use ℓ such pairs: publish 2ℓ hashes, and for each message bit reveal the matching half of the corresponding pair.

Example 35.3 (Lamport on 4 bits)**.** Suppose the message digest is 1011. The secret key is four pairs $(x_{i,0}, x_{i,1})$ for $i = 1, \ldots, 4$; the public key is the eight hashes $y_{i,b} = \mathrm{H}(x_{i,b})$. As Figure 35.4 shows, the signature for 1011 is $(x_{1,1}, x_{2,0}, x_{3,1}, x_{4,1})$ — pick the 1-half, the 0-half, the 1-half, the 1-half. The verifier hashes each revealed value and checks it equals the published hash for that bit position and bit value. A forger who wants to sign a *different* digest must produce a preimage of some $y_{i,b}$ they were never shown — exactly a preimage attack on the hash.

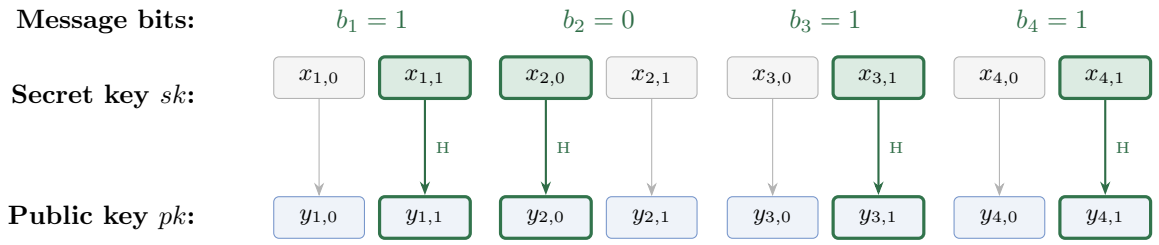


Figure 35.4: Lamport one-time signature on the 4-bit digest 1011 (Example 35.3). For each bit b_i, the signer reveals the highlighted secret preimage x_{i,b_i} while keeping the other half secret; the verifier hashes each revealed value and checks it against the corresponding public hash y_{i,b_i}.

Common trap

"One-time" is not a slogan, it is a hard rule. Sign two different messages with one Lamport key and you leak both halves of some pair: where the two digests differ in bit i, you have revealed both $x_{i,0}$ and $x_{i,1}$. An attacker can then mix-and-match revealed halves to forge signatures on digests that combine bits from the two you signed. Reusing a one-time key is the hash-signature analogue of reusing a one-time-pad key or an ECDSA nonce: a catastrophic, total break, not a gradual erosion. This is why the entire engineering effort in this family is about safely getting *many* signatures out of *many* one-time keys.

35.6.2 From one signature to a tree: Merkle, XMSS, SPHINCS+

One Lamport key signs once. To sign 2^h messages, generate 2^h one-time key pairs and compress all their public keys into a single short public key using a *Merkle tree*: hash the public keys into the leaves, hash pairs upward, and publish only the root. A signature now carries (a) a one-time signature from one leaf, plus (b) the *authentication path* — the sibling hashes along the route from that leaf to the root — so the verifier can recompute the root and confirm the leaf belongs to your tree.

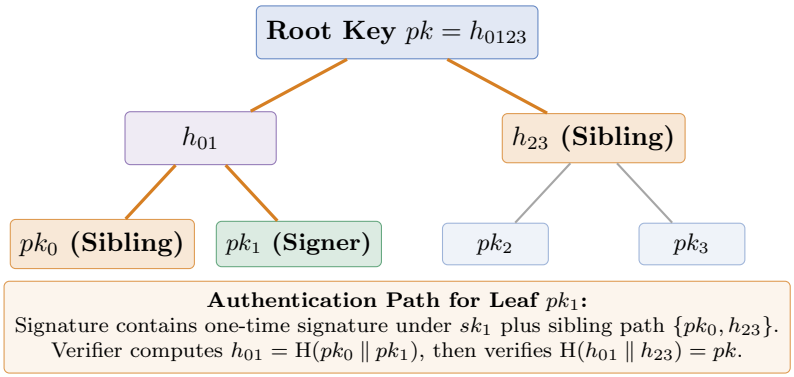


Figure 35.5: Merkle tree signature hierarchy. To authenticate message signatures across 2^h one-time Lamport key pairs, public keys are formed into a Merkle hash tree. When leaf pk_1 signs a message, the signer provides the one-time signature plus the authentication path (pk_0 and h_{23}). The verifier re-computes hashes up to the root, validating authenticity without trusting any public key except the single published root.

The remaining wrinkle is *state*: you must never reuse a leaf, so the signer has to remember which leaves are spent. A signer that forgets — restored from a backup, cloned to two servers — can reuse a leaf and break itself. **XMSS** and **LMS** (RFC 8391, RFC 8554) are the standardized stateful tree schemes; they are excellent where you can guarantee state, such as signing firmware images from a controlled build server. **SPHINCS+** — standardized as **SLH-DSA**, FIPS 205 — removes the state by picking the one-time leaf *pseudorandomly from the message* and stacking trees of trees (a hypertree) so the chance of a damaging collision is negligible without bookkeeping. The price is size: SLH-DSA signatures run tens of kilobytes. You pay in bytes for the privilege of trusting only a hash.

In the real world

Stateful hash-based signatures are already the recommended tool for *firmware and software signing*, where the threat is precisely "harvest now, forge later" over device lifetimes of decades, and where a controlled signing server can manage state safely. NIST's SP 800-208 approves XMSS and LMS for exactly this. SLH-DSA is the conservative fallback for general signatures when you want maximal confidence and can tolerate large signatures — a belt-and-suspenders choice for long-lived roots of trust, such as the key that signs your other keys.

35.7 Migration, hybrids, and timelines

35.7.1 The standardized algorithms

As of 2024, the headline NIST standards are: **FIPS 203 (ML-KEM / Kyber)** for key establishment, **FIPS 204 (ML-DSA / Dilithium)** and **FIPS 205 (SLH-DSA / SPHINCS+)** for signatures, with **FN-DSA (Falcon)** following. NIST also ran a separate "on-ramp" for additional signatures to diversify away from lattices, and selected code-based and other KEM candidates (such as the conservative, large-key Classic McEliece) for niche use where key size matters less than decades of cryptanalytic confidence. The portfolio is deliberately not all-lattice: if a structural break hits lattices, the world wants a non-lattice scheme already on the shelf.

35.7.2 Hybrid key exchange: belt and suspenders

Deploying a brand-new assumption (Module-LWE) as your *only* protection is itself a risk — young schemes have bugs and parameter mistakes, and a structural break is conceivable. The standard mitigation is a *hybrid*: run a classical key exchange (X25519, say) *and* a post-quantum one (ML-KEM) and combine both shared secrets through a key-derivation function:

$$K = \mathrm{KDF}\big(\,\text{X25519 secret} \,\|\, \text{ML-KEM secret}\,\big).$$

The session key is safe if *either* ingredient is safe. A quantum computer breaks X25519 but not ML-KEM; a future cryptanalytic break of ML-KEM still leaves X25519, which holds against every classical adversary. You only lose if both fall, which is the conjunction of two independent failures (Figure 35.6). This is exactly what shipped: TLS 1.3 deployments rolled out the hybrid `X25519MLKEM768` group across major browsers and servers starting in 2024, so a large fraction of web traffic already negotiates a post-quantum-protected handshake today.

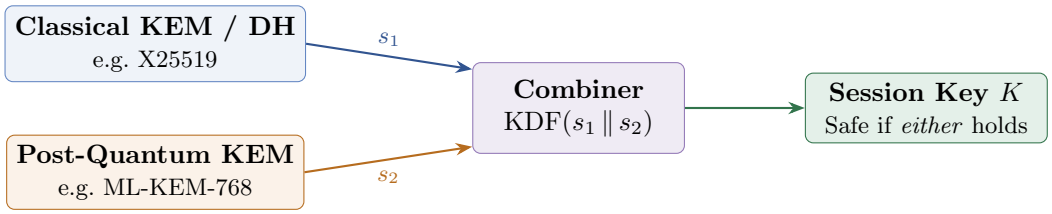


Figure 35.6: Hybrid key exchange (`X25519MLKEM768`). Classical and post-quantum shared secrets s_1 and s_2 are fed together into a key-derivation function, so the resulting session key K remains indistinguishable from random as long as at least one of the two underlying problems stays hard.

In the real world

The hybrid is live in places you use daily. Chrome, Firefox, and Cloudflare enabled `X25519MLKEM768` (and its predecessor `X25519Kyber768`) for TLS, so a sizable share of HTTPS connections are already quantum-resistant on the key-exchange side. Signal deployed PQXDH — a post-quantum extension of its X3DH key agreement that mixes a Kyber/ML-KEM secret into the initial handshake — so the *forward* secret of a Signal session resists future quantum decryption even though the long-term identity signatures are still classical. SSH (OpenSSH) defaults to a hybrid `sntrup761x25519` key exchange. The pattern is identical everywhere: protect the ephemeral key agreement first, because that is what "harvest now, decrypt later" targets; migrate signatures and certificates more slowly, because forging a signature requires the quantum computer to exist *now*, not later.

Focus

Notice the priority ordering, and why it is correct. *Confidentiality* is urgent because of store-and-decrypt: traffic recorded today is exposed the day the machine arrives, so the key exchange must go post-quantum first. *Authentication* (signatures, certificates) is less urgent because a forged signature is only useful at the moment of the connection — you cannot retroactively forge yesterday's handshake. An attacker cannot wait for a future quantum computer to impersonate a server in a session that already happened. So migrate KEMs now; migrate signature infrastructure (which is bigger and slower, given certificate chains and root programs)

on a longer schedule.

35.7.3 Timelines, without hand-waving

How long until a cryptographically relevant quantum computer (CRQC) exists? No one knows, and anyone who gives you a confident year is selling something. What we can say with precision is the engineering gap. Today's quantum processors have on the order of hundreds to low thousands of noisy physical qubits with error rates around 10^{-3}. Breaking RSA-2048 with Shor needs a few thousand *logical* qubits, and error correction inflates each logical qubit to roughly 10^3–10^4 physical qubits, putting the requirement near *millions* of high-quality physical qubits running stably for hours. That is many orders of magnitude beyond current hardware, and the gap is dominated by error rates, not raw qubit counts — which is why "qubit count" headlines are a poor proxy for cryptographic danger.

Estimates from serious groups cluster in the range of one to three decades, with wide error bars and a real tail probability of "sooner" if error correction improves faster than expected. The defender's calculus does not actually depend on resolving that uncertainty. Mosca's inequality states it cleanly: if X is how long your data must stay confidential, Y is how long migration takes, and Z is the time until a CRQC exists, then you have a problem whenever

$$X + Y > Z.$$

For data that must stay secret twenty years, with a migration that realistically takes five to ten years across a large organization, $X + Y$ already reaches into the late 2040s — comfortably inside the plausible range for Z. That is why migration is a present-day program, not a future one, even though the breaking machine is not here. The threat is real today not because the computer exists, but because the secrets and the recordings do.

Common trap

Two failure modes bracket the sensible path. *Complacency*: "no quantum computer exists, so do nothing" ignores store-and-decrypt and the multi-year reality of migration; by the time the machine is announced it is far too late to start. *Panic*: ripping out classical crypto for unvetted post-quantum schemes deployed alone has already caused real incidents — early non-standard parameter sets, implementation bugs, and at least one reference implementation with a timing side channel in the very rejection-sampling step that was supposed to add safety. The measured answer is the hybrid: deploy post-quantum *alongside* classical, using standardized parameters and audited constant-time code, so a flaw in the new scheme cannot undo the protection of the old one while the new one matures.

35.8 Where this leaves us

Quantum computers threaten public-key cryptography in a specific, structural way: Shor's algorithm dissolves the abelian hidden-period problems — factoring and discrete log — that RSA and elliptic curves are built on, and no key size repairs that. Grover's algorithm only dents symmetric cryptography, and doubling the key size (AES-256, SHA-384/512) restores the margin. The replacements avoid abelian period structure: lattice problems (SVP/CVP, packaged as LWE and its ring/module

variants) power the efficient new standards ML-KEM and ML-DSA, while hash-based signatures (XMSS, LMS, SLH-DSA) offer maximal confidence at the cost of size and, sometimes, state. The migration is already underway in TLS, Signal, and SSH, run as hybrids so that a stumble in the young schemes cannot cost you the protection of the old ones. The machine that forces all this is not here. The data it would read is.

With public-key primitives and their quantum-resistant successors in hand, Chapter 36 explores how cryptographic hashes, digital signatures, and consensus protocols combine to build decentralized state in blockchains.

35.9 Exercises

Exercise 35.1 (Shor by hand)**.** Let $N = 21$ and $a = 2$. Compute the order r of 2 modulo 21 by listing powers. Verify r is even and that $2^{r/2} \not\equiv \pm 1$, then compute $\gcd(2^{r/2} - 1, 21)$ and $\gcd(2^{r/2} + 1, 21)$ to factor 21. Repeat with $a = 5$ and report whether the lucky conditions hold.

Exercise 35.2 (When Shor's luck fails)**.** Find an a coprime to $N = 15$ whose order modulo 15 is even such that $a^{r/2} \equiv -1 \pmod{15}$ (Hint: try $a = 14$), so that the order-finding fails to yield a non-trivial factor. Explain in one sentence why this condition blocks the construction, and what the algorithm does in that case.

Exercise 35.3 (Grover arithmetic)**.** A cipher uses a 112-bit key. Estimate the number of operations a classical brute force and an ideal Grover search each require, expressed as powers of two. If a designer wants 128 bits of security against a quantum adversary using Grover, what is the smallest key length they should choose, and why is 256 the usual round number?

Exercise 35.4 (Why the error matters)**.** In the LWE setup, suppose all errors e_i are exactly 0. Given n samples $(\mathbf{a}_i, b_i)$ with $\mathbf{a}_i \in \mathbb{Z}/q\mathbb{Z}^n$, describe precisely how to recover the secret $\mathbf{s}$ in polynomial time, and state how large n must be relative to the dimension. Then explain why adding even a small random error to each b_i defeats your method.

Exercise 35.5 (Decrypt a tiny LWE ciphertext)**.** Using the instance of Example 35.2 ($q = 17$, $\mathbf{s} = (4, 1, 11)$), an encryptor sends $\mathbf{u} = (2, 2, 2)$ and $v = 14$ for some bit $\mu \in \{0, 1\}$. Compute $v - \langle \mathbf{s}, \mathbf{u} \rangle \bmod 17$, fold the result into $[-8, 8]$, and decide whether $\mu = 0$ (result near 0) or $\mu = 1$ (result near $\lfloor 17/2 \rfloor = 8$).

Exercise 35.6 (Break a two-time Lamport key)**.** A Lamport key signs 3-bit digests, with secret pairs $(x_{i,0}, x_{i,1})$ for $i = 1, 2, 3$. The owner mistakenly signs digest 010 and then digest 100 with the same key. List exactly which secret values $x_{i,b}$ an attacker now holds, and determine which two additional digests (besides 010 and 100) the attacker can forge valid signatures for.

Exercise 35.7 (Merkle authentication path)**.** A Merkle-tree signature scheme has 8 leaves (one-time public keys $pk_0, \ldots, pk_7$) and a tree of height 3. The signer uses leaf pk_5. List, in order from leaf to root, exactly which sibling hashes form the authentication path, and write out the sequence of hash computations the verifier performs to recompute the root.

Exercise 35.8 (Hybrid security argument)**.** A hybrid key exchange derives its session key as $K = \mathrm{KDF}(s_1 \,\|\, s_2)$, where s_1 comes from X25519 and s_2 from ML-KEM. Argue carefully that an adversary who can break X25519 (e.g. with a quantum computer) but not ML-KEM learns nothing about K, assuming the KDF is a good pseudorandom function. Then state the single condition under which the hybrid fails.

Exercise 35.9 (Mosca's inequality)**.** Your organization stores medical records that must remain confidential for 25 years. A full cryptographic migration is estimated to take 8 years. Suppose expert consensus puts the arrival of a cryptographically relevant quantum computer at 20 years out. Using $X + Y > Z$, determine whether you have a problem, and compute the latest year you could have started migration while still protecting all data through its confidentiality lifetime (take "now" as year 0).

Exercise 35.10 (Spot the structure)**.** For each problem, state whether Shor's algorithm solves it in polynomial time, and give a one-line reason based on whether it is an abelian hidden-period/subgroup problem: (a) factoring an RSA modulus; (b) elliptic-curve discrete log; (c) finding the shortest vector in a 400-dimensional lattice; (d) inverting SHA-256; (e) computing a discrete log in $(\mathbb{Z}/p\mathbb{Z})^*$.

Exercise 35.11 (Decryption failure as a leak)**.** Explain why a lattice KEM with a non-negligible decryption-failure probability is dangerous even if each individual failure seems harmless. Sketch how an attacker who can submit ciphertexts and observe whether decryption succeeded could, over many queries, learn information about the secret key. (This is why standardized parameters drive the failure rate below about 2^{-130}.)

Chapter 36

The Cryptography Behind Blockchains

A blockchain is not a cryptographic primitive. It is a database — an append-only ledger — that a crowd of mutually suspicious strangers maintains without a referee. What lets that crowd agree on a single history, and lets any reader check that history without trusting anyone, is cryptography: hash functions chain the records together, Merkle trees let one number stand in for millions of transactions, digital signatures prove who authorized what, and a hash puzzle (proof of work) makes rewriting the past expensive. Almost every tool in this book shows up. Nothing here is new; what is new is the *assembly*.

This chapter takes the primitives apart and shows how they snap together. We will hash a tiny block by hand, build a four-leaf Merkle tree and verify one leaf with a short proof, derive a Bitcoin-style address from a public key, sign a toy transaction with ECDSA, and mine a block against a deliberately easy target so you can watch the nonce search happen. Along the way we will be exact about a point that gets lost in hype: the cryptography secures *integrity and authorization*, and nothing else. It does not make a coin valuable, a contract legal, or a key un-loseable. Confusing those is how people lose money.

36.1 The job: agreement without a referee

Start with the Alice/Bob/Eve story, adapted. Alice wants to pay Bob ten coins. In a bank, a trusted server debits Alice's balance and credits Bob's; the server's word is final. Take the server away and replace it with a public network anyone can join, including Eve, who would happily spend the same coin twice or rewrite yesterday's payments. We need a way for everyone to agree on *one* ordered list of transactions — the ledger — such that:

- only the owner of a coin can spend it (*authorization*),
- once a transaction is buried in the history, changing it is infeasible (*integrity / immutability*),
- any newcomer can download the ledger and verify it from scratch, trusting no single participant (*public verifiability*).

Intuition

Picture a shared notebook passed around a table of people who do not trust each other. Three rules make it work. (1) Each entry is signed by its author with a signature nobody can forge, so you cannot spend my coins. (2) Each page carries a fingerprint of the previous page, so tearing out or editing an old page changes every fingerprint after it — the tampering is loud, not silent. (3) Writing a new page costs real effort (solving a puzzle), so an attacker who wants to rewrite history must redo all that effort faster than the rest of the table combined. None of these three is a new invention; they are signatures, hash chaining, and proof of work. The blockchain is the way they interlock.

The cryptography delivers (1) and (2) outright. Point (3), and the "faster than everyone combined" part, is *consensus* — an economic and game-theoretic argument layered on top of the crypto, not a theorem about hash functions. Keep that seam in mind; we will return to it when we separate what the math guarantees from what it does not.

History

The chaining-by-hash idea predates Bitcoin by nearly two decades. In 1990–91 Stuart Haber and W. Scott Stornetta published a scheme for timestamping digital documents by linking each document's hash to the previous one, so that the position of a record in the chain fixed its time of creation; they later folded in Merkle trees (Ralph Merkle's hash-tree patent dates to 1979) to batch many documents per link. In 2008 a pseudonymous author writing as Satoshi Nakamoto combined hash chaining, Merkle trees, ECDSA signatures, and a proof-of-work consensus rule (drawing on Adam Back's 1997 Hashcash) into Bitcoin, whose whitepaper cites Haber and Stornetta directly. The novelty was not any single primitive but the consensus rule that let an open, permissionless set of participants agree on which chain is the real one. That design sparked long-running debates over proof-of-work's energy cost versus proof-of-stake, and over how to scale throughput without making full-node verification too expensive for ordinary users.

36.2 Hashes: the glue

Everything rests on a collision-resistant hash function (Chapter 14). Bitcoin uses SHA-256, almost always doubled — $\mathrm{H}(x) = \mathrm{SHA256}(\mathrm{SHA256}(x))$, written SHA-256d — and RIPEMD-160 for addresses; Ethereum uses Keccak-256. The exact choice matters less than the properties we lean on. Recall them, because each blockchain mechanism relies on a specific one:

- **Preimage resistance**: given a digest y, you cannot find any x with $\mathrm{H}(x) = y$. This is what makes proof of work a search rather than an inversion.
- **Second-preimage resistance**: given x, you cannot find $x' \neq x$ with the same digest. This is what stops you from swapping a block's contents while keeping its fingerprint.
- **Collision resistance**: you cannot find *any* pair $x \neq x'$ that collide. Merkle trees and commitments need this; a collision would let someone open a commitment two ways.

A hash is also, informally, a *binding commitment*: publishing $\mathrm{H}(x)$ pins down x without revealing it, and you cannot later claim it was some other x'. That single sentence is the engine of the whole

construction. A block header commits to its transactions; a chain of headers commits to all of history; an address commits to a public key.

Connection

Two facts from Chapter 14 carry the load here. First, the birthday bound: finding a collision in an n-bit hash costs about $2^{n/2}$ work, not 2^n, which is why blockchains use 256-bit digests — 2^{128} is the security floor, and that is why a 160-bit address is considered marginal but a 128-bit one would be reckless. Second, modular-exponentiation and elliptic-curve costs from the algorithms companion: signing and verification are a handful of curve operations, cheap enough to do per transaction, while a brute-force key search is 2^{128} curve operations, infeasible. The whole edifice is a bet that 2^{128} is and stays out of reach.

36.3 Binding a block to its predecessor

A *block* bundles a batch of transactions plus a small *header*. The header is the part that gets hashed and chained. A simplified Bitcoin header contains:

- the hash of the *previous* block's header (`prev`),
- the *Merkle root* of this block's transactions (`root`, Section 36.4),
- a *timestamp*,
- the difficulty *target* (Section 36.7),
- a *nonce* — the number miners vary to solve the puzzle.

The block's identity is ID = H(header). Because the header includes `prev`, each block's ID depends on the one before it, and that one on the one before *it*, all the way back to the first block (the *genesis* block, which has no predecessor and hardcodes `prev` to all zeros).

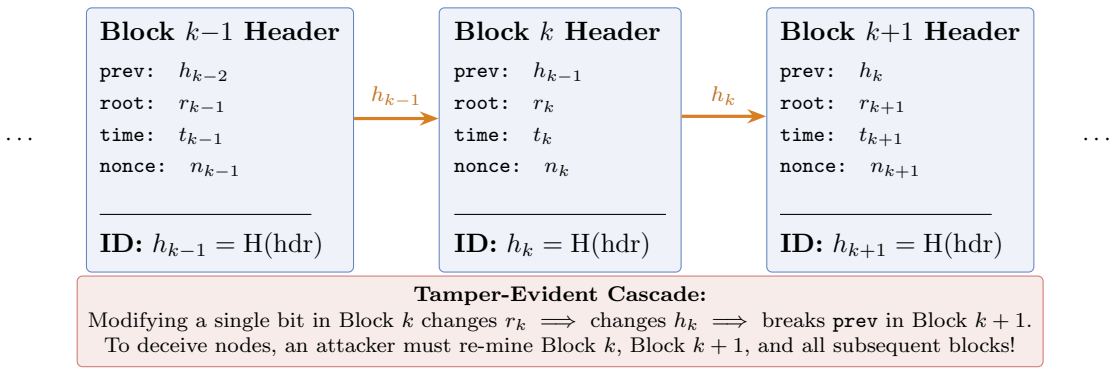


Figure 36.1: Cryptographic block chaining. Each header commits to its transactions via a Merkle root and incorporates the exact cryptographic digest of the prior header in its `prev` field. Any retrospective alteration creates an immediate mismatch that invalidates every succeeding block.

Why it works

Why does chaining make the past tamper-evident? Suppose Eve wants to alter one transaction inside block k. That changes block k's Merkle root, hence its header, hence its ID h_k. But block $k{+}1$ stored the *old* h_k in its `prev` field. For the chain to stay valid, Eve must update block $k{+}1$'s `prev` to the new h_k — which changes block $k{+}1$'s ID — which forces an update to block $k{+}2$, and so on to the tip. By second-preimage resistance she cannot find a different block k body that keeps h_k the same. So a one-transaction edit cascades into re-deriving every subsequent block. With proof of work each of those re-derivations costs the puzzle's worth of computation, which is the point: integrity is cheap to *check* and expensive to *forge*.

Let us make this concrete with absurdly small numbers so you can do the hashing in spirit, if not literally. We will use a toy hash $h(x) = \mathrm{SHA256}(x)$ but only look at the first few hex digits. Take three blocks whose "bodies" are just strings.

Example 36.1 (A three-block toy chain)**.** Let block 0 be the genesis: body `"genesis"`, `prev` $= 0$. Compute $h_0 = \mathrm{H}(\texttt{"0"} \,\|\, \texttt{"genesis"})$. Block 1 has body `"Alice->Bob 10"` and `prev` $= h_0$, giving $h_1 = \mathrm{H}(h_0 \,\|\, \texttt{"Alice->Bob 10"})$. Block 2 has body `"Bob->Carol 4"` and `prev` $= h_1$. Now if Eve rewrites block 1's body to `"Alice->Eve 10"`, recomputing gives a new $h_1' \neq h_1$. Block 2's stored `prev` is still the old h_1, so a verifier recomputing block 2's expected `prev` sees a mismatch and rejects the chain. Eve must also recompute h_2, and if there were a block 3, that too, and so on. The implement-it box below runs exactly this.

36.4 Merkle trees: one root for a million transactions

A block can hold thousands of transactions. We do not want the header to grow with them, and we want a lightweight client — say a phone wallet — to be able to prove "transaction T is in block k" while downloading only the small header, not the whole block. Both needs are met by a *Merkle tree*, a binary tree of hashes named for Ralph Merkle.

36.4.1 Construction

Take the list of transactions $T_0, T_1, \ldots, T_{m-1}$. Hash each one to get the *leaves* $\ell_i = \mathrm{H}(T_i)$. Then pair the leaves and hash each pair to get the next level up: $\mathrm{H}(\ell_0 \,\|\, \ell_1)$, $\mathrm{H}(\ell_2 \,\|\, \ell_3)$, and so on. Repeat, halving the count each level, until one hash remains: the *Merkle root*. If a level has an odd number of nodes, duplicate the last one (Bitcoin's convention) so it pairs with itself.

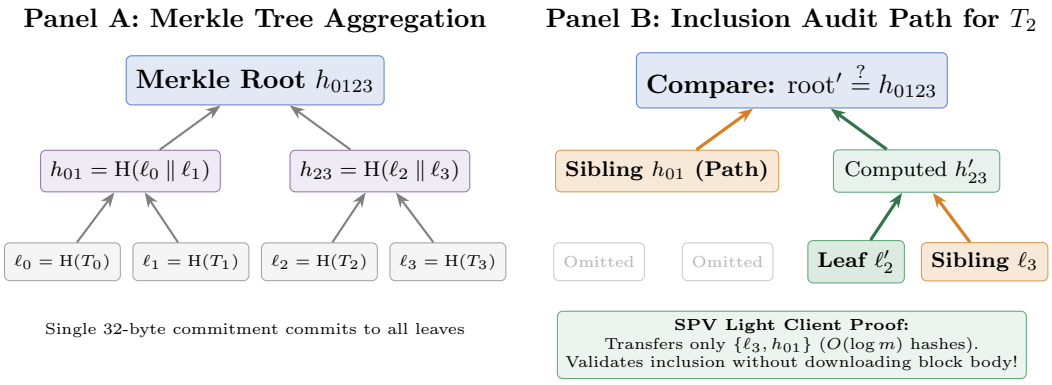


Figure 36.2: Merkle tree commitment and verification. Panel A depicts the full tree where m transaction leaves are paired and hashed up to the 32-byte Merkle root stored in the block header. Panel B illustrates an SPV inclusion proof for transaction T_2: the light client receives only the audit siblings ℓ_3 and h_{01} (shaded orange), recomputes $h'_{23} = \mathrm{H}(\ell'_2 \,\|\, \ell_3)$ and $\mathrm{root}' = \mathrm{H}(h_{01} \,\|\, h'_{23})$, and confirms equality with the known block root.

36.4.2 Membership proofs

The payoff is the *Merkle proof* (also called an inclusion or audit proof). To convince someone that T_2 is one of the leaves under a root they already trust, you do not send all the transactions. You send T_2 plus the *sibling hashes* along the path from T_2's leaf up to the root — one hash per level. The verifier recomputes the path and checks it lands on the trusted root.

For the tree in Figure 36.2, the proof for T_2 is the pair $(\ell_3,\ h_{01})$ together with which side each sits on. The verifier computes

$$\ell'_2 = \mathrm{H}(T_2), \quad h'_{23} = \mathrm{H}(\ell'_2 \,\|\, \ell_3), \quad \mathrm{root}' = \mathrm{H}(h_{01} \,\|\, h'_{23}),$$

and accepts iff $\mathrm{root}' = \mathrm{root}$. Two hashes sent, two hashes computed, for a four-leaf tree; in general a tree of m leaves needs a proof of only $\lceil \log_2 m \rceil$ hashes. A block with a million transactions yields a 20-hash proof — about 640 bytes — versus tens of megabytes for the full block.

Why it works

Why is the proof sound — why can't a prover fake membership for a transaction that is not in the tree? Because forging a valid path to a fixed root requires a hash collision. Suppose Eve wants the verifier to accept a bogus $T_2^\star$. She must supply sibling hashes so that the recomputed root equals the real root. But the real root was the hash of the real $h_{01} \,\|\, h_{23}$. To hit it with a different left-or-right input is a (second-)preimage break on the top hash; to instead make h'_{23} equal the real h_{23} from a different ℓ'_2 is a collision on that node's hash. Collision resistance forbids both. So under a hash we believe is collision resistant, the only way to produce an accepting proof for $T_2^\star$ is for $T_2^\star$ to genuinely be the leaf — the tree is a *binding* commitment to its entire leaf set.

Example 36.2 (A four-leaf tree, worked with small hex)**.** We will use a 1-byte toy hash $H(x) = \mathrm{SHA256}(x) \bmod 256$, written as two hex digits, just to make the arithmetic visible (a real tree uses the full 256-bit output). Suppose the four transactions hash to leaves

$$\ell_0 = \texttt{a3}, \quad \ell_1 = \texttt{7c}, \quad \ell_2 = \texttt{1f}, \quad \ell_3 = \texttt{d8}.$$

Then $h_{01} = H(\texttt{a3} \,\|\, \texttt{7c})$, say $= \texttt{4b}$, and $h_{23} = H(\texttt{1f} \,\|\, \texttt{d8})$, say $= \texttt{e0}$, and the root is $H(\texttt{4b} \,\|\, \texttt{e0})$, say $= \texttt{29}$. The proof for T_2 is $\{\ell_3 = \texttt{d8}$ (right), $h_{01} = \texttt{4b}$ (left)$\}$. A verifier holding only the root `29` and the transaction T_2 computes $\ell_2 = H(T_2) = \texttt{1f}$, then $h'_{23} = H(\texttt{1f} \,\|\, \texttt{d8}) = \texttt{e0}$, then $\text{root}' = H(\texttt{4b} \,\|\, \texttt{e0}) = \texttt{29}$, matches, and accepts — never having seen T_0, T_1, T_3.

In the real world

This is exactly how Bitcoin's *Simplified Payment Verification* (SPV) clients work, the design Nakamoto called out in the whitepaper. A phone wallet stores only the 80-byte block headers (a few dozen megabytes for the whole chain) and asks a full node for a Merkle proof that its transaction is in a block. It checks the proof against the header's root and confirms the header is buried under enough later blocks. It never downloads the multi-hundred-gigabyte full chain. Ethereum pushes the idea further with *Merkle–Patricia tries* that commit not just to a list of transactions but to the entire world state — every account balance and contract storage slot — so a light client can be given a proof of any single account's balance. Certificate Transparency logs (Chapter 24) and the `git` version-control system are Merkle trees too: a git commit hash is a root over the whole repository tree.

Common trap

Merkle trees have bitten implementers. Bitcoin's tree duplicates the last node when a level has an odd count, and this created *CVE-2012-2459*: a block could be padded with a duplicate transaction so two different transaction lists produced the *same* root, letting an attacker craft a valid-looking-but-invalid block that split nodes' views. The deeper lesson is that a Merkle root only commits to the *multiset of leaf hashes in their tree positions*, not to a canonical list; if your encoding lets two distinct lists map to one tree, you have handed an attacker a collision the hash function never gave them. Always domain-separate leaf hashes from internal-node hashes (prefix a leaf with a `0x00` byte and an internal node with `0x01`) so a leaf can never be reinterpreted as an internal node — a *second-preimage* attack on naive Merkle trees that the prefix trick defeats.

Implement it

A toy chain with Merkle roots and a tamper check. **Learning only — this omits real serialization, signatures, consensus, and uses Python's slow hashing; never run a real ledger on it.**

```python
import hashlib

def H(b: bytes) -> bytes:
    # SHA-256d, Bitcoin style. Domain-separate leaves vs nodes in real code.
    return hashlib.sha256(hashlib.sha256(b).digest()).digest()

def merkle_root(txs):
    level = [H(b"\x00" + t.encode()) for t in txs]   # 0x00 = leaf tag
    if not level:
        return H(b"")
    while len(level) > 1:
        if len(level) % 2:           # odd: duplicate last (Bitcoin convention)
            level.append(level[-1])
```

```
        level = [H(b"\x01" + level[i] + level[i+1])   # 0x01 = internal tag
                 for i in range(0, len(level), 2)]
    return level[0]

def block_id(prev, root, nonce):
    return H(prev + root + nonce.to_bytes(8, "big"))

# Build a 3-block chain.
chain = []
prev = bytes(32)                                      # genesis prev = 32 zero bytes
for txs in [["genesis"], ["Alice->Bob 10"], ["Bob->Carol 4", "Dave->Eve 1"]]:
    root = merkle_root(txs)
    bid  = block_id(prev, root, nonce=0)
    chain.append({"prev": prev, "root": root, "txs": txs, "id": bid})
    prev = bid

def verify(chain):
    prev = bytes(32)
    for b in chain:
        if b["prev"] != prev:                         # broken link?
            return False
        if b["root"] != merkle_root(b["txs"]):        # tampered body?
            return False
        if b["id"] != block_id(b["prev"], b["root"], 0):
            return False
        prev = b["id"]
    return True

print(verify(chain))                  # True
chain[1]["txs"] = ["Alice->Eve 10"]   # Eve rewrites history
print(verify(chain))                  # False: root no longer matches, link snaps
```

36.5 Keys, addresses, and wallets

A coin is controlled by whoever can produce a valid signature under a particular key. So "owning" bitcoin means holding a private key. The chain of derivations from key to address is pure cryptography, and getting it right is what separates a wallet from a way to lose money.

36.5.1 From private key to public key

Bitcoin and Ethereum both use the elliptic curve `secp256k1` (Chapter 21), defined over the prime field $\mathbb{F}_p$ with

$$p = 2^{256} - 2^{32} - 977, \qquad y^2 = x^3 + 7 \pmod{p},$$

and a fixed base point G of large prime order $n \approx 2^{256}$. A private key is a random integer d with $1 \le d < n$. The public key is the curve point

$$Q = dG,$$

G added to itself d times under the curve's group law. Recovering d from Q is the elliptic-curve discrete log problem, believed to cost about 2^{128} operations — the security floor of the whole system. The private key is 32 bytes of entropy; lose it and the coins are gone, leak it and they are stolen. There is no reset.

Example 36.3 (A microscopic curve to fix the idea)**.** `secp256k1`'s numbers are too big to trace, so work over $\mathbb{F}_{17}$ with $y^2 = x^3 + 7$. Checking points, $(1, 5)$ is on the curve since $5^2 = 25 \equiv 8$ and $1^3 + 7 = 8$. Take $G = (1, 5)$. Using the curve addition rules from Chapter 21, doubling gives $2G$, tripling $3G$, and so on, cycling through the group of points. If your private key is $d = 4$, your public key is $Q = 4G$, computed by two doublings. An attacker who sees Q but not d would, on this tiny curve, just try all d — there are only a handful — which is exactly why real curves use $n \approx 2^{256}$: the only known attack is to try (cleverly) all d, and there are too many.

36.5.2 From public key to address

You do not publish Q directly as your payment handle; you publish a *hash* of it, called an *address*. Figure 36.3 shows the full pipeline. Bitcoin's legacy "P2PKH" address is

$$\text{address} = \text{Base58Check}\big(\,\texttt{0x00} \,\|\, \text{RIPEMD160}(\text{SHA256}(Q))\,\big),$$

that is, SHA-256 then RIPEMD-160 (jointly nicknamed HASH160) to get a 20-byte fingerprint, prefixed with a version byte, then Base58Check-encoded with a 4-byte checksum so typos do not send coins into the void. Ethereum addresses are the last 20 bytes of Keccak256(Q).

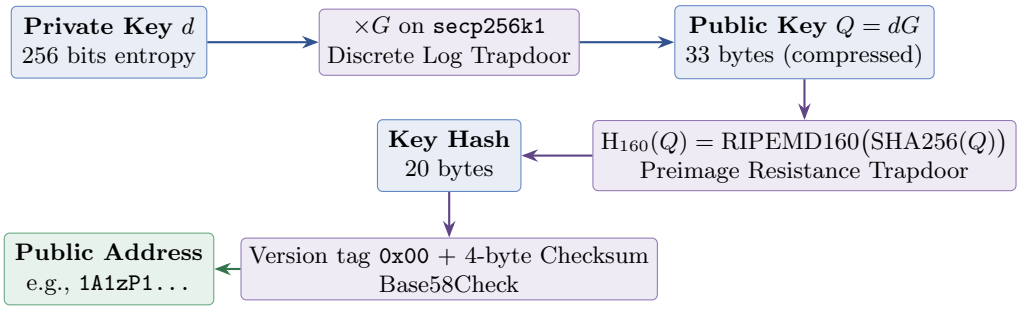


Figure 36.3: The one-way derivation pipeline from private key to payment address. Each operational stage is mathematically irreversible: curve multiplication conceals d under the elliptic-curve discrete log problem, and the dual-hash (H_{160}) pipeline conceals Q behind preimage resistance until the coins are spent.

Why it works

Why hash the public key instead of using it directly? Three reasons. (1) *Brevity*: 20 bytes is shorter than a 33-byte compressed public key, friendlier to type and to print as a QR code. (2) *A second wall*: if elliptic-curve discrete log were ever broken, an address whose public key has never been revealed still hides Q behind a preimage-resistant hash — the public key only becomes visible *when you spend*, so an unspent, never-reused address has a hash-based backstop. This is the practical case against address reuse. (3) *Checksumming*: the Base58Check wrapper carries a checksum that catches mistyped addresses before funds move. Note the security floor for the hash wall is 2^{80} (RIPEMD-160 is 160 bits, collisions at 2^{80}),

weaker than the curve's 2^{128}; this is a known and debated soft spot, mitigated in practice by not reusing addresses.

36.5.3 Wallets and key derivation

A *wallet* is a key manager, not a place where coins live — the coins are entries on the chain; the wallet holds the keys that authorize spending them. Managing one random key per address is unwieldy, so modern wallets are *hierarchical deterministic* (HD, BIP-32): a single master seed deterministically derives an unlimited tree of keys via repeated HMAC-SHA512. The seed itself is usually encoded as a *mnemonic* (BIP-39) — the famous twelve or twenty-four English words — whose entropy plus a checksum maps to the seed.

Intuition

The whole hierarchy — millions of addresses across many coins — collapses back to twelve words. That compactness is both its strength and its hazard. Back up the words once and you can regenerate every key forever, on any compatible wallet, even if your device is destroyed. But anyone who reads those words owns everything the seed controls, instantly and irreversibly. Because the mnemonic serves as the root of a deterministic tree rather than an arbitrary password, "rotating" it requires transferring all funds to a newly generated tree. Treat the twelve words the way you would treat the only key to a vault with no locksmith.

In the real world

HD derivation has a subtle but important split: *hardened* vs *non-hardened* child keys. Non-hardened derivation lets you compute child *public* keys from a parent public key without the private key — useful for a webshop that wants a fresh receiving address per customer without keeping any private key on the public server. The catch: if an attacker learns one non-hardened child *private* key and the parent *public* key, the BIP-32 math lets them recover the parent private key and thus every sibling. Hardened derivation breaks that link by feeding the parent private key into the HMAC, at the cost of not supporting public-only derivation. Real wallets use hardened derivation for the account-level nodes and non-hardened only for the leaf addresses under an account, so a leaked leaf key cannot climb the tree.

36.6 Signatures: authorizing a spend

A transaction is an explicit authorization: "consume these specific unspent outputs that I control, and create new unspent outputs assigned to these recipients." In the Unspent Transaction Output (UTXO) model, coins do not exist as account balances; they exist as atomic, discrete chunks of value called UTXOs. To spend a UTXO, the transaction must satisfy the locking script associated with it, which almost universally requires presenting a valid cryptographic signature under the private key matching the locked address.

Figure 36.4 illustrates this ledger mechanism. Panel A traces a legitimate spend consuming an existing UTXO, while Panel B demonstrates why double-spending fails immediately at the peer-to-peer relay layer.

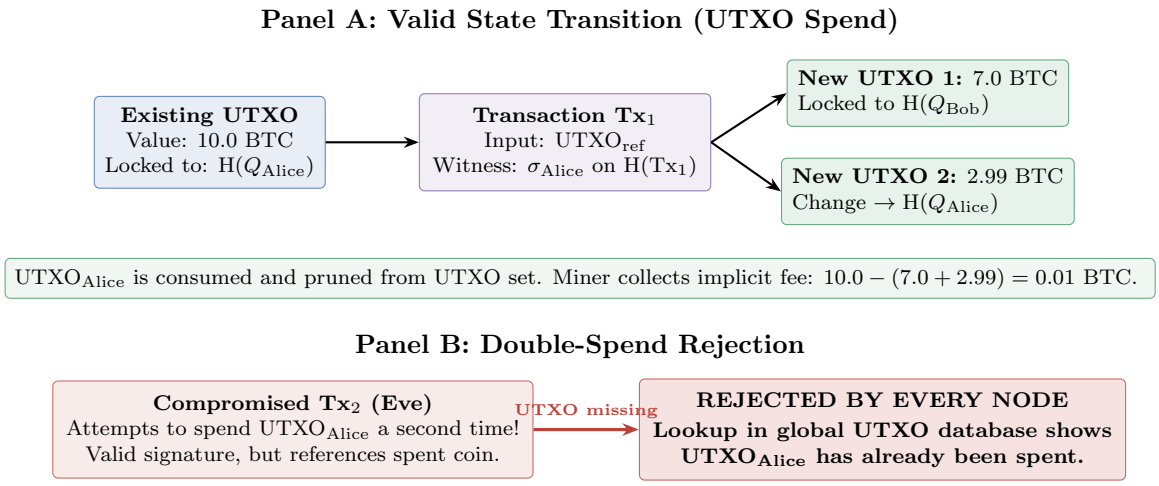


Figure 36.4: The Unspent Transaction Output (UTXO) accounting model and double-spending prevention. In Panel A, an unspent output is consumed in its entirety, validated by Alice's signature, and transmuted into new unspent outputs for Bob and Alice's change address. In Panel B, any concurrent or subsequent attempt to spend the same referenced UTXO fails consensus validation because full nodes track the global unspent set and reject previously spent references.

36.6.1 ECDSA

Bitcoin's original signature scheme is ECDSA. With private key d, public key $Q = dG$, group order n, and message hash $z = \mathrm{H}(\text{transaction})$ reduced mod n, signing produces a pair (r, s):

1. Pick a random per-signature *nonce* k, with $1 \le k < n$.
2. Compute the point $kG = (x_1, y_1)$ and set $r = x_1 \bmod n$. If $r = 0$, restart.
3. Compute $s = k^{-1}(z + r\,d) \bmod n$. If $s = 0$, restart.
4. The signature is (r, s).

Verification, given (r, s), z, and Q: compute $u_1 = z\,s^{-1} \bmod n$, $u_2 = r\,s^{-1} \bmod n$, form the point $u_1G + u_2Q = (x_1, y_1)$, and accept iff $x_1 \bmod n = r$.

Why it works

The verification identity falls out of the algebra. From $s = k^{-1}(z+rd)$ we get $k = s^{-1}(z+rd) = s^{-1}z+s^{-1}rd = u_1+u_2d \pmod n$. Multiply both sides by G: $kG = u_1G+u_2(dG) = u_1G+u_2Q$. So the point the verifier reconstructs from public data, $u_1G + u_2Q$, equals the point kG the signer used, and its x-coordinate mod n is exactly r. Only someone who knows d can produce an s that makes this close, because s binds z, r, and d together through the secret k; forging (r, s) for a chosen z without d requires solving the elliptic-curve discrete log.

Attack

Nonce reuse is fatal. The per-signature k must be *secret* and *never reused*. Reuse k across two signatures (r, s_1) and (r, s_2) on messages $z_1 \neq z_2$ — note the r is identical, a dead giveaway

(Figure 36.5) — and the private key falls out by hand:

$$s_1 - s_2 = k^{-1}(z_1 - z_2) \;\Rightarrow\; k = \frac{z_1 - z_2}{s_1 - s_2} \bmod n, \qquad d = \frac{s_1 k - z_1}{r} \bmod n.$$

This is not theoretical. In 2010 the Sony PlayStation 3 used a *fixed* k for its ECDSA code-signing key; the master key was extracted and the console's chain of trust collapsed. In 2013 a flaw in Android's `SecureRandom` produced repeated k values in some wallet apps and real bitcoin was stolen from the affected addresses. The fix is *deterministic nonces* (RFC 6979): derive k via HMAC from the key d and message z so it is reproducible, unique per message, and never depends on a possibly-broken RNG. Modern Bitcoin signing uses RFC 6979 universally.

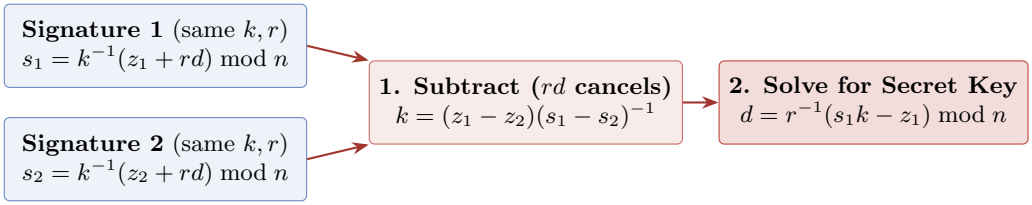


Figure 36.5: Why ECDSA nonce reuse is fatal. Two signatures generated with the same nonce k share the same r and give two linear equations in two unknowns (k and d): subtracting eliminates rd to expose k, which immediately reveals the long-term private key d.

Example 36.4 (ECDSA on a toy curve)**.** Take a curve with group order $n = 19$ and a base point G, private key $d = 7$. Suppose for message hash $z = 11$ the signer's RNG yields $k = 3$, and kG has x-coordinate $x_1 = 10$, so $r = 10 \bmod 19 = 10$. Then $s = k^{-1}(z + rd) \bmod 19$. Compute $k^{-1} = 3^{-1} \bmod 19 = 13$ (since $3 \cdot 13 = 39 \equiv 1$). And $z + rd = 11 + 10 \cdot 7 = 81 \equiv 81 - 4 \cdot 19 = 5 \pmod{19}$. So $s = 13{\cdot}5 = 65 \equiv 65 - 3{\cdot}19 = 8 \pmod{19}$. Signature $(r, s) = (10, 8)$. A verifier computes $s^{-1} = 8^{-1} \bmod 19 = 12$ (since $8 \cdot 12 = 96 \equiv 1$), then $u_1 = zs^{-1} = 11 \cdot 12 = 132 \equiv 132 - 6 \cdot 19 = 18$, $u_2 = rs^{-1} = 10 \cdot 12 = 120 \equiv 120 - 6 \cdot 19 = 6$, forms $18G + 6Q$, and checks its x-coordinate mod 19 equals 10. It does, because $u_1 + u_2 d = 18 + 6 \cdot 7 = 60 \equiv 60 - 3 \cdot 19 = 3 = k \pmod{19}$, so $18G + 6Q = 3G = kG$, whose x-coordinate is the 10 we started with.

36.6.2 Schnorr signatures

In 2021 Bitcoin's Taproot upgrade added Schnorr signatures (BIP-340) alongside ECDSA. Schnorr is older, simpler, and has a clean security proof under the discrete-log assumption (ECDSA's security proof is messier). For private key d, public key $Q = dG$, message m:

1. Pick nonce k, compute $R = kG$.
2. Compute challenge $e = \mathrm{H}(R \,\|\, Q \,\|\, m) \bmod n$.
3. Set $s = k + e\,d \bmod n$. Signature is (R, s).

Verify by checking $sG \stackrel{?}{=} R + eQ$. The check holds because $sG = (k + ed)G = kG + e(dG) = R + eQ$. Schnorr's headline feature is *linearity*: signatures add. Two signers with keys d_1, d_2 can produce a single Schnorr signature valid under the combined key $Q_1 + Q_2$ (this is *MuSig*, done carefully to resist a rogue-key attack). On-chain, a 3-of-3 multisig or a Lightning channel close looks like one ordinary

signature — smaller, cheaper, and more private, since you cannot tell a multisig from a single-sig spend.

Focus

Both schemes prove the same thing: *the signer knew the private key d for this public key, and authorized this exact message m.* Change one satoshi of the transaction and z (or e) changes, so the old signature no longer verifies — signatures bind to the precise bytes. What signatures do *not* prove is that the spend is wise, legal, or what the signer intended; a wallet that displays the wrong amount, or malware that swaps the destination address before signing, gets a perfectly valid signature on a transaction that robs you. The cryptography authenticates bytes, not intentions.

36.7 Proof of work: making history expensive to rewrite

Signatures and hash chaining make tampering *detectable*. They do not by themselves stop an attacker from broadcasting an entirely alternative chain and claiming it is the real one. With no referee, which chain is canonical? Bitcoin's answer: the chain with the most cumulative *proof of work*. Producing each block requires solving a hash puzzle that has no shortcut, so the longest valid chain is the one the most computation went into, and rewriting old blocks means out-computing the entire rest of the network.

36.7.1 The puzzle

A block is valid only if its header hash, read as a 256-bit number, is *below a target* $\mathcal{T}$:

$$\mathrm{H}(\text{header}) < \mathcal{T}.$$

The header includes the nonce, so miners try nonce after nonce (and tweak other malleable fields when the nonce space is exhausted) until the hash comes out small enough (Figure 36.6). Because the hash is preimage-resistant, there is no way to compute a winning nonce directly; you can only guess and check. Lowering $\mathcal{T}$ makes wins rarer and the puzzle harder; the network adjusts $\mathcal{T}$ every 2016 blocks so blocks arrive about every ten minutes on average regardless of total mining power.

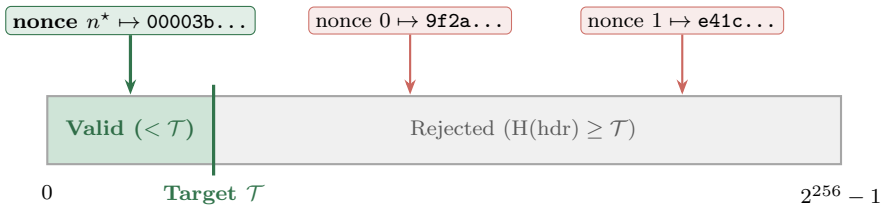


Figure 36.6: Proof-of-work as a target-threshold search in $\{0, \ldots, 2^{256} - 1\}$. Each candidate nonce maps to a pseudorandom point along the 256-bit interval; only hashes landing in the green zone $[0, \mathcal{T})$ (equivalently, starting with enough leading zero bits) produce a valid block.

Intuition

"Below the target" is the same as "the hash starts with enough zero bits." If the target is $2^{256}/D$, then a random hash lands below it with probability $1/D$, so you expect to try about D nonces per block. Each try is one hash; finding a hash with the first 20 bits zero takes about $2^{20} \approx 10^6$ tries, the first 76 bits zero (a recent real Bitcoin difficulty) takes about 2^{76}. There is no cleverness available — preimage resistance guarantees the best strategy is brute force — which is precisely why the work is a meaningful, unforgeable measure of effort. You cannot fake having done 2^{76} hashes; the winning nonce is the receipt.

Connection

The nonce search is a Bernoulli process: each hash is an independent trial that succeeds with probability $q = \mathcal{T}/2^{256}$. The number of tries until the first success is geometric with mean $1/q$, and over many blocks the inter-block times are (to good approximation) exponential — a Poisson process of block discoveries. That is the probability companion's machinery applied directly: difficulty retargeting is just steering the rate parameter of a Poisson process toward one event per ten minutes. The same memorylessness explains why "this block is taking a while" tells you nothing about when the next one arrives.

Example 36.5 (Mining a toy block)**.** Set an easy target: "hash must start with four zero hex digits" ($\mathcal{T} = 2^{256}/2^{16}$, so $q = 1/65536$). The header is the string `"prev|root|"` followed by a nonce. Try nonce $0, 1, 2, \ldots$, hashing `"...|0"`, `"...|1"`, until SHA-256d of the header begins `0000`. You expect about 65,536 tries; on a laptop this finishes in a fraction of a second, and you can literally watch the nonce climb. Real Bitcoin uses the same loop with $\mathcal{T}$ vastly smaller, which is why it takes the combined planet's worth of ASICs ten minutes instead of your laptop a millisecond.

Implement it

Mining the toy block from Example 36.5. **Learning only — a real miner is optimized ASIC firmware; this is to feel the search.**

```python
import hashlib

def mine(prev_hex, root_hex, leading_zeros=2):
    prefix = "0" * leading_zeros
    nonce = 0
    while True:
        header = f"{prev_hex}|{root_hex}|{nonce}".encode()
        h = hashlib.sha256(hashlib.sha256(header).digest()).hexdigest()
        if h.startswith(prefix):
            return nonce, h
        nonce += 1

nonce, h = mine("00ab", "4f12", leading_zeros=4)   # ~65536 expected tries
print("nonce:", nonce, "hash:", h)                  # hash begins 0000...
# Verifying is one hash; finding the nonce was ~65536. That asymmetry IS the point.
```

36.7.2 Why rewriting is hard: the longest-chain rule

Rule-following miners always extend the chain with the most work. To rewrite block k, an attacker must redo block k's proof of work *and* every block after it, then catch up to and overtake the protocol-following tip — which keeps growing while the attacker computes. If the attacker controls a fraction β of total hash power and the rest of the network $1-\beta$, the attacker's private chain falls behind in expectation whenever $\beta < 1/2$. The gap to overcome grows with each confirmation, so the probability of a successful rewrite drops roughly geometrically in the number of blocks built on top — which is why merchants "wait for six confirmations."

Attack

The 51% attack. If $\beta > 1/2$, the attacker can on average out-produce the rest of the network, build a longer alternative chain, and replace the public history — enabling *double-spends*: pay a merchant on the public chain, take the goods, then publish a longer chain in which that payment never happened. What it does *not* grant is the ability to forge signatures, mint coins out of thin air, or spend others' addresses — those are protected by ECDSA and the issuance rules, which a longer chain cannot violate and still be valid. A 51% attacker can reorder, censor, and reverse *recent* transactions involving their own coins, not break cryptography. Smaller chains with little total hash power have suffered real 51% attacks (Ethereum Classic, Bitcoin Gold); Bitcoin's hash rate makes it prohibitively expensive, though that is an economic guarantee, not a mathematical one. Proof-of-stake chains (Ethereum since 2022) replace hash power with staked capital and slashing penalties, shifting the same "majority can rewrite" threshold from compute to coins.

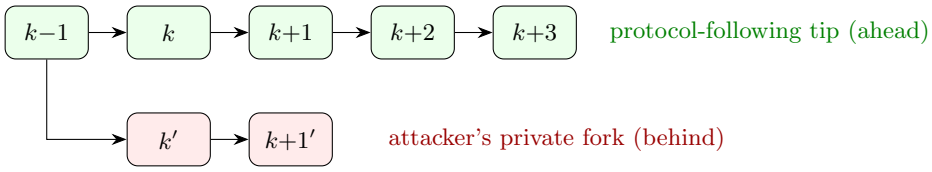


Figure 36.7: An attacker forking from block $k-1$ must outrun the protocol-following chain. While the attacker built two blocks, the rest of the network built four. With under half the hash power, the attacker's fork falls further behind every block, so deeply-buried history is effectively permanent.

36.8 What the cryptography does and does not guarantee

This is the section to read twice. Blockchain marketing routinely claims properties the math never promised. Be precise about the boundary.

What the cryptography *does* guarantee (assuming SHA-256, RIPEMD-160, and `secp256k1` hold, and a protocol-following majority for consensus):

- *Integrity of recorded history.* Hash chaining plus proof of work make editing a buried transaction infeasible. The ledger is tamper-evident, and tamper-resistant in proportion to the work piled on top.
- *Authorization.* Only the holder of a private key can produce a spending signature for its address. Coins cannot be stolen without the key (or a break in ECDSA).

- *Public verifiability.* Anyone can re-derive every hash, check every signature, and confirm every Merkle proof from scratch, trusting no participant. Correctness is checkable, not asserted.
- *Commitment / non-repudiation of the bytes.* A signed transaction is a binding statement: the signer cannot later deny authorizing those exact bytes.

What it does *not* guarantee:

- *Value.* No theorem says a coin is worth anything. Cryptography secures the ledger's *integrity*, not the *price* or liquidity of what it records. A perfectly secured ledger of worthless tokens is still worthless.
- *Correctness of intent.* Signatures bind bytes, not meaning. Malware that rewrites the recipient address before you sign, or a smart contract with a logic bug, produces a cryptographically flawless theft. The 2016 DAO hack drained tens of millions of dollars through a contract reentrancy bug — the signatures and hashes were all valid; the *logic* was wrong.
- *Privacy.* The default ledger is public and pseudonymous, not anonymous. Addresses are not names, but transaction-graph analysis routinely de-anonymizes users. Confidentiality requires extra machinery (zero-knowledge proofs, Chapter 31; mixing; confidential transactions).
- *Key safety.* Lose the private key (or the BIP-39 mnemonic) and the coins are unrecoverable — there is no password reset and no support line. Leak it and they are stolen. Millions of bitcoin are estimated to be permanently lost to forgotten keys. The cryptography has no opinion about your backups.
- *Legality or off-chain truth.* The chain can record a transaction that is fraud, theft, or fiction. It guarantees the *record* is consistent, not that the recorded claim is true or lawful. "Code is law" is a slogan, not a fact.
- *Liveness against a hash-power majority, forever.* The protocol-following-majority assumption is economic, not mathematical. A 51% attacker breaks no cipher but can reorder and reverse recent history. Security here is "expensive to attack," not "impossible."

Common trap

The most expensive misunderstanding in the space is conflating "the cryptography is unbreakable" with "my money is safe." The cryptography being sound is necessary, not sufficient. Real losses overwhelmingly come from the *edges* the math does not touch: phishing the seed phrase, malware swapping the destination address at signing time, exchange custodians getting hacked, smart-contract bugs, lost backups, and social-engineering a wallet's recovery flow. A cryptographer's job on a blockchain team is as much about these seams — key management, signing-device UX, contract auditing — as about the curve arithmetic.

In the real world

Where these primitives show up beyond cryptocurrency. *Certificate Transparency* (Chapter 24) is a Merkle-tree append-only log of every TLS certificate issued, letting anyone detect a rogue certificate; it borrows the blockchain's tamper-evident-log idea without the coin or the proof of work. *Git* chains commits by hash exactly as a blockchain chains blocks — a commit hash is a root over the tree plus the parent commit hash, which is why rewriting git history changes every later commit ID. Supply-chain and software-provenance systems

(Sigstore, in-toto) use Merkle logs to bind artifacts to their build records. The reusable lesson: *a hash-chained Merkle log gives you a tamper-evident, publicly verifiable history.* Whether you bolt on a coin and a proof-of-work consensus depends on whether you have a trusted operator (then you do not need them) or a permissionless crowd (then you do).

36.9 Putting it together: the life of a transaction

Trace one payment through every primitive in this chapter.

1. **Keys.** Alice's wallet holds a private key d; her address is HASH160(dG), Base58Check-encoded. Bob gives Alice his address.

2. **Build.** Alice's wallet constructs a transaction: inputs referencing coins she controls, outputs paying Bob's address (and change back to herself).

3. **Sign.** The wallet hashes the transaction to z and produces an ECDSA (or Schnorr) signature with d, using an RFC 6979 deterministic nonce. The signature proves authorization without revealing d.

4. **Broadcast.** The signed transaction floods the peer-to-peer network. Every node checks the signature against the input public keys and confirms the coins are unspent.

5. **Merkle.** A miner collects valid transactions into a block, hashes each, and builds the Merkle tree; the root goes in the header.

6. **Chain & mine.** The header includes the previous block's hash (chaining) and a nonce; the miner searches nonces until H(header) $< \mathcal{T}$ (proof of work).

7. **Confirm.** The block propagates; nodes verify the proof of work, the Merkle root, and every signature, then extend their chain. Each subsequent block buries Alice's payment deeper, making reversal exponentially harder.

8. **Verify later.** Bob's phone, an SPV client, holds only headers. It asks for a Merkle proof that Alice's transaction is in the block and checks it against the header's root — never downloading the full chain.

Six primitives, one payment: hash functions glue and commit, Merkle trees compress and prove membership, elliptic-curve keys identify, signatures authorize, hash chaining makes the past tamper-evident, and proof of work makes rewriting it expensive. Remove any one and the structure fails: no signatures and anyone spends your coins; no chaining and the past is editable; no proof of work and any node rewrites history for free; no Merkle trees and light clients must trust full nodes. The blockchain is the interlock.

Correct algorithms on paper still have to run on physical hardware. In Part VI we turn to what happens when they do: Chapter 37 opens with side-channel and timing attacks, showing how execution time, cache hits, and power draw can leak a secret key without touching the math.

Exercises

Exercise 36.1 (Chaining cascade)**.** A chain has blocks $0, 1, 2, 3, 4$, each header containing `prev` (the previous header's hash) and a body. Eve edits the body of block 2. List exactly which block IDs change and which `prev` fields a verifier finds inconsistent. If each block required W units of proof-of-work, how much work must Eve redo to present a self-consistent altered chain of the same length?

Exercise 36.2 (Merkle proof by hand)**.** Using the toy 1-byte hash of Example 36.2 with leaves $\ell_0 = \texttt{a3}, \ell_1 = \texttt{7c}, \ell_2 = \texttt{1f}, \ell_3 = \texttt{d8}$ and the intermediate values given there, write down the Merkle proof for T_1 (not T_2). State which sibling goes on which side, and show the three-step recomputation a verifier performs to reach the root `29`.

Exercise 36.3 (Odd leaves)**.** A block has five transactions. Draw the Merkle tree using Bitcoin's "duplicate the last node" rule for odd levels. How many hashes are in the inclusion proof for the third transaction? Then explain in one or two sentences how the duplication rule enabled CVE-2012-2459, and how a leaf/internal domain-separation tag prevents it.

Exercise 36.4 (Address derivation)**.** Given a compressed public key as 33 bytes, write the sequence of operations (which hash, in which order, what prefix byte, what encoding) to produce a Bitcoin P2PKH address. Why is the public key hashed rather than published directly? Give two distinct reasons, and state the bit-security of the RIPEMD-160 wall against collisions.

Exercise 36.5 (Break ECDSA nonce reuse)**.** Two ECDSA signatures share the same r: (r, s_1) on z_1 and (r, s_2) on z_2, with group order n. Derive k and then the private key d in terms of r, s_1, s_2, z_1, z_2. Then plug in $n = 19$, $r = 10$, $z_1 = 11$, $s_1 = 8$, $z_2 = 4$, $s_2 = 12$ and recover d as a concrete number. (Check your answer against the signing equation.)

Exercise 36.6 (Verify a toy ECDSA signature)**.** With $n = 19$, suppose a signature is $(r, s) = (10, 8)$ on $z = 11$, and you are told $u_1 + u_2 d \equiv 3 \pmod{19}$ where d is the signer's key. Compute u_1 and u_2 from (r, s, z), and confirm the verification point reconstructs kG with $k = 3$. (You may reuse the inverses from Example 36.4.)

Exercise 36.7 (Schnorr verification)**.** For Schnorr with private key d, $Q = dG$, nonce k, $R = kG$, challenge $e = \mathrm{H}(R \,\|\, Q \,\|\, m)$, and $s = k + ed \bmod n$: prove algebraically that the check $sG = R + eQ$ always holds for a correctly formed signature. Then explain why an attacker who does not know d cannot produce a valid (R, s) for a chosen m without solving discrete log.

Exercise 36.8 (Mining probability)**.** A proof-of-work target requires the double-SHA-256 of the header to start with w zero *bits*. Model each nonce as an independent trial. (a) What is the success probability q per nonce? (b) What is the expected number of nonces tried before a success? (c) For $w = 20$, roughly how many tries, and how does this compare to a real Bitcoin block needing on the order of $w = 76$? (d) Using the geometric/exponential model, argue why the time since the last block tells you nothing about when the next one arrives.

Exercise 36.9 (51% scope)**.** An attacker controls 60% of a chain's hash power. For each of the following, state whether the attacker can do it and why: (a) reverse a transaction confirmed one block ago to double-spend their own coins; (b) spend coins sitting at an address whose private key they do not hold; (c) create 10,000 new coins beyond the issuance schedule; (d) censor a specific user's transactions for as long as they hold the majority. Tie each answer to whether it is blocked by *cryptography* or only by *majority-following consensus*.

Exercise 36.10 (What the math does not cover)**.** For each loss scenario, name the failure and state whether the cryptography of this chapter was broken or merely bypassed: (a) a user types their twelve-word mnemonic into a phishing site; (b) malware swaps the destination address in the wallet UI just before the user signs; (c) a smart contract with a reentrancy bug is drained; (d) the coin's market price falls to zero. What single sentence summarizes the boundary these four share?

Exercise 36.11 (Light-client trust)**.** An SPV phone wallet stores only 80-byte headers and verifies payments via Merkle proofs. (a) Roughly how many bytes of headers for a chain of 800,000 blocks? (b) What exactly does the wallet trust, and what does it verify itself? (c) Describe one thing an SPV client *cannot* detect that a full node can (hint: think about whether a header's transactions obey the issuance and validity rules), and how committing to the full state (Ethereum's approach) narrows that gap.

Part VI

Attacks and Safe Implementation

Chapter 37

Side-Channel and Timing Attacks

Every chapter before this one lived in a clean world. We proved that AES is a strong pseudorandom permutation, that RSA rests on factoring, that ECDSA rests on the elliptic-curve discrete log, and we drew threat models in which the adversary sees ciphertexts and signatures and nothing else. In that world a scheme is secure when the underlying math is hard and the reductions go through. The catch is that nobody runs math. They run code, on a processor, in a box that draws power, radiates electromagnetic fields, heats up, takes time, shares a cache with other programs, and occasionally glitches. The clean abstraction $c = \mathrm{Enc}_k(m)$ hides a physical machine, and that machine leaks.

A *side channel* is any observable behavior of an implementation that depends on the secret, but that the security proof never accounted for because the proof reasoned about inputs and outputs, not about the machine in between. How long did the comparison take? How much current did the chip draw on that clock cycle? Did a memory access hit the cache or miss it? Did decryption return "bad padding" or "bad MAC"? None of these are part of the ciphertext. All of them can carry the key out of the box, one bit at a time, while the proof on paper remains perfectly valid. The cipher was never broken. The implementation was.

This chapter is about that gap, and it is the most humbling subject in the book. We will start with timing – the cleanest, most teachable channel – and build a working mental model of why an `if` statement on a secret is a leak. We will write a tiny string comparison that leaks a MAC byte by byte, then learn to write the constant-time version. From there we widen out: cache attacks that recover AES keys from a process that never shares memory with you, power and electromagnetic analysis that read a smart card's key off an oscilloscope, fault attacks that extract an RSA key from a *single* corrupted signature, and the two padding-oracle disasters that defined a decade of TLS bugs – Bleichenbacher against RSA and Lucky 13 against CBC. The unifying lesson is blunt: **secure on paper is necessary and nowhere near sufficient**, and the discipline that closes the gap – constant-time programming – is a craft you have to learn deliberately, because every instinct a normal programmer has runs against it.

37.1 The Shape of the Problem

Let us fix the cast. Alice runs a server holding a secret key k. Eve is a client. She cannot read Alice's memory, cannot see k, cannot break AES or RSA. What she *can* do is interact with Alice's

implementation and measure something about how it behaves: how long a response takes, how much the machine's shared cache got disturbed, how much power the chip drew. The question of this chapter is whether those measurements, none of which are "the ciphertext," leak k.

Intuition

Consider a combination safe whose dial emits an audible click at each correct digit. An observer listening to the clicks can determine each digit independently: a search over 10^6 combinations collapses to $10 \times 6 = 60$ trials. The underlying mathematical structure of the combination lock is unchanged, but an unintentional physical emission converts a multiplicative search into an additive one. Side-channel attacks operate on this principle: physical behavior correlated with secret state collapses an otherwise infeasible brute-force search into a tractable one.

That collapse, from multiplicative to additive search, is the recurring mathematical signature of a side channel and worth stating sharply. A secret with n independent components, each from a set of size s, has s^n possible values. Guessing it blind costs $O(s^n)$. But if the implementation leaks, for each component, whether your guess of that component is right – independently of the others – then you can fix the components one at a time, paying $O(s)$ per component and $O(s \cdot n)$ total. For a 16-byte MAC that is the difference between $256^{16} \approx 10^{38}$ and $256 \times 16 = 4096$. The leak does not shave a few bits off the search; it changes the exponent into a coefficient.

Focus

The threat model is the core asymmetry. "Secure" in the earlier chapters meant: an adversary who sees inputs and outputs learns nothing. The side-channel threat model adds observables the proof omitted – time, power, EM, cache state, error messages, fault behavior. A scheme can be IND-CCA secure in the standard model and still hand Eve the key through one of these channels, because IND-CCA never promised anything about *how long* decryption takes. Evaluating implementations under an expanded threat model ensures security analysis accounts for real-world execution environments rather than idealized mathematical abstractions.

History

The discipline was born in 1996 when Paul Kocher published a timing attack on implementations of RSA, Diffie-Hellman, and DSS, showing that the time to compute a modular exponentiation $g^d \bmod N$ leaks the private exponent d bit by bit. The community's first reaction was disbelief that something so abstractly secure could fall to a stopwatch. Two years later Kocher, Joshua Jaffe, and Benjamin Jun introduced *differential power analysis*, pulling DES and later AES keys off smart cards by averaging power traces, while Daniel Bleichenbacher showed that error messages distinguishing valid from invalid PKCS#1 v1.5 padding turned RSA decryption into an oracle. By the 2000s remote timing attacks (Brumley–Boneh 2003) and cache attacks (Bernstein, Osvik–Shamir–Tromer, Percival) recovered keys across network links and process boundaries, and by the 2010s padding oracles (the BEAST/CRIME/Lucky 13/POODLE parade) were breaking TLS in the field. Spectre and Meltdown in 2018 showed the CPU itself – speculative execution – was a side channel, spurring dedicated instructions (AES-NI) and formally verified constant-time libraries (HACL*, fiat-crypto). The throughline across thirty years: the math held every time, and the implementation leaked every time.

37.2 Timing Attacks: When Time Is a Variable

Start with the cleanest channel. A program takes time. If the time it takes depends on a secret, then measuring the time measures a function of the secret. That is the entire principle. The art is in finding code whose running time depends on the key in an exploitable way – and such code is everywhere, because the most natural way to write almost anything makes the running time data-dependent.

37.2.1 The leaky string comparison

Here is the canonical example, and it is so common it is almost a rite of passage. Alice's server receives a message m together with a claimed MAC tag t'. It computes the correct tag $t = \mathrm{MAC}_k(m)$ and checks whether $t' = t$. The check is a string comparison. The natural string comparison – the one in every standard library, the one you would write without thinking – looks like this:

```
def insecure_equal(a, b):
    if len(a) != len(b):
        return False
    for i in range(len(a)):
        if a[i] != b[i]:      # bails out at the FIRST mismatch
            return False
    return True
```

Read the loop carefully. It returns the moment it finds the first differing byte. If t' and t disagree in byte 0, the loop runs one iteration. If they agree in bytes 0 through 4 and disagree in byte 5, the loop runs six iterations. **The running time of the comparison is proportional to the length of the matching prefix.** The comparison was supposed to reveal exactly one bit – equal or not equal – but its timing reveals *how much* of the tag is correct. That is the click on the safe.

Now Eve attacks. She wants to forge a tag for a message m of her choosing without knowing k. She cannot compute $\mathrm{MAC}_k(m)$, but she can guess it byte by byte using the timing leak.

Attack

Forging a MAC through a timing oracle. Eve fixes m and will discover the correct 16-byte tag $t = \mathrm{MAC}_k(m)$ one byte at a time.
Step 1: guess byte 0. She sends (m, t') for all 256 values of $t'[0]$, padding the rest arbitrarily, and *times each rejection*. For 255 of the guesses the loop bails at byte 0. For the one correct guess of $t'[0]$, the loop gets to byte 1 before bailing – so that rejection takes measurably longer. Eve picks the byte value whose rejection was slowest. She has learned $t[0]$ with 256 queries.
Step 2: with $t[0]$ fixed and correct, she repeats for byte 1: 256 guesses, the correct one makes the loop reach byte 2, slowest wins. And so on.
Total cost: about $256 \times 16 = 4096$ queries to recover the full 16-byte tag, versus the $256^{16} \approx 3.4 \times 10^{38}$ of blind forgery. Once she has t, she submits (m, t) and it verifies. She forged a MAC for a message of her choice without the key – purely from timing.

Walking question

Why does Eve win each byte by picking the *slowest* response, not the fastest? Because a longer matching prefix means the loop runs more iterations before returning, which costs more time. The correct guess for the current byte is precisely the one that lets the loop advance past it. So at each position the right byte is the one that makes the server work a hair harder, and "a hair harder" shows up as "a hair slower." Eve is climbing the tag one rung at a time, and each rung is marked by a small step up in latency.

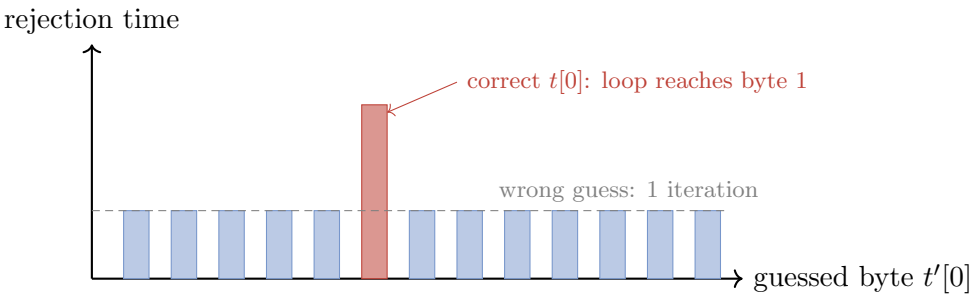


Figure 37.1: Timing signal of the leaky comparison (Section 37.2.1) while Eve probes byte 0. Every wrong guess bails at the first byte and takes the same short time; the one correct guess lets the loop advance one more iteration, producing a measurably taller bar. Eve picks the slowest.

37.2.2 The hard part is the noise

The objection writes itself: one byte comparison is a handful of nanoseconds, and the network jitter between Eve and Alice is milliseconds – a million times larger. Surely the signal drowns. It would, if Eve measured once. The defense against noise is the oldest trick in experimental science: *average*. Network delay is (mostly) zero-mean noise added to a fixed signal; averaging N independent samples shrinks the noise's standard deviation by $\sqrt{N}$ while leaving the signal intact.

Connection

This is exactly the law-of-large-numbers / central-limit machinery from the probability companion. Suppose the true timing difference between a correct and incorrect byte guess is a constant Δ, and each measurement is corrupted by independent noise with standard deviation σ. The mean of N samples has standard deviation $\sigma/\sqrt{N}$. To resolve a signal of size Δ against noise σ, Eve needs the mean's error well below Δ, i.e.

$$\frac{\sigma}{\sqrt{N}} \ll \Delta \quad \Longrightarrow \quad N \gg \left(\frac{\sigma}{\Delta}\right)^2.$$

The number of samples grows with the *square* of the noise-to-signal ratio. A million-to-one ratio of jitter to signal costs on the order of 10^{12} samples per byte if done naively – but attackers use clever statistics (the "Box test," percentile filtering, comparing distributions rather than means) to cut this dramatically, and researchers have demonstrated remote timing attacks across real networks. The point of the $\sqrt{N}$ law is that noise buys you safety only logarithmically slowly. It does not make the leak go away; it makes it more expensive, and "more expensive" is not a security guarantee.

In the real world

"Surely the network hides it" was demolished in 2003 by Brumley and Boneh, who recovered an RSA private key from an OpenSSL server *over a network*, exploiting timing differences in the modular reduction inside RSA-CRT decryption. In 2011 Brumley and Tuveri did the same for ECDSA over TLS. The repeated lesson: jitter raises the query count, it does not close the channel. If your security depends on an attacker not being patient, you have no security.

37.2.3 Constant-time comparison

The fix is to make the comparison's running time independent of the data. Never branch on a secret; never return early. Compare *all* bytes every time, accumulating the result without leaking where the first difference was.

```
def constant_time_equal(a, b):
    # Assumes len(a) == len(b); leaking the length is usually acceptable,
    # leaking WHERE bytes differ is not.
    if len(a) != len(b):
        return False
    result = 0
    for x, y in zip(a, b):
        result |= x ^ y       # 0 only if every byte matched
    return result == 0
```

Implement it

Why this is constant-time (in the byte count). The loop always runs the full length – no early return. Each iteration does the same work: one XOR, one OR. The accumulator `result` ends at 0 if and only if every `x ^ y` was 0, i.e. every byte matched; otherwise some bit is set. Because *which* bytes differed and *how many* differed never alter the control flow or instruction sequence, the execution time carries zero information about the matching prefix. The final `result == 0` is a single comparison on an accumulated mask that reveals only whether the buffers were identical.

Learning only. This Python is for understanding the principle. Real Python is the wrong tool: the interpreter, integer caching, and garbage collector add their own timing variation, and `==` on the final int may itself short-circuit at the C level. In production call a vetted primitive – `hmac.compare_digest` in Python, `CRYPTO_memcmp` in OpenSSL, `subtle.ConstantTimeCompare` in Go, the `subtle` crate in Rust. Never roll your own for real code.

Common trap

Constant-time comparison fixes *tag verification*. It does nothing for the dozens of other places a real protocol branches on secrets: the modular exponentiation in RSA, the conditional subtraction in a Montgomery reduction, the table lookup indexed by a key byte in a naive AES, the "is this padding valid" check we will dissect later. "I used `compare_digest`" is the beginning of constant-time discipline, not the end. The leak is wherever the secret steers the machine.

37.2.4 Timing in the math itself: square-and-multiply

The comparison leak is easy to picture; the deeper problem is that the cryptographic *algorithms* themselves are naturally data-dependent. The classic is modular exponentiation by square-and-multiply, the heart of RSA and Diffie-Hellman. To compute $g^d \bmod N$ you scan the bits of the secret exponent d from most significant to least:

```
def modexp_leaky(g, d, N):
    r = 1
    for bit in bits_of(d):        # MSB to LSB
        r = (r * r) % N           # SQUARE, every iteration
        if bit == 1:
            r = (r * g) % N       # MULTIPLY, only when the bit is 1
    return r
```

Look at the branch. Every bit costs a square. A *set* bit costs an additional multiply. So the total time is roughly (number of bits) squarings plus (number of *one*-bits) multiplies – and the running time leaks the Hamming weight of the private exponent, while finer timing (or the power trace we will meet shortly) can leak the bits in order. Kocher's original 1996 attack worked exactly here.

Example 37.1. Take a toy exponent $d = 13 = 1101_2$ and trace the multiplies. Bits MSB to LSB are $1, 1, 0, 1$.

$$\begin{aligned}
&\text{bit } 1: \quad r \leftarrow r^2 = 1, && \text{bit} = 1: \quad r \leftarrow r \cdot g = g \\
&\text{bit } 1: \quad r \leftarrow r^2 = g^2, && \text{bit} = 1: \quad r \leftarrow r \cdot g = g^3 \\
&\text{bit } 0: \quad r \leftarrow r^2 = g^6, && \text{bit} = 0: \quad \text{(no multiply)} \\
&\text{bit } 1: \quad r \leftarrow r^2 = g^{12}, && \text{bit} = 1: \quad r \leftarrow r \cdot g = g^{13}.
\end{aligned}$$

Four squarings, three multiplies – because d has three one-bits. An exponent of the same length but with $d = 8 = 1000_2$ runs four squarings and *one* multiply. Same input size, different running time, and the difference is a direct readout of the secret's bit pattern.

Why it works

The constant-time countermeasure here is the *Montgomery ladder*, which performs one squaring *and* one multiplication on every bit, regardless of the bit's value, using the bit only to choose which of two registers gets updated – and even that choice is done with a constant-time conditional swap, not a branch. Every iteration looks identical from the outside: same operations, same memory pattern, same time. The exponent no longer modulates the work. You pay a constant factor (you always do the multiply, even for zero bits) in exchange for closing the channel. That trade – spend cycles to erase data dependence – is the entire economics of constant-time code.

In the real world

This is not academic. OpenSSL, BoringSSL, libsodium, and every serious crypto library implement RSA and EC scalar multiplication with constant-time exponentiation ladders and "blinding" (multiply the input by a random factor before exponentiating, divide it out after, so the attacker times a randomized input they cannot predict). When a library ships a CVE titled "timing side channel in ⟨ECDSA nonce / RSA decryption / …⟩," it almost always means some code path slipped back into a data-dependent branch or a non-constant-time modular reduction. The fixes are exactly the ladder-and-blind discipline described here.

37.3 Cache Attacks: The Memory System Is Watching

Timing told us *when* the machine did more work. Cache attacks tell us *where* in memory the machine looked – and for many ciphers, where you look is a function of the key. This channel is more insidious because it works *across processes*: Eve's program and Alice's program can share nothing – no memory, no key, no files – and Eve can still extract Alice's AES key purely by watching how Alice's memory accesses disturb a cache they happen to share on the same physical CPU.

37.3.1 Why the cache leaks

A CPU cache is a small fast memory that holds recently used lines of main memory. An access to a cached address is fast (a *hit*, a few cycles); an access to an uncached address is slow (a *miss*, hundreds of cycles) and pulls the line into the cache, evicting something else. The cache is shared among programs on the same core (and the last-level cache across cores). That sharing is the channel: by measuring which lines are fast or slow to access, one program infers which lines *another* program recently touched.

Now connect that to a cipher. Many fast AES implementations use precomputed lookup tables ("T-tables") and index them by key-dependent bytes: a line like `T[ state[i] ^ key[i] ]`. The *table index* – hence which cache line gets loaded – depends directly on a byte of the key XORed with a byte of state. If Eve can tell which table lines Alice accessed, she learns constraints on `state ^ key`, and with known or chosen plaintext she solves for the key.

Attack

Prime+Probe on an AES T-table. Eve and Alice run on the same machine; they share the last-level cache. Eve has no access to Alice's memory or key.

1. *Prime.* Eve fills the cache sets that the AES tables map to with her own data, so she knows exactly what is cached.
2. *Trigger.* Eve gets Alice to encrypt a (known) plaintext block. Alice's AES does table lookups `T[p[i] ^ k[i]]`, loading those specific lines and *evicting* Eve's data from exactly the cache sets those indices map to.
3. *Probe.* Eve re-accesses her primed data and times each set. Sets that are now *slow* were evicted – meaning Alice's lookup landed there – revealing the high bits of `p[i] ^ k[i]` (the bits that pick the cache line). Since Eve knows `p[i]`, she learns the high bits of `k[i]`.

Repeat over many plaintexts and table positions; the constraints pin down the key bytes. Osvik, Shamir, and Tromer (2006) recovered a full AES-128 key this way in a few hundred encryptions on a stock PC. `Flush+Reload` is a sharper variant when memory is shared (e.g. a shared library): flush a specific line with `clflush`, let the victim run, then time reloading that exact line – fast means the victim used it.

Intuition

Picture a library reading room with a shared shelf of reference books. You cannot see the scholar at the next desk or read his notes. But you arrange all the books neatly (prime), let

him work (he pulls down whichever volumes his research needs), then check which books are now out of place (probe). The books he disturbed tell you the topic he is studying – without ever reading over his shoulder. AES T-table lookups are the disturbed books; the topic is the key.

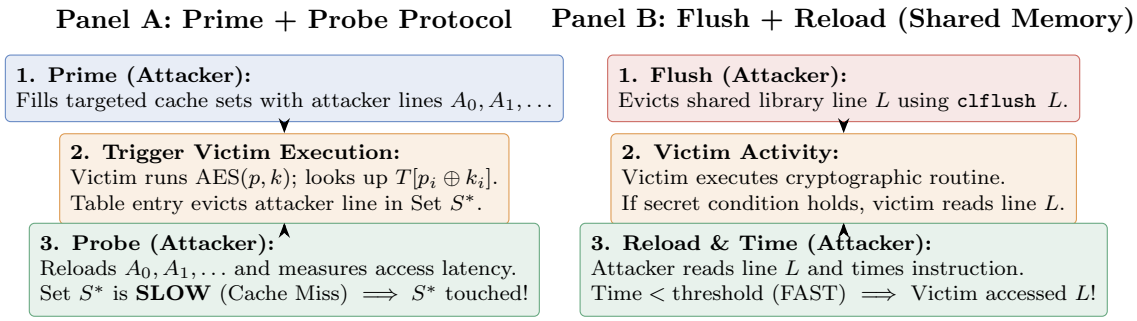


Figure 37.2: Microarchitectural cache-timing attack mechanics. Panel A traces Prime+Probe: the attacker fills cache sets, allows the victim cipher to run, and measures which sets suffered eviction. Panel B traces Flush+Reload in shared-memory environments: line L is flushed; a subsequent low-latency reload proves the victim accessed that specific address.

Why it works

The defense is to stop indexing memory by secrets. Three lines of attack: (1) Bit-sliced or vector implementations that compute the S-box with arithmetic/logic instead of a table lookup – no key-dependent address, no cache footprint. (2) The AES-NI hardware instructions (`aesenc` and friends on x86, and ARM's crypto extensions), which perform a round in fixed-function silicon with no data-dependent memory access at all; this is why AES-NI is both faster *and* safer than table AES. (3) If you must use tables, ensure the whole table fits in a single cache line or is accessed in a way that touches every line every time – expensive and fragile. The clean answer that won is AES-NI: when the hardware does the cipher, there is no table to spy on.

In the real world

Cache attacks demonstrate why hardware AES-NI instructions serve as an essential security requirement, not merely a speed optimization, and why cloud providers enforce core isolation: in a shared-tenant cloud, Eve's VM and Alice's VM can land on the same physical core and share the last-level cache. Cross-VM key recovery via Flush+Reload has been demonstrated against real crypto libraries. Mitigations in the field include constant-time / bit-sliced implementations, cache partitioning, disabling memory deduplication (which enabled Flush+Reload across VMs), and core isolation for sensitive workloads. Spectre and Meltdown (2018) exploit cache side channels: speculative execution touches memory it should not, and the *cache footprint* of that speculation is read out by Flush+Reload, leaking data across security boundaries the architecture promised to keep separate.

37.4 Power and Electromagnetic Analysis

If Eve has physical access to the device – a smart card, a hardware security module, a payment terminal, an IoT chip on her bench – she can measure something even more direct than time or cache state: the *instantaneous power* the chip draws, or the electromagnetic field it radiates, sampled by an oscilloscope at every clock cycle. CMOS logic burns power when its transistors switch, and the amount of switching depends on the data being processed. The power trace is a high-resolution movie of the computation, and the secret is in the frame.

37.4.1 Simple power analysis (SPA)

Sometimes you can just *look* at the trace. Recall square-and-multiply: a squaring and a multiply are different operations taking different amounts of work, so they draw different power signatures. On a naive implementation, a single power trace of one RSA decryption shows a sequence of "square" and "square-multiply" bumps – and reading off which bumps have the extra multiply reads off the bits of the private exponent *directly from the picture*. No statistics, no averaging, one trace. That is SPA.

Intuition

A squaring is one operation; a square-followed-by-multiply is two. On the oscilloscope the second one is visibly wider – a double hump instead of a single. Bit $= 1$ looks different from bit $= 0$ to the naked eye. You read the secret exponent off the screen like Morse code: a long blip is a one, a short blip is a zero. This is why the Montgomery ladder (always square *and* multiply) matters for power as much as for timing: it makes every bit's trace identical, so there is nothing to read.

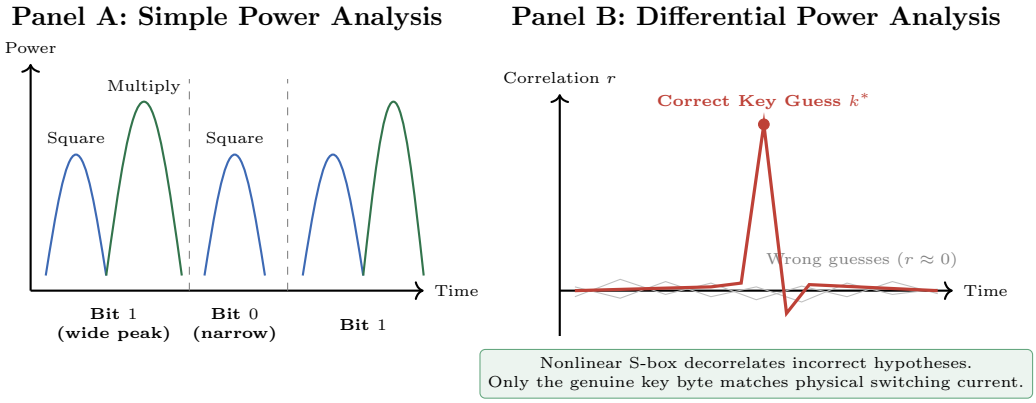


Figure 37.3: Power side-channel analysis. Panel A illustrates Simple Power Analysis (SPA) on unmitigated modular exponentiation: squarings and multiplications produce distinct electrical profiles, permitting direct visual extraction of exponent bits. Panel B illustrates Differential Power Analysis (DPA): statistical correlation of measured power against hypothetical intermediate Hamming weights yields a singular correlation spike exclusively at the true key candidate.

37.4.2 Differential power analysis (DPA)

When the per-bit signal is buried in noise too deep for the eye, DPA digs it out statistically, and it is genuinely clever. The attacker does not need to understand the chip's electrical details. She needs only a *leakage model* – a guess at how power correlates with data – and many traces.

The standard model: the power a chip draws at the moment it computes an intermediate value v is correlated with the *Hamming weight* of v (number of one-bits), because more one-bits means more transistors charged high. So if Eve can predict an intermediate value v as a function of the known plaintext and a *small piece* of the key, she can split her traces into groups by predicted Hamming weight and look for a power difference. Here is the divide-and-conquer that makes it devastating.

Attack

DPA on the first round of AES. The first-round S-box output for byte i is $v_i = S(p_i \oplus k_i)$, where p_i is a known plaintext byte and k_i is one *unknown key byte*. Eve attacks k_i in isolation – only 256 possibilities.

1. Collect T power traces, each with its known plaintext $p^{(j)}$, $j = 1 \dots T$.
2. For each *guess* $\hat{k}_i \in \{0, \dots, 255\}$, predict the intermediate $\hat{v}^{(j)} = S(p_i^{(j)} \oplus \hat{k}_i)$ for every trace, and compute its Hamming weight as the model's predicted power.
3. Correlate the predicted power against the measured power (at the cycle where the S-box happens) across all T traces – e.g. Pearson correlation (this is CPA, correlation power analysis, the modern form).
4. For the *wrong* key guesses, the prediction is decorrelated from reality: correlation ≈ 0. For the *one correct* guess of k_i, the predicted intermediate equals the real one, so the model matches the measurements and the correlation spikes.
5. The guess with the highest correlation is k_i. Repeat for all 16 key bytes *independently*.

Total work: $16 \times 256 = 4096$ key-byte hypotheses tested by correlation, instead of 2^{128}. Smart cards have fallen to a few thousand traces.

Why it works

Why does only the correct key byte light up? Because AES's S-box is a good nonlinear function: $S(p \oplus \hat{k})$ for a wrong $\hat{k}$ is, as p varies, statistically unrelated to $S(p \oplus k)$ for the true k. Their Hamming weights are essentially independent, so the model built on a wrong guess has no correlation with the real power. Only when $\hat{k} = k$ does the predicted intermediate *equal* the actual intermediate the chip computed, at which point the model's Hamming weight tracks the real switching activity and the correlation is large. The S-box's nonlinearity, a strength against cryptanalysis, is what makes the correct key the unique correlation peak – the cipher's good design sharpens the side channel.

Connection

DPA is applied statistics. The correlation peak at the right key emerges from the same signal-in-noise math as the timing attack: each trace is signal plus independent noise, and correlating over T traces makes the true-key peak rise out of the noise floor at a rate governed by $\sqrt{T}$

(see the probability companion on correlation and the central limit theorem). Doubling your trace count does not double your reach – it improves the signal-to-noise of the peak by $\sqrt{2}$. The defender's job is to lower the per-trace signal (so you need impractically many traces) by masking and shuffling, discussed next.

Why it works

Countermeasures: masking and hiding. *Masking* randomizes the intermediates: instead of computing on v, compute on $v \oplus r$ for a fresh random mask r (carried correctly through the cipher and removed at the end). Now the leaked Hamming weight is of $v \oplus r$, which to an attacker who does not know r is random – the correlation with any key-dependent prediction of v collapses. *Hiding* reduces the signal directly: add noise, randomize the *order* of independent operations (shuffling the 16 S-box evaluations so the attacker does not know which cycle is byte i), insert random delays, or balance the hardware so every bit switch draws equal power (dual-rail logic). Real secure elements stack several of these, and DPA-resistance is a certification requirement (Common Criteria, FIPS 140) for payment and identity chips.

In the real world

Power and EM analysis is the threat model for anything an adversary can hold: EMV chip cards, SIM cards, TPMs, hardware wallets, set-top-box and game-console security chips, automotive immobilizers. EM has a nasty advantage over power – it needs no electrical contact (a near-field probe over the package suffices) and can be *localized* to the region of the die doing the crypto, improving signal. The entire smart-card security industry exists to make these attacks cost more (in traces, equipment, and expertise) than the secret is worth, because you cannot make a physical chip leak *nothing* – you can only push the attack beyond economic reach.

37.5 Fault Attacks: Breaking the Machine on Purpose

The channels so far were *passive*: Eve observed. Fault attacks are *active*: Eve makes the chip *compute wrong* – by glitching the clock, dropping the supply voltage for a nanosecond, zapping the die with a laser pulse or an EM spike, or heating it – and then exploits the erroneous output. A surprising amount of cryptography shatters when you can induce one well-placed error, because the difference between a correct and a faulty result is itself a leak of the secret.

37.5.1 The Bellcore attack on RSA-CRT

The classic fault attack on RSA-CRT is striking enough to trace in full: a *single* faulty RSA signature factors the modulus.

RSA signing computes $s = m^d \bmod N$ where $N = pq$. For speed, virtually every implementation uses the Chinese Remainder Theorem (CRT): compute the signature mod p and mod q separately, then recombine. Define

$$s_p = m^d \bmod p, \qquad s_q = m^d \bmod q,$$

and recombine with CRT coefficients into s with $s \equiv s_p \pmod p$ and $s \equiv s_q \pmod q$. This is about four times faster than one exponentiation mod N, which is why it is universal.

Now suppose Eve induces a fault during the computation of s_p *only* – a single glitch corrupts the mod-p half – so the device outputs a wrong signature $\hat{s}$ with

$$\hat{s} \equiv \hat{s}_p \pmod p \quad \text{(garbage)}, \qquad \hat{s} \equiv s_q \pmod q \quad \text{(still correct)}.$$

The faulty signature is wrong modulo p but *right* modulo q. Watch what that hands Eve (Figure 37.4).

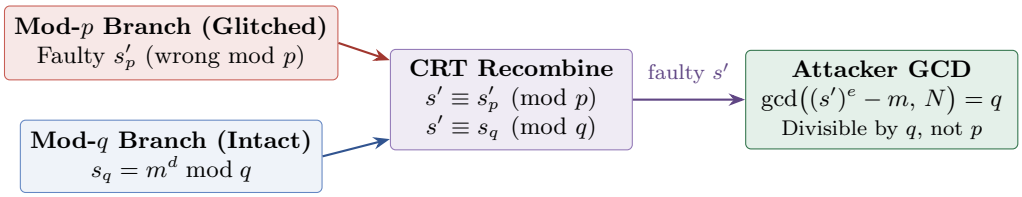


Figure 37.4: The Bellcore fault attack on RSA-CRT. A single hardware fault in the mod-p exponentiation leaves the recombined signature s' valid modulo q but invalid modulo p; consequently $(s')^e - m$ is a multiple of q and not p, so one gcd with $N = pq$ immediately reveals q.

Why it works

One faulty signature factors N. A correct signature satisfies $s^e \equiv m \pmod N$. The faulty one satisfies $\hat{s}^e \equiv m \pmod q$ (correct mod q) but $\hat{s}^e \not\equiv m \pmod p$ (faulted mod p). Therefore

$$\hat{s}^e - m \equiv 0 \pmod q, \qquad \hat{s}^e - m \not\equiv 0 \pmod p.$$

A number that is $\equiv 0 \pmod q$ but $\not\equiv 0 \pmod p$ shares the factor q with $N = pq$ but not the factor p. So

$$\boxed{q = \gcd(\hat{s}^e - m,\ N)}$$

and one Euclidean GCD – microseconds – yields q, hence $p = N/q$, hence the entire private key. Eve needs the public (N, e), the message m, and *one* faulty signature $\hat{s}$. The math that should have protected the key is undone by a single induced error. (If Eve also has a correct signature s for the same m, an even simpler form works: $q = \gcd(s - \hat{s}, N)$, since $s - \hat{s} \equiv 0 \pmod q$ but $\not\equiv 0 \pmod p$.)

Example 37.2. Toy RSA: $p = 11$, $q = 13$, $N = 143$, $e = 7$. The private exponent is $d = 103$ (since $7 \cdot 103 = 721 = 1 + 6 \cdot 120$ and $\varphi(N) = 120$). Sign $m = 50$.

Correct CRT halves: $s_p = 50^{103} \bmod 11$ and $s_q = 50^{103} \bmod 13$. Now suppose the mod-p half is faulted to $\hat{s}_p = 3$ (some wrong value) while s_q stays correct. CRT recombination produces some $\hat{s}$ with $\hat{s} \equiv 3 \pmod{11}$ and $\hat{s} \equiv s_q \pmod{13}$. Eve computes

$$\hat{s}^{\,e} - m \pmod N \quad \text{and then} \quad \gcd(\hat{s}^e - m,\ 143)\,.$$

Because the result is divisible by 13 (correct mod q) but not by 11 (faulted mod p), the GCD returns $13 = q$. Then $p = 143/13 = 11$. The key falls out of a single bad signature and one GCD – trace it with these numbers and the mechanism is undeniable.

Why it works

The defense is to verify before releasing. The fix is humble: after computing a signature, *check it* with the public key ($s^e \stackrel{?}{=} m \bmod N$) before outputting it. A faulted signature fails this check and is suppressed – Eve never sees $\hat{s}$, so she has nothing to GCD. More robust schemes compute the CRT halves redundantly and compare, or use "infective" computation where a fault scrambles the output into uselessness rather than into something exploitable. The general principle for fault resistance: **never emit a result you have not checked**, because a faulted output still carries algebraic information about the secret key.

Attack

Fault attacks beyond RSA. *Differential fault analysis* (DFA) on AES: induce a single-byte fault in an intermediate round, collect the correct and faulty ciphertexts, and the difference propagates through the known final rounds in a way that, after a small search, pins down the last round key (and thus the master key) from as few as one or two faulty encryptions. *Signature-verification skipping*: glitch the branch that checks a signature so it returns "valid" regardless – the classic way to defeat secure boot on game consoles and locked phones, where the goal is not to recover a key but to make the device accept unsigned code. *Safe-error attacks*: induce a fault in a dummy operation; if the output is still correct, the operation was a dummy (so the corresponding secret bit was, say, 0), leaking the bit through whether the fault *mattered*.

In the real world

Fault injection is the daily reality of high-security hardware evaluation. Labs maintain laser-fault benches, EM-glitch coils, and voltage/clock glitchers to certify that smart cards and secure elements survive. Countermeasures shipped in real silicon: voltage and clock-frequency sensors that halt on anomaly, light sensors under the package to detect decapping, redundant computation with comparison, error-detecting codes on the datapath, randomized timing so the attacker cannot aim a glitch at the right cycle, and – for RSA – the verify-before-output check above. "Tamper-resistant" on a datasheet means, in large part, "fault-resistant."

37.6 Padding Oracles: When an Error Message Is the Channel

The side channels so far were physical or microarchitectural. The last and most consequential class is purely logical: the implementation tells the attacker *why* it rejected a ciphertext, and that "why" – one bit of distinction, "bad padding" versus "bad something-else" – is enough to decrypt or forge. These *padding oracle* attacks broke real RSA and real TLS in the field, repeatedly, for twenty years.

37.6.1 The AES-CBC padding oracle

Recall CBC decryption. Ciphertext blocks $C_1, C_2, \ldots$ (with IV $= C_0$) decrypt as

$$P_i = D_k(C_i) \oplus C_{i-1},$$

where D_k is the block cipher's decryption. Because plaintext must fill whole blocks, it is padded – the standard is PKCS#7: if b bytes are needed, append b copies of the byte value b. So one missing

byte is padded `01`; two missing bytes `02 02`; and so on up to a full block `10 10 ... 10` (sixteen $0x10$ bytes). On decryption the receiver checks that the trailing bytes form valid PKCS#7 padding and strips them.

Here is the fatal design: an early TLS (and countless homemade protocols) returned a *distinguishable* error – one message for "padding invalid," another for "padding valid but MAC failed" (or it leaked the distinction through timing). That one bit is an oracle: a function $\mathcal{O}(C)$ that returns whether the decryption of C has valid padding. Eve, who cannot break AES, decrypts *any* ciphertext using only this oracle.

Attack

Decrypting one block with a padding oracle. Eve wants the plaintext $P_2 = D_k(C_2) \oplus C_1$ of a captured block C_2. Write $I_2 = D_k(C_2)$ (the cipher's raw decryption of C_2, which she cannot compute directly), so $P_2 = I_2 \oplus C_1$. Eve will recover I_2 byte by byte by sending C_2 preceded by a *forged* previous block C' that she controls, and asking the oracle whether the padding is valid.

Last byte first. Eve sends the two-block ciphertext (C', C_2) for a chosen C'. The decrypted plaintext of C_2 in this forged message is $P' = I_2 \oplus C'$. She tweaks the last byte $C'[15]$ through all 256 values until the oracle says "valid padding." Valid padding almost always means the last byte decrypted to `01` (a one-byte pad). So at the accepted byte value,

$$I_2[15] \oplus C'[15] = \texttt{0x01} \quad \Longrightarrow \quad I_2[15] = C'[15] \oplus \texttt{0x01}.$$

She has the last byte of I_2 – and thus $P_2[15] = I_2[15] \oplus C_1[15]$, using the *real* previous block C_1. About 256 queries (often fewer; on average 128).

Next byte. To attack $I_2[14]$, she forces the last byte to pad value `02` by setting $C'[15] = I_2[15] \oplus \texttt{0x02}$ (she knows $I_2[15]$ now), then varies $C'[14]$ until the oracle accepts – which means the last two bytes are `02 02`, giving $I_2[14] = C'[14] \oplus \texttt{0x02}$. Continue down to byte 0: about $16 \times 256 \approx 4096$ queries to recover the entire block, with *zero* knowledge of the key.

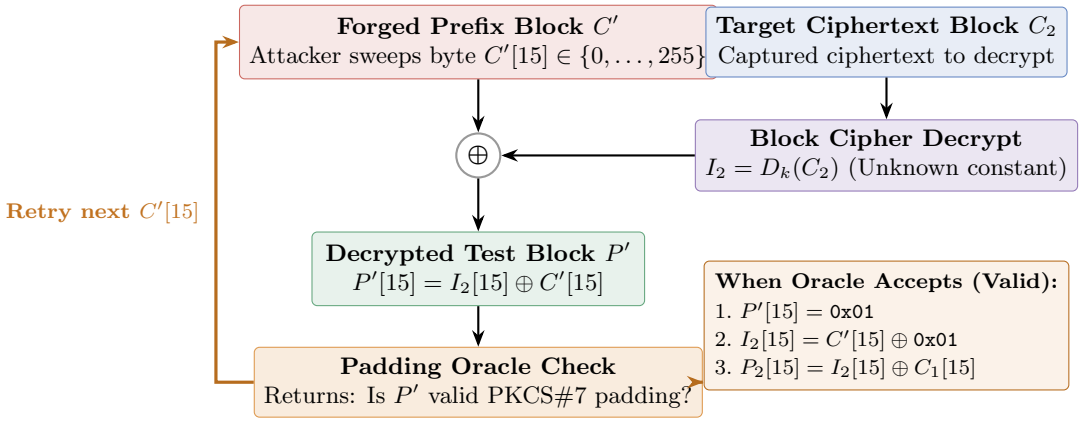


Figure 37.5: CBC padding oracle exploitation. The adversary precedes captured block C_2 with a manipulated block C'. Sweeping $C'[15]$ until the oracle signals valid padding steers the decrypted byte to the known constant `0x01`, algebraically disclosing the internal block cipher output $I_2[15]$ and the true plaintext byte $P_2[15]$ without the secret key.

Walking question

Why does "valid padding" pin down a byte? Because PKCS#7 padding values are *known constants*. When Eve forces the last decrypted byte to be `01`, she has set up an equation $I_2[15] \oplus C'[15] = \texttt{0x01}$ with exactly one unknown, $I_2[15]$, and she controls $C'[15]$, so she solves it. Rather than revealing plaintext directly, the oracle signals when the decrypted byte matches a known padding constant, allowing the adversary to back-solve the cipher's internal intermediate state I_2. Because recovering I_2 requires no knowledge of the key, the adversary recovers the plaintext block as $P_2 = I_2 \oplus C_1$.

Intuition

The oracle answers one tiny question – "is the padding well-formed?" – and you would swear that leaks nothing about the plaintext. It leaks everything, because Eve gets to *choose* the question's context. By controlling the previous block she controls what the last byte decrypts to, and the oracle confirms when she has steered it to a value she already knows (`01`, then `02 02`, ...). Each confirmation is one solved equation in one unknown. The general rule to remember: **any** observable that distinguishes "valid" from "invalid" on attacker-chosen ciphertexts is a decryption oracle waiting to happen.

Implement it

Toy padding oracle (learning only). The skeleton, to make the loop concrete – never run anything like this on a real channel, and never *be* the server that answers these queries:

```
# LEARNING ONLY. Eve has oracle(c) -> True iff CBC-decrypt(c) has valid PKCS#7
    padding.
def recover_block(C1, C2, oracle):        # returns plaintext of C2
    I2 = bytearray(16)                    # the cipher's raw decrypt D_k(C2)
    Cp = bytearray(16)                    # forged "previous" block we control
    for pos in range(15, -1, -1):         # last byte to first
        pad = 16 - pos                    # target pad value: 1, then 2, ...
        for j in range(pos + 1, 16):      # set already-known tail to the pad value
            Cp[j] = I2[j] ^ pad
        for guess in range(256):
            Cp[pos] = guess
            if oracle(bytes(Cp) + bytes(C2)):
                I2[pos] = guess ^ pad     # solved one byte of I2
                break
    return bytes(b ^ c for b, c in zip(I2, C1))    # P2 = I2 XOR real C1
```

The whole attack is one bit of feedback per query, amplified by attacker-chosen context into full plaintext recovery.

Why it works

The structural fix is encrypt-then-MAC. The disease is that the receiver *decrypts and parses padding on unauthenticated ciphertext*, so the attacker's chosen ciphertext is processed and an oracle answer leaks. In encrypt-then-MAC, the receiver first verifies a MAC over the ciphertext; if the MAC fails, it rejects *before decrypting at all*, so no padding check is

ever reached and there is no oracle. This is one of the central reasons AEAD (Chapter 16) authenticates the ciphertext: a forged or tweaked ciphertext is rejected by the tag check, uniformly, with one error and constant time, long before any structure-dependent processing. Padding oracles are what you get when you decrypt things you have not authenticated.

37.6.2 Bleichenbacher: the RSA padding oracle that will not die

CBC was not the first padding oracle. In 1998 Daniel Bleichenbacher found one against *RSA encryption* with PKCS#1 v1.5 padding, and it is so durable it has been re-broken in TLS roughly once a decade (DROWN in 2016, ROBOT in 2017) because the vulnerable code path keeps coming back.

Attack

Bleichenbacher's "million message" attack. In RSA PKCS#1 v1.5 encryption, a message is padded to `00 02 [random nonzero bytes] 00 [message]` before being raised to the public exponent. On decryption the server checks that the result starts with `00 02`. Many servers (or the TLS handshake's behavior) revealed – through an error message, a timeout, or a connection reset – whether a decrypted ciphertext had the correct `00 02` prefix. That is the oracle: $\mathcal{O}(c)$ returns whether $c^d \bmod N$ begins with `00 02`, i.e. whether the decrypted value lies in the interval $[2B, 3B)$ where $B = 2^{8(\ell-2)}$ for an ℓ-byte modulus.
Eve wants to decrypt a target ciphertext $c_0 = m^e \bmod N$ without the key. Exploiting RSA's malleability, she sends $c_0 \cdot s^e \bmod N$ for chosen multipliers s; this decrypts to $m \cdot s \bmod N$. Each oracle hit tells her $m \cdot s \bmod N \in [2B, 3B)$, which is a *linear constraint* narrowing the interval that must contain m. By cleverly choosing successive s values she shrinks the candidate interval for m geometrically until a single value remains – the plaintext (often a TLS pre-master secret, i.e. the session key). The original attack took on the order of a million oracle queries; modern refinements need far fewer.

Why it works

Bleichenbacher is the same shape as the CBC oracle, lifted into RSA's multiplicative structure. There, attacker-chosen *previous blocks* steered a known padding byte; here, attacker-chosen *multipliers* s steer the decrypted value, and "valid `00 02` prefix" replaces "valid PKCS#7." Both turn a one-bit validity oracle on chosen ciphertexts into full recovery; both exist because the implementation reveals *why* it rejected. The countermeasures are equally telling: PKCS#1 v1.5 encryption is replaced by RSA-OAEP (which has a security proof against chosen-ciphertext attack), and where v1.5 must persist for compatibility, servers must make the padding-valid and padding-invalid paths *indistinguishable* – identical timing, identical error, no early-out – which is exactly the constant-time discipline from the start of this chapter applied to error handling. Even better, modern TLS prefers (EC)DHE key exchange, which never RSA-encrypts a secret to the server at all, deleting the oracle's home.

37.6.3 Lucky 13: the padding oracle hiding in the timing

Unifying error messages so that invalid padding and invalid MAC return the same error code does not eliminate the oracle if the two rejection paths take different amounts of time. In 2013, Nadhem

AlFardan and Kenny Paterson showed that timing differences in TLS-CBC decryption still expose padding validity. This is Lucky 13, and it combines both major themes of this chapter – padding oracles and timing leaks – in a single attack.

Attack

Lucky 13 against TLS-CBC (MAC-then-encrypt). Old TLS used MAC-then-encrypt: compute MAC over the plaintext, append it, pad the result, then CBC-encrypt. On decryption the receiver CBC-decrypts, removes padding, then MACs the rest and compares. The error was unified – same "bad_record_mac" alert either way – so the naive padding oracle was closed. But the *time* to produce that error was not constant.

The MAC is HMAC over the data, and HMAC's running time depends on how many compression-function blocks it processes. After removing padding, the amount of data left to MAC depends on *how many padding bytes were stripped* – which depends on the (attacker-influenced) decrypted padding. When Eve manipulates a ciphertext so that decryption yields, say, a single `00` byte interpreted as 1 byte of padding, the MAC runs over more data (and one more compression block) than when it yields a long valid-looking pad. That difference – a single SHA-1 compression call, a few hundred nanoseconds – depends on the plaintext bytes. Timing the unified error reconstructs the padding-validity bit, and the CBC padding-oracle attack proceeds as before. The name: it took advantage of a lucky coincidence in the 13-byte TLS header plus MAC arithmetic that placed a compression-block boundary just right.

Why it works

Lucky 13 is the capstone because it fuses two earlier lessons. It is a *padding oracle* (the secret is which padding got stripped) delivered through a *timing channel* (how long HMAC ran), defeating a defense (unify the error message) that addressed only the logical channel and forgot the physical one. The countermeasure makes decryption time independent of padding length: always processing a fixed number of MAC compression blocks regardless of how much padding was stripped, and discarding the excess in constant time. This requires performing dummy operations so that the running time carries no information about plaintext structure, reinforcing the necessity of modern AEAD modes over legacy CBC MAC-then-encrypt constructions.

Common trap

Notice the escalation across the three padding attacks. CBC oracle: a *distinct error message* leaks the bit – fix by unifying the error. Bleichenbacher: even an error *or a timeout or a reset* leaks the bit – fix by making every path identical. Lucky 13: even with identical errors, the *time* leaks the bit – fix by constant-time error handling. Each fix closes one channel and reveals the next one underneath. There is no point at which "just hide the error" is enough. The only robust answer is structural: authenticate first (encrypt-then-MAC / AEAD) so the attacker's chosen ciphertexts are rejected uniformly before any secret-dependent processing happens at all.

37.7 Constant-Time Programming: The Discipline

Every defense in this chapter converged on one idea: make the implementation's observable behavior independent of secret data. That idea has a name and a set of rules, and it is the single most important practical skill in this chapter. The rules are simple to state and unnatural to follow, because they forbid the exact constructs that make ordinary code fast and readable.

Formal

The constant-time rules. For any value that depends on a secret (a key, a plaintext under attack, a nonce, a private exponent, an intermediate of these), the implementation must ensure that none of the following *depend on that secret*:

1. **Control flow.** No branch (`if`, loop bound, early `return`, `switch`) whose condition is a secret. Branches change timing, instruction-cache footprint, and power.
2. **Memory access addresses.** No array index, table lookup, or pointer computed from a secret. Addresses change cache footprint (the cache-attack channel) and sometimes timing.
3. **Variable-time instructions.** Avoid instructions whose latency depends on operands – on some CPUs integer division, and historically multiplication, take data-dependent time. Use only fixed-latency operations on secrets.

Equivalently: a secret may flow only into operations that take the same time and touch the same memory regardless of its value – bitwise logic, addition, fixed-latency multiply, and constant-time *selection*.

The replacement for a secret-dependent branch is a *branchless select* (Figure 37.6): compute *both* possibilities and pick one with arithmetic on a mask, so the same instructions run every time.

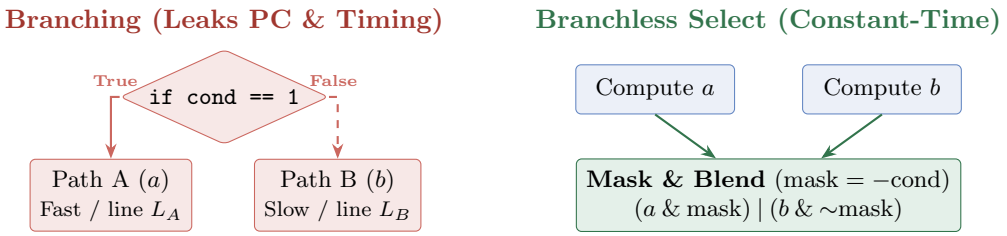


Figure 37.6: Secret-dependent branching versus constant-time selection. A conditional branch executes only one arm, leaking the secret condition through execution time, branch-predictor state, and instruction-cache lines; branchless selection evaluates both inputs and blends them with an all-ones or all-zeros bitmask so identical instructions run every time.

Implement it

Branchless conditional, the core mechanism of constant-time code (learning only).

```
# Select a or b WITHOUT branching on the secret bit.
# Build a full-width mask from a 0/1 condition, then blend.
def ct_select(cond, a, b, width=32):
    # cond is 0 or 1 (secret). mask = all-ones if cond==1 else all-zeros.
```

```
    mask = -cond & ((1 << width) - 1)   # -1 -> 0xFFFF...; 0 -> 0x0000...
    return (a & mask) | (b & ~mask & ((1 << width) - 1))

# Constant-time "is x zero?" without `if x == 0`:
def ct_is_zero(x, width=32):
    # fold all bits down; result bit 0 is 1 iff x was 0
    x &= (1 << width) - 1
    return 1 ^ ((x | (-x & ((1 << width) - 1))) >> (width - 1) & 1)
```

Both possibilities are evaluated; the secret only chooses the mask, and masking-and-blending runs identically for every secret value. The same trick gives constant-time table lookup: to read `T[i]` for secret `i`, read *every* entry and select with a per-index mask – $O(n)$ work to hide which index you wanted.

Learning only. Real constant-time code is written in C or assembly with great care, because high-level languages and optimizing compilers actively fight you: a compiler is free to "optimize" your branchless select back into a branch, or to introduce a data-dependent short-circuit, because the language standard says nothing about timing. Production code uses audited libraries (libsodium, BearSSL, HACL*, fiat-crypto), compiler barriers, and verification tools.

Common trap

The compiler is not your friend here. Constant-time source code is not the same as constant-time machine code. Optimizing compilers reason about *values*, never about *time*, so they may turn your carefully branchless `ct_select` into a conditional move on one target and a predicted branch on another, or replace a constant-time multiply with a shift-and-add that leaks. "Constant-time" is a property of the emitted instructions on a specific CPU, not of your C. This is why serious projects verify the binary – with tools like `ctgrind` / Valgrind-based taint tracking, `dudect` (statistically testing whether timing depends on input), `ct-verif`, or by writing the primitive in assembly and auditing it – rather than trusting that the source "looks constant-time."

Intuition

The mental flip you must make: in ordinary programming, doing less work is a virtue – bail early, skip the unnecessary, branch around the common case. In constant-time crypto, doing *the same work every time* is the virtue, even when most of it is wasted, because variation *is* the leak. The constant-time comparison reads every byte even after a mismatch. The Montgomery ladder multiplies even for zero bits. The Lucky-13 fix MACs phantom blocks and throws them away. You are deliberately spending cycles to buy silence. "Wasteful but uniform" beats "efficient but talkative" whenever the data is secret.

In the real world

Where this lives in the real world: TLS libraries (OpenSSL/BoringSSL/LibreSSL) ship constant-time modular exponentiation, constant-time RSA padding checks (the post-Bleichenbacher "always do the full work and return a random secret on failure" pattern), and constant-time CBC-MAC processing (the post-Lucky-13 fix). Curve25519 and Ed25519 were *designed* for constant-time implementation – their formulas avoid secret-dependent branches

and the Montgomery ladder is natural – which is a large part of why they displaced older curves with awkward special cases. Signal, WireGuard, and modern SSH lean on these constant-time-by-design primitives. The trend in the field is to push correctness further: `fiat-crypto` machine-generates field arithmetic with a proof of constant-time behavior, and HACL* provides formally verified, constant-time crypto used in Firefox and the Linux kernel. The endgame is to make "constant-time" a checked property of the shipped code, not a hopeful comment.

37.8 Why "Secure on Paper" Is Not Enough

A security proof is a statement about an idealized *model*: in that model the adversary observes only specified inputs and outputs, the underlying problem is hard, and the scheme is provably secure. In practice, however, a physical adversary may observe execution time, power consumption, electromagnetic radiation, cache state, error messages, and the results of induced faults—observables omitted from the abstract model and therefore unaddressed by the proof.

Focus

A proof of security is conditional: *relative to a threat model*, the scheme is secure. Side channels are observables the threat model left out. So the correct reading of every theorem is, "secure against an adversary limited to the modeled interface." Deployment widens the interface. The discipline of this chapter is the work of either (a) narrowing the real interface back down to the modeled one – constant-time code removes the timing/cache observables; verify-before-output removes the fault observable; encrypt-then-MAC removes the padding observable – or (b) widening the *model* to include the leak and proving security against it (leakage-resilient cryptography, masking with proofs in the probing model). You close the gap from one side or the other; you do not get to ignore that it exists.

Intuition

Mathematical security of an abstract primitive does not imply side-channel resistance of a physical implementation. A protocol provably immune to mathematical cryptanalysis can still leak its key through execution time, power consumption, cache contention, or error messages. Mitigating these implementation-level side channels requires constant-time programming, hardware masking, and uniform error handling.

Pause

Pause and consolidate. A side channel is any secret-dependent observable the security proof omitted. The signature move of every attack is collapsing a multiplicative search (s^n) into an additive one ($s \cdot n$) by leaking each secret component independently. Timing leaks through data-dependent branches and operations; the fix is constant-time code (compare all bytes, ladder all bits). Cache leaks through secret-dependent memory addresses; the fix is no secret-indexed lookups (bit-slicing, AES-NI). Power/EM leaks through data-dependent switching; the fix is masking and hiding. Faults leak through the difference between correct and corrupted output; the fix is verify-before-output and redundancy. Padding oracles leak through

any "valid/invalid" distinction on chosen ciphertexts – message, timeout, or timing; the fix is authenticate-first (AEAD / encrypt-then-MAC) and constant-time error handling. Underneath all of it: secure on paper is necessary, not sufficient, and constant-time programming is the craft that makes the paper proof describe the running machine.

Beyond physical and microarchitectural side channels, real systems fail when engineers use sound primitives outside their preconditions – reusing nonces, choosing ECB mode, or getting the order of encryption and authentication backwards. In Chapter 38, we catalog these common pitfalls before turning in Chapter 39 to how to implement and deploy cryptography safely.

Exercises

Exercise 37.1. Take `insecure_equal` from Section 37.2.1. Suppose each byte comparison takes exactly 1 time unit and the function returns immediately on the first mismatch. The true 4-byte tag is $t = (\texttt{A3}, \texttt{17}, \texttt{C9}, \texttt{2E})$. Eve probes byte 0 with all 256 values. Give the running time the server reports for the guess $t'[0] = \texttt{A3}$ versus any wrong guess, and explain how Eve identifies byte 0 from those times. Then state the total number of queries to recover all 4 bytes and compare it to blind forgery (256^4).

Exercise 37.2. A timing attack has a true signal of $\Delta = 20$ ns between a correct and incorrect byte guess, and per-measurement noise with standard deviation $\sigma = 5{,}000$ ns. Using $N \gg (\sigma/\Delta)^2$, estimate the order of magnitude of samples needed per byte to resolve the signal by simple averaging. If averaging 10^4 samples takes one second, roughly how long does recovering one byte take? Comment on why attackers invest in better statistics than the plain mean.

Exercise 37.3. Trace `modexp_leaky` on exponent $d = 22 = 10110_2$. List, in order, the sequence of squarings and multiplications. How many of each? An attacker watching a *different* run observes the operation sequence (e.g. via SPA) `SM, SM, S, SM, S` (S = square, SM = square-then-multiply). Reading SM as a one-bit and S as a zero-bit, what is that exponent's bit pattern, and what is its value d? Explain why the Montgomery ladder makes both $d = 22$ and $d = 16$ produce identical observable traces.

Exercise 37.4. In a Prime+Probe attack on an AES T-table, the table has 256 one-byte entries, and the cache line holds 16 bytes, so accessing entry j loads the line $\lfloor j/16 \rfloor$ – there are 16 distinct lines. Eve observes that for a known plaintext byte $p_i = \texttt{0x5A}$, the lookup `T[p_i XOR k_i]` landed in cache line 3. Which 16 values of `p_i XOR k_i` are consistent with line 3, and what does this tell Eve about the high nibble of k_i? How many such observations (with varying p_i) does she roughly need to pin k_i to a single value?

Exercise 37.5. In a DPA/CPA attack on AES round one, the leakage model is the Hamming weight of $S(p_i \oplus k_i)$. Explain why correlating the model against measured power produces a sharp peak only at the correct $\hat{k}_i = k_i$ and near-zero correlation elsewhere. In your answer, use the fact that the AES S-box is highly nonlinear. Then explain precisely how Boolean masking (computing on $S(p_i \oplus k_i) \oplus r$ for secret random r) destroys the correlation, and why the attacker cannot simply also guess r.

Exercise 37.6. With toy RSA $p = 11$, $q = 13$, $N = 143$, $e = 7$, message $m = 50$: the correct signature is $s = 50^{103} \bmod 143$. Suppose a fault produces $\hat{s}$ that is correct mod 13 but wrong mod 11. Show, with the congruences, why $\gcd(\hat{s}^7 - 50,\ 143) = 13$. Then suppose instead Eve has both

the correct s and the faulty $\hat{s}$; show why $\gcd(s - \hat{s},\ 143) = 13$ as well. State the one-line defense that prevents Eve from ever obtaining $\hat{s}$.

Exercise 37.7. In the CBC padding-oracle attack, Eve is recovering byte 15 (the last byte) of $I_2 = D_k(C_2)$. She varies $C'[15]$ and the oracle accepts at $C'[15] = \texttt{0x8B}$. (i) What is $I_2[15]$? (ii) If the genuine previous block has $C_1[15] = \texttt{0x40}$, what is the plaintext byte $P_2[15]$? (iii) To then attack byte 14, what value must she force $C'[15]$ to, and why? (iv) Explain in one or two sentences why encrypt-then-MAC eliminates the oracle entirely.

Exercise 37.8. Bleichenbacher's oracle returns whether $c^d \bmod N$ begins with the bytes `00 02`, equivalently whether the decrypted integer lies in $[2B, 3B)$ for $B = 2^{8(\ell-2)}$. Eve sends $c_0 \cdot s^e \bmod N$ for chosen s. (i) What does the decryption of this modified ciphertext equal in terms of the target plaintext m? (ii) When the oracle accepts, what interval constraint does Eve learn about $m \cdot s \bmod N$, and hence about m? (iii) Explain why RSA-OAEP is not vulnerable to this attack, and why modern TLS sidesteps it entirely by preferring (EC)DHE key exchange.

Exercise 37.9. Lucky 13 defeated a server that returned the *same* error for bad padding and bad MAC. (i) If the error message is identical, what observable still distinguishes the two cases, and why does it differ? (ii) Explain why the difference is tied to the number of HMAC compression-function blocks processed after padding removal. (iii) Describe the constant-time fix in terms of doing fixed, possibly wasted, work. (iv) Why does switching to an AEAD scheme make this entire class of bug impossible?

Exercise 37.10. Constant-time selection without branching. (i) Verify that `ct_select` from Section 37.7 returns a when `cond=1` and b when `cond=0`, by computing the mask in each case for `width=8`, $a = \texttt{0xF0}$, $b = \texttt{0x0F}$. (ii) Write a constant-time expression for $\max(x, y)$ on w-bit unsigned integers that branches on no secret (hint: derive a sign-based mask from $x - y$). (iii) Explain why testing the *source code* for constant-time behavior is insufficient, and name one tool that tests the compiled binary instead.

Exercise 37.11. You are reviewing a homemade protocol: it CBC-encrypts with AES, appends an unkeyed CRC32 for "integrity," returns distinct errors for "decryption failed" and "CRC failed," and signs with RSA-CRT without verifying the signature before output. Identify *three* distinct side-channel or implementation vulnerabilities in this design, name the attack each enables (from this chapter), and give the one structural change that fixes the most problems at once.

Chapter 38

Common Pitfalls

Almost nobody who loses a cryptographic battle loses it on the mathematics. AES has stood for two decades. RSA-2048 will not be factored by anyone reading this. The discrete logarithm in a 256-bit elliptic curve group is as hard today as it was when the curve was standardized. And yet systems built on exactly these primitives get broken constantly – WiFi networks read off the air, session cookies forged, firmware decrypted, password databases cracked wholesale. The break almost never touches the prime. It happens in the joints: the nonce that was reused, the IV that was predictable, the comparison that returned early, the key that was hardcoded in the binary, the checksum mistaken for a MAC, the same key pressed into two jobs it was never meant to share.

Each pitfall in this chapter has compromised production systems, and each stems from a common failure mode: employing a sound cryptographic primitive outside its operational requirements. A primitive guarantees security only when its preconditions are met—such as unique nonces, uniform keys, constant-time comparisons, and strict key separation. Violating these preconditions voids the security guarantees.

We will work most of these with small numbers, because the failures are not subtle once you watch them happen. Reusing a stream-cipher keystream once turns two ciphertexts into a crossword puzzle you can solve by hand. Encrypting a structured document in ECB mode leaves the structure visible in the ciphertext. Comparing a MAC tag byte-by-byte with early exit leaks the correct tag one byte at a time. None of these need deep theory to exploit – they need only that you stop assuming and start looking. By the end you should have an instinct that fires whenever you see a counter that might repeat, a random value that might be predictable, or a primitive doing two jobs at once.

38.1 The Shape of Every Pitfall

Before the catalog, the pattern. Every cryptographic primitive comes with a contract – a precise statement of what it guarantees and what it demands in return. The one-time pad guarantees perfect secrecy and demands a uniform key as long as the message, used once. AES-GCM guarantees confidentiality and integrity and demands a nonce that never repeats under a given key. A MAC guarantees that a tag cannot be forged and demands that the verifier compare tags without leaking timing. HMAC guarantees collision-resistant authentication and demands that you not reuse the same key for unrelated purposes.

Intuition

A cryptographic primitive is a contract with preconditions, not an unconditional guarantee. Saying "it's AES-encrypted" is like saying "it's bolted down" – the bolt only holds if it went through the hole and into the frame. The math gives you a guarantee with a precondition attached, and the entire discipline of using crypto safely is keeping the preconditions true. Eve does not attack the guarantee. She attacks the precondition you forgot was there.

Why does this pattern dominate real-world failures? Because the preconditions are invisible at the call site. The function is named `encrypt(key, nonce, message)` and it returns a ciphertext whether or not the nonce is fresh. Nothing crashes. Nothing warns. The contract violation is silent, and the system ships, and it looks perfect in testing because testing never reuses a nonce on purpose. The failure surfaces only when an adversary – who reads the spec more carefully than the implementer did – goes looking for exactly the violated clause.

Focus

The recurring lesson of this chapter: **cryptographic correctness is not testable by example.** A function that returns plausible ciphertext for every input you try can still be catastrophically broken, because the break lives in a relationship *between* inputs (two messages under one nonce), or in a property you cannot observe from the output (constant-time behavior), or in a misuse that only an adversary triggers (chosen-ciphertext queries). You cannot unit-test your way to a secure system. You verify the contract is honored, every clause, every call.

We now walk the catalog. The ordering is rough: we start with the misuses that leak plaintext directly, move through integrity and key-management failures, and end with the meta-pitfall of building your own primitives.

38.2 Nonce and IV Reuse

A nonce – "number used once" – is the freshness input that lets a deterministic cipher produce different ciphertexts for the same plaintext. An IV (initialization vector) plays the same role for block-cipher modes. The contract is in the name: used once. Break it and the consequences range from "leaks whether two messages are equal" to "recover the plaintext" to "recover the authentication key and forge arbitrary messages." This is, by volume of real-world damage, the single most destructive pitfall in the book.

38.2.1 The two-time pad

Start with the cleanest case, a stream cipher. Recall the structure: from a key K and a nonce N the cipher generates a keystream $\mathrm{KS}(K, N)$ and encrypts by XOR, $c = m \oplus \mathrm{KS}(K, N)$. The keystream is as long as the message and looks uniformly random to anyone without K. One message, one fresh N: this is as good as a one-time pad. Reuse N for two messages and you fall into the classic stream-cipher trap, the *two-time pad*.

Suppose two plaintexts m_1 and m_2 are encrypted under the same key and the same nonce:

$$c_1 = m_1 \oplus \mathrm{KS}(K, N), \qquad c_2 = m_2 \oplus \mathrm{KS}(K, N).$$

The attacker does not know K and cannot compute KS. But she does not need to. XOR the two ciphertexts:

$$c_1 \oplus c_2 = (m_1 \oplus \mathrm{KS}) \oplus (m_2 \oplus \mathrm{KS}) = m_1 \oplus m_2.$$

The keystream cancels. The attacker now holds $m_1 \oplus m_2$, the XOR of two plaintexts, with the key removed entirely. Everything that follows is linguistics and arithmetic, not cryptography.

Walking question

Why is $m_1 \oplus m_2$ already a disaster, when it is not either plaintext? Because natural-language plaintext is enormously redundant. ASCII text uses a few dozen of the 256 byte values; spaces are everywhere; letters cluster. XOR a space (`0x20`) with a lowercase letter and you get its uppercase counterpart, which is itself a printable letter – a tell-tale signature that one of the two bytes was a space. "Crib-dragging" a guessed word across $m_1 \oplus m_2$ and checking whether the other stream stays readable peels both messages apart. Hold the idea: $m_1 \oplus m_2$ leaks because plaintext is not random, and XOR preserves that structure.

Example 38.1 (Cracking a two-time pad by hand)**.** Take a tiny keystream KS = the bytes `6A 1F 22 0C` (in hex), and two 4-byte plaintexts in ASCII:

$$m_1 = \texttt{"HOME"} = \texttt{48 4F 4D 45}, \qquad m_2 = \texttt{"WORK"} = \texttt{57 4F 52 4B}.$$

Encrypt both with the *same* keystream:

$$c_1 = m_1 \oplus \mathrm{KS} = \texttt{22 50 6F 49}, \qquad c_2 = m_2 \oplus \mathrm{KS} = \texttt{3D 50 70 47}.$$

Eve sees c_1, c_2, not the keystream. She computes

$$c_1 \oplus c_2 = \texttt{1F 00 1F 0E}.$$

The second byte is `00`: the two plaintexts share that byte exactly (both `O`). Now Eve guesses that m_1 starts with `H` (`0x48`). Then byte 0 of m_2 is `0x48` $\oplus$ `0x1F` = `0x57` = `W` – a printable letter, the guess survives. She guesses $m_1[3]$ = `E` (`0x45`); then $m_2[3]$ = `0x45` $\oplus$ `0x0E` = `0x4B` = `K`. Both words fall out together. No key was ever recovered; the keystream's secrecy became irrelevant the moment it was reused.

Attack

MS-PPTP and WEP, the same bug twice. Microsoft's PPTP VPN encrypted both directions of a session with RC4 under keys derived from the user's password, and in early versions effectively reused keystream across the two directions, yielding $m_1 \oplus m_2$ for client and server traffic. WEP (early WiFi, Chapter 26) used a 24-bit IV prepended to a fixed key as RC4's seed. With only 2^{24} IVs and a busy access point cycling through them in hours, IV collisions are guaranteed by the birthday bound – and each collision is a two-time pad. Attackers collected colliding packets, XORed them, and read traffic off the air. The cipher (RC4) was not broken here; the *nonce discipline* was.

38.2.2 CTR mode is a stream cipher, so the same rule binds

CTR mode turns a block cipher into a stream cipher: the keystream block at position i is $E_K(N \,\|\, i)$, the encryption of the nonce concatenated with a counter. It inherits the stream-cipher contract exactly. Reuse the pair (K, N) for two messages and the counter blocks line up identically, so $c_1 \oplus c_2 = m_1 \oplus m_2$ block for block. There is nothing special about CTR's protection here; it is a one-time pad whose pad is $E_K(N \,\|\, i)$, and one-time means one time.

38.2.3 The GCM catastrophe: from confidentiality loss to forgery

AES-GCM raises the stakes. GCM is CTR mode for confidentiality plus a polynomial MAC called GHASH for integrity, sharing one key. The MAC works in the field $\mathbb{F}_{2^{128}}$: it evaluates a polynomial whose coefficients are the ciphertext blocks at a secret point $H = E_K(0^{128})$, then masks the result with $E_K(N \,\|\, 1)$ to produce the tag. The authentication key H is fixed by K; the per-message mask depends on the nonce.

Reuse a nonce in GCM and two separate things break. First, confidentiality: it is still CTR mode, so $c_1 \oplus c_2 = m_1 \oplus m_2$, the two-time pad. Second, and far worse, *integrity collapses*. With two messages encrypted under the same (K, N), the per-message mask $E_K(N \,\|\, 1)$ is identical for both. Subtracting the two tag equations cancels the mask and leaves the attacker a polynomial equation in the single unknown H over $\mathbb{F}_{2^{128}}$. Solving it (finding the roots of a known polynomial over a finite field is easy) recovers H. With H in hand the attacker can compute the GHASH of *any* ciphertext she likes and re-mask it – she can forge valid tags for messages of her choosing. One repeated nonce in GCM does not merely leak two plaintexts; it hands the attacker the integrity key for the whole session.

Why it works

Why does GCM punish nonce reuse so much harder than a plain stream cipher? Because GCM's integrity rests on the secrecy of a single fixed value H, and the only thing hiding H in the tag equation is the per-nonce mask. A fresh nonce gives a fresh, independent mask each time, so each tag is a one-time-pad-protected evaluation and reveals nothing about H. Reuse the nonce and two tags share one mask; the difference of the two equations is mask-free and exposes H. The structure that makes GCM fast (one fixed multiplier H, a cheap per-nonce mask) is exactly the structure that makes nonce reuse fatal. This is worked in full in Chapter 16; the takeaway for the field guide is blunt: *a GCM nonce must never repeat under a key, not once, not ever.*

Focus

GCM nonces are 96 bits. If you generate them randomly, the birthday bound says a collision becomes likely after about 2^{48} messages – which sounds enormous until you picture a busy server. The safe pattern is a *deterministic counter* nonce (message 1, 2, 3, ...), never random, never reset, and a hard stop before the counter could wrap or the key could be copied to a second machine that counts independently. Virtual-machine snapshots and process forks are the classic way a counter silently rewinds and starts repeating – more on that under randomness below.

Implement it

A misuse-resistant alternative exists: *nonce-misuse-resistant* AEAD such as AES-GCM-SIV. It derives the per-message values from a hash of the nonce *and* the plaintext, so repeating a nonce on two different messages degrades gracefully (it leaks only whether the messages were equal) instead of catastrophically. Here is the shape of the defensive idea, not the real construction:

```
# LEARNING ONLY -- not GCM-SIV, not secure, do not deploy.
# Illustrates: derive the "nonce" from the message so identical
# (nonce, message) pairs are the only thing that ever collides.
import hashlib

def synthetic_iv(key, nonce, message):
    # A real SIV uses a PRF/MAC keyed separately; this is a toy.
    h = hashlib.sha256(key + nonce + message).digest()
    return h[:12]            # 96-bit synthetic IV bound to the plaintext

# Two DIFFERENT messages under the same nonce now get DIFFERENT IVs,
# so the keystream differs and there is no two-time pad.
```

The lesson is the design principle: if you cannot guarantee nonce uniqueness operationally, choose a primitive whose failure mode under reuse is mild. Reach for a vetted GCM-SIV library, never this snippet.

38.2.4 Predictable IVs: the CBC case and BEAST

CBC mode demands a different property than CTR: its IV must be *unpredictable*, not merely unique. CBC encrypts block i as $c_i = E_K(m_i \oplus c_{i-1})$, with $c_0 = \text{IV}$. Figure 38.1 shows the chaining: each plaintext block is masked by the previous ciphertext block before encryption, and the IV is the mask for the first block. The first block is $E_K(m_1 \oplus \text{IV})$. If the attacker can *predict* the IV that will be used for a message before that message is encrypted, and she can also influence the plaintext (a chosen-plaintext setting common on the web), she can choose m_1 to cancel her knowledge and test guesses about secret data.

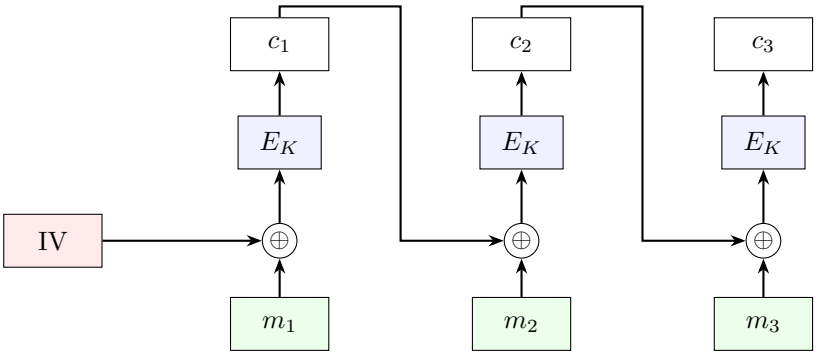


Figure 38.1: CBC encryption. Each plaintext block is XORed with the previous ciphertext block before the cipher is applied; the IV seeds the first block. If an attacker can *predict* the IV (or, as in BEAST, the chained block reused from the previous record) and also choose plaintext, she can cancel the mask and probe secret bytes – which is why a CBC IV must be unpredictable, not merely unique.

This is the BEAST attack on TLS 1.0, which used the *last ciphertext block of the previous record* as the IV of the next – a value the attacker sees on the wire and can therefore predict. The fix in TLS 1.1+ was to make each record's IV explicit and random. The general rule: a unique-but-predictable IV is fine for CTR and fatal for CBC.

Common trap

"Unique" and "unpredictable" are different requirements, and the right one depends on the mode. CTR/GCM: unique is enough, predictable is fine (a counter is ideal). CBC: must be unpredictable (random), because a predictable IV plus chosen plaintext leaks. Using a counter as a CBC IV is a real bug even though the counter never repeats. Read the mode's contract; do not assume one freshness rule fits all.

38.3 ECB Mode: Encryption That Preserves Pictures

Electronic Codebook mode is the naive way to encrypt a long message with a block cipher: split into blocks, encrypt each independently under the same key, concatenate. $c_i = E_K(m_i)$. It is deterministic and stateless, which is precisely why it is broken. Because E_K is a fixed permutation, *equal plaintext blocks produce equal ciphertext blocks.* Every repetition in the plaintext shows through as a repetition in the ciphertext. ECB does not hide patterns; it relabels them.

Intuition

ECB is a substitution cipher whose alphabet is the set of all 2^{128} blocks instead of the 26 letters. We learned in Chapter 2 that monoalphabetic substitution falls to frequency analysis because it preserves the structure of the message. ECB has exactly the same flaw at block granularity. The alphabet is too big to frequency-analyze a block at a time, but any plaintext with repeated blocks – a bitmap, a fixed-format record, a database with repeated fields – broadcasts its repetition straight into the ciphertext.

The famous demonstration is the ECB penguin: encrypt a bitmap image of the Linux mascot in

ECB and the outline of the penguin remains plainly visible, because large monochrome regions are runs of identical blocks that encrypt to identical ciphertext. The colors scramble; the shape survives. Confidentiality that leaks the shape of the secret is not confidentiality.

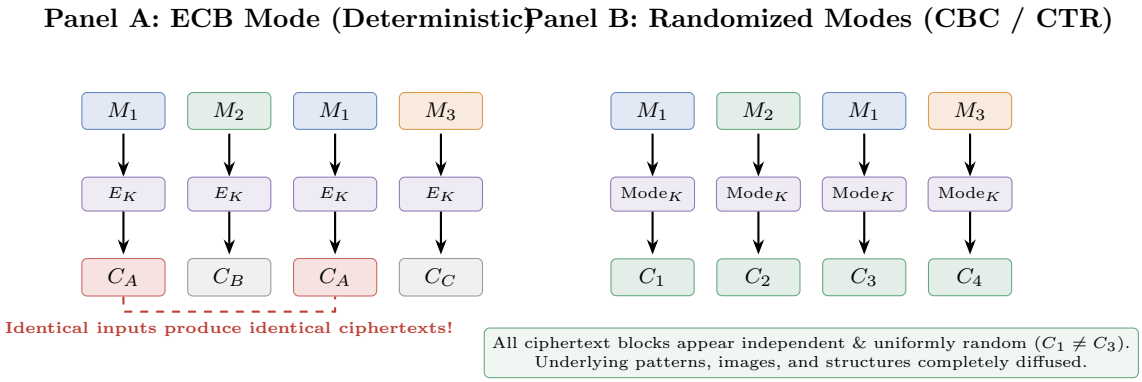


Figure 38.2: ECB mode pattern preservation versus randomized mode diffusion. In Panel A, deterministic block-by-block ECB encryption maps identical plaintext blocks (M_1) to identical ciphertexts (C_A), preserving outlines, repeated fields, and bitmaps (the "ECB penguin" effect). In Panel B, semantic security is restored by stateful or randomized modes (CBC with random IV or CTR with unique nonce), which map identical inputs to distinct pseudorandom blocks.

Example 38.2 (ECB leaks a fixed-format record)**.** Use a toy 4-bit block cipher (block = one hex digit) with the permutation E_K that maps

$$\texttt{1}\to\texttt{7},\quad \texttt{4}\to\texttt{2},\quad \texttt{5}\to\texttt{E},\quad \texttt{A}\to\texttt{9}$$

(only these entries matter here). Encrypt the record `4A4A1554` block by block:

$$\texttt{4A4A1554} \xrightarrow{E_K} \texttt{29297EE2}.$$

Look at the ciphertext `2929 7EE2` without knowing the key. You can immediately read off: positions 0 and 2 are equal, positions 1 and 3 are equal, positions 5 and 6 are equal, and position 7 equals position 0. The plaintext's repetition pattern is fully exposed. If this record were "`FIELDA FIELDA ...`" you would know two fields are identical – maybe two transactions with the same amount, or two users with the same password block. The cipher did its one-block job perfectly and leaked anyway.

Attack

ECB cut-and-paste. Determinism plus block independence lets an attacker *rearrange* ciphertext. Suppose a cookie is `user=alice&role=user` encrypted in ECB with 8-byte blocks aligned so that `role=user` and a forged `role=admin` fall on block boundaries. An attacker who can get the system to encrypt a chosen username can harvest the ciphertext block for `admin`, then splice it over the `user` block in her own cookie. Because each block decrypts independently, the spliced ciphertext decrypts to a valid `role=admin` token. No key needed – ECB's block independence is the whole vulnerability.

Focus

The fix is never "encrypt harder." It is to use a mode that randomizes – CBC with a random IV, or better, an AEAD like GCM or ChaCha20-Poly1305 that gives you confidentiality *and* integrity. ECB has exactly one defensible use in modern practice: encrypting a single block of already-random data (e.g., wrapping a random key), where there is no structure to leak and only one block to repeat. For anything with length or structure, ECB is a bug.

38.4 Getting Encrypt-and-Authenticate Backwards

We devoted Chapter 16 to authenticated encryption; here is the pitfall distilled. There are three ways to combine an encryption scheme with a MAC, and two of them are landmines (Figure 38.3):

- **Encrypt-then-MAC** (EtM): compute $c = \mathrm{Enc}_K(m)$, then $t = \mathrm{MAC}_{K'}(c)$, send $c \,\|\, t$. Verify the tag *before* decrypting. This is the provably sound order.
- **MAC-then-Encrypt** (MtE): compute $t = \mathrm{MAC}_{K'}(m)$, then $c = \mathrm{Enc}_K(m \,\|\, t)$. Used by old TLS. Fragile.
- **Encrypt-and-MAC** (E&M): send $\mathrm{Enc}_K(m) \,\|\, \mathrm{MAC}_{K'}(m)$. Used by SSH. The MAC is computed over plaintext, so it can leak plaintext equality and is not generically secure.

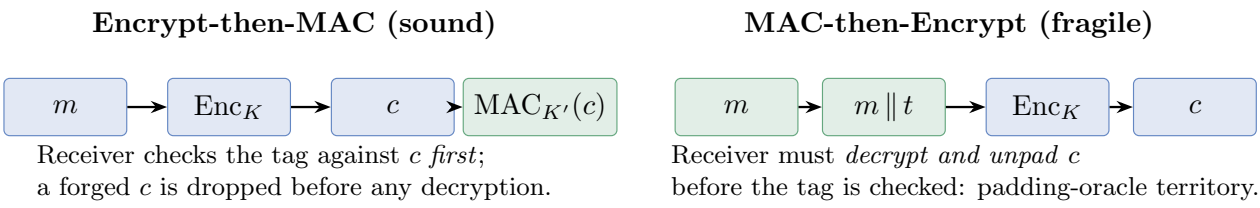


Figure 38.3: The order decides what the receiver must do to a forged ciphertext. Encrypt-then-MAC verifies the tag over the ciphertext bytes the attacker controls, rejecting forgeries before decryption. MAC-then-Encrypt forces decryption and unpadding first, which is the foothold the padding oracle exploits.

The deep reason EtM wins: it lets the receiver *reject forged ciphertexts before touching the decryption logic*. The tag is checked against bytes the attacker fully controls (the ciphertext), and only if it verifies does any plaintext-dependent processing happen. In MtE and E&M the receiver must decrypt (or process plaintext) before or while checking integrity, which opens the door to attacks that learn from *how* decryption fails.

Attack

The padding oracle. CBC mode needs the plaintext padded to a block boundary, typically PKCS#7: if you need p bytes of padding, every padding byte equals p. On receipt the decryptor strips and *checks* the padding. In a MAC-then-Encrypt design (TLS $\leq$ 1.0), the decryptor unpads *before* verifying the MAC. If it reveals – via an error message, or merely by responding faster or slower – whether the padding was well-formed, it has handed the

attacker an oracle: "does this ciphertext decrypt to valid padding?" By flipping bytes in the preceding ciphertext block (CBC malleability) and watching the oracle, the attacker recovers the plaintext one byte at a time, about 128 tries per byte, with no key. This is Vaudenay's padding-oracle attack; its TLS incarnations are POODLE and Lucky Thirteen (where the "oracle" was a timing difference of microseconds). EtM closes it: a forged ciphertext fails the MAC and is dropped before any padding is ever examined.

Example 38.3 (One byte from a padding oracle)**.** Take a CBC ciphertext as two blocks c_0c_1 with a one-byte block for clarity (block size 1, so each "block" is a single byte). Decryption gives $m_1 = D_K(c_1) \oplus c_0$. The attacker does not know $D_K(c_1)$; call it the unknown intermediate $z = D_K(c_1)$, so the genuine plaintext byte is $m_1 = z \oplus c_0$. She does not need K; she needs z. She replaces c_0 with a guess g and submits $g\,c_1$. The oracle decrypts to $z \oplus g$ and reports whether that is valid PKCS#7 padding – for a one-byte block, valid padding means the byte equals `0x01`. So the oracle says "valid" exactly when $z \oplus g =$ `0x01`, i.e. when $g = z \oplus$ `0x01`. She walks g from `0x00` to `0xFF`; the single value that returns "valid" reveals $z = g \oplus$ `0x01`. With z known she recovers the real byte $m_1 = z \oplus c_0$ using the *original* c_0. Concretely, suppose the true intermediate is $z =$ `0x6D`. The guess $g =$ `0x6C` gives $z \oplus g =$ `0x01`: valid. So $z =$ `0x6C` $\oplus$ `0x01` $=$ `0x6D`, and if the genuine $c_0 =$ `0x28` then $m_1 =$ `0x6D` $\oplus$ `0x28` $=$ `0x45` $=$ `E`. At most 256 queries spent one byte; no key was ever touched. Real blocks are 16 bytes, so the attacker peels byte 16 first (forcing padding `0x01`), then byte 15 (forcing the last two bytes to `0x02 0x02`), and so on up the block – 256 guesses per byte, $\leq 16 \times 256 = 4096$ per block.

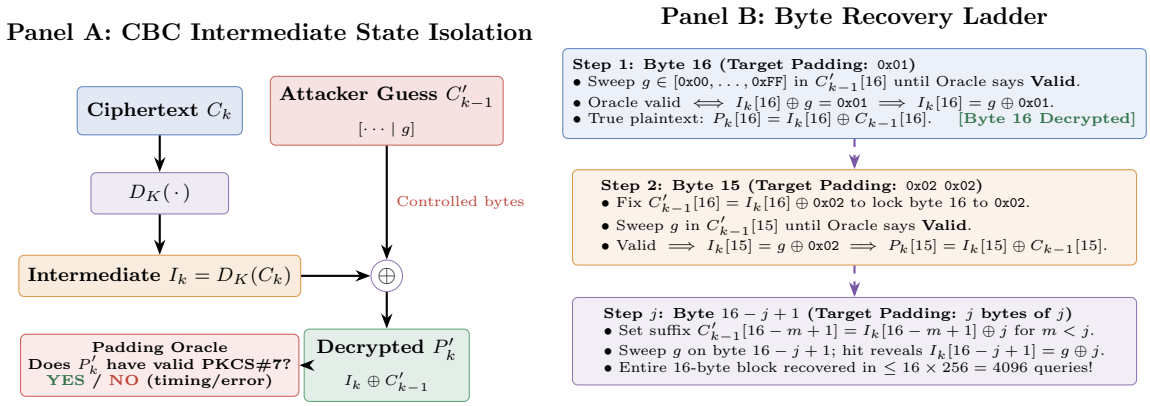


Figure 38.4: CBC padding oracle step-by-step byte recovery ladder. In Panel A, an attacker modifies the preceding ciphertext block C'_{k-1} fed into the decryptor. The underlying intermediate block $I_k = D_K(C_k)$ is fixed. By testing values of guess byte g against the padding oracle, the attacker isolates the exact intermediate byte value. In Panel B, the attack ascends from byte 16 to byte 1: once $I_k[16]$ is known, byte 16 is fixed to pad value `0x02`, allowing byte 15 to be probed. At most $16 \times 256 = 4096$ total queries peel the entire 16-byte block without touching the secret key K.

Why it works

Why does *order* carry so much weight? Because integrity must guard the decryptor's inputs, and the decryptor's inputs are ciphertext bytes. EtM checks the tag against exactly those bytes, so a tampered ciphertext is rejected with no information leaked beyond "invalid." MtE checks the tag against the plaintext, which the receiver can only see after running the very

decryption-and-unpadding code the attacker is trying to probe. The attacker gets to make the receiver process her chosen ciphertext and learn from the outcome. The cleanest defense is not to assemble these parts at all: use a single AEAD primitive that bakes EtM in and gives you one function with the right order already inside it.

Common trap

Even "Encrypt-then-MAC" has a sub-pitfall: **MAC the entire ciphertext, including the IV/nonce and any associated header.** If the IV is sent in the clear but excluded from the MAC, an attacker can tamper with it and, for CBC, scramble the first plaintext block in a controlled way while the tag still verifies. AEAD constructions handle this for you by taking "associated data" that is authenticated but not encrypted. Roll your own EtM and it is easy to leave the IV unauthenticated.

38.5 Weak, Predictable, and Duplicated Randomness

Chapter 17 argued the stakes; here is the pitfall in operational form. Three failure modes recur: a generator with too little entropy (predictable output), a generator seeded identically on two machines (duplicated output), and a per-message secret reused because a counter rewound (repeated output). All three convert a sound algorithm into a public decryption procedure.

38.5.1 Predictable seeds

The textbook blunder is seeding a cryptographic generator from the wall clock, the process ID, or a non-cryptographic PRNG. If a session key is drawn from a generator seeded with the current time in seconds, an attacker searches the plausible timestamp window rather than a 2^{128} keyspace. A key generated "in the last day" has at most $\log_2(86400) \approx 16.4$ bits of real entropy regardless of how many bits it occupies. The attacker enumerates the seeds, not the keys.

Example 38.4 (A 128-bit key worth 17 bits)**.** Suppose `key = SHA256(str(int(time.time())))[:16]` and the attacker knows the key was made sometime yesterday. There are 86,400 seconds in a day. She computes H of each of the 86,400 candidate timestamps, derives each candidate key, and trial-decrypts. On a laptop this is well under a second. The key "has 128 bits" in the sense that it is 16 bytes long, and roughly 17 bits in the only sense that matters – how many guesses break it.

Attack

Debian OpenSSL, 2006–2008. A well-meaning patch removed a line that mixed uninitialized memory into the OpenSSL entropy pool, on the theory that reading uninitialized memory is undefined behavior (it is). The side effect: the only remaining entropy fed to the generator was the process ID, about 15 bits. Every key generated on a patched Debian or Ubuntu system for nearly two years – SSH host keys, TLS certificates, OpenVPN keys – came from one of roughly 2^{15} possible streams. Attackers precomputed the entire set of possible keys. A 2048-bit RSA key with 15 bits of seed entropy is a 15-bit key. The math was untouched; the entropy was gone.

38.5.2 Duplicated randomness across devices

Embedded devices – routers, IoT cameras, set-top boxes – often generate their TLS or SSH keys on first boot, before any entropy has accumulated: no disk, no user input, no network traffic yet, just a freshly reset clock. Thousands of identical units run the same firmware and reach the key-generation step in nearly the same low-entropy state, so they generate *the same keys*, or keys sharing a prime factor.

Why it works

Shared RSA factors are an especially cruel form of this. If two devices' RSA moduli $N_1 = p \cdot q_1$ and $N_2 = p \cdot q_2$ happen to share one prime p (because the weak generator produced p on both), then anyone who collects both public keys computes $\gcd(N_1, N_2) = p$ in microseconds and factors *both* moduli. A 2012 internet-wide scan did exactly this: it collected millions of public RSA keys, ran pairwise GCDs, and recovered the private keys of a small but real fraction of TLS and SSH hosts – not by factoring anything hard, but because their "random" primes were not random across the population. The hardness of factoring a single modulus is irrelevant when two moduli share a factor by accident.

Connection

The birthday bound from the companion probability book governs when duplicated randomness becomes likely. Draw n values from a space of size M and a collision appears around $n \approx \sqrt{M}$. For a 24-bit WEP IV ($M = 2^{24}$) that is about $2^{12} = 4096$ packets – minutes of traffic. For 96-bit random GCM nonces ($M = 2^{96}$) it is about 2^{48} messages. The same $\sqrt{M}$ law tells you both why WEP collapses instantly and why random GCM nonces are merely risky rather than instantly fatal. Sizing a nonce or a key is always a birthday-bound calculation.

38.5.3 The forked-counter trap

A deterministic counter nonce is the safe way to feed GCM – until the process that owns the counter is duplicated. Fork a process, snapshot a virtual machine, restore from backup, or load-balance the same key across two servers, and you now have two independent copies of the counter, both starting from the same value, both about to emit nonce 1, 2, 3 again. Identical nonces, same key, different messages: the GCM catastrophe of section 38.2.3, triggered by an operations decision that had nothing to do with cryptography.

Focus

The defensive rules: (1) seed CSPRNGs from the operating system's entropy source (`getrandom`, `/dev/urandom`, `CryptGenRandom`), never from time or PID; (2) on embedded devices, block key generation until the entropy pool is initialized, or provision keys at the factory from a good source; (3) never let a nonce counter be copied – bind it to a unique per-instance identifier, or switch to a misuse-resistant mode if duplication is possible; (4) reseed after a fork (modern libraries detect forks for you). The unifying instinct: ask "what is the smallest set this secret could actually be drawn from?" and make sure that set is astronomically large.

38.6 The Signature Nonce: One Number Leaks the Private Key

The nonce-reuse pitfall has a second, sharper edge in (EC)DSA signatures, and it deserves its own section because the consequence is total: a single repeated nonce – or even a slightly biased one – recovers the long-term *private signing key*, extending far beyond the compromised message. DSA and ECDSA sign a message hash h with private key d by choosing a per-signature secret nonce k and computing

$$r = (k \cdot G)_x \bmod n, \qquad s = k^{-1}(h + d \cdot r) \bmod n,$$

where G is the curve's base point and n its order. The nonce k must be uniformly random and *never reused*. The verifier never sees k; it is as secret as d itself, and for the same reason.

Why it works

Why is a reused k instantly fatal? Suppose two messages with hashes h_1, h_2 are signed under the same key d and the *same* nonce k. Then both signatures share the same r (because r depends only on k), and

$$s_1 = k^{-1}(h_1 + dr), \qquad s_2 = k^{-1}(h_2 + dr) \pmod n.$$

Subtract: $s_1 - s_2 = k^{-1}(h_1 - h_2)$, an equation with one unknown k. Solve it: $k = (h_1 - h_2)(s_1 - s_2)^{-1} \bmod n$. Now plug k back into either signature equation and solve for the private key: $d = (s_1 k - h_1)\, r^{-1} \bmod n$. Two signatures, a handful of modular operations, and the attacker holds d – she can now forge a signature on *anything*. The math of ECDSA was never touched; the freshness of one number was.

Example 38.5 (Recovering a private key from two signatures)**.** Work over the toy order $n = 101$ (a prime, so every nonzero element is invertible mod 101). The signer's secret is $d = 7$ and – through a bug – it reuses $k = 5$ for two messages with hashes $h_1 = 13$ and $h_2 = 27$. The shared r is some fixed value from k; take $r = 9$. Then

$$s_1 = 5^{-1}(13 + 7 \cdot 9) \bmod 101, \qquad s_2 = 5^{-1}(27 + 7 \cdot 9) \bmod 101.$$

Since $5^{-1} \equiv 81 \pmod{101}$ (because $5 \cdot 81 = 405 = 4 \cdot 101 + 1$), we get $s_1 = 81 \cdot 76 \bmod 101$. Now $81 \cdot 76 = 6156 = 60 \cdot 101 + 96$, so $s_1 = 96$. And $s_2 = 81 \cdot 90 \bmod 101$; $81 \cdot 90 = 7290 = 72 \cdot 101 + 18$, so $s_2 = 18$. The attacker sees $(r, s_1) = (9, 96)$ and $(r, s_2) = (9, 18)$ – the repeated r is the tell. She recovers k:

$$k = (h_1 - h_2)(s_1 - s_2)^{-1} = (13 - 27)(96 - 18)^{-1} = (-14) \cdot 78^{-1} \bmod 101.$$

Compute $78^{-1} \equiv 79 \pmod{101}$ (check $78 \cdot 79 = 6162 = 61 \cdot 101 + 1$). Then $k = (-14) \cdot 79 = -1106 \equiv -1106 + 11 \cdot 101 = 5 \pmod{101}$, matching the reused nonce. Now recover the private key from $d = (s_1 k - h_1)\, r^{-1}$:

$$d = (96 \cdot 5 - 13) \cdot 9^{-1} = 467 \cdot 9^{-1} \equiv 63 \cdot 45 \pmod{101},$$

using $467 \equiv 63$ and $9^{-1} \equiv 45 \pmod{101}$ (since $9 \cdot 45 = 405 \equiv 1$). Finally $63 \cdot 45 = 2835 = 28 \cdot 101 + 7$, so $d = 7$ – the signer's secret, pulled out by hand from two signatures. The procedure is entirely mechanical: a repeated r in two ECDSA signatures means the private key is a few modular inversions away.

Attack

PS3, Android Bitcoin wallets, and beyond. Sony signed PlayStation 3 firmware with ECDSA but used a *constant* nonce k for every signature. Two signed binaries were enough for the `fail0verflow` group to recover Sony's master private signing key in 2010, after which anyone could sign code the console would accept – a fleet-wide, unpatchable break. The same bug drained real money: in 2013 a flaw in Android's `SecureRandom` made some Bitcoin wallets reuse ECDSA nonces across transactions; attackers scanned the public blockchain for two signatures sharing an r, recovered the private keys, and swept the coins. The blockchain made every signature public, so the attack was a database query plus the arithmetic above. The defense is *deterministic* nonces (RFC 6979), which derive k as a PRF of the message and the private key – unique per message by construction, never dependent on a possibly-broken RNG.

Common trap

It is worse than "do not repeat k." Even a *biased* nonce – a few predictable top bits, a k drawn from a slightly-too-small range – leaks the key, given enough signatures, via lattice (hidden-number-problem) attacks. So "I made k random but only 250 bits instead of 256" is still a break. This is why RFC 6979 deterministic nonces (or modern designs like Ed25519, which derives the nonce by hashing the message with a secret prefix) are the standard: they remove the RNG from the signing path entirely, so there is no bias and no reuse to get wrong.

38.7 Hardcoded and Reused Keys

A key embedded in source code, firmware, or a compiled binary cannot remain secret. Relying on obfuscation or the difficulty of reverse engineering merely delays discovery rather than providing security. Kerckhoffs's principle, restated for the modern device: assume the adversary holds your firmware, because she does.

Attack

One hardcoded key across an entire fleet. A line of home routers ships with a single firmware image containing a hardcoded AES key used to encrypt the remote-management channel. The key is the same on every unit. An attacker buys one router for $30, dumps the flash, extracts the key with `strings` and a little patience, and now decrypts and forges management traffic for *every* router of that model on the internet. The pattern repeats across IoT cameras (hardcoded credentials), point-of-sale terminals (a global master key), and game consoles (a signing key extracted from one unit breaks the platform). One extraction, fleet-wide compromise.

Common trap

Closely related: **default keys and credentials** that nobody changes, and **secrets committed to version control**. An API key, a private key, or a database password pushed to a public Git repository is harvested by automated scanners within minutes of the commit landing – and rewriting history does not help, because the bad commit was already cloned.

Treat any secret that has touched a repository, a log file, a chat message, or a screenshot as burned: rotate it, do not hide it.

Implement it

The fix is to derive or provision keys, never to embed them. Per-device keys from a hardware secure element, keys fetched from a key-management service at runtime, or keys derived from a user secret via a slow KDF – anything but a constant in the binary. The shape of provisioning:

```python
# LEARNING ONLY -- illustrative, not production key management.
import os, hashlib

# BAD: same key in every shipped unit.
SECRET_KEY = b"hardcoded_master_key_1234567890!"   # never do this

# BETTER: a unique per-device key the manufacturer does not retain,
# stored in a secure element, or derived at provisioning time.
def provision_device_key(device_serial: bytes) -> bytes:
    # A real system uses an HSM-held root + a proper KDF (HKDF).
    root = os.environ["PROVISIONING_ROOT_KEY"].encode()  # never in the image
    return hashlib.sha256(root + device_serial).digest()
```

The principle is the lesson, not the code: the value that protects unit N must not protect unit M, and no secret that breaks the fleet should live where the fleet can be disassembled.

38.8 Reusing One Key for Two Purposes

Keys are typed, even when the API does not say so. A key for encryption is not a key for authentication; a signing key is not a decryption key; a key for "messages from A to B" is not a key for "messages from B to A." Using one key in two roles can let the two roles interfere, and the interference is often exploitable.

Why it works

The classic abstract example: the same RSA key pair used both to *sign* and to *decrypt*. RSA signing computes $s = m^d \bmod N$; RSA decryption computes $m = c^d \bmod N$ – the *same* private operation $x \mapsto x^d$. So an attacker who wants the decryption of a ciphertext c can ask the victim to *sign* the value c (perhaps disguised as an "innocuous" document), and the resulting "signature" $c^d \bmod N$ *is* the decryption she wanted. The signing oracle and the decryption oracle are the same oracle when the key is shared. The fix is domain separation: separate key pairs for separate functions, and padding schemes (PSS for signing, OAEP for encryption) that make the two operations incompatible even if a key leaks between them.

The same disease appears in symmetric systems. If you derive one key and use it both as the AES key and the HMAC key in a hand-rolled encrypt-then-MAC, structural relationships between the two algorithms can leak. If both directions of a connection share one key and one nonce space, a message Alice sent can be *reflected* back to her as if Bob had sent it – a reflection attack – because nothing in the key distinguishes the two endpoints.

Attack

Reflection across a shared key. Two parties authenticate each other with a challenge-response over a single shared symmetric key: "I send you a random r, you return $\mathrm{MAC}_K(r)$ to prove you know K." If both sides use the same K and the same construction, Eve can open two connections to the server, take the server's challenge r from connection 1, send it as *her* challenge on connection 2, get the server to compute $\mathrm{MAC}_K(r)$ for her, and replay it back on connection 1 – authenticating as a legitimate party without knowing K. This vulnerability arises because a single unseparated key serves both directions, allowing the adversary to use the server as an authentication oracle against itself.

Focus

The discipline is **key separation via a KDF**. From one master secret, derive distinct sub-keys with a labeled key-derivation function: $k_{\text{enc}} = \mathrm{HKDF}(\text{master}, \texttt{"encryption"})$, $k_{\text{mac}} = \mathrm{HKDF}(\text{master}, \texttt{"authentication"})$, and separate keys per direction, per purpose, per protocol version. The labels guarantee the sub-keys are independent: knowledge of one tells you nothing about another. TLS 1.3 takes this to its conclusion, deriving a whole tree of purpose-specific keys from the handshake secret, each with its own label. One secret in, many independent keys out, no two roles ever sharing material.

38.9 Broken Primitives: MD5, SHA-1, DES, RC4

Primitives have lifespans. A hash or cipher that was respectable in 1995 can be a liability in 2025 because attacks improve and hardware gets faster. Continuing to use a primitive after it is broken is a pitfall of inertia, and the broken ones still appear in real systems through copied code, legacy compatibility, and "it still passes the tests."

History

A rough timeline of the fallen. *MD5* (1992): collisions found in 2004; by 2008 researchers forged a rogue certificate-authority certificate using an MD5 collision, and the Flame malware (2012) used an MD5 collision to forge a Microsoft code-signing certificate. *SHA-1* (1995): theoretically weakened by 2005, with the first concrete collision (SHAttered) demonstrated by Google and CWI in 2017, then a *chosen-prefix* collision in 2020 that broke the PGP web of trust's use of it. *DES* (1977): its 56-bit key was brute-forced in public by 1998 (the EFF "Deep Crack" machine, under a day); 3DES patched the key length but is now retired for its 64-bit block. *RC4* (1987, leaked 1994): biases in its keystream made it broken in WEP, then in TLS (the 2013 "RC4 NOMORE" and related attacks), and it is now prohibited.

38.9.1 What "broken" means, precisely

"Broken" is not binary, and the distinctions matter for risk. For a hash, the three properties degrade independently:

- **Collision resistance** (hard to find any $x \neq y$ with $\mathrm{H}(x) = \mathrm{H}(y)$) is the weakest to defend and

the first to fall. MD5 and SHA-1 are collision-broken. This breaks *signatures* and *certificates*, where an attacker who controls both colliding inputs can get one signed and present the other.

- **Second-preimage resistance** (given x, hard to find $y \neq x$ with the same hash) is stronger; MD5/SHA-1 are not practically second-preimage-broken, so an attacker cannot yet match a hash of a *given* file she did not choose.
- **Preimage resistance** (given h, hard to find any x with $\mathrm{H}(x) = h$) is the strongest. Even MD5 is not practically preimage-broken.

Common trap

This gradation explains a subtle real-world rule. Using MD5 to verify a download against a hash *you computed yourself from the genuine file* relies on second-preimage resistance, which MD5 still has – so it is weak but not instantly exploitable. Using MD5 in a *signature or certificate*, where an attacker can craft two colliding inputs, relies on collision resistance, which MD5 lacks – so it is catastrophically broken. Same algorithm, opposite verdicts, because the threat model invokes different properties. "Is MD5 broken?" has no answer without "for what?" Still: do not use MD5 or SHA-1 for anything new. The collision break already destroys their most common security use, and migration is cheap.

Example 38.6 (Why a hash collision forges a certificate)**.** A certificate authority signs the hash of a certificate's contents: $\mathrm{Sign}_{\mathrm{CA}}(\mathrm{H}(\mathrm{cert}))$. Suppose an attacker uses an MD5 collision to build two certificates C_{benign} and C_{evil} with $\mathrm{H}(C_{\mathrm{benign}}) = \mathrm{H}(C_{\mathrm{evil}})$ – the benign one a plausible request for `shop.example.com`, the evil one a certificate granting her a sub-CA that can sign *any* domain. She submits C_{benign}; the CA validates it and signs its hash. Because the hashes are equal, that same signature is also a valid signature on C_{evil}. She lifts the signature onto the evil certificate and now holds a CA-signed credential for the entire internet. This is essentially the 2008 rogue-CA demonstration. Collision resistance was the only thing standing between a routine signature and a master key, and MD5 did not have it.

Focus

The current safe defaults: hashing – SHA-256, SHA-3, or BLAKE2/3; symmetric encryption – AES (128 or 256) or ChaCha20, always inside an AEAD; key sizes – 128-bit symmetric minimum, RSA $\geq$ 2048 (3072 for long-term), elliptic curves over $\geq$ 256-bit fields. The meta-rule: track the deprecation announcements (NIST, the TLS working group, your library's changelog) and treat "this primitive is being phased out" as a maintenance ticket, not a someday. Crypto agility – the ability to swap a primitive without re-architecting – is itself a design goal.

38.10 Insecure Comparison: Leaking Secrets Through Time

Verifying a MAC, a password hash, or an API token means comparing a computed value against a provided one. The obvious comparison – walk both byte arrays, return `false` at the first mismatch – is a security bug, because the time it takes leaks how many leading bytes matched. An attacker who can measure response time (even noisily, by averaging many requests) recovers the secret one byte at a time.

Intuition

Picture guessing a combination lock that clicks louder the moment you hit a correct digit in sequence. You do not need to try all 10^4 combinations; you try the first digit's ten values listening for the click, fix it, move to the second, and so on – $10 \times 4 = 40$ tries instead of 10^4. Early-exit comparison is that louder click. It converts a brute-force search over the whole secret (exponential) into a digit-at-a-time search (linear). The leak is not in *what* the function returns – always just true or false – but in *how long* it takes to say it.

Attack

Byte-at-a-time tag recovery. A server checks an HMAC tag on incoming requests with a standard `==` (early-exit) comparison. The attacker wants a valid tag for a forged message. She fixes byte 0 of her guessed tag to `0x00`, sends the request, times the response; tries `0x01`, times it; ... through `0xFF`. For exactly one value, the server compares one byte further before failing, so the response is measurably slower. That value is the correct byte 0. She locks it in and repeats for byte 1, then byte 2. A 16-byte tag falls in about $16 \times 256 = 4096$ measured requests (times averaging for noise) instead of 2^{128}. The MAC is cryptographically perfect; the *comparison* leaked it. This was a real vulnerability in several web frameworks' signed-cookie verification.

Why it works

The fix is **constant-time comparison**: a routine whose running time depends only on the *length* of the inputs, never on their contents. The standard trick is to OR together the XOR of every byte pair and check the accumulator once at the end – every byte is always examined, so there is no early exit to time.

$$\text{diff} = \bigvee_{i=0}^{n-1} (a_i \oplus b_i), \qquad \text{equal} \iff \text{diff} = 0.$$

Each $a_i \oplus b_i$ is zero exactly when the bytes match; OR-ing accumulates "any mismatch anywhere" into a single value examined only after the full pass. The loop runs the same number of steps regardless of where (or whether) a mismatch occurs.

Implement it

```
# LEARNING ONLY. Use hmac.compare_digest or your library's
# constant-time equal in real code -- a Python loop is not
# truly constant-time at the hardware level.

def insecure_equal(a, b):          # DO NOT USE -- early exit leaks timing
    if len(a) != len(b):
        return False
    for x, y in zip(a, b):
        if x != y:
            return False           # returns sooner the earlier the mismatch
    return True

```

```python
def constant_time_equal(a, b):      # examine every byte, decide once
    if len(a) != len(b):
        return False                # length leak is usually acceptable
    diff = 0
    for x, y in zip(a, b):
        diff |= x ^ y               # accumulate; never branch on content
    return diff == 0

# In real Python:  import hmac;  hmac.compare_digest(tag, expected)
```

The warning is double here: the insecure version leaks immediately, and even the "constant-time" loop in interpreted Python cannot guarantee hardware-level constant time – branch prediction, caching, and the interpreter leak. Use the platform primitive (`hmac.compare_digest`, `CRYPTO_memcmp`, `subtle.ConstantTimeCompare`). The toy code shows the *idea*; the library gives you the guarantee.

Connection

Timing leaks connect directly to the broader subject of side channels—power analysis, cache-timing on table lookups, branch prediction—covered in Chapter 37. The unifying observation: a cryptographic function is more than its input-output map. Its *physical execution* – time, power, electromagnetic emission, cache state – is also an output, and one the security proof never accounted for. Constant-time programming is the discipline of making sure those extra outputs carry no secret-dependent information.

38.11 Rolling Your Own Crypto

Building cryptographic primitives or protocols independently rather than adopting vetted standards introduces severe vulnerabilities. A primitive gains trust only after years of sustained evaluation by the broader cryptographic community. A newly invented scheme has typically faced scrutiny only from its author, whose perspective is constrained by the design assumptions they intended to enforce.

Intuition

"I can't break it" and "it can't be broken" are wildly different claims. The gap between them is the entire history of cryptanalysis. Every broken cipher in this chapter looked unbreakable to its designer. Schneier's law: anyone can invent a cipher they personally cannot break. Security is a *negative* property – the absence of any working attack – and you cannot establish a negative by failing to find a positive. Only sustained adversarial scrutiny by people who *want* you to fail can build confidence, and only the standard primitives have received it.

The danger is sharpest because home-grown crypto often *looks* fine. It encrypts; the ciphertext looks random; it decrypts back correctly; the tests pass. None of that is evidence of security, because, as we established at the top of the chapter, cryptographic correctness is not testable by example. The XOR-with-a-repeating-key "cipher," ECB "encryption," a hand-rolled MAC that is really just a hash of key-then-message (vulnerable to length-extension), a custom key-exchange missing authentication – each passes every functional test and falls to a standard attack.

Attack

Length extension on a home-rolled MAC. A tempting "MAC" is $t = \mathrm{H}(K \,\|\, m)$ with a Merkle–Damgård hash like SHA-256. It looks like it authenticates m with K. But Merkle–Damgård hashes process message blocks sequentially and expose their internal state as the output. Given t and the length of $K \,\|\, m$, an attacker can set the hash's internal state to t and *continue* hashing with an appended suffix s, producing a valid tag $t' = \mathrm{H}(K \,\|\, m \,\|\, \mathrm{pad} \,\|\, s)$ for the extended message – without knowing K. The fix is the standard construction HMAC, $\mathrm{H}((K \oplus \mathrm{opad}) \,\|\, \mathrm{H}((K \oplus \mathrm{ipad}) \,\|\, m))$, whose nesting defeats length extension. Someone rolling their own MAC reaches for the obvious $\mathrm{H}(K \,\|\, m)$ and ships a forgeable design; using HMAC means never meeting this attack.

In the real world

The graveyard of home-grown crypto is large and well-documented. WEP's RC4-plus-CRC integrity (forgeable; see Chapter 26). Telegram's bespoke MTProto protocol drew repeated criticism for deviating from standard AEAD and authentication patterns. Countless "encryption" features in commercial products turned out to be XOR with a fixed key, or AES in ECB, or a Caesar cipher with extra steps. The pattern is not that the designers were unintelligent – many were skilled engineers – but that they treated a cryptographic protocol as ordinary software, where "it works in testing" means "it works." In crypto it does not.

Focus

The constructive version of "don't roll your own" is not "don't use cryptography." It is **use the right level of abstraction**: a vetted high-level library (libsodium / NaCl, the platform's authenticated-encryption API, a maintained TLS stack) that exposes "seal this message to this recipient" rather than "here is a block cipher, good luck." These libraries make the secure choice the default and the insecure choice hard to express – no ECB, no nonce reuse, no key reuse, no insecure comparison, because the API does not let you. The next chapter, on implementing crypto safely, is about choosing and using those tools. Rolling your own is the pitfall; reaching for the well-worn library is the cure for nearly every pitfall in this chapter at once.

In the real world

How the survivors avoid these traps. TLS 1.3 is, in large part, a list of these pitfalls with the lessons applied: it removed CBC and the MAC-then-Encrypt modes (padding oracles gone), removed RC4 and export ciphers (broken primitives gone), mandates AEAD with carefully managed nonces (no two-time pads), derives a tree of purpose-separated keys via HKDF (no key reuse), and uses constant-time implementations in its reference libraries (no timing leaks). The Signal protocol layers a fresh key per message via its ratchet (forward secrecy and no key reuse), authenticates everything, and ships as a single audited library. WireGuard deliberately offers *no* algorithm choices – one AEAD, one hash, one curve – so there is no broken option to misconfigure. Each is a case study in turning "common pitfall" into "impossible by construction."

38.12 A Field Checklist

Carry this list to any cryptographic code review. Each item is a pitfall from this chapter phrased as a question that has a right answer.

- **Nonces/IVs**: is every nonce unique under its key? For CBC, is the IV unpredictable (random)? Can a counter be duplicated by a fork, snapshot, or restore? Could random 96-bit nonces collide at this message volume?
- **Mode**: is ECB used anywhere on structured or multi-block data? Is encryption randomized (IND-CPA)? Is an AEAD used rather than encryption alone?
- **Integrity ordering**: is it Encrypt-then-MAC (or a single AEAD)? Does the MAC cover the IV and associated header? Is the tag checked before any decryption or unpadding?
- **Randomness**: does every key, nonce, and salt come from the OS CSPRNG? Is anything seeded from time, PID, or a non-crypto PRNG? Do embedded devices block until entropy is ready?
- **Key management**: are any keys hardcoded, defaulted, or committed to a repo? Is each key used for exactly one purpose, derived via a labeled KDF? Separate keys per direction?
- **Primitives**: any MD5, SHA-1, DES/3DES, or RC4? Are key sizes at current minimums? Is there a migration path (crypto agility)?
- **Comparisons**: are all secret comparisons (tags, tokens, hashes) constant-time via the platform primitive?
- **Provenance**: is every primitive a standard, vetted construction from a maintained library, or did someone roll their own?

Pause

Notice the through-line. Every question above is a precondition from some primitive's contract, made checkable. You do not need to be a cryptanalyst to catch most real-world breaks – you need to read each primitive's contract and verify, clause by clause, that the code keeps its end of the bargain. The mathematicians secured the primitives. Keeping the preconditions true is your job, and it is where systems actually live or die.

Exercises

Exercise 38.1 (Crack a two-time pad)**.** Two ASCII plaintexts of equal length are encrypted with the same stream-cipher keystream. You intercept

$$c_1 = \texttt{0x37 0x1D 0x0A 0x1B 0x09}, \qquad c_2 = \texttt{0x2B 0x1D 0x0B 0x07 0x09}.$$

(a) Compute $c_1 \oplus c_2$. (b) Which byte positions are identical between m_1 and m_2, and how do you know without the key? (c) If you learn $m_1 =$ `"HELLO"`, recover m_2 and the keystream. (Hint: `H`=`0x48`.)

Exercise 38.2 (GCM nonce reuse)**.** Explain in your own words why a single nonce repetition in AES-GCM lets an attacker forge tags on *new* messages, whereas a single nonce repetition in a plain

CTR-mode stream cipher (no MAC) only leaks $m_1 \oplus m_2$. Your answer should identify the value H, say why two same-nonce tags expose it, and state why a fresh nonce per message keeps H hidden.

Exercise 38.3 (Read the ECB pattern)**.** A toy 4-bit block cipher in ECB mode produces the ciphertext `2929 7EE2` (one hex digit per block). Without the key, state everything you can deduce about the plaintext's structure. Then explain why CBC mode with a random IV would leak none of it, and write the CBC encryption of a 2-block message $m_1 m_2$ symbolically.

Exercise 38.4 (Padding oracle reasoning)**.** A CBC-mode service decrypts a ciphertext, strips PKCS#7 padding, and returns "bad padding" or "bad MAC" as distinct errors, checking padding *before* the MAC. (a) Explain how an attacker uses the two distinct responses to recover one plaintext byte. (b) Roughly how many oracle queries to recover a 16-byte block? (c) Which combination order (EtM/MtE/E&M) does this service use, and which order would close the hole, and why?

Exercise 38.5 (Shared RSA factors)**.** A weak generator produces two RSA moduli that happen to share a prime: $N_1 = 1{,}593{,}211 = p \cdot q_1$ and $N_2 = 1{,}597{,}247 = p \cdot q_2$, where $p = 1009$ (so $q_1 = 1579$ and $q_2 = 1583$). (a) Run the Euclidean algorithm on (N_1, N_2) to recover p without knowing it in advance. (b) Divide to get q_1 and q_2, factoring both moduli. (c) Explain why this attack costs essentially nothing across millions of collected public keys, and exactly what property of the random generator caused it.

Exercise 38.6 (Entropy of a time-seeded key)**.** A service generates a session key as `SHA256(str(unix_seconds))` truncated to 128 bits. (a) An attacker knows the key was created within a known 10-minute window. How many candidate keys must she try? (b) Express that as bits of effective entropy, $\log_2$ of the count. (c) The key is 128 bits long; why is its security nowhere near 2^{128}? (d) Give a one-line fix.

Exercise 38.7 (Constant-time comparison)**.** (a) Trace the running time of an early-exit byte comparison for the inputs (`AB CD`, `AB CE`) versus (`AB CD`, `FF FF`): which returns sooner, and what does that leak? (b) Write out the value of the accumulator diff $= \bigvee_i (a_i \oplus b_i)$ for the comparison of `12 34` against `12 35`. (c) Explain why a byte-at-a-time timing attack needs about $n \cdot 256$ queries for an n-byte secret, not 256^n.

Exercise 38.8 (Length extension)**.** A system authenticates messages with $t = \mathrm{H}(K \,\|\, m)$ using a Merkle–Damgård hash. (a) Explain why an attacker who knows t, m, and $|K|$ can compute a valid tag for $m \,\|\, \mathrm{pad} \,\|\, s$ for any suffix s, without knowing K. (b) What internal property of the hash makes this possible? (c) State the HMAC construction and explain in one sentence why its nesting blocks the attack.

Exercise 38.9 (Key reuse across purposes)**.** A protocol uses one RSA key pair for both signing documents and decrypting messages, with no padding (textbook RSA), so both operations are $x \mapsto x^d \bmod N$. (a) Show how an attacker who can submit documents for signing can use the signing service to decrypt an intercepted ciphertext c. (b) Why does adding distinct, incompatible padding (PSS for signatures, OAEP for encryption) break the attack even if the same modulus is shared? (c) State the general principle this illustrates.

Exercise 38.10 (Audit a snippet)**.** You find this pseudocode in a code review:

```
key = MD5(password)
iv = 0   (a constant)
ct = AES_CBC(key, iv, plaintext)
tag = MD5(key + ct)
send(ct, tag);   verify with tag == MD5(key + ct) using ==.
```

List every pitfall from this chapter that appears (there are at least six), and for each give a one-line fix.

Exercise 38.11 (Recover an ECDSA key from a reused nonce)**.** A signer uses ECDSA over the toy order $n = 101$ and reuses one nonce k for two messages, producing $(r, s_1) = (9, 96)$ and $(r, s_2) = (9, 18)$ for hashes $h_1 = 13$ and $h_2 = 27$. (a) From the two signatures alone, what one feature tells you the nonce was reused? (b) Recover k using $k = (h_1 - h_2)(s_1 - s_2)^{-1} \bmod n$. (c) Recover the private key $d = (s_1 k - h_1)\, r^{-1} \bmod n$. (d) State why a single bias in k (not even a full repeat) can still leak d, and name the deterministic-nonce fix.

Exercise 38.12 (Walk a padding oracle one byte)**.** A CBC service with one-byte blocks decrypts $g\, c_1$ to $D_K(c_1) \oplus g$ and reports "valid padding" exactly when that byte equals `0x01`. The unknown intermediate is $z = D_K(c_1)$, and the genuine $c_0 =$ `0x28`. (a) As the attacker sweeps g from `0x00` to `0xFF`, which equation must hold for the oracle to say "valid"? (b) If the oracle fires at $g =$ `0x6C`, what is z? (c) Recover the genuine plaintext byte $m_1 = z \oplus c_0$ and give its ASCII character. (d) For a real 16-byte block, why are at most 16×256 queries needed, not 256^{16}?

Exercise 38.13 (Prove determinism is insecure)**.** Prove that any *deterministic* encryption scheme fails the IND-CPA game (the adversary wins with probability 1). Then explain how this single theorem condemns ECB, textbook RSA, and any cipher used with a fixed IV, in one stroke.

Chapter 39

Implementing Cryptography Safely

Every chapter so far taught you how a cryptographic machine works on the inside. You can now trace AES through its rounds, run RSA with small primes by hand, see why GCM dies on a repeated nonce, and follow a TLS 1.3 handshake message by message. That knowledge is real and it matters. But there is a gap between understanding a primitive and shipping a system that uses it, and that gap is where almost all real-world cryptographic failures live. The math is rarely what breaks. What breaks is the wiring: a nonce that repeats because a counter reset, a key that lived in a log file, a comparison that returned early and leaked a tag byte at a time, a downgrade that nobody tested, a "temporary" ECB mode that shipped to production.

The conventional advice "do not roll your own crypto" highlights a genuine risk, but provides little constructive guidance. The operational rule is more specific: *do not implement primitives, do compose audited high-level constructions, and treat every place where you must make a choice as a place where you can make a mistake.* The craft of safe implementation is the discipline of removing your own choices — nonces, modes, padding, key sizes, comparison routines — and handing them to code that has already been attacked by people smarter and meaner than your future self at 3 a.m.

We will work through the library hierarchy (why libsodium and Tink exist and what "misuse-resistant" actually buys you), key management and rotation with a worked key hierarchy, secure defaults, testing against published test vectors, a real threat-modeling pass on a small system, the genuinely rare cases where you do touch a primitive, and a one-page checklist you can paste into a code review. There is less new mathematics here than anywhere else in the book and more hard-won engineering judgment. Both keep secrets. Only one of them is usually the reason your secret leaked.

39.1 Defining the Core Rule

Let us start with the slogan and dismantle it carefully, because half of using it well is knowing exactly what it forbids.

Focus

"Don't roll your own crypto" decomposed. The slogan bundles three very different activities that deserve different verdicts.

1. **Designing a new primitive** — a new cipher, hash, or signature scheme. For almost everyone, almost always: *never*. New primitives need years of public cryptanalysis before anyone should trust them. This is a research activity, not an engineering one.
2. **Implementing a standard primitive** — your own AES, your own Curve25519. *Almost never.* The algorithm is public and correct on paper, but a correct-looking implementation can still leak the key through timing, cache behavior, or a faulty bignum routine. We will see when — rarely — this is justified.
3. **Composing primitives into a protocol** — gluing key exchange, encryption, and authentication into a secure channel. This is the dangerous one precisely because it looks like ordinary programming. Most "roll your own" disasters live here: people who would never write their own AES happily invent their own handshake, their own MAC-then-encrypt order, their own session resumption. *Use a vetted construction instead.*

The corrected rule: **compose audited high-level constructions; do not design or implement primitives; and minimize the choices you are allowed to make at every layer.**

Why is this so much harder than normal software? In ordinary engineering, a bug announces itself. The button does not click, the total comes out wrong, the test goes red. Cryptographic bugs are silent. A system that uses a fixed nonce, or that never actually verifies a signature, or that compares MACs with a short-circuiting string equality, *works perfectly* in every functional test. Encrypt, decrypt, round-trip succeeds, ship it. The defect is invisible until an adversary who is looking for exactly that defect finds it, and by then the failure is a breach, not a failed unit test. The feedback loop that keeps normal code roughly correct is missing here. That single fact — correctness is not testable by the people writing the code — is the primary reason to lean on artifacts that hostile experts have already tested.

Intuition

Safe cryptographic engineering requires using certified primitives and standard protocol compositions. Much like structural engineering codes govern how standardized materials must be assembled to withstand loads, cryptographic standards provide vetted construction patterns that prevent systemic failures under adversarial conditions.

History

The line "anyone can invent a cipher they cannot break themselves" is folk wisdom usually traced to Bruce Schneier's *Memo to the Amateur Cipher Designer* (1998), and the underlying point is older — it is Auguste Kerckhoffs's principle from 1883 turned into career advice. The history of broken homebrew crypto is long and consistent: 1999's 802.11 WEP paired RC4 with a non-cryptographic CRC-32 checksum and 24-bit nonces that inevitably collided; Adobe's 2013 password "encryption" used ECB mode with unsalted password hints sitting right beside it; the MS-CHAPv2 handshake and countless custom "obfuscation" layers in mobile apps fall in an afternoon. In response, Daniel J. Bernstein, Tanja Lange, and Peter Schwabe released NaCl in 2008 to pioneer "misuse-resistant" APIs that remove configuration knobs, and Google followed with Tink in 2017. The pattern in field failures is almost never "they used a weak cipher." It is almost always "they used a fine cipher and wired it up wrong."

39.2 The Library Hierarchy: Where to Stand

Cryptographic APIs come in layers, and the single most consequential decision you make is *which layer you build on*. Higher is safer, because higher means fewer decisions left to you. Picture a ladder (Figure 39.1); you want to stand as near the top as your problem allows.

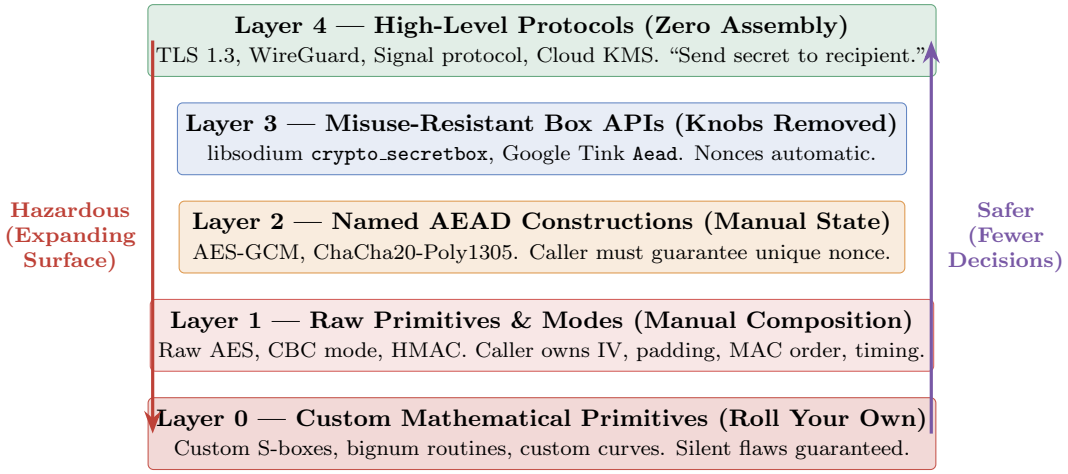


Figure 39.1: The cryptographic API ladder. As you ascend from Layer 0 to Layer 4, the library assumes responsibility for error-prone mechanics (constant-time branches, IV generation, padding, and MAC ordering), narrowing your attack surface to near zero. Every parameter eliminated from an API signature is an exploit vector eliminated from production.

The mistakes scale with the layer. At Layer 4 there is almost nothing left to get wrong; you call code whose job is to be secure end to end. At Layer 3 you choose a key and pass plaintext; the API picks the nonce, the mode, the tag, and refuses to let you reuse a nonce. At Layer 2 you have re-acquired one sharp edge — the nonce — and GCM punishes a repeat catastrophically (recall Chapter 16). At Layer 1 every sharp edge is back: you choose the mode, generate the IV, add the padding, decide MAC-then-encrypt versus encrypt-then-MAC, and write the tag comparison. Layer 0 adds the algorithm itself. **Each step down the ladder multiplies the number of independent ways to be silently wrong.**

Intuition

Read the ladder this way: every API parameter you are *required* to fill in is a place the library has delegated a security decision to you. Count the parameters. `secretbox(message, key)` delegates almost nothing. `aes_gcm_encrypt(message, key, nonce, aad)` delegates the nonce — exactly the parameter with the famous catastrophic failure mode. "How many ways can I hold this wrong?" is, to a first approximation, "how many arguments does it take?"

39.2.1 NaCl and libsodium: the box that hides the nonce

NaCl ("salt," the Networking and Cryptography Library) was Daniel J. Bernstein, Tanja Lange, and Peter Schwabe's deliberate answer to the misuse problem. libsodium is its portable, packaged, widely audited descendant. Its philosophy is this whole chapter compressed into an API: *pick good*

primitives once, expose them through operations that are hard to misuse, and delete the knobs. There is no cipher selection, no mode menu, no negotiation. Secret-key authenticated encryption is a single function.

Implement it

Learning only — this is illustrative, but here the lesson *is* to use the real library. The point of the snippet is how little you are asked to decide, not to teach you to reimplement anything.

Listing 39.1: libsodium-style authenticated encryption via PyNaCl. Notice how few decisions you are handed.

```python
import nacl.secret, nacl.utils

# A 32-byte key from a CSPRNG. Never type a key in; never derive it from
# a password without a real KDF (see the key-management section).
key = nacl.utils.random(nacl.secret.SecretBox.KEY_SIZE)
box = nacl.secret.SecretBox(key)

# Encrypt. We do NOT pass a nonce: SecretBox draws a fresh random one and
# prepends it to the ciphertext. One catastrophic decision removed from us.
ct = box.encrypt(b"transfer 100 to account 7")

# Decrypt. The nonce rides along with the ciphertext; the tag is checked.
# If a single bit was flipped, this RAISES -- it never returns garbage.
pt = box.decrypt(ct)
assert pt == b"transfer 100 to account 7"
```

Look at what is *absent*. You did not pick a cipher (it is XSalsa20-Poly1305). You did not pick a mode. You did not choose between encrypt-then-MAC and the alternatives — the construction is fixed and correct. You did not generate a nonce, so you cannot reuse one. The decryption either returns the exact plaintext or throws; there is no code path that hands you unauthenticated bytes to "check later." Every one of those absences corresponds to a class of CVEs that simply cannot occur in code written against this API. That is what *misuse-resistant* means: not "hard to break if used correctly," but "hard to use incorrectly in the first place."

Focus

Why XSalsa20 and not raw AES-GCM here? The leading `X` means *extended nonce* — 192 bits instead of 96. With a 192-bit random nonce you can pick nonces at random forever and the chance of a collision stays negligible (a birthday bound over 2^{192}; see the connection box). That is precisely what lets `SecretBox` generate nonces randomly without bookkeeping. AES-GCM's 96-bit nonce is too short for that, which is why AEAD-at-Layer-2 forces you to manage a counter. The library chose a primitive whose parameters make the safe usage the only usage. Design choices propagate.

Connection

The random-nonce argument is a birthday-bound calculation from the companion probability book. With an n-bit nonce drawn uniformly, the expected number of draws before a collision is about $\sqrt{2^n} = 2^{n/2}$. For $n = 96$ that is 2^{48} messages — reachable by a busy service, which is why GCM nonces are managed, not random. For $n = 192$ it is 2^{96} — astronomically out of reach, which is why XSalsa20's extended nonce can be random. The whole API ergonomics question reduces to one square-root.

39.2.2 Tink: types that refuse to be confused

Google's Tink (in Java, C++, Go, Python) attacks the same problem from the type-system side. Its central ideas are worth naming because they generalize beyond any one library.

Keys are objects, not byte strings. In Tink a key is a `Keyset` that carries its own algorithm, parameters, and key-id metadata. You cannot accidentally feed an HMAC key into AES-GCM, because the type that comes out of the keyset *is* an `Aead` or a `Mac`, never raw bytes. A whole genus of bugs — "key confusion," where the same key material gets used for two purposes — is unrepresentable.

Operations are interfaces, not algorithms. You ask for an `Aead` and call `encrypt(plaintext, associatedData)`. You did not name AES-GCM; the keyset did. Swapping AES-GCM for ChaCha20-Poly1305, or rotating to a new key, is a keyset change with no call-site edits. Crypto-agility without per-call decisions.

Keysets carry rotation built in. A keyset can hold several keys at once, one marked *primary*. New data is encrypted under the primary; incoming data is decrypted under whichever key-id its header names. Rotation becomes "add a key, promote it to primary, later retire the old one" — and old ciphertext keeps decrypting the entire time. We will see why that staging matters when we work through rotation by hand.

Intuition

libsodium removes decisions by deleting knobs; Tink removes decisions by making the wrong combination a type error your compiler rejects before the code runs. Same goal, two routes: *the safe path is the path of least resistance.* If your crypto API makes you stop and think hard about nonces and modes on every call, it is, by this standard, a badly designed API — and you are one tired afternoon away from a vulnerability.

39.3 Misuse-Resistance Made Concrete

It is worth seeing, with small numbers, exactly which mistake each design choice forecloses. We will take the single most expensive misuse in modern symmetric crypto — nonce reuse in a counter-based AEAD — and watch a misuse-resistant API prevent it.

Recall from Chapter 16 that GCM (and any CTR-based scheme) generates a keystream as a function of the key and nonce alone. Encrypt two messages m_1, m_2 under the same key and *same nonce* and

you get keystream S used twice:

$$c_1 = m_1 \oplus S, \qquad c_2 = m_2 \oplus S \quad \Longrightarrow \quad c_1 \oplus c_2 = m_1 \oplus m_2.$$

The key vanished; an eavesdropper learns the XOR of two plaintexts, and for GCM specifically the authentication key is also exposed, letting the attacker forge. Let us make the confidentiality half tangible with one byte.

Example 39.1 (Nonce reuse leaks plaintext, by hand)**.** Work in single bytes. Suppose the keystream byte for a given (key, nonce) is $S =$ `0xA3`. Message one is the ASCII letter `'Y'` = `0x59`; message two is `'N'` = `0x4E` (a yes/no vote, say). Reusing the nonce:

$$c_1 = \texttt{0x59} \oplus \texttt{0xA3} = \texttt{0xFA}, \qquad c_2 = \texttt{0x4E} \oplus \texttt{0xA3} = \texttt{0xED}.$$

Eve never learns S. She computes

$$c_1 \oplus c_2 = \texttt{0xFA} \oplus \texttt{0xED} = \texttt{0x17} = \texttt{0x59} \oplus \texttt{0x4E} = \text{`Y'} \oplus \text{`N'}.$$

`0x17` is the XOR of the two votes. Given that the plaintexts are one of two known ASCII letters, that single value tells Eve the two votes differed — and a sliver of known plaintext (a fixed header, a guessed word) peels the rest apart. One repeated nonce, and the "encrypted" channel is a crib. Now imagine the messages are kilobytes of structured English: classical two-time-pad cryptanalysis recovers both in full.

Here is the point of the whole section. At Layer 2 (raw AES-GCM) this disaster is one off-by-one counter away — a process restart that resets a nonce counter to zero, a forked worker that inherits the same counter, a virtual-machine snapshot restored twice. At Layer 3, the API never lets you supply the nonce, so the bug is *unreachable from your code*. You did not get more careful; the design got the carefulness for you.

Common trap

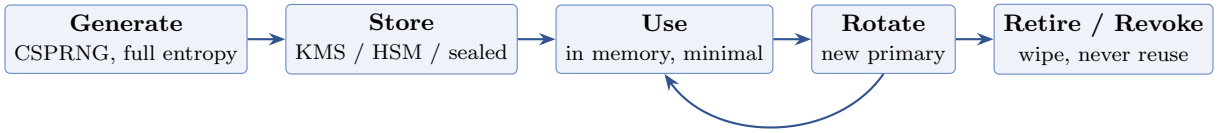
The nastiest version of this bug is the VM-snapshot or fork: your nonce counter is perfectly correct, monotone, never reused — within one process. Then the deploy system clones the running machine, or your worker pool forks after seeding the counter, and two "different" processes march the same counter from the same start. Every functional test passes. The collision happens only in production, only under scale, only once it matters. This is exactly the class of failure that a 192-bit random nonce (XSalsa20) or a managed-keyset design sidesteps by construction.

39.4 Key Management: The Part Nobody Tests

A cipher with a key is a machine with a secret. If the secret is mishandled, the machine's mathematical strength is irrelevant. Key management is where most real-world crypto actually lives and dies, and it is the part with the fewest unit tests, because "the key never leaked" is not something a test suite can assert.

39.4.1 The key lifecycle

Every key passes through stages, and each stage has its own failure mode.



Generate from a cryptographically secure RNG with full entropy — `/dev/urandom`, `getrandom()`, the OS CSPRNG. Never from `rand()`, never from a timestamp, never a hand-typed string. **Store** so the key is not sitting in a config file, an environment variable dumped to logs, or a git repository. **Use** for the shortest time and smallest scope possible, ideally never copying the raw key out of a hardware security module. **Rotate** on a schedule and after any suspected exposure. **Retire** by wiping the material and never, ever reusing it for a new purpose.

Common trap

The most common key-management failure is the most boring one: the key in source control. A long-lived API key or signing key committed to git, then the repo goes public or a laptop is stolen, and the key is in history forever — removing the file does not remove it from the commit graph. The fix is procedural, not mathematical: secrets never touch the repo, they come from a secrets manager or environment injected at deploy time, and a pre-commit scanner (gitleaks, trufflehog) blocks the commit. No cipher protects you from publishing the key.

39.4.2 Key hierarchies and key wrapping

You almost never encrypt data directly under a long-lived master key. Instead you build a *hierarchy*: one well-protected root, many cheap leaves. This is *envelope encryption*, and it solves several problems at once.

The pattern: a **master key** (also called a key-encryption key, KEK) lives in an HSM or a managed KMS and never leaves it. For each chunk of data — a file, a record, a session — you generate a fresh **data-encryption key** (DEK), encrypt the data with the DEK, then ask the KMS to encrypt ("wrap") the DEK under the KEK. You store the ciphertext and the wrapped DEK together; the bare DEK is wiped from memory. To read, you send the wrapped DEK to the KMS, it unwraps with the KEK, and you decrypt (Figure 39.2).

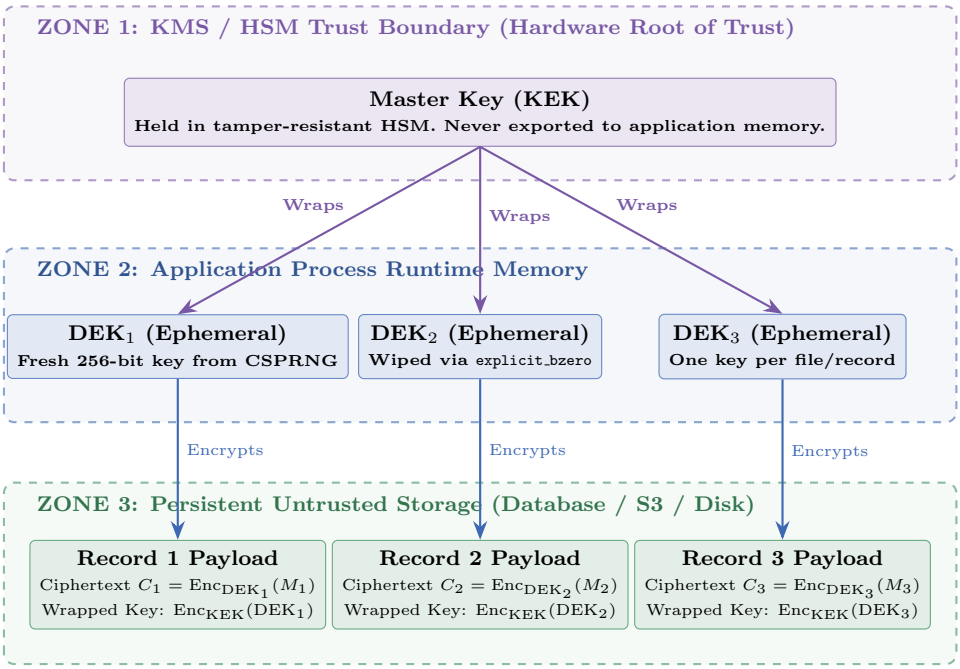


Figure 39.2: Envelope encryption architecture across three security boundaries. In Zone 1, the Master Key (KEK) is locked inside an audited hardware security module (KMS/HSM) and never touches host memory. In Zone 2, ephemeral Data Encryption Keys (DEKs) are generated on the fly to encrypt individual application objects, then wiped immediately. In Zone 3, the untrusted storage layer persists the ciphertext along with the wrapped key $\mathrm{Enc}_{\mathrm{KEK}}(\mathrm{DEK})$. A breach of persistent storage yields only encrypted keys and ciphertexts; rotation of the master key requires re-wrapping small 32-byte DEKs without rewriting gigabytes of ciphertext.

Why bother? Three concrete wins. *Blast radius:* the bare key that ever touches your application memory is a single-file DEK; compromise it and you lose one file, not the archive. *Cheap rotation:* to rotate the master key you re-wrap the small DEKs under the new KEK without re-encrypting terabytes of data — you touch kilobytes of wrapped keys, not the data itself. *Separation of duty:* the KMS enforces who may unwrap, with an audit log, while your app never sees the master key at all.

Example 39.2 (Envelope encryption with tiny numbers)**.** We will use a toy XOR "cipher" so the arithmetic is visible; a real system uses AES-GCM throughout. Let the master key be $K = \texttt{0x5C}$ (one byte). To store the message byte $m = \texttt{0x41}$ (`'A'`):

1. Draw a fresh DEK from the CSPRNG: $D = \texttt{0x1B}$.
2. Encrypt data with the DEK: $c = m \oplus D = \texttt{0x41} \oplus \texttt{0x1B} = \texttt{0x5A}$.
3. Wrap the DEK under the master key: $w = D \oplus K = \texttt{0x1B} \oplus \texttt{0x5C} = \texttt{0x47}$.
4. Wipe D. Store the pair $(c, w) = (\texttt{0x5A}, \texttt{0x47})$.

To read back: unwrap $D = w \oplus K = \texttt{0x47} \oplus \texttt{0x5C} = \texttt{0x1B}$, then decrypt $m = c \oplus D = \texttt{0x5A} \oplus \texttt{0x1B} = \texttt{0x41} =$ 'A'. Now rotate the master key to $K' = \texttt{0x33}$. We do *not* touch c. We only re-wrap: unwrap with the old K to recover $D = \texttt{0x1B}$, then re-wrap $w' = D \oplus K' = \texttt{0x1B} \oplus \texttt{0x33} = \texttt{0x28}$. Store $(\texttt{0x5A}, \texttt{0x28})$. The 1-byte ciphertext was never rewritten; only the 1-byte wrapped key changed.

Scale c to a terabyte and this approach avoids re-encrypting the bulk ciphertext during master key rotation.

Focus

Toy warning, sharpened. The XOR above is for arithmetic visibility only — XOR-wrapping is a one-time pad and reusing K across many DEKs would leak their XORs exactly as in Example 39.1. Real envelope encryption wraps with an AEAD (AES-GCM or AES-KW, the NIST key-wrap mode), which authenticates the wrap and is safe under reuse of the KEK across many DEKs. Never wrap keys by XOR. The structure of the hierarchy is the lesson; the primitive must be a real AEAD.

39.4.3 Rotation, staged so nothing breaks

Rotation is where good intentions cause outages. Swap the key in one atomic step and every piece of data encrypted under the old key suddenly will not decrypt. The fix is to let old and new keys coexist, identified by a key-id carried in the ciphertext header — exactly Tink's multi-key keyset (Figure 39.3).

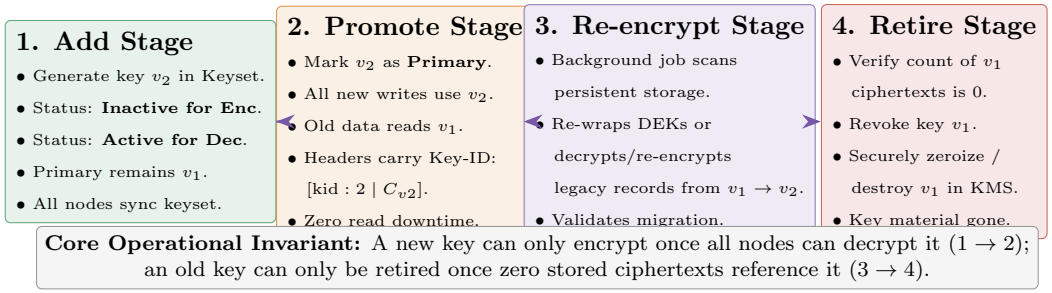


Figure 39.3: Staged key rotation timeline. Rotating a cryptographic key in production requires maintaining backward decryption compatibility while shifting write traffic. In Stage 1, key v_2 is distributed to all decrypting nodes before use. In Stage 2, v_2 is promoted to primary for new encryptions while v_1 continues decrypting historical records via ciphertext key-IDs. In Stage 3, a background process re-wraps legacy records. In Stage 4, once zero active records rely on v_1, it is safely retired and erased.

The invariant that keeps the system online: **you may start encrypting under a new key only after every consumer can decrypt it, and you may remove an old key only after no ciphertext still references it.** Each ciphertext names its key-id, so the decryptor always knows which key to reach for. Skip the staging — promote and retire in one move — and you have manufactured an outage or, worse, a silent data loss when the old key is gone and old records will not open.

39.4.4 Memory Hygiene: Dead-Store Elimination and Zeroization

When a cryptographic function finishes, its raw keys, intermediate expanded states, and plaintexts linger in heap buffers or stack frames. If those memory pages are later reallocated without clearing,

leaked via core dumps, inspected by debugging tools, or swapped to an unencrypted disk, the key material is exposed long after the operation finished.

The naive defense in C or C++ is:

```
void decrypt_and_process(const uint8_t *ciphertext, size_t len) {
    uint8_t key[32];
    derive_key(key);
    aes_gcm_decrypt(ciphertext, len, key);
    memset(key, 0, sizeof(key)); // DANGER: Compiler optimizer deletes this!
}
```

Common trap

The Dead-Store Elimination (DSE) trap. Under standard compiler optimizations (`-O2`, `-O3`), GCC and Clang analyze data dependencies. The compiler observes that the array `key` is an automatic stack variable that is never read again after `memset` before the function returns. Under the C "as-if" rule, writes to variables that are never subsequently read have no observable effect on program behavior. The optimizer silently removes the `memset` call entirely. The compiled binary leaves the 256-bit key sitting naked on the stack frame.

To enforce guaranteed zeroization that survives aggressive optimization:

- **C11 Annex K:** `memset_s(key, sizeof(key), 0, sizeof(key))`, which the ISO C standard mandates must never be optimized away.
- **POSIX / BSD / Linux glibc 2.25+:** `explicit_bzero(key, sizeof(key))`.
- **OpenSSL / BoringSSL:** `OPENSSL_cleanse(key, sizeof(key))`, which uses volatile pointer arithmetic or compiler barriers to prevent DSE.
- **Windows API:** `SecureZeroMemory(key, sizeof(key))`.

Focus

Defense in depth: memory locking with `mlock`. Operating systems relieve physical RAM pressure by paging memory blocks to swap partitions on disk. If a page holding cryptographic keys is swapped out, plaintext key material is written to non-volatile storage, surviving host reboot. High-assurance cryptographic services call `mlock()` / `VirtualLock()` on memory regions holding keys to instruct the kernel virtual memory manager never to write those pages to disk. Combine with disabling core dumps (`setrlimit(RLIMIT_CORE, 0)`) to ensure crash dumps do not persist transient keys to disk.

In the real world

How the cloud KMSs do it. AWS KMS, Google Cloud KMS, and HashiCorp Vault all implement exactly this. You get a key with versions; "rotate" adds a new version and makes it primary; ciphertext blobs embed the version id so old data keeps decrypting indefinitely; the master key never leaves the service boundary (often a FIPS-validated HSM). Envelope encryption is the default API: `GenerateDataKey` hands you a fresh DEK *and* its wrapped form in one call, and you are expected to throw away the plaintext DEK the moment you finish the operation. The entire design is the key hierarchy of Example 39.2, productized.

39.5 Secure Defaults and Crypto-Agility

A library's defaults are a security decision made on behalf of every user who never changes them — which is most of them. The discipline is: **the default configuration should be the secure configuration**, and the insecure options should be hard to reach or absent.

This is why modern designs delete the algorithm menu. Old TLS let the client and server *negotiate* a cipher suite from a long list including export-grade 40-bit keys and broken modes; the negotiation itself became the attack surface (FREAK, Logjam, and the downgrade attacks of Chapters 16 and 25). TLS 1.3 cut the list to a handful of AEAD suites and removed negotiable weak options entirely. There is no insecure choice to be tricked into, because the insecure choices are gone.

Common trap

Agility is a liability if it includes the past. "Crypto-agility" — the ability to swap algorithms — is genuinely necessary; you will need to migrate to post-quantum primitives this decade. But agility that keeps the old algorithm *selectable* is a downgrade waiting to happen. The right shape is a versioned *suite*: version 1 is "X25519 + AES-GCM," version 2 is "ML-KEM + AES-GCM," and a peer either speaks version 2 or it does not — there is no mix-and-match menu to negotiate downward. The keyset/suite carries the choice; the wire protocol does not haggle over primitives.

Intuition

A secure default is a gift to the 90% of users who copy your example code verbatim and ship it. They will not read the security section. They will not tune the parameters. Whatever your constructor does with no arguments is what runs in production. Treat the zero-argument path as your most important security feature, because it is the one almost everyone actually uses.

39.6 Testing Crypto: Vectors, Properties, and Failure Paths

We said correctness is not testable by the implementer. That is true for *security*, but ordinary correctness — "does this code compute AES the way the standard says" — is very testable, and you must, because a one-bit implementation error can void all the security. The tools are test vectors, round-trip and property tests, and — the one people forget — tests of the *rejection* path.

39.6.1 Known-answer test vectors

Standards bodies publish *test vectors*: fixed inputs with their exact expected outputs, computed by the reference implementation. NIST's CAVP/ACVP vectors, Project Wycheproof from Google, RFC appendices — they exist so your implementation, or your wiring of a library, can be checked against ground truth. A passing vector test proves you compute the same function the rest of the world computes.

Implement it

Learning only. This shows the *shape* of vector testing, not a way to trust homemade crypto. Check a real implementation against published vectors; never ship the toy.

Listing 39.2: Running a known-answer vector. Real suites have thousands of these, including deliberately malformed inputs.

```
# One ChaCha20-Poly1305 test vector (from RFC 8439). The hex strings below
# are TRUNCATED for the page -- the real vector has the full 32-byte key and
# the full expected ct+tag. A vector is:
#   key, nonce, aad, plaintext -> expected ciphertext+tag.
vectors = [
    {
      "key":   bytes.fromhex("808182838485868788898a8b..."),  # full = 32 bytes
      "nonce": bytes.fromhex("070000004041424344454647"),     # 12 bytes
      "aad":   bytes.fromhex("50515253c0c1c2c3c4c5c6c7"),
      "pt":    b"Ladies and Gentlemen of the class of '99...",
      "ct_tag": bytes.fromhex("d31a8d34648e60db7b86afbc..."),  # full expected
    },
    # ... thousands more, including malformed inputs that MUST be rejected ...
]

def run_vectors(aead_encrypt, aead_decrypt, vectors):
    for v in vectors:
        out = aead_encrypt(v["key"], v["nonce"], v["aad"], v["pt"])
        assert out == v["ct_tag"], "encryption disagrees with the standard"
        back = aead_decrypt(v["key"], v["nonce"], v["aad"], v["ct_tag"])
        assert back == v["pt"],    "decryption disagrees with the standard"
```

39.6.2 Property tests and the rejection path

Beyond fixed vectors, two kinds of generated tests catch whole classes of bug.

Round-trip property: for random keys, nonces, and messages, $\mathrm{Dec}_k(\mathrm{Enc}_k(m)) = m$. Easy, and it catches encoding bugs and length-handling errors. But notice what it does *not* catch: a decrypt routine that skips tag verification round-trips perfectly. Round-trip success is necessary, never sufficient.

Rejection property: this is the one that matters and the one people skip. Take a valid ciphertext, flip one bit anywhere — in the body, in the nonce, in the tag, in the associated data — and assert that decryption *fails*. A shocking number of crypto bugs are decryptors that do not actually check the thing they claim to check. The only way to catch "the signature is never really verified" is a test that feeds a *bad* signature and demands rejection.

Attack

The verify that never verified. A recurring real-world bug: a signature- or MAC-checking routine that returns "valid" on every input because of an inverted boolean, an exception swallowed by a bare `except: pass`, or a function that computes the expected tag and then

forgets to compare it. Apple's 2014 "goto fail" was a sibling of this — a duplicated `goto` skipped the final certificate check, so TLS accepted forged certificates. Every round-trip test passed; legitimate traffic worked perfectly. Only a *negative* test — feed a forged certificate, assert it is rejected — would have caught it. Always test that bad inputs fail, not only that good inputs succeed.

Common trap

Constant-time comparison, or you test your way to a timing oracle. When you do compare a tag or MAC, you must compare in *constant time* — a routine that inspects every byte regardless of where the first mismatch is. The natural `a == b` on byte strings short-circuits at the first differing byte, so its running time leaks how many leading bytes matched. An attacker who can time verification recovers a valid tag byte by byte, turning a forgery from 2^{128} guesses into a few thousand timed queries (Chapter 37 shows the full attack). Use the library's `constant_time_compare` / `sodium_memcmp` / `hmac.compare_digest`; never `==`.

Connection

A timing-oracle attack collapses the search cost across components. A blind forgery is a 2^{128} brute force — hopeless. A byte-at-a-time timing oracle turns it into 16 independent searches of at most 256 values each — about 128 tries per byte on average, so roughly $16 \times 128 \approx 2^{11}$ timed queries. The math of the tag did not change; the comparison routine handed the attacker a way to decompose one impossible search into sixteen trivial ones. Constant-time code denies that decomposition.

39.7 Constant-Time Discipline and Branch Elimination

Every programmer is taught to optimize code with early exits and conditional branches: if a condition is false, skip the work; if a match is found, return immediately. In cryptography, that instinct is lethal. The physical CPU does not compute in an abstract mathematical ether. It executes on silicon pipelines with branch prediction units (BPUs), instruction caches, out-of-order execution windows, and arithmetic logic units (ALUs).

When code branches on a condition that depends on secret data — a key bit, a padding byte, an authentication tag — the CPU's branch predictor records the outcome in internal history tables. An adversary sharing the CPU core (e.g., in a cloud environment) or timing network packet responses can observe whether the branch was taken.

39.7.1 The Constant-Time Multiplexer

The fundamental building block of branchless programming is the *constant-time conditional select* (or multiplexer). Suppose we have two w-bit values a and b, and a secret decision flag $c \in \{0, 1\}$. We wish to compute:

$$r = \begin{cases} a & \text{if } c = 1, \\ b & \text{if } c = 0, \end{cases}$$

without executing an `if/else` statement or conditional jump instruction.

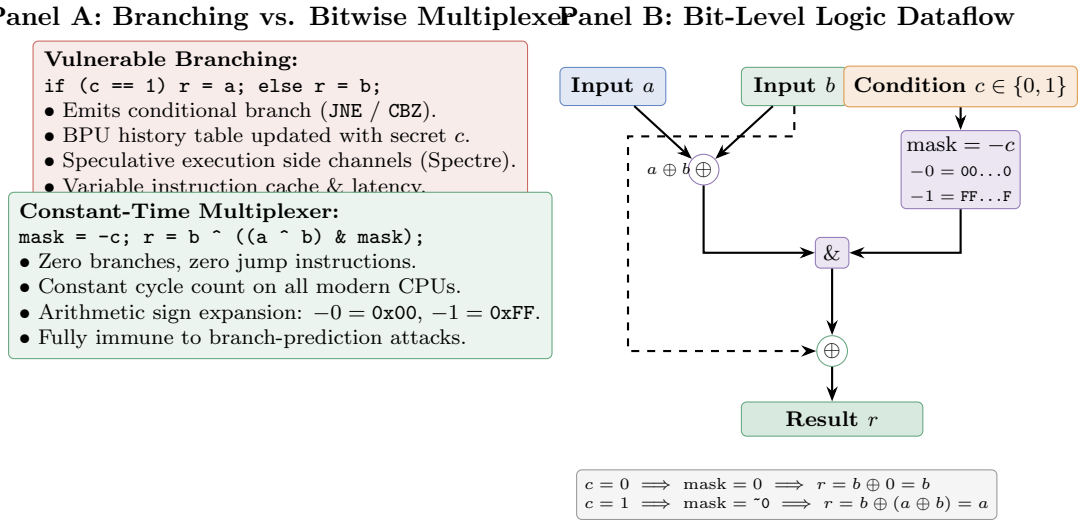


Figure 39.4: Constant-time multiplexer bit-level logic diagram ($r = b \oplus ((a \oplus b)\ \&\ -c)$). In Panel A, conditional branching code (`if/else`) introduces variable cycle latency and alters branch predictor state, creating exploitable timing side channels. In Panel B, the selection is performed using straight-line bitwise operations. Two's complement negation converts a single bit condition $c \in \{0, 1\}$ into an all-zero or all-one bitmask (mask $= -c$). The bitwise formula computes $r = b \oplus ((a \oplus b)\ \&\ \text{mask})$, which evaluates to b when $c = 0$ and a when $c = 1$ in identical clock cycles on every modern CPU.

The mechanism relies on two's complement sign expansion and bitwise Boolean logic (Figure 39.4):

1. **Mask generation via negation:** Compute mask $= -c$. In two's complement arithmetic:

$$-0 = 0 = \texttt{0x00...00}, \qquad -1 = \texttt{0xFF...FF} \quad \text{(all bits set to 1)}.$$

2. **Bitwise multiplexing:** Compute:

$$r = b \oplus \big((a \oplus b)\ \&\ \text{mask}\big).$$

Trace both cases:

- **When $c = 0$:** mask $=$ `0x00...00`. Then $(a \oplus b)\ \&\ 0 = 0$, giving:

$$r = b \oplus 0 = b.$$

- **When $c = 1$:** mask $=$ `0xFF...FF`. Then $(a \oplus b)\ \&$ `~0` $= (a \oplus b)$, giving:

$$r = b \oplus (a \oplus b) = a.$$

Every CPU instruction executed — negation, XOR, AND, XOR — takes fixed cycle latency regardless of whether c is 0 or 1. No jump is taken; no branch predictor entry is dirtied; no speculative window is opened.

39.8 Threat Modeling: Decide What You Are Defending Against

You cannot build a secure system without first writing down what "secure" means for it — against whom, protecting what, assuming what. Skip this and you will either under-protect (no defense against an attacker who is clearly in scope) or over-engineer (HSMs and key ceremonies to guard a public RSS feed). A threat model is four questions answered in plain language before any code.

1. **What are you protecting?** The assets: user messages, password database, signing key, session tokens. Be specific; "the data" is not an asset.

2. **Who is the adversary, and what can they do?** The capabilities: a passive eavesdropper on the wire? An active attacker who can inject and reorder (Dolev–Yao)? A malicious insider with database read access? An attacker who can time your server? Capabilities, not personas.

3. **What must hold for you to be secure?** The security goals tied to assets: confidentiality of messages, integrity of transfers, authenticity of the server, forward secrecy if a key later leaks.

4. **What is explicitly *out* of scope?** The assumptions you are making. "We do not defend against a compromised client device." "We assume the CSPRNG is sound." Writing these assumptions down explicitly separates documented trust boundaries from hidden blind spots.

Example 39.3 (A threat model for a small message store)**.** Suppose you are building a service where users store notes and read them back later. A one-page model:

Assets: the note contents (confidentiality), and the fact that a returned note is the one the user actually stored (integrity).

Adversaries and capabilities: (a) a network attacker who can read and modify traffic between client and server — handled by TLS, in scope; (b) an attacker who steals a database backup — in scope, so notes must be encrypted at rest under a key *not* in that backup; (c) an attacker who fully compromises the running server, including memory and the live KEK — declared *out* of scope for this tier of product.

Goals: TLS 1.3 for the channel; per-note envelope encryption with DEKs wrapped by a KMS-held KEK for at-rest confidentiality and integrity; the KEK lives in the KMS so a stolen backup is useless without KMS access.

Out of scope, stated: a live-server compromise that can call the KMS to unwrap; client-device malware; traffic-analysis (sizes and timing). These are acknowledged, not defended, at this tier.

Notice the model immediately *decides the architecture*: adversary (b) is the reason DEKs are wrapped by a KMS key rather than a key stored next to the data (Figure 39.5). Without writing down (b) you might have "encrypted at rest" with the key in the same backup — security theater that the model exposes in one line.

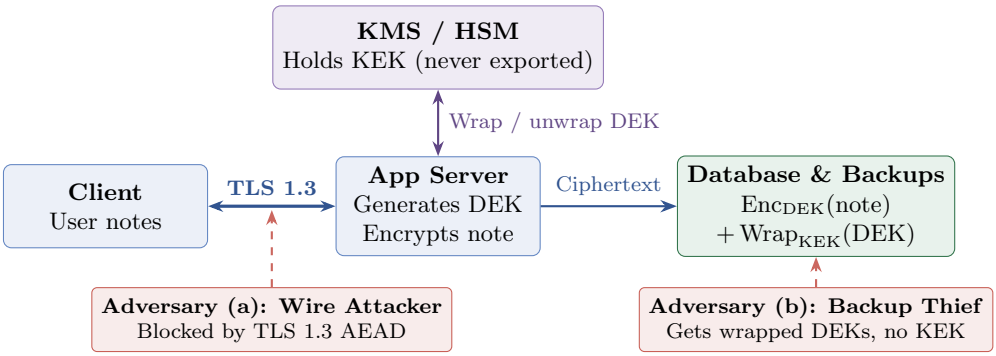


Figure 39.5: Architecture driven by the threat model of Example 39.3. Adversary (a) on the network is stopped by TLS 1.3; Adversary (b) who steals a database backup gets only ciphertexts and wrapped DEKs because the KEK remains isolated in the KMS.

Intuition

A realistic threat model establishes clear security boundaries: defining which attacks must be defended against and which are out of scope. Designing security mechanisms without explicitly documenting these adversarial capabilities leads to either unaddressed vulnerabilities or misallocated effort.

39.9 When (Almost Never) to Implement a Primitive

Having said "almost never" many times, intellectual integrity demands we name the cases where touching a primitive is defensible — and surround each with the conditions that make it not reckless.

- **You are a cryptographer doing research**, designing or analyzing a scheme for public scrutiny. The deliverable is a paper and a reference implementation meant to be attacked, not a product feature.
- **No vetted library exists for your platform.** A genuinely novel environment — a tiny microcontroller, an exotic enclave, a new language with no binding to libsodium/BoringSSL — may force your hand. The right move is usually to port a vetted implementation, not to write fresh code, and to get it reviewed and tested against full vector suites and constant-time checks before trusting it.
- **You are deliberately learning** — working the examples in a book like this one. This is valuable and you should do it. The discipline is to *label* the result as a learning artifact and never let it cross into a real system. That is exactly why every code box here carries a learning-only warning.

If you do find yourself at Layer 0 for a real deployment, the bar is high: a public, peer-reviewed algorithm (never a secret design); constant-time code with no secret-dependent branches or memory indices (Chapter 37); the full published vector suite passing, including malformed-input rejection; a defended RNG; secure memory handling that wipes keys and resists being swapped to disk; and, non-negotiably, an independent expert audit. If you cannot fund the audit, you cannot afford to roll the primitive. This requirement reflects the reality that undetected failures in custom implementations carry severe operational risk.

In the real world

The libraries you should be reaching for are themselves the product of this discipline done right. BoringSSL is Google's audited fork of OpenSSL, born after Heartbleed showed what a single unchecked length in a widely-trusted C codebase can do. libsodium is a hardened, portable packaging of NaCl. `ring` and RustCrypto bring memory-safe implementations with constant-time intent baked in. Each represents many person-years of expert attention, fuzzing, formal verification, and audit — precisely the work you are *not* going to replicate over a weekend, and the reason standing on their shoulders is not laziness but engineering.

39.10 A Practical Checklist

Print this. Tape it next to the review queue. It compresses the chapter into questions you can answer yes/no during a code review, grouped by the failure classes we have walked through.

Focus

Shipping-cryptography checklist.
Library and layer

- Am I using a vetted, audited, maintained library — not my own primitive?
- Am I standing on the highest layer that solves my problem (a box/AEAD API, not raw modes)?
- Is every algorithm choice a current standard (AES-GCM or ChaCha20-Poly1305 for AEAD, HKDF for derivation, X25519/ML-KEM for exchange, Ed25519 for signatures, Argon2/scrypt for passwords)? No DES, RC4, MD5, SHA-1, ECB, or textbook RSA.

Keys

- Are keys generated from a CSPRNG with full entropy — never a password directly, never a constant, never `rand()`?
- Are passwords run through a real KDF (Argon2id, scrypt, or PBKDF2) with a per-user salt — never hashed bare?
- Are long-lived keys in a KMS/HSM, out of source control, out of logs, out of environment dumps? Is a secret-scanner in the commit hook?
- Is each key used for exactly one purpose? No key reuse across schemes.
- Is there a staged rotation plan (add, promote, re-encrypt, retire) with key-ids in the ciphertext?

Nonces, modes, and composition

- Are nonces/IVs unique per (key, message) — counter-managed for 96-bit GCM, random only with extended nonces? Survives restarts, forks, and VM snapshots?
- Is encryption authenticated (AEAD)? Never confidentiality without integrity.
- If composing by hand (you should not be), is it encrypt-then-MAC, never MAC-then-encrypt or encrypt-and-MAC?

Comparison, randomness, and side channels

- Are all secret comparisons constant-time (`compare_digest`/`sodium_memcmp`)? Never `==` on a tag.
- Is there no secret-dependent branch or memory index in code you wrote?
- Is the RNG the OS CSPRNG, seeded, never reused after fork?

Defaults, testing, and modeling

- Is the zero-argument / default path the secure path?
- Do tests run published vectors *and* negative tests (tampered input MUST be rejected; a forged signature MUST fail)?
- Is there a written threat model naming assets, adversary capabilities, goals, and explicit out-of-scope assumptions?
- Is there a plan for crypto-agility (versioned suites, no downgrade menu) toward post-quantum?

In the real world

What the systems you trust actually do. Signal stands on libsignal (a vetted protocol implementation) rather than reinventing the ratchet per platform. WireGuard ships a single fixed cipher suite — Curve25519, ChaCha20-Poly1305, BLAKE2s — with no negotiation, so there is no downgrade to attack; "no algorithm agility on the wire" is a stated design goal. Apple and Google's mobile keystores keep device keys in dedicated secure hardware (Secure Enclave, Titan/StrongBox), so even a fully compromised OS cannot export them — the key-hierarchy and out-of-scope-compromise ideas from our threat model, in silicon. age and Tink give application developers misuse-resistant file and data encryption with managed keys. Across all of them the through-line is identical to this chapter: *choose strong primitives once, expose a narrow safe interface, and refuse to let the caller make the dangerous decision.*

39.11 What to Carry Forward

The mathematics in this book is the reason cryptography *can* work. This chapter is the reason it sometimes does not. Keep the central asymmetry in view: the cipher almost never fails, whereas the wiring around it fails silently whenever an adversary probes an overlooked assumption. Reliable cryptographic engineering means putting fewer decisions in application code and delegating them to libraries that have already survived sustained review.

Stand as high on the API ladder as your problem allows. Reach for libsodium or Tink, or higher still for a whole protocol you do not assemble. Let the library own the nonce, the mode, the comparison, the composition. Manage keys with a hierarchy and a KMS, rotate in stages, and keep secrets out of the repo. Make the default the secure default. Test the rejection path, not only the round trip. Write the threat model before the code. And treat "implement a primitive yourself" as a decision that needs an audit budget, not a free afternoon. Do that, and the mathematics of the preceding chapters gets to do its job — which is, in the end, the only job: to keep the secret.

To keep all the supporting mathematics in one place for quick lookup, the first appendix gathers the foundational number theory — from the Euclidean algorithm and Bézout's identity to Euler's

totient theorem and primality testing — that underpins public-key cryptography.

Exercises

Exercise 39.1 (Nonce reuse by hand)**.** A CTR-based AEAD produces keystream byte $S = \texttt{0x6D}$ for a given (key, nonce). Two messages are sent under the same key and the *same* nonce: $m_1 =$ 'T' $= \texttt{0x54}$ and $m_2 =$ 'F' $= \texttt{0x46}$ (a true/false bit). Compute c_1, c_2, and $c_1 \oplus c_2$. Verify $c_1 \oplus c_2 = m_1 \oplus m_2$ and explain, in one sentence, what Eve learns without ever recovering S.

Exercise 39.2 (Envelope re-wrap)**.** Using the toy XOR scheme of Example 39.2: master key $K = \texttt{0x2A}$, data byte $m = \texttt{0x6B}$, fresh DEK $D = \texttt{0x39}$. (a) Compute the stored pair (c, w). (b) The master key rotates to $K' = \texttt{0x91}$. Compute the new wrapped key w' *without* changing c. (c) State the one quantity that gets rewritten on a master-key rotation and the one that does not, and explain why this is the central payoff of envelope encryption.

Exercise 39.3 (Count the rope)**.** For each API signature, list the security-relevant decision(s) the caller is forced to make, and rank the three from most to least misuse-resistant:
(a) `secretbox(message, key)`;
(b) `aes_gcm_encrypt(message,key, nonce, aad)`;
(c) `aes_cbc_encrypt(message,key, iv)` followed by a separate `hmac(...)` you must order and compare yourself. Justify the ranking by the "count the arguments" heuristic and name the specific catastrophic failure each delegated decision enables.

Exercise 39.4 (The verify that never verified)**.** Here is a sketch of a MAC check:

```
def verify(key, msg, tag):
    expected = hmac(key, msg)
    return True
```

(a) Which test — round-trip or rejection — catches this bug, and which one passes despite it? (b) Write, in words, the single negative test case that exposes it. (c) Relate this to Apple's "goto fail": what was structurally the same?

Exercise 39.5 (Timing oracle arithmetic)**.** A 16-byte authentication tag is verified with a comparison that short-circuits at the first differing byte, and an attacker can measure the verification time. (a) Estimate the number of timed queries to recover the full tag byte by byte, and compare it to the 2^{128} cost of a blind forgery. (b) Name the one-line code change that defeats the attack and explain why it removes the leak. (c) Connect this to the search-cost-collapse idea from the connection box.

Exercise 39.6 (Write a threat model)**.** You are building a password manager: users store credentials, encrypted, synced across their devices through your server. Write a one-page threat model in the four-question form of Example 39.3. Be explicit about whether a *server compromise* is in scope, and let that decision drive where the master key lives and what the server can and cannot decrypt. Identify at least one adversary you place *out* of scope and say why.

Exercise 39.7 (Staged rotation invariant)**.** A team rotates an encryption key by deleting the old key and installing the new one in a single deploy. (a) Describe precisely what happens to data encrypted under the old key, and what the user experiences. (b) State the two-part safety invariant from Section 39.4 that the team violated. (c) Give the corrected four-phase sequence and explain what the key-id in each ciphertext header is for.

Exercise 39.8 (Defaults as a security feature)**.** Library A's AEAD constructor with no arguments selects AES-GCM and generates a random 96-bit nonce per message from a global counter. Library B's selects XSalsa20-Poly1305 with a 192-bit random nonce. (a) Using the birthday bound from the connection box, estimate how many messages each can safely encrypt before nonce collision becomes a realistic worry. (b) Argue which default is more misuse-resistant for a developer who never reads the docs, and tie your answer to the "zero-argument path" principle.

Exercise 39.9 (Justify the exception)**.** Give a concrete scenario in which implementing a standard primitive yourself (Layer 0) is defensible, then list every condition from Section 39.9 that must hold before the result may touch a real system. For one of those conditions — constant-time code — give an example of a seemingly innocent line that violates it (a secret-dependent branch or a table lookup indexed by secret data) and explain the leak.

Exercise 39.10 (Audit a snippet, end to end)**.** You are reviewing a service that: (i) reads a 16-byte AES key from an environment variable set in the deploy YAML committed to git; (ii) encrypts records with AES-CBC and a fixed all-zero IV; (iii) stores the ciphertext with no MAC; (iv) verifies an API token with `if token == stored_token`. Identify every distinct vulnerability against the checklist in Section 39.10, and for each give the one-line remediation. (There are at least five.)

Part VII

Mathematical Background

Chapter 40

Appendix: Number Theory Reference

Throughout the preceding chapters, our cryptographic constructions relied on the algebraic and modular properties of the integers. This appendix gathers, in one place, the number-theoretic machinery that the rest of the book leans on. It is meant to be read straight through once and then kept open beside the cryptography chapters as a lookup table. Where the main text proves a result slowly and motivates it with an adversarial game, here we state it crisply, prove it when the proof is short and illuminating, and trace concrete numerical examples so each formula connects directly to tangible computation. Every fact below shows up in a real protocol: the extended Euclidean algorithm computes RSA private keys, Euler's theorem proves RSA decryption undoes encryption, the Chinese Remainder Theorem quadruples RSA signing speed, Miller-Rabin is how your TLS library finds the primes in the first place, and quadratic residues underlie everything from Rabin encryption to point compression on elliptic curves.

Nothing here requires a specialized pure mathematics background. A compact set of foundational theorems, combined in different ways, generates almost all of classical public-key cryptography. Trace them on small numbers, and you can see directly why each construction works.

40.1 Notation and conventions

Throughout, $\mathbb{Z} = \{\ldots, -2, -1, 0, 1, 2, \ldots\}$ is the integers and $\mathbb{N} = \{0, 1, 2, \ldots\}$ the non-negative integers. Lowercase letters a, b, c, m, n, p, q denote integers; p and q are almost always prime. We write:

- $a \mid b$ for "a divides b": there is an integer k with $b = ak$.
- $\gcd(a, b)$ for the greatest common divisor, and $\operatorname{lcm}(a, b)$ for the least common multiple.
- $a \equiv b \pmod{n}$ for "$n \mid (a - b)$", read "a is congruent to b modulo n".
- $\mathbb{Z}/n\mathbb{Z}$ for the ring of residues $\{0, 1, \ldots, n-1\}$ under addition and multiplication mod n.
- $(\mathbb{Z}/n\mathbb{Z})^*$ for the group of *units* mod n — those residues a with $\gcd(a, n) = 1$ — under multiplication.
- $\varphi(n)$ for Euler's totient, the size of $(\mathbb{Z}/n\mathbb{Z})^*$.

- $\mathrm{ord}_n(a)$ for the multiplicative order of a modulo n.
- $a^{-1} \bmod n$ for the multiplicative inverse of a modulo n, when it exists.

A residue and its representative are conflated freely: "$3 \in \mathbb{Z}/7\mathbb{Z}$" means the class of all integers congruent to 3 mod 7, but we name it by its smallest non-negative member. When we say "reduce mod n" we mean replace an integer by the unique representative in $\{0, \ldots, n-1\}$.

Intuition

Modular arithmetic is clock arithmetic. On a 12-hour clock, $7+8=15$ becomes 3 because you wrap around at 12. Everything in this appendix is the consequence of one idea: once you only care about remainders after dividing by n, the integers collapse into a finite world of exactly n values, and that finite world is rich enough to hide secrets in.

40.2 Divisibility and the division algorithm

Theorem 40.1 (Division algorithm)**.** *For any integer a and any positive integer n, there exist unique integers q (quotient) and r (remainder) with*

$$a = qn + r, \qquad 0 \le r < n.$$

The remainder r is what we call $a \bmod n$. Existence is just "keep subtracting n until you land in $[0, n)$"; uniqueness follows because two remainders differing by less than n whose difference is divisible by n must be equal. Worked: $a = 47$, $n = 9$ gives $47 = 5 \cdot 9 + 2$, so $47 \bmod 9 = 2$. For negatives we still demand $r \ge 0$: $-47 = (-6) \cdot 9 + 7$, so $-47 \bmod 9 = 7$, *not* -2. This sign convention matters in code; see the trap below.

Common trap

In C, Java, and JavaScript the `%` operator follows the sign of the *dividend*, so `-47 % 9` returns -2, not the mathematician's 7. Cryptographic code that indexes tables or computes Legendre symbols with a raw `%` on a possibly-negative value will silently read the wrong slot. The fix is `((a % n) + n) % n`, or use a big-integer library whose `mod` already normalizes to $[0, n)$. Python's `%` already does the right thing.

40.3 Greatest common divisor and the Euclidean algorithm

The greatest common divisor $\gcd(a, b)$ is the largest integer dividing both a and b (with $\gcd(0, 0)$ left undefined and $\gcd(a, 0) = |a|$). Two numbers with $\gcd = 1$ are *coprime*. The Euclidean algorithm computes the gcd using one fact:

$$\gcd(a, b) = \gcd(b,\ a \bmod b).$$

Any common divisor of a and b divides $a - qb = a \bmod b$, and vice versa, so the set of common divisors never changes as we replace (a, b) by $(b, a \bmod b)$; the second coordinate shrinks every step until it hits 0, and then the first coordinate is the gcd.

Example 40.1 (Euclid on 1071 and 462)**.**

$$\begin{aligned}\gcd(1071, 462) &= \gcd(462,\ 1071 \bmod 462) = \gcd(462, 147)\\ &= \gcd(147,\ 462 \bmod 147) = \gcd(147, 21)\\ &= \gcd(21,\ 147 \bmod 21) = \gcd(21, 0) = 21.\end{aligned}$$

Connection

The Euclidean algorithm is very fast: the number of steps is $O(\log \min(a, b))$, and the worst case is consecutive Fibonacci numbers (Lamé's theorem). The companion algorithms book analyzes this; the upshot for cryptography is that gcd, modular inverse, and therefore RSA key generation are all cheap even on 4096-bit numbers. Contrast that with factoring those same numbers, which nobody knows how to do quickly — the whole field lives in that asymmetry.

40.3.1 Bézout and the extended Euclidean algorithm

Theorem 40.2 (Bézout's identity)**.** *For integers a, b not both zero, there exist integers s, t (the* Bézout coefficients*) with*

$$as + bt = \gcd(a, b).$$

The extended Euclidean algorithm finds s and t alongside the gcd by carrying the combination through every step. This is the single most useful subroutine in public-key crypto, because the case $\gcd(a, n) = 1$ gives $as + nt = 1$, i.e. $as \equiv 1 \pmod{n}$, so $s = a^{-1} \bmod n$.

Example 40.2 (Extended Euclid: inverse of 7 mod 26)**.** We track triples (r, s, t) with $r = 7s + 26t$. Start from $(26, 0, 1)$ and $(7, 1, 0)$; each row subtracts q times the next.

remainder r	quotient q	s	t
26		0	1
7		1	0
5	3	-3	1
2	1	4	-1
1	2	-11	3
0	2		

The last nonzero remainder is $1 = \gcd(7, 26)$, with $s = -11$, $t = 3$: check $7 \cdot (-11) + 26 \cdot 3 = -77 + 78 = 1$. So $7^{-1} \equiv -11 \equiv 15 \pmod{26}$, and indeed $7 \cdot 15 = 105 = 4 \cdot 26 + 1$.

Implement it

A compact iterative extended-gcd in Python. It returns (g, s, t) with $as + bt = g$.

```
def ext_gcd(a, b):
    old_r, r = a, b
    old_s, s = 1, 0
    old_t, t = 0, 1
    while r != 0:
        q = old_r // r
        old_r, r = r, old_r - q * r
```

```
        old_s, s = s, old_s - q * s
        old_t, t = t, old_t - q * t
    return old_r, old_s, old_t          # g, s, t  with a*s + b*t = g

def inverse(a, n):
    g, s, _ = ext_gcd(a % n, n)
    if g != 1:
        raise ValueError("no inverse: gcd != 1")
    return s % n                        # normalize into [0, n)

# inverse(7, 26) -> 15
```

LEARNING ONLY. Real libraries use constant-time, side-channel-hardened modular inverses (e.g. Fermat-based or a binary GCD with branchless updates) because the loop count above leaks information about the secret operand through timing. Never ship this for key material.

40.4 Modular arithmetic: the ring $\mathbb{Z}/n\mathbb{Z}$

Congruence respects addition, subtraction, and multiplication: if $a \equiv a'$ and $b \equiv b' \pmod{n}$, then $a \pm b \equiv a' \pm b'$ and $ab \equiv a'b' \pmod{n}$. So we can reduce "early and often" — replace any intermediate result by its remainder without changing the final answer. This is why modular exponentiation of huge numbers never overflows: you reduce after each multiply.

Walking question

Why can't we always divide? In $\mathbb{Z}/n\mathbb{Z}$, division by a means multiplying by a^{-1}, and that inverse exists *iff* $\gcd(a, n) = 1$. In $\mathbb{Z}/12\mathbb{Z}$ the element 3 has no inverse: $3 \cdot k$ runs through $\{0, 3, 6, 9\}$ and never hits 1. Trace it: $3 \cdot 1 = 3$, $3 \cdot 2 = 6$, $3 \cdot 3 = 9$, $3 \cdot 4 = 12 \equiv 0$, then it repeats. The multiples of a non-coprime element are trapped on a sublattice. Only the units — the residues coprime to n — are invertible.

Definition 40.1 (Units)**.** $(\mathbb{Z}/n\mathbb{Z})^* = \{a \in \mathbb{Z}/n\mathbb{Z} : \gcd(a, n) = 1\}$ is the group of units under multiplication mod n. It is closed, associative, has identity 1, and every element has an inverse (by Bézout). When $n = p$ is prime, every nonzero residue is a unit, so $(\mathbb{Z}/p\mathbb{Z})^* = \{1, 2, \ldots, p-1\}$ and $\mathbb{Z}/p\mathbb{Z}$ is a field.

40.4.1 Fast modular exponentiation

Computing $a^e \bmod n$ by multiplying a by itself e times is hopeless when e has hundreds of digits. Square-and-multiply does it in $O(\log e)$ multiplications by reading the exponent's binary digits. To compute $a^{13} \bmod n$, note $13 = 1101_2$, so $a^{13} = a^8 \cdot a^4 \cdot a^1$: square repeatedly to get a, a^2, a^4, a^8 and multiply in the ones that the bits select.

Example 40.3 ($3^{13} \bmod 7$)**.** Repeated squaring mod 7: $3^1 = 3$, $3^2 = 9 \equiv 2$, $3^4 \equiv 2^2 = 4$, $3^8 \equiv 4^2 = 16 \equiv 2$. Bits of $13 = 1101_2$ pick a^8, a^4, a^1: $3^{13} \equiv 2 \cdot 4 \cdot 3 = 24 \equiv 3 \pmod{7}$. (Sanity check via Fermat below: $3^6 \equiv 1$, so $3^{13} = 3^{12} \cdot 3 \equiv 1 \cdot 3 = 3$.)

Implement it

```python
def mod_exp(a, e, n):
    result = 1
    a = a % n
    while e > 0:
        if e & 1:                  # current low bit is set
            result = (result * a) % n
        a = (a * a) % n            # square the base
        e >>= 1                    # shift to next bit
    return result
# mod_exp(3, 13, 7) -> 3
```

LEARNING ONLY. The branch `if e & 1` makes the running time depend on the secret exponent's bits — a classic timing/power side channel that has broken real RSA and ECC implementations. Production code uses Montgomery ladders or constant-time windowed exponentiation. Python's built-in `pow(a, e, n)` is the right call for learning, and is itself not constant-time.

40.5 Fermat's little theorem and Euler's theorem

Theorem 40.3 (Fermat's little theorem)**.** *If p is prime and $p \nmid a$, then*

$$a^{p-1} \equiv 1 \pmod{p}.$$

Equivalently $a^p \equiv a \pmod{p}$ for every *integer a.*

Why it works

Here is the one-line proof worth remembering. Multiply the nonzero residues $\{1, 2, \ldots, p-1\}$ each by a. Since a is a unit, the products $\{a \cdot 1, \ldots, a \cdot (p-1)\}$ are the same set, just shuffled. Multiply both sets together:

$$(p-1)! \equiv a^{p-1}\,(p-1)! \pmod{p}.$$

Now $(p-1)!$ is coprime to p, so cancel it: $a^{p-1} \equiv 1$. The whole theorem is "relabeling a set does not change its product."

Theorem 40.4 (Euler's theorem)**.** *If $\gcd(a, n) = 1$ then*

$$a^{\varphi(n)} \equiv 1 \pmod{n},$$

where $\varphi(n) = |(\mathbb{Z}/n\mathbb{Z})^*|$ is Euler's totient.

The proof is identical: multiplication by a unit a permutes $(\mathbb{Z}/n\mathbb{Z})^*$, so the product of all units equals $a^{\varphi(n)}$ times itself, and cancellation gives the result. Fermat is the special case $n = p$, where $\varphi(p) = p - 1$.

40.5.1 Computing the totient

φ is multiplicative on coprime arguments and easy on prime powers:

$$\varphi(p) = p - 1, \qquad \varphi(p^k) = p^k - p^{k-1} = p^k\Big(1 - \tfrac{1}{p}\Big), \qquad \varphi(mn) = \varphi(m)\varphi(n) \ \text{ if } \gcd(m,n) = 1.$$

So from the prime factorization $n = \prod_i p_i^{k_i}$,

$$\varphi(n) = n \prod_i \Big(1 - \tfrac{1}{p_i}\Big).$$

Example 40.4 (φ of a few numbers)**.** $\varphi(10) = \varphi(2)\varphi(5) = 1 \cdot 4 = 4$ (the units are $1, 3, 7, 9$). $\varphi(36) = \varphi(4)\varphi(9) = 2 \cdot 6 = 12$. For an RSA-style modulus $n = pq$ with distinct primes, $\varphi(n) = (p-1)(q-1)$: e.g. $p = 61, q = 53$ give $n = 3233$ and $\varphi(n) = 60 \cdot 52 = 3120$.

In the real world

Euler's theorem is the engine of RSA correctness. With $n = pq$ and exponents e, d chosen so that $ed \equiv 1 \pmod{\varphi(n)}$, decryption recovers the message because $m^{ed} = m^{1+k\varphi(n)} = m \cdot (m^{\varphi(n)})^k \equiv m \pmod{n}$. The private key d is literally $e^{-1} \bmod \varphi(n)$, computed with the extended Euclidean algorithm of §40.3.1. Knowing $\varphi(n)$ is equivalent to factoring n — which is why p, q must stay secret.

40.6 The Chinese Remainder Theorem

Theorem 40.5 (CRT)**.** *Let $n_1, \dots, n_k$ be pairwise coprime and $N = \prod_i n_i$. Then the system*

$$x \equiv a_1 \pmod{n_1}, \quad \dots, \quad x \equiv a_k \pmod{n_k}$$

has a unique solution modulo N. Equivalently, the map $x \mapsto (x \bmod n_1, \dots, x \bmod n_k)$ is a ring isomorphism $\mathbb{Z}/N\mathbb{Z} \cong \mathbb{Z}/n_1\mathbb{Z} \times \cdots \times \mathbb{Z}/n_k\mathbb{Z}$.

The isomorphism view is the one to internalize: arithmetic mod N *is* component-wise arithmetic on the small moduli. Add, multiply, exponentiate each coordinate independently, then glue back. The gluing is explicit. For two moduli, write $N = n_1 n_2$ and find $m_1 = n_2^{-1} \bmod n_1$, $m_2 = n_1^{-1} \bmod n_2$; then

$$x = a_1\, n_2\, m_1 + a_2\, n_1\, m_2 \pmod{N}.$$

Example 40.5 (Solve $x \equiv 2$ (5), $x \equiv 3$ (7))**.** Here $n_1 = 5, n_2 = 7, N = 35$. Compute $m_1 = 7^{-1} \bmod 5$: $7 \equiv 2$, and $2 \cdot 3 = 6 \equiv 1$, so $m_1 = 3$. Compute $m_2 = 5^{-1} \bmod 7$: $5 \cdot 3 = 15 \equiv 1$, so $m_2 = 3$. Then

$$x = 2 \cdot 7 \cdot 3 + 3 \cdot 5 \cdot 3 = 42 + 45 = 87 \equiv 87 - 2 \cdot 35 = 17 \pmod{35}.$$

Check: $17 \bmod 5 = 2$ and $17 \bmod 7 = 3$. Correct.

The isomorphism $\mathbb{Z}/35\mathbb{Z} \cong \mathbb{Z}/5\mathbb{Z} \times \mathbb{Z}/7\mathbb{Z}$ says each $x \in \{0, \dots, 34\}$ sits at a unique cell of a 5×7 grid indexed by its two remainders, and every cell is hit exactly once. Figure 40.1 plots a few values; the pair $(2, 3)$ pins down $x = 17$.

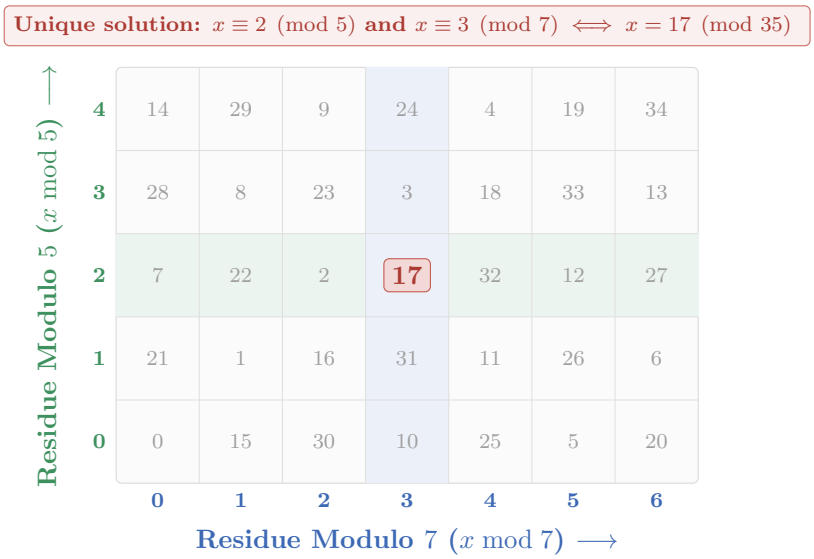


Figure 40.1: The Chinese Remainder Theorem isomorphism $\mathbb{Z}/35\mathbb{Z} \cong \mathbb{Z}/5\mathbb{Z} \times \mathbb{Z}/7\mathbb{Z}$. Every residue $x \in \{0, \ldots, 34\}$ corresponds bijectively to a single cell determined by its coordinate remainders. The system $x \equiv 2 \pmod 5$ (row 2) and $x \equiv 3 \pmod 7$ (column 3) intersects at the single highlighted solution $x = 17$.

In the real world

RSA decryption and signing run about four times faster with CRT. Instead of one exponentiation mod n (a $2k$-bit number), you do two exponentiations mod p and mod q (each k-bit), then glue. The cost of an exponentiation grows roughly with the cube of the bit length, so halving the modulus size cuts each piece to $1/8$, and two of them total $1/4$. OpenSSL, BoringSSL, and every serious RSA implementation store the CRT parameters $d_p = d \bmod (p-1)$, $d_q = d \bmod (q-1)$, and $q^{-1} \bmod p$ in the private key for exactly this reason.

Attack

CRT-RSA has a notorious failure mode. If a hardware fault (a glitch, a cosmic ray, a deliberate laser) corrupts the computation modulo p but not modulo q, the faulty signature $\hat{s}$ satisfies $\hat{s}^e \equiv m \pmod q$ but $\hat{s}^e \not\equiv m \pmod p$. Then

$$\gcd(\hat{s}^e - m,\ n) = q,$$

handing the attacker a prime factor and thus the entire private key from *one* faulty signature. This is the Boneh-DeMillo-Lipton fault attack. Defenses verify the signature (recompute $\hat{s}^e \bmod n$ and compare to m) before releasing it.

40.7 Orders, cyclic groups, and primitive roots

Definition 40.2 (Multiplicative order)**.** For $a \in (\mathbb{Z}/n\mathbb{Z})^*$, the *order* $\mathrm{ord}_n(a)$ is the least positive integer d with $a^d \equiv 1 \pmod n$.

By Euler's theorem $a^{\varphi(n)} \equiv 1$, and a short argument shows $\mathrm{ord}_n(a) \mid \varphi(n)$: divide $\varphi(n)$ by d with remainder r, and $a^r \equiv 1$ would follow, contradicting minimality of d unless $r = 0$. So orders are always divisors of the group size — a special case of Lagrange's theorem.

Example 40.6 (Orders in $(\mathbb{Z}/7\mathbb{Z})^*$)**.** Here $\varphi(7) = 6$, so every order divides 6. Powers of 3: $3, 2, 6, 4, 5, 1$ — order 6. Powers of 2: $2, 4, 1$ — order 3. Powers of 6: $6, 1$ — order 2. The possible orders $1, 2, 3, 6$ all appear, and 3 runs through *all* six units, so 3 is a generator.

Definition 40.3 (Primitive root)**.** A *primitive root* modulo n is an element $g \in (\mathbb{Z}/n\mathbb{Z})^*$ of order $\varphi(n)$; its powers generate the whole group, making $(\mathbb{Z}/n\mathbb{Z})^*$ *cyclic* with generator g.

Theorem 40.6 (Existence of primitive roots)**.** *$(\mathbb{Z}/n\mathbb{Z})^*$ is cyclic — a primitive root exists — exactly when $n \in \{1, 2, 4, p^k, 2p^k\}$ for an odd prime p. In particular every prime modulus p has a primitive root, and $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic of order $p - 1$.*

That last sentence is what Diffie-Hellman and ElGamal stand on: a prime p gives a cyclic group in which exponentiation is a permutation but its inverse (the discrete logarithm) is believed hard.

Why it works

A cyclic group of order m has exactly $\varphi(m)$ generators, and for each divisor $d \mid m$ exactly $\varphi(d)$ elements of order d. Why $\varphi(m)$ generators? If g generates, then g^k generates iff $\gcd(k, m) = 1$ — otherwise g^k cycles through a proper subgroup. In $(\mathbb{Z}/7\mathbb{Z})^*$ ($m = 6$) there are $\varphi(6) = 2$ primitive roots, namely 3 and 5, matching the count from the example above.

Connection

To *test* whether g is a primitive root mod p you do not check all $p - 2$ powers. Factor $p - 1 = \prod q_i^{e_i}$ and verify $g^{(p-1)/q_i} \not\equiv 1 \pmod{p}$ for each prime q_i. If none of those drops to 1, then $\mathrm{ord}_p(g)$ cannot be a proper divisor of $p - 1$, so it equals $p - 1$. This is why "safe primes" $p = 2q + 1$ (with q prime) are popular: $p - 1 = 2q$ has only two prime factors, so generator testing is trivial and the only subgroups have orders $1, 2, q, 2q$.

40.8 Primality testing

Generating RSA or Diffie-Hellman parameters means producing large primes — often thousands of bits. Trial division is hopeless past a few digits. Probabilistic tests built on Fermat's theorem handle prime generation in practice.

40.8.1 The Fermat test and its blind spot

If n is prime, Fermat says $a^{n-1} \equiv 1 \pmod{n}$ for all a coprime to n. So pick a random base a; if $a^{n-1} \not\equiv 1$, then n is *definitely composite* (a is a "witness"). If it equals 1, n is probably prime. The blind spot:

Attack

Carmichael numbers are composite n for which $a^{n-1} \equiv 1 \pmod{n}$ for *every* a coprime to n — the Fermat test never finds a witness no matter how many bases you try. The smallest is $561 = 3 \cdot 11 \cdot 17$. There are infinitely many. A naive Fermat-only primality check will cheerfully accept 561 as prime, and an RSA modulus built on such a "prime" factors instantly. This is why production code uses Miller-Rabin, which has no Carmichael blind spot.

40.8.2 Miller-Rabin

Miller-Rabin strengthens the Fermat test by looking inside the squaring chain. Write $n - 1 = 2^s\, d$ with d odd. For a base a, compute $x = a^d \bmod n$ and then square it s times. If n is prime, the sequence $a^d, a^{2d}, \ldots, a^{2^s d} = a^{n-1}$ must end at 1, and the first time it reaches 1 it must do so *from* -1 (because in a field the only square roots of 1 are ± 1). So n *passes* for base a if either

$$a^d \equiv 1 \pmod{n}, \qquad \text{or} \qquad a^{2^r d} \equiv -1 \pmod{n} \quad \text{for some } 0 \le r < s.$$

Otherwise a is a witness and n is composite. The key theorem: for an odd composite n, at least $3/4$ of the bases in $\{1, \ldots, n-1\}$ are witnesses. So k independent random bases that all pass leave a failure probability of at most 4^{-k}.

Example 40.7 (Miller-Rabin catches 561)**.** $n = 561$, so $n - 1 = 560 = 2^4 \cdot 35$, giving $s = 4$, $d = 35$. Take base $a = 2$. Compute $2^{35} \bmod 561 = 263$ (not ± 1). Square: $263^2 = 69169 \equiv 166 \pmod{561}$. Square: $166^2 = 27556 \equiv 67$. Square: $67^2 = 4489 \equiv 1 \pmod{561}$. We reached 1, but the value just before it was 67, not $560 \equiv -1$. A nontrivial square root of 1 appeared — impossible in a field — so 2 is a witness and 561 is composite. The Fermat test missed this because $2^{560} \equiv 1$; Miller-Rabin caught the illegal square root on the way up (Figure 40.2).

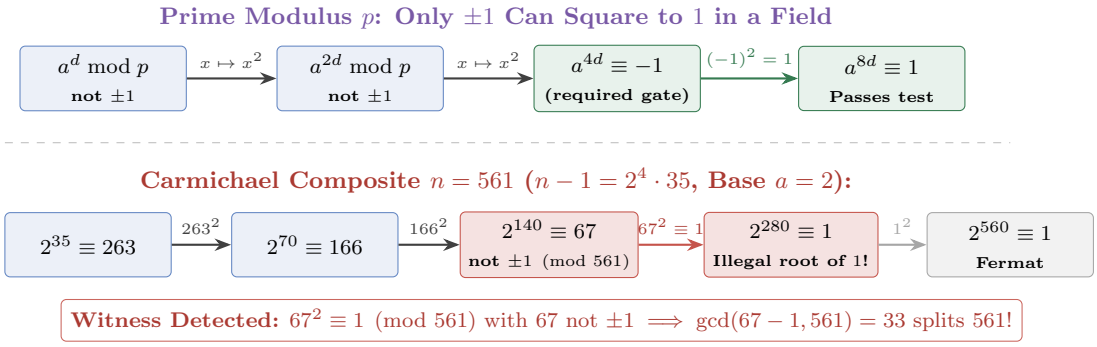


Figure 40.2: The Miller–Rabin squaring chain versus the Fermat test. In a prime field $\mathbb{Z}/p\mathbb{Z}$ (top), any sequence of squarings that reaches 1 from a non-trivial value must pass through -1, because $x^2 - 1 = (x-1)(x+1)$ has only two roots. For the Carmichael number $n = 561$ with base $a = 2$ (bottom), the Fermat test checks only the final value $2^{560} \equiv 1$ and is fooled, whereas Miller–Rabin inspects each intermediate squaring and catches $67 \not\equiv \pm 1$ squaring directly to 1.

Implement it

```python
import random

def miller_rabin(n, rounds=40):
    if n < 2:            return False
    for p in (2,3,5,7,11,13,17,19,23,29,31,37):
        if n % p == 0:   return n == p
    # write n-1 = 2^s * d  with d odd
    d, s = n - 1, 0
    while d % 2 == 0:
        d //= 2; s += 1
    for _ in range(rounds):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(s - 1):
            x = (x * x) % n
            if x == n - 1:
                break
        else:
            return False           # a is a witness: composite
    return True                    # probably prime
```

LEARNING ONLY. This uses non-cryptographic `random` and is not constant-time. For real key generation, draw bases from a CSPRNG and follow standards (FIPS 186-5 specifies round counts by key size); libraries also combine Miller-Rabin with a Lucas test (the Baillie-PSW combination) so that no known composite passes.

History

Distinguishing prime numbers from composites without factoring goes back to Pierre de Fermat (1640) and Leonhard Euler (1736). In 1976 Gary Miller designed a polynomial-time primality test based on non-trivial modular square roots of unity whose correctness relied on the unproven Generalized Riemann Hypothesis (under GRH, testing bases up to $2(\ln n)^2$ suffices). In 1980 Michael Rabin turned Miller's test into an unconditional randomized algorithm where every composite fails for at least three-fourths of bases, sparking early debate over whether security-critical keys could rely on probabilistic tests with a $\leq 4^{-k}$ error bound. A deterministic polynomial-time test (AKS, 2002, by Manindra Agrawal, Neeraj Kayal, and Nitin Saxena) finally settled the theoretical question — PRIMES is in **P** — but it is far slower in practice than Miller–Rabin; cryptographers instead run enough random Miller–Rabin rounds that the error probability sits far below hardware fault rates.

40.9 Quadratic residues

Definition 40.4 (Quadratic residue)**.** For odd prime p and $a \not\equiv 0$, a is a *quadratic residue* (QR) mod p if $x^2 \equiv a \pmod{p}$ has a solution; otherwise a *quadratic non-residue* (QNR).

Exactly half of the nonzero residues are QRs: the squaring map $x \mapsto x^2$ is two-to-one (since x and $-x$ share a square), so its image has size $(p-1)/2$. Modulo 7: the squares of $1, \ldots, 6$ are $1, 4, 2, 2, 4, 1$, so the QRs are $\{1, 2, 4\}$ and the QNRs are $\{3, 5, 6\}$.

Definition 40.5 (Legendre symbol)**.**

$$\left(\frac{a}{p}\right) = \begin{cases} 1 & a \text{ is a QR mod } p, \\ -1 & a \text{ is a QNR mod } p, \\ 0 & p \mid a. \end{cases}$$

Theorem 40.7 (Euler's criterion)**.** *For odd prime p and $p \nmid a$,*

$$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod{p}.$$

Why: $a^{(p-1)/2}$ squares to $a^{p-1} \equiv 1$, so it is a square root of 1, hence ± 1. It is $+1$ precisely when a is a square, because if $a = g^{2j}$ (a generator power with even exponent) then $a^{(p-1)/2} = g^{j(p-1)} \equiv 1$, while odd exponents give -1. This is a fast QR test: one modular exponentiation. Check mod 7: $3^3 = 27 \equiv 6 \equiv -1$, so 3 is a QNR (matches the list); $2^3 = 8 \equiv 1$, so 2 is a QR.

Example 40.8 (Computing $\left(\frac{2}{7}\right)$ and a square root)**.** 2 is a QR mod 7. When $p \equiv 3 \pmod 4$ — and 7 is — a square root is just $a^{(p+1)/4}$: here $2^{(7+1)/4} = 2^2 = 4$, and indeed $4^2 = 16 \equiv 2 \pmod 7$. The other root is $-4 \equiv 3$; check $3^2 = 9 \equiv 2$. For $p \equiv 1 \pmod 4$ you need the Tonelli-Shanks algorithm instead.

In the real world

Quadratic residues are everywhere. Elliptic-curve *point compression* stores only the x-coordinate plus one sign bit: the receiver recovers y by taking a modular square root of $x^3 + ax + b$, which exists only when that value is a QR — the sign bit picks which root. The *Goldwasser-Micali* cryptosystem encrypts a 0 bit as a random QR and a 1 bit as a QNR modulo $n = pq$, and its security rests on the *quadratic residuosity problem*: deciding QR-ness modulo a composite without the factorization is believed hard. *Rabin encryption* squares the message mod n, and decrypting means taking square roots mod p and q and CRT-gluing the four roots. And the Jacobi symbol (the multiplicative extension of Legendre to composite moduli) lets you compute these symbols fast *without* factoring, via quadratic reciprocity.

Common trap

The Jacobi symbol $\left(\frac{a}{n}\right)$ for composite n can equal $+1$ even when a is a *non*-residue mod n (it only tracks the product of Legendre symbols, so two -1's cancel). So a Jacobi value of $+1$ does *not* prove a is a square modulo a composite. Goldwasser-Micali exploits exactly this gap: it hides bits inside the pseudosquares (Jacobi $+1$ but not a true QR), which are indistinguishable from real squares without the factorization.

40.10 Polynomial Rings, Quotients, and Cyclotomic Reductions

Just as integers wrap around a modulus n to form the finite ring $\mathbb{Z}/n\mathbb{Z}$, polynomials wrap around an irreducible polynomial $f(x)$ to form polynomial quotient rings. These rings undergird modern

lattice-based post-quantum cryptography (Section 35.4) and secret-sharing multiplication protocols (Section 34.5).

40.10.1 The Polynomial Ring $\mathbb{F}[x]$

Let $\mathbb{F}$ be a field (such as $\mathbb{Z}/p\mathbb{Z}$ for prime p). The set of polynomials with coefficients in $\mathbb{F}$, denoted $\mathbb{F}[x]$, consists of all expressions $a(x) = \sum_{i=0}^{d} a_i x^i$ with $a_i \in \mathbb{F}$. Addition and multiplication follow standard polynomial algebra, with coefficient operations performed strictly within $\mathbb{F}$.

Theorem 40.8 (Polynomial Division Algorithm)**.** *For any $a(x), b(x) \in \mathbb{F}[x]$ with $b(x) \neq 0$, there exist unique polynomials $q(x)$ (quotient) and $r(x)$ (remainder) such that:*

$$a(x) = q(x) \cdot b(x) + r(x), \qquad \text{where } \deg(r) < \deg(b) \text{ or } r(x) = 0.$$

When coefficients belong to a field $\mathbb{F}$, polynomial long division is identical to integer long division because leading coefficients can always be inverted.

40.10.2 Quotient Rings and the Cyclotomic Modulus $x^n + 1$

In post-quantum lattice cryptography (specifically Ring-LWE, ML-KEM, and ML-DSA), we work in the quotient polynomial ring:

$$\mathcal{R}_q = \mathbb{Z}/q\mathbb{Z}[x]/(x^n + 1),$$

where n is chosen as a power of two (typically $n = 256$) and q is a prime.

Every element in $\mathcal{R}_q$ is uniquely represented by a polynomial of degree at most $n - 1$:

$$a(x) = a_0 + a_1 x + a_2 x^2 + \cdots + a_{n-1} x^{n-1}, \qquad a_i \in \mathbb{Z}/q\mathbb{Z}.$$

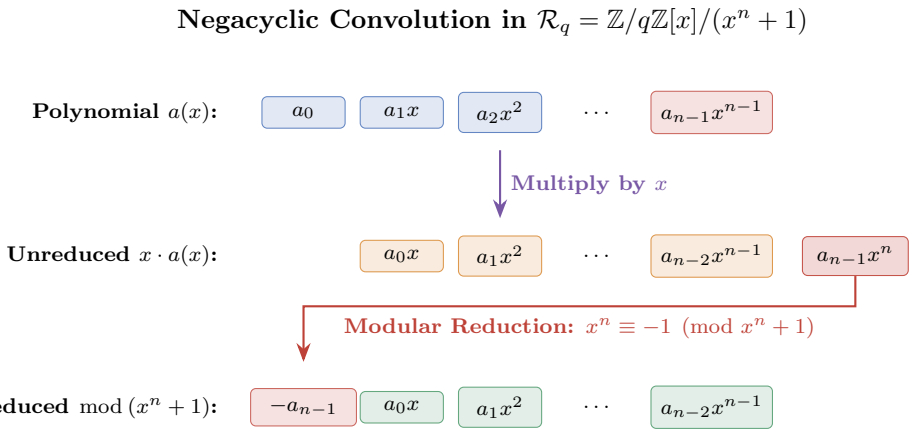


Figure 40.3: Negacyclic polynomial reduction in the cyclotomic quotient ring $\mathcal{R}_q = \mathbb{Z}/q\mathbb{Z}[x]/(x^n+1)$. Multiplying a polynomial by x shifts each coefficient to the next higher power of x. The top coefficient $a_{n-1}x^n$ exceeds degree $n-1$; reduction modulo $x^n + 1$ replaces x^n with -1, wrapping the leading coefficient back to the constant position with a sign inversion ($-a_{n-1}$). This negacyclic convolution property allows lattice-based post-quantum schemes (Kyber/Dilithium) to implement polynomial multiplications in $O(n \log n)$ time using the Number Theoretic Transform (NTT).

The reduction modulo $x^n + 1$ has an elegant geometric property:

$$x^n \equiv -1 \pmod{x^n + 1}.$$

Whenever multiplication creates terms of degree $\geq n$, higher powers wrap around with a sign flip:

$$x^n \mapsto -1, \quad x^{n+1} \mapsto -x, \quad x^{n+2} \mapsto -x^2, \quad \ldots$$

This sign-flipping wrap-around is called *negacyclic convolution* (Figure 40.3). It turns polynomial multiplication into matrix-vector multiplication over a skew-circulant lattice, enabling rapid $O(n \log n)$ evaluation via the discrete Fourier transform analogue: the Number Theoretic Transform (NTT).

40.11 Lagrange Polynomial Interpolation

The Fundamental Theorem of Algebra guarantees that a non-zero polynomial of degree at most $k-1$ over a field $\mathbb{F}$ has at most $k-1$ distinct roots. A direct consequence is that any k distinct evaluations:

$$(x_1, y_1), \quad (x_2, y_2), \quad \ldots, \quad (x_k, y_k), \qquad x_i \in \mathbb{F},\ y_i \in \mathbb{F},$$

uniquely determine a polynomial $f(x)$ of degree at most $k-1$. If two polynomials of degree $\leq k-1$ agreed on k points, their difference would have k roots and thus must be zero.

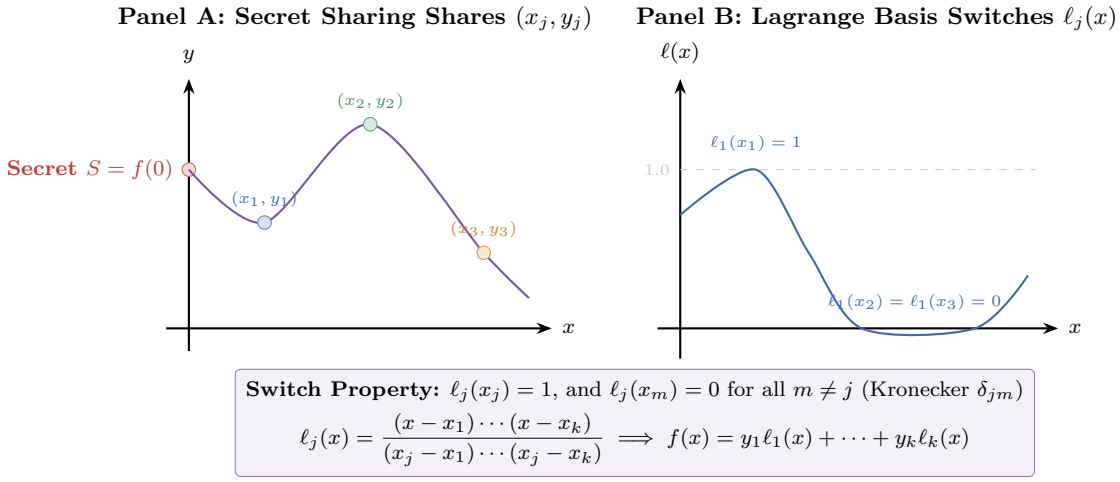


Figure 40.4: Lagrange polynomial interpolation basis switch mechanism. In Panel A, three shares (x_j, y_j) uniquely identify a degree-2 polynomial whose y-intercept reveals secret $S = f(0)$. In Panel B, Lagrange interpolation decomposes the polynomial into basis polynomials $\ell_j(x)$. Each basis polynomial acts as an arithmetic switch: it equals 1 at node x_j and precisely vanishes (0) at every other node x_m, eliminating linear-algebra matrix inversion.

Rather than setting up a system of k equations and inverting a Vandermonde matrix, Joseph-Louis Lagrange (1795) constructed the interpolating polynomial directly using basis polynomials that act as orthogonal switches (Figure 40.4).

40.11.1 The Lagrange Basis Construction

Define the j-th *Lagrange basis polynomial* as:

$$\ell_j(x) = \prod_{\substack{m=1 \\ m \neq j}}^{k} \frac{x - x_m}{x_j - x_m}.$$

Notice how the formula acts as an arithmetic switch:

- **At $x = x_j$:** Every factor in the numerator equals the corresponding denominator $(x_j - x_m)$, so:
$$\ell_j(x_j) = \prod_{m \neq j} 1 = 1.$$
- **At $x = x_m$ for any $m \neq j$:** The numerator contains the term $(x_m - x_m) = 0$, so:
$$\ell_j(x_m) = 0.$$

Since $\ell_j(x_m) = \delta_{jm}$ (the Kronecker delta), the unique interpolating polynomial is:

$$f(x) = \sum_{j=1}^{k} y_j \, \ell_j(x).$$

40.11.2 Application: Reconstructing Shamir's Secret at $x = 0$

In Shamir's (k, n)-threshold secret sharing scheme (Section 32.5), the shared secret is the constant term $S = f(0)$. Evaluating the Lagrange formula at $x = 0$ yields:

$$S = f(0) = \sum_{j=1}^{k} y_j \, \ell_j(0), \qquad \text{where } \ell_j(0) = \prod_{\substack{m=1 \\ m \neq j}}^{k} \frac{-x_m}{x_j - x_m} = \prod_{\substack{m=1 \\ m \neq j}}^{k} \frac{x_m}{x_m - x_j}.$$

Example 40.9 (Lagrange secret recovery in $\mathbb{Z}/11\mathbb{Z}$)**.** Suppose a $(2, 3)$-threshold scheme over $\mathbb{Z}/11\mathbb{Z}$ produces two shares: $(x_1, y_1) = (1, 8)$ and $(x_2, y_2) = (3, 5)$. The polynomial has degree ≤ 1. Compute the basis coefficients at 0:

$$\ell_1(0) = \frac{x_2}{x_2 - x_1} = \frac{3}{3 - 1} = \frac{3}{2} \equiv 3 \cdot 6 = 18 \equiv 7 \pmod{11}.$$

$$\ell_2(0) = \frac{x_1}{x_1 - x_2} = \frac{1}{1 - 3} = \frac{1}{-2} \equiv \frac{1}{9} \equiv 5 \pmod{11}.$$

(Notice $\ell_1(0) + \ell_2(0) = 7 + 5 = 12 \equiv 1 \pmod{11}$, as required.) Now recover the secret:

$$S = y_1\ell_1(0) + y_2\ell_2(0) = 8 \cdot 7 + 5 \cdot 5 = 56 + 25 = 81 \equiv 4 \pmod{11}.$$

Verification: The line passing through $(1, 8)$ and $(3, 5)$ in $\mathbb{Z}/11\mathbb{Z}$ has slope $m = (5 - 8)/(3 - 1) = -3/2 \equiv 8/2 = 4$. Then $f(x) = 4x + b$. With $f(1) = 4(1) + b = 8 \implies b = 4$. Thus $f(0) = 4$, exactly matching the Lagrange reconstruction!

40.12 Quick reference

Focus

Core identities.

$\gcd(a,b) = as + bt$	(Bézout; extended Euclid finds s,t)
$a^{-1} \bmod n$ exists $\iff \gcd(a,n) = 1$	($s \bmod n$ from above)
$a^{p-1} \equiv 1 \pmod p$	(Fermat, $p \nmid a$)
$a^{\varphi(n)} \equiv 1 \pmod n$	(Euler, $\gcd(a,n) = 1$)
$\varphi(n) = n\prod_{p\mid n}(1-\frac{1}{p})$	(totient from factorization)
$\mathrm{ord}_n(a) \mid \varphi(n)$	(orders divide the group size)
$\mathbb{Z}/N\mathbb{Z} \cong \prod_i \mathbb{Z}/n_i\mathbb{Z}$	(CRT, n_i pairwise coprime)
$\left(\frac{a}{p}\right) \equiv a^{(p-1)/2} \pmod p$	(Euler's criterion)

Task	**Tool**	**Cost**
$\gcd(a,b)$	Euclidean algorithm	$O(\log \min(a,b))$ steps
$a^{-1} \bmod n$	extended Euclid	$O(\log n)$ steps
$a^e \bmod n$	square-and-multiply	$O(\log e)$ multiplications
system of $x \bmod n_i$	CRT reconstruction	$O(k)$ inverses
"is n prime?"	Miller-Rabin (k rounds)	error $\le 4^{-k}$
"is a a square mod p?"	Euler's criterion	one exponentiation
$\sqrt{a} \bmod p$, $p \equiv 3\ (4)$	$a^{(p+1)/4}$	one exponentiation

40.13 Worked end-to-end: a complete RSA toy with these tools

To see every piece fire at once, run RSA with $p = 61,\ q = 53$ (the classic textbook numbers).

1. **Modulus and totient.** $n = 61 \cdot 53 = 3233$, and $\varphi(n) = 60 \cdot 52 = 3120$ (totient of a product, §40.5.1).
2. **Public exponent.** Choose $e = 17$; check $\gcd(17, 3120) = 1$ with Euclid.
3. **Private exponent.** $d = 17^{-1} \bmod 3120$ via extended Euclid (§40.3.1). Running it gives $d = 2753$; verify $17 \cdot 2753 = 46801 = 15 \cdot 3120 + 1 \equiv 1$.
4. **Encrypt** $m = 65$: $c = 65^{17} \bmod 3233$ by square-and-multiply $= 2790$.
5. **Decrypt** with CRT for speed (§40.6). Compute $d_p = d \bmod (p-1) = 2753 \bmod 60 = 53$ and $d_q = d \bmod (q-1) = 2753 \bmod 52 = 49$. Then

$$m_p = c^{d_p} \bmod 61 = 2790^{53} \bmod 61 = 4, \qquad m_q = c^{d_q} \bmod 53 = 2790^{49} \bmod 53 = 12.$$

CRT-glue $x \equiv 4\ (61),\ x \equiv 12\ (53)$: with $61^{-1} \bmod 53 = 20$ and $53^{-1} \bmod 61 = 38$, reconstruct $m = 4 \cdot 53 \cdot 38 + 12 \cdot 61 \cdot 20 = 8056 + 14640 = 22696 \equiv 65 \pmod{3233}$. The Euler-theorem identity $m^{ed} \equiv m$ guarantees this.

Every line used a tool from this appendix: factorization-based totient, extended Euclid for the inverse, fast exponentiation, CRT for the speedup (Figure 40.5). That is the whole appendix in miniature, and it is exactly what your TLS library does — just with 2048-bit primes instead of two-digit ones.

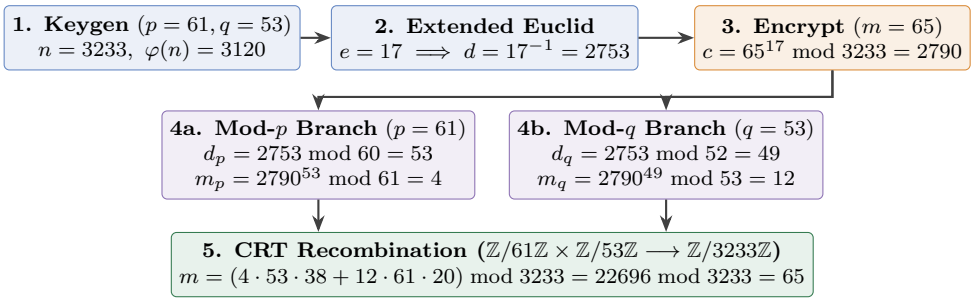


Figure 40.5: End-to-end RSA key generation, encryption, and Chinese Remainder Theorem (CRT) fast decryption for $p = 61, q = 53$. Instead of computing $2790^{2753} \bmod 3233$ directly, CRT decryption projects the ciphertext into $\mathbb{Z}/61\mathbb{Z}$ and $\mathbb{Z}/53\mathbb{Z}$ with reduced exponents $d_p = 53$ and $d_q = 49$, then glues the two residues $(m_p, m_q) = (4, 12)$ back into $\mathbb{Z}/3233\mathbb{Z}$ to recover $m = 65$.

With integer arithmetic, prime testing, and modular rings in hand, the next appendix turns to abstract groups, rings, finite fields $\mathbb{F}_{2^k}$, and the probability tools used across security proofs.

40.14 Exercises

Exercise 40.1. Use the Euclidean algorithm to compute $\gcd(2024, 748)$, showing each remainder. Then run extended Euclid to find integers s, t with $2024s + 748t = \gcd$.

Exercise 40.2. Compute $11^{-1} \bmod 26$ two ways: (a) by extended Euclid, and (b) by Euler's theorem, using $11^{\varphi(26)-1} \bmod 26$. Confirm both give the same residue, and state $\varphi(26)$.

Exercise 40.3. Compute $7^{222} \bmod 11$. First reduce the exponent using Fermat's little theorem, then finish by hand. (Hint: what is $7^{10} \bmod 11$?)

Exercise 40.4. Solve the system $x \equiv 1 \pmod 3$, $x \equiv 2 \pmod 4$, $x \equiv 3 \pmod 5$ for the unique $x \in \{0, \ldots, 59\}$. Verify your answer against all three congruences.

Exercise 40.5. List the order of every element of $(\mathbb{Z}/11\mathbb{Z})^*$. How many primitive roots does 11 have, and does this match the $\varphi(\varphi(11))$ prediction? Exhibit one primitive root.

Exercise 40.6. Verify that $561 = 3 \cdot 11 \cdot 17$ is a Carmichael number by checking the Korselt criterion: n is squarefree and $(p - 1) \mid (n - 1)$ for each prime factor p. Then find a single Miller-Rabin witness for 561 other than the base 2 used in Example 40.7.

Exercise 40.7. Determine all quadratic residues modulo 13 by squaring $1, \ldots, 12$. Then use Euler's criterion to confirm whether 5 and 6 are residues, and for each residue among them give a square root.

Exercise 40.8. (Break it.) An RSA signer uses CRT and a fault corrupts the mod-p branch, producing a signature $\hat{s}$ with $\hat{s}^e \equiv m \pmod q$ but $\hat{s}^e \not\equiv m \pmod p$. Explain why $\gcd(\hat{s}^e - m,\ n)$ recovers the factor q, and why verifying the signature before output prevents the leak. Reference Theorem 40.5 in your reasoning.

Exercise 40.9. (Prove it.) Show that if $\gcd(m,n)=1$ then $\varphi(mn)=\varphi(m)\varphi(n)$, using the CRT isomorphism $\mathbb{Z}/mn\mathbb{Z}\cong\mathbb{Z}/m\mathbb{Z}\times\mathbb{Z}/n\mathbb{Z}$ and the fact that an element is a unit iff it is a unit in each coordinate.

Exercise 40.10. Let $p=23$, so $p-1=22=2\cdot 11$. Use the generator test of §40.7 (check $g^{(p-1)/2}$ and $g^{(p-1)/11}$) to decide whether $g=5$ is a primitive root mod 23, without listing all 22 powers.

Chapter 41

Appendix: Algebra and Probability Reference

Following the number-theoretic tools of the previous appendix, this appendix is a compact, self-contained refresher for the two broader mathematical languages the rest of the book speaks: the *algebra* of groups, rings, and fields (the stage on which RSA, Diffie–Hellman, and elliptic curves live), and the *probability* of uniform sampling, entropy, and collisions (the stage on which keys, nonces, and hash security live). Nothing here is new if you read the core chapters; it is gathered in one place so you can look it up without hunting.

The treatment is deliberately terse but worked. Every abstract definition is followed by a tiny concrete object you can hold in your head, and every theorem you actually use in cryptography is stated precisely with a one-line reason it is true. Where a proof would take a page, we point to the companion volume and give the intuition instead.

Reading note

If you want the full development with all proofs, see the companion volumes: *Probability From the Ground Up* for distributions, entropy, and the birthday/collision machinery, and *Algorithms From the Ground Up* for the cost of modular exponentiation, GCD, and polynomial multiplication. This appendix gives you the statements; those books give you the derivations. Cross-references appear in Connection notes throughout.

History

Modern cryptography was born from the collision of two intellectual revolutions: abstract algebra and mathematical probability.
In 1832, on the eve of his fatal duel at age twenty, Évariste Galois scribbled the foundations of group and field theory to solve a 300-year-old puzzle: why polynomial equations of degree five or higher possess no general solution by radicals, unlike the quadratic, Cardano's cubic, and Ferrari's quartic formulas. Galois realized that whether an equation can be solved is governed not by algebraic calculation, but by the symmetry group of permutations of its roots. His manuscript was initially dismissed as incomprehensible and lay dormant until Joseph Liouville published it in 1846. Meanwhile, Leonhard Euler and Carl Friedrich Gauss turned modular

arithmetic from isolated calculation tricks into the systematic theory of congruence classes. When modern cryptographers needed finite mathematical worlds where one-way trapdoors could live, they turned directly to Galois's finite fields $\mathbb{F}_{p^m}$ to power the byte substitutions of AES and the coordinate grids of elliptic curves, and to Gauss's cyclic groups $(\mathbb{Z}/p\mathbb{Z})^*$ to build the Diffie–Hellman key exchange.

Probability followed an equally formative trajectory. In 1654, Blaise Pascal and Pierre de Fermat resolved the Chevalier de Méré's puzzle of dividing stakes in an interrupted game of dice, inventing mathematical expectation. In the 1760s and 1770s, Thomas Bayes and Pierre-Simon Laplace inverted this logic, creating conditional probability to infer unseen causes from observed evidence. In 1948, Claude Shannon brought probability into communications engineering, formalizing entropy and proving that perfect secrecy requires key entropy at least equal to message entropy.

Yet applying probability to security brought its own counter-intuitive surprises. Shannon's average entropy can look comfortably high even when a single dominant password occurs half the time. Because real adversaries attack the single most probable choice rather than drawing average samples, cryptographers had to introduce min-entropy $H_\infty(X) = -\log_2 \max p_x$ to quantify worst-case predictability. Together, these algebraic and probabilistic tools provide the shared vocabulary of this book, and they prepare us for the high-dimensional lattice geometries and quantum-resistant protocols of modern cryptography.

41.1 The big picture: why cryptographers care about algebra

Almost every public-key scheme has the same shape. You have a set with an operation. Composing elements is cheap; *undoing* the composition is (we hope) hard. Encryption walks forward along the easy direction; security rests on the adversary being stuck trying to walk backward.

- In RSA the set is $(\mathbb{Z}/N\mathbb{Z})^*$ under multiplication; "forward" is raising to the public exponent e, "backward" is taking the e-th root, which is tied to factoring N.
- In Diffie–Hellman the set is a cyclic group $\mathbb{G}$; "forward" is exponentiation $g \mapsto g^x$, "backward" is the discrete logarithm.
- In elliptic-curve crypto the set is the points on a curve under a geometric addition law; "forward" is scalar multiplication $P \mapsto kP$, "backward" is again a discrete log.

To even *state* these schemes you need the vocabulary of groups (for the multiplicative and additive structures), rings (for arithmetic mod N, where you add *and* multiply but cannot always divide), and fields (for AES's byte arithmetic and for the coordinate fields of elliptic curves). That is the tour below.

Intuition

A useful mental hierarchy: a **group** gives you one good operation (you can combine and undo). A **ring** gives you two operations (add and multiply) but division may fail. A **field** is a ring where division by anything nonzero works — it behaves like ordinary fractions. Each step up adds power and removes freedom: there are many groups, fewer rings worth naming, and the finite fields are pinned down completely (one for each prime power). Figure 41.1 shows

the containment.

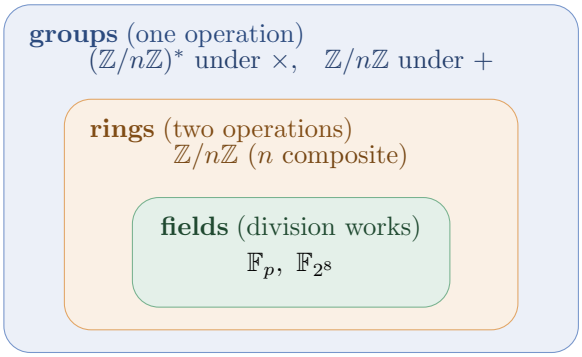


Figure 41.1: The algebraic hierarchy. Each inner box has more structure: a field is a special ring, and a ring's additive (or a field's multiplicative) part is a group. Cryptography lives at every level.

41.2 Groups

Definition 41.1 (Group)**.** A *group* is a set G with a binary operation $\cdot$ satisfying:

1. **Closure:** $a \cdot b \in G$ for all $a, b \in G$.
2. **Associativity:** $(a \cdot b) \cdot c = a \cdot (b \cdot c)$.
3. **Identity:** there is $e \in G$ with $e \cdot a = a \cdot e = a$.
4. **Inverses:** every a has some a^{-1} with $a \cdot a^{-1} = e$.

If additionally $a \cdot b = b \cdot a$ for all a, b, the group is *abelian* (commutative). Nearly every group in cryptography is abelian.

Two notations coexist. In *multiplicative* notation the operation is $\cdot$, the identity is 1, and repeated operation is a power a^k. In *additive* notation the operation is $+$, the identity is 0, and repeated operation is a multiple $k \cdot a$. They describe the same idea; elliptic curves use additive notation, $(\mathbb{Z}/N\mathbb{Z})^*$ uses multiplicative.

Example 41.1 (Canonical cryptographic groups)**.**

- $\mathbb{Z}/n\mathbb{Z}$ under addition: $\{0, 1, \ldots, n-1\}$ with addition mod n. Identity 0; inverse of a is $n - a$. Always a group, for every n.
- $(\mathbb{Z}/n\mathbb{Z})^*$ under multiplication: the residues mod n that are *coprime* to n, with multiplication mod n. Identity 1. The coprimality requirement is exactly what guarantees inverses exist (see §41.3). Its size is $\varphi(n)$, Euler's totient.

Take $n = 10$. Then $(\mathbb{Z}/10\mathbb{Z})^* = \{1, 3, 7, 9\}$, since those are the residues sharing no factor with 10. Check that it is closed: $3 \cdot 7 = 21 \equiv 1$, $3 \cdot 3 = 9$, $7 \cdot 9 = 63 \equiv 3$, all land back in the set. Here $\varphi(10) = 4$.

41.2.1 Order, generators, and cyclic groups

Definition 41.2 (Order of an element)**.** The *order* of $a \in G$, written $\operatorname{ord}(a)$, is the smallest positive integer k with $a^k = e$ (or $k \cdot a = 0$ additively). The *order of the group* $|G|$ is its number of elements.

Example 41.2 (Orders in $(\mathbb{Z}/10\mathbb{Z})^*$)**.** Powers of 3 mod 10: $3^1 = 3$, $3^2 = 9$, $3^3 = 27 \equiv 7$, $3^4 = 81 \equiv 1$. So $\operatorname{ord}(3) = 4$. Because $4 = |(\mathbb{Z}/10\mathbb{Z})^*|$, the powers of 3 sweep out the whole group: 3 is a *generator*.

Definition 41.3 (Cyclic group, generator)**.** G is *cyclic* if some $g \in G$ has $\operatorname{ord}(g) = |G|$; then every element is a power of g, written $G = \langle g \rangle$, and g is a *generator* (or *primitive root*).

Focus

Cyclic groups are the algebraic setting for discrete-log cryptography. A generator g gives a bijection between exponents $\{0, 1, \ldots, |G| - 1\}$ and group elements: $x \mapsto g^x$. Diffie–Hellman is exactly the statement that this map is easy forward and (in the right group) hard to invert.

Theorem 41.1 (Lagrange's theorem)**.** *In a finite group G, the order of every element divides $|G|$. Equivalently $a^{|G|} = e$ for all $a \in G$.*

The reason is a counting trick: an element of order k generates a subgroup of exactly k elements, and a finite group splits into equal-sized translated copies (*cosets*) of any subgroup, so k must divide $|G|$. Two cryptographic payoffs fall straight out.

Corollary 41.1 (Euler and Fermat)**.** *For $\gcd(a, n) = 1$, $a^{\varphi(n)} \equiv 1 \pmod{n}$ (Euler). When $n = p$ is prime this is $a^{p-1} \equiv 1 \pmod{p}$ (Fermat's little theorem).*

These two congruences are why RSA decryption works and why so many primality and inverse computations terminate where they do.

Connection

The cost of computing g^x by repeated squaring — $O(\log x)$ multiplications — is derived in *Algorithms From the Ground Up* (fast exponentiation). The number-theoretic facts that φ is multiplicative and that $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic are proved in the companion *Appendix: Number Theory Reference*.

41.2.2 Subgroups and the structure of $(\mathbb{Z}/p\mathbb{Z})^*$

A *subgroup* is a subset that is itself a group under the same operation. By Lagrange its size divides $|G|$. For a prime p, the group $(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic of order $p - 1$, and its subgroups are exactly one for each divisor of $p - 1$. Cryptographers exploit this: to get a group of large *prime* order q (which kills small-subgroup attacks), pick a "safe prime" $p = 2q + 1$ and work in the order-q subgroup of $(\mathbb{Z}/p\mathbb{Z})^*$.

Common trap

Working in $(\mathbb{Z}/p\mathbb{Z})^*$ *as a whole* is dangerous when $p - 1$ has small prime factors: the Pohlig–Hellman attack solves the discrete-log piece by piece across those small subgroups and re-

combines via the Chinese Remainder Theorem, turning a "hard" log into many easy ones. Restricting to a prime-order subgroup removes the small factors. This is also why an attacker who can confine you to a tiny subgroup (a *small-subgroup confinement* attack) can leak bits of your secret — always validate that a received element has the expected large order.

41.3 Rings: where you can add and multiply

Definition 41.4 (Commutative ring with unity)**.** A *ring* R is a set with two operations, $+$ and $\cdot$, such that $(R, +)$ is an abelian group with identity 0, multiplication is associative and has an identity 1, and the *distributive law* $a(b + c) = ab + ac$ holds. If multiplication also commutes, the ring is *commutative*. All rings in this book are commutative with unity.

The prototype is $\mathbb{Z}/n\mathbb{Z} = \{0, 1, \ldots, n-1\}$ with addition and multiplication mod n. You can always add, subtract, and multiply. What you cannot always do is *divide*.

Definition 41.5 (Unit, zero divisor)**.** An element a is a *unit* if it has a multiplicative inverse a^{-1}. It is a *zero divisor* if $a \neq 0$ yet $ab = 0$ for some $b \neq 0$.

In $\mathbb{Z}/n\mathbb{Z}$ the element a is a unit exactly when $\gcd(a, n) = 1$, and the units form the group $(\mathbb{Z}/n\mathbb{Z})^*$ of §41.2. The non-units (other than 0) are zero divisors.

Example 41.3 (Why 4 has no inverse mod 10)**.** $\gcd(4, 10) = 2 \neq 1$, so 4 is not a unit. Indeed $4 \cdot 5 = 20 \equiv 0 \pmod{10}$, so 4 and 5 are zero divisors: multiplying by 4 collapses distinct inputs to the same output, and a collapsing map cannot be undone. Contrast $\gcd(3, 10) = 1$: the extended Euclidean algorithm gives $3 \cdot 7 = 21 \equiv 1$, so $3^{-1} \equiv 7$.

Why it works

Why coprimality is exactly the condition for invertibility. If $\gcd(a, n) = 1$, Bezout's identity gives integers u, v with $au + nv = 1$; reducing mod n leaves $au \equiv 1$, so $u = a^{-1}$. Conversely, if $\gcd(a, n) = d > 1$, then a times anything is a multiple of d, so it can never equal 1 mod n unless $d \mid 1$. The extended Euclidean algorithm *computes* the u, v, which is how implementations find modular inverses (and RSA private exponents) in $O(\log n)$ steps.

Connection

The extended Euclidean algorithm and its running time are in *Algorithms From the Ground Up*. The fact that $\mathbb{Z}/n\mathbb{Z}$ splits as a product $\mathbb{Z}/p_1^{e_1}\mathbb{Z} \times \cdots$ via the Chinese Remainder Theorem — the engine behind fast RSA decryption — is in the number-theory appendix.

41.4 Fields

Definition 41.6 (Field)**.** A *field* is a commutative ring in which every nonzero element is a unit. So you can add, subtract, multiply, and divide by anything nonzero — arithmetic behaves like the rationals or reals.

The single most useful fact:

Theorem 41.2 ($\mathbb{Z}/p\mathbb{Z}$ is a field iff p is prime)**.** *$\mathbb{Z}/n\mathbb{Z}$ is a field if and only if n is prime. When p is prime we write the field as $\mathbb{F}_p$.*

If p is prime, every nonzero residue is coprime to p, hence a unit (by §41.3); so every nonzero element is invertible and $\mathbb{Z}/p\mathbb{Z}$ is a field. If n is composite, $n = ab$ with $1 < a, b < n$ gives zero divisors a, b, and a zero divisor can never be a unit — so $\mathbb{Z}/n\mathbb{Z}$ is not a field.

Example 41.4 ($\mathbb{F}_5$ by hand)**.** In $\mathbb{F}_5 = \{0, 1, 2, 3, 4\}$ every nonzero element has an inverse: $1^{-1} = 1$, $2^{-1} = 3$ (since $2 \cdot 3 = 6 \equiv 1$), $3^{-1} = 2$, $4^{-1} = 4$ (since $4 \cdot 4 = 16 \equiv 1$). So $3/4 = 3 \cdot 4^{-1} = 3 \cdot 4 = 12 \equiv 2$. Division just works.

41.4.1 Finite fields of prime-power order

Prime fields $\mathbb{F}_p$ are not the only finite fields. There is exactly one finite field of size q for every prime power $q = p^m$ (and none of any other size), all called $\mathbb{F}_q$ or $\mathrm{GF}(q)$. The case cryptographers meet constantly is $\mathbb{F}_{2^m}$ — AES lives in $\mathbb{F}_{2^8}$ and many MACs and codes live in $\mathbb{F}_{2^{128}}$.

You build $\mathbb{F}_{p^m}$ not from integers mod p^m (that ring has zero divisors and is *not* a field) but from *polynomials* over $\mathbb{F}_p$ reduced modulo an *irreducible* polynomial of degree m — the polynomial analogue of taking integers mod a prime.

Definition 41.7 (Polynomial ring and irreducibility)**.** $\mathbb{F}_p[x]$ is the ring of polynomials with coefficients in $\mathbb{F}_p$, added and multiplied as usual but with coefficient arithmetic done mod p. A nonconstant $f(x)$ is *irreducible* over $\mathbb{F}_p$ if it cannot be written as a product of two lower-degree polynomials in $\mathbb{F}_p[x]$ — the polynomial version of "prime."

Theorem 41.3 (Construction of $\mathbb{F}_{p^m}$)**.** *If $f(x) \in \mathbb{F}_p[x]$ is irreducible of degree m, then $\mathbb{F}_p[x]/(f(x))$ — the polynomials of degree $< m$, with multiplication reduced mod f — is a field of p^m elements. Different choices of f give isomorphic ("relabeled-identical") fields.*

The reason it is a field exactly parallels the integer case: irreducibility of f plays the role of primality of p, so the only polynomial sharing a factor with f is a multiple of f (which is 0 in the quotient), and the extended Euclidean algorithm *for polynomials* produces inverses.

41.4.2 Worked example: arithmetic in $\mathbb{F}_{2^3}$

Take $p = 2$, $m = 3$, and the irreducible polynomial $f(x) = x^3 + x + 1$. Elements are the 8 polynomials of degree < 3 over $\mathbb{F}_2$; written as bit-triples they are $000, 001, \ldots, 111$. Coefficient arithmetic is mod 2, so *addition is XOR.*

Addition. $(x^2 + 1) + (x + 1) = x^2 + x =$ in bits $101 \oplus 011 = 110$. No carries, no reduction — adding in $\mathbb{F}_{2^m}$ is just XOR of the bit-strings.

Multiplication. Compute $(x^2 + 1) \cdot (x + 1)$:

$$(x^2 + 1)(x + 1) = x^3 + x^2 + x + 1.$$

This has degree 3, so reduce mod f. Since $x^3 + x + 1 \equiv 0$, we have $x^3 \equiv x + 1$. Substitute:

$$x^3 + x^2 + x + 1 \equiv (x + 1) + x^2 + x + 1 = x^2 + 2x + 2 \equiv x^2 \pmod 2,$$

because $2x$ and 2 vanish mod 2. So the product is x^2, i.e. bits 100.

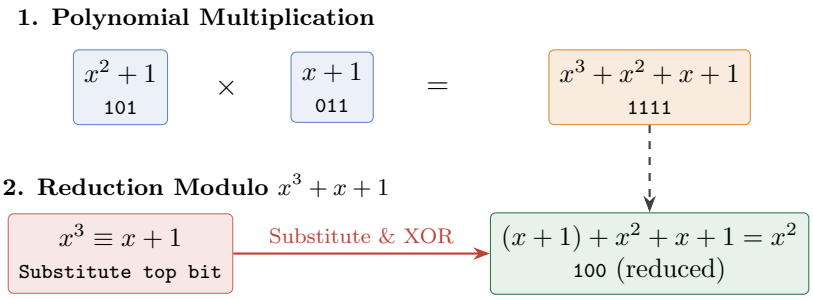


Figure 41.2: Polynomial multiplication in the finite field $\mathbb{F}_{2^3}$. The product of `101` and `011` yields a degree-3 polynomial `1111`. Because the irreducible modulus is x^3+x+1, the x^3 term wraps around and is replaced by $x+1$, causing the lower terms to cancel out under modulo-2 addition (XOR), leaving x^2 (`100`).

Intuition

The reduction step "$x^3 \equiv x+1$" is the polynomial twin of "$10 \equiv 0 \bmod 10$." Whenever a product overflows degree m, you fold the top term back in using the relation the irreducible polynomial gives you. In $\mathbb{F}_{2^8}$ (AES) the relation is $x^8 \equiv x^4+x^3+x+1$, which is the famous reduction by the byte `0x11B`.

Implement it

Multiplication in $\mathbb{F}_{2^8}$ with AES's reduction polynomial `0x11B`, the "Russian peasant" way: shift-and-XOR, folding in the modulus whenever bit 8 appears.

```
def gf_mul(a, b):
    # Multiply two bytes in GF(2^8) with AES modulus x^8+x^4+x^3+x+1 (0x11B).
    # LEARNING ONLY: not constant-time -- the data-dependent branch below
    # leaks the operands through timing. Never use for real crypto.
    p = 0
    for _ in range(8):
        if b & 1:            # add current shifted a (XOR = add in GF(2))
            p ^= a
        b >>= 1
        carry = a & 0x80     # will a overflow past bit 7?
        a <<= 1
        if carry:            # fold in the modulus: subtract x^8+x^4+x^3+x+1
            a ^= 0x11B
    return p & 0xFF

assert gf_mul(0x53, 0xCA) == 0x01   # 0x53 and 0xCA are inverses in AES's field
```

Warning: this is toy code to show the algebra. A production implementation uses constant-time table lookups or carry-less multiplication (`PCLMULQDQ`) to avoid the timing side channel flagged above.

In the real world

Where finite fields show up. AES does every `MixColumns` step as matrix multiplication over $\mathbb{F}_{2^8}$. AES-GCM's authentication tag is a polynomial evaluated over $\mathbb{F}_{2^{128}}$ (a universal hash). Reed–Solomon codes (CDs, QR codes, deep-space links) and Shamir secret sharing both interpolate polynomials over a finite field. Elliptic curves are defined by an equation whose coordinates live in $\mathbb{F}_p$ or $\mathbb{F}_{2^m}$. If you understand $\mathbb{F}_{2^3}$ above, you understand the algebra under all of these — only the sizes change.

41.5 Vector Spaces, Lattices, and Determinants

Modern post-quantum cryptography (Section 35.4) departs from the one-dimensional multiplicative groups of RSA and Diffie–Hellman. Instead, it builds security upon geometric objects in high-dimensional spaces: *vector spaces* and *lattices*.

41.5.1 Vector Spaces

A *vector space* V over a field $\mathbb{F}$ is an abelian group under addition equipped with scalar multiplication by elements of $\mathbb{F}$ satisfying associativity, distributivity, and an identity scalar $1 \cdot \mathbf{v} = \mathbf{v}$.

A set of vectors $\{\mathbf{b}_1, \ldots, \mathbf{b}_k\}$ is *linearly independent* if $\sum_{i=1}^{k} c_i \mathbf{b}_i = \mathbf{0}$ implies $c_1 = \cdots = c_k = 0$. A maximal linearly independent set forms a *basis*, and the number of basis vectors is the *dimension* n. In Euclidean space $\mathbb{R}^n$, scalars are real numbers; in coding theory and lattice cryptography, scalars frequently belong to $\mathbb{Z}/q\mathbb{Z}$ or $\mathbb{F}_2$.

41.5.2 Lattices and the Fundamental Domain

A *lattice* $\mathcal{L} \subset \mathbb{R}^n$ is a discrete additive subgroup of $\mathbb{R}^n$. Given n linearly independent basis vectors $\mathbf{B} = (\mathbf{b}_1, \ldots, \mathbf{b}_n)$, the lattice generated by $\mathbf{B}$ consists of all *integer* linear combinations:

$$\mathcal{L}(\mathbf{B}) = \left\{ \sum_{i=1}^{n} z_i \mathbf{b}_i \;\middle|\; z_i \in \mathbb{Z} \right\}.$$

Unlike a vector space, which is a continuous continuum, a lattice is a periodic grid of isolated points separated by empty space.

Definition 41.8 (Fundamental Parallelotope and Determinant)**.** The *fundamental parallelotope* (or fundamental domain) of a basis $\mathbf{B}$ is:

$$\mathcal{P}(\mathbf{B}) = \left\{ \sum_{i=1}^{n} t_i \mathbf{b}_i \;\middle|\; 0 \le t_i < 1 \right\}.$$

The *lattice determinant* $\det(\mathcal{L})$ is the n-dimensional volume of $\mathcal{P}(\mathbf{B})$:

$$\det(\mathcal{L}) = \mathrm{vol}(\mathcal{P}(\mathbf{B})) = |\det(\mathbf{B})|.$$

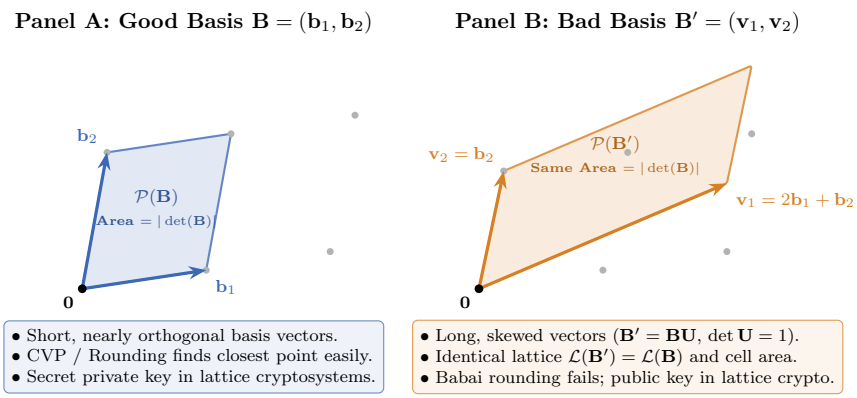


Figure 41.3: Lattice geometry, fundamental parallelotope, and determinant volume. A lattice $\mathcal{L} \subset \mathbb{R}^n$ is the discrete set of all integer linear combinations of basis vectors. In Panel A, a "good" basis consists of short, nearly orthogonal vectors; the fundamental cell $\mathcal{P}(\mathbf{B})$ is compact, making the Closest Vector Problem (CVP) easy to solve. In Panel B, an integer unimodular transformation ($\mathbf{B}' = \mathbf{BU}$ with $\det(\mathbf{U}) = \pm 1$) produces an equivalent "bad" basis spanning the identical grid of points with the identical determinant volume $\text{vol}(\mathcal{P}) = |\det(\mathbf{B})|$, but whose skewed geometry hides the nearest lattice point from efficient rounding algorithms.

The volume of $\mathcal{P}(\mathbf{B})$ is an intrinsic geometric invariant of the lattice itself. Two bases $\mathbf{B}$ and $\mathbf{B}'$ generate the same lattice if and only if $\mathbf{B}' = \mathbf{BU}$ for some unimodular integer matrix $\mathbf{U}$ ($\det(\mathbf{U}) = \pm 1$). Because $|\det(\mathbf{B}')| = |\det(\mathbf{B})| \cdot |\det(\mathbf{U})| = |\det(\mathbf{B})|$, every basis of $\mathcal{L}$ encloses the exact same volume (Figure 41.3).

Focus

The geometric trapdoor in lattice crypto. While all bases generate the same grid with the same cell volume, their shapes differ wildly:

- A **good basis** (Panel A) consists of short, nearly mutually orthogonal vectors. Given a point $\mathbf{t} = \mathbf{v} + \mathbf{e}$ perturbed by a small error $\mathbf{e}$, the closest lattice vector $\mathbf{v}$ can be efficiently recovered by simple rounding (Babai's algorithm). This good basis serves as the *private key*.
- A **bad basis** (Panel B) consists of long, highly skewed vectors. Simple rounding on a bad basis produces coordinates whose errors explode exponentially with dimension n. This bad basis serves as the *public key*.

Because Shor's quantum algorithm requires an underlying periodic abelian group structure, the geometric hardness of finding short vectors in high-dimensional lattices ($n \geq 512$) remains unbroken by quantum computers.

41.6 Bitwise Arithmetic and Two's Complement

At the silicon hardware layer, ciphers do not execute abstract algebra; they manipulate fixed-width binary registers of size $w \in \{8, 16, 32, 64\}$ bits. Understanding the algebra of two's complement and bitwise logic is essential for constant-time cryptographic implementation (Section 39.7).

41.6.1 The Two's Complement Ring $\mathbb{Z}/2^w\mathbb{Z}$

In a register of width w, arithmetic addition and multiplication occur modulo 2^w. To represent signed integers within the range $[-2^{w-1}, 2^{w-1}-1]$, two's complement defines negative numbers by their additive inverses in $\mathbb{Z}/2^w\mathbb{Z}$.

Theorem 41.4 (Two's Complement Negation)**.** *For any integer $x \in [0, 2^{w-1}-1]$, the negative $-x$ in a w-bit register satisfies:*

$$-x \equiv 2^w - x = (2^w - 1 - x) + 1 = \mathrm{NOT}(x) + 1 \pmod{2^w},$$

where $\mathrm{NOT}(x)$ *denotes bitwise bit-inversion (the one's complement).*

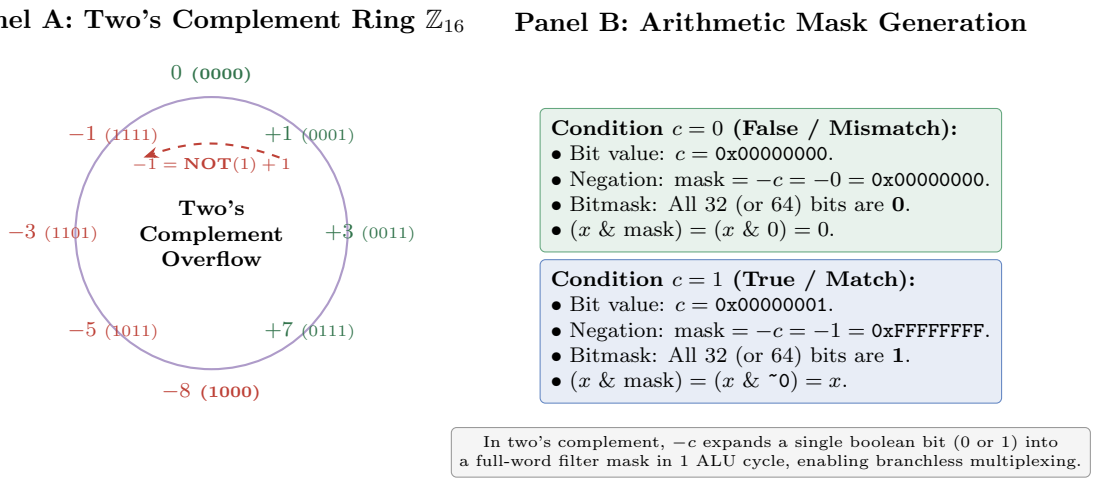


Figure 41.4: Two's complement integer arithmetic and arithmetic mask generation. In Panel A, w-bit words form a modular ring $\mathbb{Z}_{2^w}$. Negation $-x$ is defined algebraically by $\mathrm{NOT}(x) + 1 \pmod{2^w}$. In Panel B, this property turns arithmetic negation into a constant-time mask generator: negating a condition bit $c \in \{0, 1\}$ produces an all-zero word (for $c = 0$) or an all-one word (for $c = 1$), forming the foundation of branchless multiplexers ($r = b \oplus ((a \oplus b)\ \&\ -c)$).

41.6.2 Constant-Time Arithmetic Masking

The two's complement property gives a clean mathematical bridge between boolean logic and bitwise selection:

- If $c = 0$, then $-c = -0 =$ `0x00...00` (all bits 0).
- If $c = 1$, then $-c = -1 =$ `0xFF...FF` (all bits 1).

This single negation operation expands a 1-bit predicate into a full-width bitmask without emitting a conditional jump or branching instruction (Figure 41.4). Combined with bitwise XOR (vector addition over $\mathbb{F}_2^w$) and bitwise AND (projection), it allows conditional choices to be evaluated in identical clock cycle counts on every CPU architecture.

41.7 A pocket card of algebraic facts

Fact	Where it is used
$a^{-1} \bmod n$ exists iff $\gcd(a, n) = 1$	RSA key gen, modular division
$a^{\varphi(n)} \equiv 1 \pmod{n}$ (Euler)	RSA correctness
$a^{p-1} \equiv 1 \pmod{p}$ (Fermat)	primality tests, $a^{-1} = a^{p-2}$
$(\mathbb{Z}/p\mathbb{Z})^*$ is cyclic of order $p-1$	Diffie–Hellman, ElGamal
$\mathrm{ord}(g) \mid \lvert G\rvert$ (Lagrange)	subgroup choice, Pohlig–Hellman
$\mathbb{Z}/n\mathbb{Z}$ is a field iff n prime	choosing curve/field parameters
$\mathbb{F}_{p^m} = \mathbb{F}_p[x]/(f)$, f irreducible	AES, GCM, Reed–Solomon, sharing
Add in $\mathbb{F}_{2^m}$ is bitwise XOR	every byte-level cipher operation
$\det(\mathcal{L}) = \lvert\det(\mathbf{B})\rvert$	lattice volume, basis reduction
$-c \in \{\texttt{0x00}, \texttt{0xFF}\}$ for $c \in \{0,1\}$	constant-time branchless multiplexers

41.8 Probability: the cryptographer's slice

Cryptography uses a focused but foundational domain of probability: *uniform* sampling (good keys, good nonces), *entropy* (how much real unpredictability you have), and *collision* bounds (when do random draws repeat). We collect those here.

41.8.1 Probability spaces and uniform distributions

A finite *probability space* is a finite set Ω of outcomes with weights $\Pr[\omega] \geq 0$ summing to 1. An *event* is a subset $A \subseteq \Omega$ with $\Pr[A] = \sum_{\omega \in A} \Pr[\omega]$. A *random variable* is a function $X : \Omega \to \mathbb{R}$ (or into any set).

The distribution cryptography cares about most is the *uniform* one: $\Pr[\omega] = 1/|\Omega|$ for every outcome. Writing $X \xleftarrow{\$} S$ means X is drawn uniformly from the set S. A 128-bit key chosen uniformly from $\{0,1\}^{128}$ is the canonical example: every one of the 2^{128} strings is equally likely, and *that* is what makes guessing it hopeless.

Definition 41.9 (Statistical (total variation) distance)**.** For distributions P, Q over the same finite set S,

$$\Delta(P, Q) \;=\; \tfrac{1}{2} \sum_{s \in S} \lvert\, P(s) - Q(s) \,\rvert \;=\; \max_{A \subseteq S} \lvert\, P(A) - Q(A) \,\rvert.$$

It is 0 when the distributions are identical and 1 when their supports are disjoint.

Statistical distance is the yardstick for "looks uniform." A PRG output is good if its distribution is statistically (or, weaker but enough, computationally) close to uniform; an attacker who cannot tell the two apart cannot exploit the difference. The triangle inequality $\Delta(P, R) \leq \Delta(P, Q) + \Delta(Q, R)$ lets you chain these closeness arguments through a proof — the "hybrid argument" you meet in security reductions.

Example 41.5 (A biased coin is detectable)**.** Let P be a fair bit ($\Pr[0] = \Pr[1] = \frac{1}{2}$) and Q a coin with $\Pr[1] = 0.51$. Then $\Delta(P, Q) = \frac{1}{2}(|0.5 - 0.49| + |0.5 - 0.51|) = 0.01$. Small — but over many samples a distinguisher that always guesses "Q when it sees more 1s" gains an edge that grows, which is exactly how statistical tests catch a broken RNG.

41.8.2 Expectation, variance, and the union bound

For a real random variable X, the *expectation* is $\mathbb{E}[X] = \sum_x x \cdot \Pr[X = x]$, and it is *linear*: $\mathbb{E}[X+Y] = \mathbb{E}[X]+\mathbb{E}[Y]$ *even when* X, Y *are dependent.* Linearity is the most useful tool in elementary probability — the birthday count below uses it directly.

Two inequalities provide essential bounds in security proofs.

Theorem 41.5 (Union bound)**.** *For any events* $A_1, \ldots, A_k$, $\Pr[A_1 \cup \cdots \cup A_k] \le \sum_i \Pr[A_i]$.

Theorem 41.6 (Markov's inequality)**.** *For a nonnegative* X *and* $t > 0$, $\Pr[X \ge t] \le \mathbb{E}[X]/t$.

The union bound is how we say "if no single bad event is likely, then *some* bad event happening is still unlikely" — e.g. bounding the chance that any of q encryption queries reuses a nonce. Markov turns an expectation into a tail guarantee when that is all you have.

41.8.3 Independence and conditional probability

Events A, B are *independent* if $\Pr[A \cap B] = \Pr[A]\Pr[B]$. The *conditional* probability is $\Pr[A \mid B] = \Pr[A \cap B]/\Pr[B]$, and Bayes's rule rearranges it:

$$\Pr[A \mid B] = \frac{\Pr[B \mid A]\ \Pr[A]}{\Pr[B]}.$$

Independence is what lets a one-time-pad ciphertext reveal nothing: if the key bit is uniform and independent of the message bit, the XOR is uniform regardless of the message, so Pr[message | ciphertext] = Pr[message] — perfect secrecy is exactly a statement about conditional probability.

Connection

Full constructions of probability spaces, the proofs that expectation is linear and that Markov/Chebyshev/Chernoff hold, and the law of large numbers (why statistical tests on RNGs converge) are developed in *Probability From the Ground Up*. Here we only need the statements.

41.8.4 Entropy: measuring unpredictability

A key is only as strong as it is unpredictable, and "unpredictable" has a precise measure. Let X be a random variable taking value x with probability p_x.

Definition 41.10 (Shannon and min-entropy)**.** The *Shannon entropy* (in bits) is

$$H(X) = -\sum_x p_x \log_2 p_x,$$

the average surprise. The *min-entropy* is

$$H_\infty(X) = -\log_2\Big(\max_x p_x\Big),$$

governed by the single most likely value.

Common trap

For *key material the quantity that matters is min-entropy, not Shannon entropy.* Shannon entropy is an average; an attacker does not attack the average, they attack the easiest case. If one password is chosen 50% of the time and a million others split the rest, the Shannon entropy looks healthy but $H_\infty = -\log_2(0.5) = 1$ bit: a single guess succeeds half the time. Always size key-derivation and password requirements against H_∞.

Example 41.6 (Entropy of a uniform key)**.** For X uniform on $\{0,1\}^n$, every value has $p_x = 2^{-n}$, so $H(X) = H_\infty(X) = n$ bits — the two notions coincide exactly when the distribution is uniform. This is the formal sense in which "a 128-bit key has 128 bits of security against guessing." Any nonuniformity strictly lowers H_∞, which is why CSPRNG output must be statistically close to uniform (§41.8.1).

In the real world

When you run `ssh-keygen` or a TLS library samples a session key, it pulls bytes from the OS CSPRNG (`getrandom`, `/dev/urandom`), which is seeded to have full min-entropy. Password managers target ~ 128 bits of min-entropy by drawing characters uniformly from a known alphabet — e.g. 20 characters from a 90-symbol set gives $20\log_2 90 \approx 130$ bits. The moment a generator's output is biased (Debian's 2008 OpenSSL bug, which shrank the key space to a few thousand values) the effective entropy collapses and keys become guessable.

41.8.5 The birthday bound: when do random draws collide?

The single most-cited probability fact in cryptography. Draw q values uniformly and independently from a set of size N. How large must q be before two draws *collide* (coincide)?

Theorem 41.7 (Birthday bound)**.** *With q uniform independent draws from N values, the probability of at least one collision satisfies*

$$\Pr[\textit{collision}] \;\le\; \binom{q}{2}\frac{1}{N} \;=\; \frac{q(q-1)}{2N},$$

and is well-approximated by $1 - e^{-q^2/(2N)}$. The probability crosses $1/2$ around $q \approx 1.177\sqrt{N}$, i.e. at roughly $\sqrt{N}$ draws (Figure 41.5).

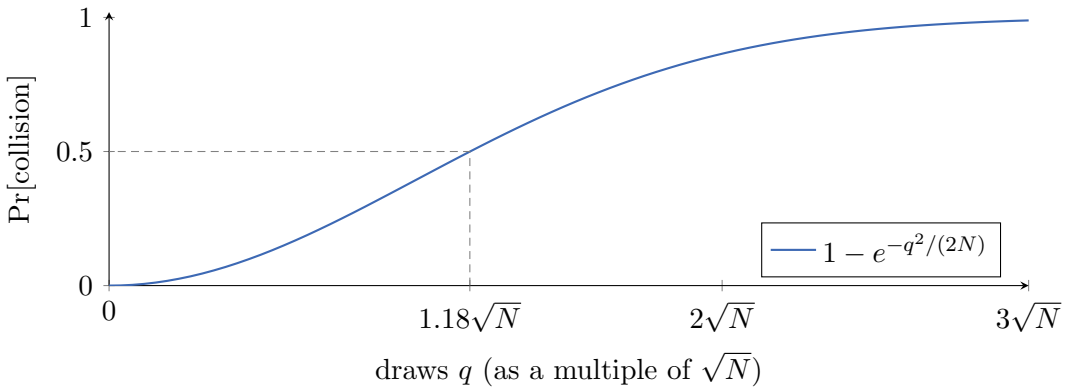


Figure 41.5: Collision probability against the number of draws, with q measured in units of $\sqrt{N}$. The curve crosses one half at about $1.18\sqrt{N}$ draws, regardless of how large N is — the square-root law in one picture.

Why it works

Why $\sqrt{N}$ and not N. A collision needs a *pair*, and there are $\binom{q}{2} \approx q^2/2$ pairs among q draws. Each specific pair coincides with probability $1/N$. By linearity of expectation, the expected number of colliding pairs is $\binom{q}{2}/N$; the union bound turns that into the upper bound above. When the expected number of collisions reaches about 1 — which happens at $q^2/2 \approx N$, i.e. $q \approx \sqrt{2N}$ — collisions become likely. The counterintuitive part is that you only need on the order of $\sqrt{N}$ draws, not N, because the number of *pairs* grows quadratically.

Example 41.7 (Classic birthdays and a 64-bit tag)**.** With $N = 365$ days, $1.177\sqrt{365} \approx 22.5$ — just 23 people give a better than even chance two share a birthday. Now scale up: a 64-bit value has $N = 2^{64}$, so collisions become likely around $q \approx 2^{32}$ — about four billion draws. That is why a 64-bit authentication tag or a 64-bit block cipher (such as 3DES) is risky at high volume, and why modern designs use 128-bit tags ($\sqrt{N} = 2^{64}$ draws, comfortably out of reach).

Attack

Birthday attack on hash collisions. To forge a document collision against an n-bit hash, an attacker does *not* search 2^n inputs. They generate about $2^{n/2}$ variants of an innocuous message and $2^{n/2}$ variants of a malicious one (whitespace, synonyms), hash them all, and look for any matching pair. By the birthday bound a collision is likely after $\sim 2^{n/2}$ work. This halves the effective security: MD5's 128-bit digest gives only $\sim 2^{64}$ collision resistance (and is broken outright in practice), which is precisely why collision-resistant hashes need an output *twice* the target security level — SHA-256 for 128-bit collision resistance.

Implement it

Watch the birthday bound appear empirically: draw random values and count how many draws until the first collision, averaged over trials.

```
import random, math

def first_collision(N, trials=2000):
```

```
    # Average number of uniform draws from [0, N) until the first repeat.
    # LEARNING ONLY: a demonstration, not a cryptographic RNG test.
    total = 0
    for _ in range(trials):
        seen = set()
        while True:
            x = random.randrange(N)
            if x in seen:
                break
            seen.add(x)
            total += 1
    return total / trials

for N in (365, 4096, 65536):
    print(N, round(first_collision(N)), "predicted", round(math.sqrt(math.pi*N/2)))
# Empirical count tracks sqrt(pi*N/2), the birthday-bound mean.
```

Warning: Python's `random` is a Mersenne Twister — fine for this illustration, never for keys or nonces. Use `secrets` or `os.urandom` for anything security-relevant.

41.8.6 The negligible-function discipline

Security definitions need a notion of "so small we do not care." A function $\mathsf{negl}(\lambda)$ is *negligible* if it shrinks faster than any inverse polynomial: for every c, eventually $\mathsf{negl}(\lambda) < \lambda^{-c}$. Think $2^{-\lambda}$ or $\lambda^{-\log \lambda}$. The whole apparatus of provable security reads "every PPT adversary wins with probability at most $\mathsf{negl}(\lambda)$." Negligible functions are closed under addition and under multiplication by polynomials, which is exactly what makes union-bound and hybrid arguments compose without the error blowing up.

Connection

The asymptotics behind "negligible," and the formal definition of a PPT (probabilistic polynomial-time) machine, sit at the boundary of *Probability From the Ground Up* and *Algorithms From the Ground Up*; Chapter 8 of this book builds the security games on top of them.

41.9 A pocket card of probability facts

Fact	Where it is used
$X \xleftarrow{\$} \{0,1\}^n$ has $H = H_\infty = n$	key/nonce strength
Min-entropy $H_\infty = -\log_2 \max_x p_x$	passwords, KDF sizing
$\Delta(P,Q) = \frac{1}{2}\sum\lvert P - Q\rvert$	PRG/indistinguishability proofs
Triangle ineq. for Δ	hybrid arguments
Union bound $\Pr[\cup A_i] \le \sum \Pr[A_i]$	bounding many bad events
Linearity of $\mathbb{E}$ (always)	counting collisions
Collision likely at $q \approx \sqrt{N}$	tag/block/hash sizing
n-bit hash $\Rightarrow 2^{n/2}$ collision work	choosing digest length
negl: smaller than any $1/\mathrm{poly}$	security definitions

41.10 How the two halves meet

The algebra gives you objects whose forward operation is easy and backward operation is hard; the probability gives you the language to say *how* hard, and to size your parameters so that "hard" stays hard. A discrete-log group of order q resists generic attacks up to about $\sqrt{q}$ work — a *birthday bound* again (Pollard's rho), which is why a 256-bit elliptic curve targets 128-bit security. RSA needs a ~ 3072-bit modulus for the same 128 bits because factoring is sub-exponential, not square-root. In both cases you are balancing an algebraic hardness assumption against a probabilistic attack cost. Hold the two pocket cards side by side and most parameter choices in the book stop looking arbitrary and follow directly from the arithmetic.

41.11 Exercises

Exercise 41.1. List the elements of $(\mathbb{Z}/18\mathbb{Z})^*$ and verify $|(\mathbb{Z}/18\mathbb{Z})^*| = \varphi(18)$. Find the order of 5 in this group, and decide whether 5 is a generator.

Exercise 41.2. Using the extended Euclidean algorithm, compute $17^{-1} \bmod 43$. Check your answer by multiplying. Then explain in one sentence why gcd being 1 was necessary.

Exercise 41.3. Show directly that $\mathbb{Z}/12\mathbb{Z}$ is not a field by exhibiting two nonzero elements whose product is 0. Which elements of $\mathbb{Z}/12\mathbb{Z}$ *are* units?

Exercise 41.4. In $\mathbb{F}_{2^3}$ with modulus $f(x) = x^3 + x + 1$, compute $(x^2 + x) \cdot (x + 1)$ and reduce. Then find the multiplicative inverse of x (hint: what does $x \cdot x^2$ reduce to?).

Exercise 41.5. The AES field $\mathbb{F}_{2^8}$ uses the reduction polynomial $x^8 + x^4 + x^3 + x + 1$. Verify that multiplying the byte `0x02` by an element is "shift left by one, and XOR `0x1B` if the high bit was set." Compute $\texttt{0x02} \cdot \texttt{0x80}$ and $\texttt{0x02} \cdot \texttt{0x57}$.

Exercise 41.6. A coin lands heads with probability 0.6. Compute its Shannon entropy and its min-entropy in bits, and explain which one you would use to budget guessing resistance.

Exercise 41.7. You generate random 40-bit nonces. Using the birthday approximation, roughly how many nonces can you issue before the probability of a repeat reaches $1/2$? Restate your answer as "about $2^{?}$ nonces."

Exercise 41.8. Prove that if P, Q, R are distributions on a finite set, then $\Delta(P, R) \le \Delta(P, Q) + \Delta(Q, R)$. (Use the definition $\Delta = \frac{1}{2}\sum|\cdot|$ and the ordinary triangle inequality on each term.)

Exercise 41.9. An attacker wants a collision in a 96-bit hash. About how much work (as a power of two) does the birthday attack of §41.8.5 require, and how many bits of collision security does a 96-bit digest therefore provide? Is that enough for 128-bit-security applications?

Exercise 41.10. Using the union bound, show that if each of q independent encryptions draws a nonce uniformly from N values, the probability that *any* two share a nonce is at most $q^2/(2N)$. For $N = 2^{96}$, how large can q be while keeping this below 2^{-32}?

Exercise 41.11. Explain, using min-entropy, why a key derived from a password with 20 bits of min-entropy through any deterministic KDF still has at most 20 bits of security, no matter how strong the KDF is. What does this tell you about the role of a slow KDF (such as Argon2)?